Building Java Programs

A BACK TO BASICS APPROACH

5e

Stuart Reges / Marty Stepp

Table of Contents

Building Java Programs A Back to Basics Approach
Preface
- New to This Edition
- Features from Prior Editions
- Layers and Dependencies
- Supplements
- MyLab Programming
- VideoNotes

Location of Video Notes in the Text http://www.pearson.com/cs-resources
Brief Contents
Contents
Chapter 1 Introduction to Java Programming
- Introduction
- 1.1 Basic Computing Concepts
  - Why Programming?
  - Hardware and Software
  - The Digital Realm
  - The Process of Programming
  - Why Java?
  - The Java Programming Environment
- 1.2 And Now—Java
  - String Literals (Strings)
  - System.out.println
  - Escape Sequences
  - print versus println
  - Identifiers and Keywords
  - A Complex Example: DrawFigures1
  - Comments and Readability
- 1.3 Program Errors
  - Syntax Errors

Logic Errors (Bugs)
1.4 Procedural Decomposition
Static Methods
Flow of Control
Methods That Call Other Methods
An Example Runtime Error
1.5 Case Study: DrawFigures
Structured Version
Final Version without Redundancy
Analysis of Flow of Execution
Chapter Summary
Self-Check Problems
Section 1.1: Basic Computing Concepts
Section 1.2: And Now—Java
Section 1.3: Program Errors
Section 1.4: Procedural Decomposition
Exercises
Programming Projects
Chapter 2 Primitive Data and Definite Loops
Introduction
2.1 Basic Data Concepts
Primitive Types
Expressions
JShell
Literals
Arithmetic Operators
Precedence
Mixing Types and Casting
2.2 Variables
Assignment/Declaration Variations
String Concatenation
Increment/Decrement Operators
Variables and Mixing Types

- 2.3 The for Loop
  - Tracing for Loops
  - for Loop Patterns
  - Nested for Loops
- 2.4 Managing Complexity
  - Scope
  - Pseudocode
  - The Table Technique
  - Class Constants
- 2.5 Case Study: Hourglass Figure
  - Problem Decomposition and Pseudocode
  - Initial Structured Version
  - Adding a Class Constant
  - Further Variations
- Chapter Summary
- Self-Check Problems
  - Section 2.1: Basic Data Concepts
  - Section 2.2: Variables
  - Section 2.3: The for Loop
  - Section 2.4: Managing Complexity
- Exercises
- Programming Projects

Chapter 3 Introduction to Parameters and Objects

- Introduction
- 3.1 Parameters
  - The Mechanics of Parameters
  - Limitations of Parameters
  - Multiple Parameters
  - Parameters versus Constants
  - Overloading of Methods
- 3.2 Methods That Return Values
  - The Math Class
  - Defining Methods That Return Values

- 3.3 Using Objects
  - String Objects
  - Interactive Programs and Scanner Objects
  - Sample Interactive Program
- 3.4 Case Study: Projectile Trajectory
  - Unstructured Solution
  - Structured Solution
- Chapter Summary
- Self-Check Problems
  - Section 3.1: Parameters
  - Section 3.2: Methods That Return Values
  - Section 3.3: Using Objects
- Exercises
- Programming Projects

Supplement 3G Graphics (Optional)

- Introduction
- 3G.1 Introduction to Graphics
  - DrawingPanel
  - Drawing Lines and Shapes
  - Colors
  - Drawing with Loops
  - Text and Fonts
  - Images
- 3G.2 Procedural Decomposition with Graphics
  - A Larger Example: DrawDiamonds
- 3G.3 Case Study: Pyramids
  - Unstructured Partial Solution
  - Generalizing the Drawing of Pyramids
  - Complete Structured Solution
- Chapter Summary
- Self-Check Problems
  - Section 3G.1: Introduction to Graphics
- Exercises

- Programming Projects
- Chapter 4 Conditional Execution
  - Introduction
  - 4.1 if/else Statements
    - Relational Operators
    - Nested if/else Statements
    - Object Equality
    - Factoring if/else Statements
    - Testing Multiple Conditions
  - 4.2 Cumulative Algorithms
    - Cumulative Sum
    - Min/Max Loops
    - Cumulative Sum with if
    - Roundoff Errors
  - 4.3 Text Processing
    - The char Type
    - char versus int
    - Cumulative Text Algorithms
    - System.out.printf
  - 4.4 Methods with Conditional Execution
    - Preconditions and Postconditions
    - Throwing Exceptions
    - Revisiting Return Values
    - Reasoning about Paths
  - 4.5 Case Study: Body Mass Index
    - One-Person Unstructured Solution
    - Two-Person Unstructured Solution
    - Two-Person Structured Solution
    - Procedural Design Heuristics
  - Chapter Summary
  - Self-Check Problems
    - Section 4.1: if/else Statements
    - Section 4.2: Cumulative Algorithms

Section 4.3: Text Processing
Section 4.4: Methods with Conditional Execution
Exercises
Programming Projects
Chapter 5 Program Logic and Indefinite Loops
Introduction
5.1 The while Loop
A Loop to Find the Smallest Divisor
Random Numbers
Simulations
do/while Loop
5.2 Fencepost Algorithms
Fencepost with if
Sentinel Loops
5.3 The boolean Type
Logical Operators
Short-Circuited Evaluation
boolean Variables and Flags
Boolean Zen
Negating Boolean Expressions
5.4 User Errors
Scanner Lookahead
Handling User Errors
5.5 Assertions and Program Logic
Reasoning about Assertions
A Detailed Assertions Example
5.6 Case Study: NumberGuess
Initial Version without Hinting
Randomized Version with Hinting
Final Robust Version
Chapter Summary
Self-Check Problems
Section 5.1: The while Loop

Section 5.2: Fencepost Algorithms
Section 5.3: The boolean Type
Section 5.4: User Errors
Section 5.5: Assertions and Program Logic
Exercises
Programming Projects
Chapter 6 File Processing
Introduction
6.1 File-Reading Basics
Data, Data Everywhere
Files and File Objects
Reading a File with a Scanner
6.2 Details of Token-Based Processing
Structure of Files and Consuming Input
Scanner Parameters
Paths and Directories
A More Complex Input File
6.3 Line-Based Processing
String Scanners and Line/Token Combinations
6.4 Advanced File Processing
Output Files with PrintStream
Guaranteeing That Files Can Be Read
6.5 Case Study: Zip Code Lookup
Chapter Summary
Self-Check Problems
Section 6.1: File-Reading Basics
Section 6.2: Details of Token-Based Processing
Section 6.3: Line-Based Processing
Section 6.4: Advanced File Processing
Exercises
Programming Projects
Chapter 7 Arrays
Introduction

7.1 Array Basics
- Constructing and Traversing an Array
- Accessing an Array
- Initializing Arrays
- A Complete Array Program
- Random Access
- Arrays and Methods
- The For-Each Loop
- The Arrays Class

7.2 Array-Traversal Algorithms
- Printing an Array
- Searching and Replacing
- Testing for Equality
- Reversing an Array
- String Traversal Algorithms
- Functional Approach

7.3 Reference Semantics
- Multiple Objects

7.4 Advanced Array Techniques
- Shifting Values in an Array
- Arrays of Objects
- Command-Line Arguments
- Nested Loop Algorithms

7.5 Multidimensional Arrays
- Rectangular Two-Dimensional Arrays
- Jagged Arrays

7.6 Arrays of Pixels

7.7 Case Study: Benford's Law
- Tallying Values
- Completing the Program

Chapter Summary

Self-Check Problems
- Section 7.1: Array Basics

Section 7.2: Array-Traversal Algorithms
Section 7.3: Reference Semantics
Section 7.4: Advanced Array Techniques
Section 7.5: Multidimensional Arrays
Exercises
Programming Projects
Chapter 8 Classes
Introduction
8.1 Object-Oriented Programming
Classes and Objects
Point Objects
8.2 Object State and Behavior
Object State: Fields
Object Behavior: Methods
The Implicit Parameter
Mutators and Accessors
The toString Method
8.3 Object Initialization: Constructors
The Keyword this
Multiple Constructors
8.4 Encapsulation
Private Fields
Class Invariants
Changing Internal Implementations
8.5 Case Study: Designing a Stock Class
Object-Oriented Design Heuristics
Stock Fields and Method Headers
Stock Method and Constructor Implementation
Chapter Summary
Self-Check Problems
Section 8.1: Object-Oriented Programming
Section 8.2: Object State and Behavior
Section 8.3: Object Initialization: Constructors

Section 8.4: Encapsulation
Section 8.5: Case Study: Designing a Stock Class
Exercises
Programming Projects
Chapter 9 Inheritance and Interfaces
Introduction
9.1 Inheritance Basics
Nonprogramming Hierarchies
Extending a Class
Overriding Methods
9.2 Interacting with the Superclass
Calling Overridden Methods
Accessing Inherited Fields
Calling a Superclass's Constructor
DividendStock Behavior
The Object Class
The equals Method
The instanceof Keyword
9.3 Polymorphism
Polymorphism Mechanics
Interpreting Inheritance Code
Interpreting Complex Calls
9.4 Inheritance and Design
A Misuse of Inheritance
Is-a Versus Has-a Relationships
Graphics2D
9.5 Interfaces
An Interface for Shapes
Implementing an Interface
Benefits of Interfaces
9.6 Case Study: Financial Class Hierarchy
Designing the Classes
Redundant Implementation

Abstract Classes
Chapter Summary
Self-Check Problems
Section 9.1: Inheritance Basics
Section 9.2: Interacting with the Superclass
Section 9.3: Polymorphism
Section 9.4: Inheritance and Design
Section 9.5: Interfaces
Section 9.6: Case Study: Financial Class Hierarchy
Exercises
Programming Projects
Chapter 10 ArrayLists
Introduction
10.1 ArrayLists
Basic ArrayList Operations
ArrayList Searching Methods
A Complete ArrayList Program
Adding to and Removing from an ArrayList
Initializing an ArrayList
Using the For-Each Loop with ArrayLists
Wrapper Classes
10.2 The Comparable Interface
Natural Ordering and compareTo
Implementing the Comparable Interface
10.3 Case Study: Ranked Choice Voting
Ballot Class
Counting Votes
Multiple Rounds
Chapter Summary
Self-Check Problems
Section 10.1: ArrayLists
Section 10.2: The Comparable Interface
Exercises

- Programming Projects
- Chapter 11 Java Collections Framework
  - Introduction
  - 11.1 Lists
    - Collections
    - LinkedList versus ArrayList
    - Iterators
    - Abstract Data Types (ADTs)
    - LinkedList Case Study: Sieve
  - 11.2 Sets
    - Set Concepts
    - TreeSet versus HashSet
    - Set Operations
    - Set Case Study: Lottery
  - 11.3 Maps
    - Basic Map Operations
    - Map Views (keySet and values)
    - TreeMap versus HashMap
    - Map Case Study: WordCount
    - Collection Overview
  - Chapter Summary
  - Self-Check Problems
    - Section 11.1: Lists
    - Section 11.2: Sets
    - Section 11.3: Maps
  - Exercises
  - Programming Projects
- Chapter 12 Recursion
  - Introduction
  - 12.1 Thinking Recursively
    - A Nonprogramming Example
    - An Iterative Solution Converted to Recursion
    - Structure of Recursive Solutions

- 12.2 A Better Example of Recursion
  - Mechanics of Recursion
- 12.3 Recursive Functions and Data
  - Integer Exponentiation
  - Greatest Common Divisor
  - Directory Crawler
  - Helper Methods
- 12.4 Recursive Graphics
- 12.5 Recursive Backtracking
  - A Simple Example: Traveling North/East
  - 8 Queens Puzzle
  - Solving Sudoku Puzzles
- 12.6 Case Study: Prefix Evaluator
  - Infix, Prefix, and Postfix Notation
  - Evaluating Prefix Expressions
  - Complete Program
- Chapter Summary
- Self-Check Problems
  - Section 12.1: Thinking Recursively
  - Section 12.2: A Better Example of Recursion
  - Section 12.3: Recursive Functions and Data
  - Section 12.4: Recursive Graphics
  - Section 12.5: Recursive Backtracking
- Exercises
- Programming Projects

Chapter 13 Searching and Sorting

- Introduction
- 13.1 Searching and Sorting in the Java Class Libraries
  - Binary Search
  - Sorting
  - Shuffling
  - Custom Ordering with Comparators
- 13.2 Program Complexity

Empirical Analysis
Complexity Classes
13.3 Implementing Searching and Sorting Algorithms
Sequential Search
Binary Search
Recursive Binary Search
Searching Objects
Selection Sort
13.4 Case Study: Implementing Merge Sort
Splitting and Merging Arrays
Recursive Merge Sort
Complete Program
Chapter Summary
Self-Check Problems
Section 13.1: Searching and Sorting in the Java Class Libraries
Section 13.2: Program Complexity
Section 13.3: Implementing Searching and Sorting Algorithms
Section 13.4: Case Study: Implementing Merge Sort
Exercises
Programming Projects
Chapter 14 Stacks and Queues
Introduction
14.1 Stack/Queue Basics
Stack Concepts
Queue Concepts
14.2 Common Stack/Queue Operations
Transferring between Stacks and Queues
Sum of a Queue
Sum of a Stack
14.3 Complex Stack/Queue Operations
Removing Values from a Queue

Comparing Two Stacks for Similarity
14.4 Case Study: Expression Evaluator
Splitting into Tokens
The Evaluator
Chapter Summary
Self-Check Problems
Section 14.1: Stack/Queue Basics
Section 14.2: Common Stack/Queue Operations
Section 14.3: Complex Stack/Queue Operations
Exercises
Programming Projects
Chapter 15 Implementing a Collection Class
Introduction
15.1 Simple ArrayIntList
Adding and Printing
Thinking about Encapsulation
Dealing with the Middle of the List
Another Constructor and a Constant
Preconditions and Postconditions
15.2 A More Complete ArrayIntList
Throwing Exceptions
Convenience Methods
15.3 Advanced Features
Resizing When Necessary
Adding an Iterator
15.4 ArrayList<E>
Chapter Summary
Self-Check Problems
Section 15.1: Simple ArrayIntList
Section 15.2: A More Complete ArrayIntList
Section 15.3: Advanced Features
Section 15.4: ArrayList<E>
Exercises

- Programming Projects
- Chapter 16 Linked Lists
  - Introduction
  - 16.1 Working with Nodes
    - Constructing a List
    - List Basics
    - Manipulating Nodes
    - Traversing a List
  - 16.2 A Linked List Class
    - Simple LinkedIntList
    - Appending add
    - The Middle of the List
  - 16.3 A Complex List Operation
    - Inchworm Approach
  - 16.4 An IntList Interface
  - 16.5 LinkedList<E>
    - Linked List Variations
    - Linked List Iterators
    - Other Code Details
  - Chapter Summary
  - Self-Check Problems
    - Section 16.1: Working with Nodes
    - Section 16.2: A Linked List Class
    - Section 16.3: A Complex List Operation
    - Section 16.4: An IntList Interface
    - Section 16.5: LinkedList<E>
  - Exercises
  - Programming Projects
- Chapter 17 Binary Trees
  - Introduction
  - 17.1 Binary Tree Basics
    - Node and Tree Classes
  - 17.2 Tree Traversals

Constructing and Viewing a Tree
17.3 Common Tree Operations
Sum of a Tree
Counting Levels
Counting Leaves
17.4 Binary Search Trees
The Binary Search Tree Property
Building a Binary Search Tree
The Pattern x = change(x)
Searching the Tree
Binary Search Tree Complexity
17.5 SearchTree<E>
Chapter Summary
Self-Check Problems
Section 17.1: Binary Tree Basics
Section 17.2: Tree Traversals
Section 17.3: Common Tree Operations
Section 17.4: Binary Search Trees
Section 17.5: SearchTree<E>
Exercises
Programming Projects
Chapter 18 Advanced Data Structures
Introduction
18.1 Hashing
Array Set Implementations
Hash Functions and Hash Tables
Collisions
Rehashing
Hashing Non-Integer Data
Hash Map Implementation
18.2 Priority Queues and Heaps
Priority Queues
Introduction to Heaps

Removing from a Heap
Adding to a Heap
Array Heap Implementation
Heap Sort
Chapter Summary
Self-Check Problems
Section 18.1: Hashing
Section 18.2: Priority Queues and Heaps
Exercises
Programming Projects
Chapter 19 Functional Programming with Java 8
Introduction
19.1 Effect-Free Programming
19.2 First-Class Functions
Lambda Expressions
19.3 Streams
Basic Idea
Using Map
Using Filter
Using Reduce
Optional Results
19.4 Function Closures
19.5 Higher-Order Operations on Collections
Working with Arrays
Working with Lists
Working with Files
19.6 Case Study: Perfect Numbers
Computing Sums
Incorporating Square Root
Just Five and Leveraging Concurrency
Chapter Summary
Self-Check Problems
19.1 Effect-Free Programming

19.2 First-Class Functions
19.3 Streams
19.4 Function Closures
19.5 Higher-Order Operations on Collections
Exercises
Programming Projects
Appendix A Java Summary
Java Keywords
Primitive Types
Arithmetic Operators
Relational Operators
Logical Operators
Operator Precedence
Wrapper Classes
Syntax Templates
Useful Methods of ArrayList Objects
Useful Methods of the Character Class
Useful Methods of the Collection Interface
Useful Methods of the Collections Class
Useful Methods of DrawingPanel Objects
Useful Methods of File Objects
Useful Methods of Graphics Objects
Useful Methods of Iterator Objects
Useful Methods of Map Objects
Constants and Useful Methods of the Math Class
Useful Methods of the Object Class
Useful Methods of Point Objects
Useful Methods of Random Objects
Useful Methods of Scanner Objects
Useful Methods of String Objects
Appendix B The Java API Specification and Javadoc Comments
The Java API Specification
Parameters

- Returns
- Throws
- Deprecated
- Writing Javadoc Comments

Appendix C Additional Java Syntax
- Primitive Types: byte, short, long, float
- Ternary Operator ? :
- Exiting a Loop: break and continue
- The switch Statement
- The try/catch Statement
- The assert Statement
- Enumerations: enum
- Packages
- Protected and Default Access

Index
- A
- B
- C
- D
- E
- F
- G
- H
- I
- J
- K
- L
- M
- N
- O
- P
- Q
- R

S
T
U
V
W
Y
Z

Building Java Programs

A BACK TO BASICS APPROACH

5e

Stuart Reges / Marty Stepp

Building Java Programs

A Back to Basics Approach

Fifth Edition

Stuart Reges
University of Washington
Marty Stepp
Stanford University

SVP, Courseware Portfolio Management: Marcia Horton

Portfolio Manager: Matt Goldstein

Portfolio Manager Assistant: Meghan Jacoby

VP, Product Marketing: Roxanne McCarley

Director of Field Marketing: Tim Galligan

Product Marketing Manager: Yvonne Vannatta

Field Marketing Manager: Demetrius Hall

Marketing Assistant: Jon Bryant

Managing Content Producer: Scott Disanno

VP, Production & Digital Studio: Ruth Berry

Project Manager: Lakeside Editorial Services L.L.C.

Senior Specialist, Program Planning and Support: Deidra Headlee

Cover Design: Jerilyn Bockorick

R&P Manager: Ben Ferrini

R&P Project Manager: Lav Kush Sharma/Integra Publishing Services, Inc.

Cover Art: Marcell Faber/Shutterstock

Full-Service Project Management: Integra Software Services Pvt. Ltd.

Composition: Integra Software Services Pvt. Ltd.

Printer/Binder: LSC Communications

Cover Printer: Phoenix Color

Text Font: Monotype

The authors and publisher of this book have used their best efforts in preparing this book. These efforts include the development, research, and testing of the theories and programs to determine their effectiveness. The authors and publisher make no warranty of any kind, expressed or implied, with regard to these programs or to the documentation contained in this book. The authors and publisher shall not be liable in any event for incidental or consequential damages in connection with, or arising out of, the furnishing, performance, or use of these programs.

Copyright © 2020, 2017, 2014 and 2011 Pearson Education, Inc. or its affiliates. All rights reserved. Printed in the United States of America. This publication is protected by copyright, and permission should be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions department, please visit www.pearsonhighed.com/permissions/.

PEARSON, and MyLab Programming are exclusive trademarks in the U.S. and/or other countries owned by Pearson Education, Inc. or its affiliates.

Unless otherwise indicated herein, any third-party trademarks that may appear in this work are the property of their respective owners and any references to third-party trademarks, logos or other trade dress are for demonstrative or descriptive purposes only. Such references are not intended to imply any sponsorship, endorsement, authorization, or promotion of Pearson's products by the owners of such marks, or any relationship between the owner and Pearson Education, Inc. or its affiliates, authors, licensees or distributors.

Library of Congress Cataloging-in-Publication Data

Names: Reges, Stuart, author. | Stepp, Martin, author.

Title: Building Java programs: a back to basics approach / Stuart Reges, University of Washington, Marty Stepp, Stanford University.

Description: Fifth edition. | Hoboken, New Jersey: Pearson, 2019. | Includes index.

Identifiers: LCCN 2018050748 | ISBN 9780135471944 | ISBN 013547194X

Subjects: LCSH: Java (Computer program language)

Classification: LCC QA76.73.J38 R447 2019 | DDC 005.13/3—dc23 LC record available at https://lccn.loc.gov/2018050748

1 19

ISBN 10: 0- 13-547194- X

ISBN 13: 978-0-13-547194- 4

MyLab Programming

Through the power of practice and immediate personalized feedback, MyLab™ Programming helps students master programming fundamentals and build computational thinking skills.

PROGRAMMING PRACTICE

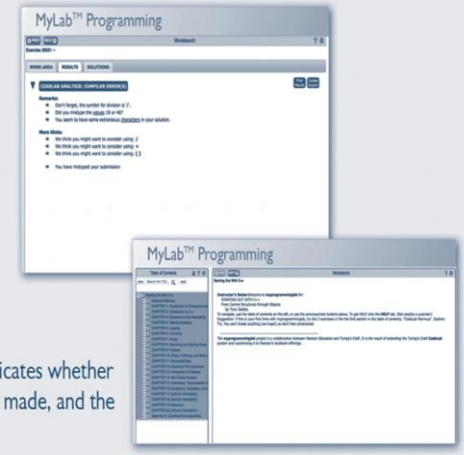
With MyLab Programming, your students will gain first-hand programming experience in an interactive online environment.

IMMEDIATE, PERSONALIZED FEEDBACK

MyLab Programming automatically detects errors in the logic and syntax of their code submission and offers targeted hints that enables students to figure out what went wrong and why.

GRADUATED COMPLEXITY

MyLab Programming breaks down programming concepts into short, understandable sequences of exercises. Within each sequence the level and sophistication of the exercises increase gradually but steadily.



DYNAMIC ROSTER

Students' submissions are stored in a roster that indicates whether the submission is correct, how many attempts were made, and the actual code submissions from each attempt.

PEARSON eTEXT

The Pearson eText gives students access to their textbook anytime, anywhere.

STEP-BY-STEP VIDEONOTE TUTORIALS

These step-by-step video tutorials enhance the programming concepts presented in select Pearson textbooks.

For more information and titles available with **MyLab Programming,** please visit **www.pearson.com/mylab/programming**

Copyright © 2018 Pearson Education, Inc. or its affiliate(s). All rights reserved. HELO88173 • 11/15

ALWAYS LEARNING

Pearson

1-1 Full Alternative Text

Lời nói đầu

Phiên bản thứ năm được chỉnh sửa lại của giáo trình "Building Java Programs" được thiết kế để sử dụng cho hai khóa học nhập môn khoa học máy tính. Chúng tôi đã thử nghiệm trên lớp với hàng ngàn sinh viên đại học, hầu hết không phải là chuyên ngành khoa học máy tính, trong chuỗi khóa học CS1-CS2 tại Đại học Washington. Các khóa học này đang có lượng ghi danh kỷ lục, và các trường khác đã áp dụng giáo trình của chúng tôi cũng báo cáo rằng sinh viên đang thành công với phương pháp tiếp cận của chúng tôi.

Các khóa học nhập môn khoa học máy tính thường được coi là những khóa học "khó khăn" với tỷ lệ thất bại cao. Nhưng như Douglas Adams đã nói trong "The Hitchhiker’s Guide to the Galaxy," "Đừng hoảng sợ." Sinh viên có thể làm chủ được tài liệu này nếu họ học nó một cách từ từ. Giáo trình của chúng tôi sử dụng cách tiếp cận từng lớp để giới thiệu cú pháp và khái niệm mới qua nhiều chương.

Giáo trình của chúng tôi sử dụng cách tiếp cận “objects later”, trong đó các nguyên tắc lập trình và phân tích thủ tục được dạy trước khi đi sâu vào lập trình hướng đối tượng. Chúng tôi đã đề xuất cách tiếp cận này, đôi khi gọi là “trở lại cơ bản,” và qua nhiều năm kinh nghiệm, chúng tôi đã thấy rằng một loạt các nhà khoa học, kỹ sư và những người khác có thể học cách lập trình theo cách thủ tục. Khi chúng ta đã xây dựng được nền tảng vững chắc về các kỹ thuật thủ tục, chúng ta sẽ chuyển sang lập trình hướng đối tượng. Cuối khóa học, sinh viên sẽ học được cả hai phong cách lập trình.

Ngôn ngữ Java luôn tiến hóa, và chúng tôi đã tập trung vào các tính năng mới được bổ sung trong Java 8 đến 10 trong các phiên bản gần đây. Trong ấn bản thứ tư, chúng tôi đã thêm một chương mới, Chương 19, về các tính năng lập trình hàm của Java được giới thiệu trong Java 8. Trong ấn bản này, chúng tôi tích hợp công cụ JShell được giới thiệu trong Java 9.

Những điểm mới trong phiên bản này

Dưới đây là những thay đổi chính trong phiên bản thứ năm của chúng tôi:

- Tích hợp JShell: Java 9 đã giới thiệu JShell, một tiện ích với vòng lặp đọc-đánh giá-in (REPL) tương tác, giúp dễ dàng nhập các biểu thức Java và ngay lập tức thấy kết quả của chúng. Chúng tôi thấy JShell là một công cụ học tập hữu ích cho phép sinh viên khám phá các khái niệm Java mà không cần tạo một chương trình hoàn chỉnh. Chúng tôi giới thiệu JShell trong Chương 2 và tích hợp các ví dụ JShell trong từng chương suốt giáo trình.

- Cải tiến phần vòng lặp trong Chương 2: Chúng tôi đã thêm các phần và hình minh họa mới trong Chương 2 để giúp sinh viên hiểu về vòng lặp for và tạo bảng tìm mẫu trong vòng lặp lồng nhau. Nội dung mới này dựa trên các tương tác của chúng tôi với sinh viên khi họ giải quyết các bài toán lập trình với vòng lặp từ sớm trong khóa học.

- Cải tiến các nghiên cứu trường hợp, ví dụ, và các nội dung khác: Chúng tôi đã viết lại hoặc chỉnh sửa các phần của các

chương khác nhau dựa trên phản hồi từ sinh viên và giảng viên. Chúng tôi cũng đã viết lại nghiên cứu trường hợp Chương 10 (ArrayLists) với một chương trình mới tập trung vào bầu cử và bỏ phiếu lựa chọn xếp hạng.

- Cập nhật cú pháp và cách sử dụng bộ sưu tập: Các phiên bản Java gần đây đã giới thiệu cú pháp và tính năng mới liên quan đến bộ sưu tập, chẳng hạn như toán tử "diamond operator" <>; các giao diện bộ sưu tập như Lists, Sets, và Maps; và các phương thức bộ sưu tập mới. Chúng tôi đã cập nhật các Chương 10 và 11 về bộ sưu tập để thảo luận về các tính năng mới này, và sử dụng cú pháp "diamond operator" với bộ sưu tập trong phần còn lại của giáo trình.

- Mở rộng các bài tự kiểm tra và bài tập lập trình: Với mỗi phiên bản mới, chúng tôi thêm các bài tập lập trình mới vào cuối mỗi chương. Có khoảng năm mươi bài toán và bài tập mỗi chương, tất cả đều đã được kiểm tra trên lớp với sinh viên thực và có giải pháp cung cấp cho giảng viên trên trang web của chúng tôi.

- Dự án lập trình mới: Một số chương đã có thêm các dự án lập trình mới, chẳng hạn như dự án bỏ phiếu lựa chọn xếp hạng trong Chương 10.

Các tính năng từ các phiên bản trước

Các tính năng sau đã được giữ lại từ các phiên bản trước:

- Tập trung vào giải quyết vấn đề: Nhiều sách giáo khoa tập trung vào chi tiết ngôn ngữ khi giới thiệu các cấu trúc mới. Chúng tôi tập trung vào giải quyết vấn đề. Với mỗi cấu trúc mới, có thể giải quyết được những vấn đề gì? Những khó khăn nào người mới học có thể gặp phải trong quá trình học? Những cách phổ biến nhất để sử dụng một cấu trúc mới là gì?

- Nhấn mạnh tư duy thuật toán: Cách tiếp cận theo quy trình của chúng tôi cho phép nhấn mạnh vào giải quyết vấn đề theo thuật toán: chia nhỏ một vấn đề lớn thành các vấn đề nhỏ hơn, sử dụng mã giả (pseudocode) để tinh chỉnh thuật toán, và đối mặt với thách thức của việc biểu diễn một chương trình lớn theo thuật toán.

- Cách tiếp cận từng lớp: Lập trình bằng Java liên quan đến nhiều khái niệm khó có thể học được cùng một lúc. Dạy Java cho người mới bắt đầu giống như xây một ngôi nhà từ những quân bài. Mỗi quân bài mới phải được đặt cẩn thận. Nếu quá trình này bị vội vã và bạn cố gắng đặt quá nhiều quân bài cùng lúc, toàn bộ cấu trúc sẽ sụp đổ. Chúng tôi dạy các khái niệm mới một cách từ từ, từng lớp một, cho phép sinh viên mở rộng sự hiểu biết của mình với tốc độ phù hợp.

- Nghiên cứu trường hợp: Chúng tôi kết thúc hầu hết các chương với một nghiên cứu trường hợp quan trọng, cho thấy cách phát triển một chương trình phức tạp theo từng giai đoạn và cách kiểm tra nó khi đang được phát triển. Cấu trúc này cho phép chúng tôi minh họa mỗi cấu trúc lập trình mới trong một ngữ cảnh phong phú mà không thể đạt được

với các ví dụ mã ngắn. Một số nghiên cứu trường hợp đã được mở rộng và cải tiến trong phiên bản thứ hai.

- Tiện ích như một giáo trình CS1+CS2: Trong các phiên bản gần đây, chúng tôi đã thêm các chương mở rộng phạm vi của sách để bao quát tất cả các chủ đề từ khóa học thứ hai trong khoa học máy tính của chúng tôi, làm cho sách có thể sử dụng cho một chuỗi hai khóa học. Các chương 12–19 khám phá đệ quy, tìm kiếm và sắp xếp, ngăn xếp và hàng đợi, triển khai bộ sưu tập, danh sách liên kết, cây nhị phân, bảng băm, heap, và nhiều hơn nữa. Chương 12 cũng có thêm một phần về đệ quy truy hồi ngược, một kỹ thuật mạnh mẽ để khám phá tập hợp các khả năng giải quyết các vấn đề như 8 Queens và Sudoku.

- Năm nay cũng đánh dấu sự ra mắt của sách giáo trình mới của chúng tôi, "Building Python Programs," mang cách tiếp cận “trở lại cơ bản” của chúng tôi đến ngôn ngữ Python. Trong những năm gần đây, Python đã chứng kiến sự gia tăng phổ biến trong các lớp học khoa học máy tính nhập môn. Chúng tôi nhận thấy rằng tài liệu và cách tiếp cận của chúng tôi hoạt động tốt trong Python cũng như trong Java, và chúng tôi rất vui mừng cung cấp lựa chọn hai ngôn ngữ cho giảng viên và sinh viên.

Các lớp và sự phụ thuộc

Nhiều sách nhập môn khoa học máy tính tập trung vào ngôn ngữ, nhưng các chương đầu của cuốn sách này được xây dựng theo từng lớp. Ví dụ, Java có nhiều cấu trúc điều khiển (bao gồm vòng lặp for, vòng lặp while, và câu lệnh if/else), và nhiều sách bao

gồm tất cả các cấu trúc điều khiển này trong một chương duy nhất. Mặc dù điều này có thể hợp lý với những ai đã biết lập trình, nhưng lại có thể gây choáng ngợp cho người mới bắt đầu học lập trình. Chúng tôi thấy rằng hiệu quả hơn nhiều khi chia nhỏ các cấu trúc điều khiển này vào các chương khác nhau để sinh viên học từng cấu trúc một, thay vì cố gắng học tất cả cùng lúc.

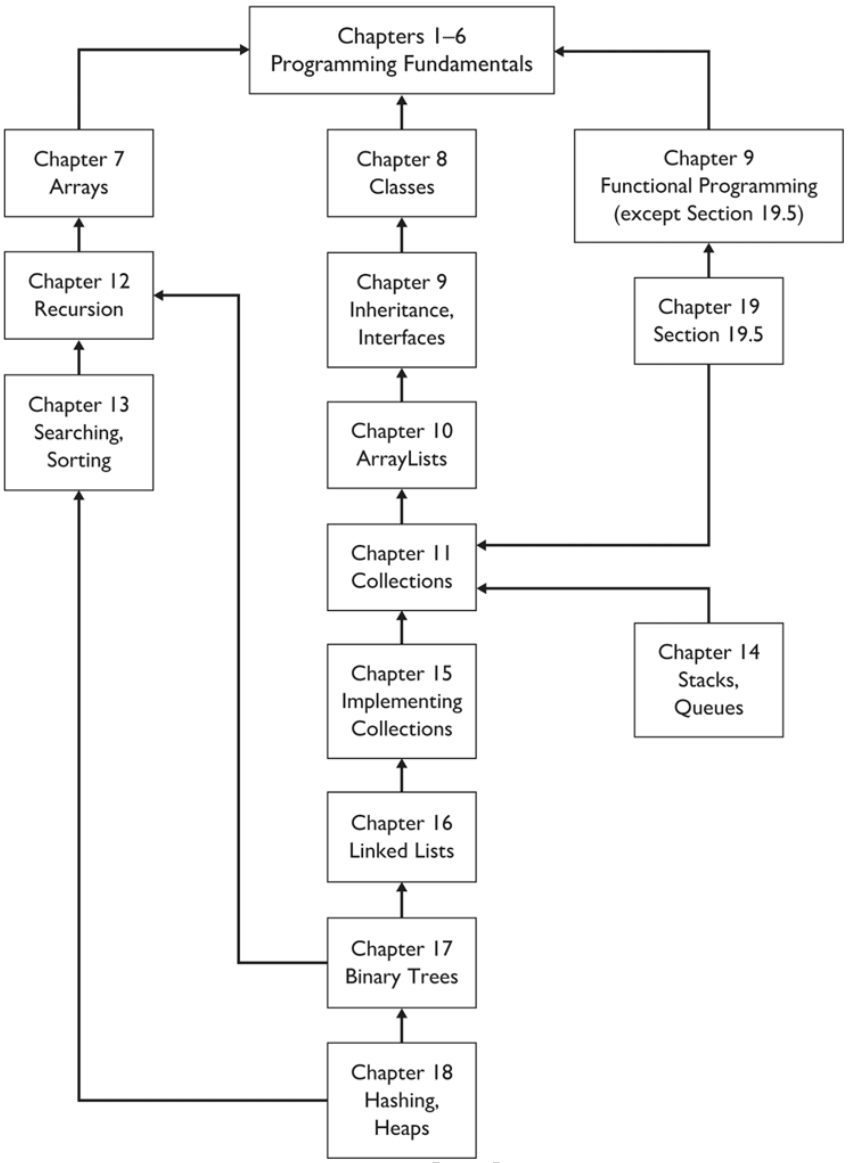
Bảng dưới đây cho thấy cách tiếp cận theo từng lớp hoạt động trong sáu chương đầu:

Chương	**Luồng điều khiển**	**Dữ liệu**	**Kỹ thuật lập trình**	**Nhập/Xuất**
1	phương thức	chuỗi ký tự	phân tích thủ tục	println, print
2	vòng lặp xác định (for)	biến, biểu thức, int, double	biến cục bộ, hằng lớp, mã giả	
3	giá trị trả về	sử dụng đối tượng	tham số	nhập từ bàn phím, đồ họa 2D (tùy chọn)
4	điều kiện (if/else)	char	điều kiện trước/sau, ném ngoại lệ	printf
5	vòng lặp không xác định (while)	boolean	khẳng định, chương trình bền vững	

Chương	Luồng điều khiển	Dữ liệu	Kỹ thuật lập trình	Nhập/Xuất
6		Scanner	xử lý tệp theo dòng/nhãn	I/O tệp

Các Chương 1–6 được thiết kế để học theo thứ tự, với sự linh hoạt hơn bắt đầu từ Chương 7. Chương 6 có thể bỏ qua, mặc dù nghiên cứu trường hợp trong Chương 7 liên quan đến việc đọc từ tệp, một chủ đề được đề cập trong Chương 6.

Dưới đây là biểu đồ phụ thuộc của cuốn sách:



1-2 Văn Bản Thay Thế Đầy Đủ

Tài liệu bổ sung

Trang web: http://www.buildingjavaprograms.com/

Đáp án cho tất cả các bài tự kiểm tra đều có trên trang web của chúng tôi và có thể truy cập bởi bất kỳ ai. Trang web của chúng tôi cung cấp các tài nguyên bổ sung sau cho sinh viên:

- Các chương bổ sung chỉ có trực tuyến, chẳng hạn như chương về tạo giao diện người dùng đồ họa (GUI).
- Mã nguồn và các tệp dữ liệu cho tất cả các nghiên cứu trường hợp và các ví dụ chương trình hoàn chỉnh khác.
- Lớp DrawingPanel được sử dụng trong Phụ lục 3G đồ họa tùy chọn.

Trang web của chúng tôi cũng cung cấp các tài nguyên bổ sung sau cho giáo viên:

- Slide PowerPoint phù hợp cho bài giảng.
- Giải pháp cho các bài tập và dự án lập trình, cùng với tài liệu đặc tả bài tập về nhà cho nhiều dự án.
- Các bài kiểm tra mẫu và đáp án.
- Bài tập phòng thí nghiệm bổ sung và bài tập lập trình kèm đáp án.
- Công cụ tạo bài tập phòng thí nghiệm đóng để tạo tài liệu bài tập với các bài tập do giảng viên chọn và tích hợp với giáo trình.

Để truy cập tài nguyên cho giảng viên, liên hệ với chúng tôi tại địa chỉ email: authors@buildingjavaprograms.com. Các tài liệu tương tự cũng có sẵn tại: http://www.pearsonhighered.com/cs-resources. Để hỏi các câu hỏi khác liên quan đến tài nguyên, hãy liên hệ với đại diện bán hàng của Pearson.

MyLab Programming

MyLab Programming là một công cụ luyện tập và đánh giá trực tuyến giúp sinh viên nắm bắt hoàn toàn logic, ngữ nghĩa, và cú pháp của lập trình. Thông qua các bài tập thực hành và phản hồi cá nhân hóa ngay lập tức, MyLab Programming cải thiện khả năng lập trình của sinh viên mới bắt đầu, những người thường gặp khó khăn với các khái niệm và mô hình cơ bản của các ngôn ngữ lập trình cấp cao phổ biến. Là một công cụ tự học và làm bài tập, khóa học MyLab Programming bao gồm hàng trăm bài tập thực hành nhỏ được tổ chức theo cấu trúc của giáo trình này. Đối với sinh viên, hệ thống tự động phát hiện lỗi trong logic và cú pháp của các bài nộp mã và cung cấp gợi ý nhắm mục tiêu giúp sinh viên tự tìm ra lỗi sai và lý do tại sao. Đối với giảng viên, một sổ điểm toàn diện theo dõi các câu trả lời đúng và sai và lưu trữ mã mà sinh viên đã nhập để xem lại.

Để xem bản trình diễn đầy đủ, xem phản hồi từ giảng viên và sinh viên, hoặc để áp dụng MyLab Programming cho khóa học của bạn, truy cập trang web sau:
www.pearson.com/mylab/programming

VideoNotes

Chúng tôi đã ghi lại một loạt các video hướng dẫn để đi kèm với giáo trình. Các video này có sẵn tại trang web:
http://www.pearsonhighered.com/cs-resources

Khoảng 3–4 video được đăng cho mỗi chương. Một biểu tượng trong lề trang sẽ chỉ ra khi nào có VideoNote cho một chủ đề nhất định. Trong mỗi video, chúng tôi dành từ 5–15 phút để đi qua một khái niệm hoặc vấn đề cụ thể, nói về những thách thức và phương pháp cần thiết để giải quyết nó. Những video này là tài liệu bổ sung tốt cho hướng dẫn được đưa ra trong các lớp học giảng dạy và trong giáo trình. Bản sao mới của giáo trình của bạn có một mã truy cập cho phép bạn xem các video này.

Lời Cảm Ơn
Trước hết, chúng tôi muốn gửi lời cảm ơn đến nhiều đồng nghiệp, sinh viên và trợ giảng đã sử dụng và đóng góp ý kiến cho các bản thảo ban đầu của cuốn sách này. Chúng tôi không thể viết cuốn sách này nếu không có sự đóng góp của họ. Đặc biệt, chúng tôi cảm ơn Hélène Martin, người đã cẩn thận đọc qua các phiên bản đầu tiên của các chương trong ấn bản đầu tiên để tìm ra lỗi và xác định những phần cần cải thiện. Chúng tôi cũng muốn cảm ơn giảng viên Benson Limketkai vì đã dành nhiều giờ thực hiện việc đọc và kiểm tra kỹ thuật cho ấn bản thứ hai.
Thứ hai, chúng tôi muốn cảm ơn đội ngũ các nhà đánh giá tài năng đã hướng dẫn chúng tôi trong quá trình tạo ra cuốn sách giáo khoa này:

- Greg Anderson, Đại học Bang Weber
- Delroy A. Brinkerhoff, Đại học Bang Weber
- Ed Brunjes, Cao đẳng Cộng đồng Miramar
- Tom Capaul, Đại học Đông Washington
- Tom Cortina, Đại học Carnegie Mellon
- Charles Dierbach, Đại học Towson
- H.E. Dunsmore, Đại học Purdue
- Michael Eckmann, Cao đẳng Skidmore
- Mary Anne Egan, Cao đẳng Siena
- Leonard J. Garrett, Đại học Temple
- Ahmad Ghafarian, Đại học Cao đẳng & Bang North Georgia
- Raj Gill, Cao đẳng Cộng đồng Anne Arundel
- Michael Hostetler, Đại học Park
- David Hovemeyer, Đại học York, Pennsylvania
- Chenglie Hu, Cao đẳng Carroll
- Philip Isenhour, Viện Bách khoa Virginia
- Andree Jacobson, Đại học New Mexico
- David C. Kamper, Sr., Đại học Đông Bắc Illinois
- Simon G.M. Koo, Đại học San Diego

- Evan Korth, Đại học New York
- Joan Krone, Đại học Denison
- John H.E.F. Lasseter, Đại học Fairfield
- Eric Matson, Đại học Bang Wright
- Kathryn S. McKinley, Đại học Texas, Austin
- Jerry Mead, Đại học Bucknell
- George Medelinskas, Cao đẳng Cộng đồng Bắc Essex
- John Neitzke, Đại học Truman
- Dale E. Parson, Đại học Kutztown
- Richard E. Pattis, Đại học Carnegie Mellon
- Frederick Pratter, Đại học Đông Oregon
- Roger Priebe, Đại học Texas, Austin
- Dehu Qi, Đại học Lamar
- John Rager, Cao đẳng Amherst
- Amala V.S. Rajan, Đại học Middlesex
- Craig Reinhart, Đại học Lutheran California
- Mike Scott, Đại học Texas, Austin
- Alexa Sharp, Cao đẳng Oberlin
- Tom Stokke, Đại học Bắc Dakota
- Leigh Ann Sudol, Trường Trung học Fox Lane
- Ronald F. Taylor, Đại học Bang Wright
- Andy Ray Terrel, Đại học Chicago
- Scott Thede, Đại học DePauw
- Megan Thomas, Đại học Bang California, Stanislaus
- Dwight Tuinstra, Đại học Bang New York Potsdam
- Jeannie Turner, Trường Sayre
- Tammy VanDeGrift, Đại học Portland
- Thomas John VanDrunen, Cao đẳng Wheaton
- Neal R. Wagner, Đại học Texas, San Antonio
- Jiangping Wang, Đại học Webster
- Yang Wang, Đại học Bang Missouri
- Stephen Weiss, Đại học Bắc Carolina tại Chapel Hill
- Laurie Werner, Đại học Miami

- Dianna Xu, Cao đẳng Bryn Mawr
- Carol Zander, Đại học Washington, Bothell

Cuối cùng, chúng tôi muốn cảm ơn đội ngũ tuyệt vời tại Pearson đã giúp sản xuất cuốn sách này. Michelle Brown, Jeff Holcomb, Maurene Goo, Patty Mahtani, Nancy Kotary, và Kathleen Kenny đã làm việc rất tốt trong việc chuẩn bị ấn bản đầu tiên. Các biên tập viên và nhân viên của Aptara Corp, bao gồm Heather Sisan, Brian Baker, Brendan Short, và Rachel Head, đã phát hiện ra nhiều lỗi và cải thiện chất lượng bài viết. Marilyn Lloyd và Chelsea Bell đã phục vụ tốt trong vai trò quản lý dự án và trợ lý biên tập tương ứng ở các phiên bản trước. Đối với sự giúp đỡ của họ với ấn bản thứ ba, chúng tôi muốn cảm ơn Kayla Smith-Tarbox, Quản lý Dự án Sản xuất, và Jenah Blitz-Stoehr, Trợ lý Biên tập Khoa học Máy tính. Mohinder Singh và đội ngũ tại Aptara, Inc., cũng đã rất hữu ích trong sản xuất cuối cùng của ấn bản thứ ba. Đối với công việc tuyệt vời của họ trong việc sản xuất ấn bản thứ tư và thứ năm, chúng tôi cảm ơn Louise Capulli và đội ngũ của Lakeside Editorial Services, cùng với Carole Snyder tại Pearson. Đặc biệt cảm ơn biên tập viên chính của chúng tôi tại Pearson, Matt Goldstein, người đã tin tưởng vào ý tưởng của cuốn sách từ ngày đầu tiên. Chúng tôi không thể hoàn thành công việc này mà không có tất cả sự làm việc chăm chỉ và hỗ trợ của họ.

Stuart Reges
Marty Stepp

Vị trí của Video Notes trong sách

http://www.pearson.com/cs-resources

Chương 1: Trang 31, 40
Chương 2: Trang 65, 76, 92, 100, 115
Chương 3: Trang 146, 161, 166, 173, 178
Chương 3G: Trang 202, 220
Chương 4: Trang 248, 256, 283
Chương 5: Trang 329, 333, 337, 339, 362
Chương 6: Trang 401, 413, 427
Chương 7: Trang 464, 470, 488, 510
Chương 8: Trang 540, 552, 560, 573
Chương 9: Trang 602, 615, 631
Chương 10: Trang 679, 686, 694
Chương 11: Trang 723, 737, 745
Chương 12: Trang 773, 781, 818
Chương 13: Trang 842, 845, 852
Chương 14: Trang 897, 904
Chương 15: Trang 939, 945, 949
Chương 16: Trang 982, 989, 1002
Chương 17: Trang 1048, 1049, 1059
Chương 18: Trang 1085, 1104

Mục lục ngắn gọn

Chương 1: Giới thiệu về lập trình Java - Trang 1
Chương 2: Dữ liệu sơ cấp và vòng lặp xác định - Trang 63
Chương 3: Giới thiệu về tham số và đối tượng - Trang 142
Phụ lục 3G: Đồ họa (tùy chọn) - Trang 201
Chương 4: Thực thi có điều kiện - Trang 243
Chương 5: Logic chương trình và vòng lặp vô hạn - Trang 320
Chương 6: Xử lý tệp - Trang 392
Chương 7: Mảng - Trang 447
Chương 8: Lớp - Trang 535
Chương 9: Kế thừa và Giao diện - Trang 592
Chương 10: ArrayLists - Trang 667
Chương 11: Java Collections Framework - Trang 722
Chương 12: Đệ quy - Trang 763
Chương 13: Tìm kiếm và Sắp xếp - Trang 840
Chương 14: Ngăn xếp và Hàng đợi - Trang 892
Chương 15: Triển khai một lớp Collection - Trang 931
Chương 16: Danh sách liên kết - Trang 975
Chương 17: Cây nhị phân - Trang 1028
Chương 18: Cấu trúc dữ liệu nâng cao - Trang 1083
Chương 19: Lập trình hàm với Java 8 - Trang 1119

Phụ lục A: Tóm tắt Java - Trang 1161
Phụ lục B: Đặc tả API Java và Ghi chú Javadoc - Trang 1176
Phụ lục C: Cú pháp Java bổ sung - Trang 1182
Mục lục: Trang 1191

Nội dung chi tiết

Chương 1: Giới thiệu về lập trình Java - Trang 1
1.1 Các khái niệm cơ bản về máy tính - Trang 2
Tại sao lập trình? - Trang 2
Phần cứng và Phần mềm - Trang 3
Thế giới số - Trang 4
Quy trình lập trình - Trang 6
Tại sao Java? - Trang 7
Môi trường lập trình Java - Trang 8
1.2 Và bây giờ—Java - Trang 10
Chuỗi kí tự (Strings) - Trang 14
System.out.println - Trang 15
Chuỗi thoát (Escape Sequences) - Trang 15
print so với println - Trang 17
Định danh và từ khóa - Trang 18
Ví dụ phức tạp: DrawFigures1 - Trang 20
Ghi chú và tính dễ đọc - Trang 21
1.3 Lỗi chương trình - Trang 24
Lỗi cú pháp - Trang 24
Lỗi logic (Bugs) - Trang 28
1.4 Phân rã thủ tục - Trang 28
Phương thức tĩnh - Trang 31
Luồng điều khiển - Trang 34
Các phương thức gọi phương thức khác - Trang 36
Một ví dụ về lỗi runtime - Trang 39
1.5 Nghiên cứu trường hợp: DrawFigures - Trang 40
Phiên bản có cấu trúc - Trang 41
Phiên bản cuối cùng không có sự dư thừa - Trang 43
Phân tích luồng thực thi - Trang 44

Chương 2: Dữ liệu sơ cấp và vòng lặp xác định - Trang 63
2.1 Khái niệm cơ bản về dữ liệu - Trang 64
Kiểu dữ liệu nguyên thủy - Trang 64
Biểu thức - Trang 65
JShell - Trang 67
Hằng số - Trang 68
Toán tử số học - Trang 69
Thứ tự ưu tiên - Trang 72
Kết hợp kiểu và Ép kiểu - Trang 74
2.2 Biến - Trang 76
Biến thể của Gán/Khai báo - Trang 81
Ghép nối Chuỗi - Trang 84
Toán tử tăng/giảm - Trang 87
Biến và Kết hợp kiểu - Trang 90
2.3 Vòng lặp for - Trang 92
Dò dấu vòng lặp for - Trang 94
Mẫu vòng lặp for - Trang 98
Vòng lặp for lồng nhau - Trang 100
2.4 Quản lý sự phức tạp - Trang 103
Phạm vi - Trang 103
Mã giả (Pseudocode) - Trang 108
Kỹ thuật bảng - Trang 110
Hằng số lớp - Trang 113
2.5 Nghiên cứu trường hợp: Hình đồng hồ cát - Trang 115
Phân rã vấn đề và Mã giả - Trang 115
Phiên bản có cấu trúc ban đầu - Trang 117
Thêm Hằng số lớp - Trang 119
Các biến thể tiếp theo - Trang 122

Chương 3: Giới thiệu về Tham số và Đối tượng 142

- 3.1 Tham số 143
- Cơ chế của Tham số 146
- Hạn chế của Tham số 150

- Nhiều Tham số 153
- Tham số so với Hằng số 156
- Nạp chồng Phương thức 156

3.2 Phương thức Trả về Giá trị 157

- Lớp Math 158
- Định nghĩa Phương thức Trả về Giá trị 161

3.3 Sử dụng Đối tượng 165

- Đối tượng String 166
- Chương trình Tương tác và Đối tượng Scanner 173
- Ví dụ Chương trình Tương tác 176

3.4 Nghiên cứu trường hợp: Đường bay của Vật thể 178

- Giải pháp Không cấu trúc 182
- Giải pháp Có cấu trúc 184

Phụ lục 3G Đồ họa (Tùy chọn) 201

- 3G.1 Giới thiệu về Đồ họa 202
- DrawingPanel 202
- Vẽ Đường thẳng và Hình dạng 203
- Màu sắc 208
- Vẽ với Vòng lặp 211
- Văn bản và Phông chữ 215
- Hình ảnh 218

3G.2 Phân rã Thủ tục với Đồ họa 220

- Ví dụ Lớn hơn: DrawDiamonds 220

3G.3 Nghiên cứu trường hợp: Kim tự tháp 224

- Giải pháp Một phần Không cấu trúc 224
- Tổng quát hóa việc Vẽ Kim tự tháp 226
- Giải pháp Hoàn chỉnh Có cấu trúc 228

Chương 4: Thực thi Điều kiện 243

- 4.1 Câu lệnh if/else 244
- Toán tử Quan hệ 246
- Lồng câu lệnh if/else 248
- So sánh Đối tượng 255
- Phân tích Câu lệnh if/else 256

- Kiểm tra Nhiều Điều kiện 258

4.2 Thuật toán Tích lũy 259

- Tích lũy Tổng 259
- Vòng lặp Min/Max 261
- Tích lũy Tổng với if 265
- Lỗi làm tròn 267

4.3 Xử lý Văn bản 270

- Kiểu char 270
- char so với int 271
- Thuật toán Tích lũy Văn bản 272
- System.out.printf 274

4.4 Phương thức với Thực thi Điều kiện 279

- Tiền điều kiện và Hậu điều kiện 279
- Ném Ngoại lệ 279
- Xem lại Giá trị Trả về 283
- Suy luận về Các Đường đi 288

4.5 Nghiên cứu trường hợp: Chỉ số Khối Cơ thể (BMI) 290

- Giải pháp Không cấu trúc cho Một Người 291
- Giải pháp Không cấu trúc cho Hai Người 294
- Giải pháp Có cấu trúc cho Hai Người 296
- Kinh nghiệm Thiết kế Thủ tục 300

Chương 5: Logic Chương trình và Vòng lặp Không xác định 320

- 5.1 Vòng lặp while 321
- Vòng lặp Tìm Ước nhỏ nhất 322
- Số ngẫu nhiên 325
- Mô phỏng 329
- Vòng lặp do/while 331

5.2 Thuật toán Cột hàng rào 333

- Cột hàng rào với if 334
- Vòng lặp Sentinel 337

5.3 Kiểu boolean 339

- Toán tử Logic 340

- Đánh giá Ngắn mạch 343
- Biến boolean và Cờ hiệu 348
- Zen của Boolean 350
- Phủ định Biểu thức Boolean 353

5.4 Lỗi Người dùng 354

- Scanner Lookahead 354
- Xử lý Lỗi Người dùng 356

5.5 Khẳng định và Logic Chương trình 358

- Suy luận về Khẳng định 360
- Ví dụ Chi tiết về Khẳng định 362

5.6 Nghiên cứu trường hợp: Đoán số (NumberGuess) 366

- Phiên bản Ban đầu không Có gợi ý 366
- Phiên bản Ngẫu nhiên có Gợi ý 369
- Phiên bản Cuối cùng Mạnh mẽ 372

Chương 6: Xử lý Tệp tin 392

- 6.1 Cơ bản về Đọc Tệp tin 393
- Dữ liệu, Dữ liệu ở Khắp nơi 393
- Tệp tin và Đối tượng Tệp tin 393
- Đọc Tệp tin với Scanner 396

6.2 Chi tiết về Xử lý Dựa trên Token 401

- Cấu trúc của Tệp tin và Tiêu thụ Đầu vào 403
- Tham số Scanner 407
- Đường dẫn và Thư mục 409
- Tệp tin Đầu vào Phức tạp hơn 412

6.3 Xử lý Dựa trên Dòng 413

- Scanner Chuỗi và Kết hợp Dòng/Token 415

6.4 Xử lý Tệp tin Nâng cao 420

- Tệp tin Đầu ra với PrintStream 420
- Đảm bảo Rằng Tệp tin Có thể Đọc được 424

6.5 Nghiên cứu trường hợp: Tra cứu Mã Bưu điện 427

Chương 7: Mảng 447

- 7.1 Cơ bản về Mảng 448
- Tạo và Duyệt qua Mảng 448

- Truy cập vào Mảng 452
- Khởi tạo Mảng 455
- Chương trình Mảng Hoàn chỉnh 456
- Truy cập Ngẫu nhiên 461
- Mảng và Phương thức 464
- Vòng lặp For-Each 467
- Lớp Arrays 468

7.2 Thuật toán Duyệt qua Mảng 470

- In Mảng 471
- Tìm kiếm và Thay thế 473
- Kiểm tra Sự bằng nhau 475
- Đảo ngược Mảng 477
- Thuật toán Duyệt qua Chuỗi 481
- Tiếp cận Theo Chức năng 482

7.3 Tham chiếu Ngữ nghĩa 484

- Nhiều Đối tượng 486

7.4 Kỹ thuật Mảng Nâng cao 488

- Dịch Chuyển Giá trị trong Mảng 488
- Mảng của Đối tượng 493
- Tham số Dòng Lệnh 494
- Thuật toán Vòng lặp Lồng nhau 495

7.5 Mảng Đa chiều 497

- Mảng Hai chiều Hình chữ nhật 497
- Mảng Jagged 499

7.6 Mảng của Pixel 504

7.7 Nghiên cứu Trường hợp: Luật Benford 509

- Tính Tổng Giá trị 510
- Hoàn thành Chương trình 514

Chương 8: Lớp 535

- 8.1 Lập trình Hướng đối tượng 536
- Lớp và Đối tượng 537
- Đối tượng Point 539

8.2 Trạng thái và Hành vi Đối tượng 540

- Trạng thái Đối tượng: Thuộc tính 541
- Hành vi Đối tượng: Phương thức 543
- Tham số Ngầm định 546
- Phương thức Thay đổi và Truy cập 548
- Phương thức toString 550

8.3 Khởi tạo Đối tượng: Constructor 552
- Từ khóa this 557
- Nhiều Constructor 559

8.4 Đóng gói 560
- Thuộc tính Private 561
- Bất biến của Lớp 567
- Thay đổi Thực thi Nội bộ 571

8.5 Nghiên cứu Trường hợp: Thiết kế Lớp Stock 573
- Kinh nghiệm Thiết kế Hướng đối tượng 574
- Thuộc tính và Tiêu đề Phương thức của Stock 576
- Thực thi Phương thức và Constructor của Stock 578

Chương 9: Kế thừa và Interface 592
- 9.1 Cơ bản về Kế thừa 593
- Các Hệ thống Phân cấp không Lập trình 594
- Mở rộng Lớp 596
- Ghi đè Phương thức 600

9.2 Tương tác với Lớp Cha 602
- Gọi Phương thức Ghi đè 602
- Truy cập Thuộc tính Thừa kế 603
- Gọi Constructor của Lớp Cha 605
- Hành vi của DividendStock 607
- Lớp Object 609
- Phương thức equals 610
- Từ khóa instanceof 613

9.3 Đa hình 615
- Cơ chế Đa hình 618
- Giải thích Mã Kế thừa 620
- Giải thích Lời gọi Phức tạp 622

9.4 Kế thừa và Thiết kế 625
- Sử dụng Sai Kế thừa 625
- Quan hệ Is-a và Has-a 628
- Graphics2D 629

9.5 Interface 631
- Một Interface cho Hình dạng 632
- Thực thi Interface 634
- Lợi ích của Interface 637

9.6 Nghiên cứu Trường hợp: Hệ thống Phân cấp Lớp Tài chính 639
- Thiết kế Các Lớp 640
- Thực thi Dư thừa 644
- Lớp Trừu tượng 647

Chương 10: ArrayLists 667
- 10.1 ArrayLists 668
- Các Hoạt động Cơ bản trên ArrayList 669
- Phương thức Tìm kiếm ArrayList 674
- Chương trình ArrayList Hoàn chỉnh 677
- Thêm vào và Xóa khỏi ArrayList 679
- Khởi tạo ArrayList 683
- Sử dụng Vòng lặp For-Each với ArrayList 684
- Lớp Wrapper 686

10.2 Interface Comparable 689
- Sắp xếp Tự nhiên và compareTo 691
- Thực thi Interface Comparable 694

10.3 Nghiên cứu Trường hợp: Bỏ phiếu Lựa chọn Xếp hạng 701
- Lớp Ballot 702
- Đếm Phiếu 705
- Nhiều Vòng 709**
- **Chương 11: Khung làm việc Java Collections 722**
- 11.1 Danh sách 723
- Collections 723

- So sánh LinkedList và ArrayList 724
- Iterators 727
- Kiểu Dữ liệu Trừu tượng (ADTs) 731
- Nghiên cứu Trường hợp LinkedList: Sieve 734
- **11.2 Sets 737**
- Khái niệm về Set 738
- So sánh TreeSet và HashSet 740
- Các thao tác trên Set 741
- Nghiên cứu Trường hợp Set: Lottery 743
- **11.3 Maps 745**
- Các thao tác cơ bản trên Map 746
- Các Lượt xem của Map (keySet và values) 748
- So sánh TreeMap và HashMap 749
- Nghiên cứu Trường hợp Map: WordCount 750
- Tổng quan về Collection 753
- **Chương 12: Đệ quy 763**
- 12.1 Suy nghĩ Đệ quy 764
- Ví dụ không lập trình 764
- Chuyển đổi Giải pháp Lặp thành Đệ quy 767
- Cấu trúc của Giải pháp Đệ quy 769
- **12.2 Một Ví dụ Tốt hơn về Đệ quy 771**
- Cơ chế của Đệ quy 773
- **12.3 Hàm và Dữ liệu Đệ quy 781**
- Lũy thừa Số nguyên 781
- Ước chung lớn nhất 784
- Duyệt Thư mục 790
- Phương thức Hỗ trợ 794
- **12.4 Đồ họa Đệ quy 797**
- **12.5 Backtracking Đệ quy 801**
- Ví dụ Đơn giản: Đi về Phía Bắc/Đông 802
- Bài toán 8 Quân Hậu 807
- Giải các Câu đố Sudoku 814
- **12.6 Nghiên cứu Trường hợp: Bộ tính Toán tiền tố 818**

- Ký hiệu Infix, Prefix, và Postfix 818
- Đánh giá các Biểu thức Prefix 819
- Chương trình Hoàn chỉnh 822
- **Chương 13: Tìm kiếm và Sắp xếp 840**
- 13.1 Tìm kiếm và Sắp xếp trong Thư viện Lớp Java 841
- Tìm kiếm Nhị phân 842
- Sắp xếp 845
- Trộn 846
- Sắp xếp Tùy chỉnh với Comparators 848
- **13.2 Độ phức tạp của Chương trình 852**
- Phân tích Thực nghiệm 855
- Các Lớp Độ phức tạp 858
- **13.3 Thực thi Thuật toán Tìm kiếm và Sắp xếp 861**
- Tìm kiếm Tuần tự 861
- Tìm kiếm Nhị phân 862
- Tìm kiếm Nhị phân Đệ quy 865
- Tìm kiếm Đối tượng 868
- Sắp xếp Lựa chọn 869
- **13.4 Nghiên cứu Trường hợp: Thực thi Merge Sort 873**
- Chia và Gộp Mảng 873
- Merge Sort Đệ quy 876
- Chương trình Hoàn chỉnh 879
- **Chương 14: Stacks và Queues 892**
- 14.1 Khái niệm Cơ bản về Stack/Queue 893
- Khái niệm về Stack 893
- Khái niệm về Queue 896
- **14.2 Các Thao tác Thông thường trên Stack/Queue 897**
- Chuyển đổi giữa Stacks và Queues 899
- Tổng của một Queue 900
- Tổng của một Stack 901
- **14.3 Các Thao tác Phức tạp trên Stack/Queue 904**
- Loại bỏ Giá trị khỏi Queue 904
- So sánh Hai Stacks để Tìm Sự tương đồng 906

- **14.4 Nghiên cứu Trường hợp: Bộ tính Biểu thức 908**
- Phân tách thành Các Token 909
- Bộ tính 914

Chương 15: Triển khai một Lớp Bộ Sưu Tập 931 15.1 Simple ArrayIntList 932
Thêm và In 932
Suy nghĩ về Đóng gói 938
Xử lý Phần Giữa của Danh sách 939
Một Constructor khác và một Hằng số 944
Điều kiện Tiên quyết và Điều kiện Hậu 945
15.2 Một ArrayIntList Hoàn thiện hơn 949
Ném Ngoại lệ 949
Phương thức Tiện lợi 952
15.3 Các Tính năng Nâng cao 955
Thay đổi Kích thước Khi Cần thiết 955
Thêm một Iterator 957
15.4 ArrayList<E> 963
Chương 16: Danh sách Liên kết 975
16.1 Làm việc với Nodes 976
Xây dựng một Danh sách 977
Các Khái niệm Cơ bản về Danh sách 979
Thao tác với Nodes 982
Duyệt qua Danh sách 985
16.2 Một Lớp Danh sách Liên kết 989
Simple LinkedIntList 989
Append add 991
Phần Giữa của Danh sách 995
16.3 Một Hoạt động Phức tạp với Danh sách 1002
Phương pháp Inchworm 1007
16.4 Giao diện IntList 1008
16.5 LinkedList<E> 1011
Biến thể của Danh sách Liên kết 1012
Các Iterator của Danh sách Liên kết 1015

Các Chi tiết Mã Khác 1017
Chương 17: Cây Nhị Phân 1028
17.1 Các Khái niệm Cơ bản về Cây Nhị Phân 1029
Lớp Node và Cây 1032
17.2 Duyệt Cây 1033
Xây dựng và Xem Cây 1039
17.3 Các Hoạt động Thường gặp với Cây 1048
Tổng của Cây 1048
Đếm Số Mức 1049
Đếm Lá 1051
17.4 Cây Tìm Kiếm Nhị Phân 1052
Thuộc tính của Cây Tìm Kiếm Nhị Phân 1053
Xây dựng Cây Tìm Kiếm Nhị Phân 1055
Mẫu x = change(x) 1059
Tìm kiếm trong Cây 1062
Độ Phức tạp của Cây Tìm Kiếm Nhị Phân 1066
17.5 SearchTree<E> 1067
Chương 18: Cấu trúc Dữ liệu Nâng cao 1083
18.1 Hashing 1084
Các Triển khai Bộ Set Bằng Mảng 1084
Hàm Băm và Bảng Băm 1085
Xung đột 1087
Rehashing 1092
Hashing Dữ liệu Không phải Số nguyên 1095
Triển khai Hash Map 1098
18.2 Hàng đợi Ưu tiên và Heaps 1099
Hàng đợi Ưu tiên 1099
Giới thiệu về Heaps 1101
Xóa khỏi Heap 1103
Thêm vào Heap 1104
Triển khai Heap Bằng Mảng 1106
Heap Sort 1110
Chương 19: Lập trình Hàm với Java 8 1119

19.1 Lập trình Không có Tác dụng Phụ 1120
19.2 Hàm là Đối tượng Hạng Nhất 1123
Biểu thức Lambda 1126
19.3 Streams 1129
Ý tưởng Cơ bản 1129
Sử dụng Map 1131
Sử dụng Filter 1132
Sử dụng Reduce 1134
Kết quả Tùy chọn 1135
19.4 Khóa Hàm 1136
19.5 Các Hoạt động Bậc Cao trên Bộ Sưu Tập 1139
Làm việc với Mảng 1140
Làm việc với Danh sách 1141
Làm việc với Tệp tin 1145
19.6 Nghiên cứu Trường hợp: Số Hoàn Hảo 1146
Tính Tổng 1147
Kết hợp Căn bậc Hai 1150
Chỉ Năm và Tận dụng Tính Song Song 1153
Phụ lục A: Tóm tắt Java 1161
Phụ lục B: Đặc tả API Java và Nhận xét Javadoc 1176
Phụ lục C: Cú pháp Java Bổ sung 1182
Mục lục 1191

Chương 1: Giới thiệu về Lập trình Java

1.1 Các Khái niệm Cơ bản về Máy tính
- Tại sao cần Lập trình?
- Phần cứng và Phần mềm
- Thế giới Số
- Quá trình Lập trình
- Tại sao lại là Java?
- Môi trường Lập trình Java

1.2 Và bây giờ—Java
- Chuỗi ký tự (Strings)
- System.out.println
- Các Chuỗi Ký tự Thoát
- Sự khác biệt giữa print và println
- Tên định danh và Từ khóa
- Một Ví dụ Phức tạp: DrawFigures1
- Nhận xét và Tính Đọc được

1.3 Lỗi trong Chương trình
- Lỗi Cú pháp
- Lỗi Logic (Bugs)

1.4 Phân rã Thủ tục
- Các Phương thức tĩnh
- Dòng Điều khiển
- Các Phương thức Gọi Phương thức Khác
- Một Ví dụ về Lỗi Runtime

1.5 Nghiên cứu Trường hợp: DrawFigures
- Phiên bản Cấu trúc
- Phiên bản Cuối cùng không có sự Trùng lặp
- Phân tích Dòng chảy của Thực thi

Giới thiệu

Chương này bắt đầu với việc xem xét một số thuật ngữ cơ bản về máy tính và lập trình máy tính. Nhiều khái niệm này sẽ xuất hiện trong các chương sau, vì vậy việc xem lại chúng trước khi bắt đầu tìm hiểu chi tiết về cách lập trình bằng Java sẽ rất hữu ích.
Chúng ta sẽ bắt đầu khám phá Java bằng cách xem xét các chương trình đơn giản tạo ra đầu ra. Cuộc thảo luận này sẽ cho phép chúng ta khám phá nhiều yếu tố chung của tất cả các chương trình Java, trong khi làm việc với các chương trình có cấu trúc tương đối đơn giản.
Sau khi đã xem xét các yếu tố cơ bản của chương trình Java, chúng ta sẽ khám phá kỹ thuật phân rã thủ tục bằng cách học cách chia nhỏ một chương trình Java thành nhiều phương thức. Sử dụng kỹ thuật này, chúng ta có thể chia nhỏ các nhiệm vụ phức tạp thành các nhiệm vụ con dễ quản lý hơn và tránh được sự trùng lặp trong các giải pháp chương trình.

1.1 Các Khái niệm Cơ bản về Máy tính

Máy tính hiện diện khắp nơi trong cuộc sống hàng ngày của chúng ta, và nhờ vào Internet, chúng mang lại cho chúng ta truy cập vào nguồn thông tin gần như vô hạn. Một số thông tin này là tin tức cần thiết, như tiêu đề trên trang web tin tức yêu thích của bạn. Máy tính cho phép chúng ta chia sẻ ảnh với gia đình và tìm đường đến tiệm pizza gần nhất cho bữa tối.
Nhiều vấn đề trong thế giới thực đang được giải quyết bởi máy tính, một số trong đó không giống nhiều với chiếc máy tính trên bàn hoặc trên đùi của bạn. Máy tính cho phép chúng ta giải trình tự gen người và tìm kiếm các mẫu DNA trong đó. Máy tính trong những chiếc xe hơi được sản xuất gần đây giám sát trạng thái và chuyển động của mỗi chiếc xe, và máy tính đang giúp một số

chiếc xe tự lái. Các máy nghe nhạc kỹ thuật số và các thiết bị di động như iPhone của Apple thực sự có máy tính bên trong vỏ nhỏ của chúng. Thậm chí, robot hút bụi Roomba còn chứa một máy tính với các chỉ dẫn phức tạp về cách tránh đồ nội thất khi làm sạch sàn của bạn.
Nhưng điều gì làm cho một chiếc máy tính trở thành máy tính? Một chiếc máy tính bỏ túi có phải là máy tính không? Một con người với giấy và bút có phải là máy tính không? Các phần tiếp theo sẽ cố gắng trả lời câu hỏi này trong khi giới thiệu một số thuật ngữ cơ bản giúp bạn chuẩn bị học lập trình.

Tại sao cần Lập trình?

Tại hầu hết các trường đại học, khóa học đầu tiên về khoa học máy tính là một khóa học lập trình. Nhiều nhà khoa học máy tính cảm thấy không thoải mái với điều này vì nó khiến mọi người nghĩ rằng khoa học máy tính chỉ là lập trình. Mặc dù đúng là nhiều nhà khoa học máy tính được đào tạo dành nhiều thời gian để lập trình, nhưng còn rất nhiều khía cạnh khác trong lĩnh vực này. Vậy tại sao chúng ta lại học lập trình trước?

Một nhà khoa học máy tính ở Stanford tên là Don Knuth đã trả lời câu hỏi này bằng cách nói rằng điểm chung của hầu hết các nhà khoa học máy tính là chúng ta đều làm việc với các thuật toán theo cách nào đó.

Thuật toán

Là mô tả từng bước về cách thực hiện một nhiệm vụ.

Knuth là một chuyên gia về thuật toán, vì vậy ông tự nhiên có xu hướng coi chúng là trung tâm của khoa học máy tính. Tuy nhiên,

ông khẳng định rằng điều quan trọng nhất không phải là các thuật toán, mà là quá trình tư duy mà các nhà khoa học máy tính sử dụng để phát triển chúng. Theo Knuth,

"Người ta thường nói rằng một người không thực sự hiểu điều gì đó cho đến khi dạy nó cho người khác. Thực ra, một người không thực sự hiểu điều gì đó cho đến khi dạy nó cho một chiếc máy tính, tức là thể hiện nó dưới dạng một thuật toán."
(Knuth, Don. *Selected Papers on Computer Science*. Stanford, CA: Center for the Study of Language and Information, 1996.)

Knuth đang mô tả một quá trình tư duy chung cho hầu hết khoa học máy tính, mà ông gọi là tư duy thuật toán. Chúng ta học lập trình không phải vì đó là khía cạnh quan trọng nhất của khoa học máy tính, mà vì đó là cách tốt nhất để giải thích cách tiếp cận mà các nhà khoa học máy tính sử dụng để giải quyết vấn đề.

Khái niệm về thuật toán giúp hiểu được máy tính là gì và khoa học máy tính thực chất là gì. Một từ điển lớn định nghĩa từ "computer" (máy tính) là "một thiết bị tính toán." Theo định nghĩa này, mọi loại thiết bị đều có thể được coi là máy tính, bao gồm máy tính cầm tay, hệ thống định vị GPS, và đồ chơi trẻ em như Furby. Trước khi máy tính điện tử ra đời, con người thường được gọi là "computers." Ví dụ, nhà toán học thế kỷ 19 Charles Peirce ban đầu được thuê để làm việc cho chính phủ Mỹ với tư cách là "Trợ lý Máy tính" vì công việc của ông liên quan đến thực hiện các tính toán toán học.

Như vậy, theo nghĩa rộng, từ "computer" có thể áp dụng cho nhiều thiết bị. Nhưng khi các nhà khoa học máy tính đề cập đến một máy tính, chúng ta thường nghĩ đến một thiết bị tính toán phổ quát có thể được lập trình để thực hiện bất kỳ thuật toán nào. Do đó, khoa học máy tính là nghiên cứu về các thiết bị tính toán và nghiên cứu về chính quá trình tính toán, bao gồm cả thuật toán.

Thuật toán được thể hiện dưới dạng chương trình máy tính, và đó là nội dung mà cuốn sách này sẽ đề cập đến. Nhưng trước khi chúng ta xem cách lập trình, sẽ rất hữu ích nếu xem lại một số khái niệm cơ bản về máy tính.

Phần cứng và Phần mềm

Một máy tính là một cỗ máy thao tác dữ liệu và thực hiện các danh sách lệnh được gọi là chương trình.

Chương trình

Là danh sách các lệnh để máy tính thực hiện.

Một đặc điểm chính phân biệt máy tính với một cỗ máy đơn giản hơn như máy tính bỏ túi là tính linh hoạt của nó. Cùng một chiếc máy tính có thể thực hiện nhiều nhiệm vụ khác nhau (chơi game, tính toán thuế thu nhập, kết nối với các máy tính khác trên thế giới), tùy thuộc vào chương trình mà nó đang chạy vào thời điểm đó. Máy tính có thể chạy không chỉ các chương trình hiện có trên nó, mà còn cả các chương trình mới chưa được viết ra.

Các thành phần vật lý tạo nên một chiếc máy tính được gọi chung là phần cứng. Một trong những phần cứng quan trọng nhất là bộ xử lý trung tâm, hay còn gọi là CPU. CPU là "bộ não" của máy tính: Nó thực thi các lệnh. Cũng quan trọng là bộ nhớ của máy tính (thường được gọi là bộ nhớ truy cập ngẫu nhiên, hay RAM, vì máy tính có thể truy cập bất kỳ phần nào của bộ nhớ đó vào bất kỳ thời điểm nào). Máy tính sử dụng bộ nhớ của nó để lưu trữ các chương trình đang được thực thi, cùng với dữ liệu của chúng. RAM có dung lượng hạn chế và không giữ lại nội dung của nó khi máy tính tắt. Do đó, các máy tính thường sử dụng ổ đĩa cứng như một khu vực lưu trữ lâu dài lớn hơn.

Các chương trình máy tính được gọi chung là phần mềm. Phần mềm chính chạy trên máy tính là hệ điều hành. Hệ điều hành cung cấp một môi trường trong đó nhiều chương trình có thể chạy đồng thời; nó cũng cung cấp cầu nối giữa các chương trình, phần cứng và người dùng (người sử dụng máy tính). Các chương trình chạy trong hệ điều hành thường được gọi là ứng dụng.

Khi người dùng chọn một chương trình để hệ điều hành chạy (ví dụ, bằng cách nhấp đúp vào biểu tượng của chương trình trên màn hình), một số điều xảy ra: Các lệnh cho chương trình đó được tải vào bộ nhớ của máy tính từ ổ đĩa cứng, hệ điều hành phân bổ bộ nhớ cho chương trình sử dụng, và các lệnh để chạy chương trình được đưa từ bộ nhớ vào CPU và thực thi tuần tự.

Thế giới Kỹ thuật số

Trong phần trước, chúng ta đã thấy rằng máy tính là một thiết bị đa năng có thể được lập trình. Bạn thường nghe mọi người gọi máy tính hiện đại là máy tính số vì cách chúng hoạt động.

Kỹ thuật số

Dựa trên các con số tăng theo các bước rời rạc, chẳng hạn như các số nguyên 0, 1, 2, 3, v.v.

Vì máy tính là kỹ thuật số, nên mọi thứ được lưu trữ trên máy tính đều được lưu dưới dạng một chuỗi các số nguyên. Điều này bao gồm mọi chương trình và mọi dữ liệu. Ví dụ, một tệp MP3 đơn giản chỉ là một chuỗi dài các số nguyên lưu trữ thông tin âm thanh. Ngày nay, chúng ta đã quen với nhạc số, hình ảnh số và phim số, nhưng vào những năm 1940, khi những chiếc máy tính

đầu tiên được chế tạo, ý tưởng lưu trữ dữ liệu phức tạp dưới dạng số nguyên là khá mới lạ.

Không chỉ là máy tính kỹ thuật số, lưu trữ mọi thông tin dưới dạng số nguyên, mà chúng còn là nhị phân, nghĩa là chúng lưu trữ các số nguyên dưới dạng các số nhị phân.

Số nhị phân

Một con số chỉ bao gồm các chữ số 0 và 1, còn được gọi là số hệ nhị phân.

Con người thường làm việc với các số thập phân hoặc số hệ 10, phù hợp với sinh lý của chúng ta (10 ngón tay và 10 ngón chân). Tuy nhiên, khi chúng ta thiết kế những chiếc máy tính đầu tiên, chúng ta muốn có các hệ thống dễ tạo ra và rất đáng tin cậy. Hóa ra, việc xây dựng các hệ thống này trên cơ sở hiện tượng nhị phân (ví dụ, mạch điện đóng hoặc mở) đơn giản hơn so với có 10 trạng thái khác nhau mà cần phải phân biệt (ví dụ, 10 mức điện áp khác nhau).

Từ góc độ toán học, bạn có thể lưu trữ mọi thứ bằng số nhị phân cũng dễ dàng như bạn có thể sử dụng số hệ 10. Nhưng vì dễ dàng hơn để xây dựng một thiết bị vật lý sử dụng số nhị phân, nên đó là lý do tại sao máy tính sử dụng chúng.

Tuy nhiên, điều này cũng có nghĩa là những người không quen với máy tính sẽ thấy những quy ước này không quen thuộc. Vì vậy, đáng để dành một chút thời gian để xem lại cách các số nhị phân hoạt động. Để đếm bằng số nhị phân, giống như với số hệ 10, bạn bắt đầu từ 0 và đếm lên, nhưng bạn sẽ hết chữ số nhanh hơn nhiều. Vì vậy, khi đếm bằng nhị phân, bạn nói:

0

1

Và bạn đã hết chữ số. Điều này giống như việc đạt đến số 9 khi bạn đếm bằng hệ 10. Sau khi hết chữ số, bạn chuyển sang chữ số tiếp theo. Vì vậy, hai số nhị phân tiếp theo là:

10

11

Và một lần nữa, bạn đã hết chữ số. Điều này giống như việc đạt đến số 99 trong hệ 10. Một lần nữa, bạn chuyển sang chữ số tiếp theo để tạo thành số ba chữ số 100. Trong hệ nhị phân, bất cứ khi nào bạn thấy một dãy các số 1, chẳng hạn như 111111, bạn biết rằng bạn chỉ còn một bước nữa là các chữ số đó sẽ chuyển thành 0 với một chữ số 1 thêm vào phía trước, giống như trong hệ 10, khi bạn thấy một con số như 999999, bạn biết rằng bạn chỉ còn một bước nữa là tất cả các chữ số đó sẽ chuyển thành 0 với một chữ số 1 thêm vào phía trước.

Bảng 1.1 cho thấy cách đếm đến số 8 trong hệ 10 bằng cách sử dụng hệ nhị phân.

Bảng 1.1 Số Thập phân so với Số Nhị phân

Thập phân	**Nhị phân**
0	0
1	1
2	10

Thập phân	Nhị phân
3	11
4	100
5	101
6	110
7	111
8	1000

Chúng ta có thể rút ra một số nhận xét hữu ích về các số nhị phân. Hãy chú ý trong bảng rằng các số nhị phân 1, 10, 100, và 1000 đều là các lũy thừa hoàn hảo của 2 (2^0, 2^1, 2^2, 2^3). Tương tự như trong hệ 10, chúng ta nói về các chữ số hàng đơn vị, hàng chục, hàng trăm, v.v., chúng ta có thể nghĩ đến trong hệ nhị phân các chữ số hàng đơn vị, hàng hai, hàng bốn, hàng tám, hàng mười sáu, v.v.

Các nhà khoa học máy tính nhanh chóng nhận thấy cần phải đề cập đến kích thước của các lượng nhị phân khác nhau, vì vậy họ đã phát minh ra thuật ngữ "bit" để chỉ một chữ số nhị phân đơn và thuật ngữ "byte" để chỉ 8 bit. Để nói về các đơn vị bộ nhớ lớn, họ đã phát minh ra các thuật ngữ "kilobyte" (KB), "megabyte" (MB), "gigabyte" (GB), v.v. Nhiều người nghĩ rằng các đơn vị này tương ứng với hệ mét, trong đó "kilo" có nghĩa là 1000, nhưng điều đó chỉ đúng một phần. Chúng ta sử dụng thực tế là 2^10 xấp xỉ bằng 1000 (thực tế là bằng 1024). Bảng 1.2 cho thấy một số đơn vị lưu trữ bộ nhớ phổ biến:

Bảng 1.2 Các Đơn vị Lưu trữ Bộ nhớ

Đơn vị đo lường	Lũy thừa của 2	Giá trị thực tế	Ví dụ
kilobyte (KB)	2^10	1024	Bài viết 500 từ (3 KB)
megabyte (MB)	2^20	1.048.576	Sách điển hình (1 MB) hoặc bài hát (5 MB)
gigabyte (GB)	2^30	1.073.741.824	Phim điển hình (4,7 GB)
terabyte (TB)	2^40	1.099.511.627.776	20 triệu cuốn sách trong Thư viện Quốc hội (20 TB)
petabyte (PB)	2^50	1.125.899.906.842.624	10 tỷ bức ảnh kỹ thuật số (1,5 PB)

Quy trình Lập trình

Từ "code" mô tả các đoạn mã chương trình ("bốn dòng mã này") hoặc hành động lập trình ("Hãy lập trình đoạn này bằng Java"). Khi một chương trình đã được viết xong, bạn có thể thực thi nó.

Thực thi Chương trình

Hành động thực hiện các lệnh chứa trong một chương trình. Quá trình thực thi thường được gọi là "chạy". Thuật ngữ này cũng có thể được sử dụng như một động từ ("Khi chương trình của tôi chạy, nó có gì đó lạ") hoặc như một danh từ ("Lần chạy cuối của chương trình của tôi đã tạo ra các kết quả này").

Một chương trình máy tính được lưu trữ bên trong dưới dạng một chuỗi các số nhị phân, được gọi là ngôn ngữ máy của máy tính. Vào những ngày đầu, các lập trình viên nhập trực tiếp các số như thế này vào máy tính. Rõ ràng, đây là một cách lập trình máy tính tẻ nhạt và khó hiểu, và chúng ta đã phát minh ra nhiều cơ chế để đơn giản hóa quy trình này.

Các lập trình viên hiện đại viết bằng các ngôn ngữ lập trình bậc cao, chẳng hạn như Java. Các chương trình như vậy không thể chạy trực tiếp trên máy tính: Chúng phải được dịch sang một dạng khác bởi một chương trình đặc biệt gọi là trình biên dịch.

Trình biên dịch

Một chương trình dịch một chương trình máy tính được viết bằng một ngôn ngữ sang một chương trình tương đương bằng một ngôn ngữ khác (thường, nhưng không phải luôn luôn, dịch từ ngôn ngữ bậc cao sang ngôn ngữ máy). Một trình biên dịch dịch trực tiếp sang ngôn ngữ máy sẽ tạo ra một chương trình có thể thực thi trực tiếp trên máy tính, được gọi là tệp thực thi. Chúng ta gọi các trình biên dịch như vậy là trình biên dịch "native" vì chúng biên dịch mã xuống mức thấp nhất có thể (ngôn ngữ máy native của máy tính).

Cách tiếp cận này hoạt động tốt khi bạn biết chính xác máy tính nào bạn muốn sử dụng để chạy chương trình của mình. Nhưng nếu bạn muốn thực thi chương trình trên nhiều máy tính khác nhau thì sao? Bạn sẽ cần một trình biên dịch tạo ra các kết quả

ngôn ngữ máy khác nhau cho từng loại máy tính. Các nhà thiết kế Java đã quyết định sử dụng một cách tiếp cận khác. Họ rất quan tâm đến việc các chương trình của mình có thể chạy trên nhiều máy tính khác nhau, vì họ muốn tạo ra một ngôn ngữ hoạt động tốt trên web.

Thay vì biên dịch sang ngôn ngữ máy, các chương trình Java biên dịch thành cái được gọi là bytecode Java. Một bộ bytecode có thể thực thi trên nhiều máy khác nhau. Những bytecode này đại diện cho một mức trung gian: Chúng không phải là ngôn ngữ bậc cao như Java hoặc ngôn ngữ máy thấp. Thực tế, chúng là ngôn ngữ máy của một máy tính lý thuyết gọi là Máy ảo Java (JVM).

Máy ảo Java

Một máy tính lý thuyết mà ngôn ngữ máy của nó là tập hợp các bytecode Java. JVM không phải là một máy thực sự, nhưng nó tương tự như một máy tính. Khi chúng ta biên dịch các chương trình đến mức này, không còn nhiều công việc còn lại để chuyển các bytecode Java thành các lệnh máy thực tế.

Để thực thi một chương trình Java, bạn cần một chương trình khác sẽ thực thi các bytecode Java. Các chương trình như vậy được gọi chung là các "runtime Java", và môi trường tiêu chuẩn được Oracle Corporation phân phối được gọi là Môi trường Runtime Java (JRE).

Runtime Java

Một chương trình thực thi các bytecode Java đã biên dịch. Hầu hết mọi người đều có runtime Java trên máy tính của mình, ngay cả khi họ không biết về nó. Ví dụ, Mac OS X của Apple bao gồm

một runtime Java, và nhiều ứng dụng Windows cũng cài đặt một runtime Java.

Tại sao chọn Java?

Khi Sun Microsystems phát hành Java vào năm 1995, họ đã công bố một tài liệu gọi là "white paper" mô tả ngôn ngữ lập trình mới của mình. Câu quan trọng nhất từ tài liệu đó có thể là:

Java: Một ngôn ngữ đơn giản, hướng đối tượng, am hiểu mạng, thông dịch, mạnh mẽ, an toàn, trung lập về kiến trúc, di động, hiệu năng cao, đa luồng và động.

Câu này bao gồm nhiều lý do tại sao Java là một ngôn ngữ lập trình nhập môn tốt. Đầu tiên, Java khá đơn giản cho người mới học và nó tiếp cận lập trình hướng đối tượng, một phong cách viết chương trình đã được chứng minh là rất thành công trong việc tạo ra các hệ thống phần mềm lớn và phức tạp.

Java cũng bao gồm một lượng lớn phần mềm đã được viết sẵn mà các lập trình viên có thể sử dụng để nâng cao chương trình của mình. Những thành phần phần mềm có sẵn này thường được gọi là thư viện. Ví dụ, nếu bạn muốn viết một chương trình kết nối với một trang web trên Internet, Java có sẵn một thư viện để đơn giản hóa việc kết nối cho bạn. Java có các thư viện để vẽ giao diện người dùng đồ họa (GUI), truy xuất dữ liệu từ cơ sở dữ liệu và thực hiện các phép tính toán phức tạp, cùng nhiều chức năng khác. Những thư viện này được gọi chung là thư viện lớp Java.

Thư viện lớp Java

Tập hợp các mã Java có sẵn cung cấp các giải pháp cho các vấn đề lập trình thông thường. Sự phong phú của các thư viện lớp Java là một yếu tố cực kỳ quan trọng trong sự phát triển của Java thành một ngôn ngữ phổ biến. Các thư viện lớp Java trong phiên bản 10 bao gồm hơn 6000 mục.

Một lý do khác để sử dụng Java là nó có một cộng đồng lập trình viên sôi động. Tài liệu trực tuyến và các hướng dẫn phong phú có sẵn để giúp lập trình viên học các kỹ năng mới. Nhiều tài liệu trong số này được viết bởi Oracle, bao gồm một tài liệu tham khảo toàn diện về thư viện lớp Java gọi là Đặc tả API (API là viết tắt của Giao diện lập trình ứng dụng).

Java cực kỳ độc lập với nền tảng; không giống như các chương trình viết bằng nhiều ngôn ngữ khác, cùng một chương trình Java có thể được thực thi trên nhiều hệ điều hành khác nhau, chẳng hạn như Windows, Linux và Mac OS X.

Java được sử dụng rộng rãi cho cả nghiên cứu và ứng dụng kinh doanh, điều đó có nghĩa là có rất nhiều công việc lập trình hiện có trên thị trường cho các lập trình viên Java có kỹ năng. Một tìm kiếm trên Google với cụm từ "Java jobs" trả về khoảng 816 triệu kết quả tại thời điểm viết tài liệu này.

Môi trường lập trình Java

Bạn phải làm quen với cài đặt máy tính của mình trước khi bắt đầu lập trình. Mỗi máy tính cung cấp một môi trường phát triển chương trình khác nhau, nhưng có một số yếu tố chung đáng được lưu ý. Dù bạn sử dụng môi trường nào, bạn sẽ tuân theo ba bước cơ bản:

1. Nhập một chương trình dưới dạng một lớp Java.

2. Biên dịch tệp chương trình.

3. Chạy phiên bản đã biên dịch của chương trình.

Đơn vị lưu trữ cơ bản trên hầu hết các máy tính là một tệp. Mỗi tệp có một tên. Tên tệp kết thúc bằng một phần mở rộng, là phần của tên tệp theo sau dấu chấm. Phần mở rộng của tệp cho biết loại dữ liệu chứa trong tệp. Ví dụ, các tệp có phần mở rộng .doc là tài liệu Microsoft Word và các tệp có phần mở rộng .mp3 là tệp âm thanh MP3.

Các tệp chương trình Java mà bạn tạo phải sử dụng phần mở rộng .java. Khi bạn biên dịch một chương trình Java, bytecode Java kết quả được lưu trong một tệp có cùng tên và phần mở rộng .class.

Hầu hết các lập trình viên Java sử dụng cái được gọi là Môi trường Phát triển Tích hợp, hoặc IDE, cung cấp một môi trường tất cả trong một để tạo, chỉnh sửa, biên dịch và thực thi các tệp chương trình. Một số lựa chọn phổ biến cho các lớp học nhập môn khoa học máy tính là Eclipse, IntelliJ, NetBeans, jGRASP, DrJava, BlueJ và TextPad. Giáo viên của bạn sẽ cho bạn biết môi trường nào bạn nên sử dụng.

Hãy thử nhập chương trình đơn giản sau trong IDE của bạn (số dòng không phải là một phần của chương trình mà chỉ để giúp bạn theo dõi):

```
1 public class Hello {

2     public static void main(String[] args) {

3         System.out.println("Hello, world!");
```

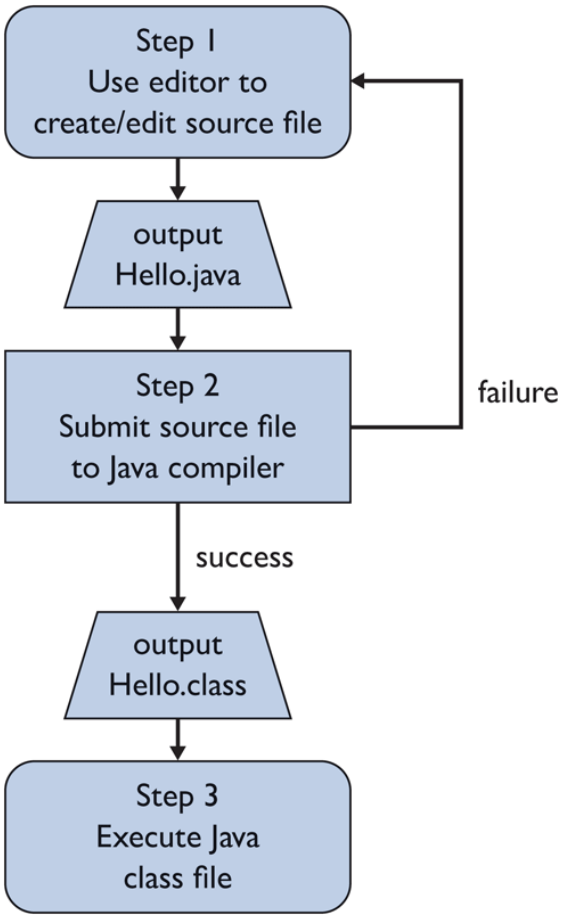
```
4     }

5 }
```

Đừng lo lắng về các chi tiết của chương trình này ngay bây giờ. Chúng ta sẽ khám phá chúng trong phần tiếp theo.

Khi bạn đã tạo tệp chương trình của mình, chuyển sang bước 2 và biên dịch nó. Lệnh để biên dịch sẽ khác nhau trong mỗi môi trường phát triển, nhưng quy trình là giống nhau (các lệnh phổ biến là "compile" hoặc "build"). Nếu có bất kỳ lỗi nào được báo cáo, quay lại trình soạn thảo, sửa lỗi và thử biên dịch lại chương trình. (Chúng ta sẽ thảo luận chi tiết về lỗi sau trong chương này.)

Khi bạn đã biên dịch thành công chương trình của mình, bạn đã sẵn sàng chuyển sang bước 3, chạy chương trình. Một lần nữa, lệnh này sẽ khác nhau tùy theo từng môi trường, nhưng quy trình là tương tự (lệnh phổ biến là "run"). Sơ đồ trong Hình 1.1 tóm tắt các bước bạn sẽ thực hiện khi tạo một chương trình gọi là Hello.java.

Hình 1.1 Quá trình tạo và thực thi một chương trình Java



Hình 1.1 Văn bản thay thế đầy đủ

Trong một số IDE, hai bước đầu tiên được kết hợp lại. Trong các môi trường này, quá trình biên dịch trở nên gia tăng; trình biên dịch sẽ cảnh báo bạn về lỗi khi bạn đang gõ mã. Thông thường, không cần thiết phải yêu cầu chính thức một môi trường như vậy biên dịch chương trình của bạn vì nó đã biên dịch khi bạn gõ.

Khi chương trình của bạn được thực thi, nó thường sẽ tương tác với người dùng theo một cách nào đó. Chương trình Hello.java liên quan đến một cửa sổ trên màn hình được gọi là bảng điều khiển.

Cửa sổ điều khiển

Một cửa sổ chỉ có văn bản đặc biệt trong đó các chương trình Java tương tác với người dùng.

Cửa sổ điều khiển là một cơ chế tương tác cổ điển trong đó máy tính hiển thị văn bản trên màn hình và đôi khi chờ người dùng gõ phản hồi. Điều này được gọi là tương tác bảng điều khiển hoặc tương tác terminal. Văn bản mà máy tính in ra cửa sổ điều khiển được gọi là đầu ra của chương trình. Bất kỳ thứ gì do người dùng gõ vào đều được gọi là đầu vào của bảng điều khiển.

Để giữ mọi thứ đơn giản, hầu hết các chương trình mẫu trong sách này liên quan đến tương tác bảng điều khiển. Giữ cho tương tác đơn giản sẽ cho phép bạn tập trung chú ý và nỗ lực vào các khía cạnh khác của lập trình.

1.2 Bây giờ—Java

Đã đến lúc xem xét một chương trình Java hoàn chỉnh. Trong ngôn ngữ lập trình Java, không có gì có thể tồn tại ngoài một lớp.

Lớp

Một đơn vị mã là khối xây dựng cơ bản của các chương trình Java.

Khái niệm về lớp phong phú hơn nhiều so với điều này, như bạn sẽ thấy khi chúng ta đến Chương 8, nhưng hiện tại tất cả những gì bạn cần biết là mỗi chương trình Java của bạn sẽ được lưu trữ trong một lớp.

Có một truyền thống trong khoa học máy tính là khi bạn mô tả một ngôn ngữ lập trình mới, bạn nên bắt đầu với một chương trình tạo ra một dòng đầu ra duy nhất với dòng chữ "Hello, world!" Truyền thống "hello world" đã bị phá vỡ bởi nhiều tác giả sách

Java vì chương trình này không ngắn và đơn giản khi viết bằng Java như khi viết bằng các ngôn ngữ khác, nhưng chúng ta vẫn sẽ sử dụng nó ở đây.

Dưới đây là chương trình "hello world" của chúng ta:

```
1 public class Hello {

2     public static void main(String[] args) {

3         System.out.println("Hello, world!");

4     }

5 }
```

Chương trình này định nghĩa một lớp gọi là Hello. Oracle đã thiết lập quy ước rằng tên lớp luôn bắt đầu bằng chữ cái viết hoa, điều này làm cho nó dễ dàng nhận ra. Java yêu cầu tên lớp và tên tệp phải khớp nhau, vì vậy chương trình này phải được lưu trữ trong một tệp gọi là Hello.java. Bạn không cần phải hiểu tất cả các chi tiết của chương trình này ngay bây giờ, nhưng bạn cần hiểu cấu trúc cơ bản.

Dạng cơ bản của một lớp Java như sau:

```
public class <name> {

    <method>

    <method>

    ...
```

```
    <method>

}
```

Loại mô tả này được gọi là mẫu cú pháp vì nó mô tả dạng cơ bản của một cấu trúc Java. Java có các quy tắc xác định cú pháp hợp pháp hoặc ngữ pháp của nó. Mỗi khi chúng ta giới thiệu một yếu tố mới của Java, chúng ta sẽ bắt đầu bằng việc xem xét mẫu cú pháp của nó. Theo quy ước, chúng tôi sử dụng các ký tự nhỏ hơn (<) và lớn hơn (>) trong mẫu cú pháp để chỉ ra các mục cần được điền vào (trong trường hợp này, tên của lớp và các phương thức). Khi chúng tôi viết “...” trong danh sách các phần tử, chúng tôi đang chỉ ra rằng bất kỳ số lượng phần tử nào cũng có thể được bao gồm.

Dòng đầu tiên của lớp được gọi là phần đầu lớp. Từ khóa public trong phần đầu chỉ ra rằng lớp này có sẵn cho bất kỳ ai sử dụng. Lưu ý rằng mã chương trình trong một lớp được bao bọc bởi các ký tự ngoặc nhọn ({}). Những ký tự này được sử dụng trong Java để nhóm các đoạn mã liên quan lại với nhau. Trong trường hợp này, các dấu ngoặc nhọn chỉ ra rằng mọi thứ được định nghĩa bên trong chúng đều là một phần của lớp public này.

Vậy chính xác thì có thể xuất hiện gì bên trong dấu ngoặc nhọn? Những gì có thể được chứa trong một lớp? Rất nhiều thứ, nhưng hiện tại, chúng ta sẽ giới hạn trong các phương thức. Các phương thức là đơn vị mã nhỏ nhất tiếp theo trong Java, sau các lớp. Một phương thức đại diện cho một hành động hoặc phép tính duy nhất cần thực hiện.

Phương thức

Một đơn vị chương trình đại diện cho một hành động hoặc tính toán cụ thể. Các phương thức đơn giản giống như các động từ: chúng ra lệnh cho máy tính thực hiện một hành động nào đó. Bên trong dấu ngoặc nhọn của một lớp, bạn có thể định nghĩa nhiều phương thức khác nhau. Tối thiểu, một chương trình hoàn chỉnh yêu cầu một phương thức đặc biệt được gọi là phương thức main. Nó có cú pháp như sau:

```
public static void main(String[] args) {

    <câu lệnh>;

    <câu lệnh>;

    ...

    <câu lệnh>;

}
```

Cũng như dòng đầu tiên của một lớp được gọi là phần đầu lớp, dòng đầu tiên của một phương thức được gọi là phần đầu phương thức. Phần đầu của main khá phức tạp. Hầu hết mọi người đều ghi nhớ nó như một loại câu thần chú ma thuật. Bạn muốn mở cửa vào hang động của Ali Baba? Bạn nói: "Mở ra, vừng ơi!" Bạn muốn tạo ra một chương trình Java có thể thực thi? Bạn nói: public static void main(String[] args). Một nhóm giáo viên Java đã chế giễu điều này bằng một trang web có tên là publicstaticvoidmain.com.

Chỉ ghi nhớ các câu thần chú ma thuật không bao giờ là thỏa mãn, đặc biệt là đối với các nhà khoa học máy tính, những người thích hiểu mọi thứ đang diễn ra trong chương trình của họ. Nhưng đây là một nơi mà Java thể hiện khía cạnh xấu xí của nó,

và bạn sẽ phải chấp nhận điều đó. Các lập trình viên mới, giống như những người lái xe mới, phải học cách sử dụng một thứ phức tạp mà không cần hiểu đầy đủ cách thức hoạt động của nó. May mắn thay, khi bạn kết thúc cuốn sách này, bạn sẽ hiểu từng phần của câu thần chú đó.

Hãy chú ý rằng phương thức main có một bộ dấu ngoặc nhọn riêng. Chúng lại được sử dụng để nhóm, chỉ ra rằng mọi thứ xuất hiện giữa chúng đều là một phần của phương thức main. Các dòng giữa dấu ngoặc nhọn chỉ định chuỗi các hành động mà máy tính cần thực hiện khi nó thực thi phương thức. Chúng tôi gọi những điều này là các câu lệnh của phương thức.

Câu lệnh

Một đoạn mã có thể thực thi đại diện cho một lệnh hoàn chỉnh.

Mỗi câu lệnh được kết thúc bằng dấu chấm phẩy. Chương trình mẫu “hello world” chỉ có một câu lệnh duy nhất được gọi là câu lệnh println:

```
System.out.println("Hello, world!");
```

Chú ý rằng câu lệnh này kết thúc bằng dấu chấm phẩy. Dấu chấm phẩy có một trạng thái đặc biệt trong Java; nó được sử dụng để kết thúc các câu lệnh theo cùng cách mà dấu chấm kết thúc câu trong tiếng Anh.

Trong chương trình cơ bản “hello world”, chỉ có một lệnh để tạo ra một dòng đầu ra, nhưng hãy xem xét biến thể sau đây (được gọi là Hello2), có bốn dòng mã để thực thi trong phương thức main:

```
1 public class Hello2 {
```

```
2     public static void main(String[] args) {

3         System.out.println("Hello, world!");

4         System.out.println();

5         System.out.println("This program produces four");

6         System.out.println("lines of output.");

7     }

8 }
```

Chú ý rằng có bốn dấu chấm phẩy trong phương thức main, một ở cuối mỗi câu lệnh println. Các câu lệnh được thực thi theo thứ tự xuất hiện của chúng, từ đầu đến cuối, vì vậy chương trình Hello2 tạo ra đầu ra sau:

```
Hello, world!

This program produces four

lines of output.
```

Hãy tóm tắt các cấp độ khác nhau mà chúng ta vừa xem xét:

- Một chương trình Java được lưu trữ trong một lớp.
- Bên trong lớp, có các phương thức. Tối thiểu, một chương trình hoàn chỉnh yêu cầu một phương thức đặc biệt được gọi là main.

- Bên trong một phương thức như main, có một chuỗi các câu lệnh, mỗi câu lệnh đại diện cho một lệnh duy nhất mà máy tính cần thực thi.

Có thể sẽ lạ khi đặt dấu ngoặc nhọn mở ở cuối một dòng thay vì trên một dòng riêng biệt. Một số người sẽ sử dụng kiểu thụt lề này cho chương trình thay vì:

```
1 public class Hello3

2 {

3     public static void main(String[] args)

4     {

5         System.out.println("Hello, world!");

6     }

7 }
```

Những người khác nhau sẽ có các lựa chọn khác nhau về vị trí của dấu ngoặc nhọn. Phong cách chúng tôi sử dụng tuân theo các quy ước mã hóa Java chính thức của Oracle, nhưng phong cách khác cũng có người ủng hộ. Thường thì mọi người sẽ tranh luận một cách đam mê rằng cách này tốt hơn cách kia, nhưng thực ra đó chỉ là vấn đề sở thích cá nhân vì mỗi lựa chọn đều có một số ưu điểm và nhược điểm. Giảng viên của bạn có thể yêu cầu một phong cách cụ thể; nếu không, bạn nên chọn một phong cách mà bạn cảm thấy thoải mái và sau đó sử dụng nó một cách nhất quán.

Bây giờ bạn đã thấy tổng quan về cấu trúc, hãy xem xét một số chi tiết của các chương trình Java.

BẠN CÓ BIẾT?

Hello, World!

Truyền thống "hello world" được bắt đầu bởi Brian Kernighan và Dennis Ritchie. Ritchie đã phát minh ra một ngôn ngữ lập trình được gọi là C vào những năm 1970 và cùng với Kernighan, đồng tác giả của cuốn sách đầu tiên mô tả về C, xuất bản vào năm 1978. Chương trình hoàn chỉnh đầu tiên trong cuốn sách của họ là một chương trình "hello world". Kernighan và Ritchie, cũng như cuốn sách của họ *The C Programming Language*, đã được yêu mến gọi là "K & R" từ đó trở đi.

Nhiều ngôn ngữ lập trình lớn đã vay mượn cú pháp cơ bản của C như một cách để tận dụng sự phổ biến của C và khuyến khích các lập trình viên chuyển sang nó. Cả ngôn ngữ C++ và Java đều vay mượn rất nhiều cú pháp cốt lõi từ C.

Kernighan và Ritchie cũng có một phong cách đặc biệt về vị trí của dấu ngoặc nhọn và thụt lề của các chương trình mà đã trở thành “phong cách K & R.” Đây là phong cách mà Oracle khuyến nghị và chúng tôi sử dụng trong cuốn sách này.

Chuỗi Ký Tự (Strings)

Khi bạn viết các chương trình Java (chẳng hạn như chương trình “hello world” ở trên), bạn sẽ thường muốn bao gồm một số văn bản cụ thể để gửi tới cửa sổ điều khiển làm đầu ra. Các lập trình viên truyền thống gọi văn bản này là một chuỗi (string) vì nó được

tạo thành từ một chuỗi các ký tự mà chúng ta xâu chuỗi lại với nhau. Đặc tả ngôn ngữ Java sử dụng thuật ngữ chuỗi ký tự (string literals).

Trong Java, bạn xác định một chuỗi ký tự bằng cách bao quanh văn bản bằng dấu ngoặc kép, như trong:

```
"This is a bunch of text surrounded by quotation marks."
```

Bạn phải sử dụng dấu ngoặc kép, không phải dấu ngoặc đơn. Dòng sau đây không phải là một chuỗi ký tự hợp lệ:

```
'Bad stuff here.'
```

Dòng sau đây là một chuỗi ký tự hợp lệ:

```
"This is a string even with 'these' quotes inside."
```

Chuỗi ký tự không được phép trải dài qua nhiều dòng của một chương trình. Dòng sau đây không phải là một chuỗi ký tự hợp lệ:

```
"This is really


 bad stuff



 right here."
```

System.out.println

Như bạn đã thấy, phương thức main của một chương trình Java chứa một loạt các câu lệnh để máy tính thực hiện. Chúng được thực thi tuần tự, bắt đầu từ câu lệnh đầu tiên, sau đó là câu lệnh thứ hai, thứ ba và tiếp tục cho đến khi câu lệnh cuối cùng được thực thi. Một trong những câu lệnh đơn giản và phổ biến nhất là System.out.println, được sử dụng để tạo ra một dòng đầu ra. Đây là một "câu thần chú" khác mà bạn nên ghi nhớ. Tính đến thời điểm này, Google liệt kê khoảng 8.000.000 trang web đề cập đến System.out.println. Điều quan trọng cần nhớ về câu lệnh này là nó được sử dụng để tạo ra một dòng đầu ra gửi tới cửa sổ điều khiển.

Dạng đơn giản nhất của câu lệnh println không có gì bên trong dấu ngoặc đơn và tạo ra một dòng trống làm đầu ra:

```
System.out.println();
```

Bạn cần bao gồm dấu ngoặc đơn ngay cả khi không có gì để đặt bên trong. Lưu ý dấu chấm phẩy ở cuối dòng. Tất cả các câu lệnh trong Java phải được kết thúc bằng dấu chấm phẩy.

Tuy nhiên, thông thường bạn sẽ sử dụng println để xuất một dòng văn bản:

```
System.out.println("This line uses the println method.");
```

Câu lệnh trên yêu cầu máy tính tạo ra dòng đầu ra sau đây:

```
This line uses the println method.
```

Mỗi câu lệnh println tạo ra một dòng đầu ra khác nhau. Ví dụ, hãy xem xét ba câu lệnh sau:

```
System.out.println("This is the first line of output.");

System.out.println();

System.out.println("This is the third, below a blank line.");
```

Thực thi các câu lệnh này sẽ tạo ra ba dòng đầu ra sau đây (dòng thứ hai là dòng trống):

```
This is the first line of output.



This is the third, below a blank line.
```

Chuỗi thoát (Escape Sequences)

Bất kỳ hệ thống nào liên quan đến việc trích dẫn văn bản đều sẽ dẫn đến một số tình huống khó khăn. Ví dụ, các chuỗi ký tự (string literals) được chứa trong dấu ngoặc kép, vậy làm thế nào để bạn có thể bao gồm dấu ngoặc kép bên trong một chuỗi ký tự? Các chuỗi ký tự cũng không được phép ngắt dòng, vậy làm sao để bạn có thể thêm một dòng ngắt bên trong một chuỗi ký tự? Giải pháp là chèn các chuỗi thoát (escape sequences) vào chuỗi ký tự. Các chuỗi thoát là các chuỗi gồm hai ký tự được sử dụng để đại diện cho các ký tự đặc biệt. Tất cả đều bắt đầu bằng ký tự gạch chéo ngược (). Bảng 1.3 liệt kê một số chuỗi thoát phổ biến.

Bảng 1.3 Các chuỗi thoát phổ biến

Chuỗi thoát	**Đại diện cho**
\t	ký tự tab
\n	ký tự xuống dòng
"	dấu ngoặc kép
\	ký tự gạch chéo ngược

Hãy nhớ rằng mỗi chuỗi gồm hai ký tự này thực sự chỉ đại diện cho một ký tự duy nhất. Ví dụ, hãy xem xét câu lệnh sau:

```
System.out.println("What \"characters\" does this \\ print?");
```

Nếu bạn thực thi câu lệnh này, bạn sẽ nhận được đầu ra sau:

```
What "characters" does this \ print?
```

Chuỗi ký tự trong lệnh println có ba chuỗi thoát, mỗi chuỗi gồm hai ký tự và tạo ra một ký tự duy nhất trong đầu ra.

Mặc dù bản thân chuỗi ký tự không thể kéo dài qua nhiều dòng (nghĩa là bạn không thể sử dụng dấu xuống dòng bên trong một chuỗi ký tự để buộc nó xuống dòng), bạn có thể sử dụng chuỗi thoát \n để nhúng các ký tự xuống dòng vào trong chuỗi. Điều này dẫn đến một tình huống lạ lùng khi một câu lệnh println có thể tạo ra nhiều hơn một dòng đầu ra.

Ví dụ, hãy xem xét câu lệnh này:

```
System.out.println("This\nproduces 3 lines\nof output.");
```

Nếu bạn thực thi câu lệnh này, bạn sẽ nhận được đầu ra sau:

```
This
produces 3 lines
of output.
```

Lệnh println tự nó tạo ra một dòng đầu ra, nhưng chuỗi ký tự chứa hai ký tự xuống dòng khiến nó bị chia thành tổng cộng ba dòng đầu ra. Để tạo ra cùng một đầu ra mà không có các ký tự xuống dòng, bạn sẽ phải thực hiện ba lệnh println riêng biệt.

Đây là một thói quen lập trình khác nhau tùy theo sở thích. Một số người (bao gồm cả các tác giả) thấy khó đọc các chuỗi ký tự chứa các chuỗi thoát \n, nhưng những người khác lại thích viết ít dòng code hơn. Một lần nữa, bạn nên tự quyết định khi nào cần sử dụng chuỗi thoát xuống dòng.

print và println

Java có một biến thể của lệnh println gọi là print, cho phép bạn tạo đầu ra trên dòng hiện tại mà không chuyển sang dòng mới.

Lệnh println thực hiện hai việc khác nhau: nó gửi đầu ra đến dòng hiện tại, sau đó chuyển đến đầu của một dòng mới. Lệnh print chỉ thực hiện bước đầu tiên. Do đó, một loạt các lệnh print sẽ tạo ra đầu ra trên cùng một dòng. Chỉ có lệnh println mới hoàn thành dòng hiện tại và bắt đầu một dòng mới. Ví dụ, hãy xem xét sáu câu lệnh sau:

```
System.out.print("To be ");
System.out.print("or not to be.");
System.out.print("That is ");
System.out.println("the question.");
System.out.print("This is");
System.out.println(" for the whole family!");
```

Những câu lệnh này tạo ra hai dòng đầu ra. Hãy nhớ rằng mỗi câu lệnh println tạo ra chính xác một dòng đầu ra; vì có hai câu lệnh println ở đây, nên có hai dòng đầu ra. Sau khi câu lệnh đầu tiên được thực thi, dòng hiện tại trông như thế này:

```
To be
^
```

Mũi tên dưới dòng đầu ra chỉ ra vị trí mà đầu ra sẽ được gửi tới tiếp theo. Chúng ta có thể đơn giản hóa việc thảo luận nếu chúng ta coi mũi tên là con trỏ đầu ra. Lưu ý rằng con trỏ đầu ra nằm ở cuối dòng này và nó xuất hiện sau một khoảng trắng. Lý do là lệnh vừa rồi là print (không chuyển sang dòng mới) và chuỗi ký tự trong lệnh print kết thúc bằng một khoảng trắng. Java sẽ không chèn khoảng trắng cho bạn trừ khi bạn yêu cầu cụ thể. Sau lệnh print tiếp theo, dòng trông như thế này:

```
To be or not to be.
^
```

Không có khoảng trắng ở cuối vì chuỗi ký tự trong lệnh print thứ hai kết thúc bằng dấu chấm, không phải khoảng trắng. Sau lệnh print tiếp theo, dòng trông như thế này:

```
To be or not to be.That is
^
```

Không có khoảng cách giữa dấu chấm và từ "That" vì không có khoảng trắng trong các lệnh print, nhưng có một khoảng trắng ở cuối chuỗi ký tự trong câu lệnh thứ ba. Sau khi câu lệnh tiếp theo được thực thi, đầu ra trông như thế này:

```
To be or not to be.That is the question.
^
```

Vì câu lệnh thứ tư là một lệnh println, nó hoàn thành dòng đầu ra và đặt con trỏ tại đầu dòng thứ hai. Câu lệnh tiếp theo là một lệnh print khác, tạo ra đầu ra như sau:

```
To be or not to be.That is the question.

This is
^
```

Lệnh println cuối cùng hoàn thành dòng thứ hai và đặt con trỏ đầu ra tại đầu dòng mới:

```
To be or not to be.That is the question.

This is for the whole family!
^
```

Sáu câu lệnh này tương đương với hai câu lệnh đơn sau:

```
System.out.println("To be or not to be.That is the question.");
System.out.println("This is for the whole family!");
```

Sử dụng lệnh print và println cùng nhau để tạo ra các dòng như thế này có thể trông hơi kỳ quặc, nhưng bạn sẽ thấy rằng có những ứng dụng thú vị hơn của lệnh print trong chương tiếp theo. Hãy nhớ rằng có thể có một lệnh println trống:

```
System.out.println();
```

Vì không có gì bên trong dấu ngoặc để được ghi ra dòng đầu ra, lệnh này đặt con trỏ đầu ra tại đầu dòng tiếp theo. Nếu có các lệnh print trước lệnh println trống này, nó sẽ hoàn thành dòng được tạo ra bởi các lệnh print đó. Nếu không có lệnh print nào

trước đó, nó sẽ tạo ra một dòng trống. Một lệnh print trống là vô nghĩa và không hợp lệ.

Định danh và Từ khóa
Các từ được sử dụng để đặt tên cho các phần của một chương trình Java được gọi là định danh.
Định danh
Tên được gán cho một thực thể trong chương trình, chẳng hạn như một lớp hoặc phương thức. Các định danh phải bắt đầu bằng một chữ cái, sau đó có thể là bất kỳ số lượng chữ cái hoặc chữ số nào. Dưới đây là các định danh hợp pháp:
sql
Copy code
first hiThere numStudents TwoBy4
Đặc tả ngôn ngữ Java định nghĩa tập hợp các chữ cái bao gồm ký tự gạch dưới và ký tự đô-la (_ và $), điều này có nghĩa là các định danh sau cũng hợp pháp:
bash
Copy code
two_plus_two _count $2donuts MAX_COUNT
Dưới đây là các định danh không hợp pháp:
Copy code
two+two hi there hi-There 2by4
Java có quy ước về cách viết hoa được tuân thủ khá nhất quán bởi các lập trình viên. Tất cả các tên lớp nên bắt đầu bằng một chữ cái viết hoa, như với các lớp Hello, Hello2, và Hello3 đã được giới thiệu trước đó. Tên của các phương thức nên bắt đầu bằng chữ cái viết thường, như trong phương thức main. Khi bạn ghép nhiều từ lại với nhau để tạo thành tên lớp hoặc tên phương thức, hãy viết hoa chữ cái đầu tiên của mỗi từ sau từ đầu tiên. Trong chương tiếp theo, chúng ta sẽ thảo luận về hằng số, có một quy ước viết hoa khác, với tất cả các chữ cái viết hoa và các từ được ngăn cách bởi dấu gạch dưới. Những quy ước khác

nhau này có thể dường như là những hạn chế tẻ nhạt, nhưng việc sử dụng cách viết hoa nhất quán trong mã của bạn cho phép người đọc nhanh chóng nhận diện các phần tử mã khác nhau. Ví dụ, giả sử rằng bạn sẽ ghép các từ “all my children” thành một định danh. Kết quả sẽ là:

- AllMyChildren cho tên lớp (mỗi từ bắt đầu bằng chữ cái viết hoa)
- allMyChildren cho tên phương thức (bắt đầu bằng chữ cái viết thường, các từ sau viết hoa)
- ALL_MY_CHILDREN cho tên hằng số (tất cả chữ cái viết hoa, với các từ ngăn cách bởi dấu gạch dưới; sẽ được mô tả trong Chương 2)

Java phân biệt chữ hoa và chữ thường, vì vậy các định danh class, Class, CLASS, và cLASs được coi là khác nhau. Hãy ghi nhớ điều này khi bạn đọc các thông báo lỗi từ trình biên dịch. Con người giỏi trong việc hiểu những gì bạn viết, ngay cả khi bạn viết sai chính tả hoặc mắc phải các lỗi nhỏ như thay đổi cách viết hoa của một từ. Tuy nhiên, những lỗi như vậy sẽ khiến trình biên dịch Java hoàn toàn bối rối.

Đừng ngần ngại sử dụng các định danh dài. Các tên mô tả càng chi tiết, thì càng dễ dàng cho mọi người (bao gồm cả bạn) đọc các chương trình của bạn. Các định danh mô tả đáng giá với thời gian bạn bỏ ra để gõ chúng. Ví dụ, lớp String của Java có một phương thức gọi là compareToIgnoreCase.

Tuy nhiên, hãy lưu ý rằng Java có một tập hợp các định danh được định nghĩa trước gọi là từ khóa, được dành riêng cho các mục đích cụ thể. Khi bạn đọc cuốn sách này, bạn sẽ học được nhiều từ khóa này và cách sử dụng chúng. Bạn chỉ có thể sử dụng các từ khóa cho các mục đích dự định của chúng. Bạn phải cẩn thận để tránh sử dụng những từ này trong tên của các định danh. Ví dụ, nếu bạn đặt tên cho một phương thức là short hoặc try, điều này sẽ gây ra vấn đề, vì short và try là các từ khóa đã

được dành riêng. Bảng 1.4 cho thấy danh sách đầy đủ của các từ khóa đã được dành riêng.

Bảng 1.4 Danh sách từ khóa của Java

Từ khóa				
abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Một Ví dụ Phức Tạp: DrawFigures1

Câu lệnh println có thể được sử dụng để vẽ các hình bằng văn bản làm đầu ra. Hãy xem xét ví dụ chương trình phức tạp hơn sau đây (lưu ý rằng nó sử dụng hai câu lệnh println trống để tạo ra các dòng trống):

lại để loại bỏ các dòng trống:

```
public class DrawFigures1 {
    public static void main(String[] args) {
        System.out.println("    /\\");
        System.out.println("  /  \\");
        System.out.println(" /    \\");
        System.out.println(" \\    /");
        System.out.println("  \\  /");
        System.out.println("   \\/");
        System.out.println(" \\    /");
```

```
        System.out.println("  \\  /");
        System.out.println("   \\/");
        System.out.println("   /\\");
        System.out.println("  /  \\");
        System.out.println(" /    \\");
        System.out.println("   /\\");
        System.out.println("  /  \\");
        System.out.println(" /    \\");
        System.out.println("+------+");
        System.out.println("|      |");
        System.out.println("|      |");
        System.out.println("+------+");
        System.out.println("|United|");
        System.out.println("|States|");
        System.out.println("+------+");
        System.out.println("|      |");
        System.out.println("|      |");
        System.out.println("+------+");
        System.out.println("   /\\");
        System.out.println("  /  \\");
        System.out.println(" /    \\");
    }
}
```

Dưới đây là đầu ra mà chương trình tạo ra. Lưu ý rằng chương trình bao gồm các ký tự dấu gạch chéo ngược đôi (\\), nhưng đầu ra chỉ có các ký tự dấu gạch chéo ngược đơn. Đây là một ví dụ về chuỗi thoát, như đã được mô tả trước đó.

```
  /\
 /  \
/    \
\    /
 \  /
  \/
```

```
 \    /
  \  /
   \/
   /\
  /  \
 /    \
   /\
  /  \
 /    \
+------+
|      |
|      |
+------+
|United|
|States|
+------+
|      |
|      |
+------+
   /\
  /  \
 /    \
```

Chú Thích và Tính Dễ Đọc

Java là một ngôn ngữ có định dạng tự do. Điều này có nghĩa là bạn có thể chèn nhiều hay ít khoảng trắng và dòng trống tùy thích, miễn là bạn đặt ít nhất một khoảng trắng hoặc dấu câu giữa các từ. Tuy nhiên, bạn nên nhớ rằng cách bố trí của chương trình có thể tăng cường (hoặc làm giảm) tính dễ đọc của nó. Chương trình sau đây là hợp pháp nhưng khó đọc:

```
public class Ugly {
    public static void main(String[] args) {
        System.out.println("How short I am!");
```

```
    }
}
```

Dưới đây là một số quy tắc đơn giản mà bạn nên tuân theo để làm cho các chương trình của mình dễ đọc hơn:

- Đặt tiêu đề lớp và phương thức trên các dòng riêng biệt.
- Không đặt quá một câu lệnh trên mỗi dòng.
- Thụt lề chương trình đúng cách. Khi xuất hiện dấu ngoặc nhọn mở {, hãy tăng mức thụt lề của các dòng tiếp theo. Khi xuất hiện dấu ngoặc nhọn đóng }, hãy giảm mức thụt lề. Thụt lề các câu lệnh bên trong dấu ngoặc nhọn theo số lượng khoảng trắng nhất quán (một lựa chọn phổ biến là bốn khoảng trắng mỗi mức thụt lề).
- Sử dụng các dòng trống để tách biệt các phần của chương trình (ví dụ: các phương thức).

Viết lại chương trình Ugly theo các quy tắc này sẽ cho ra đoạn mã sau:

```
public class Ugly {
    public static void main(String[] args) {
        System.out.println("How short I am!");
    }
}
```

Các chương trình Java được viết tốt có thể rất dễ đọc, nhưng đôi khi bạn sẽ muốn bao gồm một số giải thích không phải là một phần của chính chương trình. Bạn có thể chú thích các chương trình bằng cách đặt các ghi chú được gọi là chú thích (comments) vào chúng.

Chú Thích

Đoạn văn bản mà lập trình viên thêm vào chương trình để giải thích mã của họ. Trình biên dịch sẽ bỏ qua các chú thích này.

Có hai dạng chú thích trong Java. Trong dạng đầu tiên, bạn mở chú thích bằng dấu gạch chéo theo sau là dấu sao và đóng nó bằng dấu sao theo sau là dấu gạch chéo:

```
/* như thế này */
```

Bạn không được đặt khoảng trắng giữa dấu gạch chéo và dấu sao:

```
/ * điều này là sai * /
```

Bạn có thể đặt hầu như bất kỳ văn bản nào bạn thích, bao gồm nhiều dòng, bên trong chú thích:

```
/* Thaddeus Martin

   Assignment #1

   Instructor: Professor Walingford

   Grader:     Bianca Montgomery */
```

Điều duy nhất bạn không được phép đặt bên trong chú thích là các ký tự kết thúc chú thích. Đoạn mã sau là không hợp lệ:

```
/* Chú thích này có dấu gạch chéo dấu sao /*/ bên trong,

   dẫn đến việc đóng chú thích sớm. Điều này là sai. */
```

Java cũng cung cấp dạng chú thích thứ hai cho các chú thích ngắn, chỉ một dòng. Bạn có thể sử dụng hai dấu gạch chéo liên tiếp để chỉ ra rằng phần còn lại của dòng hiện tại (tất cả mọi thứ bên phải của hai dấu gạch chéo) là một chú thích. Ví dụ, bạn có thể đặt chú thích sau một câu lệnh:

```
System.out.println("You win!"); // Tốt lắm!
```

Hoặc bạn có thể tạo một chú thích trên dòng riêng:

```
// giới thiệu với người dùng

System.out.println("Chào mừng đến với trò chơi blackjack.");

System.out.println();

System.out.println("Để tôi giải thích quy tắc.");
```

Bạn thậm chí có thể tạo các khối chú thích một dòng:

```
// Thaddeus Martin
// Assignment #1
// Instructor:  Professor Walingford
// Grader:      Bianca Montgomery
```

Một số người thích sử dụng dạng chú thích đầu tiên cho các chú thích nhiều dòng nhưng sử dụng dạng thứ hai an toàn hơn vì bạn không phải nhớ đóng chú thích. Nó cũng giúp chú thích nổi bật hơn. Đây là một trường hợp khác mà nếu giảng viên của bạn không yêu cầu bạn sử dụng một kiểu chú thích cụ thể, bạn nên tự quyết định kiểu nào bạn thích và sử dụng nó một cách nhất quán.
Đừng nhầm lẫn chú thích với văn bản trong các câu lệnh println. Văn bản của chú thích sẽ không được hiển thị dưới dạng đầu ra khi chương trình chạy. Chú thích chỉ tồn tại để giúp người đọc kiểm tra và hiểu chương trình.
Một ý tưởng hay là bao gồm chú thích ở đầu mỗi tệp lớp để chỉ ra lớp đó làm gì. Bạn cũng có thể muốn thêm thông tin về bạn là ai, khóa học bạn đang theo học, tên giảng viên và/hoặc người chấm điểm, ngày tháng, v.v. Bạn cũng nên chú thích từng phương thức để chỉ ra nó làm gì.
Chú thích trở nên hữu ích hơn trong các chương trình lớn và phức tạp hơn, cũng như trong các chương trình sẽ được xem hoặc sửa đổi bởi nhiều lập trình viên. Chú thích rõ ràng rất hữu ích để giải thích cho người khác, hoặc cho chính bạn sau này, chương trình của bạn đang làm gì và tại sao nó lại làm như vậy.
Ngoài hai dạng chú thích đã được thảo luận, Java còn hỗ trợ một kiểu chú thích đặc biệt gọi là chú thích Javadoc. Định dạng của chúng phức tạp hơn, nhưng chúng có ưu điểm là bạn có thể sử dụng một chương trình để trích xuất các chú thích để tạo các tệp HTML phù hợp để đọc bằng trình duyệt web. Chú thích Javadoc hữu ích trong lập trình nâng cao và sẽ được thảo luận chi tiết hơn trong Phụ lục B.

1.3 Lỗi Chương Trình

Năm 1949, Maurice Wilkes, một trong những người tiên phong trong lĩnh vực máy tính, đã bày tỏ một suy nghĩ vẫn còn đúng đến ngày nay: Ngay khi bắt đầu lập trình, chúng tôi ngạc nhiên phát hiện rằng việc viết đúng chương trình không dễ như chúng tôi tưởng. Việc gỡ lỗi phải được khám phá. Tôi còn nhớ khoảnh khắc chính xác khi tôi nhận ra rằng một phần lớn cuộc đời tôi từ đó trở đi sẽ dành để tìm lỗi trong các chương trình của chính mình.

Bạn cũng sẽ phải đối mặt với thực tế này khi học lập trình. Bạn sẽ mắc lỗi, giống như mọi lập trình viên trong lịch sử, và bạn sẽ cần có các chiến lược để loại bỏ những lỗi đó. May mắn thay, máy tính có thể giúp bạn với một phần công việc này.

Có ba loại lỗi mà bạn sẽ gặp phải khi viết chương trình:

- **Lỗi cú pháp** xảy ra khi bạn sử dụng sai Java. Chúng giống như lỗi ngữ pháp và sẽ bị trình biên dịch Java phát hiện.
- **Lỗi logic** xảy ra khi bạn viết mã không thực hiện đúng nhiệm vụ mà nó dự định thực hiện.
- **Lỗi thời gian chạy** là lỗi logic nghiêm trọng đến mức Java dừng chương trình của bạn khi đang thực thi.

Lỗi Cú Pháp

Con người thường khá dễ dãi về các lỗi nhỏ trong lời nói. Ví dụ, nhân vật Yoda sẽ mất điểm vì ngữ pháp kỳ lạ của mình trong bất kỳ lớp viết văn nào, nhưng chúng ta vẫn hiểu ý của ông ta.

Trình biên dịch Java sẽ không dễ dãi như vậy. Trình biên dịch báo lỗi cú pháp khi cố gắng chuyển đổi chương trình của bạn từ Java sang bytecodes nếu chương trình của bạn vi phạm bất kỳ quy tắc ngữ pháp nào của Java. Ví dụ, nếu bạn đặt nhầm một dấu chấm phẩy trong chương trình của mình, trình biên dịch có thể sẽ báo một loạt lỗi tùy thuộc vào điều mà nó nghĩ là sai với chương trình của bạn.

Một chương trình tạo ra lỗi biên dịch không thể được thực thi. Nếu bạn gửi chương trình của mình cho trình biên dịch và nó báo

lỗi, bạn phải sửa lỗi đó và gửi lại chương trình. Bạn sẽ không thể tiếp tục cho đến khi chương trình của bạn không còn lỗi biên dịch. Một số môi trường phát triển, chẳng hạn như Eclipse, giúp bạn dễ dàng phát hiện lỗi cú pháp bằng cách gạch chân chúng khi bạn viết chương trình. Điều này giúp bạn dễ dàng xác định chính xác vị trí lỗi.

Bạn có thể mắc lỗi trước cả khi bắt đầu viết chương trình, nếu bạn chọn sai tên tệp cho nó.

LỖI LẬP TRÌNH THƯỜNG GẶP: Tên Tệp Không Khớp Với Tên Lớp

Như đã đề cập trước đó, Java yêu cầu tên lớp của chương trình và tên tệp phải khớp nhau. Ví dụ, một chương trình bắt đầu bằng public class Hello phải được lưu trong một tệp có tên là Hello.java.

Nếu bạn sử dụng sai tên tệp (ví dụ, lưu nó dưới dạng WrongFileName.java), bạn sẽ nhận được thông báo lỗi như sau:

```
WrongFileName.java:1: error: class Hello is public,
    should be declared in a file named Hello.java
public class Hello {
       ^
1 error
```

Tên tệp chỉ là thử thách đầu tiên. Một số lỗi khác có thể tồn tại trong chương trình Java của bạn. Một trong những lỗi cú pháp phổ biến nhất là viết sai chính tả. Bạn có thể mắc lỗi dấu câu, chẳng hạn như thiếu dấu chấm phẩy. Cũng rất dễ quên một từ, chẳng hạn như từ khóa bắt buộc.

Các thông báo lỗi mà trình biên dịch đưa ra có thể có ích hoặc không. Nếu bạn không hiểu nội dung của thông báo lỗi, hãy tìm dấu mũ (^) dưới dòng, chỉ vào vị trí trong dòng mà trình biên dịch gặp vấn đề. Điều này có thể giúp bạn xác định vị trí mà từ khóa bắt buộc có thể bị thiếu.

LỖI LẬP TRÌNH THƯỜNG GẶP: Viết Sai Chính Tả

Java (như hầu hết các ngôn ngữ lập trình) rất khắt khe về chính tả. Bạn cần phải viết đúng chính tả từng từ, bao gồm cả việc viết hoa đúng cách. Giả sử, ví dụ, bạn thay thế câu lệnh println trong chương trình "hello world" bằng dòng sau:

```
System.out.pruntln("Hello, world!");
```

Khi bạn cố gắng biên dịch chương trình này, nó sẽ tạo ra thông báo lỗi tương tự như sau:

```
Hello.java:3: error: cannot find symbol
symbol : method pruntln(java.lang.String)
location: variable out of type PrintStream
        System.out.pruntln("Hello, world!");
                   ^
1 error
```

Dòng đầu tiên của kết quả này cho biết rằng lỗi xảy ra trong tệp Hello.java ở dòng 3 và lỗi là trình biên dịch không thể tìm thấy một ký hiệu. Dòng thứ hai cho biết ký hiệu mà nó không thể tìm thấy là một phương thức có tên pruntln. Đó là vì không có phương thức nào như vậy; phương thức có tên là println. Thông báo lỗi có thể có nhiều dạng khác nhau tùy thuộc vào những gì bạn đã viết sai chính tả. Ví dụ, bạn có thể quên viết hoa từ System:

```
system.out.println("Hello, world!");
```

Bạn sẽ nhận được thông báo lỗi sau:

```
Hello.java:3: error: package system does not exist
        system.out.println("Hello, world!");
              ^
1 error
```

Một lần nữa, dòng đầu tiên cho biết rằng lỗi xảy ra ở dòng 3 của tệp Hello.java. Thông báo lỗi ở đây hơi khác một chút, cho biết rằng nó không thể tìm thấy một package có tên system. Dòng thứ hai và thứ ba của thông báo lỗi này bao gồm dòng mã ban đầu với một mũi tên (dấu mũ) chỉ vào nơi mà trình biên dịch bị nhầm lẫn. Các lỗi của trình biên dịch không phải lúc nào cũng rõ ràng,

nhưng nếu bạn chú ý đến vị trí mà mũi tên đang chỉ, bạn sẽ có ý tưởng khá tốt về nơi lỗi xảy ra.
Nếu bạn vẫn không thể tìm ra lỗi, hãy thử xem xét số dòng của lỗi và so sánh nội dung của dòng đó với các dòng tương tự trong các chương trình khác. Bạn cũng có thể nhờ người khác, chẳng hạn như giảng viên hoặc trợ giảng, kiểm tra chương trình của mình.

LỖI LẬP TRÌNH THƯỜNG GẶP: Quên Dấu Chấm Phẩy

Tất cả các câu lệnh trong Java phải kết thúc bằng dấu chấm phẩy, nhưng rất dễ quên đặt dấu chấm phẩy ở cuối câu lệnh, như trong chương trình sau:

```
1  public class MissingSemicolon {
2      public static void main(String[] args) {
3          System.out.println("A rose by any other name")
4          System.out.println("would smell as sweet");
5      }
6  }
```

Trong trường hợp này, trình biên dịch sẽ tạo ra thông báo lỗi tương tự như sau:

```
MissingSemicolon.java:3: error: ';' expected
        System.out.println("A rose by any other name")
                                                     ^
1 error
```

LỖI LẬP TRÌNH THƯỜNG GẶP: Quên Từ Khóa Bắt Buộc

Một lỗi cú pháp phổ biến khác là quên một từ khóa bắt buộc khi bạn đang gõ chương trình, chẳng hạn như static hoặc class. Hãy kiểm tra kỹ các chương trình của bạn so với các ví dụ trong sách giáo khoa để đảm bảo bạn không bỏ sót một từ khóa quan trọng. Trình biên dịch sẽ đưa ra các thông báo lỗi khác nhau tùy thuộc vào từ khóa nào bị thiếu, nhưng các thông báo này có thể khó hiểu. Ví dụ, bạn có thể viết một chương trình gọi là Bug4 và quên

từ khóa class khi viết tiêu đề của lớp. Trong trường hợp này, trình biên dịch sẽ cung cấp thông báo lỗi sau:

```
Bug4.java:1: error: class, interface, or enum expected
public Bug4 {
       ^
1 error
```

Tuy nhiên, nếu bạn quên từ khóa void khi khai báo phương thức main, trình biên dịch sẽ tạo ra một thông báo lỗi khác:

```
Bug5.java:2: error: invalid method declaration; return type
required
    public static main(String[] args) {
                  ^
1 error
```

Một lỗi cú pháp phổ biến khác nữa là quên đóng một chuỗi ký tự. Một quy tắc hữu ích là lỗi đầu tiên được trình biên dịch báo cáo là lỗi quan trọng nhất. Các lỗi còn lại có thể là kết quả của lỗi đầu tiên đó. Nhiều lập trình viên thậm chí không quan tâm đến các lỗi sau lỗi đầu tiên, vì sửa lỗi đó và biên dịch lại có thể làm cho các lỗi khác biến mất.

LỖI LẬP TRÌNH THƯỜNG GẶP: Không Đóng Chuỗi Ký Tự Hoặc Bình Luận

Mọi chuỗi ký tự đều phải có dấu mở và dấu đóng, nhưng rất dễ quên dấu ngoặc kép đóng. Ví dụ, bạn có thể viết như sau:

```
System.out.println("Hello, world!);
```

Điều này dẫn đến ba thông báo lỗi khác nhau, mặc dù chỉ có một lỗi cú pháp cơ bản:

```
Hello.java:3: error: unclosed string literal
        System.out.println("hello world);
                           ^
Hello.java:3: error: ';' expected
        System.out.println("hello world);
                                    ^
```

```
Hello.java:5: error: reached end of file while parsing
    }
    ^
3 errors
```

Trong trường hợp này, thông báo lỗi đầu tiên khá rõ ràng, bao gồm cả một mũi tên chỉ vào phần đầu của chuỗi ký tự chưa được đóng. Thông báo lỗi thứ hai do lỗi đầu tiên gây ra. Bởi vì chuỗi ký tự chưa được đóng, trình biên dịch không nhận ra dấu ngoặc đơn phải và dấu chấm phẩy xuất hiện ở cuối dòng.

Một vấn đề tương tự xảy ra khi bạn quên đóng một bình luận nhiều dòng bằng cách viết */, như trong dòng đầu tiên của chương trình sau:

```
/* This is a bad program.

public class Bad {
   public static void main(String[] args) {
       System.out.println("Hi there.");
   }
} /* end of program */
```

Tệp trước không phải là một chương trình; nó chỉ là một bình luận dài. Vì bình luận ở dòng đầu tiên chưa được đóng, toàn bộ chương trình đã bị nuốt chửng.

May mắn thay, nhiều chương trình soạn thảo Java sử dụng màu sắc để giúp bạn nhận biết các phần của chương trình một cách trực quan. Thường thì, nếu bạn quên đóng chuỗi ký tự hoặc bình luận, phần còn lại của chương trình sẽ chuyển sang màu sai, điều này có thể giúp bạn phát hiện ra lỗi.

Java được thiết kế cho một loại phân rã khác, hướng đối tượng hơn. Thay vì nghĩ về vấn đề như một loạt các hành động cần thực hiện, chúng ta nghĩ về nó như một tập hợp các đối tượng cần tương tác với nhau.

Với tư cách là một nhà khoa học máy tính, bạn nên quen thuộc với cả hai loại giải quyết vấn đề. Cuốn sách này bắt đầu với sự

phân rã theo thủ tục và dành nhiều chương để thành thạo các khía cạnh khác nhau của cách tiếp cận thủ tục. Chỉ sau khi bạn đã thực hành lập trình theo thủ tục một cách kỹ lưỡng, chúng ta mới quay lại chú ý đến phân rã đối tượng và lập trình hướng đối tượng.

Ví dụ về phân rã thủ tục, hãy xem xét vấn đề nướng một chiếc bánh. Bạn có thể chia vấn đề này thành các vấn đề con sau:

- Làm bột bánh.
- Nướng bánh.
- Làm kem phủ.
- Phủ kem lên bánh.

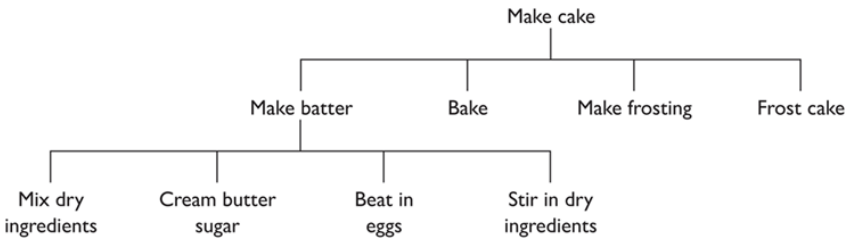
Mỗi tác vụ trong số bốn tác vụ này đều có các chi tiết liên quan. Để làm bột bánh, ví dụ, bạn thực hiện các bước sau:

- Trộn các nguyên liệu khô.
- Đánh kem bơ và đường.
- Đánh trứng vào hỗn hợp.
- Khuấy đều các nguyên liệu khô.

Vì vậy, bạn chia tác vụ tổng thể thành các tác vụ con, sau đó tiếp tục chia thành các tác vụ con nhỏ hơn nữa. Cuối cùng, bạn đạt đến các mô tả đơn giản đến mức không cần giải thích thêm (tức là các phép nguyên thủy).

Một sơ đồ phân rã một phần của quá trình "Làm bánh" được hiển thị trong Hình 1.2. "Làm bánh" là hoạt động ở mức cao nhất. Nó được định nghĩa theo bốn hoạt động cấp thấp hơn gọi là "Làm bột", "Nướng", "Làm kem phủ", và "Phủ kem lên bánh." Hoạt động "Làm bột" được định nghĩa theo các hoạt động cấp thấp hơn, và điều tương tự cũng có thể được thực hiện cho ba hoạt động còn lại. Sơ đồ này được gọi là sơ đồ cấu trúc và nhằm mục đích hiển thị cách một vấn đề được chia nhỏ thành các vấn đề con. Trong sơ đồ này, bạn cũng có thể biết thứ tự thực hiện các hoạt động bằng cách đọc từ trái sang phải. Điều này không đúng với hầu hết các sơ đồ cấu trúc khác. Để xác định thứ tự thực hiện

thực sự của các chương trình con, bạn thường phải tham khảo chương trình chính nó.
Hình 1.2: Phân rã tác vụ "Làm bánh"



Hình 1.2 Văn bản mô tả đầy đủ

Một thuật ngữ giải quyết vấn đề cuối cùng liên quan đến quá trình lập trình. Các lập trình viên chuyên nghiệp phát triển chương trình theo từng giai đoạn. Thay vì cố gắng tạo ra một chương trình hoàn chỉnh ngay lập tức, họ chọn một phần của vấn đề để thực hiện trước. Sau đó, họ thêm phần khác, và phần khác nữa. Chương trình tổng thể được xây dựng từ từ, từng phần một. Quy trình này được gọi là cải tiến theo từng bước hoặc tinh chỉnh theo từng bước.

Cải tiến theo từng bước

Quá trình tạo ra một chương trình theo từng giai đoạn, thêm chức năng mới ở mỗi giai đoạn. Một đặc điểm quan trọng của mỗi bước lặp lại là bạn có thể kiểm tra nó để đảm bảo rằng phần đó hoạt động trước khi tiếp tục.

Bây giờ, hãy xem một cấu trúc sẽ cho phép bạn cải tiến từng bước các chương trình Java của bạn để cải thiện cấu trúc và giảm sự dư thừa của chúng: các phương thức tĩnh.

Các phương thức tĩnh

Java được thiết kế cho các đối tượng, và lập trình trong Java thường liên quan đến việc phân rã một vấn đề thành nhiều đối tượng khác nhau, mỗi đối tượng có các phương thức thực hiện các nhiệm vụ cụ thể. Bạn sẽ thấy cách điều này hoạt động trong các chương sau, nhưng bây giờ, chúng ta sẽ khám phá phân rã theo quy trình. Chúng ta sẽ hoãn việc xem xét một số chi tiết của Java trong khi chúng ta thảo luận về lập trình nói chung.

Hãy xem chương trình sau, nó vẽ hai hộp văn bản trên console:

```
1 // Chương trình này vẽ hai hình hộp.

2 public class DrawBoxes {

3     public static void main(String[] args) {

4         System.out.println("+------+");

5         System.out.println("|      |");

6         System.out.println("|      |");

7         System.out.println("+------+");

8         System.out.println();

9         System.out.println("+------+");

10        System.out.println("|      |");

11        System.out.println("|      |");

12        System.out.println("+------+");
```

```
13    }

14 }
```

Chương trình hoạt động đúng, nhưng bốn dòng được sử dụng để vẽ hộp xuất hiện hai lần. Sự dư thừa này là không mong muốn vì nhiều lý do. Ví dụ, bạn có thể muốn thay đổi diện mạo của các hộp, trong trường hợp đó bạn sẽ phải thực hiện tất cả các chỉnh sửa hai lần. Ngoài ra, bạn có thể muốn vẽ thêm các hộp, điều này sẽ yêu cầu bạn phải gõ thêm các bản sao của (hoặc sao chép và dán) các dòng dư thừa.

Chúng ta có thể cải thiện chương trình bằng cách giới thiệu một lệnh mới để vẽ hộp và sau đó thực hiện lệnh đó hai lần. Java không có lệnh "vẽ một hộp", nhưng bạn có thể tạo một lệnh như vậy. Một lệnh được đặt tên như vậy gọi là phương thức tĩnh.

Phương thức tĩnh

Một khối các câu lệnh Java được đặt tên. Các phương thức tĩnh là các đơn vị của phân rã theo quy trình. Chúng ta thường phân rã một lớp thành nhiều phương thức tĩnh, mỗi phương thức giải quyết một phần của vấn đề tổng thể. Ví dụ, đây là một phương thức tĩnh để vẽ một hộp:

```
public static void drawBox() {

    System.out.println("+------+");

    System.out.println("|      |");

    System.out.println("|      |");

    System.out.println("+------+");
```

```
}
```

Bạn đã thấy một phương thức tĩnh gọi là main trong các chương trình trước đó. Nhớ lại rằng phương thức main có hình thức sau:

```
public static void main(String[] args) {

    <statement>;

    <statement>;

    ...

    <statement>;

}
```

Các phương thức tĩnh mà bạn viết sẽ có cấu trúc tương tự:

```
public static void <name>() {

    <statement>;

    <statement>;

    ...

    <statement>;

}
```

Dòng đầu tiên được gọi là tiêu đề phương thức. Bạn chưa cần phải hiểu hoàn toàn ý nghĩa của từng phần trong tiêu đề này trong Java; hiện tại, chỉ cần nhớ rằng bạn sẽ cần viết public static

void, theo sau là tên bạn muốn đặt cho phương thức, theo sau là một cặp dấu ngoặc đơn. Tóm tắt, đây là ý nghĩa của các từ trong tiêu đề:

- Từ khóa public chỉ ra rằng phương thức này có sẵn để được sử dụng bởi tất cả các phần của chương trình của bạn. Tất cả các phương thức bạn viết sẽ là public.

- Từ khóa static chỉ ra rằng đây là một phương thức tĩnh (theo kiểu quy trình, không phải theo kiểu đối tượng). Hiện tại, tất cả các phương thức bạn viết sẽ là static, cho đến khi bạn học về việc định nghĩa đối tượng trong Chương 8.

- Từ khóa void chỉ ra rằng phương thức này thực thi các câu lệnh nhưng không trả về giá trị nào. (Các phương thức khác mà bạn sẽ thấy sau này tính toán và trả về giá trị.)

- <name> (ví dụ, drawBox) là tên của phương thức.

- Các dấu ngoặc đơn trống chỉ định một danh sách (trong trường hợp này, một danh sách trống) các giá trị được gửi đến phương thức của bạn như là đầu vào; các giá trị này gọi là tham số và sẽ không được đưa vào các phương thức của bạn cho đến Chương 3.

Việc bao gồm từ khóa static cho mỗi phương thức bạn định nghĩa có thể cảm thấy rườm rà. Các sách giáo khoa Java khác thường không thảo luận về các phương thức tĩnh sớm như chúng tôi làm ở đây; thay vào đó, chúng cho thấy các kỹ thuật khác để phân rã các vấn đề. Nhưng dù các phương thức tĩnh yêu cầu một chút công sức để tạo ra, chúng là công cụ mạnh mẽ và hữu ích để cải thiện các chương trình Java cơ bản.

Sau tiêu đề trong phương thức mẫu của chúng tôi, một loạt các câu lệnh println tạo thành phần thân của phương thức tĩnh này. Giống như phương thức main, các câu lệnh của phương thức này được thực thi theo thứ tự từ đầu đến cuối.

Bằng cách định nghĩa phương thức drawBox, bạn đã đặt một cái tên đơn giản cho chuỗi các câu lệnh println này. Giống như việc nói với trình biên dịch Java, "Mỗi khi tôi bảo bạn 'drawBox', tôi thực sự muốn bạn thực thi các câu lệnh println trong phương thức drawBox." Nhưng lệnh sẽ không được thực thi trừ khi phương thức main của chúng tôi rõ ràng nói rằng nó muốn làm như vậy. Hành động thực thi một phương thức tĩnh được gọi là gọi phương thức.

Gọi phương thức

Là một lệnh để thực thi một phương thức khác, làm cho tất cả các câu lệnh bên trong phương thức đó được thực thi.

Để thực thi lệnh drawBox, hãy thêm dòng này vào phương thức main của chương trình:

```
drawBox();
```

Vì chúng ta muốn thực thi lệnh drawBox hai lần (để vẽ hai hộp), phương thức main nên chứa hai lệnh gọi đến phương thức drawBox. Chương trình sau sử dụng phương thức drawBox để tạo ra cùng một đầu ra như chương trình DrawBoxes gốc:

```
1 // Vẽ hai hình hộp sử dụng phương thức tĩnh.

2 public class DrawBoxes2 {

3     public static void main(String[] args) {
```

```
4         drawBox();
5         System.out.println();
6         drawBox();
7     }
8     public static void drawBox() {
9         System.out.println("+------+");
10        System.out.println("|      |");
11        System.out.println("|      |");
12        System.out.println("+------+");
13    }
14 }
```

Dòng chảy điều khiển

Điều khó hiểu nhất về các phương thức tĩnh là các chương trình với các phương thức tĩnh không thực thi tuần tự từ trên xuống dưới. Thay vào đó, mỗi lần chương trình gặp một lệnh gọi phương thức tĩnh, việc thực thi chương trình "nhảy" đến phương thức tĩnh đó, thực thi từng câu lệnh trong phương thức đó theo thứ tự, và sau đó "nhảy" trở lại điểm mà lệnh gọi bắt đầu và tiếp tục thực thi. Thứ tự trong đó các câu lệnh của chương trình được thực thi được gọi là dòng chảy điều khiển của chương trình.

Dòng chảy điều khiển

Thứ tự trong đó các câu lệnh của một chương trình Java được thực thi.

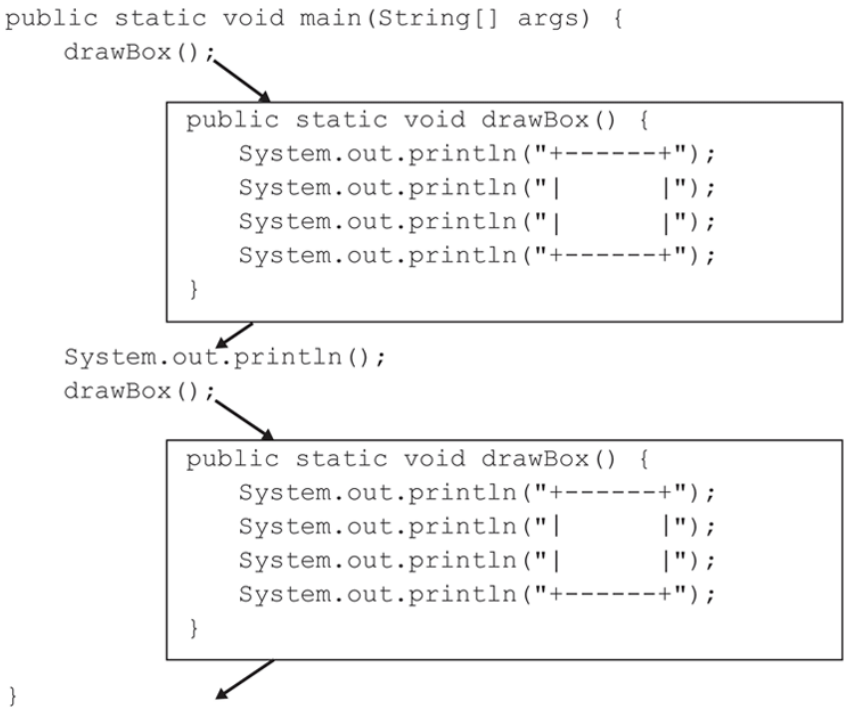
Hãy xem dòng chảy điều khiển của chương trình DrawBoxes2 được trình bày trước đó. Nó có hai phương thức. Phương thức đầu tiên là phương thức main quen thuộc, và phương thức thứ hai là drawBox. Như trong bất kỳ chương trình Java nào, việc thực thi bắt đầu với phương thức main:

```
public static void main(String[] args) {

    drawBox();

    System.out.println();

    drawBox();

}
```

Theo một nghĩa nào đó, việc thực thi chương trình này là tuần tự: Mỗi câu lệnh được liệt kê trong phương thức main được thực thi lần lượt, từ câu lệnh đầu tiên đến câu lệnh cuối cùng.

Nhưng phương thức main này bao gồm hai lệnh gọi khác nhau đến phương thức drawBox. Chương trình này sẽ thực hiện ba việc khác nhau: thực thi drawBox, thực thi một lệnh println, sau đó thực thi drawBox một lần nữa.

Sơ đồ dưới đây chỉ ra dòng chảy điều khiển được tạo ra bởi chương trình này.



1.4-5 Văn bản mô tả đầy đủ

Theo sơ đồ, bạn có thể thấy rằng chín câu lệnh println được thực thi. Đầu tiên, bạn chuyển điều khiển đến phương thức drawBox và thực thi bốn câu lệnh của nó. Sau đó, bạn trở về phương thức main và thực thi câu lệnh println của nó. Sau đó, bạn chuyển điều khiển lần thứ hai đến drawBox và một lần nữa thực thi bốn câu lệnh của nó. Việc thực hiện các lệnh gọi phương thức này gần giống như việc sao chép và dán mã của phương thức vào phương thức main. Kết quả là, chương trình này có hành vi giống hệt như phương thức main gồm chín dòng của chương trình DrawBoxes:

```
public static void main(String[] args) {
   System.out.println("+------+");
   System.out.println("|      |");
   System.out.println("|      |");
   System.out.println("+------+");
   System.out.println();
   System.out.println("+------+");
   System.out.println("|      |");
```

```
    System.out.println("|      |");
    System.out.println("+------+");
}
```

Phiên bản này đơn giản hơn về mặt dòng chảy điều khiển, nhưng phiên bản đầu tiên tránh được sự dư thừa khi các câu lệnh println xuất hiện nhiều lần. Nó cũng cung cấp cái nhìn rõ hơn về cấu trúc của giải pháp. Trong phiên bản gốc, rõ ràng có một nhiệm vụ phụ gọi là drawBox đang được thực hiện hai lần. Ngoài ra, mặc dù phiên bản cuối cùng của phương thức main có ít dòng mã hơn so với chương trình DrawBoxes2, hãy xem xét điều gì sẽ xảy ra nếu bạn muốn thêm một hộp thứ ba vào đầu ra. Bạn sẽ phải thêm năm câu lệnh println cần thiết một lần nữa, trong khi ở các chương trình sử dụng phương thức drawBox, bạn chỉ cần thêm một câu lệnh println nữa và một lệnh gọi phương thức thứ ba. Java cho phép bạn định nghĩa các phương thức theo bất kỳ thứ tự nào bạn thích. Một quy ước phổ biến là đặt phương thức main là phương thức đầu tiên hoặc cuối cùng trong lớp. Trong sách giáo khoa này, chúng tôi sẽ thường đặt main trước, nhưng các chương trình sẽ hoạt động giống nhau nếu chúng tôi đổi thứ tự. Ví dụ, chương trình được sửa đổi sau đây hoạt động giống hệt như chương trình DrawBoxes2 trước đó:

```
1 // Vẽ hai hình hộp sử dụng phương thức tĩnh.
2 public class DrawBoxes3 {
3     public static void drawBox() {
4         System.out.println("+------+");
5         System.out.println("|      |");
6         System.out.println("|      |");
7         System.out.println("+------+");
8     }
9     public static void main(String[] args) {
10        drawBox();
11        System.out.println();
12        drawBox();
```

```
13    }
14 }
```

Phương thức main luôn là điểm khởi đầu cho việc thực thi chương trình, và từ điểm khởi đầu đó, bạn có thể xác định thứ tự mà các phương thức khác được gọi.

Các phương thức gọi các phương thức khác

Phương thức main không phải là nơi duy nhất bạn có thể gọi một phương thức khác. Thực tế, bất kỳ phương thức nào cũng có thể gọi bất kỳ phương thức nào khác. Do đó, dòng chảy điều khiển có thể trở nên khá phức tạp. Hãy xem xét ví dụ sau đây, một chương trình hơi lạ. Chúng tôi sử dụng các từ vô nghĩa (“foo”, “bar”, “baz”, và “mumble”) có chủ đích vì chương trình không có mục đích làm cho nó có ý nghĩa.

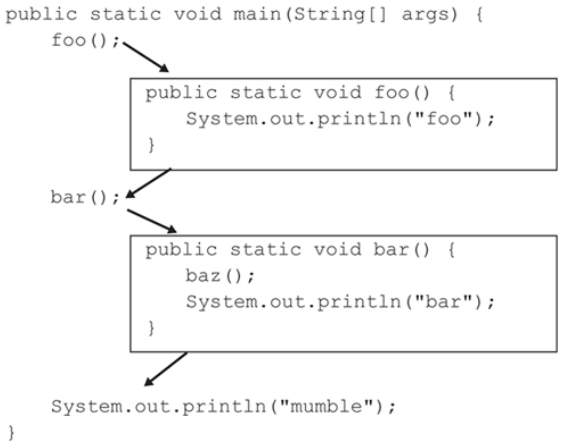
```
1 public class FooBarBazMumble {

2     public static void main(String[] args) {

3         foo();

4         bar();

5         System.out.println("mumble");

6     }

7     public static void foo() {

8         System.out.println("foo");

9     }
```

```
10    public static void bar() {
11        baz();
12        System.out.println("bar");
13    }
14    public static void baz() {
15        System.out.println("baz");
16    }
17 }
```

Bạn không thể dễ dàng biết được chương trình này sản xuất ra đầu ra gì, vì vậy hãy khám phá chi tiết những gì chương trình đang làm. Hãy nhớ rằng Java luôn bắt đầu với phương thức gọi là main. Trong chương trình này, phương thức main gọi phương thức foo và phương thức bar, sau đó thực thi một câu lệnh println:

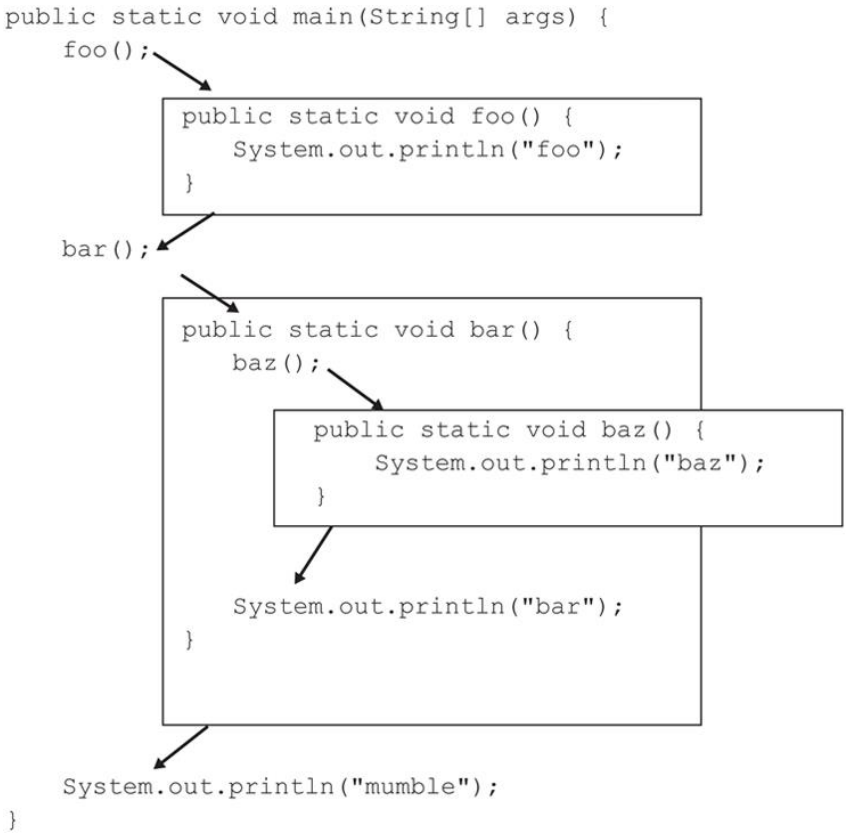
```
public static void main(String[] args) {
    foo();
    bar();
    System.out.println("mumble");
}
```

Mỗi lệnh gọi phương thức này sẽ mở rộng thành nhiều câu lệnh hơn. Hãy mở rộng các lệnh gọi phương thức foo và bar trước:




```
public static void foo() {

    System.out.println("foo");

}



public static void bar() {

    baz();

    System.out.println("bar");

}
```

Điều này giúp chúng ta làm rõ hơn bức tranh về dòng chảy điều khiển, nhưng lưu ý rằng bar gọi phương thức baz, vì vậy chúng ta cũng phải mở rộng điều đó.



Cuối cùng, chúng ta đã hoàn tất bức tranh về dòng chảy điều khiển của chương trình này. Như vậy, chương trình sản xuất đầu ra sau:

```
foo

baz

bar

mumble
```

Chúng ta sẽ thấy một ví dụ hữu ích hơn về các phương thức gọi các phương thức khi chúng ta đi qua nghiên cứu tình huống ở cuối chương.

BẠN CÓ BIẾT?

Từ điển Hacker Mới

Các nhà khoa học máy tính và lập trình viên máy tính sử dụng rất nhiều thuật ngữ mà có thể gây nhầm lẫn cho người mới bắt đầu. Một nhóm các chuyên gia phần mềm do Eric Raymond dẫn đầu đã tập hợp nhiều thuật ngữ chuyên ngành trong một cuốn sách gọi là *Từ điển Hacker Mới*. Bạn có thể mua cuốn sách này, hoặc bạn có thể duyệt nó trực tuyến tại trang web của Eric: http://catb.org/esr/jargon/html/frames.html.

Ví dụ, nếu bạn tra cứu từ “foo”, bạn sẽ thấy định nghĩa sau: “Được sử dụng rất chung để làm tên mẫu cho bất kỳ thứ gì, đặc biệt là các chương trình và tệp.” Nói cách khác, khi chúng ta cần tìm một từ vô nghĩa, chúng ta sử dụng “foo”.

Từ điển Hacker Mới chứa rất nhiều thông tin lịch sử về nguồn gốc của các thuật ngữ chuyên ngành. Mục từ “foo” bao gồm một cuộc thảo luận dài về thuật ngữ kết hợp “foobar” và cách nó trở nên phổ biến trong giới kỹ sư.

Nếu bạn muốn biết thêm về nội dung, hãy xem các mục từ “bug”, “hacker”, “bogosity”, và “bogo-sort”.

Nguồn: Raymond, Eric S., *Từ điển Hacker Mới*, trang 38, © 1991 Massachusetts Institute of Technology, theo sự cho phép của The MIT Press.

Một ví dụ về lỗi thời gian chạy

Lỗi thời gian chạy xảy ra khi một lỗi khiến chương trình của bạn không thể tiếp tục thực thi. Điều gì có thể gây ra điều này? Một ví dụ là nếu bạn yêu cầu máy tính tính toán một giá trị không hợp lệ, chẳng hạn như 1 chia cho 0. Một ví dụ khác là nếu chương trình của bạn cố gắng đọc dữ liệu từ một tệp không tồn tại.

Chúng ta chưa thảo luận về cách tính toán giá trị hoặc đọc tệp, nhưng có một cách bạn có thể "vô tình" gây ra lỗi thời gian chạy. Cách để làm điều này là viết một phương thức tĩnh gọi chính nó. Nếu bạn làm như vậy, chương trình của bạn sẽ không dừng lại, vì phương thức sẽ tiếp tục gọi chính nó vô hạn, cho đến khi máy tính hết bộ nhớ. Khi điều này xảy ra, chương trình sẽ in ra một số lượng lớn các dòng đầu ra và sau đó dừng thực thi với thông báo lỗi gọi là StackOverflowError. Dưới đây là một ví dụ:

```
1 // Chương trình này chứa một hàm gọi chính nó

2 // vô hạn, gây ra lỗi thời gian chạy.

3 public class Infinite {

4     public static void main(String[] args) {

5         oops();

6     }

7     public static void oops() {

8         System.out.println("Make it stop!");

9         oops();
```

```
10    }

11 }
```

Chương trình không may này sản xuất đầu ra sau (với các nhóm dòng giống hệt nhau được đại diện bởi “...”):

```
Make it stop!

Make it stop!

Make it stop!

Make it stop!

Make it stop!

Make it stop!

Make it stop!

Make it stop!

Make it stop!

...

Make it stop!

Exception in thread "main" java.lang.StackOverflowError

        at
sun.nio.cs.SingleByteEncoder.encodeArrayLoop(Unknown
Source)
```

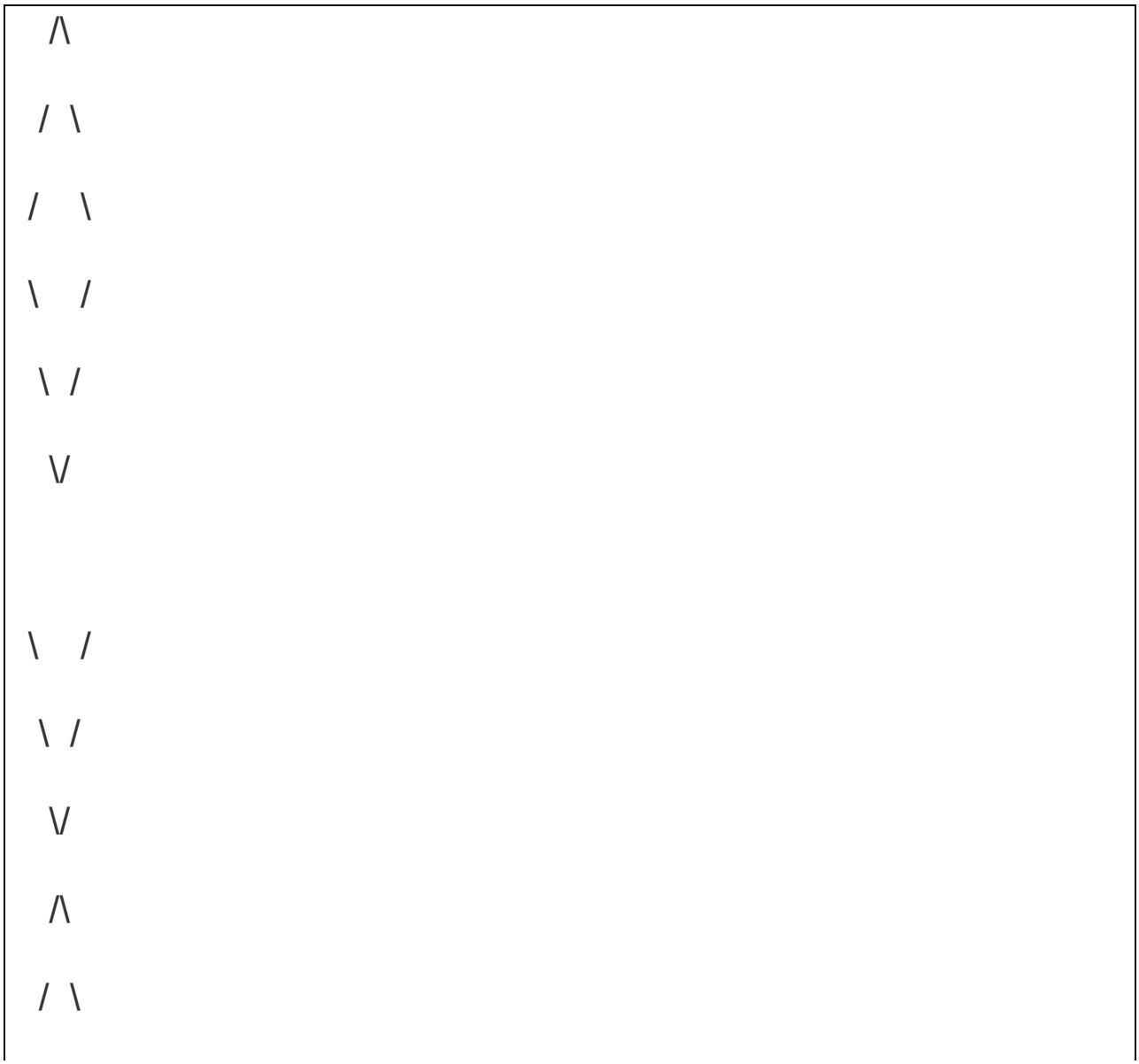
```
        at sun.nio.cs.SingleByteEncoder.encodeLoop(Unknown
Source)

        at java.nio.charset.CharsetEncoder.encode(Unknown
Source)

        at
sun.nio.cs.StreamEncoder$CharsetSE.implWrite(Unknown
Source)

        at sun.nio.cs.StreamEncoder.write(Unknown Source)

        at java.io.OutputStreamWriter.write(Unknown Source)

        at java.io.BufferedWriter.flushBuffer(Unknown Source)

        at java.io.PrintStream.newLine(Unknown Source)

        at java.io.PrintStream.println(Unknown Source)

        at Infinite.oops(Infinite.java:7)

        at Infinite.oops(Infinite.java:8)

        at Infinite.oops(Infinite.java:8)

        at Infinite.oops(Infinite.java:8)

        at ...
```

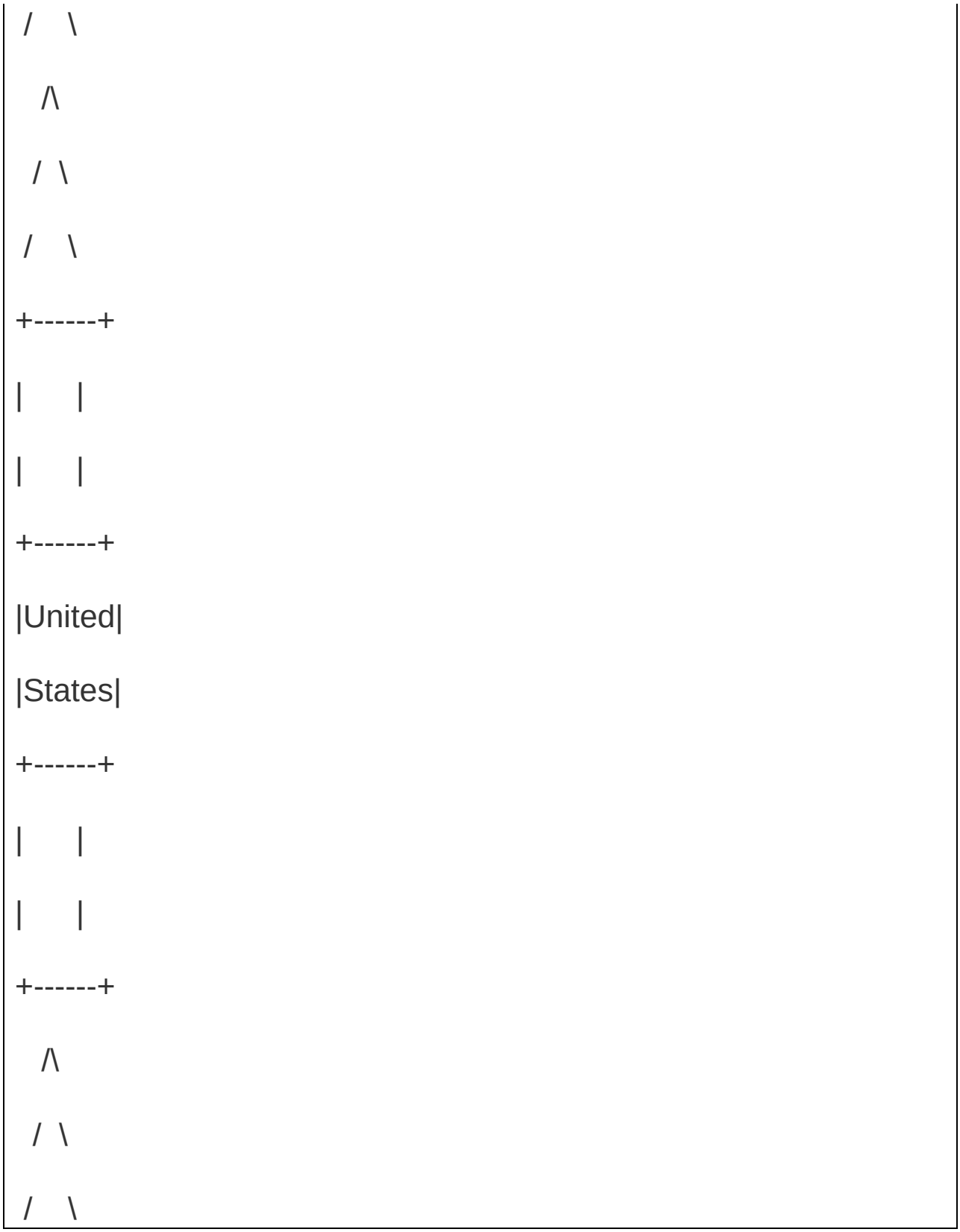
Lỗi thời gian chạy là điều không may mà bạn sẽ phải đối mặt khi học lập trình. Bạn sẽ phải cẩn thận đảm bảo rằng các chương trình của bạn không chỉ biên dịch thành công mà còn không chứa bất kỳ lỗi nào gây ra lỗi thời gian chạy. Cách phổ biến nhất để

phát hiện và sửa lỗi thời gian chạy là chạy chương trình nhiều lần để kiểm tra hành vi của nó.

1.5 Nghiên cứu tình huống: Vẽ Hình

Trước đó trong chương, bạn đã thấy một chương trình gọi là DrawFigures1 tạo ra đầu ra sau:




+------+
|United|
|States|
+------+
+------+

Chương trình này thực hiện điều đó bằng cách sử dụng một chuỗi dài các câu lệnh println trong phương thức chính (main). Trong phần này, bạn sẽ cải thiện chương trình bằng cách sử dụng các phương thức tĩnh để phân tách quy trình, nhằm nắm bắt cấu trúc và loại bỏ sự trùng lặp. Sự trùng lặp có thể rõ ràng hơn, nhưng hãy bắt đầu bằng cách cải thiện cách chương trình nắm bắt cấu trúc của nhiệm vụ tổng thể.

Phiên bản có cấu trúc

Nếu bạn nhìn kỹ vào đầu ra, bạn sẽ thấy rằng nó có một cấu trúc mà sẽ là mong muốn để nắm bắt trong cấu trúc chương trình. Đầu ra được chia thành ba hình con: hình thoi, chữ X, và tên lửa.

Bạn có thể chỉ ra cấu trúc của chương trình tốt hơn bằng cách chia nó thành các phương thức tĩnh. Vì có ba hình con, bạn có thể tạo ba phương thức, một cho mỗi hình con. Chương trình sau đây tạo ra đầu ra giống như DrawFigures1:

```
1 // Chương trình này vẽ ba hình văn bản sử dụng

2 // phương thức tĩnh để đại diện cho mỗi hình.

3 public class DrawFigures2 {

4     public static void main(String[] args) {

5         drawDiamond();

6         drawX();

7         drawRocket();

8     }
```

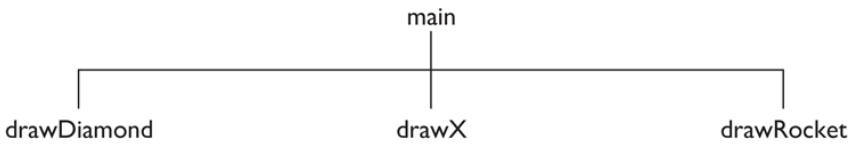
```
9    public static void drawDiamond() {
10       System.out.println("   /\\");
11       System.out.println("  /  \\");
12       System.out.println(" /    \\");
13       System.out.println(" \\    /");
14       System.out.println("  \\  /");
15       System.out.println("   \\/");
16       System.out.println();
17   }
18   public static void drawX() {
19       System.out.println(" \\    /");
20       System.out.println("  \\  /");
21       System.out.println("   \\/");
22       System.out.println("    /\\");
23       System.out.println("   /  \\");
24       System.out.println("  /    \\");
25       System.out.println();
```

```
26    }
27    public static void drawRocket() {
28        System.out.println("    /\\");
29        System.out.println("  /  \\");
30        System.out.println(" /    \\");
31        System.out.println("+------+");
32        System.out.println("|      |");
33        System.out.println("|      |");
34        System.out.println("+------+");
35        System.out.println("|United|");
36        System.out.println("|States|");
37        System.out.println("+------+");
38        System.out.println("|      |");
39        System.out.println("|      |");
40        System.out.println("+------+");
41        System.out.println("    /\\");
42        System.out.println("  /  \\");
```

```
43        System.out.println(" /    \\");

44    }

45 }
```

Chương trình này xuất hiện trong một lớp gọi là DrawFigures2 và có bốn phương thức tĩnh được định nghĩa trong đó. Phương thức tĩnh đầu tiên là phương thức chính (main) thông thường, gọi ba phương thức. Ba phương thức được gọi bởi phương thức chính xuất hiện tiếp theo.

Hình 1.3 là sơ đồ cấu trúc cho phiên bản này của chương trình. Lưu ý rằng nó có hai cấp độ cấu trúc. Vấn đề tổng thể được chia thành ba nhiệm vụ con.



Phiên bản cuối cùng không bị trùng lặp

Chương trình vẫn có thể được cải thiện thêm. Mỗi hình con trong ba hình có các phần tử riêng lẻ, và một số phần tử đó xuất hiện trong nhiều hơn một trong ba hình con. Chương trình in ra nhóm các dòng trùng lặp sau nhiều lần:

```
  /\

 /  \

/    \
```

Một phiên bản tốt hơn của chương trình trước đó thêm một phương thức bổ sung cho mỗi phần đầu ra trùng lặp. Các phần trùng lặp là nửa trên và nửa dưới của hình thoi và hộp được sử dụng trong tên lửa. Đây là chương trình được cải thiện:

```
1 // Chương trình này vẽ ba hình văn bản sử dụng các phương thức tĩnh

2 // để tạo cấu trúc và tránh sự trùng lặp.

3 public class DrawFigures3 {

4     public static void main(String[] args) {

5         drawDiamond();

6         drawX();

7         drawRocket();

8     }

9     public static void drawDiamond() {

10        drawCone();

11        drawV();

12        System.out.println();

13    }

14    public static void drawX() {
```

```
15        drawV();
16        drawCone();
17        System.out.println();
18    }
19    public static void drawRocket() {
20        drawCone();
21        drawBox();
22        System.out.println("|United|");
23        System.out.println("|States|");
24        drawBox();
25        drawCone();
26        System.out.println();
27    }
28    public static void drawBox() {
29        System.out.println("+------+");
30        System.out.println("|      |");
31        System.out.println("|      |");
```

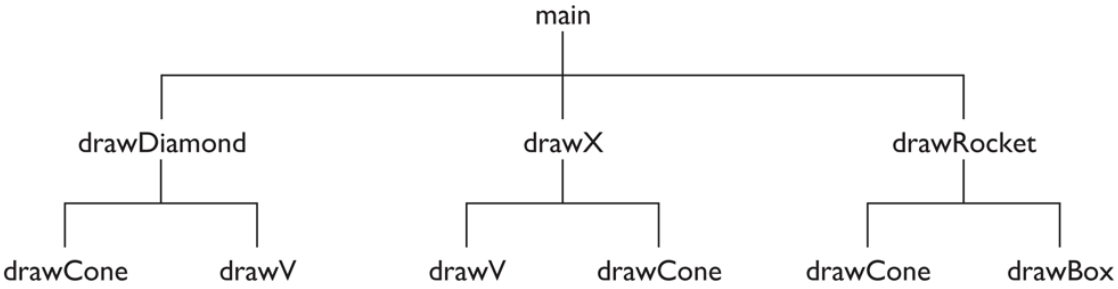
```
32        System.out.println("+------+");
33    }
34    public static void drawCone() {
35        System.out.println("   /\\");
36        System.out.println("  /  \\");
37        System.out.println(" /    \\");
38    }
39    public static void drawV() {
40        System.out.println(" \\    /");
41        System.out.println("  \\  /");
42        System.out.println("   \\/");
43    }
44 }
```

Chương trình này, hiện gọi là DrawFigures3, có bảy phương thức tĩnh được định nghĩa trong đó. Phương thức tĩnh đầu tiên là phương thức chính (main), gọi ba phương thức. Ba phương thức này lại gọi ba phương thức khác, và các phương thức này xuất hiện tiếp theo.

Phân Tích Luồng Thực Thi

Sơ đồ cấu trúc trong Hình 1.4 cho thấy các phương thức tĩnh nào mà phương thức main gọi và các phương thức tĩnh nào mà mỗi phương thức đó gọi. Như bạn có thể thấy, chương trình này có ba cấp độ cấu trúc và hai cấp độ phân rã. Nhiệm vụ tổng thể được chia thành ba nhiệm vụ con, mỗi nhiệm vụ con lại có hai nhiệm vụ nhỏ hơn.

Hình 1.4 Phân rã của DrawFigures3



Hình 1.4 Văn Bản Thay Thế Đầy Đủ

Một chương trình có các phương thức có luồng điều khiển phức tạp hơn so với chương trình không có chúng, nhưng các quy tắc vẫn khá đơn giản. Hãy nhớ rằng khi một phương thức được gọi, máy tính sẽ thực thi các câu lệnh trong thân của phương thức đó. Sau đó, máy tính sẽ tiếp tục thực hiện câu lệnh tiếp theo sau lệnh gọi phương thức. Cũng cần nhớ rằng máy tính luôn bắt đầu với phương thức main, thực thi các câu lệnh từ đầu đến cuối.

Vì vậy, để thực thi chương trình DrawFigures3, máy tính trước tiên thực thi phương thức main. Sau đó, phương thức main lần lượt thực thi thân của phương thức drawDiamond. Phương thức drawDiamond thực thi các phương thức drawCone và drawV (theo thứ tự đó). Khi drawDiamond kết thúc thực thi, điều khiển chuyển sang câu lệnh tiếp theo trong thân của phương thức main: lệnh gọi phương thức drawX.

Một phân tích hoàn chỉnh về luồng điều khiển từ phương thức tĩnh này sang phương thức tĩnh khác trong DrawFigures3 như sau:

1. main
2. drawDiamond
3. drawCone
4. drawV
5. drawX
6. drawV
7. drawCone
8. drawRocket
9. drawCone
10. drawBox
11. drawBox
12. drawCone

Hãy nhớ rằng thứ tự bạn định nghĩa các phương thức không nhất thiết phải song song với thứ tự mà chúng được thực thi. Thứ tự thực thi được xác định bởi thân của phương thức main và bởi thân của các phương thức được gọi từ main. Khai báo phương thức tĩnh giống như một mục từ trong từ điển—nó định nghĩa một từ, nhưng không chỉ rõ cách từ đó sẽ được sử dụng. Thân của

phương thức main của chương trình này yêu cầu thực thi drawDiamond trước, sau đó drawX, và cuối cùng drawRocket. Đây là thứ tự thực thi, không phụ thuộc vào thứ tự mà các phương thức được định nghĩa.

Java cho phép bạn định nghĩa các phương thức theo bất kỳ thứ tự nào bạn muốn. Bắt đầu với main ở đầu và làm việc xuống các phương thức cấp thấp hơn là một cách tiếp cận phổ biến, nhưng nhiều người thích ngược lại, đặt các phương thức cấp thấp trước và main ở cuối. Java không quan tâm bạn sử dụng thứ tự nào, vì vậy bạn có thể quyết định cho riêng mình và làm những gì bạn nghĩ là tốt nhất. Tuy nhiên, sự nhất quán là quan trọng, để bạn có thể dễ dàng tìm thấy một phương thức sau này trong một chương trình lớn.

Quan trọng là các chương trình DrawFigures1, DrawFigures2, và DrawFigures3 đều tạo ra cùng một kết quả trên bảng điều khiển. Mặc dù DrawFigures1 có thể là chương trình dễ đọc nhất đối với người mới bắt đầu, nhưng DrawFigures2 và đặc biệt là DrawFigures3 có nhiều ưu điểm hơn. Một giải pháp được cấu trúc tốt dễ hiểu hơn và các phương thức trở thành một cách giải thích chương trình. Ngoài ra, các chương trình với phương thức linh hoạt hơn và dễ dàng thích ứng với các nhiệm vụ tương tự nhưng khác nhau. Bạn có thể lấy bảy phương thức được định nghĩa trong DrawFigures3 và viết một chương trình mới để tạo ra một kết quả lớn hơn và phức tạp hơn. Xây dựng các phương thức tĩnh để tạo ra các lệnh mới tăng cường tính linh hoạt của bạn mà không làm tăng sự phức tạp không cần thiết. Ví dụ, bạn có thể thay thế phương thức main bằng một phiên bản gọi các phương thức khác theo thứ tự mới sau đây. Chương trình sẽ tạo ra kết quả gì?

```
public static void main(String[] args) {

```

```
    drawCone();

    drawCone();

    drawRocket();

    drawX();

    drawRocket();

    drawDiamond();

    drawBox();

    drawDiamond();

    drawX();

    drawRocket();

}
```

Tóm tắt Chương

Máy tính thực thi các tập hợp lệnh gọi là chương trình. Máy tính lưu trữ thông tin nội bộ dưới dạng các chuỗi số 0 và 1 (các số nhị phân).

Lập trình và khoa học máy tính liên quan đến các thuật toán, là các mô tả từng bước để giải quyết vấn đề.

Java là một ngôn ngữ lập trình hướng đối tượng hiện đại được phát triển bởi Sun Microsystems, hiện thuộc sở hữu của Oracle

Corporation, có một tập hợp lớn các thư viện mà bạn có thể sử dụng để xây dựng các chương trình phức tạp.

Một chương trình được chuyển đổi từ văn bản thành các lệnh máy tính bởi một chương trình khác gọi là trình biên dịch. Trình biên dịch của Java chuyển các chương trình Java thành một định dạng đặc biệt gọi là mã byte Java, được thực thi bằng một chương trình đặc biệt gọi là Môi trường Thực thi Java (Java Runtime Environment).

Các lập trình viên Java thường hoàn thành công việc của họ bằng cách sử dụng một trình soạn thảo gọi là Môi trường Phát triển Tích hợp (IDE). Các lệnh có thể khác nhau từ môi trường này sang môi trường khác, nhưng quy trình ba bước luôn được thực hiện:

1. Gõ chương trình dưới dạng một lớp Java.
2. Biên dịch tệp chương trình.
3. Chạy phiên bản đã biên dịch của chương trình.

Java sử dụng lệnh gọi là System.out.println để hiển thị văn bản trên màn hình console.

Các từ viết trong chương trình có thể mang ý nghĩa khác nhau. Từ khóa là các từ được đặt sẵn đặc biệt thuộc ngôn ngữ. Các định danh là các từ do lập trình viên định nghĩa để đặt tên cho các thực thể trong chương trình. Các từ cũng có thể được đưa vào chuỗi, là các đoạn văn bản có thể được in ra màn hình console.

Các chương trình Java có sự định dạng và bố trí hợp lý sẽ dễ đọc hơn đối với lập trình viên. Đọc dễ hơn cũng được cải thiện bằng cách viết các ghi chú gọi là chú thích bên trong chương trình.

Ngôn ngữ Java có cú pháp, hoặc một tập hợp các lệnh hợp lệ có thể được sử dụng. Một chương trình Java không tuân theo cú

pháp đúng sẽ không được biên dịch. Một chương trình có thể biên dịch nhưng viết sai có thể vẫn chứa lỗi gọi là ngoại lệ xảy ra khi chương trình chạy. Một loại lỗi thứ ba là lỗi logic hoặc lỗi ý định. Loại lỗi này xảy ra khi chương trình chạy nhưng không thực hiện những gì lập trình viên mong muốn.

Các lệnh trong chương trình được gọi là câu lệnh. Một lớp có thể nhóm các câu lệnh thành các lệnh lớn hơn gọi là phương thức tĩnh. Các phương thức tĩnh giúp lập trình viên nhóm mã thành các phần có thể tái sử dụng. Một phương thức tĩnh quan trọng phải có trong mọi chương trình được gọi là main.

Cải tiến lặp đi lặp lại là quá trình xây dựng chương trình từng phần một, kiểm tra chương trình ở mỗi bước trước khi tiến đến bước tiếp theo.

Các nhiệm vụ lập trình phức tạp nên được chia nhỏ thành các nhiệm vụ chính mà máy tính phải thực hiện. Quá trình này gọi là phân rã quy trình. Việc sử dụng chính xác các phương thức tĩnh hỗ trợ phân rã quy trình.

Các vấn đề tự kiểm tra

Phần 1.1: Khái niệm tính toán cơ bản

1. Tại sao máy tính sử dụng các số nhị phân?
2. Chuyển đổi từng số thập phân sau đây thành số nhị phân tương ứng của nó: a. 6
   b. 44
   c. 72
   d. 131
3. Giá trị thập phân tương ứng của từng số nhị phân sau đây là gì? a. 100
   b. 1011

c. 101010
d. 1001110

4. Bằng lời của bạn, mô tả một thuật toán để nướng bánh quy. Giả sử bạn có một số lượng lớn bạn bè đói bụng, vì vậy bạn sẽ muốn làm nhiều mẻ bánh quy!

5. Sự khác biệt giữa tệp MyProgram.java và tệp MyProgram.class là gì?

Phần 1.2: Và bây giờ—Java

6. Những gì sau đây có thể được sử dụng trong chương trình Java dưới dạng định danh? println first-name AnnualSalary "hello" ABC
isTheAnswer for sum_of_data _average B4

7. Cú pháp đúng để xuất một thông điệp là gì? a. System.println(Hello, world!);
b. System.println.out('Hello, world!');
c. System.println("Hello, world!");
d. System.out.println("Hello, world!");
e. Out.system.println"(Hello, world!)";

8. Đầu ra được tạo ra từ các câu lệnh sau là gì?
System.out.println("\"Quotes\"");
System.out.println("Slashes \\/");
System.out.println("How '\"confounding' \"\\\" it is!");

9. Đầu ra được tạo ra từ các câu lệnh sau là gì?
System.out.println("name\tage\theight");
System.out.println("Archie\t17\t5'9\"");
System.out.println("Betty\t17\t5'6\"");
System.out.println("Jughead\t16\t6'");

10. Đầu ra được tạo ra từ các câu lệnh sau là gì?
System.out.println("Shaq is 7'1");

```
System.out.println("The string \"\" is an empty message.");
System.out.println("\\'\"\"");
```

11. Đầu ra được tạo ra từ các câu lệnh sau là gì?

```
System.out.println("\ta\tb\tc");
System.out.println("\\\\");
System.out.println("'");
System.out.println("\"\"\"");
System.out.println("C:\\nin\\the downward spiral");
```

12. Đầu ra được tạo ra từ các câu lệnh sau là gì?

```
System.out.println("Dear \"DoubleSlash\" magazine,");
System.out.println();
System.out.println("\tYour publication confuses me. Is it");
System.out.println("a \\\\ slash or a //// slash?");
System.out.println("\nSincerely,");
System.out.println("Susan \"Suzy\" Smith");
```

13. Dãy các câu lệnh println nào sẽ tạo ra đầu ra sau đây?

```
"Several slashes are sometimes seen,"
said Sally. "I've said so." See?
\ / \ // \\ ///
```

14. Dãy các câu lệnh println nào sẽ tạo ra đầu ra sau đây?

```
This is a test of your
knowledge of "quotes" used
in 'string literals.'
You're bound to "get it right"
if you read the section on
"quotes."
```

15. Viết một câu lệnh println tạo ra đầu ra sau đây: / \ // \ /// \\

16. Viết lại mã sau dưới dạng một dãy các câu lệnh System.out.println tương đương (tức là không sử dụng bất kỳ câu lệnh System.out.print nào):

```
System.out.println("Twas ");
System.out.print("brillig and the");
System.out.println(" ");
System.out.print(" slithy toves did");
System.out.print(" ");
System.out.println("gyre and");
System.out.println("gimble");
System.out.println();
System.out.println("in the wabe.");
```

17. Đầu ra của chương trình sau là gì? Lưu ý rằng chương trình chứa một số chú thích. 1

```
public class Commentary {

    public static void main(String[] args) {

        System.out.println("some lines of code");

        System.out.println("have // characters on them");

        System.out.println("which means ");  // that they are
comments

        // System.out.println("written by the programmer.");

        System.out.println("lines can also");

        System.out.println("have /* and */ characters");

        /* System.out.println("which represents");

        System.out.println("a multi-line style");

        */ System.out.println("of comment.");
```

```
    }
}
```

Mục 1.3: Lỗi trong chương trình

18. Tìm ba lỗi trong chương trình sau:

```
public MyProgram {
public static void main(String[] args) {
   System.out.println("Đây là một bài kiểm tra của")
   System.out.Println("hệ thống phát sóng khẩn cấp.")
}
}
```

19. Tìm bốn lỗi trong chương trình sau:

```
public class SecretMessage {
public static main(string[] args) {
   System.out.println("Nói bạn bè")
   System.out.println("và nhập);
}
}
```

20. Tìm bốn lỗi trong chương trình sau:

```
public class FamousSpeech
public static void main(String[]) {
   System.out.println("Bốn mươi năm trước,")
   System.out.println("các tổ phụ của chúng tôi đã đem đến")
   System.out.println("trên lục địa này một quốc gia mới")
   System.out.println("được hình thành trong tự do,")
   System.out.println("và cam kết với quan điểm")
   System.out.println("rằng");    /* phần này nên
   System.out.println("tất cả");       thực sự nói,
   System.out.println("những người");       "tất cả là NGƯỜI!" */
   System.out.println("là")
   System.out.println("được tạo ra")
   System.out.println("bình đẳng")
}
```

```
}
```

Mục 1.4: Phân tích quy trình

21. Đâu là tiêu đề phương thức sử dụng cú pháp đúng trong các tiêu đề sau? a. public static example() { b. public static void example() { c. public void static example() { d. public static example void[] { e. public void static example{} (
22. Đầu ra của chương trình sau là gì? (Bạn có thể vẽ sơ đồ cấu trúc trước.)

```
public class Tricky {
public static void main(String[] args) {
   message1();
   message2();
   System.out.println("Hoàn thành main.");
}
public static void message1() {
   System.out.println("Đây là message1.");
}
public static void message2() {
   System.out.println("Đây là message2.");
   message1();
   System.out.println("Hoàn thành message2.");
}
}
```

23. Đầu ra của chương trình sau là gì? (Bạn có thể vẽ sơ đồ cấu trúc trước.)

```
public class Strange {
public static void first() {
   System.out.println("Bên trong phương thức first");
}
public static void second() {
   System.out.println("Bên trong phương thức second");
   first();
}
public static void third() {
   System.out.println("Bên trong phương thức third");
```

```
    first();
    second();
}
public static void main(String[] args) {
    first();
    third();
    second();
    third();
}
}
```

24. Đầu ra của chương trình trên sẽ như thế nào nếu phương thức third chứa các câu lệnh sau?

```
public static void third() {
    first();
    second();
    System.out.println("Bên trong phương thức third");
}
```

25. Đầu ra của chương trình Strange sẽ như thế nào nếu phương thức main chứa các câu lệnh sau? (Sử dụng phiên bản gốc của third, không phải phiên bản đã chỉnh sửa từ bài tập gần nhất.)

```
public static void main(String[] args) {
    second();
    first();
    second();
    third();
}
```

26. Đầu ra của chương trình sau là gì? (Bạn có thể vẽ sơ đồ cấu trúc trước.)

```
public class Confusing {
public static void method2() {
    method1();
    System.out.println("Tôi là phương thức 2.");
}
public static void method3() {
```

```
    method2();
    System.out.println("Tôi là phương thức 3.");
    method1();
}
public static void method1() {
    System.out.println("Tôi là phương thức 1.");
}
public static void main(String[] args) {
    method1();
    method3();
    method2();
    method3();
}
}
```

27. Đầu ra của chương trình trên sẽ như thế nào nếu phương thức method3 chứa các câu lệnh sau?

```
public static void method3() {
    method1();
    method2();
    System.out.println("Tôi là phương thức 3.");
}
```

28. Đầu ra của chương trình Confusing sẽ như thế nào nếu phương thức main chứa các câu lệnh sau? (Sử dụng phiên bản gốc của method3, không phải phiên bản đã chỉnh sửa từ bài tập gần nhất.)

```
public static void main(String[] args) {
    method2();
    method1();
    method3();
    method2();
}
```

29. Chương trình sau chứa ít nhất 10 lỗi cú pháp. Những lỗi đó là gì?

```
public class LotsOfErrors {
public static main(String args) {
```

```
    System.println(Hello, world!);
    message()
}
public static void message {
    System.out println("Chương trình này chắc chắn không");
    System.out.println("có bất kỳ "errors" nào trong nó");
}
}
```

30. Xem xét chương trình sau, được lưu vào file có tên Example.java:

```
public class Example {
public static void displayRule() {
    System.out.println("Quy tắc đầu tiên");
    System.out.println("của Câu lạc bộ Java là,");
    System.out.println();
    System.out.println("bạn không nói về Câu lạc bộ Java.");
}
public static void main(String[] args) {
    System.out.println("Các quy tắc của Câu lạc bộ Java.");
    displayRule();
    displayRule();
}
}
```

Điều gì sẽ xảy ra nếu thực hiện từng thay đổi sau đối với chương trình Example? Ví dụ, có thể không có hiệu ứng gì, lỗi cú pháp, hoặc đầu ra chương trình khác? Xem xét từng thay đổi một cách độc lập.

a. Thay đổi dòng 1 thành: public class Demonstration b. Thay đổi dòng 9 thành: public static void MAIN(String[] args) { c. Thêm một dòng mới sau dòng 11 là: System.out.println(); d. Thay đổi dòng 2 thành: public static void printMessage() { e. Thay đổi dòng 2 thành: public static void showMessage() { và thay đổi các dòng 11 và 12 thành: showMessage(); f. Thay thế các dòng 3–4 bằng: System.out.println("Quy tắc đầu tiên của Câu lạc bộ Java là,");

31. Chương trình sau hợp lệ theo các quy tắc cú pháp của Java, nhưng khó đọc do cách bố trí và thiếu chú thích. Định dạng lại chương trình theo các quy tắc được đưa ra trong chương này và thêm một tiêu đề chú thích ở đầu chương trình.

```
public class GiveAdvice {
public static void main(String[] args) {
   System.out.println("Các chương trình có thể dễ hoặc");
   System.out.println("khó đọc, tùy thuộc");
   System.out.println("vào định dạng của chúng.");
   System.out.println("Mọi người, bao gồm cả bạn,");
   System.out.println("sẽ hạnh phúc hơn nếu bạn chọn");
   System.out.println("định dạng chương trình của bạn.");
}
}
```

32. Chương trình sau hợp lệ theo các quy tắc cú pháp của Java, nhưng khó đọc do cách bố trí và thiếu chú thích. Định dạng lại chương trình theo các quy tắc được đưa ra trong chương này và thêm một tiêu đề chú thích ở đầu chương trình.

```
public class Messy {
public static void main(String[] args) {
   message();
   System.out.println();
   message();
}
public static void message() {
   System.out.println("Tôi thật sự ước rằng");
   System.out.println("tôi đã định dạng mã nguồn của mình");
   System.out.println("đúng cách!");
}
}
```

Bài tập

1. Viết một chương trình Java hoàn chỉnh gọi là Stewie để in ra đầu ra sau:

```
public class Stewie {
   public static void main(String[] args) {
      System.out.println("//////////////////////");
      System.out.println("|| Victory is mine! ||");
      System.out.println("\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\");
```

```
    }
}
```

2. Viết một chương trình Java hoàn chỉnh gọi là Spikey để in ra đầu ra sau:

```
public class Spikey {
    public static void main(String[] args) {
        System.out.println(" \\/");
        System.out.println(" \\\\//");
        System.out.println("\\\\\\//");
        System.out.println("///\\\\\\");
        System.out.println(" //\\\\");
        System.out.println("  /\\");
    }
}
```

3. Viết một chương trình Java hoàn chỉnh gọi là WellFormed để in ra đầu ra sau:

```
public class WellFormed {
    public static void main(String[] args) {
        System.out.println("A well–formed Java program has");
        System.out.println();
        System.out.println("a main method with { and }");
        System.out.println("braces.");
        System.out.println();
        System.out.println("A System.out.println statement");
        System.out.println("has ( and ) and usually a");
        System.out.println("String that starts and ends");
        System.out.println("with a \" character.");
        System.out.println();
        System.out.println("(But we type \\\" instead!)");
    }
}
```

4. Viết một chương trình Java hoàn chỉnh gọi là Difference để in ra đầu ra sau:

```
public class Difference {
    public static void main(String[] args) {
        System.out.println("What is the difference between");
```

```
        System.out.println();
        System.out.println("a ' and a \"? Or between a \" and a \\\"?");
        System.out.println();
        System.out.println("One is what we see when we're typing our program.");
        System.out.println();
        System.out.println("The other is what appears on the \"console.\"");
    }
}
```

5. Viết một chương trình Java hoàn chỉnh gọi là MuchBetter để in ra đầu ra sau:

```
public class MuchBetter {
    public static void main(String[] args) {
        System.out.println("A \"quoted\" String is");
        System.out.println();
        System.out.println("'much' better if you learn");
        System.out.println("the rules of \"escape sequences.\"");
        System.out.println();
        System.out.println("Also, \"\" represents an empty String.");
        System.out.println();
        System.out.println("Don't forget: use \\\" instead of \" !");
        System.out.println();
        System.out.println("" is not the same as \"");
    }
}
```

6. Viết một chương trình Java hoàn chỉnh gọi là Meta có đầu ra là mã nguồn của một chương trình Java in ra “Hello, world!” như là đầu ra của nó:

```
public class Meta {
    public static void main(String[] args) {
        System.out.println("public class HelloWorld {");
        System.out.println("    public static void main(String[] args) {");
        System.out.println("        System.out.println(\"Hello, world!\");");
        System.out.println("    }");
        System.out.println("}");
    }
```

```
}
```

7. Viết một chương trình Java hoàn chỉnh gọi là Mantra để in ra đầu ra sau. Sử dụng ít nhất một phương thức tĩnh ngoài main.

```
public class Mantra {
    public static void main(String[] args) {
        printMantra();
    }

    public static void printMantra() {
        System.out.println("There's one thing every coder must understand:");
        System.out.println();
        System.out.println("The System.out.println command.");
        System.out.println();
        System.out.println("There's one thing every coder must understand:");
        System.out.println();
        System.out.println("The System.out.println command.");
    }
}
```

8. Viết một chương trình Java hoàn chỉnh gọi là Stewie2 để in ra đầu ra sau. Sử dụng ít nhất một phương thức tĩnh ngoài main.

```
public class Stewie2 {
    public static void main(String[] args) {
        printBanner();
        printBanner();
        printBanner();
        printBanner();
        printBanner();
    }

    public static void printBanner() {
        System.out.println("//////////////////////");
        System.out.println("|| Victory is mine! ||");
        System.out.println("\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\");
```

```
    }
}
```

9. Viết một chương trình gọi là Egg để hiển thị đầu ra sau:

```
public class Egg {
    public static void main(String[] args) {
        System.out.println("_______");
        System.out.println();
        System.out.println("/       \\");
        System.out.println();
        System.out.println("/         \\");
        System.out.println();
        System.out.println("-\"-'-\"-'-\"-");
        System.out.println();
        System.out.println("\\         /");
        System.out.println();
        System.out.println(" \\_______/");
    }
}
```

10. Chỉnh sửa chương trình từ bài tập trước để trở thành chương trình mới Egg2 hiển thị đầu ra sau. Sử dụng các phương thức tĩnh khi cần thiết.

```
public class Egg2 {
    public static void main(String[] args) {
        printTop();
        printBottom();
        printTop();
        printBottom();
        printTop();
        printBottom();
    }

    public static void printTop() {
        System.out.println("_______");
        System.out.println();
```

```
        System.out.println("/       \\");
        System.out.println();
        System.out.println("/         \\");
    }

    public static void printBottom() {
        System.out.println();
        System.out.println("-\"-'-\"-'-\"-");
        System.out.println();
        System.out.println("\\         /");
        System.out.println();
        System.out.println(" \\_______/");
    }
}
```

11. Viết một chương trình Java gọi là TwoRockets để tạo ra đầu ra sau. Sử dụng các phương thức tĩnh để thể hiện cấu trúc và loại bỏ sự lặp lại trong giải pháp của bạn. Lưu ý rằng có hai tên lửa cạnh nhau. Những sự lặp lại nào bạn có thể loại bỏ bằng cách sử dụng các phương thức tĩnh? Những sự lặp lại nào không thể loại bỏ?

```
public class TwoRockets {
    public static void main(String[] args) {
        printRocket();
        printRocket();
    }

    public static void printRocket() {
        System.out.println("   /\\       /\\");
        System.out.println("  /  \\     /  \\");
        System.out.println(" /    \\   /    \\");
        System.out.println("+------+ +------+");
        System.out.println("|      | |      |");
        System.out.println("|      | |      |");
        System.out.println("+------+ +------+");
        System.out.println("|United| |United|");
```

```
        System.out.println("|States| |States|");
        System.out.println("+------+ +------+");
        System.out.println("|      | |      |");
        System.out.println("|      | |      |");
        System.out.println("+------+ +------+");
        System.out.println("   /\\       /\\");
        System.out.println("  /  \\     /  \\");
        System.out.println(" /    \\   /    \\");
    }
}
```

12. Viết một chương trình gọi là FightSong để tạo ra đầu ra sau. Sử dụng ít nhất hai phương thức tĩnh để thể hiện cấu trúc và loại bỏ sự lặp lại trong giải pháp của bạn.

```
public class FightSong {
    public static void main(String[] args) {
        printVerse();
        printChorus();
        printVerse();
        printChorus();
        printVerse();
        printChorus();
    }

    public static void printVerse() {
        System.out.println("Go, team, go!");
        System.out.println();
        System.out.println("You can do it.");
        System.out.println();
    }

    public static void printChorus() {
        System.out.println("You're the best,");
        System.out.println();
```

```
        System.out.println("In the West.");
        System.out.println();
    }
}
```

13. Viết một chương trình Java gọi là StarFigures để tạo ra đầu ra sau. Sử dụng các phương thức tĩnh để thể hiện cấu trúc và loại bỏ sự lặp lại trong giải pháp của bạn.

```
public class StarFigures {
    public static void main(String[] args) {
        printStars();
        printStars();
        printStar();
        printStar();
        printStars();
        printStars();
        printStar();
        printStar();
    }

    public static void printStars() {
        System.out.println("*****");
        System.out.println("*****");
    }

    public static void printStar() {
        System.out.println(" * * ");
        System.out.println("  *  ");
    }
}
```

14. Viết một chương trình Java gọi là Lanterns để tạo ra đầu ra sau. Sử dụng các phương thức tĩnh để thể hiện cấu trúc và loại bỏ sự lặp lại trong giải pháp của bạn.

```
public class Lanterns {
    public static void main(String[] args) {
        printTop();
```

```
        printMiddle();
        printBottom();
    }

    public static void printTop() {
        System.out.println("    *****");
        System.out.println("  *********");
        System.out.println("*************");
    }

    public static void printMiddle() {
        System.out.println();
        System.out.println("    *****");
        System.out.println("  *********");
        System.out.println("*************");
        System.out.println("* | | | | *");
        System.out.println("*************");
    }

    public static void printBottom() {
        System.out.println();
        System.out.println("    *****");
        System.out.println("  *********");
        System.out.println("*************");
        System.out.println("    *****");
        System.out.println("    *****");
    }
}
```

15. Viết một chương trình Java gọi là EggStop để tạo ra đầu ra sau. Sử dụng các phương thức tĩnh để thể hiện cấu trúc và loại bỏ sự lặp lại trong giải pháp của bạn.

```
public class EggStop {
    public static void main(String[] args) {
        printEgg();
```

```
        printStop();
        printEgg();
    }

    public static void printEgg() {
        System.out.println("  ________");
        System.out.println(" /        \\");
        System.out.println("/          \\");
        System.out.println("\\          /");
        System.out.println(" \\________/");
    }

    public static void printStop() {
        System.out.println("+-------+");
        System.out.println("  ________");
        System.out.println(" /        \\");
        System.out.println("/          \\");
        System.out.println("|   STOP   |");
        System.out.println("\\          /");
        System.out.println(" \\________/");
    }
}
```

16. Viết một chương trình gọi là Shining để in dòng chữ sau 1000 lần:

```
public class Shining {
    public static void main(String[] args) {
        for (int i = 0; i < 1000; i++) {
            printLine();
        }
    }

    public static void printLine() {
        System.out.println("All work and no play makes Jack a dull boy.");
    }
```

```
}
```

17. Viết một chương trình gọi là FarewellGoodBye để in lời bài hát sau. Sử dụng các phương thức tĩnh để thể hiện cấu trúc và loại bỏ sự lặp lại trong giải pháp của bạn.

```
public class FarewellGoodBye {
    public static void main(String[] args) {
        printFarewell();
        printGoodbye();
        printFarewell();
        printGoodbye();
        printFarewell();
    }

    public static void printFarewell() {
        System.out.println("Farewell,");
        System.out.println();
        System.out.println("goodbye,");
    }

    public static void printGoodbye() {
        System.out.println("au revoir,");
        System.out.println("good night!");
        System.out.println();
        System.out.println("It's time, to go,");
        System.out.println("and I'll be out of sight!");
        System.out.println();
        System.out.println("Farewell,");
        System.out.println();
        System.out.println("goodbye,");
    }
}
```

Dự án lập trình

1. Viết một chương trình để viết từ "MISSISSIPPI" bằng chữ cái khối như sau (mỗi chữ cái trên một dòng):

```
public class Mississipi {
    public static void main(String[] args) {
        printM();
        printI();
        printS();
        printP();
    }

    public static void printM() {
        System.out.println("M     M    IIIII     SSSSS     PPPPPP");
        System.out.println("MM   MM      I      S     S    P     P");
        System.out.println("M M M M      I      S          P     P");
        System.out.println("M  M  M      I       SSSSS     PPPPPP");
        System.out.println("M     M      I            S    P");
        System.out.println("M     M      I      S     S    P");
        System.out.println("M     M    IIIII     SSSSS     P");
    }

    public static void printI() {
        System.out.println("I");
    }

    public static void printS() {
        System.out.println("SSSSS");
    }

    public static void printP() {
        System.out.println("PPPPPP");
    }
}
```

2. Đôi khi chúng ta viết các bức thư tương tự cho những người khác nhau. Ví dụ, bạn có thể viết cho cha mẹ của bạn để kể về các lớp học và bạn bè của bạn và yêu cầu tiền; bạn có thể viết cho một người bạn về đời sống tình cảm, các lớp học, và sở thích của bạn; và bạn có thể viết cho anh trai của bạn về sở thích và bạn bè của bạn và yêu cầu tiền. Viết một chương trình in các bức thư tương tự như vậy cho ba người mà bạn chọn. Mỗi bức thư nên có ít nhất một đoạn chung với từng bức thư còn lại. Chương trình chính của bạn nên có ba lệnh gọi phương thức, một cho mỗi người mà bạn đang viết. Cố gắng tách các nhiệm vụ lặp lại thành các phương thức.

3. Viết một chương trình tạo ra đầu ra là lời bài hát “There Was an Old Lady Who Swallowed a Fly” của Simms Taback. Sử dụng các phương thức cho mỗi đoạn và điệp khúc. Các phương thức của bạn nên nắm bắt cấu trúc của các đoạn bài hát cũng như tránh sự lặp lại giữa các dòng và nhóm dòng tương tự.

4. Viết một chương trình tạo ra đầu ra là lời bài hát “The Twelve Days of Christmas.” (Các phương thức tĩnh đơn giản hóa nhiệm vụ này.) Đây là hai đoạn đầu tiên và đoạn cuối của bài hát:

```
public class TwelveDaysOfChristmas {
   public static void main(String[] args) {
      printVerse1();
      printVerse2();
      printVerse12();
   }

   public static void printVerse1() {
      System.out.println("On the first day of Christmas,");
      System.out.println("my true love sent to me");
      System.out.println("a partridge in a pear tree.");
   }

   public static void printVerse2() {
      System.out.println("On the second day of Christmas,");
      System.out.println("my true love sent to me");
      System.out.println("two turtle doves, and");
      System.out.println("a partridge in a pear tree.");
```

```
    }

    public static void printVerse12() {
        System.out.println("On the twelfth day of Christmas,");
        System.out.println("my true love sent to me");
        System.out.println("Twelve drummers drumming,");
        System.out.println("eleven pipers piping,");
        System.out.println("ten lords a-leaping,");
        System.out.println("nine ladies dancing,");
        System.out.println("eight maids a-milking,");
        System.out.println("seven swans a-swimming,");
        System.out.println("six geese a-laying,");
        System.out.println("five golden rings,");
        System.out.println("four calling birds,");
        System.out.println("three French hens,");
        System.out.println("two turtle doves, and");
        System.out.println("a partridge in a pear tree.");
    }
}
```

5. Viết một chương trình tạo ra đầu ra là lời bài hát “The House That Jack Built.” Sử dụng các phương thức cho mỗi đoạn và cho văn bản lặp lại. Dưới đây là lời bài hát để sử dụng:

```
public class HouseThatJackBuilt {
    public static void main(String[] args) {
        printVerse1();
        printVerse2();
        printVerse3();
        printVerse4();
        printVerse5();
        printVerse6();
    }

    public static void printVerse1() {
        System.out.println("This is the house that Jack built.");
```

```
}

public static void printVerse2() {
    System.out.println("This is the malt");
    System.out.println("That lay in the house that Jack built.");
}

public static void printVerse3() {
    System.out.println("This is the rat,");
    System.out.println("That ate the malt");
    System.out.println("That lay in the house that Jack built.");
}

public static void printVerse4() {
    System.out.println("This is the cat,");
    System.out.println("That killed the rat,");
    System.out.println("That ate the malt");
    System.out.println("That lay in the house that Jack built.");
}

public static void printVerse5() {
    System.out.println("This is the dog,");
    System.out.println("That worried the cat,");
    System.out.println("That killed the rat,");
    System.out.println("That ate the malt");
    System.out.println("That lay in the house that Jack built.");
}

public static void printVerse6() {
    System.out.println("This is the cow with the crumpled horn,");
    System.out.println("That tossed the dog,");
    System.out.println("That worried the cat,");
    System.out.println("That killed the rat,");
```

```
        System.out.println("That ate the malt");
        System.out.println("That lay in the house that Jack built.");
    }
}
```

6. Viết một chương trình tạo ra đầu ra là lời bài hát “Bought Me a Cat.” Sử dụng các phương thức cho mỗi đoạn và cho văn bản lặp lại.

7. Viết một chương trình tạo ra đầu ra là lời của bài hát hài hước sau. Sử dụng các phương thức cho mỗi đoạn và cho văn bản lặp lại. Dưới đây là lời bài hát đầy đủ:

```
public class BoughtMeACat {
    public static void main(String[] args) {
        printVerse1();
        printVerse2();
        printVerse3();
        printVerse4();
    }

    public static void printVerse1() {
        System.out.println("I once wrote a program that wouldn't compile");
        System.out.println("I don't know why it wouldn't compile,");
        System.out.println("My TA just smiled.");
    }

    public static void printVerse2() {
        System.out.println("My program did nothing");
        System.out.println("So I started typing.");
        System.out.println("I added System.out.println(\"I <3 coding\"),");
        System.out.println("I don't know why it wouldn't compile,");
        System.out.println("My TA just smiled.");
    }

    public static void printVerse3() {
        System.out.println("\"Parse error,\" cried the compiler");
        System.out.println("Luckily I'm such a code baller.");
```

```
        System.out.println("I added a backslash to escape the quotes,");
        System.out.println("I added System.out.println(\"I <3 coding\"),");
        System.out.println("I don't know why it wouldn't compile,");
        System.out.println("My TA just smiled.");
    }

    public static void printVerse4() {
        System.out.println("Now the compiler wanted an identifier");
        System.out.println("And I thought the situation was getting dire.");
        System.out.println("I added a main method with its String[] args,");
        System.out.println("I added a backslash to escape the quotes,");
        System.out.println("I added System.out.println(\"I <3 coding\"),");
        System.out.println("I don't know why it wouldn't compile,");
        System.out.println("My TA just smiled.");
        System.out.println("Java complained it expected an enum");
        System.out.println("Boy, these computers really are dumb!");
        System.out.println("I added a public class and called it Scum,");
        System.out.println("I added a main method with its String[] args,");
        System.out.println("I added a backslash to escape the quotes,");
        System.out.println("I added System.out.println(\"I <3 coding\"),");
        System.out.println("I don't know why it wouldn't compile,");
        System.out.println("My TA just smiled.");
    }
}
```

Chương 2: Dữ liệu Nguyên thủy và Vòng lặp Xác định

2.1 Khái niệm Cơ bản về Dữ liệu • Các Kiểu Dữ liệu Nguyên thủy • Biểu thức • JShell • Hằng số • Toán tử Số học • Thứ tự Ưu tiên • Kết hợp Kiểu Dữ liệu và Ép kiểu
2.2 Biến • Các Biến thể của Gán/ Khai báo • Nối Chuỗi • Toán tử Tăng/Giảm • Biến và Kết hợp Kiểu Dữ liệu
2.3 Vòng lặp for • Theo dõi Vòng lặp for • Các Mẫu Vòng lặp for • Vòng lặp for Lồng nhau
2.4 Quản lý Độ phức tạp • Phạm vi • Pseudocode (Mã giả) • Kỹ thuật Bảng • Hằng số của Lớp
2.5 Nghiên cứu Trường hợp: Hình đồng hồ cát • Phân tách Vấn đề và Mã giả • Phiên bản Cấu trúc Ban đầu • Thêm Hằng số của Lớp • Các Biến thể Khác

Giới thiệu

Bây giờ bạn đã biết về cấu trúc cơ bản của các chương trình Java, bạn đã sẵn sàng để học cách giải quyết các vấn đề phức tạp hơn. Trong thời gian này, chúng ta vẫn sẽ tập trung vào các chương trình tạo ra kết quả đầu ra, nhưng chúng ta sẽ bắt đầu khám phá một số khía cạnh của lập trình đòi hỏi kỹ năng giải quyết vấn đề. Nửa đầu của chương này sẽ bổ sung hai lĩnh vực quan trọng. Đầu tiên, nó sẽ xem xét các biểu thức, được sử dụng để thực hiện các phép tính đơn giản trong Java, đặc biệt là những phép tính liên quan đến dữ liệu số. Thứ hai, nó sẽ thảo luận về các yếu tố chương trình được gọi là biến, có thể thay đổi giá trị khi chương trình chạy. Nửa sau của chương sẽ giới thiệu cấu trúc điều khiển đầu tiên của bạn: vòng lặp for. Bạn sử dụng cấu trúc này để lặp lại các hành động trong một chương trình. Điều này hữu ích bất cứ khi nào bạn tìm thấy một mẫu trong một nhiệm vụ như việc tạo ra một hình phức tạp, vì bạn có thể sử dụng vòng lặp for để lặp lại hành động nhằm tạo ra mẫu đó. Thách thức là tìm ra từng mẫu và xác định những hành động lặp lại nào sẽ tái tạo nó. Vòng lặp for là một cấu trúc điều khiển linh hoạt có thể được sử dụng cho nhiều nhiệm vụ. Trong chương này, chúng ta sẽ sử dụng nó cho các vòng lặp xác định, nơi bạn biết chính xác số lần bạn muốn thực hiện một nhiệm vụ cụ thể. Trong Chương 5, chúng ta sẽ thảo luận cách viết các vòng lặp vô định, nơi bạn không biết trước bao nhiêu lần cần thực hiện một nhiệm vụ.

2.1 Khái niệm Cơ bản về Dữ liệu

Chương trình thao tác với thông tin, và thông tin có nhiều hình thức khác nhau. Java là một ngôn ngữ an toàn về kiểu dữ liệu, điều này có nghĩa là bạn phải xác định rõ loại thông tin mà bạn định thao tác và nó đảm bảo rằng bạn thao tác dữ liệu một cách hợp lý. Mọi thứ mà bạn

thao tác trong một chương trình Java đều thuộc một kiểu nhất định, và bạn sẽ thường xuyên phải chỉ định cho Java biết kiểu dữ liệu mà bạn định sử dụng.

Kiểu Dữ liệu

Tên gọi cho một danh mục các giá trị dữ liệu có liên quan với nhau, chẳng hạn như kiểu int trong Java, được sử dụng để biểu diễn các giá trị số nguyên. Một quyết định đã được đưa ra từ sớm trong quá trình thiết kế Java là hỗ trợ hai loại dữ liệu khác nhau: dữ liệu nguyên thủy và đối tượng. Các nhà thiết kế đã đưa ra quyết định này hoàn toàn dựa trên hiệu suất, nhằm giúp các chương trình Java chạy nhanh hơn. Thật không may, điều này có nghĩa là bạn phải học hai bộ quy tắc về cách dữ liệu hoạt động, nhưng đây là một trong những lúc bạn phải trả giá nếu muốn sử dụng một ngôn ngữ lập trình mạnh mẽ. Để dễ dàng hơn một chút, chúng ta sẽ học về các kiểu dữ liệu nguyên thủy trước trong chương này; trong chương tiếp theo, chúng ta sẽ chuyển sự chú ý đến đối tượng.

Kiểu Nguyên thủy

Có tám kiểu dữ liệu nguyên thủy trong Java: boolean, byte, char, double, float, int, long, và short. Trong đó, bốn kiểu được coi là cơ bản: boolean, char, double, và int. Bốn kiểu còn lại là các biến thể dành cho các chương trình có yêu cầu đặc biệt. Bốn kiểu cơ bản mà chúng ta sẽ tìm hiểu được liệt kê trong Bảng 2.1.

Bảng 2.1: Các Kiểu Dữ liệu Nguyên thủy Thường Dùng trong Java

Kiểu	Mô tả	Ví dụ
int	số nguyên (số nguyên toàn phần)	42, −3, 18, 20493, 0
double	số thực	7.35, 14.9, −19.83423
char	ký tự đơn	'a', 'X', '!'
boolean	giá trị logic	true, false

Tên các kiểu (int, double, char, và boolean) là các từ khóa của Java mà bạn sẽ sử dụng trong các chương trình của mình để cho trình biên dịch biết rằng bạn định sử dụng loại dữ liệu đó. Có thể bạn sẽ thấy lạ khi phải sử dụng một kiểu cho số nguyên và một kiểu khác cho số thực. Chẳng phải mọi số nguyên đều là số thực hay sao? Câu trả lời là đúng, nhưng đây là hai loại số cơ bản khác nhau. Sự khác biệt này lớn đến mức chúng ta thậm chí còn phân biệt trong ngôn ngữ thường ngày. Chúng ta không hỏi, “Bạn có bao nhiêu chị em?” hay “Bạn nặng bao nhiêu?” mà là “Bạn có bao nhiêu chị em gái?” hoặc “Bạn nặng bao nhiêu?”. Chúng ta nhận ra rằng số lượng chị em gái là số nguyên rời rạc (0 chị em, 1 chị em, 2 chị em, 3 chị em, v.v.), và chúng ta sử dụng từ “bao nhiêu” cho các số nguyên (“Bạn có bao nhiêu chị em?”). Tương tự, chúng ta

nhận ra rằng trọng lượng có thể thay đổi bởi những lượng rất nhỏ (175 pounds so với 175.5 pounds so với 175.25 pounds, v.v.), và chúng ta sử dụng từ “bao nhiêu” cho các số thực (“Bạn nặng bao nhiêu?”).

Trong lập trình, sự phân biệt này còn quan trọng hơn nữa, vì số nguyên và số thực được biểu diễn theo những cách khác nhau trong bộ nhớ máy tính: Số nguyên được lưu trữ chính xác, trong khi số thực được lưu trữ dưới dạng gần đúng với một số lượng chữ số giới hạn. Bạn sẽ thấy rằng việc lưu trữ giá trị dưới dạng gần đúng có thể dẫn đến các lỗi làm tròn khi bạn sử dụng các giá trị thực.

Tên gọi double cho các giá trị số thực không phải là trực quan. Đó là một ngẫu nhiên của lịch sử, giống như cách chúng ta vẫn nói về việc “quay số” trên điện thoại dù điện thoại hiện đại không có quay số. Ngôn ngữ lập trình C đã giới thiệu một kiểu gọi là float (viết tắt của “floating-point number” - số dấu chấm động) để lưu trữ các số thực. Nhưng float có độ chính xác hạn chế, nên một kiểu khác được giới thiệu, gọi là double (viết tắt của “double precision,” có nghĩa là nó có độ chính xác gấp đôi so với float đơn giản). Khi bộ nhớ trở nên rẻ hơn, người ta bắt đầu sử dụng double làm mặc định cho các giá trị dấu chấm động. Nhìn lại, có thể tốt hơn nếu sử dụng từ float cho những gì hiện gọi là double và một từ như “half” cho các giá trị có độ chính xác thấp hơn, nhưng rất khó để thay đổi những thói quen đã ăn sâu vào tiềm thức. Vì vậy, các ngôn ngữ lập trình sẽ tiếp tục sử dụng từ double cho các số dấu chấm động, và mọi người sẽ vẫn nói về việc “quay số” trên điện thoại ngay cả khi họ chưa bao giờ chạm vào một chiếc điện thoại có quay số.

Biểu thức

Khi bạn viết chương trình, bạn sẽ thường xuyên cần bao gồm các giá trị và phép tính. Thuật ngữ kỹ thuật cho những yếu tố này là biểu thức.

Biểu thức

Một giá trị đơn giản hoặc một tập hợp các phép toán tạo ra một giá trị.

Biểu thức đơn giản nhất là một giá trị cụ thể, như 42 hoặc 28.9. Chúng ta gọi chúng là “giá trị trực tiếp,” hoặc literals. Các biểu thức phức tạp hơn liên quan đến việc kết hợp các giá trị đơn giản. Giả sử, chẳng hạn, bạn muốn biết bạn có bao nhiêu chai nước. Nếu bạn có hai thùng 6 chai, bốn thùng 4 chai, và hai chai lẻ, bạn có thể tính tổng số chai với biểu thức sau:

(2 * 6) + (4 * 4) + 2

Lưu ý rằng chúng ta sử dụng dấu hoa thị (*) để biểu thị phép nhân và sử dụng dấu ngoặc đơn để nhóm các phần của biểu thức. Máy tính xác định giá trị của một biểu thức bằng cách đánh giá nó.

Đánh giá

Quá trình nhận giá trị của một biểu thức.

Giá trị thu được khi một biểu thức được đánh giá được gọi là kết quả.

Các biểu thức phức tạp được hình thành bằng cách sử dụng các toán tử.

Toán tử

Một ký hiệu đặc biệt (như + hoặc *) được sử dụng để chỉ ra một phép toán cần thực hiện trên một hoặc nhiều giá trị.

Các giá trị được sử dụng trong biểu thức được gọi là toán hạng. Ví dụ, hãy xem xét các biểu thức đơn giản sau:

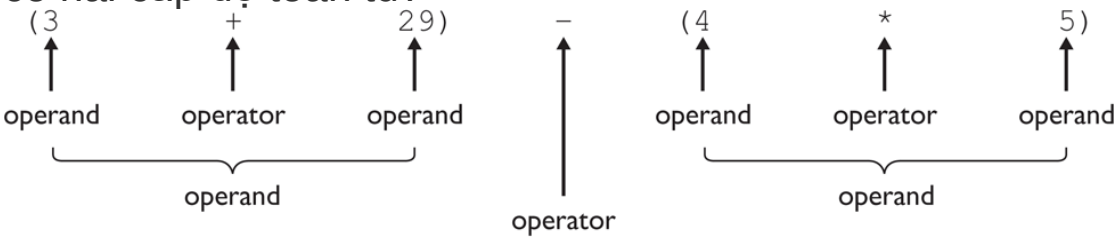
3 + 29

4 * 5

Các toán tử ở đây là + và *, và các toán hạng là những con số đơn giản.

2.1-2 Văn bản Thay thế Đầy đủ

Khi bạn tạo các biểu thức phức tạp, các biểu thức đơn giản này có thể trở thành toán hạng cho các toán tử khác. Ví dụ, biểu thức:
(3 + 29) − (4 * 5)
có hai cấp độ toán tử.



2.1-3 Văn bản Thay thế Đầy đủ

Toán tử cộng có các toán hạng đơn giản là 3 và 29, và toán tử nhân có các toán hạng đơn giản là 4 và 5, nhưng toán tử trừ có các toán hạng là các biểu thức được đặt trong dấu ngoặc với các toán tử riêng của chúng. Do đó, các biểu thức phức tạp có thể

được xây dựng từ các biểu thức nhỏ hơn. Ở mức thấp nhất, bạn có các con số đơn giản. Những con số này được sử dụng như các toán hạng để tạo ra các biểu thức phức tạp hơn, sau đó có thể được sử dụng như các toán hạng trong các biểu thức phức tạp hơn nữa.
Có nhiều điều bạn có thể làm với các biểu thức. Một trong những điều đơn giản nhất bạn có thể làm là in giá trị của một biểu thức bằng cách sử dụng câu lệnh println. Ví dụ, nếu bạn nói:

```
System.out.println(42);
System.out.println(2 + 2);
```

Bạn sẽ nhận được hai dòng kết quả sau:

```
42
4
```

Lưu ý rằng đối với câu lệnh println thứ hai, máy tính tính toán biểu thức (cộng 2 và 2) và in kết quả (trong trường hợp này là 4).
Trong phần tiếp theo, chúng ta sẽ thảo luận về JShell, một chương trình cho phép bạn gõ các biểu thức Java và xem kết quả của chúng ngay lập tức.
Bạn sẽ thấy nhiều toán tử khác nhau khi tiếp tục qua cuốn sách này, tất cả đều có thể được sử dụng để tạo thành các biểu thức. Các biểu thức có thể phức tạp tùy ý, với nhiều toán tử như bạn muốn. Vì lý do đó, khi chúng tôi nói, “Một biểu thức có thể được sử dụng ở đây,” chúng tôi có nghĩa là bạn có thể sử dụng các biểu thức tùy ý bao gồm cả các biểu thức phức tạp cũng như các giá trị đơn giản.

JShell

Bắt đầu từ Java 9, Bộ công cụ phát triển Java (JDK) hiện bao gồm một công cụ gọi là JShell mà bạn có thể sử dụng để khám phá cú pháp và các tính năng của Java. JShell là một chương trình với một dấu nhắc, nơi bạn có thể nhập các biểu thức Java tùy ý. Khi bạn nhấn Enter, biểu thức bạn vừa nhập sẽ ngay lập tức được tính toán và kết quả của nó sẽ được hiển thị. Loại công

cụ này thường được gọi là Vòng lặp Đọc-Tính-Toán (Read-Evaluate-Print Loop) hoặc viết tắt là REPL.

Vòng lặp Đọc-Tính-Toán (REPL)

Là một chương trình nhắc người dùng nhập một biểu thức mã cụ thể, sau đó thực thi biểu thức và hiển thị kết quả của nó ngay lập tức.

REPL rất hữu ích khi bạn đang học lập trình hoặc học một ngôn ngữ mới vì bạn có thể thử các lệnh nhanh chóng và nhận được phản hồi tức thì. Trong đề xuất chính thức về việc thêm công cụ này vào Java, các nhà phát triển của ngôn ngữ viết, “Phản hồi ngay lập tức rất quan trọng khi học một ngôn ngữ lập trình và các API của nó. Lý do hàng đầu mà các trường học viện dẫn để chuyển khỏi Java như một ngôn ngữ giảng dạy là các ngôn ngữ khác có REPL và có mức độ khó khăn thấp hơn để thực hiện chương trình ‘Hello, world!’ ban đầu.”

JShell được tích hợp với một số chương trình trình chỉnh sửa Java. (Trong một số trình chỉnh sửa, shell được đặt tên khác như “Interactions Pane.”) Tham khảo tài liệu của trình chỉnh sửa của bạn hoặc tài liệu của JShell để tìm cách tốt nhất để khởi chạy JShell trên máy của bạn.

Khi bạn khởi chạy JShell lần đầu, bạn sẽ thấy một dấu nhắc trông giống như sau:

```
|  Welcome to JShell -- Version 10.0.2
|  For an introduction type: /help intro
jshell>
```

Bạn có thể nhập một biểu thức tại dấu nhắc, và JShell sẽ tính toán và hiển thị kết quả của nó (đầu vào của người dùng được in đậm):

```
jshell> 1 + 2
$1 ==> 3
```

Trong tương tác trên, JShell đã tính toán kết quả của 1 + 2 và hiển thị nó là 3. Kết quả được gọi là $1 vì JShell gán một ID số cho mỗi kết quả mà nó tính toán trong trường hợp bạn muốn

tham khảo lại kết quả đó sau này. Bạn thường có thể bỏ qua các ID này và tập trung vào các giá trị đã tính toán.
JShell có thể tính toán các kết quả của các biểu thức phức tạp. Ví dụ, đây là kết quả của một biểu thức từ phần trước:

```
jshell> (3 + 29) - (4 * 5)
$2 ==> 12
```

JShell có rất nhiều tính năng và tùy chọn. Ví dụ, bạn có thể nhấn các mũi tên Lên và Xuống trên bàn phím của mình để gọi lại các biểu thức trước đó bạn đã nhập. Khi bạn không còn muốn sử dụng JShell nữa, bạn có thể gõ /exit để thoát khỏi công cụ:

```
jshell> /exit
```

Khi bạn đang học các khái niệm lập trình mới và các tính năng của Java, hãy cân nhắc khởi chạy JShell để thử nghiệm chúng. Đây là một công cụ hữu ích để giúp bạn xác minh và nâng cao hiểu biết về các khái niệm lập trình. Chúng tôi cũng sẽ chỉ ra các ví dụ về tương tác với JShell trong suốt phần còn lại của cuốn sách này khi giới thiệu các khái niệm mới.

Literals

Các biểu thức đơn giản nhất đề cập đến các giá trị trực tiếp bằng cách sử dụng cái gọi là literals. Một literal số nguyên (được coi là thuộc loại int) là một chuỗi chữ số có hoặc không có dấu hiệu dẫn đầu:

```
3    482    −29434    0    92348    +9812
```

Một literal số thực (được coi là thuộc loại double) bao gồm một dấu thập phân:

```
298.4    0.284    207.    .2843    −17.452    −.98
```

Lưu ý rằng 207. được coi là double mặc dù nó trùng với một số nguyên, bởi vì có dấu thập phân. Literals thuộc loại double cũng có thể được biểu diễn dưới dạng ký hiệu khoa học (một số theo sau là e và một số nguyên):

```
2.3e4    1e-5    3.84e92    2.458e12
```

Số đầu tiên trong những số này biểu thị 2.3 nhân với 10 mũ 4, tức là bằng 23,000. Mặc dù giá trị này trùng với một số nguyên, nó được coi là thuộc loại double vì nó được biểu diễn dưới dạng ký hiệu khoa học. Số thứ hai biểu thị 1 nhân với 10 mũ -5, bằng 0.00001. Số thứ ba biểu thị 3.84 nhân với 10 mũ 92. Số thứ tư biểu thị 2.458 nhân với 10 mũ 12.

Chúng ta đã thấy rằng thông tin văn bản có thể được lưu trữ trong các chuỗi literals lưu trữ một chuỗi ký tự. Trong các chương sau, chúng ta sẽ khám phá cách xử lý một chuỗi ký tự theo từng ký tự. Mỗi ký tự như vậy thuộc loại char. Một literal ký tự được đặt trong dấu nháy đơn và chỉ bao gồm một ký tự:

```
'a'    'm'    'X'    '!'    '3'    '\\'
```

Tất cả các ví dụ này đều thuộc loại char. Lưu ý rằng ví dụ cuối cùng sử dụng chuỗi thoát để biểu diễn ký tự dấu gạch chéo ngược. Bạn thậm chí có thể tham chiếu đến ký tự dấu nháy đơn bằng cách sử dụng chuỗi thoát:

```
'\''
```

Cuối cùng, loại nguyên thủy boolean lưu trữ thông tin logic. Chúng ta sẽ không khám phá việc sử dụng loại boolean cho đến khi chúng ta đến Chương 4 và thấy cách đưa các bài kiểm tra logic vào chương trình của chúng ta, nhưng để hoàn thiện, chúng tôi bao gồm các giá trị literal boolean ở đây. Logic chỉ liên quan đến hai khả năng: true và false. Đây là hai từ khóa của Java và cũng là hai giá trị literal của loại boolean:

```
true    false
```

Toán tử số học

Các toán tử số học cơ bản được hiển thị trong Bảng 2.2. Toán tử cộng và trừ, tất nhiên, sẽ quen thuộc với bạn, cũng như dấu hoa thị cho toán tử nhân và dấu gạch chéo cho toán tử chia. Tuy nhiên, như bạn sẽ thấy, Java có hai phép chia khác nhau. Phép chia dư hoặc phép mod có thể sẽ mới mẻ đối với bạn.

Bảng 2.2 Toán tử số học trong Java

Toán tử	Ý nghĩa	Ví dụ	Kết quả
+	Phép cộng	2 + 2	4
−	Phép trừ	53 − 18	35
*	Phép nhân	3 * 8	24
/	Phép chia	4.8 / 2.0	2.4
%	Phép chia dư	19 % 5	4

Phép chia gây ra một vấn đề khi các toán hạng là số nguyên. Khi bạn chia 119 cho 5, chẳng hạn, bạn không nhận được kết quả là số nguyên. Do đó, kết quả của phép chia số nguyên được biểu diễn dưới dạng hai số nguyên khác nhau, thương số và số dư:

```
119 / 5 = 23 (thương số) với 4 (số dư)
```

Về mặt các toán tử số học:

```
119 / 5 = 23
119 % 5 = 4
```

Các toán tử chia này sẽ quen thuộc nếu bạn nhớ lại cách thực hiện các phép chia dài:

```
31341079    102          59¯          34          25¯
```

Ở đây, chia 1079 cho 34 cho kết quả là 31 với số dư là 25. Sử dụng các toán tử số học, vấn đề sẽ được mô tả như sau:

```
1079 / 34 = 31
1079 % 34 = 25
```

Sẽ mất một thời gian để quen với phép chia số nguyên trong Java. Khi bạn sử dụng toán tử chia (/), điều quan trọng cần ghi nhớ là nó sẽ bỏ qua bất kỳ phần thập phân nào. Vì vậy, nếu bạn tưởng tượng tính toán một câu trả lời trên máy tính, hãy nghĩ đến việc bỏ qua bất kỳ phần nào sau dấu thập phân:

- 19 / 5 là 3.8 trên máy tính, vì vậy 19 / 5 = 3
- 207 / 10 là 20.7 trên máy tính, vì vậy 207 / 10 = 20
- 3 / 8 là 0.375 trên máy tính, vì vậy 3 / 8 = 0

Toán tử dư (%) thường được gọi là "toán tử mod" hoặc đơn giản là "mod". Toán tử mod cho bạn biết bao nhiêu số đã bị bỏ qua bởi

toán tử chia cắt bỏ. Ví dụ, với các ví dụ trước, bạn có thể tính toán kết quả của mod như được hiển thị trong Bảng 2.3.

Bảng 2.3 Ví dụ về Toán tử Mod

Vấn đề mod	Chia trước	Chia bằng số nào?	Còn lại bao nhiêu?	Kết quả
19 % 5	19 / 5 = 3	3 * 5 = 15	19 - 15 = 4	4
207 % 10	207 / 10 = 20	20 * 10 = 200	207 - 200 = 7	7
3 % 8	3 / 8 = 0	0 * 8 = 0	3 - 0 = 3	3

Trong mỗi trường hợp, bạn phải tìm hiểu bao nhiêu của số đã được tính toán bởi toán tử chia. Toán tử mod cho bạn phần dư (số còn lại). Khi bạn đặt điều này vào công thức, bạn có thể nghĩ rằng toán tử mod hoạt động như sau:

```
x % y = x − (x / y) * y
```

Có thể có kết quả là 0 đối với toán tử mod. Điều này xảy ra khi một số chia đều cho số khác. Ví dụ, mỗi biểu thức sau đây đều cho kết quả là 0 vì số thứ hai chia đều cho số thứ nhất:

```
28 % 7
95 % 5
44 % 2
```

Một số trường hợp đặc biệt đáng lưu ý vì chúng không phải lúc nào cũng hiển nhiên đối với các lập trình viên mới bắt đầu:

- Tử số nhỏ hơn mẫu số: Trong trường hợp này, phép chia tạo ra 0 và mod tạo ra số ban đầu. Ví dụ: 7 / 10 = 0 và 7 % 10 = 7.
- Tử số bằng 0: Trong trường hợp này, cả phép chia và mod đều trả về 0. Ví dụ: 0 / 10 và 0 % 10 đều cho kết quả là 0.
- Mẫu số bằng 0: Trong trường hợp này, cả phép chia và mod đều không được định nghĩa và tạo ra lỗi runtime. Ví dụ: một chương trình cố gắng thực hiện 7 / 0 hoặc 7 % 0 sẽ tạo ra lỗi ArithmeticException.

Toán tử mod có nhiều ứng dụng hữu ích trong các chương trình máy tính. Dưới đây là một vài ý tưởng:

- Kiểm tra xem một số là số chẵn hay lẻ (number % 2 = 0 đối với số chẵn, number % 2 = 1 đối với số lẻ).
- Tìm các chữ số riêng lẻ của một số (ví dụ: number % 10 là chữ số cuối cùng).
- Tìm bốn chữ số cuối của số an sinh xã hội (number % 10000).

Toán tử dư có thể được sử dụng với số thực cũng như với số nguyên, và nó hoạt động tương tự: Bạn cân nhắc bao nhiêu còn lại khi bạn lấy đi nhiều giá trị "nguyên" nhất có thể. Ví dụ, biểu thức 10.2 % 2.4 = 0.6 vì bạn có thể lấy đi bốn giá trị 2.4 từ 10.2, để lại 0.6 còn lại.

Đối với các giá trị số thực (giá trị kiểu double), toán tử chia thực hiện phép chia "bình thường". Vì vậy, mặc dù biểu thức 119 / 5 = 23, biểu thức 119.0 / 5.0 = 23.8.

Bạn có thể kiểm tra các toán tử chia bằng cách sử dụng JShell. Dưới đây là một ví dụ tương tác:

```
jshell> 19 / 5
$1 ==> 3
jshell> 1079 % 34
$2 ==> 25
jshell> 7 / 10
$3 ==> 0
jshell> 7 % 10
$4 ==> 7
jshell> 0 % 10
$5 ==> 0
jshell> 125.0 / 5.0 / 10.0
$6 ==> 2.5
```

Thứ tự ưu tiên

Các biểu thức Java giống như các cụm danh từ phức tạp trong tiếng Anh. Các cụm từ này có thể gây ra sự mơ hồ. Ví dụ, hãy xem xét cụm từ “người đàn ông trên đồi cạnh sông với kính viễn vọng.” Con sông ở gần ngọn đồi hay gần người đàn ông? Người đàn ông đang cầm kính viễn vọng, hay kính viễn vọng nằm trên đồi, hoặc ở dưới sông? Chúng ta không biết cách nhóm các phần khác nhau lại với nhau.

Bạn có thể gặp phải sự mơ hồ tương tự nếu dấu ngoặc không được sử dụng để nhóm các phần của một biểu thức Java. Ví dụ, biểu thức 2 + 3 * 4 có hai toán tử. Phép toán nào được thực hiện trước? Bạn có thể hiểu theo hai cách:

```
2 + 3 * 4
(2 + 3) * 4
```

Cách đầu tiên cho kết quả là 20, trong khi cách thứ hai cho kết quả là 14. Để giải quyết sự mơ hồ, Java có các quy tắc về thứ tự ưu tiên để xác định cách nhóm các phần khác nhau lại với nhau.

Thứ tự ưu tiên

Sức mạnh liên kết của một toán tử, quyết định cách nhóm các phần của một biểu thức.

Máy tính áp dụng các quy tắc về thứ tự ưu tiên khi cách nhóm các toán tử trong một biểu thức là mơ hồ. Một toán tử có thứ tự ưu tiên cao sẽ được đánh giá trước, sau đó đến các toán tử có thứ tự ưu tiên thấp hơn. Trong một mức độ ưu tiên nhất định, các toán tử được đánh giá theo một hướng, thường là từ trái sang phải.

Đối với các biểu thức số học, có hai mức độ ưu tiên. Các toán tử nhân (*, /, %) có mức độ ưu tiên cao hơn so với các toán tử cộng và trừ (+, −). Do đó, biểu thức 2 + 3 * 4 được hiểu là:

```
2 + (3 * 4)
```

Trong cùng một mức độ ưu tiên, các toán tử số học được đánh giá từ trái sang phải. Điều này thường không ảnh hưởng đến kết quả cuối cùng, nhưng thỉnh thoảng có. Hãy xem xét ví dụ sau:

```
40 − 25 − 9
```

Biểu thức này được đánh giá như sau:

```
(40 − 25) − 9 = 15 − 9 = 6
```

Bạn sẽ có một kết quả khác nếu phép trừ thứ hai được đánh giá trước.

Bạn luôn có thể ghi đè thứ tự ưu tiên bằng dấu ngoặc đơn. Ví dụ, nếu bạn thực sự muốn phép trừ thứ hai được đánh giá trước, bạn có thể buộc điều đó xảy ra bằng cách đưa vào dấu ngoặc đơn:

```
40 − (25 − 9) = 40 − 16 = 24
```

Một khái niệm khác trong số học là cộng và trừ đơn (unary), chúng chỉ lấy một toán hạng, khác với các toán tử nhị phân mà chúng ta đã thấy cho đến nay (ví dụ: *, /, và cả + và − nhị phân), tất cả đều lấy hai toán hạng. Ví dụ, chúng ta có thể tìm giá trị phủ định của 8 bằng cách hỏi −8. Các toán tử đơn này có mức độ ưu tiên cao hơn các toán tử nhân. Do đó, chúng ta có thể tạo ra các biểu thức như sau:

```
12 * −8
```

Biểu thức này đánh giá thành −96.

Chúng ta sẽ thấy nhiều loại toán tử trong các chương tiếp theo. Bảng 2.4 là bảng thứ tự ưu tiên bao gồm các toán tử số học. Khi chúng ta giới thiệu thêm các toán tử khác, chúng ta sẽ cập nhật bảng này để bao gồm chúng. Bảng này được sắp xếp từ thứ tự ưu tiên cao nhất đến thấp nhất và cho thấy rằng Java sẽ nhóm các phần của một biểu thức trước tiên bằng cách sử dụng các toán tử đơn, sau đó sử dụng các toán tử nhân, và cuối cùng sử dụng các toán tử cộng.

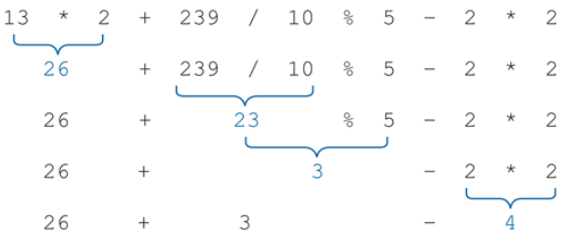
Bảng 2.4 Thứ tự ưu tiên của các toán tử trong Java

Mô tả	Toán tử
Toán tử đơn	+, −
Toán tử nhân	*, /, %
Toán tử cộng và trừ	+, −

Trước khi chúng ta rời khỏi chủ đề này, hãy xem một biểu thức phức tạp và xem cách nó được đánh giá từng bước một. Xem xét biểu thức sau:

```
13 * 2 + 239 / 10 % 5 − 2 * 2
```

Biểu thức này có tổng cộng sáu toán tử: hai phép nhân, một phép chia, một phép mod, một phép trừ và một phép cộng. Các phép toán nhân, chia và mod sẽ được thực hiện trước, vì chúng có thứ tự ưu tiên cao hơn, và chúng sẽ được thực hiện từ trái sang phải vì chúng đều ở cùng một mức độ ưu tiên.



Bây giờ chúng ta sẽ đánh giá các toán tử cộng từ trái sang phải:

```
26 + 3 - 4
```

Kết hợp các loại và Ép kiểu

Bạn sẽ thường gặp tình huống kết hợp các giá trị của các loại khác nhau và muốn chuyển đổi từ loại này sang loại khác. Java có các quy tắc đơn giản để tránh nhầm lẫn và cung cấp một cơ chế để yêu cầu chuyển đổi giá trị từ loại này sang loại khác.

Hai loại thường được kết hợp là int và double. Ví dụ, bạn có thể yêu cầu Java tính toán 2 * 3.6. Biểu thức này bao gồm giá trị nguyên int là 2 và giá trị thực double là 3.6. Trong trường hợp này, Java sẽ chuyển đổi int thành double và thực hiện phép tính hoàn toàn với các giá trị double; đây luôn là quy tắc khi Java gặp một int tại nơi cần một double.

Điều này đặc biệt quan trọng khi bạn hình thành các biểu thức liên quan đến phép chia. Nếu hai toán hạng đều là int, Java sẽ sử dụng phép chia nguyên (loại bỏ phần thập phân). Tuy nhiên, nếu

một trong hai toán hạng là double, Java sẽ thực hiện phép chia thực. Ví dụ:

```
23 / 4 // Kết quả là 5
```

Nhưng tất cả các biểu thức sau đây đều cho kết quả là 5.75:
markdown

```
23.0 / 4
23. / 4
23 / 4.0
23 / 4.
23. / 4.
23.0 / 4.0
```

Đôi khi bạn muốn Java chuyển đổi ngược lại, tức là từ double sang int. Bạn có thể yêu cầu Java thực hiện chuyển đổi này bằng cách sử dụng cast. Hãy nghĩ về việc “ép kiểu” như việc “thay đổi cái nhìn về một giá trị”. Bạn yêu cầu cast bằng cách đặt tên loại mà bạn muốn chuyển đổi vào trong ngoặc đơn trước giá trị cần ép kiểu. Ví dụ:

```
(int) 4.75
```

sẽ cho ra giá trị int là 4. Khi bạn ép kiểu một giá trị double sang int, nó sẽ loại bỏ bất kỳ phần thập phân nào.
Nếu bạn muốn ép kiểu kết quả của một biểu thức, bạn cần cẩn thận sử dụng dấu ngoặc đơn. Ví dụ, giả sử bạn có một số cuốn sách, mỗi cuốn rộng 0.15 feet và bạn muốn biết bao nhiêu cuốn sẽ vừa với một kệ sách rộng 2.5 feet. Bạn có thể thực hiện phép chia trực tiếp 2.5 / 0.15, nhưng phép này sẽ cho ra một kết quả double nằm giữa 16 và 17. Người Mỹ thường dùng cụm từ "16 and change" để diễn tả một giá trị lớn hơn 16 nhưng không lớn bằng 17. Trong trường hợp này, chúng ta không quan tâm đến phần "change"; chúng ta chỉ muốn tính phần 16. Bạn có thể hình thành biểu thức sau:

```
jshell> (int) 2.5 / 0.15
$1 ==> 13.333333333333334
```

Thật không may, biểu thức này cho kết quả sai vì phép ép kiểu được áp dụng cho giá trị ngay sau nó (ở đây là giá trị 2.5). Điều này chuyển đổi 2.5 thành số nguyên 2, chia cho 0.15 và cho kết quả là 13.33, không phải là một số nguyên và không phải là kết quả đúng. Thay vào đó, bạn nên hình thành biểu thức sau:

```
jshell> (int) (2.5 / 0.15)
$2 ==> 16
```

Biểu thức này đầu tiên thực hiện phép chia để có được 16 và phần thừa, sau đó ép kiểu giá trị đó sang int bằng cách loại bỏ phần thập phân. Nó cho ra giá trị int là 16, là kết quả mà bạn đang tìm kiếm.

2.2 Biến

Dữ liệu nguyên thủy có thể được lưu trữ trong bộ nhớ máy tính dưới dạng biến.

Biến

Một vị trí bộ nhớ có tên và kiểu dữ liệu, lưu trữ một giá trị.

Hãy tưởng tượng bộ nhớ máy tính giống như một bảng tính khổng lồ có nhiều ô, nơi dữ liệu có thể được lưu trữ. Khi bạn tạo một biến trong Java, bạn đang yêu cầu nó dành riêng một trong những ô đó cho biến mới này. Ban đầu, ô này sẽ trống, nhưng bạn sẽ có tùy chọn lưu trữ một giá trị trong ô đó. Và giống như bảng tính, bạn sẽ có tùy chọn thay đổi giá trị trong ô đó sau này. Java cẩn thận hơn một chút so với bảng tính ở chỗ nó yêu cầu bạn phải nói rõ loại dữ liệu bạn sẽ lưu trữ trong ô. Ví dụ, nếu bạn muốn lưu trữ một số nguyên, bạn cần nói với Java rằng bạn định sử dụng kiểu int. Nếu bạn muốn lưu trữ một giá trị thực, bạn cần nói với Java rằng bạn định sử dụng kiểu double. Bạn cũng phải quyết định tên sẽ sử dụng khi bạn muốn tham chiếu đến vị trí bộ nhớ này. Các quy tắc thông thường của Java về định danh áp dụng (tên phải bắt đầu bằng một chữ cái, sau đó có thể là bất kỳ kết hợp nào của chữ cái và chữ số). Quy ước chuẩn trong Java là bắt đầu tên biến bằng chữ cái viết thường, như number hoặc

digits, và viết hoa bất kỳ từ nào tiếp theo, như trong numberOfDigits.

Để khám phá cách sử dụng biến cơ bản, hãy xem xét một chương trình tính chỉ số khối cơ thể (BMI) của một người. Các chuyên gia y tế sử dụng số này để tư vấn cho mọi người về việc liệu họ có bị thừa cân hay không. Dựa trên chiều cao và cân nặng của một người, chúng ta có thể tính BMI của người đó. Một chương trình BMI đơn giản, do đó, sẽ có ba biến cho ba thông tin này. Có một số chi tiết mà chúng ta cần thảo luận về biến, nhưng sẽ hữu ích khi nhìn vào một chương trình hoàn chỉnh trước để xem tổng thể. Chương trình sau đây tính toán và in BMI cho một người có chiều cao 5 feet 10 inches và cân nặng 195 pounds:

```
1  // Tính toán Chỉ số khối cơ thể (BMI) cho một người cụ thể.
 2  public class BMICalculator {
 3      public static void main(String[] args) {
 4          // khai báo biến
 5          double height;
 6          double weight;
 7          double bmi;
 8          // tính toán BMI
 9          height = 70;
10           weight = 195;
11           bmi = weight / (height * height) * 703;
12           // in kết quả
13           System.out.println("Current BMI:");
14           System.out.println(bmi);
15      }
16  }
```

Lưu ý rằng chương trình bao gồm các dòng trống để tách các phần và các nhận xét để chỉ ra các phần khác nhau của chương trình làm gì. Nó tạo ra đầu ra sau:

```
Current BMI:
27.976530612244897
```

Bây giờ hãy xem xét chi tiết của chương trình này để hiểu cách hoạt động của biến. Trước khi các biến có thể được sử dụng trong một chương trình Java, chúng phải được khai báo. Dòng mã khai báo biến được gọi là khai báo biến.

Khai báo

Một yêu cầu để dành một biến mới với tên và kiểu dữ liệu cụ thể. Mỗi biến chỉ được khai báo một lần. Nếu bạn khai báo một biến nhiều hơn một lần, bạn sẽ nhận được thông báo lỗi từ trình biên dịch Java.

Các khai báo biến đơn giản có dạng:

<type> <name>;

như trong ba khai báo ở đầu chương trình mẫu của chúng ta:

double height;
double weight;
double bmi;

Lưu ý rằng một khai báo biến, giống như một câu lệnh, kết thúc bằng dấu chấm phẩy. Các khai báo này có thể xuất hiện ở bất kỳ đâu mà một câu lệnh có thể xuất hiện. Khai báo chỉ ra kiểu dữ liệu và tên của biến. Hãy nhớ rằng tên của mỗi kiểu dữ liệu nguyên thủy là một từ khóa trong Java (int, double, char, boolean). Chúng ta đã sử dụng từ khóa double để định nghĩa kiểu dữ liệu của ba biến này.

Khi một biến được khai báo, Java sẽ dành một vị trí bộ nhớ để lưu trữ giá trị của nó. Tuy nhiên, với dạng khai báo biến đơn giản được sử dụng trong chương trình của chúng ta, Java không lưu trữ giá trị ban đầu trong các vị trí bộ nhớ này. Chúng ta gọi chúng là các biến chưa được khởi tạo, và chúng tương tự như các ô trống trong bảng tính:

Vậy làm thế nào để chúng ta đưa giá trị vào các ô đó?

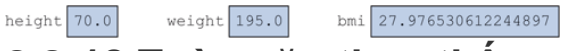
Cách dễ nhất để làm điều này là sử dụng câu lệnh gán. Cú pháp

tổng quát của câu lệnh gán là
<variable> = <expression>;
như trong
height = 70;
Câu lệnh này lưu giá trị 70 vào vị trí bộ nhớ cho biến height, chỉ ra rằng người này cao 70 inch (tương đương 5 feet 10 inch). Chúng ta thường sử dụng cụm từ “nhận” hoặc “được gán” khi đọc câu lệnh như thế này, chẳng hạn như “height nhận giá trị 70” hoặc “height được gán giá trị 70.”
Khi câu lệnh được thực thi, máy tính đầu tiên sẽ đánh giá biểu thức ở phía bên phải; sau đó, nó lưu kết quả vào vị trí bộ nhớ cho biến đã cho. Trong trường hợp này, biểu thức chỉ là một giá trị đơn giản, vì vậy sau khi máy tính thực thi câu lệnh này, bộ nhớ sẽ trông như thế này:
Lưu ý rằng giá trị được lưu dưới dạng 70.0 vì biến height là kiểu double. Biến height bây giờ đã được khởi tạo, nhưng các biến weight và bmi vẫn chưa được khởi tạo. Câu lệnh gán thứ hai gán giá trị cho weight:
weight = 195;
Sau khi thực thi câu lệnh này, bộ nhớ trông như thế này:
Câu lệnh gán thứ ba bao gồm một công thức (một biểu thức để đánh giá):
bmi = weight / (height * height) * 703;
Để tính giá trị của biểu thức này, máy tính chia trọng lượng cho bình phương của chiều cao và sau đó nhân kết quả của phép toán đó với giá trị 703. Kết quả được lưu trong biến bmi. Vì vậy, sau khi máy tính thực thi câu lệnh gán thứ ba, bộ nhớ trông như thế này:
Hai dòng cuối cùng của chương trình báo cáo kết quả BMI bằng cách sử dụng các câu lệnh println:
System.out.println("Current BMI:");
System.out.println(bmi);

Lưu ý rằng chúng ta có thể bao gồm một biến trong câu lệnh println theo cách tương tự như chúng ta bao gồm các giá trị đơn giản và các biểu thức khác để in ra.
Như tên gọi của nó, một biến có thể có các giá trị khác nhau vào các thời điểm khác nhau. Ví dụ, hãy xem xét một biến thể sau đây của chương trình BMI, tính toán một BMI mới giả sử người đó đã giảm 15 pound (từ 195 pound xuống còn 180 pound).

```
// Tính chỉ số khối cơ thể (BMI) cho hai người cụ thể.
public class BMICalculator2 {
    public static void main(String[] args) {
        // Khai báo biến
        double height;
        double weight;
        double bmi;

        // Tính BMI
        height = 70;
        weight = 195;
        bmi = weight / (height * height) * 703;

        // In kết quả
        System.out.println("Previous BMI:");
        System.out.println(bmi);

        // Tính lại BMI
        weight = 180;
        bmi = weight / (height * height) * 703;

        // Báo cáo kết quả mới
        System.out.println("Current BMI:");
        System.out.println(bmi);
    }
}
```

Chương trình bắt đầu theo cùng một cách, thiết lập ba biến với các giá trị sau và báo cáo giá trị BMI ban đầu này:



2.2-12 Toàn văn thay thế

Nhưng chương trình mới sau đó bao gồm câu lệnh gán sau:

weight = 180;

Điều này thay đổi giá trị của biến weight:

2.2-13 Toàn văn thay thế

Bạn có thể nghĩ rằng điều này cũng sẽ thay đổi giá trị của biến bmi. Rốt cuộc, trước đó trong chương trình chúng ta đã nói rằng điều sau đây phải đúng:

bmi = weight / (height * height) * 703;

Đây là nơi mà phép ẩn dụ về bảng tính không còn chính xác nữa. Một bảng tính có thể lưu trữ các công thức trong các ô của nó và khi bạn cập nhật một ô, nó có thể khiến giá trị trong các ô khác được cập nhật. Điều này không đúng trong Java.

Bạn cũng có thể bị nhầm lẫn bởi việc sử dụng dấu bằng để gán. Đừng nhầm lẫn câu lệnh này với một câu lệnh biểu thị sự bằng nhau. Câu lệnh gán không đại diện cho một mối quan hệ đại số. Trong đại số, bạn có thể nói

x = y + 2

Trong toán học, bạn tuyên bố một cách dứt khoát rằng x bằng y cộng thêm hai, một thực tế đúng bây giờ và mãi mãi. Nếu x thay đổi, y sẽ thay đổi theo, và ngược lại. Câu lệnh gán trong Java thì rất khác.

Câu lệnh gán là một lệnh thực hiện một hành động tại một thời điểm cụ thể. Nó không đại diện cho một mối quan hệ bền vững giữa các biến. Đó là lý do tại sao chúng ta thường nói “nhận” hoặc “được gán” thay vì nói “bằng” khi chúng ta đọc các câu lệnh gán.

Quay trở lại chương trình, việc đặt lại biến weight không đặt lại biến bmi. Để tính toán lại bmi dựa trên giá trị mới của weight,

chúng ta phải bao gồm câu lệnh gán thứ hai:
weight = 180;
bmi = weight / (height * height) * 703;
Nếu không, biến bmi sẽ lưu trữ cùng một giá trị như trước. Điều đó sẽ là một kết quả khá đáng buồn để báo cáo cho một người vừa giảm 15 pound. Bằng cách bao gồm cả hai câu lệnh này, chúng ta đặt lại cả hai biến weight và bmi để bộ nhớ trông như thế này:

2.2-14 Toàn văn thay thế

Kết quả đầu ra của phiên bản mới của chương trình là:

```
Previous BMI:
27.976530612244897
Current BMI:
25.82448979591837
```

Một câu lệnh gán rất phổ biến chỉ ra sự khác biệt giữa các mối quan hệ đại số và câu lệnh chương trình là:
x = x + 1;
Hãy nhớ đừng nghĩ về điều này như “x bằng x + 1.” Không có số nào thỏa mãn phương trình đó. Chúng ta sử dụng từ “nhận” để đọc điều này như “x nhận giá trị của x cộng thêm một.” Điều này có vẻ là một câu lệnh khá kỳ lạ, nhưng bạn nên có thể giải mã nó dựa trên các quy tắc đã nêu trước đó. Giả sử giá trị hiện tại của x là 19. Để thực thi câu lệnh, trước tiên bạn đánh giá biểu thức để thu được kết quả là 20. Máy tính lưu trữ giá trị này trong biến được đặt tên ở bên trái, x. Do đó, câu lệnh này thêm một vào giá trị của biến. Chúng ta gọi đây là việc tăng giá trị của x. Nó là một thao tác lập trình cơ bản vì nó là tương đương lập trình của việc đếm (1, 2, 3, 4, v.v.). Câu lệnh sau đây là một biến thể đếm ngược, mà chúng ta gọi là giảm giá trị của một biến:
x = x − 1;
Chúng ta sẽ thảo luận chi tiết hơn về việc tăng và giảm giá trị trong phần sau của chương này.

Biến thể của Gán/Khai báo

Java là một ngôn ngữ phức tạp cung cấp rất nhiều sự linh hoạt cho lập trình viên. Trong phần trước, chúng ta đã thấy hình thức đơn giản nhất của việc khai báo và gán biến, nhưng có nhiều biến thể khác nhau về chủ đề này. Không có gì sai khi bạn bám vào hình thức đơn giản nhất khi bạn đang học, nhưng bạn sẽ gặp các hình thức khác khi đọc các chương trình của người khác, vì vậy bạn sẽ muốn hiểu chúng có nghĩa gì.

Biến thể đầu tiên là Java cho phép bạn cung cấp giá trị ban đầu cho một biến vào thời điểm bạn khai báo nó. Cú pháp như sau: <type> <name> = <expression>;

như trong double height = 70;

double weight = 195;

double bmi = weight / (height * height) * 703;

Biến thể này kết hợp khai báo và gán trong một dòng mã. Hai câu lệnh gán đầu tiên có các số đơn giản sau dấu bằng, nhưng câu lệnh thứ ba có một biểu thức phức tạp sau dấu bằng. Ba câu lệnh này có hiệu quả tương tự như việc cung cấp ba khai báo sau đó là ba câu lệnh gán: double height;

double weight;

double bmi;

height = 70;

weight = 195;

bmi = weight / (height * height) * 703;

Một biến thể khác là khai báo nhiều biến có cùng kiểu dữ liệu trong một câu lệnh duy nhất. Cú pháp là: <type> <name>, <name>, <name>, ..., <name>;

như trong double height, weight;

Ví dụ này khai báo hai biến khác nhau, cả hai đều thuộc kiểu double. Lưu ý rằng kiểu dữ liệu chỉ xuất hiện một lần, ở đầu của khai báo.

Biến thể cuối cùng là sự kết hợp của hai hình thức trước. Bạn có thể khai báo nhiều biến, tất cả đều cùng kiểu, và bạn có thể khởi

tạo chúng cùng một lúc. Ví dụ, bạn có thể nói: double height = 70, weight = 195;
Câu lệnh này khai báo hai biến double là height và weight và gán cho chúng giá trị ban đầu (70 và 195, tương ứng). Java thậm chí còn cho phép bạn kết hợp giữa việc khởi tạo và không khởi tạo, như trong: double height = 70, weight = 195, bmi;
Câu lệnh này khai báo ba biến double có tên height, weight, và bmi và cung cấp giá trị ban đầu cho hai trong số chúng (height và weight). Biến bmi không được khởi tạo.
Kể từ Java 10, bạn có thể khai báo kiểu dữ liệu của một biến là var và trình biên dịch Java sẽ tự động suy luận kiểu dữ liệu thích hợp. Ví dụ, các khai báo: var age = 42;
var height = 70.5;
tương đương với: int age = 42;
double height = 70.5;
Tác giả thích phong cách chuẩn với các kiểu biến được đặt tên rõ ràng, vì vậy chúng tôi sẽ không sử dụng var trong các ví dụ của chúng tôi trong văn bản này.

LỖI LẬP TRÌNH PHỔ BIẾN

Khai báo cùng một biến hai lần

Một trong những điều cần ghi nhớ khi bạn đang học là bạn chỉ có thể khai báo một biến cho mỗi biến cụ thể. Bạn có thể gán giá trị cho nó bao nhiêu lần tùy thích sau khi đã khai báo, nhưng khai báo chỉ nên xuất hiện một lần. Hãy nghĩ về khai báo biến giống như việc nhận phòng vào khách sạn và gán giá trị giống như việc ra vào phòng của bạn. Bạn phải nhận phòng trước để nhận chìa khóa phòng, nhưng sau đó bạn có thể ra vào bất cứ lúc nào bạn muốn. Nếu bạn cố gắng nhận phòng lần thứ hai, khách sạn có thể sẽ hỏi bạn xem bạn có thực sự muốn trả tiền cho một phòng thứ hai không.
Nếu bạn khai báo một biến hơn một lần, Java sẽ tạo ra một lỗi biên dịch. Ví dụ, giả sử chương trình của bạn chứa các dòng sau:

```
int x = 13;
System.out.println(x);
int x = 2;  // dòng này sẽ không biên dịch
System.out.println(x);
```

Dòng đầu tiên là ổn. Nó khai báo một biến nguyên có tên x và khởi tạo nó với giá trị 13. Dòng thứ hai cũng ổn, vì nó chỉ đơn giản là in giá trị của x. Nhưng dòng thứ ba sẽ tạo ra một thông báo lỗi cho biết rằng "x đã được định nghĩa." Nếu bạn muốn thay đổi giá trị của x, bạn cần sử dụng một câu lệnh gán đơn giản thay vì khai báo biến:

```
int x = 13;
System.out.println(x);
x = 2;
System.out.println(x);
```

Chúng tôi đã đề cập đến "câu lệnh gán," nhưng thực ra gán là một toán tử, không phải là một câu lệnh. Khi bạn gán một giá trị cho một biến, toàn bộ biểu thức đánh giá thành giá trị vừa được gán. Điều này có nghĩa là bạn có thể tạo ra các biểu thức có toán tử gán được nhúng bên trong chúng. Không giống như hầu hết các toán tử khác, toán tử gán đánh giá từ phải sang trái, điều này cho phép lập trình viên viết các câu lệnh như sau:

```
int x, y, z;
x = y = z = 2 * 5 + 4;
```

Bởi vì toán tử gán đánh giá từ phải sang trái, câu lệnh này tương đương với:

```
x = (y = (z = 2 * 5 + 4));
```

Biểu thức 2 * 5 + 4 được đánh giá thành 14. Giá trị này được gán cho z. Việc gán này tự nó là một biểu thức được đánh giá thành 14, sau đó được gán cho y. Việc gán cho y cũng được đánh giá thành 14, sau đó được gán cho x. Kết quả là cả ba biến đều được gán giá trị 14.

Nối chuỗi

Trong Chương 1, bạn đã thấy rằng bạn có thể xuất chuỗi ký tự bằng cách sử dụng System.out.println. Bạn cũng có thể xuất các biểu thức số bằng cách sử dụng System.out.println:

```
System.out.println(12 + 3 - 1);
```

Câu lệnh này yêu cầu máy tính đầu tiên đánh giá biểu thức, thu được giá trị 14, sau đó in giá trị đó ra cửa sổ console. Thường thì bạn sẽ muốn xuất nhiều giá trị trên một dòng, nhưng tiếc là bạn chỉ có thể truyền một giá trị duy nhất cho println. Để khắc phục hạn chế này, Java cung cấp một cơ chế đơn giản gọi là **nối chuỗi** để ghép nhiều phần lại thành một chuỗi ký tự dài.

Nối chuỗi

Kết hợp nhiều chuỗi thành một chuỗi đơn, hoặc kết hợp một chuỗi với dữ liệu khác thành một chuỗi mới, dài hơn. Toán tử cộng (+) nối các phần lại với nhau. Điều này tạo thành một biểu thức có thể đánh giá. Ngay cả khi biểu thức bao gồm cả số và văn bản, nó vẫn có thể được đánh giá giống như các biểu thức số mà chúng ta đã khám phá. Ví dụ, hãy xem xét đoạn sau:

```
"I have " + 3 + " things to concatenate"
```

Bạn phải chú ý kỹ đến dấu ngoặc kép trong một biểu thức như thế này để xác định các phần nào là "bên trong" chuỗi ký tự và các phần nào là bên ngoài. Biểu thức này bắt đầu với văn bản "I have " (bao gồm cả khoảng trống ở cuối), sau đó là dấu cộng và giá trị nguyên 3. Java chuyển đổi số nguyên thành dạng văn bản ("3") và nối hai phần lại với nhau thành "I have 3". Theo sau 3 là một dấu cộng và một chuỗi ký tự khác, " things to concatenate" (bắt đầu với khoảng trống). Phần này được gắn vào cuối của chuỗi trước đó để tạo thành chuỗi "I have 3 things to concatenate".

Bạn có thể thấy kết quả của việc nối chuỗi trong JShell:

```
jshell> "I have " + 3 + " things to concatenate"
$1 ==> "I have 3 things to concatenate"
```

Vì biểu thức này tạo ra một chuỗi nối đơn, chúng ta có thể bao gồm nó trong câu lệnh println:

```
System.out.println("I have " + 3 + " things to concatenate");
```

Câu lệnh này tạo ra một dòng kết quả:

```
I have 3 things to concatenate
```

Nối chuỗi thường được sử dụng để báo cáo giá trị của một biến. Ví dụ, hãy xem xét chương trình sau đây tính toán số giờ, phút, và giây trong một năm chuẩn:

```
1  // In ra các chuyển đổi giữa các đơn vị thời gian.
 2  public class Time {
 3      public static void main(String[] args) {
 4          int hours = 365 * 24;
 5          int minutes = hours * 60;
 6          int seconds = minutes * 60;
 7          System.out.println("Hours in a year = " + hours);
 8          System.out.println("Minutes in a year = " + minutes);
 9          System.out.println("Seconds in a year = " + seconds);
10      }
11  }
```

Lưu ý rằng ba lệnh println ở cuối mỗi cái đều có một chuỗi ký tự được nối với một biến. Chương trình này tạo ra kết quả sau:

```
Hours in a year = 8760
Minutes in a year = 525600
Seconds in a year = 31536000
```

Bạn có thể sử dụng nối chuỗi để tạo ra các biểu thức phức tạp. Ví dụ, nếu bạn có các biến x, y, và z và bạn muốn in giá trị của chúng dưới dạng tọa độ với dấu ngoặc đơn và dấu phẩy, bạn có thể nói:

```
System.out.println("(" + x + ", " + y + ", " + z + ")");
```

Nếu x, y, và z có các giá trị lần lượt là 8, 19, và 23, câu lệnh này sẽ xuất ra chuỗi " (8, 19, 23)".

Toán tử + được sử dụng để nối chuỗi có cùng mức ưu tiên như toán tử + trong số học thông thường, điều này có thể gây ra một số nhầm lẫn. Hãy xem xét ví dụ sau:

```
2 + 3 + " hello " + 7 + 2 * 3
```

Biểu thức này có bốn toán tử cộng và một toán tử nhân. Vì ưu tiên, chúng ta sẽ đánh giá phép nhân trước:

```
2 + 3 + " hello " + 7 + 2 * 3
```

Nhóm này có thể trông kỳ lạ, nhưng đó là những gì quy tắc ưu tiên yêu cầu: Chúng ta không đánh giá bất kỳ toán tử cộng nào cho đến khi chúng ta đã đánh giá tất cả các toán tử nhân. Sau khi xử lý phép nhân, chúng ta còn lại bốn toán tử cộng. Chúng sẽ được đánh giá từ trái sang phải.

Phép cộng đầu tiên liên quan đến hai giá trị nguyên. Mặc dù biểu thức tổng thể bao gồm một chuỗi ký tự, vì phép cộng nhỏ này chỉ có hai số nguyên, chúng ta thực hiện phép cộng số nguyên:

```
2 + 3 + " hello " + 7 + 6
```

Phép cộng tiếp theo liên quan đến việc thêm số nguyên 5 vào chuỗi ký tự " hello ". Nếu một trong hai toán hạng là một chuỗi ký tự, chúng ta thực hiện phép nối chuỗi. Vì vậy, trong trường hợp này, chúng ta chuyển đổi số nguyên thành dạng văn bản ("5") và gắn các phần lại với nhau để tạo thành một giá trị chuỗi mới:

```
"5 hello " + 7 + 6
```

Bạn có thể nghĩ rằng Java sẽ cộng 7 và 6 lại với nhau giống như cách nó đã cộng 2 và 3 để thành 5. Nhưng nó không hoạt động như vậy. Các quy tắc ưu tiên rất đơn giản và Java tuân theo chúng một cách nhất quán. Ưu tiên cho chúng ta biết rằng các toán tử cộng được đánh giá từ trái sang phải, vì vậy trước tiên chúng ta nối chuỗi "5 hello " với 7. Đó là một sự kết hợp khác của một chuỗi và một số nguyên, vì vậy Java chuyển đổi số nguyên thành dạng văn bản ("7") và nối hai phần lại với nhau để tạo thành một chuỗi mới:

```
"5 hello 7" + 6
```

Bây giờ chỉ còn lại một phép cộng duy nhất, mà một lần nữa liên quan đến sự kết hợp chuỗi/số nguyên. Chúng ta chuyển đổi số nguyên thành dạng văn bản ("6") và nối hai phần lại với nhau để tạo thành một chuỗi mới:

```
"5 hello 76"
```

Rõ ràng, các biểu thức như vậy có thể gây nhầm lẫn, nhưng bạn sẽ không muốn trình biên dịch Java phải cố gắng đoán ý bạn là gì. Công việc của chúng ta như là lập trình viên sẽ dễ dàng hơn nếu chúng ta biết rằng trình biên dịch sẽ tuân theo các quy tắc đơn giản một cách nhất quán. Bạn có thể làm cho biểu thức rõ ràng hơn và chỉ định cách nó được đánh giá bằng cách thêm dấu ngoặc đơn. Ví dụ, nếu chúng ta thực sự muốn Java cộng 7 và 6 lại với nhau thay vì nối chúng riêng lẻ, chúng ta có thể viết lại biểu thức ban đầu theo cách rõ ràng hơn nhiều như sau:

```
(2 + 3) + " hello " + (7 + 2 * 3)
```

Vì có dấu ngoặc đơn, Java sẽ đánh giá hai phần số học của biểu thức này trước, sau đó nối kết quả với chuỗi ở giữa. Biểu thức này sẽ đánh giá thành "5 hello 13".

Toán tử Tăng/Giảm

Ngoài toán tử gán tiêu chuẩn, Java có một số toán tử đặc biệt hữu ích cho một số thao tác phổ biến trong lập trình. Như đã đề cập trước đó, bạn thường sẽ cần tăng giá trị của một biến lên một lượng nhất định, một thao tác gọi là tăng (increment). Bạn cũng thường sẽ cần giảm giá trị của một biến đi một lượng nhất định, gọi là giảm (decrement). Để thực hiện việc này, bạn viết các câu lệnh như sau:

```
x = x + 1;
y = y - 1;
z = z + 2;
```

Tương tự, bạn sẽ thường muốn nhân đôi hoặc nhân ba giá trị của một biến hoặc giảm giá trị của nó đi một nửa, trong trường hợp đó bạn có thể viết mã như sau:

```
x = x * 2;
y = y * 3;
z = z / 2;
```

Java có một cách viết tắt cho các tình huống này. Bạn kết hợp ký tự toán tử (+, -, *, v.v.) với dấu bằng để có một toán tử gán đặc biệt (+=, -=, *=, v.v.). Cách viết này cho phép bạn viết lại các câu lệnh gán như trên thành:

```
x += 1;
y -= 1;
z += 2;
x *= 2;
y *= 3;
z /= 2;
```

Cách viết này là một chi tiết khác cần học về Java, nhưng nó có thể làm cho mã dễ đọc hơn. Hãy coi câu lệnh x += 2 như câu "cộng 2 vào x". Điều này ngắn gọn hơn so với việc nói x = x + 2. Java thậm chí còn có một cách viết ngắn gọn hơn cho trường hợp cụ thể khi bạn muốn tăng lên 1 hoặc giảm đi 1. Trong trường hợp này, bạn có thể sử dụng các toán tử tăng và giảm (++ và --). Ví dụ:

```
x++;
y--;
```

Thực tế, có hai dạng khác nhau của mỗi toán tử này, vì bạn cũng có thể đặt toán tử trước biến:

```
++x;
--y;
```

Hai phiên bản của ++ được gọi là toán tử tiền tăng (++x) và hậu tăng (x++). Tương tự, hai phiên bản của -- được gọi là toán tử tiền giảm (--x) và hậu giảm (x--). Sự khác biệt giữa tiền và hậu không quan trọng khi bạn sử dụng chúng như các câu lệnh độc lập, như trong hai ví dụ trên. Sự khác biệt chỉ xuất hiện khi bạn nhúng các câu lệnh này vào các biểu thức phức tạp hơn, điều mà chúng tôi không khuyến nghị.

Bây giờ chúng ta đã thấy một số toán tử mới, điều đáng để quay lại là vấn đề về thứ tự ưu tiên. Bảng 2.5 cho thấy phiên bản cập nhật của bảng thứ tự ưu tiên của toán tử Java, bao gồm cả các toán tử gán và toán tử tăng, giảm. Lưu ý rằng các toán tử tăng và giảm được nhóm với các toán tử đơn và có mức ưu tiên cao nhất.

Bảng 2.5: Thứ tự ưu tiên của toán tử Java

Mô tả	**Toán tử**
Toán tử đơn	++, --, +, -
Toán tử nhân chia	*, /, %
Toán tử cộng trừ	+, -
Toán tử gán	=, +=, -=, *=, /=, %=

Bạn có thể khai báo và sử dụng các biến trong JShell. Lệnh JShell /vars hiển thị tất cả các biến mà bạn đã khai báo. Lưu ý trong tương tác dưới đây rằng đầu ra của /vars cũng liệt kê các ID như $1, $2, v.v., được tự động tạo ra nếu bạn không đặt tên biến cho một biểu thức đã cho. Đây thực ra là các biến bạn có thể sử dụng nếu bạn muốn tham chiếu đến một phép tính trước đó.

```
jshell> int age = 42;
age ==> 42

jshell> double height = 63.5;
height ==> 63.5

jshell> age * 2
$3 ==> 84

jshell> /vars
|    int age = 42
|    double height = 63.5
|    int $3 = 84
```

```
jshell> $3 + 2
$4 ==> 86
```

BẠN CÓ BIẾT?

Toán tử ++ và --

Toán tử ++ và -- lần đầu tiên được giới thiệu trong ngôn ngữ lập trình C. Java có chúng vì những người thiết kế ngôn ngữ đã quyết định sử dụng cú pháp của C làm cơ sở cho cú pháp Java. Nhiều ngôn ngữ đã thực hiện lựa chọn tương tự, bao gồm C++ và C#. Các lập trình viên C thường tự hào rằng những toán tử này cho phép bạn viết mã rất ngắn gọn, nhưng nhiều người khác cảm thấy rằng chúng có thể làm mã phức tạp không cần thiết. Trong cuốn sách này, chúng tôi luôn sử dụng những toán tử này như các câu lệnh riêng biệt để rõ ràng, nhưng để đầy đủ, chúng ta sẽ xem xét tùy chọn khác ở đây.

Các biến thể trước (pre-) và sau (post-) đều có tác dụng chung—hai toán tử tăng ++ sẽ tăng giá trị của một biến và hai toán tử giảm -- sẽ giảm giá trị của một biến—nhưng chúng khác nhau về giá trị mà chúng đánh giá. Khi bạn tăng hoặc giảm, thực sự có hai giá trị liên quan: giá trị ban đầu mà biến có trước khi thực hiện phép tăng hoặc giảm, và giá trị cuối cùng mà biến có sau khi thực hiện phép tăng hoặc giảm. Các phiên bản sau (post-) đánh giá giá trị ban đầu (cũ hơn) và các phiên bản trước (pre-) đánh giá giá trị cuối cùng (mới hơn).

Xem xét ví dụ sau:

```
int x = 10;
int y = 20;
int z = ++x * y--;
```

Giá trị nào được gán cho z? Câu trả lời là 220. Phép gán thứ ba tăng x lên 11 và giảm y xuống 19, nhưng khi tính giá trị của z, nó sử dụng giá trị mới của x (++x) nhân với giá trị cũ của y (y--), là 11 nhân với 20, hay 220.

Có một cách nhớ đơn giản cho điều này: Khi bạn thấy x++, hãy đọc là "cho tôi x, sau đó tăng", và khi thấy ++x, hãy đọc là "tăng, sau đó cho tôi x". Một cách nhớ khác có thể hữu ích là nhớ rằng C++ là một tên không hay cho một ngôn ngữ lập trình. Biểu thức “C++” sẽ được hiểu là “đánh giá giá trị cũ của C và sau đó tăng C”. Nói cách khác, mặc dù bạn đang cố gắng tạo ra thứ gì đó mới và khác biệt, bạn thực sự vẫn bị mắc kẹt với ngôn ngữ cũ. Ngôn ngữ bạn muốn là ++C, sẽ là một ngôn ngữ mới và cải tiến hơn so với cái cũ. Một số người đã gợi ý rằng có lẽ Java là ++C.

Biến và Pha Trộn Kiểu Dữ Liệu

Bạn đã biết rằng khi bạn khai báo một biến, bạn phải cho Java biết kiểu dữ liệu mà nó sẽ lưu trữ. Ví dụ, bạn có thể khai báo một biến kiểu int cho các giá trị nguyên hoặc kiểu double cho các giá trị thực. Tình huống khá rõ ràng khi bạn chỉ có số nguyên hoặc số thực, nhưng chuyện gì xảy ra khi bạn bắt đầu pha trộn các kiểu dữ liệu? Ví dụ, đoạn mã sau rõ ràng là ổn:

```
int x;
double y;
x = 2 + 3;
y = 3.4 * 2.9;
```

Ở đây, chúng ta có một biến số nguyên mà chúng ta gán một giá trị nguyên và một biến double mà chúng ta gán một giá trị double. Nhưng điều gì sẽ xảy ra nếu chúng ta làm ngược lại?

```
int x;
double y;
x = 3.4 * 2.9;  // không hợp lệ
y = 2 + 3;      // hợp lệ
```

Như các chú thích chỉ ra, bạn không thể gán một biến số nguyên với một giá trị double, nhưng bạn có thể gán một biến double với một giá trị nguyên. Hãy xem xét trường hợp thứ hai trước. Biểu thức 2 + 3 được đánh giá là số nguyên 5. Giá trị này không phải là một số thực (double), nhưng mọi số nguyên đều là giá trị thực,

vì vậy Java dễ dàng chuyển đổi số nguyên thành số thực. Thuật ngữ kỹ thuật là Java "nâng cấp" số nguyên thành double. Trường hợp đầu tiên có vấn đề hơn. Biểu thức 3.4 * 2.9 được đánh giá là giá trị double là 9.86. Giá trị này không thể được lưu trữ trong một số nguyên vì nó không phải là số nguyên. Nếu bạn muốn thực hiện loại thao tác này, bạn sẽ phải yêu cầu Java chuyển đổi giá trị này thành một số nguyên. Như đã mô tả trước đó, bạn có thể ép kiểu (cast) một giá trị double thành int, điều này sẽ cắt bỏ mọi thứ sau dấu thập phân:

```
x = (int) (3.4 * 2.9);  // bây giờ hợp lệ
```

Câu lệnh này trước tiên đánh giá 3.4 * 2.9 để nhận được 9.86 và sau đó cắt bỏ giá trị này để nhận được số nguyên 9.

LỖI LẬP TRÌNH PHỔ BIẾN

Quên Ép Kiểu (Cast)

Chúng ta thường viết các chương trình liên quan đến việc kết hợp các kiểu dữ liệu int và double, vì vậy rất dễ mắc lỗi khi kết hợp hai kiểu dữ liệu này. Ví dụ, giả sử bạn muốn tính toán tỷ lệ phần trăm số câu trả lời đúng trên một bài kiểm tra của học sinh, với tổng số câu hỏi trong bài kiểm tra và số câu hỏi mà học sinh trả lời đúng. Bạn có thể khai báo các biến sau:

```
int totalQuestions;
int numRight;
double percent;
```

Giả sử hai biến đầu tiên được khởi tạo như sau:

```
totalQuestions = 73;
numRight = 59;
```

Làm thế nào để bạn tính toán tỷ lệ phần trăm số câu mà học sinh trả lời đúng? Bạn chia số câu đúng cho tổng số câu hỏi và nhân với 100 để chuyển thành phần trăm:

```
percent = numRight / totalQuestions * 100;  // không đúng
```

Đáng tiếc, nếu bạn in ra giá trị của biến percent sau khi thực thi dòng mã này, bạn sẽ thấy rằng nó có giá trị là 0.0. Nhưng rõ ràng là học sinh đã trả lời đúng hơn 0%.
Vấn đề đến từ phép chia nguyên (integer division). Biểu thức bạn đang sử dụng bắt đầu với hai giá trị int:

```
numRight / totalQuestions
```

Điều này có nghĩa là bạn đang tính toán:

```
59 / 73
```

Điều này đánh giá thành 0 với phép chia nguyên. Một số học sinh sửa lỗi này bằng cách thay đổi kiểu của tất cả các biến thành double. Điều đó sẽ giải quyết vấn đề ngay lập tức, nhưng đó không phải là lựa chọn tốt từ quan điểm phong cách. Tốt nhất là sử dụng kiểu dữ liệu phù hợp nhất cho dữ liệu, và số câu hỏi trong bài kiểm tra chắc chắn sẽ là một số nguyên.
Bạn có thể cố gắng sửa lỗi này bằng cách thay đổi giá trị 100 thành 100.0:

```
percent = numRight / totalQuestions * 100.0;  // không đúng
```

nhưng điều này không giúp ích vì phép chia được thực hiện trước. Tuy nhiên, nó sẽ hoạt động nếu bạn đặt 100.0 trước:

```
percent = 100.0 * numRight / totalQuestions;
```

Bây giờ phép nhân được thực hiện trước phép chia, điều này có nghĩa là mọi thứ sẽ được chuyển đổi thành double.
Đôi khi bạn có thể sửa lỗi như thế này thông qua việc sắp xếp lại công thức một cách thông minh, nhưng bạn không muốn phụ thuộc vào sự thông minh đó. Đây là một nơi tốt để sử dụng ép kiểu (cast). Ví dụ, quay lại công thức ban đầu, bạn có thể ép kiểu từng biến int thành double:

```
percent = (double) numRight / (double) totalQuestions * 100.0;
```

Bạn cũng có thể tận dụng thực tế rằng một khi bạn đã ép kiểu một trong hai biến này thành double, phép chia sẽ được thực hiện với double. Vì vậy, bạn có thể, ví dụ, chỉ cần ép kiểu giá trị đầu tiên thành double:

```
percent = (double) numRight / totalQuestions * 100.0;
```

2.3 Vòng lặp for

Lập trình thường liên quan đến việc chỉ định các nhiệm vụ lặp lại. Vòng lặp for giúp tránh sự dư thừa này bằng cách thực thi lặp lại một dãy các câu lệnh qua một phạm vi giá trị cụ thể. Giả sử bạn muốn viết ra các bình phương của năm số nguyên đầu tiên. Bạn có thể viết một chương trình như sau:

```
1  // Một chương trình dư thừa để tính bình phương các số
nguyên.
2  public class WriteSquares {
3      public static void main(String[] args) {
4          System.out.println(1 + " squared = " + (1 * 1));
5          System.out.println(2 + " squared = " + (2 * 2));
6          System.out.println(3 + " squared = " + (3 * 3));
7          System.out.println(4 + " squared = " + (4 * 4));
8          System.out.println(5 + " squared = " + (5 * 5));
9      }
10 }
```

Chương trình này sẽ tạo ra kết quả sau:

```
1 squared = 1
2 squared = 4
3 squared = 9
4 squared = 16
5 squared = 25
```

Nhưng cách tiếp cận này rất tẻ nhạt. Chương trình có năm câu lệnh rất giống nhau. Chúng đều có dạng:

```
System.out.println(number + " squared = " + (number * number));
```

với number là 1, 2, 3, 4, hoặc 5. Vòng lặp for giúp tránh sự dư thừa này. Đây là một chương trình tương đương sử dụng vòng lặp for:

```
1  // Một chương trình tính bình phương các số nguyên bằng
vòng lặp for.
```

```
2  public class WriteSquares2 {
3      public static void main(String[] args) {
4          for (int i = 1; i <= 5; i++) {
5              System.out.println(i + " squared = " + (i * i));
6          }
7      }
8  }
```

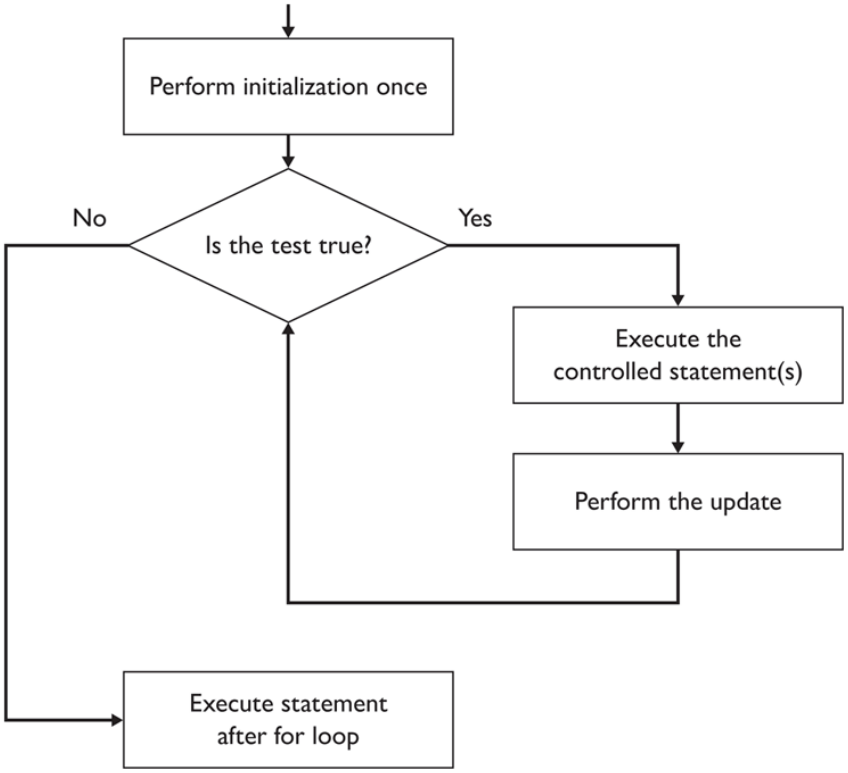
Chương trình này khởi tạo một biến gọi là i với giá trị là 1. Sau đó, nó lặp lại việc thực thi câu lệnh println miễn là giá trị của biến i nhỏ hơn hoặc bằng 5. Sau mỗi lần thực thi println, nó đánh giá biểu thức i++ để tăng giá trị của i.

Cú pháp tổng quát của vòng lặp for là như sau:

```
for (<khởi tạo>; <kiểm tra tiếp tục>; <cập nhật>) {
    <câu lệnh>;
    <câu lệnh>;
    ...
    <câu lệnh>;
}
```

Bạn luôn phải bao gồm từ khóa for và dấu ngoặc đơn. Bên trong dấu ngoặc đơn có ba phần khác nhau, được phân cách bằng dấu chấm phẩy: khởi tạo, kiểm tra tiếp tục, và cập nhật. Sau đó là một cặp dấu ngoặc nhọn bao quanh tập hợp các câu lệnh. Vòng lặp for điều khiển các câu lệnh bên trong dấu ngoặc nhọn. Chúng ta gọi các câu lệnh được điều khiển là thân của vòng lặp. Ý tưởng là chúng ta thực thi thân vòng lặp nhiều lần, như được xác định bởi sự kết hợp của ba phần khác.

Hình dưới đây cho thấy các bước mà Java tuân theo để thực thi vòng lặp for. Nó thực hiện bất kỳ sự khởi tạo nào mà bạn đã yêu cầu một lần trước khi vòng lặp bắt đầu thực thi. Sau đó, nó lặp lại việc thực hiện kiểm tra tiếp tục mà bạn đã cung cấp.



Hình 2.1 Văn bản thay thế đầy đủ

Nếu bài kiểm tra tiếp tục (continuation test) cho kết quả là true, nó sẽ thực thi các câu lệnh được điều khiển một lần và thực hiện phần cập nhật (update). Sau đó, nó thực hiện lại bài kiểm tra. Nếu kết quả lại là true, nó sẽ thực thi lại các câu lệnh và thực hiện cập nhật lại. Lưu ý rằng phần cập nhật được thực hiện sau khi các câu lệnh được điều khiển đã được thực thi. Khi bài kiểm tra cho kết quả false, Java kết thúc việc thực thi vòng lặp và chuyển sang câu lệnh tiếp theo sau vòng lặp.

Vòng lặp for là ví dụ đầu tiên về cấu trúc điều khiển (control structure) mà chúng ta sẽ nghiên cứu.

Cấu trúc điều khiển

Một cấu trúc cú pháp điều khiển các câu lệnh khác.

Bạn nên cẩn thận sử dụng thụt đầu dòng để chỉ ra các câu lệnh được điều khiển. Trong trường hợp của vòng lặp for, tất cả các câu lệnh trong thân vòng lặp (body of the loop) đều được thụt đầu dòng như một cách để chỉ ra rằng chúng nằm “bên trong” vòng lặp.

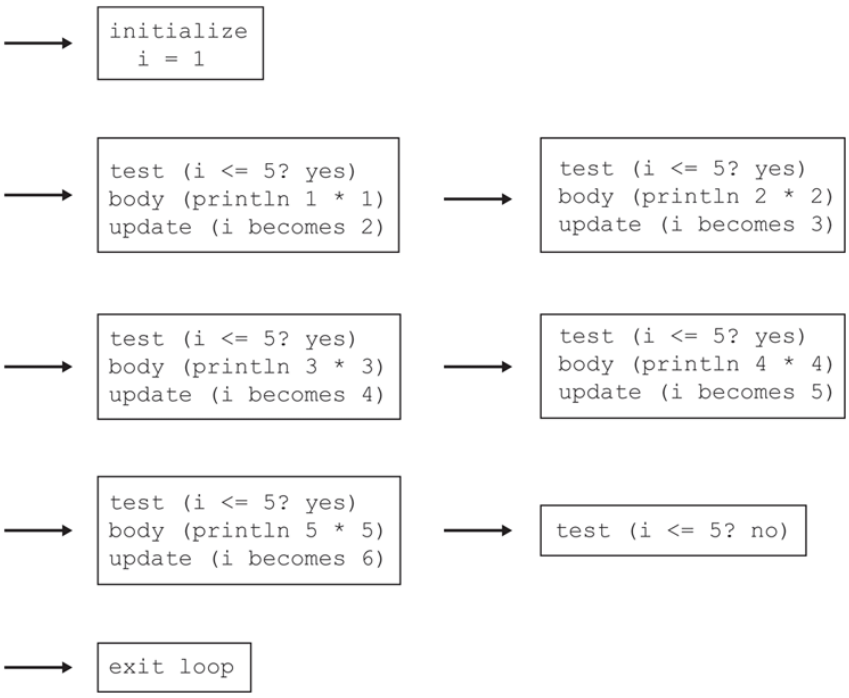
Dò tìm vòng lặp for

Hãy xem xét kỹ lưỡng vòng lặp for của chương trình WriteSquares2:

```
for (int i = 1; i <= 5; i++) {

    System.out.println(i + " squared = " + (i * i));

}
```

Trong vòng lặp này, phần khởi tạo (initialization) int i = 1 khai báo một biến nguyên i và gán cho nó giá trị 1. Bài kiểm tra tiếp tục i <= 5 cho biết rằng chúng ta nên tiếp tục thực thi miễn là i nhỏ hơn hoặc bằng 5. Điều này có nghĩa là khi i lớn hơn 5, chúng ta sẽ dừng việc thực thi thân vòng lặp. Phần cập nhật i++ sẽ tăng giá trị của i thêm một mỗi lần, đưa i đến gần hơn với giá trị lớn hơn 5. Sau năm lần thực thi thân vòng lặp và năm lần cập nhật, i sẽ lớn hơn 5 và vòng lặp sẽ kết thúc. Hình 2.2 sẽ dò tìm chi tiết quá trình này.

Hình 2.2 Dò tìm chi tiết của vòng lặp for



Hình 2.2 Văn bản thay thế đầy đủ

Java cho phép linh hoạt lớn trong việc quyết định những gì cần đưa vào phần khởi tạo (initialization) và cập nhật (update), do đó chúng ta có thể sử dụng vòng lặp for để giải quyết tất cả các nhiệm vụ lập trình khác nhau. Tuy nhiên, lúc này, chúng ta sẽ giới hạn mình ở một loại vòng lặp cụ thể, khai báo và khởi tạo một biến duy nhất được sử dụng để điều khiển vòng lặp. Biến này thường được gọi là biến vòng lặp (loop variable). Trong bài kiểm tra, chúng ta so sánh biến vòng lặp với một giá trị mong muốn cuối cùng, và trong phần cập nhật, chúng ta thay đổi giá trị của biến vòng lặp, thường là tăng nó lên 1. Những vòng lặp này rất phổ biến trong lập trình. Theo quy ước, chúng ta thường sử dụng các tên như i, j, và k cho các biến vòng lặp.

Mỗi lần thực thi câu lệnh được điều khiển của một vòng lặp được gọi là một lần lặp (iteration) của vòng lặp (ví dụ, “Vòng lặp đã hoàn thành sau bốn lần lặp”). Iteration cũng có thể ám chỉ việc lặp

lại nói chung (ví dụ, “Tôi đã giải quyết vấn đề bằng cách sử dụng iteration”).

Xem xét một vòng lặp for khác:

```
for (int i = −100; i <= 100; i++) {

    System.out.println(i + " squared = " + (i * i));

}
```

Vòng lặp này thực hiện tổng cộng 201 lần, tạo ra các bình phương của tất cả các số nguyên từ −100 đến +100 bao gồm. Các giá trị được sử dụng trong phần khởi tạo và bài kiểm tra có thể là bất kỳ số nguyên nào. Trên thực tế, chúng có thể là các biểu thức số nguyên bất kỳ:

```
for (int i = (2 + 2); i <= (17 * 3); i++) {

    System.out.println(i + " squared = " + (i * i));

}
```

Vòng lặp này sẽ tạo ra các bình phương của tất cả các số nguyên từ 4 đến 51 bao gồm. Các dấu ngoặc đơn xung quanh các biểu thức không cần thiết nhưng cải thiện tính dễ đọc. Xem xét vòng lặp sau:

```
for (int i = 1; i <= 30; i++) {

    System.out.println("+--------+");

}
```

Vòng lặp này tạo ra 30 dòng đầu ra, tất cả đều giống hệt nhau. Nó hơi khác so với vòng lặp trước đó vì câu lệnh được điều khiển bởi vòng lặp for không tham chiếu đến biến vòng lặp. Do đó,

```
for (int i = −30; i <= −1; i++) {

    System.out.println("+--------+");

}
```

cũng tạo ra cùng một đầu ra. Hành vi của một vòng lặp như vậy chỉ được xác định bởi số lần lặp nó thực hiện. Số lần lặp được tính bằng công thức:

```
<giá trị kết thúc> − <giá trị bắt đầu> + 1
```

Rất đơn giản để thấy rằng vòng lặp đầu tiên lặp lại 30 lần, do đó tốt hơn là sử dụng vòng lặp đó.

Bây giờ hãy xem xét một số trường hợp biên. Xem xét vòng lặp này:

```
for (int i = 1; i <= 1; i++) {

    System.out.println("+--------+");

}
```

Theo quy tắc của chúng ta, vòng lặp này sẽ lặp một lần, và nó thực sự làm như vậy. Nó khởi tạo biến i thành 1 và kiểm tra xem giá trị này có nhỏ hơn hoặc bằng 1 hay không, điều này đúng. Vì vậy, nó thực thi câu lệnh println, tăng i, và kiểm tra lại. Lần thứ hai

kiểm tra, nó phát hiện ra rằng i không còn nhỏ hơn hoặc bằng 1, vì vậy nó dừng thực thi. Bây giờ hãy xem xét vòng lặp này:

```
for (int i = 1; i <= 0; i++) {

    System.out.println("+--------+"); // không bao giờ thực thi

}
```

Vòng lặp này không thực hiện bất kỳ lần lặp nào. Nó sẽ không gây ra lỗi thực thi; nó chỉ đơn giản là không thực thi phần thân. Nó khởi tạo biến thành 1 và kiểm tra xem giá trị này có nhỏ hơn hoặc bằng 0 hay không. Điều này không đúng, vì vậy thay vì thực thi các câu lệnh trong phần thân, nó dừng ở đó.

Khi bạn tạo một vòng lặp for, bạn có thể bao gồm nhiều hơn một câu lệnh bên trong dấu ngoặc nhọn. Xem xét, ví dụ, đoạn mã sau:

```
for (int i = 1; i <= 20; i++) {

    System.out.println("Hi!");

    System.out.println("Ho!");

}
```

Điều này sẽ tạo ra 20 cặp dòng, dòng đầu tiên có từ “Hi!” và dòng thứ hai có từ “Ho!”.

Khi một vòng lặp for điều khiển một câu lệnh duy nhất, bạn không cần phải bao gồm dấu ngoặc nhọn. Dấu ngoặc nhọn chỉ cần thiết trong các tình huống như ví dụ trước, khi bạn có nhiều hơn một câu lệnh mà bạn muốn vòng lặp điều khiển. Tuy nhiên, quy ước

mã hóa của Java bao gồm dấu ngoặc nhọn ngay cả khi chỉ có một câu lệnh, và chúng ta tuân theo quy ước này trong cuốn sách này. Có hai lợi ích của quy ước này:

- Bao gồm dấu ngoặc nhọn ngăn ngừa các lỗi trong tương lai. Ngay cả khi bạn chỉ cần một câu lệnh trong phần thân của vòng lặp của bạn bây giờ, mã của bạn có thể thay đổi theo thời gian. Việc có dấu ngoặc nhọn ở đó đảm bảo rằng, nếu bạn thêm một câu lệnh bổ sung vào phần thân sau này, bạn sẽ không vô tình quên bao gồm chúng. Nói chung, việc bao gồm dấu ngoặc nhọn trước là rẻ hơn so với việc tìm ra lỗi khó hiểu sau này.

- Luôn bao gồm dấu ngoặc nhọn giúp giảm bớt mức độ chi tiết bạn phải xem xét khi học các cấu trúc điều khiển mới. Mất thời gian để nắm vững chi tiết của bất kỳ cấu trúc điều khiển mới nào, và sẽ dễ dàng hơn để nắm vững những chi tiết đó nếu bạn không phải nghĩ về việc khi nào nên bao gồm và khi nào không nên bao gồm dấu ngoặc nhọn.

LỖI LẬP TRÌNH PHỔ BIẾN

Quên dấu ngoặc nhọn

Bạn nên sử dụng thụt lề để chỉ ra thân (body) của một vòng lặp for, nhưng chỉ thụt lề thôi là không đủ. Java bỏ qua thụt lề khi quyết định cách nhóm các câu lệnh khác nhau. Giả sử, chẳng hạn, bạn viết mã sau:

```
for (int i = 1; i <= 20; i++)

    System.out.println("Hi!");

    System.out.println("Ho!");
```

Việc thụt lề báo hiệu cho người đọc rằng cả hai câu lệnh println đều nằm trong thân của vòng lặp for, nhưng không có dấu ngoặc nhọn nào để báo hiệu điều đó cho Java. Kết quả là, mã này được diễn giải như sau:

```
for (int i = 1; i <= 20; i++) {

    System.out.println("Hi!");

}

System.out.println("Ho!");
```

Chỉ có câu lệnh println đầu tiên được coi là nằm trong thân của vòng lặp for. Câu lệnh println thứ hai được coi là nằm ngoài vòng lặp. Vì vậy, mã này sẽ tạo ra 20 dòng đầu ra đều ghi "Hi!" theo sau là một dòng đầu ra ghi "Ho!". Để bao gồm cả hai câu lệnh println trong thân, bạn cần thêm dấu ngoặc nhọn xung quanh chúng:

```
for (int i = 1; i <= 20; i++) {

    System.out.println("Hi!");

    System.out.println("Ho!");

}
```

Mẫu vòng lặp for

Nói chung, nếu bạn muốn một vòng lặp lặp lại chính xác n lần, bạn sẽ sử dụng một trong hai vòng lặp tiêu chuẩn. Dạng tiêu chuẩn đầu tiên trông giống như các vòng lặp mà bạn đã thấy:

```
for (int <biến> = 1; <biến> <= n; <biến>++) {

    <câu lệnh>;

    <câu lệnh>;

    ...

    <câu lệnh>;

}
```

Rõ ràng là vòng lặp này thực thi n lần, vì nó bắt đầu từ 1 và tiếp tục miễn là nó nhỏ hơn hoặc bằng n. Ví dụ, vòng lặp này in ra các số từ 1 đến 10:

```
for (int i = 1; i <= 10; i++) {

    System.out.print(i + " ");

}
```

Vì nó sử dụng print thay vì println, nó tạo ra một dòng đầu ra duy nhất:

```
1 2 3 4 5 6 7 8 9 10
```

Tuy nhiên, thường tiện lợi hơn khi bắt đầu đếm từ 0 thay vì từ 1. Điều này đòi hỏi thay đổi trong bài kiểm tra vòng lặp để cho phép bạn dừng khi n nhỏ hơn một:

```
for (int <biến> = 0; <biến> < n; <biến>++) {

    <câu lệnh>;
```

```
    <câu lệnh>;

    ...

    <câu lệnh>;

}
```

Lưu ý rằng trong dạng này, khi bạn khởi tạo biến bằng 0, bạn kiểm tra xem nó có nhỏ hơn n không. Cả hai dạng đều thực thi chính xác n lần, mặc dù có một số tình huống mà vòng lặp bắt đầu từ 0 hoạt động tốt hơn. Ví dụ, vòng lặp này thực thi 10 lần giống như vòng lặp trước:

```
for (int i = 0; i < 10; i++) {

    System.out.print(i + " ");

}
```

Vì nó bắt đầu từ 0 thay vì từ 1, nó tạo ra một chuỗi 10 giá trị khác nhau:

```
0 1 2 3 4 5 6 7 8 9
```

Thường thì bạn sẽ sử dụng vòng lặp bắt đầu từ 0 hoặc 1 để thực hiện một thao tác nào đó một số lần cố định. Nhưng cũng có một biến thể nhỏ mà đôi khi hữu ích. Thay vì chạy vòng lặp theo chiều tiến, chúng ta có thể chạy nó theo chiều lùi. Thay vì bắt đầu từ 1 và thực thi cho đến khi đạt đến n, bạn bắt đầu từ n và tiếp tục thực thi cho đến khi đạt đến 1. Bạn có thể làm điều này bằng cách sử dụng phép giảm (decrement) thay vì phép tăng

(increment), vì vậy chúng ta đôi khi gọi đây là vòng lặp giảm (decrementing loop).

Đây là dạng chung của vòng lặp giảm:

```
for (int <biến> = n; <biến> >= 1; <biến>--) {

    <câu lệnh>;

    <câu lệnh>;

    ...

    <câu lệnh>;

}
```

Ví dụ, đây là một vòng lặp giảm thực thi 10 lần:

```
for (int i = 10; i >= 1; i--) {

    System.out.print(i + " ");

}
```

Vì nó chạy ngược, nó in ra các giá trị theo thứ tự ngược lại:

```
10 9 8 7 6 5 4 3 2 1
```

JShell cho phép bạn viết các vòng lặp for. Vì một vòng lặp trải dài nhiều dòng, trình shell hiển thị một dấu nhắc tiếp tục ...> trên mỗi dòng tiếp theo cho đến khi bạn hoàn thành vòng lặp bằng cách viết dấu ngoặc nhọn } đóng, như được hiển thị trong tương tác

sau. Khi bạn hoàn thành việc viết vòng lặp hoàn chỉnh, vòng lặp sẽ thực thi.

```
jshell>



for (int i = 1; i <= 10; i++) {

  ...> System.out.print(i + "! ");

  ...> }
```

Sẽ tạo ra đầu ra:

```
1! 2! 3! 4! 5! 6! 7! 8! 9! 10!
```

Đoạn mã đã được giữ nguyên với các dòng mã không bị rớt hàng, các chỉ mục dòng tương ứng và các dòng trống đã được loại bỏ.

Vòng lặp for lồng nhau

Vòng lặp for điều khiển một câu lệnh, và bản thân vòng lặp for cũng là một câu lệnh, có nghĩa là một vòng lặp for có thể điều khiển một vòng lặp for khác. Ví dụ, bạn có thể viết mã như sau:

```
for (int i = 1; i <= 10; i++) {

    for (int j = 1; j <= 5; j++) {

        System.out.println("Hi there.");
```

```
    }

}
```

Mã này có lẽ dễ đọc hơn từ bên trong ra ngoài. Câu lệnh println tạo ra một dòng đầu ra duy nhất. Vòng lặp j bên trong thực thi câu lệnh này năm lần, tạo ra năm dòng đầu ra. Vòng lặp i bên ngoài thực thi vòng lặp bên trong 10 lần, tạo ra 10 tập hợp mỗi tập gồm 5 dòng, hay 50 dòng đầu ra. Mã trước đó, do đó, tương đương với:

```
for (int i = 1; i <= 50; i++) {

    System.out.println("Hi there.");

}
```

Ví dụ này cho thấy rằng một vòng lặp for có thể được điều khiển bởi một vòng lặp for khác. Vòng lặp như vậy được gọi là vòng lặp lồng nhau. Tuy nhiên, ví dụ này không quá thú vị, vì vòng lặp lồng nhau có thể được loại bỏ.

Bây giờ bạn đã biết cách viết vòng lặp for, bạn sẽ muốn có khả năng tạo ra các dòng đầu ra phức tạp từng phần bằng cách sử dụng lệnh print. Nhớ lại từ Chương 1 rằng lệnh print in ra trên dòng đầu ra hiện tại mà không chuyển sang một dòng đầu ra mới. Ví dụ, nếu bạn muốn tạo ra một dòng đầu ra có 80 dấu sao, bạn có thể sử dụng lệnh print để in từng dấu sao một và thực thi 80 lần thay vì sử dụng một lệnh println duy nhất.

Hãy xem một vòng lặp lồng nhau thú vị hơn sử dụng lệnh print:

```
for (int i = 1; i <= 6; i++) {
```

```
    for (int j = 1; j <= 3; j++) {

        System.out.print(j + " ");

    }

}
```

Chúng ta có thể một lần nữa đọc mã này từ bên trong ra ngoài. Vòng lặp bên trong in ra giá trị của biến vòng lặp j khi nó thay đổi từ 1 đến 3. Vòng lặp bên ngoài thực thi điều này sáu lần khác nhau. Kết quả là chúng ta có sáu chuỗi "1 2 3" liên tiếp trong đầu ra:

```
1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
```

Mã này in tất cả đầu ra của nó trên một dòng duy nhất. Hãy xem xét một đoạn mã kết hợp giữa print và println để tạo ra nhiều dòng đầu ra:

```
for (int i = 1; i <= 6; i++) {

    for (int j = 1; j <= 10; j++) {

        System.out.print("*");

    }

    System.out.println();

}
```

Khi bạn viết mã có chứa các vòng lặp lồng nhau, bạn phải cẩn thận thụt lề mã đúng cách để làm rõ cấu trúc. Ở cấp độ ngoài

cùng, đoạn mã trước đó là một vòng lặp for đơn giản thực thi sáu lần:

```
for (int i = 1; i <= 6; i++) {

    ...

}
```

Chúng ta sử dụng thụt lề cho các câu lệnh bên trong vòng lặp for này để làm rõ rằng chúng là thân (body) của vòng lặp này. Bên trong, chúng ta tìm thấy hai câu lệnh: một vòng lặp for khác và một println. Hãy xem xét vòng lặp for bên trong:

```
for (int j = 1; j <= 10; j++) {

    System.out.print("*");

}
```

Vòng lặp này được điều khiển bởi vòng lặp for bên ngoài, đó là lý do tại sao nó được thụt lề, nhưng bản thân nó cũng điều khiển một câu lệnh (câu lệnh print), vì vậy chúng ta có thêm một cấp độ thụt lề khác. Việc thụt lề này cho thấy rằng câu lệnh print được điều khiển bởi vòng lặp for bên trong, và vòng lặp này được điều khiển bởi vòng lặp for bên ngoài. Vậy vòng lặp bên trong này làm gì? Nó in ra 10 dấu sao trên dòng đầu ra hiện tại. Tất cả chúng đều xuất hiện trên cùng một dòng vì chúng ta đang sử dụng print thay vì println. Lưu ý rằng sau vòng lặp này, chúng ta thực hiện một lệnh println:

```
System.out.println();
```

Tác động của vòng lặp for tiếp theo lệnh println là chúng ta có được một dòng đầu ra với 10 dấu sao trên đó. Nhưng hãy nhớ rằng các câu lệnh này nằm trong một vòng lặp ngoài thực thi sáu lần, vì vậy chúng ta sẽ có được sáu dòng đầu ra, mỗi dòng có 10 dấu sao:

```
**********

**********

**********

**********

**********

**********
```

Hãy xem xét một biến thể khác. Trong đoạn mã trên, vòng lặp for bên trong luôn thực hiện chính xác cùng một việc: in ra chính xác 10 dấu sao trên một dòng đầu ra. Nhưng điều gì sẽ xảy ra nếu chúng ta thay đổi điều kiện kiểm tra cho vòng lặp for bên trong để sử dụng biến của vòng lặp for bên ngoài (i)?

```
for (int i = 1; i <= 6; i++) {

    for (int j = 1; j <= i; j++) {

        System.out.print("*");

    }

    System.out.println();
```

```
}
```

Trong phiên bản cũ, vòng lặp bên trong luôn thực thi 10 lần, tạo ra 10 dấu sao trên mỗi dòng đầu ra. Với điều kiện kiểm tra mới (j <= i), vòng lặp bên trong sẽ thực thi số lần bằng giá trị của i trong mỗi lần lặp. Nhưng i đang thay đổi: Nó lần lượt nhận các giá trị 1, 2, 3, 4, 5, và 6. Trong lần lặp đầu tiên của vòng lặp ngoài, khi i là 1, điều kiện j <= i thực chất là đang kiểm tra j <= 1, và nó tạo ra một dòng với một dấu sao trên đó. Trong lần lặp thứ hai của vòng lặp ngoài, khi i là 2, điều kiện này đang kiểm tra j <= 2, và nó tạo ra một dòng với hai dấu sao trên đó. Trong lần lặp thứ ba của vòng lặp ngoài, khi i là 3, điều kiện này đang kiểm tra j <= 3, và nó tạo ra một dòng với ba dấu sao trên đó. Điều này tiếp tục cho đến lần lặp thứ sáu.

Nói cách khác, mã này tạo ra một hình tam giác như đầu ra:

```
*

**

***

****

*****

******
```

2.4 Quản lý Độ phức tạp

Bạn đã học về một số cấu trúc lập trình mới trong chương này, và đã đến lúc ghép các mảnh lại với nhau để giải quyết một số nhiệm vụ phức tạp. Như chúng tôi đã chỉ ra trong Chương 1,

Brian Kernighan, một trong những đồng tác giả của cuốn *The C Programming Language*, đã nói rằng "Kiểm soát độ phức tạp là cốt lõi của lập trình máy tính." Trong phần này, chúng ta sẽ xem xét một số kỹ thuật mà các nhà khoa học máy tính sử dụng để giải quyết các vấn đề phức tạp mà không bị choáng ngợp bởi độ phức tạp.

Phạm vi

Khi các chương trình trở nên dài hơn, khả năng các phần khác nhau của chương trình can thiệp vào nhau tăng lên. Java giúp chúng ta quản lý vấn đề tiềm ẩn này bằng cách thực thi các quy tắc về phạm vi.

Phạm vi

Phần của chương trình mà trong đó một khai báo cụ thể có hiệu lực. Như bạn đã thấy, khi nói đến việc khai báo các phương thức tĩnh, bạn có thể đặt chúng theo bất kỳ thứ tự nào. Phạm vi của một phương thức tĩnh là toàn bộ lớp mà nó xuất hiện. Biến thì khác. Quy tắc đơn giản là phạm vi của một khai báo biến kéo dài từ điểm mà nó được khai báo cho đến dấu ngoặc nhọn đóng bao quanh nó. Nói cách khác, hãy tìm cặp dấu ngoặc nhọn bao quanh trực tiếp khai báo biến đó. Phạm vi của biến là từ điểm mà nó được khai báo cho đến dấu ngoặc nhọn đóng.

Quy tắc phạm vi này có một số hệ quả. Hãy xem xét điều này có ý nghĩa gì đối với các phương thức khác nhau. Mỗi phương thức có tập hợp dấu ngoặc nhọn riêng của nó để chỉ định các câu lệnh sẽ được thực thi khi phương thức được gọi. Bất kỳ biến nào được khai báo bên trong dấu ngoặc nhọn của một phương thức sẽ không có sẵn bên ngoài phương thức đó. Chúng tôi gọi các biến này là biến cục bộ và gọi quá trình giới hạn phạm vi của chúng là *cục bộ hóa biến*.

Biến cục bộ

Một biến được khai báo bên trong một phương thức và chỉ có thể truy cập được trong phương thức đó.

Cục bộ hóa biến

Khai báo biến trong phạm vi nhỏ nhất (cục bộ nhất) có thể.

Nhìn chung, bạn sẽ muốn khai báo biến trong phạm vi cục bộ nhất có thể. Bạn có thể tự hỏi tại sao chúng ta lại muốn cục bộ hóa biến chỉ trong một phương thức. Tại sao không khai báo mọi thứ trong một phạm vi ngoài? Điều đó chắc chắn có vẻ đơn giản hơn, nhưng có một số nhược điểm quan trọng. Cục bộ hóa biến có thể dẫn đến một số sự trùng lặp (và có thể là nhầm lẫn) nhưng cung cấp nhiều bảo mật hơn. Để dễ hiểu hơn, hãy xem xét việc sử dụng tủ lạnh trong ký túc xá. Mỗi phòng trong ký túc xá có thể có một tủ lạnh riêng, nhưng nếu bạn ở ngoài phòng, bạn sẽ không biết liệu trong đó có tủ lạnh hay không. Nội dung của phòng bị ẩn khỏi bạn.

Các chương trình Java sử dụng biến để lưu trữ giá trị giống như sinh viên sử dụng tủ lạnh để lưu trữ bia, kem và các thứ có giá trị khác. Lần cuối chúng tôi ở trong ký túc xá, chúng tôi nhận thấy hầu hết các phòng cá nhân đều có tủ lạnh. Điều này có vẻ rất dư thừa, nhưng lý do là rõ ràng. Nếu bạn muốn đảm bảo an ninh cho một thứ gì đó, bạn đặt nó ở nơi không ai khác có thể lấy được. Bạn sẽ sử dụng các biến cục bộ trong chương trình của mình theo cách tương tự. Nếu mỗi phương thức riêng lẻ có các biến cục bộ của riêng nó, bạn sẽ không phải lo lắng về sự can thiệp có thể xảy ra từ các phần khác của chương trình.

Hãy xem xét một ví dụ đơn giản liên quan đến hai phương thức:

```
// Chương trình này không biên dịch được.



public class ScopeExample {

    public static void main(String[] args) {

        int x = 3;

        int y = 7;

        computeSum();

    }



    public static void computeSum() {

        int sum = x + y; // không hợp lệ, x/y không nằm trong phạm
vi

        System.out.println("sum = " + sum);

    }

}
```

Trong ví dụ này, phương thức main khai báo các biến cục bộ x và y và gán giá trị ban đầu cho chúng. Sau đó, nó gọi phương thức computeSum. Bên trong phương thức này, chúng ta cố gắng sử dụng giá trị của x và y để tính tổng. Tuy nhiên, vì các biến x và y là biến cục bộ của phương thức main và không có sẵn trong

phương thức computeSum, điều này không hoạt động. (Trong chương tiếp theo, chúng ta sẽ xem xét một kỹ thuật cho phép một phương thức truyền giá trị cho một phương thức khác.)

Chương trình sẽ tạo ra thông báo lỗi như sau:

```
ScopeExample.java:10: error: cannot find symbol

symbol  : variable x

location: class ScopeExample

        int sum = x + y;  // không hợp lệ, x/y không nằm trong phạm
vi

                ^



ScopeExample.java:10: error: cannot find symbol

symbol  : variable y

location: class ScopeExample

        int sum = x + y;  // không hợp lệ, x/y không nằm trong phạm
vi

                    ^
```

Điều quan trọng là phải hiểu về phạm vi khi thảo luận về các biến cục bộ của một phương thức so với phương thức khác. Phạm vi cũng có những tác động đối với những gì xảy ra bên trong một phương thức duy nhất. Bạn đã thấy rằng dấu ngoặc nhọn được

sử dụng để nhóm một loạt các câu lệnh lại với nhau. Nhưng bạn có thể có dấu ngoặc nhọn bên trong dấu ngoặc nhọn, và điều này dẫn đến một số vấn đề về phạm vi. Ví dụ, hãy xem xét đoạn mã sau:

```
for (int i = 1; i <= 5; i++) {

    int squared = i * i;

    System.out.println(i + " squared = " + squared);

}
```

Đây là một biến thể của đoạn mã mà chúng ta đã xem xét trước đó trong chương để in ra bình phương của năm số nguyên đầu tiên. Trong phiên bản này, một biến có tên là squared được sử dụng để theo dõi bình phương của biến vòng lặp for. Đoạn mã này hoạt động tốt, nhưng hãy xem xét biến thể sau:

```
for (int i = 1; i <= 5; i++) {

    int squared = i * i;

    System.out.println(i + " squared = " + squared);

}



System.out.println("Last square = " + squared); // không hợp lệ
```

Đoạn mã này tạo ra lỗi biên dịch. Biến squared được khai báo bên trong vòng lặp for. Nói cách khác, dấu ngoặc nhọn chứa nó là dấu ngoặc nhọn của vòng lặp. Nó không thể được sử dụng

ngoài phạm vi này, vì vậy khi bạn cố gắng tham chiếu đến nó ngoài vòng lặp, bạn sẽ gặp lỗi biên dịch.

Nếu vì lý do nào đó bạn cần viết mã như thế này để truy cập biến sau vòng lặp, bạn phải khai báo biến trong phạm vi ngoài trước vòng lặp:

```
int squared = 0; // khai báo bây giờ nằm trong phạm vi ngoài



for (int i = 1; i <= 5; i++) {

    squared = i * i; // thay đổi thành câu lệnh gán

    System.out.println(i + " squared = " + squared);

}



System.out.println("Last square = " + squared); // bây giờ hợp lệ
```

Có một vài trường hợp đặc biệt về phạm vi, và vòng lặp for là một trong số đó. Khi một biến được khai báo trong phần khởi tạo của vòng lặp for, phạm vi của nó chỉ là vòng lặp for đó (ba phần trong tiêu đề vòng lặp for và các câu lệnh được điều khiển bởi vòng lặp for). Điều đó có nghĩa là bạn có thể sử dụng cùng một tên biến trong nhiều vòng lặp for:

```
for (int i = 1; i <= 10; i++) {

    System.out.println(i + " squared = " + (i * i));
```

```
}



for (int i = 1; i <= 10; i++) {

    System.out.println(i + " cubed = " + (i * i * i));

}
```

Biến i được khai báo hai lần trong đoạn code trước đó, nhưng vì phạm vi của mỗi biến chỉ nằm trong vòng lặp for mà nó được khai báo, điều này không gây ra vấn đề gì. (Nó giống như có hai phòng ký túc xá, mỗi phòng có tủ lạnh riêng.) Tất nhiên, bạn không thể làm điều này với các vòng lặp lồng nhau. Ví dụ, đoạn code sau sẽ không biên dịch được:

```
for (int i = 1; i <= 5; i++) {

    for (int i = 1; i <= 10; i++) { // illegal

        System.out.println("hi there.");

    }

}
```

Khi Java gặp vòng lặp for bên trong, nó sẽ thông báo rằng biến i đã được khai báo trong phạm vi này. Bạn không thể khai báo cùng một biến hai lần trong cùng một phạm vi. Bạn phải đặt hai tên khác nhau để phân biệt chúng, giống như khi có hai người tên Carl trong cùng một gia đình, họ thường được gọi là "Carl Junior" và "Carl Senior" để tránh nhầm lẫn.

Một biến vòng lặp được sử dụng trong vòng lặp for không cần phải khai báo trong phần khởi tạo của vòng lặp. Bạn có thể tách việc khai báo biến vòng lặp for ra khỏi việc khởi tạo biến, như trong đoạn mã sau:

```
int i;

for (i = 1; i <= 5; i++) {

    System.out.println(i + " squared = " + (i * i));

}
```

Việc làm như vậy mở rộng phạm vi của biến đến cuối tập hợp dấu ngoặc nhọn bao quanh. Một lợi thế của cách tiếp cận này là nó cho phép bạn tham chiếu đến giá trị cuối cùng của biến vòng lặp sau vòng lặp. Thông thường bạn sẽ không thể làm điều này, vì phạm vi của biến vòng lặp sẽ bị giới hạn trong vòng lặp. Tuy nhiên, việc khai báo biến vòng lặp bên ngoài vòng lặp là một thói quen nguy hiểm và nó cung cấp một ví dụ điển hình về các vấn đề bạn có thể gặp phải khi không cục bộ hóa các biến. Xem xét đoạn code sau, ví dụ:

```
int i;

for (i = 1; i <= 5; i++) {

    for (i = 1; i <= 10; i++) {

        System.out.println("hi there.");

    }

}
```

Như đã đề cập trước đó, bạn không nên sử dụng cùng một biến vòng lặp khi có các vòng lặp lồng nhau. Nhưng khác với ví dụ trước, đoạn code này vẫn biên dịch được, vì ở đây việc khai báo biến nằm ngoài vòng lặp for bên ngoài. Vì vậy, thay vì nhận được thông báo lỗi hữu ích từ trình biên dịch Java, bạn sẽ nhận được một chương trình có lỗi. Bạn có thể nghĩ rằng việc đọc các vòng lặp này sẽ tạo ra 50 dòng đầu ra, nhưng thực tế nó chỉ tạo ra 10 dòng đầu ra. Vòng lặp bên trong tăng biến i cho đến khi nó trở thành 11, và điều đó làm cho vòng lặp bên ngoài kết thúc sau chỉ một lần lặp. Nó thậm chí còn tệ hơn nếu bạn đảo ngược thứ tự của các vòng lặp này:

```
int i;

for (i = 1; i <= 10; i++) {

    for (i = 1; i <= 5; i++) {

        System.out.println("hi there.");

    }

}
```

Đoạn mã này tạo ra một vòng lặp vô hạn.

Vòng lặp vô hạn

Một vòng lặp không bao giờ kết thúc. Vòng lặp này là vô hạn vì bất kể vòng lặp bên ngoài làm gì với biến i, vòng lặp bên trong luôn đặt lại giá trị của nó về 1 và lặp cho đến khi nó trở thành 6. Vòng lặp bên ngoài sau đó tăng biến này lên 7 và thấy rằng 7 nhỏ hơn hoặc bằng 10, vì vậy nó luôn quay lại vòng lặp bên trong, và một lần nữa đặt lại biến về 1 và lặp đến 6. Quá trình này tiếp tục

vô hạn. Đây là các loại vấn đề nhiễu mà bạn có thể gặp phải khi không cục bộ hóa các biến.

LỖI LẬP TRÌNH PHỔ BIẾN Tham chiếu đến Biến Vòng lặp Sai

Đoạn mã sau nhằm in ra một tam giác các dấu sao. Tuy nhiên, nó có một lỗi nhỏ khiến nó in các dấu sao vô hạn:

```
for (int i = 1; i <= 6; i++) {

    for (int j = 1; j <= i; i++) {

        System.out.print("*");

    }

    System.out.println();

}
```

Vấn đề nằm ở dòng thứ hai, trong phần cập nhật của vòng lặp for bên trong. Người lập trình dự định viết j++ nhưng thay vào đó lại vô tình viết i++. Dưới đây là bảng theo dõi mã (Table 2.6).

Bảng 2.6 Theo dõi Vòng lặp for Lồng nhau

Bước	Mã	Mô tả
Khởi tạo	int i = 1;	biến i được tạo và khởi tạo giá trị là 1

Bước	**Mã**	**Mô tả**
Khởi tạo	int j = 1;	biến j được tạo và khởi tạo giá trị là 1
Kiểm tra	j <= i	đúng vì 1 <= 1, do đó chúng ta vào vòng lặp bên trong
Thân vòng	{...}	thực hiện lệnh in với j bằng 1
Cập nhật	i++	tăng i, biến thành 2
Kiểm tra	j <= i	đúng vì 1 <= 2, do đó chúng ta vào vòng lặp bên trong
Thân vòng	{...}	thực hiện lệnh in với j bằng 1
Cập nhật	i++	tăng i, biến thành 3
...	...	...

Biến j đáng lẽ phải tăng, nhưng thay vào đó i lại tăng. Hiệu ứng của lỗi này là biến j không bao giờ được tăng trong vòng lặp bên trong, và do đó việc kiểm tra j <= i không bao giờ sai, vì vậy vòng lặp bên trong không kết thúc.

Dưới đây là một đoạn mã lỗi khác. Đoạn mã này cố gắng in ra một hộp dấu sao 6×4, nhưng nó cũng in vô hạn:

```
for (int i = 1; i <= 6; i++) {
```

```
    for (int j = 1; i <= 4; j++) {

        System.out.print("*");

    }

    System.out.println();

}
```

Vấn đề nằm ở dòng thứ hai, lần này là trong phần kiểm tra của vòng lặp for bên trong. Người lập trình dự định viết j <= 4 nhưng thay vào đó lại vô tình viết i <= 4. Vì giá trị của i không bao giờ tăng trong vòng lặp bên trong, việc kiểm tra i <= 4 không bao giờ sai, vì vậy vòng lặp bên trong một lần nữa không kết thúc.

Pseudocode

Khi bạn viết các thuật toán phức tạp hơn, bạn sẽ thấy rằng không thể viết ngay toàn bộ thuật toán. Thay vào đó, bạn sẽ dần dần sử dụng kỹ thuật viết mã giả (pseudocode).

Mã giả (Pseudocode)

Mô tả thuật toán giống tiếng Anh. Lập trình với mã giả liên quan đến việc liên tục tinh chỉnh một mô tả không chính thức cho đến khi nó dễ dàng chuyển đổi thành mã Java. Ví dụ, bạn có thể mô tả vấn đề vẽ một hộp như sau:

```
Vẽ một hộp với 50 dòng và 30 cột dấu hoa thị.
```

Mặc dù câu lệnh này mô tả hình dạng, nhưng nó không đưa ra chỉ dẫn cụ thể về cách vẽ nó (tức là thuật toán nào để sử dụng). Bạn sẽ vẽ hình dạng dòng theo dòng hay cột theo cột? Trong

Java, các hình dạng như thế này phải được tạo ra dòng theo dòng, vì khi một dòng đã được in ra, bạn không thể thay đổi nó. Không có lệnh nào để quay lại một dòng trước đó trong đầu ra. Do đó, bạn phải in toàn bộ dòng đầu tiên, sau đó là dòng thứ hai, v.v. Vì vậy, phân chia công việc của bạn cho các hình dạng như thế này sẽ theo hướng dòng. Do đó, một phiên bản của câu lệnh gần hơn với Java là:

```
for (mỗi dòng trong 50 dòng) {

    vẽ một dòng với 30 dấu hoa thị.

}
```

Câu lệnh này có thể cụ thể hơn bằng cách giới thiệu ý tưởng về việc viết lặp đi lặp lại một ký tự duy nhất trên dòng đầu ra và sau đó chuyển sang dòng mới:

```
for (mỗi dòng trong 50 dòng) {

    for (mỗi cột trong 30 cột) {

        viết một dấu hoa thị trên dòng đầu ra.

    }

    chuyển sang dòng đầu ra mới.

}
```

Sử dụng mã giả, bạn có thể dần dần chuyển đổi một mô tả tiếng Anh thành một cái gì đó dễ dàng chuyển đổi thành chương trình Java. Những ví dụ đơn giản mà chúng ta đã xem qua không đáng

để áp dụng mã giả, vì vậy bây giờ chúng ta sẽ xem xét vấn đề tạo ra một hình phức tạp hơn:

```
*********
 *******
  *****
   ***
    *
```

Hình này cũng phải được tạo ra dòng theo dòng:

```
for (mỗi dòng trong 5 dòng) {
    vẽ một dòng của tam giác.
}
```

Không may, mỗi dòng lại khác nhau. Do đó, bạn phải nghĩ ra một quy tắc chung phù hợp cho tất cả các dòng. Dòng đầu tiên của hình này có một chuỗi dấu hoa thị mà không có khoảng trắng ở đầu. Mỗi dòng tiếp theo có một chuỗi khoảng trắng theo sau bởi một chuỗi dấu hoa thị. Với một chút tưởng tượng, bạn có thể nói rằng dòng đầu tiên có 0 khoảng trắng theo sau bởi một chuỗi dấu hoa thị. Điều này cho phép bạn viết một quy tắc chung để tạo ra hình này:

```
for (mỗi dòng trong 5 dòng) {
    viết một số khoảng trắng (có thể là 0) trên dòng đầu ra.
```

```
    viết một số dấu hoa thị trên dòng đầu ra.

    chuyển sang dòng đầu ra mới.

}
```

Để tiến hành, bạn phải xác định một quy tắc cho số lượng khoảng trắng và một quy tắc cho số lượng dấu hoa thị. Bạn muốn tìm mối quan hệ giữa số dòng và hai giá trị này. Đây là toán học đơn giản, vì các giá trị này liên quan một cách tuyến tính. Bạn thường cần tìm biểu thức cho những mối quan hệ tuyến tính này, vì vậy đáng để khám phá kỹ thuật chung trước khi hoàn thành mã giả này.

Kỹ thuật lập bảng

Bạn có thể nhớ từ lớp toán rằng các mối quan hệ tuyến tính được biểu diễn bằng cách sử dụng độ dốc và điểm giao nhau. Với mục đích của chúng ta, sẽ dễ dàng hơn nếu nghĩ về hai giá trị này như một hệ số nhân và một hằng số. Ví dụ, xem xét các cột giá trị trong Bảng 2.7, đại diện cho số lượng ký tự trên mỗi dòng cho một hình giả định.

Bảng 2.7 Số lượng ký tự

Dòng	Giá trị 1	Giá trị 2	Giá trị 3	Giá trị 4
1	6	3	10	4
2	10	6	8	3
3	14	9	6	2

Dòng	Giá trị 1	Giá trị 2	Giá trị 3	Giá trị 4
4	18	12	4	1
5	22	15	2	0

Đối với các chương trình vẽ hình trong chương này, bạn thường có một vòng lặp bên ngoài tạo ra đầu ra dòng theo dòng. Do đó, bạn thường có một biến gọi là dòng (line) nhận các giá trị tuần tự bắt đầu từ 1, như trong cột đầu tiên của bảng. Nhiệm vụ của bạn là đưa ra các biểu thức dự đoán các giá trị trong các cột khác. Có hai bước để xác định biểu thức thích hợp cho một cột giá trị:

- Đầu tiên, chọn hệ số nhân thích hợp bằng cách xem các giá trị tăng hoặc giảm bao nhiêu từ hàng này sang hàng khác trong cột đó.

- Sau đó, chọn hằng số thích hợp để thêm vào biểu thức, để tạo ra chuỗi giá trị đúng.

Nhìn vào cột có nhãn giá trị 1. Lưu ý rằng các giá trị tăng thêm 4 mỗi lần khi bạn di chuyển từ một hàng sang hàng khác. Nếu bạn định sử dụng biến dòng, biến tăng thêm 1 mỗi lần, để tạo ra các số tăng thêm 4 mỗi lần, bạn sẽ cần nhân nó với 4. Vì vậy, áp dụng bước đầu tiên, bạn có biểu thức (4 * dòng). Nhưng biểu thức này không đủ. Khi dòng là 1, (4 * dòng) là 4 khi bạn muốn giá trị là 6. Bạn sửa điều này ở bước thứ hai. Bạn cần thêm giá trị nào vào 4 để có được giá trị 6? Câu trả lời là 2. Vì vậy, biểu thức chính xác để sử dụng là (4 * dòng + 2). Bạn có thể cắm các giá trị khác nhau của dòng để xác minh rằng biểu thức này đánh giá được 6 khi dòng là 1, 10 khi dòng là 2, và cứ thế.

Nếu bạn nhìn vào cột có nhãn giá trị 2, bạn sẽ thấy rằng các số tăng thêm 3 mỗi lần, có nghĩa là bạn muốn một hệ số nhân là 3. Điều đó dẫn đến biểu thức (3 * dòng). Trong bước thứ hai, bạn cần chọn hằng số thích hợp để thêm vào biểu thức này để làm cho nó hoạt động, nhưng biểu thức này đã hoạt động, tạo ra 3 khi dòng là 1, 6 khi dòng là 2, và cứ thế. Đây là một trường hợp đặc biệt khi hằng số đơn giản là 0 vì không cần điều chỉnh nào sau khi bạn đã chọn hệ số nhân thích hợp.

Các giá trị không phải lúc nào cũng tăng lên. Trong cột có nhãn giá trị 3, bạn sẽ nhận thấy rằng các giá trị giảm 2 mỗi lần, có nghĩa là bạn muốn một hệ số nhân là -2. Điều đó dẫn đến biểu thức (-2 * dòng). Khi dòng là 1, biểu thức này sẽ đánh giá là -2 khi bạn muốn giá trị là 10. Điều đó có nghĩa là bạn cần thêm 12 vào biểu thức, dẫn đến (-2 * dòng + 12). Nhiều lập trình viên sẽ sắp xếp lại điều này thành (12 - 2 * dòng), nhưng bất kỳ biểu thức nào cũng hoạt động tốt.

Các giá trị trong cột có nhãn giá trị 4 giảm đi 1 mỗi lần, có nghĩa là bạn cần một hệ số nhân là -1. Điều đó dẫn đến biểu thức (-1 * dòng). Biểu thức này đánh giá là -1 khi dòng là 1, trong khi bạn muốn giá trị là 4. Điều đó có nghĩa là bạn cần thêm 5 vào biểu thức, dẫn đến (-1 * dòng + 5). Một lần nữa, hầu hết các lập trình viên sẽ sắp xếp lại để đơn giản hóa điều này thành (5 - dòng), nhưng bất kỳ phiên bản nào của biểu thức cũng hoạt động.

Để áp dụng kỹ thuật này cho bài toán vẽ hình nón mà chúng ta đã thực hiện trong phần trước, trước tiên chúng ta tạo một bảng có các cột cho số khoảng trắng và dấu hoa thị trên mỗi dòng của đầu ra. Những giá trị này được bao gồm trong Bảng 2.8.

Bảng 2.8 Phân tích hình

Dòng	Khoảng trắng	Dấu hoa thị
1	0	9
2	1	7
3	2	5
4	3	3
5	4	1

Lưu ý rằng các giá trị trong cột có nhãn khoảng trắng tăng thêm 1 mỗi lần, có nghĩa là bạn muốn một hệ số nhân là 1. Nhưng bạn muốn có 0 khoảng trắng khi dòng là 1, có nghĩa là bạn cần thêm giá trị -1 vào biểu thức. Điều đó tạo ra biểu thức (1 * dòng - 1), hoặc đơn giản là (dòng - 1).

Các giá trị trong cột có nhãn dấu hoa thị giảm đi 2 mỗi lần, có nghĩa là chúng ta muốn một hệ số nhân là -2. Điều đó sẽ tạo ra -2 khi dòng là 1, trong khi chúng ta muốn có giá trị 9, có nghĩa là chúng ta cần thêm một hằng số là 11. Điều đó tạo ra biểu thức (-2 * dòng + 11) hoặc (11 - 2 * dòng).

Sử dụng các biểu thức này, chúng ta có thể cải thiện mã giả của mình như sau:

```
for (dòng từ 1 đến 5) {

    viết (dòng − 1) khoảng trắng trên dòng đầu ra.

    viết (11 − 2 * dòng) dấu hoa thị trên dòng đầu ra.
```

```
    chuyển sang dòng đầu ra mới.
}
```

Mã giả này đơn giản để chuyển đổi thành một chương trình:

```
1  // Sử dụng vòng lặp lồng nhau để vẽ một hình chữ 'V'.
 2  public class DrawV {
 3      public static void main(String[] args) {
 4          for (int line = 1; line <= 5; line++) {
 5              for (int i = 1; i <= (line − 1); i++) {
 6                  System.out.print(" ");
 7              }
 8              for (int i = 1; i <= (11 − 2 * line); i++) {
 9                  System.out.print("*");
10              }
11              System.out.println();
12          }
13      }
14  }
```

Đôi khi chúng ta quản lý sự phức tạp bằng cách tận dụng công việc mà chúng ta đã hoàn thành. Ví dụ, bạn sẽ tạo ra hình này như thế nào?

```
   *
  ***
 *****
*******
```

Bạn có thể theo quy trình giống như trước đây và tìm các biểu thức mới để tạo ra số lượng khoảng trắng và dấu hoa thị phù hợp. Tuy nhiên, có một cách dễ dàng hơn. Hình này giống như hình trước đó, ngoại trừ các dòng xuất hiện theo thứ tự ngược lại. Đây là một nơi tốt để sử dụng vòng lặp giảm dần để chạy vòng lặp for ngược lại: Thay vì bắt đầu từ 1 và tăng lên đến 5 với cập nhật ++, bạn có thể bắt đầu từ 5 và giảm xuống 1 bằng cách sử dụng cập nhật −−.

Cách đơn giản để tạo ra tam giác hướng lên trên là với đoạn mã sau:

```
1  // Sử dụng vòng lặp lồng nhau để vẽ một hình nón tam giác.
2  public class DrawCone {
3      public static void main(String[] args) {
4          for (int line = 5; line >= 1; line--) {
```

```
5             for (int i = 1; i <= (line - 1); i++) {
6                 System.out.print(" ");
7             }
8             for (int i = 1; i <= (11 - 2 * line); i++) {
9                 System.out.print("*");
10             }
11             System.out.println();
12         }
13     }
14 }
```

Hằng số lớp

Chương trình DrawCone trong phần trước vẽ một hình nón với năm dòng. Làm thế nào bạn có thể chỉnh sửa nó để tạo ra một hình nón với ba dòng? Suy nghĩ đầu tiên của bạn có thể là chỉ cần thay đổi số 5 trong mã thành 3. Tuy nhiên, điều đó sẽ khiến chương trình tạo ra đầu ra sau:

```
  *****

 *******

*********
```

Điều này rõ ràng là sai. Nếu bạn nghiên cứu hình học của hình vẽ, bạn sẽ thấy rằng vấn đề là việc sử dụng số 11 trong biểu thức tính toán số lượng dấu hoa thị cần in. Số 11 xuất phát từ công thức này:

```
2 * (số dòng) + 1
```

Vì vậy, khi số dòng là năm, giá trị phù hợp là 11, nhưng khi số dòng là ba, giá trị phù hợp là 7. Các lập trình viên gọi những con số như thế này là **số ma thuật**. Chúng là "ma thuật" theo nghĩa chúng dường như làm cho chương trình hoạt động, nhưng định nghĩa của chúng không phải lúc nào cũng rõ ràng. Nhìn vào chương trình DrawCone, bạn có thể tự hỏi, “Tại sao là 5? Tại sao là 11? Tại sao là 3? Tại sao là 7? Tại sao lại là tôi?”

Để làm cho các chương trình dễ đọc hơn và dễ thay đổi hơn, bạn nên tránh sử dụng số ma thuật bất cứ khi nào có thể. Bạn có thể làm điều đó bằng cách lưu trữ các số ma thuật này. Bạn có thể sử dụng biến để lưu trữ các giá trị này, nhưng điều đó có thể gây hiểu lầm, vì bạn đang cố gắng biểu diễn các giá trị không thay đổi. May mắn thay, Java cung cấp một giải pháp thay thế: Bạn có thể khai báo các giá trị tương tự như biến nhưng được đảm bảo có giá trị không đổi. Không ngạc nhiên khi chúng được gọi là **hằng số**. Chúng ta thường định nghĩa **hằng số lớp**, có thể truy cập từ mọi nơi trong lớp.

Hằng số lớp

Là một giá trị có tên không thể thay đổi. Hằng số lớp có thể truy cập từ bất kỳ đâu trong lớp (tức là phạm vi của nó là toàn bộ lớp).

Bạn có thể chọn một tên mô tả cho hằng số để giải thích nó đại diện cho điều gì. Bạn sau đó có thể sử dụng tên đó thay vì gọi giá

trị cụ thể để làm cho chương trình của bạn dễ đọc hơn và dễ thay đổi hơn. Ví dụ, trong chương trình DrawCone, bạn có thể muốn giới thiệu một hằng số gọi là LINES đại diện cho số dòng (nhớ rằng trong Chương 1 chúng ta sử dụng tất cả các chữ cái viết hoa cho tên hằng số). Bạn có thể sử dụng hằng số này thay thế cho số ma thuật 5 và như một phần của biểu thức để tính toán một giá trị. Cách tiếp cận này cho phép bạn thay thế số ma thuật 11 bằng công thức từ đó nó được dẫn xuất (2 * LINES + 1).

Hằng số được khai báo với từ khóa final, cho biết giá trị của chúng không thể thay đổi sau khi được gán, như trong:

```
final int LINES = 5;
```

Bạn có thể khai báo một hằng số ở bất kỳ đâu bạn có thể khai báo một biến, nhưng vì hằng số thường được sử dụng bởi nhiều phương thức khác nhau, chúng ta thường khai báo chúng ngoài các phương thức. Điều này lại khiến chúng ta gặp lại từ khóa static. Nếu bạn muốn các phương thức static của mình có thể truy cập các hằng số, chính các hằng số cũng phải là static. Tương tự như vậy, cũng như chúng ta khai báo các phương thức của mình là public, chúng ta thường khai báo các hằng số của mình là public. Sau đây là cú pháp chung cho việc định nghĩa hằng số:

```
public static final <type> <name> = <expression>;
```

Ví dụ, đây là định nghĩa cho hai hằng số:

```
public static final int HEIGHT = 10;

public static final int WIDTH = 20;
```

Những định nghĩa này tạo ra các hằng số gọi là HEIGHT và WIDTH sẽ luôn có giá trị lần lượt là 10 và 20. Đây được gọi là **hằng số lớp**, vì chúng được khai báo trong phạm vi bên ngoài nhất của lớp, cùng với các phương thức của lớp. Theo cách đó, chúng có thể nhìn thấy trong mỗi phương thức của lớp.

Chúng ta đã đề cập rằng chúng ta có thể tránh sử dụng một số ma thuật trong chương trình DrawCone bằng cách giới thiệu một hằng số cho số dòng. Đây là cách định nghĩa hằng số đó:

```
public static final int LINES = 5;
```

Bây giờ chúng ta có thể thay thế số 5 trong vòng lặp ngoài cùng bằng hằng số này và thay thế số 11 trong vòng lặp lồng trong thứ hai bằng biểu thức 2 * LINES + 1. Kết quả là chương trình sau:

```
1  // Sử dụng vòng lặp lồng nhau để vẽ một hình nón tam giác.
2  // Phiên bản này sử dụng hằng số lớp.
3  public class DrawCone2 {
4      public static final int LINES = 5;
5      public static void main(String[] args) {
6          for (int line = LINES; line >= 1; line--) {
7              for (int i = 1; i <= (line - 1); i++) {
8                  System.out.print(" ");
9              }
```

```
10              int stars = 2 * LINES + 1 - 2 * line;
11              for (int i = 1; i <= stars; i++) {
12                  System.out.print("*");
13              }
14              System.out.println();
15          }
16      }
17  }
```

Lưu ý rằng trong chương trình này, biểu thức cho số lượng dấu hoa thị đã trở nên đủ phức tạp để chúng ta giới thiệu một biến cục bộ gọi là stars để lưu trữ giá trị. Ưu điểm của chương trình này là nó dễ đọc hơn và dễ thay đổi hơn. Một thay đổi đơn giản đối với hằng số LINES sẽ làm cho nó tạo ra một hình có số dòng khác nhau.

2.5 Nghiên cứu tình huống: Hình Đồng Hồ Cát

Bây giờ chúng ta sẽ xem xét một ví dụ phức tạp hơn. Để giải quyết nó, chúng ta sẽ tuân theo ba bước cơ bản:

1. Phân chia nhiệm vụ thành các công việc con, mỗi công việc sẽ trở thành một phương thức tĩnh (static).

2. Đối với mỗi công việc con, tạo bảng cho hình vẽ và tính toán các công thức cho từng cột của bảng dựa trên số dòng.

3. Chuyển đổi các bảng thành mã vòng lặp for thực tế cho mỗi phương thức.

Đầu ra mà chúng ta muốn tạo ra như sau:

```
+------+

|\..../|

| \../ |

|  \/  |

|  /\  |

| /..\ |

|/....\|

+------+
```

Phân Tích Vấn Đề và Mã Giả

Để tạo ra hình vẽ này, trước tiên bạn phải chia nó thành các hình con. Khi làm như vậy, bạn nên tìm các dòng có sự tương đồng theo cách này hay cách khác. Dòng đầu tiên và dòng cuối cùng giống hệt nhau. Ba dòng sau dòng đầu tiên đều phù hợp với một mẫu, và ba dòng tiếp theo phù hợp với một mẫu khác:

```
+------+

      dòng đầu tiên

|\..../|
```

```
| \../ |
      nửa trên
|  \/  |
|  /\  |
| /..\ |
      nửa dưới
|/....\|
+------+
      dòng cuối cùng
```

Vì vậy, bạn có thể phân chia vấn đề tổng thể như sau:

- Vẽ một dòng liền.
- Vẽ nửa trên của hình đồng hồ cát.
- Vẽ nửa dưới của hình đồng hồ cát.
- Vẽ một dòng liền.

Bạn nên giải quyết từng vấn đề con một cách độc lập. Cuối cùng, bạn sẽ muốn tích hợp một hằng số lớp để làm cho chương trình linh hoạt hơn, nhưng trước tiên hãy giải quyết vấn đề mà không cần lo lắng về việc sử dụng hằng số.

Nhiệm vụ vẽ dòng liền có thể được xác định cụ thể hơn là:

- Viết một dấu cộng trên dòng đầu ra.
- Viết 6 dấu gạch ngang trên dòng đầu ra.
- Viết một dấu cộng trên dòng đầu ra.
- Chuyển sang một dòng đầu ra mới.

Bộ hướng dẫn này dễ dàng chuyển thành một phương thức tĩnh:

```
public static void drawLine() {
    System.out.print("+");
    for (int i = 1; i <= 6; i++) {
        System.out.print("-");
    }
    System.out.println("+");
}
```

Nửa trên của hình đồng hồ cát phức tạp hơn. Đây là một dòng điển hình:

```
| \../ |
```

Có bốn ký tự riêng biệt, được phân tách bởi các khoảng trắng và dấu chấm.

Vì vậy, một bản phác thảo ban đầu trong mã giả có thể trông như thế này:

```
for (mỗi dòng trong 3 dòng) {
    viết một thanh dọc trên dòng đầu ra;
    viết một số khoảng trắng trên dòng đầu ra;
    viết một dấu gạch chéo ngược trên dòng đầu ra;
    viết một số dấu chấm trên dòng đầu ra;
    viết một dấu gạch chéo trên dòng đầu ra;
    viết một số khoảng trắng trên dòng đầu ra;
    viết một thanh dọc trên dòng đầu ra;
    chuyển sang một dòng đầu ra mới;
}
```

Một lần nữa, bạn có thể tạo một bảng để xác định các biểu thức cần thiết. Việc viết các ký tự riêng lẻ sẽ dễ dàng chuyển thành Java, nhưng bạn cần cụ thể hơn về các khoảng trắng và dấu chấm. Mỗi dòng trong nhóm này chứa hai bộ khoảng trắng và một bộ dấu chấm. Bảng 2.9 cho thấy số lượng cần sử dụng.

Bảng 2.9 Phân Tích Hình Vẽ

Dòng	Khoảng trắng	Dấu chấm	Khoảng trắng
1	0	4	0

Dòng	Khoảng trắng	Dấu chấm	Khoảng trắng
2	1	2	1
3	2	0	2

Hai bộ khoảng trắng tuân theo quy tắc (dòng - 1), và số lượng dấu chấm là (6 - 2 * dòng). Do đó, mã giả nên được viết như sau:

```
for (dòng từ 1 đến 3) {
    viết một thanh dọc trên dòng đầu ra;
    viết `(dòng - 1)` khoảng trắng trên dòng đầu ra;
    viết một dấu gạch chéo ngược trên dòng đầu ra;
    viết `(6 - 2 * dòng)` dấu chấm trên dòng đầu ra;
    viết một dấu gạch chéo trên dòng đầu ra;
    viết `(dòng - 1)` khoảng trắng trên dòng đầu ra;
    viết một thanh dọc trên dòng đầu ra;
    chuyển sang một dòng đầu ra mới;
}
```

Phiên bản có cấu trúc ban đầu

Mã giả cho nửa trên của hình đồng hồ cát dễ dàng được dịch thành một phương thức tĩnh có tên là drawTop. Một giải pháp

tương tự cũng tồn tại cho nửa dưới của hình đồng hồ cát. Khi kết hợp lại, chương trình trông như sau:

```
1  // Vẽ hình đồng hồ cát sử dụng các vòng lặp lồng nhau.
2  public class Hourglass {
3      public static void main(String[] args) {
4          drawLine();
5          drawTop();
6          drawBottom();
7          drawLine();
8      }
9
10     // Tạo ra một dòng liền
11     public static void drawLine() {
12         System.out.print("+");
13         for (int i = 1; i <= 6; i++) {
14             System.out.print("−");
15         }
16         System.out.println("+");
```

```
17    }
18
19    // Tạo ra nửa trên của hình đồng hồ cát
20    public static void drawTop() {
21        for (int line = 1; line <= 3; line++) {
22            System.out.print("|");
23            for (int i = 1; i <= (line − 1); i++) {
24                System.out.print(" ");
25            }
26            System.out.print("\\");
27            for (int i = 1; i <= (6 − 2 * line); i++) {
28                System.out.print(".");
29            }
30            System.out.print("/");
31            for (int i = 1; i <= (line − 1); i++) {
32                System.out.print(" ");
33            }
```

```
34            System.out.println("|");
35        }
36    }
37
38    // Tạo ra nửa dưới của hình đồng hồ cát
39    public static void drawBottom() {
40        for (int line = 1; line <= 3; line++) {
41            System.out.print("|");
42            for (int i = 1; i <= (3 − line); i++) {
43                System.out.print(" ");
44            }
45            System.out.print("/");
46            for (int i = 1; i <= 2 * (line − 1); i++) {
47                System.out.print(".");
48            }
49            System.out.print("\\");
50            for (int i = 1; i <= (3 − line); i++) {
```

```
51                System.out.print(" ");

52            }

53            System.out.println("|");

54        }

55    }

56 }
```

Thêm một Hằng Số Lớp

Chương trình Hourglass tạo ra đầu ra mong muốn, nhưng nó không linh hoạt lắm. Điều gì sẽ xảy ra nếu chúng ta muốn tạo ra một hình tương tự có kích thước khác? Vấn đề ban đầu liên quan đến một hình đồng hồ cát có ba dòng ở nửa trên và ba dòng ở nửa dưới. Điều gì sẽ xảy ra nếu chúng ta muốn có đầu ra như sau, với bốn dòng ở nửa trên và bốn dòng ở nửa dưới?

```
+--------+
|\....../|
| \..../ |
|  \../  |
|   \/   |
|   /\   |
|  /..\  |
| /....\ |
|/......\|
+--------+
```

Rõ ràng là chương trình sẽ hữu ích hơn nếu chúng ta có thể làm cho nó linh hoạt đủ để tạo ra bất kỳ đầu ra nào. Chúng ta làm

điều đó bằng cách loại bỏ các con số ma thuật với việc giới thiệu một hằng số lớp. Bạn có thể nghĩ rằng chúng ta cần giới thiệu hai hằng số—một cho chiều cao và một cho chiều rộng—nhưng do tính đều đặn của hình này, chiều cao được xác định bởi chiều rộng và ngược lại. Do đó, chúng ta chỉ cần giới thiệu một hằng số lớp duy nhất. Hãy sử dụng chiều cao của các nửa đồng hồ cát:

```
public static final int SUB_HEIGHT = 4;
```

Chúng ta đã đặt tên hằng số là SUB_HEIGHT thay vì HEIGHT vì nó đề cập đến chiều cao của mỗi nửa, thay vì toàn bộ hình. Chú ý cách chúng ta sử dụng ký tự gạch dưới để phân tách các từ khác nhau trong tên của hằng số.

Vậy, làm thế nào để chúng ta sửa đổi chương trình gốc để tích hợp hằng số này? Chúng ta sẽ xem xét chương trình để tìm bất kỳ con số ma thuật nào và chèn hằng số hoặc một biểu thức liên quan đến hằng số ở nơi phù hợp. Ví dụ, cả hai phương thức drawTop và drawBottom đều có một vòng lặp for thực hiện 3 lần để tạo ra 3 dòng đầu ra. Chúng ta thay đổi nó thành 4 để tạo ra 4 dòng đầu ra, và tổng quát hơn, chúng ta thay đổi nó thành SUB_HEIGHT để tạo ra số dòng đầu ra tương ứng với SUB_HEIGHT.

Ở các phần khác của chương trình, chúng ta phải cập nhật các công thức của mình cho số dấu gạch ngang, khoảng trắng và dấu chấm. Đôi khi chúng ta có thể sử dụng các phỏng đoán có giáo dục để tìm ra cách điều chỉnh công thức đó để sử dụng hằng số. Nếu bạn không thể đoán được công thức phù hợp, bạn có thể sử dụng kỹ thuật bảng để tìm ra công thức thích hợp. Sử dụng đầu ra mới này với chiều cao phụ là 4, bạn có thể cập nhật các công thức khác nhau trong chương trình. Bảng 2.10 cho thấy các công thức khác nhau.

Bảng 2.10 Phân tích các hình có chiều cao khác nhau

Chiều cao phụ	Gạch ngang trong drawLine	Khoảng trống trong drawTop	Dấu chấm trong drawTop	Khoảng trống trong drawBottom	Dấu chấm trong drawBottom
3	6	line − 1	6 − 2 * line	3 − line	2 * (line − 1)
4	8	line − 1	8 − 2 * line	4 − line	2 * (line − 1)

Sau đó, chúng ta sẽ xem qua từng công thức và tìm ra cách thay thế nó bằng một công thức mới liên quan đến hằng số. Số lượng dấu gạch ngang tăng thêm 2 khi chiều cao phụ tăng thêm 1, vì vậy chúng ta cần một hệ số nhân là 2. Biểu thức 2 * SUB_HEIGHT tạo ra các giá trị chính xác. Số lượng khoảng trắng trong drawTop không thay đổi theo chiều cao phụ, vì vậy biểu thức không cần phải thay đổi. Số lượng dấu chấm trong drawTop liên quan đến số 6 cho chiều cao phụ là 3 và số 8 cho chiều cao phụ là 4. Một lần nữa, chúng ta cần một hệ số nhân là 2, vì vậy chúng ta sử dụng biểu thức 2 * SUB_HEIGHT − 2 * line. Số lượng khoảng trắng trong drawBottom liên quan đến giá trị 3 cho chiều cao phụ là 3 và giá trị 4 cho chiều cao phụ là 4, vì vậy biểu thức tổng quát là SUB_HEIGHT − line. Số lượng dấu chấm trong drawBottom không thay đổi khi chiều cao phụ thay đổi.

Đây là phiên bản mới của chương trình với một hằng số lớp cho chiều cao phụ. Nó sử dụng giá trị SUB_HEIGHT là 4, nhưng chúng ta có thể thay đổi nó thành 3 để tạo ra phiên bản nhỏ hơn hoặc thành một giá trị khác để tạo ra một phiên bản khác của hình.

```
1  // Vẽ hình đồng hồ cát sử dụng các vòng lặp lồng nhau.
2  // Phiên bản thứ hai sử dụng một hằng số lớp.
3  public class Hourglass2 {
4      public static final int SUB_HEIGHT = 4;
5      public static void main(String[] args) {
```

```
6           drawLine();
7           drawTop();
8           drawBottom();
9           drawLine();
10      }
11      // Tạo ra một dòng liền
12      public static void drawLine() {
13          System.out.print("+");
14          for (int i = 1; i <= (2 * SUB_HEIGHT); i++) {
15              System.out.print("−");
16          }
17          System.out.println("+");
18      }
19      // Tạo ra nửa trên của hình đồng hồ cát
20      public static void drawTop() {
21          for (int line = 1; line <= SUB_HEIGHT; line++) {
22              System.out.print("|");
23              for (int i = 1; i <= (line − 1); i++) {
24                  System.out.print(" ");
25              }
26              System.out.print("\\");
27              int dots = 2 * SUB_HEIGHT − 2 * line;
28              for (int i = 1; i <= dots; i++) {
29                  System.out.print(".");
30              }
31              System.out.print("/");
32              for (int i = 1; i <= (line − 1); i++) {
33                  System.out.print(" ");
34              }
35              System.out.println("|");
36          }
37      }
38      // Tạo ra nửa dưới của hình đồng hồ cát
```

```
39    public static void drawBottom() {
40        for (int line = 1; line <= SUB_HEIGHT; line++) {
41            System.out.print("|");
42            for (int i = 1; i <= (SUB_HEIGHT − line); i++) {
43                System.out.print(" ");
44            }
45            System.out.print("/");
46            for (int i = 1; i <= 2 * (line − 1); i++) {
47                System.out.print(".");
48            }
49            System.out.print("\\");
50            for (int i = 1; i <= (SUB_HEIGHT − line); i++) {
51                System.out.print(" ");
52            }
53            System.out.println("|");
54        }
55    }
56 }
```

Chú ý rằng hằng số SUB_HEIGHT được khai báo với phạm vi toàn bộ lớp, thay vì cục bộ trong các phương thức riêng lẻ. Mặc dù việc cục bộ hóa biến là một ý tưởng hay, nhưng điều đó không đúng với các hằng số. Chúng ta cục bộ hóa biến để tránh sự can thiệp tiềm ẩn, nhưng lập luận đó không áp dụng cho hằng số, vì chúng được đảm bảo không thay đổi. Một lập luận khác cho việc sử dụng biến cục bộ là nó làm cho các phương thức tĩnh độc lập hơn. Lập luận này có giá trị nhất định khi áp dụng cho hằng số, nhưng không đủ mạnh. Đúng là hằng số lớp giới thiệu sự phụ thuộc giữa các phương thức, nhưng đôi khi đó chính là điều bạn muốn. Ví dụ, ba phương thức trong Hourglass2 không nên độc lập với nhau về kích thước của hình. Mỗi phần hình phụ phải sử dụng cùng một hằng số kích thước. Hãy tưởng tượng thảm họa tiềm ẩn nếu mỗi phương thức có SUB_HEIGHT riêng, mỗi

phương thức có giá trị khác nhau—không phần nào sẽ khớp nhau.

Các Biến Thể Khác

Giải pháp mà chúng ta đã đạt được có thể trông phức tạp, nhưng nó dễ dàng thích ứng hơn với một nhiệm vụ mới so với chương trình ban đầu của chúng ta. Ví dụ, giả sử rằng bạn muốn tạo ra đầu ra sau:

```
+----------+
|\......../|
| \....../ |
|  \..../  |
|   \../   |
|    \/    |
|    /\    |
|   /..\   |
|  /....\  |
| /......\ |
|/........\|
+----------+
|    /\    |
|   /..\   |
|  /....\  |
| /......\ |
|/........\|
|\......../|
| \....../ |
|  \..../  |
|   \../   |
|    \/    |
+----------+
```

Đầu ra này sử dụng chiều cao phụ là 5 và bao gồm cả mô hình kim cương và mô hình X. Bạn có thể tạo ra đầu ra này bằng cách thay đổi hằng số SUB_HEIGHT thành 5:

```
public static final int SUB_HEIGHT = 5;
```

và viết lại phương thức main như sau để tạo ra cả mô hình X gốc và mô hình kim cương mới, mà bạn có thể có chỉ bằng cách đảo ngược thứ tự của các lệnh gọi trên hai nửa:

```
public static void main(String[] args) {
    drawLine();
    drawTop();
    drawBottom();
    drawLine();
    drawBottom();
    drawTop();
    drawLine();
}
```

Tóm tắt Chương

Java nhóm dữ liệu thành các kiểu. Có hai loại dữ liệu chính: dữ liệu nguyên thủy và đối tượng. Các kiểu nguyên thủy bao gồm int (số nguyên), double (số thực), char (ký tự văn bản riêng lẻ), và boolean (giá trị logic).

Các giá trị và phép toán được gọi là biểu thức. Các biểu thức đơn giản nhất là các giá trị riêng lẻ, còn được gọi là hằng số. Một số ví dụ về hằng số là: 42, 3.14, 'Q', và false. Các biểu thức có thể chứa các toán tử, như trong (3 + 29) − 4 * 5. Phép toán chia là đặc biệt vì nó được chia thành phép chia nguyên (/) và phép chia lấy phần dư (%).

Các quy tắc thứ tự ưu tiên xác định thứ tự mà các toán tử sẽ được đánh giá trong các biểu thức phức tạp. Phép nhân và chia được thực hiện trước phép cộng và trừ. Dấu ngoặc đơn có thể được sử dụng để ép buộc một thứ tự đánh giá cụ thể.

Dữ liệu có thể được chuyển đổi từ kiểu này sang kiểu khác bằng một phép toán gọi là ép kiểu.

Biến là các vị trí trong bộ nhớ nơi có thể lưu trữ các giá trị. Một biến được khai báo với một tên và một kiểu. Bất kỳ giá trị dữ liệu nào có kiểu tương thích có thể được lưu trữ trong bộ nhớ của biến và sử dụng sau này trong chương trình.
Dữ liệu nguyên thủy có thể được in trên bảng điều khiển bằng phương thức System.out.println, giống như chuỗi văn bản. Một chuỗi có thể được kết nối với một giá trị khác (nối chuỗi) bằng toán tử + để tạo ra một chuỗi lớn hơn. Tính năng này cho phép bạn in các biểu thức phức tạp bao gồm số và văn bản trên bảng điều khiển.
Một vòng lặp được sử dụng để thực thi một nhóm các câu lệnh nhiều lần. Vòng lặp for là một loại vòng lặp có thể được sử dụng để áp dụng cùng một câu lệnh qua một phạm vi số hoặc lặp lại các câu lệnh một số lần nhất định. Một vòng lặp có thể chứa một vòng lặp khác, được gọi là vòng lặp lồng nhau.
Một biến tồn tại từ dòng mà nó được khai báo đến dấu ngoặc nhọn phải bao quanh nó. Phạm vi này, còn được gọi là phạm vi của biến, tạo thành phần của chương trình mà biến có thể được sử dụng hợp pháp. Một biến được khai báo bên trong một phương thức hoặc vòng lặp được gọi là biến cục bộ. Một biến cục bộ chỉ có thể được sử dụng trong phương thức hoặc vòng lặp của nó.
Một thuật toán có thể dễ dàng viết hơn nếu bạn viết trước một mô tả bằng tiếng Anh về nó. Mô tả như vậy cũng được gọi là mã giả.
Các giá trị hằng quan trọng được viết vào một chương trình nên được khai báo như là các hằng số lớp, cả để giải thích tên và giá trị của chúng và để dễ dàng thay đổi giá trị của chúng sau này.

Bài Tập Tự Kiểm Tra
Phần 2.1: Các Khái Niệm Dữ Liệu Cơ Bản

1. Những giá trị nào sau đây là hợp lệ cho kiểu int?
    - 22, 1.5, −1, 2.3, 10.0, 5., −6875309, '7'
2. Kết quả của biểu thức sau là gì?

- 1 + 2 * 3 + 7 * 2 % 5
- a. 1
- b. 2
- c. 5
- d. 11
- e. 21

3. Dò theo các bước tính toán của các biểu thức sau và cho kết quả:
   - a. 2 + 3 * 4 − 6
   - b. 14 / 7 * 2 + 30 / 5 + 1
   - c. (12 + 3) / 4 * 2
   - d. (238 % 10 + 3) % 7
   - e. (18 − 7) * (43 % 10)
   - f. 2 + 19 % 5 − (11 * (5 / 2))
   - g. 813 % 100 / 3 + 2.4
   - h. 26 % 10 % 4 * 3
   - i. 22 + 4 * 2
   - j. 23 % 8 % 3
   - k. 12 − 2 − 3
   - l. 6 / 2 + 7 / 3
   - m. 6 * 7 % 4
   - n. 3 * 4 + 2 * 3
   - o. 177 % 100 % 10 / 2
   - p. 89 % (5 + 5) % 5
   - q. 392 / 10 % 10 / 2
   - r. 8 * 2 − 7 / 4
   - s. 37 % 20 % 3 * 4
   - t. 17 % 10 / 4
4. Dò theo các bước tính toán của các biểu thức sau và cho kết quả:
   - a. 4.0 / 2 * 9 / 2
   - b. 2.5 * 2 + 8 / 5.0 + 10 / 3
   - c. 12 / 7 * 4.4 * 2 / 4

- d. 4 * 3 / 8 + 2.5 * 2
- e. (5 * 7.0 / 2 − 2.5) / 5 * 2
- f. 41 % 7 * 3 / 5 + 5 / 2 * 2.5
- g. 10.0 / 2 / 4
- h. 8 / 5 + 13 / 2 / 3.0
- i. (2.5 + 3.5) / 2
- j. 9 / 4 * 2.0 − 5 / 4
- k. 9 / 2.0 + 7 / 3 − 3.0 / 2
- l. 813 % 100 / 3 + 2.4
- m. 27 / 2 / 2.0 * (4.3 + 1.7) − 8 / 3
- n. 53 / 5 / (0.6 + 1.4) / 2 + 13 / 2
- o. 2 * 3 / 4 * 2 / 4.0 + 4.5 − 1
- p. 89 % 10 / 4 * 2.0 / 5 + (1.5 + 1.0 / 2) * 2

5. Dò theo các bước tính toán của các biểu thức sau và cho kết quả:
   - a. 2 + 2 + 3 + 4
   - b. "2 + 2" + 3 + 4
   - c. 2 + " 2 + 3 " + 4
   - d. 3 + 4 + " 2 + 2"
   - e. "2 + 2 " + (3 + 4)
   - f. "(2 + 2) " + (3 + 4)
   - g. "hello 34 " + 2 * 4
   - h. 2 + "(int) 2.0" + 2 * 2 + 2
   - i. 4 + 1 + 9 + "." + (-3 + 10) + 11 / 3
   - j. 8 + 6 * −2 + 4 + "0" + (2 + 5)
   - k. 1 + 1 + "8 − 2" + (8 − 2) + 1 + 1
   - l. 5 + 2 + "(1 + 1)" + 4 + 2 * 3
   - m. "1" + 2 + 3 + "4" + 5 * 6 + "7" + (8 + 9)

Phần 2.2: Biến

6. Lựa chọn nào sau đây là cú pháp đúng để khai báo một biến số thực tên là grade và khởi tạo giá trị của nó là 4.0?
   - a. int grade: 4.0;

- b. grade = double 4.0;
- c. double grade = 4.0;
- d. grade = 4;
- e. 4.0 = grade;

7. Giả sử bạn đang viết một chương trình thể dục cá nhân lưu trữ tuổi, giới tính, chiều cao (tính bằng feet hoặc mét), và cân nặng (tính theo pound hoặc kilogram) của người dùng. Hãy khai báo các biến với tên và kiểu dữ liệu phù hợp để lưu trữ thông tin này.
8. Giả sử bạn đang viết một chương trình lưu trữ năm học của một sinh viên (Năm nhất, Năm hai, Năm ba, hoặc Năm tư), số lượng khóa học mà sinh viên đang theo học, và điểm GPA của họ trên thang điểm 4.0. Hãy khai báo các biến với tên và kiểu dữ liệu phù hợp để lưu trữ thông tin này.
9. Giả sử bạn có một biến int tên là number. Biểu thức Java nào tạo ra chữ số cuối cùng của số (chữ số hàng đơn vị)?
10. Chương trình sau đây chứa 9 lỗi! Các lỗi đó là gì?

```
public class Oops2 {
   public static void main(String[] args) {
      int x;
      System.out.println("x is" x);
      int x = 15.2; // set x to 15.2
      System.out.println("x is now + x");
      int y; // set y to 1 more than x
      y = int x + 1;
      System.out.println("x and y are " + x + and + y);
   }
}
```

11. Giả sử bạn có một biến int tên là number. Biểu thức Java nào tạo ra chữ số thứ hai từ cuối của số (chữ số hàng chục)? Biểu thức nào tạo ra chữ số thứ ba từ cuối của số (chữ số hàng trăm)?
12. Giá trị của biến x sau khi đoạn mã sau thực thi là gì?

```
int x = 3;
x = x + 2;
x = x + x;
```

- a. 3
- b. 5
- c. 7
- d. 10
- e. 12

13. Giá trị của a, b, và c sau các câu lệnh sau là gì?

```
int a = 5;
int b = 10;
int c = b;
a = a + 1;
b = b - 1;
c = c + a;
```

14. Giá trị của first và second sau đoạn mã sau là gì? Bạn sẽ mô tả tác dụng của các câu lệnh trong bài tập này như thế nào?

```
int first = 8;
int second = 19;
first = first + second;
second = first − second;
first = first − second;
```

15. Viết lại mã từ bài tập trước để ngắn gọn hơn, bằng cách khai báo các biến cùng nhau và sử dụng các toán tử gán đặc biệt (ví dụ: +=, −=, *=, và /=) nếu thích hợp.

16. Giá trị của i, j, và k sau các câu lệnh sau là gì?

```
int i = 2;
int j = 3;
int k = 4;
int x = i + j + k;
i = x - i - j;
j = x - j - k;
```

```
k = x - i - k;
```

17. Kết quả đầu ra từ đoạn mã sau là gì?

```
int max;
int min = 10;
max = 17 − 4 / 10;
max = max + 6;
min = max − min;
System.out.println(max * 2);
System.out.println(max + min);
System.out.println(max);
System.out.println(min);
```

18. Giả sử bạn có một biến số thực x. Viết biểu thức Java tính giá trị y sau đây chỉ sử dụng toán tử * bốn lần:

- y = 12.3x^4 − 9.1x^3 + 19.3x^2 − 4.6x + 34.2

19. Chương trình sau đây lặp lại một cách thừa thãi cùng một biểu thức nhiều lần. Chỉnh sửa chương trình để loại bỏ tất cả các biểu thức thừa thãi bằng cách sử dụng các biến với kiểu dữ liệu thích hợp.

```
public class ComputePay {
   public static void main(String[] args) {
      // Tính lương làm việc dựa trên số giờ làm việc mỗi ngày
      System.out.println("Tổng số giờ làm việc của tôi:");
      System.out.println(4 + 5 + 8 + 4);
      System.out.println("Lương theo giờ của tôi:");
      System.out.println("$8.75");
      System.out.println("Tổng tiền lương của tôi:");
      System.out.println((4 + 5 + 8 + 4) * 8.75);
      System.out.println("Thuế của tôi:");
      System.out.println((4 + 5 + 8 + 4) * 8.75 * 0.20);
   }
}
```

20. Chương trình sau đây lặp lại một cách thừa thãi cùng một biểu thức nhiều lần. Chỉnh sửa chương trình để loại bỏ

tất cả các biểu thức thừa thãi bằng cách sử dụng các biến với kiểu dữ liệu thích hợp. // Chương trình này tính tổng số tiền phải trả cho một bữa ăn, // với giả định thuế 8% và tiền tip 15%.

```
public class Receipt {
    public static void main(String[] args) {
        System.out.println("Tổng phụ:");
        System.out.println(38 + 40 + 30);
        System.out.println("Thuế:");
        System.out.println((38 + 40 + 30) * .08);
        System.out.println("Tip:");
        System.out.println((38 + 40 + 30) * .15);
        System.out.println("Tổng cộng:");
        System.out.println(38 + 40 + 30 +
                           (38 + 40 + 30) * .08 +
                           (38 + 40 + 30) * .15);
    }
}
```

Mục 2.3: Vòng lặp for

21. Hoàn thành đoạn mã sau, thay thế phần "FINISH ME" bằng mã của bạn:

```
public class Count2 {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            System.out.println("2 times " + i + " = " + (2 * i));
        }
    }
}
```

Kết quả đầu ra mong muốn:

```
2 times 1 = 2
2 times 2 = 4
2 times 3 = 6
```

```
2 times 4 = 8
```

22. Giả sử bạn có một biến count sẽ nhận các giá trị 1, 2, 3, 4, và tiếp theo. Bạn sẽ tạo các biểu thức theo count để tạo ra các dãy số khác nhau. Ví dụ, để có dãy số 2, 4, 6, 8, 10, 12,..., bạn sẽ sử dụng biểu thức (2 * count). Điền vào bảng sau với biểu thức để tạo ra mỗi dãy số.

Dãy số	**Biểu thức**
a. 2, 4, 6, 8, 10, 12, ...	2 * count
b. 4, 19, 34, 49, 64, 79, ...	15 * count - 11
c. 30, 20, 10, 0, −10, −20, ...	30 - 10 * count
d. −7, −3, 1, 5, 9, 13, ...	4 * count - 11
e. 97, 94, 91, 88, 85, 82, ...	100 - 3 * count

23. Hoàn thành đoạn mã sau:

```
for (int i = 1; i <= 6; i++) {
   System.out.println(-4 + 18 * (i - 1));
}
```

Kết quả đầu ra mong muốn:

```
-4
14
32
50
68
86
```

24. Kết quả của phương thức oddStuff sau là gì?

```
public static void oddStuff() {
   int number = 4;
   for (int count = 1; count <= number; count++) {
      System.out.println(number);
      number = number / 2;
   }
}
```

Kết quả đầu ra:

```
4
2
1
```

25. Kết quả của vòng lặp sau là gì?

```
int total = 25;
for (int number = 1; number <= (total / 2); number++) {
    total = total - number;
    System.out.println(total + " " + number);
}
```

Kết quả đầu ra:

```
24 1
22 2
19 3
15 4
10 5
4 6
```

26. Kết quả của vòng lặp sau là gì?

```
System.out.println("+---+");
for (int i = 1; i <= 3; i++) {
    System.out.println("\\   /");
    System.out.println("/   \\");
}
System.out.println("+---+");
```

Kết quả đầu ra:

```
+---+
\   /
/   \
\   /
/   \
\   /
/   \
```

```
+---+
```

27. Kết quả của vòng lặp sau là gì?

```
for (int i = 1; i <= 3; i++)
    System.out.println("How many lines");
System.out.println("are printed?");
```

Kết quả đầu ra:

```
How many lines
How many lines
How many lines
are printed?
```

28. Kết quả của vòng lặp sau là gì?

```
System.out.print("T-minus ");
for (int i = 5; i >= 1; i--) {
    System.out.print(i + ", ");
}
System.out.println("Blastoff!");
```

Kết quả đầu ra:

```
T-minus 5, 4, 3, 2, 1, Blastoff!
```

29. Kết quả của chuỗi vòng lặp sau là gì?

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= 10; j++) {
        System.out.print((i * j) + " ");
    }
    System.out.println();
}
```

Kết quả đầu ra:

```
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
```

30. Kết quả của chuỗi vòng lặp sau là gì?

```
for (int i = 1; i <= 10; i++) {
    for (int j = 1; j <= 10 - i; j++) {
        System.out.print(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

Kết quả đầu ra:

```
         *
        ***
       *****
      *******
     *********
    ***********
   *************
  ***************
 *****************
*******************
```

31. Kết quả của chuỗi vòng lặp sau là gì?

```
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 3; j++) {
        for (int k = 1; k <= 4; k++) {
            System.out.print("*");
        }
        System.out.print("!");
    }
    System.out.println();
}
```

Kết quả đầu ra:

```
****!****!****!
****!****!****!
```

32. Kết quả của chuỗi vòng lặp sau là gì? Lưu ý rằng mã giống với bài tập trước, nhưng vị trí của các dấu ngoặc đã thay đổi.

```
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 3; j++) {
        for (int k = 1; k <= 4; k++) {
            System.out.print("*");
        }
    }
    System.out.print("!");
    System.out.println();
}
```

Kết quả đầu ra:

```
**********!
**********!
```

33. Kết quả của chuỗi vòng lặp sau là gì? Lưu ý rằng mã giống với bài tập trước, nhưng vị trí của các dấu ngoặc đã thay đổi.

```
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 3; j++) {
        for (int k = 1; k <= 4; k++) {
            System.out.print("*");
            System.out.print("!");
        }
        System.out.println();
    }
}
```

Kết quả đầu ra:

```
*!*!*!*!
*!*!*!*!
*!*!*!*!
*!*!*!*!
*!*!*!*!
```

```
*!*!*!*!
```

Mục 2.4: Quản lý Độ phức tạp

34. Giả sử bạn đang cố gắng viết một chương trình để tạo ra kết quả đầu ra sau:

```
1 3 5 7 9 11 13 15 17 19 21
1 3 5 7 9 11
```

Chương trình sau đây là một giải pháp thử nghiệm, nhưng nó chứa bốn lỗi lớn. Xác định tất cả chúng:

```
public class BadNews {
   public static final int MAX_ODD = 21;
   public static void writeOdds() {
      for (int count = 1; count <= (MAX_ODD - 2); count++) {
         System.out.print(count + " ");
         count = count + 2;
      }
      System.out.print(count + 2);
   }
   public static void main(String[] args) {
      writeOdds();
      MAX_ODD = 11;  // Error 1: MAX_ODD là hằng số (final) nên không thể thay đổi giá trị.
      writeOdds();  // Error 2: Chạy lại hàm với giá trị MAX_ODD đã bị thay đổi là không đúng.
   }
}
```

Lỗi chính:

1. Không thể thay đổi giá trị của MAX_ODD vì đó là một hằng số (final).
2. Không nên chạy lại writeOdds() sau khi đã thay đổi giá trị MAX_ODD.
3. count = count + 2 nằm trong vòng lặp làm cho count tăng không đúng cách.

4. Phần System.out.print(count + 2) sẽ in ra một số không phải là số lẻ cuối cùng, mà là số lớn hơn số lẻ cuối cùng.
5. Kết quả của phương thức unknown sau là gì?

```
public class Strange {
   public static final int MAX = 5;
   public static void unknown() {
      int number = 0;
      for (int count = MAX; count >= 1; count--) {
         number += (count * count);
      }
      System.out.println("The result is: " + number);
   }
   public static void main(String[] args) {
      unknown();
   }
}
```

Kết quả đầu ra:

```
The result is: 55
```

36. Giả sử bạn có một biến line sẽ nhận các giá trị 1, 2, 3, 4,... và một hằng số lớp tên là SIZE nhận một trong hai giá trị. Bạn sẽ tạo ra các biểu thức dựa trên line và SIZE để tạo ra các chuỗi số lượng ký tự khác nhau. Điền vào bảng sau với biểu thức để tạo ra mỗi chuỗi số.

Giá trị dòng	**Giá trị SIZE**	**Số lượng ký tự**	**Biểu thức**
1, 2, 3, 4, 5, 6,...	1	4, 6, 8, 10, 12, 14,...	2 * line + 2 * SIZE
1, 2, 3, 4, 5, 6,...	2	6, 8, 10, 12, 14, 16,...	2 * line + 2 * SIZE
1, 2, 3, 4, 5, 6,...	3	13, 17, 21, 25, 29, 33,...	4 * line + 9

Giá trị dòng	Giá trị SIZE	Số lượng ký tự	Biểu thức
1, 2, 3, 4, 5, 6,...	5	19, 23, 27, 31, 35, 39,...	4 * line + 15
1, 2, 3, 4, 5, 6,...	4	10, 9, 8, 7, 6, 5,...	10 - line
1, 2, 3, 4, 5, 6,...	9	20, 19, 18, 17, 16, 15,...	20 - line

37. Viết một bảng xác định các biểu thức cho số lượng mỗi loại ký tự trên mỗi dòng trong 6 dòng kết quả đầu ra sau.

```
!!!!!!!!!!!!!!!!!!!!
\\!!!!!!!!!!!!!!!!!!//
\\\\!!!!!!!!!!!!!!////
\\\\\\!!!!!!!!!!!!//////
\\\\\\\\!!!!!!!!////////
\\\\\\\\\\!!//////////
```

Dòng	Số lượng \	Số lượng !	Số lượng //
1	0	22	0
2	2	18	2
3	4	14	4
4	6	10	6
5	8	6	8
6	10	2	10

38. Giả sử chương trình đã được viết ra để tạo ra kết quả đầu ra như trong bài trước. Bây giờ tác giả muốn chương trình có thể mở rộng bằng cách sử dụng một hằng số lớp tên là SIZE. Kết quả trước đó sử dụng chiều cao cố định là 6 vì có 6 dòng. Dưới đây là kết quả cho chiều cao cố định là 4. Tạo một bảng mới để chỉ ra các biểu thức cho số lượng ký tự ở kích thước mới này là 4 và so sánh các bảng này để

tìm ra các biểu thức cho bất kỳ kích thước nào bằng cách sử dụng hằng số SIZE.

```
!!!!!!!!!!!!!
\\!!!!!!!!!!//
\\\\!!!!!!////
\\\\\\!!//////
```

Dòng	Số lượng \	Số lượng !	Số lượng //
1	0	14	0
2	2	10	2
3	4	6	4
4	6	2	6

Bài tập

1. Trong vật lý, một phương trình hữu ích để tìm vị trí s của một vật thể trong chuyển động thẳng vào thời điểm t, dựa trên vị trí ban đầu s0, vận tốc ban đầu v0, và gia tốc a, là phương trình sau:

s = s0 + v0*t + 1/2*a*t^2

Viết mã để khai báo các biến s0, v0, a, và t, sau đó viết mã để tính s dựa trên các giá trị này.

2. Viết vòng lặp for tạo ra kết quả sau:

```
1 4 9 16 25 36 49 64 81 100
```

Để thêm thử thách, hãy thử sửa mã sao cho không cần sử dụng toán tử nhân *. (Điều này có thể thực hiện được! Gợi ý: Hãy xem xét sự khác biệt giữa các số liền kề.)

3. Các số Fibonacci là một dãy số nguyên trong đó hai phần tử đầu tiên là 1, và mỗi phần tử tiếp theo là tổng của hai phần tử trước đó. Định nghĩa toán học của mỗi số Fibonacci thứ k là:

```
F(k) = {F(k−1) + F(k−2), k > 2
        1, k ≤ 2}
```

Mười hai số Fibonacci đầu tiên là:

```
1 1 2 3 5 8 13 21 34 55 89 144
```

Viết vòng lặp for để tính toán và in ra mười hai số Fibonacci đầu tiên.

4. Viết vòng lặp for lồng nhau để tạo ra kết quả sau:

```
*****
*****
*****
*****
```

5. Viết vòng lặp for lồng nhau để tạo ra kết quả sau:

```
*
**
***
****
*****
```

6. Viết vòng lặp for lồng nhau để tạo ra kết quả sau:

```
1
22
333
4444
55555
666666
7777777
```

7. Viết vòng lặp for lồng nhau để tạo ra kết quả sau:

```
    1
   2
  3
 4
5
```

8. Viết vòng lặp for lồng nhau để tạo ra kết quả sau:

```
    1
   22
  333
```

```
 4444
55555
```

9. Viết vòng lặp for lồng nhau để tạo ra kết quả sau, mỗi dòng rộng 40 ký tự:

```
----------------------------------------
  _-^-  _-^-  _-^-  _-^-  _-^-  _-^-  _-^-  _-^-  _-^-  _-^-
_  _  _  _  _  _  _  _  _  _  _  _  _  _  _  _  _  _  _  _  _
1122334455667788990011223344556677889900
----------------------------------------
```

10. Thông thường, một chương trình bắt đầu bằng cách in ra danh sách các số một chữ số đang quay vòng, tăng dần, như một hướng dẫn trực quan để đánh số các cột của đầu ra. Với ý tưởng này, hãy viết vòng lặp for lồng nhau để tạo ra kết quả sau, mỗi dòng rộng 60 ký tự:

```
      |         |         |         |         |         |
12345678901234567890123456789012345678901234567890123
34567890
```

11. Sửa đổi mã từ bài tập trước để có thể dễ dàng thay đổi để hiển thị một phạm vi số khác (thay vì 1234567890) và số lần lặp lại các số đó khác nhau (thay vì tổng cộng 60 ký tự), với các thanh dọc vẫn khớp chính xác. Sử dụng hằng số lớp thay vì "số ma thuật". Đây là một số ví dụ về đầu ra có thể được tạo ra bằng cách thay đổi các hằng số của bạn:

```
    |   |   |   |   |   |   |   |   |   |
12340123401234012340123401234012340123401234012340
```

```
      |       |       |       |       |       |
12345670123456701234567012345670123456701234567012345
670
```

12. Viết vòng lặp for lồng nhau để tạo ra kết quả sau:

```
000111222333444555666777888999
000111222333444555666777888999
000111222333444555666777888999
```

13. Sửa đổi mã để tạo ra kết quả sau:

```
999998888777776666655555444443333322221111100000
999998888777776666655555444443333322221111100000
999998888777776666655555444443333322221111100000
999998888777776666655555444443333322221111100000
999998888777776666655555444443333322221111100000
```

14. Sửa đổi mã để tạo ra kết quả sau:

```
999999999888888887777777666666555554444333221
999999999888888887777777666666555554444333221
999999999888888887777777666666555554444333221
999999999888888887777777666666555554444333221
```

15. Viết một phương thức gọi là printDesign để tạo ra kết quả sau. Sử dụng vòng lặp for lồng nhau để nắm bắt cấu trúc của hình:

```
-----1-----
----333----
---55555---
--7777777--
-999999999-
```

16. Viết một chương trình Java gọi là SlashFigure để tạo ra kết quả sau. Sử dụng vòng lặp for lồng nhau để nắm bắt cấu trúc của hình. (Xem thêm các bài tự kiểm tra 34 và 35.)

```
!!!!!!!!!!!!!!!!!!!!
\\!!!!!!!!!!!!!!!!!!//
\\\\!!!!!!!!!!!!!!!!////
\\\\\\!!!!!!!!!!!!!!//////
\\\\\\\\!!!!!!!!!!!!////////
\\\\\\\\\\!!!!!!!!!!//////////
```

17. Sửa đổi chương trình SlashFigure từ bài tập trước để trở thành một chương trình mới gọi là SlashFigure2 sử dụng hằng số toàn cục cho chiều cao của hình. (Bạn có thể muốn tạo bảng vòng lặp trước.) Kết quả trước đó sử dụng chiều

cao cố định là 6. Dưới đây là các kết quả cho chiều cao cố định là 4 và 8:

```
Chiều cao 4:     Chiều cao 8:
!!!!!!!!!!!!!
\\!!!!!!!!!!//
\\\\!!!!!!////
\\\\\\!!//////
```

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
\\!!!!!!!!!!!!!!!!!!!!!!!!!!!//
\\\\!!!!!!!!!!!!!!!!!!!!!!!////
\\\\\\!!!!!!!!!!!!!!!!!!!!!//////
\\\\\\\!!!!!!!!!!!!!!!!!////////
\\\\\\\\\!!!!!!!!!!!!!!!!!!!!!!!
\\\\\\\\\\\!!!!!!!!!!!!!!!!!!!!!
\\\\\\\\\\\\\!!!!!!!!!!!!!!!!!!!
\\\\\\\\\\\\\\\!!!!!!!!!!!!!!!!!
```

18. Viết một thuật toán giả mã để tạo ra hình sau làm đầu ra:

```
+===+===+
|   |   |
|   |   |
|   |   |
+===+===+
|   |   |
|   |   |
|   |   |
+===+===+
```

19. Sử dụng giả mã của bạn từ bài tập trước để viết một chương trình Java gọi là Window tạo ra hình trên làm đầu ra. Sử dụng vòng lặp for lồng nhau để in các phần lặp lại của hình. Sau khi bạn làm cho nó hoạt động, thêm một hằng số lớp để kích thước của hình có thể được thay đổi đơn giản bằng cách thay đổi giá trị của hằng số.

20. Viết một chương trình Java gọi là StarFigure để tạo ra kết quả sau. Sử dụng vòng lặp for lồng nhau để nắm bắt cấu trúc của hình:

```
//////////////////\\\\\\\\\\\\\\\\\\
////////////********\\\\\\\\\\\\
////////**********\\\\\\\\
////**********\\\\
**********
```

21. Sửa đổi chương trình StarFigure từ bài tập trước để trở thành một chương trình mới gọi là StarFigure2 sử dụng hằng số toàn cục cho chiều cao của hình. (Bạn có thể muốn tạo bảng vòng lặp trước.) Kết quả trước đó sử dụng chiều cao cố định là 5. Dưới đây là các kết quả cho chiều cao cố định là 3 và 6:

```
Chiều cao 3:     Chiều cao 6:
//////////\\\\\\\\\\
///////********\\\\\\\\
```

```
///////////////\\\\\\\\\\\\\\\\\\
//////////********\\\\\\\\\\
////////**********\\\\\\\\\\
////**********\\\\\\
///**********\\\
**********
```

22. Viết một chương trình Java gọi là DollarFigure để tạo ra kết quả sau. Sử dụng vòng lặp for lồng nhau để nắm bắt cấu trúc của hình:

```
$$$$$$**********$$$$$$
**$$$$$**********$$$$$**
****$$$$**********$$$$****
******$$$********$$$******
********$$******$$********
**********$$****$$**********
```

```
**********$**$**********
```

23. Sửa đổi chương trình DollarFigure từ bài tập trước để trở thành một chương trình mới gọi là DollarFigure2 sử dụng hằng số toàn cục cho chiều cao của hình. Kết quả trước đó sử dụng chiều cao cố định là 7.
24. Viết một chương trình gọi là TwoRectangles sử dụng hai hằng số lớp WIDTH và HEIGHT để vẽ hai hình chữ nhật bằng các dấu sao với các kích thước đã cho. Hình chữ nhật đầu tiên phải nằm sát lề trái, và hình chữ nhật thứ hai phải được thụt vào ngang bằng với chiều rộng đã cho. Ví dụ, nếu WIDTH và HEIGHT lần lượt là 7 và 4, chương trình phải vẽ hình sau:

```
*******
*******
*******
*******
      *******
      *******
      *******
      *******
```

Bài tập lập trình

1. Viết chương trình sử dụng vòng lặp lồng nhau để tạo ra kết quả sau:

```
****** /////////// ******
*****  //////////\\  *****
****   ////////\\\\   ****
***    //////\\\\\\    ***
**     ////\\\\\\\\     **
*      //\\\\\\\\\\      *
      \\\\\\\\\\\\
```

2. Viết chương trình sử dụng vòng lặp lồng nhau để tạo ra kết quả sau:

```
+------+
|  ^^  |
| ^  ^ |
|^    ^|
|  ^^  |
| ^  ^ |
|^    ^|
+------+
|v    v|
| v  v |
|  vv  |
|v    v|
| v  v |
|  vv  |
+------+
```

3. Viết chương trình sử dụng vòng lặp lồng nhau để tạo ra kết quả sau:

```
+---------+
|    *    |
|   /*\   |
|  //*\\  |
| ///*\\\ |
| \\\*/// |
|  \\*//  |
|   \*/   |
|    *    |
+---------+
| \\\*/// |
|  \\*//  |
|   \*/   |
|    *    |
|    *    |
|   /*\   |
```

```
|  //*\\  |
| ///*\\\ |
+---------+
```

4. Viết chương trình sử dụng vòng lặp lồng nhau để tạo ra hình đồng hồ cát như sau:

```
|''''''''''''''''''|
 \::::::::/
  \::::::/
   \::::/
    \::/
     ||
    /::\
   /::::\
  /::::::\
 /::::::::\
|''''''''''''''''''|
```

5. Viết chương trình sử dụng vòng lặp lồng nhau để tạo ra kết quả sau. Sử dụng hằng số lớp để thay đổi số bậc thang trong hình:

```
                o  *******
               /|\ *     *
               / \ *     *
             o  ******     *
            /|\ *          *
            / \ *          *
          o  ******         *
         /|\ *             *
         / \ *             *
       o  ******           *
      /|\ *                *
      / \ *                *
   o  ******               *
  /|\ *                    *
```

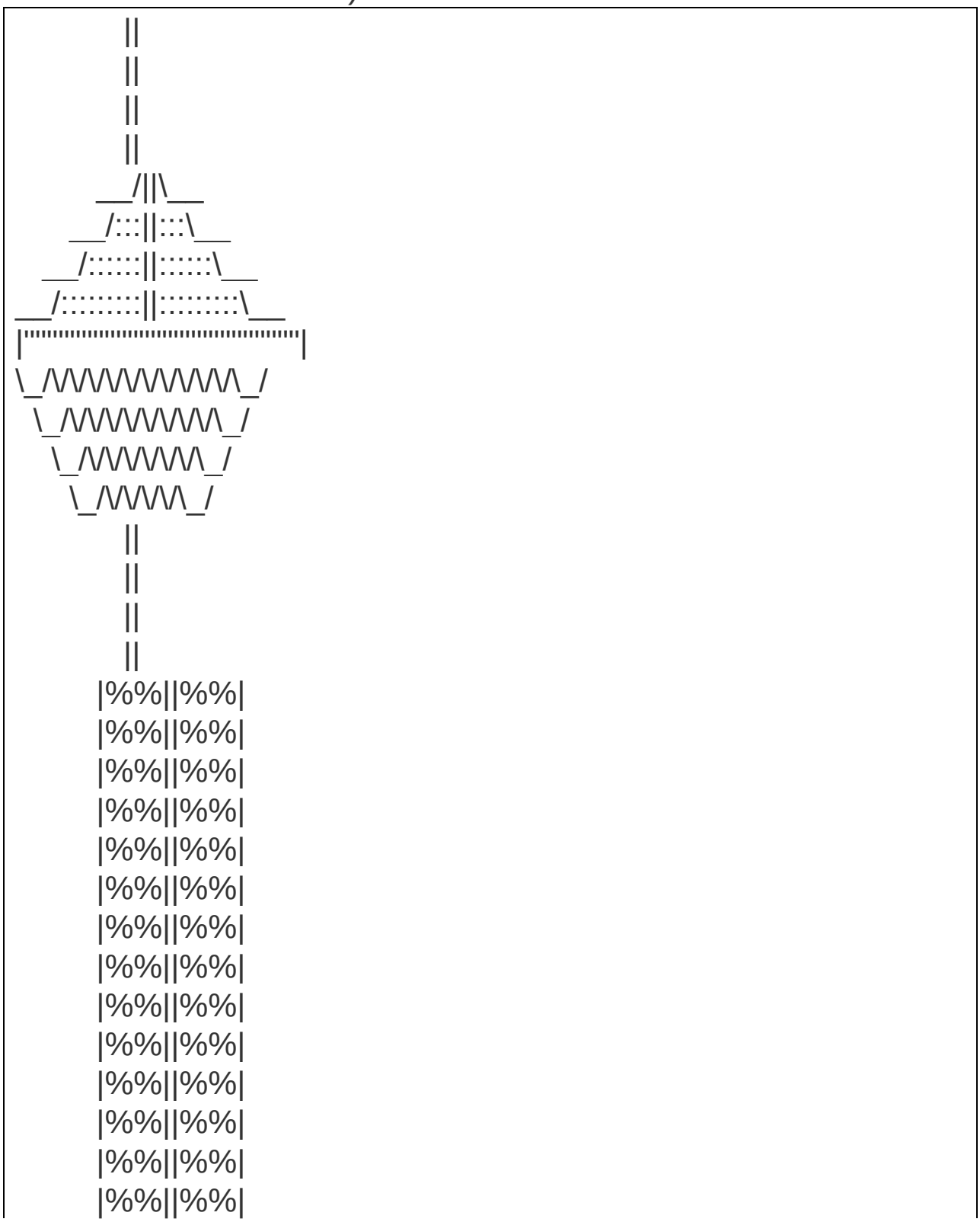
```
 / \ *                    *
*********************************
```

6. Viết chương trình sử dụng vòng lặp lồng nhau để tạo ra hình tàu vũ trụ như sau. Sử dụng hằng số lớp để thay đổi kích thước tàu (kết quả dưới đây có kích thước là 3):

```
     /**\
    //**\\
   ///**\\\
  ////**\\\\
 /////**\\\\\
+=*=*=*=*=*=*+
|../\..../\..|
|./\/\../\/\.|
|/\/\/\/\/\/\|
|\/\/\/\/\/\/|
|.\/\..\/\/.|
|..\/....\/..|
+=*=*=*=*=*=*+
|\/\/\/\/\/\/|
|.\/\..\/\/.|
|..\/....\/..|
|../\..../\..|
|./\/\../\/\.|
|/\/\/\/\/\/\|
+=*=*=*=*=*=*+
     /**\
    //**\\
   ///**\\\
  ////**\\\\
 /////**\\\\\
```

7. Viết chương trình sử dụng vòng lặp lồng nhau để tạo ra hình giống như Tháp Space Needle ở Seattle như sau. Sử dụng

hằng số lớp để thay đổi kích thước hình (kết quả dưới đây có kích thước là 4):


||
||
||
||
__/||__
__/:::||:::__
__/::::::||::::::__
__/:::::::::||:::::::::__
|''''''

```
        |%%||%%|
        |%%||%%|
        |%%||%%|
         __/||\__
      __/:::||:::\__
   __/::::::||::::::\__
__/::::::::||::::::::\__
|''''''
```

- Cơ chế của Tham số
- Hạn chế của Tham số
- Nhiều Tham số
- So sánh Tham số và Hằng số
- Nạp chồng phương thức

3.2 Phương thức Trả về Giá trị

- Lớp Math
- Định nghĩa Phương thức Trả về Giá trị

3.3 Sử dụng Đối tượng

- Đối tượng String
- Chương trình Tương tác và Đối tượng Scanner
- Ví dụ Chương trình Tương tác

3.4 Nghiên cứu tình huống: Quỹ đạo Vật phóng

- Giải pháp chưa cấu trúc
- Giải pháp có cấu trúc

Giới thiệu

Chương 2 đã giới thiệu các kỹ thuật để quản lý sự phức tạp, bao gồm việc sử dụng các hằng số lớp, giúp chương trình trở nên linh hoạt hơn. Chương này khám phá một kỹ thuật mạnh mẽ hơn để đạt được sự linh hoạt đó. Tại đây, bạn sẽ học cách sử dụng tham số để tạo ra các phương thức giải quyết không chỉ một nhiệm vụ duy nhất mà còn là toàn bộ các nhiệm vụ tương tự. Việc tạo ra các phương thức như vậy yêu cầu bạn phải tổng quát hóa, hoặc nhìn vượt ra ngoài một nhiệm vụ cụ thể để tìm ra danh mục nhiệm vụ chung mà nó thể hiện. Khả năng tổng quát hóa là một trong những phẩm chất quan trọng nhất của một kỹ sư phần mềm giỏi, và kỹ thuật tổng quát hóa mà bạn sẽ học trong chương này là một trong những kỹ thuật mạnh mẽ nhất mà lập trình viên sử dụng. Sau khi khám phá về tham số, chúng ta sẽ thảo luận về một số vấn đề khác liên quan đến phương thức, chẳng hạn như khả năng của một phương thức để trả về một giá trị.

Chương này sau đó giới thiệu ý tưởng về các đối tượng và cách sử dụng chúng trong các chương trình Java. Chúng ta sẽ không khám phá chi tiết về định nghĩa đối tượng ngay lập tức, nhưng chúng ta muốn bắt đầu sử dụng đối tượng từ sớm. Một trong những tính năng hấp dẫn nhất của Java là nó đi kèm với một thư viện phong phú các đối tượng được định nghĩa sẵn có thể được sử dụng để giải quyết nhiều nhiệm vụ lập trình phổ biến. Chương này kết thúc với việc khám phá một loại đối tượng rất quan trọng được gọi là Scanner. Bằng cách sử dụng đối tượng Scanner, bạn có thể viết các chương trình lấy giá trị từ người dùng. Tính năng này sẽ cho phép bạn viết các chương trình tương tác, yêu cầu nhập liệu cũng như xuất kết quả.

3.1 Tham số

Con người rất giỏi trong việc học các nhiệm vụ mới. Khi chúng ta học, chúng ta thường phát triển một giải pháp chung cho một loạt các nhiệm vụ liên quan. Ví dụ, ai đó có thể yêu cầu bạn tiến lên 10 bước hoặc 20 bước. Đây là những nhiệm vụ khác nhau, nhưng cả hai đều liên quan đến việc tiến một số bước nhất định. Chúng ta coi hành động này là một nhiệm vụ duy nhất là tiến bước, và chúng ta hiểu rằng số bước sẽ thay đổi từ nhiệm vụ này sang nhiệm vụ khác. Trong thuật ngữ lập trình, chúng ta gọi số bước là một tham số cho phép chúng ta tổng quát hóa nhiệm vụ.

Tham số (Parameterize)

Bất kỳ đặc điểm nào trong tập hợp các đặc điểm giúp phân biệt các thành viên khác nhau của một nhóm nhiệm vụ. Tham số hóa một nhiệm vụ là xác định một tập hợp các tham số của nó. Để có ví dụ lập trình, hãy quay lại chương trình DrawFigure2 của Chương 2. Nó thực hiện nhiệm vụ của mình một cách hiệu quả, nhưng có nhiều khía cạnh của chương trình này mà chúng ta có thể cải thiện. Ví dụ, có nhiều chỗ trong vòng lặp for mà một khoảng trắng được ghi ra. Cách tiếp cận này là dư thừa và có thể

được hợp nhất vào một phương thức duy nhất thực hiện tất cả các nhiệm vụ ghi khoảng trắng.
Mỗi nhiệm vụ ghi khoảng trắng đòi hỏi một số khoảng trắng khác nhau, vì vậy bạn cần một cách nào đó để nói cho phương thức biết cần viết bao nhiêu khoảng trắng. Các phương thức mà bạn đã viết cho đến nay có cơ chế gọi đơn giản, trong đó bạn nói:

```
writeSpaces();
```

Một cách tiếp cận có thể là đặt một biến với một giá trị cụ thể trước khi phương thức được gọi:

```
int number = 10;
writeSpaces();
```

Sau đó, phương thức có thể xem giá trị của biến number để biết cần viết bao nhiêu khoảng trắng. Tuy nhiên, cách tiếp cận này sẽ không hoạt động. Nhớ lại từ Chương 2 rằng các quy tắc phạm vi xác định nơi các biến có thể được truy cập. Theo các quy tắc đó, biến number sẽ là một biến cục bộ trong main và không thể được nhìn thấy bên trong writeSpaces.
Thay vào đó, bạn có thể chỉ định một hoặc nhiều tham số cho một phương thức. Ý tưởng là thay vì viết một phương thức thực hiện chỉ một phiên bản của một nhiệm vụ, bạn viết một phiên bản linh hoạt hơn để giải quyết một loạt các nhiệm vụ liên quan mà tất cả khác nhau bởi một hoặc nhiều tham số. Trong trường hợp của phương thức writeSpaces, tham số là số khoảng trắng cần ghi. Dưới đây là định nghĩa của writeSpaces với một tham số cho số khoảng trắng cần ghi:

```
public static void writeSpaces(int number) {
    for (int i = 1; i <= number; i++) {
        System.out.print(" ");
    }
}
```

Tham số xuất hiện trong tiêu đề của phương thức, sau tên và bên trong dấu ngoặc đơn mà bạn đã để trống cho đến thời điểm này. Phương thức writeSpaces sử dụng một tham số gọi là number

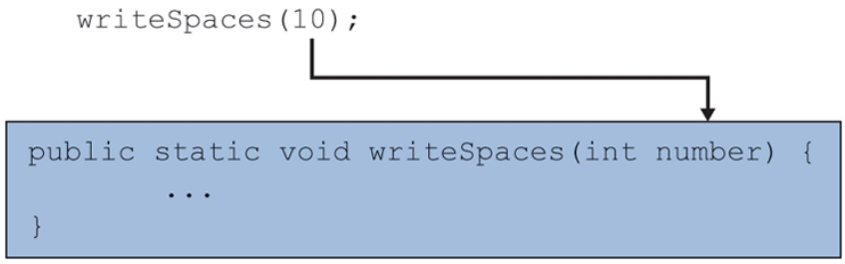
kiểu int. Như chúng ta đã chỉ ra trước đó, bạn không còn có thể gọi phương thức có tham số bằng cách chỉ sử dụng tên của nó:

```
writeSpaces();
```

Bây giờ bạn phải nói điều gì đó như:

```
writeSpaces(10);
```

Khi một lệnh gọi như vậy được thực hiện, giá trị 10 sẽ được sử dụng để khởi tạo tham số number. Bạn có thể coi đây là thông tin được truyền vào phương thức từ lệnh gọi:



3.1-1 Văn bản thay thế đầy đủ

Tham số number là một biến cục bộ, nhưng nó nhận giá trị ban đầu từ lệnh gọi. Gọi phương thức này với giá trị 10 tương đương với việc thêm khai báo sau vào đầu phương thức writeSpaces:

```
int number = 10;
```

Tất nhiên, cơ chế này linh hoạt hơn so với một khai báo biến cụ thể, bởi vì bạn có thể thay thế bằng:

```
writeSpaces(20);
```

và nó sẽ tương đương với việc bạn đã nói:

```
int number = 20;
```

ở đầu phương thức. Bạn thậm chí có thể sử dụng một biểu thức số nguyên trong lệnh gọi:

```
writeSpaces(3 * 4 – 5);
```

Trong trường hợp này, Java sẽ tính toán biểu thức để lấy giá trị 7 và sau đó gọi writeSpaces, khởi tạo number thành 7.

Các nhà khoa học máy tính sử dụng từ “tham số” rộng rãi để chỉ cả những gì xuất hiện trong tiêu đề phương thức (tham số hình

thức) và những gì xuất hiện trong lệnh gọi phương thức (tham số thực).

Tham số hình thức
Một biến xuất hiện bên trong dấu ngoặc đơn trong tiêu đề của một phương thức, được sử dụng để tổng quát hóa hành vi của phương thức.

Tham số thực
Một giá trị cụ thể hoặc biểu thức xuất hiện bên trong dấu ngoặc đơn trong lệnh gọi phương thức.
Thuật ngữ “tham số hình thức” không mô tả mục đích của nó. Một tên gọi tốt hơn có thể là “tham số tổng quát hóa”. Trong phương thức writeSpaces, number là tham số tổng quát hóa xuất hiện trong khai báo phương thức. Nó là một chỗ giữ chỗ cho một giá trị chưa xác định. Các giá trị xuất hiện trong các lệnh gọi phương thức là các tham số thực, bởi vì mỗi lệnh gọi chỉ định một nhiệm vụ cụ thể để thực hiện. Nói cách khác, mỗi lệnh gọi cung cấp một giá trị thực để điền vào chỗ giữ chỗ.
Từ “đối số” thường được sử dụng như một từ đồng nghĩa với “tham số”, như trong “Đây là các đối số mà tôi đang truyền cho phương thức này”. Một số người thích sử dụng từ “đối số” để chỉ tham số thực và từ “tham số” để chỉ tham số hình thức.
Hãy xem xét một ví dụ về cách bạn có thể sử dụng phương thức writeSpaces này. Nhớ lại rằng chương trình DrawFigure2 có phương thức sau, được gọi là drawTop: // tạo nửa trên của hình đồng hồ cát

```
public static void drawTop() {
    for (int line = 1; line <= SUB_HEIGHT; line++) {
        System.out.print("|");
        for (int i = 1; i <= (line – 1); i++) {
            System.out.print(" ");
        }
```

```
        System.out.print("\\");
        int dots = 2 * SUB_HEIGHT – 2 * line;
        for (int i = 1; i <= dots; i++) {
            System.out.print(".");
        }
        System.out.print("/");
        for (int i = 1; i <= (line – 1); i++) {
            System.out.print(" ");
        }
        System.out.println("|");
    }
}
```

Sử dụng phương thức writeSpaces, bạn có thể viết lại như sau:

```
public static void drawTop() {
    for (int line = 1; line <= SUB_HEIGHT; line++) {
        System.out.print("|");
        writeSpaces(line – 1);
        System.out.print("\\");
        int dots = 2 * SUB_HEIGHT – 2 * line;
        for (int i = 1; i <= dots; i++) {
            System.out.print(".");
        }
        System.out.print("/");
        writeSpaces(line – 1);
        System.out.println("|");
    }
}
```

Lưu ý rằng writeSpaces được gọi hai lần khác nhau, chỉ định số lượng khoảng trắng cần thiết trong mỗi trường hợp. Bạn có thể sửa đổi phương thức drawBottom từ chương trình DrawFigure2 tương tự để đơn giản hóa nó.

Bạn có thể viết toàn bộ phương thức trong JShell để kiểm tra các tham số. Dưới đây là một phương thức gọi là writeStars in ra một số lượng dấu sao nhất định ra màn hình:

```
jshell>
public static void writeStars(int number) {
    for (int i = 1; i <= number; i++) {
        System.out.print("*");
    }
}
| đã tạo phương thức writeStars(int)
```

```
jshell>
writeStars(5);
```

```
jshell>
writeStars(8);
********
```

Cơ chế của Tham số

Khi Java thực thi một lệnh gọi phương thức, nó sẽ khởi tạo các tham số của phương thức. Đối với mỗi tham số, Java đầu tiên đánh giá biểu thức được truyền vào như là tham số thực, sau đó sử dụng kết quả đó để khởi tạo biến cục bộ có tên được xác định bởi tham số hình thức. Hãy sử dụng một ví dụ để làm rõ quá trình này:

```
// Minh họa tham số với một phương thức để viết khoảng trắng.
public class ParameterExample {
    public static void main(String[] args) {
        int spaces1 = 3;
        int spaces2 = 5;

        System.out.print("*");
        writeSpaces(spaces1);
        System.out.println("*");
```

```
        System.out.print("!");
        writeSpaces(spaces2);
        System.out.println("!");

        System.out.print("'");
        writeSpaces(8);
        System.out.println("'");

        System.out.print("<");
        writeSpaces(spaces1 * spaces2 - 5);
        System.out.println(">");
    }

    // viết "number" khoảng trắng trên dòng đầu ra hiện tại
    public static void writeSpaces(int number) {
        for (int i = 1; i <= number; i++) {
            System.out.print(" ");
        }
    }
}
```

Trong hai dòng đầu tiên của phương thức main, máy tính tìm thấy các chỉ thị để cấp phát và khởi tạo hai biến:

Ba dòng mã tiếp theo tạo ra một dòng đầu ra với ba khoảng trắng được bao quanh bởi các dấu sao:

```
System.out.print("*");
writeSpaces(spaces1);
System.out.println("*");
```

Bạn có thể thấy các dấu sao xuất hiện từ đâu, nhưng hãy xem xét lệnh gọi phương thức tạo ra các khoảng trắng. Khi Java thực thi lệnh gọi writeSpaces, nó phải thiết lập tham số của nó. Để thiết

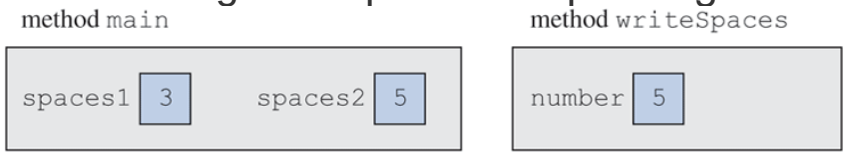
lập tham số, Java đầu tiên đánh giá biểu thức được truyền vào như là tham số thực. Biểu thức chỉ đơn giản là biến spaces1, có giá trị là 3. Do đó, biểu thức này được đánh giá là 3. Java sử dụng kết quả này để khởi tạo một biến cục bộ gọi là number.
Sơ đồ sau cho thấy bộ nhớ của máy tính sẽ trông như thế nào khi phương thức writeSpaces được gọi lần đầu tiên. Vì có hai phương thức liên quan (main và writeSpaces), sơ đồ sẽ chỉ ra các biến nào là cục bộ của main (spaces1 và spaces2) và các biến nào là cục bộ của writeSpaces (tham số number):

3.1-3 Văn bản thay thế đầy đủ

Hiệu ứng cuối cùng của quá trình này là phương thức writeSpaces có một bản sao cục bộ của giá trị được lưu trữ trong biến spaces1 từ phương thức main. Lệnh println đi sau lệnh gọi writeSpaces sẽ đặt một dấu sao ở cuối dòng và hoàn thành dòng đầu ra.
Hãy cùng theo dõi ba dòng mã tiếp theo:

```
System.out.print("!");
writeSpaces(spaces2);
System.out.println("!");
```

Dòng đầu tiên in ra một dấu chấm than trên dòng thứ hai của đầu ra, sau đó gọi lại phương thức writeSpaces, lần này với biến spaces2 làm tham số thực. Máy tính đánh giá biểu thức này và thu được kết quả là 5. Giá trị này được sử dụng để khởi tạo number. Do đó, lần này nó tạo ra một bản sao của giá trị được lưu trữ trong biến spaces2 từ phương thức main:



3.1-4 Văn bản thay thế đầy đủ

Vì number có giá trị khác lần này (5 thay vì 3), phương thức tạo ra số lượng khoảng trắng khác nhau. Sau khi phương thức thực thi, lệnh println kết thúc dòng đầu ra với dấu chấm than thứ hai. Dưới đây là ba dòng mã tiếp theo:

```
System.out.print("");
writeSpaces(8);
System.out.println("");
```

Đoạn mã này in một dấu nháy đơn ở đầu dòng thứ ba của đầu ra và sau đó lại gọi phương thức writeSpaces. Lần này, nó sử dụng số nguyên 8 làm biểu thức, có nghĩa là nó khởi tạo tham số number như một bản sao của số 8:

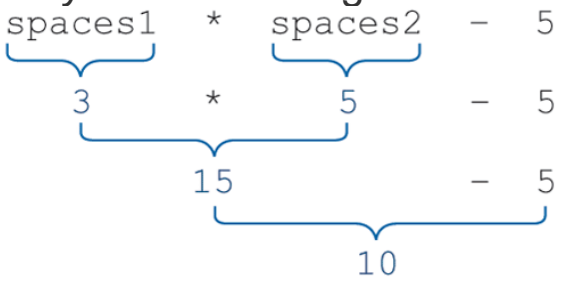
3.1-5 Văn bản thay thế đầy đủ

Một lần nữa, phương thức sẽ hoạt động khác nhau do giá trị khác của number. Nó in ra tám khoảng trắng trên dòng và hoàn thành việc thực thi. Sau đó, lệnh println kết thúc dòng đầu ra bằng cách in thêm một dấu nháy đơn ở cuối dòng.

Cuối cùng, ba dòng mã cuối trong phương thức main là:

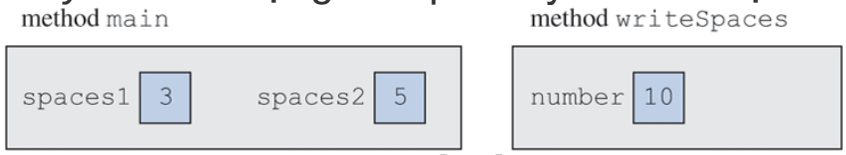
```
System.out.print("<");
writeSpaces(spaces1 * spaces2 - 5);
System.out.println(">");
```

Đoạn mã này in ký tự "nhỏ hơn" ở đầu dòng thứ tư của đầu ra và sau đó thực hiện một lần gọi cuối cùng đến phương thức writeSpaces. Lần này, tham số thực là một biểu thức, không chỉ là một biến hoặc giá trị số. Vì vậy, trước khi thực hiện lệnh gọi, máy tính sẽ đánh giá biểu thức để xác định giá trị của nó:



3.1-6 Văn bản thay thế đầy đủ

Máy tính sử dụng kết quả này để khởi tạo number:



3.1-7 Văn bản thay thế đầy đủ

Bây giờ number là một bản sao của giá trị được mô tả bởi biểu thức phức tạp này. Do đó, tổng đầu ra của chương trình này là:

```
*  *
!   !
'    '

<      >
```

LỖI LẬP TRÌNH PHỔ BIẾN

Nhầm lẫn giữa Tham số Thực và Tham số Hình thức

Nhiều học sinh quen với việc nhìn thấy khai báo của tham số hình thức và nhầm lẫn tin rằng cú pháp của chúng giống hệt như khi truyền tham số thực. Một lỗi phổ biến là viết kiểu dữ liệu của biến khi truyền nó vào tham số:

```
writeSpaces(int spaces1); // điều này không hoạt động
```

Sự nhầm lẫn này là do tham số của kiểu dữ liệu được viết trong khai báo của phương thức, như thế này:

```
public static void writeSpaces(int number)
```

Các kiểu dữ liệu phải được viết khi khai báo biến hoặc tham số, nhưng khi biến được sử dụng, chẳng hạn như khi mã gọi phương thức và truyền biến làm tham số thực, kiểu dữ liệu không được viết. Tham số thực không phải là khai báo; do đó, kiểu dữ liệu không nên được viết trước chúng:

```
writeSpaces(spaces1); // tốt hơn nhiều!
```

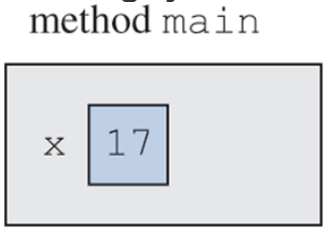
Hạn chế của Tham số

Chúng ta đã thấy rằng một tham số có thể được sử dụng để cung cấp đầu vào cho một phương thức. Nhưng mặc dù bạn có thể sử dụng tham số để gửi giá trị vào một phương thức, bạn không thể sử dụng tham số để nhận giá trị ra khỏi phương thức.

Khi một tham số được thiết lập, một biến cục bộ được tạo và được khởi tạo với giá trị đang được truyền dưới dạng tham số thực. Hiệu ứng cuối cùng là biến cục bộ là một bản sao của giá trị đến từ bên ngoài. Vì nó là biến cục bộ, nó không thể ảnh hưởng đến bất kỳ biến nào bên ngoài phương thức. Hãy xem xét chương trình mẫu sau:

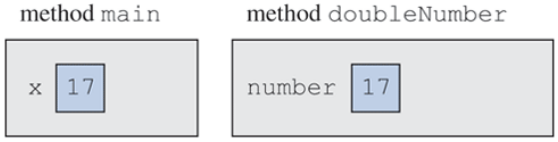
```
1  // Một minh họa về những hạn chế của tham số.
2  public class ParameterExample2 {
3      public static void main(String[] args) {
4          int x = 17;
5          doubleNumber(x);
6          System.out.println("x = " + x);
7          System.out.println();
8          int number = 42;
9          doubleNumber(number);
10          System.out.println("number = " + number);
11     }
12
13     public static void doubleNumber(int number) {
14         System.out.println("Giá trị ban đầu = " + number);
15         number = number * 2;
16         System.out.println("Giá trị cuối cùng = " + number);
17     }
18 }
```

Chương trình này bắt đầu bằng việc khai báo và khởi tạo một biến nguyên có tên là x với giá trị 17:



Chương trình sau đó gọi phương thức doubleNumber, truyền x làm tham số. Giá trị của x được sử dụng để khởi tạo tham số

number như là một biến cục bộ của phương thức doubleNumber:



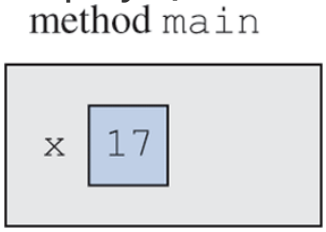
3.1-9 Văn bản thay thế đầy đủ

Chương trình sau đó thực thi các lệnh bên trong hàm doubleNumber. Đầu tiên, doubleNumber in ra giá trị ban đầu của biến number (17). Sau đó, nó nhân đôi number:

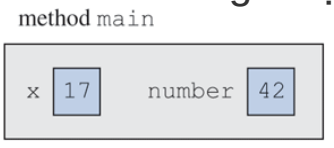
3.1-10 Văn bản thay thế đầy đủ

Lưu ý rằng điều này không ảnh hưởng đến biến x. Tham số có tên là number chỉ là một bản sao của x, vì vậy mặc dù chúng bắt đầu giống nhau, việc thay đổi giá trị của number sẽ không ảnh hưởng đến x. Tiếp theo, doubleNumber báo cáo giá trị mới của number (34).

Tại thời điểm này, doubleNumber hoàn tất việc thực thi và chúng ta quay lại hàm main:

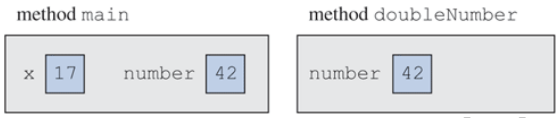


Phương thức tiếp theo trong phương thức main báo cáo giá trị của x, là 17. Sau đó, nó khai báo và khởi tạo một biến có tên là number với giá trị 42:



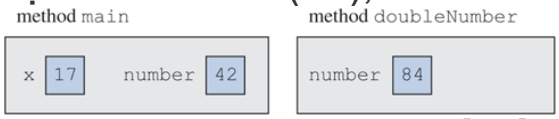
3.1-12 Văn bản thay thế đầy đủ

Lệnh tiếp theo gọi lại phương thức doubleNumber, lần này truyền vào giá trị của number. Đây là một tình huống kỳ lạ vì tham số có cùng tên với biến trong main, nhưng Java không quan tâm. Nó luôn tạo ra một biến cục bộ mới cho phương thức doubleNumber:



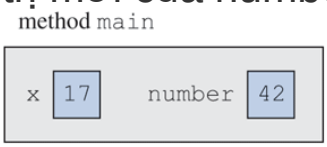
3.1-13 Văn bản thay thế đầy đủ

Vì vậy, tại thời điểm này có hai biến khác nhau được gọi là number, một trong mỗi phương thức. Bây giờ là lúc để thực thi các lệnh của doubleNumber một lần nữa. Đầu tiên nó báo cáo giá trị của number (42), sau đó nhân đôi giá trị này:



3.1-14 Văn bản thay thế đầy đủ

Một lần nữa, lưu ý rằng việc nhân đôi number bên trong doubleNumber không ảnh hưởng đến biến number ban đầu trong main. Đây là các biến riêng biệt. Phương thức sau đó báo cáo giá trị mới của number (84) và quay trở lại main:



3.1-15 Văn bản thay thế đầy đủ

Chương trình sau đó báo cáo giá trị của number và kết thúc. Vì vậy, đầu ra tổng thể của chương trình như sau:

```
System.out.println("x = " + x);
System.out.println("number = " + number);
```

Kết quả:

```
Initial value = 17
Final value = 34
x = 17

Initial value = 42
Final value = 84
number = 42
```

Các thao tác cục bộ của tham số không thay đổi các biến này bên ngoài phương thức. Việc các biến được sao chép là một khía cạnh quan trọng của tham số. Mặt tích cực, chúng ta biết rằng các biến được bảo vệ khỏi sự thay đổi vì các tham số là bản sao của bản gốc. Mặt tiêu cực, điều này có nghĩa là mặc dù tham số cho phép chúng ta gửi giá trị vào một phương thức, chúng không cho phép chúng ta lấy giá trị ra khỏi phương thức.

Nhiều Tham Số

Cho đến nay, việc thảo luận về cú pháp tham số của chúng ta vẫn còn khá không chính thức. Đã đến lúc chúng ta cần viết chính xác hơn cú pháp mà chúng ta sử dụng để khai báo các phương thức tĩnh với tham số. Dưới đây là cú pháp:

```
public static void <name>(<type> <name>, ..., <type> <name>) {
   <statement>;
   <statement>;
   ...
   <statement>;
}
```

Mẫu này chỉ ra rằng chúng ta có thể khai báo bao nhiêu tham số tùy thích bên trong dấu ngoặc đơn xuất hiện sau tên của một phương thức trong phần khai báo. Chúng ta sử dụng dấu phẩy để tách các tham số khác nhau.

Ví dụ về một phương thức với nhiều tham số, hãy xem xét một biến thể của phương thức writeSpaces. Việc sử dụng phương thức này để viết các số lượng khoảng trắng khác nhau là tiện lợi, nhưng nó luôn viết khoảng trắng. Sẽ thế nào nếu chúng ta muốn 18 dấu hoa thị, 23 dấu chấm hoặc 17 dấu hỏi? Chúng ta có thể tổng quát hóa nhiệm vụ này hơn nữa bằng cách để phương thức nhận hai tham số—một ký tự và số lần cần viết ký tự đó:

```
public static void writeChars(char ch, int number) {
   for (int i = 1; i <= number; i++) {
      System.out.print(ch);
```

```
    }
}
```

Ký tự cần được in là một tham số kiểu char, mà chúng ta sẽ thảo luận chi tiết hơn trong chương tiếp theo. Nhớ rằng các ký tự chữ được bao quanh bởi dấu nháy đơn.

Cú pháp mẫu để gọi một phương thức chấp nhận tham số là:

```
<method name>(<expression>, <expression>, ..., <expression>);
```

Bằng cách gọi phương thức writeChars, bạn có thể viết mã như sau:

```
writeChars('=', 20);
System.out.println();

for (int i = 1; i <= 10; i++) {
    writeChars('>', i);
    writeChars(' ', 20 - 2 * i);
    writeChars('<', i);
    System.out.println();
}
```

Mã này tạo ra kết quả sau:

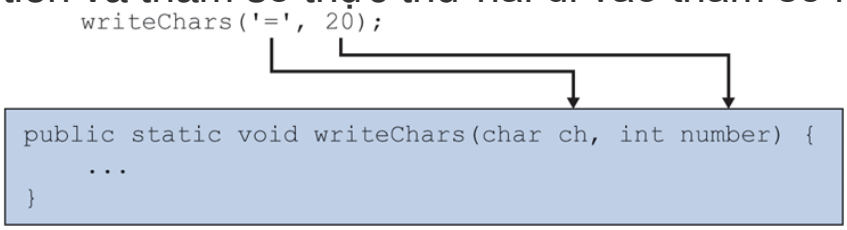
```
====================
>              <
>>            <<
>>>          <<<
>>>>        <<<<
>>>>>      <<<<<
>>>>>>    <<<<<<
>>>>>>>  <<<<<<<
>>>>>>>> <<<<<<<<
>>>>>>>>><<<<<<<<<
```

Bằng cách sử dụng phương thức writeChars, chúng ta có thể viết một phiên bản tốt hơn của phương thức drawTop từ chương trình DrawFigure2 của Chương 2. Chúng ta đã thấy rằng việc sử dụng

writeSpaces có thể loại bỏ hai trong số các vòng lặp bên trong, nhưng điều này vẫn để lại vòng lặp bên trong để in các dấu chấm. Bây giờ, chúng ta có thể loại bỏ cả ba vòng lặp bên trong và tạo ra một phiên bản dễ đọc hơn nhiều của phương thức:

```
public static void drawTop() {
    for (int line = 1; line <= SUB_HEIGHT; line++) {
        System.out.print("|");
        writeChars(' ', line - 1);
        System.out.print("\\");
        writeChars('.', 2 * SUB_HEIGHT - 2 * line);
        System.out.print("/");
        writeChars(' ', line - 1);
        System.out.println("|");
    }
}
```

Bạn có thể bao gồm bao nhiêu tham số tùy thích khi định nghĩa một phương thức. Mỗi lần gọi phương thức phải cung cấp chính xác số lượng tham số đó, theo thứ tự tương ứng. Ví dụ, hãy xem xét lệnh gọi đầu tiên của writeChars trong đoạn mã trước đó và phần khai báo cho writeChars. Java xếp các tham số theo thứ tự tuần tự (với tham số thực đầu tiên đi vào tham số hình thức đầu tiên và tham số thực thứ hai đi vào tham số hình thức thứ hai).



3.1-16 Văn bản thay thế đầy đủ

Chúng ta đã thấy rằng các phương thức có thể gọi các phương thức khác; điều này cũng đúng với các phương thức có tham số. Ví dụ, dưới đây là một phương thức để vẽ một hộp với chiều cao và chiều rộng cho trước bằng cách gọi phương thức writeChars:

```
public static void drawBox(int height, int width) {
```

```
    // Vẽ đỉnh hộp
    writeChars('*', width);
    System.out.println();

    // Vẽ các dòng giữa
    for (int i = 1; i <= height - 2; i++) {
        System.out.print('*');
        writeChars(' ', width - 2);
        System.out.println("*");
    }

    // Vẽ đáy hộp
    writeChars('*', width);
    System.out.println();
}
```

Lưu ý rằng phương thức drawBox được truyền các giá trị cho tham số height và width, và các tham số này được sử dụng để tạo các biểu thức được truyền vào writeChars. Ví dụ, bên trong vòng lặp for, chúng ta gọi writeChars và yêu cầu nó tạo ra width - 2 khoảng trắng. (Chúng ta trừ 2 vì in một dấu sao ở đầu và cuối dòng.) Dưới đây là một ví dụ gọi phương thức:

```
drawBox(5, 10);
```

Mã này tạo ra kết quả sau:

```
**********
*        *
*        *
*        *
**********
```

Khi viết các phương thức chấp nhận nhiều tham số, phần khai báo phương thức có thể trở nên rất dài. Thông thường, chúng ta ngắt dòng (những dòng vượt quá khoảng 80 ký tự) bằng cách chèn một dấu xuống dòng sau một toán tử hoặc tham số và thụt

lề dòng tiếp theo bằng cách gấp đôi chiều rộng thụt lề thông thường:

```
// phần khai báo phương thức này quá dài, vì vậy chúng ta sẽ ngắt dòng
public static void printTriangle(int xCoord1, int yCoord1,
        int xCoord2, int yCoord2, int xCoord3, int yCoord3) {
    ...
}
```

Tham Số so với Hằng Số

Trong Chương 2, bạn đã thấy rằng các hằng số lớp là một cơ chế hữu ích để tăng tính linh hoạt cho chương trình của bạn. Bằng cách sử dụng các hằng số này, bạn có thể dễ dàng sửa đổi chương trình để hoạt động khác đi. Tham số cung cấp nhiều tính linh hoạt hơn, và nhiều hơn thế nữa. Hãy xem xét phương thức writeSpaces. Giả sử bạn viết nó bằng cách sử dụng một hằng số lớp:

```
public static final int NUMBER_OF_SPACES = 10;
```

Cách tiếp cận này sẽ cho phép bạn tạo ra số lượng khoảng trắng khác nhau, nhưng nó có một hạn chế lớn: Hằng số chỉ có thể thay đổi từ lần thực thi này sang lần thực thi khác; nó không thể thay đổi trong một lần thực thi duy nhất. Nói cách khác, bạn có thể thực thi chương trình một lần với một giá trị, chỉnh sửa chương trình, biên dịch lại, và sau đó thực thi nó một lần nữa với một giá trị khác, nhưng bạn không thể sử dụng các giá trị khác nhau trong một lần thực thi chương trình sử dụng hằng số lớp. Tham số linh hoạt hơn. Bởi vì bạn chỉ định giá trị sẽ được sử dụng mỗi khi gọi phương thức, bạn có thể sử dụng nhiều giá trị khác nhau trong một lần thực thi chương trình. Như bạn đã thấy, bạn có thể gọi phương thức nhiều lần khác nhau trong một lần thực thi chương trình và để nó hoạt động khác nhau mỗi lần. Tuy nhiên, việc sử dụng tham số yêu cầu lập trình viên phải làm nhiều việc hơn so với việc sử dụng hằng số lớp. Nó làm cho phần khai

báo phương thức và các lệnh gọi phương thức trở nên tẻ nhạt hơn, chưa kể đến việc làm cho quá trình thực thi (và do đó, gỡ lỗi) trở nên phức tạp hơn.
Vì vậy, bạn có thể sẽ tìm thấy cơ hội để sử dụng mỗi kỹ thuật. Quy tắc cơ bản là sử dụng một hằng số lớp khi bạn chỉ muốn thay đổi giá trị từ lần thực thi này sang lần thực thi khác. Nếu bạn muốn sử dụng các giá trị khác nhau trong một lần thực thi, hãy sử dụng tham số.

Quá Tải Phương Thức

Bạn thường muốn tạo ra các biến thể nhẹ của cùng một phương thức bằng cách truyền các tham số khác nhau. Ví dụ, bạn có thể có một phương thức drawBox cho phép bạn chỉ định chiều cao và chiều rộng cụ thể, nhưng cũng có thể bạn muốn có một phiên bản khác vẽ một hộp với kích thước mặc định. Nói cách khác, đôi khi bạn muốn chỉ định các giá trị này, như trong:

```
drawBox(8, 10);
```

và những lúc khác bạn chỉ muốn vẽ một hộp với chiều cao và chiều rộng tiêu chuẩn:

```
drawBox();
```

Một số ngôn ngữ lập trình yêu cầu bạn phải đặt tên khác nhau cho các phiên bản này, chẳng hạn như drawBox và drawDefaultBox. Như bạn có thể tưởng tượng, việc nghĩ ra tên mới cho mỗi biến thể sẽ trở nên tẻ nhạt. May mắn thay, Java cho phép bạn có nhiều phương thức với cùng một tên, miễn là chúng có các tham số khác nhau. Điều này được gọi là **quá tải** (overloading). Yêu cầu chính cho việc quá tải là các phương thức khác nhau mà bạn định nghĩa phải có **chữ ký phương thức** khác nhau.

Quá Tải Phương Thức

Khả năng định nghĩa hai hoặc nhiều phương thức khác nhau với cùng một tên nhưng có chữ ký phương thức khác nhau.

Chữ Ký Phương Thức
Tên của một phương thức, cùng với số lượng và kiểu của các tham số.
Hai phiên bản drawBox trong ví dụ rõ ràng có chữ ký phương thức khác nhau, vì một phiên bản có hai tham số và phiên bản kia có không có tham số. Điều này sẽ rõ ràng từ bất kỳ lệnh gọi nào của phương thức xem nên sử dụng phiên bản nào: Nếu bạn thấy hai tham số, bạn sẽ thực thi phiên bản có hai tham số; nếu bạn thấy không có tham số, bạn sẽ thực thi phiên bản không có tham số.
Tình huống sẽ trở nên phức tạp hơn khi quá tải liên quan đến cùng số lượng tham số, nhưng điều này lại là một trong những ứng dụng hữu ích nhất của quá tải. Ví dụ, phương thức println thực sự là một chuỗi các phương thức quá tải. Chúng ta có thể gọi println truyền vào một chuỗi (String), một số nguyên (int), một số thực (double), v.v. Tính linh hoạt này được triển khai dưới dạng một chuỗi các phương thức khác nhau, tất cả đều nhận một tham số: Một phiên bản nhận một chuỗi, một phiên bản khác nhận một số nguyên, một phiên bản khác nhận một số thực, v.v. Rõ ràng, bạn sẽ làm những điều hơi khác nhau để in một trong các loại dữ liệu này so với loại khác, đó là lý do tại sao nó hữu ích khi có các phiên bản khác nhau của phương thức này.

3.2 Các Phương Thức Trả Về Giá Trị
Các phương thức mà chúng ta đã xem xét gần đây là các phương thức hành động, thực hiện một số nhiệm vụ cụ thể. Bạn có thể nghĩ chúng như những lệnh mà bạn có thể đưa ra cho ai đó, chẳng hạn như "Vẽ một hộp" hoặc "Vẽ một tam giác." Tham số cho phép các lệnh này linh hoạt hơn, chẳng hạn như "Vẽ một hộp có kích thước 10x20."
Bạn cũng sẽ muốn viết các phương thức tính toán giá trị. Các phương thức này giống như các câu hỏi, chẳng hạn như "Căn bậc hai của 2.5 là bao nhiêu?" hoặc "Kết quả khi bạn lũy thừa 2.3

lên mũ 4 là gì?" Hãy xem xét, ví dụ, một phương thức gọi là sqrt có nhiệm vụ tính căn bậc hai của một số.
Có thể bạn nghĩ rằng cách viết một phương thức như vậy sẽ là để nó chấp nhận một tham số kiểu double và in căn bậc hai của nó ra màn hình console. Nhưng bạn có thể muốn sử dụng căn bậc hai này như một phần của một biểu thức hoặc phép tính lớn hơn, chẳng hạn như giải phương trình bậc hai hoặc tính khoảng cách giữa các điểm trên mặt phẳng x/y.
Giải pháp tốt hơn sẽ là một lệnh tính căn bậc hai truyền vào số cần tính làm tham số và trả về kết quả căn bậc hai của nó cho chương trình. Bạn có thể sử dụng kết quả này như một phần của biểu thức, lưu trữ nó trong một biến, hoặc in nó ra màn hình console. Một lệnh như vậy là một loại phương thức mới được gọi là **phương thức trả về giá trị**.

Trả Về Giá Trị
Để gửi một giá trị ra ngoài như kết quả của một phương thức có thể được sử dụng trong một biểu thức trong chương trình của bạn. Phương thức void không trả về bất kỳ giá trị nào.
Nếu bạn có một phương thức như vậy, bạn có thể yêu cầu căn bậc hai của 2.5 bằng cách viết mã như sau:

```
// giả sử bạn có một phương thức gọi là sqrt
double answer = sqrt(2.5);
```

Phương thức sqrt có một tham số (số mà bạn muốn tìm căn bậc hai), và nó cũng trả về một giá trị (căn bậc hai). Tham số thực 2.5 "đi vào" phương thức, và căn bậc hai được đưa ra. Trong đoạn mã trên, kết quả trả về được lưu trữ trong một biến gọi là answer.
Bạn có thể biết một phương thức có trả về giá trị hay không bằng cách nhìn vào tiêu đề của nó. Tất cả các phương thức mà bạn đã viết cho đến nay đều bắt đầu với public static void. Từ void được biết đến như là kiểu trả về của phương thức.
Kiểu trả về void hơi kỳ lạ vì, như tên của nó ngụ ý, phương thức không trả về gì cả. Một phương thức có thể trả về bất kỳ kiểu hợp

pháp nào: một int, một double, hoặc bất kỳ kiểu nào khác. Trong trường hợp của phương thức sqrt, bạn muốn nó trả về một double, vì vậy bạn sẽ viết tiêu đề của nó như sau:

```
public static double sqrt(double n)
```

Giống như trường hợp trước đó, từ đi sau public static là kiểu trả về của phương thức:

```
public static
double
⌣

return type
sqrt(double n)
```

May mắn thay, bạn không thực sự cần phải viết một phương thức để tính căn bậc hai của một số, vì Java đã có sẵn một phương thức như vậy. Phương thức này được bao gồm trong một lớp gọi là Math bao gồm nhiều phương thức tính toán hữu ích. Vì vậy, trước khi chúng ta thảo luận chi tiết về cách viết các phương thức trả về giá trị, hãy khám phá lớp Math và những gì nó cung cấp.

Lớp Math

Trong Chương 1, chúng ta đã đề cập rằng rất nhiều mã được định nghĩa trước, được gọi chung là các thư viện lớp Java, đã được viết cho Java. Một trong những lớp hữu ích nhất là Math. Nó bao gồm các hằng số toán học được định nghĩa trước và một số lượng lớn các hàm toán học phổ biến. Lớp Math sẽ có sẵn trên bất kỳ máy nào mà Java được cài đặt đúng cách.

Như chúng ta đã lưu ý ở phần trước, lớp Math có một phương thức gọi là sqrt tính căn bậc hai của một số. Phương thức này có tiêu đề như sau:

```
public static double sqrt(double n)
```

Tiêu đề này cho biết rằng phương thức được gọi là sqrt, nó nhận một tham số kiểu double, và nó trả về một giá trị kiểu double.

Thật không may, bạn không thể gọi trực tiếp phương thức này bằng cách tham chiếu đến nó như sqrt vì nó nằm trong một lớp khác. Bất cứ khi nào bạn muốn tham chiếu đến một thứ gì đó được khai báo trong một lớp khác, bạn sử dụng ký pháp dấu chấm:

```
<class name>.<element>
```

Vì vậy, bạn sẽ tham chiếu đến phương thức này như là Math.sqrt. Dưới đây là một chương trình mẫu sử dụng phương thức này:

```
1  // Tính căn bậc hai bằng cách sử dụng lớp Math.
2  public class WriteRoots {
3      public static void main(String[] args) {
4          for (int i = 1; i <= 20; i++) {
5              double root = Math.sqrt(i);
6              System.out.println("sqrt(" + i + ") = " + root);
7          }
8      }
9  }
```

Chương trình này tạo ra kết quả sau:

```
sqrt(1) = 1.0
sqrt(2) = 1.4142135623730951
sqrt(3) = 1.7320508075688772
sqrt(4) = 2.0
sqrt(5) = 2.23606797749979
sqrt(6) = 2.449489742783178
sqrt(7) = 2.6457513110645907
sqrt(8) = 2.8284271247461903
sqrt(9) = 3.0
sqrt(10) = 3.1622776601683795
sqrt(11) = 3.3166247903554
sqrt(12) = 3.4641016151377544
sqrt(13) = 3.605551275463989
sqrt(14) = 3.7416573867739413
```

```
sqrt(15) = 3.872983346207417
sqrt(16) = 4.0
sqrt(17) = 4.123105625617661
sqrt(18) = 4.242640687119285
sqrt(19) = 4.358898943540674
sqrt(20) = 4.47213595499958
```

Chú ý rằng chúng ta đã truyền một giá trị kiểu int vào Math.sqrt, nhưng tiêu đề cho biết rằng nó mong đợi một giá trị kiểu double. Hãy nhớ rằng nếu Java đang mong đợi một double và nhận được một int, nó sẽ chuyển đổi int thành một double tương ứng.

Lớp Math cũng định nghĩa hai hằng số thường được sử dụng: E và π (xem Bảng 3.1). Theo quy ước của Java, chúng ta sử dụng tất cả các chữ cái viết hoa cho tên của chúng và tham chiếu đến chúng là Math.E và Math.PI.

Bảng 3.1 Các Hằng Số Toán Học

Hằng số	**Mô tả**
E	cơ sở dùng trong logarit tự nhiên (2.71828...)
PI	tỷ lệ giữa chu vi của một đường tròn với đường kính của nó (3.14159...)

Bảng 3.2 liệt kê một số phương thức tĩnh hữu ích nhất từ lớp Math. Bạn có thể xem danh sách đầy đủ các phương thức được định nghĩa trong lớp Math bằng cách kiểm tra tài liệu API cho phiên bản Java của bạn. API mô tả cách sử dụng các thư viện tiêu chuẩn có sẵn cho các lập trình viên Java. Nó có thể hơi choáng ngợp, vì các thư viện Java rất rộng lớn. Hãy dạo quanh một chút nếu bạn có hứng thú, nhưng đừng nản lòng vì có quá nhiều thư viện để lựa chọn trong Java.

Bảng 3.2 Các Phương Thức Tĩnh Hữu Ích Trong Lớp Math

Phương thức	**Mô tả**	**Ví dụ**
abs	giá trị tuyệt đối	Math.abs(-308) trả về 308

Phương thức	Mô tả	Ví dụ
ceil	làm tròn lên	Math.ceil(2.13) trả về 3.0
cos	cosine (radian)	Math.cos(Math.PI) trả về -1.0
exp	lũy thừa cơ số e	Math.exp(1) trả về 2.7182818284590455
floor	làm tròn xuống	Math.floor(2.93) trả về 2.0
log	logarit cơ số e	Math.log(Math.E) trả về 1.0
log10	logarit cơ số 10	Math.log10(1000) trả về 3.0
max	giá trị lớn nhất giữa hai giá trị	Math.max(45, 207) trả về 207
min	giá trị nhỏ nhất giữa hai giá trị	Math.min(3.8, 2.75) trả về 2.75
pow	lũy thừa	Math.pow(3, 4) trả về 81.0
random	giá trị ngẫu nhiên	Math.random() trả về một giá trị double ngẫu nhiên k sao cho 0.0 ≤ k < 1.0
round	làm tròn số thực tới số nguyên gần nhất	Math.round(2.718) trả về 3
sin	sine (radian)	Math.sin(0) trả về 0.0
sqrt	căn bậc hai	Math.sqrt(2) trả về 1.4142135623730951
toDegrees	chuyển đổi từ radian sang độ	Math.toDegrees(Math.PI) trả về 180.0
toRadians	chuyển đổi từ độ sang radian	Math.toRadians(270.0) trả về 4.71238898038469

Tương tác sau đây trong JShell minh họa một số phương thức Math:

```
jshell> Math.ceil(2.13)
$1 ==> 3.0

jshell> Math.pow(2, 5)
$2 ==> 32.0

jshell> Math.random()
$3 ==> 0.6051518556439361

jshell> Math.sin(Math.PI / 2)
$4 ==> 1.0

jshell> Math.pow(2, 2) + Math.pow(3, 2)
$5 ==> 13.0
```

Nếu bạn xem xét API Math, bạn sẽ nhận thấy rằng lớp Math có nhiều phương thức quá tải. Ví dụ, có một phiên bản của phương thức giá trị tuyệt đối (Math.abs) cho số nguyên và một phiên bản khác cho số double. Các quy tắc quyết định phương thức nào được gọi khá phức tạp, vì vậy chúng ta sẽ không bàn về chúng ở đây. Tuy nhiên, ý tưởng cơ bản là Java sẽ cố gắng tìm phương thức phù hợp nhất. Hầu hết thời gian, bạn không cần phải nghĩ nhiều về vấn đề này; bạn có thể để Java chọn cho bạn, và nó thường sẽ chọn đúng.

Định Nghĩa Các Phương Thức Trả Về Giá Trị

Bạn có thể tự viết các phương thức trả về giá trị bằng cách sử dụng một câu lệnh đặc biệt được gọi là câu lệnh return. Ví dụ, chúng ta thường sử dụng một phương thức trả về giá trị để biểu diễn một phương trình. Có một câu chuyện nổi tiếng về nhà toán học Carl Friedrich Gauss minh họa cho việc sử dụng một phương thức như vậy. Khi Gauss còn là một cậu bé, giáo viên của ông yêu cầu cả lớp cộng tất cả các số nguyên từ 1 đến 100, nghĩ rằng điều này sẽ mất một thời gian dài để hoàn thành. Gauss ngay lập

tức tìm ra một công thức và đưa ra câu trả lời cho giáo viên. Ông sử dụng một thủ thuật đơn giản là cộng hai bản sao của chuỗi lại với nhau, một bản theo thứ tự xuôi và một bản theo thứ tự ngược. Phương pháp này cho phép ông ghép đôi các giá trị từ hai bản sao sao cho tổng của chúng là như nhau:

Chuỗi đầu tiên	Chuỗi thứ hai	Tổng
1	100	101
2	99	101
3	98	101
4	97	101
. . .	. . .	. . .
100	1	101

Mỗi giá trị trong cột bên phải bằng 101 và có 100 hàng trong bảng này, do đó tổng chung là 100×101=10,100. Tất nhiên, đó là tổng của hai bản sao của chuỗi, vì vậy câu trả lời thực sự là một nửa số đó. Bằng cách sử dụng cách tiếp cận này, Gauss đã xác định rằng tổng của 100 số nguyên đầu tiên là 5050. Khi chuỗi đi từ 1 đến n, tổng sẽ là (n+1)×n/2.

Chúng ta có thể sử dụng công thức của Gauss để viết một phương thức tính tổng của n số nguyên đầu tiên:

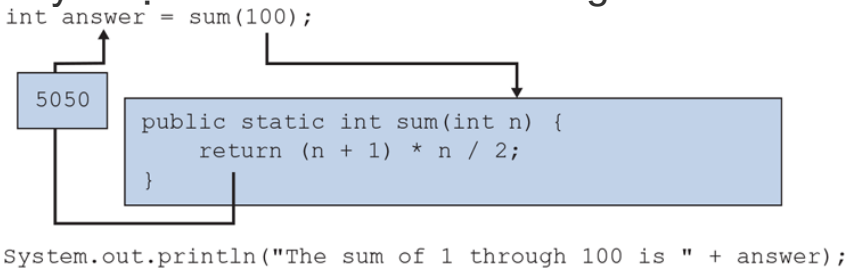
```
// trả về tổng của các số nguyên từ 1 đến N, bao gồm cả N
public static int sum(int n) {
    return (n + 1) * n / 2;
}
```

Phương thức sum có thể được sử dụng bởi phương thức main trong mã như sau:

```
int answer = sum(100);
System.out.println("Tổng của các số từ 1 đến 100 là " + answer);
```

Sơ đồ của những gì xảy ra khi mã này được thực thi như sau. Phương thức được gọi với tham số n được khởi tạo bằng 100.

Cắm giá trị này vào công thức, chúng ta có giá trị là 5050, giá trị này được trả về và lưu trữ trong biến có tên là answer.



3.2-19 Văn bản Thay thế Hoàn chỉnh

Lưu ý một lần nữa rằng trong phần khai báo của phương thức, từ khóa quen thuộc void (chỉ ra rằng không có giá trị trả về) đã được thay thế bằng từ int. Hãy nhớ rằng khi bạn khai báo một phương thức trả về một giá trị, bạn phải cho Java biết kiểu giá trị mà nó sẽ trả về. Do đó, chúng ta có thể cập nhật mẫu cú pháp của phương thức tĩnh một lần nữa để làm rõ rằng phần khai báo bao gồm kiểu trả về (void nếu không có):

```
public static <type> <name>(<type> <name>, ..., <type> <name>)
{
   <statement>;
   <statement>;
   ...
   <statement>;
}
```

Cú pháp của câu lệnh return là:

```
return <expression>;
```

Khi Java gặp một câu lệnh return, nó sẽ đánh giá biểu thức đã cho và ngay lập tức kết thúc phương thức, trả về giá trị mà nó thu được từ biểu thức đó. Do đó, không hợp lệ khi có bất kỳ câu lệnh nào khác sau một câu lệnh return; return phải là câu lệnh cuối cùng trong phương thức của bạn. Cũng là lỗi nếu một phương thức Java có kiểu trả về khác void mà kết thúc mà không có câu lệnh return.

Có những ngoại lệ đối với các quy tắc trước đây, như bạn sẽ thấy sau này. Ví dụ, có thể một phương thức có nhiều hơn một câu lệnh return; điều này sẽ được đề cập trong chương tiếp theo, khi chúng ta thảo luận về việc thực thi điều kiện bằng cách sử dụng câu lệnh if và if/else.

Hãy xem một ví dụ khác về phương thức trả về giá trị. Trong "Phù thủy xứ Oz", Bù nhìn sau khi nhận được bằng tốt nghiệp đã thể hiện trí thông minh của mình bằng cách mô tả một công thức sử dụng tổng căn bậc hai của hai cạnh bất kỳ của một tam giác cân. Có lẽ ông ấy đang cố gắng nêu định lý Pythagoras, mặc dù không rõ liệu các nhà biên kịch có tệ về toán học hay họ đang bình luận về giá trị của bằng cấp. Trong một tập phim của "The Simpsons", Homer đã lặp lại công thức sai của Bù nhìn sau khi đeo một cặp kính của Henry Kissinger mà anh ta tìm thấy trong phòng tắm tại nhà máy điện hạt nhân Springfield.

Định lý Pythagoras đúng đề cập đến các tam giác vuông và nói rằng độ dài cạnh huyền của một tam giác vuông bằng căn bậc hai của tổng các bình phương của hai cạnh còn lại. Nếu bạn biết độ dài của hai cạnh a và b của một tam giác vuông và muốn tìm độ dài cạnh thứ ba c, bạn có thể tính nó như sau:

```
c = Math.sqrt(a * a + b * b);
```

LỖI LẬP TRÌNH PHỔ BIẾN Bỏ qua Giá Trị Trả Về

Khi bạn gọi một phương thức trả về giá trị, kỳ vọng là bạn sẽ làm gì đó với giá trị được trả về. Bạn có thể in ra nó, lưu trữ nó trong một biến, hoặc sử dụng nó như một phần của biểu thức lớn hơn. Việc gọi phương thức và bỏ qua giá trị được trả về từ nó là hợp pháp (nhưng không khôn ngoan):

```
sum(1000);   // không làm gì cả
```

Lệnh gọi trên không in ra tổng hoặc không có bất kỳ hiệu ứng rõ rệt nào. Nếu bạn muốn in giá trị, bạn phải bao gồm câu lệnh println:

```
int answer = sum(1000);    // tốt hơn
```

```
System.out.println("Tổng đến 1000 là " + answer);
```

Một dạng ngắn hơn của mã sửa đổi sẽ là:

```
System.out.println("Tổng đến 1000 là " + sum(1000));
```

Giả sử bạn muốn in ra độ dài của các cạnh huyền của hai tam giác vuông, một với chiều dài cạnh là 5 và 12, và một cái khác với chiều dài cạnh là 3 và 4. Bạn có thể viết mã như sau:

```
double c1 = Math.sqrt(Math.pow(5, 2) + Math.pow(12, 2));
System.out.println("Cạnh huyền 1 = " + c1);

double c2 = Math.sqrt(Math.pow(3, 2) + Math.pow(4, 2));
System.out.println("Cạnh huyền 2 = " + c2);
```

Mã trên đúng, nhưng nó hơi khó đọc, và bạn sẽ phải lặp lại phép toán phức tạp tương tự một lần nữa nếu bạn muốn bao gồm tam giác thứ ba. Một giải pháp tốt hơn là tạo ra một phương thức tính toán và trả về độ dài cạnh huyền khi cho chiều dài của hai cạnh kia làm tham số. Phương thức như vậy sẽ trông như thế này:

```
public static double hypotenuse(double a, double b) {
    double c = Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
    return c;
}
```

Phương thức này có thể được sử dụng để tạo ra một phương thức chính ngắn gọn và dễ đọc hơn, như sau:

```
1 // Tính độ dài cạnh huyền của tam giác bằng cách sử dụng giá trị trả về.
2 public class Triangles {
3     public static void main(String[] args) {
4         System.out.println("Cạnh huyền 1 = " + hypotenuse(5, 12));
5         System.out.println("Cạnh huyền 2 = " + hypotenuse(3, 4));
6     }
7     public static double hypotenuse(double a, double b) {
8         return Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
```

```
9     }
10 }
```

Một vài biến thể của chương trình này có thể xảy ra. Ví dụ, không cần thiết phải lưu trữ giá trị trả về của phương thức hypotenuse vào biến c. Nếu bạn thích, bạn có thể tính toán và trả về giá trị trong một dòng. Trong trường hợp này, thân của phương thức hypotenuse sẽ trở thành:

```
return Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
```

Ngoài ra, một số lập trình viên tránh sử dụng Math.pow cho các lũy thừa nhỏ như 2 và chỉ thực hiện nhân tay. Sử dụng cách tiếp cận đó, thân của phương thức hypotenuse sẽ trông như thế này:

```
return Math.sqrt(a * a + b * b);
```

LỖI LẬP TRÌNH PHỔ BIẾN

Câu lệnh sau Return

Việc đặt các câu lệnh khác ngay sau một câu lệnh return là không hợp pháp, vì những câu lệnh đó sẽ không bao giờ được truy cập hoặc thực thi. Những lập trình viên mới thường vô tình làm điều này khi cố gắng in giá trị của một biến sau khi trả về. Giả sử bạn đã viết phương thức hypotenuse nhưng vô tình viết các tham số cho Math.pow theo thứ tự sai, vì vậy phương thức không đưa ra kết quả đúng. Bạn sẽ cố gắng gỡ lỗi bằng cách in giá trị của c đang được trả về. Dưới đây là mã lỗi:

```
// cố gắng tìm lỗi trong phiên bản lỗi của hypotenuse

public static double hypotenuse(double a, double b) {
    double c = Math.sqrt(Math.pow(2, a) + Math.pow(2, b));
    return c;

    System.out.println(c);  // điều này không hoạt động
}
```

Trình biên dịch sẽ báo lỗi về câu lệnh println vì không thể truy cập được, do nó nằm sau một câu lệnh return. Lỗi trình biên dịch sẽ trông như thế này:

```
Triangles.java:10: error: unreachable statement
    System.out.println(c);
    ^

Triangles.java:11: error: missing return statement
    }
    ^

2 errors
```

Cách sửa là di chuyển câu lệnh println lên trước câu lệnh return trong phương thức:

```
public static double hypotenuse(double a, double b) {
    double c = Math.sqrt(Math.pow(2, a) + Math.pow(2, b));
    System.out.println(c);  // tốt hơn
    return c;
}
```

3.3 Sử Dụng Đối Tượng

Chúng ta đã dành khá nhiều thời gian để thảo luận về các kiểu dữ liệu nguyên thủy trong Java và cách chúng hoạt động, nên bây giờ là lúc chúng ta bắt đầu nói về đối tượng và cách chúng hoạt động.

Ý tưởng về đối tượng xuất phát từ việc quan sát rằng khi chúng ta bắt đầu làm việc với các loại dữ liệu mới (số nguyên, số thực, ký tự, văn bản, v.v.), chúng ta thường viết rất nhiều phương thức để thao tác trên dữ liệu đó. Thay vì tách rời hoàn toàn các thao tác cơ bản khỏi dữ liệu, có vẻ hợp lý hơn khi kết hợp chúng lại với nhau. Việc đóng gói dữ liệu và các thao tác vào một thực thể duy nhất là ý tưởng trung tâm của đối tượng. Một đối tượng lưu

trữ một số dữ liệu và có các phương thức hành động trên dữ liệu của nó.

Đối Tượng
Một thực thể lập trình chứa trạng thái (dữ liệu) và hành vi (phương thức).
Như chúng ta đã nói trong Chương 1, các lớp là các khối xây dựng cơ bản của các chương trình Java. Nhưng các lớp cũng phục vụ một mục đích khác: để mô tả các loại đối tượng mới.

Lớp
Một danh mục hoặc loại đối tượng.
Khi được sử dụng theo cách này, một lớp giống như bản thiết kế của đối tượng sẽ trông như thế nào. Một khi bạn đã cung cấp cho Java bản thiết kế, bạn có thể yêu cầu nó tạo ra các đối tượng thực tế phù hợp với bản thiết kế đó. Chúng ta đôi khi gọi các đối tượng riêng lẻ này là các thể hiện của lớp. Chúng ta có xu hướng sử dụng các từ “thể hiện” và “đối tượng” thay thế cho nhau.
Khái niệm này khó hiểu trong trừu tượng. Để giúp bạn hiểu được nó có nghĩa là gì và nó hoạt động như thế nào, chúng ta sẽ xem xét một số lớp khác nhau. Tuân thủ theo ý tưởng tập trung vào các khái niệm cơ bản trước, trong chương này chúng ta sẽ nghiên cứu cách sử dụng các đối tượng hiện có mà đã là một phần của Java, nhưng chúng ta chưa học cách định nghĩa các loại đối tượng mới của riêng mình. Chúng ta sẽ thực hiện điều đó trong Chương 8, sau khi chúng ta đã có thời gian thực hành sử dụng các đối tượng.
Sử dụng đối tượng khác với sử dụng các kiểu nguyên thủy, vì vậy chúng ta sẽ phải giới thiệu một số cú pháp và khái niệm mới. Sẽ tốt hơn nếu Java có một mô hình nhất quán để sử dụng tất cả các loại dữ liệu, nhưng nó không có. Do đó, nếu bạn muốn hiểu cách chương trình của mình hoạt động, bạn sẽ phải học hai bộ quy tắc: một cho các kiểu nguyên thủy và một cho các đối tượng.

Đối Tượng String

Đối tượng String là một trong những loại đối tượng hữu ích và được sử dụng phổ biến nhất trong Java, vì vậy chúng là một điểm khởi đầu tốt. Tuy nhiên, chúng không phải là ví dụ tốt nhất về đối tượng, vì có rất nhiều quy tắc đặc biệt chỉ áp dụng cho các chuỗi. Trong phần tiếp theo, chúng ta sẽ xem xét một loại đối tượng điển hình hơn.

Một tính chất đặc biệt của đối tượng String là có các hằng số biểu diễn chúng (hằng chuỗi). Chúng ta đã sử dụng chúng trong các câu lệnh println từ Chương 1. Điều mà chúng ta chưa thảo luận là các giá trị hằng số này biểu diễn các đối tượng thuộc loại String (các thể hiện của lớp String). Ví dụ, theo cách mà bạn có thể nói:

```
int x = 8;
```

Bạn cũng có thể nói:

```
String s = "hello there";
```

Bạn có thể khai báo các biến thuộc loại String và sử dụng câu lệnh gán để gán giá trị cho các biến này. Bạn cũng có thể viết mã liên quan đến các biểu thức String:

```
String s1 = "hello";
String s2 = "there";
String combined = s1 + " " + s2;
```

Đoạn mã này định nghĩa hai chuỗi, mỗi chuỗi biểu diễn một từ, và một chuỗi thứ ba biểu diễn sự kết hợp của hai từ với một khoảng trắng ở giữa. Bạn sẽ nhận thấy rằng kiểu String được viết hoa (như tên của tất cả các loại đối tượng trong Java), không giống như các kiểu nguyên thủy như double và int.

Những ví dụ này chưa cho thấy điều gì đặc biệt về đối tượng String, nhưng chúng ta đang dần tiếp cận điều đó. Hãy nhớ rằng ý tưởng đằng sau đối tượng là kết hợp các thao tác cơ bản với chính dữ liệu, giống như cách chúng ta chế tạo ô tô với các bộ điều khiển tích hợp sẵn. Dữ liệu được lưu trữ trong một đối tượng String là một chuỗi các ký tự. Có rất nhiều thao tác mà bạn có thể

muốn thực hiện trên chuỗi các ký tự này. Ví dụ, bạn có thể muốn biết có bao nhiêu ký tự trong chuỗi đó. Đối tượng String có một phương thức length trả về thông tin này.
Nếu phương thức length là tĩnh, bạn sẽ gọi nó bằng cách nói như sau:

```
length(s);  // điều này không hợp lệ
```

Nhưng khi bạn thực hiện các thao tác trên đối tượng, bạn sử dụng một cú pháp khác. Các đối tượng lưu trữ dữ liệu và phương thức, vì vậy phương thức để báo cáo độ dài của chuỗi thực sự tồn tại bên trong đối tượng String đó. Để gọi một phương thức của đối tượng, bạn viết tên của biến trước, sau đó là dấu chấm, và sau đó là tên của phương thức:

```
s.length();
```

Hãy nghĩ về điều này như việc nói chuyện với đối tượng String. Khi bạn hỏi s.length(), bạn đang nói: “Này, s. Tôi đang nói chuyện với bạn. Độ dài của bạn là bao nhiêu?” Tất nhiên, các đối tượng String khác nhau có độ dài khác nhau, vì vậy bạn sẽ nhận được các câu trả lời khác nhau khi giao tiếp với các đối tượng String khác nhau.
Cú pháp chung để gọi một phương thức của một đối tượng là:

```
<variable>.<method name>(<expression>, <expression>, ..., <expression>);
```

Ví dụ, tương tác JShell sau tạo ra hai biến String và yêu cầu mỗi biến cho biết độ dài của nó:

```
jshell> String s1 = "hello";
s1 ==> "hello";

jshell> String s2 = "how are you?";
s2 ==> "how are you?";

jshell> s1.length();
$3 ==> 5;
```

```
jshell> s2.length();
$4 ==> 12;
```

Bạn có thể muốn làm gì khác với một đối tượng String? Với phương thức length, bạn có thể biết có bao nhiêu ký tự trong một chuỗi, nhưng còn việc lấy các ký tự riêng lẻ thì sao? Có một số cách để thực hiện điều này, nhưng một trong những cách phổ biến nhất là sử dụng một phương thức gọi là charAt trả về ký tự tại một vị trí cụ thể trong chuỗi.

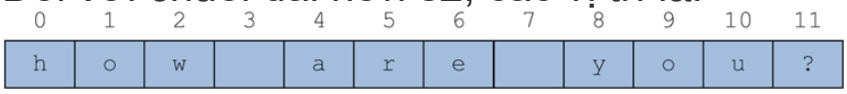
Điều này dẫn chúng ta đến vấn đề về cách xác định vị trí trong một chuỗi. Rõ ràng là có ký tự đầu tiên, ký tự thứ hai, v.v., nên có lý khi sử dụng một số nguyên để chỉ định một vị trí cụ thể. Chúng ta gọi đây là chỉ số (index).

Chỉ Số (Index)

Một số nguyên được sử dụng để chỉ định một vị trí trong một chuỗi giá trị. Java thường sử dụng đánh chỉ số bắt đầu từ 0 (với 0 là giá trị chỉ số đầu tiên, sau đó là 1, 2, 3, v.v.). Mỗi ký tự của một đối tượng String được gán một giá trị chỉ số, bắt đầu từ 0. Ví dụ, đối với biến s1 tham chiếu đến chuỗi "hello", các chỉ số là:

Có thể trực quan khi cho rằng chữ cái “h” ở vị trí 1, nhưng có những lợi ích khi bắt đầu với chỉ số 0. Đây là một quy ước đã được các nhà thiết kế ngôn ngữ C áp dụng và đã được các nhà thiết kế ngôn ngữ C++ và Java tuân theo, vì vậy đây là một quy ước mà bạn sẽ phải làm quen.

Đối với chuỗi dài hơn s2, các vị trí là:



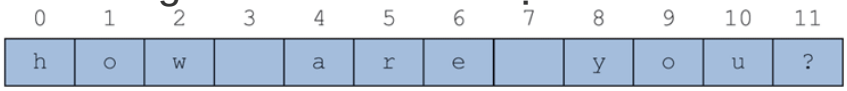
Lưu ý rằng các khoảng trắng trong chuỗi này cũng có vị trí (ở đây là vị trí 3 và 7). Cũng lưu ý rằng các chỉ số của một chuỗi luôn nằm trong khoảng từ 0 đến một giá trị nhỏ hơn độ dài của chuỗi. Sử dụng phương thức charAt, bạn có thể yêu cầu các ký tự cụ thể của một chuỗi. Kiểu trả về là char. Ví dụ, nếu bạn yêu cầu

s1.charAt(1) bạn sẽ nhận được 'e' (chữ ‘e’ trong “hello”). Nếu bạn yêu cầu s2.charAt(5), bạn sẽ nhận được 'r' (chữ ‘r’ trong “how are you?”). Đối với bất kỳ chuỗi String nào, nếu bạn yêu cầu charAt(0), bạn sẽ nhận được ký tự đầu tiên của chuỗi.
Khi làm việc với các đối tượng String, bạn thường thấy hữu ích khi viết một vòng lặp for để xử lý các ký tự khác nhau của chuỗi. Bởi vì các chuỗi được đánh chỉ số bắt đầu từ 0, việc này dễ viết hơn với các vòng lặp for bắt đầu từ 0 thay vì 1. Xem xét, ví dụ, tương tác JShell sau đây in ra các ký tự riêng lẻ của s1:

```
jshell> String s1 = "hello";
s1 ==> "hello";

jshell> for (int i = 0; i < s1.length(); i++) {
        System.out.println(i + ": " + s1.charAt(i));
        }

0: h
1: e
2: l
3: l
4: o
```

Nhớ rằng khi chúng ta bắt đầu vòng lặp từ 0, chúng ta thường kiểm tra với dấu nhỏ hơn < thay vì nhỏ hơn hoặc bằng <=. Chuỗi s1 có năm ký tự, vì vậy lệnh gọi s1.length() sẽ trả về 5. Nhưng vì chỉ số đầu tiên là 0, chỉ số cuối cùng sẽ là một giá trị nhỏ hơn 5, tức là 4. Quy ước này mất một thời gian để làm quen, nhưng đánh chỉ số bắt đầu từ 0 được sử dụng trong suốt Java, vì vậy cuối cùng bạn sẽ nắm vững nó.
Một phương thức String hữu ích khác là substring. Nó nhận hai tham số nguyên đại diện cho chỉ số bắt đầu và kết thúc. Khi bạn gọi phương thức substring, bạn cung cấp hai chỉ số này: chỉ số của ký tự đầu tiên mà bạn muốn và chỉ số ngay sau chỉ số cuối cùng mà bạn muốn.

Nhớ rằng chuỗi s2 có các vị trí như sau:



3.3-22 Văn bản thay thế đầy đủ

Giả sử bạn muốn lấy từng từ trong chuỗi này. Để lấy từ đầu tiên, “how,” bạn sẽ yêu cầu:

```
jshell> String s2 = "how are you?";
s2 ==> "how are you?";

jshell> s2.substring(0, 3);
$2 ==> "how";
```

Nhớ rằng giá trị thứ hai mà bạn truyền vào phương thức substring phải là một vị trí ngay sau phần cuối của chuỗi con mà bạn đang tạo. Vì vậy, mặc dù có một khoảng trắng ở vị trí 3 trong chuỗi gốc, nó sẽ không phải là phần bạn nhận được từ lệnh gọi substring. Thay vào đó, bạn sẽ nhận được tất cả các ký tự ngay trước vị trí 3. Tuân theo quy tắc này nghĩa là đôi khi bạn sẽ đưa ra một vị trí cho substring mà tại đó không có ký tự nào. Ví dụ, ký tự cuối cùng trong chuỗi mà s2 tham chiếu đến là ở chỉ số 11 (dấu chấm hỏi). Nếu bạn muốn lấy chuỗi con “you?” bao gồm cả dấu chấm hỏi, bạn sẽ yêu cầu:

```
jshell> s2.substring(8, 12);
$3 ==> "you?";
```

Không có ký tự nào tại vị trí 12 trong s2, nhưng lệnh gọi này yêu cầu các ký tự bắt đầu từ vị trí 8 cho đến ngay trước vị trí 12, vì vậy điều này thực sự có ý nghĩa.

Bạn cần cẩn thận về các chỉ số mà bạn sử dụng. Với phương thức substring, bạn có thể yêu cầu vị trí ngay sau phần cuối của chuỗi, nhưng bạn không thể yêu cầu bất cứ điều gì vượt quá điều đó. Ví dụ, nếu bạn chỉ định 13 làm chỉ số kết thúc trong ví dụ trước, mã sẽ tạo ra một lỗi thực thi:

```
jshell> s2.substring(8, 13);
```

```
|  java.lang.StringIndexOutOfBoundsException thrown:
|  begin 8, end 13, length 12
|        at String.checkBoundsBeginEnd (String.java:3107)
|        at String.substring (String.java:1873)
|        at (#4:1);
```

Tương tự, nếu bạn yêu cầu charAt tại một vị trí không tồn tại, chương trình của bạn sẽ tạo ra một lỗi thực thi. Những lỗi này được gọi là ngoại lệ (exceptions). Ngoại lệ là các lỗi xảy ra khi chương trình đang chạy, như đã đề cập trong Chương 1.

Ngoại lệ (Exception)

Một lỗi xảy ra trong thời gian chạy (runtime error) ngăn chương trình tiếp tục thực thi bình thường.

Chúng ta nói rằng một ngoại lệ được ném ra khi gặp phải lỗi. Khi một ngoại lệ được ném ra, Java sẽ kiểm tra xem bạn có viết mã để xử lý nó hay không. Nếu không, chương trình sẽ dừng lại và bạn sẽ thấy một cái gọi là "stack trace" hoặc "back trace". "Stack trace" hiển thị chuỗi các phương thức đã được gọi, theo thứ tự ngược lại. Trong trường hợp chỉ số chuỗi không hợp lệ, ngoại lệ sẽ in ra một thông báo như sau trên bảng điều khiển:

```
Exception in thread "main"
    java.lang.StringIndexOutOfBoundsException:
    String index out of range: 13
    at java.lang.String.substring(Unknown Source)
    at ExampleProgram.main(ExampleProgram.java:3)
```

Bạn có thể sử dụng một chuỗi (String) làm tham số cho phương thức. Ví dụ, chương trình sau sử dụng các tham số chuỗi để in ra một thông điệp tùy chỉnh cho một người dựa trên tên của họ:

```
1  // Chương trình này minh họa cách sử dụng tham số String.
2  public class FormLetter {
3      public static void main(String[] args) {
4          letter("Philip", "Smith");
5          letter("Jean", "Geges");
```

```
6     }
7     public static void letter(String first, String last) {
8         System.out.println("Dear " + first + ",");
9         System.out.println("Check out this amazing offer! 20%");
10        System.out.println("off all items. A great deal, as");
11        System.out.println("sure as your name is " + last + "!");
12        System.out.println();
13    }
14 }
```

Nó tạo ra đầu ra như sau:

```
Dear Philip,
Check out this amazing offer! 20%
off all items. A great deal, as
sure as your name is Smith!

Dear Jean,
Check out this amazing offer! 20%
off all items. A great deal, as
sure as your name is Geges!
```

Bảng 3.3 liệt kê một số phương thức hữu ích mà bạn có thể gọi trên các đối tượng String.

Bảng 3.3: Các phương thức hữu ích của đối tượng String

Phương thức	**Mô tả**	**Ví dụ (giả sử s là "hello")**
charAt(index)	Ký tự tại một chỉ số cụ thể	s.charAt(1) trả về 'e'
endsWith(text)	Chuỗi có kết thúc bằng một văn bản nào đó hay không	s.endsWith("llo") trả về true
indexOf(text)	Vị trí của một ký tự hoặc chuỗi cụ thể (–1 nếu không có)	s.indexOf("o") trả về 4

Phương thức	Mô tả	Ví dụ (giả sử s là "hello")
length()	Số ký tự trong chuỗi	s.length() trả về 5
replace(s1, s2)	Thay thế tất cả các chuỗi con bằng một chuỗi khác	s.replace("l", "yy") trả về "heyyyyo"
startsWith(text)	Chuỗi có bắt đầu bằng một văn bản nào đó hay không	s.startsWith("hi") trả về false
substring(start, stop)	Các ký tự từ chỉ số bắt đầu đến ngay trước chỉ số kết thúc	s.substring(1, 3) trả về "el"
toLowerCase()	Tạo một chuỗi mới với tất cả các ký tự viết thường	s.toLowerCase() trả về "hello"
toUpperCase()	Tạo một chuỗi mới với tất cả các ký tự viết hoa	s.toUpperCase() trả về "HELLO"

Chuỗi trong Java là bất biến, có nghĩa là một khi đã được tạo, giá trị của chúng không thể thay đổi.

Đối tượng bất biến (Immutable Object)
Một đối tượng mà giá trị không thể thay đổi.
Có thể bạn sẽ thấy lạ khi các chuỗi là bất biến nhưng lại có các phương thức như toUpperCase và toLowerCase. Nhưng nếu bạn đọc kỹ mô tả trong bảng, bạn sẽ thấy rằng các phương thức này thực sự không thay đổi đối tượng String; thay vào đó, chúng trả về một chuỗi mới. Xem xét đoạn mã sau:

```
String s = "Hello, Maria";
s.toUpperCase();
System.out.println(s);
```

Bạn có thể nghĩ rằng điều này sẽ biến chuỗi s thành phiên bản viết hoa của nó, nhưng thực tế không phải vậy. Dòng mã thứ hai

tạo ra một chuỗi mới có phiên bản viết hoa của giá trị của s, nhưng chúng ta không làm gì với giá trị mới này. Để biến chuỗi thành viết hoa, bạn cần lưu chuỗi mới này vào một biến khác hoặc gán lại biến s để trỏ đến chuỗi mới:

```
String s = "Hello, Maria";
s = s.toUpperCase();
System.out.println(s);
```

Phiên bản này của mã tạo ra đầu ra như sau:

```
HELLO, MARIA
```

Các phương thức toUpperCase và toLowerCase rất hữu ích khi bạn muốn thực hiện các so sánh chuỗi mà bỏ qua sự khác biệt giữa các chữ cái viết hoa và viết thường.

Một phương thức hữu ích khác trong đối tượng String là replace. Phương thức này nhận hai tham số: một chuỗi để tìm kiếm, và một chuỗi mới để thay thế tất cả các lần xuất hiện của nó.

Đoạn tương tác JShell sau đây minh họa phương thức replace:

```
jshell> String name = "Tweedle Dee";
name ==> "Tweedle Dee";

jshell> name = name.replace("ee", "oo");
name ==> "Twoodle Doo";

jshell> name = name.replace("w", "");
name ==> "Toodle Doo";
```

Chương Trình Tương Tác và Đối Tượng Scanner

Như bạn đã thấy, bạn có thể dễ dàng tạo ra đầu ra trong cửa sổ console bằng cách gọi System.out.println và System.out.print. Bạn cũng có thể viết các chương trình tạm dừng và chờ người dùng nhập phản hồi. Những chương trình như vậy được gọi là chương trình tương tác, và các phản hồi do người dùng nhập vào được gọi là **đầu vào console**.

Đầu vào console

Phản hồi được người dùng nhập vào khi một chương trình tương tác tạm dừng để nhận đầu vào.

Khi bạn tham chiếu đến System.out, bạn đang truy cập một đối tượng trong lớp System được gọi là luồng đầu ra tiêu chuẩn, hoặc gọi tắt là “standard out”. Có một đối tượng tương ứng cho đầu vào tiêu chuẩn được gọi là System.in, nhưng Java không được thiết kế cho đầu vào console, và System.in chưa bao giờ thực sự dễ sử dụng cho mục đích này. May mắn thay, có một cách dễ dàng hơn để đọc đầu vào console: Đối tượng Scanner.

Hầu hết các đối tượng phải được tạo rõ ràng bằng cách gọi một phương thức đặc biệt được gọi là **constructor**.

Constructor (Xây dựng)

Cú pháp đặc biệt được sử dụng để tạo và khởi tạo một đối tượng. Các đối tượng trong chương trình Java phải được tạo trước khi chúng có thể được sử dụng.

Nhớ rằng một lớp giống như một bản thiết kế cho một nhóm đối tượng. Gọi một constructor giống như gửi một yêu cầu đến nhà máy yêu cầu nó làm theo bản thiết kế để tạo ra một đối tượng thực tế mà bạn có thể thao tác. Khi gửi yêu cầu đến nhà máy, đôi khi bạn cần chỉ định một số tham số (ví dụ, bạn muốn đối tượng có màu gì).

Trong Java, các constructor được gọi bằng từ khóa đặc biệt new, theo sau là kiểu của đối tượng và bất kỳ tham số cần thiết nào. Ví dụ, để tạo một đối tượng Scanner cụ thể, bạn phải cung cấp thông tin về nguồn đầu vào. Đặc biệt, bạn phải cung cấp một luồng đầu vào. Để đọc từ cửa sổ console, truyền System.in:

```
Scanner console = new Scanner(System.in);
```

Sau khi đã tạo đối tượng Scanner, bạn có thể yêu cầu nó trả về một giá trị của một kiểu cụ thể. Một số phương thức, tất cả đều bắt đầu bằng từ “next”, có sẵn để lấy các loại giá trị khác nhau. Bảng 3.4 liệt kê chúng.

Bảng 3.4: Các phương thức của Scanner

Phương thức	Mô tả
next()	Đọc và trả về token tiếp theo dưới dạng chuỗi
nextDouble()	Đọc và trả về một giá trị kiểu double
nextInt()	Đọc và trả về một giá trị kiểu int
nextLine()	Đọc và trả về dòng tiếp theo của đầu vào dưới dạng chuỗi

Thông thường, bạn sẽ sử dụng biến để theo dõi các giá trị được trả về bởi các phương thức này. Ví dụ, bạn có thể nói:

```
int n = console.nextInt();
```

Lệnh gọi phương thức nextInt của đối tượng console sẽ tạm dừng để nhận đầu vào từ người dùng. Bất cứ khi nào máy tính tạm dừng để nhận đầu vào, nó sẽ tạm dừng cho toàn bộ dòng đầu vào. Nói cách khác, nó sẽ chờ cho đến khi người dùng nhấn phím Enter trước khi tiếp tục thực thi chương trình.
Bạn có thể sử dụng lớp Scanner để đọc đầu vào từng dòng một bằng cách sử dụng phương thức nextLine, mặc dù hiện tại chúng ta sẽ không sử dụng nextLine nhiều. Các phương thức “next” khác đều dựa trên token (tức là chúng đọc các phần tử riêng lẻ của đầu vào thay vì toàn bộ dòng).

Token
Một phần tử duy nhất của đầu vào (ví dụ, một từ, một số).
Mặc định, Scanner sử dụng khoảng trắng để tách các token.

Khoảng trắng
Các ký tự khoảng trắng, tab, và ký tự xuống dòng.
Đối tượng Scanner xem xét những gì người dùng nhập và sử dụng khoảng trắng trên dòng đầu vào để chia nó thành các token riêng lẻ. Ví dụ, dòng đầu vào

```
hello     there. "how are"        "you?"  all-one-token
```

sẽ được chia thành sáu token:
sql
Copy code
hello
there.
"how
are"
"you?"
all-one-token

Chú ý rằng Scanner bao gồm các ký tự dấu câu như dấu chấm, dấu hỏi, và dấu ngoặc kép trong các token mà nó tạo ra. Nó cũng bao gồm dấu gạch ngang, vì vậy do không có khoảng trắng ở giữa để chia nó thành các token khác nhau, chúng ta chỉ nhận được một token cho “all-one-token”. Có thể kiểm soát cách mà một Scanner biến đổi các đầu vào thành các token (một quá trình được gọi là token hóa đầu vào), nhưng chúng ta sẽ không làm bất cứ điều gì phức tạp như vậy.
Có thể đọc nhiều hơn một giá trị từ một Scanner, như sau:

```
double x = console.nextDouble();
double y = console.nextDouble();
```

Vì có hai lệnh gọi khác nhau đến phương thức nextDouble của đối tượng console, đoạn mã này sẽ khiến máy tính tạm dừng cho đến khi người dùng nhập hai giá trị số. Các giá trị có thể được nhập trên cùng một dòng hoặc trên các dòng khác nhau. Nói chung, máy tính tiếp tục tạm dừng để nhận đầu vào từ người dùng cho đến khi nó lấy được các giá trị mà bạn đã yêu cầu Scanner lấy.
Nếu người dùng nhập vào một thứ gì đó không phải là một số nguyên khi bạn gọi nextInt, như “XYZZY”, đối tượng Scanner sẽ tạo ra một ngoại lệ. Nhớ lại từ phần về đối tượng String rằng ngoại lệ là các lỗi thời gian chạy dừng việc thực thi chương trình. Trong trường hợp này, bạn sẽ thấy đầu ra lỗi thời gian chạy như sau:

```
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Unknown Source)
    at java.util.Scanner.next(Unknown Source)
    at java.util.Scanner.nextInt(Unknown Source)
    at Example.main(Example.java:13)
```

Bạn sẽ thấy trong một chương sau cách kiểm tra lỗi của người dùng. Trong thời gian chờ đợi, chúng ta sẽ giả định rằng người dùng cung cấp đầu vào phù hợp.

Chương Trình Tương Tác Mẫu

Sử dụng lớp Scanner, chúng ta có thể viết một chương trình tương tác hoàn chỉnh thực hiện một phép tính hữu ích cho người dùng. Nếu bạn từng mua nhà, bạn sẽ muốn biết khoản thanh toán thế chấp hàng tháng của mình là bao nhiêu. Dưới đây là một chương trình hoàn chỉnh yêu cầu thông tin về khoản vay và in ra khoản thanh toán hàng tháng:

```
1    // Chương trình này yêu cầu thông tin về khoản vay và
2    // tính toán khoản thanh toán thế chấp hàng tháng.

4    import java.util.*; // sử dụng Scanner

6    public class Mortgage {

7        public static void main(String[] args) {
8            Scanner console = new Scanner(System.in);

10           // lấy các giá trị
11           System.out.println("Chương trình này tính toán khoản
thanh toán " +
12                              "thế chấp hàng tháng.");
13           System.out.print("Số tiền vay    : ");
14           double loan = console.nextDouble();
15           System.out.print("Số năm vay     : ");
```

```
16          int years = console.nextInt();
17          System.out.print("Lãi suất       : ");
18          double rate = console.nextDouble();
19          System.out.println();

21          // tính toán kết quả và báo cáo
22          int n = 12 * years;
23          double c = rate / 12.0 / 100.0;
24          double payment = loan * c * Math.pow(1 + c, n) /
25                          (Math.pow(1 + c, n) - 1);
26          System.out.println("Khoản thanh toán = $" + (int)
payment);
27      }
28  }
```

Dưới đây là một ví dụ thực thi chương trình (người dùng nhập dữ liệu được in đậm):

```
Chương trình này tính toán khoản thanh toán thế chấp hàng
tháng.
Số tiền vay    : **275000**
Số năm vay     : **30**
Lãi suất       : **4.25**

Khoản thanh toán = $1352
```

Điều đầu tiên chúng ta làm trong chương trình là tạo một đối tượng Scanner, mà chúng ta sẽ sử dụng để nhập dữ liệu từ console. Tiếp theo, chúng ta giải thích những gì chương trình sẽ làm, in mô tả ra console. Điều này rất quan trọng đối với các chương trình tương tác. Bạn không muốn chương trình tạm dừng để nhận dữ liệu từ người dùng mà không giải thích cho họ biết điều gì sẽ xảy ra.

Dưới đây là vài cặp lệnh như thế này:

```
System.out.print("Số tiền vay    : ");
double loan = console.nextDouble();
```

Lệnh đầu tiên gọi là **prompt**, một yêu cầu thông tin từ người dùng. Chúng ta sử dụng lệnh print thay vì println để người dùng nhập giá trị ngay trên cùng dòng với prompt (tức là ở bên phải của prompt). Lệnh thứ hai gọi phương thức nextDouble của đối tượng console để đọc một giá trị kiểu double từ người dùng. Giá trị này được lưu trữ trong một biến gọi là loan. Mẫu prompt/đọc lệnh này rất phổ biến trong các chương trình tương tác.
Sau khi yêu cầu các giá trị, chương trình tính toán một số giá trị. Công thức để tính toán khoản thanh toán thế chấp hàng tháng liên quan đến số tiền vay, tổng số tháng (gọi là n), và lãi suất hàng tháng (gọi là c). Công thức thanh toán được cho bởi phương trình sau:

```
payment = loan * c * (1 + c)^n / ((1 + c)^n - 1)
```

Bạn sẽ thấy trong chương trình rằng chúng ta sử dụng phương thức Math.pow để tính lũy thừa và chuyển công thức này thành biểu thức Java.
Dòng cuối cùng của chương trình in ra khoản thanh toán hàng tháng. Bạn có thể nghĩ rằng chúng ta chỉ cần nói:

```
System.out.println("Khoản thanh toán = $" + payment);
```

Tuy nhiên, vì khoản thanh toán được lưu trữ trong một biến kiểu double, câu lệnh như vậy sẽ in tất cả các chữ số của số đó. Ví dụ, đối với nhật ký trên, nó sẽ in ra:

```
Khoản thanh toán = $1352.8347004685593
```

Đó là một đầu ra khá kỳ lạ đối với một người đã quen với việc tính toán tiền đô la và xu. Đối với mục đích của chương trình đơn giản này, dễ dàng hơn là ép kiểu double sang int và chỉ báo cáo số tiền đô la của khoản thanh toán:

```
System.out.println("Khoản thanh toán = $" + (int) payment);
```

Hầu hết mọi người cố gắng tìm hiểu khoản thanh toán thế chấp của họ không quan tâm nhiều đến xu, do đó chương trình vẫn hữu ích. Trong phần tiếp theo, chúng ta sẽ xem cách làm tròn một số double đến hai chữ số thập phân.

Có một điều mới ở đầu tệp lớp này gọi là khai báo import. Nhớ rằng Java có một số lượng lớn các lớp được bao gồm trong những gì được gọi chung là thư viện lớp Java. Để giúp quản lý các lớp này, Java cung cấp một đơn vị tổ chức gọi là gói (package). Các lớp liên quan được kết hợp với nhau thành một gói duy nhất. Ví dụ, lớp Scanner được lưu trữ trong một gói gọi là java.util. Các chương trình Java thường không truy cập được vào một gói trừ khi chúng bao gồm khai báo import.

Gói (Package)

Một tập hợp các lớp Java liên quan.

Khai Báo Import

Yêu cầu truy cập vào một gói Java cụ thể. Chúng ta chưa cần khai báo import vì Java tự động import mọi lớp được lưu trữ trong một gói gọi là java.lang. Gói java.lang bao gồm các lớp cơ bản mà hầu hết các chương trình Java có khả năng sử dụng (ví dụ: System, String, Math). Vì Java không tự động import java.util, bạn phải tự làm điều này.

Java cho phép bạn sử dụng dấu sao để import tất cả các lớp từ một gói:

```
import java.util.*;
```

Nhưng một số người thích chỉ rõ từng lớp mà họ import. Khai báo import cho phép bạn chỉ import một lớp duy nhất từ một gói, như trong:

```
import java.util.Scanner;
```

Vấn đề là khi bạn bắt đầu import một lớp từ một gói, bạn có thể sẽ muốn import thêm các lớp khác. Chúng ta sẽ sử dụng phiên bản dấu sao của import trong cuốn sách này để giữ mọi thứ đơn giản.

3.4 Nghiên Cứu Trường Hợp: Quỹ Đạo Vật Thể

Đã đến lúc kết hợp các yếu tố của chương này với một ví dụ phức tạp hơn, sẽ liên quan đến các tham số, các phương thức trả về giá trị, các phép tính toán học, và việc sử dụng đối tượng Scanner để nhập liệu từ console.

Sinh viên ngành vật lý thường được yêu cầu tính toán quỹ đạo mà một vật thể sẽ theo, dựa trên vận tốc ban đầu và góc ban đầu của nó so với phương ngang. Ví dụ, vật thể có thể là một quả bóng mà ai đó đã đá. Chúng ta muốn tính toán đường đi của nó dựa trên lực hấp dẫn của Trái Đất. Để giữ cho phép tính đơn giản, chúng ta sẽ bỏ qua lực cản không khí.

Có một số câu hỏi liên quan đến vấn đề này mà chúng ta có thể muốn trả lời:

- Khi nào vật thể đạt đến điểm cao nhất?
- Nó đạt đến độ cao bao nhiêu?
- Mất bao lâu để nó quay lại mặt đất?
- Nó hạ cánh cách vị trí phóng bao xa?

Có một số cách để trả lời những câu hỏi này. Một cách đơn giản là cung cấp một bảng hiển thị quỹ đạo từng bước, chỉ ra vị trí x, vị trí y, và thời gian đã trôi qua.

Để tạo ra một bảng như vậy, chúng ta cần lấy ba giá trị từ người dùng: vận tốc ban đầu, góc so với phương ngang, và số bước để hiển thị trong bảng mà chúng ta sẽ tạo ra. Chúng ta có thể yêu cầu vận tốc theo mét/giây hoặc feet/giây, nhưng vì đây là một vấn đề vật lý, chúng ta sẽ tuân theo hệ mét và yêu cầu theo mét/giây.

Chúng ta cũng phải suy nghĩ về cách chỉ định góc. Thật không may, hầu hết các phương thức Java hoạt động trên góc yêu cầu góc theo radian thay vì độ. Chúng ta có thể yêu cầu góc theo radian, nhưng điều đó sẽ rất bất tiện cho người dùng, vì họ sẽ phải tự chuyển đổi. Thay vào đó, chúng ta có thể cho phép người dùng nhập góc theo độ và sau đó chuyển đổi nó sang radian bằng phương thức Math.toRadians.

Vì vậy, phần tương tác của chương trình sẽ trông như thế này:

```
Scanner console = new Scanner(System.in);

System.out.print("vận tốc (mét/giây)? ");

double velocity = console.nextDouble();

System.out.print("góc (độ)? ");

double angle = Math.toRadians(console.nextDouble());

System.out.print("số bước để hiển thị? ");

int steps = console.nextInt();
```

Lưu ý rằng đối với vận tốc và góc, chúng ta gọi phương thức nextDouble của đối tượng console, vì chúng ta muốn cho phép người dùng chỉ định bất kỳ số nào (bao gồm số có dấu thập

phân), nhưng đối với số bước, chúng ta gọi nextInt, vì số dòng trong bảng của chúng ta cần phải là một số nguyên.

Hãy nhìn kỹ hơn vào dòng mã này:

```
double angle = Math.toRadians(console.nextDouble());
```

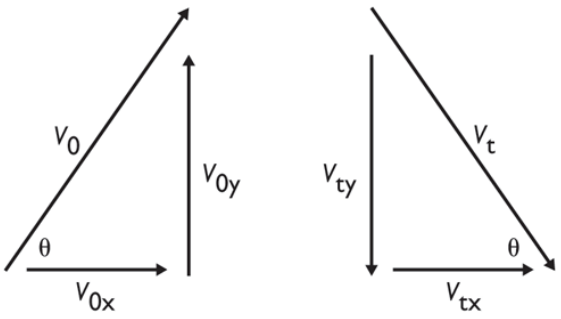
Một số người mới bắt đầu có thể viết điều này dưới dạng hai bước riêng biệt:

```
double angleInDegrees = console.nextDouble();

double angle = Math.toRadians(angleInDegrees);
```

Cả hai cách đều hoạt động và hợp lý, nhưng hãy nhớ rằng bạn không cần phải chia thao tác này thành hai bước riêng biệt. Bạn có thể viết nó dưới dạng một dòng mã gọn hơn.

Khi chúng ta đã lấy được các giá trị này từ người dùng, chúng ta sẵn sàng bắt đầu các phép tính cho bảng quỹ đạo. Vị trí x/y của vật thể tại mỗi khoảng thời gian được xác định bởi vận tốc của nó trong mỗi chiều và gia tốc của vật thể do lực hấp dẫn gây ra. Hình 3.1 cho thấy vận tốc ban đầu v_0 và góc θ của vật thể ngay khi nó được ném và vận tốc cuối cùng v_t ngay khi nó chạm đất.

Hình 3.1 Vận Tốc Ban Đầu và Cuối Cùng của Vật Thể



Hình 3.1 Văn bản thay thế đầy đủ

Chúng ta cần tính toán thành phần x của vận tốc so với thành phần y của vận tốc. Từ vật lý, chúng ta biết rằng có thể tính toán chúng như sau:

```
double xVelocity = velocity * Math.cos(angle);

double yVelocity = velocity * Math.sin(angle);
```

Vì chúng ta bỏ qua khả năng có lực cản không khí, vận tốc x sẽ không thay đổi. Tuy nhiên, vận tốc y sẽ chịu tác động của lực hấp dẫn. Vật lý cho chúng ta biết rằng trên bề mặt Trái Đất, gia tốc do lực hấp dẫn vào khoảng 9,81 mét/giây². Đây là một giá trị phù hợp để định nghĩa là hằng số của lớp:

```
public static final double ACCELERATION = -9.81;
```

Lưu ý rằng chúng ta định nghĩa gia tốc hấp dẫn là một số âm vì nó làm giảm vận tốc y của vật thể (kéo nó xuống thay vì đẩy nó lên).

Mục tiêu của chúng ta là hiển thị x, y, và thời gian đã trôi qua khi vật thể bay lên và quay trở lại. Vận tốc y giảm dần cho đến khi nó bằng 0. Từ vật lý, chúng ta biết rằng đồ thị của vật thể sẽ đối xứng. Vật thể sẽ bay lên cho đến khi vận tốc y của nó bằng 0, sau đó sẽ theo một đường tương tự quay xuống với thời gian bằng nhau. Do đó, thời gian tổng cộng có thể tính như sau:

```
double totalTime = -2.0 * yVelocity / ACCELERATION;
```

Bây giờ, làm thế nào để chúng ta tính các giá trị của x, y, và thời gian đã trôi qua để đưa vào bảng? Việc tính toán hai giá trị trong

số này tương đối đơn giản. Chúng ta muốn có các bước thời gian đều đặn cho mỗi mục trong bảng, vì vậy chúng ta có thể tính toán gia số thời gian bằng cách chia tổng thời gian cho số bước chúng ta muốn đưa vào bảng:

```
double timeIncrement = totalTime / steps;
```

Như đã đề cập trước đó, vận tốc x không thay đổi, vì vậy đối với mỗi gia số thời gian này, chúng ta di chuyển cùng một khoảng cách theo hướng x:

```
double xIncrement = xVelocity * timeIncrement;
```

Giá trị khó tính toán ở đây là vị trí y. Do gia tốc gây ra bởi lực hấp dẫn, vận tốc y thay đổi theo thời gian. Nhưng từ vật lý, chúng ta có công thức chung để tính sự thay đổi vị trí của một vật thể khi biết vận tốc v, thời gian t, và gia tốc a:

```
displacement = v * t + 0.5 * a * t * t;
```

Trong trường hợp của chúng ta, vận tốc mà chúng ta muốn là vận tốc y và gia tốc là từ hằng số lực hấp dẫn của Trái Đất. Dưới đây là mô tả mã giả về cách tạo bảng:

```
x = 0.0;

y = 0.0;

t = 0.0;



System.out.println("step\tx\ty\ttime");
```

```
System.out.println("0\t0.0\t0.0\t0.0");



for (int i = 1; i <= steps; i++) {

    t += timeIncrement;

    x += xIncrement;

    y = yVelocity * t + 0.5 * ACCELERATION * t * t;

    System.out.println(i + "\t" + round2(x) + "\t" +

                       round2(y) + "\t" + round2(t));

}
```

Chúng ta khá gần với mã Java thực tế ở đây, nhưng chúng ta cần suy nghĩ về cách báo cáo các giá trị của x, y, và t trong bảng. Tất cả chúng sẽ có kiểu double, có nghĩa là chúng có thể tạo ra một số lượng lớn chữ số sau dấu thập phân. Nhưng chúng ta không quan tâm đến việc hiển thị tất cả các chữ số đó, vì chúng không đặc biệt liên quan và vì các phép tính của chúng ta không chính xác đến mức đó.

Trước khi hoàn thành mã cho bảng, hãy suy nghĩ về vấn đề chỉ hiển thị một số chữ số của một số. Ý tưởng là cắt bớt các chữ số để chúng ta không thấy tất cả chúng. Một cách để cắt bớt là chuyển kiểu double sang int, điều này cắt bớt tất cả các chữ số sau dấu thập phân. Chúng ta có thể làm điều đó, nhưng có lẽ chúng ta muốn giữ ít nhất một số chữ số đó. Ví dụ, chúng ta có thể muốn báo cáo hai chữ số sau dấu thập phân. Mẹo là mang hai chữ số mà chúng ta muốn sang phía khác của dấu thập phân.

Chúng ta có thể làm điều đó bằng cách nhân với 100 và sau đó chuyển kiểu về int:

```
(int) (n * 100.0);
```

Biểu thức này đưa chúng ta các chữ số mong muốn, nhưng bây giờ dấu thập phân đang ở sai vị trí. Ví dụ, nếu n ban đầu là 3.488834, biểu thức trên sẽ đưa ra 348. Chúng ta phải chia kết quả này cho 100 để biến nó trở lại thành số 3.48:

```
(int) (n * 100.0) / 100.0;
```

Trong khi làm điều này, chúng ta có thể thực hiện một cải tiến cuối cùng. Lưu ý rằng số ban đầu là 3.488834. Nếu chúng ta chỉ cắt bớt, chúng ta sẽ được 3.48, nhưng thực sự số này gần với 3.49 hơn. Chúng ta có thể làm tròn số bằng cách gọi phương thức Math.round trên số thay vì chuyển kiểu:

```
Math.round(n * 100.0) / 100.0;
```

Đây là một thao tác mà chúng ta có thể muốn thực hiện trên nhiều số, vì vậy nó xứng đáng được đưa vào một phương thức:

```
public static double round2(double n) {

    return Math.round(n * 100.0) / 100.0;

}
```

Quay trở lại mã giả cho bảng, chúng ta có thể kết hợp các cuộc gọi phương thức round2 để tiến gần hơn đến mã Java thực tế:

```
x = 0.0;
```

```
y = 0.0;

t = 0.0;



System.out.println("step\tx\ty\ttime");

System.out.println("0\t0.0\t0.0\t0.0");



for (int i = 1; i <= steps; i++) {

    t += timeIncrement;

    x += xIncrement;

    y = yVelocity * t + 0.5 * ACCELERATION * t * t;

    System.out.println(i + "\t" + round2(x) + "\t" +

                       round2(y) + "\t" + round2(t));

}
```

Sẽ thật tuyệt nếu các giá trị trong bảng được căn chỉnh. Để có được các số thẳng hàng gọn gàng, chúng ta sẽ phải sử dụng định dạng đầu ra, điều mà chúng ta sẽ thảo luận trong Chương 4. Hiện tại, ít nhất chúng ta có thể làm cho các số thẳng hàng trong các cột bằng cách ngăn cách chúng bằng các ký tự tab. Hãy nhớ rằng chuỗi thoát \t đại diện cho một tab đơn.

Nếu chúng ta có một bảng với các cột, thì cũng có ý nghĩa để có một tiêu đề cho bảng. Và có lẽ chúng ta muốn đưa vào một dòng trong bảng hiển thị điều kiện ban đầu, nơi mà x, y, và thời gian đều bằng 0. Vì vậy, chúng ta có thể mở rộng mã giả thành mã Java sau:

```
double x = 0.0;

double y = 0.0;

double t = 0.0;



System.out.println("step\tx\ty\ttime");

System.out.println("0\t0.0\t0.0\t0.0");



for (int i = 1; i <= steps; i++) {

    t += timeIncrement;

    x += xIncrement;

    y = yVelocity * t + 0.5 * ACCELERATION * t * t;

    System.out.println(i + "\t" + round2(x) + "\t" +

                       round2(y) + "\t" + round2(t));

}
```

Giải pháp không cấu trúc

Chúng ta có thể kết hợp tất cả các phần này để tạo thành một chương trình hoàn chỉnh. Trước tiên, hãy xem phiên bản không cấu trúc, bao gồm hầu hết mã trong phương thức main. Phiên bản này cũng bao gồm một số câu lệnh println mới ở đầu để giới thiệu ngắn gọn cho người dùng:

```
1   // Chương trình này tính toán quỹ đạo của một vật thể.

2   import java.util.*; // để sử dụng Scanner

3   public class Projectile {

4       // hằng số cho gia tốc Trái Đất theo mét/giây²

5       public static final double ACCELERATION = -9.81;

6       public static void main(String[] args) {

7           Scanner console = new Scanner(System.in);

8           System.out.println("Chương trình này tính toán quỹ đạo
của một vật thể");

9           System.out.println("dựa trên vận tốc ban đầu và góc của
nó");

10          System.out.println("so với phương ngang.");

11          System.out.println();

12          System.out.print("vận tốc (mét/giây)? ");

13          double velocity = console.nextDouble();
```

```
14          System.out.print("góc (độ)? ");
15          double angle = Math.toRadians(console.nextDouble());
16          System.out.print("số bước hiển thị? ");
17          int steps = console.nextInt();
18          System.out.println();
19          double xVelocity = velocity * Math.cos(angle);
20          double yVelocity = velocity * Math.sin(angle);
21          double totalTime = -2.0 * yVelocity / ACCELERATION;
22          double timeIncrement = totalTime / steps;
23          double xIncrement = xVelocity * timeIncrement;
24          double x = 0.0;
25          double y = 0.0;
26          double t = 0.0;
27          System.out.println("bước\tx\ty\tthời gian");
28          System.out.println("0\t0.0\t0.0\t0.0");
29          for (int i = 1; i <= steps; i++) {
30              t += timeIncrement;
31              x += xIncrement;
```

```
32              y = yVelocity * t + 0.5 * ACCELERATION * t * t;
33              System.out.println(i + "\t" + round2(x) + "\t" +
34                                      round2(y) + "\t" + round2(t));
35          }
36      }
37      public static double round2(double n) {
38          return Math.round(n * 100.0) / 100.0;
39      }
40  }
```

Dưới đây là một ví dụ về việc thực thi chương trình:

```
Chương trình này tính toán quỹ đạo của một vật thể
dựa trên vận tốc ban đầu và góc của nó
so với phương ngang.
vận tốc (mét/giây)? 30
góc (độ)? 50
số bước hiển thị? 10
bước   x       y       thời gian
0      0.0     0.0     0.0
```

1	9.03	9.69	0.47
2	18.07	17.23	0.94
3	27.1	22.61	1.41
4	36.14	25.84	1.87
5	45.17	26.92	2.34
6	54.21	25.84	2.81
7	63.24	22.61	3.28
8	72.28	17.23	3.75
9	81.31	9.69	4.22
10	90.35	0.0	4.69

Từ nhật ký thực thi, bạn có thể thấy rằng vật thể đạt chiều cao tối đa là 26.92 mét sau 2.34 giây (bước thứ năm) và nó chạm đất cách vị trí ban đầu 90.35 mét sau 4.69 giây (bước thứ mười).

Phiên bản chương trình này hoạt động, nhưng chúng ta thường không muốn bao gồm quá nhiều mã trong phương thức main. Phần tiếp theo sẽ khám phá cách chia nhỏ chương trình thành các phần nhỏ hơn.

Giải pháp có cấu trúc

Có ba khối mã chính trong phương thức main của chương trình Projectile: một loạt các câu lệnh println để giới thiệu chương trình cho người dùng, một loạt các câu lệnh để yêu cầu người dùng nhập ba giá trị dùng để tạo bảng, và sau đó là mã sản xuất bảng.

Vì vậy, trong mã giả, cấu trúc tổng thể sẽ trông như sau:

- giới thiệu.
- yêu cầu nhập vận tốc, góc, và số bước.
- tạo bảng.

Bước đầu tiên và thứ ba có thể dễ dàng chuyển thành phương thức, nhưng bước ở giữa thì không. Bước này yêu cầu người dùng nhập các giá trị mà chúng ta cần để tạo bảng. Nếu chúng ta chuyển nó thành một phương thức, nó sẽ phải trả về ba giá trị cho main. Một phương thức chỉ có thể trả về một giá trị, vì vậy đáng tiếc là chúng ta không thể chuyển bước này thành phương thức. Chúng ta có thể chia nó thành ba phương thức khác nhau, mỗi phương thức dành cho một trong ba giá trị, nhưng mỗi phương thức đó sẽ chỉ có hai dòng, vì vậy không rõ liệu việc làm như vậy có cải thiện cấu trúc tổng thể không.

Cải tiến chính mà chúng ta có thể thực hiện là tách phần giới thiệu và phần bảng thành các phương thức riêng biệt. Một cải tiến khác là chuyển công thức dịch chuyển vật lý thành một phương thức riêng. Luôn là một ý tưởng tốt để chuyển các phương trình thành các phương thức. Giới thiệu các phương thức đó, chúng ta có phiên bản có cấu trúc của chương trình như sau:

```
1   // Chương trình này tính toán quỹ đạo của một vật thể.
```

```
2   import java.util.*; // để sử dụng Scanner
3   public class Projectile2 {
4       // hằng số cho gia tốc Trái Đất theo mét/giây²
5       public static final double ACCELERATION = -9.81;
6       public static void main(String[] args) {
7           Scanner console = new Scanner(System.in);
8           giveIntro();
9           System.out.print("vận tốc (mét/giây)? ");
10          double velocity = console.nextDouble();
11          System.out.print("góc (độ)? ");
12          double angle = Math.toRadians(console.nextDouble());
13          System.out.print("số bước hiển thị? ");
14          int steps = console.nextInt();
15          System.out.println();
16          printTable(velocity, angle, steps);
17      }
18      // in ra bảng hiển thị quỹ đạo của vật thể với vận tốc và góc
ban đầu và bao gồm số bước được chỉ định trong bảng
```

```
19      public static void printTable(double velocity, double angle,
int steps) {
20          double xVelocity = velocity * Math.cos(angle);
21          double yVelocity = velocity * Math.sin(angle);
22          double totalTime = -2.0 * yVelocity / ACCELERATION;
23          double timeIncrement = totalTime / steps;
24          double xIncrement = xVelocity * timeIncrement;
25          double x = 0.0;
26          double y = 0.0;
27          double t = 0.0;
28          System.out.println("bước\tx\ty\tthời gian");
29          System.out.println("0\t0.0\t0.0\t0.0");
30          for (int i = 1; i <= steps; i++) {
31              t += timeIncrement;
32              x += xIncrement;
33              y = displacement(yVelocity, t, ACCELERATION);
34              System.out.println(i + "\t" + round2(x) + "\t" +
35                                 round2(y) + "\t" + round2(t));
```

```
36          }
37      }
38      // giới thiệu ngắn gọn cho người dùng
39      public static void giveIntro() {
40          System.out.println("Chương trình này tính toán quỹ đạo của một vật thể");
41          System.out.println("dựa trên vận tốc ban đầu và góc của nó");
42          System.out.println("so với phương ngang.");
43          System.out.println();
44      }
45      // trả về độ dịch chuyển theo phương thẳng đứng cho một vật thể với vận tốc ban đầu v, thời gian t đã trôi qua, và gia tốc a
46      public static double displacement(double v, double t, double a) {
47          return v * t + 0.5 * a * t * t;
48      }
49      // làm tròn số n đến 2 chữ số sau dấu thập phân
50      public static double round2(double n) {
```

```
51          return Math.round(n * 100.0) / 100.0;

52      }

53  }
```

Phiên bản này thực thi giống như phiên bản trước đó.

Tóm tắt Chương

Các phương thức có thể được viết để chấp nhận các tham số, đó là tập hợp các đặc điểm phân biệt các thành viên khác nhau của một họ nhiệm vụ. Các tham số cho phép giá trị dữ liệu truyền vào phương thức, từ đó có thể thay đổi cách thực thi phương thức. Một phương thức được khai báo với một tập hợp tham số có thể thực hiện cả một họ nhiệm vụ tương tự thay vì chỉ thực hiện chính xác một nhiệm vụ.

Khi các giá trị nguyên thủy như kiểu int hoặc double được truyền làm tham số, giá trị của chúng được sao chép vào phương thức. Các tham số nguyên thủy gửi giá trị vào phương thức nhưng không gửi ra khỏi nó; phương thức có thể sử dụng các giá trị dữ liệu nhưng không thể ảnh hưởng đến giá trị của bất kỳ biến nào bên ngoài nó.

Hai phương thức có thể có cùng tên nếu chúng khai báo các tham số khác nhau. Điều này được gọi là quá tải (overloading).

Các phương thức có thể được viết để trả về giá trị cho mã gọi chúng. Tính năng này cho phép phương thức thực hiện một tính toán phức tạp và sau đó cung cấp kết quả của nó trở lại mã gọi.

Loại giá trị trả về phải được khai báo trong phần đầu của phương thức và được gọi là kiểu trả về của phương thức.

Java có một lớp gọi là Math chứa một số phương thức tĩnh hữu ích mà bạn có thể sử dụng trong chương trình của mình, chẳng hạn như lũy thừa, căn bậc hai và logarit.

Một đối tượng là một thực thể kết hợp dữ liệu và các thao tác. Một số đối tượng trong Java bao gồm String, đó là chuỗi các ký tự văn bản, và Scanner, dùng để đọc dữ liệu đầu vào từ người dùng.

Các đối tượng chứa các phương thức thực hiện hành vi của chúng. Để gọi một phương thức trên một đối tượng, hãy viết tên đối tượng, tiếp theo là dấu chấm, và sau đó là tên phương thức.

Một đối tượng String giữ một chuỗi các ký tự. Các ký tự có các chỉ số, bắt đầu từ 0 cho ký tự đầu tiên.

Một ngoại lệ (exception) là một lỗi xảy ra khi chương trình thực hiện một hành động bất hợp pháp và không thể tiếp tục thực thi bình thường.

Một số chương trình là tương tác và phản hồi lại đầu vào từ người dùng. Những chương trình này nên in ra một thông báo cho người dùng, còn được gọi là prompt, yêu cầu đầu vào.

Java có một lớp gọi là Scanner dùng để đọc dữ liệu đầu vào từ bàn phím. Một Scanner có thể đọc các mẩu dữ liệu khác nhau (còn gọi là token) từ một nguồn đầu vào. Nó có thể đọc từng token một hoặc toàn bộ một dòng một lúc.

Các bài tập tự kiểm tra

Phần 3.1: Tham số

1. Cú pháp đúng cho tiêu đề phương thức có tham số là gì?
    - a. public static void example(x, y) {
    - b. public static (int x, int y) example() {
    - c. public static void example(int x, y) {
    - d. public static void example(x: int, y: int) {
    - e. public static void example(int x, int y) {
2. Kết quả nào được tạo ra bởi chương trình sau?

```
1 public class MysteryNums {
2   public static void main(String[] args) {
3       int x = 15;
4       sentence(x, 42);
5       int y = x - 5;
6       sentence(y, x + y);
7   }
8   public static void sentence(int num1, int num2) {
```

```
9       System.out.println(num1 + " " + num2);
10  }
11 }
```

3. Chương trình sau có 9 lỗi. Đó là những lỗi nào?

```
1 public class Oops3 {
2   public static void main() {
3       double bubble = 867.5309;
4       double x = 10.01;
5       printer(double x, double y);
6       printer(x);
7       printer("barack", "obama");
8       System.out.println("z = " + z);
9   }
10  public static void printer(x, y double) {
11      int z = 5;
12      System.out.println("x = " + double x + " and y = " + y);
13      System.out.println("The value from main is: " + bubble);
```

```
14  }
15 }
```

4. Kết quả nào được tạo ra bởi chương trình sau?

```
1 public class Odds {
2   public static void main(String[] args) {
3       printOdds(3);
4       printOdds(17 / 2);
5       int x = 25;
6       printOdds(37 - x + 1);
7   }
8   public static void printOdds(int n) {
9       for (int i = 1; i <= n; i++) {
10          int odd = 2 * i - 1;
11          System.out.print(odd + " ");
12      }
13      System.out.println();
14  }
```

```
15 }
```

5. Kết quả nào được tạo ra bởi chương trình sau?

```
1 public class Weird {
2   public static void main(String[] args) {
3       int number = 8;
4       halfTheFun(11);
5       halfTheFun(2 - 3 + 2 * 8);
6       halfTheFun(number);
7       System.out.println("number = " + number);
8   }
9   public static void halfTheFun(int number) {
10      number = number / 2;
11      for (int count = 1; count <= number; count++) {
12          System.out.print(count + " ");
13      }
14      System.out.println();
15  }
```

```
16 }
```

6. Kết quả nào được tạo ra bởi chương trình sau?

```
1 public class MysteryNumbers {

2   public static void main(String[] args) {

3       String one = "two";

4       String two = "three";

5       String three = "1";

6       int number = 20;

7       sentence(one, two, 3);

8       sentence(two, three, 14);

9       sentence(three, three, number + 1);

10      sentence(three, two, 1);

11      sentence("eight", three, number / 2);

12  }

13  public static void sentence(String three, String one, int
number) {

14      System.out.println(one + " times " + three + " = " + (number
* 2));
```

```
15  }

16 }
```

7. Kết quả nào được tạo ra bởi chương trình sau?

```
1 public class MysteryWho {

2   public static void main(String[] args) {

3       String whom = "her";

4       String who = "him";

5       String it = "who";

6       String he = "it";

7       String she = "whom";

8       sentence(he, she, it);

9       sentence(she, he, who);

10      sentence(who, she, who);

11      sentence(it, "stu", "boo");

12      sentence(it, whom, who);

13  }

14  public static void sentence(String she, String who, String
whom) {
```

```
15      System.out.println(who + " and " + whom + " like " + she);
16  }
17 }
```

8. Kết quả nào được tạo ra bởi chương trình sau?

```
1 public class MysteryTouch {
2   public static void main(String[] args) {
3       String head = "shoulders";
4       String knees = "toes";
5       String elbow = "head";
6       String eye = "eyes and ears";
7       String ear = "eye";
8       touch(ear, elbow);
9       touch(elbow, ear);
10      touch(head, "elbow");
11      touch(eye, eye);
12      touch(knees, "Toes");
13      touch(head, "knees " + knees);
```

```
14  }
15  public static void touch(String elbow, String ear) {
16      System.out.println("touch your " + elbow + " to your " + ear);
17  }
18 }
```

9. Kết quả nào được tạo ra bởi chương trình sau?

```
1 public class MysterySoda {
2   public static void main(String[] args) {
3       String soda = "Coke";
4       String pop = "Pepsi";
5       String Coke = "pop";
6       String Pepsi = "soda";
7       String say = pop;
8       carbonated(Coke, soda, pop);
9       carbonated(pop, Pepsi, Pepsi);
10      carbonated("pop", pop, "Kool-Aid");
11      carbonated(say, "say", pop);
```

```
12  }

13  public static void carbonated(String Coke, String soda, String
pop) {

14      System.out.println("say " + soda + " not " + pop + " or " +
Coke);

15  }

16 }
```

10. Viết một phương thức có tên printStrings nhận một chuỗi và số lần lặp làm tham số và in chuỗi đó với số lần đã cho, với khoảng trắng sau mỗi lần. Ví dụ, lệnh gọi printStrings("abc", 5); sẽ in ra kết quả sau:

```
abc abc abc abc abc
```

11. Lệnh System.out.println hoạt động trên nhiều loại giá trị khác nhau, như số nguyên hoặc số thực. Thuật ngữ dùng để chỉ một phương thức như vậy là gì?

Phần 3.2: Phương thức trả về giá trị

12. Có gì sai trong chương trình sau?

```
1  public class Temperature {
2    public static void main(String[] args) {
3        double tempf = 98.6;
4        double tempc = 0.0;
5        ftoc(tempf, tempc);
6        System.out.println("Nhiệt độ cơ thể ở C là: " + tempc);
7    }
8    // Chuyển đổi nhiệt độ từ Fahrenheit sang Celsius
```

```
9    public static void ftoc(double tempf, double tempc) {
10       tempc = (tempf - 32) * 5 / 9;
11   }
12 }
```

13. Đánh giá các biểu thức sau:

- a. Math.abs(–1.6)
- b. Math.abs(2 + –4)
- c. Math.pow(6, 2)
- d. Math.pow(5 / 2, 6)
- e. Math.ceil(9.1)
- f. Math.ceil(115.8)
- g. Math.max(7, 4)
- h. Math.min(8, 3 + 2)
- i. Math.min(–2, –5)
- j. Math.sqrt(64)
- k. Math.sqrt(76 + 45)
- l. 100 + Math.log10(100)
- m. 13 + Math.abs(–7) – Math.pow(2, 3) + 5
- n. Math.sqrt(16) * Math.max(Math.abs(–5), Math.abs(–3))
- o. 7 – 2 + Math.log10(1000) + Math.log(Math.pow(Math.E, 5))
- p. Math.max(18 – 5, Math.ceil(4.6 * 3))

14. Kết quả nào được tạo ra bởi chương trình sau?

```
1  public class MysteryReturn {
2    public static void main(String[] args) {
3        int x = 1, y = 2, z = 3;
4        z = mystery(x, z, y);
5        System.out.println(x + " " + y + " " + z);
6        x = mystery(z, z, x);
7        System.out.println(x + " " + y + " " + z);
8        y = mystery(y, y, z);
9        System.out.println(x + " " + y + " " + z);
10   }
```

```
11   public static int mystery(int z, int x, int y) {
12       z--;
13       x = 2 * y + z;
14       y = x - 1;
15       System.out.println(y + " " + z);
16       return x;
17   }
18 }
```

15. Viết kết quả của từng biểu thức sau. Lưu ý rằng giá trị của biến chỉ thay đổi nếu bạn gán lại nó bằng cách sử dụng toán tử =.

```
double grade = 2.7;
Math.round(grade);                          // grade =
grade = Math.round(grade);                  // grade =
double min = Math.min(grade, Math.floor(2.9));    //   min =
double x = Math.pow(2, 4);                  //     x =
x = Math.sqrt(64);                          //     x =
int count = 25;
Math.sqrt(count);                           // count =
count = (int) Math.sqrt(count);             // count =
int a = Math.abs(Math.min(-1, -3));         //     a =
```

16. Viết một phương thức có tên min nhận ba số nguyên làm tham số và trả về giá trị nhỏ nhất trong ba giá trị đó. Ví dụ, một lệnh gọi min(3, -2, 7) sẽ trả về -2, và một lệnh gọi min(19, 27, 6) sẽ trả về 6. Sử dụng Math.min để viết giải pháp của bạn.
17. Viết một phương thức có tên countQuarters nhận một số nguyên đại diện cho số cent làm tham số và trả về số lượng đồng xu 25 cent được biểu diễn bởi số cent đó. Đừng tính bất kỳ đô la nào, vì chúng sẽ được phát hành dưới dạng tờ đô la. Ví dụ, countQuarters(64) sẽ trả về 2, vì 64 cent tương đương với 2 đồng xu 25 cent với 14 cent còn lại. Một

lệnh gọi countQuarters(1278) sẽ trả về 3, vì sau khi trừ 12 đô la, còn lại 3 đồng xu 25 cent trong 78 cent còn lại.

Phần 3.3: Sử dụng đối tượng

18. **Đầu ra của đoạn mã sau sẽ là gì?**

```
String first = "James";
String last = "Kirk";
String middle = "T.";
System.out.println(last);
System.out.println("My name is " + first);
System.out.println(first + " " + last);
System.out.println(last + ", " + first + " " + middle);
System.out.println(middle + " is for Tiberius");
```

19. **Giả sử rằng các biến sau đã được khai báo:**

```
//       index 0123456789012345
String str1 = "Frodo Baggins";
String str2 = "Gandalf the GRAY";
```

Đánh giá các biểu thức sau: a. str1.length()
b. str1.charAt(7)
c. str2.charAt(0)
d. str1.indexOf("o")
e. str2.toUpperCase()
f. str1.toLowerCase().indexOf("B")
g. str1.substring(4)
h. str2.substring(3, 14)
i. str2.replace("a", "oo")
j. str2.replace("gray", "white")
k. "str1".replace("r", "range")

20. **Giả sử rằng các biến sau đã được khai báo:**

```
String str1 = "Q.E.D.";
String str2 = "Arcturan Megadonkey";
String str3 = "Sirius Cybernetics Corporation";
```

Đánh giá các biểu thức sau: a. str1.length()

b. str2.length()
c. str1.toLowerCase()
d. str2.toUpperCase()
e. str1.substring(2, 4)
f. str2.substring(10, 14)
g. str1.indexOf("D")
h. str1.indexOf(".")
i. str2.indexOf("donkey")
j. str3.indexOf("X")
k. str2 + str3.charAt(17)
l. str3.substring(9, str3.indexOf("e"))
m. str3.substring(7, 12)
n. str2.toLowerCase().substring(9, 13) + str3.substring(18, str3.length() - 7)

21. **Xem xét chuỗi sau:**

```
String quote = "Four score and seven years ago";
```

Biểu thức nào tạo ra chuỗi mới là "SCORE"? Biểu thức nào tạo ra "four years"?

22. **Xem xét đoạn mã sau:**

```
Scanner console = new Scanner(System.in);
System.out.print("How much money do you have? ");
double money = console.nextDouble();
```

Mô tả điều gì sẽ xảy ra khi người dùng nhập các giá trị sau. Nếu mã chạy thành công, mô tả giá trị sẽ được lưu trữ trong biến money: a. 34.50
b. 6
c. $25.00
d. million
e. 100*5
f. 600x000
g. none
h. 645

23. **Viết mã Java để đọc một số nguyên từ người dùng, sau đó in số đó nhân đôi. Bạn có thể giả định rằng người dùng nhập một số nguyên hợp lệ.**
24. **Xem xét chương trình sau. Sửa đổi mã để sử dụng Scanner để nhắc người dùng nhập giá trị của low và high.**

```
public class SumNumbers {
   public static void main(String[] args) {
      Scanner console = new Scanner(System.in);
      System.out.print("low? ");
      int low = console.nextInt();
      System.out.print("high? ");
      int high = console.nextInt();
      int sum = 0;
      for (int i = low; i <= high; i++) {
         sum += i;
      }
      System.out.println("sum = " + sum);
   }
}
```

Dưới đây là một ví dụ về thực thi trong đó người dùng yêu cầu tính tổng của các giá trị từ 1 đến 10:

```
low?
1
high?
10
sum = 55
```

25. **Viết mã Java yêu cầu người dùng nhập một cụm từ và một số lần lặp lại, sau đó in cụm từ đó số lần yêu cầu. Dưới đây là một ví dụ đối thoại với người dùng:**

```
What is your phrase?
His name is Robert Paulson.
How many times should I repeat it?
```

```
3
His name is Robert Paulson.
His name is Robert Paulson.
His name is Robert Paulson.
```

Bài tập

1. **Viết một phương thức gọi là printNumbers chấp nhận một số tối đa làm đối số và in ra từng số từ 1 đến số tối đa đó, bao gồm cả số tối đa, trong ngoặc vuông. Ví dụ, hãy xem xét các lệnh gọi sau:**

```
printNumbers(15);
printNumbers(5);
```

Các lệnh gọi này sẽ tạo ra kết quả sau:

```
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15]
[1] [2] [3] [4] [5]
```

Bạn có thể giả định rằng giá trị truyền vào printNumbers là 1 hoặc lớn hơn.

2. **Viết một phương thức gọi là printPowersOf2 chấp nhận một số tối đa làm đối số và in ra từng lũy thừa của 2 từ 2^0 (1) đến lũy thừa tối đa đó, bao gồm cả lũy thừa tối đa. Ví dụ, hãy xem xét các lệnh gọi sau:**

```
printPowersOf2(3);
printPowersOf2(10);
```

Các lệnh gọi này sẽ tạo ra kết quả sau:

```
1 2 4 8
1 2 4 8 16 32 64 128 256 512 1024
```

Bạn có thể giả định rằng giá trị truyền vào printPowersOf2 là 0 hoặc lớn hơn. (Lớp Math có thể giúp bạn giải quyết vấn đề này. Nếu sử dụng nó, bạn có thể cần phải chuyển đổi kết quả từ kiểu double sang int để không thấy .0 sau mỗi số trong

đầu ra. Cũng hãy thử viết chương trình này mà không sử dụng lớp Math.)

3. **Viết một phương thức gọi là printPowersOfN chấp nhận một cơ số và một số mũ làm đối số và in ra từng lũy thừa của cơ số từ cơ số^0 (1) đến lũy thừa tối đa đó, bao gồm cả lũy thừa tối đa. Ví dụ, hãy xem xét các lệnh gọi sau:**

```
printPowersOfN(4, 3);
printPowersOfN(5, 6);
printPowersOfN(-2, 8);
```

Các lệnh gọi này sẽ tạo ra kết quả sau:

```
1 4 16 64
1 5 25 125 625 3125 15625
1 -2 4 -8 16 -32 64 -128 256
```

Bạn có thể giả định rằng số mũ truyền vào printPowersOfN có giá trị là 0 hoặc lớn hơn. (Lớp Math có thể giúp bạn giải quyết vấn đề này. Nếu sử dụng nó, bạn có thể cần phải chuyển đổi kết quả từ kiểu double sang int để không thấy .0 sau mỗi số trong đầu ra. Cũng hãy thử viết chương trình này mà không sử dụng lớp Math.)

4. **Viết một phương thức gọi là printSquare chấp nhận một số nguyên nhỏ nhất và lớn nhất và in ra một hình vuông gồm các dòng số tăng dần. Dòng đầu tiên bắt đầu từ số nhỏ nhất, và mỗi dòng tiếp theo bắt đầu từ số lớn hơn tiếp theo. Dãy số trên một dòng sẽ quay lại số nhỏ nhất sau khi đạt đến số lớn nhất. Ví dụ, lệnh gọi**

```
printSquare(3, 7);
```

sẽ tạo ra kết quả sau:

```
34567
45673
56734
67345
73456
```

Nếu số lớn nhất nhỏ hơn số nhỏ nhất, phương thức sẽ không tạo ra kết quả nào.

5. **Viết một phương thức gọi là printGrid chấp nhận hai số nguyên đại diện cho số hàng và cột và in ra một lưới số từ 1 đến (hàng * cột) theo thứ tự cột chính. Ví dụ, lệnh gọi**

```
printGrid(4, 6);
```

sẽ tạo ra kết quả sau:

```
1 5 9 13 17 21
2 6 10 14 18 22
3 7 11 15 19 23
4 8 12 16 20 24
```

6. **Viết một phương thức gọi là largerAbsVal chấp nhận hai số nguyên làm tham số và trả về giá trị tuyệt đối lớn hơn trong hai số đó. Lệnh gọi largerAbsVal(11, 2) sẽ trả về 11, và lệnh gọi largerAbsVal(4, -5) sẽ trả về 5.**
7. **Viết một biến thể của phương thức largestAbsVal từ bài tập trước đó, chấp nhận ba số nguyên làm tham số và trả về giá trị tuyệt đối lớn nhất trong ba số đó. Ví dụ, lệnh gọi largestAbsVal(7, -2, -11) sẽ trả về 11, và lệnh gọi largestAbsVal(-4, 5, 2) sẽ trả về 5.**
8. **Viết một phương thức gọi là quadratic giải phương trình bậc hai và in ra các nghiệm của phương trình. Nhớ lại rằng một phương trình bậc hai là một phương trình đa thức theo biến x dưới dạng ax^2 + bx + c = 0. Công thức giải phương trình bậc hai là**

```
x = (–b ± √(b^2–4ac)) / 2a
```

Dưới đây là một số ví dụ về phương trình và nghiệm của chúng:

```
x^2–7x+12: x = 4, x = 3
x^2–3x+2: x = -2, x = -1
```

Phương thức của bạn sẽ chấp nhận các hệ số a, b và c làm tham số và in ra các nghiệm của phương trình. Bạn có thể

giả định rằng phương trình có hai nghiệm thực, mặc dù về mặt toán học điều này không phải lúc nào cũng đúng.

9. **Viết một phương thức gọi là lastDigit trả về chữ số cuối cùng của một số nguyên. Ví dụ, lastDigit(3572) sẽ trả về 2. Phương thức này cũng hoạt động với các số âm. Ví dụ, lastDigit(-947) sẽ trả về 7.**
10. **Viết một phương thức gọi là area chấp nhận bán kính của một hình tròn làm tham số và trả về diện tích của hình tròn. Ví dụ, lệnh gọi area(2.0) sẽ trả về 12.566370614359172. Nhớ rằng diện tích có thể được tính bằng cách lấy pi (π) nhân với bình phương của bán kính và Java có một hằng số gọi là Math.PI.**
11. **Viết một phương thức gọi là distance chấp nhận bốn tọa độ số nguyên x1, y1, x2, y2 làm tham số và tính khoảng cách giữa các điểm (x1, y1) và (x2, y2) trên mặt phẳng Cartesian. Phương trình tính khoảng cách là**

d = √((x2–x1)^2 + (y2–y1)^2)

Ví dụ, lệnh gọi distance(1, 0, 4, 4) sẽ trả về 5.0 và lệnh gọi distance(10, 2, 3, 15) sẽ trả về 14.7648230602334.

12. **Viết một phương thức gọi là scientific chấp nhận một cơ số số thực và một số mũ làm tham số và tính toán cơ số nhân với 10 mũ, như đã thấy trong ký hiệu khoa học. Ví dụ, lệnh gọi scientific(6.23, 5) sẽ trả về 623000.0 và lệnh gọi scientific(1.9, –2) sẽ trả về 0.019.**
13. **Viết một phương thức gọi là pay chấp nhận hai tham số: một số thực đại diện cho lương của một trợ giảng (TA) và một số nguyên đại diện cho số giờ mà trợ giảng đã làm việc trong tuần này. Phương thức sẽ trả về số tiền cần trả cho trợ giảng. Ví dụ, lệnh gọi pay(5.50, 6) sẽ trả về 33.0. Trợ giảng sẽ nhận lương "làm thêm giờ" là 1.5 lần mức lương bình thường cho bất kỳ giờ làm việc nào vượt quá 8 giờ. Ví dụ, lệnh gọi pay(4.00, 11) sẽ trả về (4.00*8)+(6.00*3) hoặc 50.0.**

14. **Viết một phương thức gọi là cylinderSurfaceArea chấp nhận bán kính và chiều cao làm tham số và trả về diện tích bề mặt của một hình trụ có các kích thước đó. Ví dụ, lệnh gọi cylinderSurfaceArea(3.0, 4.5) sẽ trả về 141.3716694115407. Công thức tính diện tích bề mặt của một hình trụ có bán kính r và chiều cao h là**

diện tích bề mặt = 2πr^2 + 2πrh

15. **Viết một phương thức gọi là sphereVolume chấp nhận bán kính làm tham số và trả về thể tích của một hình cầu có bán kính đó. Ví dụ, lệnh gọi sphereVolume(2.0) sẽ trả về 33.510321638291124. Công thức tính thể tích của một hình cầu có bán kính r là**

thể tích = (4/3)πr^3

16. **Viết một phương thức gọi là triangleArea chấp nhận ba độ dài của ba cạnh của một tam giác làm tham số và trả về diện tích của tam giác có các cạnh đó. Ví dụ, lệnh gọi triangleArea(8, 5.2, 7.1) sẽ trả về 18.151176098258745. Để tính diện tích, sử dụng công thức của Heron, công thức này dựa trên giá trị tính toán s, một độ dài bằng một nửa chu vi của tam giác:**

diện tích = √(s(s−a)(s−b)(s−c)) với s = (a+b+c)/2

17. **Viết một phương thức gọi là padString chấp nhận hai tham số: một chuỗi và một số nguyên đại diện cho chiều dài. Phương thức sẽ đệm chuỗi bằng các khoảng trắng cho đến khi chiều dài của chuỗi đạt đến chiều dài được cho. Ví dụ, padString("hello", 8) sẽ trả về "hello ". (Phương thức này hữu ích khi cố gắng in ra đầu ra thẳng hàng theo chiều ngang.) Nếu chiều dài của chuỗi đã bằng hoặc lớn hơn chiều dài của tham số, phương thức của bạn nên trả về chuỗi gốc. Ví dụ, padString("congratulations", 10) sẽ trả về "congratulations".**

18. **Viết một phương thức gọi là vertical chấp nhận một chuỗi làm tham số và in từng chữ cái của chuỗi đó trên các dòng riêng biệt. Ví dụ, một lệnh gọi vertical("hey now") sẽ tạo ra kết quả sau:**

```
h
e
y
n
o
w
```

19. **Viết một phương thức gọi là printReverse chấp nhận một chuỗi làm tham số và in các ký tự theo thứ tự ngược lại. Ví dụ, một lệnh gọi printReverse("hello there!") sẽ in "!ereht olleh". Nếu chuỗi rỗng được truyền vào, phương thức sẽ không tạo ra kết quả nào.**
20. **Viết một phương thức gọi là inputBirthday chấp nhận một Scanner cho giao diện điều khiển làm tham số và nhắc người dùng nhập tháng, ngày và năm sinh, sau đó in ngày sinh theo định dạng phù hợp. Dưới đây là một ví dụ đối thoại với người dùng:**

```
Bạn sinh vào ngày nào của tháng? 8
Tên tháng bạn sinh là gì? May
Bạn sinh vào năm nào? 1981
Bạn sinh ngày 8 tháng 5, 1981. Bạn già rồi!
```

21. **Viết một phương thức gọi là processName chấp nhận một Scanner cho giao diện điều khiển làm tham số và nhắc người dùng nhập họ tên đầy đủ, sau đó in tên theo thứ tự ngược lại (tức là, họ trước, tên sau). Dưới đây là một ví dụ đối thoại với người dùng:**

```
Vui lòng nhập họ tên đầy đủ: Sammy Jankis
Tên của bạn theo thứ tự ngược lại là Jankis, Sammy
```

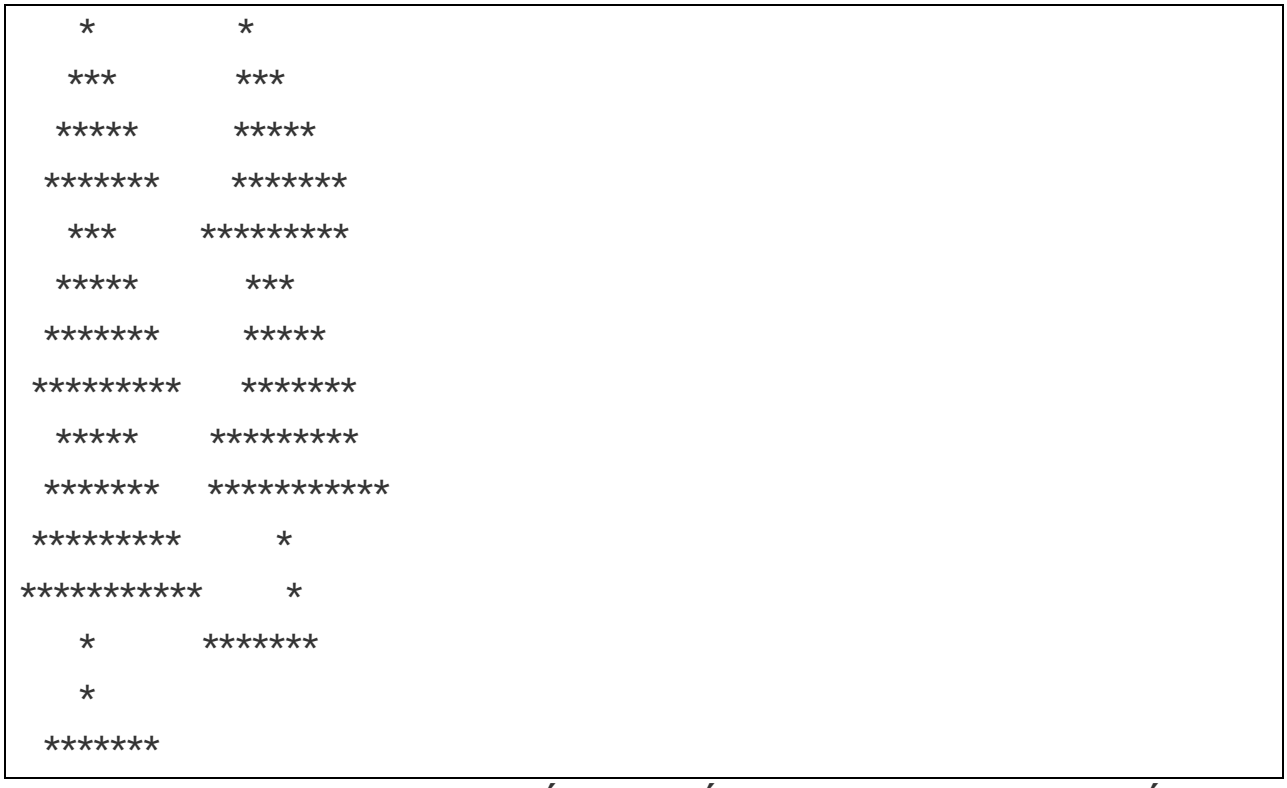
22. **Viết một chương trình xuất ra "The Name Game", trong đó người dùng nhập họ và tên, và một bài hát về**

tên của họ sẽ được in ra. Sử dụng phương thức để tránh lặp lại mã.

23. **Viết một phương thức gọi là printIndexed chấp nhận một chuỗi làm tham số và in ra các ký tự của chuỗi đó theo thứ tự, tiếp theo là các chỉ số của chúng theo thứ tự ngược lại. Ví dụ, lệnh gọi printIndexed("ZELDA") sẽ in Z4E3L2D1A0 lên giao diện điều khiển.**

Dự án Lập trình

1. Viết một chương trình tạo ra hình ảnh cây thông Giáng sinh làm đầu ra. Chương trình cần có một phương thức với hai tham số: một cho số lượng phân đoạn của cây và một cho chiều cao của mỗi phân đoạn. Ví dụ, cây bên trái có ba phân đoạn với chiều cao 4 và cây bên phải có hai phân đoạn với chiều cao 5:



2. Một ngân hàng cung cấp lãi suất 6.5% cho tài khoản tiết kiệm, lãi suất cộng dồn hàng năm. Tạo một bảng hiển thị số tiền mà một người sẽ tích lũy được trong khoảng thời gian

25 năm, giả sử người đó đầu tư ban đầu $1000 và gửi thêm $100 mỗi năm sau năm đầu tiên. Bảng của bạn nên hiển thị số dư hiện tại, lãi suất, tiền gửi mới và số dư mới cho mỗi năm.

3. Viết một chương trình hiển thị tổng số quà mà người trong bài hát "The Twelve Days of Christmas" nhận được vào mỗi ngày, như được chỉ ra trong Bảng 3.5.

Bảng 3.5 Twelve Days of Christmas

Ngày	Số quà nhận được	Tổng số quà
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15
...	...	...

4. Viết một chương trình yêu cầu nhập độ dài các cạnh của một tam giác và báo cáo ba góc của tam giác đó.
5. Viết một chương trình tính khoảng cách hình cầu giữa hai điểm trên bề mặt Trái Đất, dựa trên vĩ độ và kinh độ của chúng. Điều này hữu ích vì nó cho biết khoảng cách giữa hai thành phố nếu nhân khoảng cách với bán kính Trái Đất, khoảng 6372.795 km.

Giả sử φ1, λ1, và φ2, λ2 là vĩ độ và kinh độ của hai điểm, tương ứng. △λ là chênh lệch kinh độ, và △σ là chênh lệch/góc đo trong radian, có thể được xác định như sau từ định luật cosin hình cầu:

$$\triangle\sigma = \arccos(\sin\varphi_1 * \sin\varphi_2 + \cos\varphi_1 * \cos\varphi_2 * \cos\triangle\lambda)$$

Ví dụ, xem xét vĩ độ và kinh độ của hai thành phố lớn:

- Nashville, TN: N 36◦7.2′, W 86◦40.2′
- Los Angeles, CA: N 33◦56.4′, W 118◦24.0′

Bạn cần chuyển đổi các tọa độ này sang radian trước khi có thể sử dụng chúng trong công thức. Sau khi chuyển đổi, các tọa độ trở thành:

- Nashville: φ1 = 36.12◦ = 0.6304 rad, Δ1 = −86.67◦ = −1.5127 rad
- Los Angeles: φ2 = 33.94◦ = 0.5924 rad, Δ2 = −118.40◦ = −2.0665 rad

Sử dụng các giá trị này trong phương trình khoảng cách góc, bạn nhận được:

```
r△σ = 6372.795 × 0.45306 = 2887.259 km
```

Vì vậy, khoảng cách giữa hai thành phố này là khoảng 2887 km, hay 1794 dặm. (Lưu ý: Để giải quyết vấn đề này, bạn sẽ cần sử dụng phương thức Math.acos, trả về góc arccosine trong radian.)

6. Viết một chương trình tạo ra các lịch tháng làm đầu ra. Chương trình của bạn nên có một phương thức tạo ra lịch tháng như ví dụ dưới đây, với các tham số chỉ định số ngày trong tháng và ngày của Chủ Nhật đầu tiên trong tháng đó. Trong tháng được hiển thị dưới đây, các giá trị này lần lượt là 31 và 6.

```
Sun   Mon   Tue   Wed   Thu   Fri   Sat
+-----+-----+-----+-----+-----+-----+-----+
|     |     |  1  |  2  |  3  |  4  |  5  |
|  6  |  7  |  8  |  9  | 10  | 11  | 12  |
| 13  | 14  | 15  | 16  | 17  | 18  | 19  |
| 20  | 21  | 22  | 23  | 24  | 25  | 26  |
| 27  | 28  | 29  | 30  | 31  |     |     |
+-----+-----+-----+-----+-----+-----+-----+
```

Một phần khó của chương trình này là làm cho các cột khác nhau thẳng hàng với độ rộng phù hợp. Chúng ta sẽ học các cách định dạng đầu ra tốt hơn trong chương sau. Hiện tại, bạn có thể sao chép phương thức trợ giúp sau vào chương trình của mình và gọi nó để chuyển đổi một số thành chuỗi được căn trái với độ rộng nhất định. Ví dụ, lệnh gọi System.out.print(padded(7, 5)); sẽ in " 7" (số 7 với bốn dấu cách phía trước).

```
// Trả về chuỗi của số n, căn trái với dấu cách cho đến khi đạt độ rộng nhất định.
```

```
public static String padded(int n, int width) {
    String s = "" + n;
    for (int i = s.length(); i < width; i++) {
        s = " " + s;
    }
    return s;
}
```

Supplement 3G Graphics (Optional)

3G.1 Introduction to Graphics
• [redacted]
• Drawing Lines and Shapes
• Colors
• Drawing with Loops
• Text and Fonts
• Images
3G.2 Procedural Decomposition with Graphics
• A Larger Example: [redacted]
3G.3 Case Study: Pyramids
• Unstructured Partial Solution
• Generalizing the Drawing of Pyramids
• Complete Structured Solution

Introduction

One of the most compelling reasons to learn about using objects is that they allow us to draw graphics in Java. Graphics are used for games, computer animations, and modern graphical user interfaces (GUIs), and to render complex images. Graphics are also a good way to practice the use of parameters as discussed in the previous chapter.

In this optional supplement we will examine a few of the basic classes from Java's graphical framework and use them to draw patterned two-dimensional figures of shapes and text onto the screen.

3G.1 Introduction to Graphics

Java's original graphical tools were collectively known as the Abstract Window Toolkit (AWT). The classes associated with the AWT reside in the package [redacted]. In order to create graphical programs like the ones you'll see in this section, you'll need to include the following [redacted] declaration at the top of your programs:

[redacted]

To keep things simple, we'll use a custom class called [redacted] that was written by the authors of this textbook to simplify some of the more esoteric details of Java graphics. Its core code is less than a page long, so we aren't hiding much. You won't need to import [redacted], but you will need to place the file [redacted] in the same folder as your program. If you want to test the [redacted]'s functionality in JShell, run JShell from the same folder where [redacted] is located, then type [redacted] at the prompt.

The drawing panel keeps track of the overall image, but the actual drawing will be done with an object of type [redacted]. The [redacted] class is also part of the [redacted] library.

DrawingPanel

You can create a graphical window on your screen by constructing a [redacted] object. You must specify the width and height of the drawing area. When the following line executes, a window appears immediately on the screen:

[redacted]

[redacted] objects have two public methods, listed in Table 3G.1.

Table 3G.1 Useful Methods for [redacted]

[redacted]	
[redacted]	returns a [redacted] object that can be used to draw onto the panel
[redacted]	sets the background color of the panel to the given color (the default is white)

The typical way that you'll use [redacted] will be to construct a panel of a particular height and width, set its background color (if you don't want the default white background), and then draw something on it using its [redacted] object. The [redacted] appears on the screen at the time that you construct it.

All coordinates are specified as integers. Each (x, y) position corresponds to a different pixel on the computer screen. The word *pixel* is shorthand for “picture element” and represents a single dot on the computer screen.

Pixel

A single small dot on a computer screen.

The coordinate system assigns the upper-left corner of a panel the position (0, 0). As you move to the right of this position, the x value increases. As you move down from this position, the y value increases. For example, suppose that you construct a object with a width of 200 pixels and a height of 100 pixels. The upper-left corner will have the coordinates (0, 0), and the lower-right corner will have the coordinates (199, 99) as shown in Figure 3G.1.

Figure 3G.1 The (x, y) Coordinate Space

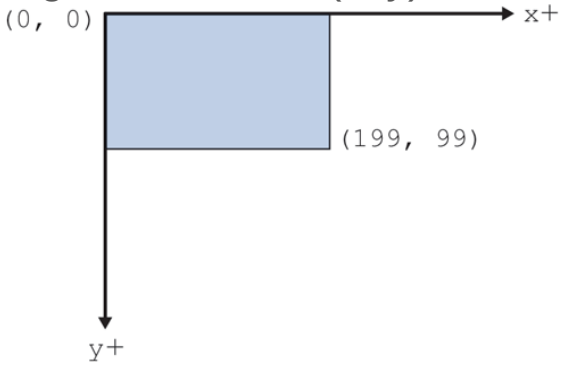


Figure 3G.1 Full Alternative Text

This is likely to be confusing at first, because you're probably used to coordinate systems where y values decrease as you move down. However, you'll soon get the hang of it.

Drawing Lines and Shapes

So, how do you actually draw something? To draw shapes and lines, you don't talk directly to the , but rather to a related object of type . Think of the as a canvas and the object as the paintbrush. The class has a method called that returns its object:

One of the simplest drawing commands is , which takes four integer arguments.

For example, the method call:

draws a line from the point (x1, y1) to the point (x2, y2). The method is just one of many commands a object understands; see Table 3G.2 for others. The

object has many more methods in addition to the ones discussed here. You can read about them in the Java API documentation.

Table 3G.2 Some Useful Methods of [redacted] Objects

[redacted]	
[redacted]	draws a line between the points $(x1, y1)$ and $(x2, y1)$
[redacted]	draws the outline of the largest oval that fits within the specified rectangle
[redacted]	draws the outline of the specified rectangle
[redacted]	draws the given text with its lower-left corner at (x, y)
[redacted]	fills the largest oval that fits within the specified rectangle using the current color
[redacted]	fills the specified rectangle using the current color
[redacted]	sets this graphics context's current color to the specified color (all subsequent graphics operations using this graphics context use this specified color)
[redacted]	sets this graphics context's current font to the specified font (all subsequent strings drawn using this graphics context use this specified font)

Here is a sample program that puts these pieces together:

```
import

public class

public static void
```

```
new
```

When you run this program, the window shown in Figure 3G.2 appears. Though it isn't text on the console, as in previous chapters, we'll still refer to this as the "output" of the program.

Figure 3G.2 Output of

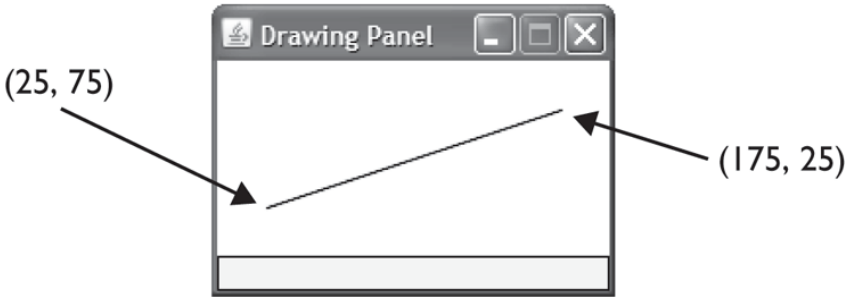


Figure 3G.2 Full Alternative Text
(Java can be run on a variety of systems. Depending on your operating system, your output may differ slightly from the screenshots in this chapter.)

The first statement in [redacted] constructs a [redacted] with a width of 200 and a height of 100. Once it has been constructed, the window will pop up on the screen. The second statement draws a line from (25, 75) to (175, 25). The first point is in the lower-left part of the window (25 over from the left, 75 down from the top). The second point is in the upper-right corner (175 over from the left, 25 down from the top).

Notice these particular lines of code:

[redacted]

You might wonder why you can't just say:

The problem is that there are two different objects involved in this program: the [redacted] itself (the canvas) and the [redacted] object associated with the panel (the paintbrush). The panel doesn't know how to draw a line; only the [redacted] object knows how to do this. You have to be careful to make sure that you are talking to the right object when you give a command.

This requirement can be confusing, but it is common in Java programs. In fact, in a typical Java program, there are hundreds (if not thousands) of objects interacting with each other. These interactions aren't so unlike interactions between people. If you want to schedule a meeting, a busy corporate executive might tell you, "Talk to my secretary about that." Or if you're asking difficult legal questions, a person might tell you, "Talk to my lawyer about that." In this case, the [redacted] doesn't know how to draw, so if it could talk it would say, "Talk to my [redacted] object about that."

It's also legal to use the [redacted] object without storing it in a variable, like this:

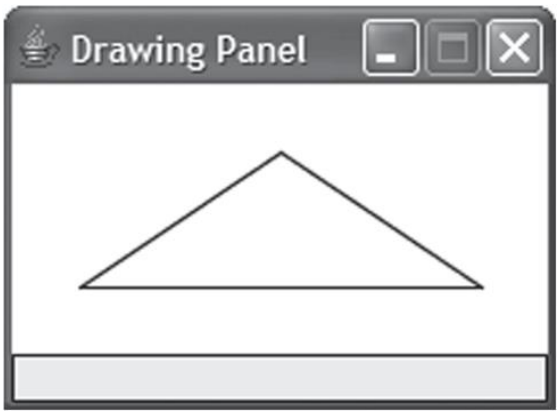
But you'll often want to send several commands to the [redacted] object, so it's more convenient to give it a name and store it in a variable.

Let’s look at a more complicated example:

```
import

public class

public static void

new
```

This program draws three different lines to form a triangle, as shown in Figure 3G.3. The lines are drawn between three different points. In the lower-left corner we have the point (25, 75). In the middle at the top we have the point (100, 25). And in the lower-right corner we have the point (175, 75). The various calls on [redacted] simply draw the lines that connect these three points.

Figure 3G.3 Output of [redacted]



The [redacted] object also has methods for drawing particular shapes. For example, you can draw rectangles with the [redacted] method:

[redacted]

This draws a rectangle with upper-left coordinates (x, y) and the given height and width.

Another figure you'll often want to draw is a circle or, more generally, an oval. But how do you specify where it appears and how big it is? What you actually specify is what is known as the "bounding rectangle" of the circle or oval. Java will draw the largest oval possible that fits inside that rectangle. So, the method call:

[redacted]

draws the largest oval that fits within the rectangle with upper-left coordinates (x, y) and the given height and width.

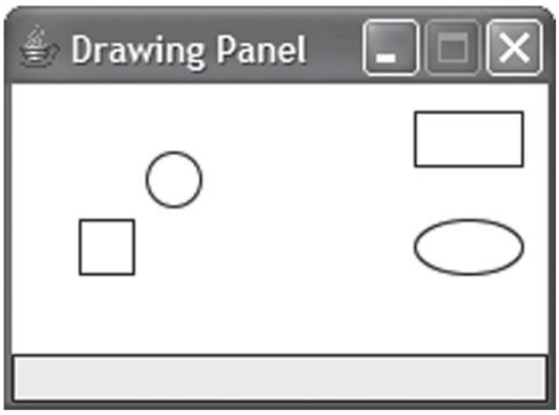
Notice that the first two values passed to [redacted] and [redacted] are coordinates, while the next two values are a width and a height. For example, here is a short program that draws two rectangles and two ovals:

```
import

public class

public static void
```

```
new
```

Figure 3G.4 shows the output of the program.

Figure 3G.4 Output of



The first rectangle has its upper-left corner at the coordinates (25, 50). Its width and height are each 20, so this is a square. The coordinates of its lower-right corner would be (45, 70), or 20 more than the (x, y) coordinates of the upper-left corner. The program also draws a rectangle with its upper-left corner at (150, 10) that has a width of 40 and a height of 20 (wider than it is tall). The bounding rectangle of the first oval has upper-left coordinates (50, 25) and a width and height of 20. In other words, it's a circle. The bounding rectangle of the second oval has upper-left coordinates (150, 50), a width of 40, and a height of 20 (it's an oval that is wider than it is tall).

Sometimes you don't just want to draw the outline of a shape; you want to paint the entire area with a particular color. There are variations of the [redacted] and [redacted] methods known as [redacted] and [redacted] that do exactly that, drawing a rectangle or oval and filling it in with the current color of paint (the default is black). Let's change two of the calls in the previous program to be "fill" operations instead of "draw" operations:

[redacted]

```
import

public class

public static void

new
```

```
g.fillRect(25, 50, 20, 20);









g.fillOval(150, 50, 40, 20);
```

Now we get the output shown in Figure 3G.5 instead.

Figure 3G.5 Output of

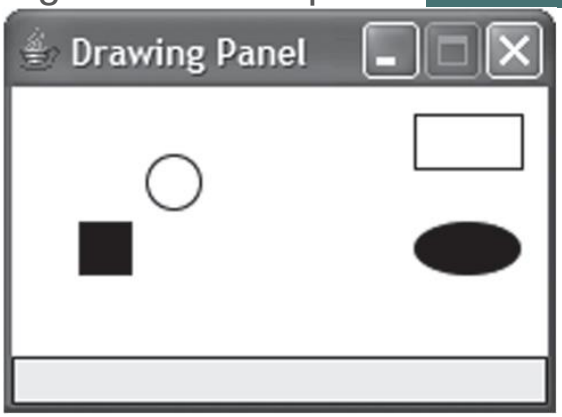


Figure 3G.5 Full Alternative Text

Colors

All of the shapes and lines drawn by the preceding programs were black, and all of the panels had a white background. These are the default colors, but you can change the background color of the panel, and you can change the color being used by the [redacted] object as many times as you like. To change these colors, you use the standard [redacted] class, which is part of the [redacted] package.

There are a number of predefined colors that you can refer to directly. They are defined as class constants in the [redacted] class (a lot like the constants we used in Chapter 2). The names of these constants are all in uppercase and are self-explanatory. To refer to one of these colors, you have to precede it with the class name and a dot, as in [redacted] or [redacted]. The predefined [redacted] constants are listed in Table 3G.3.

Table 3G.3 Color Constants

[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	
[redacted]	[redacted]	

As mentioned earlier, the [redacted] object has a method that can be used to change the background color that covers the entire panel:

Likewise, the [redacted] object has a method that can be used to change the current color that draws or fills shapes and lines:

Calling [redacted] is like dipping your paintbrush in a different color of paint. From that point on, all drawing and filling will be done in the specified color. For example, here is another version of the previous program that uses a cyan (light blue) background color and fills in the oval and square with white instead of black:

```
public class




public static void









panel.setBackground(Color.CYAN);
```

```
g.setColor(Color.WHITE);
```

This program produces the output shown in Figure 3G.6. (The figures shown in this textbook may not match the colors you would see on your screen.)

Figure 3G.6 Output of

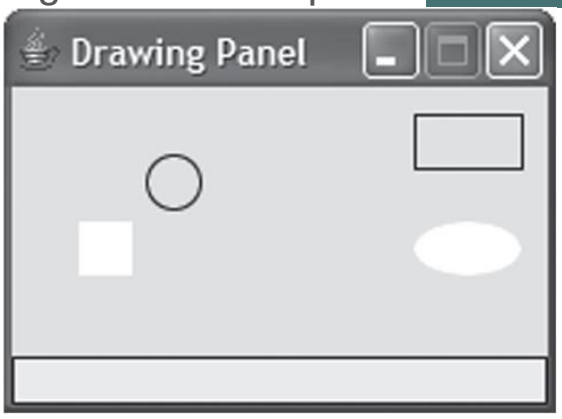


Figure 3G.6 Full Alternative Text

Notice that you tell the panel to set the background color, while you tell the [redacted] object to set the foreground color. The reasoning is that the background color is a property of the entire window, while the foreground color affects only the particular shapes that you draw.

Notice also that the order of the calls has been rearranged. The two drawing commands appear first, then the call on [redacted] that changes the color to white, then the two filling commands. This ensures that the drawing is done in black and the filling is done in white. The order of operations is very important in these drawing programs, so you'll have to keep track of what your current color is each time you give a new command to draw or fill something.

COMMON PROGRAMMING ERROR

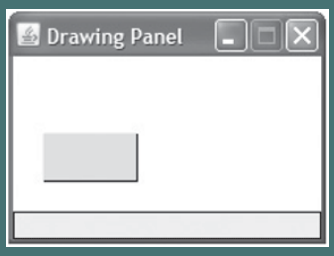
Misunderstanding Draw vs. Fill

[redacted]

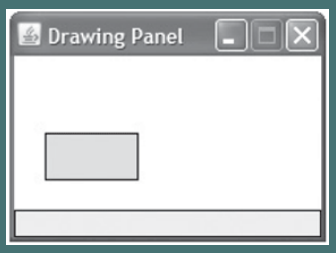
Some new programmers think that a shape must be drawn (such as with [redacted]) before it can be filled in (such as with [redacted]). This is not the case. In fact, when you are trying to draw an outlined shape, this is exactly the wrong thing to do. Suppose you want to draw a 60×30 green rectangle with a black border at (20, 50). You might write the following code:

[redacted]

However, the fill command covers the same pixels as the draw command, and the green interior will be drawn over the black outline, leading to the following appearance:



Instead, the code should fill the interior of the rectangle first, then draw the black outline, to make sure that the outline shows on top of the filling. The following is the correct code and its output:



Drawing with Loops

In each of the preceding examples we used simple constants for the drawing and filling commands, but it is possible to use expressions. For example, suppose that we stick with our [redacted] size of 200 pixels wide and 100 pixels tall and we want to produce a diagonal series of four rectangles that extend from the upper-left corner to the lower-right corner, each with a white oval inside. In other words, we want to produce the output shown in Figure 3G.7.

Figure 3G.7 Desired Output of [redacted]

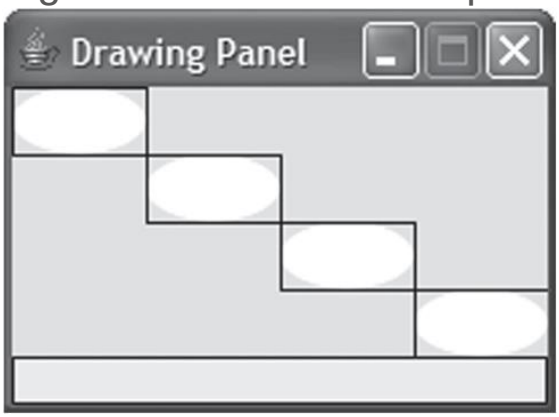


Figure 3G.7 Full Alternative Text

The overall width of 200 and overall height of 100 are divided evenly into four rectangles, which means that they must all be 50 pixels wide and 25 pixels high. So, width and height values for the four rectangles are the same, but the positions of their upper-left corners are different. The first rectangle's upper-left corner is at (0, 0), the second is at (50, 25), the third is at (100, 50), and the fourth is at (150, 75). We need to write code to generate these different coordinates.

This is a great place to use a [redacted] loop. Using the techniques introduced in Chapter 2, we can make a table and develop a formula for the coordinates. In this case it is easier to have the loop start with 0 rather than 1, which will often be the case with drawing programs. Here is a program that makes a good first stab at generating the desired output:

```
import
```

```
public class


public static void


new


for

int
```

This program produces the output shown in Figure 3G.8.

Figure 3G.8 Output of

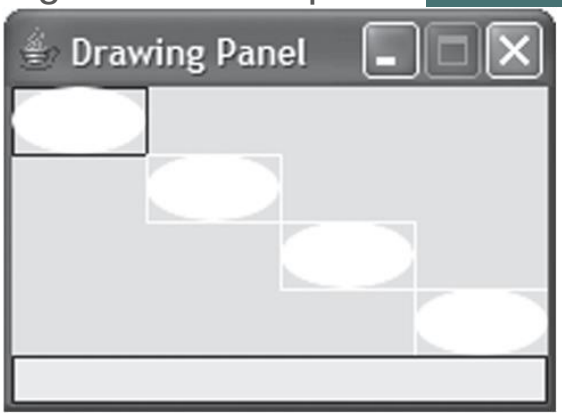


Figure 3G.8 Full Alternative Text

The coordinates and sizes are right, but not the colors. Instead of getting four black rectangles with white ovals inside, we're getting one black rectangle and three white rectangles. That's because we only have one call on inside the loop. Initially the color will be set to black, which is why the first rectangle comes out black. But once we make a call on changing the

color to white, every subsequent drawing and filling command is done in white, including the second, third, and fourth rectangles.

So, we need to include calls to set the color to black, to draw the rectangles, and to set the color to white to draw the filled ovals. While we're at it, it's a good idea to switch the order of these tasks. The rectangles and ovals overlap slightly, and we would rather have the rectangle drawn over the oval than the other way around. The following program produces the correct output:

```
import

public class

public static void
```

```
for

int



g.setColor(Color.WHITE);



g.setColor(Color.BLACK);
```

It’s also possible to create custom objects of your own, rather than using the constant colors provided in the class. Computer monitors use red, green, and blue (RGB) as their primary colors, so when you construct a object you pass your own parameter values for the redness, greenness, and blueness of the color:

The red/green/blue components should be integer values between and . The higher the value, the more of that color is mixed in. All values produce black, and all values produce white. Values of produce a pure green, while values of make a dark purple color (because red and blue are mixed). Search for “RGB table” in your favorite search engine to find tables of many common colors.

The following program demonstrates the use of custom colors. It uses a class constant for the number of rectangles to draw and produces a blend of colors from black to white:

```
import

public class

public static final int

public static void
```

```
new

new

for

int

int

new
```

This program produces the output shown in Figure 3G.9.

Figure 3G.9 Output of

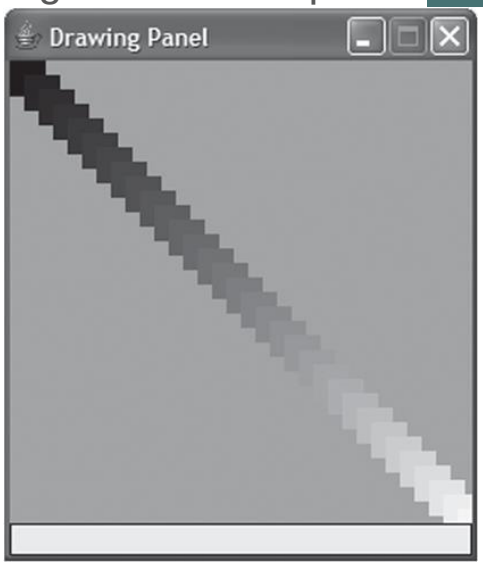


Figure 3G.9 Full Alternative Text

It is also legal to store a object into a variable or pass it as a parameter. For example, we could have written the coloring code in the preceding program as follows:

We will use this idea later when parameterizing colors in this chapter's Case Study.

Text and Fonts

Another drawing command worth mentioning can be used to include text in your drawings. The [redacted] method of the [redacted] object draws the given [redacted] with its lower-left corner at coordinates (x, y):

This is a slightly different convention than we used for [redacted]. With [redacted], we specified the coordinates of the upper-left corner. Here we specify the coordinates of the lower-left corner. By default, text is drawn approximately 10 pixels high. Here is a sample program that uses a loop to draw a particular [redacted] 10 different times, each time indenting it 5 pixels to the right and moving it down 10 pixels from the top:

```
import

public class

public static void

new
```

```
for

int
```

This program produces the output shown in Figure 3G.10.

Figure 3G.10 Output of

Fonts are used to describe different styles for writing characters on the screen. If you'd like to change the style or size of the onscreen text, you can use the [redacted] method of the [redacted] object.

Font

[redacted]

An overall design for a set of text characters, including the style, size, weight, and appearance of each character.

This method changes the text size and style in which strings are drawn.

The parameter to [redacted] is a [redacted] object. A [redacted] object is constructed by passing three parameters—the font's name as a [redacted], its style (such as bold or italic), and its size as an integer:

[redacted]

Common font styles such as bold are implemented as constants in the [redacted] class. The available constants and some popular font names are listed in Tables 3G.4 and 3G.5.

Table 3G.4 Useful Constants of the [redacted] Class

[redacted]	
[redacted]	Bold text
[redacted]	*Italic text*
[redacted]	*Bold/Italic text*
[redacted]	Plain text

Table 3G.5 Common Font Names

[redacted]	
[redacted]	a typewriter font, such as [redacted]
[redacted]	a font without curves (serifs) at letter edges, such as Arial
[redacted]	a font with curved edges, such as Times New Roman

As in the case of colors, setting the font affects only strings that are drawn after the font is set. The following program sets several fonts and uses them to draw strings:

[redacted]

```
import

public class

public static void

new
```

new

new

new

This program produces the output shown in Figure 3G.11.

Figure 3G.11 Output of

Figure 3G.11 Full Alternative Text

Images

The is also capable of displaying images loaded from files in formats such as JPEG, PNG, and GIF. To display an image, first you must find an image file (such as one on the internet or on your computer) and place it into the same directory as your code project. Images are displayed in two steps: first the image must be loaded from the hard drive into an object, and then your panel's object can display the image.

The x and y coordinates passed when drawing the image represent its top/left corner pixel position.

There are a few quirks to the syntax. One is that we use the [redacted] to load the image, while you use the [redacted] object to draw it. It's easy to accidentally get the two mixed up. Also, unlike the other drawing commands, [redacted] requires you to pass the [redacted] as a last parameter to the method. This is required by Java's [redacted] class in order for the code to compile.

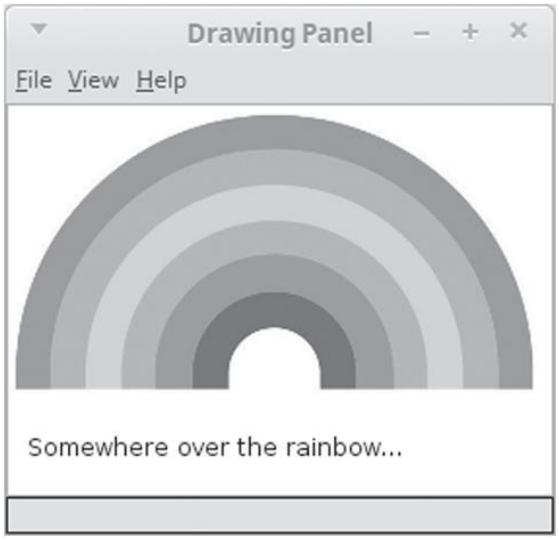
For example, the following program loads an image that looks like a drawing of a rainbow and draws it onto the [redacted] with a text string underneath it.

```
public class
```

```
public static void




new
```

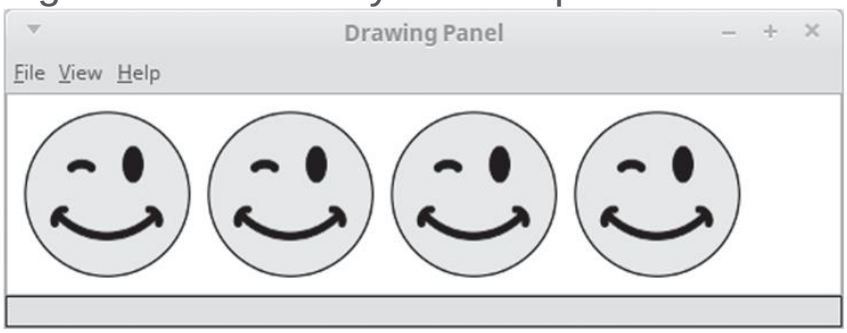
The program produces the output shown in Figure 3G.12.

Figure 3G.12 Output of



If you want to draw the same image multiple times on the panel, you don't need to repeat the [redacted] part of the process. It is much more efficient to load the image a single time and then draw it as many times as you like. The following code would draw several copies of an image in a file named [redacted], which is 100×100 pixels in size. The output of this code is shown in Figure 3G.13.

Figure 3G.13 Smiley face output



[redacted]

3G.2 Procedural Decomposition with Graphics

If you write complex drawing programs, you will want to break them down into several static methods to structure the code and to remove redundancy. When you do this, you'll have to pass the [redacted] object to each static method that you introduce. For a quick example, the [redacted] program from the previous section could be split into a [redacted] method and a [redacted] method, as follows:

```
import

public class
```

```
public static void

new

drawText(g);

public static void

Graphics g
```

```
for

int
```

This program produces the same output as the original program (Figure 3G.10).

The program wouldn’t compile without passing [redacted] to the [redacted] method, because [redacted] is needed to call drawing methods such as [redacted] and [redacted].

A Larger Example: [redacted]

Now let’s consider a slightly more complicated task: drawing the largest diamond figure that will fit into a box of a particular size. The largest diamond that can fit into a box of size 50×50 is shown in Figure 3G.14.

Figure 3G.14 Diamond

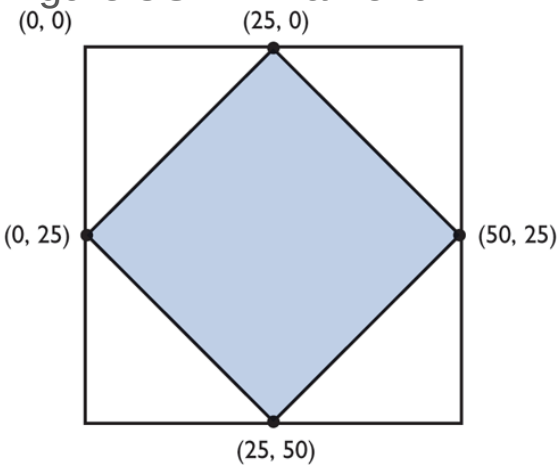


Figure 3G.14 Full Alternative Text

The code to draw such a diamond would be the following:

Now imagine that we wish to draw three such 50×50 diamonds at different locations. We can turn our diamond-drawing code into a [redacted] method that we'll call three times. Since each diamond will be in a different position, we can pass the x- and y-coordinates as parameters to our [redacted] method.

A diamond enclosed by a box with top-left corner at the location (78, 22) is shown in Figure 3G.15.

Figure 3G.15 Diamond at (78, 22)

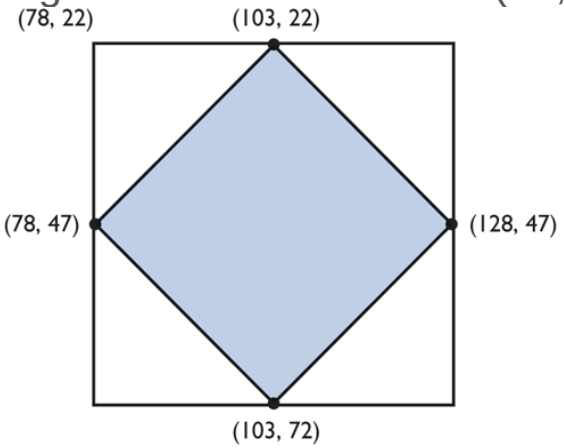


Figure 3G.15 Full Alternative Text

The code to draw this diamond would be the following:

[redacted]

As you can see, the parameter values passed to the [redacted] and [redacted] methods are very similar to those of the first diamond, except that they're shifted by 78 in the x-direction and 22 in the y-direction (except for the third and fourth parameters to [redacted], since these are the rectangle's width and height). This (78, 22) shift is called an *offset.*

We can generalize the coordinates to pass to [redacted]'s drawing commands so that they'll work with any diamond if we pass that diamond's top-left x- and y-offset. For example, we'll generalize the line from (0, 25) to (25, 0) in the first diamond and from (78, 47) to (103, 22) in the second diamond by saying that it is a line from (x, y+25) to (x+25, y), where (x, y) is the offset of the given diamond.

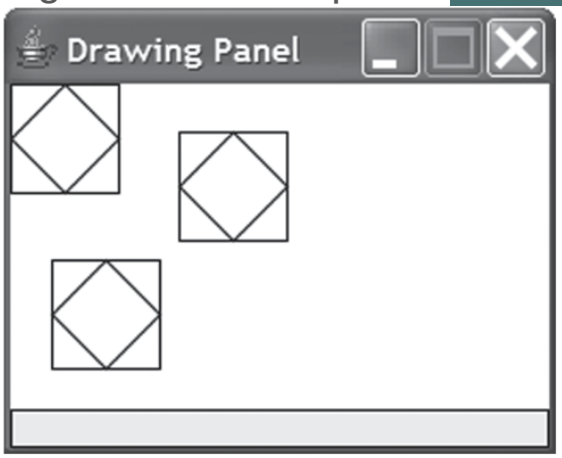
The following program uses the [redacted] method to draw three diamonds without redundancy:

```
public class


public static void


new
```

```
public static void

int

int
```

This program produces the output shown in Figure 3G.16.

Figure 3G.16 Output of



It's possible to draw patterned figures in loops and to have one drawing method call another. For example, if we want to draw five diamonds, starting at (12, 15) and spaced 60 pixels apart, we just need a loop that repeats five times and shifts the x-coordinate by 60 each time. Here's an example loop:

If we created another method to draw the line of five diamonds, we could call it from to draw many lines of diamonds. Here's

a modified version of the [redacted] program with two graphical methods:

```
import

public class

public static void

new
```

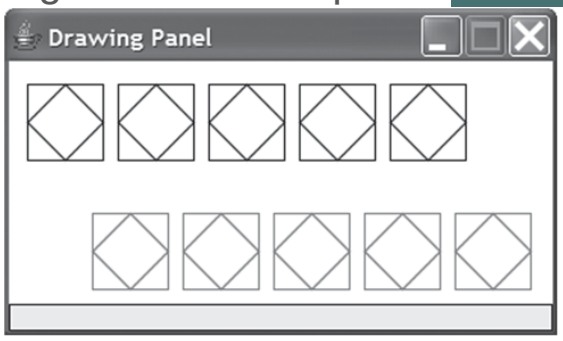
```
public static void


int

int
```

```
for

int
```

```
public static void

int

int
```

This program produces the output shown in Figure 3G.17.

Figure 3G.17 Output of



3G.3 Case Study: Pyramids

Imagine that you've been asked to write a program that will draw the images in Figure 3G.18 onto a [redacted].

Figure 3G.18 Desired Pyramids Output

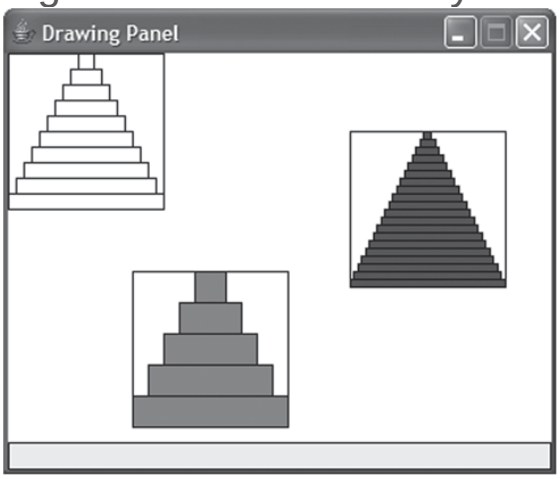


Figure 3G.18 Full Alternative Text

The overall drawing panel has a size of 350×250. Each pyramid is 100 pixels high and 100 pixels wide. The pyramids consist of centered flights of colored stairs that widen toward the bottom, with black outlines around each stair. Table 3G.6 lists the attributes of each pyramid.

Table 3G.6 Pyramid Attributes

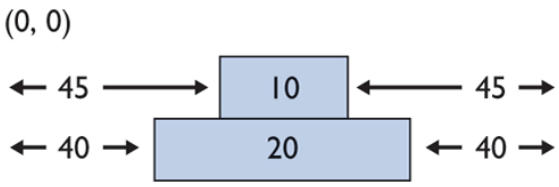
[redacted]

white	(0, 0)	10 stairs	10 pixels
red	(80, 140)	5 stairs	20 pixels
blue	(220, 50)	20 stairs	5 pixels

Unstructured Partial Solution

When trying to solve a larger and more complex problem like this, it's important to tackle it piece by piece and make iterative enhancements toward a final solution. Let's begin by trying to draw the top-left white pyramid correctly.

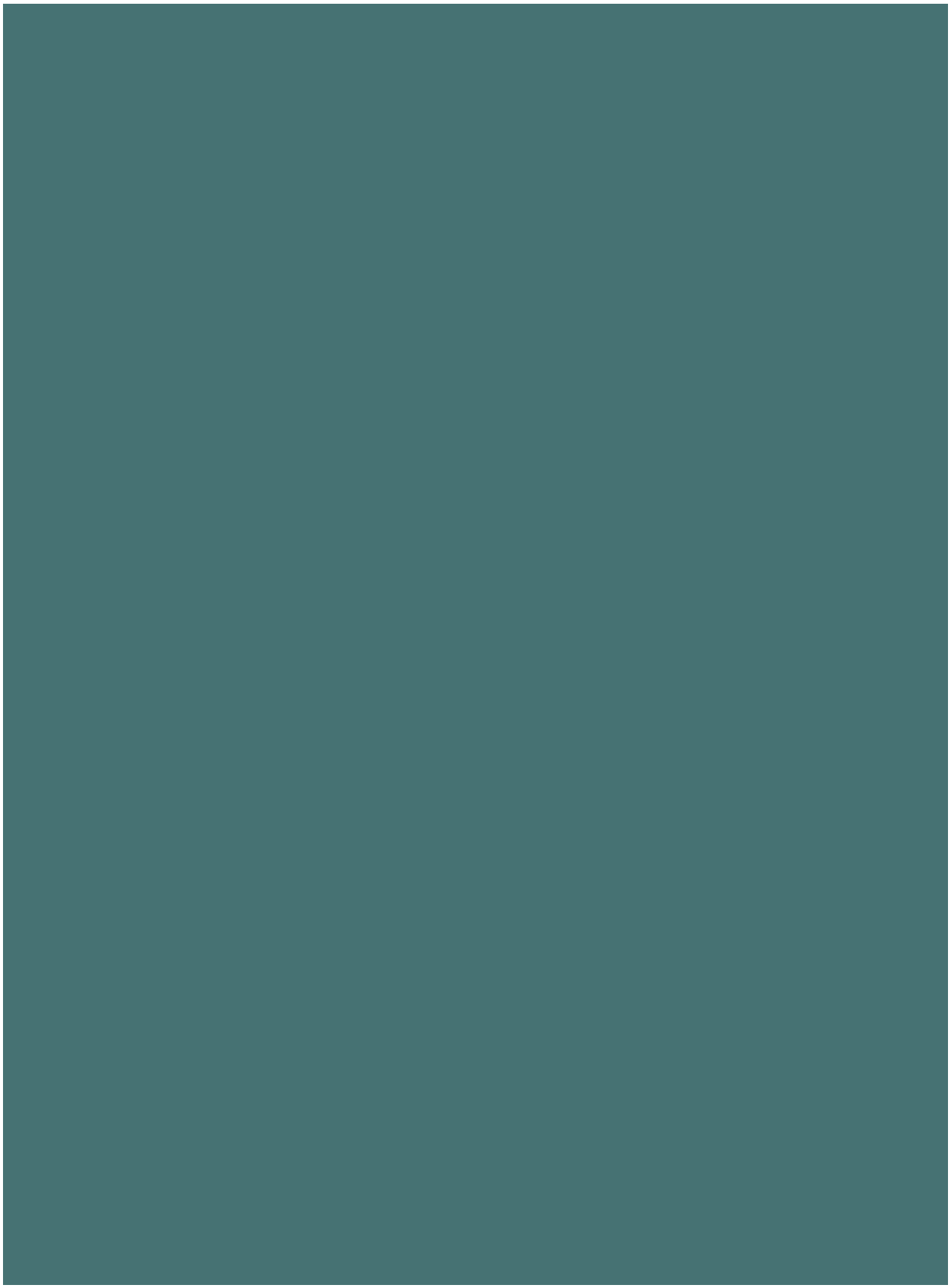
Each stair is centered horizontally within the pyramid. The top stair is 10 pixels wide. Therefore, it is surrounded by 90/2 or 45 pixels of empty space on either side. That means that the 10×10 rectangle's top-left corner is at (45, 0). The second stair is 20 pixels wide, meaning that it's surrounded by 80/2 or 40 pixels on each side:



3G.3-9 Full Alternative Text

The following program draws the white pyramid in the correct position:

```
import


public class


public static void


new
```

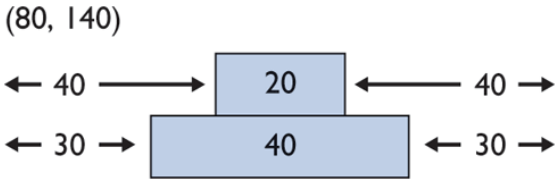


Looking at the code, it's clear that there's a lot of redundancy among the 10 lines to draw the stairs. Examining the patterns of numbers in each column reveals that the x value decreases by 5 each time, the y value increases by 10 each time, the width increases by 10 each time, and the height stays the same.

Another way of describing a stair's x value is to say that it is half of the overall 100 minus the stair's width. With that in mind, the following loop draws the 10 stairs without the previous redundancy:

Generalizing the Drawing of Pyramids

Next let's add code to draw the bottom (red) pyramid. Its (x, y) position is (80, 140) and it has only five stairs. That means each stair is twice as tall and wide as those in the white pyramid.



3G.3-10 Full Alternative Text

Given this information, we can determine that the top stair's upper-left corner is at (120, 140) and its size is 20×20, the second stair's upper-left corner is at (110, 160) and its size is 40×20, and so on.

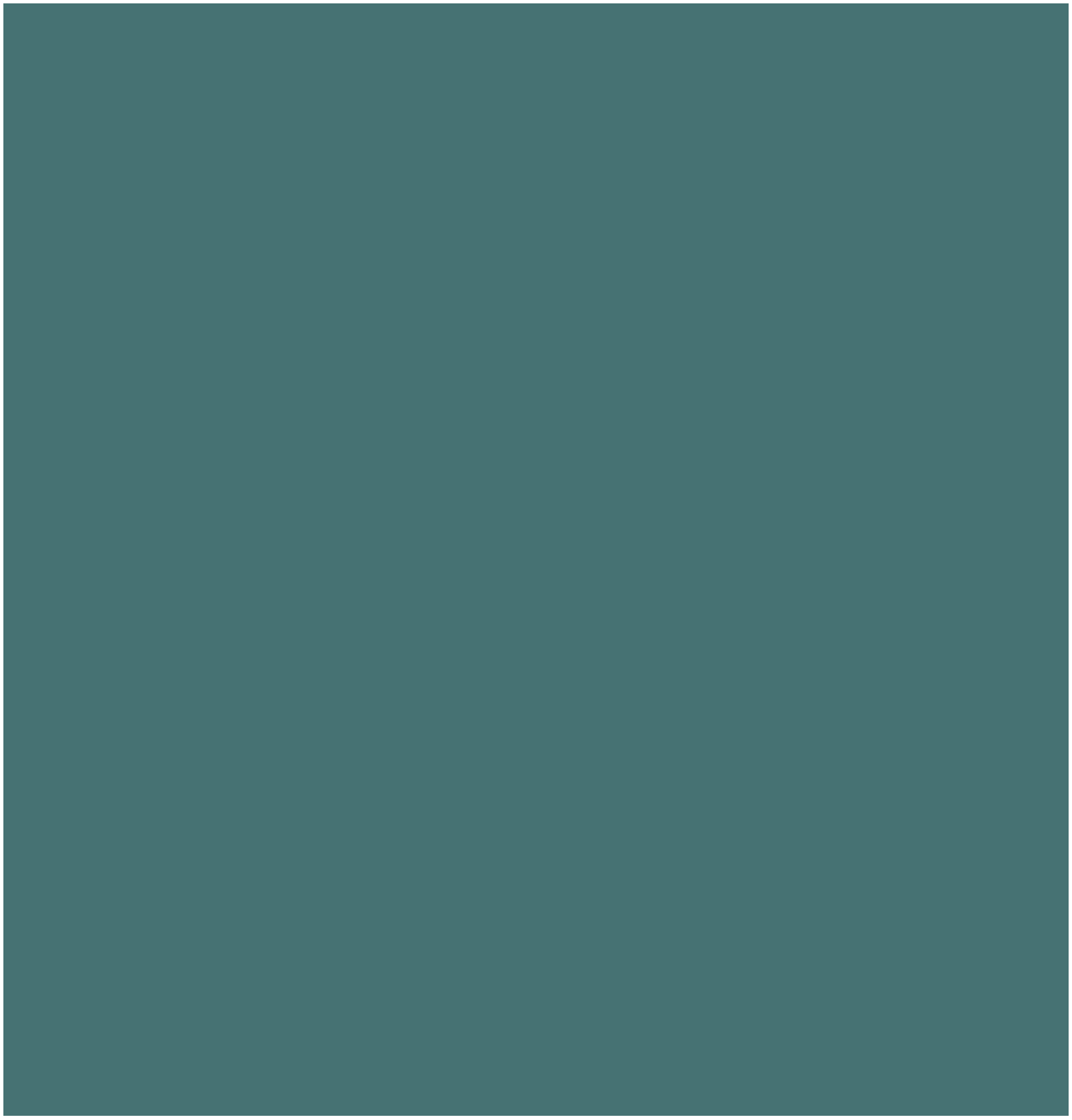
For the moment, let's focus on getting the coordinates of the stairs right and not worry about the red fill color. Here is a redundant bit of code to draw the red pyramid's stairs, without the coloring:

Again we have redundancy among the five lines to draw the stairs, so let's look for a pattern. We'll use a loop to eliminate the redundancy like we did for the last pyramid, but with appropriate modifications. Each stair's height is now 20 pixels, and each stair's width is now 20 times the number for that stair. The x- and y-coordinates are a bit trickier. The x-coordinate formula is similar to the [redacted] from before, but this time it must be shifted right by 80 to account for the position of its bounding box's top-left corner. The y-coordinate must similarly be shifted downward by 140 pixels. Here's the correct loop:

```
5

20

20
```

Can you spot the pattern between the two [redacted] loops used to draw the stairs of the pyramids? The x- and y-coordinates differ only in the addition of the offset from (0, 0) in the second loop. The stairs' widths and heights differ only in that one pyramid's stairs are 20 pixels tall and the other pyramid's stairs are 10 pixels tall (the result of dividing the overall size of 100 by the number of stairs).

Using the preceding information, let's turn the code for drawing a pyramid into a method that we can call three times to avoid redundancy. The parameters will be the (x, y) coordinates of the top-left corner of the pyramid's bounding box and the number of stairs in the pyramid. We'll also need to pass [redacted] as a parameter so that we can draw onto the [redacted]. We'll modify the [redacted] loop to compute the stair height first, then use the height to compute the stair width, and finally use the width and height to help compute the (x, y) coordinates of the stair. Here's the code:



The preceding code is now generalized to draw a pyramid at any location with any number of stairs. But one final ingredient is missing: the ability to give a different color to each pyramid.

Complete Structured Solution

The preceding code is correct except that it doesn't allow us to draw the pyramids in the proper colors. Let's add an additional parameter, a [redacted], to our method and use it to fill the pyramid stairs as needed. We'll pass [redacted] as this parameter's value for the first white pyramid; it'll fill the stairs with white, even though this isn't necessary.

The way to draw a filled shape with an outline of a different color is to first fill the shape, then use the outline color to draw the same shape. For example, to get red rectangles with black outlines, first we'll use [redacted] with red, then we'll use [redacted] with black with the same parameters.

Here's the new version of the [redacted] method that uses the fill color as a parameter:

Using this method, we can now draw all three pyramids easily by calling [redacted] three times with the appropriate parameters:

One last improvement we can make to our [redacted] program is to turn the overall pyramid size of 100 into a constant, so there aren’t so many 100s lying around in the code. Here is the complete program:

```
import

public class

public static final int

public static void
```

```
new
```

```
public static void
```

```
int

int

int

for

int

int

int
```

```
int


int
```

Chapter Summary

[redacted] is a custom class provided by the authors to easily show a graphical window on the screen. A [redacted] contains a [redacted] object that can be used to draw lines, text, and shapes on the screen using different colors.

A [redacted] object has many useful methods for drawing shapes and lines, such as [redacted], [redacted], and [redacted]. Shapes can be “drawn” (drawing only the outline) or “filled” (coloring the entire shape).

The [redacted] object can write text on the screen with its [redacted] method. You can specify different font styles and sizes with the [redacted] method.

Graphical programs that are decomposed into methods must pass appropriate parameters to those methods (for example, the [redacted] object, as well as any (x, y) coordinates, sizes, or other values that guide the figures to be drawn).

Self-Check Problems

Section 3G.1: Introduction to Graphics

1. Which of the following is the correct syntax to draw a rectangle?

   a. [redacted]
   b. [redacted]
   c. [redacted]
   d. [redacted]
   e. [redacted]

2. There are two mistakes in the following code, which attempts to draw a line from coordinates (50, 86) to (20, 35). What are they?

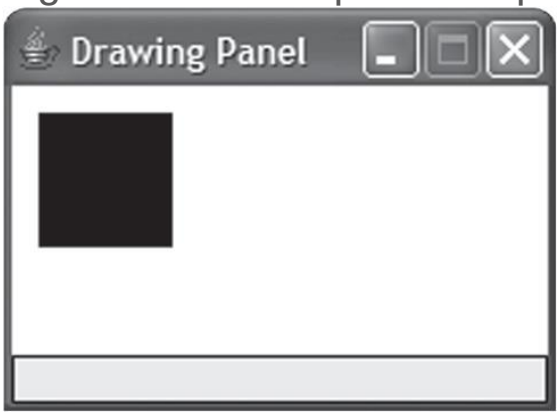
[redacted]

3. The following code attempts to draw a black-filled outer rectangle with a white-filled inner circle inside it:

[redacted]

However, the graphical output looks like Figure 3G.19 instead. What must be changed for it to look as intended?

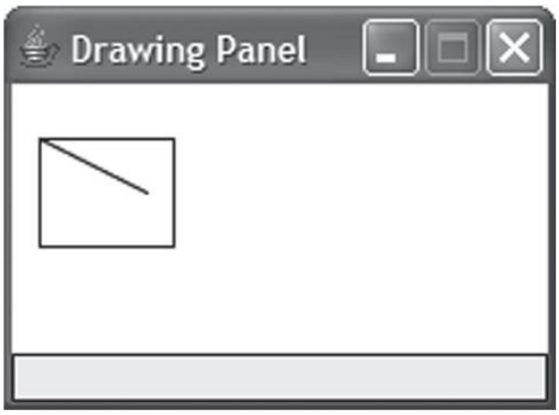
Figure 3G.19 Graphical output of Self-Check 3G.3



4. The following code attempts to draw a black rectangle from (10, 20) to (50, 40) with a line across its diagonal:

However, the graphical output looks like Figure 3G.20 instead. What must be changed for it to look as intended?

Figure 3G.20 Graphical output of Self-Check 3G.4



5. What sort of figure will be drawn by the following program? Can you draw a picture that will approximately match its appearance without running it first?

```
import

public class

public static void
```

```
new

for

int
```

Exercises

1. Write a program that uses the [redacted] to draw Figure 3G.21.

Figure 3G.21 Expected graphical output of Exercise 3G.1

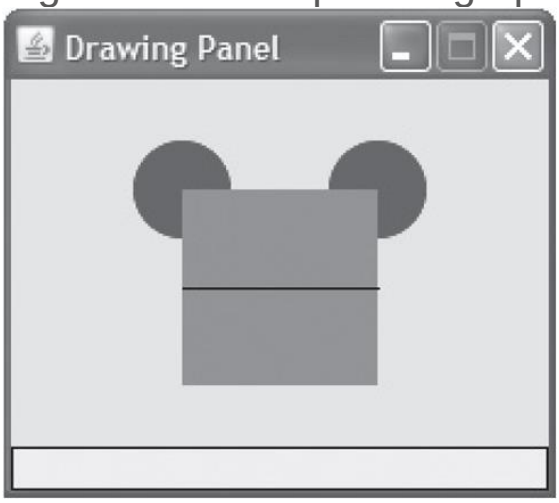


Figure 3G.21 Full Alternative Text

The window is 220 pixels wide and 150 pixels tall. The background is yellow. There are two blue ovals of size 40×40 pixels. They are 80 pixels apart, and the left oval's top-left corner is located at position (50, 25). There is a red square whose top two corners exactly intersect the centers of the two ovals. Lastly, there is a black horizontal line through the center of the square.

2. Modify your program from the previous exercise to draw the figure by a method called [redacted]. The method should accept three parameters: the [redacted] of the [redacted] on which to draw, and a pair of (x, y)

coordinates specifying the location of the top-left corner of the figure. Use the following heading for your method:

```
```

Set your [redacted]'s size to 450×150 pixels, and use your [redacted] method to place two figures on it, as shown in Figure 3G.22. One figure should be at position (50, 25) and the other should be at position (250, 45).

Figure 3G.22 Expected graphical output of Exercise 3G.2

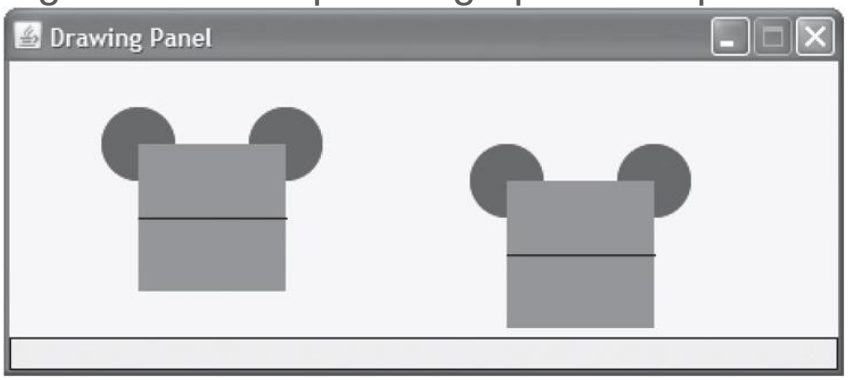


Figure 3G.22 Full Alternative Text

3. Suppose you have the following existing program called [redacted] that uses the [redacted] to draw the face figure shown in Figure 3G.23. Modify the program to draw the modified output shown in Figure 3G.24. Do so by writing a parameterized method that draws a face at different positions. The window size should be changed to 320×180 pixels, and the two faces' top-left corners are at (10, 30) and (150, 50).

Figure 3G.23 Initial graphical output of Exercise 3G.3

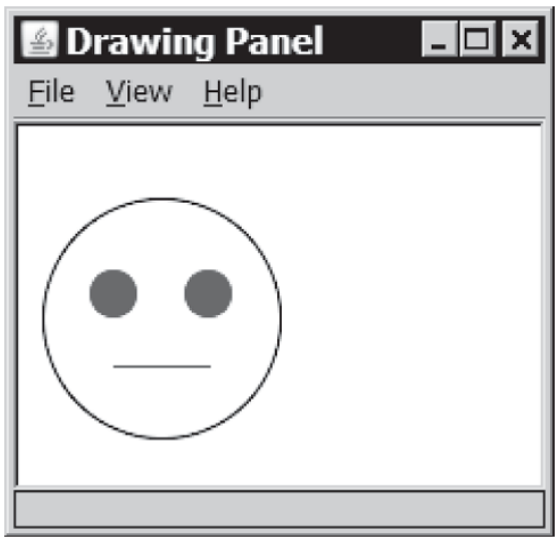
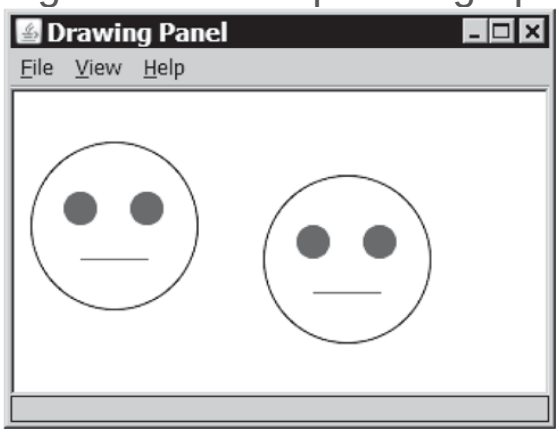


Figure 3G.24 Expected graphical output of Exercise 3G.3

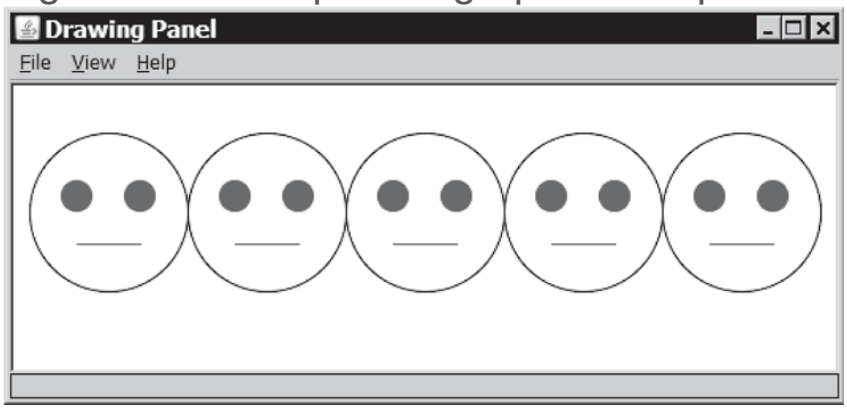



```
public class

public static void
```

new

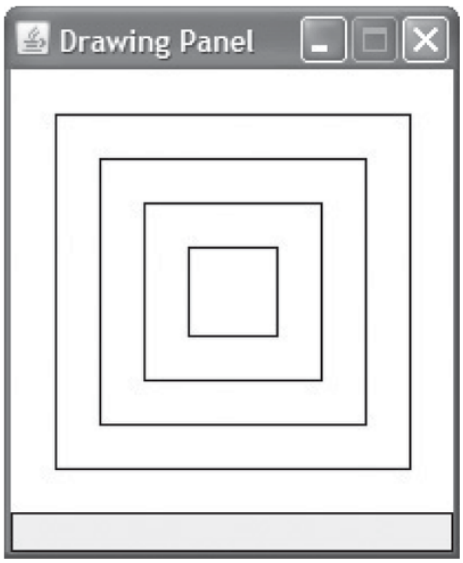
4. Modify your previous [redacted] program to draw the new output shown in Figure 3G.25. The window size should be changed to 520×180 pixels, and the faces' top-left corners are at (10, 30), (110, 30), (210, 30), (310, 30), and (410, 30).

Figure 3G.25 Expected graphical output of Exercise 3G.4



5. Write a program called [redacted] that uses the [redacted] to draw Figure 3G.26.

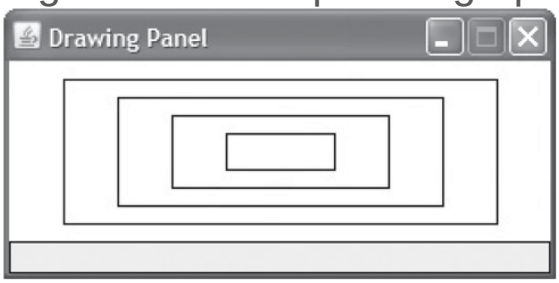
Figure 3G.26 Expected graphical output of Exercise 3G.5



The window is 200 pixels wide and 200 pixels tall. The background is white and the foreground is black. There are 20 pixels between each of the four rectangles, and the rectangles are concentric (their centers are at the same point). Use a loop to draw the repeated rectangles.

6. Modify your [redacted] program from the previous exercise so that it has a method that accepts parameters for the window width and height and displays the rectangles at the appropriate sizes. For example, if your method was called with values of 300 and 100, the window would look like Figure 3G.27.

Figure 3G.27 Expected graphical output of Exercise 3G.6



7. Write a program called [redacted] that uses the [redacted] to draw the shape shown in Figure 3G.28.

Figure 3G.28 Expected graphical output of Exercise 3G.7

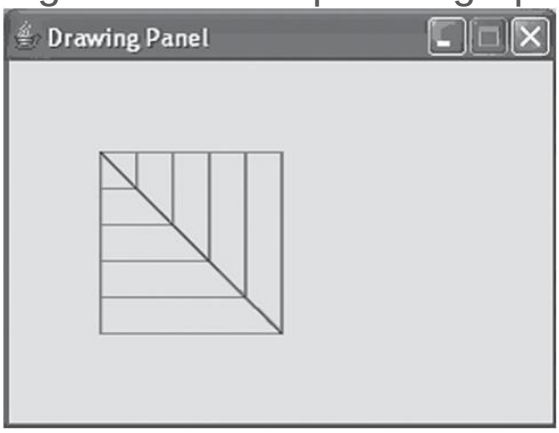


Figure 3G.28 Full Alternative Text

The [redacted] is 300 pixels wide by 200 pixels high. Its background is cyan. The horizontal and vertical lines are drawn in red and the diagonal line is drawn in black. The upper-left corner of the diagonal line is at (50, 50). Successive horizontal and vertical lines are spaced 20 pixels apart.

8. Modify your code from the previous exercise to produce the pattern shown in Figure 3G.29.

Figure 3G.29 Expected graphical output of Exercise 3G.8

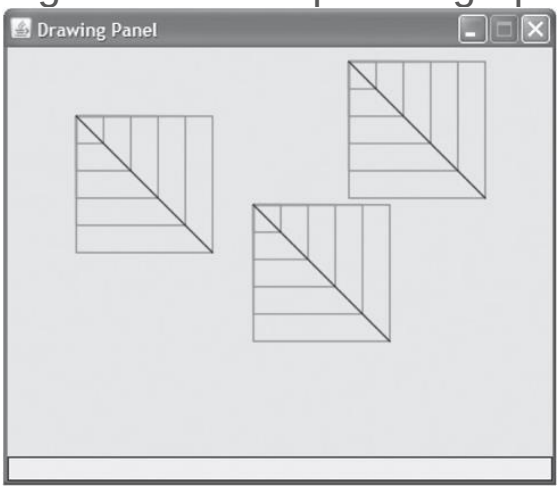


Figure 3G.29 Full Alternative Text

The [redacted] is now 400×300 pixels in size. The first figure is at the same position, (50, 50). The other figures are at positions (250, 10) and (180, 115), respectively. Use one or more parameterized static methods to reduce the redundancy of your solution.

9. Modify your code from the previous exercise to produce the pattern shown in Figure 3G.30.

Figure 3G.30 Expected graphical output of Exercise 3G.9

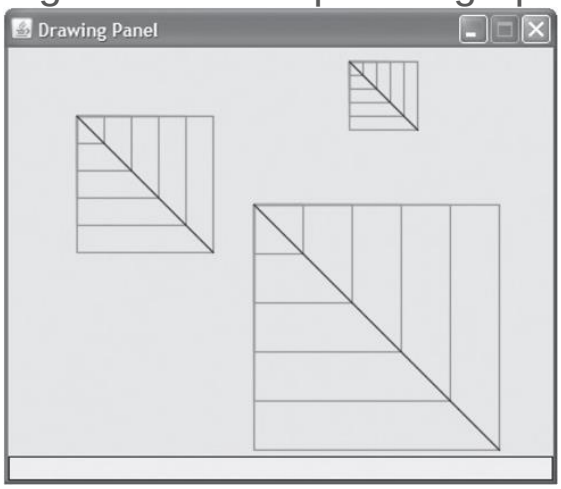


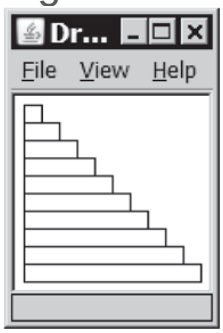
Figure 3G.30 Full Alternative Text

The [redacted] is the same except that now each figure has a different size. The left figure has its original size of 100, the top-right figure has a size of 50, and the bottom-right figure has a size of 180. Use parameterized static methods to reduce the redundancy of your solution.

10. Write a program called [redacted] that uses the [redacted] to draw the figure shown in Figure 3G.31. The first stair's top-left corner is at position (5, 5). The first stair is 10×10 pixels in size. Each stair is 10 pixels wider than the one above it. Make a table with the (x, y) coordinates and (width×height) sizes of the first five

stairs. Note which values change and which ones stay the same.

Figure 3G.31 Expected graphical output of Exercise 3G.10



11. Modify your previous [redacted] program to draw each of the outputs shown in Figure 3G.32. Modify only the body of your loop. (You may want to make a new table to find the expressions for x, y, *width*, and *height* for each new output.)

Figure 3G.32 Expected graphical outputs of Exercise 3G.11

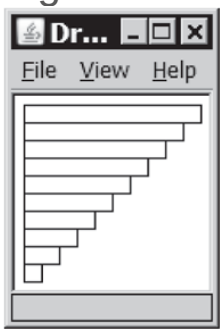
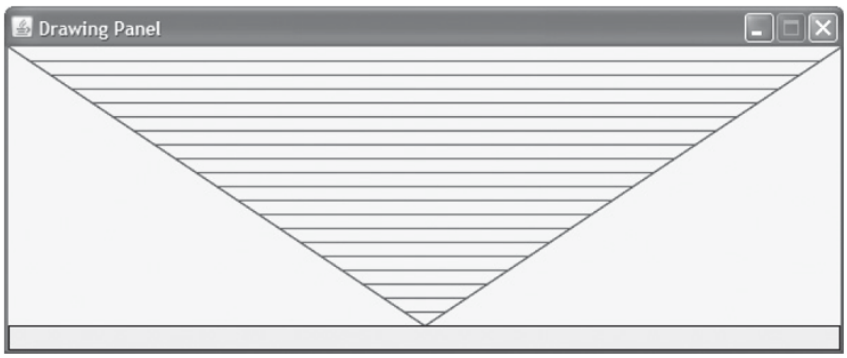


Figure 3G.32 Full Alternative Text

12. Write a program called [redacted] that uses the [redacted] to draw the figure shown in Figure 3G.33.

Figure 3G.33 Expected graphical output of Exercise 3G.12



The window is 600×200 pixels in size. The background is yellow and the lines are blue. The lines are 10 pixels apart vertically, and the diagonal lines intersect at the bottom of the figure in its horizontal center.

13. Write a program called [redacted] that uses the [redacted] to draw the figure shown in Figure 3G.34. Though the figure looks to contain curves, it is entirely made of straight lines.

Figure 3G.34 Expected graphical output of Exercise 3G.13

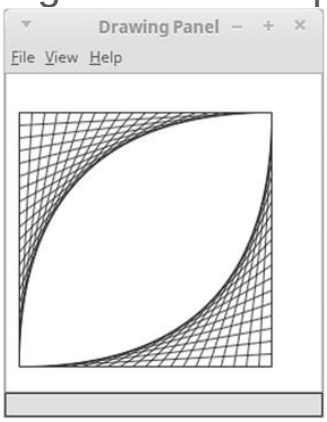


Figure 3G.34 Full Alternative Text

The window is 250×250 pixels in size. There is an outer rectangle from (10, 30) to (210, 230), and a set of black lines drawn around the edges every 10 pixels. For example, along the top-left there is a line from (10, 200) to (20, 30), a line from (10, 190) to (30, 30), a

line from (10, 180) to (40, 30),... and along the bottom-right there is a line from (20, 210) to (210, 200), a line from (30, 210) to (210, 190), and so on.

Programming Projects

1. Write a program that draws the patterns shown in Figure 3G.35 onto a [redacted].

Figure 3G.35 Expected graphical output of Programming Project 3G.1

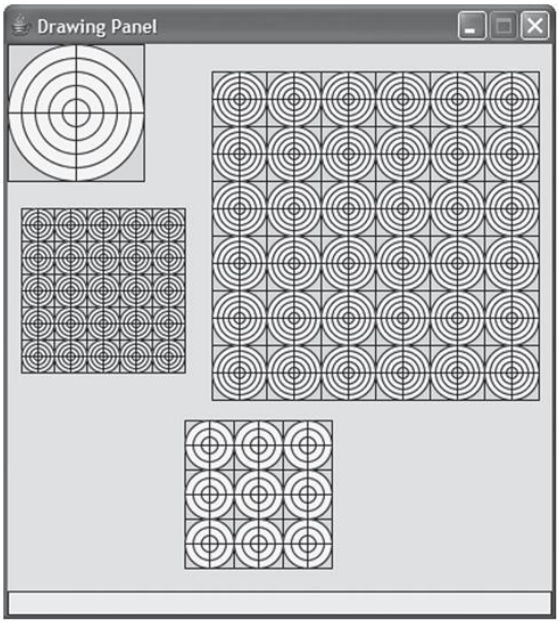


Figure 3G.35 Full Alternative Text

The [redacted]'s size is 400×400 pixels and its background color is cyan. It contains four figures of concentric yellow circles with black outlines, all surrounded by a green rectangle with a black outline. The four figures on your [redacted] should have the properties shown in Table 3G.7.

Table 3G.7 Circle Figure Properties

top left	(0, 0)	100×100	5	1×1
bottom left	(10, 120)	24×24	4	5×5
top right	(150, 20)	40×40	5	6×6
bottom right	(130, 275)	36×36	3	3×3

Break down your program into methods for drawing one subfigure as well as larger grids of subfigures, such as the 5×5 grid at (10, 120).

2. Write a program that draws the image shown in Figure 3G.36 onto a [redacted] of size 200×200. Each stamp is 50×50 pixels in size.

Figure 3G.36 Expected graphical output of Programming Project 3G.2

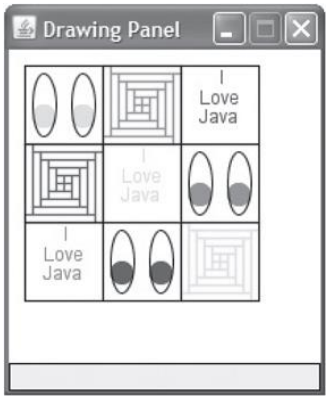


Figure 3G.36 Full Alternative Text

3. Write a program that draws checkerboards like these shown in Figure 3G.37 onto a [redacted] of size 420×300.

Figure 3G.37 Expected graphical output of Programming Project 3G.3

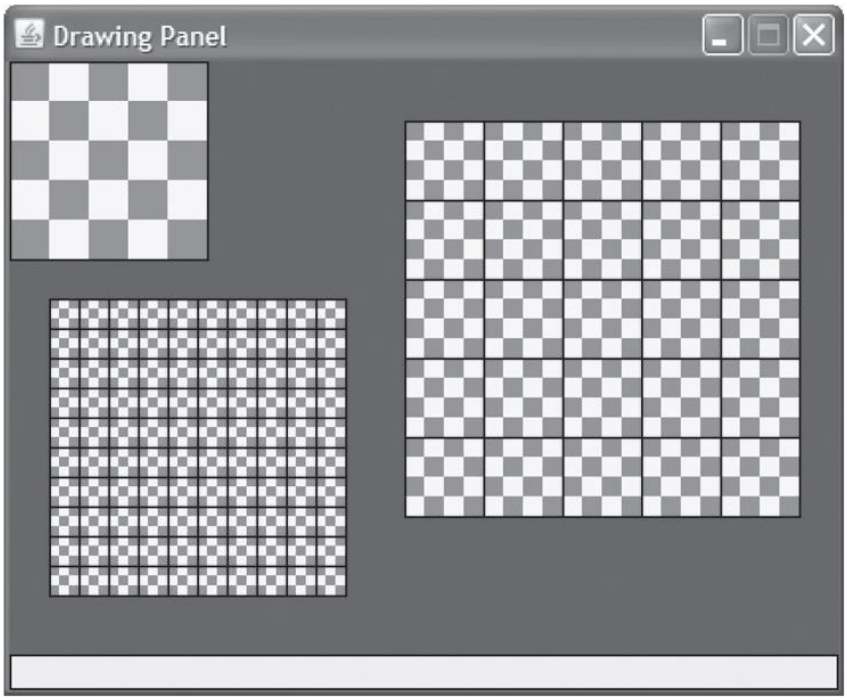
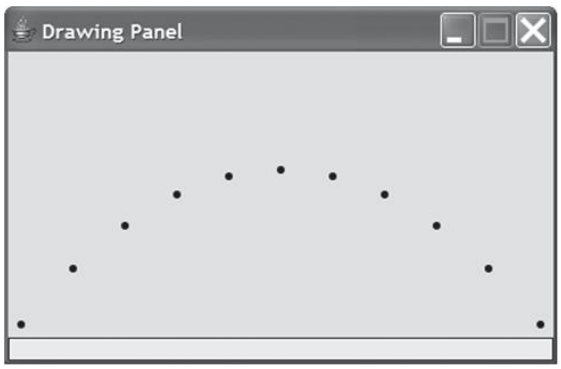


Figure 3G.37 Full Alternative Text

4. Write a modified version of the [redacted] case study program from Chapter 3 that draws a graph of the projectile's flight onto a [redacted] of size 420×220. For example, the panel shown in Figure 3G.38 draws a projectile with an initial velocity of 30 meters per second, an angle of 50 degrees, and 10 steps.

Figure 3G.38 Expected graphical output of Programming Project 3G.4



5. Write a program that draws the image shown in Figure 3G.39 onto a [redacted] of size 650×400. The image represents a famous optical illusion called the “Cafe Wall,” in which a series of straight squares appears to be slanted.

Figure 3G.39 Expected graphical output of Programming Project 3G.5

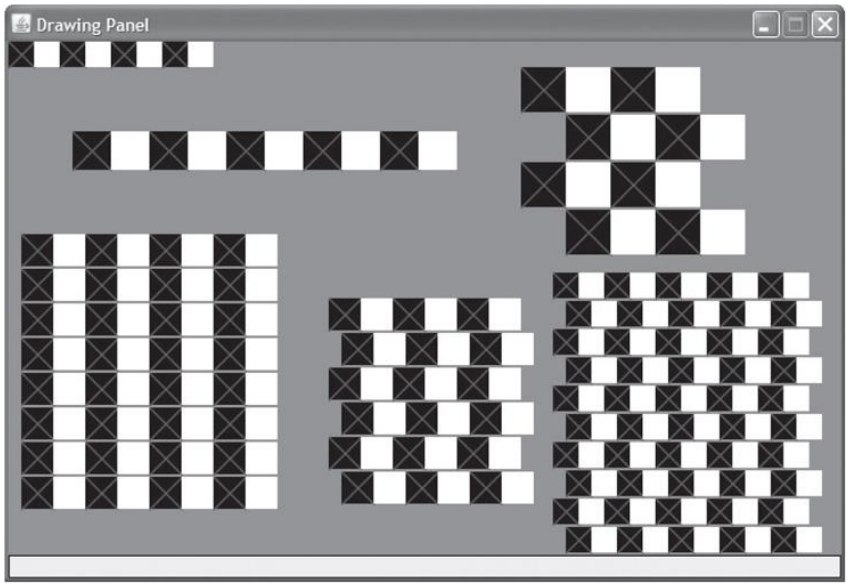


Figure 3G.39 Full Alternative Text

The image has a gray background and many rows of black and white squares with a blue X drawn through each black square.

The two free-standing rows in the diagram have the following properties:

Table 3G.8 Cafe Wall Row Properties

upper-left	(0, 0)	4	20
mid-left	(50, 70)	5	30

The diagram has four grids of rows of squares, with 2 pixels of vertical space between adjacent rows. A key aspect of the optical illusion is that every other row is shifted horizontally by a particular offset. The four grids have the following properties:

Table 3G.9 Cafe Wall Grid Properties

lower-left	(10, 150)	4	25	0
lower-middle	(250, 200)	3	25	10
lower-right	(425, 180)	5	20	10
upper-right	(400, 20)	2	35	35

Chương 4: Thực thi có điều kiện

4.1 Câu lệnh if/else

- Toán tử quan hệ
- Câu lệnh if/else lồng nhau
- So sánh đối tượng
- Tối ưu hóa câu lệnh if/else
- Kiểm tra nhiều điều kiện

4.2 Thuật toán tích lũy

- Tích lũy tổng
- Vòng lặp Min/Max
- Tích lũy tổng với if
- Lỗi làm tròn

4.3 Xử lý văn bản

- Kiểu char
- char so với int
- Thuật toán tích lũy văn bản
- System.out.printf

4.4 Phương thức với thực thi có điều kiện

- Điều kiện trước và sau
- Ném ngoại lệ
- Xem xét lại giá trị trả về
- Lý luận về các đường dẫn

4.5 Nghiên cứu tình huống: Chỉ số khối cơ thể (BMI)

- Giải pháp không cấu trúc cho một người
- Giải pháp không cấu trúc cho hai người
- Giải pháp cấu trúc cho hai người
- Nguyên tắc thiết kế thủ tục

Giới thiệu

Trong các chương trước, bạn đã thấy cách giải quyết các vấn đề lập trình phức tạp bằng cách sử dụng vòng lặp for để lặp lại một số tác vụ nhiều lần. Bạn cũng đã thấy cách thêm một số tính linh hoạt vào chương trình của mình bằng cách sử dụng hằng số lớp và cách đọc các giá trị được nhập vào bởi người dùng bằng đối tượng Scanner. Bây giờ, chúng ta sẽ khám phá một kỹ thuật mạnh mẽ hơn để viết mã có thể thích ứng với các tình huống khác nhau.

Trong chương này, chúng ta sẽ xem xét thực thi có điều kiện dưới dạng cấu trúc điều khiển được gọi là câu lệnh if/else. Với câu lệnh if/else, bạn có thể hướng dẫn máy tính thực hiện các dòng mã khác nhau tùy thuộc vào việc các điều kiện nhất định có đúng hay không. Câu lệnh if/else, giống như vòng lặp for, mạnh mẽ đến mức bạn sẽ tự hỏi làm thế nào mà mình có thể viết chương trình mà không có nó.

Chương này cũng sẽ mở rộng hiểu biết của bạn về các tình huống lập trình phổ biến. Nó bao gồm việc khám phá các kỹ thuật vòng lặp mà chúng ta chưa xem xét và thảo luận về các vấn đề xử lý văn bản. Việc thêm thực thi có điều kiện vào kho tàng của bạn cũng sẽ yêu cầu chúng ta xem lại các phương thức, tham số và giá trị trả về để bạn có thể hiểu rõ hơn về một số điểm chính. Chương kết thúc với một số quy tắc giúp chúng ta thiết kế các chương trình thủ tục tốt hơn.

4.1 Câu lệnh if/else

Bạn sẽ thường xuyên thấy mình viết mã mà bạn muốn thực thi chỉ một số lần nhất định. Ví dụ, nếu bạn đang viết một chương trình chơi game, bạn có thể muốn in một thông báo mỗi khi người dùng đạt được điểm cao mới và lưu trữ điểm đó. Bạn có thể thực hiện điều này bằng cách đặt hai dòng mã cần thiết vào bên trong câu lệnh if:

```
if (currentScore > maxScore) {

    System.out.println("A new high score!");

    maxScore = currentScore;
```

```
}
```

Ý tưởng là bạn sẽ muốn thực hiện hai dòng mã bên trong câu lệnh if, nhưng không phải luôn luôn. Bài kiểm tra trong dấu ngoặc đơn xác định xem các câu lệnh bên trong câu lệnh if có được thực thi hay không. Nói cách khác, bài kiểm tra mô tả các điều kiện mà chúng ta muốn thực thi mã.

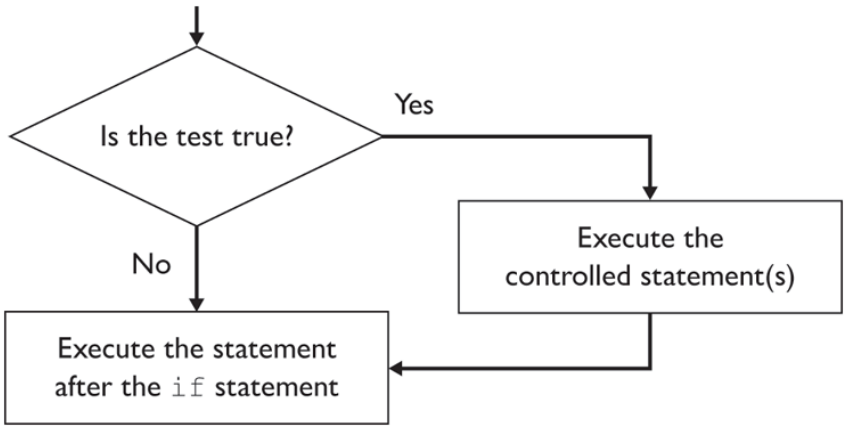
Dạng chung của câu lệnh if như sau:

```
if (<test>) {

    <statement>;

    <statement>;

    ...

    <statement>;

}
```

Câu lệnh if, giống như vòng lặp for, là một cấu trúc điều khiển. Lưu ý rằng chúng ta lại thấy một từ khóa Java (if) đi kèm với dấu ngoặc đơn và một bộ dấu ngoặc nhọn bao quanh một loạt các câu lệnh được kiểm soát.

Sơ đồ trong Hình 4.1 chỉ ra luồng điều khiển của câu lệnh if đơn giản. Máy tính thực hiện bài kiểm tra và nếu nó đánh giá là đúng, máy tính sẽ thực thi các câu lệnh được kiểm soát. Nếu bài kiểm tra đánh giá là sai, máy tính sẽ bỏ qua các câu lệnh được kiểm soát.

Hình 4.1: Luồng điều khiển của câu lệnh if



Hình 4.1 Văn bản thay thế đầy đủ

Bạn sẽ sử dụng câu lệnh if đơn giản khi bạn có đoạn mã mà bạn muốn thực thi đôi khi và bỏ qua ở các lần khác. Java cũng có một biến thể gọi là câu lệnh if/else cho phép bạn chọn giữa hai lựa chọn. Giả sử, ví dụ, bạn muốn đặt một biến gọi là answer bằng căn bậc hai của một số:

```
answer = Math.sqrt(number);
```

Bạn không muốn yêu cầu lấy căn bậc hai nếu số đó là số âm. Để tránh vấn đề tiềm ẩn này, bạn có thể sử dụng câu lệnh if đơn giản:

```
if (number >= 0) {

    answer = Math.sqrt(number);

}
```

Đoạn mã này sẽ tránh việc yêu cầu lấy căn bậc hai của một số âm, nhưng giá trị nào sẽ được gán cho answer nếu number là số âm? Trong trường hợp này, bạn có thể muốn gán một giá trị cho answer bằng cách nào đó. Giả sử bạn muốn answer bằng –1 khi

number là số âm. Bạn có thể biểu diễn cặp lựa chọn này với câu lệnh if/else sau:

```
if (number >= 0) {

    answer = Math.sqrt(number);

} else {

    answer = –1;

}
```

Câu lệnh if/else cung cấp hai lựa chọn và thực thi một trong hai. Vì vậy, trong đoạn mã trên, bạn biết rằng answer sẽ được gán một giá trị bất kể number là số dương hay số âm.

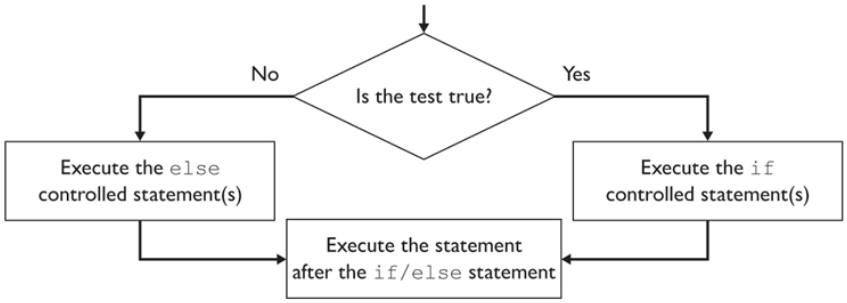
Dạng chung của câu lệnh if/else là:

```
if (<test>) {

    <statement>;

    <statement>;

    ...

    <statement>;

} else {

    <statement>;

    <statement>;
```

```
    ...

    <statement>;

}
```

Cấu trúc điều khiển này khá đặc biệt ở chỗ nó có hai tập hợp câu lệnh được kiểm soát và hai từ khóa khác nhau (if và else). Hình 4.2 chỉ ra luồng điều khiển. Máy tính thực hiện bài kiểm tra và, tùy thuộc vào việc mã có đánh giá là đúng hay sai, sẽ thực thi một trong hai nhóm câu lệnh.

Hình 4.2: Luồng điều khiển của câu lệnh if/else



Hình 4.2 Văn bản thay thế đầy đủ

Cũng như trong trường hợp của vòng lặp for, nếu bạn có một câu lệnh duy nhất để thực thi, bạn không cần phải bao gồm dấu ngoặc nhọn. Tuy nhiên, theo quy ước của Java, dấu ngoặc nhọn nên được bao gồm ngay cả khi bạn không cần đến chúng, và chúng tôi tuân theo quy ước đó trong cuốn sách này.

Toán tử quan hệ

Một câu lệnh if/else được kiểm soát bởi một điều kiện. Các kiểm tra đơn giản so sánh hai biểu thức để xem chúng có liên quan theo một cách nào đó hay không. Các kiểm tra này tự bản thân là các biểu thức có dạng sau và đánh giá là đúng hoặc sai:

```
<biểu thức> <toán tử quan hệ> <biểu thức>
```

Để đánh giá một kiểm tra theo dạng này, trước tiên hãy đánh giá hai biểu thức, sau đó xem liệu quan hệ đã cho có tồn tại giữa giá trị bên trái và giá trị bên phải hay không. Nếu quan hệ tồn tại, kiểm tra đánh giá là đúng. Nếu không, kiểm tra đánh giá là sai. Các toán tử quan hệ được liệt kê trong Bảng 4.1. Lưu ý rằng toán tử so sánh bằng bao gồm hai dấu bằng (==), để phân biệt với toán tử gán (=).

Bảng 4.1 Toán tử quan hệ

Toán tử	Ý nghĩa	Ví dụ	Giá trị
==	bằng với	2 + 2 == 4	đúng
!=	không bằng	3.2 != 4.1	đúng
<	nhỏ hơn	4 < 3	sai
>	lớn hơn	4 > 3	đúng
<=	nhỏ hơn hoặc bằng	2 <= 0	sai
>=	lớn hơn hoặc bằng	2.4 >= 1.6	đúng

Bởi vì chúng ta sử dụng các toán tử quan hệ như một cách mới để tạo biểu thức, chúng ta phải xem xét lại độ ưu tiên. Bảng 4.2 là một phiên bản cập nhật của Bảng 2.5, bao gồm các toán tử mới này. Bạn sẽ thấy rằng, về mặt kỹ thuật, các phép so sánh bằng có một mức độ ưu tiên khác biệt một chút so với các toán tử quan hệ khác, nhưng cả hai tập hợp toán tử đều có độ ưu tiên thấp hơn các toán tử số học.

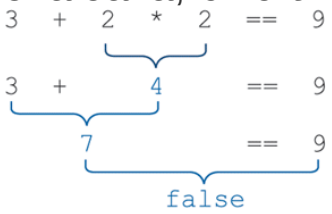
Bảng 4.2 Độ ưu tiên của toán tử trong Java

Mô tả	Toán tử
toán tử đơn hạng	++, --, +, –
toán tử nhân chia	*, /, %
toán tử cộng trừ	+, –
toán tử quan hệ	<, >, <=, >=
toán tử so sánh bằng	==, !=
toán tử gán	=, +=, –=, *=, /=, %=

Hãy xem một ví dụ. Biểu thức sau đây được tạo thành từ các hằng số 3, 2, và 9 và bao gồm các phép cộng, nhân và so sánh bằng:

```
3 + 2 * 2 == 9
```

Phép toán nào sẽ được thực hiện trước? Bởi vì các toán tử quan hệ có mức độ ưu tiên thấp hơn các toán tử số học, phép nhân sẽ được thực hiện trước, sau đó là phép cộng, rồi mới đến phép kiểm tra bằng. Nói cách khác, Java sẽ thực hiện tất cả các phép toán "toán học" trước khi kiểm tra bất kỳ mối quan hệ nào. Quy tắc ưu tiên này giải phóng bạn khỏi việc phải đặt dấu ngoặc đơn xung quanh các biểu thức bên trái và bên phải của một kiểm tra sử dụng toán tử quan hệ. Khi bạn tuân theo các quy tắc ưu tiên của Java, biểu thức mẫu sẽ được đánh giá như sau:



4.1-3 Văn bản thay thế đầy đủ

Bạn có thể đặt các biểu thức tùy ý ở hai bên của toán tử quan hệ, miễn là các kiểu dữ liệu tương thích. Đây là một ví dụ với các biểu thức phức tạp ở hai bên:

```
(2 – 3 * 8) / (435 % (7 * 2)) <= 3.8 – 4.5 / (2.2 * 3.8)
```

Bạn có thể kiểm tra các toán tử quan hệ trong JShell, như minh họa trong tương tác sau:

```
jshell>

int a = 3;

a ==> 3
```

```
jshell>

int b = 2;

b ==> 2

jshell>

a > b

$3 ==> true

jshell>

a > b * b

$4 ==> false

jshell>

a - 1 == b

$5 ==> true
```

Một hạn chế của các toán tử quan hệ là chúng chỉ nên được sử dụng với dữ liệu nguyên thủy. Sau này trong chương này, chúng ta sẽ nói về cách so sánh đối tượng để kiểm tra sự bằng nhau, và trong một chương sau, chúng ta sẽ thảo luận về cách thực hiện so sánh lớn hơn và nhỏ hơn trên các đối tượng.

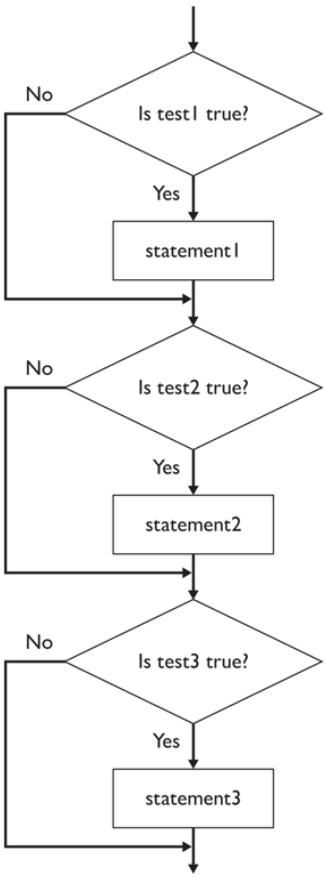
Lồng các câu lệnh if/else

Nhiều người mới bắt đầu viết mã trông như thế này:

```
if (<test1>) {

    <statement1>;

}

if (<test2>) {

    <statement2>;

}

if (<test3>) {

    <statement3>;

}
```

Cấu trúc tuần tự này là thích hợp nếu bạn muốn thực thi bất kỳ sự kết hợp nào của ba câu lệnh. Ví dụ, bạn có thể viết mã này trong một chương trình cho một bảng câu hỏi với ba phần tùy chọn, bất kỳ sự kết hợp nào trong số đó đều có thể áp dụng cho một người cụ thể.

Hình 4.3 minh họa luồng của mã if tuần tự. Lưu ý rằng máy tính có thể thực thi không một câu lệnh nào (nếu tất cả các kiểm tra đều sai), chỉ một câu lệnh (nếu chỉ có một kiểm tra là đúng), hoặc nhiều câu lệnh (nếu nhiều kiểm tra là đúng).



Hình 4.3 Văn bản thay thế đầy đủ

Tuy nhiên, trong nhiều trường hợp, bạn chỉ muốn thực thi một trong số các câu lệnh. Trong những trường hợp như vậy, tốt hơn là lồng các câu lệnh if, xếp chồng chúng lên nhau:

```
if (<test1>) {

    <statement1>;

} else {

    if (<test2>) {

        <statement2>;
```

```
    } else {
        if (<test3>) {
            <statement3>;
        }
    }
}
```

Khi sử dụng cấu trúc này, bạn có thể chắc chắn rằng máy tính sẽ chỉ thực thi tối đa một câu lệnh: câu lệnh tương ứng với kiểm tra đầu tiên cho kết quả true. Nếu không có kiểm tra nào cho kết quả true, không câu lệnh nào được thực thi. Nếu mục tiêu của bạn là chỉ thực thi tối đa một câu lệnh, cấu trúc này phù hợp hơn so với các câu lệnh if tuần tự. Nó làm giảm khả năng xảy ra lỗi và đơn giản hóa quá trình kiểm tra.

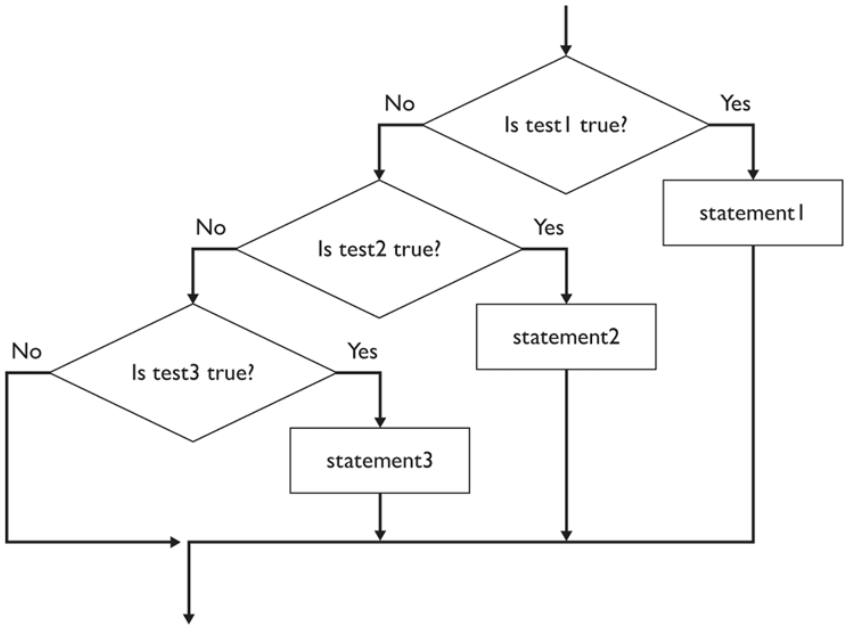
Như bạn thấy, việc lồng các câu lệnh if như thế này dẫn đến rất nhiều thụt lề. Việc thụt lề này không mấy hữu ích, vì cấu trúc này thực sự được thiết kế để cho phép lựa chọn một trong nhiều lựa chọn thay thế. Phong cách K&R có một giải pháp cho vấn đề này. Nếu một else được theo sau bởi một if, chúng ta đặt chúng trên cùng một dòng:

```
if (<test1>) {
    <statement1>;
} else if (<test2>) {
    <statement2>;
```

```
} else if (<test3>) {

    <statement3>;

}
```

Khi bạn tuân theo quy ước này, các câu lệnh khác nhau đều xuất hiện ở cùng một mức độ thụt lề. Chúng tôi khuyến nghị rằng các câu lệnh if/else lồng nhau nên được thụt lề theo cách này.

Hình 4.4 minh họa luồng của mã if/else lồng nhau. Lưu ý rằng có thể thực thi một trong các câu lệnh được kiểm soát (câu lệnh đầu tiên cho kết quả true) hoặc không thực thi câu lệnh nào (nếu không có kiểm tra nào cho kết quả true).



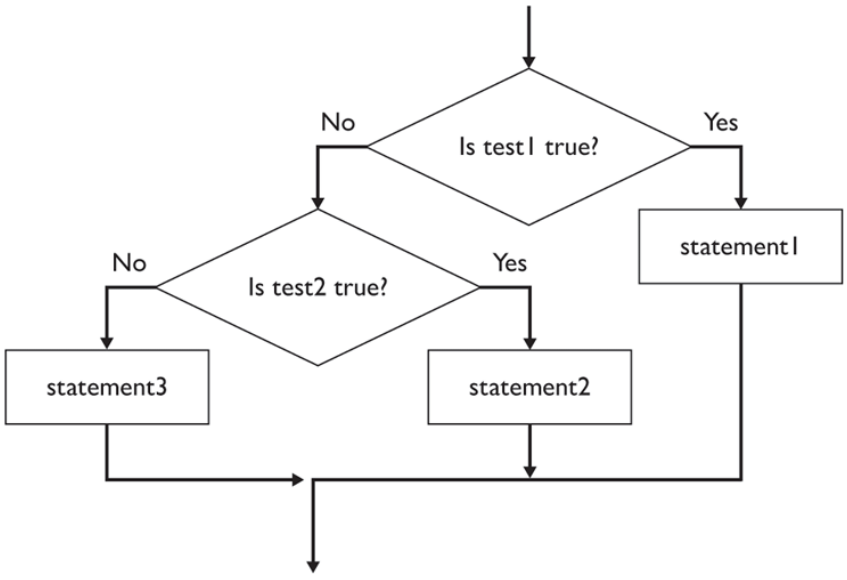
Hình 4.4 Văn bản thay thế đầy đủ

Trong một biến thể của cấu trúc này, câu lệnh cuối cùng được điều khiển bởi else thay vì một điều kiện kiểm tra:

```
if (<test1>) {

    <statement1>;

} else if (<test2>) {

    <statement2>;

} else {

    <statement3>;

}
```

Trong cấu trúc này, máy tính sẽ luôn chọn nhánh cuối cùng khi tất cả các điều kiện kiểm tra đều không thỏa mãn, do đó, cấu trúc sẽ luôn thực thi chính xác một trong ba câu lệnh. Hình 4.5 cho thấy luồng của mã if/else lồng nhau đã được sửa đổi này.

Hình 4.5 Luồng của các if lồng nhau kết thúc bằng else



Hình 4.5 Văn bản thay thế đầy đủ

Để khám phá các biến thể này, hãy xem xét nhiệm vụ để máy tính xác định liệu một số là dương, âm hay bằng không. Bạn có thể cấu trúc nhiệm vụ này bằng ba câu lệnh if đơn giản như sau:

```
if (number > 0) {

    System.out.println("Number is positive.");

}

if (number == 0) {

    System.out.println("Number is zero.");

}

if (number < 0) {

    System.out.println("Number is negative.");

}
```

Để xác định bao nhiêu trong số các câu lệnh println có thể được thực thi, bạn phải dừng lại và suy nghĩ về các điều kiện kiểm tra. Nhưng bạn không nên cần phải nỗ lực nhiều như vậy để hiểu đoạn mã này. Đoạn mã sẽ rõ ràng hơn nếu bạn lồng các câu lệnh if:

```
if (number > 0) {

    System.out.println("Number is positive.");

} else if (number == 0) {
```

```
    System.out.println("Number is zero.");

} else if (number < 0) {

    System.out.println("Number is negative.");

}
```

Tuy nhiên, giải pháp này có một vấn đề. Bạn biết rằng bạn muốn thực thi một và chỉ một câu lệnh println, nhưng cấu trúc lồng này không ngăn cản khả năng không có câu lệnh nào được thực thi (điều này sẽ xảy ra nếu tất cả ba điều kiện kiểm tra đều thất bại). Tất nhiên, với các điều kiện kiểm tra này, điều đó sẽ không bao giờ xảy ra: Nếu một số không phải là dương cũng không phải là bằng không, thì nó phải là âm. Do đó, điều kiện kiểm tra cuối cùng ở đây là không cần thiết và gây hiểu lầm. Bạn phải suy nghĩ về các điều kiện kiểm tra để xác định liệu có thể tất cả ba điều kiện kiểm tra đều thất bại và tất cả các nhánh đều bị bỏ qua hay không.

Trong trường hợp này, giải pháp tốt nhất là sử dụng cách tiếp cận if/else lồng nhau với một nhánh cuối cùng luôn được thực thi nếu hai điều kiện kiểm tra đầu tiên thất bại:

```
if (number > 0) {

    System.out.println("Number is positive.");

} else if (number == 0) {

    System.out.println("Number is zero.");

} else {
```

```
    System.out.println("Number is negative.");

}
```

Bạn có thể nhìn lướt qua cấu trúc này và ngay lập tức nhận ra rằng chỉ có một câu lệnh println sẽ được thực thi. Bạn không cần phải xem xét các điều kiện kiểm tra để nhận ra điều này; đó là một thuộc tính của loại cấu trúc if/else lồng nhau này. Nếu bạn muốn, bạn có thể thêm một chú thích để làm rõ những gì đang diễn ra:

```
if (number > 0) {

    System.out.println("Number is positive.");

} else if (number == 0) {

    System.out.println("Number is zero.");

} else {

    // number must be negative

    System.out.println("Number is negative.");

}
```

Một lợi ích cuối cùng của cách tiếp cận này là hiệu quả. Khi mã sử dụng ba câu lệnh if đơn giản, máy tính sẽ luôn thực hiện cả ba điều kiện kiểm tra. Khi mã sử dụng cách tiếp cận if/else lồng nhau, máy tính chỉ thực hiện các điều kiện kiểm tra cho đến khi tìm thấy một điều kiện phù hợp, điều này sử dụng tài nguyên tốt hơn. Ví dụ, trong mã trên, chúng ta chỉ cần thực hiện một điều

kiện kiểm tra đối với các số dương và tối đa hai điều kiện kiểm tra cho toàn bộ quá trình.

Khi bạn gặp tình huống viết mã để lựa chọn giữa các khả năng như thế này, bạn cần phân tích vấn đề cụ thể để xác định bao nhiêu nhánh bạn muốn thực thi. Nếu không quan trọng nhánh nào được chọn, hãy sử dụng các câu lệnh if liên tiếp. Nếu bạn muốn một hoặc không có nhánh nào được chọn, hãy sử dụng các câu lệnh if/else lồng nhau với một điều kiện kiểm tra cho mỗi câu lệnh. Nếu bạn muốn chỉ có một nhánh được chọn, hãy sử dụng các câu lệnh if/else lồng nhau với nhánh cuối cùng được kiểm soát bởi else thay vì một điều kiện kiểm tra. Bảng 4.3 tóm tắt các lựa chọn này.

Bảng 4.3 Các tùy chọn if/else

Tình huống	**Cấu trúc**	**Hình thức cơ bản**
Bạn muốn thực thi bất kỳ sự kết hợp nào của các câu lệnh được kiểm soát	Các câu lệnh if liên tiếp	if (<test1>) { <statement1>; }
		if (<test2>) { <statement2>; }
		if (<test3>) { <statement3>; }

Tình huống	Cấu trúc	Hình thức cơ bản
Bạn muốn thực thi không có hoặc một trong các câu lệnh được kiểm soát	Các câu lệnh if lồng kết thúc bằng điều kiện kiểm tra	if (<test1>) { <statement1>; }
		else if (<test2>) { <statement2>; }
		else if (<test3>) { <statement3>; }
Bạn muốn thực thi chính xác một trong các câu lệnh được kiểm soát	Các câu lệnh if lồng kết thúc bằng else	if (<test1>) { <statement1>; }
		else if (<test2>) { <statement2>; }
		else { <statement3>; }

LỖI LẬP TRÌNH PHỔ BIẾN

Chọn sai cấu trúc if/else

Giả sử rằng giáo viên của bạn đã nói rằng điểm số sẽ được xác định như sau:

- A cho điểm số ≥ 90
- B cho điểm số ≥ 80
- C cho điểm số ≥ 70
- D cho điểm số ≥ 60
- F cho điểm số < 60

Bạn có thể chuyển đổi thang điểm này thành mã như sau:

```
String grade;


if (score >= 90) {
    grade = "A";
}
if (score >= 80) {
    grade = "B";
}
if (score >= 70) {
    grade = "C";
}
```

```
if (score >= 60) {
    grade = "D";
}
if (score < 60) {
    grade = "F";
}
```

Tuy nhiên, nếu bạn cố gắng sử dụng biến grade sau đoạn mã này, bạn sẽ nhận được thông báo lỗi từ trình biên dịch:

```
variable grade might not have been initialized
```

Đây là một gợi ý rằng có vấn đề. Trình biên dịch Java đang nói rằng nó tin rằng có những đường dẫn thông qua đoạn mã này sẽ khiến biến grade không được khởi tạo. Trên thực tế, biến này sẽ luôn được khởi tạo, nhưng trình biên dịch không thể nhận ra điều đó. Chúng ta có thể khắc phục vấn đề này bằng cách cung cấp giá trị ban đầu cho grade:

```
String grade = "no grade";
```

Thay đổi này cho phép đoạn mã được biên dịch. Nhưng nếu bạn biên dịch và chạy chương trình, bạn sẽ thấy rằng nó chỉ cho ra hai loại điểm: D và F. Bất kỳ ai có điểm số ít nhất là 60 đều nhận được D và ai có điểm dưới 60 đều nhận được F. Và mặc dù trình biên dịch phàn nàn rằng có một đường dẫn cho phép grade không được khởi tạo, nhưng không ai bao giờ nhận được điểm "no grade".

Vấn đề ở đây là bạn muốn thực thi chính xác một trong các câu lệnh gán giá trị, nhưng khi bạn sử dụng các câu lệnh if liên tiếp, có thể chương trình sẽ thực thi nhiều câu lệnh trong số đó liên tiếp. Ví dụ, nếu một học sinh có điểm số là 95, điểm của học sinh đó được đặt là "A", sau đó đặt lại là "B", rồi đặt lại là "C", và cuối cùng là "D". Bạn có thể khắc phục vấn đề này bằng cách sử dụng cấu trúc if/else lồng nhau:

```
String grade;



if (score >= 90) {

    grade = "A";

} else if (score >= 80) {

    grade = "B";

} else if (score >= 70) {

    grade = "C";

} else if (score >= 60) {

    grade = "D";

} else {

    // score < 60

    grade = "F";
```

```
}
```

Bây giờ bạn không cần phải đặt grade thành "no grade" nữa vì trình biên dịch có thể thấy rằng bất kể đường dẫn nào được theo, biến grade sẽ được gán giá trị (chính xác một trong các nhánh sẽ được thực thi).

So sánh Đối tượng

Trước đó trong chương, bạn đã thấy rằng bạn có thể sử dụng các toán tử == và != để kiểm tra sự bằng nhau và không bằng nhau của dữ liệu nguyên thủy. Tuy nhiên, các toán tử này không hoạt động như bạn mong đợi khi kiểm tra sự bằng nhau của các đối tượng như chuỗi (string). Bạn sẽ cần học một cách mới để kiểm tra sự bằng nhau của các đối tượng.

Ví dụ, bạn có thể viết mã như sau để đọc một từ từ bảng điều khiển và gọi một trong hai phương thức khác nhau tùy thuộc vào việc người dùng trả lời "yes" hay "no". Nếu người dùng không nhập từ nào trong hai từ đó, đoạn mã này sẽ in ra một thông báo lỗi:

```
System.out.print("yes or no? ");

String s = console.next();

if (s == "yes") {

    processYes();

} else if (s == "no") {

    processNo();
```

```
} else {

    System.out.println("You didn't type yes or no");

}
```

Đáng tiếc, đoạn mã này không hoạt động. Bất kể người dùng nhập gì, chương trình này luôn in ra “You didn’t type yes or no”. Chúng ta sẽ khám phá chi tiết trong Chương 8 lý do tại sao đoạn mã này không hoạt động. Hiện tại, điều quan trọng cần biết là Java cung cấp một cách thứ hai để kiểm tra sự bằng nhau, cách này được sử dụng cho các đối tượng. Mỗi đối tượng Java đều có một phương thức gọi là equals nhận một đối tượng khác làm đối số. Bạn có thể sử dụng phương thức này để hỏi một đối tượng xem nó có bằng một đối tượng khác hay không. Ví dụ, chúng ta có thể sửa đoạn mã trước đó như sau:

```
System.out.print("yes or no? ");

String s = console.next();

if (s.equals("yes")) {

    processYes();

} else if (s.equals("no")) {

    processNo();

} else {

    System.out.println("You didn't type yes or no");

}
```

Hãy nhớ rằng khi bạn làm việc với chuỗi, bạn nên luôn gọi phương thức equals thay vì sử dụng ==.

Lớp String cũng có một biến thể đặc biệt của phương thức equals gọi là equalsIgnoreCase bỏ qua sự khác biệt về chữ hoa và chữ thường. Ví dụ, bạn có thể viết lại đoạn mã trước đó như sau để nhận diện các phản hồi như “Yes”, “YES”, “No”, “NO”, “yES”, và tương tự:

```
System.out.print("yes or no? ");

String s = console.next();

if (s.equalsIgnoreCase("yes")) {

    processYes();

} else if (s.equalsIgnoreCase("no")) {

    processNo();

} else {

    System.out.println("You didn't type yes or no");

}
```

Tối ưu hóa câu lệnh if/else

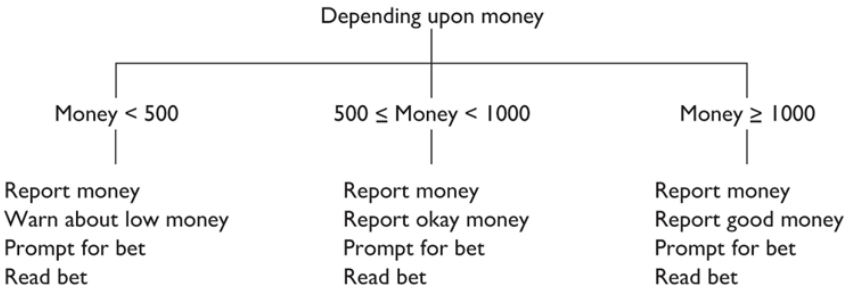
Giả sử bạn đang viết một chương trình chơi trò chơi cá cược với người dùng và bạn muốn đưa ra các cảnh báo khác nhau về số tiền mặt mà người dùng còn lại. Cấu trúc if/else lồng sau đây

phân biệt ba trường hợp khác nhau: tiền dưới 500 đô la, được coi là thấp; tiền từ 500 đến 1000 đô la, được coi là vừa đủ; và tiền trên 1000 đô la, được coi là tốt. Lưu ý rằng người dùng nhận được lời khuyên khác nhau trong mỗi trường hợp:

```
if (money < 500) {

    System.out.println("You have $" + money + " left.");

    System.out.print("Cash is dangerously low. Bet carefully.");

    System.out.print("How much do you want to bet? ");

    bet = console.nextInt();

} else if (money < 1000) {

    System.out.println("You have $" + money + " left.");

    System.out.print("Cash is somewhat low. Bet moderately.");

    System.out.print("How much do you want to bet? ");

    bet = console.nextInt();

} else {

    System.out.println("You have $" + money + " left.");

    System.out.print("Cash is in good shape. Bet liberally.");

    System.out.print("How much do you want to bet? ");

    bet = console.nextInt();
```

```
}
```

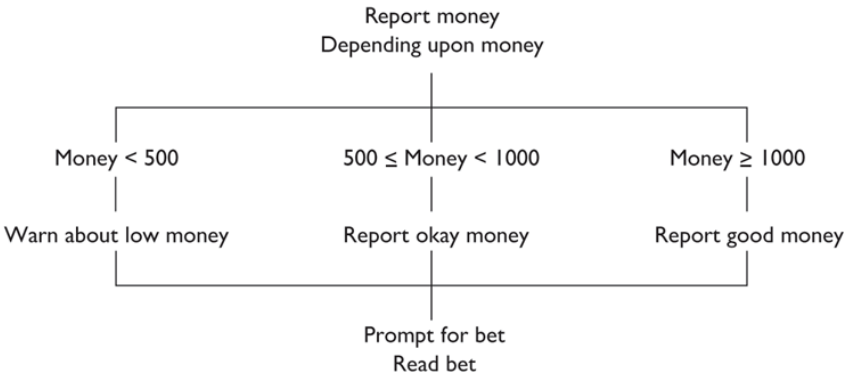
Cấu trúc này lặp lại và có thể được làm cho hiệu quả hơn bằng cách sử dụng một kỹ thuật gọi là tối ưu hóa. Sử dụng kỹ thuật đơn giản này, bạn có thể loại bỏ những đoạn mã giống nhau từ các nhánh khác nhau của cấu trúc if/else. Trong đoạn mã trên, ba nhánh khác nhau có thể được thực thi, tùy thuộc vào giá trị của biến money. Bắt đầu bằng cách ghi lại các chuỗi hành động đang được thực hiện trong mỗi nhánh và so sánh chúng.



Hình 4.6 Văn bản thay thế đầy đủ

Bạn có thể tối ưu hóa cả ở đầu và cuối của một cấu trúc như thế này. Nếu bạn nhận thấy rằng câu lệnh đầu tiên trong mỗi nhánh là giống nhau, bạn có thể đưa nó ra khỏi phần phân nhánh và đặt nó trước nhánh. Tương tự, nếu câu lệnh cuối cùng trong mỗi nhánh là giống nhau, bạn có thể đưa nó ra khỏi phần phân nhánh và đặt nó sau vòng lặp. Bạn có thể tối ưu hóa câu lệnh đầu tiên trong mỗi nhánh và hai câu lệnh cuối cùng, như trong Hình 4.7.

Hình 4.7 Các nhánh if/else sau khi tối ưu hóa



Hình 4.7 Văn bản thay thế đầy đủ

Do đó, đoạn mã trước có thể được rút gọn thành phiên bản ngắn gọn hơn như sau:

```
System.out.println("You have $" + money + " left.");
if (money < 500) {
    System.out.print("Cash is dangerously low. Bet carefully.");
} else if (money < 1000) {
    System.out.print("Cash is somewhat low. Bet moderately.");
} else {
    System.out.print("Cash is in good shape. Bet liberally.");
}
System.out.print("How much do you want to bet? ");
bet = console.nextInt();
```

Kiểm tra nhiều điều kiện

Khi bạn viết một chương trình, bạn thường thấy mình muốn kiểm tra nhiều hơn một điều kiện. Ví dụ, giả sử bạn muốn chương trình thực hiện một hành động cụ thể nếu một số nằm trong khoảng từ 1 đến 10. Bạn có thể nói:

```
if (number >= 1) {
    if (number <= 10) {
        doSomething();
    }
}
```

Trong những dòng mã này, bạn phải viết hai câu lệnh: một để kiểm tra xem số đó có lớn hơn hoặc bằng 1 hay không và một để kiểm tra xem số đó có nhỏ hơn hoặc bằng 10 hay không. Java cung cấp một cách thay thế hiệu quả hơn: Bạn có thể kết hợp hai phép kiểm tra này bằng cách sử dụng một toán tử được gọi là toán tử AND logic, được viết bằng hai ký tự ampersand không có khoảng cách giữa chúng (&&). Sử dụng toán tử AND, chúng ta có thể viết lại đoạn mã trên một cách đơn giản hơn:

```
if (number >= 1 && number <= 10) {
    doSomething();
}
```

Như tên gọi của nó, toán tử AND tạo ra một phép kiểm tra yêu cầu cả hai phần của phép kiểm tra phải đánh giá là đúng. Có một toán tử tương tự gọi là toán tử OR logic, nó đánh giá là đúng nếu một trong hai phép kiểm tra đánh giá là đúng. Toán tử OR logic được viết bằng hai ký tự thanh dọc (||). Ví dụ, nếu bạn muốn kiểm tra xem một biến số có bằng 1 hoặc 2 hay không, bạn có thể nói:

```
if (number == 1 || number == 2) {
    processNumber(number);
}
```

Chúng ta sẽ khám phá toán tử AND logic và toán tử OR logic chi tiết hơn trong chương tiếp theo.

4.2 Thuật toán Cộng dồn

Càng lập trình nhiều, bạn sẽ càng nhận thấy rằng có những mẫu xuất hiện thường xuyên. Nhiều thuật toán phổ biến liên quan đến việc tích lũy kết quả từng bước một. Trong phần này, chúng ta sẽ khám phá một số thuật toán cộng dồn phổ biến nhất.

Thuật toán Cộng dồn

Một thao tác trong đó giá trị tổng thể được tính toán dần dần, thường sử dụng vòng lặp. Ví dụ, bạn có thể sử dụng một thuật

toán cộng dồn trên một tập hợp các số để tính giá trị trung bình hoặc tìm số lớn nhất.

Tổng cộng dồn

Bạn thường muốn tìm tổng của một loạt số. Một cách để làm điều này là khai báo một biến khác nhau cho mỗi giá trị bạn muốn bao gồm, nhưng đó không phải là một giải pháp thực tế: Nếu bạn phải cộng một trăm số với nhau, bạn sẽ không muốn khai báo một trăm biến khác nhau. May mắn thay, có một cách đơn giản hơn. Bí quyết là giữ một bảng tính kết quả và xử lý từng số một. Ví dụ, để cộng vào một biến gọi là sum, bạn sẽ viết dòng mã sau:

```
sum = sum + next;
```

Ngoài ra, bạn có thể sử dụng toán tử gán ngắn gọn:

```
sum += next;
```

Câu lệnh trên lấy giá trị hiện tại của sum, cộng với giá trị của một biến gọi là next, và lưu kết quả là giá trị mới của sum. Thao tác này được thực hiện cho mỗi số cần cộng. Lưu ý rằng khi bạn thực hiện câu lệnh này lần đầu tiên, sum chưa có giá trị. Để giải quyết điều này, bạn khởi tạo sum với một giá trị không ảnh hưởng đến kết quả: 0. Đây là mô tả bằng giả mã của thuật toán cộng dồn:

```
sum = 0.
for (all numbers to sum) {
    obtain "next".
    sum += next.
}
```

Để triển khai thuật toán này, bạn phải quyết định số lần thực hiện vòng lặp và cách lấy giá trị tiếp theo. Đây là một chương trình tương tác yêu cầu người dùng nhập số lượng số cần cộng và nhập các số đó:

```
1  // Tìm tổng của một dãy số.
2
3  import java.util.*;
```

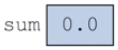
```
4
5  public class ExamineNumbers1 {
6      public static void main(String[] args) {
7          System.out.println("This program adds a sequence of");
8          System.out.println("numbers.");
9          System.out.println();
10
11         Scanner console = new Scanner(System.in);
12
13         System.out.print("How many numbers do you have? ");
14         int totalNumber = console.nextInt();
15
16         double sum = 0.0;
17         for (int i = 1; i <= totalNumber; i++) {
18             System.out.print("    #" + i + "? ");
19             double next = console.nextDouble();
20             sum += next;
21         }
22         System.out.println();
23
24         System.out.println("sum = " + sum);
25     }
26 }
```

Chương trình sẽ thực thi như sau (như thường lệ, đầu vào của người dùng được in đậm):

```
This program adds a sequence of
numbers.
How many numbers do you have? 6
#1? 3.2
#2? 4.7
#3? 5.1
#4? 9.0
```

```
#5? 2.4
#6? 3.1
sum = 27.5
```

Hãy theo dõi chi tiết quá trình thực thi. Trước khi chúng ta vào vòng lặp for, chúng ta khởi tạo biến sum với giá trị 0.0:



Trong lần thực thi đầu tiên của vòng lặp for, chúng ta đọc vào giá trị 3.2 từ người dùng và cộng giá trị này vào biến sum:

Lần thứ hai, chúng ta đọc vào giá trị 4.7 và cộng nó vào giá trị của sum:

Chú ý rằng sum bây giờ đã bao gồm cả hai số mà người dùng đã nhập, bởi vì chúng ta đã cộng giá trị mới, 4.7, vào giá trị cũ, 3.2. Lần thứ ba, chúng ta cộng thêm giá trị 5.1:

Chú ý rằng biến sum bây giờ chứa tổng của ba số đầu tiên (3.2 + 4.7 + 5.1). Sau đó, chúng ta đọc vào giá trị 9.0 và cộng nó vào sum:

Sau đó, chúng ta cộng vào giá trị thứ năm, 2.4:

Cuối cùng, chúng ta cộng vào giá trị thứ sáu, 3.1:

Chúng ta sau đó thoát khỏi vòng lặp for và in ra giá trị của sum. Có một vấn đề thú vị về phạm vi biến trong chương trình này. Chú ý rằng biến sum được khai báo bên ngoài vòng lặp, trong khi biến next được khai báo bên trong vòng lặp. Chúng ta không có lựa chọn nào khác ngoài việc khai báo sum bên ngoài vòng lặp bởi vì nó cần được khởi tạo và nó được sử dụng sau vòng lặp. Nhưng biến next chỉ được sử dụng bên trong vòng lặp, nên nó có thể được khai báo trong phạm vi bên trong đó. Tốt nhất là khai báo các biến trong phạm vi trong cùng có thể.

Thuật toán tính tổng tích lũy và các biến thể của nó sẽ rất hữu ích trong nhiều nhiệm vụ lập trình mà bạn sẽ giải quyết. Làm thế nào để bạn tính tích lũy? Dưới đây là mã giả:

```
product = 1.
for (all numbers to multiply) {
    obtain "next".
    product = product * next.
}
```

Vòng lặp Tìm Giá Trị Lớn Nhất/Nhỏ Nhất

Một nhiệm vụ lập trình phổ biến khác là theo dõi giá trị lớn nhất và/hoặc nhỏ nhất trong một dãy số. Ví dụ, hãy xem xét nhiệm vụ xác định liệu có khả thi để xây dựng một khu vực sống trên Mặt Trăng cho con người. Một trở ngại là nhiệt độ trung bình hàng ngày trên Mặt Trăng là −50 độ Fahrenheit. Nhưng một vấn đề lớn hơn nhiều là phạm vi nhiệt độ rộng lớn; từ −240 độ đến 250 độ Fahrenheit.

Để tính giá trị lớn nhất của một dãy số, bạn có thể theo dõi giá trị lớn nhất mà bạn đã thấy và sử dụng một câu lệnh if để cập nhật giá trị lớn nhất nếu bạn gặp phải một giá trị mới lớn hơn giá trị lớn nhất hiện tại. Cách tiếp cận này có thể được mô tả bằng mã giả như sau:

```
initialize max.
for (all numbers to examine) {
    obtain "next".
    if (next > max) {
        max = next.
    }
}
```

Khởi tạo giá trị lớn nhất không đơn giản như nghe có vẻ. Ví dụ, người mới học thường khởi tạo max là 0. Nhưng nếu dãy số bạn đang xem xét hoàn toàn là các số âm thì sao? Ví dụ, bạn có thể cần tìm giá trị lớn nhất trong dãy số này:

−84, −7, −14, −39, −410, −17, −41, −9
Giá trị lớn nhất trong dãy này là −7, nhưng nếu bạn khởi tạo max là 0, chương trình sẽ báo sai 0 là giá trị lớn nhất.
Có hai giải pháp cổ điển cho vấn đề này. Đầu tiên, nếu bạn biết phạm vi của các số mà bạn đang xem xét, bạn có thể chọn một giá trị phù hợp cho max. Trong trường hợp đó, bạn có thể đặt max là giá trị nhỏ nhất trong phạm vi. Điều này nghe có vẻ ngược đời vì thường chúng ta nghĩ giá trị lớn nhất phải lớn, nhưng ý tưởng là đặt max là giá trị nhỏ nhất có thể để bất kỳ giá trị nào lớn hơn sẽ khiến max được đặt lại thành giá trị đó. Ví dụ, nếu bạn biết dãy số trước đó là nhiệt độ trong độ Fahrenheit, bạn sẽ biết rằng nhiệt độ không thể nhỏ hơn 0 tuyệt đối (khoảng −460 độ Fahrenheit), do đó bạn có thể khởi tạo max thành giá trị đó.
Khả năng thứ hai là khởi tạo max thành giá trị đầu tiên trong dãy. Điều này không phải lúc nào cũng thuận tiện vì nó có nghĩa là phải lấy một trong các số nằm ngoài vòng lặp.
Khi kết hợp hai khả năng này, mã giả trở thành:

```
initialize max either to lowest possible value or to first value.
for (all numbers to examine) {
    obtain "next".
    if (next > max) {
        max = next.
    }
}
```

Mã giả để tính giá trị nhỏ nhất là một biến thể nhẹ của mã này:

```
initialize min either to highest possible value or to first value.
for (all numbers to examine) {
    obtain "next".
    if (next < min) {
        min = next.
    }
}
```

Để giúp bạn hiểu rõ hơn, hãy đưa mã giả này vào thực tế với một vấn đề thực sự. Trong toán học, có một vấn đề mở liên quan đến các dãy số gọi là dãy số hailstone. Các dãy số này thường tăng và giảm theo các mẫu khó dự đoán, tương tự như quá trình hình thành mưa đá.
Một dãy số hailstone là một dãy số trong đó mỗi giá trị x được theo sau bởi: (3x+1), nếu x lẻ; (x/2) nếu x chẵn.
Ví dụ, nếu bạn bắt đầu với số 7 và xây dựng một dãy số có độ dài 10, bạn sẽ có dãy số: 7, 22, 11, 34, 17, 52, 26, 13, 40, 20
Trong dãy này, các giá trị lớn nhất và nhỏ nhất lần lượt là 52 và 7. Nếu bạn kéo dài tính toán này thành một dãy có độ dài 20, bạn sẽ có dãy số: 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1, 4, 2, 1
Trong trường hợp này, các giá trị lớn nhất và nhỏ nhất lần lượt là 52 và 1.
Bạn sẽ nhận thấy rằng khi 1, 2, hoặc 4 xuất hiện trong dãy, dãy sẽ lặp lại. Người ta cho rằng tất cả các số nguyên cuối cùng đều đạt tới 1, giống như những hạt mưa đá rơi xuống mặt đất. Đây là một vấn đề chưa được giải quyết trong toán học. Không ai có thể bác bỏ nó, nhưng cũng không ai có thể chứng minh nó.
Hãy viết một phương thức nhận giá trị bắt đầu và độ dài dãy và in ra các giá trị lớn nhất và nhỏ nhất thu được trong một dãy hailstone được tạo thành từ giá trị bắt đầu đó và có độ dài đó.
Phương thức của chúng ta sẽ như thế này:

```
public static void printHailstoneMaxMin(int value, int length) {

    int min = value;
    int max = value;

    for (int i = 1; i <= length - 1; i++) {

        if (value % 2 == 0) {
            value = value / 2;
```

```
        } else {
            value = 3 * value + 1;
        }

        if (value > max) {
            max = value;
        } else if (value < min) {
            min = value;
        }
    }

    System.out.println("max = " + max);
    System.out.println("min = " + min);
}
```

Tổng cộng với điều kiện if

Bây giờ, hãy khám phá cách bạn có thể sử dụng các câu lệnh if/else để tạo ra những biến thể thú vị của thuật toán tổng cộng. Giả sử bạn muốn đọc một dãy số và tính trung bình. Nhiệm vụ này có vẻ như là một biến thể đơn giản của mã tổng cộng. Bạn có thể tính trung bình bằng cách chia tổng cho số lượng số:

```
double average = sum / totalNumber;
System.out.println("average = " + average);
```

Nhưng có một vấn đề nhỏ với mã này. Giả sử khi chương trình hỏi người dùng có bao nhiêu số cần xử lý, người dùng nhập 0. Khi đó, chương trình sẽ không vào vòng lặp tổng cộng, và mã của bạn sẽ cố gắng tính giá trị của 0 chia cho 0. Java sẽ in ra rằng trung bình là NaN, một thông báo bí ẩn có nghĩa là "Not a Number." Sẽ tốt hơn nếu chương trình in ra một thông báo khác cho biết rằng không có số nào để tính trung bình. Bạn có thể sử dụng câu lệnh if/else cho mục đích này:

```
if (totalNumber <= 0) {
    System.out.println("No numbers to average");
```

```
} else {
    double average = sum / totalNumber;
    System.out.println("average = " + average);
}
```

Một cách sử dụng khác của câu lệnh if là đếm số lượng số âm mà người dùng nhập vào. Bạn sẽ thường xuyên muốn đếm số lần một sự kiện xảy ra trong một chương trình. Mục tiêu này dễ dàng đạt được với câu lệnh if và một biến số nguyên gọi là bộ đếm. Bạn bắt đầu bằng cách khởi tạo bộ đếm về 0:

```
int negatives = 0;
```

Bạn có thể sử dụng bất kỳ tên nào bạn muốn cho biến này. Ở đây chúng ta sử dụng "negatives" vì đó là những gì bạn đang đếm. Bước quan trọng khác là tăng giá trị của bộ đếm trong vòng lặp nếu nó vượt qua bài kiểm tra:

```
if (next < 0) {
    negatives++;
}
```

Khi bạn kết hợp tất cả lại và chỉnh sửa nhận xét cùng phần mở đầu, bạn sẽ có biến thể sau của chương trình tổng cộng:

```
// Tìm trung bình của một dãy số cũng như
// báo cáo có bao nhiêu số do người dùng nhập là số âm.

import java.util.*;

public class ExamineNumbers2 {
    public static void main(String[] args) {
        System.out.println("This program examines a sequence");
        System.out.println("of numbers to find the average");
        System.out.println("and count how many are negative.");
        System.out.println();

        Scanner console = new Scanner(System.in);
        System.out.print("How many numbers do you have? ");
```

```
        int totalNumber = console.nextInt();

        int negatives = 0;
        double sum = 0.0;

        for (int i = 1; i <= totalNumber; i++) {
            System.out.print("    #" + i + "? ");
            double next = console.nextDouble();
            sum += next;
            if (next < 0) {
                negatives++;
            }
        }

        System.out.println();

        if (totalNumber <= 0) {
            System.out.println("No numbers to average");
        } else {
            double average = sum / totalNumber;
            System.out.println("average = " + average);
        }

        System.out.println("# of negatives = " + negatives);
    }
}
```

Khi chương trình chạy, kết quả sẽ trông như sau:

```
This program examines a sequence
of numbers to find the average
and count how many are negative.

How many numbers do you have?
8
```

```
   #1?
2.5

   #2?
9.2

   #3?
-19.4

   #4?
208.2

   #5?
42.3

   #6?
92.7

   #7?
-17.4

   #8?
8

average = 40.7625
# of negatives = 2
```

Lỗi làm tròn

Khi bạn khám phá các thuật toán tích lũy, bạn sẽ phát hiện ra một vấn đề cụ thể mà bạn nên hiểu. Ví dụ, hãy xem xét đoạn mã sau của chương trình ExamineNumbers2 với các đầu vào khác nhau từ người dùng:

Chương trình này kiểm tra một dãy số để tìm trung bình và đếm số lượng số âm.
Bạn có bao nhiêu số? 4
bash
Copy code
#1?
2.1
bash
Copy code
#2?
-3.8
bash
Copy code
#3?
5.4
bash
Copy code
#4?
7.4
Trung bình = 2.7750000000000004 Số lượng số âm = 1
Nếu bạn sử dụng máy tính, bạn sẽ thấy rằng bốn số này có tổng là 11.1. Nếu bạn chia số này cho 4, bạn sẽ nhận được 2.775. Tuy nhiên, Java lại báo cáo kết quả là 2.7750000000000004. Tại sao lại có quá nhiều số 0 và tại sao số cuối là 4? Câu trả lời là số thực có thể dẫn đến lỗi làm tròn.

Lỗi làm tròn

Một lỗi số học xảy ra vì các số thực được lưu trữ dưới dạng gần đúng thay vì giá trị chính xác. Lỗi làm tròn thường nhỏ và có thể xảy ra theo cả hai hướng (cao hơn hoặc thấp hơn một chút). Trong trường hợp trên, chúng ta đã gặp phải một lỗi làm tròn cao hơn một chút.
Số thực được lưu trữ trong một định dạng tương tự như ký hiệu khoa học, với một tập hợp các chữ số và một số mũ. Hãy xem xét

cách bạn sẽ lưu trữ giá trị một phần ba trong ký hiệu khoa học sử dụng cơ số 10. Bạn sẽ ghi rằng số đó là 3.33333 (lặp lại) nhân với 10 mũ -1. Tuy nhiên, chúng ta không thể lưu trữ một số lượng vô hạn các chữ số trên máy tính, vì vậy chúng ta phải dừng việc lặp lại các số 3 tại một điểm nào đó. Giả sử chúng ta có thể lưu trữ 10 chữ số, thì giá trị của một phần ba sẽ được lưu trữ là 3.333333333 nhân với 10 mũ -1. Nếu chúng ta nhân số đó với 3, chúng ta sẽ không nhận được 1 mà thay vào đó nhận được 9.999999999 nhân với 10 mũ -1 (tương đương với 0.9999999999).

Bạn có thể tự hỏi tại sao các số mà chúng ta sử dụng trong ví dụ trước lại gây ra vấn đề khi chúng không có bất kỳ chữ số lặp lại nào. Bạn phải nhớ rằng máy tính lưu trữ các số ở dạng cơ số 2. Các số như 2.1 và 5.4 có thể trông giống như những số đơn giản ở cơ số 10, nhưng chúng có các chữ số lặp lại khi được lưu trữ ở cơ số 2.

Lỗi làm tròn có thể dẫn đến những kết quả khá bất ngờ. Ví dụ, hãy xem xét chương trình ngắn sau:

java

Copy code

```
// Chương trình này minh họa lỗi làm tròn.
public class Roundoff {
    public static void main(String[] args) {
        double n = 1.0;
        for (int i = 1; i <= 10; i++) {
            n += 0.1;
            System.out.println(n);
        }
    }
}
```

Chương trình này thể hiện một phép tổng tích lũy cổ điển với một vòng lặp cộng thêm 0.1 vào số n mỗi khi vòng lặp thực thi. Chúng ta bắt đầu với n bằng 1.0 và vòng lặp lặp lại 10 lần, mà chúng ta

có thể mong đợi sẽ in ra các số 1.1, 1.2, 1.3, và cứ thế cho đến 2.0. Tuy nhiên, nó tạo ra đầu ra sau:
Copy code
1.1
1.2000000000000002
1.3000000000000003
1.4000000000000004
1.5000000000000004
1.6000000000000005
1.7000000000000006
1.8000000000000007
1.9000000000000008
2.000000000000001
Vấn đề xảy ra vì 0.1 không thể được lưu trữ chính xác trong cơ số 2 (nó tạo ra một tập hợp các chữ số lặp lại, giống như một phần ba trong cơ số 10). Mỗi lần qua vòng lặp, lỗi này được cộng dồn, dẫn đến lỗi làm tròn ngày càng tệ hơn.
Một ví dụ khác, hãy xem xét việc cộng các giá trị của một xu, một xu niken, một xu hào, và một đồng xu quarter. Nếu chúng ta sử dụng các biến kiểu int, chúng ta sẽ nhận được câu trả lời chính xác bất kể thứ tự cộng các số:
java
Copy code
jshell> int cents1 = 1 + 5 + 10 + 25;
cents1 ==> 41

jshell> int cents2 = 25 + 10 + 5 + 1;
cents2 ==> 41
Bất kể thứ tự, những con số này luôn cộng lại thành 41 xu.
Nhưng giả sử thay vì nghĩ về những giá trị này là các đồng xu nguyên, chúng ta nghĩ về chúng là các phần của một đô la được lưu trữ dưới dạng số thực:
java

Copy code
```
jshell> double dollars1 = 0.01 + 0.05 + 0.10 + 0.25;
dollars1 ==> 0.41000000000000003

jshell> double dollars2 = 0.25 + 0.10 + 0.05 + 0.01;
dollars2 ==> 0.41
```
Mặc dù chúng ta đang cộng chính xác các số như nhau, nhưng thứ tự cộng khác nhau lại tạo ra sự khác biệt. Nguyên nhân là do lỗi làm tròn.
Có một số bài học từ điều này:

- Hãy nhận thức rằng khi bạn lưu trữ các giá trị số thực (ví dụ: double), bạn đang lưu trữ các giá trị gần đúng chứ không phải chính xác. Nếu bạn cần lưu trữ giá trị chính xác, hãy lưu nó dưới dạng kiểu int.
- Đừng ngạc nhiên khi thấy các số hơi lệch so với giá trị mong đợi.
- Đừng mong đợi có thể so sánh các biến kiểu double cho sự bằng nhau.

Để thực hiện theo điểm thứ ba, hãy xem xét những gì mã trước đó sẽ dẫn đến nếu chúng ta thực hiện kiểm tra sau:
java
Copy code
```
if (dollars1 == dollars2) {
   ...
}
```
Kiểm tra này sẽ đánh giá là sai vì các giá trị rất gần nhau, nhưng không đủ gần để Java coi chúng là bằng nhau. Chúng ta hiếm khi sử dụng một bài kiểm tra cho sự bằng nhau tuyệt đối khi làm việc với double. Thay vào đó, chúng ta có thể sử dụng một kiểm tra như sau để xem các số có gần nhau không:
java
Copy code
```
if (Math.abs(dollars1 - dollars2) < 0.001) {
```

```
    ...
}
```
Chúng ta sử dụng phương thức giá trị tuyệt đối (abs) từ lớp Math để tìm độ lớn của sự khác biệt và sau đó kiểm tra xem nó có nhỏ hơn một giá trị nhỏ nào đó (trong trường hợp này là 0.001).
Sau đó trong chương này, chúng ta sẽ giới thiệu một biến thể của print/println gọi là printf, giúp dễ dàng in ra các số như thế này mà không có quá nhiều chữ số thừa.

Xử lý văn bản
Các lập trình viên thường gặp các vấn đề yêu cầu họ tạo, chỉnh sửa, kiểm tra và định dạng văn bản. Tổng thể, chúng ta gọi những công việc này là xử lý văn bản.
Xử lý văn bản
Chỉnh sửa và định dạng các chuỗi văn bản. Trong phần này, chúng ta sẽ tìm hiểu chi tiết hơn về kiểu nguyên thủy char và giới thiệu lệnh mới là System.out.printf. Cả hai công cụ này đều rất hữu ích cho các công việc xử lý văn bản.
Kiểu char
Kiểu nguyên thủy char đại diện cho một ký tự đơn trong văn bản. Bạn có thể tạo các biến, tham số và giá trị trả về thuộc kiểu char nếu bạn muốn. Các giá trị kiểu char được biểu thị bằng cách đặt ký tự trong dấu nháy đơn:
java
Copy code
```
char ch = 'A';
```
Cũng hợp lệ khi tạo giá trị char đại diện cho một chuỗi thoát:
java
Copy code
```
char newline = '\n';
```
Trong chương trước, chúng ta đã thảo luận về các đối tượng String. Sự khác biệt giữa char và String là một điều tinh tế gây nhầm lẫn cho nhiều lập trình viên Java mới. Sự khác biệt chính là

String là một đối tượng, trong khi char là một giá trị nguyên thủy. char chiếm một lượng bộ nhớ rất nhỏ, nhưng nó không có phương thức nào. Bảng 4.4 tóm tắt một số khác biệt giữa hai kiểu này.

Bảng 4.4: Khác biệt giữa char và String

char	String
Giá trị kiểu	nguyên thủy
Sử dụng bộ nhớ	2 byte
Phương thức	không có
Số lượng ký tự	đúng 1
Bao quanh bởi	dấu nháy đơn: 'c'
So sánh	<, >=, ==, ...

Tại sao Java có hai kiểu dữ liệu cho dữ liệu tương tự như vậy? Kiểu char tồn tại chủ yếu vì lý do lịch sử; nó xuất hiện từ các ngôn ngữ cũ hơn như C, đã ảnh hưởng đến thiết kế của Java.

Vậy tại sao người ta lại sử dụng kiểu char khi đã có String? Thông thường, cần sử dụng char vì một số phương thức trong API của Java sử dụng nó như một tham số hoặc kiểu trả về. Tuy nhiên, cũng có một số trường hợp mà việc sử dụng char có thể hữu ích hoặc đơn giản hơn so với String.

Các ký tự của một chuỗi String được lưu trữ bên trong đối tượng dưới dạng các giá trị kiểu char. Bạn có thể truy cập các ký tự riêng lẻ thông qua phương thức charAt của đối tượng, phương thức này nhận một chỉ số số nguyên làm tham số và trả về ký tự tại chỉ số đó. Chúng ta thường lặp qua một chuỗi để kiểm tra hoặc thay đổi các ký tự của nó. Ví dụ, phương thức sau in mỗi ký tự của một chuỗi trên một dòng riêng biệt:

java

Copy code

```
public static void printVertical(String message) {
    for (int i = 0; i < message.length(); i++) {
        char ch = message.charAt(i);
```

```
        System.out.println(ch);
    }
}
```

char và int

Các giá trị kiểu char được lưu trữ bên trong như các số nguyên. Một sơ đồ mã hóa tiêu chuẩn gọi là Unicode xác định giá trị số nguyên nào đại diện cho mỗi ký tự. (Unicode sẽ được trình bày chi tiết hơn sau trong chương này.) Vì các ký tự thực chất là các số nguyên, Java tự động chuyển đổi một giá trị kiểu char thành int bất cứ khi nào nó mong đợi một số nguyên:

java

Copy code

```
int letter = 'a' + 2; // lưu trữ giá trị 99
```

Hóa ra giá trị số nguyên của 'a' là 97, vì vậy kết quả của biểu thức là 99, được lưu trữ dưới dạng ký tự 'c'. Tương tự, một số nguyên có thể được chuyển đổi thành char bằng cách sử dụng ép kiểu. (Ép kiểu là cần thiết như một lời hứa với trình biên dịch, vì không phải mọi giá trị số nguyên đều tương ứng với một ký tự hợp lệ.) Dưới đây là một ví dụ về đoạn mã sử dụng ép kiểu để chuyển đổi giá trị số nguyên thành giá trị kiểu char:

java

Copy code

```
int code = 66;
char grade = (char) code; // lưu trữ 'B'
```

Vì các giá trị kiểu char thực chất là các số nguyên, chúng cũng có thể được so sánh bằng cách sử dụng các toán tử quan hệ như < hoặc ==. Ngoài ra, chúng có thể được sử dụng trong các vòng lặp để bao quát các phạm vi ký tự. Ví dụ, đoạn mã sau in ra mọi chữ cái của bảng chữ cái:

java

Copy code

```
for (char letter = 'a'; letter <= 'z'; letter++) {
    System.out.print(letter);
```

}
java
Copy code
if (grade == 'B') {... // true
Bạn có thể tìm hiểu thêm về sự tương đương giữa ký tự và số nguyên bằng cách tìm kiếm trên web các bảng Unicode.

Thuật toán Xử lý Văn bản Cộng dồn

Chuỗi ký tự thường được sử dụng trong các thuật toán cộng dồn như đã thảo luận ở các phần trước của chương này. Ví dụ, bạn có thể lặp qua các ký tự của một chuỗi để tìm một chữ cái cụ thể. Phương thức sau đây nhận vào một chuỗi và một ký tự, sau đó trả về số lần xuất hiện của ký tự trong chuỗi:

java
Copy code
```
public static int count(String text, char c) {
    int found = 0;
    for (int i = 0; i < text.length(); i++) {
        if (text.charAt(i) == c) {
            found++;
        }
    }
    return found;
}
```

Bạn có thể nối một ký tự với một chuỗi bằng cách sử dụng toán tử +. Sử dụng ý tưởng này, bạn có thể tạo một chuỗi bằng cách lặp qua các ký tự và nối chúng lại, bắt đầu với một chuỗi rỗng và nối các ký tự riêng lẻ trong vòng lặp. Đây được gọi là cộng dồn nối chuỗi. Phương thức sau đây nhận vào một chuỗi và trả về các ký tự của chuỗi đó theo thứ tự ngược lại:

java
Copy code
```
public static String reverse(String phrase) {
```

```
    String result = "";
    for (int i = 0; i < phrase.length(); i++) {
        result = phrase.charAt(i) + result;
    }
    return result;
}
```

Ví dụ, lệnh gọi reverse("Tin man") sẽ trả về "nam niT".

Vì char là một kiểu nguyên thủy, bạn không thể sử dụng cú pháp dấu chấm để gọi các phương thức như với một giá trị kiểu String. Thay vào đó, Java cung cấp một lớp gọi là Character, có một số phương thức tĩnh cho phép bạn kiểm tra loại ký tự (số, chữ cái, v.v.) và thực hiện các chuyển đổi khác nhau. Một số phương thức hữu ích nhất của lớp Character được liệt kê trong Bảng 4.5.

Bảng 4.5: Các Phương thức Hữu ích của Lớp Character

Phương thức	Mô tả	Ví dụ
getNumericValue(ch)	Chuyển đổi một ký tự trông giống số thành số tương ứng	Character.getNumericValue('6') trả về 6
isDigit(ch)	Kiểm tra ký tự có phải là một trong các chữ số từ '0' đến '9' hay không	Character.isDigit('X') trả về false
isLetter(ch)	Kiểm tra ký tự có phải là chữ cái từ 'a' đến	Character.isLetter('f') trả về true

Phương thức	Mô tả	Ví dụ
	'z' hoặc 'A' đến 'Z'	
isLowerCase(ch)	Kiểm tra ký tự có phải là chữ cái viết thường	Character.isLowerCase('Q') trả về false
isUpperCase(ch)	Kiểm tra ký tự có phải là chữ cái viết hoa	Character.isUpperCase('Q') trả về true
toLowerCase(ch)	Chuyển đổi ký tự thành chữ cái viết thường	Character.toLowerCase('Q') trả về 'q'
toUpperCase(ch)	Chuyển đổi ký tự thành chữ cái viết hoa	Character.toUpperCase('x') trả về 'X'

Phương thức sau đây đếm số lượng chữ cái trong một chuỗi, bỏ qua tất cả các ký tự không phải là chữ cái như dấu câu, số và khoảng trắng:

java

Copy code

```
public static int countLetters(String phrase) {
    int count = 0;
    for (int i = 0; i < phrase.length(); i++) {
        char ch = phrase.charAt(i);
        if (Character.isLetter(ch)) {
```

```
            count++;
        }
    }
    return count;
}
```

Ví dụ, lệnh gọi countLetters("gr8 JoB!") sẽ trả về 5.

System.out.printf

Cho đến nay, chúng ta đã sử dụng System.out.println và System.out.print để xuất kết quả ra màn hình. Có một phương thức thứ ba, System.out.printf, phức tạp hơn một chút nhưng mang lại cho chúng ta những khả năng mới hữu ích. Chữ "f" trong printf đại diện cho "formatted" (định dạng), ngụ ý rằng System.out.printf cho phép bạn kiểm soát nhiều hơn về định dạng của kết quả in ra.

Hãy tưởng tượng bạn muốn in một bảng cửu chương từ 1 đến 10. Đoạn mã sau in ra các số đúng, nhưng không trông đẹp lắm:

java

Copy code

```
for (int i = 1; i <= 10; i++) {
    for (int j = 1; j <= 10; j++) {
        System.out.print(i * j + " ");
    }
    System.out.println();
}
```

Kết quả là:

Copy code

```
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
6 12 18 24 30 36 42 48 54 60
7 14 21 28 35 42 49 56 63 70
```

8 16 24 32 40 48 56 64 72 80
9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100
Lưu ý rằng các số không thẳng hàng dọc.
Chúng ta có thể ngăn cách các số bằng tab, điều này sẽ tốt hơn. Nhưng cách này không cung cấp nhiều kiểm soát về giao diện của bảng. Mọi số sẽ cách nhau đúng 8 khoảng trắng trên màn hình và các số sẽ xuất hiện căn lề trái. Sẽ rất phiền phức nếu cố gắng căn lề phải các số một cách thủ công, vì bạn sẽ phải sử dụng các câu lệnh if/else để kiểm tra xem một số có nằm trong một khoảng nhất định hay không và nếu cần, thêm các khoảng trắng vào.

BẠN CÓ BIẾT?
ASCII và Unicode
Chúng ta lưu trữ dữ liệu trên máy tính dưới dạng số nhị phân (chuỗi các số 0 và 1). Để lưu trữ dữ liệu văn bản, chúng ta cần một hệ thống mã hóa để cho biết chuỗi các số 0 và 1 nào được sử dụng cho bất kỳ ký tự nào. Hãy nghĩ về nó như một vòng mã hóa khổng lồ, ví dụ: "Nếu bạn muốn lưu trữ chữ cái viết thường ‘a,’ hãy sử dụng chuỗi 01100001."
Vào đầu những năm 1960, IBM đã phát triển một hệ thống mã hóa có tên là EBCDIC, hoạt động tốt với các thẻ đục lỗ của công ty, những thẻ đã được sử dụng hàng thập kỷ trước khi máy tính được phát minh. Nhưng sau đó rõ ràng rằng EBCDIC không phải là hệ thống mã hóa thuận tiện cho các lập trình viên máy tính. Có những khoảng trống trong chuỗi làm cho các ký tự như 'i' và 'j' xuất hiện cách xa nhau mặc dù chúng đứng cạnh nhau.
Năm 1967, Hiệp hội Tiêu chuẩn Hoa Kỳ đã công bố một hệ thống được gọi là ASCII (được phát âm là "AS-kee") đã được sử dụng rộng rãi kể từ đó. ASCII là viết tắt của "Mã Tiêu chuẩn Hoa Kỳ cho Trao đổi Thông tin." Ở dạng ban đầu, ASCII xác định 128 ký tự mà mỗi ký tự có thể được lưu trữ bằng 7 bit dữ liệu.

Vấn đề lớn nhất của ASCII là nó là mã dành cho Hoa Kỳ. Có rất nhiều ký tự được sử dụng phổ biến ở các quốc gia khác không được bao gồm trong ASCII. Ví dụ, ký tự bảng Anh (£) và biến thể chữ n của tiếng Tây Ban Nha (ñ) không có trong 128 ký tự ASCII tiêu chuẩn. Nhiều nỗ lực đã được thực hiện để mở rộng ASCII, tăng gấp đôi lên 256 ký tự để bao gồm nhiều ký tự đặc biệt này. Tuy nhiên, hóa ra thậm chí 256 ký tự cũng không đủ để nắm bắt sự đa dạng đáng kinh ngạc của giao tiếp con người.
Vào thời điểm Java được tạo ra, một tập đoàn các chuyên gia phần mềm đã giới thiệu một tiêu chuẩn mới để mã hóa ký tự được gọi là Unicode. Họ quyết định rằng 7 bit của ASCII tiêu chuẩn và 8 bit của ASCII mở rộng là không đủ và không đặt giới hạn về số bit họ có thể sử dụng để mã hóa ký tự. Tại thời điểm viết bài này, tập đoàn đã xác định được hơn 110.000 ký tự, cần hơn 16 bit để lưu trữ. Unicode bao gồm các ký tự được sử dụng trong hầu hết các ngôn ngữ hiện đại và thậm chí một số ngôn ngữ cổ đại. Chữ tượng hình Ai Cập đã được thêm vào năm 2007, mặc dù nó vẫn chưa bao gồm chữ tượng hình Maya và tập đoàn đã từ chối đề xuất bao gồm các ký tự Klingon.
Những nhà thiết kế Java đã sử dụng Unicode làm tiêu chuẩn cho kiểu char, điều đó có nghĩa là các chương trình Java có khả năng thao tác với toàn bộ các ký tự. May mắn thay, tập đoàn Unicode đã quyết định tích hợp các mã ASCII, do đó ASCII có thể được coi là một phần phụ của Unicode. Nếu bạn tò mò về thứ tự thực tế của các ký tự trong ASCII, hãy tìm kiếm "bảng mã ASCII" trên công cụ tìm kiếm yêu thích của bạn và bạn sẽ tìm thấy hàng triệu kết quả để khám phá.
Một cách dễ dàng hơn nhiều để in các giá trị được căn chỉnh trong các trường có chiều rộng cố định là sử dụng lệnh System.out.printf. Phương thức printf chấp nhận một chuỗi đặc biệt gọi là chuỗi định dạng, chỉ định hình thức tổng quát của đầu ra, theo sau là bất kỳ tham số nào cần được đưa vào đầu ra:
java

Copy code
System.out.printf(<chuỗi định dạng>, <tham số>,..., <tham số>);
Chuỗi định dạng giống như một chuỗi bình thường, ngoại trừ nó có thể chứa các trình giữ chỗ gọi là định dạng đặc tả, cho phép bạn chỉ định vị trí mà giá trị của một biến cần được chèn vào, cùng với định dạng mà bạn muốn áp dụng cho giá trị đó. Các định dạng đặc tả bắt đầu bằng ký hiệu % và kết thúc bằng một chữ cái chỉ loại giá trị, chẳng hạn như d cho số nguyên thập phân (int) hoặc f cho số thực (các số thực thuộc kiểu double). Hãy xem xét câu lệnh printf sau:
java
Copy code
int x = 38, y = -152;
System.out.printf("location: (%d, %d)\n", x, y);
Câu lệnh này sẽ tạo ra đầu ra sau:
java
Copy code
location: (38, -152)
Ký hiệu %d không thực sự được in mà thay vào đó được thay thế bằng tham số tương ứng được viết sau chuỗi định dạng. Số lượng định dạng đặc tả trong chuỗi định dạng phải khớp với số lượng tham số theo sau nó. Định dạng đặc tả đầu tiên sẽ được thay thế bằng tham số đầu tiên, định dạng đặc tả thứ hai sẽ được thay thế bằng tham số thứ hai, và cứ tiếp tục như vậy.
System.out.printf là một phương thức đặc biệt vì nó có thể chấp nhận số lượng tham số khác nhau.
Lệnh printf giống như System.out.print ở chỗ nó không chuyển sang dòng mới trừ khi bạn chỉ rõ điều đó. Lưu ý rằng trong mã trước đó, chúng ta đã kết thúc chuỗi định dạng với \n để hoàn thành dòng đầu ra.
Vì định dạng đặc tả sử dụng ký tự % như một ký tự đặc biệt, nếu bạn muốn in ký hiệu % trong một câu lệnh printf, hãy viết hai ký tự % liên tiếp. Ví dụ:

java
Copy code
```
int score = 87;
System.out.printf("You got %d%% on the exam!\n", score);
```
Mã này sẽ tạo ra đầu ra sau:
java
Copy code
```
You got 87% on the exam!
```
Một định dạng đặc tả có thể chứa thông tin sau ký hiệu % để chỉ định chiều rộng, độ chính xác và căn chỉnh của giá trị được in. Ví dụ, %8d chỉ định một số nguyên được căn chỉnh bên phải trong một trường có chiều rộng 8 ký tự, và %12.4f chỉ định một giá trị double được căn chỉnh bên phải trong một trường có chiều rộng 12 ký tự, được làm tròn đến bốn chữ số sau dấu thập phân. Bảng 4.6 liệt kê một số định dạng đặc tả phổ biến mà bạn có thể muốn sử dụng trong chương trình của mình.

Định dạng	**Kết quả**
%d	Số nguyên
%8d	Số nguyên, căn phải, trường rộng 8 ký tự
%-6d	Số nguyên, căn trái, trường rộng 6 ký tự
%f	Số thực
%12f	Số thực, căn phải, trường rộng 12 ký tự
%.2f	Số thực, làm tròn đến chữ số thập phân thứ hai
%16.3f	Số thực, làm tròn đến chữ số thập phân thứ ba, trường rộng 16 ký tự
%s	Chuỗi
%8s	Chuỗi, căn phải, trường rộng 8 ký tự
%-9s	Chuỗi, căn trái, trường rộng 9 ký tự

Ví dụ, giả sử rằng các biến sau đã được khai báo để đại diện cho thông tin về một sinh viên:

java
Copy code
int score = 87;
double gpa = 3.18652;
String name = "Jessica";
Đoạn mã sau in các biến trên với nhiều định dạng đặc tả:
java
Copy code
System.out.printf("student name: %10s\n", name);
System.out.printf("exam score: %10d\n", score);
System.out.printf("GPA: %10.2f\n", gpa);
Mã này sẽ tạo ra đầu ra sau:
java
Copy code
student name: Jessica
exam score: 87
GPA: 3.19
Ba giá trị được căn chỉnh theo mép phải của chúng, vì chúng ta in tất cả với chiều rộng 10 ký tự. Phương thức printf làm cho việc căn chỉnh các giá trị trong cột dễ dàng theo cách này. Lưu ý rằng GPA của sinh viên được làm tròn thành 3.19, do đặc tả 2 trong định dạng của biến đó. Định dạng đặc tả 10.2 làm cho giá trị phù hợp trong một khu vực rộng 10 ký tự với chính xác 2 chữ số sau dấu thập phân.
Bây giờ, hãy quay lại ví dụ bảng nhân của chúng ta. Bây giờ chúng ta đã biết về printf, chúng ta có thể in bảng với các số được căn chỉnh bên phải tương đối dễ dàng. Chúng ta sẽ căn chỉnh các số vào các trường rộng 5 ký tự:
java
Copy code
```
for (int i = 1; i <= 10; i++) {
    for (int j = 1; j <= 10; j++) {
        System.out.printf("%5d", i * j);
```

```
    }
    System.out.println();
}
```

Mã này tạo ra đầu ra sau:

java

Copy code

```
 1    2    3    4    5    6    7    8    9    10
 2    4    6    8   10   12   14   16   18    20
 3    6    9   12   15   18   21   24   27    30
 4    8   12   16   20   24   28   32   36    40
 5   10   15   20   25   30   35   40   45    50
 6   12   18   24   30   36   42   48   54    60
 7   14   21   28   35   42   49   56   63    70
 8   16   24   32   40   48   56   64   72    80
 9   18   27   36   45   54   63   72   81    90
10   20   30   40   50   60   70   80   90   100
```

Phương thức printf cũng có thể giải quyết vấn đề với chương trình Roundoff được giới thiệu trước đó trong chương này. Việc sửa độ chính xác của giá trị double đảm bảo rằng nó sẽ được làm tròn để tránh những lỗi làm tròn nhỏ do số học của double. Dưới đây là chương trình đã sửa:

java

Copy code

```
1  // Sử dụng System.out.printf để khắc phục lỗi làm tròn.
2  public class Roundoff2 {
3      public static void main(String[] args) {
4          double n = 1.0;
5          for (int i = 1; i <= 10; i++) {
6              n += 0.1;
7              System.out.printf("%3.1f\n", n);
8          }
9      }
10 }
```

Chương trình này tạo ra đầu ra sau:
java
Copy code
1.1
1.2
1.3
1.4
1.5
1.6
1.7
1.8
1.9
2.0

4.4 Các Phương Thức Với Điều Kiện Thực Thi

Chúng ta đã giới thiệu rất nhiều thông tin về các phương thức trong Chương 3, bao gồm cách sử dụng tham số để truyền giá trị vào phương thức và cách sử dụng câu lệnh return để phương thức trả về một giá trị. Bây giờ, khi chúng ta đã giới thiệu về thực thi có điều kiện, chúng ta cần quay lại các vấn đề này để bạn có thể hiểu sâu hơn về chúng.

Điều kiện trước và Điều kiện sau

Mỗi khi bạn viết một phương thức, bạn nên suy nghĩ chính xác về điều mà phương thức đó cần phải hoàn thành. Bạn có thể mô tả cách một phương thức hoạt động bằng cách mô tả các điều kiện trước phải đúng trước khi phương thức thực thi và các điều kiện sau sẽ đúng sau khi phương thức đã thực thi xong.

Điều kiện trước

Là điều kiện phải đúng trước khi một phương thức thực thi để đảm bảo rằng phương thức có thể hoàn thành nhiệm vụ của nó.

Điều kiện sau

Là điều kiện mà phương thức đảm bảo sẽ đúng sau khi nó hoàn thành thực thi, miễn là các điều kiện trước đã đúng trước khi phương thức được gọi.
Ví dụ, nếu bạn mô tả nhiệm vụ của một người trên dây chuyền lắp ráp ô tô, bạn có thể sử dụng điều kiện sau như: "Các bu lông cố định lốp trước bên trái đã được gắn vào xe và chặt chẽ." Nhưng điều kiện sau không phải là toàn bộ câu chuyện. Nhân viên trên dây chuyền lắp ráp phụ thuộc vào nhau. Một công nhân không thể thêm bu lông và siết chặt chúng nếu lốp trái không ở đó hoặc nếu không có bu lông. Vì vậy, công nhân lắp ráp có thể có các điều kiện trước như: "Lốp trái đã được gắn đúng cách vào xe, có ít nhất tám bu lông trong hộp cung cấp, và có một cờ lê hoạt động." Bạn mô tả đầy đủ nhiệm vụ bằng cách nói rằng công nhân có thể làm cho các điều kiện sau đúng nếu các điều kiện trước đúng trước khi bắt đầu.
Giống như các công nhân trên dây chuyền lắp ráp, các phương thức cần làm việc cùng nhau, mỗi phương thức giải quyết một phần nhiệm vụ để tất cả chúng có thể giải quyết nhiệm vụ tổng thể. Các điều kiện trước và sau mô tả sự phụ thuộc lẫn nhau giữa các phương thức.

Ném ngoại lệ

Chúng ta đã thấy một số trường hợp trong đó Java có thể ném ra một ngoại lệ. Ví dụ, nếu chúng ta có một Scanner trên bảng điều khiển và chúng ta gọi nextInt, chương trình sẽ ném ra một ngoại lệ nếu người dùng nhập vào thứ gì đó không phải là một số nguyên. Trong Phụ lục C, chúng ta sẽ xem xét cách bạn có thể xử lý ngoại lệ. Bây giờ, chúng ta chỉ muốn khám phá một số cách mà ngoại lệ có thể xảy ra và cách bạn có thể muốn tạo ra chúng trong mã của mình.
Lý tưởng nhất là các chương trình thực thi mà không sinh ra bất kỳ lỗi nào, nhưng trên thực tế, có rất nhiều vấn đề phát sinh. Nếu bạn yêu cầu người dùng nhập một số nguyên, người dùng có thể

vô tình hoặc thậm chí cố ý nhập thứ gì đó không phải là số nguyên. Hoặc mã của bạn có thể có lỗi.
Chương trình sau luôn ném ra một ngoại lệ vì nó cố gắng tính giá trị của 1 chia cho 0, điều này không xác định về mặt toán học:
java
Copy code
```
public class CauseException {
    public static void main(String[] args) {
        int x = 1 / 0;
        System.out.println(x);
    }
}
```
Khi bạn chạy chương trình, bạn sẽ nhận được thông báo lỗi sau:
php
Copy code
```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at CauseException.main(CauseException.java:3)
```
Vấn đề xảy ra ở dòng 3, khi bạn yêu cầu Java tính toán một giá trị không thể lưu trữ dưới dạng số nguyên. Java phải làm gì với giá trị đó? Nó ném ra một ngoại lệ, dừng chương trình thực thi và cảnh báo bạn rằng đã xảy ra một ngoại lệ số học trong khi chương trình đang thực thi dòng mã cụ thể đó.
Đáng lưu ý là phép chia cho 0 không phải lúc nào cũng tạo ra ngoại lệ. Bạn sẽ không gặp ngoại lệ nếu bạn thực thi dòng mã sau:
java
Copy code
```
double x = 1.0 / 0.0;
```
Trong trường hợp này, chương trình thực thi bình thường và xuất ra giá trị Infinity. Điều này là do các số dấu phẩy động tuân theo một tiêu chuẩn từ Viện Kỹ sư Điện và Điện tử (IEEE) định nghĩa

chính xác điều gì sẽ xảy ra trong những trường hợp này, và có các giá trị đặc biệt biểu thị vô cực và "NaN" (không phải số).
Bạn có thể muốn tự ném ngoại lệ trong mã của mình. Đặc biệt, sẽ là ý tưởng tốt nếu ném ngoại lệ khi một điều kiện trước thất bại. Ví dụ, giả sử bạn muốn viết một phương thức để tính giai thừa của một số nguyên. Giai thừa được định nghĩa như sau:
css
Copy code
n! (đọc là "n giai thừa") = 1 * 2 * 3 * ... * n
Bạn có thể viết một phương thức Java sử dụng tích lũy để tính toán kết quả này:
java
Copy code
```
// pre : n >= 0
// post: trả về giai thừa của n (n!)
public static int factorial(int n) {
    if (n < 0) {
        throw new IllegalArgumentException("negative n: " + n);
    }
    int product = 1;
    for (int i = 2; i <= n; i++) {
        product = product * i;
    }
    return product;
}
```
Bạn có thể kiểm tra phương thức này với các giá trị khác nhau bằng một vòng lặp:
java
Copy code
```
for (int i = 0; i <= 10; i++) {
    System.out.println(i + "! = " + factorial(i));
}
```
Vòng lặp này sẽ tạo ra đầu ra sau:

yaml
Copy code
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
Dường như lạ lùng khi phương thức giai thừa trả về 1 khi nó được yêu cầu tính 0!, nhưng đó thực sự là một phần của định nghĩa toán học của hàm giai thừa. Nó trả về 1 vì biến cục bộ product trong phương thức giai thừa được khởi tạo bằng 1, và vòng lặp không được thực thi khi tham số n có giá trị 0. Vì vậy, đây thực sự là hành vi mong muốn cho 0!.
Nhưng điều gì sẽ xảy ra nếu bạn được yêu cầu tính giai thừa của một số âm? Phương thức này vẫn trả về giá trị 1. Định nghĩa toán học của giai thừa nói rằng hàm này không được định nghĩa cho các giá trị âm của n, vì vậy thực sự nó không nên tính toán kết quả khi n là âm. Chỉ chấp nhận các số bằng 0 hoặc dương là một điều kiện trước của phương thức này, có thể được mô tả trong tài liệu:
java
Copy code
// pre : n >= 0
// post: trả về giai thừa của n (n!)
public static int factorial(int n) {
    if (n < 0) {
        throw new IllegalArgumentException("negative n: " + n);

```
    }
    int product = 1;
    for (int i = 2; i <= n; i++) {
        product = product * i;
    }
    return product;
}
```

Bạn không cần một câu lệnh else sau if ném ra ngoại lệ, vì khi một ngoại lệ được ném ra, nó dừng việc thực thi của phương thức. Vì vậy, nếu ai đó gọi phương thức giai thừa với giá trị âm của n, Java sẽ không bao giờ thực thi mã tiếp theo sau câu lệnh throw.

Bạn có thể kiểm tra mã này với phương thức main sau:

java

Copy code

```
public static void main(String[] args) {
    System.out.println(factorial(-1));
}
```

Khi bạn thực thi chương trình này, nó sẽ dừng thực thi và in ra thông báo sau:

mathematica

Copy code

```
Exception in thread "main" java.lang.IllegalArgumentException: negative n: -1
    at Factorial2.factorial(Factorial2.java:8)
    at Factorial2.main(Factorial2.java:3)
```

Thông báo này chỉ ra rằng chương trình Factorial2 đã dừng chạy vì một ngoại lệ IllegalArgumentException được ném ra với giá trị âm của n là -1. Hệ thống sau đó hiển thị cho bạn một dấu vết ngược để chỉ ra cách nó đến đó. Tham số không hợp lệ xuất hiện ở dòng 8 của phương thức factorial của lớp Factorial2. Nó đến đó vì một cuộc gọi ở dòng 3 của phương thức main của lớp

Factorial2. Loại thông tin này rất hữu ích khi bạn muốn tìm lỗi trong chương trình của mình.
Ném ngoại lệ là một ví dụ của lập trình phòng thủ. Chúng ta không có ý định để lỗi xảy ra trong các chương trình mà chúng ta viết, nhưng chúng ta chỉ là con người, vì vậy chúng ta muốn xây dựng các cơ chế giúp phản hồi khi chúng ta mắc lỗi. Viết mã kiểm tra các giá trị được truyền vào các phương thức và ném ra IllegalArgumentException khi một giá trị không phù hợp là một cách tuyệt vời để cung cấp phản hồi đó.

Xem lại Giá Trị Trả Về

Trong Chương 3, chúng ta đã xem xét một số ví dụ về các phương thức tính toán đơn giản trả về một giá trị, như trong phương thức để tìm tổng của n số nguyên đầu tiên:
java
Copy code
```
public static int sum(int n) {
    return (n + 1) * n / 2;
}
```
Bây giờ, khi bạn đã biết cách viết các câu lệnh if/else, chúng ta có thể xem xét một số ví dụ thú vị hơn liên quan đến giá trị trả về. Ví dụ, trong chương này, bạn đã thấy rằng lớp Math có một phương thức gọi là max trả về giá trị lớn hơn trong hai giá trị. Thực tế có hai phiên bản khác nhau của phương thức này, một phiên bản tìm giá trị lớn hơn của hai số nguyên và một phiên bản tìm giá trị lớn hơn của hai số dấu phẩy động. Hãy nhớ rằng khi hai phương thức có cùng tên (nhưng khác tham số), điều này được gọi là nạp chồng (overloading).
Hãy viết phiên bản riêng của phương thức max trả về giá trị lớn hơn của hai số nguyên. Phần khai báo của nó sẽ trông như thế này:
java

Copy code
```
public static int max(int x, int y) {
    ...
}
```
Chúng ta muốn trả về hoặc x hoặc y, tùy thuộc vào giá trị nào lớn hơn. Đây là lúc thích hợp để sử dụng cấu trúc if/else:
java
Copy code
```
public static int max(int x, int y) {
    if (x > y) {
        return x;
    } else {
        return y;
    }
}
```
Đoạn mã này bắt đầu bằng việc kiểm tra xem x có lớn hơn y không. Nếu có, máy tính thực thi nhánh đầu tiên bằng cách trả về x. Nếu không, máy tính thực thi nhánh else bằng cách trả về y. Nhưng điều gì xảy ra nếu x và y bằng nhau? Đoạn mã trước đó sẽ thực thi nhánh else khi các giá trị bằng nhau, nhưng thực tế không quan trọng nhánh trả về nào được thực thi khi x và y bằng nhau.
Hãy nhớ rằng khi Java thực thi một câu lệnh return, phương thức sẽ dừng thực thi. Nó giống như một lệnh yêu cầu Java "thoát khỏi phương thức này ngay bây giờ." Điều đó có nghĩa là phương thức này cũng có thể được viết như sau:
java
Copy code
```
public static int max(int x, int y) {
    if (x > y) {
        return x;
    }
    return y;
```

}
Phiên bản mã này có hành vi tương đương vì câu lệnh return x trong câu lệnh if sẽ khiến Java thoát khỏi phương thức ngay lập tức và không thực thi câu lệnh return theo sau câu lệnh if. Mặt khác, nếu chúng ta không vào trong câu lệnh if, chúng ta sẽ trực tiếp thực thi câu lệnh return y.
Việc bạn chọn sử dụng phiên bản đầu tiên hay phiên bản thứ hai trong chương trình của mình phụ thuộc vào sở thích cá nhân. Cấu trúc if/else làm rõ rằng phương thức đang lựa chọn giữa hai lựa chọn, nhưng một số người thích phiên bản thứ hai vì nó ngắn gọn hơn.
Một ví dụ khác là phương thức indexOf của lớp String. Bạn có thể sử dụng các biểu thức như sau để xác định vị trí của một ký tự cụ thể trong một chuỗi:
java
Copy code
String s = "four score and seven years ago";
int r = s.indexOf('r');
int v = s.indexOf('v');
Mã này gán r là 3 vì 3 là chỉ số của lần xuất hiện đầu tiên của chữ cái 'r' trong chuỗi. Nó gán v là 17 vì đó là chỉ số của lần xuất hiện đầu tiên của chữ cái 'v' trong chuỗi.
Phương thức indexOf là một phần của lớp String, nhưng hãy xem cách chúng ta có thể viết một phương thức khác thực hiện cùng nhiệm vụ. Phương thức của chúng ta sẽ được gọi khác đi vì nó là một phương thức tĩnh nằm ngoài đối tượng String. Chúng ta sẽ phải truyền vào cả chuỗi và ký tự:
java
Copy code
int r = indexOf('r', s);
int v = indexOf('v', s);
Vì vậy, phần khai báo cho phương thức của chúng ta sẽ như sau:
java

Copy code
```
public static int indexOf(char ch, String s) {
    ...
}
```
Hãy nhớ rằng khi một phương thức trả về một giá trị, chúng ta phải bao gồm kiểu trả về sau từ khóa public static. Trong trường hợp này, chúng ta đã chỉ ra rằng phương thức trả về một giá trị kiểu int vì chỉ số sẽ là một số nguyên.
Nhiệm vụ này có thể được giải quyết khá tốt bằng một vòng lặp for duyệt qua từng chỉ số có thể có từ đầu đến cuối. Chúng ta có thể mô tả điều này bằng mã giả như sau:
java
Copy code
```
for (int i = 0; i < s.length(); i++) {
    if (s.charAt(i) == ch) {
        return i;
    }
}
```
Để hiểu đoạn mã này, bạn cần hiểu cách câu lệnh return hoạt động. Ví dụ, nếu chuỗi s là chuỗi từ ví dụ của chúng ta ("four score...") và chúng ta đang tìm kiếm ký tự 'r', chúng ta biết rằng khi i bằng 3, chúng ta sẽ thấy rằng s.charAt(3) bằng 'r'. Trường hợp đó sẽ khiến mã của chúng ta thực thi câu lệnh return, nghĩa là:
java
Copy code
```
return 3;
```
Khi câu lệnh return được thực thi, Java sẽ thoát khỏi phương thức ngay lập tức, điều đó có nghĩa là chúng ta thoát khỏi vòng lặp và trả về 3 làm kết quả. Mặc dù vòng lặp thông thường sẽ tăng i lên 4 và tiếp tục, nhưng mã của chúng ta sẽ không làm điều đó vì chúng ta đã gặp câu lệnh return.

Chỉ có một điều thiếu sót trong mã của chúng ta. Nếu chúng ta cố gắng biên dịch nó như hiện tại, chúng ta sẽ nhận được thông báo lỗi từ trình biên dịch Java:
kotlin
Copy code
missing return statement
Thông báo lỗi này xuất hiện vì chúng ta chưa nói cho Java biết phải làm gì nếu chúng ta không bao giờ tìm thấy ký tự mà chúng ta đang tìm kiếm. Trong trường hợp đó, chúng ta sẽ thực thi vòng lặp for hoàn toàn và đạt đến cuối phương thức mà không trả về giá trị nào. Điều này không thể chấp nhận được. Nếu chúng ta nói rằng phương thức trả về một giá trị int, chúng ta phải đảm bảo rằng mọi đường dẫn trong phương thức đều trả về một giá trị int. Nếu chúng ta không tìm thấy ký tự, chúng ta muốn trả về một giá trị đặc biệt nào đó để chỉ ra rằng ký tự không được tìm thấy. Chúng ta không thể sử dụng giá trị 0, vì 0 là một chỉ số hợp lệ cho một chuỗi (chỉ số của ký tự đầu tiên). Vì vậy, quy ước trong Java là trả về -1 nếu ký tự không được tìm thấy. Rất dễ dàng để thêm mã cho câu lệnh trả về này sau vòng lặp for:
java
Copy code

```
public static int indexOf(char ch, String s) {
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == ch) {
            return i;
        }
    }
    return -1;
}
```

Lỗi Lập Trình Phổ Biến
Lỗi: Chỉ Mục Chuỗi Vượt Ngoài Phạm Vi

Rất dễ quên rằng chỉ mục cuối cùng của một chuỗi có độ dài n thực sự là n-1. Quên điều này có thể khiến bạn viết các vòng lặp xử lý văn bản không chính xác như sau: // Phiên bản này của mã có lỗi! // Kiểm tra điều kiện nên là i < s.length()
java
Copy code
```
public static int indexOf(char ch, String s) {
    for (int i = 0; i <= s.length(); i++) {
        if (s.charAt(i) == ch) {
            return i;
        }
    }
    return -1;
}
```
Chương trình sẽ ném ra một ngoại lệ nếu vòng lặp vượt quá cuối chuỗi. Trong lần lặp cuối cùng, giá trị của biến i sẽ bằng với s.length(). Khi thực hiện câu lệnh if, chương trình sẽ ném ra ngoại lệ với thông báo lỗi như sau:
arduino
Copy code
```
Exception in thread "main"
    java.lang.StringIndexOutOfBoundsException:
    String index out of range: 11
        at java.lang.String.charAt(Unknown Source)
        at
OutOfBoundsExample.indexOf(OutOfBoundsExample.java:9)
        at
OutOfBoundsExample.main(OutOfBoundsExample.java:4)
```
Điều thú vị về lỗi trong ví dụ này là nó chỉ xảy ra nếu chuỗi không chứa ký tự ch. Nếu ch có trong chuỗi, kiểm tra if sẽ đúng với một trong những chỉ mục hợp lệ trong s, vì vậy mã sẽ trả về chỉ mục đó. Chỉ khi tất cả các ký tự từ s đã được kiểm tra mà không tìm thấy ch, vòng lặp mới thực hiện vòng lặp cuối cùng gây ra lỗi.

Có vẻ lạ là chúng ta không kiểm tra câu lệnh return cuối cùng trả về -1, nhưng hãy nhớ rằng vòng lặp for thử mọi chỉ mục có thể của chuỗi để tìm ký tự. Nếu ký tự xuất hiện ở bất kỳ đâu trong chuỗi, câu lệnh return bên trong vòng lặp sẽ được thực thi và chúng ta sẽ không bao giờ đến câu lệnh return sau vòng lặp. Cách duy nhất để đến câu lệnh return sau vòng lặp là khi ký tự không xuất hiện trong chuỗi đã cho.

Lập Luận Về Các Đường Dẫn

Sự kết hợp của if/else và return rất mạnh mẽ. Nó cho phép bạn giải quyết nhiều vấn đề phức tạp dưới dạng một phương thức nhận một số đầu vào và tính toán kết quả. Nhưng bạn phải cẩn thận suy nghĩ về các đường dẫn khác nhau tồn tại trong mã bạn viết. Ban đầu, quá trình này có vẻ khó chịu, nhưng khi bạn quen với nó, bạn sẽ thấy rằng nó cho phép bạn đơn giản hóa mã của mình.

Ví dụ, giả sử rằng chúng ta muốn chuyển đổi điểm số SAT thành một xếp hạng để sử dụng trong việc xét tuyển đại học. Mỗi phần của SAT có điểm từ 200 đến 800, vì vậy tổng điểm chung dao động từ 400 đến 1600. Giả sử một trường đại học giả định chia dải này thành ba nhóm, với tổng điểm dưới 800 được coi là không cạnh tranh, từ 800 đến dưới 1200 được coi là cạnh tranh, và từ 1200 đến 1600 được coi là rất cạnh tranh.

Hãy viết một phương thức có tên là rating để nhận tổng điểm SAT làm tham số và trả về chuỗi với văn bản phù hợp. Chúng ta có thể sử dụng toán tử AND được mô tả trước đó để viết một cấu trúc if/else có các kiểm tra cho từng dải này:

java

Copy code

```
public static String rating(int totalSAT) {
    if (totalSAT >= 400 && totalSAT < 800) {
        return "not competitive";
    } else if (totalSAT >= 800 && totalSAT < 1200) {
        return "competitive";
```

```
    } else if (totalSAT >= 1200 && totalSAT <= 1600) {
        return "highly competitive";
    }
}
```

Phương thức này đã được viết một cách hợp lý với các kiểm tra cụ thể cho từng trường hợp, nhưng nó không biên dịch được. Trình biên dịch chỉ ra rằng cuối phương thức có một "thiếu câu lệnh return." Điều này có vẻ kỳ lạ vì có ba câu lệnh return khác nhau trong phương thức này. Chúng ta đã bao gồm một câu lệnh return cho mỗi trường hợp khác nhau, vậy tại sao lại có lỗi biên dịch?

Khi trình biên dịch gặp một phương thức mà dự kiến sẽ trả về một giá trị, nó tính toán mọi đường dẫn có thể qua phương thức đó và đảm bảo rằng mỗi đường dẫn kết thúc bằng một lệnh return. Phương thức chúng ta đã viết có bốn đường dẫn qua nó. Nếu kiểm tra đầu tiên thành công, phương thức sẽ trả về "not competitive". Nếu không, nếu kiểm tra thứ hai thành công, phương thức sẽ trả về "competitive". Nếu cả hai kiểm tra đó thất bại nhưng kiểm tra thứ ba thành công, phương thức sẽ trả về "highly competitive". Nhưng nếu cả ba kiểm tra đều thất bại, đó sẽ là đường dẫn thứ tư không có câu lệnh return liên kết với nó. Thay vào đó, chúng ta sẽ đạt đến cuối phương thức mà không trả về giá trị nào. Điều đó không được chấp nhận, đó là lý do tại sao trình biên dịch tạo ra thông báo lỗi.

Việc xử lý một trường hợp thứ tư có vẻ khó chịu vì chúng ta biết rằng tổng điểm SAT sẽ luôn nằm trong khoảng từ 400 đến 1600. Mã của chúng ta đã bao phủ tất cả các trường hợp mà chúng ta mong đợi cho phương thức này, nhưng điều đó không đủ. Java yêu cầu chúng ta bao phủ mọi trường hợp có thể xảy ra.

Hiểu điều này có thể đơn giản hóa mã bạn viết. Nếu bạn nghĩ về các đường dẫn và trường hợp, bạn thường có thể loại bỏ mã không cần thiết. Đối với phương thức của chúng ta, nếu chúng ta thực sự muốn chỉ trả về một trong ba giá trị khác nhau, thì chúng

ta không cần kiểm tra thứ ba. Chúng ta có thể làm nhánh cuối cùng của cấu trúc if/else đơn giản thành else:
java
Copy code
```
public static String rating(int totalSAT) {
    if (totalSAT >= 400 && totalSAT < 800) {
        return "not competitive";
    } else if (totalSAT >= 800 && totalSAT < 1200) {
        return "competitive";
    } else { // totalSAT >= 1200
        return "highly competitive";
    }
}
```
Phiên bản này của phương thức biên dịch và trả về chuỗi phù hợp cho mỗi trường hợp khác nhau. Chúng ta đã loại bỏ kiểm tra cuối cùng vì chúng ta biết rằng chúng ta chỉ muốn có ba đường dẫn qua phương thức. Một khi chúng ta đã chỉ định hai đường dẫn, thì mọi thứ còn lại phải là một phần của đường dẫn thứ ba. Chúng ta có thể tiến thêm một bước nữa. Chúng ta đã viết một phương thức biên dịch và tính toán đúng câu trả lời, nhưng chúng ta có thể làm cho nó đơn giản hơn nữa. Hãy xem xét kiểm tra đầu tiên, ví dụ. Tại sao chúng ta phải kiểm tra tổng điểm có lớn hơn hoặc bằng 400 không? Nếu chúng ta kỳ vọng rằng tổng điểm sẽ luôn nằm trong khoảng từ 400 đến 1600, thì chúng ta có thể đơn giản kiểm tra xem tổng điểm có nhỏ hơn 800 không. Tương tự, để kiểm tra cho dải rất cạnh tranh, chúng ta có thể đơn giản kiểm tra xem điểm số có ít nhất là 1200 không. Trong ba dải, đây là hai trường hợp dễ kiểm tra nhất. Vì vậy, chúng ta có thể đơn giản hóa phương thức này hơn nữa bằng cách bao gồm các kiểm tra cho hai dải đầu tiên và giả định rằng tất cả các tổng điểm khác đều nằm trong dải giữa:
java
Copy code

```
public static String rating(int totalSAT) {
    if (totalSAT < 800) {
        return "not competitive";
    } else if (totalSAT >= 1200) {
        return "highly competitive";
    } else {  // 800 <= totalSAT < 1200
        return "competitive";
    }
}
```

Mỗi khi bạn viết một phương thức như thế này, bạn nên suy nghĩ về các trường hợp khác nhau và tìm ra những trường hợp nào dễ kiểm tra nhất. Điều này sẽ giúp bạn tránh viết một kiểm tra rõ ràng cho trường hợp phức tạp nhất. Như trong các ví dụ này, nên bao gồm một nhận xét trên nhánh else cuối cùng để mô tả trường hợp cụ thể đó bằng tiếng Anh.

Trước khi chúng ta kết thúc ví dụ này, đáng để suy nghĩ về điều gì xảy ra khi phương thức được truyền vào một tổng điểm SAT không hợp lệ. Nếu nó được truyền một tổng điểm nhỏ hơn 400, thì nó sẽ phân loại nó là không cạnh tranh, và nếu nó được truyền một tổng điểm lớn hơn 1600, nó sẽ phân loại nó là rất cạnh tranh. Những câu trả lời này không phải là tệ cho chương trình, nhưng điều đúng đắn cần làm là ghi chú rằng có một điều kiện tiên quyết về tổng điểm. Ngoài ra, chúng ta có thể thêm một kiểm tra bổ sung cho trường hợp đặc biệt này và ném ra một ngoại lệ nếu điều kiện tiên quyết bị vi phạm. Kiểm tra các giá trị không hợp lệ là một trường hợp trong đó phép OR logic là phù hợp vì các giá trị không hợp lệ sẽ hoặc quá thấp hoặc quá cao (nhưng không phải cả hai):

java

Copy code

```
// pre: 400 <= totalSAT <= 1600 (ném IllegalArgumentException nếu không hợp lệ)
```

```
public static String rating(int totalSAT) {
    if (totalSAT < 400 || totalSAT > 1600) {
        throw new IllegalArgumentException("total: " + totalSAT);
    } else if (totalSAT < 800) {
        return "not competitive";
    } else if (totalSAT >= 1200) {
        return "highly competitive";
    } else {  // 800 <= totalSAT < 1200
        return "competitive";
    }
}
```

Nghiên cứu trường hợp: Chỉ số khối cơ thể

Chỉ số khối cơ thể cá nhân đã trở thành một thước đo phổ biến về tình trạng sức khỏe tổng quát. Trang web của Trung tâm Kiểm soát và Phòng ngừa Dịch bệnh (CDC) về chỉ số khối cơ thể giải thích: Chỉ số khối cơ thể (BMI) là một con số được tính toán từ cân nặng và chiều cao của một người. BMI cung cấp một chỉ số đáng tin cậy về lượng mỡ cơ thể cho hầu hết mọi người và được sử dụng để sàng lọc các danh mục cân nặng có thể dẫn đến các vấn đề sức khỏe.

Nó cũng trở nên phổ biến để so sánh các số liệu thống kê của hai hoặc nhiều cá nhân đối đầu nhau trong một “thách thức về thể lực”, hoặc so sánh hai bộ số liệu cho cùng một người để hiểu xem BMI của người đó sẽ thay đổi như thế nào nếu người đó giảm cân. Trong phần này, chúng ta sẽ viết một chương trình nhắc người dùng nhập chiều cao và cân nặng của hai người và báo cáo kết quả tổng thể cho hai người đó. Dưới đây là một ví dụ thực thi chương trình mà chúng ta muốn viết:

Chương trình này đọc dữ liệu cho hai người và tính toán chỉ số khối cơ thể của họ và trạng thái cân nặng. Nhập thông tin người tiếp theo:

java

Copy code

height (in inches)?
73.5

weight (in pounds)?
230

Nhập thông tin người tiếp theo:
height (in inches)?
71

weight (in pounds)?
220.5

Person #1 body mass index = 29.93
overweight
Person #2 body mass index = 30.75
obese
Trong Chương 1, chúng ta đã giới thiệu ý tưởng về việc phát triển dần dần, trong đó bạn phát triển một chương trình phức tạp theo từng giai đoạn. Mỗi lập trình viên chuyên nghiệp đều sử dụng kỹ thuật này, vì vậy điều quan trọng là phải học cách áp dụng nó vào các chương trình bạn viết.
Trong trường hợp này, chúng ta cuối cùng muốn chương trình của mình giải thích cho người dùng những gì nó làm và tính toán kết quả BMI cho hai người khác nhau. Chúng ta cũng muốn chương trình được cấu trúc tốt. Nhưng chúng ta không cần phải làm tất cả cùng một lúc. Thực tế, nếu chúng ta cố gắng làm như vậy, chúng ta có thể bị choáng ngợp bởi các chi tiết. Trong khi viết chương trình này, chúng ta sẽ đi qua ba giai đoạn khác nhau:

1. Đầu tiên, chúng ta sẽ viết một chương trình tính toán kết quả cho chỉ một người, mà không có phần giới thiệu. Chúng ta sẽ không lo lắng về cấu trúc chương trình ngay lúc này.

2. Tiếp theo, chúng ta sẽ viết một chương trình hoàn chỉnh tính toán kết quả cho hai người, bao gồm phần giới thiệu. Một lần nữa, chúng ta sẽ không lo lắng về cấu trúc chương trình vào lúc này.
3. Cuối cùng, chúng ta sẽ tập hợp một chương trình hoàn chỉnh và có cấu trúc tốt.

Giải pháp không có cấu trúc cho một người

Ngay cả phiên bản đầu tiên của chương trình cũng sẽ yêu cầu nhập liệu từ người dùng, vì vậy chúng ta cần xây dựng một đối tượng Scanner để đọc từ bàn phím:

java
Copy code

```
Scanner console = new Scanner(System.in);
```

Để tính toán BMI cho một cá nhân, chúng ta cần biết chiều cao và cân nặng của người đó. Đây là một nhiệm vụ "nhắc và đọc" khá đơn giản. Quyết định duy nhất ở đây là loại biến nào cần sử dụng để lưu trữ chiều cao và cân nặng. Mọi người thường nói về chiều cao và cân nặng bằng số nguyên, nhưng câu hỏi cần đặt ra là liệu có hợp lý khi cho phép mọi người sử dụng phân số không. Mọi người có bao giờ mô tả chiều cao của mình bằng nửa inch không? Câu trả lời là có. Mọi người có bao giờ mô tả cân nặng của mình bằng nửa pound không? Một lần nữa câu trả lời là có. Vì vậy, hợp lý khi lưu trữ các giá trị dưới dạng double, để cho phép mọi người nhập cả giá trị nguyên và phân số:

java
Copy code

```
System.out.println("Nhập thông tin người tiếp theo:");
System.out.print("height (in inches)? ");
double height1 = console.nextDouble();
System.out.print("weight (in pounds)? ");
double weight1 = console.nextDouble();
```

Một khi chúng ta có chiều cao và cân nặng của người đó, chúng ta có thể tính toán BMI của họ. Trang web của CDC cung cấp công thức BMI sau đây cho người lớn:
java
Copy code
```
double bmi1 = weight1 / (height1 * height1) * 703;
```
Nếu bạn nhìn kỹ vào ví dụ thực thi, bạn sẽ thấy rằng chúng ta muốn in các dòng trống để tách các phần khác nhau của tương tác với người dùng. Phần giới thiệu kết thúc bằng một dòng trống, sau đó có một dòng trống sau phần “nhắc và đọc”. Vì vậy, sau khi thêm một dòng println trống và đặt tất cả các phần này lại với nhau, phương thức main của chúng ta sẽ trông như sau:
java
Copy code
```
public static void main(String[] args) {
    Scanner console = new Scanner(System.in);
    System.out.println("Nhập thông tin người tiếp theo:");
    System.out.print("height (in inches)? ");
    double height1 = console.nextDouble();
    System.out.print("weight (in pounds)? ");
    double weight1 = console.nextDouble();
    double bmi1 = weight1 / (height1 * height1) * 703;
    System.out.println();
    ...
}
```
Chương trình này nhắc nhập giá trị và tính toán BMI. Bây giờ chúng ta cần thêm mã để báo cáo kết quả. Chúng ta có thể sử dụng println để in chỉ số BMI:
java
Copy code
```
System.out.println("Person #1 body mass index = " + bmi1);
```
Điều này sẽ hoạt động, nhưng nó sẽ tạo ra kết quả như sau:
java

Copy code
Person #1 body mass index = 29.9301217085473 68
Chuỗi số dài sau dấu thập phân gây phân tâm và ngụ ý một mức độ chính xác mà chúng ta không có. Sẽ hợp lý và thu hút người dùng hơn khi chỉ liệt kê một vài chữ số sau dấu thập phân. Đây là nơi tốt để sử dụng printf:
java
Copy code
System.out.printf("Person #1 body mass index = %5.2f\n", bmi1);
Trong ví dụ thực thi, chúng ta cũng thấy báo cáo về trạng thái cân nặng của người đó. Trang web của CDC bao gồm thông tin trong Bảng 4.7. Có bốn mục trong bảng này, vì vậy chúng ta cần bốn câu lệnh println khác nhau cho bốn khả năng. Chúng ta sẽ muốn sử dụng câu lệnh if hoặc if/else để kiểm soát bốn câu lệnh println này. Trong trường hợp này, chúng ta biết rằng chúng ta muốn in chính xác một trong bốn khả năng. Do đó, hợp lý nhất là sử dụng cấu trúc if/else lồng nhau và kết thúc bằng else.

Bảng 4.7: Trạng thái cân nặng theo BMI

BMI	Trạng thái cân nặng
dưới 18.5	thiếu cân
18.5–24.9	bình thường
25.0–29.9	thừa cân
30.0 trở lên	béo phì

Nhưng chúng ta sử dụng kiểm tra nào cho if/else lồng nhau? Nếu bạn nhìn kỹ vào Bảng 4.7, bạn sẽ thấy có một số khoảng trống. Ví dụ, nếu BMI của bạn là 24.95 thì sao? Số đó không nằm trong khoảng từ 18.5 đến 24.9 và cũng không nằm trong khoảng từ 25.0 đến 29.9. Rõ ràng CDC dự định bảng của họ được hiểu theo cách hơi khác. Khoảng này có lẽ được cho là từ 18.5 đến 24.999999 (lặp lại), nhưng điều đó sẽ trông khá lạ trong bảng. Thực tế, nếu bạn hiểu các câu lệnh if/else lồng nhau, đây là một trường hợp trong đó cấu trúc if/else lồng nhau diễn đạt các khả

năng rõ ràng hơn so với bảng như của CDC. Cấu trúc if/else lồng nhau trông như sau:
java
Copy code
```
if (bmi1 < 18.5) {
    System.out.println("underweight");
} else if (bmi1 < 25) {
    System.out.println("normal");
} else if (bmi1 < 30) {
    System.out.println("overweight");
} else {  // bmi1 >= 30
    System.out.println("obese");
}
```
Vậy là, đặt tất cả lại với nhau, chúng ta có một phiên bản hoàn chỉnh của chương trình đầu tiên:
java
Copy code
```
// Chương trình này tính chỉ số khối cơ thể (BMI) cho một người.
import java.util.*;

public class BMI1 {
    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);
        System.out.println("Nhập thông tin người tiếp theo:");
        System.out.print("height (in inches)? ");
        double height1 = console.nextDouble();
        System.out.print("weight (in pounds)? ");
        double weight1 = console.nextDouble();
        double bmi1 = weight1 / (height1 * height1) * 703;
        System.out.println();
        System.out.printf("Person #1 body mass index = %5.2f\n",
bmi1);
        if (bmi1 < 18.5) {
```

```
            System.out.println("underweight");
        } else if (bmi1 < 25) {
            System.out.println("normal");
        } else if (bmi1 < 30) {
            System.out.println("overweight");
        } else {  // bmi1 >= 30
            System.out.println("obese");
        }
    }
}
```

Dưới đây là một ví dụ thực thi của chương trình:

java

Copy code

```
Enter next person's information:
height (in inches)?
73.5

weight (in pounds)?
230

Person #1 body mass index = 29.93
overweight
```

Giải pháp không cấu trúc cho hai người

Bây giờ chúng ta đã có một chương trình tính chỉ số BMI và trạng thái cân nặng của một người, hãy mở rộng nó để xử lý hai người khác nhau. Các lập trình viên có kinh nghiệm có lẽ sẽ bắt đầu bằng cách thêm cấu trúc vào chương trình trước khi cố gắng xử lý hai bộ dữ liệu, nhưng các lập trình viên mới sẽ dễ dàng hơn nếu xem xét giải pháp không cấu trúc trước.

Để làm cho chương trình này xử lý hai người, chúng ta có thể sao chép và dán nhiều đoạn mã và thực hiện các sửa đổi nhỏ. Ví

dụ, thay vì sử dụng các biến gọi là height1, weight1 và bmi1, đối với người thứ hai, chúng ta sẽ sử dụng các biến height2, weight2 và bmi2.
Chúng ta cũng cần cẩn thận để thực hiện từng bước theo đúng thứ tự. Nhìn vào ví dụ thực thi, bạn sẽ thấy rằng chương trình nhắc nhập dữ liệu cho cả hai người trước và sau đó báo cáo kết quả cho cả hai. Do đó, chúng ta không thể chỉ sao chép toàn bộ chương trình và dán thêm một bản sao; chúng ta phải sắp xếp lại thứ tự của các câu lệnh sao cho tất cả các lời nhắc nhập liệu xảy ra trước và tất cả các báo cáo kết quả xảy ra sau.
Chúng ta cũng đã quyết định rằng khi chuyển sang giai đoạn thứ hai này, chúng ta sẽ thêm mã để giới thiệu chương trình. Mã này nên xuất hiện ở đầu chương trình và nên bao gồm một câu lệnh println trống để tạo ra một dòng trống ngăn cách phần giới thiệu với phần còn lại của tương tác với người dùng.
Chúng ta kết hợp các yếu tố này vào một chương trình hoàn chỉnh:
java
Copy code
```
// Chương trình này tính chỉ số khối cơ thể (BMI) cho hai người.
import java.util.*;
public class BMI2 {
    public static void main(String[] args) {
        System.out.println("Chương trình này đọc dữ liệu cho hai
người và tính chỉ số BMI cũng như trạng thái cân nặng của họ.");
        System.out.println();
        Scanner console = new Scanner(System.in);
        System.out.println("Nhập thông tin người thứ nhất:");
        System.out.print("Chiều cao (in inches)? ");
        double height1 = console.nextDouble();
        System.out.print("Cân nặng (in pounds)? ");
        double weight1 = console.nextDouble();
        double bmi1 = weight1 / (height1 * height1) * 703;
```

```
        System.out.println();
        System.out.println("Nhập thông tin người thứ hai:");
        System.out.print("Chiều cao (in inches)? ");
        double height2 = console.nextDouble();
        System.out.print("Cân nặng (in pounds)? ");
        double weight2 = console.nextDouble();
        double bmi2 = weight2 / (height2 * height2) * 703;
        System.out.println();
        System.out.printf("Chỉ số BMI của người thứ nhất =
%5.2f\n", bmi1);
        if (bmi1 < 18.5) {
            System.out.println("Thiếu cân");
        } else if (bmi1 < 25) {
            System.out.println("Bình thường");
        } else if (bmi1 < 30) {
            System.out.println("Thừa cân");
        } else {
            System.out.println("Béo phì");
        }
        System.out.printf("Chỉ số BMI của người thứ hai = %5.2f\n",
bmi2);
        if (bmi2 < 18.5) {
            System.out.println("Thiếu cân");
        } else if (bmi2 < 25) {
            System.out.println("Bình thường");
        } else if (bmi2 < 30) {
            System.out.println("Thừa cân");
        } else {
            System.out.println("Béo phì");
        }
    }
}
```

Chương trình này biên dịch và hoạt động. Khi chúng ta thực thi nó, chúng ta có được chính xác tương tác mà mình mong muốn. Tuy nhiên, chương trình thiếu cấu trúc. Tất cả mã đều xuất hiện trong phương thức main, và có sự dư thừa đáng kể. Điều này không nên là một điều bất ngờ, vì chúng ta đã tạo ra phiên bản này bằng cách sao chép và dán. Mỗi khi bạn thấy mình sử dụng sao chép và dán, bạn nên tự hỏi liệu có một cách tốt hơn để giải quyết vấn đề hay không. Thường thì có một cách tốt hơn.

Giải pháp có cấu trúc cho hai người

Hãy khám phá cách phương thức tĩnh (static methods) có thể cải thiện cấu trúc của chương trình. Khi xem xét mã, bạn sẽ nhận thấy có rất nhiều sự dư thừa. Ví dụ, chúng ta có hai đoạn mã trông giống như sau:

java

Copy code

```
System.out.println("Nhập thông tin của người tiếp theo:");

System.out.print("Chiều cao (in inches)? ");

double height1 = console.nextDouble();

System.out.print("Cân nặng (in pounds)? ");

double weight1 = console.nextDouble();

double bmi1 = weight1 / (height1 * height1) * 703;

System.out.println();
```

Điểm khác biệt duy nhất giữa hai đoạn mã này là đoạn đầu sử dụng các biến height1, weight1, bmi1, và đoạn thứ hai sử dụng các biến height2, weight2, và bmi2. Chúng ta có thể loại bỏ sự dư thừa bằng cách chuyển đoạn mã này vào một phương thức mà

chúng ta có thể gọi hai lần. Vì vậy, bước đầu tiên, chúng ta có thể biến đoạn mã này thành một dạng chung hơn như phương thức sau:
java
Copy code
```
public static void getBMI(Scanner console) {

    System.out.println("Nhập thông tin của người tiếp theo:");

    System.out.print("Chiều cao (in inches)? ");

    double height = console.nextDouble();

    System.out.print("Cân nặng (in pounds)? ");

    double weight = console.nextDouble();

    double bmi = weight / (height * height) * 703;

    System.out.println();
}
```
Chúng ta phải truyền vào Scanner từ phương thức main. Ngoài ra, chúng ta đã làm cho tất cả các biến trở thành cục bộ trong phương thức này. Từ main, chúng ta có thể gọi phương thức này hai lần:
java
Copy code
```
getBMI(console);

getBMI(console);
```
Tuy nhiên, việc giới thiệu thay đổi này sẽ làm hỏng phần còn lại của mã. Nếu chúng ta cố biên dịch và chạy chương trình, chúng

ta sẽ nhận được thông báo lỗi trong main bất cứ khi nào tham chiếu đến các biến bmi1 và bmi2.
Vấn đề là phương thức tính giá trị BMI mà chúng ta cần sử dụng sau này trong chương trình. Chúng ta có thể khắc phục điều này bằng cách cho phép phương thức trả về giá trị BMI mà nó tính toán:
java
Copy code

```
public static double getBMI(Scanner console) {

    System.out.println("Nhập thông tin của người tiếp theo:");

    System.out.print("Chiều cao (in inches)? ");

    double height = console.nextDouble();

    System.out.print("Cân nặng (in pounds)? ");

    double weight = console.nextDouble();

    double bmi = weight / (height * height) * 703;

    System.out.println();

    return bmi;
}
```

Lưu ý rằng tiêu đề phương thức giờ đây liệt kê kiểu trả về là double. Chúng ta cũng phải thay đổi main. Chúng ta không thể chỉ gọi phương thức hai lần như cách chúng ta gọi một phương thức void. Bởi vì mỗi lần gọi trả về một kết quả BMI mà chương trình sẽ cần sau này, chúng ta phải lưu trữ kết quả trả về từ phương thức trong một biến cho mỗi lần gọi:
java

Copy code
```
double bmi1 = getBMI(console);

double bmi2 = getBMI(console);
```
Hãy nghiên cứu kỹ thay đổi này, vì kỹ thuật này có thể là một trong những thử thách khó khăn nhất đối với những người mới học. Khi chúng ta viết phương thức, chúng ta phải đảm bảo rằng nó trả về kết quả BMI. Khi chúng ta viết lệnh gọi, chúng ta phải đảm bảo rằng chúng ta lưu trữ kết quả trong một biến để có thể truy cập sau này.
Sau khi thực hiện sửa đổi này, chương trình sẽ biên dịch và chạy đúng cách. Nhưng còn một sự dư thừa rõ ràng khác trong phương thức main: Cấu trúc if/else lồng nhau xuất hiện hai lần. Sự khác biệt duy nhất giữa chúng là trong một trường hợp chúng ta sử dụng biến bmi1, và trong trường hợp khác chúng ta sử dụng biến bmi2. Cấu trúc này có thể dễ dàng được tổng quát hóa bằng một tham số:
java
Copy code
```
public static void reportStatus(double bmi) {

    if (bmi < 18.5) {

        System.out.println("Thiếu cân");

    } else if (bmi < 25) {

        System.out.println("Bình thường");

    } else if (bmi < 30) {

        System.out.println("Thừa cân");
```

```
    } else {  // bmi >= 30

        System.out.println("Béo phì");

    }
}
```

Sử dụng phương thức này, chúng ta có thể thay thế mã trong main bằng hai lệnh gọi:

java
Copy code

```
System.out.printf("Chỉ số BMI của người thứ nhất = %5.2f\n", bmi1);

reportStatus(bmi1);

System.out.printf("Chỉ số BMI của người thứ hai = %5.2f\n", bmi2);

reportStatus(bmi2);
```

Thay đổi đó loại bỏ sự dư thừa trong chương trình, nhưng chúng ta vẫn có thể sử dụng các phương thức tĩnh để cải thiện chương trình bằng cách chỉ rõ cấu trúc rõ ràng hơn. Tốt nhất là giữ cho phương thức main ngắn gọn nếu có thể, để phản ánh cấu trúc tổng thể của chương trình. Vấn đề chia thành ba giai đoạn chính: phần giới thiệu, tính toán BMI, và báo cáo kết quả. Chúng ta đã có một phương thức để tính toán BMI, nhưng chúng ta chưa giới thiệu các phương thức cho phần giới thiệu và báo cáo kết quả. Việc thêm các phương thức này khá đơn giản.

Còn một phương thức khác mà chúng ta nên thêm vào chương trình. Chúng ta đang sử dụng công thức từ trang web CDC để tính toán BMI của một cá nhân dựa trên chiều cao và cân nặng của người đó. Bất cứ khi nào bạn lập trình một công thức, sẽ là

một ý tưởng tốt để giới thiệu một phương thức cho công thức đó để dễ dàng phát hiện và để nó có một tên gọi.
Áp dụng tất cả các ý tưởng này, chúng ta sẽ có phiên bản chương trình như sau:
java
Copy code

```
// Chương trình này tính chỉ số khối cơ thể (BMI) cho hai người.
// Phiên bản này bao gồm nhiều phương thức khác ngoài phương thức main.
import java.util.*;

public class BMI3 {

    public static void main(String[] args) {
        giveIntro();
        Scanner console = new Scanner(System.in);
        double bmi1 = getBMI(console);
        double bmi2 = getBMI(console);
        reportResults(bmi1, bmi2);
    }

    // Giới thiệu chương trình tới người dùng
    public static void giveIntro() {
        System.out.println("Chương trình này đọc dữ liệu cho hai người và tính chỉ số BMI cũng như trạng thái cân nặng của họ.");
        System.out.println();
    }

    // Nhập số liệu của một người và trả về chỉ số BMI
    public static double getBMI(Scanner console) {
        System.out.println("Nhập thông tin của người tiếp theo:");
        System.out.print("Chiều cao (in inches)? ");
        double height = console.nextDouble();
```

```
        System.out.print("Cân nặng (in pounds)? ");
        double weight = console.nextDouble();
        double bmi = BMIFor(height, weight);
        System.out.println();
        return bmi;
    }

    // Phương thức này chứa công thức BMI để chuyển đổi chiều
cao (inches) và cân nặng (pounds) thành BMI
    public static double BMIFor(double height, double weight) {
        return weight / (height * height) * 703;
    }

    // Báo cáo các giá trị BMI và trạng thái cân nặng
    public static void reportResults(double bmi1, double bmi2) {
        System.out.printf("Chỉ số BMI của người thứ nhất =
%5.2f\n", bmi1);
        reportStatus(bmi1);
        System.out.printf("Chỉ số BMI của người thứ hai = %5.2f\n",
bmi2);
        reportStatus(bmi2);
    }

    // Báo cáo trạng thái cân nặng dựa trên giá trị BMI
    public static void reportStatus(double bmi) {
        if (bmi < 18.5) {
            System.out.println("Thiếu cân");
        } else if (bmi < 25) {
            System.out.println("Bình thường");
        } else if (bmi < 30) {
            System.out.println("Thừa cân");
        } else {  // bmi >= 30
            System.out.println("Béo phì");
```

```
      }
   }
}
```
Giải pháp này tương tác với người dùng theo cùng một cách và tạo ra kết quả giống như giải pháp không có cấu trúc, nhưng nó có một cấu trúc rõ ràng hơn. Chương trình không có cấu trúc có vẻ đơn giản hơn, nhưng giải pháp có cấu trúc dễ bảo trì hơn nếu chúng ta muốn mở rộng chương trình hoặc thực hiện các sửa đổi khác. Những lợi ích của cấu trúc không quá quan trọng trong các chương trình ngắn, nhưng chúng trở nên cần thiết khi các chương trình trở nên dài hơn và phức tạp hơn.

Nguyên tắc thiết kế thủ tục

Có rất nhiều cách để chia nhỏ (phân rã) một vấn đề thành các phương thức, nhưng có một số tập hợp phương thức tốt hơn các tập hợp khác. Phân rã thường mơ hồ và đầy thách thức, đặc biệt là đối với các chương trình lớn có hành vi phức tạp. Tuy nhiên, phần thưởng sẽ xứng đáng với công sức, vì một chương trình được thiết kế tốt sẽ dễ hiểu hơn và có tính mô-đun cao hơn. Những đặc điểm này rất quan trọng khi các lập trình viên làm việc cùng nhau hoặc khi cần xem lại một chương trình đã viết trước đó để thêm hành vi mới hoặc sửa đổi mã hiện có. Không có thiết kế nào là hoàn hảo, nhưng trong phần này, chúng ta sẽ thảo luận về một số nguyên tắc chỉ đạo (heuristics) để phân rã hiệu quả các chương trình lớn thành các phương thức.

Hãy xem xét một cách triển khai thay thế, có cấu trúc kém của chương trình BMI dành cho một người. Chúng ta sẽ sử dụng chương trình này như một ví dụ phản chứng, làm nổi bật các phần mà nó vi phạm các nguyên tắc của chúng ta và đưa ra lý do tại sao nó tệ hơn so với phiên bản hoàn chỉnh trước đó của chương trình BMI.

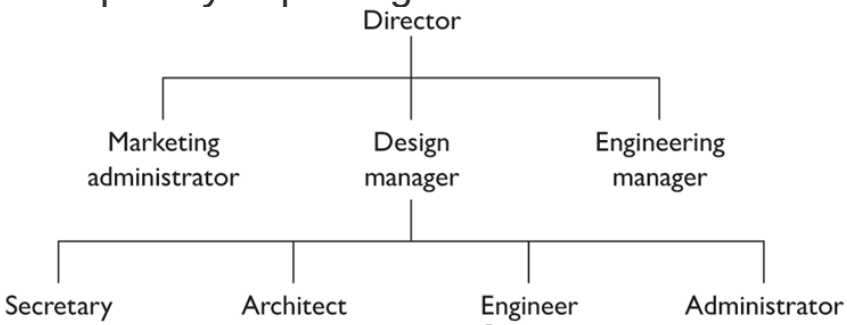
java

Copy code

```
1  // Một phiên bản được thiết kế kém của chương trình nghiên cứu trường hợp BMI.
2
3  import java.util.*;
4
5  public class BadBMI {
6      public static void main(String[] args) {
7          System.out.println("Chương trình này đọc dữ liệu cho một người");
8          System.out.println("và tính chỉ số khối cơ thể và trạng thái cân nặng của họ.");
9          System.out.println();
10         Scanner console = new Scanner(System.in);
11         person(console);
12     }
13
14     public static void person(Scanner console) {
15         System.out.println("Nhập thông tin của người tiếp theo:");
16         System.out.print("Chiều cao (in inches)? ");
17         double height = console.nextDouble();
18         getWeight(console, height);
19     }
20
21     public static void getWeight(Scanner console, double height) {
22         System.out.print("Cân nặng (in pounds)? ");
23         double weight = console.nextDouble();
24         reportStatus(console, height, weight);
25     }
26
27     public static void reportStatus(Scanner console, double height, double weight) {
```

```
28          double bmi = weight / (height * height) * 703;
29          System.out.println("Chỉ số BMI của người thứ nhất = " +
bmi);
30          if (bmi < 18.5) {
31              System.out.println("Thiếu cân");
32          } else if (bmi < 25) {
33              System.out.println("Bình thường");
34          } else if (bmi < 30) {
35              System.out.println("Thừa cân");
36          } else {
37              System.out.println("Béo phì");
38          }
39      }
40 }
```

Các phương thức của một chương trình giống như các nhân viên trong một công ty. Tác giả của một chương trình hoạt động như một giám đốc công ty, quyết định tạo ra những vị trí nhân viên nào, cách nhóm các nhân viên lại với nhau thành các đơn vị làm việc, giao công việc nào cho nhóm nào và cách các nhóm tương tác với nhau. Giả sử một giám đốc công ty chia công việc thành ba bộ phận chính, trong đó hai bộ phận được giám sát bởi các nhà quản lý cấp trung.



4.5-16 Toàn văn thay thế

Một cấu trúc tốt sẽ phân công rõ ràng nhiệm vụ cho từng nhóm, tránh để bất kỳ cá nhân hoặc nhóm nào phải làm quá nhiều công việc, và duy trì cân bằng giữa người lao động và quản lý. Những

hướng dẫn này dẫn đến nguyên tắc thiết kế thủ tục đầu tiên của chúng ta.

1. Mỗi phương thức nên có một tập hợp trách nhiệm mạch lạc. Trong ví dụ tương tự với một công ty, mỗi nhóm nhân viên phải có một ý tưởng rõ ràng về công việc mà nó cần thực hiện. Nếu bất kỳ nhóm nào không có trách nhiệm rõ ràng, giám đốc công ty sẽ khó theo dõi được ai đang làm công việc gì. Khi một công việc mới xuất hiện, hai bộ phận có thể cùng cố gắng giành lấy nó, hoặc một công việc có thể bị bỏ qua mà không có bộ phận nào đảm nhiệm.

Khái niệm tương tự trong lập trình là mỗi phương thức nên có một mục đích rõ ràng và một tập hợp trách nhiệm. Đặc điểm này của chương trình máy tính được gọi là tính liên kết.

Liên kết: Một đặc điểm mong muốn mà các trách nhiệm của một phương thức hoặc quy trình có mối liên hệ chặt chẽ với nhau.

Một quy tắc chung là bạn nên có thể tóm tắt mỗi phương thức của mình trong một câu duy nhất, chẳng hạn như “Mục đích của phương thức này là…”. Viết một câu như vậy là cách tốt để phát triển một bình luận cho phần đầu của phương thức. Đó là một dấu hiệu xấu khi bạn gặp khó khăn trong việc mô tả phương thức trong một câu hoặc khi câu đó dài và sử dụng từ “và” nhiều lần. Những dấu hiệu này có thể có nghĩa là phương thức quá lớn, quá nhỏ, hoặc không thực hiện một tập hợp công việc gắn kết.

Các phương thức của ví dụ BadBMI có tính liên kết kém. Mục đích của phương thức person không rõ ràng và getWeight có lẽ quá đơn giản để trở thành một phương thức riêng. Phương thức reportStatus sẽ dễ đọc hơn nếu việc tính toán BMI là một phương thức riêng, vì công thức này khá phức tạp.

Một ứng dụng tinh tế hơn của nguyên tắc đầu tiên này là không phải mọi phương thức đều phải tạo ra đầu ra. Đôi khi một phương thức có thể tái sử dụng hơn nếu nó chỉ đơn giản tính toán một kết quả phức tạp và trả về nó thay vì in ra kết quả đã tính toán. Định dạng này để lại sự lựa chọn cho người gọi là in kết quả hoặc sử dụng nó để thực hiện các tính toán tiếp theo. Trong chương trình BadBMI, phương thức reportStatus vừa tính toán vừa in ra BMI của người dùng. Chương trình sẽ linh hoạt hơn nếu có một phương thức chỉ đơn giản tính toán và trả về giá trị BMI, như BMIFor trong phiên bản BMI3 của mã. Một phương thức như vậy có thể có vẻ đơn giản vì thân của nó chỉ là một dòng, nhưng nó có một mục đích rõ ràng và gắn kết: bắt giữ một biểu thức phức tạp được sử dụng nhiều lần trong chương trình.

2. Không nên có một phương thức nào làm quá nhiều phần việc của toàn bộ nhiệm vụ. Một phân chia của công ty không thể được kỳ vọng sẽ thiết kế và xây dựng toàn bộ dòng sản phẩm cho cả năm. Hệ thống này sẽ làm quá tải phân chia đó và sẽ khiến các bộ phận khác không có đủ công việc để làm. Nó cũng sẽ làm khó khăn cho việc giao tiếp giữa các bộ phận, vì quá nhiều thông tin quan trọng và trách nhiệm sẽ tập trung vào quá ít người.

Tương tự, một phương thức không nên chiếm phần lớn chương trình. Nguyên tắc này tự nhiên theo sau nguyên tắc đầu tiên về tính liên kết, vì một phương thức làm quá nhiều việc không thể có tính liên kết. Chúng ta đôi khi gọi những phương thức như vậy là phương thức “làm mọi thứ” vì chúng làm gần như tất cả mọi thứ liên quan đến việc giải quyết vấn đề. Bạn có thể đã viết một phương thức “làm mọi thứ” nếu một trong những phương thức của bạn dài hơn nhiều so với các phương thức khác, tích trữ hầu hết các biến và dữ liệu, hoặc chứa phần lớn logic và vòng lặp.

Trong chương trình BadBMI, phương thức person là một ví dụ về phương thức làm mọi thứ. Thực tế này có thể khiến bạn ngạc nhiên, vì phương thức này không quá dài. Nhưng một lệnh gọi tới person dẫn đến nhiều lệnh gọi khác mà cuối cùng thực hiện tất cả các công việc cho chương trình.

3. Tối thiểu hóa sự phụ thuộc và gắn kết giữa các phương thức. Một công ty sẽ năng suất hơn nếu mỗi phân chia của nó có thể hoạt động độc lập phần lớn khi hoàn thành các nhiệm vụ nhỏ. Các phân chia của công ty cần phải giao tiếp và phụ thuộc lẫn nhau, nhưng sự giao tiếp đó đi kèm với chi phí. Tương tác liên phòng ban thường được giảm thiểu và giữ ở các cuộc họp vào những thời điểm và địa điểm cụ thể.

Khi lập trình, chúng ta cố gắng tránh các phương thức có sự gắn kết chặt chẽ.

Gắn kết: Một trạng thái không mong muốn trong đó hai phương thức hoặc quy trình phụ thuộc chặt chẽ vào nhau.

Các phương thức có sự gắn kết nếu không thể dễ dàng gọi một phương thức mà không gọi phương thức kia. Một cách để xác định mức độ gắn kết giữa hai phương thức là xem xét tập hợp các tham số mà một phương thức truyền cho phương thức kia. Một phương thức chỉ nên chấp nhận một tham số nếu dữ liệu đó cần được cung cấp từ bên ngoài và chỉ khi dữ liệu đó cần thiết để hoàn thành nhiệm vụ của phương thức. Nói cách khác, nếu một dữ liệu có thể được tính toán hoặc thu thập bên trong phương thức, hoặc nếu dữ liệu đó không được sử dụng bởi phương thức, thì không nên khai báo nó như một tham số của phương thức.

Một cách quan trọng để giảm sự gắn kết giữa các phương thức là sử dụng câu lệnh return để gửi thông tin trở lại người gọi. Một phương thức nên trả về giá trị kết quả nếu nó tính toán một cái gì

đó có thể hữu ích cho các phần sau của chương trình. Vì mong muốn có phương thức gắn kết và tự chứa, nên thường tốt hơn cho chương trình trả về kết quả thay vì gọi thêm các phương thức khác và truyền kết quả dưới dạng tham số cho chúng.

Không có phương thức nào trong chương trình BadBMI trả về giá trị. Mỗi phương thức truyền tham số cho các phương thức tiếp theo, nhưng không có phương thức nào trả về giá trị. Đây là một cơ hội bị mất vì một số giá trị (như chiều cao, cân nặng, hoặc BMI của người dùng) sẽ được xử lý tốt hơn dưới dạng giá trị trả về.

4. Phương thức chính nên là một bản tóm tắt ngắn gọn của toàn bộ chương trình. Người đứng đầu mỗi nhóm lớn hoặc bộ phận của công ty giả định của chúng ta sẽ báo cáo với giám đốc nhóm. Nếu bạn nhìn vào các nhóm kết nối trực tiếp với giám đốc ở cấp cao nhất của sơ đồ công ty, bạn có thể thấy một bản tóm tắt về công việc tổng thể: thiết kế, kỹ thuật, và tiếp thị. Cấu trúc này giúp giám đốc giữ nhận thức về những gì mỗi nhóm đang làm. Nhìn vào cấu trúc cấp cao nhất cũng có thể giúp nhân viên có cái nhìn tổng quan về các mục tiêu của công ty.

Phương thức chính của một chương trình giống như giám đốc ở chỗ nó bắt đầu nhiệm vụ tổng thể và thực hiện các công việc phụ. Phương thức chính nên đọc như một bản tóm tắt về hành vi tổng thể của chương trình. Các lập trình viên có thể hiểu mã của nhau bằng cách xem phương thức chính để có cảm nhận về những gì chương trình đang làm.

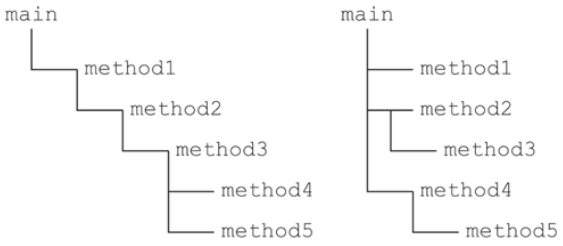
Một lỗi phổ biến ngăn cản phương thức chính trở thành bản tóm tắt tốt của chương trình là sự tồn tại của một phương thức “làm mọi thứ”. Khi phương thức chính gọi nó, phương thức “làm mọi thứ” sẽ thực hiện hầu hết hoặc toàn bộ công việc thực sự.

Một lỗi khác là thiết lập chương trình theo cách mà nó mắc phải tình trạng “chaining”.

Chaining: Một thiết kế không mong muốn trong đó một chuỗi các phương thức gọi lẫn nhau mà không trả lại luồng điều khiển tổng thể về phương thức chính.

Chương trình mắc phải tình trạng chaining nếu phần cuối của mỗi phương thức chỉ đơn giản gọi phương thức tiếp theo. Chaining thường xảy ra khi một lập trình viên mới không hiểu đầy đủ về câu lệnh return và cố gắng tránh sử dụng chúng bằng cách truyền nhiều tham số xuống cho phần còn lại của chương trình. Hình 4.8 cho thấy một chương trình giả định với hai thiết kế. Luồng gọi trong một chương trình bị chaining tồi tệ có thể giống như sơ đồ bên trái.

Hình 4.8: Mã mẫu với chaining (bên trái) và không chaining (bên phải).



Hình 4.8 Toàn văn thay thế

Chương trình BadBMI gặp phải vấn đề lớn về chaining. Mỗi phương thức thực hiện một lượng công việc nhỏ và sau đó gọi phương thức tiếp theo, truyền thêm các tham số xuống chuỗi. Phương thức chính (main) gọi person, person gọi getWeight, getWeight gọi reportStatus. Không bao giờ luồng thực thi quay trở lại main giữa chừng tính toán. Do đó, khi bạn đọc main, bạn

không có một ý tưởng rõ ràng về những tính toán sẽ được thực hiện.

Một phương thức không nên gọi phương thức khác chỉ đơn giản để chuyển sang công việc tiếp theo. Một luồng điều khiển mong muốn hơn là để main quản lý việc thực hiện tổng thể các nhiệm vụ trong chương trình, như được chỉ ra trong chương trình BMI3 và ở phía bên phải của Hình 4.8. Hướng dẫn này không có nghĩa là luôn xấu khi một phương thức gọi phương thức khác; điều này chấp nhận được khi một phương thức gọi phương thức khác khi phương thức thứ hai là một nhiệm vụ phụ trong nhiệm vụ tổng thể của phương thức thứ nhất, chẳng hạn như trong BMI3 khi phương thức reportResults gọi reportStatus.

1. Dữ liệu nên được "sở hữu" ở mức thấp nhất có thể. Các quyết định trong một công ty nên được đưa ra ở cấp thấp nhất trong hệ thống tổ chức. Ví dụ, một quản trị viên cấp thấp có thể tự quyết định cách thực hiện công việc của mình mà không cần phải liên tục hỏi ý kiến quản lý. Nhưng quản trị viên này không đủ thông tin hoặc chuyên môn để thiết kế toàn bộ dòng sản phẩm; nhiệm vụ thiết kế này sẽ được giao cho cấp cao hơn như quản lý. Nguyên tắc chính là mỗi nhiệm vụ công việc nên được giao cho người có cấp thấp nhất trong hệ thống tổ chức có thể xử lý chính xác nó.

Nguyên tắc này có hai ứng dụng trong chương trình máy tính. Đầu tiên là phương thức main nên tránh thực hiện các nhiệm vụ cấp thấp càng nhiều càng tốt. Ví dụ, trong một chương trình tương tác, main không nên đọc phần lớn dữ liệu đầu vào của người dùng hoặc chứa nhiều câu lệnh println.

Ứng dụng thứ hai là các biến nên được khai báo và khởi tạo trong phạm vi hẹp nhất có thể. Một thiết kế kém là để main (hoặc một phương thức cấp cao khác) đọc tất cả dữ liệu đầu vào, thực

hiện các tính toán nặng nề, sau đó truyền dữ liệu kết quả dưới dạng tham số cho các phương thức cấp thấp khác nhau. Một thiết kế tốt hơn là sử dụng các phương thức cấp thấp để đọc và xử lý dữ liệu, và chỉ trả lại dữ liệu cho main nếu chúng cần thiết cho một nhiệm vụ phụ sau này trong chương trình.

Đó là dấu hiệu của quyền sở hữu dữ liệu kém khi cùng một tham số phải được truyền qua nhiều lần gọi phương thức, chẳng hạn như biến height trong chương trình BadBMI. Nếu bạn đang truyền cùng một tham số qua nhiều cấp độ gọi, có lẽ dữ liệu đó nên được đọc và khởi tạo bởi một trong các phương thức cấp thấp hơn (trừ khi đó là một đối tượng dùng chung như Scanner).

Tóm tắt Chương

Một câu lệnh if cho phép bạn viết mã sẽ chỉ thực thi nếu một điều kiện nhất định được thỏa mãn. Một câu lệnh if/else cho phép bạn thực thi một đoạn mã nếu điều kiện được thỏa mãn, và một đoạn mã khác nếu điều kiện không được thỏa mãn. Các điều kiện là các biểu thức Boolean và có thể được viết bằng các toán tử quan hệ như <, >=, và !=. Bạn có thể kiểm tra nhiều điều kiện bằng cách sử dụng các toán tử && và ||.

Bạn có thể lồng các câu lệnh if/else để kiểm tra một loạt các điều kiện và thực thi đoạn mã phù hợp dựa trên điều kiện nào là đúng.

Toán tử == kiểm tra sự bằng nhau của dữ liệu nguyên thủy không hoạt động như mong đợi với các đối tượng, vì vậy chúng ta kiểm tra sự bằng nhau của các đối tượng bằng cách gọi phương thức equals của chúng.

Mã chung xuất hiện trong mọi nhánh của một câu lệnh if/else nên được tách ra để không bị lặp lại nhiều lần trong mã.

Các thuật toán cộng dồn tính toán các giá trị một cách liên tiếp. Một vòng lặp cộng dồn khai báo một biến sum và thêm dần giá trị vào biến đó trong vòng lặp.

Vì kiểu double không lưu trữ chính xác tất cả các giá trị, nên các lỗi làm tròn nhỏ có thể xảy ra khi máy tính thực hiện các phép tính trên các số thực. Tránh những lỗi này bằng cách cung cấp một khoảng sai số nhỏ trong mã của bạn cho các giá trị gần với các giá trị mà bạn mong đợi.

Kiểu char đại diện cho các ký tự riêng lẻ của văn bản. Mỗi chữ cái của một chuỗi (String) được lưu trữ nội bộ dưới dạng giá trị char, và bạn có thể sử dụng phương thức charAt của chuỗi để truy cập các ký tự này bằng một chỉ số.

Phương thức System.out.printf in văn bản định dạng. Bạn có thể chỉ định các chuỗi định dạng phức tạp để kiểm soát độ rộng, căn chỉnh, và độ chính xác của việc in các giá trị.

Bạn có thể "ném" (phát sinh) ngoại lệ trong mã của riêng bạn. Kỹ thuật này có thể hữu ích nếu mã của bạn gặp phải một điều kiện lỗi không thể khắc phục được, chẳng hạn như việc truyền một giá trị đối số không hợp lệ cho một phương thức.

Tự Kiểm Tra Các Vấn Đề

Mục 4.1: Câu lệnh if/else

1. Dịch mỗi câu tiếng Anh sau thành các kiểm tra logic có thể được sử dụng trong câu lệnh if/else. Viết câu lệnh if phù hợp với kiểm tra logic của bạn. Giả sử rằng ba biến kiểu int là x, y, và z đã được khai báo. a. z là số lẻ. b. z không lớn hơn

căn bậc hai của y. c. y là số dương. d. Hoặc x hoặc y là số chẵn, và cái kia là số lẻ. e. y là bội số của z. f. z không bằng không. g. y lớn hơn z về độ lớn. h. x và z có dấu ngược nhau. i. y là số không âm và có một chữ số. j. z là số không âm. k. x là số chẵn. l. x gần giá trị của y hơn z.

2. Với các khai báo biến:

java

Copy code

int x = 4;

int y = –3;

int z = 4;

Kết quả của các biểu thức quan hệ sau là gì? a. x == 4 b. x == y c. x == z d. y == z e. x + y > 0 f. x – z != 0 g. y * y <= z h. y / y == 1 i. x * (y + 2) > y - (y + z) * 2

3. Câu tiêu đề nào của câu lệnh if sau đây sử dụng cú pháp đúng? a. if x = 10 then { b. if [x == 10] { c. if (x => y) { d. if (x equals 42) { e. if (x == y) {

4. Chương trình sau đây chứa 7 lỗi! Chúng là gì?

java

Copy code

public class Oops4 {

```
    public static void main(String[] args) {
        int a = 7, b = 42;
        minimum(a, b);
        if {smaller = a} {
            System.out.println("a is the smallest!");
        }
    }

    public static void minimum(int a, int b) {
        if (a < b) {
            int smaller = a;
        } else (a => b) {
            int smaller = b;
        }
        return int smaller;
    }
}
```

5. Xem xét phương thức sau:

java

Copy code

```
public static void ifElseMystery1(int x, int y) {
    int z = 4;
    if (z <= x) {
        z = x + 1;
    } else {
        z = z + 9;
    }
    if (z <= y) {
        y++;
    }
    System.out.println(z + " " + y);
}
```

Kết quả đầu ra là gì cho từng lệnh gọi sau? a. ifElseMystery1(3, 20); b. ifElseMystery1(4, 5); c. ifElseMystery1(5, 5); d. ifElseMystery1(6, 10);

6. Xem xét phương thức sau:

java

Copy code

```
public static void ifElseMystery2(int a, int b) {
    if (a * 2 < b) {
        a = a * 3;
    } else if (a > b) {
        b = b + 3;
    }
    if (b < a) {
        b++;
    } else {
        a—;
    }
    System.out.println(a + " " + b);
}
```

Kết quả đầu ra là gì cho từng lệnh gọi sau? a. ifElseMystery2(10, 2); b. ifElseMystery2(3, 8); c. ifElseMystery2(4, 4); d. ifElseMystery2(10, 30);

7. Viết mã Java để đọc một số nguyên từ người dùng, sau đó in "even" nếu số đó là số chẵn hoặc "odd" nếu ngược lại. Bạn có thể giả định rằng người dùng nhập vào một số nguyên hợp lệ.

8. Mã sau chứa lỗi logic:

java

Copy code

```
Scanner console = new Scanner(System.in);

System.out.print("Type a number: ");

int number = console.nextInt();

if (number % 2 == 0) {

    if (number % 3 == 0) {

        System.out.println("Divisible by 6.");

    } else {

        System.out.println("Odd.");

    }

}
```

Kiểm tra mã và mô tả một trường hợp mà mã sẽ in ra điều gì đó không đúng về số đã nhập. Giải thích lý do và sau đó sửa lỗi logic trong mã.

9. Mô tả vấn đề với đoạn mã sau:

java

Copy code

```
Scanner console = new Scanner(System.in);

System.out.print("What is your favorite color?");

String name = console.next();

if (name == "blue") {

    System.out.println("Mine, too!");

}
```

10. Tách mã lặp ra khỏi ví dụ sau bằng cách chuyển nó ra khỏi câu lệnh if/else, vẫn giữ nguyên kết quả đầu ra.

java

Copy code

```
if (x < 30) {

    a = 2;

    x++;
```

```
    System.out.println("Java is awesome! " + x);
} else {
    a = 2;
    System.out.println("Java is awesome! " + x);
}
```

11. Mã sau được cấu trúc kém:

```
java
Copy code
int sum = 1000;
Scanner console = new Scanner(System.in);
System.out.print("Is your money multiplied 1 or 2 times? ");
int times = console.nextInt();
if (times == 1) {
    System.out.print("And how much are you contributing? ");
    int donation = console.nextInt();
    sum = sum + donation;
    count1++;
```

```
    total = total + donation;
}
if (times == 2) {
    System.out.print("And how much are you contributing? ");
    int donation = console.nextInt();
    sum = sum + 2 * donation;
    count2++;
    total = total + donation;
}
```

Viết lại mã để nó có cấu trúc tốt hơn và tránh sự lặp lại. Để đơn giản hóa, bạn có thể giả định rằng người dùng luôn nhập 1 hoặc 2. (Mã cần được chỉnh sửa như thế nào để xử lý bất kỳ số nào mà người dùng có thể nhập?)

12. Mã sau được cấu trúc kém:

java

Copy code

```
Scanner console = new Scanner(System.in);
System.out.print("How much will John be spending? ");
double amount = console.nextDouble();
```

```
System.out.println();

int numBills1 = (int) (amount / 20.0);

if (numBills1 * 20.0 < amount) {

    numBills1++;

}

System.out.print("How much will Jane be spending? ");

amount = console.nextDouble();

System.out.println();

int numBills2 = (int) (amount / 20.0);

if (numBills2 * 20.0 < amount) {

    numBills2++;

}

System.out.println("John needs " + numBills1 + " bills");

System.out.println("Jane needs " + numBills2 + " bills");
```

Viết lại mã để nó có cấu trúc tốt hơn và tránh sự lặp lại. Bạn có thể muốn giới thiệu một phương thức để giúp nắm bắt mã lặp.

13. Viết một đoạn mã để đọc mô tả màu ngắn gọn từ người dùng và in ra phiên bản dài hơn tương đương. Các tên màu chấp nhận được là B cho Blue, G cho Green, và R cho Red.

Nếu người dùng nhập một thứ khác ngoài B, G, hoặc R, chương trình sẽ in ra thông báo lỗi. Làm cho chương trình của bạn không phân biệt chữ hoa chữ thường để người dùng có thể nhập một chữ cái hoa hoặc thường. Dưới đây là một số ví dụ thực hiện:

css

Copy code

What color do you want? B

You have chosen Blue.

What color do you want? g

You have chosen Green.

What color do you want? Bork

Unknown color: Bork

14. Viết một đoạn mã để đọc mô tả ngắn gọn của một quân bài và in ra phiên bản dài hơn. Mô tả ngắn gọn là hạng của quân bài (2 đến 10, J, Q, K, hoặc A) theo sau là bộ bài của nó (C, D, H, hoặc S). Bạn nên mở rộng mô tả ngắn thành dạng "<Hạng> của <Bộ>". Bạn có thể giả định rằng người dùng nhập vào dữ liệu hợp lệ. Dưới đây là hai ví dụ thực hiện:

css

Copy code

Enter a card: 9 S

Nine of

Mục 4.2: Thuật toán tích lũy

15. **Sai sót trong đoạn code sau là gì, khi nó cố gắng cộng tất cả các số từ 1 đến một giá trị tối đa được cho? Hãy mô tả cách sửa lỗi của đoạn code này.**

java

Copy code

```
public static int sumTo(int n) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    return sum;
}
```

16. **Sai sót trong đoạn code sau là gì, khi nó cố gắng trả về số lượng các ước số của một số nguyên n được cho? Hãy mô tả cách sửa lỗi của đoạn code này.**

java

Copy code

```
public static int countFactors(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        if (n % i == 0) {
            count++;
        }
    }
    return count;
}
```

17. **Viết code để thực hiện tích lũy bằng cách nhân với nhiều số được đọc từ console.**

java

Copy code

```
import java.util.Scanner;
```

```
public static int cumulativeProduct() {

    Scanner scanner = new Scanner(System.in);

    int product = 1;

    System.out.println("Enter numbers to multiply (enter 0 to
stop):");

    while (true) {

        int num = scanner.nextInt();

        if (num == 0) break;

        product *= num;

    }

    scanner.close();

    return product;

}
```

18. **Biểu thức sau đây phải bằng 6.8, nhưng trong Java lại không như vậy. Tại sao?**

java

Copy code

0.2 + 1.2 + 2.2 + 3.2;

Lý do: Java sử dụng kiểu số thực dấu phẩy động, điều này có thể dẫn đến sai số nhỏ trong việc lưu trữ và tính toán các số thập phân.

19. **Đoạn code sau được dự định để in ra một thông báo, nhưng thực tế không tạo ra kết quả nào. Hãy mô tả cách sửa lỗi của đoạn code để in ra thông báo như mong đợi.**

java

Copy code

```
double gpa = 3.2;

if (Math.abs(gpa * 3 - 9.6) < 0.0001) {

    System.out.println("You earned enough credits.");

}
```

Mục 4.3: Xử lý văn bản

20. **Kết quả đầu ra của chương trình sau là gì?**

java

Copy code

```
public class CharMystery {

    public static void printRange(char startLetter, char endLetter) {
```

```
        for (char letter = startLetter; letter <= endLetter; letter++) {
            System.out.print(letter);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        printRange('e', 'g');
        printRange('n', 's');
        printRange('z', 'a');
        printRange('q', 'r');
    }
}
```

21. **Viết một câu lệnh if để kiểm tra xem một chuỗi có bắt đầu bằng chữ cái in hoa hay không.**

java

Copy code

```
if (Character.isUpperCase(s.charAt(0))) {
```

```
    // Thực hiện hành động khi chuỗi bắt đầu bằng chữ in hoa
}
```

22. **Sai sót trong đoạn code sau là gì, khi nó cố gắng đếm số lần xuất hiện của chữ cái 'e' trong một chuỗi, không phân biệt chữ hoa thường?**

java

Copy code

```
int count = 0;
for (int i = 0; i < s.length(); i++) {
    if (Character.toLowerCase(s.charAt(i)) == 'e') {
        count++;
    }
}
```

23. **Giả sử một chuỗi được lưu trữ trong biến name chứa tên và họ của một người (ví dụ: “Marla Singer”). Viết biểu thức để tạo ra họ trước, sau đó là chữ cái đầu của tên (ví dụ: “Singer, M.”).**

java

Copy code

```
String lastNameFirst = name.substring(name.indexOf(' ') + 1) + ", " + name.charAt(0) + ".";
```

24. **Viết code để kiểm tra một chuỗi và xác định có bao nhiêu chữ cái của nó đến từ nửa sau của bảng chữ cái (tức là có giá trị từ 'n' trở đi). So sánh không phân biệt chữ hoa thường, nghĩa là các giá trị từ 'N' đến 'Z' cũng được tính. Giả sử rằng mọi ký tự trong chuỗi đều là chữ cái.**

java

Copy code

```
int count = 0;

for (int i = 0; i < s.length(); i++) {

    char c = Character.toLowerCase(s.charAt(i));

    if (c >= 'n') {

        count++;

    }

}
```

Mục 4.4: Phương pháp với Điều kiện Thực thi

25. **Xem xét một phương thức printTriangleType nhận ba tham số là ba số nguyên đại diện cho độ dài các cạnh của một tam giác và in ra loại tam giác mà các cạnh này tạo thành. Ba loại tam giác là đều, cân và thường. Tam giác đều có ba cạnh bằng nhau, tam giác cân có hai cạnh bằng nhau, và tam giác thường có ba cạnh có độ dài khác nhau. Tuy nhiên, một số giá trị số nguyên nhất định (hoặc kết hợp các giá trị) sẽ không hợp lệ và không thể đại diện cho các cạnh của một tam giác thực sự. Các giá trị này là gì? Bạn sẽ mô tả điều kiện tiên quyết của phương thức printTriangleType như thế nào?**
26. **Xem xét một phương thức getGrade nhận một số nguyên đại diện cho phần trăm điểm số của học sinh trong một khóa học và trả về điểm số của học sinh đó trên thang điểm từ 0.0 (thất bại) đến 4.0 (hoàn hảo). Điều kiện tiên quyết của phương thức này là gì?**
27. **Phương thức sau đây cố gắng trả về giá trị trung vị (giá trị ở giữa) của ba giá trị số nguyên, nhưng nó chứa lỗi logic. Trong những trường hợp nào phương thức trả về kết quả sai? Làm thế nào để sửa lỗi trong đoạn mã này?**

java
Copy code

```
public static int medianOf3(int n1, int n2, int n3) {
    if (n1 < n2) {
        if (n2 < n3) {
            return n2;
        } else {
            return n3;
        }
    } else {
```

```
        if (n1 < n3) {
            return n1;
        } else {
            return n3;
        }
    }
}
```

28. **Một trong những bài tập ở Chương 3 yêu cầu bạn viết một phương thức để tìm nghiệm của phương trình bậc hai dưới dạng ax^2 + bx + c = 0. Phương thức quadratic được truyền các giá trị a, b, và c, sau đó áp dụng công thức bậc hai sau:**

text
Copy code
x = −b ± √(b^2 − 4ac) / 2a

Trong điều kiện nào công thức này sẽ thất bại? Hãy sửa đổi phương thức quadratic để nó sẽ từ chối các giá trị a, b hoặc c không hợp lệ bằng cách ném ra ngoại lệ. (Nếu bạn không hoàn thành bài tập trong chương trước, chỉ cần viết tiêu đề của phương thức và mã ném ngoại lệ.)

29. **Xem xét phương thức Java sau đây, được viết không chính xác:**

java
Copy code

```
// Phương thức này nên trả về số lượng các đối số là số lẻ.
public static void printNumOdd(int n1, int n2, int n3) {
    int count = 0;
    if (n1 % 2 != 0) {
        count++;
    } else if (n2 % 2 != 0) {
        count++;
    } else if (n3 % 2 != 0) {
        count++;
```

```
    }
    System.out.println(count + " of the 3 numbers are odd.");
}
```

Trong những trường hợp nào phương thức này in ra câu trả lời đúng, và khi nào nó sẽ in ra câu trả lời sai? Cần thay đổi gì để sửa lỗi trong đoạn mã này? Bạn có nghĩ ra cách nào để viết đúng mã mà không cần sử dụng câu lệnh if/else không?

Bài tập

1. **Viết một phương thức gọi là fractionSum nhận một tham số số nguyên n và trả về tổng của n số hạng đầu tiên của dãy dưới dạng số thực (double):**

text
Copy code
∑(i=1 đến n) của 1/i

Nói cách khác, phương thức nên tạo ra dãy số sau: 1 + 1/2 + 1/3 + 1/4 + 1/5 + ... Bạn có thể giả định rằng tham số n là số không âm.

2. **Viết một phương thức gọi là repl nhận một chuỗi và một số lần lặp làm tham số, và trả về chuỗi đó được nối lại nhiều lần tương ứng. Ví dụ, lời gọi repl("hello", 3) sẽ trả về "hellohellohello". Nếu số lần lặp bằng 0 hoặc ít hơn, phương thức nên trả về một chuỗi rỗng.**
3. **Viết một phương thức gọi là season nhận hai tham số là hai số nguyên đại diện cho tháng và ngày, và trả về một chuỗi chỉ định mùa cho tháng và ngày đó. Giả sử rằng tháng được chỉ định bằng một số nguyên từ 1 đến 12 (1 là tháng Một, 2 là tháng Hai, v.v.) và ngày của tháng là một số từ 1 đến 31. Nếu ngày thuộc khoảng từ 16/12 đến 15/3, phương thức nên trả về "winter". Nếu ngày thuộc khoảng từ 16/3 đến 15/6, phương thức nên trả về "spring". Nếu ngày thuộc khoảng từ 16/6 đến 15/9, phương thức nên trả về "summer". Và nếu ngày thuộc khoảng từ 16/9 đến 15/12, phương thức nên trả về "fall".**

4. **Viết một phương thức gọi là daysInMonth nhận một tham số là tháng (một số nguyên từ 1 đến 12) và trả về số ngày trong tháng đó trong năm nay. Ví dụ, lời gọi daysInMonth(9) sẽ trả về 30 vì tháng Chín có 30 ngày. Giả sử rằng đoạn mã này không được chạy trong một năm nhuận (tháng Hai luôn có 28 ngày). Bảng sau liệt kê số ngày trong mỗi tháng:**

text
Copy code
Tháng 1 Jan 2 Feb 3 Mar 4 Apr 5 May 6 Jun 7 Jul 8 Aug 9 Sep 10 Oct 11 Nov 12 Dec
Ngày 31 28 31 30 31 30 31 31 30 31 30 31

5. **Viết một phương thức gọi là pow nhận hai tham số là cơ số và số mũ, và trả về cơ số được nâng lên lũy thừa của số mũ đó. Ví dụ, lời gọi pow(3, 4) sẽ trả về 3 * 3 * 3 * 3, hoặc 81. Giả sử rằng cơ số và số mũ là số không âm.**
6. **Viết một phương thức gọi là printRange nhận hai số nguyên làm tham số và in ra dãy số nằm giữa hai số đó, cách nhau bằng dấu cách. In ra một dãy tăng dần nếu tham số đầu tiên nhỏ hơn tham số thứ hai; nếu không, in ra một dãy giảm dần. Nếu hai số giống nhau, thì số đó sẽ được in ra. Dưới đây là một số lời gọi mẫu cho printRange:**

java
Copy code
printRange(2, 7);
printRange(19, 11);
printRange(5, 5);

Kết quả đầu ra từ các lời gọi này sẽ là các dãy số sau:

text
Copy code
2 3 4 5 6 7

19 18 17 16 15 14 13 12 11
5

7. **Viết một phương thức tĩnh gọi là xo nhận một tham số số nguyên size và in ra một hình vuông gồm các ký tự size x size, trong đó tất cả các ký tự là "o" ngoại trừ một mô hình chữ "x" gồm các ký tự "x" được vẽ từ các góc của hình vuông. Ở dòng đầu tiên, ký tự đầu tiên và cuối cùng là "x"; ở dòng thứ hai, ký tự thứ hai và ký tự gần cuối cùng là "x"; và tiếp tục như vậy. Dưới đây là hai kết quả đầu ra mẫu:**

text
Copy code

```
xo(5);      xo(6);
xoox       xooox
oxoxo      oxoxo
ooxoo      ooxxoo
oxoxo      ooxxoo
xoox       oxoxo
           xooox
```

8. **Viết một phương thức gọi là smallestLargest nhận một tham số Scanner từ console và yêu cầu người dùng nhập vào các số, sau đó in ra số nhỏ nhất và lớn nhất trong tất cả các số mà người dùng đã cung cấp. Bạn có thể giả định rằng người dùng nhập vào một số hợp lệ lớn hơn 0 cho số lượng số cần đọc. Dưới đây là một mẫu thực thi:**

text
Copy code

```
Bạn muốn nhập bao nhiêu số?
4

Số 1:
5
```

Số 2:
11

Số 3:
-2

Số 4:
3

Số nhỏ nhất = -2
Số lớn nhất = 11

9. **Viết một phương thức gọi là evenSumMax nhận một tham số Scanner từ console. Phương thức này sẽ yêu cầu người dùng nhập vào một số lượng số nguyên, sau đó yêu cầu người dùng nhập vào các số nguyên tương ứng. Sau khi người dùng đã nhập tất cả các số nguyên, phương thức này sẽ in ra tổng của tất cả các số chẵn mà người dùng đã nhập, cùng với số chẵn lớn nhất. Bạn có thể giả định rằng người dùng sẽ nhập ít nhất một số nguyên không âm và chẵn. Dưới đây là một ví dụ đối thoại:**

text
Copy code
Bạn muốn nhập bao nhiêu số nguyên?
4

Số nguyên tiếp theo?
2

Số nguyên tiếp theo?
9

Số nguyên tiếp theo?
18

Số nguyên tiếp theo?
4

Tổng số chẵn = 24, Số chẵn lớn nhất = 18

10. **Viết một phương thức gọi là printGPA nhận một tham số Scanner từ console và tính toán điểm trung bình (GPA) của một sinh viên. Người dùng sẽ nhập vào một dòng dữ liệu chứa tên của sinh viên, sau đó là một số đại diện cho số điểm, tiếp theo là các điểm số nguyên tương ứng. Dưới đây là hai ví dụ đối thoại:**

text
Copy code
Nhập hồ sơ sinh viên:
Maria 5 72 91 84 89 78

Điểm của Maria là 82.8

Nhập hồ sơ sinh viên:
Jordan 4 86 71 62 90

Điểm của Jordan là 77.25

Điểm của Maria là 82.8 vì trung bình của (72+91+84+89+78)/5 bằng 82.8.

11. **Viết một phương thức gọi là longestName nhận một tham số Scanner từ console và một số nguyên n, sau đó yêu cầu người dùng nhập vào n tên, và in ra tên dài nhất (tên chứa nhiều ký tự nhất) theo định dạng dưới đây, có thể được tạo ra từ lời gọi longestName(console, 4):**

text

Copy code
Tên #1?
Roy

Tên #2?
DANE

Tên #3?
sTeFaNiE

Tên #4?
Mariana

Tên Stefanie là dài nhất

12. **Viết phương thức gọi là printTriangleType đã được nhắc đến trong Bài tập Tự kiểm tra 25. Phương thức này nhận ba tham số là ba số nguyên đại diện cho độ dài các cạnh của một tam giác và in ra loại tam giác mà các cạnh này tạo thành. Dưới đây là một số lời gọi mẫu cho printTriangleType:**

java
Copy code
printTriangleType(5, 7, 7);
printTriangleType(6, 6, 6);
printTriangleType(5, 7, 8);
printTriangleType(2, 18, 2);

Kết quả đầu ra từ các lời gọi này sẽ là:

text
Copy code
cân
đều
thường
cân

Phương thức của bạn nên ném ra ngoại lệ IllegalArgumentException nếu nhận các giá trị không hợp lệ, chẳng hạn như một cạnh có độ dài lớn hơn tổng của hai cạnh còn lại, điều này là không thể trong một tam giác. Ví dụ, lời gọi printTriangleType(2, 18, 2); nên ném ra một ngoại lệ.

13. **Viết một phương thức gọi là average nhận hai tham số là hai số nguyên và trả về trung bình của hai số nguyên đó.**
14. **Sửa đổi phương thức pow của bạn từ Bài tập 5 để tạo ra một phương thức mới gọi là pow2 sử dụng kiểu double cho tham số đầu tiên và hoạt động đúng với các số âm. Ví dụ, lời gọi pow2(-4.0, 3) nên trả về -4.0 * -4.0 * -4.0, hoặc -64.0, và lời gọi pow2(4.0, -2) nên trả về 1 / 16, hoặc 0.0625.**
15. **Viết một phương thức gọi là getGrade nhận một số nguyên đại diện cho điểm số của học sinh trong một khóa học và trả về điểm số của học sinh đó trên thang điểm từ 0.0 (thất bại) đến 4.0 (hoàn hảo). Giả sử rằng các điểm số nằm trong khoảng từ 0 đến 100 và các điểm số được tính dựa trên thang điểm sau:**

text
Copy code

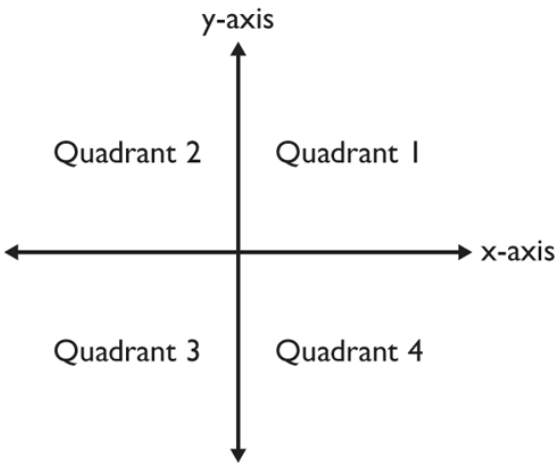
Điểm số	Điểm
<60	0.0
60–62	0.7
63	0.8
64	0.9
65	1.0
...	
92	3.7
93	3.8
94	3.9
>=95	4.0

Để thêm phần thử thách, hãy làm cho phương thức của bạn ném ra một ngoại lệ IllegalArgumentException nếu người dùng truyền vào một điểm số nhỏ hơn 0 hoặc lớn hơn 100.

16. **Viết một phương thức gọi là printPalindrome nhận một tham số Scanner từ console, yêu cầu người dùng nhập vào một hoặc nhiều từ, và in ra liệu chuỗi đã nhập có phải là một chuỗi đối xứng (palindrome) hay không (tức là chuỗi được đọc giống nhau khi từ trái sang phải hoặc từ phải sang trái, như "abba" hoặc "racecar"). Để thêm phần thử thách, hãy làm cho mã không phân biệt chữ hoa chữ thường, để các từ như "Abba" và "Madam" sẽ được coi là palindrome.**
17. **Viết một phương thức gọi là stutter nhận một tham số chuỗi và trả về chuỗi đó với các ký tự được lặp lại hai lần. Ví dụ, stutter("Hello!") sẽ trả về "HHeellllloo!!".**
18. **Viết một phương thức gọi là wordCount nhận một tham số chuỗi và trả về số từ trong chuỗi đó. Một từ là một chuỗi gồm một hoặc nhiều ký tự không phải khoảng trắng (bất kỳ ký tự nào ngoại trừ ' '). Ví dụ, lời gọi wordCount("hello") sẽ trả về 1, lời gọi wordCount("how are you?") sẽ trả về 3, lời gọi wordCount(" this string has wide spaces ") sẽ trả về 5, và lời gọi wordCount(" ") sẽ trả về 0.**
19. **Viết một phương thức gọi là quadrant nhận hai tham số là hai giá trị kiểu double đại diện cho một điểm (x, y) và trả về số thứ tự của góc phần tư mà điểm đó nằm trong. Nhớ rằng các góc phần tư được đánh số từ 1 đến 4 theo chiều ngược kim đồng hồ, với góc phần tư phía trên bên phải được đánh số 1 và các góc phần tư tiếp theo được đánh số theo chiều ngược kim đồng hồ:**

text
Copy code
Quadrant 1 | Quadrant 2

Quadrant 4 | Quadrant 3
4o



Chú ý rằng góc phần tư được xác định bằng cách xem xét tọa độ x và y là số dương hay số âm. Trả về 0 nếu điểm nằm trên trục x hoặc y. Ví dụ, lời gọi quadrant(-2.3, 3.5) sẽ trả về 2 và lời gọi quadrant(4.5, -4.5) sẽ trả về 4.

1. **Viết một phương thức tên là numUnique nhận ba số nguyên làm tham số và trả về số lượng các số nguyên duy nhất trong ba số đó. Ví dụ, lời gọi numUnique(18, 3, 4) sẽ trả về 3 vì các tham số có ba giá trị khác nhau. Ngược lại, lời gọi numUnique(6, 7, 6) sẽ trả về 2 vì chỉ có hai số duy nhất trong ba tham số: 6 và 7.**
2. **Viết một phương thức tên là perfectNumbers nhận một số nguyên maximum làm tham số và in ra tất cả các "số hoàn hảo" từ 1 đến maximum. Một số hoàn hảo là một số nguyên bằng tổng của các ước số nguyên dương khác ngoài 1 và chính nó. Ví dụ, số 28 là một số hoàn hảo vì 1 + 2 + 4 + 7 + 14 = 28. Lời gọi perfectNumbers(500); sẽ tạo ra kết quả sau:**

plaintext
Copy code
Perfect numbers up to 500: 6 28 496

3. **Viết một phương thức tên là printAcronym nhận một chuỗi làm tham số và in ra chữ cái đầu tiên của mỗi từ trong chuỗi như một chữ viết tắt. Ví dụ, lời gọi printAcronym("Breath of the Wild") sẽ in ra "BotW". Bạn có thể giả định rằng chuỗi chứa ít nhất một từ và không có khoảng trắng ở đầu hoặc cuối.**

Dự án lập trình:

1. **Viết một chương trình yêu cầu người dùng nhập một số và hiển thị nó dưới dạng số La Mã.**
2. **Viết một chương trình yêu cầu người dùng nhập một ngày (tháng, ngày, năm) và cho biết ngày đó là thứ mấy trong tuần. Có thể hữu ích khi biết rằng ngày 1 tháng 1 năm 1601 là thứ Hai.**
3. **Viết một chương trình so sánh hai ứng viên đại học. Chương trình sẽ yêu cầu người dùng nhập điểm GPA, SAT, và điểm thi ACT của mỗi ứng viên, sau đó cho biết ứng viên nào có điểm số cao hơn dựa trên các số liệu này.**
4. **Viết một chương trình yêu cầu người dùng nhập ngày sinh của hai người (tháng và ngày), cùng với tháng và ngày hiện tại. Chương trình sẽ tính toán xem còn bao nhiêu ngày nữa là đến sinh nhật của mỗi người và sinh nhật nào đến sớm hơn. Gợi ý: Sẽ dễ dàng hơn nếu bạn chuyển đổi mỗi ngày thành một "ngày tuyệt đối" trong năm, từ 1 đến 365.**
5. **Viết một chương trình tính toán điểm số của một sinh viên trong một khóa học. Điểm khóa học bao gồm ba phần: bài tập về nhà, thi giữa kỳ, và thi cuối kỳ. Chương trình sẽ yêu cầu người dùng nhập tất cả thông tin cần thiết để tính điểm, chẳng hạn như số lượng bài tập về nhà, số điểm đạt được và số điểm tối đa của mỗi bài tập, điểm thi giữa kỳ và điểm thi cuối kỳ, và liệu mỗi kỳ thi có được điều chỉnh điểm hay không (nếu có, điều chỉnh**

bao nhiêu). Hãy cân nhắc viết một phiên bản của chương trình này báo cáo số điểm thi cuối kỳ mà sinh viên cần để đạt được một điểm khóa học cụ thể.

6. **Một kỹ thuật hữu ích để phát hiện lỗi đánh máy là sử dụng số kiểm tra (check digit). Ví dụ, giả sử một trường học gán một số gồm sáu chữ số cho mỗi học sinh. Một chữ số thứ bảy có thể được xác định từ các chữ số khác bằng cách sử dụng công thức sau:**

plaintext
Copy code
Chữ số thứ 7 = (1*(chữ số 1) + 2*(chữ số 2) + ... + 6*(chữ số 6)) % 10

Khi người dùng nhập số học sinh, họ sẽ nhập tất cả bảy chữ số. Nếu số được nhập sai, chữ số kiểm tra sẽ không khớp trong 90% các trường hợp. Viết một chương trình tương tác yêu cầu người dùng nhập một số học sinh gồm sáu chữ số và báo cáo chữ số kiểm tra cho số đó, sử dụng công thức trên.

7. **Viết một chương trình hiển thị tam giác Pascal:**

plaintext
Copy code

```
                    1
                  1   1
                1   2   1
              1   3   3   1
            1   4   6   4   1
          1   5  10  10   5   1
        1   6  15  20  15   6   1
      1   7  21  35  35  21   7   1
    1   8  28  56  70  56  28   8   1
  1   9  36   84 126 126  84  36  9  1
1  10  45 120 210 252 210 120  45  10   1
```

Sử dụng System.out.printf để định dạng đầu ra thành các cột có độ rộng 4.

8. **Viết một chương trình tạo ra một mật mã Caesar cho một chuỗi thông điệp. Mật mã Caesar, hay còn gọi là mật mã quay vòng, được hình thành bằng cách dịch mỗi chữ cái của một thông điệp theo một số lượng nhất định. Ví dụ, nếu bạn dịch chuyển 3 ký tự, mỗi chữ A sẽ thành D; mỗi chữ B sẽ thành E; và cứ thế tiếp tục. Đến cuối bảng chữ cái, bạn sẽ quay lại đầu: X thành A; Y thành B; và Z thành C. Chương trình của bạn nên yêu cầu người dùng nhập một thông điệp và một số lượng để dịch chuyển mỗi chữ cái, sau đó xuất ra thông điệp đã được mã hóa.**

plaintext
Copy code
Your message?
Attack zerg at dawn

Encoding key?
3

Your message:
DWWDFN CHUJ DW GDZQ

Chương 5: Logic Chương Trình và Vòng Lặp Vô Hạn

5.1 Vòng lặp while

- Một Vòng lặp để Tìm Ước số Nhỏ nhất
- Số Ngẫu nhiên
- Mô phỏng

- Vòng lặp do/while

5.2 Thuật toán Fencepost

- Fencepost với if
- Vòng lặp Sentinel

5.3 Kiểu boolean

- Toán tử Logic
- Đánh giá Ngắn mạch (Short-Circuited Evaluation)
- Biến boolean và Cờ hiệu
- Boolean Zen
- Phủ định Biểu thức Boolean

5.4 Lỗi Người Dùng

- Scanner Lookahead
- Xử lý Lỗi Người Dùng

5.5 Kiểm định và Logic Chương trình

- Lập luận về Kiểm định
- Ví dụ Chi tiết về Kiểm định

5.6 Nghiên cứu tình huống: NumberGuess

- Phiên bản ban đầu không có gợi ý
- Phiên bản ngẫu nhiên có gợi ý
- Phiên bản cuối cùng mạnh mẽ

Giới thiệu

Chương này bắt đầu bằng việc xem xét một cấu trúc mới gọi là vòng lặp while, cho phép bạn lặp lại một số lần không xác định. Vòng lặp while sẽ giúp bạn giải quyết một lớp vấn đề lập trình mới, trong đó bạn không biết trước số lần vòng lặp cần thực hiện. Ví dụ, các chương trình chơi game thường sử dụng vòng lặp while vì không thể biết trước cách người dùng sẽ chơi game như thế nào. Vì chúng ta sẽ khám phá các chương trình chơi game, chúng ta cũng sẽ tìm hiểu cách tạo ra các số ngẫu nhiên trong chương trình Java. Chúng ta cũng sẽ khám phá một lớp thuật toán khác gọi là thuật toán fencepost, thường xuất hiện trong các nhiệm vụ lập trình vòng lặp.

Chương này sau đó sẽ thảo luận về kiểu nguyên thủy thứ tư mà chúng ta sẽ xem xét chi tiết, đó là boolean. Kiểu boolean được sử dụng để lưu trữ thông tin logic (đúng/sai). Khi hiểu rõ về kiểu boolean, bạn sẽ có thể viết các vòng lặp phức tạp liên quan đến nhiều bài kiểm tra.

Tiếp theo, chúng ta sẽ nhanh chóng xem xét chủ đề quan trọng là xử lý lỗi người dùng.

Chương này kết thúc với một cuộc thảo luận về kiểm định. Sử dụng kiểm định, bạn có thể lập luận về các thuộc tính chính thức của chương trình (những gì là đúng tại các điểm khác nhau trong quá trình thực thi chương trình).

5.1 Vòng lặp while

Các vòng lặp for mà chúng ta đã viết từ Chương 2 là các vòng lặp khá đơn giản, thực hiện một số lần lặp có thể dự đoán được. Nhớ lại rằng chúng ta gọi chúng là vòng lặp xác định vì chúng ta biết trước khi vòng lặp bắt đầu thực thi rằng chúng sẽ thực hiện bao nhiêu lần. Bây giờ chúng ta muốn chuyển sự chú ý đến các vòng lặp vô hạn, tức là các vòng lặp thực hiện một số lần không xác định. Các vòng lặp vô hạn thường xuất hiện trong các chương trình tương tác và xử lý tệp. Ví dụ, bạn không biết trước người dùng có thể muốn chơi một trò chơi bao nhiêu lần, và bạn sẽ không biết trước khi xem xét một tệp rằng nó chứa bao nhiêu dữ liệu.

Vòng lặp while là vòng lặp vô hạn đầu tiên mà chúng ta sẽ nghiên cứu. Nó có cú pháp như sau:

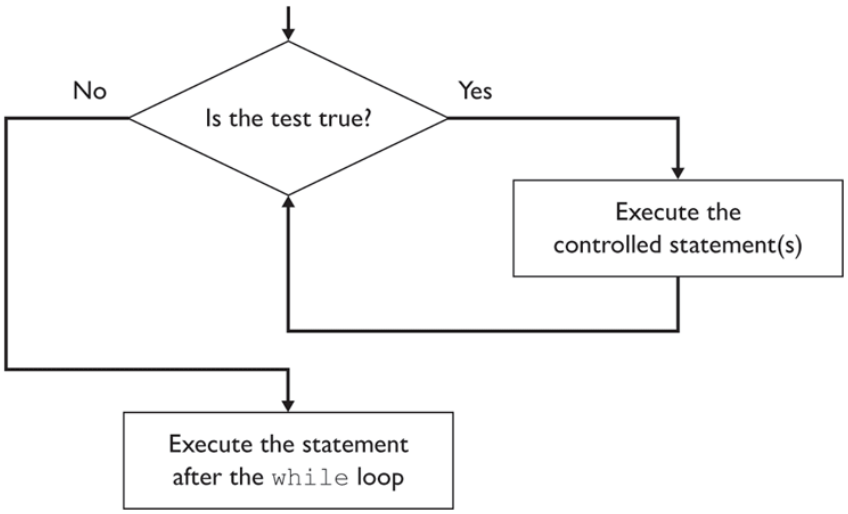
java

Copy code

```
while (<điều kiện kiểm tra>) {
    <câu lệnh>;
    <câu lệnh>;
    ...
    <câu lệnh>;
}
```

Hình 5.1 minh họa luồng điều khiển của vòng lặp while. Vòng lặp thực hiện kiểm tra của nó và nếu kiểm tra đánh giá là đúng, nó sẽ thực thi các câu lệnh được kiểm soát. Nó lặp lại việc kiểm tra và thực thi nếu kiểm tra đánh giá là đúng. Chỉ khi kiểm tra đánh giá là sai thì vòng lặp mới kết thúc.

Hình 5.1: Luồng điều khiển của vòng lặp while.



Hình 5.1 Văn bản thay thế đầy đủ

Như Hình 5.1 cho thấy, vòng lặp while thực hiện kiểm tra ở đầu vòng lặp, trước khi phần thân của vòng lặp được thực thi. Vòng lặp while sẽ không thực thi các câu lệnh điều khiển của nó nếu kiểm tra của nó đánh giá là sai ngay lần đầu tiên.

Dưới đây là một ví dụ về vòng lặp while:

java

Copy code

int number = 1;

```
while (number <= 200) {

    number = number * 2;

}
```

Vòng lặp này khởi tạo một biến số nguyên gọi là number bằng 1 và sau đó nhân đôi nó khi nó nhỏ hơn hoặc bằng 200. Bề ngoài, thao tác này giống như việc sử dụng câu lệnh if:

java

Copy code

```
int number = 1;

if (number <= 200) {

    number = number * 2;

}
```

Sự khác biệt giữa hai dạng này là vòng lặp while thực thi nhiều lần, lặp lại cho đến khi kiểm tra đánh giá là sai. Câu lệnh if chỉ thực thi câu lệnh nhân đôi một lần, để lại number bằng 2. Vòng lặp while thực thi câu lệnh nhân đôi nhiều lần, với number lần lượt nhận các giá trị 1, 2, 4, 8, 16, 32, 64, 128 và 256. Vòng lặp không dừng lại cho đến khi kiểm tra đánh giá là sai. Nó thực thi câu lệnh gán tám lần và kết thúc khi number được đặt thành giá trị 256 (lũy thừa đầu tiên của 2 lớn hơn 200).

Dưới đây là một vòng lặp while chứa hai câu lệnh:

java

Copy code

```
int number = 1;

while (number <= max) {

    System.out.println("Hi there");

    number++;

}
```

Vòng lặp while này gần giống với vòng lặp for sau:

java

Copy code

```
for (int number = 1; number <= max; number++) {

    System.out.println("Hi there");

}
```

Sự khác biệt duy nhất giữa hai vòng lặp này là phạm vi của biến number. Trong vòng lặp while, number được khai báo trong phạm vi bên ngoài vòng lặp. Trong vòng lặp for, number được khai báo bên trong vòng lặp.

Một Vòng lặp để Tìm Ước số Nhỏ nhất

Giả sử bạn muốn tìm ước số nhỏ nhất của một số khác 1. Bảng 5.1 đưa ra các ví dụ về những gì bạn đang tìm kiếm.

Bảng 5.1: Ví dụ về các Ước số

Số	Ước số	Ước số nhỏ nhất
10	2 * 5	2
15	3 * 5	3
25	5 * 5	5
31	31	31
77	7 * 11	7

Dưới đây là mô tả bằng mã giả về cách bạn có thể tìm giá trị này:

pseudocode

Copy code

bắt đầu với divisor là 2.

while (giá trị hiện tại của divisor không hoạt động) {

    tăng divisor.

}

Bạn không bắt đầu với divisor là 1 vì bạn đang tìm ước số đầu tiên lớn hơn 1. Để tinh chỉnh mã giả này, bạn cần phải rõ ràng hơn về điều gì làm cho một ước số hoạt động. Một ước số của một số không có số dư khi số đó được chia cho ước số đó. Bạn có thể viết lại quy tắc này dưới dạng mã giả sau:

pseudocode

Copy code

bắt đầu với divisor là 2.

while (số dư của number/divisor không bằng 0) {

    tăng divisor.

}

Vấn đề này có thể được giải quyết bằng cách sử dụng toán tử mod, toán tử này trả về số dư của phép chia nguyên. Vòng lặp while sau đây thực hiện nhiệm vụ này:

java

Copy code

```
int divisor = 2;

while (number % divisor != 0) {

    divisor++;

}
```

Một vấn đề mà bạn sẽ gặp phải khi viết vòng lặp while là vòng lặp vô hạn nổi tiếng. Hãy xem xét đoạn mã sau:

java

Copy code

```
int number = 1;

while (number > 0) {

    number++;

}
```

Vì number bắt đầu là một giá trị dương và vòng lặp làm cho nó lớn hơn, vòng lặp này sẽ tiếp tục vô thời hạn. Bạn phải cẩn thận khi xây dựng vòng lặp while của mình để tránh các tình huống trong đó một đoạn mã không bao giờ kết thúc. Mỗi khi bạn viết một vòng lặp while, bạn nên xem xét khi nào và cách nó sẽ kết thúc.

LỖI LẬP TRÌNH PHỔ BIẾN

Vòng Lặp Vô Hạn

Rất dễ viết một vòng lặp while không bao giờ kết thúc. Một trong những lý do khiến lỗi này dễ mắc phải là vì vòng lặp while không có bước cập nhật trong phần tiêu đề của nó như vòng lặp for. Điều quan trọng đối với lập trình viên là phải bao gồm một bước cập nhật chính xác vì bước này cần thiết để cuối cùng làm cho bài kiểm tra của vòng lặp thất bại.

Xem xét đoạn mã sau, được thiết kế để yêu cầu người dùng nhập một số và liên tục in ra số đó chia đôi cho đến khi giá trị là 0. Nỗ lực đầu tiên này không biên dịch:

java

Copy code

```
Scanner console = new Scanner(System.in);

System.out.print("Type a number: ");



// đoạn mã này không biên dịch

while (number > 0) {

    int number = console.nextInt();

    System.out.println(number / 2);

}
```

Vấn đề với đoạn mã trên là biến number cần phải nằm trong phạm vi khi thực hiện kiểm tra của vòng lặp, vì vậy nó không thể được khai báo bên trong vòng lặp. Một nỗ lực không chính xác để sửa lỗi biên dịch này là cắt và dán dòng khởi tạo number ra ngoài vòng lặp:

java

Copy code

```
// đoạn mã này có vòng lặp vô hạn

int number = console.nextInt(); // chuyển ra ngoài vòng lặp

while (number > 0) {
```

```
    System.out.println(number / 2);

}
```

Phiên bản mã này có vòng lặp vô hạn; nếu vòng lặp được nhập, nó sẽ không bao giờ kết thúc. Vấn đề này phát sinh vì không có bước cập nhật bên trong thân vòng lặp while để thay đổi giá trị của number. Nếu number lớn hơn 0, vòng lặp sẽ tiếp tục in giá trị của nó và kiểm tra điều kiện của vòng lặp, và điều kiện kiểm tra sẽ đánh giá là đúng mỗi lần.

Phiên bản mã sau đây giải quyết vấn đề vòng lặp vô hạn. Vòng lặp chứa một bước cập nhật trong mỗi lần lặp lại, chia đôi giá trị số nguyên và lưu trữ giá trị mới của nó. Nếu số nguyên chưa đạt tới 0, vòng lặp sẽ lặp lại:

java

Copy code

```
// đoạn mã này hoạt động chính xác

int number = console.nextInt(); // chuyển ra ngoài vòng lặp

while (number > 0) {

    number = number / 2; // bước cập nhật: chia đôi

    System.out.println(number);

}
```

Ý tưởng chính là mọi thân vòng lặp while nên chứa mã để cập nhật các điều kiện được kiểm tra trong điều kiện kiểm tra của

vòng lặp. Nếu điều kiện kiểm tra của vòng lặp while kiểm tra giá trị của một biến, thân vòng lặp nên gán lại một giá trị mới có ý nghĩa cho biến đó.

Số Ngẫu Nhiên

Chúng ta thường muốn các chương trình của mình thể hiện hành vi có vẻ ngẫu nhiên. Ví dụ: chúng ta muốn các chương trình chơi game tạo ra một số cho người dùng đoán, xáo trộn một bộ bài, chọn một từ từ danh sách để người dùng đoán, v.v. Các chương trình, bản chất của chúng là có thể dự đoán được và không ngẫu nhiên. Nhưng chúng ta có thể tạo ra các giá trị có vẻ ngẫu nhiên. Những giá trị như vậy được gọi là số giả ngẫu nhiên vì chúng được tạo ra theo thuật toán.

Số Giả Ngẫu Nhiên

Số giả ngẫu nhiên là các số bắt nguồn từ các thuật toán có thể dự đoán và được xác định rõ ràng, nhưng lại mô phỏng các thuộc tính của các số được chọn ngẫu nhiên.

Java cung cấp nhiều cơ chế để lấy số giả ngẫu nhiên. Một tùy chọn là gọi phương thức random từ lớp Math để lấy giá trị ngẫu nhiên thuộc loại double với thuộc tính sau:

java

Copy code

0.0 ≤ Math.random() < 1.0

Phương thức này cung cấp một cách nhanh chóng và dễ dàng để lấy số ngẫu nhiên, và bạn có thể sử dụng phép nhân để thay đổi phạm vi của các số mà phương thức tạo ra. Java cũng cung cấp

một lớp gọi là Random có thể dễ sử dụng hơn. Lớp này được bao gồm trong gói java.util, vì vậy bạn phải thêm một khai báo import ở đầu chương trình để sử dụng nó.

Đối tượng Random có nhiều phương thức hữu ích liên quan đến việc tạo số giả ngẫu nhiên, được liệt kê trong Bảng 5.2. Mỗi lần bạn gọi một trong các phương thức này, Java sẽ tạo ra và trả về một số ngẫu nhiên mới thuộc loại được yêu cầu.

Bảng 5.2: Các Phương Thức Hữu Ích của Đối Tượng Random

Phương thức	**Mô tả**
nextInt()	Số nguyên ngẫu nhiên từ –2^31 đến (2^31 – 1)
nextInt(max)	Số nguyên ngẫu nhiên từ 0 đến (max – 1)
nextDouble()	Số thực ngẫu nhiên từ 0.0 (bao gồm) đến 1.0 (không bao gồm)
nextBoolean()	Giá trị logic ngẫu nhiên true hoặc false

Để tạo số ngẫu nhiên, đầu tiên bạn tạo một đối tượng Random:

java

Copy code

```
Random r = new Random();
```

Bạn có thể gọi phương thức nextInt của đối tượng này, truyền vào một số nguyên tối đa. Số trả về sẽ nằm trong khoảng từ 0 (bao gồm) đến số tối đa (không bao gồm). Ví dụ: nếu bạn gọi nextInt(100), bạn sẽ nhận được một số trong khoảng từ 0 đến 99. Bạn có thể cộng thêm 1 vào số đó để có phạm vi từ 1 đến 100.

Tương tác sau đây trong JShell tạo ra một đối tượng Random và sử dụng nó để tạo ra một số số ngẫu nhiên từ 1-10. Lưu ý rằng mỗi lần gọi nextInt sẽ tạo ra một số nguyên ngẫu nhiên mới:

java

Copy code

```
jshell> Random r = new Random();

r ==> java.util.Random@5c7fa833



jshell> r.nextInt(10)

$2 ==> 2



jshell> r.nextInt(10)

$3 ==> 9



jshell> r.nextInt(10)
```

$4 ==> 9

jshell> r.nextInt(10)

$5 ==> 0

jshell> r.nextInt(10)

$6 ==> 5

Hãy xem một chương trình đơn giản chọn số từ 1 đến 10 cho đến khi gặp một số cụ thể. Chúng ta sẽ sử dụng đối tượng Random để tạo các số giả ngẫu nhiên. Vòng lặp của chúng ta sẽ trông giống như sau (trong đó number là giá trị mà người dùng yêu cầu chúng ta tạo):

java

Copy code

```
int result;

while (result != number) {
    result = r.nextInt(10) + 1; // số ngẫu nhiên từ 1–10
    System.out.println("next number = " + result);
```

}

Lưu ý rằng chúng ta phải khai báo biến result bên ngoài vòng lặp while, vì result xuất hiện trong điều kiện kiểm tra của vòng lặp while. Mã trên có cách tiếp cận đúng, nhưng Java sẽ không chấp nhận nó. Mã này sẽ tạo ra một thông báo lỗi rằng biến result có thể chưa được khởi tạo. Đây là một ví dụ về một vòng lặp cần được khởi động trước.

Khởi Tạo Vòng Lặp

Khởi tạo biến trước vòng lặp để "mồi" và đảm bảo rằng vòng lặp được thực thi. Chúng ta muốn đặt biến result thành một giá trị nào đó để vòng lặp được thực thi, nhưng giá trị này không quan trọng miễn là nó giúp chúng ta vào vòng lặp. Tuy nhiên, cần cẩn thận để không đặt nó thành giá trị mà người dùng muốn chúng ta tạo ra. Trong chương trình này, chúng ta xử lý các giá trị từ 1 đến 10, vì vậy chúng ta có thể đặt result thành một giá trị như -1, rõ ràng nằm ngoài phạm vi này. Chúng ta đôi khi gọi đây là một giá trị "giả định" vì chúng ta không thực sự xử lý nó. Sau này trong chương này, chúng ta sẽ thấy một biến thể của vòng lặp while không yêu cầu loại mồi này. Sau đây là giải pháp chương trình hoàn chỉnh:

java

Copy code

```
 1 // Picks random numbers from 1-10 until a given value is chosen.

 2 import java.util.*;
```

```
3
4 public class Pick {
5     public static void main(String[] args) {
6         System.out.println("This program picks numbers from");
7         System.out.println("1 to 10 until a particular");
8         System.out.println("number comes up.");
9         System.out.println();
10         Scanner console = new Scanner(System.in);
11         Random r = new Random();
12         System.out.print("Pick a number between 1 and 10—> ");
13         int number = console.nextInt();
14         int result = –1; // set to –1 to make sure we enter the loop
15         int count = 0;
16         while (result != number) {
17             result = r.nextInt(10) + 1; // random number from 1–10
18             System.out.println("next number = " + result);
19             count++;
```

```
20         }

21         System.out.println("Your number came up after " + count
+ " times");

22     }

23 }
```

Tùy thuộc vào chuỗi số được trả về bởi đối tượng Random, chương trình có thể kết thúc việc chọn số đã cho một cách nhanh chóng, như trong lần chạy mẫu sau:

typescript

Copy code

This program picks numbers from

1 to 10 until a particular

number comes up.

Pick a number between 1 and 10—> 2

next number = 7

next number = 8

next number = 2

Your number came up after 3 times

Cũng có thể chương trình sẽ mất một thời gian để chọn số đó, như trong lần chạy mẫu sau:

java

Copy code

This program picks numbers from

1 to 10 until a particular

number comes up.

Pick a number between 1 and 10—> 10

next number = 9

next number = 7

next number = 7

next number = 5

next number = 8

next number = 8

next number = 1

next number = 5

next number = 1

next number = 9

next number = 7

next number = 10

Your number came up after 12 times

LỖI LẬP TRÌNH PHỔ BIẾN

Sử Dụng Sai Đối Tượng Random

Một đối tượng Random chọn một số nguyên ngẫu nhiên mới mỗi khi phương thức nextInt được gọi. Khi sinh viên cố gắng tạo ra một giá trị ngẫu nhiên có giới hạn, chẳng hạn như một giá trị lẻ, đôi khi họ viết mã như sau:

java

Copy code

```
// đoạn mã này chứa lỗi

Random r = new Random();

if (r.nextInt() % 2 == 0) {

    System.out.println("Even number: " + r.nextInt());

} else {
```

```
    System.out.println("Odd number: " + r.nextInt());

}
```

Đoạn mã trên thất bại trong nhiều trường hợp vì đối tượng Random tạo ra một số nguyên ngẫu nhiên cho kiểm tra if/else, sau đó tạo ra một số khác để sử dụng trong câu lệnh println được chọn để thực thi. Ví dụ, kiểm tra if có thể lấy được giá trị ngẫu nhiên là 47 từ đối tượng Random. Kiểm tra sẽ phát hiện rằng 47 % 2 không bằng 0, vì vậy mã sẽ tiến hành đến câu lệnh else. Câu lệnh println sau đó sẽ thực thi một lần gọi nextInt khác, điều này sẽ trả về một số hoàn toàn khác (giả sử, 128). Kết quả đầu ra của đoạn mã sẽ là tuyên bố kỳ lạ sau:

yaml

Copy code

```
Odd number: 128
```

Giải pháp cho vấn đề này là lưu số nguyên ngẫu nhiên đã chọn vào một biến và chỉ gọi nextInt một lần nữa nếu thực sự cần một số nguyên ngẫu nhiên khác. Đoạn mã sau đây thực hiện nhiệm vụ này:

java

Copy code

```
// đoạn mã này hoạt động chính xác

Random r = new Random();

int n = r.nextInt(); // lưu số ngẫu nhiên vào biến
```

```
if (n % 2 == 0) {
    System.out.println("Even number: " + n);
} else {
    System.out.println("Odd number: " + n);
}
```

Mô phỏng

Khoa học và kỹ thuật truyền thống liên quan nhiều đến tương tác thực tế. Các nhà khoa học thực hiện các thí nghiệm để kiểm tra giả thuyết của họ, và các kỹ sư tạo ra các nguyên mẫu để kiểm tra thiết kế của họ. Tuy nhiên, ngày càng có nhiều nhà khoa học và kỹ sư chuyển sang sử dụng máy tính để tăng năng suất bằng cách thực hiện các mô phỏng trước để khám phá các khả năng trước khi thực hiện thí nghiệm thực tế hoặc xây dựng nguyên mẫu thực tế. Một nhà khoa học máy tính nổi tiếng tên là Jeanette Wing đã lập luận rằng việc tăng cường sử dụng tính toán bởi các nhà khoa học và kỹ sư sẽ dẫn đến việc tư duy tính toán được coi là cơ bản, giống như việc đọc, viết, và số học ngày nay. Từ quan điểm lập trình, hai thành phần chính trong một mô phỏng là các số ngẫu nhiên giả và các vòng lặp. Một số mô phỏng có thể được viết bằng vòng lặp for, nhưng thường chúng ta sử dụng vòng lặp while vì mô phỏng nên được chạy vô hạn cho đến khi một điều kiện nào đó được thỏa mãn. Như một ví dụ đơn giản, hãy xem cách chúng ta mô phỏng việc tung hai con xúc xắc cho đến khi tổng của chúng bằng 7. Chúng ta có thể sử dụng một đối tượng Random để mô phỏng các con xúc xắc, gọi nó một lần cho mỗi con xúc xắc. Chúng ta muốn lặp lại cho đến khi tổng bằng 7 và

chúng ta có thể in các kết quả khác nhau khi chạy mô phỏng. Dưới đây là một nỗ lực đầu tiên tốt:

java

Copy code

```
Random r = new Random();

while (sum != 7) {

    int roll1 = r.nextInt(6) + 1;

    int roll2 = r.nextInt(6) + 1;

    int sum = roll1 + roll2;

    System.out.println(roll1 + " + " + roll2 + " = " + sum);

}
```

Mã trên tạo ra lỗi biên dịch sau:

vbnet

Copy code

```
Dice.java:7: error: cannot find symbol

symbol : variable sum

location: class Dice

    while (sum != 7) {
```

^

1 error

Vấn đề là kiểm tra vòng lặp while tham chiếu đến biến sum, nhưng biến này được khai báo bên trong thân vòng lặp. Chúng ta không thể khai báo biến trong phạm vi nội bộ vì chúng ta cần tham chiếu đến nó trong kiểm tra vòng lặp. Vì vậy, chúng ta phải di chuyển khai báo biến trước vòng lặp. Chúng ta cũng phải cung cấp cho biến một giá trị ban đầu để đảm bảo rằng nó vào vòng lặp. Đây là một ví dụ khác về lúc chúng ta cần khởi tạo vòng lặp:

java

Copy code

```
Random r = new Random();

int sum = 0; // đặt giá trị 0 để đảm bảo vào được vòng lặp

while (sum != 7) {

    int roll1 = r.nextInt(6) + 1;

    int roll2 = r.nextInt(6) + 1;

    sum = roll1 + roll2;

    System.out.println(roll1 + " + " + roll2 + " = " + sum);

}
```

Phiên bản này của mã biên dịch và hoạt động đúng. Dưới đây là một lần thực thi mẫu:

Copy code

1 + 4 = 5

5 + 6 = 11

1 + 3 = 4

4 + 3 = 7

Vòng lặp do/while

Vòng lặp while là vòng lặp vô định tiêu chuẩn, nhưng Java cung cấp nhiều lựa chọn thay thế. Phần này giới thiệu vòng lặp do/while. Các biến thể khác được bao gồm trong Phụ lục C. Như chúng ta đã thấy, vòng lặp while kiểm tra ở "đầu" của vòng lặp, trước khi thực thi câu lệnh kiểm soát của nó. Java có một lựa chọn thay thế gọi là vòng lặp do/while kiểm tra ở "cuối" của vòng lặp. Vòng lặp do/while có cú pháp sau:

java

Copy code

```
do {
    <statement>;
    ...
    <statement>;
} while (<test>);
```

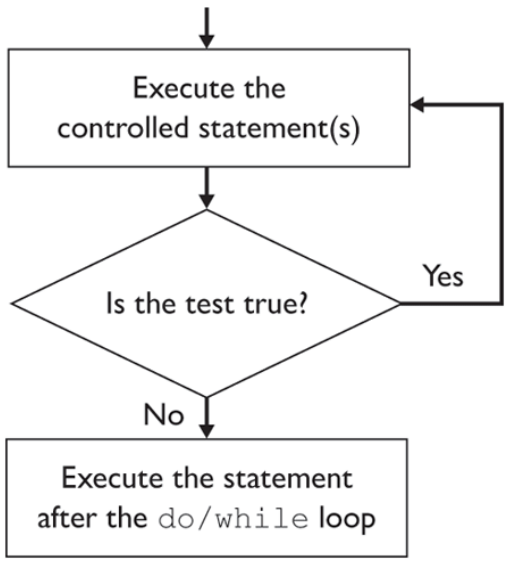
Dưới đây là một đoạn mã mẫu sử dụng vòng lặp do/while:

java

Copy code

```
int number = 1;

do {

    number *= 2;

} while (number <= 200);
```

Vòng lặp này cho kết quả giống như vòng lặp while tương ứng, nhân đôi giá trị của biến number cho đến khi giá trị của nó đạt 256, đó là lũy thừa đầu tiên của 2 lớn hơn 200. Nhưng không giống như vòng lặp while, vòng lặp do/while luôn thực thi câu lệnh kiểm soát của nó ít nhất một lần. Sơ đồ trong Hình 5.2 cho thấy luồng điều khiển trong một vòng lặp do/while.



Hình 5.2 Văn bản thay thế đầy đủ Vòng lặp do/while rất hữu ích trong các tình huống mà bạn biết rằng bạn phải thực hiện vòng lặp ít nhất một lần. Ví dụ, trong phần trước, chúng ta đã viết đoạn mã sau để mô phỏng việc tung hai con xúc xắc cho đến khi bạn nhận được tổng bằng 7:

java

Copy code

```
Random r = new Random();

int sum = 0; // đặt giá trị ban đầu để đảm bảo chúng ta vào được vòng lặp



while (sum != 7) {

    int roll1 = r.nextInt(6) + 1; // tung con xúc xắc thứ nhất

    int roll2 = r.nextInt(6) + 1; // tung con xúc xắc thứ hai

    sum = roll1 + roll2;

    System.out.println(roll1 + " + " + roll2 + " = " + sum);

}
```

Chúng ta đã phải khởi tạo vòng lặp bằng cách đặt sum là 0 để đảm bảo máy tính sẽ vào vòng lặp. Với vòng lặp do/while, chúng ta có thể loại bỏ việc khởi tạo:

java

Copy code

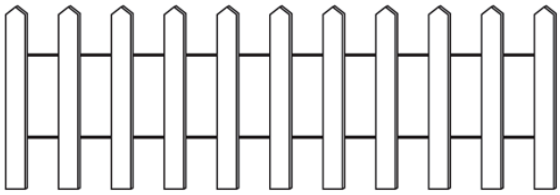
```
Random r = new Random();

int sum;



do {

    int roll1 = r.nextInt(6) + 1; // tung con xúc xắc thứ nhất

    int roll2 = r.nextInt(6) + 1; // tung con xúc xắc thứ hai

    sum = roll1 + roll2;

    System.out.println(roll1 + " + " + roll2 + " = " + sum);

} while (sum != 7);
```

Trong phiên bản này, chúng ta luôn thực hiện thân của vòng lặp ít nhất một lần, điều này giúp gán giá trị cho biến sum trước khi nó đạt đến điều kiện kiểm tra vòng lặp, giờ đây nằm ở cuối vòng lặp. Có rất nhiều vấn đề lập trình mà việc sử dụng vòng lặp do/while là phù hợp. Các vòng lặp này thường hữu ích trong các chương trình tương tác, nơi mà bạn biết rằng bạn muốn thực hiện một việc gì đó ít nhất một lần. Ví dụ, bạn có thể có một vòng lặp cho phép người dùng chơi một trò chơi nhiều lần; bạn có thể khá chắc chắn rằng người dùng sẽ muốn chơi ít nhất một lần. Tương tự, nếu bạn đang chơi một trò chơi đoán với người dùng, bạn sẽ luôn phải nhận ít nhất một lần đoán.

5.2 Thuật toán Fencepost

Một vấn đề lập trình phổ biến liên quan đến một loại vòng lặp đặc biệt được gọi là vòng lặp "fencepost". Hãy xem xét vấn đề sau: Bạn muốn dựng một hàng rào dài 100 yard và muốn cài đặt một cột mỗi 10 yard. Bạn cần bao nhiêu cột? Nếu bạn thực hiện phép chia nhanh trong đầu, bạn có thể nghĩ rằng bạn cần 10 cột, nhưng thực tế bạn cần 11 cột. Đó là vì hàng rào bắt đầu và kết thúc với các cột. Nói cách khác, một hàng rào trông như Hình 5.3.

Hình 5.3 Một hàng rào điển hình



Vì bạn muốn có cột ở cả bên trái và bên phải xa nhất, bạn không thể sử dụng vòng lặp đơn giản sau (nó sẽ không cài đặt cột cuối cùng):

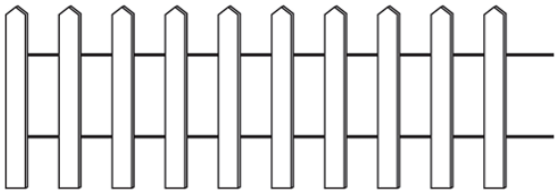
java

Copy code

```
for (the length of the fence) {

    plant a post;

    attach some wire;

}
```

Nếu bạn sử dụng vòng lặp trên, bạn sẽ có một hàng rào trông như Hình 5.4.

Hình 5.4 Một hàng rào bị lỗi



Việc thay đổi thứ tự của hai thao tác không giúp ích gì, vì khi đó bạn sẽ bỏ lỡ cột đầu tiên. Vấn đề với vòng lặp này là nó tạo ra số lượng cột giống như số đoạn dây, nhưng chúng ta biết rằng cần thêm một cột nữa. Đó là lý do tại sao vấn đề này đôi khi còn được gọi là vấn đề "vòng lặp rưỡi" — chúng ta muốn thực hiện một nửa vòng lặp này (trồng một cột) thêm một lần nữa. Một giải pháp là trồng một trong các cột trước hoặc sau vòng lặp. Giải pháp thông thường là thực hiện điều đó trước:

java

Copy code

```
plant a post;

for (the length of the fence) {

    attach some wire;

    plant a post;

}
```

Lưu ý rằng thứ tự của hai thao tác trong thân vòng lặp giờ đã được đảo ngược vì cột đầu tiên được trồng trước khi vào vòng lặp. Ví dụ đơn giản, hãy xem xét vấn đề viết ra các số nguyên từ

1 đến 10, cách nhau bởi dấu phẩy. Nói cách khác, chúng ta muốn có kết quả sau:

Copy code

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Nhiệm vụ này là một ví dụ cổ điển về vấn đề "fencepost" vì chúng ta muốn viết ra 10 số nhưng chỉ có 9 dấu phẩy. Trong thuật ngữ "fencepost", việc viết một số là phần "cột" của nhiệm vụ và việc viết dấu phẩy là phần "dây". Vì vậy, thực hiện mã giả mà chúng ta đã phát triển, chúng ta in số đầu tiên trước khi vào vòng lặp và đảo ngược thứ tự các thao tác trong vòng lặp bằng cách in một dấu phẩy trước một số:

java

Copy code

```
System.out.print(1);

for (int i = 2; i <= 10; i++) {

    System.out.print(", " + i);

}

System.out.println();
```

Fencepost với câu lệnh if

Nhiều vòng lặp "fencepost" mà bạn viết sẽ yêu cầu thực hiện điều kiện. Thực tế, vấn đề "fencepost" tự nó có thể được giải quyết

bằng một câu lệnh if. Hãy nhớ rằng giải pháp cổ điển cho "fencepost" là xử lý cột đầu tiên trước khi vòng lặp bắt đầu:

java

Copy code

```
plant a post;

for (the length of the fence) {

    attach some wire;

    plant a post;

}
```

Giải pháp này giải quyết vấn đề, nhưng nó có thể gây nhầm lẫn vì bên trong vòng lặp, các bước dường như bị đảo ngược thứ tự. Bạn có thể sử dụng câu lệnh if và giữ nguyên thứ tự ban đầu của các bước:

java

Copy code

```
for (the length of the fence) {

    plant a post;

    if (this isn't the last post) {

        attach some wire;
```

```
    }

}
```

Biến thể này không được sử dụng thường xuyên như giải pháp cổ điển vì nó liên quan đến cả việc kiểm tra vòng lặp và kiểm tra bên trong vòng lặp. Thường thì các kiểm tra này gần như giống nhau, vì vậy việc kiểm tra cùng một điều hai lần mỗi khi vòng lặp thực thi là không hiệu quả. Nhưng sẽ có những tình huống mà bạn có thể sử dụng cách tiếp cận này. Ví dụ, trong cách tiếp cận cổ điển, các dòng mã tương ứng với việc trồng một cột bị lặp lại. Nếu bạn đang viết một chương trình mà bước này yêu cầu nhiều mã, bạn có thể quyết định rằng việc đặt câu lệnh if bên trong vòng lặp là cách tiếp cận tốt hơn, ngay cả khi nó dẫn đến việc kiểm tra nhiều hơn. Ví dụ, hãy xem xét việc viết một phương thức gọi là multiprint để in một chuỗi một số lần cụ thể. Giả sử bạn muốn kết quả được in trên một dòng riêng, trong ngoặc vuông, và cách nhau bởi dấu phẩy. Dưới đây là hai lệnh gọi ví dụ:

java

Copy code

```
multiprint("please", 4);

multiprint("beetlejuice", 3);
```

Bạn sẽ mong đợi các lệnh gọi này tạo ra kết quả sau:

csharp

Copy code

```
[please, please, please, please]
```

[beetlejuice, beetlejuice, beetlejuice]

Nỗ lực đầu tiên của bạn để viết phương thức này có thể là một vòng lặp đơn giản in dấu ngoặc vuông bên ngoài vòng lặp và in chuỗi cùng với dấu phẩy bên trong vòng lặp:

java

Copy code

```
public static void multiprint(String s, int times) {
    System.out.print("[");
    for (int i = 1; i <= times; i++) {
        System.out.print(s + ", ");
    }
    System.out.println("]");
}
```

Thật không may, đoạn mã này tạo ra một dấu phẩy dư thừa sau giá trị cuối cùng:

csharp

Copy code

```
[please, please, please, please, ]

[beetlejuice, beetlejuice, beetlejuice, ]
```

Vì dấu phẩy là dấu phân cách, bạn muốn in một chuỗi nhiều hơn dấu phẩy (ví dụ: hai dấu phẩy để ngăn cách ba lần xuất hiện của "beetlejuice"). Bạn có thể sử dụng giải pháp cổ điển cho vấn đề "fencepost" để đạt được hiệu quả này bằng cách in một chuỗi bên ngoài vòng lặp và đảo ngược thứ tự in bên trong vòng lặp:

java

Copy code

```
public static void multiprint(String s, int times) {
    System.out.print("[" + s);
    for (int i = 2; i <= times; i++) {
        System.out.print(", " + s);
    }
    System.out.println("]");
}
```

Lưu ý rằng vì bạn đang in một trong những chuỗi trước khi vòng lặp bắt đầu, bạn phải điều chỉnh vòng lặp để nó không in nhiều chuỗi như trước đây. Điều chỉnh biến i bắt đầu từ 2 để tính đến giá trị đầu tiên được in trước vòng lặp. Tuy nhiên, giải pháp này cũng không hoạt động đúng. Hãy xem xét điều gì xảy ra khi bạn yêu cầu phương thức in một chuỗi không lần nào, như trong:

java

Copy code

multiprint("please don't", 0);

Lệnh này tạo ra kết quả sai:

csharp

Copy code

[please don't]

Bạn muốn người dùng có thể yêu cầu in chuỗi không lần nào, vì vậy phương thức không nên tạo ra kết quả sai. Vấn đề là giải pháp cổ điển cho vấn đề "fencepost" liên quan đến việc in một giá trị trước khi vòng lặp bắt đầu. Để làm cho phương thức hoạt động đúng cho trường hợp không lần, bạn có thể bao gồm câu lệnh if/else:

java

Copy code

```
public static void multiprint(String s, int times) {
    if (times == 0) {
        System.out.println("[]");
    } else {
        System.out.print("[" + s);
        for (int i = 2; i <= times; i++) {
            System.out.print(", " + s);
```

```
        }
        System.out.println("]");
    }
}
```

Ngoài ra, bạn có thể đặt câu lệnh if bên trong vòng lặp (cách tiếp cận kiểm tra kép):

java

Copy code

```
public static void multiprint(String s, int times) {
    System.out.print("[");
    for (int i = 1; i <= times; i++) {
        System.out.print(s);
        if (i < times) {
            System.out.print(", ");
        }
    }
    System.out.println("]");
}
```

Mặc dù phiên bản mã trước đó thực hiện kiểm tra tương tự hai lần trong mỗi lần lặp, nhưng nó đơn giản hơn so với việc sử dụng giải pháp "fencepost" cổ điển và trường hợp đặc biệt của nó. Không có giải pháp nào tốt hơn giải pháp khác, vì cả hai đều có sự đánh đổi. Nếu bạn nghĩ rằng mã sẽ được thực thi thường xuyên và vòng lặp sẽ lặp lại nhiều lần, bạn có thể nghiêng về việc sử dụng giải pháp hiệu quả. Ngược lại, bạn có thể chọn mã đơn giản hơn.

Vòng lặp Sentinel

Giả sử bạn muốn đọc một loạt các số từ người dùng và tính tổng của chúng. Bạn có thể yêu cầu người dùng biết trước số lượng số cần đọc, như chúng ta đã làm trong chương trước, nhưng điều đó không phải lúc nào cũng thuận tiện. Nếu người dùng có một danh sách dài các số cần nhập và không đếm trước chúng thì sao? Một cách để giải quyết vấn đề không cần yêu cầu số lượng là chọn một giá trị nhập đặc biệt để báo hiệu kết thúc việc nhập. Chúng ta gọi giá trị này là **sentinel**.

Sentinel
Giá trị đặc biệt báo hiệu kết thúc việc nhập.

Ví dụ, bạn có thể yêu cầu người dùng nhập giá trị -1 để dừng việc nhập số. Nhưng làm thế nào để bạn cấu trúc mã của mình để sử dụng sentinel này? Nói chung, bạn sẽ muốn sử dụng cách tiếp cận sau:

java

Copy code

```
sum = 0;

while (chưa gặp sentinel) {

    prompt and read;

    add it to the sum;

}
```

Cách tiếp cận này không hoàn toàn đúng. Ví dụ, giả sử người dùng nhập các số 10, 42, 5, và -1. Như mã giả trên đã chỉ ra, chúng ta sẽ nhắc và đọc từng giá trị trong bốn giá trị này và thêm chúng vào tổng cho đến khi gặp giá trị sentinel là -1. Chúng ta khởi tạo tổng là 0, do đó phép tính sẽ là (10 + 42 + 5 + -1), tức là 56. Nhưng kết quả đúng phải là 57. Giá trị sentinel -1 không nên được bao gồm trong tổng. Vấn đề này là một vấn đề cổ điển "fencepost" hoặc "vòng lặp rưỡi": Bạn muốn nhắc và đọc một loạt các số, bao gồm cả sentinel, và bạn muốn tính tổng hầu hết các số, nhưng không muốn thêm sentinel vào tổng. Giải pháp "fencepost" thông thường hoạt động: Chúng ta chèn lệnh nhắc và đọc đầu tiên trước vòng lặp và đảo ngược thứ tự của hai bước trong thân vòng lặp:

java

Copy code

```
sum = 0;

prompt and read;

while (chưa gặp sentinel) {
```

```
    add it to the sum;
    prompt and read;
}
```

Bạn có thể cải tiến mã giả này bằng cách giới thiệu một biến cho số được đọc từ người dùng:

java

Copy code

```
sum = 0;
prompt and read a value into n;
while (n không phải là sentinel) {
    add n to the sum;
    prompt and read a value into n;
}
```

Mã giả này khá dễ dịch sang mã Java:

java

Copy code

```
Scanner console = new Scanner(System.in);
int sum = 0;
```

```
System.out.print("next integer (–1 to quit)? ");

int number = console.nextInt();

while (number != -1) {

    sum += number;

    System.out.print("next integer (–1 to quit)? ");

    number = console.nextInt();

}

System.out.println("sum = " + sum);
```

Khi mã trên được thực thi, quá trình tương tác có thể trông như thế này:

vbnet

Copy code

```
next integer (–1 to quit)? 10

next integer (–1 to quit)? 20

next integer (–1 to quit)? 30

next integer (–1 to quit)? 40

next integer (–1 to quit)? 50

next integer (–1 to quit)? 60
```

next integer (–1 to quit)? -1

sum = 210

Một số học sinh nhận thấy rằng chương trình sẽ hoạt động tương tự nếu thay vì có lệnh nhắc và đọc trước khi vòng lặp bắt đầu, chúng ta khởi tạo number là 0. Điều đó đúng trong trường hợp đặc biệt này, nhưng nó không tổng quát. Ví dụ, nếu bạn muốn tính giá trị lớn nhất hoặc nhỏ nhất, bạn sẽ không muốn bao gồm 0 như một giá trị để xem xét. Vì vậy, mặc dù đoạn mã cụ thể này có thể được đơn giản hóa, chúng ta ưa thích trình bày một giải pháp hoạt động cho tất cả các nhiệm vụ tích lũy.

Kiểu dữ liệu boolean

George Boole là một nhà logic học tài giỏi đến mức Java có một kiểu dữ liệu được đặt theo tên của ông. Kiểu dữ liệu boolean trong Java được sử dụng để mô tả các quan hệ logic đúng/sai. Hãy nhớ rằng boolean là một trong những kiểu dữ liệu nguyên thủy, giống như int, double và char.

Người mới thường thắc mắc tại sao các nhà khoa học máy tính lại quan tâm đến logic. Câu trả lời là logic là nền tảng của lập trình, giống như vật lý là nền tảng của kỹ thuật. Các kỹ sư học vật lý vì họ muốn xây dựng các hiện vật trong thế giới thực theo quy luật của vật lý. Nếu bạn không hiểu vật lý, có thể bạn sẽ xây một cây cầu bị sụp đổ. Các nhà khoa học máy tính cũng xây dựng các hiện vật, nhưng trong một thế giới ảo được điều khiển bởi các quy luật của logic. Nếu bạn không hiểu logic, có thể bạn sẽ xây dựng các chương trình máy tính bị sụp đổ.

Mà không nhận ra, bạn đã sử dụng các kiểu boolean. Tất cả các cấu trúc điều khiển mà chúng ta đã xem xét—câu lệnh if/else, vòng lặp for và vòng lặp while—đều được điều khiển bởi các biểu thức chỉ định các bài kiểm tra. Ví dụ, biểu thức number % 2 == 0 là một kiểm tra về tính chia hết cho 2. Đây cũng là một biểu thức boolean. Biểu thức boolean nhằm nắm bắt các khái niệm về đúng và sai, vì vậy không có gì ngạc nhiên khi miền của kiểu boolean chỉ có hai giá trị: true và false. Từ true và false là các từ khóa trong Java. Chúng là các giá trị literal của kiểu boolean. Tất cả các biểu thức boolean khi được đánh giá sẽ trả về một trong hai giá trị literal này. Đừng nhầm lẫn các giá trị đặc biệt này với các chuỗi literal "true" và "false". Bạn không cần dấu ngoặc kép khi tham chiếu đến các literal boolean.

Khi bạn viết một chương trình xử lý các giá trị số, bạn thường muốn tính một giá trị và lưu trữ nó trong một biến hoặc viết một phương thức nắm bắt một công thức phức tạp. Chúng ta thường làm điều tương tự với kiểu boolean. Chúng ta có thể muốn ghi lại một giá trị boolean trong một biến và thường viết các phương thức trả về kết quả boolean. Ví dụ, chúng ta đã thấy rằng lớp String có các phương thức trả về kết quả boolean, bao gồm startsWith, endsWith, equals, và equalsIgnoreCase.

Để hiểu rõ hơn, hãy nhớ lại những gì những thuật ngữ này có nghĩa với kiểu int. Các giá trị literal của kiểu int bao gồm 0, 1, 2, và như vậy. Bởi vì đây là các literal của kiểu int, bạn có thể viết các biểu thức như các biểu thức sau với chúng:

java

Copy code

```
int number1 = 1;
```

int number2 = 0;

Hãy xem xét những gì bạn có thể làm với các biến của kiểu boolean. Giả sử bạn định nghĩa hai biến boolean, test1 và test2. Các biến này chỉ có thể có hai giá trị: true và false. Bạn có thể nói:

java

Copy code

boolean test1 = true;

boolean test2 = false;

Bạn cũng có thể viết một câu lệnh sao chép giá trị của một biến boolean sang biến khác, giống như với các biến của bất kỳ kiểu nào khác:

java

Copy code

test1 = test2;

Hơn nữa, bạn biết rằng câu lệnh gán có thể sử dụng các biểu thức như:

java

Copy code

number1 = 2 + 2;

và rằng các kiểm tra đơn giản mà bạn đã sử dụng là các biểu thức boolean. Điều đó có nghĩa là bạn có thể viết các câu lệnh như các câu lệnh sau:

java

Copy code

```
test1 = (2 + 2 == 4);

test2 = (3 * 100 < 250);
```

Những câu lệnh gán này nói rằng, "Hãy đặt biến boolean này theo giá trị đúng hoặc sai được trả về bởi kiểm tra sau đây." Câu lệnh đầu tiên đặt biến test1 thành true, bởi vì kiểm tra này được đánh giá là đúng. Câu lệnh thứ hai đặt biến test2 thành false, bởi vì kiểm tra thứ hai được đánh giá là sai. Bạn không cần phải sử dụng dấu ngoặc đơn, nhưng chúng làm cho các câu lệnh dễ đọc hơn.

Rõ ràng, câu lệnh gán là một trong những phép toán bạn có thể thực hiện trên các biến kiểu boolean.

Các toán tử logic

Trong Java, bạn có thể tạo ra các biểu thức Boolean phức tạp bằng cách sử dụng các toán tử logic, được hiển thị trong Bảng 5.3.

Bảng 5.3: Các toán tử logic

Toán tử	Ý nghĩa	Ví dụ	Giá trị
&&	AND (phép hội)	(2 == 2) && (3 < 4)	true
`		`	OR (phép tuyển)
!	NOT (phép phủ định)	!(2 == 2)	false

Toán tử NOT (!) đảo ngược giá trị đúng/sai của toán hạng. Nếu một biểu thức được đánh giá là true, phủ định của nó sẽ là false, và ngược lại. Bạn có thể diễn đạt mối quan hệ này trong một bảng chân lý. Bảng chân lý dưới đây có hai cột, một cột dành cho biến và một cột dành cho phủ định của nó. Đối với mỗi giá trị của biến, bảng hiển thị giá trị tương ứng của phủ định.

Bảng chân lý cho NOT (!)

p	!p
true	false
false	true

Ngoài toán tử phủ định, còn có hai kết nối logic bạn sẽ sử dụng, AND (&&) và OR (||). Bạn sử dụng các kết nối này để liên kết hai biểu thức Boolean, tạo ra một biểu thức Boolean mới. Bảng chân lý sau đây cho thấy toán tử AND chỉ được đánh giá là true khi cả hai toán hạng đều là true.

Bảng chân lý cho AND (&&)

p	q	p && q
true	true	true
true	false	false
false	true	false
false	false	false

Bảng chân lý sau đây cho thấy toán tử OR được đánh giá là true trừ khi cả hai toán hạng đều là false.

Bảng chân lý cho OR (||)

p	q	p || q
true	true	true
true	false	true
false	true	true
false	false	false

Toán tử OR trong Java có ý nghĩa hơi khác so với từ "hoặc" trong tiếng Anh. Trong tiếng Anh, bạn có thể nói "Tôi sẽ học tối nay hoặc tôi sẽ đi xem phim." Một trong hai điều đó sẽ đúng, nhưng không phải cả hai. Toán tử OR giống như biểu thức "và/hoặc" trong tiếng Anh: Nếu một hoặc cả hai toán hạng là true, toàn bộ mệnh đề là true.

Bạn thường sử dụng các toán tử logic khi điều bạn muốn diễn đạt không thể giảm xuống thành một bài kiểm tra đơn lẻ. Ví dụ, như chúng ta đã thấy trong chương trước, nếu bạn muốn thực hiện một thao tác cụ thể khi một số nằm trong khoảng từ 1 đến 10, bạn có thể viết:

java

Copy code

```
if (number >= 1) {
    if (number <= 10) {
        doSomething();
    }
}
```

Nhưng bạn có thể nói điều này dễ dàng hơn bằng cách sử dụng AND logic:

java

Copy code

```
if (number >= 1 && number <= 10) {
    doSomething();
}
```

Mọi người thường xuyên sử dụng các từ "và" và "hoặc," nhưng Java chỉ cho phép bạn sử dụng chúng theo nghĩa logic nghiêm ngặt. Hãy cẩn thận không viết mã như sau:

java

Copy code

```
// đoạn mã này không thể biên dịch

if (x == 1 || 2 || 3) {

    doSomething();

}
```

Trong tiếng Anh, chúng ta sẽ đọc câu này là "x bằng 1 hoặc 2 hoặc 3," điều này có ý nghĩa đối với chúng ta, nhưng không có ý nghĩa đối với Java. Bạn cũng có thể bị cám dỗ viết mã như sau:

java

Copy code

```
// đoạn mã này không thể biên dịch

if (1 <= x <= 10) {

    doSomethingElse();

}
```

Trong toán học, biểu thức này sẽ có ý nghĩa và sẽ kiểm tra liệu x có nằm trong khoảng từ 1 đến 10 bao gồm cả hai đầu hay không. Tuy nhiên, biểu thức này không có ý nghĩa trong Java. Bạn chỉ có thể sử dụng các toán tử AND và OR logic để kết hợp một chuỗi các biểu thức Boolean. Nếu không, máy tính sẽ không hiểu bạn đang muốn nói gì. Để diễn đạt ý "1 hoặc 2 hoặc 3", hãy kết hợp ba biểu thức Boolean khác nhau với các toán tử OR logic:

java

Copy code

```
if (x == 1 || x == 2 || x == 3) {

    doSomething();

}
```

Để diễn đạt ý "trong khoảng từ 1 đến 10 bao gồm cả hai đầu", hãy kết hợp hai biểu thức Boolean với AND logic:

java

Copy code

```
if (1 <= x && x <= 10) {

    doSomethingElse();

}
```

Bây giờ khi chúng ta đã giới thiệu các toán tử logic AND, OR và NOT, đã đến lúc xem lại bảng thứ tự ưu tiên của chúng. Toán tử NOT xuất hiện ở đầu bảng, với mức độ ưu tiên cao nhất. Hai toán tử logic còn lại có mức độ ưu tiên khá thấp, thấp hơn các toán tử số học và so sánh, nhưng cao hơn các toán tử gán. Toán tử AND có mức độ ưu tiên cao hơn một chút so với toán tử OR. Bảng 5.4 bao gồm các toán tử mới này.

Bảng 5.4: Thứ tự ưu tiên của toán tử Java

Mô tả	Toán tử
Toán tử đơn nguyên	!, ++, —, +, –
Toán tử nhân chia	*, /, %
Toán tử cộng trừ	+, –
Toán tử quan hệ	<, >, <=, >=
Toán tử bằng nhau	==, !=
Toán tử logic AND	&&
Toán tử logic OR	`
Toán tử gán	=, +=, –=, *=, /=, %=, &&=, `

Theo các quy tắc ưu tiên này, khi Java đánh giá một biểu thức như biểu thức sau, máy tính sẽ đánh giá NOT trước, AND thứ hai, và sau đó là OR:

java

Copy code

```
if (test1 || !test2 && test3) {

    doSomething();

}
```

Đánh giá ngắn mạch

Trong phần này, chúng ta sẽ khám phá việc sử dụng các toán tử logic để giải quyết một tác vụ lập trình phức tạp, và chúng ta sẽ giới thiệu một tính chất quan trọng của các toán tử này. Chúng ta sẽ viết một phương thức có tên là firstWord nhận một chuỗi String làm tham số và trả về từ đầu tiên trong chuỗi đó. Để giữ mọi thứ đơn giản, chúng ta sẽ chấp nhận quy ước rằng một chuỗi String được tách ra thành các từ riêng lẻ bởi khoảng trắng. Nếu chuỗi không có từ nào, phương thức nên trả về một chuỗi rỗng. Dưới đây là một vài ví dụ gọi phương thức:

Gọi phương thức	**Giá trị trả về**
firstWord("four score and seven years")	"four"
firstWord("all-one-word-here")	"all-one-word-here"
firstWord(" lots of space here")	"lots"
firstWord(" ")	" "

Hãy nhớ rằng chúng ta có thể gọi phương thức substring để lấy ra một phần của chuỗi. Chúng ta truyền hai tham số vào phương thức substring: chỉ số bắt đầu của phần chuỗi con và chỉ số ngay sau phần kết thúc của chuỗi con. Nếu chuỗi được lưu trữ trong một biến có tên là s, nhiệm vụ của chúng ta về cơ bản giảm xuống các bước sau:

1. Đặt start là chỉ số đầu tiên của từ.
2. Đặt stop là chỉ số ngay sau từ.
3. Trả về s.substring(start, stop).

Ở mức độ đơn giản ban đầu, hãy giả sử rằng chỉ số bắt đầu là 0. Chỉ số bắt đầu này sẽ không hoạt động đối với các chuỗi bắt đầu bằng khoảng trắng, nhưng nó sẽ cho phép chúng ta tập trung vào bước thứ hai trong mã giả. Xem xét một chuỗi bắt đầu bằng "four score". Nếu chúng ta kiểm tra từng ký tự riêng lẻ của chuỗi và chỉ số của chúng, chúng ta sẽ thấy mô hình sau đây:

java
Copy code
"four score"

Đánh giá ngắn mạch

Trong phần này, chúng ta sẽ khám phá việc sử dụng các toán tử logic để giải quyết một tác vụ lập trình phức tạp và giới thiệu một tính chất quan trọng của các toán tử này. Chúng ta sẽ viết một phương thức gọi là firstWord nhận một chuỗi String làm tham số và trả về từ đầu tiên trong chuỗi đó. Để giữ mọi thứ đơn giản, chúng ta sẽ sử dụng quy ước rằng một chuỗi String được chia thành các từ riêng lẻ bởi khoảng trắng. Nếu chuỗi không có từ nào, phương thức nên trả về một chuỗi rỗng. Dưới đây là một số ví dụ gọi phương thức:

Gọi phương thức	**Giá trị trả về**
firstWord("four score and seven years")	"four"
firstWord("all-one-word-here")	"all-one-word-here"
firstWord(" lots of space here")	"lots"
firstWord(" ")	" "

Chúng ta có thể sử dụng phương thức substring để lấy ra một phần của chuỗi. Để làm điều này, ta truyền hai tham số vào phương thức substring: chỉ số bắt đầu của phần chuỗi con và chỉ số ngay sau phần kết thúc của chuỗi con. Nếu chuỗi được lưu trữ trong biến s, nhiệm vụ của chúng ta là:

1. Đặt start là chỉ số đầu tiên của từ.
2. Đặt stop là chỉ số ngay sau từ.
3. Trả về s.substring(start, stop).

Giả sử rằng chỉ số bắt đầu là 0. Chỉ số này sẽ không hoạt động đối với các chuỗi bắt đầu bằng khoảng trắng, nhưng nó sẽ cho phép chúng ta tập trung vào bước thứ hai trong mã giả. Xem xét một chuỗi bắt đầu bằng "four score". Nếu kiểm tra từng ký tự của chuỗi và chỉ số của chúng, ta sẽ thấy mô hình sau:

java

Copy code

"four score"

Để đặt stop ở vị trí ngay sau từ đầu tiên, chúng ta cần tìm khoảng trắng đầu tiên trong chuỗi. Chúng ta sử dụng vòng lặp while để lặp qua các ký tự cho đến khi gặp khoảng trắng:
java
Copy code
```
int stop = 0;
while (s.charAt(stop) != ' ') {
    stop++;
}
```
Mã này dễ dàng chuyển đổi sang Java. Kết hợp với giả định rằng start là 0, chúng ta có:
java
Copy code
```
public static String firstWord(String s) {
    int start = 0;
    int stop = 0;
    while (s.charAt(stop) != ' ') {
        stop++;
    }
    return s.substring(start, stop);
}
```
Phiên bản này hoạt động cho nhiều trường hợp, bao gồm chuỗi mẫu của chúng ta, nhưng không hoạt động cho tất cả các chuỗi. Nó có hai hạn chế chính. Đầu tiên, chúng ta giả định rằng chuỗi không bắt đầu bằng khoảng trắng, do đó cần phải sửa hạn chế này. Thứ hai, phiên bản này không hoạt động cho chuỗi chỉ có một từ. Ví dụ, nếu chúng ta thực hiện với chuỗi "four", nó sẽ gây ra ngoại lệ StringIndexOutOfBoundsException vì chỉ số 4 không hợp lệ. Ngoại lệ này xảy ra vì mã của chúng ta giả định rằng sẽ tìm thấy một khoảng trắng, nhưng không có khoảng trắng trong chuỗi "four". Vì vậy, stop được tăng lên đến 4, và một ngoại lệ bị ném ra vì không có ký tự tại chỉ số 4. Điều này đôi khi được gọi là “chạy khỏi cuối chuỗi.”

Để khắc phục vấn đề này, chúng ta cần kết hợp kiểm tra liên quan đến độ dài của chuỗi. Nhiều người mới bắt đầu cố gắng làm điều này bằng cách sử dụng sự kết hợp của while và if, như trong đoạn mã sau:
java
Copy code
```
int stop = 0;
while (stop < s.length()) {
    if (s.charAt(stop) != ' ') {
        stop++;
    }
}
```
Mã này hoạt động cho chuỗi một từ như "four" vì khi stop bằng với độ dài của chuỗi, chúng ta thoát khỏi vòng lặp. Tuy nhiên, nó không hoạt động cho các trường hợp có nhiều từ như "four score". Chúng ta bị mắc kẹt trong vòng lặp vô tận vì khi stop bằng 4, chúng ta ngừng tăng giá trị stop, nhưng vẫn bị mắc kẹt trong vòng lặp vì điều kiện kiểm tra yêu cầu tiếp tục chừng nào stop nhỏ hơn độ dài của chuỗi. Cách tiếp cận này đã gây ra lỗi nổi tiếng vào ngày 31 tháng 12 năm 2008, khi các máy nghe nhạc Zune trên toàn thế giới ngừng hoạt động.
Điều cần nhận ra là trong trường hợp này, chúng ta cần sử dụng hai điều kiện khác nhau để kiểm soát vòng lặp. Chúng ta chỉ muốn tiếp tục tăng giá trị stop nếu biết rằng chưa gặp khoảng trắng và chưa đạt đến cuối chuỗi. Chúng ta có thể diễn đạt ý tưởng đó bằng cách sử dụng toán tử logic AND:
java
Copy code
```
int stop = 0;
while (s.charAt(stop) != ' ' && stop < s.length()) {
    stop++;
}
```

Thật không may, ngay cả kiểm tra này cũng không hoạt động. Nó diễn đạt đúng hai điều kiện, vì chúng ta muốn chắc chắn rằng chưa gặp khoảng trắng và muốn đảm bảo rằng chưa đạt đến cuối chuỗi. Nhưng hãy nghĩ về điều gì xảy ra khi chúng ta đến cuối chuỗi. Giả sử s là "four" và stop bằng 3. Chúng ta thấy rằng ký tự tại chỉ số 3 không phải là khoảng trắng và stop nhỏ hơn độ dài của chuỗi, vì vậy chúng ta tăng thêm một lần nữa và stop trở thành 4. Khi vòng lặp quay lại, chúng ta kiểm tra xem s.charAt(4) có phải là khoảng trắng không. Kiểm tra này gây ra ngoại lệ vì chỉ số không hợp lệ. Chúng ta cũng kiểm tra xem stop có nhỏ hơn 4 không, điều này không đúng, nhưng kiểm tra đó đến quá muộn để tránh ngoại lệ.
Java cung cấp một giải pháp cho tình huống này. Các toán tử logic && và || sử dụng đánh giá ngắn mạch.

Đánh giá ngắn mạch
Tính chất của các toán tử logic && và || ngăn không cho toán hạng thứ hai được đánh giá nếu kết quả tổng thể đã rõ ràng từ giá trị của toán hạng đầu tiên. Trong trường hợp của chúng ta, chúng ta đang thực hiện hai phép kiểm tra khác nhau và yêu cầu kết quả logic AND của hai phép kiểm tra này. Nếu bất kỳ phép kiểm tra nào thất bại, kết quả tổng thể sẽ là false, do đó nếu phép kiểm tra đầu tiên thất bại, không cần thực hiện phép kiểm tra thứ hai. Vì đánh giá ngắn mạch—nghĩa là kết quả tổng thể đã rõ ràng từ phép kiểm tra đầu tiên—chúng ta không thực hiện phép kiểm tra thứ hai. Nói cách khác, việc thực hiện và đánh giá phép kiểm tra thứ hai bị ngăn chặn (ngắn mạch) bởi thực tế là phép kiểm tra đầu tiên thất bại.
Điều này có nghĩa là chúng ta cần đảo ngược thứ tự của hai phép kiểm tra:
java
Copy code
int stop = 0;

```
while (stop < s.length() && s.charAt(stop) != ' ') {
    stop++;
}
```
Nếu chúng ta thực hiện lại cùng một kịch bản với stop bằng 3, chúng ta sẽ vượt qua cả hai phép kiểm tra này và tăng stop lên 4. Sau đó, khi vòng lặp tiếp tục, chúng ta đầu tiên kiểm tra xem stop có nhỏ hơn s.length() không. Nếu không, phép kiểm tra này sẽ đánh giá là false. Kết quả là, Java biết rằng biểu thức tổng thể sẽ đánh giá là false và không bao giờ đánh giá phép kiểm tra thứ hai. Thứ tự các sự kiện này ngăn ngừa ngoại lệ xảy ra, vì chúng ta không bao giờ kiểm tra xem s.charAt(4) có phải là khoảng trắng không.
Giải pháp này cho chúng ta một phiên bản thứ hai của phương thức:
java
Copy code
```
public static String firstWord(String s) {
    int start = 0;
    int stop = 0;
    while (stop < s.length() && s.charAt(stop) != ' ') {
        stop++;
    }
    return s.substring(start, stop);
}
```
Nhưng nhớ rằng chúng ta đã giả định rằng từ đầu tiên bắt đầu ở vị trí 0. Điều này không nhất thiết phải đúng. Ví dụ, nếu chúng ta truyền một chuỗi bắt đầu bằng nhiều khoảng trắng, phương thức này sẽ trả về một chuỗi rỗng. Chúng ta cần sửa đổi mã để nó bỏ qua bất kỳ khoảng trắng dẫn đầu nào. Để đạt được mục tiêu đó, cần thêm một vòng lặp khác. Dưới đây là mã mẫu đầu tiên:
java
Copy code
```
int start = 0;
```

```
while (s.charAt(start) == ' ') {
    start++;
}
```
Mã này hoạt động cho hầu hết các chuỗi, nhưng nó thất bại trong hai trường hợp quan trọng. Kiểm tra vòng lặp giả định rằng chúng ta sẽ tìm thấy một ký tự không phải khoảng trắng. Điều gì xảy ra nếu chuỗi chỉ toàn khoảng trắng? Trong trường hợp đó, chúng ta sẽ chạy khỏi cuối chuỗi, tạo ra ngoại lệ StringIndexOutOfBoundsException. Và nếu chuỗi rỗng từ đầu? Chúng ta sẽ gặp lỗi ngay lập tức khi hỏi về s.charAt(0), vì không có ký tự nào tại chỉ số 0.
Chúng ta có thể quyết định rằng các trường hợp này là lỗi. Rốt cuộc, làm sao bạn có thể trả về từ đầu tiên nếu không có từ nào? Do đó, chúng ta có thể ghi nhận một điều kiện tiên quyết rằng chuỗi phải chứa ít nhất một ký tự không phải khoảng trắng, và ném ngoại lệ nếu phát hiện ra không có. Một cách tiếp cận khác là trả về một chuỗi rỗng trong những trường hợp này.
Để xử lý khả năng chuỗi rỗng, chúng ta cần sửa đổi vòng lặp của mình để kết hợp kiểm tra độ dài của chuỗi. Nếu thêm kiểm tra này vào cuối phép kiểm tra của vòng lặp while, chúng ta sẽ có mã sau:
java
Copy code
```
int start = 0;
while (s.charAt(start) == ' ' && start < s.length()) {
    start++;
}
```
Nhưng mã này có cùng một lỗi như trước. Nó được cho là ngăn chặn các vấn đề khi start bằng với độ dài của chuỗi, nhưng khi tình huống này xảy ra, một ngoại lệ StringIndexOutOfBoundsException sẽ bị ném ra trước khi máy tính đến phép kiểm tra độ dài của chuỗi. Vì vậy, các phép kiểm

tra này cũng cần được đảo ngược để tận dụng đánh giá ngắn mạch:
java
Copy code
```
int start = 0;
while (start < s.length() && s.charAt(start) == ' ') {
    start++;
}
```
Để kết hợp những dòng mã này với mã trước, chúng ta phải thay đổi khởi tạo của stop. Chúng ta không còn muốn tìm kiếm từ đầu chuỗi nữa. Thay vào đó, cần khởi tạo stop bằng với start. Kết hợp các phần này lại, chúng ta có phiên bản sau của phương thức:
java
Copy code
```
public static String firstWord(String s) {
    int start = 0;
    while (start < s.length() && s.charAt(start) == ' ') {
        start++;
    }
    int stop = start;
    while (stop < s.length() && s.charAt(stop) != ' ') {
        stop++;
    }
    return s.substring(start, stop);
}
```
Phiên bản này hoạt động trong tất cả các trường hợp, bỏ qua bất kỳ khoảng trắng dẫn đầu nào và trả về một chuỗi rỗng nếu không có từ nào để trả về.

Biến boolean và cờ hiệu

Tất cả các câu lệnh if/else đều được kiểm soát bởi các kiểm tra Boolean. Các kiểm tra này có thể là các biến boolean hoặc các biểu thức Boolean. Ví dụ, hãy xem xét đoạn mã sau:

java
Copy code
```
if (number > 0) {
    System.out.println("positive");
} else {
    System.out.println("not positive");
}
```
Đoạn mã này có thể được viết lại như sau:
java
Copy code
```
boolean positive = (number > 0);
if (positive) {
    System.out.println("positive");
} else {
    System.out.println("not positive");
}
```
Sử dụng các biến boolean giúp tăng cường khả năng đọc của chương trình vì nó cho phép bạn đặt tên cho các kiểm tra. Hãy xem xét loại mã bạn sẽ tạo ra cho một chương trình hẹn hò. Bạn có thể có một số biến kiểu số nguyên mô tả một số thuộc tính của một người: looks để lưu trữ ước tính sơ bộ về sắc đẹp (trên thang điểm 1–10); IQ để lưu trữ chỉ số thông minh; income để lưu trữ thu nhập hàng năm; và snothers để theo dõi bạn thân thiết (viết tắt của "significant other"). Với các biến này để xác định các thuộc tính của một người, bạn có thể phát triển các kiểm tra khác nhau về tính phù hợp. Khi viết chương trình, bạn có thể sử dụng các biến boolean để đặt tên cho các kiểm tra đó, giúp mã trở nên dễ đọc hơn rất nhiều:
java
Copy code
```
boolean cute = (looks >= 9);
boolean smart = (IQ > 125);
boolean rich = (income > 100000);
```

```
boolean available = (snothers == 0);
boolean awesome = cute && smart && rich && available;
```
Bạn có thể sử dụng một loại biến boolean đặc biệt gọi là cờ hiệu (flag). Thông thường, chúng ta sử dụng cờ hiệu trong các vòng lặp để ghi lại các điều kiện lỗi hoặc báo hiệu hoàn thành. Các cờ hiệu khác nhau kiểm tra các điều kiện khác nhau. Ví dụ, hãy xem xét một trọng tài trong một trận đấu thể thao, người quan sát một hành động vi phạm nào đó và ném cờ nếu nó xảy ra. Bạn đôi khi nghe thấy một bình luận viên nói: "Có một lá cờ được ném ra trong trận đấu."
Hãy đưa một cờ hiệu vào mã tổng lũy kế mà chúng ta đã thấy trong chương trước:
java
Copy code
```
double sum = 0.0;
for (int i = 1; i <= totalNumber; i++) {
    System.out.print("    #" + i + "? ");
    double next = console.nextDouble();
    sum += next;
}
System.out.println("sum = " + sum);
```
Giả sử chúng ta muốn biết liệu tổng số có bao giờ trở nên âm hay không. Lưu ý rằng tình huống này không giống với tình huống tổng số kết thúc ở giá trị âm. Giống như số dư tài khoản ngân hàng, tổng số có thể chuyển đổi qua lại giữa âm và dương. Khi bạn thực hiện một loạt các khoản tiền gửi và rút tiền, ngân hàng sẽ theo dõi liệu bạn có rút tiền quá mức trong suốt quá trình hay không. Sử dụng một cờ hiệu boolean, chúng ta có thể sửa đổi vòng lặp trước đó để theo dõi liệu tổng số có bao giờ trở nên âm và báo cáo kết quả sau vòng lặp:
java
Copy code
```
double sum = 0.0;
```

```
boolean negative = false;
for (int i = 1; i <= totalNumber; i++) {
    System.out.print("    #" + i + "? ");
    double next = console.nextDouble();
    sum += next;
    if (sum < 0.0) {
        negative = true;
    }
}
System.out.println("sum = " + sum);
if (negative) {
    System.out.println("Sum went negative");
} else {
    System.out.println("Sum never went negative");
}
```

Lưu ý rằng câu lệnh if bên trong vòng lặp không có else vì bạn không muốn đặt lại cờ hiệu về false sau khi đã thấy một giá trị âm.

Zen của Boolean

Năm 1974, Robert Pirsig đã bắt đầu một xu hướng văn hóa với cuốn sách của ông "Zen và Nghệ thuật Bảo trì Xe Máy: Một Sự Điều Tra về Giá Trị". Rất nhiều cuốn sách sau đó đã sao chép tiêu đề với "Zen và Nghệ thuật của X", trong đó X có thể là Poker, Đan Len, Viết Lách, Foosball, Guitar, Giảng Dạy Trường Công, Kiếm Sống, Yêu Đương, Làm Chăn, Hài Độc Thoại, Kỳ Thi SAT, Cắm Hoa, Buộc Ruồi, Phân Tích Hệ Thống, Làm Cha, Viết Kịch Bản, Quản Lý Bệnh Tiểu Đường, Sự Thân Mật, Giúp Đỡ, Đấu Đường Phố, Giết Người, và vân vân. Thậm chí còn có một cuốn sách tên là "Zen và Nghệ thuật của Bất Kỳ Thứ Gì".

Bây giờ chúng ta tham gia xu hướng văn hóa này bằng cách thảo luận về Zen và nghệ thuật của kiểu boolean. Dường như phải mất một thời gian để nhiều người mới bắt đầu quen với các biểu

thức Boolean. Người mới thường viết các biểu thức quá phức tạp liên quan đến các giá trị boolean vì họ không nắm bắt được sự đơn giản có thể khi bạn hiểu cách hoạt động của kiểu boolean.
Ví dụ, giả sử bạn đang viết một chương trình chơi trò chơi liên quan đến các số có hai chữ số, mỗi số được tạo thành từ hai chữ số khác nhau. Nói cách khác, chương trình sẽ sử dụng các số như 42 được tạo thành từ hai chữ số khác biệt, nhưng không sử dụng các số như 6 (chỉ một chữ số), 394 (hơn hai chữ số), hoặc 22 (cả hai chữ số giống nhau). Bạn có thể muốn kiểm tra xem một số đã cho có hợp lệ để sử dụng trong trò chơi hay không.
Bạn có thể giới hạn bản thân trong các số có hai chữ số với một bài kiểm tra như sau: n >= 10 && n <= 99
Bạn cũng cần kiểm tra để đảm bảo rằng hai chữ số không giống nhau. Bạn có thể lấy các chữ số của một số có hai chữ số với các biểu thức n / 10 và n % 10. Vì vậy, bạn có thể mở rộng bài kiểm tra để đảm bảo rằng các chữ số không giống nhau: n >= 10 && n <= 99 && (n / 10 != n % 10)
Bài kiểm tra này là một ví dụ tốt về tình huống bạn có thể sử dụng một phương thức để nắm bắt một biểu thức Boolean phức tạp. Trả về một giá trị boolean sẽ cho phép bạn gọi phương thức bao nhiêu lần tùy thích mà không cần phải sao chép biểu thức phức tạp này mỗi lần, và bạn có thể đặt tên cho tính toán này để làm cho chương trình dễ đọc hơn.
Giả sử bạn muốn gọi phương thức isTwoUniqueDigits. Bạn muốn phương thức nhận một giá trị kiểu int và trả về true nếu int được tạo thành từ hai chữ số khác biệt và trả về false nếu không. Vậy phương thức sẽ trông như sau:
java
Copy code
```
public static boolean isTwoUniqueDigits(int n) {
    ...
}
```

Bạn sẽ viết thân phương thức này như thế nào? Chúng ta đã viết sẵn bài kiểm tra, vì vậy chúng ta chỉ cần tìm cách tích hợp nó vào phương thức. Phương thức có kiểu trả về boolean, vì vậy bạn muốn nó trả về giá trị true khi bài kiểm tra thành công và giá trị false khi bài kiểm tra thất bại. Bạn có thể viết phương thức như sau:

java

Copy code

```
public static boolean isTwoUniqueDigits(int n) {
    if (n >= 10 && n <= 99 && (n % 10 != n / 10)) {
        return true;
    } else {
        return false;
    }
}
```

Phương thức này hoạt động, nhưng nó dài dòng hơn cần thiết. Mã trước đó đánh giá bài kiểm tra mà chúng ta đã phát triển. Biểu thức đó là kiểu boolean, nghĩa là nó đánh giá thành true hoặc false. Lệnh if/else yêu cầu máy tính trả về true nếu biểu thức đánh giá thành true và trả về false nếu nó đánh giá thành false. Nhưng tại sao phải sử dụng cấu trúc này? Nếu phương thức sẽ trả về true khi biểu thức đánh giá thành true và trả về false khi biểu thức đánh giá thành false, bạn có thể chỉ cần trả về giá trị của biểu thức trực tiếp:

java

Copy code

```
public static boolean isTwoUniqueDigits(int n) {
    return (n >= 10 && n <= 99 && (n % 10 != n / 10));
}
```

Thậm chí phiên bản trước có thể được đơn giản hóa, vì dấu ngoặc đơn là không cần thiết (mặc dù chúng làm rõ hơn chính xác điều gì sẽ được phương thức trả về). Mã này đánh giá bài kiểm tra mà chúng ta đã phát triển để xác định xem một số có

được tạo thành từ hai chữ số khác biệt hay không và trả về kết quả (trả về true khi nó có, và trả về false khi nó không có).
Hãy xem xét một sự tương tự với các biểu thức số nguyên. Đối với người hiểu Zen của Boolean, phiên bản if/else của phương thức này trông kỳ quặc như mã sau:
java
Copy code
```
if (x == 1) {
    return 1;
} else if (x == 2) {
    return 2;
} else if (x == 3) {
    return 3;
} else if (x == 4) {
    return 4;
} else if (x == 5) {
    return 5;
}
```
Nếu bạn luôn muốn trả về giá trị của x, bạn chỉ cần nói: return x;
Một sự nhầm lẫn tương tự có thể xảy ra khi học sinh sử dụng các biến boolean. Trong phần trước, chúng ta đã xem xét một biến thể của thuật toán tổng cộng dồn sử dụng một biến boolean gọi là negative để theo dõi liệu tổng có bao giờ âm hay không. Chúng ta sau đó đã sử dụng một câu lệnh if/else để in ra một thông báo báo cáo kết quả:
java
Copy code
```
if (negative) {
    System.out.println("Sum went negative");
} else {
    System.out.println("Sum never went negative");
}
```
Một số người mới sẽ viết mã này như sau:

java
Copy code
```
if (negative == true) {
    System.out.println("Sum went negative");
} else {
    System.out.println("Sum never went negative");
}
```
So sánh này là không cần thiết vì lệnh if/else mong đợi một biểu thức kiểu boolean xuất hiện bên trong dấu ngoặc đơn. Một biến boolean đã là kiểu thích hợp, vì vậy chúng ta không cần kiểm tra liệu nó có bằng true hay không; nó hoặc là true hoặc không phải (trong trường hợp này là false). Đối với người hiểu Zen của Boolean, bài kiểm tra trước đó có vẻ dư thừa như việc nói:
java
Copy code
```
if ((negative == true) == true) {
    ...
}
```
Người mới cũng thường viết các bài kiểm tra như sau:
java
Copy code
```
if (negative == false) {
    ...
}
```
Điều này có vẻ hợp lý vì bài kiểm tra đang làm điều gì đó hữu ích, ở chỗ nó chuyển đổi ý nghĩa của biến boolean (đánh giá thành true nếu biến là false và đánh giá thành false nếu biến là true). Nhưng toán tử phủ định được thiết kế để làm loại chuyển đổi giá trị boolean này, vì vậy bài kiểm tra này tốt hơn nên được viết như sau:
java
Copy code
```
if (!negative) {
```

```
    ...
}
```
Bạn nên làm quen với việc đọc dấu chấm than là "không", vì vậy bài kiểm tra này sẽ được đọc là "nếu không phải negative." Đối với những người hiểu Zen của Boolean, đó là cách ngắn gọn hơn để diễn đạt bài kiểm tra hơn là kiểm tra liệu negative có bằng false hay không.

Phủ định các biểu thức Boolean

Lập trình viên thường gặp tình huống cần phải phủ định một biểu thức Boolean phức tạp. Ví dụ, thường dễ dàng hơn để suy nghĩ về một vòng lặp dựa trên điều kiện kết thúc khi ta muốn dừng vòng lặp, nhưng vòng lặp while lại yêu cầu chúng ta biểu thị mã dưới dạng điều kiện tiếp tục. Giả sử bạn muốn viết một vòng lặp để liên tục yêu cầu người dùng nhập một số nguyên cho đến khi số đó là một số có hai chữ số. Vì số có hai chữ số nằm trong khoảng từ 10 đến 99, bạn có thể sử dụng các dòng mã sau để kiểm tra xem một số có chính xác hai chữ số hay không:

java
Copy code
```
number >= 10 && number <= 99
```
Để đặt điều này vào một vòng lặp while, chúng ta phải đảo ngược phép thử vì phép thử của vòng lặp while là một phép thử điều kiện tiếp tục. Nói cách khác, chúng ta muốn ở lại trong vòng lặp khi điều này chưa đúng (khi người dùng chưa đưa ra một số có hai chữ số). Một cách tiếp cận là sử dụng toán tử NOT để phủ định biểu thức này:

java
Copy code
```
while (!(number >= 10 && number <= 99))
```
Lưu ý rằng chúng ta cần đặt toàn bộ biểu thức Boolean trong dấu ngoặc đơn và sau đó đặt toán tử NOT trước nó. Mặc dù cách tiếp

cận này hoạt động, nhưng nó thường được coi là phong cách kém. Tốt nhất là đơn giản hóa biểu thức này.
Một cách tiếp cận chung để đơn giản hóa các biểu thức như vậy đã được nhà logic học người Anh Augustus De Morgan chính thức hóa. Chúng ta có thể áp dụng một trong hai quy tắc được gọi là định luật De Morgan. Bảng 5.5 cho thấy các định luật De Morgan. Lưu ý rằng khi bạn phủ định một biểu thức Boolean, mỗi toán hạng đều bị phủ định (p trở thành !p và q trở thành !q) và toán tử logic bị đảo ngược. OR logic trở thành AND logic và ngược lại khi bạn tính toán phủ định.

Bảng 5.5 Định luật De Morgan

Biểu thức gốc	Biểu thức phủ định	Phủ định đơn giản
p		q
p && q	!(p && q)	!p

Chúng ta có thể sử dụng định luật De Morgan thứ nhất cho chương trình kiểm tra số có hai chữ số vì chúng ta đang cố gắng tìm phủ định của một biểu thức liên quan đến toán tử OR logic. Thay vì viết:

java
Copy code
```
while (!(number >= 10 && number <= 99))
```
chúng ta có thể nói:

java
Copy code
```
while (number < 10 || number > 99)
```
Mỗi kiểm tra riêng lẻ đã được phủ định và toán tử AND đã được thay thế bằng toán tử OR.
Hãy xem một ví dụ thứ hai liên quan đến định luật De Morgan còn lại. Giả sử rằng bạn muốn hỏi người dùng một câu hỏi và bạn muốn buộc người dùng phải trả lời “yes” hoặc “no”. Nếu bạn có một biến chuỗi response, bạn có thể sử dụng kiểm tra sau để mô tả điều bạn muốn là đúng:

java

Copy code
response.equals("yes") || response.equals("no")
Nếu chúng ta viết một vòng lặp để tiếp tục đọc phản hồi cho đến khi biểu thức này đánh giá là đúng, thì chúng ta muốn viết vòng lặp sao cho nó sử dụng phủ định của kiểm tra này. Vì vậy, một lần nữa, chúng ta có thể sử dụng toán tử NOT cho toàn bộ biểu thức:
java
Copy code
while (!(response.equals("yes") || response.equals("no")))
Một lần nữa, tốt nhất là đơn giản hóa biểu thức bằng cách sử dụng định luật De Morgan:
java
Copy code
while (!response.equals("yes") && !response.equals("no"))

5.4 Lỗi của Người dùng

Trong chương trước, bạn đã học rằng thực hành lập trình tốt là nghĩ về các điều kiện tiên quyết của một phương thức và đề cập đến chúng trong các chú thích cho phương thức. Bạn cũng đã học rằng trong một số trường hợp mã của bạn có thể ném ra ngoại lệ nếu điều kiện tiên quyết bị vi phạm.
Khi bạn viết các chương trình tương tác, cách tiếp cận đơn giản nhất là giả định rằng người dùng sẽ cung cấp đầu vào tốt. Bạn có thể ghi lại các điều kiện tiên quyết của mình và ném ngoại lệ khi đầu vào của người dùng không đúng như mong đợi. Tuy nhiên, nói chung, tốt hơn là viết các chương trình không đưa ra giả định về đầu vào của người dùng. Ví dụ, bạn đã thấy rằng đối tượng Scanner có thể ném ra ngoại lệ nếu người dùng nhập sai loại dữ liệu. Tốt hơn là viết các chương trình có thể xử lý lỗi của người dùng. Những chương trình như vậy được gọi là chương trình "mạnh mẽ".

Mạnh mẽ

Khả năng của một chương trình để thực thi ngay cả khi gặp phải dữ liệu không hợp lệ. Trong phần này, chúng ta sẽ khám phá cách viết các chương trình tương tác mạnh mẽ. Tuy nhiên, trước khi có thể viết mã mạnh mẽ, bạn phải hiểu một số chức năng đặc biệt của lớp Scanner.

Nhìn trước với Scanner

Lớp Scanner có các phương thức cho phép bạn thực hiện một kiểm tra trước khi đọc giá trị. Nói cách khác, nó cho phép bạn "nhìn trước khi nhảy". Với mỗi phương thức "next" của lớp Scanner, có một phương thức "has" tương ứng cho biết liệu bạn có thể thực hiện thao tác đã cho hay không. Ví dụ, bạn thường muốn đọc một số nguyên (int) bằng cách sử dụng đối tượng Scanner. Nhưng điều gì xảy ra nếu người dùng nhập một thứ gì đó không phải là một số nguyên? Scanner có một phương thức gọi là hasNextInt cho biết liệu việc đọc một số nguyên có khả thi hay không. Để xác định điều này, đối tượng Scanner nhìn vào token tiếp theo và kiểm tra xem nó có thể được diễn giải là một số nguyên hay không.

Chúng ta thường diễn giải các chuỗi ký tự nhất định như các loại dữ liệu cụ thể, nhưng khi đọc các token, chúng có thể được diễn giải theo nhiều cách khác nhau. Chương trình sau sẽ cho phép chúng ta khám phá khái niệm này:

java

Copy code

```
 1 // In ra các cách mà một token đầu vào có thể được đọc bởi Scanner.
 2 import java.util.*;
 3
 4 public class ExamineInput1 {
 5     public static void main(String[] args) {
 6         System.out.println("This program examines the ways");
 7         System.out.println("a token can be read.");
 8         System.out.println();
```

```
 9         Scanner console = new Scanner(System.in);
10         System.out.print("token? ");
11         System.out.println("  hasNextInt = " +
console.hasNextInt());
12         System.out.println("  hasNextDouble = " +
console.hasNextDouble());
13         System.out.println("  hasNext = " + console.hasNext());
14     }
15 }
```

Hãy xem một số lần thực thi mẫu. Dưới đây là đầu ra của chương trình khi chúng ta nhập token 348:

arduino

Copy code

```
This program examines the ways
a token can be read.

token?
348

  hasNextInt = true
  hasNextDouble = true
  hasNext = true
```

Như bạn mong đợi, lệnh gọi hasNextInt trả về true, có nghĩa là chúng ta có thể diễn giải token này là một số nguyên. Scanner cũng cho phép chúng ta diễn giải token này là một số thực (double), vì vậy hasNextDouble cũng trả về true. Nhưng hãy lưu ý rằng hasNext() cũng trả về true. Điều này có nghĩa là chúng ta có thể gọi phương thức next để đọc token này dưới dạng một chuỗi ký tự (String).

Dưới đây là một lần thực thi khác, lần này với token 348.2:

arduino

Copy code

```
This program examines the ways
```

a token can be read.

token?
348.2

  hasNextInt = false
  hasNextDouble = true
  hasNext = true
Token này không thể được diễn giải là một số nguyên (int), nhưng nó có thể được diễn giải là một số thực (double) hoặc một chuỗi ký tự (String). Cuối cùng, hãy xem xét lần thực thi này với token "hello":
arduino
Copy code
This program examines the ways
a token can be read.

token?
hello

  hasNextInt = false
  hasNextDouble = false
  hasNext = true
Token "hello" không thể được diễn giải là một số nguyên (int) hoặc số thực (double); nó chỉ có thể được diễn giải là một chuỗi ký tự (String).

Xử lý Lỗi Người Dùng
Xem xét đoạn mã sau:
java
Copy code
Scanner console = new Scanner(System.in);
System.out.print("How old are you? ");

int age = console.nextInt();
Điều gì sẽ xảy ra nếu người dùng nhập vào một giá trị không phải là một số nguyên (int)? Nếu điều đó xảy ra, Scanner sẽ ném ra một ngoại lệ khi gọi nextInt. Chúng ta đã thấy trong phần trước rằng có thể kiểm tra liệu token tiếp theo có thể được diễn giải là một số nguyên bằng cách sử dụng phương thức hasNextInt. Vì vậy, chúng ta có thể kiểm tra trước khi đọc một số nguyên xem người dùng đã nhập một giá trị thích hợp hay chưa.
Nếu người dùng nhập vào một giá trị không phải là số nguyên, chúng ta muốn bỏ qua đầu vào đó, in ra một thông báo lỗi nào đó và yêu cầu nhập lại một giá trị khác. Chúng ta muốn đoạn mã này thực thi trong một vòng lặp để tiếp tục bỏ qua đầu vào và tạo ra thông báo lỗi cho đến khi người dùng nhập vào giá trị hợp lệ.
Dưới đây là một thử nghiệm đầu tiên cho giải pháp bằng cách sử dụng giả mã:
vbnet
Copy code
```
while (user hasn't given us an integer) {

    prompt.

    discard input.

    generate an error message.

}

read the integer.
```
Đoạn mã này thể hiện những gì chúng ta muốn làm. Chúng ta muốn tiếp tục yêu cầu nhập, bỏ qua đầu vào, và tạo ra thông báo lỗi miễn là đầu vào không hợp lệ, và khi một giá trị hợp lệ được nhập vào, chúng ta sẽ đọc số nguyên đó. Tất nhiên, trong lần cuối cùng của vòng lặp, chúng ta không muốn bỏ qua đầu vào

hoặc tạo ra thông báo lỗi. Nói cách khác, lần cuối cùng trong vòng lặp, chúng ta chỉ muốn thực hiện bước đầu tiên (yêu cầu nhập, nhưng không bỏ qua và không tạo ra thông báo lỗi). Đây là một vấn đề điển hình của "fencepost", và chúng ta có thể giải quyết nó bằng cách đặt yêu cầu nhập ban đầu trước vòng lặp và thay đổi thứ tự các thao tác trong vòng lặp:
vbnet
Copy code
```
prompt.

while (user hasn't given us an integer) {

    discard input.

    generate an error message.

    prompt.

}

read the integer.
```
Giả mã này khá dễ dàng chuyển thành mã Java thực tế:
java
Copy code
```
Scanner console = new Scanner(System.in);
System.out.print("How old are you? ");

while (!console.hasNextInt()) {
    console.next(); // bỏ qua đầu vào
    System.out.println("Not an integer; try again.");
    System.out.print("How old are you? ");
}
```

```
int age = console.nextInt();
```
Thực tế, đây là một thao tác phổ biến đến mức nó đáng được biến thành một phương thức tĩnh:
java
Copy code
```
// prompts until a valid number is entered
public static int getInt(Scanner console, String prompt) {
    System.out.print(prompt);
    while (!console.hasNextInt()) {
        console.next(); // bỏ qua đầu vào
        System.out.println("Not an integer; try again.");
        System.out.print(prompt);
    }
    return console.nextInt();
}
```
Sử dụng phương thức này, chúng ta có thể viết lại đoạn mã ban đầu như sau:
java
Copy code
```
Scanner console = new Scanner(System.in);
int age = getInt(console, "How old are you? ");
```
Khi bạn thực thi đoạn mã này, tương tác sẽ trông như sau:
sql
Copy code
```
How old are you?
what?
Not an integer; try again.
How old are you?
18.4
Not an integer; try again.
How old are you?
ten
Not an integer; try again.
```

How old are you?
darn!
Not an integer; try again.
How old are you?
help
Not an integer; try again.
How old are you?
19

5.5 Khẳng định (Assertions) và Logic Chương trình

Các nhà logic học quan tâm đến các câu tuyên bố được gọi là khẳng định.

Khẳng định (Assertion)

Một câu tuyên bố có thể đúng hoặc sai. Các câu sau đều là khẳng định:

- 2+2 bằng 4.
- Mặt trời lớn hơn Trái Đất.
- x > 45.
- Trời đang mưa.
- Mưa ở Tây Ban Nha chủ yếu rơi trên đồng bằng.

Các câu sau không phải là khẳng định (câu đầu tiên là câu hỏi và câu thứ hai là mệnh lệnh):

- Bạn nặng bao nhiêu?
- Đưa tôi về nhà.

Một số khẳng định có thể đúng hoặc sai tùy thuộc vào ngữ cảnh:

- x > 45. (Tính đúng đắn của câu này phụ thuộc vào giá trị của x.)
- Trời đang mưa. (Tính đúng đắn của câu này phụ thuộc vào thời gian và địa điểm.)

Bạn có thể xác định chúng đúng hay sai bằng cách cung cấp ngữ cảnh:

- Khi x = 13, x > 45.

- Vào ngày 4 tháng 7 năm 1776, tại Philadelphia, trời đang mưa.

Để viết chương trình đúng và hiệu quả, bạn cần học cách đưa ra các khẳng định về chương trình của mình và hiểu rõ ngữ cảnh mà các khẳng định đó sẽ đúng. Ví dụ, nếu bạn đang cố gắng nhận một số không âm từ người dùng, bạn muốn khẳng định "Số là không âm" phải đúng. Bạn có thể sử dụng một lời nhắc đơn giản và đọc giá trị:

java
Copy code

```
System.out.print("Please give me a nonnegative number— —> ");
double number = console.nextDouble();
// số có không âm không?
```

Nhưng người dùng có thể bỏ qua yêu cầu của bạn và nhập một số âm. Thực tế, người dùng thường nhập các giá trị mà bạn không mong đợi, thường là do họ nhầm lẫn. Do sự không chắc chắn của đầu vào từ người dùng, khẳng định cụ thể này có thể đôi khi đúng và đôi khi sai. Nhưng một số phần khác trong chương trình có thể phụ thuộc vào việc khẳng định này đúng. Ví dụ, nếu bạn định lấy căn bậc hai của số đó, bạn phải đảm bảo rằng số đó không âm. Nếu không, bạn có thể gặp phải kết quả sai.

Bằng cách sử dụng vòng lặp, bạn có thể đảm bảo rằng số bạn nhận được là không âm:

java
Copy code

```
System.out.print("Please give me a nonnegative number— —> ");
double number = console.nextDouble();

while (number < 0.0) {
    System.out.print("That is a negative number. Try again— —>
");
    number = console.nextDouble();
```

```
}

// số có không âm không?
```

Bạn biết rằng số sẽ là không âm sau khi thoát khỏi vòng lặp; nếu không, chương trình sẽ không thoát khỏi vòng lặp while. Miễn là người dùng nhập các giá trị âm, chương trình của bạn sẽ ở lại trong vòng lặp while và tiếp tục yêu cầu nhập dữ liệu.
Điều này không có nghĩa là số nên là không âm sau vòng lặp. Nó có nghĩa là số chắc chắn sẽ là không âm. Bằng cách làm việc thông qua logic của chương trình, bạn có thể thấy rằng điều này là chắc chắn, một khẳng định mà bạn chắc chắn. Bạn thậm chí có thể chứng minh điều đó nếu cần. Một khẳng định như vậy được gọi là một khẳng định có thể chứng minh được.

Khẳng định có thể chứng minh được

Một khẳng định có thể được chứng minh là đúng tại một điểm cụ thể trong quá trình thực thi chương trình. Các khẳng định có thể chứng minh được giúp xác định các phần mã không cần thiết.
Xem xét các câu lệnh sau:
java
Copy code

```
int x = 0;

if (x == 0) {
    System.out.println("This is what I expect.");
} else {
    System.out.println("How can that be?");
}
```

Cấu trúc if/else là không cần thiết. Bạn biết rằng câu lệnh gán sẽ đặt giá trị x bằng 0. Kiểm tra xem x có bằng 0 hay không là thừa thãi, giống như việc nói "Trước khi tiếp tục, tôi sẽ kiểm tra rằng 2+2 bằng 4." Vì phần if của câu lệnh if/else luôn được thực thi, bạn có thể chứng minh rằng các dòng mã sau đây luôn thực hiện cùng một điều như các dòng mã trước:

java
Copy code
int x = 0;
System.out.println("This is what I expect.");
Đoạn mã này đơn giản hơn và do đó, tốt hơn. Các chương trình đã phức tạp đủ mà không cần thêm mã không cần thiết.
Khái niệm về khẳng định đã trở nên phổ biến đến mức nhiều ngôn ngữ lập trình cung cấp hỗ trợ cho việc kiểm tra các khẳng định. Java đã bổ sung hỗ trợ kiểm tra khẳng định bắt đầu từ phiên bản 1.4. Bạn có thể đọc thêm về câu lệnh assert của Java trong Phụ lục C.

Lý luận về Khẳng định (Assertions)
Tập trung vào các khẳng định xuất phát từ một lĩnh vực của khoa học máy tính được gọi là xác minh chính thức.
Xác minh chính thức (Formal Verification)
Một lĩnh vực của khoa học máy tính liên quan đến việc lý luận về các thuộc tính chính thức của chương trình để chứng minh tính đúng đắn của chương trình.
Ví dụ, hãy xem xét các thuộc tính của câu lệnh if đơn giản:
java
Copy code
if (<test>) { // test luôn luôn đúng tại đây
    ...
}
Bạn chỉ vào phần thân của câu lệnh if nếu điều kiện kiểm tra (test) là đúng, vì vậy bạn biết rằng điều kiện kiểm tra phải đúng nếu dòng mã đó được thực thi. Bạn cũng có thể rút ra kết luận tương tự về điều gì là đúng trong câu lệnh if/else:
java
Copy code
if (<test>) { // test luôn luôn đúng tại đây
    ...

```
} else {  // test không bao giờ đúng tại đây
    ...
}
```
Bạn cũng có thể rút ra kết luận tương tự về điều gì là đúng bên trong phần thân của vòng lặp while:
java
Copy code
```
while (<test>) {  // test luôn luôn đúng tại đây
    ...
}
```
Nhưng trong trường hợp của vòng lặp while, bạn có thể rút ra một kết luận mạnh mẽ hơn. Bạn biết rằng miễn là điều kiện kiểm tra cho kết quả đúng, bạn sẽ tiếp tục quay lại vòng lặp. Do đó, bạn có thể kết luận rằng sau khi vòng lặp kết thúc, điều kiện kiểm tra không còn đúng nữa:
java
Copy code
```
while (<test>) {  // test luôn luôn đúng tại đây
    ...
}
// test không bao giờ đúng tại đây
```
Điều kiện kiểm tra không thể đúng sau vòng lặp vì nếu đúng, chương trình sẽ thực hiện phần thân của vòng lặp thêm một lần nữa.
Những quan sát này về thuộc tính của các câu lệnh if, if/else, và vòng lặp while cung cấp một khởi đầu tốt cho việc chứng minh các khẳng định về chương trình. Nhưng thường thì việc chứng minh các khẳng định đòi hỏi phải phân tích sâu hơn về những gì mã thực sự thực hiện. Ví dụ, giả sử bạn có một biến x thuộc kiểu int và bạn thực thi câu lệnh if sau:
java
Copy code
```
if (x < 0) {  // x < 0 luôn luôn đúng tại đây
```

```
    x = -x;
}
// nhưng x < 0 thì sao tại đây?
```

Bình thường, bạn sẽ không thể kết luận điều gì về việc x nhỏ hơn 0 sau câu lệnh if, nhưng bạn có thể rút ra kết luận nếu bạn suy nghĩ về các trường hợp khác nhau. Nếu x lớn hơn hoặc bằng 0 trước câu lệnh if, nó sẽ vẫn lớn hơn hoặc bằng 0 sau câu lệnh if. Và nếu x nhỏ hơn 0 trước câu lệnh if, nó sẽ bằng -x sau đó. Khi x nhỏ hơn 0, thì -x lớn hơn 0. Vì vậy, trong cả hai trường hợp, bạn biết rằng sau khi câu lệnh if được thực thi, x sẽ lớn hơn hoặc bằng 0.

Các lập trình viên tự nhiên áp dụng kiểu lý luận này khi viết chương trình. Các nhà khoa học máy tính đang cố gắng tìm ra cách thực hiện loại lý luận này theo cách chính thức, có thể xác minh được.

Ví dụ chi tiết về Assertions

Để tìm hiểu thêm về assertions, hãy cùng xem xét kỹ lưỡng một đoạn mã và một số assertions mà chúng ta có thể thực hiện về đoạn mã này. Xem xét phương thức sau:

java

Copy code

```
public static void printCommonPrefix(int x, int y) {
    int z = 0;
    // Điểm A
    while (x != y) {
        // Điểm B
        z++;
        // Điểm C
        if (x > y) {
            // Điểm D
            x = x / 10;
        } else {
```

```
            // Điểm E
            y = y / 10;
        }
        // Điểm F
    }
    // Điểm G
    System.out.println("common prefix = " + x);
    System.out.println("digits discarded = " + z);
}
```

Phương thức này tìm chuỗi số đầu tiên dài nhất mà hai số có chung. Ví dụ, các số 32845 và 328929343 đều bắt đầu bằng tiền tố 328. Phương thức này sẽ báo cáo tiền tố đó và cũng sẽ báo cáo tổng số chữ số theo sau tiền tố chung và bị loại bỏ.

Chúng ta sẽ xem xét chương trình để kiểm tra xem các assertions khác nhau luôn đúng, không bao giờ đúng, hay đôi khi đúng và đôi khi sai tại các điểm khác nhau trong quá trình thực thi chương trình. Các chú thích trong phương thức chỉ ra các điểm cần quan tâm. Các assertions chúng ta sẽ xem xét là:

- x > y
- x == y
- z == 0

Thông thường, các nhà khoa học máy tính viết assertions bằng ký hiệu toán học, như z = 0, nhưng chúng ta sẽ sử dụng biểu thức Java để phân biệt assertion của sự bằng nhau với việc gán giá trị cho biến.

Chúng ta có thể ghi lại các câu trả lời trong một bảng với các từ “luôn luôn” (always), “không bao giờ” (never), hoặc “đôi khi” (sometimes). Bảng của chúng ta sẽ trông như sau:

	x > y	**x == y**	**z == 0**
Điểm A			
Điểm B			
...	...	...	...

Bắt đầu tại điểm A, điểm này xuất hiện gần đầu quá trình thực thi phương thức:

java
Copy code

```
public static void printCommonPrefix(int x, int y) {
    int z = 0;
    // Điểm A
```

Các biến x và y là các tham số và nhận giá trị từ lần gọi phương thức. Nhiều lần gọi có thể xảy ra, vì vậy chúng ta không thực sự biết gì về các giá trị của x và y. Do đó, assertion x > y có thể đúng nhưng không bắt buộc phải đúng. Assertion này đôi khi đúng, đôi khi sai tại điểm A. Tương tự, assertion x == y có thể đúng tùy thuộc vào giá trị được truyền vào phương thức, nhưng nó không bắt buộc phải đúng. Tuy nhiên, chúng ta khởi tạo biến cục bộ z với giá trị 0 ngay trước điểm A, vì vậy assertion z == 0 sẽ luôn đúng tại điểm đó trong quá trình thực thi. Vì vậy, chúng ta có thể điền dòng đầu tiên của bảng như sau:

	x > y	**x == y**	**z == 0**
Điểm A	đôi khi	đôi khi	luôn luôn

Điểm B xuất hiện ngay trong vòng lặp while:

java
Copy code

```
while (x != y) {
    // Điểm B
    z++;
    ...
}
```

Chúng ta đến điểm B chỉ bằng cách đi vào vòng lặp, có nghĩa là điều kiện kiểm tra vòng lặp đã được đánh giá là đúng. Nói cách khác, tại điểm B, luôn đúng rằng x không bằng y, vì vậy assertion x == y sẽ không bao giờ đúng tại điểm đó. Nhưng chúng ta không biết cái nào trong hai số lớn hơn. Do đó, assertion x > y đôi khi đúng và đôi khi sai.

Bạn có thể nghĩ rằng assertion z == 0 sẽ luôn đúng tại điểm B vì chúng ta đã ở điểm A ngay trước khi đến điểm B, nhưng đó không phải là câu trả lời đúng. Hãy nhớ rằng điểm B nằm trong một vòng lặp while. Trong lần lặp đầu tiên của vòng lặp, chúng ta sẽ ở điểm A ngay trước khi đến điểm B, nhưng trong các lần lặp sau của vòng lặp, chúng ta sẽ ở bên trong vòng lặp ngay trước khi đến điểm B. Và nếu bạn nhìn vào dòng mã ngay sau điểm B, bạn sẽ thấy rằng nó tăng z. Không có sự thay đổi nào khác đối với biến z bên trong vòng lặp. Do đó, mỗi lần thân vòng lặp thực thi, z sẽ tăng thêm 1. Vì vậy, z sẽ bằng 0 tại điểm B lần đầu tiên qua vòng lặp, nhưng nó sẽ là 1 trong lần lặp thứ hai, 2 trong lần lặp thứ ba, v.v. Do đó, câu trả lời đúng cho assertion z == 0 tại điểm B là đôi khi đúng, đôi khi sai. Vì vậy, dòng thứ hai của bảng nên trông như thế này:

	x > y	**x == y**	**z == 0**
Điểm B	đôi khi	không bao giờ	đôi khi

Điểm C nằm ngay sau khi biến z được tăng. Không có sự thay đổi nào đối với giá trị của x và y giữa điểm B và điểm C, vì vậy câu trả lời tương tự áp dụng cho điểm C đối với các assertions x > y và x == y. Assertion z == 0 sẽ không bao giờ đúng sau khi tăng, mặc dù z bắt đầu bằng 0 trước khi vòng lặp bắt đầu vì không có sự thao tác nào khác đối với biến bên trong vòng lặp; một khi nó được tăng, nó sẽ không bao giờ trở lại 0 nữa. Do đó, chúng ta có thể điền vào bảng cho điểm C như sau:

	x > y	**x == y**	**z == 0**
Điểm C	đôi khi	không bao giờ	không bao giờ

Các điểm D và E là một phần của câu lệnh if/else bên trong vòng lặp while, vì vậy chúng ta có thể đánh giá chúng như một cặp. Câu lệnh if/else xuất hiện ngay sau điểm C:

java

Copy code

```
// Điểm C
```

```
if (x > y) {
    // Điểm D
    x = x / 10;
} else {
    // Điểm E
    y = y / 10;
}
```

Không có sự thay đổi nào đối với các biến giữa điểm C và các điểm D và E. Java thực hiện một phép kiểm tra và phân nhánh theo một trong hai hướng. Kiểm tra if/else xác định xem x có lớn hơn y hay không. Nếu kiểm tra đúng, chúng ta sẽ đến điểm D. Nếu không, chúng ta sẽ đến điểm E. Vì vậy, đối với assertion x > y, chúng ta biết rằng nó luôn đúng tại điểm D và không bao giờ đúng tại điểm E. Assertion x == y khó làm rõ hơn một chút. Chúng ta biết rằng nó không bao giờ có thể đúng tại điểm D, nhưng liệu nó có thể đúng tại điểm E? Chỉ dựa trên phép kiểm tra if/else, câu trả lời là có. Nhưng hãy nhớ rằng tại điểm C, assertion này không bao giờ có thể đúng. Các giá trị của x và y không thay đổi giữa điểm C và điểm E, vì vậy nó vẫn không bao giờ có thể đúng.

Đối với assertion z == 0, biến z không thay đổi giữa điểm C và các điểm D và E, và z không được bao gồm trong phép kiểm tra. Vì vậy, bất cứ điều gì chúng ta biết về z trước đó vẫn giữ nguyên. Do đó, các câu trả lời đúng để điền vào cho các điểm D và E như sau:

	x > y	**x == y**	**z == 0**
Điểm D	luôn luôn	không bao giờ	không bao giờ
Điểm E	không bao giờ	không bao giờ	không bao giờ

Điểm F xuất hiện sau câu lệnh if/else. Để xác định mối quan hệ giữa x và y tại điểm F, chúng ta phải xem xét các biến đã thay đổi như thế nào. Câu lệnh if/else hoặc chia x cho 10 (nếu nó là giá trị lớn hơn) hoặc chia y cho 10 (nếu nó là giá trị lớn hơn). Vì vậy, chúng ta phải hỏi liệu assertion x > y có thể đúng tại điểm F hay

không. Câu trả lời là có. Ví dụ, x có thể là 218 và y có thể là 6 trước câu lệnh if/else. Trong trường hợp đó, x bây giờ sẽ là 21, vẫn lớn hơn y. Nhưng nó có phải luôn luôn lớn hơn y không? Không nhất thiết. Các giá trị có thể đã bị đảo ngược, trong trường hợp đó y sẽ lớn hơn x. Vì vậy, assertion đó đôi khi đúng và đôi khi sai tại điểm F.

Còn assertion x == y thì sao? Chúng ta biết nó không cần phải đúng vì chúng ta đã thấy các trường hợp trong đó x lớn hơn y hoặc y lớn hơn x. Liệu có thể nào nó đúng không? Có giá trị nào của x và y dẫn đến kết quả này không? Xem xét trường hợp trong đó x là 218 và y là 21. Sau đó, chúng ta sẽ chia x cho 10 để được 21, bằng với y. Vì vậy, assertion này cũng đôi khi đúng và đôi khi sai.

Không có thay đổi nào đối với z giữa các điểm D và E và điểm F, vì vậy chúng ta chỉ cần mang câu trả lời của mình từ các cột trước xuống. Vì vậy, chúng ta sẽ điền vào bảng như sau cho điểm F:

	x > y	**x == y**	**z == 0**
Điểm F	đôi khi	đôi khi	không bao giờ

Điểm G xuất hiện sau vòng lặp while:

```
java
Copy code
while (x != y) {
    ...
}
// Điểm G
```

Chúng ta có thể thoát khỏi vòng lặp while chỉ khi x trở nên bằng với y. Vì vậy, tại điểm G, chúng ta biết rằng assertion x == y luôn đúng. Điều đó có nghĩa là assertion x > y không bao giờ có thể đúng. Assertion z == 0 hơi khó hơn một chút. Tại điểm F, nó không bao giờ đúng, vì vậy bạn có thể tưởng tượng rằng tại điểm G, nó không bao giờ đúng. Nhưng chúng ta không nhất thiết đã ở điểm F ngay trước khi đến điểm G. Chúng ta có thể chưa bao giờ

vào vòng lặp while, trong trường hợp đó chúng ta sẽ ở điểm A ngay trước điểm G. Tại điểm A, biến z bằng 0. Do đó, câu trả lời đúng cho assertion này là đôi khi đúng, đôi khi sai tại điểm G. Hàng cuối cùng của bảng của chúng ta như sau:

	x > y	**x == y**	**z == 0**
Điểm G	không bao giờ	luôn luôn	đôi khi

Khi chúng ta kết hợp thông tin này, chúng ta có thể điền vào bảng như sau:

	x > y	**x == y**	**z == 0**
Điểm A	đôi khi	đôi khi	luôn luôn
Điểm B	đôi khi	không bao giờ	đôi khi
Điểm C	đôi khi	không bao giờ	không bao giờ
Điểm D	luôn luôn	không bao giờ	không bao giờ
Điểm E	không bao giờ	không bao giờ	không bao giờ
Điểm F	đôi khi	đôi khi	không bao giờ
Điểm G	không bao giờ	luôn luôn	đôi khi

5.6 Nghiên Cứu Tình Huống: NumberGuess

Nếu chúng ta kết hợp vòng lặp vô định, khả năng kiểm tra lỗi của người dùng và việc tạo số ngẫu nhiên, chúng ta có thể tạo ra các trò chơi đoán số trong đó máy tính nghĩ ra các số ngẫu nhiên và người dùng cố gắng đoán chúng. Hãy xem xét một trò chơi ví dụ với các quy tắc sau. Máy tính nghĩ ra một số ngẫu nhiên gồm hai chữ số nhưng giữ bí mật với người chơi. Chúng ta sẽ chỉ cho phép chương trình chấp nhận các số dương, vì vậy phạm vi số có thể chấp nhận được là từ 00 đến 99 bao gồm cả hai. Người chơi sẽ cố gắng đoán số mà máy tính đã chọn. Nếu người chơi đoán đúng, chương trình sẽ báo cáo số lần đoán mà người chơi đã thực hiện.

Để làm cho trò chơi thú vị hơn, mỗi lần người dùng nhập vào một lần đoán không chính xác, máy tính sẽ đưa ra một gợi ý. Cụ thể, máy tính sẽ cho người chơi biết có bao nhiêu chữ số từ số đoán

nằm trong đáp án đúng. Thứ tự của các chữ số không ảnh hưởng đến số lượng chữ số khớp. Ví dụ, nếu số đúng là 57 và người chơi đoán là 73, máy tính sẽ báo cáo một chữ số khớp, vì đáp án đúng chứa một số 7. Nếu người chơi tiếp theo đoán 75, máy tính sẽ báo cáo hai chữ số khớp. Lúc này, người chơi biết rằng số của máy tính phải là 57, vì 57 là số duy nhất gồm hai chữ số có các chữ số khớp với số 75.

Vì người chơi sẽ nhập nhiều thông tin vào giao diện điều khiển, rất có thể họ sẽ nhập sai số hoặc các ký hiệu không phải là số. Chúng ta muốn chương trình trò chơi đoán số của mình chống lại các lỗi nhập của người dùng.

Phiên bản ban đầu không có gợi ý

Trong các chương trước, chúng ta đã nói về ý tưởng cải tiến lặp đi lặp lại. Vì đây là một chương trình đầy thách thức, chúng ta sẽ thực hiện nó theo từng giai đoạn. Một trong những phần khó nhất của chương trình là đưa ra các gợi ý chính xác cho người chơi. Hiện tại, chúng ta chỉ cần viết một trò chơi nói với người chơi liệu họ có đúng hay không sau mỗi lần đoán và, khi trò chơi kết thúc, báo cáo số lần đoán mà người chơi đã thực hiện. Chương trình chưa xử lý được các lỗi nhập liệu của người dùng; điều đó có thể được thêm sau. Để đơn giản hóa trò chơi, thay vì để máy tính chọn số ngẫu nhiên, chúng ta sẽ chọn một giá trị đã biết cho số để dễ dàng kiểm tra mã hơn.

Vì chúng ta không biết người chơi sẽ cần bao nhiêu lần đoán trước khi đoán đúng số, có vẻ như vòng lặp chính cho trò chơi này sẽ là một vòng lặp while. Có thể sẽ cám dỗ để viết mã theo pseudocode sau:

java

Copy code

```
// pseudocode số đoán sai
```

nghĩ ra một số.

while (người dùng chưa đoán đúng số) {

    nhắc và đọc một dự đoán.

    báo cáo xem dự đoán đó đúng hay sai.

}
Nhưng vấn đề với pseudocode này là bạn không thể bắt đầu vòng lặp while nếu bạn chưa có giá trị đoán từ người chơi. Đoạn mã sau đây không biên dịch được vì biến guess chưa được khởi tạo khi vòng lặp bắt đầu:
java
Copy code
// đoạn mã này không biên dịch được

int numGuesses = 0;
int number = 42; // máy tính luôn chọn số giống nhau
int guess;

while (guess != number) {
    System.out.print("Your guess? ");
    guess = console.nextInt();
    numGuesses++;
    System.out.println("Incorrect.");
}

System.out.println("You got it right in " + numGuesses + " tries.");
Hóa ra vòng lặp đoán chính của trò chơi là một vòng lặp fencepost, vì sau mỗi lần đoán sai, chương trình phải in thông báo "Incorrect" (và sau này là một gợi ý). Đối với n lần đoán, có n-1 gợi ý. Nhớ lại pseudocode tổng quát cho các vòng lặp fencepost:
java

Copy code
đặt một cột.

for (độ dài của hàng rào) {

    gắn một số dây.

    đặt một cột.

}
Vấn đề cụ thể này là một fencepost vô định sử dụng vòng lặp while. Hãy xem xét một pseudocode cụ thể hơn. Các "cột" là những lời nhắc cho các dự đoán, và "dây" là những thông điệp "Incorrect":
java
Copy code
// pseudocode số đoán cụ thể

nghĩ ra một số.

yêu cầu dự đoán ban đầu của người chơi.

while (dự đoán không phải là số đúng) {

    thông báo cho người chơi rằng dự đoán không chính xác.

    yêu cầu một dự đoán khác.

}

báo cáo số lần đoán cần thiết.

Pseudocode này dẫn chúng ta đến việc viết chương trình Java sau. Lưu ý rằng máy tính luôn chọn giá trị 42 trong phiên bản chương trình này:
java
Copy code
```
// Trò chơi đoán hai chữ số mà không có gợi ý.

import java.util.*;

public class NumberGuess1 {
    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);
        int number = 42; // luôn chọn cùng một số
        System.out.print("Your guess? ");
        int guess = console.nextInt();
        int numGuesses = 1;

        while (guess != number) {
            System.out.println("Incorrect.");
            System.out.print("Your guess? ");
            guess = console.nextInt();
            numGuesses++;
        }

        System.out.println("You got it right in " + numGuesses + "
tries.");
    }
}
```
Chúng ta có thể kiểm tra chương trình ban đầu của mình để xác minh mã chúng ta đã viết cho đến nay. Một cuộc đối thoại mẫu trông như thế này:
css
Copy code

Your guess? 65
Incorrect.
Your guess? 12
Incorrect.
Your guess? 34
Incorrect.
Your guess? 42
You got it right in 4 tries.

Phiên bản Ngẫu nhiên với Gợi ý

Bây giờ chúng ta đã kiểm tra mã để đảm bảo vòng lặp chính của trò chơi hoạt động, hãy làm cho trò chơi ngẫu nhiên bằng cách chọn một giá trị ngẫu nhiên trong khoảng từ 00 đến 99 bao gồm cả hai. Để làm điều này, chúng ta sẽ tạo một đối tượng Random và gọi phương thức nextInt của nó, xác định giá trị tối đa. Hãy nhớ rằng giá trị được truyền vào nextInt phải lớn hơn một đơn vị so với giá trị tối đa mong muốn, vì vậy chúng ta sẽ truyền vào 100:

java
Copy code

```
// chọn một số ngẫu nhiên trong khoảng từ 00 đến 99 bao gồm cả hai
Random rand = new Random();
int number = rand.nextInt(100);
```

Tính năng quan trọng tiếp theo mà trò chơi của chúng ta cần có là đưa ra gợi ý khi người chơi đoán sai. Phần khó là xác định có bao nhiêu chữ số trong dự đoán của người chơi khớp với số đúng. Vì đoạn mã này không đơn giản để viết, chúng ta hãy tạo một phương thức gọi là matches để làm việc này cho chúng ta. Để xác định có bao nhiêu chữ số khớp, phương thức matches cần sử dụng dự đoán và số đúng làm tham số. Nó sẽ trả về số lượng chữ số khớp. Do đó, phần đầu của nó sẽ trông như sau:

java

Copy code
```
public static int matches(int number, int guess) {
    ...
}
```
Thuật toán của chúng ta phải đếm số lượng chữ số khớp. Mỗi chữ số từ dự đoán có thể khớp với một trong hai chữ số của số đúng. Vì các chữ số này độc lập với nhau — tức là việc chữ số hàng đơn vị của dự đoán có khớp hay không là độc lập với việc chữ số hàng chục có khớp hay không — chúng ta nên sử dụng các câu lệnh if tuần tự thay vì câu lệnh if/else để biểu thị các điều kiện này.
Thuật toán khớp chữ số có một trường hợp đặc biệt. Nếu người chơi đoán một số như 33 có chứa hai chữ số giống nhau, và nếu chữ số đó có trong đáp án đúng (giả sử đáp án đúng là 37), sẽ gây hiểu lầm nếu báo cáo rằng có hai chữ số khớp. Sẽ hợp lý hơn nếu chương trình báo cáo một chữ số khớp. Để xử lý trường hợp này, thuật toán của chúng ta phải kiểm tra xem dự đoán có chứa hai chữ số giống nhau không và chỉ coi chữ số thứ hai của dự đoán là khớp nếu nó khác với chữ số thứ nhất.
Dưới đây là pseudocode cho thuật toán:
java
Copy code
```
matches = 0;

if (chữ số thứ nhất của dự đoán khớp với một trong hai chữ số của số đúng) {
    chúng ta đã tìm thấy một chữ số khớp.
}

if (chữ số thứ hai của dự đoán khác với chữ số thứ nhất,
    VÀ nó khớp với một trong hai chữ số của số đúng) {
    chúng ta đã tìm thấy một chữ số khớp khác.
}
```

Chúng ta cần có khả năng tách số đúng và dự đoán thành hai chữ số để có thể so sánh chúng. Nhớ lại từ phần Boolean Zen rằng chúng ta có thể sử dụng các toán tử chia và chia dư để biểu diễn các chữ số của bất kỳ số hai chữ số nào n là n / 10 cho chữ số hàng chục và n % 10 cho chữ số hàng đơn vị.
Hãy viết câu lệnh so sánh chữ số hàng chục của dự đoán với đáp án đúng. Vì chữ số hàng chục của dự đoán có thể khớp với một trong hai chữ số của số đúng, chúng ta sẽ sử dụng kiểm tra OR với toán tử ||:
java
Copy code
```
int matches = 0;

// kiểm tra chữ số thứ nhất xem có khớp không
if (guess / 10 == number / 10 || guess / 10 == number % 10) {
    matches++;
}
```
Viết câu lệnh so sánh chữ số hàng đơn vị của dự đoán với đáp án đúng hơi phức tạp hơn, vì chúng ta phải tính đến trường hợp đặc biệt đã mô tả trước đó (trong đó cả hai chữ số của dự đoán giống nhau). Chúng ta sẽ giải quyết trường hợp này bằng cách chỉ tính chữ số thứ hai là khớp nếu nó là duy nhất và khớp với một chữ số từ đáp án đúng:
java
Copy code
```
// kiểm tra chữ số thứ hai xem có khớp không
if (guess / 10 != guess % 10 &&
    (guess % 10 == number / 10 || guess % 10 == number % 10)) {
    matches++;
}
```
Phiên bản sau của chương trình sử dụng mã gợi ý mà chúng ta vừa viết. Nó cũng thêm số được chọn ngẫu nhiên và một phần giới thiệu ngắn gọn về chương trình:

java
Copy code

```
// Trò chơi đoán số hai chữ số có gợi ý.
import java.util.*;

public class NumberGuess2 {
    public static void main(String[] args) {
        System.out.println("Try to guess my two-digit");
        System.out.println("number, and I'll tell you how");
        System.out.println("many digits from your guess");
        System.out.println("appear in my number.");
        System.out.println();

        Scanner console = new Scanner(System.in);

        // chọn một số ngẫu nhiên từ 0 đến 99 bao gồm cả hai
        Random rand = new Random();
        int number = rand.nextInt(100);

        // lấy dự đoán đầu tiên
        System.out.print("Your guess? ");
        int guess = console.nextInt();
        int numGuesses = 1;

        // đưa ra gợi ý cho đến khi đoán đúng
        while (guess != number) {
            int numMatches = matches(number, guess);
            System.out.println("Incorrect (hint: " + numMatches + "
digits match)");
            System.out.print("Your guess? ");
            guess = console.nextInt();
            numGuesses++;
        }
```

```
        System.out.println("You got it right in " + numGuesses + "
tries.");
    }

    // trả về số lượng chữ số từ dự đoán khớp với chữ số từ đáp
án đúng
    public static int matches(int number, int guess) {
        int numMatches = 0;

        if (guess / 10 == number / 10 || guess / 10 == number % 10)
{
            numMatches++;
        }

        if (guess / 10 != guess % 10 &&
            (guess % 10 == number / 10 || guess % 10 == number %
10)) {
            numMatches++;
        }

        return numMatches;
    }
}
```

Dưới đây là một nhật ký mẫu của việc thực hiện chương trình:

less

Copy code

```
Try to guess my two-digit
number, and I'll tell you how
many digits from your guess
appear in my number.

Your guess?
```

13

Incorrect (hint: 0 digits match)
Your guess?
26

Incorrect (hint: 0 digits match)
Your guess?
78

Incorrect (hint: 1 digits match)
Your guess?
79

Incorrect (hint: 1 digits match)
Your guess?
70

Incorrect (hint: 2 digits match)
Your guess?
7
You got it right in 6 tries.

Phiên bản Cuối cùng Tối ưu

Thay đổi lớn cuối cùng mà chúng ta sẽ thực hiện trong chương trình của mình là làm cho nó chống lại các đầu vào không hợp lệ từ người dùng. Có hai loại đầu vào xấu mà chúng ta có thể gặp phải:

1. Chuỗi không phải số.
2. Các số nằm ngoài phạm vi từ 0–99.

Hãy xử lý các trường hợp này từng cái một. Nhớ lại phương thức getInt đã được thảo luận trước đó trong chương này. Nó liên tục

yêu cầu người dùng nhập cho đến khi một số nguyên được nhập. Đây là phần đầu của nó:
java
Copy code
public static int getInt(Scanner console, String prompt)
Chúng ta có thể sử dụng getInt để nhận một số nguyên nằm trong khoảng từ 0 đến 99. Chúng ta sẽ liên tục gọi getInt cho đến khi số nguyên được trả về nằm trong phạm vi chấp nhận được. Điều kiện sau mà chúng ta yêu cầu trước khi có thể dừng yêu cầu đoán là:
java
Copy code
guess >= 0 && guess <= 99
Để đảm bảo rằng điều kiện sau này được đáp ứng, chúng ta có thể sử dụng một vòng lặp while để kiểm tra điều kiện ngược lại. Sử dụng định lý De Morgan, chúng ta biết rằng điều kiện ngược lại của bài kiểm tra trước đó sẽ là:
java
Copy code
guess < 0 || guess > 99
Bài kiểm tra đảo ngược này được sử dụng trong phương thức getGuess mới của chúng ta để nhận một dự đoán hợp lệ trong khoảng từ 0 đến 99. Bây giờ, bất cứ khi nào chúng ta muốn đọc đầu vào của người dùng trong chương trình chính, chúng ta sẽ gọi getGuess. Việc tách biệt yêu cầu đầu vào theo cách này rất hữu ích để đảm bảo rằng chúng ta không vô tình đếm các đầu vào không hợp lệ là các lần đoán.
Phiên bản cuối cùng của mã là như sau:
java
Copy code
// Trò chơi đoán số hai chữ số tối ưu với gợi ý.
import java.util.*;

```
public class NumberGuess3 {
    public static void main(String[] args) {
        giveIntro();
        Scanner console = new Scanner(System.in);

        // chọn một số ngẫu nhiên từ 0 đến 99 bao gồm cả hai
        Random rand = new Random();
        int number = rand.nextInt(100);

        // nhận dự đoán đầu tiên
        int guess = getGuess(console);
        int numGuesses = 1;

        // đưa ra gợi ý cho đến khi đoán đúng
        while (guess != number) {
            int numMatches = matches(number, guess);
            System.out.println("Incorrect (hint: " + numMatches + "
digits match)");
            guess = getGuess(console);
            numGuesses++;
        }

        System.out.println("You got it right in " + numGuesses + "
tries.");
    }

    public static void giveIntro() {
        System.out.println("Try to guess my two-digit");
        System.out.println("number, and I'll tell you how");
        System.out.println("many digits from your guess");
        System.out.println("appear in my number.");
        System.out.println();
    }
```

```
    // trả về số lượng chữ số khớp giữa hai số
    // tiền điều kiện: số và dự đoán là các số hai chữ số khác nhau
    public static int matches(int number, int guess) {
        int numMatches = 0;

        if (guess / 10 == number / 10 || guess / 10 == number % 10)
{
            numMatches++;
        }

        if (guess / 10 != guess % 10 && (guess % 10 == number / 10
|| guess % 10 == number % 10)) {
            numMatches++;
        }

        return numMatches;
    }

    // yêu cầu cho đến khi nhập một số trong phạm vi hợp lệ
    // hậu điều kiện: dự đoán nằm trong khoảng từ 0 đến 99
    public static int getGuess(Scanner console) {
        int guess = getInt(console, "Your guess? ");
        while (guess < 0 || guess >= 100) {
            System.out.println("Out of range; try again.");
            guess = getInt(console, "Your guess? ");
        }
        return guess;
    }

    // yêu cầu cho đến khi nhập một số hợp lệ
    public static int getInt(Scanner console, String prompt) {
        System.out.print(prompt);
```

```
        while (!console.hasNextInt()) {
            console.next(); // loại bỏ đầu vào không hợp lệ
            System.out.println("Not an integer; try again.");
            System.out.print(prompt);
        }
        return console.nextInt();
    }
}
```

Nhật ký mẫu sau đây của quá trình thực hiện cho thấy tính mạnh mẽ của chương trình khi xử lý đầu vào:

rust
Copy code

```
Try to guess my two-digit
number, and I'll tell you how
many digits from your guess
appear in my number.

Your guess?
12

Incorrect (hint: 0 digits match)
Your guess?
okay

Not an integer; try again.
Your guess?
34

Incorrect (hint: 1 digits match)
Your guess?
35

Incorrect (hint: 1 digits match)
```

Your guess?
67

Incorrect (hint: 0 digits match)
Your guess?
89

Incorrect (hint: 0 digits match)
Your guess?
3

Incorrect (hint: 2 digits match)
Your guess?
300

Out of range; try again.
Your guess?
30
You got it right in 7 tries.
Chúng ta lưu ý rằng mình đã cẩn thận chú thích mã để ghi lại các điều kiện trước và sau liên quan của các phương thức. Điều kiện trước của phương thức matches là hai tham số phải là các số hai chữ số khác nhau. Điều kiện sau của phương thức getGuess mới là nó trả về một dự đoán trong khoảng từ 0 đến 99 bao gồm cả hai. Ngoài ra, chương trình không tính đầu vào không hợp lệ (như okay và 300 trong nhật ký mẫu trước đó) là các lần đoán.

Tóm Tắt Chương
Java có một vòng lặp while bên cạnh vòng lặp for. Vòng lặp while có thể được sử dụng để viết các vòng lặp không xác định, tiếp tục thực thi cho đến khi một điều kiện nào đó không còn đúng.

"Khởi tạo vòng lặp" có nghĩa là thiết lập các giá trị của biến sẽ được sử dụng trong kiểm tra vòng lặp, để đảm bảo rằng lần kiểm tra đầu tiên sẽ thành công và vòng lặp sẽ thực thi.
Java có thể tạo ra các số ngẫu nhiên giả bằng cách sử dụng các đối tượng của lớp Random.
Vòng lặp do/while là một biến thể của vòng lặp while, thực hiện kiểm tra vòng lặp ở cuối thân vòng lặp. Một vòng lặp do/while đảm bảo rằng thân vòng lặp sẽ được thực thi ít nhất một lần.
Vòng lặp "fencepost" thực thi một “vòng lặp rưỡi” bằng cách thực hiện một phần thân của vòng lặp một lần trước khi vòng lặp bắt đầu.
Vòng lặp "sentinel" là một loại vòng lặp "fencepost" liên tục xử lý đầu vào cho đến khi nó nhận được một giá trị đặc biệt, nhưng không xử lý giá trị đặc biệt đó.
Kiểu dữ liệu nguyên thủy boolean đại diện cho các giá trị logic true hoặc false. Biểu thức boolean được sử dụng làm điều kiện kiểm tra trong câu lệnh if và vòng lặp. Các biểu thức boolean có thể sử dụng các toán tử quan hệ như < hoặc != cũng như các toán tử logic như && hoặc !.
Các kiểm tra boolean phức tạp với các toán tử logic như && hoặc || được đánh giá theo cách lười biếng: Nếu kết quả tổng thể đã rõ ràng từ việc đánh giá phần đầu tiên của biểu thức, các phần sau đó sẽ không được đánh giá. Điều này được gọi là "đánh giá ngắn mạch" (short-circuited evaluation).
Biến boolean (đôi khi được gọi là "cờ") có thể lưu trữ giá trị boolean và có thể được sử dụng làm điều kiện kiểm tra vòng lặp.
Một biểu thức boolean phức tạp có thể được phủ định bằng cách sử dụng một tập hợp các quy tắc được gọi là "Định luật De Morgan", trong đó mỗi biểu thức con bị phủ định và tất cả các phép toán AND và OR được hoán đổi.
Một chương trình mạnh mẽ kiểm tra các lỗi trong đầu vào của người dùng. Để đạt được sự mạnh mẽ tốt hơn, bạn có thể lặp lại và yêu cầu người dùng nhập lại khi họ nhập đầu vào sai. Lớp

Scanner có các phương thức như hasNextInt mà bạn có thể sử dụng để "nhìn trước" đầu vào hợp lệ.
Các câu lệnh kiểm tra (assertions) là các tuyên bố logic về một điểm cụ thể trong chương trình. Các câu lệnh kiểm tra hữu ích để chứng minh các thuộc tính về cách chương trình sẽ thực thi. Hai loại câu lệnh kiểm tra hữu ích là tiền điều kiện (preconditions) và hậu điều kiện (postconditions), đây là các tuyên bố về những gì sẽ đúng trước và sau khi một phương thức thực thi.

Bài Tập Tự Kiểm Tra

Phần 5.1: Vòng Lặp while

1. Với mỗi vòng lặp while sau đây, hãy cho biết số lần vòng lặp sẽ thực thi thân của nó. Hãy nhớ rằng các câu trả lời "không", "vô hạn" và "không xác định" là hợp lệ. Ngoài ra, kết quả đầu ra của mã trong mỗi trường hợp là gì?

a.
java
Copy code
```
int x = 1;
while (x < 100) {
    System.out.print(x + " ");
    x += 10;
}
```
b.
java
Copy code
```
int max = 10;
while (max < 10) {
    System.out.println("count down: " + max);
    max—;
}
```
c.
java
Copy code

```
int x = 250;
while (x % 3 != 0) {
    System.out.println(x);
}
```

d.

java

Copy code

```
int x = 2;
while (x < 200) {
    System.out.print(x + " ");
    x *= x;
}
```

e.

java

Copy code

```
String word = "a";
while (word.length() < 10) {
    word = "b" + word + "b";
}
System.out.println(word);
```

f.

java

Copy code

```
int x = 100;
while (x > 0) {
    System.out.println(x / 10);
    x = x / 2;
}
```

2. Chuyển đổi mỗi vòng lặp for sau đây thành một vòng lặp while tương đương:

a.

java

Copy code

```
for (int n = 1; n <= max; n++) {
    System.out.println(n);
}
```
b.
java
Copy code
```
int total = 25;
for (int number = 1; number <= (total / 2); number++) {
    total = total – number;
    System.out.println(total + " " + number);
}
```
c.
java
Copy code
```
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 3; j++) {
        for (int k = 1; k <= 4; k++) {
            System.out.print("*");
        }
        System.out.print("!");
    }
    System.out.println();
}
```
d.
java
Copy code
```
int number = 4;
for (int count = 1; count <= number; count++) {
    System.out.println(number);
    number = number / 2;
}
```
    3. Xem xét phương thức sau:
java

Copy code
```
public static void mystery(int x) {
    int y = 1;
    int z = 0;
    while (2 * y <= x) {
        y = y * 2;
        z++;
    }
    System.out.println(y + " " + z);
}
```
Với mỗi lời gọi sau, hãy chỉ ra kết quả mà phương thức trên tạo ra:
java
Copy code
```
mystery(1);
mystery(6);
mystery(19);
mystery(39);
mystery(74);
```
4. Xem xét phương thức sau:
java
Copy code
```
public static void mystery(int x) {
    int y = 0;
    while (x % 2 == 0) {
        y++;
        x = x / 2;
    }
    System.out.println(x + " " + y);
}
```
Với mỗi lời gọi sau, hãy chỉ ra kết quả mà phương thức trên tạo ra:
java

Copy code
```
mystery(19);
mystery(42);
mystery(48);
mystery(40);
mystery(64);
```
5. Xem xét mã sau:

java
Copy code
```
Random rand = new Random();
int a = rand.nextInt(100);
int b = rand.nextInt(20) + 50;
int c = rand.nextInt(20 + 50);
int d = rand.nextInt(100) – 20;
int e = rand.nextInt(10) * 4;
```
Giá trị phạm vi của mỗi biến a, b, c, d, và e có thể là gì?

6. Viết mã tạo ra một số nguyên ngẫu nhiên trong khoảng từ 0 đến 10 bao gồm cả hai.
7. Viết mã tạo ra một số nguyên lẻ ngẫu nhiên (không chia hết cho 2) trong khoảng từ 50 đến 99 bao gồm cả hai.
8. Với mỗi vòng lặp do/while sau đây, hãy cho biết số lần vòng lặp sẽ thực thi thân của nó. Hãy nhớ rằng các câu trả lời "không", "vô hạn" và "không xác định" là hợp lệ. Ngoài ra, kết quả đầu ra của mã trong mỗi trường hợp là gì?

---

1. Sử dụng vòng lặp do/while trong các ví dụ sau:

a.

java
Copy code
```
int x = 1;
do {
   System.out.print(x + " ");
   x = x + 10;
} while (x < 100);
```

b.
java
Copy code
```
int max = 10;
do {
    System.out.println("count down: " + max);
    max—;
} while (max < 10);
```
c.
java
Copy code
```
int x = 250;
do {
    System.out.println(x);
} while (x % 3 != 0);
```
d.
java
Copy code
```
int x = 100;
do {
    System.out.println(x);
    x = x / 2;
} while (x % 2 == 0);
```
e.
java
Copy code
```
int x = 2;
do {
    System.out.print(x + " ");
    x *= x;
} while (x < 200);
```
f.
java

Copy code
```
String word = "a";
do {
    word = "b" + word + "b";
} while (word.length() < 10);
System.out.println(word);
```
g.
java
Copy code
```
int x = 100;
do {
    System.out.println(x / 10);
    x = x / 2;
} while (x > 0);
```
h.
java
Copy code
```
String str = "/\\";
do {
    str += str;
} while (str.length() < 10);
System.out.println(str);
```
2. Viết một vòng lặp do/while để in ra một thông báo nhất định cho đến khi người dùng yêu cầu chương trình dừng lại. Vòng lặp do/while là phù hợp vì thông báo nên luôn được in ít nhất một lần, ngay cả khi người dùng nhập n sau khi thông báo đầu tiên xuất hiện. Thông báo cần in là như sau:
java
Copy code
```
Scanner scanner = new Scanner(System.in);
String response;
do {
    System.out.println("She sells seashells by the seashore.");
```

```
    System.out.print("Do you want to hear it again? ");
    response = scanner.next();
} while (response.equals("y"));
```

3. Viết một phương thức có tên zeroDigits nhận vào một tham số kiểu số nguyên và trả về số lượng chữ số trong số đó có giá trị bằng 0. Ví dụ, lời gọi zeroDigits(5024036) sẽ trả về 2, và zeroDigits(743) sẽ trả về 0. Lời gọi zeroDigits(0) sẽ trả về 1. (Chúng tôi đề xuất bạn sử dụng vòng lặp do/while trong giải pháp của bạn.)

java
Copy code

```
public static int zeroDigits(int number) {
    int count = 0;
    if (number == 0) return 1;
    do {
        if (number % 10 == 0) count++;
        number /= 10;
    } while (number != 0);
    return count;
}
```

4. Viết một vòng lặp do/while để in ra các số ngẫu nhiên trong khoảng từ 0 đến 1000 cho đến khi một số lớn hơn 900 được in. Ít nhất một dòng kết quả nên được in, ngay cả khi số ngẫu nhiên đầu tiên lớn hơn 900. Dưới đây là một ví dụ thực thi:

java
Copy code

```
Random rand = new Random();
int number;
do {
    number = rand.nextInt(1001);
    System.out.println("Random number: " + number);
} while (number <= 900);
```

Phần 5.2: Thuật toán Fencepost

12. Xem xét phương thức printLetters bị lỗi dưới đây, phương thức này nhận một chuỗi ký tự làm tham số và cố gắng in các ký tự của chuỗi, cách nhau bằng dấu gạch ngang. Ví dụ, khi gọi printLetters("Rabbit") thì đầu ra cần là R-a-b-b-i-t. Tuy nhiên, mã sau đây là không đúng:

java

Copy code

```
public static void printLetters(String text) {
   for (int i = 0; i < text.length(); i++) {
      System.out.print(text.charAt(i) + "–");
   }
   System.out.println(); // để kết thúc dòng kết quả
}
```

Vấn đề: Mã trên sẽ in thêm một dấu gạch ngang ở cuối kết quả. Điều này xảy ra vì vòng lặp luôn thêm dấu gạch ngang sau mỗi ký tự, kể cả ký tự cuối cùng.

Giải pháp: Chỉ thêm dấu gạch ngang giữa các ký tự, không phải sau ký tự cuối cùng. Có thể sửa mã như sau:

java

Copy code

```
public static void printLetters(String text) {
   for (int i = 0; i < text.length(); i++) {
      if (i > 0) {
         System.out.print("-");
      }
      System.out.print(text.charAt(i));
   }
   System.out.println(); // để kết thúc dòng kết quả
}
```

13. Viết một vòng lặp sentinel yêu cầu người dùng nhập số, và khi số -1 được nhập, hiển thị số lớn nhất và nhỏ nhất mà người dùng đã nhập. Dưới đây là một đoạn hội thoại mẫu:

less

Copy code

```
Nhập một số (hoặc -1 để dừng):
5
```

Nhập một số (hoặc -1 để dừng):
2

Nhập một số (hoặc -1 để dừng):
17

Nhập một số (hoặc -1 để dừng):
8

Nhập một số (hoặc -1 để dừng):
-1

Số lớn nhất là 17

Số nhỏ nhất là 2
Nếu -1 là số đầu tiên được nhập, không có số lớn nhất hoặc nhỏ nhất nào được in ra. Trong trường hợp này, hội thoại sẽ trông như sau:
css
Copy code
Nhập một số (hoặc -1 để dừng):
-1
Phần 5.3: Kiểu boolean
14. Xem xét các khai báo biến sau:
java
Copy code
int x = 27;
int y = -1;
int z = 32;
boolean b = false;
Giá trị của mỗi biểu thức Boolean dưới đây là gì?
a. !b → true
b. b || true → true

c. (x > y) && (y > z) → false

d. (x == y) || (x <= z) → true

e. !(x % 2 == 0) → true

f. (x % 2 != 0) && b → false

g. b && !b → false

h. b || !b → true

i. (x < y) == b → false

j. !(x / 2 == 13) || b || (z * 3 == 96) → true

k. (z < x) == false → true

l. !((x > 0) && (y < 0)) → false

15. Viết một phương thức có tên isVowel nhận vào một ký tự và trả về true nếu ký tự đó là nguyên âm (a, e, i, o, hoặc u). Để tăng độ khó, hãy làm cho phương thức của bạn không phân biệt chữ hoa và chữ thường.

java

Copy code

```
public static boolean isVowel(char ch) {
    ch = Character.toLowerCase(ch);
    return ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u';
}
```

16. Mã sau đây cố gắng kiểm tra một số và trả về kết quả xem số đó có phải là số nguyên tố (không có ước số nào khác ngoài 1 và chính nó) hay không. Một cờ tên prime được sử dụng. Tuy nhiên, logic Boolean không được thực hiện đúng, do đó phương thức này không luôn trả về kết quả chính xác. Trong những trường hợp nào thì phương thức báo cáo kết quả sai? Làm thế nào để mã có thể được thay đổi để luôn trả về kết quả chính xác?

java

Copy code

```
public static boolean isPrime(int n) {
    boolean prime = true;
    for (int i = 2; i < n; i++) {
        if (n % i == 0) {
            prime = false;
        } else {
            prime = true;
```

```
        }
    }
    return prime;
}
```

Vấn đề: Phương thức này sẽ trả về true nếu phép chia cuối cùng không chia hết, ngay cả khi đã tìm thấy một ước số trước đó.

Giải pháp: Loại bỏ else và chỉ cần gán prime là false nếu tìm thấy một ước số. Mã sửa đổi:

java

Copy code

```
public static boolean isPrime(int n) {
    boolean prime = true;
    for (int i = 2; i < n; i++) {
        if (n % i == 0) {
            prime = false;
            break;
        }
    }
    return prime;
}
```

17. Mã sau đây cố gắng kiểm tra một chuỗi và trả về kết quả xem chuỗi đó có chứa một ký tự nhất định hay không. Một cờ tên found được sử dụng. Tuy nhiên, logic Boolean không được thực hiện đúng, do đó phương thức này không luôn trả về kết quả chính xác. Trong những trường hợp nào thì phương thức báo cáo kết quả sai? Làm thế nào để mã có thể được thay đổi để luôn trả về kết quả chính xác?

java

Copy code

```
public static boolean contains(String str, char ch) {
    boolean found = false;
    for (int i = 0; i < str.length(); i++) {
        if (str.charAt(i) == ch) {
            found = true;
        } else {
            found = false;
```

```
        }
    }
    return found;
}
```

Vấn đề: Cờ found sẽ bị đặt lại thành false nếu ký tự tiếp theo không khớp, ngay cả khi ký tự đã được tìm thấy trước đó.

Giải pháp: Loại bỏ else và chỉ đặt found thành true nếu ký tự được tìm thấy. Mã sửa đổi:

java

Copy code

```
public static boolean contains(String str, char ch) {
    boolean found = false;
    for (int i = 0; i < str.length(); i++) {
        if (str.charAt(i) == ch) {
            found = true;
            break;
        }
    }
    return found;
}
```

18. Sử dụng “Boolean Zen,” viết phiên bản cải tiến của phương thức sau, phương thức này trả về kết quả xem chuỗi cho trước có bắt đầu và kết thúc bằng cùng một ký tự hay không:

java

Copy code

```
public static boolean startEndSame(String str) {
    return str.charAt(0) == str.charAt(str.length() - 1);
}
```

19. Sử dụng “Boolean Zen,” viết phiên bản cải tiến của phương thức sau, phương thức này trả về kết quả xem số cent cho trước có cần đồng xu lẻ (pennies) hay không (ngược lại với số cent có thể được tạo thành chính xác bằng các đồng xu khác ngoài pennies):

java

Copy code

```
public static boolean hasPennies(int cents) {
    return cents % 5 != 0;
```

}

20. Xem xét phương thức sau:

java

Copy code

```
public static int mystery(int x, int y) {
    while (x != 0 && y != 0) {
        if (x < y) {
            y -= x;
        } else {
            x -= y;
        }
    }
    return x + y;
}
```

Đối với mỗi lời gọi sau đây, xác định giá trị trả về:

- mystery(3, 3) → 3
- mystery(5, 3) → 2
- mystery(2, 6) → 2
- mystery(12, 18) → 6
- mystery(30, 75) → 15

21. Mã sau đây là phiên bản được sửa đổi một chút của mã thực tế đã có trong máy nghe nhạc Microsoft Zune vào năm 2008. Mã này cố gắng tính toán ngày hôm nay bằng cách xác định bao nhiêu năm và ngày đã trôi qua kể từ năm 1980. Giả sử tồn tại các phương thức để lấy tổng số ngày kể từ năm 1980 và để xác định một năm có phải là năm nhuận hay không:

java

Copy code

```
int days = getTotalDaysSince1980();
int year = 1980;
while (days > 365) { // trừ các năm
    if (isLeapYear(year)) {
        if (days > 366) {
            days -= 366;
            year += 1;
```

```
        }
    } else {
        days -= 365;
        year += 1;
    }
}
```

Vấn đề: Hàng nghìn máy nghe nhạc Zune đã bị khóa vào ngày 1 tháng 1 năm 2009, ngày đầu tiên sau khi kết thúc một năm nhuận kể từ khi Zune được phát hành. (Microsoft đã nhanh chóng phát hành bản vá để khắc phục sự cố này.) Vấn đề của mã trên là gì và trong những trường hợp nào nó sẽ hiển thị hành vi không chính xác? Làm thế nào để khắc phục?

Giải pháp: Nếu năm là năm nhuận, cần kiểm tra days có lớn hơn 365 thay vì 366. Mã sửa đổi:

java

Copy code

```
int days = getTotalDaysSince1980();
int year = 1980;
while (days > 365) {
    if (isLeapYear(year)) {
        if (days > 365) {
            days -= 366;
            year += 1;
        }
    } else {
        days -= 365;
        year += 1;
    }
}
```

22. Câu nào sau đây là phiên bản đảo ngược đúng của biểu thức Boolean sau đây, theo luật De Morgan?

java

Copy code

```
(2 == 3) && (-1 < 5) && isPrime(n)
```

a. (2 != 3) && (-1 > 5) && isPrime(n)

b. (2 == 3) || (-1 < 5) || isPrime(n)

c. !(2 == 3) && !(-1 < 5) && !isPrime(n)

d. (2 != 3) || (-1 >= 5) || !isPrime(n)

e. !(2 != 3) || !(-1 < 5) || isNotPrime(n)

Phần 5.4: Lỗi người dùng

23. Mã sau đây không đủ mạnh để xử lý đầu vào không hợp lệ từ người dùng. Mô tả cách thay đổi mã để nó không tiếp tục cho đến khi người dùng nhập một độ tuổi và điểm GPA hợp lệ. Giả sử bất kỳ int nào cũng là độ tuổi hợp lệ và bất kỳ double nào cũng là GPA hợp lệ.

java

Copy code

```
Scanner console = new Scanner(System.in);
System.out.print("Nhập tuổi của bạn: ");
int age = console.nextInt();
System.out.print("Nhập GPA của bạn: ");
double gpa = console.nextDouble();
```

Thách thức thêm: Sửa mã để từ chối các độ tuổi không hợp lệ (ví dụ: số nhỏ hơn 0) và GPA không hợp lệ (ví dụ: số nhỏ hơn 0.0 hoặc lớn hơn 4.0).

java

Copy code

```
Scanner console = new Scanner(System.in);
int age;
do {
    System.out.print("Nhập tuổi của bạn: ");
    age = console.nextInt();
} while (age < 0);
double gpa;
do {
    System.out.print("Nhập GPA của bạn: ");
    gpa = console.nextDouble();
} while (gpa < 0.0 || gpa > 4.0);
```

24. Xem xét mã sau:

java

Copy code

```
Scanner console = new Scanner(System.in);
System.out.print("Nhập cái gì đó cho tôi! ");
if (console.hasNextInt()) {
    int number = console.nextInt();
    System.out.println("IQ của bạn là " + number);
} else if (console.hasNext()) {
    String token = console.next();
    System.out.println("Tên của bạn là " + token);
}
```

Kết quả khi người dùng nhập các giá trị sau:

a. Jane

Kết quả: Tên của bạn là Jane

b. 56

Kết quả: IQ của bạn là 56

c. 56.2

Kết quả: Tên của bạn là 56.2

25. Viết một đoạn mã yêu cầu người dùng nhập một số và sau đó in một thông báo khác nhau tùy thuộc vào việc số đó là số nguyên hay số thực. Dưới đây là hai đoạn hội thoại mẫu:

less

Copy code

```
Nhập một số:
42.5
Bạn đã nhập số thực 42.5
```

less

Copy code

```
Nhập một số:
3
Bạn đã nhập số nguyên 3
```

26. Viết mã yêu cầu người dùng nhập ba số nguyên, tính trung bình của chúng và in trung bình. Làm cho mã của bạn đủ mạnh để xử lý đầu vào không hợp lệ. (Bạn có thể muốn sử dụng phương thức getInt được thảo luận trong chương này.)

java

Copy code

```
Scanner console = new Scanner(System.in);
int num1, num2, num3;
do {
    System.out.print("Nhập số nguyên thứ nhất: ");
    while (!console.hasNextInt()) {
        System.out.print("Vui lòng nhập một số nguyên hợp lệ: ");
        console.next();
    }
    num1 = console.nextInt();
} while (num1 < Integer.MIN_VALUE || num1 > Integer.MAX_VALUE);

do {
    System.out.print("Nhập số nguyên thứ hai: ");
    while (!console.hasNextInt()) {
        System.out.print("Vui lòng nhập một số nguyên hợp lệ: ");
        console.next();
    }
    num2 = console.nextInt();
} while (num2 < Integer.MIN_VALUE || num2 > Integer.MAX_VALUE);

do {
    System.out.print("Nhập số nguyên thứ ba: ");
    while (!console.hasNextInt()) {
        System.out.print("Vui lòng nhập một số nguyên hợp lệ: ");
        console.next();
    }
    num3 = console.nextInt();
} while (num3 < Integer.MIN_VALUE || num3 > Integer.MAX_VALUE);

double average = (num1 + num2 + num3) / 3.0;
System.out.println("Trung bình của ba số là " + average);
```

Phần 5.5: Các khẳng định và logic chương trình

27. Xác định các khẳng định khác nhau trong mã sau đây là luôn đúng, không bao giờ đúng, hoặc thỉnh thoảng đúng và thỉnh thoảng sai ở các điểm khác nhau trong quá trình thực thi chương trình. Các chú thích chỉ ra các điểm quan tâm:

java

Copy code

```
public static int mystery(Scanner console, int x) {
   int y = console.nextInt();
   int count = 0;
   // Điểm A
   while (y < x) {
      // Điểm B
      if (y == 0) {
         count++;
         // Điểm C
      }
      y = console.nextInt();
      // Điểm D
   }
   // Điểm E
   return count;
}
```

Phân loại từng khẳng định ở mỗi điểm với LUÔN, KHÔNG BAO GIỜ, hoặc THỈNH THOẢNG:

- y < x
    - Điểm A: THỈNH THOẢNG
    - Điểm B: THỈNH THOẢNG
    - Điểm C: THỈNH THOẢNG
    - Điểm D: THỈNH THOẢNG
    - Điểm E: KHÔNG BAO GIỜ
- y == 0
    - Điểm A: KHÔNG BAO GIỜ
    - Điểm B: THỈNH THOẢNG
    - Điểm C: LUÔN

    - Điểm D: KHÔNG BAO GIỜ
    - Điểm E: KHÔNG BAO GIỜ
- count > 0
    - Điểm A: KHÔNG BAO GIỜ
    - Điểm B: KHÔNG BAO GIỜ
    - Điểm C: THỈNH THOẢNG
    - Điểm D: THỈNH THOẢNG
    - Điểm E: LUÔN

28. Xác định các khẳng định khác nhau trong mã sau đây là luôn đúng, không bao giờ đúng, hoặc thỉnh thoảng đúng và thỉnh thoảng sai ở các điểm khác nhau trong quá trình thực thi chương trình. Các chú thích chỉ ra các điểm quan tâm:

java

Copy code

```
public static int mystery(int n) {
    Random r = new Random();
    int a = r.nextInt(3) + 1;
    int b = 2;
    // Điểm A
    while (n > b) {
        // Điểm B
        b = b + a;
        if (a > 1) {
            n--;
            // Điểm C
            a = r.nextInt(b) + 1;
        } else {
            a = b + 1;
            // Điểm D
        }
    }
    // Điểm E
    return n;
}
```

Phân loại từng khẳng định ở mỗi điểm với LUÔN, KHÔNG BAO GIỜ, hoặc THỈNH THOẢNG:

- n > b
    - Điểm A: LUÔN
    - Điểm B: THỈNH THOẢNG
    - Điểm C: THỈNH THOẢNG
    - Điểm D: KHÔNG BAO GIỜ
    - Điểm E: KHÔNG BAO GIỜ
- a > 1
    - Điểm A: THỈNH THOẢNG
    - Điểm B: THỈNH THOẢNG
    - Điểm C: THỈNH THOẢNG
    - Điểm D: LUÔN
    - Điểm E: KHÔNG BAO GIỜ
- b > a
    - Điểm A: THỈNH THOẢNG
    - Điểm B: THỈNH THOẢNG
    - Điểm C: THỈNH THOẢNG
    - Điểm D: THỈNH THOẢNG
    - Điểm E: KHÔNG BAO GIỜ

29. Xác định các khẳng định khác nhau trong mã sau đây là luôn đúng, không bao giờ đúng, hoặc thỉnh thoảng đúng và thỉnh thoảng sai ở các điểm khác nhau trong quá trình thực thi chương trình. Các chú thích chỉ ra các điểm quan tâm:

java

Copy code

```
public static int mystery(Scanner console) {
   int prev = 0;
   int count = 0;
   int next = console.nextInt();
   // Điểm A
   while (next != 0) {
      // Điểm B
      if (next == prev) {
```

```
            // Điểm C
            count++;
        }
        prev = next;
        next = console.nextInt();
        // Điểm D
    }
    // Điểm E
    return count;
}
```

Phân loại từng khẳng định ở mỗi điểm với LUÔN, KHÔNG BAO GIỜ, hoặc THỈNH THOẢNG:

- next == 0
    - Điểm A: KHÔNG BAO GIỜ
    - Điểm B: THỈNH THOẢNG
    - Điểm C: KHÔNG BAO GIỜ
    - Điểm D: THỈNH THOẢNG
    - Điểm E: LUÔN
- prev == 0
    - Điểm A: LUÔN
    - Điểm B: KHÔNG BAO GIỜ
    - Điểm C: KHÔNG BAO GIỜ
    - Điểm D: KHÔNG BAO GIỜ
    - Điểm E: KHÔNG BAO GIỜ
- next == prev
    - Điểm A: KHÔNG BAO GIỜ
    - Điểm B: THỈNH THOẢNG
    - Điểm C: LUÔN
    - Điểm D: THỈNH THOẢNG
    - Điểm E: KHÔNG BAO GIỜ

Bài Tập

1. Viết một phương thức gọi là showTwos để hiển thị các yếu tố của số 2 trong một số nguyên cho trước. Ví dụ, xem xét các lệnh gọi sau:

java

Copy code

```
showTwos(7);
showTwos(18);
showTwos(68);
showTwos(120);
```

Các lệnh gọi này nên tạo ra đầu ra sau:

Copy code

```
7
 = 7
18
 = 2 * 9
68
 = 2 * 2 * 17
120
 = 2 * 2 * 2 * 15
```

2. Viết một phương thức gọi là gcd nhận hai số nguyên làm tham số và trả về ước số chung lớn nhất (GCD) của hai số. GCD của hai số nguyên a và b là số nguyên lớn nhất là yếu tố của cả a và b. Một cách hiệu quả để tính GCD của hai số là sử dụng thuật toán Euclid, nó có dạng:

css

Copy code

```
GCD(a, b) = GCD(b, a % b)
GCD(a, 0) = Giá trị tuyệt đối của a
```

3. Viết một phương thức gọi là toBinary nhận một số nguyên làm tham số và trả về một chuỗi chứa biểu diễn nhị phân của số đó. Ví dụ, lệnh gọi printBinary(44) nên trả về "101100".

4. Viết một phương thức gọi là randomX để in ra các dòng chứa số lượng ngẫu nhiên các ký tự “x” (từ 5 đến 20, bao gồm cả hai) cho đến khi in ra một dòng chứa 16 ký tự hoặc nhiều hơn. Ví dụ, đầu ra có thể trông như sau:

Copy code

```
xxxxxxx
xxxxxxxxxxxxxxx
```

xxxxxxxxxxxx

xxxxxxxxxxxxxx

xxxxxx

xxxxxxxxxxx

xxxxxxxxxxxxxxxxx

5. Viết một phương thức gọi là randomLines để in từ 5 đến 10 chuỗi ngẫu nhiên của các ký tự (từ “a” đến “z”), mỗi chuỗi trên một dòng. Mỗi chuỗi nên có độ dài ngẫu nhiên lên đến 80 ký tự.

6. Viết một phương thức gọi là makeGuesses để đoán các số từ 1 đến 50 bao gồm cả hai cho đến khi đoán được ít nhất 48. Nó nên báo cáo mỗi lần đoán và tổng số lần đoán được thực hiện. Đây là một ví dụ về kết quả thực thi:

makefile

Copy code

guess = 43

guess = 47

guess = 45

guess = 27

guess = 49

total guesses = 5

7. Viết một phương thức gọi là diceSum nhận một Scanner cho console làm tham số và yêu cầu tổng số mong muốn, sau đó liên tục mô phỏng việc tung 2 viên xí ngầu sáu mặt cho đến khi tổng của chúng bằng tổng mong muốn. Đây là một ví dụ đối thoại với người dùng:

r

Copy code

Tổng số xí ngầu mong muốn:

9

4 và 3 = 7

3 và 5 = 8

5 và 6 = 11

5 và 6 = 11

1 và 5 = 6

6 và 3 = 9

8. Viết một phương thức gọi là randomWalk để thực hiện các bước của một cuộc đi bộ ngẫu nhiên một chiều. Cuộc đi bộ ngẫu nhiên nên bắt đầu từ vị trí 0. Mỗi bước, bạn nên tăng hoặc

giảm vị trí thêm 1 (mỗi khả năng với xác suất bằng nhau). Mã của bạn nên tiếp tục thực hiện các bước cho đến khi đạt được vị trí 3 hoặc −3, và sau đó báo cáo vị trí tối đa đã đạt được trong suốt quá trình đi bộ. Đầu ra nên trông như sau:

arduino

Copy code

```
position = 1
position = 0
position = –1
position = –2
position = –1
position = –2
position = –3
max position = 1
```

9. Viết một phương thức gọi là printFactors nhận một số nguyên làm tham số và sử dụng vòng lặp fencepost để in các yếu tố của số đó, cách nhau bởi từ "and". Ví dụ, các yếu tố của số 24 nên được in như sau:

Copy code

```
1 và 2 và 3 và 4 và 6 và 8 và 12 và 24
```

Bạn có thể giả định rằng giá trị của tham số lớn hơn 0, hoặc bạn có thể ném một ngoại lệ nếu nó là 0 hoặc số âm.

10. Viết một phương thức gọi là hopscotch nhận một số nguyên số lượng “hop” làm tham số và in ra một mẫu số giống như bảng nhảy dây. Một “hop” là một chuỗi ba số mà đầu ra hiển thị hai số trên một dòng, tiếp theo là một số trên một dòng riêng biệt. 0 hop là bảng đến số 1; một hop là bảng đến số 4; hai hop là bảng đến số 7; và cứ như vậy. Ví dụ, lệnh gọi hopscotch(3) nên in ra đầu ra sau:

markdown

Copy code

```
  1
2
   3
  4
5
   6
```

7
8
9
10
Lệnh gọi hopscotch(0) nên chỉ in số 1. Nếu nó nhận giá trị âm, phương thức nên không in gì cả.

11. Viết một phương thức gọi là threeHeads liên tục lật đồng xu cho đến khi kết quả của lần lật đồng xu là ba mặt ngửa liên tiếp. Bạn nên sử dụng một đối tượng Random để đảm bảo rằng mặt ngửa hoặc mặt sấp có xác suất bằng nhau. Mỗi lần đồng xu được lật, hiển thị H cho mặt ngửa hoặc T cho mặt sấp. Khi ba mặt ngửa liên tiếp được lật, phương thức nên in một thông điệp chúc mừng. Đây là một ví dụ đầu ra của lệnh gọi phương thức:

r

Copy code

T T H T T T H T H T H H H

Ba mặt ngửa liên tiếp!

12. Viết một phương thức gọi là printAverage sử dụng vòng lặp sentinel để liên tục yêu cầu người dùng nhập số. Khi người dùng nhập bất kỳ số nào nhỏ hơn 0, phương thức nên hiển thị trung bình của tất cả các số không âm đã nhập. Hiển thị trung bình dưới dạng số thực. Đây là một ví dụ đối thoại với người dùng:

less

Copy code

Nhập một số:

7

Nhập một số:

4

Nhập một số:

16

Nhập một số:

–4

Trung bình là 9.0

Nếu số đầu tiên mà người dùng nhập là số âm, không in trung bình:

css

Copy code

Nhập một số: –2

13. Viết một phương thức gọi là consecutive nhận ba số nguyên làm tham số và trả về true nếu chúng là ba số liên tiếp—tức là, nếu các số có thể được sắp xếp theo thứ tự sao cho, giả sử một số nguyên k, các giá trị của tham số là k, k + 1, và k + 2. Phương thức của bạn nên trả về false nếu các số không liên tiếp. Lưu ý rằng thứ tự không quan trọng; phương thức của bạn nên trả về kết quả giống nhau cho ba số nguyên được truyền vào theo bất kỳ thứ tự nào. Ví dụ, các lệnh gọi consecutive(1, 2, 3), consecutive(3, 2, 4), và consecutive(–10, –8, –9) sẽ trả về true. Các lệnh gọi consecutive(3, 5, 7), consecutive(1, 2, 2), và consecutive(7, 7, 9) sẽ trả về false.

14. Viết một phương thức gọi là hasMidpoint nhận ba số nguyên làm tham số và trả về true nếu một trong các số nguyên là điểm giữa giữa hai số nguyên còn lại; tức là, nếu một số nguyên nằm chính xác ở giữa hai số đó. Phương thức của bạn nên trả về false nếu không có mối quan hệ điểm giữa nào tồn tại. Ví dụ, lệnh gọi hasMidpoint(7, 4, 10) nên trả về true vì 7 nằm chính giữa 4 và 10. Ngược lại, lệnh gọi hasMidpoint(9, 15, 8) nên trả về false vì không có số nguyên nào nằm chính giữa hai số còn lại. Các số nguyên có thể được truyền theo bất kỳ thứ tự nào; điểm giữa có thể là số 1, số 2, hoặc số 3. Bạn phải kiểm tra tất cả các trường hợp. Nếu phương thức của bạn nhận ba giá trị giống nhau, hãy trả về true.

15. Viết một phương thức gọi là dominant nhận ba số nguyên làm tham số và trả về true nếu bất kỳ số nào trong ba số nguyên lớn hơn tổng của hai số còn lại. Các số nguyên có thể được truyền theo bất kỳ thứ tự nào, vì vậy giá trị lớn nhất có thể là bất kỳ số nào trong ba số. Ví dụ, lệnh gọi dominant(4, 9, 2) trả về true vì 9 lớn hơn 4 + 2. Giả sử không số nào là số âm.

16. Viết một phương thức gọi là anglePairs nhận ba góc (số nguyên), đo bằng độ, làm tham số và trả về liệu có tồn tại cả góc bổ sung và góc phụ trong số ba góc được truyền vào. Hai góc là bổ sung nếu tổng của chúng bằng chính xác 90 độ; hai góc là phụ nếu tổng của chúng bằng chính xác 180 độ. Do đó, phương thức nên trả về true nếu bất kỳ hai trong số ba góc cộng lại thành 90 độ và cũng có bất kỳ hai trong số ba góc cộng lại thành 180 độ. Ví dụ, lệnh gọi anglePairs(120, 60, 30) trả về true. Giả sử rằng mỗi góc được truyền vào là không âm.

17. Viết một phương thức gọi là monthApart nhận bốn số nguyên, m1, d1, m2, và d2, đại diện cho hai ngày lịch. Mỗi ngày bao gồm một tháng (1 đến 12) và một ngày (1 đến số ngày trong tháng đó [28–31]). Giả sử rằng tất cả các giá trị tham số được truyền vào là hợp lệ. Phương thức nên trả về true nếu hai ngày cách nhau ít nhất một tháng và false nếu không. Ví dụ, lệnh gọi monthApart(4, 15, 5, 22) sẽ trả về true trong khi lệnh gọi monthApart(9, 19, 10, 17) sẽ trả về false. Giả sử rằng tất cả các ngày trong bài toán này xảy ra trong cùng một năm. Lưu ý rằng ngày đầu tiên có thể đến trước hoặc sau ngày thứ hai.

18. Viết một phương thức gọi là digitSum nhận một số nguyên làm tham số và trả về tổng các chữ số của số đó. Ví dụ, lệnh gọi digitSum(29107) trả về 2+9+1+0+7 hoặc 19. Đối với số âm, trả về giá trị giống như số dương. Ví dụ, digitSum(-456) trả về 4+5+6 hoặc 15. Lệnh gọi digitSum(0) trả về 0.

19. Viết một phương thức gọi là firstDigit trả về chữ số đầu tiên (chữ số quan trọng nhất) của một số nguyên. Ví dụ, firstDigit(3572) nên trả về 3. Nó nên hoạt động cho các số âm cũng vậy; firstDigit(-947) nên trả về 9.

20. Viết một phương thức gọi là digitRange nhận một số nguyên làm tham số và trả về phạm vi giá trị của các chữ số của số đó. Phạm vi được định nghĩa là 1 cộng với sự khác biệt giữa giá trị chữ số lớn nhất và nhỏ nhất. Ví dụ, lệnh gọi digitRange(68437) sẽ trả về 6 vì giá trị chữ số lớn nhất là 8 và giá trị chữ số nhỏ nhất là 3, do đó 8−3+1=6. Nếu số chứa chỉ một chữ số, trả về 1. Bạn nên giải quyết bài toán này mà không sử dụng chuỗi.

21. Viết một phương thức gọi là swapDigitPairs nhận một số nguyên n làm tham số và trả về một số nguyên mới có giá trị tương tự như n nhưng với mỗi cặp chữ số được hoán đổi vị trí. Ví dụ, lệnh gọi swapDigitPairs(482596) sẽ trả về 845269. Lưu ý rằng 9 và 6 được hoán đổi, cũng như 2 và 5, và 4 và 8. Nếu số có số chữ số lẻ, giữ chữ số bên trái nhất ở vị trí gốc của nó. Ví dụ, lệnh gọi swapDigitPairs(1234567) sẽ trả về 1325476. Bạn nên giải quyết bài toán này mà không sử dụng chuỗi.

22. Viết một phương thức gọi là allDigitsOdd trả về liệu mỗi chữ số của một số nguyên dương đều là số lẻ. Trả về true nếu số chỉ bao gồm các chữ số lẻ (1, 3, 5, 7, 9) và false nếu bất kỳ chữ số nào của nó là số chẵn (0, 2, 4, 6, 8). Ví dụ, lệnh gọi allDigitsOdd(135319) trả về true nhưng allDigitsOdd(9145293) trả về false.

23. Viết một phương thức gọi là hasAnOddDigit trả về liệu một số nguyên dương cho trước có ít nhất một chữ số có giá trị là số lẻ. Trả về true nếu số có ít nhất một chữ số lẻ và false nếu không có chữ số nào là số lẻ. Ví dụ, lệnh gọi hasAnOddDigit(4822116) nên trả về true và hasAnOddDigit(2448) nên trả về false.

24. Viết một phương thức gọi là isAllVowels trả về liệu một chuỗi chỉ bao gồm các nguyên âm (a, e, i, o, u, không phân biệt chữ hoa chữ thường). Nếu và chỉ nếu mỗi ký tự của chuỗi là một nguyên âm, phương thức của bạn nên trả về true. Ví dụ, lệnh gọi isAllVowels("eIEiO") trả về true và isAllVowels("oink") trả về false. Bạn nên trả về true nếu chuỗi được truyền vào là chuỗi rỗng, vì nó không chứa bất kỳ ký tự không phải nguyên âm nào.

25. Viết một phương thức gọi là charsSorted nhận một chuỗi làm tham số và trả về true nếu các ký tự trong chuỗi xuất hiện theo thứ tự chữ cái đã được sắp xếp. Ví dụ, các lệnh gọi

charsSorted("abcde") và charsSorted("bins") nên trả về true, nhưng lệnh gọi charsSorted("beads") nên trả về false.

Programming Projects

1. Viết một chương trình tương tác để đọc các dòng đầu vào từ người dùng và chuyển mỗi dòng thành "Pig Latin." Pig Latin là tiếng Anh với âm đầu tiên được chuyển về cuối mỗi từ, sau đó thêm "ay." Các từ bắt đầu bằng nguyên âm chỉ cần thêm "ay." Ví dụ, cụm từ
    - The deepest shade of mushroom blue
    - Sẽ có dạng Pig Latin như sau:
    - e-Thay eepest-day ade-shay of-ay ushroom-may ue-blay
    - Kết thúc chương trình khi người dùng nhập vào một dòng trống.
2. Viết một trò chơi Hangman ngược, trong đó người dùng nghĩ về một từ và máy tính cố gắng đoán các chữ cái trong từ đó. Người dùng sẽ cho biết máy tính từ đó có bao nhiêu chữ cái.
3. Viết một chương trình chơi trò chơi đoán với người dùng. Chương trình sẽ tạo ra một số ngẫu nhiên từ 1 đến một giá trị tối đa nào đó (ví dụ: 100), sau đó yêu cầu người dùng đoán số đó liên tục. Khi người dùng đoán sai, trò chơi sẽ đưa ra gợi ý về việc số đúng là lớn hơn hay nhỏ hơn số người dùng đã đoán. Khi người dùng đoán đúng, chương trình sẽ in ra một thông báo cho biết số lần người dùng đã đoán.
    - Xem xét mở rộng chương trình này bằng cách cho phép chơi nhiều trò chơi cho đến khi người dùng chọn dừng và sau đó in thống kê về tổng số lần đoán và số lần đoán trung bình của người chơi.
4. Viết một chương trình chơi trò chơi đoán ngược với người dùng. Người dùng nghĩ về một số từ 1 đến 10, và máy tính cố gắng đoán nó bằng cách đoán các số ngẫu nhiên. Máy tính có thể đoán cùng một số ngẫu nhiên nhiều hơn một lần. Cuối trò chơi, chương trình sẽ báo cáo số lần đoán của máy tính. Ví dụ thực thi:
    - Chương trình này yêu cầu bạn, người dùng, chọn một số từ 1 đến 10. Sau đó, tôi, máy tính, sẽ cố gắng đoán nó.
    - Là 8 phải không? (y/n) n
    - Là 7 phải không? (y/n) n
    - Là 5 phải không? (y/n) n
    - Là 1 phải không? (y/n) n
    - Là 8 phải không? (y/n) n

- Là 1 phải không? (y/n) n
- Là 9 phải không? (y/n) y
- Tôi đã đoán đúng số 9 của bạn trong 7 lần đoán.
- Thử thách thêm: Hãy cho máy tính gợi ý xem số đúng là lớn hơn hay nhỏ hơn số mà máy tính đã đoán. Máy tính sẽ điều chỉnh phạm vi đoán ngẫu nhiên của nó dựa trên gợi ý.

5. Viết một trò chơi chơi nhiều vòng của "Kéo Búa Bao." Người dùng và máy tính sẽ chọn một trong ba vật phẩm: kéo (thắng bao nhưng thua búa), búa (thắng kéo nhưng thua bao), và bao (thắng búa nhưng thua kéo). Nếu người chơi và máy tính chọn cùng một vật phẩm, trò chơi là hòa.
    - Bạn có thể mở rộng chương trình này để bao gồm các chiến lược thuật toán khác nhau để chọn vật phẩm tốt nhất. Máy tính nên chọn ngẫu nhiên? Hay luôn chọn một vật phẩm cụ thể hoặc một mẫu lặp lại của các vật phẩm? Nên đếm số lần đối thủ chọn các vật phẩm khác nhau và dựa trên lịch sử đó để đưa ra chiến lược?
6. Viết một chương trình vẽ màn hình đồ họa của một hành trình ngẫu nhiên 2D bằng cách sử dụng DrawingPanel. Bắt đầu một điểm ảnh đi bộ ở giữa màn hình. Mỗi bước, chọn di chuyển 1 điểm ảnh lên, xuống, trái hoặc phải, sau đó vẽ lại điểm ảnh đó. (Bạn có thể vẽ một điểm ảnh đơn bằng cách vẽ một hình chữ nhật kích thước 1×1.)
7. Viết một chương trình chơi trò chơi xúc xắc “Pig.” Pig là trò chơi hai người chơi, trong đó người chơi lần lượt lăn một con xúc xắc 6 mặt; người chơi liên tục lăn xúc xắc cho đến khi xảy ra một trong hai trường hợp: người chơi chọn dừng lăn, và tổng số điểm của lần lăn đó sẽ được cộng vào tổng điểm của người chơi; hoặc nếu người chơi lăn ra số 1 ở bất kỳ lúc nào, tất cả điểm của lượt đó bị mất và lượt chơi kết thúc ngay lập tức. Người chơi đầu tiên đạt ít nhất 100 điểm sẽ thắng.

Chương 6 Xử lý Tệp tin

6.1 Các kiến thức cơ bản về Đọc Tệp tin

- Dữ liệu, Dữ liệu ở khắp mọi nơi
- Tệp tin và Đối tượng Tệp tin
- Đọc một Tệp tin với Scanner

6.2 Chi tiết về Xử lý Dựa trên Token

- Cấu trúc của Tệp tin và Tiêu thụ Dữ liệu
- Tham số của Scanner
- Đường dẫn và Thư mục
- Một Tệp đầu vào Phức tạp hơn

6.3 Xử lý Dựa trên Dòng

- Scanner Chuỗi và Kết hợp Dòng/Token

6.4 Xử lý Tệp tin Nâng cao

- Tệp đầu ra với PrintStream
- Đảm bảo rằng Tệp có thể Đọc được

6.5 Nghiên cứu Tình huống: Tìm kiếm Mã Zip

Giới thiệu

Trong Chương 3, chúng ta đã thảo luận về cách tạo một đối tượng Scanner để đọc dữ liệu từ bảng điều khiển. Bây giờ, chúng ta sẽ xem xét cách tạo các đối tượng Scanner để đọc dữ liệu từ tệp tin. Ý tưởng này khá đơn giản, nhưng Java không dễ dàng để đọc từ các tệp đầu vào. Điều này thật đáng tiếc vì nhiều vấn đề thú vị có thể được hình thành dưới dạng các nhiệm vụ xử lý tệp tin. Nhiều lớp học nhập môn khoa học máy tính đã hoàn toàn bỏ qua việc xử lý tệp tin và để chủ đề này cho khóa học thứ hai vì nó được coi là quá khó cho người mới học.

Không có gì phức tạp nội tại về xử lý tệp tin, nhưng Java không được thiết kế cho việc này và các nhà thiết kế của Java không mấy mặn mà trong việc cung cấp một giải pháp đơn giản. Tuy nhiên, họ đã giới thiệu lớp Scanner như một cách để đơn giản hóa một số chi tiết liên quan đến việc đọc tệp tin. Kết quả là việc

đọc tệp tin vẫn còn cồng kềnh trong Java, nhưng ít nhất mức độ chi tiết là có thể quản lý được.
Trước khi chúng ta bắt đầu viết các chương trình xử lý tệp tin, chúng ta phải tìm hiểu một số vấn đề liên quan đến ngoại lệ (exception) trong Java. Hãy nhớ rằng, ngoại lệ là những lỗi làm dừng việc thực thi chương trình. Trong trường hợp xử lý tệp tin, việc cố gắng mở một tệp không tồn tại hoặc cố gắng đọc vượt quá cuối tệp sẽ tạo ra một ngoại lệ.

6.1 Các Kiến thức Cơ bản về Đọc Tệp tin

Trong phần này, chúng ta sẽ xem xét các vấn đề cơ bản nhất liên quan đến xử lý tệp tin. Tệp tin là gì và tại sao chúng ta quan tâm đến chúng? Những kỹ thuật cơ bản nhất để đọc tệp tin trong một chương trình Java là gì? Khi bạn đã nắm vững những kiến thức cơ bản này, chúng ta sẽ tiếp tục thảo luận chi tiết hơn về các kỹ thuật khác nhau mà bạn có thể sử dụng để xử lý tệp tin.

Dữ liệu, Dữ liệu ở Khắp mọi nơi

Mọi người đều bị cuốn hút bởi dữ liệu. Khi lĩnh vực thống kê xuất hiện vào thế kỷ 19, đã có một sự bùng nổ trong việc thu thập và diễn giải lượng lớn dữ liệu. Mark Twain đã báo cáo rằng chính khách người Anh Benjamin Disraeli đã phàn nàn với ông: "Có ba loại dối trá: dối trá, dối trá tồi tệ và thống kê."
Sự xuất hiện của Internet chỉ làm tăng thêm ngọn lửa này. Ngày nay, mỗi người có kết nối Internet đều có quyền truy cập vào một loạt cơ sở dữ liệu chứa thông tin về mọi khía cạnh của cuộc sống của chúng ta. Dưới đây là một vài ví dụ:

- Nếu bạn truy cập vào trang web http://www.landmark-project.com và nhấp vào liên kết "Raw Data", bạn sẽ tìm thấy các tệp dữ liệu về động đất, ô nhiễm không khí, bóng chày, lao động, tội phạm, thị trường tài chính, lịch sử Hoa Kỳ, địa lý, thời tiết, công viên quốc gia, "khảo sát giá trị thế giới", và nhiều hơn nữa.

- Tại http://www.gutenberg.org, bạn sẽ tìm thấy hàng ngàn cuốn sách trực tuyến, bao gồm toàn bộ tác phẩm của Shakespeare và các tác phẩm của Sir Arthur Conan Doyle, Jane Austen, H.G. Wells, James Joyce, Albert Einstein, Mark Twain, Lewis Carroll, T.S. Eliot, Edgar Allan Poe, và nhiều tác giả khác.
- Một lượng lớn dữ liệu về hệ gen có sẵn từ các trang web như http://www.ncbi.nlm.nih.gov/guide/. Các nhà sinh học đã quyết định rằng lượng lớn dữ liệu mô tả hệ gen của con người và hệ gen của các sinh vật khác nên được công khai để mọi người có thể nghiên cứu.
- Nhiều trang web phổ biến, chẳng hạn như Cơ sở Dữ liệu Phim Trực tuyến (Internet Movie Database), cung cấp dữ liệu của họ để tải xuống dưới dạng các tệp dữ liệu đơn giản (xem http://www.imdb.com/interfaces).
- Chính phủ Hoa Kỳ sản xuất rất nhiều dữ liệu thống kê. Trang web https://www.usa.gov/statistics cung cấp một danh sách dài các tải xuống có sẵn, bao gồm bản đồ và thống kê về việc làm, khí hậu, sản xuất, nhân khẩu học, sức khỏe, tội phạm, và nhiều hơn nữa.

Tệp tin và Đối tượng Tệp tin

Khi bạn lưu trữ dữ liệu trên máy tính của mình, bạn lưu trữ nó trong một tệp tin.

Tệp tin: Một tập hợp thông tin được lưu trữ trên máy tính và được gán một tên cụ thể.

Như chúng ta đã lưu ý, mỗi tệp tin đều có một tên. Ví dụ, nếu bạn tải văn bản của *Hamlet* từ trang Gutenberg, bạn có thể lưu nó trên máy tính của mình trong một tệp tin có tên là hamlet.txt. Tên tệp tin thường kết thúc bằng một hậu tố đặc biệt cho biết loại dữ

liệu mà nó chứa hoặc định dạng mà nó đã được lưu trữ. Hậu tố này được gọi là phần mở rộng tệp tin. Bảng 6.1 liệt kê một số phần mở rộng tệp tin thông dụng.

Bảng 6.1: Phần Mở Rộng Tệp Tin Thông Dụng

Phần mở rộng	Mô tả
.txt	tệp văn bản
.java	tệp mã nguồn Java
.class	tệp mã bytecode đã biên dịch
.doc	tệp Microsoft Word
.xls	tệp Microsoft Excel
.pdf	tệp Adobe Portable Document
.mp3	tệp âm thanh
.jpg	tệp hình ảnh
.zip	tệp nén
.html	tệp ngôn ngữ đánh dấu siêu văn bản (thường là trang web)
.exe	tệp thực thi

BẠN CÓ BIẾT?

Nguồn Gốc Của Xử Lý Dữ Liệu

Lĩnh vực xử lý dữ liệu đã tồn tại trước máy tính hơn nửa thế kỷ. Người ta thường nói rằng nhu cầu là mẹ của phát minh, và sự ra đời của xử lý dữ liệu là một ví dụ tốt cho nguyên tắc này. Khủng hoảng đã dẫn đến sự ra đời của ngành công nghiệp này xuất phát từ yêu cầu trong Điều 1, Mục 2 của Hiến pháp Hoa Kỳ, quy định rằng dân số của mỗi bang sẽ quyết định số lượng đại diện mà bang đó có trong Hạ viện. Để tính toán con số chính xác, bạn cần biết dân số, vì vậy Hiến pháp quy định: “Việc đếm chính xác sẽ được thực hiện trong vòng ba năm sau cuộc họp đầu tiên của Quốc hội Hoa Kỳ, và trong mỗi khoảng thời gian mười năm tiếp theo, theo cách thức mà họ sẽ chỉ định bằng luật.”

Cuộc điều tra dân số đầu tiên đã được hoàn thành tương đối nhanh chóng vào năm 1790. Kể từ đó, cứ mỗi 10 năm, chính phủ Hoa Kỳ lại phải thực hiện một cuộc điều tra dân số đầy đủ khác. Quá trình này trở nên ngày càng khó khăn khi dân số của quốc gia tăng lên. Đến năm 1880, chính phủ nhận thấy rằng việc sử dụng các kỹ thuật đếm tay cũ đã khiến họ hầu như không thể hoàn thành cuộc điều tra dân số trong vòng 10 năm được phép. Vì vậy, chính phủ đã công bố một cuộc thi cho các nhà phát minh để đề xuất các máy móc có thể được sử dụng để đẩy nhanh quá trình này.

Herman Hollerith đã giành chiến thắng trong cuộc thi với hệ thống sử dụng thẻ đục lỗ. Các nhân viên đã đục lỗ hơn 62 triệu thẻ, sau đó được đếm bởi 100 máy đếm. Hệ thống này đã cho phép việc thống kê năm 1890 hoàn thành trong chưa đầy một nửa thời gian so với việc đếm tay các kết quả năm 1880, mặc dù dân số đã tăng 25%.

Hollerith đã phải vật lộn nhiều năm để biến phát minh của mình thành một thành công thương mại. Vấn đề lớn nhất ban đầu của ông là ông chỉ có một khách hàng: chính phủ Hoa Kỳ. Cuối cùng ông đã tìm được các khách hàng khác, và công ty mà ông thành lập đã hợp nhất với các đối thủ và phát triển thành công ty mà chúng ta biết ngày nay là International Business Machines Corporation, hay IBM.

Chúng ta nghĩ về IBM như một công ty máy tính, nhưng họ đã bán nhiều loại thiết bị xử lý dữ liệu liên quan đến thẻ Hollerith từ lâu trước khi máy tính trở nên phổ biến. Sau này, khi IBM bước chân vào lĩnh vực máy tính, họ đã sử dụng thẻ Hollerith để lưu trữ chương trình và dữ liệu. Những thẻ này vẫn được sử dụng khi một trong những tác giả của cuốn sách này tham gia lớp lập trình máy tính năm 1978.

Tệp tin có thể được phân loại thành tệp văn bản và tệp nhị phân tùy thuộc vào định dạng được sử dụng. Tệp văn bản có thể được chỉnh sửa bằng các trình soạn thảo văn bản đơn giản. Tệp nhị phân được lưu trữ bằng định dạng nội bộ yêu cầu phần mềm đặc biệt để xử lý. Tệp văn bản thường được lưu trữ với phần mở rộng .txt, nhưng các định dạng tệp khác cũng là định dạng văn bản, bao gồm các tệp .java và .html.

Để truy cập một tệp tin từ bên trong chương trình Java, bạn cần tạo một đối tượng nội bộ để đại diện cho tệp tin đó. Thư viện lớp của Java bao gồm một lớp gọi là File thực hiện nhiệm vụ này. Bạn tạo một đối tượng File bằng cách truyền vào tên của một tệp tin, như trong dòng mã sau:

java

Copy code

File f = new File("hamlet.txt");

Khi bạn đã tạo đối tượng này, bạn có thể gọi một số phương thức để thao tác với tệp tin. Ví dụ, đoạn tương tác JShell sau đây gọi các phương thức để xác định liệu tệp có tồn tại không, liệu nó có thể đọc được không, liệu nó có phải là thư mục không, độ dài của nó (tức là có bao nhiêu ký tự trong tệp), và đường dẫn tuyệt đối của nó (tức là nơi nó được lưu trữ trên máy tính). Giả sử bạn đã lưu tệp hamlet.txt trong cùng thư mục với chương trình, bạn sẽ nhận được đầu ra như sau:

jshell

Copy code

File f = new File("hamlet.txt");

f ==> hamlet.txt

f.exists()

$2 ==> true

f.canRead()

$3 ==> true

f.isDirectory()

$4 ==> false

f.length()

$5 ==> 196197

f.getAbsolutePath()

$6 ==> "/home/bjp/data/hamlet.txt"

Khi bạn muốn sử dụng các đối tượng File trong một chương trình Java thông thường, bạn sẽ cần nhập lớp từ gói java.io. Thuật ngữ “I/O” là thuật ngữ chuyên ngành được các chuyên gia khoa học máy tính sử dụng để chỉ “đầu vào/đầu ra”. Khai báo sau đây sẽ thực hiện công việc này:

java

Copy code

import java.io.*;

Việc chúng ta sử dụng lệnh new để tạo đối tượng File có thể gây hiểu lầm. Chúng ta không tạo ra một tệp tin mới thực sự trên ổ cứng bằng cách tạo đối tượng này. Đối tượng File là một đối tượng nội bộ cho phép chúng ta truy cập các tệp tin đã tồn tại trên máy tính. Sau này trong chương này, chúng ta sẽ thấy cách viết một chương trình tạo tệp tin như một đầu ra.

Bảng 6.2: Các Phương Thức Hữu Ích của Đối Tượng File

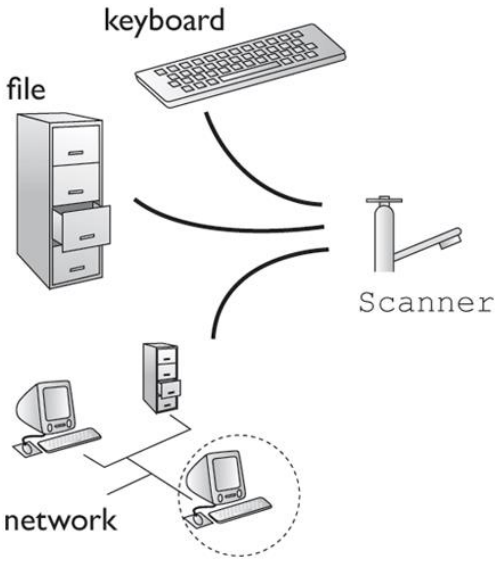
Phương thức	Mô tả
canRead()	Liệu tệp tin này có tồn tại và có thể đọc được không
delete()	Xóa tệp tin đã cho
exists()	Liệu tệp tin này có tồn tại trên hệ thống không
getAbsolutePath()	Đường dẫn đầy đủ nơi tệp tin này nằm
getName()	Tên của tệp tin này dưới dạng Chuỗi, không có đường dẫn
isDirectory()	Liệu tệp tin này có đại diện cho một thư mục trên hệ thống không
isFile()	Liệu tệp tin này có đại diện cho một tệp (không phải thư mục) trên hệ thống không
length()	Số ký tự trong tệp tin này
renameTo(file)	Thay đổi tên của tệp tin này thành tên của tệp đã cho

Đọc một Tệp tin với Scanner

Lớp Scanner mà chúng ta đã sử dụng từ Chương 3 rất linh hoạt ở chỗ các đối tượng Scanner có thể được gắn vào nhiều loại đầu vào khác nhau (xem Hình 6.1). Bạn có thể nghĩ về đối tượng Scanner như một vòi nước mà bạn có thể gắn vào một ống có nước chảy qua. Nước có thể đến từ nhiều nguồn khác nhau. Ví

dụ, trong một ngôi nhà, bạn sẽ gắn một vòi nước vào một ống dẫn nước từ nguồn cung cấp nước của thành phố hoặc từ một cái giếng, nhưng các vòi nước trong những nơi như nhà di động và máy bay có các nguồn nước khác nhau.

Hình 6.1: Scanners có thể được kết nối với nhiều nguồn đầu vào khác nhau.



Cho đến nay, chúng ta đã tạo các đối tượng Scanner bằng cách truyền System.in cho hàm dựng của Scanner:

java

Copy code

```
Scanner console = new Scanner(System.in);
```

Dòng mã này chỉ dẫn cho máy tính tạo ra một Scanner đọc từ console (tạm dừng để nhận đầu vào từ người dùng). Thay vì truyền System.in cho hàm dựng, bạn có thể truyền một đối tượng File:

java

Copy code

```
File f = new File("hamlet.txt");

Scanner input = new Scanner(f);
```

Trong trường hợp này, biến f không cần thiết, vì vậy chúng ta có thể rút gọn mã này như sau:

java

Copy code

```
Scanner input = new Scanner(new File("hamlet.txt"));
```

Dòng mã này, hoặc tương tự, sẽ xuất hiện trong tất cả các chương trình xử lý tệp của bạn. Khi chúng ta đọc từ cửa sổ console, chúng ta gọi biến Scanner là console. Khi bạn đọc từ một tệp tin đầu vào, bạn có thể muốn gọi biến này là input. Tất nhiên, bạn có thể đặt tên biến bất kỳ, miễn là bạn tham chiếu nó một cách nhất quán.

Tuy nhiên, khi bạn cố gắng biên dịch một chương trình tạo đối tượng Scanner theo cách này, bạn sẽ gặp phải một lỗi. Giả sử bạn viết phương thức main bắt đầu bằng cách mở một Scanner như sau:

java

Copy code

```
// phương thức lỗi -- không thể biên dịch
```

```
public static void main(String[] args) {

    Scanner input = new Scanner(new File("hamlet.txt"));

    ...

}
```

Chương trình này không thể biên dịch. Nó tạo ra một thông báo như sau:

java

Copy code

```
CountWords.java:8: unreported exception
java.io.FileNotFoundException;

must be caught or declared to be thrown

Scanner input = new Scanner(new File("hamlet.txt"));

                    ^

1 error
```

Vấn đề liên quan đến ngoại lệ, được mô tả trong Chương 3. Nhớ rằng ngoại lệ là lỗi ngăn chương trình tiếp tục thực thi bình thường. Trong trường hợp này, trình biên dịch lo lắng rằng có thể không tìm thấy tệp tin hamlet.txt. Điều gì sẽ xảy ra nếu điều đó xảy ra? Chương trình sẽ không có tệp để đọc, vì vậy không có cách nào tiếp tục thực thi phần còn lại của mã.

Nếu chương trình không thể định vị được tệp tin đầu vào đã chỉ định, nó sẽ tạo ra lỗi bằng cách ném ngoại lệ được gọi là FileNotFoundException. Ngoại lệ này được gọi là ngoại lệ kiểm tra.

Ngoại lệ kiểm tra: Một ngoại lệ phải được xử lý hoặc cụ thể tuyên bố trong phần đầu của phương thức có khả năng tạo ra ngoại lệ.

Vì FileNotFoundException là một ngoại lệ kiểm tra, bạn không thể chỉ bỏ qua nó. Java cung cấp một cấu trúc gọi là câu lệnh try/catch để xử lý các lỗi như vậy (được mô tả trong Phụ lục C), nhưng nó cho phép bạn tránh xử lý lỗi này miễn là bạn chỉ rõ rằng bạn không xử lý nó. Tất cả những gì bạn cần làm là bao gồm một mệnh đề throws trong phần đầu của phương thức main để chỉ rõ rằng phương thức main có thể tạo ra ngoại lệ này.

Mệnh đề throws: Một tuyên bố rằng phương thức sẽ không cố gắng xử lý một loại ngoại lệ cụ thể.

Dưới đây là cách sửa đổi phần đầu của phương thức main để bao gồm mệnh đề throws chỉ rõ rằng nó có thể ném FileNotFoundException:

java

Copy code

```
public static void main(String[] args) throws
FileNotFoundException {

    Scanner input = new Scanner(new File("hamlet.txt"));

    ...
```

}

Với mệnh đề throws, dòng mã trở nên quá dài khiến chúng ta phải chia nó thành hai dòng để phù hợp với lề sách. Trên máy tính của bạn, có lẽ bạn sẽ bao gồm tất cả trên một dòng.

Sau khi sửa đổi này, chương trình biên dịch thành công. Khi bạn đã tạo đối tượng Scanner để nó đọc từ tệp tin, bạn có thể thao tác nó giống như bất kỳ Scanner nào khác. Tất nhiên, bạn luôn nên nhắc người dùng trước khi đọc từ console để cho biết loại dữ liệu bạn muốn, nhưng khi đọc từ tệp tin, bạn không cần nhắc vì dữ liệu đã có sẵn trong tệp. Ví dụ, bạn có thể viết một chương trình như sau để đếm số lượng từ trong *Hamlet*:

java

Copy code

```
// Đếm số lượng từ trong Hamlet.

import java.io.*;

import java.util.*;



public class CountWords {

    public static void main(String[] args) throws
FileNotFoundException {

        Scanner input = new Scanner(new File("hamlet.txt"));

        int count = 0;
```

```
        while (input.hasNext()) {
            String word = input.next();
            count++;
        }
        System.out.println("total words = " + count);
    }
}
```

Lưu ý rằng bạn phải bao gồm lệnh import từ java.util cho lớp Scanner và lệnh import từ java.io cho lớp File. Chương trình tạo ra đầu ra sau:

text

Copy code

total words = 31956

LỖI LẬP TRÌNH THƯỜNG GẶP

Đọc Vượt Quá Cuối Tệp

Khi bạn học cách xử lý tệp tin đầu vào, bạn có thể sẽ viết các chương trình vô tình cố gắng đọc dữ liệu khi không còn dữ liệu nào để đọc. Ví dụ, chương trình CountWords mà chúng ta vừa thấy sử dụng một vòng lặp while để tiếp tục đọc các từ từ tệp tin

miễn là vẫn còn từ để đọc. Sau vòng lặp while, bạn có thể thêm một câu lệnh để đọc một từ nữa:

java

Copy code

```
while (input.hasNext()) {
    String word = input.next();
    count++;
}
String extra = input.next(); // không hợp lệ, không còn đầu vào
```

Dòng mã mới này khiến máy tính cố gắng đọc một từ khi không còn từ nào để đọc. Java sẽ ném ra một ngoại lệ khi điều này xảy ra:

text

Copy code

```
Exception in thread "main" java.util.NoSuchElementException
    at java.util.Scanner.throwFor(Scanner.java:817)
    at java.util.Scanner.next(Scanner.java:1317)
    at CountWords.main(CountWords.java:15)
```

Như thường lệ, thông tin quan trọng nhất xuất hiện ở cuối danh sách các số dòng này. Ngoại lệ chỉ ra rằng lỗi xảy ra tại dòng 15 của CountWords. Các số dòng khác đến từ lớp Scanner và không hữu ích.

Nếu bạn thấy mình nhận được NoSuchElementException, có thể là do bạn đã cố gắng đọc vượt quá cuối đầu vào. Scanner đang nói rằng: "Bạn đã yêu cầu tôi cung cấp một số dữ liệu, nhưng tôi không có giá trị nào như vậy để đưa cho bạn."

LỖI LẬP TRÌNH THƯỜNG GẶP
Quên "new File"

Giả sử bạn dự định tạo một đối tượng Scanner theo cách mà chúng ta đã học:

java

Copy code

```
Scanner input = new Scanner(new File("hamlet.txt"));
```

Nhưng bạn vô tình quên bao gồm đối tượng File và thay vào đó viết dòng mã này:

java

Copy code

```
Scanner input = new Scanner("hamlet.txt"); // không đúng
```

Dòng mã này có vẻ đúng vì nó đề cập đến tên của tệp tin, nhưng nó sẽ không hoạt động vì nó không bao gồm đối tượng File.

Thông thường, khi bạn mắc lỗi như thế này, Java sẽ cảnh báo bạn rằng bạn đã làm điều gì đó không hợp lệ. Tuy nhiên, trong trường hợp này, bạn sẽ không nhận được cảnh báo nào từ Java. Điều này là do, như bạn sẽ thấy sau trong chương này, có thể tạo một đối tượng Scanner từ một chuỗi (String), trong trường hợp đó, Java sẽ đọc từ chính chuỗi đó.

Nếu bạn mắc lỗi này trong chương trình CountWords, bạn sẽ nhận được đầu ra sau:

text

Copy code

total words = 1

Chương trình sẽ báo cáo chỉ một từ vì chuỗi "hamlet.txt" trông giống như một từ duy nhất đối với Scanner. Vì vậy, bất cứ khi nào bạn tạo một Scanner mà được cho là sẽ đọc từ một tệp tin đầu vào, hãy đảm bảo rằng bạn đã bao gồm lệnh gọi new File để tạo một đối tượng File thích hợp.

6.2 Chi Tiết Về Xử Lý Dựa Trên Từ Khóa

Bây giờ chúng ta đã giới thiệu một số vấn đề cơ bản liên quan đến việc đọc một tệp tin đầu vào, hãy cùng tìm hiểu chi tiết hơn về cách đọc từ một tệp. Một cách để xử lý tệp tin là xử lý từng từ khóa.

Xử Lý Dựa Trên Từ Khóa: Xử lý đầu vào theo từng từ khóa (tức là từng từ một hoặc từng số một).

Nhớ lại từ Chương 3 các phương thức chính để đọc từ khóa cho lớp Scanner:

- nextInt để đọc giá trị int
- nextDouble để đọc giá trị double
- next để đọc từ khóa tiếp theo dưới dạng String

Ví dụ, bạn có thể muốn tạo một tệp tin gọi là numbers.dat với nội dung sau:

text

Copy code

308.2 14.9 7.4

2.8

3.9 4.7 -15.4

2.8

Bạn có thể tạo tệp này bằng một trình soạn thảo như Notepad trên máy Windows hoặc TextEdit trên máy Macintosh. Sau đó, bạn có thể viết một chương trình để xử lý tệp tin đầu vào này và tạo ra một báo cáo nào đó. Ví dụ, chương trình sau đây đọc năm số đầu tiên từ tệp và báo cáo tổng của chúng:

java

Copy code

// Chương trình đọc năm số và báo cáo tổng của chúng.

import java.io.*;

```
import java.util.*;



public class ShowSum1 {

    public static void main(String[] args) throws
FileNotFoundException {

        Scanner input = new Scanner(new File("numbers.dat"));

        double sum = 0.0;

        for (int i = 1; i <= 5; i++) {

            double next = input.nextDouble();

            System.out.println("number " + i + " = " + next);

            sum += next;

        }

        System.out.println("Sum = " + sum);

    }

}
```

Chương trình này sử dụng một biến thể của mã tổng cộng từ Chương 4. Nhớ rằng bạn cần một mệnh đề throws trong phần đầu của main vì có khả năng xảy ra FileNotFoundException. Chương trình tạo ra đầu ra sau:

text

Copy code

number 1 = 308.2

number 2 = 14.9

number 3 = 7.4

number 4 = 2.8

number 5 = 3.9

Sum = 337.19999999999993

Lưu ý rằng tổng báo cáo không phải là 337.2. Kết quả này là một ví dụ khác về lỗi làm tròn (cũng được mô tả trong Chương 4).

Chương trình trước đọc chính xác năm số từ tệp. Thông thường, bạn sẽ tiếp tục đọc các số miễn là còn số để đọc trong khi một vòng lặp while đang thực thi. Nhớ rằng lớp Scanner bao gồm một loạt các phương thức hasNext song song với các phương thức next. Trong trường hợp này, nextDouble đang được sử dụng để đọc một giá trị kiểu double, vì vậy bạn có thể sử dụng hasNextDouble để kiểm tra xem có giá trị như vậy để đọc không:

java

Copy code

// Đọc một tệp tin đầu vào các số và in các số đó cùng với tổng của chúng.

```
import java.io.*;

import java.util.*;



public class ShowSum2 {

    public static void main(String[] args) throws
FileNotFoundException {

        Scanner input = new Scanner(new File("numbers.dat"));

        double sum = 0.0;

        int count = 0;

        while (input.hasNextDouble()) {

            double next = input.nextDouble();

            count++;

            System.out.println("number " + count + " = " + next);

            sum += next;

        }

        System.out.println("Sum = " + sum);

    }

}
```

Chương trình này sẽ hoạt động trên một tệp tin đầu vào có bất kỳ số lượng số nào; numbers.dat có tám số. Phiên bản này của chương trình tạo ra đầu ra sau:

text

Copy code

number 1 = 308.2

number 2 = 14.9

number 3 = 7.4

number 4 = 2.8

number 5 = 3.9

number 6 = 4.7

number 7 = -15.4

number 8 = 2.8

Sum = 329.29999999999995

Cấu Trúc Tệp Tin và Xử Lý Đầu Vào

Chúng ta thường nghĩ văn bản có dạng hai chiều, giống như một tờ giấy, nhưng từ quan điểm của máy tính, mỗi tệp tin chỉ là một chuỗi ký tự một chiều. Ví dụ, hãy xem xét tệp tin numbers.dat mà chúng ta đã sử dụng trong phần trước:

text

Copy code

308.2 14.9 7.4

2.8

3.9 4.7 -15.4

2.8

Chúng ta nghĩ rằng đây là một tệp tin có sáu dòng với văn bản chạy ngang và xuống, và có hai dòng trống ở giữa. Tuy nhiên, máy tính lại nhìn tệp tin theo cách khác. Khi bạn gõ văn bản trong tệp này, bạn đã nhấn phím Enter để chuyển sang dòng mới. Phím này chèn các ký tự "dòng mới" đặc biệt vào tệp tin. Bạn có thể chú thích tệp tin bằng các ký tự \n để chỉ ra kết thúc mỗi dòng:

text

Copy code

308.2 14.9 7.4\n

2.8\n

\n

\n

3.9 4.7 -15.4\n

2.8\n

LỖI LẬP TRÌNH THƯỜNG GẶP
Đọc Sai Loại Từ Khóa

Rất dễ viết mã vô tình đọc sai loại dữ liệu. Ví dụ, chương trình ShowSum1 luôn đọc chính xác năm giá trị double từ tệp tin đầu vào numbers.dat. Nhưng giả sử tệp tin đầu vào có một số văn bản không cần thiết:

text

Copy code

308.2 14.9 7.4

hello

2.8

3.9 4.7 -15.4

2.8

Dòng đầu tiên của tệp tin chứa ba số mà chương trình sẽ đọc đúng cách. Nhưng khi nó cố gắng đọc số thứ tư, máy tính sẽ phát hiện ra rằng từ khóa tiếp theo trong tệp tin là văn bản "hello". Từ

khóa này không thể được diễn giải như một giá trị double, vì vậy chương trình sẽ tạo ra một ngoại lệ:

text

Copy code

number 1 = 308.2

number 2 = 14.9

number 3 = 7.4

Exception in thread "main" java.util.InputMismatchException

    at java.util.Scanner.throwFor(Scanner.java:819)

    at java.util.Scanner.next(Scanner.java:1431)

    at java.util.Scanner.nextDouble(Scanner.java:2335)

    at ShowSum1.main(ShowSum1.java:13)

Một lần nữa, số dòng hữu ích xuất hiện ở cuối danh sách này. Dòng cuối cùng chỉ ra rằng ngoại lệ xảy ra tại dòng 13 của lớp ShowSum1. Các số dòng khác là từ lớp Scanner và không hữu ích.

Như bạn đã thấy trước đó, khi bạn cố gắng đọc vượt quá cuối tệp tin, Scanner sẽ ném ra một ngoại lệ NoSuchElementException. Trong trường hợp chương trình này cố gắng đọc sai loại từ khóa, Scanner sẽ ném ra một ngoại lệ InputMismatchException. Bằng cách chú ý đến loại ngoại lệ mà Scanner ném ra, bạn có thể có phản hồi tốt hơn về vấn đề là gì.

Khi bạn đánh dấu kết thúc mỗi dòng trong chương trình của mình, bạn không cần phải sử dụng biểu diễn hai chiều nữa. Bạn có thể thu gọn văn bản này thành một chuỗi ký tự một chiều:

text

Copy code

308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n

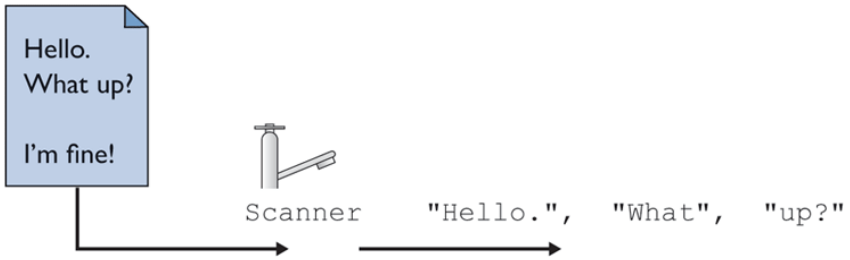
Chuỗi này là cách máy tính nhìn tệp tin: như một chuỗi ký tự một chiều bao gồm các ký tự đặc biệt đại diện cho "dòng mới". Trên một số hệ thống, bao gồm cả máy Windows, có hai ký tự "dòng mới" khác nhau, nhưng ở đây chúng ta chỉ sử dụng \n—các đối tượng như Scanner xử lý các khác biệt này cho bạn, vì vậy bạn thường có thể bỏ qua chúng. (Đối với những ai quan tâm, lời giải thích ngắn gọn là máy Windows kết thúc mỗi dòng bằng một \r theo sau bởi một \n.)

Khi xử lý một tệp tin, đối tượng Scanner theo dõi vị trí hiện tại trong tệp tin. Bạn có thể coi đây như một con trỏ đầu vào hoặc con trỏ vào tệp tin.

Con Trỏ Đầu Vào

Một con trỏ chỉ vị trí hiện tại trong một tệp tin đầu vào.

Hình 6.2: Các đối tượng Scanner xử lý tệp tin như các chuỗi ký tự một chiều và chuyển đổi nội dung của chúng thành một chuỗi các từ khóa được phân cách bởi khoảng trắng.



Hình 6.2 - Văn Bản Thay Thế Đầy Đủ

Khi một đối tượng Scanner được khởi tạo lần đầu tiên, con trỏ sẽ chỉ vào đầu tệp tin. Nhưng khi bạn thực hiện các thao tác next, con trỏ sẽ di chuyển về phía trước. Chương trình ShowSum2 từ phần trước xử lý tệp tin thông qua một loạt các lệnh gọi nextDouble. Hãy dành chút thời gian để xem xét chi tiết cách hoạt động của điều này. Một lần nữa, khi Scanner được khởi tạo lần đầu tiên, con trỏ đầu vào sẽ được đặt tại đầu tệp tin (được chỉ định bằng một mũi tên chỉ lên chỉ vào ký tự đầu tiên):

text

Copy code

↑308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n

Như con trỏ di chuyển theo tiến trình của chương trình, nó sẽ tiếp tục đọc các giá trị từ tệp và di chuyển đến vị trí kế tiếp sau mỗi thao tác nextDouble.

```
308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n
↑
input
cursor
```

Sau lần gọi đầu tiên của nextDouble, con trỏ sẽ được đặt ở giữa dòng đầu tiên, sau token “308.2”:

```
308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n
      ↑
      input
      cursor
```

6.2-4 Văn bản Thay thế Đầy đủ

Chúng ta gọi quá trình này là tiêu thụ đầu vào.

Tiêu thụ Đầu vào

Di chuyển con trỏ đầu vào về phía trước qua một số đầu vào. Quá trình tiêu thụ đầu vào không thực sự thay đổi tệp, nó chỉ thay đổi đối tượng Scanner tương ứng để nó được đặt ở một điểm khác trong tệp. Lần gọi đầu tiên của nextDouble tiêu thụ văn bản “308.2” từ tệp đầu vào và để lại con trỏ đầu vào ở ký tự đầu tiên sau token này. Lưu ý rằng điều này làm cho con trỏ đầu vào nằm ở khoảng trắng. Khi lần gọi thứ hai được thực hiện với nextDouble, Scanner sẽ bỏ qua khoảng trắng này để đến được token tiếp theo, sau đó tiêu thụ văn bản “14.9” và để lại con trỏ ở khoảng trắng tiếp theo:

```
308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n
          ↑
          input
          cursor
```

6.2-5 Văn bản Thay thế Đầy đủ

Lần gọi thứ ba của nextDouble bỏ qua khoảng trắng đó và tiêu thụ văn bản “7.4”:

```
308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n
               ↑
               input
               cursor
```

6.2-6 Văn bản Thay thế Đầy đủ

Tại thời điểm này, con trỏ đầu vào được đặt ở ký tự xuống dòng ở cuối dòng đầu tiên của đầu vào. Lần gọi thứ tư của nextDouble bỏ qua ký tự xuống dòng này và tiêu thụ văn bản “2.8”:

```
308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n
                    ↑
                    input
                    cursor
```

6.2-7 Văn bản Thay thế Đầy đủ

Lưu ý rằng khi nó bỏ qua ký tự xuống dòng đầu tiên, con trỏ đầu vào đã di chuyển vào dữ liệu được lưu trữ trên dòng thứ hai của đầu vào. Tại thời điểm này, con trỏ đầu vào được đặt ở cuối dòng thứ hai của đầu vào, vì nó đã tiêu thụ token “2.8”. Khi lần gọi thứ năm được thực hiện với nextDouble, Scanner tìm thấy ba ký tự xuống dòng liên tiếp. Điều này không phải là vấn đề đối với

Scanner, vì nó chỉ đơn giản bỏ qua bất kỳ ký tự khoảng trắng nào dẫn đầu (khoảng trắng, tab, ký tự xuống dòng) cho đến khi nó tìm thấy một token thực sự. Do đó, nó bỏ qua cả ba ký tự xuống dòng này và tiêu thụ văn bản “3.9”:

```
308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n
                           ↑
                         input
                         cursor
```

6.2-8 Văn bản Thay thế Đầy đủ

Tại thời điểm này, con trỏ đầu vào được đặt ở giữa dòng thứ năm của đầu vào. (Dòng thứ ba và thứ tư để trống). Chương trình tiếp tục đọc theo cách này, tiêu thụ ba số còn lại trong tệp đầu vào. Sau khi nó đọc token cuối cùng, “2.8”, con trỏ đầu vào được đặt ở ký tự xuống dòng ở cuối tệp:

```
308.2 14.9 7.4\n2.8\n\n\n3.9 4.7 -15.4\n2.8\n
                                      ↑
                                    input
                                    cursor
```

6.2-9 Văn bản Thay thế Đầy đủ

Nếu bạn cố gắng gọi nextDouble lần nữa, nó sẽ ném ra ngoại lệ NoSuchElementException vì không còn token nào để xử lý. Nhưng hãy nhớ rằng chương trình ShowSum2 có một vòng lặp while gọi hasNextDouble trước khi gọi nextDouble. Khi bạn gọi các phương thức như hasNextDouble, Scanner sẽ nhìn trước trong tệp để xem có token tiếp theo hay không và liệu nó có thuộc loại được chỉ định (trong trường hợp này là double) hay không. Vì vậy, chương trình ShowSum2 sẽ tiếp tục thực thi cho đến khi nó đến cuối tệp hoặc cho đến khi nó gặp phải một token không thể được diễn giải là double.

Khi con trỏ đầu vào đến ký tự xuống dòng ở cuối tệp, Scanner nhận ra rằng không còn giá trị double nào để đọc và trả về false khi hasNextDouble được gọi. Việc trả về này sẽ dừng vòng lặp while và làm cho chương trình thoát. Các đối tượng Scanner được thiết kế để xử lý tệp theo cách tiến lên phía trước. Chúng cung cấp một mức độ linh hoạt lớn để nhìn trước trong một tệp đầu vào, nhưng không hỗ trợ đọc đầu vào ngược lại. Không có

phương thức "trước" và không có cơ chế nào để đặt lại Scanner về đầu đầu vào. Thay vào đó, bạn sẽ phải xây dựng một đối tượng Scanner mới sẽ được đặt ở đầu tệp. Chúng ta sẽ thấy một ví dụ về kỹ thuật này trong nghiên cứu tình huống ở cuối chương.

Thông số của Scanner

Người mới bắt đầu đôi khi ngạc nhiên khi con trỏ đầu vào cho Scanner không được đặt lại từ đầu tệp khi nó được truyền như một tham số vào một phương thức. Ví dụ, hãy xem xét biến thể sau của chương trình ShowSum1. Nó có một phương thức nhận Scanner làm đầu vào và một số nguyên chỉ định số lượng số cần xử lý:

```
1 // Minh họa Scanner như một tham số cho một phương thức có thể tiêu thụ số lượng token tùy ý.
2
3 import java.io.*;
4 import java.util.*;
5
6 public class ShowSum3 {
7
8     public static void main(String[] args) throws FileNotFoundException {
9         Scanner input = new Scanner(new File("numbers.dat"));
10        processTokens(input, 2);
11        processTokens(input, 3);
12        processTokens(input, 2);
13    }
14
15    public static void processTokens(Scanner input, int n) {
16        double sum = 0.0;
17        for (int i = 1; i <= n; i++) {
```

```
18            double next = input.nextDouble();
19            System.out.println("number " + i + " = " + next);
20            sum += next;
21        }
22        System.out.println("Sum = " + sum);
23        System.out.println();
24    }
25 }
```

Phương thức main tạo một đối tượng Scanner liên kết với tệp numbers.dat. Sau đó, nó gọi phương thức processTokens nhiều lần, chỉ định số lượng token cần xử lý. Lần gọi đầu tiên yêu cầu phương thức xử lý hai token. Nó xử lý hai token đầu tiên của tệp, tạo ra kết quả sau:

typescript
Copy code

```
number 1 = 308.2
number 2 = 14.9
Sum = 323.09999999999997
```

Lần gọi thứ hai của phương thức chỉ ra rằng ba token sẽ được xử lý. Một số người mong đợi phương thức xử lý ba token đầu tiên của tệp, nhưng điều đó không xảy ra. Hãy nhớ rằng Scanner theo dõi vị trí của con trỏ đầu vào. Sau lần gọi đầu tiên của phương thức, con trỏ đầu vào được đặt sau hai token đầu tiên. Vì vậy, lần gọi thứ hai của phương thức xử lý ba token tiếp theo từ tệp, tạo ra kết quả sau:

typescript
Copy code

```
number 1 = 7.4
number 2 = 2.8
number 3 = 3.9
Sum = 14.1
```

Lần gọi cuối cùng của phương thức yêu cầu Scanner xử lý hai token tiếp theo từ tệp, vì vậy nó xử lý số thứ sáu và thứ bảy từ tệp:
typescript
Copy code
number 1 = 4.7
number 2 = −15.4
Sum = −10.7
Sau đó, chương trình kết thúc mà không xử lý số thứ tám trong tệp. Điểm quan trọng cần nhớ là Scanner theo dõi vị trí của con trỏ đầu vào, vì vậy bạn có thể xử lý tệp đầu vào từng phần một cách rất linh hoạt. Ngay cả khi Scanner được truyền như một tham số cho một phương thức, nó vẫn ghi nhớ phần nào của tệp đầu vào đã được xử lý cho đến nay.

Đường dẫn và Thư mục
Các tệp tin được nhóm vào các thư mục, còn gọi là thư mục con. Thư mục được tổ chức theo một hệ thống phân cấp, bắt đầu từ thư mục gốc ở trên cùng. Ví dụ, hầu hết các máy tính Windows có một ổ đĩa được biết đến với tên gọi C:. Ở cấp cao nhất của ổ đĩa này là thư mục gốc, mà chúng ta có thể mô tả là C:. Thư mục gốc này sẽ chứa các thư mục cấp cao khác nhau. Mỗi thư mục cấp cao có thể có các thư mục con, mỗi thư mục con này cũng có thể có các thư mục con của riêng nó, và cứ như thế. Tất cả các tệp tin được lưu trữ trong một trong những thư mục này. Mô tả cách đi từ thư mục cấp cao đến thư mục cụ thể chứa tệp tin được gọi là đường dẫn của tệp tin.
Đường dẫn tệp tin
Mô tả vị trí của một tệp tin trên máy tính, bắt đầu với ổ đĩa và bao gồm đường dẫn từ thư mục gốc đến thư mục nơi lưu trữ tệp tin. Chúng ta đọc thông tin đường dẫn từ trái sang phải. Ví dụ, nếu đường dẫn đến tệp tin là C:\school\data\hamlet.txt, chúng ta biết

rằng tệp tin nằm trên ổ đĩa C: trong một thư mục gọi là school và trong một thư mục con gọi là data.
Trong phần trước, chương trình đã sử dụng tên tệp tin numbers.dat. Khi Java gặp một tên đơn giản như vậy (còn gọi là đường dẫn tệp tin tương đối), nó sẽ tìm tệp tin trong thư mục hiện tại.

Thư mục hiện tại (còn gọi là Thư mục làm việc)

Thư mục mà Java sử dụng làm mặc định khi một chương trình sử dụng một tên tệp tin đơn giản.
Thư mục mặc định thay đổi tùy thuộc vào môi trường Java mà bạn đang sử dụng. Trong hầu hết các môi trường, thư mục hiện tại là thư mục mà chương trình của bạn xuất hiện. Chúng ta sẽ giả định rằng đây là trường hợp cho các ví dụ trong sách giáo khoa này.
Bạn cũng có thể sử dụng một tên tệp tin đầy đủ, hoặc hoàn chỉnh (đôi khi gọi là đường dẫn tệp tin tuyệt đối). Tuy nhiên, cách tiếp cận này chỉ hiệu quả khi bạn biết chính xác nơi tệp tin của bạn sẽ được lưu trữ trên hệ thống của bạn. Ví dụ, nếu bạn đang sử dụng máy Windows và bạn đã lưu trữ tệp tin trong thư mục C:\data, bạn có thể sử dụng tên tệp tin như sau:
java
Copy code
```
Scanner input = new Scanner(new File("C:/data/numbers.dat"));
```
Lưu ý rằng đường dẫn được viết với ký tự gạch chéo tiến (/) thay vì gạch chéo ngược (). Trên Windows, bạn thường sử dụng gạch chéo ngược, nhưng Java cho phép bạn sử dụng gạch chéo tiến. Nếu bạn muốn sử dụng gạch chéo ngược, bạn phải sử dụng chuỗi escape \\. Hầu hết lập trình viên chọn cách tiếp cận đơn giản hơn là sử dụng ký tự gạch chéo tiến vì Java sẽ thực hiện chuyển đổi thích hợp trên các máy Windows.
Bạn cũng có thể chỉ định một tệp tin bằng cách sử dụng đường dẫn tương đối. Để viết một đường dẫn tương đối, bạn bỏ qua việc chỉ định ổ đĩa ở đầu chuỗi. Bạn vẫn có thể chỉ định mối quan

hệ thư mục con mà sẽ tương đối so với thư mục hiện tại. Ví dụ, đường dẫn tương đối "data/numbers.dat" chỉ ra rằng có một tệp tin gọi là numbers.dat trong một thư mục con của thư mục làm việc gọi là data.
Đôi khi, thay vì viết tên đường dẫn của tệp tin trực tiếp trong mã, bạn sẽ yêu cầu người dùng nhập tên tệp tin. Ví dụ, đây là một biến thể của chương trình ShowSum2 yêu cầu người dùng nhập tên tệp tin:
java
Copy code

```
1  // Biến thể của ShowSum2 yêu cầu nhập tên tệp tin.
2
3  import java.io.*;
4  import java.util.*;
5
6  public class ShowSum4 {
7      public static void main(String[] args) throws
FileNotFoundException {
8          System.out.println("Chương trình này sẽ cộng một dãy số
từ tệp tin.");
9          System.out.println("Nhập tên tệp tin: ");
10
11         Scanner console = new Scanner(System.in);
12         String name = console.nextLine();
13         Scanner input = new Scanner(new File(name));
14         System.out.println();
15
16         double sum = 0.0;
17         int count = 0;
18
19         while (input.hasNextDouble()) {
20             double next = input.nextDouble();
21             count++;
```

```
22            System.out.println("number " + count + " = " + next);
23            sum += next;
24        }
25        System.out.println("Sum = " + sum);
26    }
27 }
```

Lưu ý rằng chương trình có hai đối tượng Scanner khác nhau: một đối tượng để đọc từ console và một đối tượng để đọc từ tệp tin. Chúng ta đọc tên tệp tin bằng cách gọi phương thức nextLine để đọc toàn bộ dòng nhập từ người dùng, cho phép người dùng nhập tên tệp tin có chứa khoảng trắng. Chú ý rằng chúng ta vẫn cần throws FileNotFoundException trong phần khai báo của main vì ngay cả khi chúng ta yêu cầu người dùng nhập tên tệp tin, không nhất thiết phải có tệp tin với tên đó.

Nếu chúng ta để chương trình này đọc từ tệp tin numbers.dat được sử dụng trước đó, nó sẽ tạo ra kết quả sau:

typescript

Copy code

```
Chương trình này sẽ cộng một dãy số từ tệp tin.
Nhập tên tệp tin:
numbers.dat

number 1 = 308.2
number 2 = 14.9
number 3 = 7.4
number 4 = 2.8
number 5 = 3.9
number 6 = 4.7
number 7 = -15.4
number 8 = 2.8
Sum = 329.29999999999995
```

Người dùng cũng có thể tùy chọn chỉ định một đường dẫn tệp tin đầy đủ:

typescript
Copy code
Chương trình này sẽ cộng một dãy số từ tệp tin.
Nhập tên tệp tin:
C:\data\numbers.dat

number 1 = 308.2
number 2 = 14.9
number 3 = 7.4
number 4 = 2.8
number 5 = 3.9
number 6 = 4.7
number 7 = -15.4
number 8 = 2.8
Sum = 329.29999999999995
Lưu ý rằng người dùng không cần phải nhập hai ký tự gạch chéo ngược để có được một gạch chéo ngược duy nhất. Đối tượng Scanner đọc đầu vào từ người dùng có thể đọc nó mà không cần chuỗi escape.

Một Tệp Nhập Phức Tạp Hơn
Giả sử một tệp nhập chứa thông tin về số giờ làm việc của mỗi nhân viên trong một công ty. Nó có thể trông như sau:
Copy code
Erica 7.5 8.5 10.25 8 8.5

Erin 10.5 11.5 12 11 10.75

Simone 8 8 8

Ryan 6.5 8 9.25 8

Kendall 2.5 3

Giả sử bạn muốn tìm tổng số giờ làm việc của mỗi cá nhân. Bạn có thể tạo một đối tượng Scanner liên kết với tệp này để giải quyết nhiệm vụ này. Khi bạn bắt đầu viết các chương trình xử lý tệp phức tạp hơn, bạn sẽ muốn chia chương trình thành các phương thức để phân chia mã thành các nhiệm vụ logic nhỏ hơn. Trong trường hợp này, bạn có thể mở tệp trong phương thức main và viết một phương thức riêng để xử lý tệp.
Hầu hết việc xử lý tệp sẽ liên quan đến vòng lặp while, vì bạn sẽ không biết trước tệp chứa bao nhiêu dữ liệu. Bạn sẽ cần viết các kiểm tra khác nhau, tùy thuộc vào tệp cụ thể đang được xử lý, nhưng chúng hầu như sẽ đều là các lời gọi đến các phương thức hasNext khác nhau của lớp Scanner. Bạn cơ bản muốn nói, "Trong khi bạn có thêm dữ liệu để tôi xử lý, hãy tiếp tục đọc." Trong trường hợp này, dữ liệu là một loạt các dòng đầu vào, mỗi dòng bắt đầu bằng một tên. Đối với chương trình này, bạn có thể giả định rằng các tên là đơn giản, không có khoảng trống ở giữa. Điều đó có nghĩa là bạn có thể đọc chúng bằng cách gọi phương thức next. Kết quả là, kiểm tra vòng lặp while liên quan đến việc kiểm tra xem có tên nào khác trong tệp nhập hay không:
java
Copy code
```
while (input.hasNext()) {
    // Xử lý người tiếp theo.
}
```
Vậy, làm thế nào để bạn xử lý mỗi người? Bạn phải đọc tên của người đó và sau đó đọc danh sách số giờ làm việc của người đó. Nếu bạn nhìn vào tệp nhập mẫu, bạn sẽ thấy rằng danh sách số giờ không phải lúc nào cũng có độ dài như nhau. Ví dụ, một số nhân viên làm việc trong năm ngày, trong khi những người khác chỉ làm việc hai hoặc ba ngày. Sự không đồng đều này là điều thường xảy ra trong các tệp nhập. Bạn có thể xử lý điều này bằng cách sử dụng một vòng lặp lồng nhau. Vòng lặp bên ngoài sẽ xử

lý từng người một và vòng lặp bên trong sẽ xử lý từng số một. Nhiệm vụ này khá đơn giản và là một tổng số tích lũy:
java
Copy code
```
double sum = 0.0;

while (input.hasNextDouble()) {
    sum += input.nextDouble();
}
```
Khi bạn kết hợp các phần của chương trình lại với nhau, bạn có chương trình hoàn chỉnh sau:
java
Copy code
```
 1 // Chương trình này đọc một tệp nhập về số giờ làm việc của các nhân viên
 2 // và báo cáo tổng số giờ làm việc của mỗi người.
 3
 4 import java.io.*;
 5 import java.util.*;
 6
 7 public class HoursWorked {
 8     public static void main(String[] args) throws FileNotFoundException {
 9         Scanner input = new Scanner(new File("hours.dat"));
10         process(input);
11     }
12
13     public static void process(Scanner input) {
14         while (input.hasNext()) {
15             String name = input.next();
16             double sum = 0.0;
17             while (input.hasNextDouble()) {
18                 sum += input.nextDouble();
```

```
19            }
20            System.out.println("Total hours worked by " + name
21                        + " = " + sum);
22        }
23    }

24 }
```

Lưu ý rằng bạn cần thêm throws FileNotFoundException trong phần khai báo của main. Bạn không cần bao gồm nó trong phương thức process vì mã mở tệp xuất hiện trong phương thức main.

Nếu bạn đặt dữ liệu đầu vào vào một tệp có tên là hours.dat và chạy chương trình, bạn sẽ nhận được kết quả sau:

csharp

Copy code

```
Total hours worked by Erica = 42.75
Total hours worked by Erin = 55.75
Total hours worked by Simone = 24.0
Total hours worked by Ryan = 31.75
Total hours worked by Kendall = 5.5
```

6.3 Xử lý Dựa trên Dòng

Cho đến nay, chúng ta đã xem xét các chương trình xử lý đầu vào từng token một. Tuy nhiên, bạn sẽ thường gặp các tệp đầu vào dựa trên dòng: Mỗi dòng đầu vào đại diện cho một trường hợp khác nhau, được xử lý riêng biệt với phần còn lại. Các loại tệp này phù hợp với một kiểu xử lý tệp thứ hai được gọi là xử lý dựa trên dòng.

Xử lý Dựa trên Dòng

Thực hành xử lý đầu vào theo từng dòng (tức là đọc toàn bộ dòng đầu vào một lúc). Hầu hết các quá trình xử lý tệp liên quan đến sự kết hợp giữa kiểu xử lý dựa trên dòng và dựa trên token, và lớp Scanner linh hoạt đủ để cho phép bạn viết các chương

trình bao gồm cả hai kiểu xử lý. Đối với xử lý dựa trên dòng, bạn sẽ sử dụng các phương thức nextLine và hasNextLine của đối tượng Scanner. Ví dụ, đây là một chương trình đọc tệp và in ra toàn bộ nội dung bằng chữ hoa:
java
Copy code
```
// Đọc một tệp và in nó ra bằng chữ hoa.
import java.io.*;
import java.util.*;

public class EchoUppercase {
    public static void main(String[] args) throws
FileNotFoundException {
        Scanner input = new Scanner(new File("poem.txt"));

        while (input.hasNextLine()) {
            String text = input.nextLine();
            System.out.println(text.toUpperCase());
        }
    }
}
```
Vòng lặp này đọc từng dòng của đầu vào và in mỗi dòng dưới dạng chữ hoa cho đến khi hết dòng để xử lý. Nó đọc từ một tệp có tên là poem.txt. Giả sử tệp có nội dung như sau:
perl
Copy code
```
My candle burns at both ends
It will not last the night;
But ah, my foes, and oh, my friends -
It gives a lovely light.

   --Edna St. Vincent Millay
```

Nếu bạn chạy chương trình trên đầu vào này, nó sẽ tạo ra đầu ra sau:
sql
Copy code
MY CANDLE BURNS AT BOTH ENDS
IT WILL NOT LAST THE NIGHT;
BUT AH, MY FOES, AND OH, MY FRIENDS -
IT GIVES A LOVELY LIGHT.
   --EDNA ST. VINCENT MILLAY
Lưu ý rằng bạn không thể hoàn thành cùng một nhiệm vụ với xử lý dựa trên token. Trong ví dụ này, dấu ngắt dòng rất quan trọng vì chúng là một phần của bài thơ. Ngoài ra, khi bạn đọc một tệp theo từng token, bạn sẽ mất khoảng trắng trong dòng vì Scanner bỏ qua bất kỳ khoảng trắng dẫn đầu nào khi nó đọc một token. Dòng cuối cùng của tệp này được thụt vào vì đó là tên của tác giả chứ không phải là một phần của bài thơ. Khoảng cách này sẽ bị mất nếu tệp được đọc dưới dạng một chuỗi các token.

Scanner Chuỗi và Kết hợp Dòng/Token

Trong phần trước, chúng ta đã xem xét một chương trình gọi là HoursWorked xử lý tệp dữ liệu sau:
Copy code
Erica 7.5 8.5 10.25 8 8.5
Erin 10.5 11.5 12 11 10.75
Simone 8 8 8
Ryan 6.5 8 9.25 8
Kendall 2.5 3
Dữ liệu là dựa trên dòng, thông tin của mỗi nhân viên xuất hiện trên một dòng đầu vào khác nhau, nhưng khía cạnh này của dữ liệu không được kết hợp vào chương trình. Chương trình đã xử lý tệp theo cách dựa trên token. Tuy nhiên, phương pháp đó sẽ không phải lúc nào cũng hoạt động. Ví dụ, hãy xem xét một thay đổi nhỏ trong dữ liệu, trong đó mỗi dòng đầu vào bắt đầu với một mã nhân viên thay vì chỉ tên:

Copy code
101 Erica 7.5 8.5 10.25 8 8.5
783 Erin 10.5 11.5 12 11 10.75
114 Simone 8 8 8
238 Ryan 6.5 8 9.25 8
156 Kendall 2.5 3
Sự bổ sung này có vẻ như là một thay đổi khá đơn giản không cần thay đổi lớn trong mã. Nhớ lại rằng chương trình sử dụng một phương thức để xử lý tệp:
java
Copy code
```
public static void process(Scanner input) {
    while (input.hasNext()) {
        String name = input.next();
        double sum = 0.0;

        while (input.hasNextDouble()) {
            sum += input.nextDouble();
        }

        System.out.println("Total hours worked by " + name + " = " +
sum);
    }
}
```
Giả sử bạn thêm một dòng mã để đọc mã nhân viên và sửa đổi câu lệnh println để báo cáo nó:
java
Copy code
```
public static void process(Scanner input) {
    while (input.hasNext()) {
        int id = input.nextInt();
        String name = input.next();
        double sum = 0.0;
```

```
        while (input.hasNextDouble()) {
            sum += input.nextDouble();
        }

        System.out.println("Total hours worked by " + name + " (id#"
+ id + ") = " + sum);
    }
}
```

Khi bạn chạy phiên bản mới của chương trình trên tệp đầu vào mới, bạn sẽ nhận được một dòng đầu ra và sau đó Java sẽ ném ra một ngoại lệ:

less

Copy code

```
Total hours worked by Erica (id#101) = 825.75
Exception in thread "main" java.util.InputMismatchException
        at java.util.Scanner.throwFor(Scanner.java:819)
        at java.util.Scanner.next(Scanner.java:1431)
        at java.util.Scanner.nextInt(Scanner.java:2040)
        ...
```

Chương trình đã đọc đúng mã nhân viên của Erica và báo cáo nó trong câu lệnh println, nhưng sau đó nó bị lỗi. Ngoài ra, lưu ý rằng chương trình báo cáo tổng số giờ làm việc của Erica là 825.75, trong khi số đúng là 42.75. Chương trình đã gặp sai sót ở đâu? Nếu bạn tính toán sự khác biệt giữa tổng số giờ báo cáo là 825.75 và tổng số đúng là 42.75 giờ, bạn sẽ thấy nó bằng 783. Con số đó xuất hiện trong tệp dữ liệu: Nó là mã nhân viên của nhân viên thứ hai, Erin. Khi chương trình đang cộng dồn giờ cho Erica, nó đã vô tình đọc mã nhân viên của Erin là nhiều giờ làm việc hơn của Erica và cộng số này vào tổng số. Đó cũng là lý do tại sao ngoại lệ xảy ra—trong vòng lặp thứ hai, chương trình cố gắng đọc mã nhân viên cho Erin khi token tiếp theo trong tệp là tên của cô ấy, không phải là một số nguyên.

Giải pháp là bằng cách nào đó để chương trình ngừng đọc khi nó đến cuối một dòng đầu vào. Thật không may, không có cách dễ dàng để làm điều này với cách tiếp cận dựa trên token. Vòng lặp hỏi liệu Scanner có giá trị double tiếp theo để đọc không, và mã nhân viên trông giống như một giá trị double có thể đọc được. Bạn có thể thử viết một bài kiểm tra phức tạp để tìm một giá trị double không phải là một số nguyên, nhưng ngay cả điều đó cũng không hiệu quả vì một số giờ là số nguyên.
Để làm cho chương trình này hoạt động, bạn cần viết một phiên bản tinh vi hơn chú ý đến dấu ngắt dòng. Chương trình phải đọc toàn bộ dòng đầu vào một lúc và xử lý dòng đó riêng lẻ. Nhớ lại phương thức main cho chương trình:
java
Copy code
```
public static void main(String[] args) throws
FileNotFoundException {
    Scanner input = new Scanner(new File("hours2.dat"));
    process(input);
}
```
Nếu bạn kết hợp một vòng lặp dựa trên dòng, bạn sẽ có phương thức sau:
java
Copy code
```
public static void main(String[] args) throws
FileNotFoundException {
    Scanner input = new Scanner(new File("hours2.dat"));

    while (input.hasNextLine()) {
        String text = input.nextLine();
        processLine(text);
    }
}
```

Đọc tệp theo từng dòng đảm bảo rằng bạn không vô tình kết hợp dữ liệu của hai nhân viên. Nhược điểm của cách tiếp cận này là bạn phải viết một phương thức gọi là processLine nhận một chuỗi làm tham số và bạn phải tách rời chuỗi đó. Nó chứa mã nhân viên, tiếp theo là tên nhân viên, tiếp theo là các con số chỉ ra số giờ mà nhân viên đã làm việc vào những ngày khác nhau. Nói cách khác, dòng đầu vào bao gồm nhiều mảnh (token) mà bạn muốn xử lý từng mảnh một. Dễ dàng hơn nhiều để xử lý dữ liệu này theo cách dựa trên token hơn là có nó trong một chuỗi.
May mắn thay, có một cách tiện lợi để làm điều này. Bạn có thể tạo một đối tượng Scanner từ một chuỗi riêng lẻ. Hãy nhớ rằng cũng như bạn có thể gắn vòi nước vào các nguồn nước khác nhau (một vòi nước trong nhà gắn với nước thành phố hoặc giếng nước so với một vòi nước trong vườn gắn với một cái xô), bạn có thể gắn một đối tượng Scanner vào các nguồn đầu vào khác nhau (đầu vào bàn phím, tệp hoặc chuỗi).
Khi bạn tạo một Scanner mới từ một chuỗi, bạn có thể sử dụng các phương thức next, nextInt, nextDouble, và các phương thức khác của Scanner trên dữ liệu chuỗi. Nói cách khác, bạn có thể đối xử với chuỗi như thể nó là dữ liệu tệp và sử dụng Scanner để tách dữ liệu. Điều này rất thuận tiện vì bạn có thể thực hiện kiểu xử lý dựa trên dòng đối với toàn bộ tệp, nhưng sau đó thực hiện kiểu xử lý dựa trên token đối với các chuỗi riêng lẻ mà bạn đọc từ tệp. Bạn sẽ sử dụng một Scanner "master" để đọc toàn bộ dòng đầu vào từ tệp, nhưng sau đó sử dụng các Scanner "slave" để tách từng dòng đầu vào riêng lẻ.
Bạn có thể thực hiện thay đổi này trong chương trình của mình bằng cách thêm phương thức sau:
java
Copy code
```
public static void processLine(String line) {
    Scanner lineScan = new Scanner(line);
```

```
    int id = lineScan.nextInt();
    String name = lineScan.next();
    double sum = 0.0;

    while (lineScan.hasNextDouble()) {
        sum += lineScan.nextDouble();
    }

    System.out.println("Total hours worked by " + name + " (id#" +
id + ") = " + sum);
}
```

Khi bạn gọi phương thức processLine, bạn sẽ chuyển nó một chuỗi như "101 Erica 7.5 8.5 10.25 8 8.5". Chương trình sẽ tạo một đối tượng Scanner từ chuỗi này và sau đó xử lý dòng bằng cách đọc nó từng token một. Một đối tượng Scanner được tạo ra từ một chuỗi sẽ coi toàn bộ chuỗi đó là nguồn đầu vào của nó, vì vậy bạn có thể sử dụng nextInt, nextDouble, và các phương thức khác để xử lý đầu vào. Vòng lặp dựa trên token sẽ ngừng ở cuối dòng, vì đó là giới hạn của đầu vào cho Scanner slave.

Khi bạn chạy phiên bản mới của chương trình trên tệp đầu vào mới, nó tạo ra đầu ra sau:

bash

Copy code

```
Total hours worked by Erica (id#101) = 42.75
Total hours worked by Erin (id#783) = 55.75
Total hours worked by Simone (id#114) = 24.0
Total hours worked by Ryan (id#238) = 31.75
Total hours worked by Kendall (id#156) = 5.5
```

Bằng cách sử dụng một vòng lặp dựa trên dòng trong main và một vòng lặp dựa trên token trong processLine, bạn có thể xử lý đầu vào một cách chính xác.

java

Copy code

```
import java.io.*;
import java.util.*;

public class TotalHoursWorked {
    public static void main(String[] args) throws
FileNotFoundException {
        Scanner input = new Scanner(new File("hours2.dat"));

        while (input.hasNextLine()) {
            String text = input.nextLine();
            processLine(text);
        }
    }

    public static void processLine(String line) {
        Scanner lineScan = new Scanner(line);

        int id = lineScan.nextInt();
        String name = lineScan.next();
        double sum = 0.0;

        while (lineScan.hasNextDouble()) {
            sum += lineScan.nextDouble();
        }

        System.out.println("Total hours worked by " + name + " (id#"
+ id + ") = " + sum);
    }
}
```

6.4 Xử lý tệp nâng cao

Trong phần này, chúng ta sẽ khám phá hai chủ đề nâng cao liên quan đến xử lý tệp: tạo tệp đầu ra và đảm bảo rằng các tệp có thể được đọc.

Tạo tệp đầu ra với PrintStream

Tất cả các chương trình mà chúng ta đã nghiên cứu từ trước đến nay đều gửi đầu ra đến cửa sổ console bằng cách gọi System.out.print hoặc System.out.println. Nhưng cũng giống như bạn có thể đọc đầu vào từ một tệp thay vì đọc từ console, bạn cũng có thể ghi đầu ra vào một tệp thay vì ghi nó vào console. Có nhiều cách để thực hiện nhiệm vụ này. Cách tiếp cận đơn giản nhất là tận dụng những gì bạn đã biết. Bạn đã học tất cả về cách hoạt động của các lệnh print và println, và bạn có thể sử dụng kiến thức đó để dễ dàng tạo các tệp đầu ra.

Nếu bạn nhìn vào tài liệu Java, bạn sẽ thấy rằng System.out là một biến lưu trữ một tham chiếu đến một đối tượng kiểu PrintStream. Các lệnh print và println mà bạn đã viết là các cuộc gọi tới các phương thức thuộc lớp PrintStream. Biến System.out lưu trữ một tham chiếu đến một đối tượng PrintStream đặc biệt được liên kết với cửa sổ console. Tuy nhiên, bạn có thể tạo các đối tượng PrintStream khác để gửi đầu ra của chúng đến các nơi khác. Giả sử, ví dụ, bạn muốn gửi đầu ra đến một tệp có tên là results.txt. Bạn có thể tạo một đối tượng PrintStream như sau:

java

Copy code

```
PrintStream output = new PrintStream(new File("results.txt"));
```

Dòng mã này trông rất giống với dòng mã mà chúng ta đã sử dụng để tạo một Scanner liên kết với một tệp đầu vào. Trong trường hợp này, máy tính đang tạo một tệp đầu ra. Nếu không có tệp như vậy, chương trình sẽ tạo nó. Nếu một tệp như vậy đã tồn tại, máy tính sẽ ghi đè lên phiên bản hiện tại. Ban đầu, tệp sẽ trống. Cuối cùng, tệp sẽ chứa những gì bạn yêu cầu nó sản xuất thông qua các cuộc gọi print và println.

Dòng mã tạo một đối tượng PrintStream có thể tạo ra một ngoại lệ nếu Java không thể tạo tệp mà bạn đã mô tả. Có nhiều lý do khiến điều này có thể xảy ra: Bạn có thể không có quyền ghi vào thư mục, hoặc tệp có thể bị khóa vì một chương trình khác đang sử dụng nó. Cũng giống như dòng mã tạo ra một Scanner dựa trên tệp, dòng mã này có khả năng ném ra ngoại lệ FileNotFoundException. Do đó, Java yêu cầu bạn phải bao gồm mệnh đề throws trong bất kỳ phương thức nào chứa dòng mã này. Cách tiếp cận đơn giản nhất là đặt dòng này trong main. Thực tế, việc bắt đầu phương thức main với các dòng mã xử lý tệp đầu vào và đầu ra là một thông lệ phổ biến.
Sau khi bạn đã tạo một đối tượng PrintStream, làm thế nào để sử dụng nó? Bạn có thể đã có một ý tưởng tốt về những gì cần làm. Chúng ta đã thực hiện các cuộc gọi tới System.out.print và System.out.println kể từ Chương 1. Nếu bạn nhớ tất cả những gì bạn biết về System.out, bạn sẽ có một ý tưởng tốt về những gì cần làm, nhưng đối với chương trình này, bạn sẽ gọi output.print thay vì System.out.print và output.println thay vì System.out.println.
Dưới đây là một ví dụ đơn giản, nhớ rằng trong Chương 1 chúng ta đã xem xét biến thể sau của chương trình "hello world" đơn giản sản xuất nhiều dòng đầu ra:
java
Copy code
```
1 public class Hello2 {
2    public static void main(String[] args) {
3        System.out.println("Hello, world!");
4        System.out.println();
5        System.out.println("This program produces four");
6        System.out.println("lines of output.");
7    }
8 }
```

Dưới đây là một biến thể gửi đầu ra của nó đến một tệp có tên là hello.txt:

java

Copy code

```
1 // Biến thể của Hello2 ghi ra tệp.
2 import java.io.*;
3 public class Hello4 {
4    public static void main(String[] args)
5        throws FileNotFoundException {
6        PrintStream output =
7            new PrintStream(new File("hello.txt"));
8        output.println("Hello, world.");
9        output.println();
10       output.println("This program produces four");
11       output.println("lines of output.");
12   }
13 }
```

Khi bạn chạy phiên bản mới này của chương trình, một điều thú vị xảy ra. Chương trình dường như không làm gì cả; không có đầu ra nào xuất hiện trên console. Bạn đã quen với việc viết các chương trình gửi đầu ra đến console nên điều này có thể gây bối rối lúc đầu. Chúng ta không thấy bất kỳ đầu ra nào trong cửa sổ console khi chúng ta chạy chương trình này vì đầu ra đã được chuyển hướng đến một tệp. Sau khi chương trình hoàn thành thực thi, bạn có thể mở tệp có tên hello.txt và bạn sẽ thấy rằng nó chứa nội dung sau:

lua

Copy code

```
Hello, world.
This program produces four
lines of output.
```

Điểm chính là mọi thứ bạn đã học được để làm với System.out, bạn cũng có thể làm với các đối tượng PrintStream được liên kết với các tệp.
Bạn cũng có thể viết các phương thức nhận các đối tượng PrintStream làm tham số. Ví dụ, hãy xem xét nhiệm vụ chỉnh sửa khoảng cách cho một loạt từ. Giả sử rằng bạn có một dòng văn bản với khoảng cách không đều, như dòng sau:
arduino
Copy code
```
a      new  nation,    conceived  in    liberty
```
Giả sử bạn muốn in dòng văn bản này với chính xác một khoảng trắng giữa mỗi cặp từ:
arduino
Copy code
```
a new nation, conceived in liberty
```
Làm thế nào để bạn làm điều đó? Giả sử bạn đang viết một phương thức được truyền một chuỗi để echo và một đối tượng PrintStream mà đầu ra sẽ được gửi tới:
java
Copy code
```
public static void echoFixed(String text, PrintStream output) {
    ...
}
```
Bạn có thể tạo một Scanner từ chuỗi và sau đó sử dụng phương thức next để đọc từng từ một. Nhớ rằng lớp Scanner bỏ qua khoảng trắng, vì vậy bạn sẽ chỉ nhận được các từ riêng lẻ mà không có tất cả các khoảng trắng giữa chúng. Khi bạn đọc các từ, bạn sẽ cần echo chúng tới đối tượng PrintStream. Đây là một nỗ lực đầu tiên:
java
Copy code
```
Scanner data = new Scanner(text);
while (data.hasNext()) {
```

```
    output.print(data.next());
}
```
Mã này làm rất tốt việc loại bỏ các chuỗi dài khoảng trắng từ chuỗi, nhưng nó đã đi quá xa: Nó loại bỏ tất cả các khoảng trắng. Để có một khoảng trắng giữa mỗi cặp từ, bạn sẽ phải bao gồm một số khoảng trắng:
java
Copy code
```
Scanner data = new Scanner(text);
while (data.hasNext()) {
    output.print(data.next() + " ");
}
```
Phương thức này tạo ra kết quả trông khá tốt, nhưng nó in một khoảng trắng thừa ở cuối dòng. Để loại bỏ khoảng trắng đó sao cho bạn chỉ thực sự có các khoảng trắng xuất hiện giữa các cặp từ, bạn sẽ phải thay đổi phương thức một chút. Đây là một vấn đề "cọc rào" cổ điển; bạn muốn in một từ nhiều hơn số khoảng trắng. Bạn có thể sử dụng giải pháp điển hình là xử lý từ đầu tiên trước khi vòng lặp bắt đầu và hoán đổi thứ tự của hai thao tác khác trong vòng lặp (in khoảng trắng và sau đó là từ):
java
Copy code
```
Scanner data = new Scanner(text);
output.print(data.next());
while (data.hasNext()) {
    output.print(" " + data.next());
}
```
Phiên bản này của chương trình hoạt động tốt trong hầu hết các trường hợp, nhưng bằng cách bao gồm giải pháp "cọc rào", là echo từ đầu tiên trước khi vòng lặp bắt đầu, bạn đã đưa vào một giả định rằng có một từ đầu tiên. Nếu chuỗi không có từ nào, cuộc gọi tới next này sẽ ném ra một ngoại lệ. Vì vậy, bạn cần một bài kiểm tra cho trường hợp chuỗi không chứa bất kỳ từ nào. Nếu

bạn cũng muốn chương trình này tạo ra một dòng đầu ra hoàn chỉnh, bạn sẽ phải bao gồm một cuộc gọi tới println để hoàn thành dòng đầu ra sau khi in các từ riêng lẻ. Kết hợp những thay đổi này, bạn có được mã sau:
java
Copy code
```
public static void echoFixed(String text, PrintStream output) {
    Scanner data = new Scanner(text);
    if (data.hasNext()) {
        output.print(data.next());
        while (data.hasNext()) {
            output.print(" " + data.next());
        }
    }
    output.println();
}
```
Lưu ý rằng bây giờ bạn đang gọi output.print và output.println thay vì gọi System.out.print và System.out.println. Một khía cạnh thú vị của phương thức này là nó có thể được sử dụng không chỉ để gửi đầu ra đến một tệp đầu ra, mà còn để gửi nó đến System.out. Phần đầu của phương thức cho biết rằng nó hoạt động trên bất kỳ đối tượng PrintStream nào, vì vậy bạn có thể gọi nó để gửi đầu ra hoặc tới một đối tượng PrintStream liên kết với một tệp hoặc tới System.out.
Chương trình hoàn chỉnh sau đây sử dụng phương thức này để chỉnh sửa khoảng cách trong toàn bộ tệp văn bản đầu vào. Để nhấn mạnh tính linh hoạt của phương thức, chương trình gửi đầu ra của nó cả vào một tệp (có tên là words2.txt) và vào console:
java
Copy code
```
1 // Chương trình này loại bỏ khoảng trắng thừa trong một tệp đầu vào.
2 import java.io.*;
```

```
3 import java.util.*;
4 public class FixSpacing {
5    public static void main(String[] args)
6        throws FileNotFoundException {
7        Scanner input = new Scanner(new File("words.txt"));
8        PrintStream output = new PrintStream(new
File("words2.txt"));
9        while (input.hasNextLine()) {
10           String text = input.nextLine();
11           echoFixed(text, output);
12           echoFixed(text, System.out);
13       }
14   }
15   public static void echoFixed(String text, PrintStream output) {
16       Scanner data = new Scanner(text);
17       if (data.hasNext()) {
18           output.print(data.next());
19           while (data.hasNext()) {
20               output.print(" " + data.next());
21           }
22       }
23       output.println();
24   }
25 }
```

Hãy xem xét tệp đầu vào sau:

```
csharp
Copy code
four         score    and
seven    years ago    our
fathers brought  forth on this     continent
a       new nation,     conceived  in    liberty
  and  dedicated   to  the  proposition    that
   all   men    are     created    equal
```

Sử dụng tệp đầu vào này, chương trình tạo ra tệp đầu ra sau, có tên là words2.txt:
csharp
Copy code
four score and
seven years ago our
fathers brought forth on this continent
a new nation, conceived in liberty
and dedicated to the proposition that
all men are created equal
Đầu ra cũng xuất hiện trong cửa sổ console.

Đảm bảo rằng tệp có thể được đọc

Các chương trình mà chúng ta đã nghiên cứu cho đến nay giả định rằng người dùng sẽ cung cấp một tên tệp hợp lệ. Nhưng nếu người dùng vô tình gõ vào tên của một tệp không tồn tại hoặc không thể đọc được vì một lý do nào đó thì sao? Trong phần này, chúng ta sẽ khám phá cách đảm bảo rằng một tệp có thể được đọc.

Hãy khám phá cách bạn có thể xử lý nhiệm vụ nhắc nhở người dùng nhập tên tệp trong cửa sổ console. Nếu người dùng không nhập một tên tệp hợp lệ, bạn có thể tiếp tục nhắc nhở cho đến khi người dùng nhập đúng tên tệp. Lớp File có một phương thức gọi là canRead mà bạn có thể sử dụng để kiểm tra xem tệp có tồn tại và có thể đọc được hay không. Bạn có thể in ra thông báo lỗi mỗi khi tệp không thể đọc, cho đến khi bạn nhận được một tên tệp hợp lệ. Tình huống này là một vấn đề "cọc rào". Nếu bạn cuối cùng nhắc nhở người dùng n lần, bạn sẽ muốn tạo ra n−1 thông báo lỗi. Bạn có thể sử dụng giải pháp "cọc rào" cổ điển là nhắc nhập tên tệp một lần trước khi vòng lặp bắt đầu:
java
Copy code
System.out.print("input file name? ");

```
File f = new File(console.nextLine());
while (!f.canRead()) {
    System.out.println("File not found. Try again.");
    System.out.print("input file name? ");
    f = new File(console.nextLine());
}
```

Mã này có thể được bao gồm trong phương thức main, nhưng có đủ mã ở đây để bạn nên đặt nó vào một phương thức riêng. Bởi vì mã này yêu cầu người dùng nhập đầu vào, nên bạn cần một Scanner để đọc từ console. Phương thức này có thể trả về một Scanner liên kết với tệp đầu vào để xử lý.

Khi bạn thử tạo một phương thức bao gồm mã này, bạn lại gặp phải vấn đề về ngoại lệ kiểm tra. Mặc dù chúng ta rất cẩn thận để đảm bảo rằng tệp tồn tại và có thể đọc được, nhưng trình biên dịch Java không biết điều đó. Từ quan điểm của nó, mã này có thể ném ra một ngoại lệ. Bạn vẫn cần bao gồm mệnh đề throws trong tiêu đề phương thức, giống như bạn đã làm với main:

java

Copy code

```
public static Scanner getInput(Scanner console)
        throws FileNotFoundException {
    System.out.print("input file name? ");
    File f = new File(console.nextLine());
    while (!f.canRead()) {
        System.out.println("File not found. Try again.");
        System.out.print("input file name? ");
        f = new File(console.nextLine());
    }
    // bây giờ chúng ta biết rằng f là một tệp có thể đọc được
    return new Scanner(f);
}
```

Dưới đây là một biến thể của chương trình CountWords mà nhắc người dùng nhập tên tệp:

java
Copy code

```java
// Biến thể của CountWords nhắc nhập tên tệp.
import java.io.*;
import java.util.*;
public class CountWords2 {
    public static void main(String[] args)
            throws FileNotFoundException {
        Scanner console = new Scanner(System.in);
        Scanner input = getInput(console);
        // và đếm số từ
        int count = 0;
        while (input.hasNext()) {
            String word = input.next();
            count++;
        }
        System.out.println("total words = " + count);
    }
    // Nhắc người dùng nhập tên tệp hợp lệ; tạo và
    // trả về một Scanner liên kết với tệp
    public static Scanner getInput(Scanner console)
            throws FileNotFoundException {
        System.out.print("input file name? ");
        File f = new File(console.nextLine());
        while (!f.canRead()) {
            System.out.println("File not found. Try again.");
            System.out.print("input file name? ");
            f = new File(console.nextLine());
        }
        // bây giờ chúng ta biết rằng f là một tệp có thể đọc được
        return new Scanner(f);
    }
}
```

Nhật ký thực thi sau đây cho thấy điều gì xảy ra khi người dùng nhập vào một số tên tệp không hợp lệ, và kết thúc bằng một tên tệp hợp lệ:
css
Copy code
input file name? amlet.txt
File not found. Try again.
input file name? hamlet.dat
File not found. Try again.
input file name? humlet.txt
File not found. Try again.
input file name? hamlet.txt
Total words = 31956
Mã mở tệp khá tiêu chuẩn và có thể được sử dụng mà không cần chỉnh sửa trong nhiều chương trình khác nhau. Chúng ta gọi đây là mã "boilerplate".

Mã "boilerplate"

Mã có xu hướng giống nhau từ chương trình này sang chương trình khác. Phương thức getInput là một ví dụ điển hình về loại mã "boilerplate" mà bạn có thể sử dụng trong nhiều chương trình xử lý tệp khác nhau.

6.5 Nghiên cứu trường hợp: Tra cứu mã Zip

Biết khoảng cách giữa hai địa điểm hóa ra là vô cùng hữu ích và có giá trị. Ví dụ, nhiều trang web hẹn hò phổ biến trên Internet cho phép bạn tìm kiếm mọi người dựa trên vị trí mục tiêu. Trên Match.com, bạn có thể tìm kiếm các đối tượng tiềm năng trong bán kính cụ thể từ một thành phố hoặc mã zip nhất định (5 dặm, 10 dặm, 15 dặm, 25 dặm, v.v.). Rõ ràng đây là một tính năng quan trọng cho một trang web hẹn hò vì mọi người chủ yếu quan tâm đến việc hẹn hò với những người sống gần họ.
Có rất nhiều ứng dụng khác của loại tìm kiếm gần gũi này. Vào những năm 1970 và 1980, có một sự bùng nổ về sự quan tâm

đến cái gọi là tiếp thị thư trực tiếp, đã tạo ra cái mà chúng ta gọi là thư rác ngày nay. Tìm kiếm gần gũi rất quan trọng trong các chiến dịch tiếp thị thư trực tiếp. Ví dụ, một cửa hàng địa phương có thể quyết định gửi một tập sách đến tất cả cư dân sống trong vòng 5 dặm từ cửa hàng. Một ứng cử viên chính trị có thể trả tiền cho một tổ chức thành viên như The Sierra Club hoặc Hiệp hội Súng Quốc gia để có được địa chỉ thư của tất cả các thành viên sống trong một khoảng cách nhất định từ một thị trấn hoặc khu vực thành phố.
Các cơ sở dữ liệu khổng lồ theo dõi khách hàng tiềm năng và cử tri. Các tổ chức tiếp thị thư trực tiếp thường muốn tìm khoảng cách giữa một trong những cá nhân này và một vị trí cố định. Các phép tính khoảng cách hầu như đều được thực hiện bằng mã zip. Có hơn 40.000 mã zip gồm năm chữ số ở Hoa Kỳ. Một số mã zip bao phủ các khu vực nông thôn khá lớn, nhưng thường mã zip xác định vị trí của bạn trong một thành phố hoặc thị trấn trong phạm vi dưới một dặm. Nếu bạn sử dụng cơ sở dữ liệu Zip+4 cụ thể hơn, bạn có thể xác định vị trí trong vòng vài khối phố.
Nếu bạn thực hiện tìm kiếm trên web về "cơ sở dữ liệu mã zip" hoặc "phần mềm mã zip", bạn sẽ thấy có rất nhiều người bán dữ liệu và phần mềm để giải thích dữ liệu. Cũng có một số cơ sở dữ liệu miễn phí, mặc dù dữ liệu không chính xác bằng. Cục điều tra dân số Hoa Kỳ là nguồn của nhiều dữ liệu miễn phí này.
Để khám phá ứng dụng này, hãy viết một chương trình tìm tất cả các mã zip trong một phạm vi nhất định của một mã zip khác. Một trang web như Match.com có thể sử dụng logic của chương trình này để tìm các đối tượng tiềm năng trong bán kính nhất định. Bạn chỉ cần bắt đầu với mã zip quan tâm, tìm tất cả các mã zip khác trong một khoảng cách cụ thể và sau đó tìm tất cả các khách hàng có những mã zip đó. Chúng tôi không có quyền truy cập vào một cơ sở dữ liệu hẹn hò lớn như Match.com, vì vậy chúng tôi sẽ chỉ thực hiện phần đầu tiên của nhiệm vụ này, tìm các mã zip trong phạm vi khoảng cách đã chỉ định.

Như đã đề cập trước đó, một số cơ sở dữ liệu mã zip miễn phí có sẵn trực tuyến. Chương trình mẫu của chúng tôi sử dụng dữ liệu do nhà phát triển phần mềm Schuyler Erle tổng hợp, dữ liệu của anh ấy được phân phối miễn phí qua giấy phép Creative Commons (lấy từ https://boutell.com/zipcodes/).
Chúng tôi đã định dạng lại dữ liệu để làm cho nó thuận tiện hơn cho chúng tôi khi làm việc với nó (một quá trình được gọi là dữ liệu munging). Chúng tôi sẽ làm việc với một tệp có tên là zipcode.txt, chứa một loạt các mục 3 dòng, mỗi mục cho một mã zip. Dòng đầu tiên chứa mã zip, dòng thứ hai chứa thành phố và tiểu bang, và dòng thứ ba chứa hai số đại diện cho vĩ độ và kinh độ của mã zip. Ví dụ: đây là một mục cho mã zip nơi ở của một trong các tác giả:
Copy code
98104
Seattle, WA
47.60252 -122.32855
Nhiệm vụ tổng thể là yêu cầu người dùng nhập mã zip mục tiêu và khoảng cách và hiển thị tất cả các mã zip trong phạm vi khoảng cách đã cho của mã zip mục tiêu. Dưới đây là một nỗ lực đầu tiên để viết mã giả cho nhiệm vụ tổng thể:

- giới thiệu chương trình cho người dùng.
- yêu cầu mã zip mục tiêu và khoảng cách.
- hiển thị các mã zip phù hợp từ tệp.

Cách tiếp cận này không hoàn toàn hoạt động. Để hiển thị kết quả phù hợp, bạn phải so sánh vị trí mục tiêu với từng mã zip khác nhau trong tệp dữ liệu. Bạn sẽ cần thông tin vĩ độ và kinh độ để thực hiện so sánh này. Nhưng khi yêu cầu người dùng, bạn chỉ yêu cầu một mã zip và khoảng cách. Bạn có thể thay đổi chương trình để yêu cầu vĩ độ và kinh độ, nhưng điều đó sẽ không thân thiện với người dùng. Hãy tưởng tượng nếu Match.com yêu cầu bạn biết vĩ độ và kinh độ của mình để tìm kiếm những người sống gần bạn.

Thay vào đó, bạn có thể sử dụng dữ liệu mã zip để tìm vĩ độ và kinh độ của mã zip mục tiêu. Kết quả là, bạn sẽ phải tìm kiếm dữ liệu hai lần. Lần đầu tiên bạn sẽ tìm kiếm mã zip mục tiêu, để có thể tìm tọa độ của nó. Lần thứ hai bạn sẽ hiển thị tất cả các mã zip trong khoảng cách mà người dùng yêu cầu. Dưới đây là phiên bản mới của mã giả:

- giới thiệu chương trình cho người dùng.
- yêu cầu mã zip mục tiêu và khoảng cách.
- tìm tọa độ cho mã zip mục tiêu.
- hiển thị các mã zip phù hợp từ tệp.

Giới thiệu chương trình và yêu cầu mã zip mục tiêu và khoảng cách là các nhiệm vụ khá đơn giản không cần giải thích chi tiết. Công việc thực sự của chương trình là giải quyết hai bước thứ ba và thứ tư trong mã giả này. Mỗi bước này đủ phức tạp để được đưa vào một phương thức tĩnh.

Đầu tiên hãy xem xét vấn đề tìm tọa độ cho mã zip mục tiêu. Bạn cần thiết lập một Scanner để đọc từ tệp, và sau đó bạn cần gọi phương thức thực hiện tìm kiếm. Nhưng phương thức tìm kiếm nên trả về thông tin gì? Bạn muốn tọa độ của mã zip mục tiêu (vĩ độ và kinh độ). Phương thức của bạn không thể trả về hai giá trị khác nhau, nhưng các tọa độ này xuất hiện trên một dòng đầu vào duy nhất, vì vậy bạn có thể trả về dòng đầu vào đó dưới dạng chuỗi. Điều đó có nghĩa là phương thức chính của bạn sẽ bao gồm mã sau:

java

Copy code

```
Scanner input = new Scanner(new File("zipcode.txt"));
String targetCoordinates = find(target, input);
```

Phương thức này sẽ đọc tệp đầu vào từng dòng một, tìm kiếm mã zip mục tiêu. Nhớ rằng mỗi mục trong tệp bao gồm ba dòng khác nhau. Do đó, bạn cần một biến thể nhỏ của vòng lặp xử lý dòng chuẩn để đọc ba dòng mỗi lần lặp:

java

Copy code
```
public static String find(String target, Scanner input) {
    while (input.hasNextLine()) {
        String zip = input.nextLine();
        String city = input.nextLine();
        String coordinates = input.nextLine();
    }
}
```
Khi đọc các mục mã zip khác nhau, bạn muốn kiểm tra từng mục để xem nó có khớp với mục tiêu không. Nhớ rằng bạn cần sử dụng phương thức equals để so sánh các chuỗi về sự bình đẳng. Nếu tìm thấy kết quả khớp, bạn có thể in nó ra và trả về tọa độ:
java
Copy code
```
public static String find(String target, Scanner input) {
    while (input.hasNextLine()) {
        String zip = input.nextLine();
        String city = input.nextLine();
        String coordinates = input.nextLine();
        if (zip.equals(target)) {
            System.out.println(zip + ": " + city);
            return coordinates;
        }
    }
}
```
Phương thức này chưa hoàn thành vì bạn phải xem xét trường hợp mã zip mục tiêu không xuất hiện trong tệp. Trong trường hợp đó, bạn thoát khỏi vòng lặp mà không trả về giá trị nào. Có rất nhiều điều mà chương trình có thể làm vào thời điểm này, chẳng hạn như in thông báo lỗi hoặc ném ngoại lệ. Để giữ mọi thứ đơn giản, hãy trả về một tập tọa độ giả. Nếu chương trình trả về vĩ độ và kinh độ là (0, 0), sẽ không có kết quả khớp trừ khi người dùng yêu cầu một phạm vi khoảng cách rất lớn (trên 4.000 dặm):

java
Copy code
```
public static String find(String target, Scanner input) {
    while (input.hasNextLine()) {
        String zip = input.nextLine();
        String city = input.nextLine();
        String coordinates = input.nextLine();
        if (zip.equals(target)) {
            System.out.println(zip + ": " + city);
            return coordinates;
        }
    }
    return "0 0";
}
```
Bây giờ bạn đã sẵn sàng viết mã sẽ hiển thị tất cả các mã zip trong khoảng cách mà người dùng yêu cầu từ mã zip mục tiêu. Lại một lần nữa, bạn phải đọc toàn bộ tệp, ba dòng tại một thời điểm. Để xác định khoảng cách giữa các tọa độ, bạn cần một công thức trong phương thức Math được gọi là acos (công thức được đưa ra trong các bài đọc).

Đầu tiên bạn sẽ muốn có được tọa độ của mã zip khác và chuyển đổi chúng thành double. Chúng tôi chia tọa độ thành hai số khác nhau và gọi phương thức Double.parseDouble để thực hiện chuyển đổi:

java
Copy code
```
public static void findAll(double lat1, double lon1, double dist,
Scanner input) {
    while (input.hasNextLine()) {
        String zip = input.nextLine();
        String city = input.nextLine();
        String coordinates = input.nextLine();
        Scanner data = new Scanner(coordinates);
```

```
        double lat2 = Double.parseDouble(data.next());
        double lon2 = Double.parseDouble(data.next());
    }
}
```

Có các hệ thống tọa độ khác nhau để đo khoảng cách lớn. Đo khoảng cách giữa hai điểm là phức tạp bởi thực tế rằng Trái Đất không phải là một hình cầu hoàn hảo. Để đơn giản hóa vấn đề, hầu hết các hệ thống tọa độ coi Trái Đất là một hình cầu với bán kính khoảng 3.959 dặm (bán kính này được gọi là bán kính của Trái Đất). Với hệ tọa độ này, bạn có thể sử dụng phương trình sau để tính khoảng cách giữa các tọa độ:

d=3959×acos[sin⁡(lat1)×sin⁡(lat2)+cos⁡(lat1)×cos⁡(lat2)×cos⁡(lon1 - lon2)]d = 3959 \times \text{{acos}}[\sin(\text{{lat1}}) \times \sin(\text{{lat2}}) + \cos(\text{{lat1}}) \times \cos(\text{{lat2}}) \times \cos(\text{{lon1 - lon2}})]d=3959×acos[sin(lat1)×sin(lat2)+cos(lat1)×cos(lat2)×cos(lon1 - lon2)]

Trong đó các tọa độ được đo bằng radian. Thư viện Math cung cấp các phương thức để thực hiện mọi tính toán trong phương trình này, nhưng phương thức Math.sin và Math.cos yêu cầu các góc theo radian, trong khi bạn có các tọa độ tính bằng độ. Một hình tròn có 2π2 \pi2π radian, tương đương 360 độ. Vì vậy, để chuyển đổi độ thành radian, bạn phải chia cho 360 và nhân với 2π2 \pi2π. Công thức chuyển đổi thành các lệnh sau:

java

Copy code

```
lat1 = lat1 * 2 * Math.PI / 360;
lon1 = lon1 * 2 * Math.PI / 360;
lat2 = lat2 * 2 * Math.PI / 360;
lon2 = lon2 * 2 * Math.PI / 360;
```

Phương thức findAll của bạn sẽ bao gồm một số toán học, nhưng thực hiện đúng phép toán này sẽ không quá khó. Bạn sẽ so sánh khoảng cách thực tế với giá trị khoảng cách mục tiêu mà người

dùng nhập. Nếu khoảng cách thực tế nhỏ hơn, hãy in mã zip và tên thành phố. Ví dụ, dưới đây là mã phương thức findAll hoàn chỉnh:
java
Copy code
```
public static void findAll(double lat1, double lon1, double dist, Scanner input) {
    lat1 = lat1 * 2 * Math.PI / 360;
    lon1 = lon1 * 2 * Math.PI / 360;
    while (input.hasNextLine()) {
        String zip = input.nextLine();
        String city = input.nextLine();
        String coordinates = input.nextLine();
        Scanner data = new Scanner(coordinates);
        double lat2 = Double.parseDouble(data.next());
        double lon2 = Double.parseDouble(data.next());
        lat2 = lat2 * 2 * Math.PI / 360;
        lon2 = lon2 * 2 * Math.PI / 360;
        double actual = 3959 * Math.acos(Math.sin(lat1) * Math.sin(lat2) + Math.cos(lat1) * Math.cos(lat2) * Math.cos(lon1 - lon2));
        if (actual < dist) {
            System.out.println(zip + ": " + city + " (" + actual + ")");
        }
    }
}
```
Một số chương trình sẽ hiển thị một danh sách rời rạc các lựa chọn, trong khi những chương trình khác sẽ yêu cầu người dùng đưa ra phạm vi giá trị. Bạn có thể thấy rằng phương pháp này yêu cầu nhiều nỗ lực xử lý hơn những gì bạn đã quen, nhưng nếu bạn cần những lựa chọn này, cách tiếp cận này rất hữu ích. Như vậy, chương trình mẫu hoàn chỉnh của chúng tôi có dạng sau:

java
Copy code

```
import java.util.*;
import java.io.*;

public class ZipLookup {
    public static void main(String[] args) throws
FileNotFoundException {
        Scanner console = new Scanner(System.in);
        System.out.print("target zip code: ");
        String target = console.nextLine();
        System.out.print("distance (in miles): ");
        double dist = console.nextDouble();
        Scanner input = new Scanner(new File("zipcode.txt"));
        String coord = find(target, input);
        Scanner data = new Scanner(coord);
        double lat1 = data.nextDouble();
        double lon1 = data.nextDouble();
        findAll(lat1, lon1, dist, new Scanner(new File("zipcode.txt")));
    }

    public static String find(String target, Scanner input) {
        while (input.hasNextLine()) {
            String zip = input.nextLine();
            String city = input.nextLine();
            String coordinates = input.nextLine();
            if (zip.equals(target)) {
                System.out.println(zip + ": " + city);
                return coordinates;
            }
        }
        return "0 0";
    }
```

```
    public static void findAll(double lat1, double lon1, double dist,
Scanner input) {
        lat1 = lat1 * 2 * Math.PI / 360;
        lon1 = lon1 * 2 * Math.PI / 360;
        while (input.hasNextLine()) {
            String zip = input.nextLine();
            String city = input.nextLine();
            String coordinates = input.nextLine();
            Scanner data = new Scanner(coordinates);
            double lat2 = Double.parseDouble(data.next());
            double lon2 = Double.parseDouble(data.next());
            lat2 = lat2 * 2 * Math.PI / 360;
            lon2 = lon2 * 2 * Math.PI / 360;
            double actual = 3959 * Math.acos(Math.sin(lat1) *
Math.sin(lat2) + Math.cos(lat1) * Math.cos(lat2) * Math.cos(lon1 -
lon2));
            if (actual < dist) {
                System.out.println(zip + ": " + city + " (" + actual + ")");
            }
        }
    }
}
```

Tóm tắt chương

Các tệp được biểu diễn trong Java dưới dạng các đối tượng File. Lớp File nằm trong gói java.io.

Một đối tượng Scanner có thể đọc đầu vào từ một tệp thay vì từ bàn phím. Công việc này được thực hiện bằng cách truyền new File(filename) vào hàm tạo của Scanner, thay vì truyền System.in.

Một ngoại lệ đã kiểm tra là một điều kiện lỗi chương trình phải được bắt hoặc khai báo để chương trình có thể biên dịch. Ví dụ:

khi tạo một Scanner để đọc một tệp, bạn phải viết cụm từ throws FileNotFoundException trong tiêu đề phương thức main.
Scanner xử lý một tệp đầu vào như một luồng dữ liệu một chiều, được đọc theo thứ tự từ đầu đến cuối. Con trỏ đầu vào sẽ tiêu thụ (di chuyển qua) các token đầu vào khi chúng được đọc và trả về chúng cho chương trình của bạn.
Một Scanner đọc một tệp sẽ sử dụng các phương thức hasNext khác nhau để phát hiện khi nào đầu vào của tệp đã được tiêu thụ hết.
Các Scanner có thể được truyền dưới dạng tham số cho các phương thức để đọc một phần hoặc toàn bộ tệp, vì chúng là các đối tượng và do đó sử dụng ngữ nghĩa tham chiếu.
Tên tệp có thể được chỉ định dưới dạng đường dẫn tương đối như data/text/numbers.dat, điều này ám chỉ một tệp có tên là numbers.dat tồn tại trong thư mục con data/text/ của thư mục hiện tại. Ngoài ra, bạn có thể chỉ định một đường dẫn đầy đủ như C:/Documents and Settings/user/My Documents/data/text/numbers.dat.
Trong nhiều tệp, đầu vào được cấu trúc theo dòng và có ý nghĩa khi xử lý các tệp đó theo từng dòng. Trong những trường hợp như vậy, thường sử dụng các vòng lặp lồng nhau: một vòng lặp ngoài lặp qua từng dòng của tệp và một vòng lặp trong xử lý các token trong từng dòng.
Xuất dữ liệu ra tệp có thể được thực hiện với một đối tượng PrintStream, được tạo ra với một đối tượng File và có các phương thức giống như System.out, chẳng hạn như println và print.

Các vấn đề kiểm tra tự động
Phần 6.1: Cơ bản về đọc tệp

1. Tệp là gì? Làm thế nào để chúng ta đọc dữ liệu từ một tệp trong Java?
2. Điều gì sai với dòng mã sau?

java
Copy code
Scanner input = new Scanner("test.dat");

- **Sai sót**: Đoạn mã này cố gắng tạo một đối tượng Scanner trực tiếp từ một chuỗi, nhưng Scanner cần một đối tượng File hoặc một InputStream làm đối số.

3. Đâu là cú pháp đúng để khai báo một Scanner để đọc tệp example.txt trong thư mục hiện tại?
    - a. Scanner input = new Scanner("C:\example.txt");
    - b. Scanner input = new Scanner(new File("example.txt"));
    - c. Scanner input = new File("\\example.txt");
    - d. File input = new Scanner("/example.txt");
    - e. Scanner input = new Scanner("C:/example.txt");
    - **Đáp án đúng**: **b.** Scanner input = new Scanner(new File("example.txt"));
4. Viết mã để tạo một đối tượng Scanner để đọc tệp input.txt, tệp này tồn tại trong cùng thư mục với chương trình của bạn.

java
Copy code
Scanner input = new Scanner(new File("input.txt"));

Phần 6.2: Chi tiết về xử lý dựa trên token

Câu 5: Với dòng đầu vào sau đây, Scanner sẽ chia dòng thành các token nào?

vbnet
Copy code
welcome...to the matrix.

- a. "welcome", "to", "the", "matrix"
- b. "welcome...to the matrix."
- c. "welcome...to", "the", "matrix."
- d. "welcome...", "to", "the matrix."

- e. "welcome", "to the matrix"

Câu 6: Với các dòng đầu vào sau đây, Scanner sẽ chia dòng thành các token nào?
Copy code
in fourteen-hundred 92

columbus sailed the ocean blue :)

- a. "in", "fourteen-hundred", "92"
- b. "in", "fourteen-hundred", "92", "columbus", "sailed", "the", "ocean", "blue", ":)"
- c. "in", "fourteen", "hundred", "92", "columbus", "sailed", "the", "ocean", "blue"
- d. "in", "fourteen-hundred", "92\ncolumbus", "sailed", "the", "ocean", "blue :)"
- e. "in fourteen-hundred 92", "columbus sailed the ocean blue :)"

Câu 7: Có bao nhiêu token trong đầu vào sau đây, và các phương thức nào của Scanner có thể được sử dụng để đọc từng token đó?
mathematica
Copy code
Hello there, how are you?

I am "very well", thank you.

12 34 5.67 (8 + 9) "10"

Câu 8: Điều gì sai với dòng mã sau đây?
java
Copy code
Scanner input = new Scanner(new File("C:\\temp\\new files\\test.dat"));
(*Gợi ý: Hãy thử in chuỗi trong dòng mã này.*)

Câu 9: Trả lời các câu hỏi sau về một chương trình Java nằm trên máy tính Windows trong thư mục C:\Documents and Settings\amanda\My Documents\programs:

- a. Hai cách hợp pháp để bạn có thể tham chiếu đến tệp C:\Documents and Settings\amanda\My Documents\programs\numbers.dat là gì?
- b. Làm thế nào để bạn tham chiếu đến tệp C:\Documents and Settings\amanda\My Documents\programs\data\homework6\input.dat?
- c. Có bao nhiêu cách hợp pháp để bạn tham chiếu đến tệp C:\Documents and Settings\amanda\My Documents\homework\data.txt?

Câu 10: Trả lời các câu hỏi sau về một chương trình Java nằm trên máy tính Linux trong thư mục /home/amanda/Documents/hw6:

- a. Hai cách hợp pháp để bạn có thể tham chiếu đến tệp /home/amanda/Documents/hw6/names.txt là gì?
- b. Làm thế nào để bạn tham chiếu đến tệp /home/amanda/Documents/hw6/data/numbers.txt?
- c. Có bao nhiêu cách hợp pháp để bạn tham chiếu đến tệp /home/amanda/download/saved.html?

Câu 11: Chương trình sau chứa 6 lỗi! Chúng là gì?

java

Copy code

```
1 public class Oops6 {
2     public static void main(String[] args) {
3         Scanner in = new Scanner("example.txt");
4         countWords(in);
5     }
6     // Đếm tổng số dòng và từ trong input scanner.
7     public static void countWords(Scanner input) {
8         Scanner input = new Scanner("example.txt");
9         int lineCount = 0;
```

```
10          int wordCount = 0;
11          while (input.nextLine()) {
12              String line = input.line(); // đọc một dòng
13              lineCount++;
14              while (line.next()) { // các token trong dòng
15                  String word = line.hasNext;
16                  wordCount++;
17              }
18          }
19      }
20 }
```

Phần 6.3: Xử lý Dựa trên Dòng

Câu 12: Cho các câu hỏi tiếp theo, hãy xem xét một tệp có tên là readme.txt với nội dung sau:

lua

Copy code

```
6.7         This file has

        several input lines.

  10 20         30    40

test
```

Đầu ra sẽ là gì khi đoạn mã sau được chạy trên tệp readme.txt?

java

Copy code

```
Scanner input = new Scanner(new File("readme.txt"));
int count = 0;
while (input.hasNextLine()) {
    System.out.println("input: " + input.nextLine());
    count++;
```

}
System.out.println(count + " total");
Câu 13: Đầu ra sẽ là gì từ mã trong bài tập trước nếu các lệnh hasNextLine và nextLine được thay thế bằng hasNext và next tương ứng?
Câu 14: Đầu ra sẽ là gì từ mã trong bài tập trước nếu các lệnh hasNextLine và nextLine được thay thế bằng hasNextInt và nextInt tương ứng? Còn với hasNextDouble và nextDouble thì sao?
Câu 15: Với nội dung tệp sau đây, đầu ra sẽ là gì từ mỗi đoạn mã sau?
markdown
Copy code
the quick brown

   fox   jumps

       over

the lazy   dog
a.
java
Copy code
Scanner input = new Scanner(new File("brownfox.txt"));
while (input.hasNextLine()) {
    String line = input.nextLine();
    System.out.println(line);
}
b.
java
Copy code
Scanner input = new Scanner(new File("brownfox.txt"));

```
while (input.hasNext()) {
    String token = input.next();
    System.out.println(token);
}
```

Câu 16: Viết một chương trình tự in nội dung của nó ra màn hình. Ví dụ, nếu chương trình được lưu trong Example.java, nó sẽ mở tệp Example.java và in nội dung của tệp ra màn hình.

Câu 17: Viết mã yêu cầu người dùng nhập tên tệp và in nội dung của tệp đó ra màn hình. Giả sử rằng tệp tồn tại. Bạn có thể đặt mã này trong một phương thức gọi là printEntireFile.

Câu 18: Viết một chương trình nhận đầu vào là các dòng văn bản như sau:

swift

Copy code

```
This is some

text here.
```

Chương trình sẽ tạo đầu ra là văn bản tương tự bên trong một khung, như sau:

sql

Copy code

```
+--------------+

| This is some |

| text here.   |

+--------------+
```

Chương trình của bạn sẽ phải giả định một độ dài dòng tối đa (ví dụ, 12 trong trường hợp này).

Phần 6.4: Xử lý Tệp Nâng Cao

Câu 19: Đối tượng nào được sử dụng để ghi đầu ra vào tệp? Đối tượng này có những phương thức nào để bạn sử dụng?

Câu 20: Viết mã để in bốn dòng văn bản sau đây vào một tệp có tên là message.txt:
csharp
Copy code
Testing,

1, 2, 3.

This is my output file.
Câu 21: Viết mã liên tục yêu cầu người dùng nhập tên tệp cho đến khi người dùng nhập tên của một tệp tồn tại trên hệ thống. Bạn có thể đặt mã này trong một phương thức gọi là getFileName, phương thức này sẽ trả về tên tệp dưới dạng chuỗi.
Câu 22: Trong bài tập 16, bạn đã viết một đoạn mã yêu cầu người dùng nhập tên tệp và in nội dung của tệp đó ra màn hình. Hãy chỉnh sửa mã của bạn để nó liên tục yêu cầu người dùng nhập tên tệp cho đến khi người dùng nhập tên của một tệp tồn tại trên hệ thống.

Bài tập

1. Viết một phương thức có tên boyGirl nhận một đối tượng Scanner đang đọc dữ liệu từ tệp chứa một chuỗi tên đi kèm với các số nguyên. Các tên lần lượt là tên con trai và con gái. Phương thức này sẽ tính toán sự chênh lệch tuyệt đối giữa tổng số của các số nguyên tương ứng với con trai và con gái. Dữ liệu đầu vào có thể kết thúc với tên con trai hoặc con gái; bạn không được giả định rằng nó chứa số tên chẵn. Ví dụ, nếu tệp đầu vào chứa văn bản sau:

Copy code
Erik 3 Rita 7 Tanner 14 Jillyn 13 Curtis 4 Stefanie 12 Ben 6
Phương thức nên xuất ra kết quả như sau, vì tổng của các số nguyên tương ứng với con trai là 27 và với con gái là 32:

sql
Copy code
4 boys, 3 girls
Difference between boys' and girls' sums: 5

2. Viết một phương thức có tên evenNumbers nhận một đối tượng Scanner đang đọc dữ liệu từ tệp chứa một chuỗi các số nguyên, và báo cáo các thống kê khác nhau về các số nguyên này lên bảng điều khiển. Báo cáo tổng số các số, tổng của các số, số lượng số chẵn và phần trăm số chẵn. Ví dụ, nếu tệp đầu vào chứa văn bản sau:

Copy code
5 7 2 8 9 10 12 98 7 14 20 22
Phương thức nên xuất ra kết quả sau:
bash
Copy code
12 numbers, sum = 214
8 evens (66.67%)

3. Viết một phương thức có tên negativeSum nhận một đối tượng Scanner đang đọc dữ liệu từ tệp chứa một chuỗi các số nguyên, và in ra thông báo cho biết liệu tổng từ số đầu tiên có bao giờ là âm không. Bạn cũng nên trả về giá trị true nếu tổng âm có thể đạt được và false nếu không. Ví dụ, giả sử tệp chứa văn bản sau:

Copy code
38 4 19 -27 -15 -3 4 19 38
Phương thức sẽ xem xét tổng của chỉ một số (38), hai số đầu tiên (38+4), ba số đầu tiên (38+4+19), và cứ thế cho đến hết. Không có tổng nào trong số này là âm, do đó phương thức sẽ xuất ra kết quả sau và trả về false:
bash
Copy code
no negative sum
Nếu tệp chứa các số sau:

Copy code
14 7 -10 9 -18 -10 17 42 98
Phương thức phát hiện rằng tổng âm −8 đạt được sau khi cộng sáu số đầu tiên. Nó nên xuất ra kết quả sau và trả về true:
bash
Copy code
sum of -8 after 6 steps

4. Viết một phương thức có tên countCoins nhận một đối tượng Scanner đại diện cho một tệp đầu vào có dữ liệu là một chuỗi các cặp token, trong đó mỗi cặp bắt đầu bằng một số nguyên và tiếp theo là loại tiền xu, có thể là “pennies” (mỗi đồng 1 cent), “nickels” (mỗi đồng 5 cent), “dimes” (mỗi đồng 10 cent), hoặc “quarters” (mỗi đồng 25 cent), không phân biệt chữ hoa hay chữ thường. Cộng giá trị tiền mặt của tất cả các đồng xu và in tổng số tiền. Ví dụ, nếu tệp đầu vào chứa văn bản sau:

Copy code
3 pennies 2 quarters 1 Pennies 23 NiCkeLs 4 DIMES
Phương thức trên sẽ xuất ra kết quả sau:
bash
Copy code
Total money: $2.09

5. Viết một phương thức có tên collapseSpaces nhận một đối tượng Scanner đại diện cho một tệp đầu vào như tham số của nó, sau đó đọc tệp và xuất ra văn bản của nó với tất cả các token được phân cách bởi khoảng trắng đơn, thu gọn bất kỳ chuỗi khoảng trắng nào thành khoảng trắng đơn. Ví dụ, hãy xem xét văn bản sau:

arduino
Copy code
many spaces on this line!
Nếu văn bản này là một dòng trong tệp, cùng dòng này nên được xuất ra như sau:

arduino
Copy code
many spaces on this line!

6. Viết một phương thức có tên readEntireFile nhận một đối tượng Scanner đại diện cho một tệp đầu vào như tham số của nó, sau đó đọc tệp và trả về toàn bộ nội dung văn bản của nó dưới dạng một chuỗi.
7. Viết một phương thức có tên flipLines nhận một đối tượng Scanner đại diện cho một tệp đầu vào và in ra bảng điều khiển nội dung của cùng tệp đó với mỗi cặp dòng được đảo ngược thứ tự. Nếu tệp chứa số dòng lẻ, giữ nguyên dòng cuối cùng. Ví dụ, nếu tệp chứa:

css
Copy code
Twas brillig and the slithy toves
did gyre and gimble in the wabe.
All mimsey were the borogroves,
and the mome raths outgrabe.
Phương thức nên xuất ra kết quả sau:
css
Copy code
did gyre and gimble in the wabe.
Twas brillig and the slithy toves
and the mome raths outgrabe.
All mimsey were the borogroves,

8. Viết một phương thức có tên doubleSpace nhận một đối tượng Scanner đại diện cho một tệp đầu vào và một đối tượng PrintStream đại diện cho một tệp đầu ra làm tham số, viết vào tệp đầu ra một phiên bản có khoảng cách gấp đôi của văn bản trong tệp đầu vào. Bạn có thể thực hiện nhiệm vụ này bằng cách chèn một dòng trống giữa mỗi dòng đầu ra.

9. Viết một phương thức có tên wordWrap nhận một đối tượng Scanner đại diện cho một tệp đầu vào như tham số của nó và xuất mỗi dòng của tệp đó lên bảng điều khiển, gói dòng chữ quá dài thành các dòng có độ dài không quá 60 ký tự. Ví dụ, nếu một dòng chứa 112 ký tự, phương thức nên thay thế nó bằng hai dòng: một dòng chứa 60 ký tự đầu tiên và dòng khác chứa 52 ký tự cuối cùng. Một dòng chứa 217 ký tự nên được gói thành bốn dòng: ba dòng dài 60 ký tự và một dòng cuối dài 37 ký tự.
10. Chỉnh sửa phương thức wordWrap trước đó để nó xuất văn bản mới đã được gói trở lại vào tệp gốc. (Hãy cẩn thận—đừng xuất vào tệp trong khi bạn đang đọc nó!) Cũng chỉnh sửa nó để sử dụng một hằng số lớp cho độ dài dòng tối đa thay vì mã hóa trực tiếp 60.
11. Chỉnh sửa phương thức wordWrap trước đó để nó chỉ gói các từ nguyên, không bao giờ cắt một từ thành hai. Giả sử rằng một từ là bất kỳ token nào được phân cách bằng khoảng trắng và tất cả các từ đều dài dưới 60 ký tự.
12. Viết một phương thức có tên stripHtmlTags nhận một đối tượng Scanner đại diện cho một tệp đầu vào chứa một trang web HTML như tham số của nó, sau đó đọc tệp và in ra văn bản của tệp đó với tất cả các thẻ HTML bị loại bỏ. Một thẻ là bất kỳ văn bản nào giữa các ký tự < và >. Ví dụ, hãy xem xét văn bản sau:

```
php
Copy code
<html>
<head>
<title>My web page</title>
</head>
<body>
<p>There are many pictures of my cat here,
as well as my <b>very cool</b> blog page,
```

which contains <font color="red">awesome
stuff about my trip to Vegas.</p>
Here’s my cat now:<img src="cat.jpg">
</body>
</html>
Nếu tệp chứa các dòng này, chương trình của bạn nên xuất ra văn bản sau:
perl
Copy code
My web page

There are many pictures of my cat here,
as well as my very cool blog page,
which contains awesome
stuff about my trip to Vegas.

Here’s my cat now:
Bạn có thể giả định rằng tệp là một tài liệu HTML định dạng tốt và không có ký tự < hoặc > nào bên trong các thẻ.

13. Viết một phương thức có tên stripComments nhận một đối tượng Scanner đại diện cho một tệp đầu vào chứa chương trình Java như tham số của nó, đọc tệp và sau đó in ra văn bản của tệp với tất cả các nhận xét bị loại bỏ. Một nhận xét là bất kỳ văn bản nào trên một dòng từ // đến cuối dòng, và bất kỳ văn bản nào giữa các ký tự /* và */. Ví dụ, hãy xem xét văn bản sau:

typescript
Copy code
import java.util.*;
/* My program
   By Kim
*/
public class Foo {

```
  public static void main(String[] args) {
    Scanner console = new Scanner(System.in); // Scanner
    process(console); /* this
    calls the process
    method */
  }
}
```

Nếu tệp chứa các dòng này, chương trình của bạn nên xuất ra văn bản sau:

typescript
Copy code

```
import java.util.*;

public class Foo {
  public static void main(String[] args) {
    Scanner console = new Scanner(System.in);
    process(console);
  }
}
```

14. Viết một phương thức có tên numberLines nhận một đối tượng Scanner đại diện cho một tệp đầu vào và một đối tượng PrintStream đại diện cho một tệp đầu ra làm tham số, và viết vào tệp đầu ra nội dung văn bản của tệp đầu vào với các số dòng, theo định dạng tương tự như kết quả từ cat -n trên Linux. Ví dụ, nếu tệp đầu vào chứa văn bản sau:

perl
Copy code

```
There are many pictures of my cat here,
as well as my very cool blog page.
Here's my cat now:
```

Phương thức của bạn nên xuất ra kết quả sau:

markdown
Copy code

1. There are many pictures of my cat here,
2. as well as my very cool blog page.
3. Here’s my cat now:

15. Viết một phương thức có tên là coinFlip nhận một đối tượng Scanner đại diện cho một tệp đầu vào chứa các kết quả tung đồng xu là mặt ngửa (H) hoặc mặt sấp (T). Xem mỗi dòng là một tập hợp các kết quả tung đồng xu riêng biệt và xuất ra số lượng và tỷ lệ phần trăm của mặt ngửa trong dòng đó. Nếu tỷ lệ này lớn hơn 50%, in ra “You win!”. Xem xét tệp sau đây:

r
Copy code
H T H H T
T t t T h H
Đối với đầu vào trên, phương thức của bạn nên tạo ra đầu ra sau:
scss
Copy code
3 heads (60.0%)
You win!

2 heads (33.3%)

16. Viết một phương thức có tên là mostCommonNames nhận một đối tượng Scanner đại diện cho một tệp đầu vào chứa các tên trên mỗi dòng được phân cách bởi dấu cách. Một số tên xuất hiện nhiều lần liên tiếp trên cùng một dòng. Đối với mỗi dòng, in ra tên xuất hiện nhiều nhất. Nếu có sự hòa, sử dụng tên xuất hiện đầu tiên với số lần xuất hiện đó; nếu tất cả các tên là duy nhất, in ra tên đầu tiên trên dòng. Ví dụ, nếu tệp có đầu vào như sau:

Copy code

Benson Eric Eric Kim Kim Kim Jenny Nancy Nancy Paul Paul
Ethan Jamie Jamie Alyssa Alyssa Helene Helene Jessica Jessica
Đối với đầu vào trên, phương thức của bạn nên tạo ra đầu ra sau:
mathematica
Copy code
Most common: Kim
Most common: Jamie

17. Viết một phương thức có tên là inputStats nhận một đối tượng Scanner đại diện cho một tệp đầu vào và báo cáo số dòng, dòng dài nhất, số lượng từ trên mỗi dòng và độ dài của từ dài nhất trên mỗi dòng. Nếu tệp chứa văn bản sau:

arduino
Copy code
Beware the Jabberwock, my son,
the jaws that bite, the claws that catch,
Beware the JubJub bird and shun
the frumious bandersnatch.
Đối với đầu vào trên, phương thức của bạn nên tạo ra đầu ra sau:
scss
Copy code
Line 1 has 5 tokens (longest = 11)
Line 2 has 8 tokens (longest = 6)
Line 3 has 6 tokens (longest = 6)
Line 4 has 3 tokens (longest = 13)
Longest line: the jaws that bite, the claws that catch,

18. Viết một phương thức có tên là plusScores nhận một đối tượng Scanner đại diện cho một tệp đầu vào chứa một loạt các dòng đại diện cho hồ sơ học sinh. Mỗi hồ sơ học sinh chiếm hai dòng đầu vào. Dòng đầu tiên có tên học sinh và dòng thứ hai có một chuỗi các ký tự cộng và trừ. Dưới đây là một đầu vào mẫu:

diff
Copy code
Kane, Erica
--+-+
Chandler, Adam
++-+
Martin, Jake
+++++++
Đối với mỗi học sinh, bạn nên tạo ra một dòng đầu ra với tên học sinh, sau đó là dấu hai chấm, tiếp theo là tỷ lệ phần trăm các ký tự cộng. Đối với đầu vào trên, phương thức của bạn nên tạo ra đầu ra sau:
yaml
Copy code
Kane, Erica: 40.0% plus
Chandler, Adam: 75.0% plus
Martin, Jake: 100.0% plus

19. Viết một phương thức có tên là leetSpeak nhận hai tham số: một đối tượng Scanner đại diện cho một tệp đầu vào và một đối tượng PrintStream đại diện cho một tệp đầu ra. Chuyển đổi văn bản của tệp đầu vào thành “leet speak,” trong đó các chữ cái khác nhau được thay thế bằng các chữ cái/số khác và xuất ra văn bản mới vào tệp đầu ra đã cho. Thay thế "o" bằng "0", "l" (chữ thường "L") bằng "1" (số một), "e" bằng "3", "a" bằng "4", "t" bằng "7", và "s" ở cuối từ bằng "Z". Giữ nguyên các dòng xuống dòng từ tệp đầu vào. Đồng thời bao bọc mỗi từ đầu vào trong dấu ngoặc đơn. Ví dụ, nếu tệp đầu vào chứa văn bản sau:

csharp
Copy code
four score and
seven years ago our
fathers brought forth on this continent

a new nation
Đối với đầu vào trên, phương thức của bạn nên tạo ra đầu ra sau trong tệp đầu ra:
scss
Copy code
(f0ur) (sc0r3) (4nd)
(s3v3n) (y34rZ) (4g0) (0ur)
(f47h3rZ) (br0ugh7) (f0r7h) (0n) (7hiZ) (c0n7in3n7)
(4) (n3w) (n47i0n)

20. Viết một phương thức có tên là pigLatin nhận một tham số là một đối tượng Scanner đại diện cho một tệp đầu vào. Phương thức của bạn nên, giữ nguyên các dòng xuống dòng, in ra văn bản của tệp đầu vào dưới dạng phiên bản đơn giản của Pig Latin, một biến thể hài hước của tiếng Anh trong đó chữ cái đầu tiên của mỗi từ được chuyển đến cuối. Quy tắc của chúng ta để chuyển đổi một từ thành Pig Latin như sau:
21. Nếu từ bắt đầu bằng một nguyên âm (a, e, i, o, u), thêm "yay" vào cuối. Ví dụ, "elephant" trở thành "elephantyay".
22. Nếu từ bắt đầu bằng một phụ âm, chuyển phụ âm đó đến cuối và thêm "ay" vào cuối. Ví dụ, "welcome" trở thành "elcomeway". Bạn cũng nên chuyển tất cả các từ thành chữ thường. Ví dụ, nếu tệp đầu vào chứa văn bản sau:

vbnet
Copy code
Shall I compare thee to
a summer's day Thou art more
lovely and more temperate
Đối với đầu vào trên, phương thức của bạn nên tạo ra đầu ra sau trên console:
vbnet
Copy code
hallsay iyay omparecay heetay otay

ayay ummer'ssay ayday houtay artyay oremay
ovelylay andyay oremay emperatetay

Dự án lập trình

1. Học sinh thường được yêu cầu viết các bài luận có số lượng từ nhất định. Đếm số từ trong một bài luận dài là một công việc tẻ nhạt, nhưng máy tính có thể giúp đỡ. Viết một chương trình đếm số lượng từ, dòng và tổng số ký tự (không bao gồm khoảng trắng) trong một bài luận, giả sử rằng các từ liên tiếp được ngăn cách bằng dấu cách hoặc ký tự xuống dòng.
2. Viết một chương trình so sánh hai tệp và in ra thông tin về sự khác biệt giữa chúng. Ví dụ, hãy xem xét tệp data1.txt với nội dung sau:

css
Copy code
This file has a great deal of

text in it which needs to

be processed.
Xem xét một tệp khác data2.txt có nội dung sau:
css
Copy code
This file has a grate deal of

text in it which needs to

bee procesed.
Một cuộc đối thoại với người dùng chạy chương trình của bạn có thể trông như sau:
mathematica
Copy code

Enter a first file name:
data1.txt

Enter a second file name:
data2.txt

Differences found:

Line 1:

< This file has a great deal of

> This file has a grate deal of

Line 4:

< be processed.

> bee procesed.

3. Viết một chương trình yêu cầu người dùng nhập tên tệp, giả sử rằng tệp đó chứa một chương trình Java. Chương trình của bạn nên đọc tệp và in ra nội dung của nó được căn chỉnh đúng cách. Khi bạn thấy ký tự dấu ngoặc nhọn mở { trong tệp, tăng mức độ căn chỉnh của bạn lên bốn khoảng trống. Khi bạn thấy ký tự dấu ngoặc nhọn đóng }, giảm mức độ căn chỉnh của bạn đi bốn khoảng trống. Bạn có thể giả định rằng tệp chỉ có một dấu ngoặc mở hoặc đóng trên mỗi dòng, rằng mỗi câu lệnh khối (chẳng hạn như if hoặc for) sử dụng dấu ngoặc thay vì bỏ qua chúng, và rằng mỗi sự xuất hiện liên quan của ký tự { hoặc } trong tệp xuất hiện ở cuối một dòng. Hãy xem xét sử dụng một hằng số lớp cho số khoảng trống cần căn chỉnh (4), để nó có thể dễ dàng thay đổi sau này.

4. Viết một chương trình đọc một tệp chứa dữ liệu về sự thay đổi mức độ phổ biến của các tên trẻ em theo thời gian và hiển thị dữ liệu về một tên cụ thể. Mỗi dòng của tệp lưu trữ một tên theo sau là các số nguyên đại diện cho mức độ phổ biến của tên đó trong mỗi thập kỷ: 1900, 1910, 1920, v.v. Các xếp hạng dao động từ 1 (phổ biến nhất) đến 1000 (ít phổ biến nhất), hoặc 0 đối với một tên ít phổ biến hơn tên thứ 1000. Các dòng sau là một mẫu của định dạng tệp:

Copy code
Sally 0 0 0 0 0 0 0 0 0 0 886

Sam 58 69 99 131 168 236 278 380 467 408 466

Samantha 0 0 0 0 0 0 272 107 26 5 7

Samir 0 0 0 0 0 0 0 0 920 0 798
Chương trình của bạn nên yêu cầu người dùng nhập một tên và tìm kiếm tên đó trong tệp:
css
Copy code
This program allows you to search through the

data from the Social Security Administration

to see how popular a particular name has been

since 1900.

Name?
Sam
Nếu tên được tìm thấy, chương trình nên hiển thị dữ liệu về tên đó trên màn hình:
markdown

Copy code
Statistics on name "Sam"

1900: 58

1910: 69

1920: 99

1930: 131

...

Chương trình này thú vị và thách thức hơn nếu bạn cũng vẽ độ phổ biến của tên trên DrawingPanel dưới dạng biểu đồ đường. Vẽ các thập kỷ trên trục x và độ phổ biến trên trục y.

5. Viết một chương trình chơi một trò chơi mà người chơi được yêu cầu điền vào các từ khác nhau của một câu chuyện gần như hoàn chỉnh mà không thể thấy phần còn lại. Sau đó, người dùng sẽ được hiển thị câu chuyện của họ, thường rất hài hước. Đầu vào cho chương trình của bạn là một tập hợp các tệp câu chuyện, mỗi tệp chứa các "chỗ trống" được bao quanh bởi < và >, chẳng hạn như:

php
Copy code
One of the most <adjective> characters in fiction is named

"Tarzan of the <plural-noun> ." Tarzan was raised by a/an

<noun> and lives in the <adjective> jungle in the

heart of darkest <place> .

Người dùng sẽ được yêu cầu điền vào mỗi chỗ trống trong câu chuyện, sau đó một tệp đầu ra được tạo ra với các chỗ trống đã được điền vào. Ví dụ:
yaml
Copy code
Input file name?
story1.txt

Please enter an adjective:
silly

Please enter a plural noun:
socks

Please enter a noun:
tree

Please enter an adjective:
tiny

Please enter a place:
Canada
Câu chuyện đầu ra sẽ là:
csharp
Copy code
One of the most silly characters in fiction is named

"Tarzan of the socks ." Tarzan was raised by a/an

tree and lives in the tiny jungle in the

heart of darkest Canada .

Chương 7: Mảng

7.1 Kiến thức cơ bản về mảng

- Tạo và duyệt qua một mảng
- Truy cập mảng
- Khởi tạo mảng
- Một chương trình hoàn chỉnh sử dụng mảng
- Truy cập ngẫu nhiên
- Mảng và phương thức
- Vòng lặp For-Each
- Lớp Arrays

7.2 Thuật toán duyệt mảng

- In một mảng
- Tìm kiếm và thay thế
- Kiểm tra sự bằng nhau
- Đảo ngược một mảng
- Thuật toán duyệt chuỗi
- Cách tiếp cận hàm

7.3 Ngữ nghĩa tham chiếu

- Nhiều đối tượng

7.4 Kỹ thuật nâng cao với mảng

- Dịch chuyển giá trị trong mảng
- Mảng của đối tượng
- Tham số dòng lệnh
- Thuật toán lồng nhau

7.5 Mảng nhiều chiều

- Mảng hai chiều hình chữ nhật
- Mảng lỡ cỡ

7.6 Mảng các điểm ảnh

7.7 Nghiên cứu tình huống: Luật Benford

- Thống kê các giá trị
- Hoàn thành chương trình

Giới thiệu

Tính tuần tự của các tệp dữ liệu hạn chế số lượng các thao tác thú vị mà bạn có thể thực hiện một cách dễ dàng. Các thuật toán mà chúng ta đã xem xét cho đến nay đều là các thuật toán tuần tự: những thuật toán có thể được thực hiện bằng cách duyệt qua từng mục dữ liệu một lần theo thứ tự. Một lớp thuật toán hoàn toàn khác có thể được thực hiện khi bạn có thể truy cập các mục dữ liệu nhiều lần và theo thứ tự bất kỳ.
Chương này xem xét một đối tượng mới gọi là mảng, cung cấp loại truy cập linh hoạt hơn này. Khái niệm về mảng không phức tạp, nhưng có thể mất một thời gian để người mới học có thể nắm vững tất cả các cách khác nhau mà mảng có thể được sử dụng. Chương bắt đầu với phần thảo luận chung về mảng, sau đó chuyển sang thảo luận về các thao tác phổ biến với mảng cũng như các kỹ thuật nâng cao về mảng. Chương cũng bao gồm thảo luận về các quy tắc đặc biệt được gọi là ngữ nghĩa tham chiếu chỉ áp dụng cho các đối tượng như mảng và chuỗi.

7.1 Kiến thức cơ bản về mảng

Mảng là một cấu trúc linh hoạt để lưu trữ một dãy các giá trị cùng loại.

Mảng

Một cấu trúc chỉ mục lưu trữ nhiều giá trị cùng loại. Các giá trị được lưu trữ trong mảng được gọi là phần tử. Các phần tử riêng lẻ được truy cập bằng cách sử dụng một chỉ mục nguyên.

Chỉ mục

Một số nguyên chỉ ra vị trí của một giá trị cụ thể trong một cấu trúc dữ liệu. Ví dụ, hãy xem xét hộp thư bưu điện. Các hộp được đánh số chỉ mục, vì vậy bạn có thể tham chiếu đến một hộp cụ thể bằng cách sử dụng mô tả như “P.O. Box 884.” Bạn đã có kinh nghiệm sử dụng chỉ mục để chỉ ra vị trí trong một chuỗi; hãy nhớ các phương thức charAt và substring. Giống như chỉ mục chuỗi, chỉ mục mảng bắt đầu từ 0. Đây là quy ước được gọi là chỉ mục bắt đầu từ 0.

Chỉ mục bắt đầu từ 0

Một cách đánh số được sử dụng trong Java, trong đó một dãy các giá trị được đánh số bắt đầu từ 0 (phần tử 0, phần tử 1, phần tử 2, v.v.). Có thể thấy rằng việc bắt đầu từ 1 sẽ tự nhiên hơn, nhưng Java sử dụng cùng một sơ đồ đánh số được sử dụng trong C và C++.

Tạo và duyệt qua một mảng

Giả sử bạn muốn lưu trữ một số đọc nhiệt độ khác nhau. Bạn có thể giữ chúng trong một loạt biến:

java

Copy code

```
double temperature1;
double temperature2;
double temperature3;
```

Đây không phải là một giải pháp tồi nếu bạn chỉ có 3 nhiệt độ, nhưng giả sử bạn cần lưu trữ 3000 nhiệt độ. Khi đó, bạn sẽ muốn một cách linh hoạt hơn để lưu trữ các giá trị. Bạn có thể lưu trữ các nhiệt độ trong một mảng.

Khi bạn sử dụng một mảng, trước tiên bạn cần khai báo một biến cho nó, vì vậy bạn phải biết loại dữ liệu nào để sử dụng. Loại dữ liệu sẽ phụ thuộc vào loại phần tử bạn muốn có trong mảng của mình. Để chỉ ra rằng bạn đang tạo một mảng, hãy thêm một cặp dấu ngoặc vuông [] sau tên loại dữ liệu. Nếu bạn đang lưu trữ các giá trị nhiệt độ, bạn cần một dãy các giá trị kiểu double, vì vậy bạn sử dụng loại double[]. Do đó, bạn có thể khai báo một biến để lưu trữ mảng của mình như sau:

java

Copy code

```
double[] temperature;
```

Mảng là đối tượng, có nghĩa là chúng phải được tạo. Chỉ khai báo một biến là không đủ để tạo đối tượng. Trong trường hợp này, bạn muốn một mảng gồm ba giá trị kiểu double, mà bạn có thể tạo như sau:

java
Copy code
```
double[] temperature = new double[3];
```
Đây là cú pháp hơi khác biệt mà bạn đã sử dụng trước đây để tạo một đối tượng mới. Nó là cú pháp đặc biệt chỉ dành cho mảng. Lưu ý rằng ở phía bên trái, bạn không đặt bất cứ thứ gì vào trong dấu ngoặc vuông, bởi vì bạn đang mô tả một loại. Biến temperature có thể tham chiếu đến bất kỳ mảng giá trị kiểu double nào, bất kể có bao nhiêu phần tử. Tuy nhiên, ở phía bên phải, bạn phải đề cập đến số phần tử cụ thể vì bạn đang yêu cầu Java tạo một đối tượng mảng thực sự và nó cần biết bao nhiêu phần tử để bao gồm.
Cú pháp chung để khai báo và tạo một mảng là:
java
Copy code
```
<element type>[] <name> = new <element type>[<length>];
```
Bạn có thể sử dụng bất kỳ loại nào làm loại phần tử, mặc dù phía bên trái và bên phải của câu lệnh này phải khớp nhau. Ví dụ, bất kỳ dòng mã nào sau đây đều là cách hợp pháp để tạo một mảng:
java
Copy code
```
int[] numbers = new int[10];       // một mảng gồm 10 số nguyên
char[] letters = new char[20];     // một mảng gồm 20 ký tự
boolean[] flags = new boolean[5];  // một mảng gồm 5 giá trị boolean
String[] names = new String[100];  // một mảng gồm 100 chuỗi
Color[] colors = new Color[50];    // một mảng gồm 50 màu
```
Một số quy tắc đặc biệt áp dụng khi bạn tạo một mảng đối tượng như mảng các chuỗi hoặc mảng các màu sắc, nhưng chúng ta sẽ thảo luận những quy tắc đó sau trong chương này.
Khi thực thi dòng mã để tạo mảng nhiệt độ, Java sẽ tạo một mảng gồm ba giá trị kiểu double, và biến temperature sẽ tham chiếu đến mảng:

java
Copy code
```
double[] temperature = new double[3];
```

7.1-1 Toàn bộ văn bản thay thế

Như bạn có thể thấy, biến temperature không phải là chính mảng mà là lưu trữ tham chiếu đến mảng. Các chỉ mục mảng được biểu thị trong dấu ngoặc vuông. Để tham chiếu đến một phần tử cụ thể của mảng, bạn kết hợp tên biến tham chiếu đến mảng (temperature) với một chỉ mục cụ thể ([0], [1], hoặc [2]). Vì vậy, có một phần tử được gọi là temperature[0], một phần tử được gọi là temperature[1], và một phần tử được gọi là temperature[2].

Trong sơ đồ mảng temperature, mỗi phần tử mảng đều có giá trị 0.0. Đây là kết quả đảm bảo khi một mảng được tạo ra. Mỗi phần tử được khởi tạo với một giá trị mặc định, quy trình này được gọi là tự động khởi tạo.

Tự động khởi tạo

Quá trình khởi tạo các biến với một giá trị mặc định, chẳng hạn như trên các phần tử của một mảng khi nó được tạo ra.

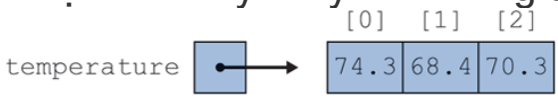
Khi Java thực hiện tự động khởi tạo, nó luôn khởi tạo với giá trị tương đương với số 0 cho loại đó. Bảng 7.1 chỉ ra các giá trị tương đương với số 0 cho các loại khác nhau. Giá trị đặc biệt null sẽ được giải thích sau trong chương này.

Bảng 7.1 Các giá trị tự động khởi tạo tương đương với số 0

Loại	Giá trị
int	0
double	0.0
char	'\0'
boolean	false
objects	null

Lưu ý rằng giá trị tương đương với số 0 cho loại double là 0.0, đó là lý do tại sao các phần tử mảng được khởi tạo với giá trị đó. Sử

dụng các chỉ mục, bạn có thể lưu trữ các giá trị nhiệt độ cụ thể có liên quan đến vấn đề này:
java
Copy code
temperature[0] = 74.3;
temperature[1] = 68.4;
temperature[2] = 70.3;
Đoạn mã này thay đổi mảng thành các giá trị sau:



7.1-3 Toàn bộ văn bản thay thế

Rõ ràng, một mảng không đặc biệt hữu ích khi bạn chỉ có ba giá trị cần lưu trữ, nhưng bạn có thể yêu cầu một mảng lớn hơn nhiều. Ví dụ, bạn có thể yêu cầu một mảng gồm 100 giá trị nhiệt độ bằng cách viết dòng mã sau:
java
Copy code
double[] temperature = new double[100];
Đây gần như là cùng một dòng mã mà bạn đã thực thi trước đó. Biến vẫn được khai báo thuộc kiểu double[], nhưng khi tạo mảng, bạn đã yêu cầu 100 phần tử thay vì 3, điều này tạo ra một mảng lớn hơn nhiều:

7.1-4 Toàn bộ văn bản thay thế

Lưu ý rằng chỉ số cao nhất là 99 thay vì 100 vì cơ chế đánh số chỉ mục bắt đầu từ 0.
Bạn không bị giới hạn chỉ sử dụng các giá trị đơn giản bên trong dấu ngoặc vuông. Bạn có thể sử dụng bất kỳ biểu thức nguyên nào. Sự linh hoạt này cho phép bạn kết hợp mảng với các vòng lặp, điều này làm đơn giản hóa đáng kể mã mà bạn viết. Ví dụ, giả sử bạn muốn đọc một loạt nhiệt độ từ một Scanner. Bạn có thể đọc từng giá trị riêng lẻ:
java

Copy code
```
temperature[0] = input.nextDouble();
temperature[1] = input.nextDouble();
temperature[2] = input.nextDouble();
...
temperature[99] = input.nextDouble();
```
Nhưng vì điều duy nhất thay đổi từ câu lệnh này sang câu lệnh khác là chỉ số, bạn có thể nắm bắt mẫu này trong một vòng lặp for với một biến điều khiển nhận các giá trị từ 0 đến 99:
java
Copy code
```
for (int i = 0; i < 100; i++) {
    temperature[i] = input.nextDouble();
}
```
Đây là một cách rất ngắn gọn để khởi tạo tất cả các phần tử của mảng. Mã trước hoạt động khi mảng có độ dài là 100, nhưng bạn có thể thay đổi mã này để phù hợp với mảng có độ dài khác. Java cung cấp một cơ chế hữu ích để làm cho mã này trở nên tổng quát hơn. Mỗi mảng tự theo dõi độ dài của nó. Bạn đang sử dụng biến temperature để tham chiếu đến mảng của mình, nghĩa là bạn có thể yêu cầu temperature.length để biết độ dài của mảng. Bằng cách sử dụng temperature.length trong điều kiện của vòng lặp for thay vì giá trị cụ thể là 100, bạn làm cho mã của mình trở nên tổng quát hơn:
java
Copy code
```
for (int i = 0; i < temperature.length; i++) {
    temperature[i] = input.nextDouble();
}
```
Lưu ý rằng quy ước mảng khác với quy ước của String. Khi bạn làm việc với biến String s, bạn yêu cầu độ dài của String bằng cách tham chiếu đến s.length(). Khi bạn làm việc với biến mảng, bạn không cần đặt dấu ngoặc đơn sau từ “length”. Đây là một

trong những sự không nhất quán không may mà lập trình viên Java cần phải nhớ.
Mã trước cung cấp một mẫu mà bạn sẽ thấy thường xuyên với mã xử lý mảng: một vòng lặp for bắt đầu từ 0 và tiếp tục trong khi biến vòng lặp nhỏ hơn độ dài của mảng, thực hiện một hành động gì đó với phần tử [i] trong thân vòng lặp. Chương trình đi qua từng phần tử của mảng theo tuần tự, chúng ta gọi quá trình này là duyệt mảng.

Duyệt Mảng

Xử lý từng phần tử mảng theo tuần tự từ đầu đến cuối.
Mẫu này rất hữu ích nên đáng để bao gồm nó dưới dạng tổng quát hơn:

java
Copy code

```
for (int i = 0; i < <array>.length; i++) {
    <do something with array[i]>;
}
```

Chúng ta sẽ thấy mẫu duyệt này lặp đi lặp lại khi khám phá các thuật toán mảng phổ biến.

Truy cập một mảng

Như đã thảo luận ở phần trước, chúng ta tham chiếu đến các phần tử mảng bằng cách kết hợp tên của biến tham chiếu đến mảng với một chỉ số nguyên bên trong dấu ngoặc vuông:

java
Copy code

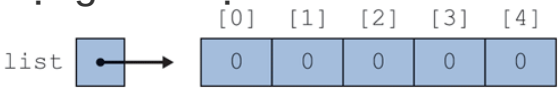
```
<array variable>[<integer expression>]
```

Lưu ý trong mô tả cú pháp này rằng chỉ số có thể là một biểu thức nguyên bất kỳ. Để khám phá tính năng này, hãy xem xét cách chúng ta sẽ truy cập các giá trị cụ thể trong một mảng số nguyên.
Giả sử rằng chúng ta tạo một mảng có độ dài 5 và điền vào đó năm số lẻ đầu tiên:

java
Copy code

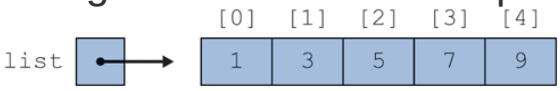
```
int[] list = new int[5];
for (int i = 0; i < list.length; i++) {
    list[i] = 2 * i + 1;
}
```

Dòng mã đầu tiên khai báo một biến list thuộc kiểu int[] tham chiếu đến một mảng có độ dài 5. Các phần tử của mảng được tự động khởi tạo về 0:



7.1-5 Toàn bộ văn bản thay thế

Sau đó, đoạn mã sử dụng vòng lặp duyệt chuẩn để điền vào mảng với các số lẻ liên tiếp:



7.1-6 Toàn bộ văn bản thay thế

Giả sử chúng ta muốn báo cáo các giá trị đầu tiên, giữa, và cuối trong danh sách. Từ việc xem xét sơ đồ trước đó, chúng ta có thể thấy rằng các giá trị này xuất hiện ở các chỉ mục 0, 2 và 4, có nghĩa là chúng ta có thể viết mã sau: // chỉ hoạt động cho một mảng có độ dài 5

java

Copy code

```
System.out.println("first = " + list[0]);
System.out.println("middle = " + list[2]);
System.out.println("last = " + list[4]);
```

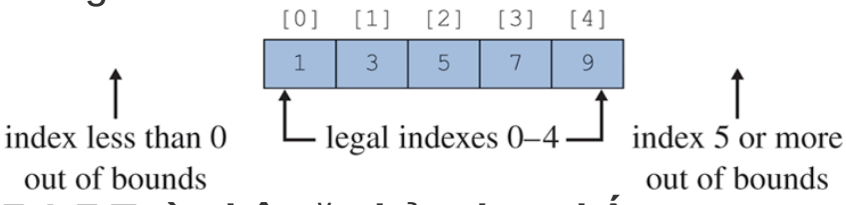
Kỹ thuật này hoạt động khi mảng có độ dài 5, nhưng giả sử rằng chúng ta sử dụng một mảng có độ dài khác? Nếu mảng có độ dài 10, ví dụ, đoạn mã này sẽ báo cáo các giá trị sai. Chúng ta cần chỉnh sửa nó để kết hợp với list.length, giống như chúng ta đã chỉnh sửa vòng lặp duyệt chuẩn.

Phần tử đầu tiên của mảng luôn ở chỉ mục 0, do đó dòng mã đầu tiên không cần thay đổi. Bạn có thể nghĩ rằng chúng ta có thể sửa dòng mã thứ ba bằng cách thay thế số 4 bằng list.length: // không hoạt động

java
Copy code
System.out.println("last = " + list[list.length]);
Tuy nhiên, mã này không hoạt động. Nguyên nhân là do đánh số chỉ mục bắt đầu từ 0. Trong ví dụ của chúng ta, giá trị cuối cùng được lưu trữ tại chỉ mục 4, không phải chỉ mục 5, khi list.length là 5. Nói chung, giá trị cuối cùng sẽ ở chỉ mục list.length – 1. Chúng ta có thể sử dụng biểu thức này trực tiếp trong câu lệnh println của chúng ta: // cái này hoạt động
java
Copy code
System.out.println("last = " + list[list.length – 1]);
Lưu ý rằng những gì xuất hiện bên trong dấu ngoặc vuông là một biểu thức nguyên (kết quả của việc trừ 1 từ list.length).
Một cách đơn giản để tìm giá trị giữa là chia độ dài của danh sách cho 2: // điều này đúng không?
java
Copy code
System.out.println("middle = " + list[list.length / 2]);
Khi list.length là 5, biểu thức này đánh giá thành 2, in ra giá trị chính xác. Nhưng nếu list.length là 10 thì sao? Trong trường hợp đó, biểu thức này đánh giá thành 5, và chúng ta sẽ in ra list[5]. Nhưng khi danh sách có độ dài chẵn, thực sự có hai giá trị ở giữa. Đối với một danh sách có độ dài 10, hai giá trị đó ở list[4] và list[5]. Nói chung, biểu thức trước đó luôn trả về giá trị thứ hai trong hai giá trị ở giữa khi danh sách có độ dài chẵn.
Nếu chúng ta muốn mã trả về giá trị đầu tiên trong hai giá trị ở giữa thay vào đó, chúng ta có thể trừ 1 từ độ dài trước khi chia nó cho 2. Đây là một tập hợp đầy đủ các câu lệnh println theo phương pháp này:
java
Copy code
System.out.println("first = " + list[0]);

System.out.println("middle = " + list[(list.length – 1) / 2]);
System.out.println("last = " + list[list.length – 1]);
Khi bạn học cách sử dụng mảng, bạn sẽ tự hỏi mình những loại thao tác nào bạn có thể thực hiện trên một phần tử mảng mà bạn đang truy cập. Ví dụ, đối với mảng số nguyên có tên list, bạn có thể làm gì với list[i]? Câu trả lời là bạn có thể làm bất cứ điều gì với list[i] mà bạn thường làm với bất kỳ biến nào thuộc kiểu int. Ví dụ, nếu bạn có một biến tên là x thuộc kiểu int, bất kỳ biểu thức nào sau đây đều hợp lệ:
java
Copy code
```
x = 3;
x++;
x *= 2;
x--;
```
Điều đó có nghĩa là các biểu thức tương tự đều hợp lệ đối với list[i] nếu list là một mảng số nguyên:
java
Copy code
```
list[i] = 3;
list[i]++;
list[i] *= 2;
list[i]--;
```
Từ góc độ của Java, vì list được khai báo thuộc kiểu int[], một phần tử mảng như list[i] là thuộc kiểu int và có thể được thao tác như vậy. Ví dụ, để tăng giá trị của mỗi phần tử trong mảng, bạn có thể sử dụng vòng lặp duyệt chuẩn:
java
Copy code
```
for (int i = 0; i < list.length; i++) {
    list[i]++;
}
```

Đoạn mã này sẽ tăng giá trị của mỗi phần tử trong mảng, biến mảng các số lẻ thành một mảng các số chẵn.
Có thể tham chiếu đến một chỉ mục không hợp lệ của mảng, trong trường hợp này Java sẽ ném ra một ngoại lệ. Ví dụ, đối với một mảng có độ dài 5, các chỉ mục hợp lệ là từ 0 đến 4. Bất kỳ số nào nhỏ hơn 0 hoặc lớn hơn 4 đều nằm ngoài giới hạn của mảng:



7.1-7 Toàn bộ văn bản thay thế
Khi bạn đang làm việc với mảng mẫu này, nếu bạn cố gắng tham chiếu đến list[-1] hoặc list[5], bạn đang cố gắng truy cập vào một phần tử mảng không tồn tại. Nếu mã của bạn thực hiện tham chiếu bất hợp pháp như vậy, Java sẽ dừng chương trình của bạn với ArrayIndexOutOfBoundsException.
Khởi tạo Mảng
Java có một cú pháp đặc biệt để khởi tạo một mảng khi bạn biết chính xác những gì bạn muốn đặt vào đó. Ví dụ, bạn có thể viết mã sau để khởi tạo một mảng số nguyên để theo dõi số ngày trong mỗi tháng ("Ba mươi ngày có tháng Chín...") và một mảng chuỗi để theo dõi các từ viết tắt cho các ngày trong tuần:
java
Copy code

```
int[] daysIn = new int[12];
daysIn[0] = 31;
daysIn[1] = 28;
daysIn[2] = 31;
daysIn[3] = 30;
daysIn[4] = 31;
daysIn[5] = 30;
daysIn[6] = 31;
daysIn[7] = 31;
```

```
daysIn[8] = 30;
daysIn[9] = 31;
daysIn[10] = 30;
daysIn[11] = 31;

String[] dayNames = new String[7];
dayNames[0] = "Mon";
dayNames[1] = "Tue";
dayNames[2] = "Wed";
dayNames[3] = "Thu";
dayNames[4] = "Fri";
dayNames[5] = "Sat";
dayNames[6] = "Sun";
```

Đoạn mã này hoạt động, nhưng nó là một cách khai báo khá tẻ nhạt cho các mảng này. Java cung cấp một cú pháp ngắn gọn hơn:

java

Copy code

```
int[] daysIn = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
String[] dayNames = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
```

Cú pháp chung cho việc khởi tạo mảng như sau:

java

Copy code

```
<element type>[] <name> = {<value>, <value>, ..., <value>};
```

Bạn sử dụng dấu ngoặc nhọn để bao bọc một chuỗi các giá trị sẽ được lưu trữ trong mảng. Thứ tự của các giá trị là quan trọng. Giá trị đầu tiên sẽ vào chỉ mục 0, giá trị thứ hai sẽ vào chỉ mục 1, và cứ tiếp tục như vậy. Java đếm bao nhiêu giá trị bạn đưa vào và xây dựng một mảng có kích thước vừa đủ. Sau đó, nó lưu các giá trị khác nhau vào các vị trí thích hợp trong mảng.

Đây là một trong hai ví dụ duy nhất mà chúng ta đã thấy trong đó Java sẽ xây dựng một đối tượng mà không cần từ khóa new. Nơi

khác mà chúng ta đã thấy điều này là với các chuỗi ký tự, trong đó Java xây dựng các đối tượng String mà không cần bạn phải gọi new. Cả hai kỹ thuật này đều là tiện ích cho lập trình viên. Những tác vụ này phổ biến đến mức các nhà thiết kế ngôn ngữ muốn làm cho chúng dễ dàng thực hiện.
Khai báo và thao tác với mảng trong JShell là một cách tốt để thử nghiệm cú pháp và tìm hiểu về cách mảng hoạt động. Cú pháp khởi tạo mảng ngắn gọn làm cho việc tạo và kiểm tra một mảng chứa một tập hợp các phần tử cho trước trở nên dễ dàng hơn.
java
Copy code

```
jshell> int[] list = {1, 3, 5, 7, 9};
list ==> int[5] {1, 3, 5, 7, 9}

jshell> list[0]
$2 ==> 1

jshell> list[4]
$3 ==> 9

jshell> list.length
$4 ==> 5

jshell> list[list.length - 1]
$5 ==> 9

jshell> list[list.length / 2]
$6 ==> 5

jshell> list[5]
|  java.lang.ArrayIndexOutOfBoundsException thrown: 5
|        at (#7:1)
```

Một Chương Trình Hoàn Chỉnh về Mảng

Hãy xem một chương trình trong đó một mảng cho phép bạn giải quyết một vấn đề mà trước đây bạn không thể giải quyết được. Nếu bạn theo dõi bất kỳ bản tin địa phương nào vào ban đêm, bạn sẽ nghe họ báo cáo nhiệt độ cao nhất trong ngày. Nó thường được báo cáo dưới dạng số nguyên, chẳng hạn như: "Nhiệt độ hôm nay lên tới 78."
Giả sử bạn muốn xem xét một loạt các nhiệt độ cao hàng ngày, tính nhiệt độ cao trung bình và đếm số ngày vượt quá nhiệt độ trung bình đó. Bạn đã sử dụng Scanner để giải quyết các vấn đề như thế này, và bạn gần như có thể giải quyết vấn đề đó theo cách này. Nếu bạn chỉ muốn biết nhiệt độ trung bình, bạn có thể sử dụng Scanner và viết một vòng lặp tổng tích lũy để tìm nó:
java
Copy code

```
1    // Đọc một loạt nhiệt độ cao và báo cáo nhiệt độ trung bình.

2

3    import java.util.*;

4

5    public class Temperature1 {

6        public static void main(String[] args) {

7            Scanner console = new Scanner(System.in);

8            System.out.print("Số ngày cần nhập nhiệt độ? ");

9            int numDays = console.nextInt();

10           int sum = 0;
```

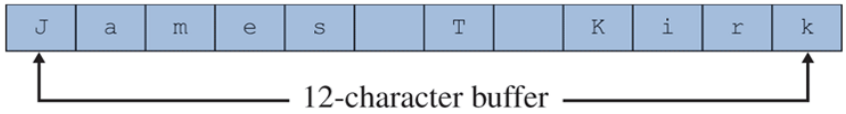
```
11          for (int i = 1; i <= numDays; i++) {

12              System.out.print("Nhiệt độ cao nhất của ngày " + i + ":
");

13              int next = console.nextInt();

14              sum += next;

15          }

16          double average = (double) sum / numDays;

17          System.out.println();

18          System.out.println("Nhiệt độ trung bình = " + average);

19      }

20  }
```

BẠN CÓ BIẾT?

Tràn Bộ Đệm

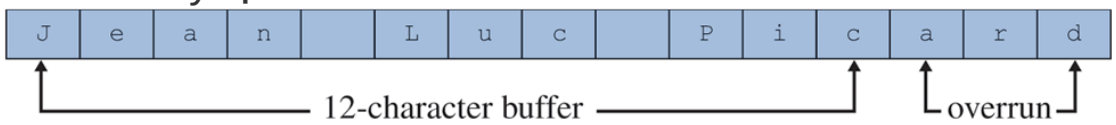
Một trong những nguyên nhân sớm nhất và vẫn phổ biến nhất của các vấn đề bảo mật máy tính là tràn bộ đệm (còn được gọi là tràn ngập bộ đệm). Tràn bộ đệm tương tự như lỗi ngoại lệ chỉ mục mảng vượt quá giới hạn. Nó xảy ra khi một chương trình ghi dữ liệu vượt ra ngoài giới hạn của bộ đệm đã được dành riêng cho dữ liệu đó.

Ví dụ, bạn có thể có không gian được phân bổ cho chuỗi “James T Kirk”, dài 12 ký tự, bao gồm cả khoảng trắng:



7.1-8 Văn Bản Thay Thế Đầy Đủ

Giả sử bạn yêu cầu máy tính ghi đè lên bộ đệm này bằng chuỗi “Jean Luc Picard”. Tên của Picard có 15 ký tự, vì vậy nếu bạn ghi tất cả các ký tự đó vào bộ đệm, bạn sẽ "tràn" nó bằng cách ghi thêm ba ký tự nữa:



7.1-9 Văn Bản Thay Thế Đầy Đủ

Ba chữ cái cuối cùng trong tên của Picard ("ard") được ghi vào một phần của bộ nhớ nằm ngoài bộ đệm. Đây là một tình huống rất nguy hiểm, vì nó sẽ ghi đè lên bất kỳ dữ liệu nào đã có sẵn ở đó. Một phép so sánh là một sinh viên nào đó lấy ba tờ giấy từ bạn và xóa bất cứ điều gì bạn đã viết trên đó. Bạn có thể đã ghi thông tin hữu ích trên những tờ giấy đó, nên việc tràn bộ đệm có thể gây ra vấn đề.

Khi việc tràn bộ đệm xảy ra do nhầm lẫn, chương trình thường dừng lại với một loại lỗi nào đó. Tuy nhiên, tràn bộ đệm đặc biệt nguy hiểm khi nó được thực hiện có chủ đích bởi một chương trình độc hại. Nếu kẻ tấn công có thể xác định đúng vị trí bộ nhớ cần ghi đè, phần mềm tấn công có thể chiếm quyền điều khiển máy tính của bạn và hướng dẫn nó thực hiện những việc bạn không yêu cầu.

Ba con sâu Internet nổi tiếng nhất được xây dựng dựa trên việc tràn bộ đệm: sâu Morris năm 1988, sâu Code Red năm 2001, và sâu SQL Slammer năm 2003.

Tràn bộ đệm thường được viết dưới dạng mã mảng. Bạn có thể tự hỏi làm thế nào một chương trình độc hại như vậy có thể được viết nếu máy tính kiểm tra giới hạn khi bạn truy cập một mảng. Câu trả lời là các ngôn ngữ lập trình cũ hơn như C và C++ không kiểm tra giới hạn khi bạn truy cập một mảng. Khi Java được thiết kế vào đầu những năm 1990, nguy cơ của tràn bộ đệm đã rõ ràng và các nhà thiết kế ngôn ngữ quyết định đưa vào kiểm tra giới hạn mảng để Java an toàn hơn. Microsoft cũng bao gồm kiểm tra giới hạn tương tự khi thiết kế ngôn ngữ C# vào cuối những năm 1990.
Chương trình trước đó đã thực hiện khá tốt. Dưới đây là một ví dụ thực thi:
plaintext
Copy code
How many days' temperatures?
5

Day 1's high temp:
78

Day 2's high temp:
81

Day 3's high temp:
75

Day 4's high temp:
79

Day 5's high temp:
71

Average = 76.8

Nhưng làm thế nào để bạn đếm được số ngày có nhiệt độ cao hơn trung bình? Bạn có thể thử kết hợp một so sánh với nhiệt độ trung bình trong vòng lặp, nhưng điều đó sẽ không hiệu quả. Vấn đề là bạn không thể tính trung bình cho đến khi bạn đã xử lý tất cả dữ liệu. Điều đó có nghĩa là bạn cần phải thực hiện một lần nữa qua dữ liệu để tính số ngày cao hơn trung bình. Bạn không thể làm điều đó với một Scanner, vì Scanner không có tùy chọn "reset" để xem lại dữ liệu lần thứ hai. Bạn sẽ phải nhắc người dùng nhập lại dữ liệu nhiệt độ lần thứ hai, điều này sẽ rất ngớ ngẩn.
May mắn thay, bạn có thể giải quyết vấn đề này bằng một mảng. Khi bạn đọc các con số và tính tổng cộng dồn, bạn có thể lấp đầy một mảng lưu trữ các nhiệt độ. Sau đó, bạn có thể sử dụng mảng này để thực hiện một lần nữa qua dữ liệu.
Trong ví dụ nhiệt độ trước đó, bạn đã sử dụng một mảng các giá trị kiểu double, nhưng ở đây bạn muốn một mảng các giá trị kiểu int. Vì vậy, thay vì khai báo một biến kiểu double[], bạn khai báo một biến kiểu int[]. Bạn đang hỏi người dùng bao nhiêu ngày dữ liệu nhiệt độ cần đưa vào, vì vậy bạn có thể tạo mảng ngay sau khi đã đọc thông tin đó:
java
Copy code
```
int numDays = console.nextInt();
int[] temps = new int[numDays];
```
Dưới đây là vòng lặp cũ:
java
Copy code
```
for (int i = 1; i <= numDays; i++) {
    System.out.print("Day " + i + "'s high temp: ");
    int next = console.nextInt();
    sum += next;
}
```

Vì bạn đang sử dụng một mảng, bạn sẽ muốn thay đổi điều này thành một vòng lặp bắt đầu từ 0 để khớp với chỉ số mảng. Nhưng chỉ vì bạn đang sử dụng chỉ số bắt đầu từ 0 bên trong chương trình không có nghĩa là bạn phải làm người dùng bối rối bằng cách hỏi "Nhiệt độ cao nhất của Ngày 0 là bao nhiêu". Bạn có thể thay đổi lệnh println để hỏi cho ngày (i + 1). Hơn nữa, bạn không cần biến next nữa vì bạn sẽ lưu trữ các giá trị trong mảng. Vì vậy, mã vòng lặp trở thành:

java
Copy code

```
for (int i = 0; i < numDays; i++) {
    System.out.print("Day " + (i + 1) + "'s high temp: ");
    temps[i] = console.nextInt();
    sum += temps[i];
}
```

Lưu ý rằng bạn đang kiểm tra xem chỉ số có nhỏ hơn numDays hay không. Sau khi vòng lặp này thực hiện xong, bạn tính trung bình như đã làm trước đây. Sau đó, bạn viết một vòng lặp mới để đếm số ngày cao hơn trung bình bằng cách sử dụng vòng lặp chuẩn của chúng ta:

java
Copy code

```
int above = 0;
for (int i = 0; i < temps.length; i++) {
    if (temps[i] > average) {
        above++;
    }
}
```

Trong vòng lặp này, điều kiện kiểm tra là temps.length. Bạn cũng có thể kiểm tra xem biến có nhỏ hơn numDays hay không; bất kỳ lựa chọn nào cũng hoạt động trong chương trình này vì chúng đều phải bằng nhau.

Nếu bạn ghép các đoạn mã khác nhau này lại với nhau và bao gồm mã để báo cáo số ngày có nhiệt độ trên mức trung bình, bạn sẽ có chương trình hoàn chỉnh sau:

java

Copy code

```
1  // Đọc một loạt các nhiệt độ cao và báo cáo
2  // trung bình và số ngày cao hơn trung bình.
3
4  import java.util.*;
5
6  public class Temperature2 {
7      public static void main(String[] args) {
8          Scanner console = new Scanner(System.in);
9          System.out.print("How many days' temperatures? ");
10         int numDays = console.nextInt();
11         int[] temps = new int[numDays];
12
13         // Ghi nhận nhiệt độ và tìm trung bình
14         int sum = 0;
15         for (int i = 0; i < numDays; i++) {
16             System.out.print("Day " + (i + 1) + "'s high temp: ");
17             temps[i] = console.nextInt();
18             sum += temps[i];
19         }
20         double average = (double) sum / numDays;
21
22         // Đếm số ngày cao hơn trung bình
23         int above = 0;
24         for (int i = 0; i < temps.length; i++) {
25             if (temps[i] > average) {
26                 above++;
27             }
28         }
```

```
29
30          // Báo cáo kết quả
31          System.out.println();
32          System.out.println("Average = " + average);
33          System.out.println(above + " days above average");
34      }
35 }
```

Dưới đây là một ví dụ thực thi chương trình:

plaintext

Copy code

```
How many days' temperatures?
9

Day 1's high temp:
75

Day 2's high temp:
78

Day 3's high temp:
85

Day 4's high temp:
71

Day 5's high temp:
69

Day 6's high temp:
82

Day 7's high temp:
74
```

Day 8's high temp:
80

Day 9's high temp:
87

Average = 77.88888888888889
5 days above average

Truy cập ngẫu nhiên
Hầu hết các thuật toán chúng ta đã thấy cho đến nay đều liên quan đến truy cập tuần tự.
Truy cập tuần tự
Thao tác các giá trị theo cách tuần tự từ đầu đến cuối. Một đối tượng Scanner thường là tất cả những gì bạn cần cho một thuật toán tuần tự, vì nó cho phép bạn truy cập dữ liệu bằng cách di chuyển từ phần tử đầu tiên đến phần tử cuối cùng. Nhưng như chúng ta đã thấy, không có cách nào để đặt lại Scanner về vị trí ban đầu. Chương trình mẫu mà chúng ta vừa nghiên cứu sử dụng một mảng để cho phép truy cập lần thứ hai qua dữ liệu, nhưng ngay cả điều này cũng về cơ bản là một phương pháp truy cập tuần tự vì nó liên quan đến hai lần truy cập qua dữ liệu theo thứ tự từ đầu đến cuối.
Một mảng là một cấu trúc dữ liệu mạnh mẽ cho phép một loại truy cập linh hoạt hơn được gọi là truy cập ngẫu nhiên:
Truy cập ngẫu nhiên
Thao tác các giá trị theo bất kỳ thứ tự nào để cho phép truy cập nhanh vào mỗi giá trị. Một mảng có thể cung cấp truy cập ngẫu nhiên vì nó được phân bổ dưới dạng một khối bộ nhớ liên tục. Máy tính có thể nhanh chóng tính toán chính xác nơi một giá trị cụ thể sẽ được lưu trữ, vì nó biết kích thước của mỗi phần tử

trong bộ nhớ và biết rằng tất cả các phần tử được phân bổ ngay cạnh nhau trong mảng.
Khi bạn làm việc với các mảng, bạn có thể nhảy xung quanh trong mảng mà không cần lo lắng về thời gian nó sẽ mất. Ví dụ, giả sử bạn đã xây dựng một mảng chứa dữ liệu nhiệt độ với 10.000 phần tử và bạn muốn in ra một số phần tử cụ thể như sau:
java
Copy code
```
System.out.println("#1394 = " + temps[1394]);
System.out.println("#6793 = " + temps[6793]);
System.out.println("#72 = " + temps[72]);
```
Đoạn mã này sẽ thực thi nhanh chóng mặc dù bạn đang yêu cầu các phần tử trong mảng cách xa nhau. Cũng lưu ý rằng bạn không phải yêu cầu chúng theo thứ tự. Bạn có thể nhảy đến phần tử 1394, sau đó nhảy tới phần tử 6793, rồi lại quay về phần tử 72. Bạn có thể truy cập các phần tử trong mảng theo bất kỳ thứ tự nào mà bạn muốn, và sẽ được truy cập nhanh.
Cuối chương này, chúng ta sẽ khám phá một số thuật toán mà sẽ rất khó để triển khai nếu không có truy cập ngẫu nhiên nhanh.

LỖI LẬP TRÌNH PHỔ BIẾN

Lỗi Off-by-One

Khi bạn chuyển chương trình Temperature1 sang một chương trình sử dụng mảng, bạn đã thay đổi vòng lặp for để bắt đầu với chỉ số 0 thay vì 1. Vòng lặp for ban đầu được viết như sau:
java
Copy code
```
for (int i = 1; i <= numDays; i++) {
    System.out.print("Day " + i + "'s high temp: ");
    int next = console.nextInt();
    sum += next;
}
```

Vì bạn đã lưu trữ các giá trị vào một mảng thay vì đọc chúng vào một biến được gọi là next, bạn đã thay thế next bằng temps[i]:
java
Copy code
```
// Sai phạm vi vòng lặp
for (int i = 1; i <= numDays; i++) {
    System.out.print("Day " + i + "'s high temp: ");
    temps[i] = console.nextInt();
    sum += temps[i];
}
```
Vì mảng được lập chỉ mục bắt đầu từ 0, bạn đã thay đổi phạm vi của vòng lặp for để bắt đầu từ 0 và điều chỉnh câu lệnh in ra. Giả sử đó là những thay đổi duy nhất bạn đã thực hiện:
java
Copy code
```
// Vẫn sai phạm vi vòng lặp
for (int i = 0; i <= numDays; i++) {
    System.out.print("Day " + (i + 1) + "'s high temp: ");
    temps[i] = console.nextInt();
    sum += temps[i];
}
```
Vòng lặp này sẽ tạo ra lỗi khi bạn chạy chương trình. Vòng lặp yêu cầu dữ liệu cho một ngày thêm và sau đó ném ra một ngoại lệ. Đây là một ví dụ:
vbnet
Copy code
```
How many days' temperatures? 5
Day 1's high temp: 82
Day 2's high temp: 80
Day 3's high temp: 79
Day 4's high temp: 71
Day 5's high temp: 75
Day 6's high temp: 83
```

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: 5
    at Temperature2.main(Temperature2.java:18)
```
Vấn đề là nếu bạn sẽ bắt đầu biến for ở 0, bạn cần kiểm tra để đảm bảo rằng nó nhỏ hơn số lần lặp mà bạn muốn. Bạn đã thay đổi 1 thành 0 nhưng để lại kiểm tra <=. Kết quả là vòng lặp đang thực hiện một lần lặp thêm và cố gắng tham chiếu đến một phần tử mảng temps[5] không tồn tại.
Đây là một lỗi off-by-one cổ điển. Cách khắc phục là thay đổi phạm vi của vòng lặp để sử dụng kiểm tra nhỏ hơn một cách nghiêm ngặt:
java
Copy code
```
// Phạm vi đúng
for (int i = 0; i < numDays; i++) {
    System.out.print("Day " + (i + 1) + "'s high temp: ");
    temps[i] = console.nextInt();
    sum += temps[i];
}
```

Mảng và Phương thức
Bạn sẽ thấy rằng khi bạn truyền một mảng làm tham số cho một phương thức, phương thức đó có khả năng thay đổi nội dung của mảng. Chúng ta sẽ tìm hiểu chi tiết sau trong chương này tại sao điều này xảy ra, nhưng hiện tại, điểm quan trọng là chỉ cần hiểu rằng các phương thức có thể thay đổi nội dung của mảng được truyền vào làm tham số.
Hãy cùng khám phá một ví dụ cụ thể để hiểu rõ hơn về cách sử dụng mảng làm tham số và giá trị trả về cho một phương thức.
Trước đó trong chương, chúng ta đã thấy mã sau để tạo một mảng số lẻ và tăng giá trị của mỗi phần tử trong mảng:
java
Copy code

```
int[] list = new int[5];
for (int i = 0; i < list.length; i++) {
    list[i] = 2 * i + 1;
}
for (int i = 0; i < list.length; i++) {
    list[i]++;
}
```

Hãy xem điều gì xảy ra khi chúng ta chuyển vòng lặp tăng giá trị vào một phương thức. Nó cần phải nhận mảng làm tham số. Chúng ta sẽ đổi tên mảng thành data thay vì list để dễ dàng phân biệt với biến mảng ban đầu. Hãy nhớ rằng mảng thuộc kiểu int[], vì vậy chúng ta sẽ viết phương thức như sau:

java

Copy code

```
public static void incrementAll(int[] data) {
    for (int i = 0; i < data.length; i++) {
        data[i]++;
    }
}
```

Bạn có thể nghĩ rằng phương thức này sẽ không có bất kỳ tác dụng nào, hoặc chúng ta phải trả về mảng để ghi nhớ sự thay đổi. Nhưng khi chúng ta sử dụng mảng làm tham số, cách tiếp cận này thực sự hoạt động. Chúng ta có thể thay thế vòng lặp tăng giá trị trong mã ban đầu bằng một lệnh gọi đến phương thức của chúng ta:

java

Copy code

```
int[] list = new int[5];
for (int i = 0; i < list.length; i++) {
    list[i] = 2 * i + 1;
}
incrementAll(list);
```

Mã này sẽ cho kết quả giống như mã ban đầu. Bài học chính từ điều này là khi chúng ta truyền một mảng làm tham số cho một phương thức, phương thức đó có khả năng thay đổi nội dung của mảng. Chúng ta không cần trả về mảng để cho phép điều này xảy ra.

Để tiếp tục với ví dụ này, hãy định nghĩa một phương thức cho mã khởi tạo để điền mảng với các số lẻ. Chúng ta có thể thực hiện điều này bằng cách chuyển vòng lặp khởi tạo vào một phương thức nhận mảng làm tham số:

java
Copy code

```
public static void fillWithOdds(int[] data) {
    for (int i = 0; i < data.length; i++) {
        data[i] = 2 * i + 1;
    }
}
```

Sau đó, chúng ta sẽ thay đổi phương thức main để gọi phương thức fillWithOdds này:

java
Copy code

```
int[] list = new int[5];
fillWithOdds(list);
incrementAll(list);
```

Giống như phương thức incrementAll, phương thức này sẽ thay đổi mảng mặc dù nó không trả về mảng. Nhưng đây không phải là cách tiếp cận tốt nhất trong tình huống này. Có vẻ lạ lùng khi phương thức fillWithOdds yêu cầu bạn tạo một mảng và truyền nó làm tham số. Tại sao fillWithOdds không tự tạo mảng? Điều đó sẽ đơn giản hóa lệnh gọi đến phương thức, đặc biệt nếu chúng ta kết thúc việc gọi nó nhiều lần.

Nếu fillWithOdds sẽ tạo mảng, nó sẽ phải trả về một tham chiếu đến mảng đó. Nếu không, chỉ có phương thức mới có tham chiếu đến mảng mới được tạo. Hiện tại, phương thức fillWithOdds giả

định rằng mảng đã được tạo, đó là lý do tại sao chúng ta đã viết hai dòng mã sau trong main:
java
Copy code
```
int[] list = new int[5];
fillWithOdds(list);
```
Nếu phương thức sẽ tạo mảng, nó không cần phải được truyền vào làm tham số, nhưng nó sẽ phải được trả về bởi phương thức. Do đó, chúng ta có thể viết lại hai dòng mã này trong main thành một dòng duy nhất:
java
Copy code
```
int[] list = fillWithOdds();
```
Tuy nhiên, bây giờ chúng ta có một tên phương thức dễ gây hiểu lầm. Phương thức này không chỉ điền vào một mảng hiện có, nó đang tạo một mảng. Cũng cần lưu ý rằng chúng ta có thể làm cho phương thức linh hoạt hơn bằng cách cho nó biết kích thước của mảng cần tạo. Vì vậy, nếu chúng ta đổi tên nó và truyền kích thước làm tham số, chúng ta sẽ gọi nó như sau:
java
Copy code
```
int[] list = buildOddArray(5);
```
Chúng ta có thể viết lại phương thức fillWithOdds để nó tạo và trả về mảng:
java
Copy code
```
public static int[] buildOddArray(int size) {
    int[] data = new int[size];
    for (int i = 0; i < data.length; i++) {
        data[i] = 2 * i + 1;
    }
    return data;
}
```

Hãy chú ý đến phần khai báo của phương thức trên. Nó không còn nhận mảng làm tham số nữa, và kiểu trả về của nó là int[] thay vì void. Nó cũng kết thúc bằng một câu lệnh return trả về tham chiếu đến mảng mà nó tạo ra.
Kết hợp tất cả lại cùng với một số mã để in nội dung của mảng, chúng ta có chương trình hoàn chỉnh sau:
java
Copy code

```
// Chương trình mẫu với mảng được truyền làm tham số
public class IncrementOdds {
    public static void main(String[] args) {
        int[] list = buildOddArray(5);
        incrementAll(list);
        for (int i = 0; i < list.length; i++) {
            System.out.print(list[i] + " ");
        }
        System.out.println();
    }

    // trả về mảng có kích thước cho trước gồm các số lẻ liên tiếp
    public static int[] buildOddArray(int size) {
        int[] data = new int[size];
        for (int i = 0; i < data.length; i++) {
            data[i] = 2 * i + 1;
        }
        return data;
    }

    // tăng mỗi phần tử của mảng thêm 1
    public static void incrementAll(int[] data) {
        for (int i = 0; i < data.length; i++) {
            data[i]++;
        }
```

```
    }
}
```
Chương trình này sẽ tạo ra kết quả sau:
Copy code
2 4 6 8 10

Vòng lặp For-Each

Java có một cấu trúc vòng lặp giúp đơn giản hóa một số vòng lặp trên mảng. Nó được gọi là vòng lặp for cải tiến, hay còn gọi là vòng lặp for-each. Bạn có thể sử dụng nó bất cứ khi nào muốn kiểm tra từng giá trị trong một mảng. Ví dụ, chương trình Temperature2 có một biến mảng gọi là temps và vòng lặp sau:
java
Copy code
```
for (int i = 0; i < temps.length; i++) {
    if (temps[i] > average) {
        above++;
    }
}
```
Chúng ta có thể viết lại điều này dưới dạng vòng lặp for-each:
java
Copy code
```
for (int n : temps) {
    if (n > average) {
        above++;
    }
}
```
Vòng lặp này thường được đọc là: “Với mỗi int n trong temps...”.
Cú pháp cơ bản của vòng lặp for-each là:
java
Copy code
```
for (<kiểu dữ liệu> <tên biến> : <mảng>) {
```

```
    <câu lệnh>;
    <câu lệnh>;
    ...
    <câu lệnh>;
}
```

Không có gì đặc biệt về tên biến, miễn là bạn giữ nó nhất quán trong thân vòng lặp. Ví dụ, vòng lặp trước đó có thể được viết với biến x thay vì biến n:

java

Copy code

```
for (int x : temps) {
    if (x > average) {
        above++;
    }
}
```

Vòng lặp for-each hữu ích nhất khi bạn chỉ muốn kiểm tra từng giá trị theo thứ tự. Có nhiều tình huống mà vòng lặp for-each không phù hợp. Ví dụ, vòng lặp sau đây sẽ nhân đôi mỗi giá trị trong một mảng gọi là list:

java

Copy code

```
for (int i = 0; i < list.length; i++) {
    list[i] *= 2;
}
```

Vì vòng lặp này đang thay đổi mảng, bạn không thể thay thế nó bằng một vòng lặp for-each:

java

Copy code

```
for (int n : list) {
    n *= 2; // chỉ thay đổi n, không thay đổi các phần tử của mảng
}
```

Như chú thích chỉ ra, vòng lặp trên chỉ nhân đôi biến n mà không thay đổi các phần tử của mảng.

Trong một số trường hợp, vòng lặp for-each không phải là lựa chọn thuận tiện nhất ngay cả khi mã liên quan đến việc kiểm tra từng phần tử của mảng theo thứ tự. Hãy xem xét, ví dụ, vòng lặp sau in ra mỗi chỉ mục của mảng cùng với giá trị của mảng được ngăn cách bởi ký tự tab:
java
Copy code
```
for (int i = 0; i < data.length; i++) {
    System.out.println(i + "\t" + data[i]);
}
```
Một vòng lặp for-each có thể được sử dụng để thay thế việc truy cập mảng:
java
Copy code
```
for (int n : data) {
    System.out.println(i + "\t" + n); // không hoàn toàn hợp lệ
}
```
Tuy nhiên, vòng lặp này sẽ gây ra vấn đề. Chúng ta muốn in ra giá trị của i, nhưng chúng ta đã loại bỏ i khi chuyển đổi truy cập mảng sang vòng lặp for-each. Chúng ta sẽ phải thêm mã phụ để theo dõi giá trị của i:
java
Copy code
```
// hợp lệ nhưng cồng kềnh
int i = 0;
for (int n : data) {
    System.out.println(i + "\t" + n);
    i++;
}
```
Trong trường hợp này, vòng lặp for-each không thực sự đơn giản hóa mọi thứ, và phiên bản ban đầu có lẽ rõ ràng hơn.

Lớp Arrays

Các mảng (arrays) có một số hạn chế quan trọng mà bạn cần hiểu. Qua nhiều năm, Java đã cố gắng khắc phục những hạn chế này bằng cách cung cấp các phương thức tiện ích khác nhau trong một lớp gọi là Arrays. Lớp này cung cấp nhiều phương thức giúp làm việc với mảng dễ dàng hơn. Lớp Arrays là một phần của gói java.util, vì vậy bạn cần phải thêm khai báo import trong bất kỳ chương trình nào sử dụng nó.
Hạn chế đầu tiên bạn cần chú ý là bạn không thể thay đổi kích thước của một mảng giữa quá trình thực thi chương trình. Hãy nhớ rằng các mảng được cấp phát như một khối bộ nhớ liền kề, vì vậy không dễ dàng để máy tính mở rộng kích thước của mảng. Nếu bạn cần một mảng lớn hơn, bạn nên tạo một mảng mới và sao chép các giá trị từ mảng cũ sang mảng mới. Phương thức Arrays.copyOf cung cấp chính xác chức năng này. Ví dụ, nếu bạn có một mảng gọi là data, bạn có thể tạo một bản sao có kích thước gấp đôi bằng dòng mã sau:
java
Copy code
```
int[] newData = Arrays.copyOf(data, 2 * data.length);
```
Nếu bạn chỉ muốn sao chép một phần của mảng, có một phương thức tương tự gọi là Arrays.copyOfRange chấp nhận một mảng, chỉ số bắt đầu và chỉ số kết thúc làm tham số.
Hạn chế thứ hai là bạn không thể in một mảng bằng cách sử dụng lệnh print hoặc println đơn giản. Bạn sẽ nhận được kết quả không mong muốn khi làm như vậy. JShell có thể hiển thị các mảng một cách đẹp mắt, nhưng nói chung các mảng không biết cách tự in dưới định dạng hữu ích.
Lớp Arrays một lần nữa cung cấp giải pháp: Phương thức Arrays.toString trả về một phiên bản định dạng thuận tiện của một mảng. Hãy xem xét, ví dụ, ba dòng mã sau:
java
Copy code
```
int[] primes = {2, 3, 5, 7, 11, 13, 17, 19, 23};
```

```
System.out.println(primes);
System.out.println(Arrays.toString(primes));
```
Kết quả đầu ra như sau:
plaintext
Copy code
```
[I@fee4648
[2, 3, 5, 7, 11, 13, 17, 19, 23]
```
Lưu ý rằng dòng đầu tiên của kết quả đầu ra không hữu ích chút nào. Tuy nhiên, dòng thứ hai cho phép chúng ta xem danh sách các số nguyên tố trong mảng vì chúng ta đã gọi Arrays.toString để định dạng mảng trước khi in nó ra.
Hạn chế thứ ba là bạn không thể so sánh các mảng để kiểm tra tính bằng nhau bằng cách sử dụng phép kiểm tra == đơn giản. Chúng ta đã thấy điều này cũng đúng với chuỗi ký tự (Strings). Nếu bạn muốn biết liệu hai mảng có chứa cùng một tập hợp giá trị hay không, bạn nên gọi phương thức Arrays.equals:
java
Copy code
```
int[] data1 = {1, 1, 2, 3, 5, 8, 13, 21};
int[] data2 = {1, 1, 2, 3, 5, 8, 13, 21};

if (Arrays.equals(data1, data2)) {
    System.out.println("Chúng lưu trữ cùng một dữ liệu");
}
```
Đoạn mã này sẽ in ra thông báo rằng các mảng lưu trữ cùng một dữ liệu. Nó sẽ không làm như vậy nếu chúng ta sử dụng phép so sánh trực tiếp với ==.
Lớp Arrays cũng cung cấp các phương thức hữu ích khác, bao gồm các phương thức để sắp xếp mảng và để lấp đầy nó với một giá trị cụ thể. Bảng 7.2 chứa danh sách một số phương thức hữu ích nhất trong lớp Arrays.
Bảng 7.2 Các phương thức hữu ích của lớp Arrays

Phương thức	Mô tả
copyOf(array, newSize)	Trả về một bản sao của mảng với kích thước mới được chỉ định
copyOfRange(array, startIndex, endIndex)	Trả về một bản sao của một phần con của mảng từ startIndex (bao gồm) đến endIndex (không bao gồm)
equals(array1, array2)	Trả về true nếu các mảng chứa các phần tử giống nhau
fill(array, value)	Đặt tất cả các phần tử của mảng thành giá trị được chỉ định
sort(array)	Sắp xếp các phần tử để chúng xuất hiện theo thứ tự tăng dần (không giảm)
toString(array)	Trả về biểu diễn dưới dạng chuỗi của mảng, chẳng hạn như [3, 5, 7]

7.2 Thuật toán Duyệt Mảng

Phần trước đã trình bày hai mẫu chuẩn để thao tác với mảng. Mẫu đầu tiên là vòng lặp duyệt, sử dụng một biến kiểu int để chỉ số từng giá trị trong mảng:

java
Copy code
```
for (int i = 0; i < array.length; i++) {
    // thực hiện một hành động nào đó với array[i]
}
```
Mẫu thứ hai là vòng lặp for-each:

java
Copy code
```
for (type name : array) {
    // câu lệnh;
    // câu lệnh;
```

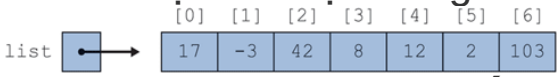
```
    ...
    // câu lệnh;
}
```

Trong phần này, chúng ta sẽ khám phá một số thuật toán phổ biến cho mảng có thể được triển khai với các mẫu này. Tất nhiên, không phải tất cả các thao tác trên mảng đều có thể được triển khai theo cách này—phần cuối sẽ có một ví dụ yêu cầu một phiên bản sửa đổi của mã chuẩn.

Chúng ta sẽ triển khai mỗi thao tác như một phương thức. Java không cho phép bạn viết mã chung cho mảng, vì vậy chúng ta phải chọn một kiểu cụ thể. Chúng ta sẽ giả định rằng bạn đang thao tác trên một mảng các giá trị kiểu int. Nếu bạn đang viết một chương trình để thao tác với loại mảng khác, bạn sẽ phải sửa đổi mã cho kiểu mà bạn đang sử dụng (ví dụ: thay đổi int[] thành double[] nếu bạn đang thao tác với một mảng các giá trị double).

In một Mảng

Giả sử bạn có một mảng các giá trị int như sau:



7.2-11 Văn Bản Thay Thế Hoàn Chỉnh

Làm thế nào để bạn in các giá trị trong mảng? Đối với các loại dữ liệu khác, bạn có thể sử dụng câu lệnh println:

java

Copy code

```
System.out.println(list);
```

Thật không may, như đã đề cập trong phần lớp Arrays của chương này, với một mảng, câu lệnh println sẽ tạo ra kết quả kỳ lạ như sau:

java

Copy code

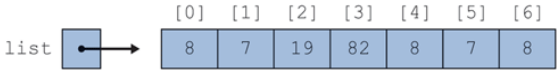
```
[I@6caf43
```

Đây không phải là đầu ra hữu ích và nó không cho chúng ta biết gì về nội dung của mảng. Chúng ta đã thấy rằng Java cung cấp một giải pháp cho vấn đề này dưới dạng một phương thức gọi là

Arrays.toString chuyển đổi mảng thành dạng văn bản tiện dụng. Bạn có thể viết lại câu lệnh println như sau để bao gồm một lời gọi đến Arrays.toString:
java
Copy code
System.out.println(Arrays.toString(list));
Dòng mã này tạo ra kết quả sau:
java
Copy code
[17, −3, 42, 8, 12, 2, 103]
Đây là một cách hợp lý để hiển thị nội dung của mảng và trong nhiều tình huống, nó sẽ là đủ. Tuy nhiên, trong những tình huống mà bạn muốn điều gì đó khác, bạn có thể viết phương thức của riêng mình.
Giả sử bạn muốn in mỗi số trên một dòng riêng. Trong trường hợp đó, bạn có thể sử dụng vòng lặp for-each để thực hiện println cho từng giá trị:
java
Copy code
```
public static void print(int[] list) {
    for (int n : list) {
        System.out.println(n);
    }
}
```
Bạn có thể gọi phương thức này với biến list:
java
Copy code
print(list);
Lời gọi này tạo ra đầu ra sau:
java
Copy code
17
−3

42
8
12
2
103
Tuy nhiên, trong một số trường hợp, vòng lặp for-each không cung cấp đúng những gì bạn muốn. Ví dụ, hãy xem cách phương thức Arrays.toString phải được viết. Nó tạo ra một danh sách các giá trị được phân tách bằng dấu phẩy, đây là một vấn đề kinh điển về việc tách dấu (ví dụ: bảy giá trị được phân tách bởi sáu dấu phẩy). Để giải quyết vấn đề này, bạn sẽ muốn sử dụng vòng lặp chỉ mục thay vì vòng lặp for-each để bạn có thể in giá trị đầu tiên trước khi vào vòng lặp:
java
Copy code
```
System.out.print(list[0]);

for (int i = 1; i < list.length; i++) {
    System.out.print(", " + list[i]);
}

System.out.println();
```
Lưu ý rằng i được khởi tạo là 1 thay vì 0 vì list[0] được in trước vòng lặp. Mã này tạo ra kết quả sau cho mảng mẫu trước đó:
java
Copy code
```
17, −3, 42, 8, 12, 2, 103
```
Tuy nhiên, ngay cả mã này cũng không hoàn toàn chính xác, vì nó giả định rằng có một list[0] để in. Có thể các mảng trống với độ dài là 0, trong trường hợp này, mã sẽ tạo ra một ArrayIndexOutOfBoundsException. Phiên bản của phương thức sau đây tạo ra đầu ra phù hợp với chuỗi do Arrays.toString tạo ra. Các câu lệnh in ngay trước và ngay sau vòng lặp đã được sửa

đổi để bao gồm dấu ngoặc vuông, và một trường hợp đặc biệt đã được thêm vào cho các mảng trống:
java
Copy code
```
public static void print(int[] list) {
    if (list.length == 0) {
        System.out.println("[]");
    } else {
        System.out.print("[" + list[0]);

        for (int i = 1; i < list.length; i++) {
            System.out.print(", " + list[i]);
        }

        System.out.println("]");
    }
}
```
Tìm kiếm và Thay thế
Thường bạn sẽ muốn tìm kiếm một giá trị cụ thể trong mảng. Ví dụ, bạn có thể muốn đếm xem một giá trị cụ thể xuất hiện bao nhiêu lần trong một mảng. Giả sử bạn có một mảng các giá trị int như sau:



7.2-12 Văn bản thay thế đầy đủ
Đếm số lần xuất hiện là một nhiệm vụ tìm kiếm đơn giản nhất, bởi vì bạn luôn kiểm tra từng giá trị trong mảng và không cần thay đổi nội dung của mảng. Bạn có thể thực hiện nhiệm vụ này bằng một vòng lặp for-each để đếm số lần xuất hiện của giá trị bạn đang tìm kiếm:
java
Copy code
```
public static int count(int[] list, int target) {
    int count = 0;
```

```
    for (int n : list) {
        if (n == target) {
            count++;
        }
    }
    return count;
}
```

Bạn có thể sử dụng phương thức này trong lời gọi sau để tìm xem có bao nhiêu số 8 trong danh sách:

java
Copy code

```
int number = count(list, 8);
```

Lời gọi này sẽ đặt giá trị number thành 3 đối với mảng mẫu, vì có ba lần xuất hiện của số 8 trong danh sách. Nếu bạn thay vào đó gọi:

java
Copy code

```
int number = count(list, 2);
```

number sẽ được đặt thành 0, bởi vì không có lần xuất hiện nào của số 2 trong danh sách.

Đôi khi, bạn muốn biết giá trị nằm ở đâu trong danh sách. Bạn có thể thực hiện nhiệm vụ này bằng cách viết một phương thức sẽ trả về chỉ mục của lần xuất hiện đầu tiên của giá trị trong danh sách. Vì bạn không biết chính xác sẽ tìm thấy giá trị ở đâu, bạn có thể thử sử dụng vòng lặp while, như trong đoạn mã giả sau:

java
Copy code

```
int i = 0;
while (we haven´t found it yet) {
    i++;
}
```

Tuy nhiên, có một cách tiếp cận đơn giản hơn. Bởi vì bạn đang viết một phương thức trả về giá trị, bạn có thể trả về chỉ mục phù

hợp ngay khi tìm thấy một kết quả trùng khớp. Điều đó có nghĩa là bạn có thể sử dụng vòng lặp duyệt chuẩn để giải quyết vấn đề này:
java
Copy code
```
public static int indexOf(int[] list, int target) {
    for (int i = 0; i < list.length; i++) {
        if (list[i] == target) {
            return i;
        }
    }
    return -1;
}
```
Hãy nhớ rằng câu lệnh return kết thúc một phương thức, vì vậy bạn sẽ thoát khỏi vòng lặp này ngay khi tìm thấy giá trị mục tiêu. Nhưng nếu giá trị không được tìm thấy thì sao? Nếu bạn duyệt toàn bộ mảng mà không tìm thấy bất kỳ kết quả trùng khớp nào, vòng lặp for sẽ hoàn thành mà không bao giờ trả về giá trị.
Có nhiều cách để xử lý khi không tìm thấy giá trị. Quy ước sử dụng trong các thư viện lớp Java là trả về giá trị -1 để chỉ ra rằng giá trị không có trong danh sách. Vì vậy, bạn có thể thêm một câu lệnh return sau vòng lặp, sẽ chỉ được thực hiện khi giá trị mục tiêu không được tìm thấy. Gộp tất cả lại với nhau, bạn có được phương thức sau:
Bạn có thể sử dụng phương thức này trong lời gọi sau để tìm lần xuất hiện đầu tiên của giá trị 7 trong danh sách:
java
Copy code
```
int position = indexOf(list, 7);
```
Lời gọi này sẽ đặt position thành 1 đối với mảng mẫu, vì lần xuất hiện đầu tiên của số 7 nằm ở chỉ mục 1. Có một lần xuất hiện khác của số 7 ở cuối mảng, tại chỉ mục 5, nhưng mã này sẽ kết

thúc ngay khi tìm thấy kết quả trùng khớp đầu tiên. Nếu bạn thay vào đó gọi:
java
Copy code
```
int position = indexOf(list, 42);
```
position sẽ được đặt thành -1 vì không có lần xuất hiện nào của số 42 trong danh sách.
Cuối cùng, hãy xem xét vấn đề thay thế tất cả các lần xuất hiện của một giá trị bằng một giá trị mới. Điều này tương tự như nhiệm vụ đếm. Bạn sẽ muốn duyệt mảng để tìm một giá trị cụ thể và thay thế giá trị đó bằng một giá trị mới khi bạn tìm thấy nó. Bạn không thể thực hiện nhiệm vụ đó với một vòng lặp for-each, bởi vì thay đổi biến vòng lặp không ảnh hưởng đến mảng. Thay vào đó, hãy sử dụng vòng lặp duyệt chuẩn:
java
Copy code
```
public static void replaceAll(int[] list, int target, int replacement) {
    for (int i = 0; i < list.length; i++) {
        if (list[i] == target) {
            list[i] = replacement;
        }
    }
}
```
Lưu ý rằng mặc dù phương thức này thay đổi nội dung của mảng, bạn không cần trả về mảng đó để thay đổi này có hiệu lực.
Như đã đề cập ở đầu phần này, các ví dụ này liên quan đến một mảng số nguyên, và bạn sẽ phải thay đổi kiểu dữ liệu nếu bạn muốn thao tác với một mảng kiểu dữ liệu khác (ví dụ, thay đổi int[] thành double[] nếu bạn có một mảng giá trị kiểu double). Nhưng sự thay đổi không đơn giản nếu bạn có một mảng các đối tượng, chẳng hạn như String. Để so sánh các giá trị String, bạn phải gọi phương thức equals thay vì sử dụng phép so sánh ==.

Đây là phiên bản đã được sửa đổi của phương thức replaceAll phù hợp cho một mảng String:
java
Copy code
```
public static void replaceAll(String[] list, String target, String replacement) {
    for (int i = 0; i < list.length; i++) {
        if (list[i].equals(target)) {
            list[i] = replacement;
        }
    }
}
```

Kiểm tra tính bằng nhau
Vì các mảng là các đối tượng, việc kiểm tra chúng có bằng nhau hay không phức tạp hơn so với kiểm tra các giá trị nguyên thủy như số nguyên và số thực. Hai mảng có giá trị tương đương nếu chúng có cùng độ dài và chứa cùng một chuỗi các giá trị. Phương thức Arrays.equals thực hiện việc kiểm tra này:
java
Copy code
```
if (Arrays.equals(list1, list2)) {
    System.out.println("The arrays are equal");
}
```
Giống như phương thức Arrays.toString, phương thức Arrays.equals thường là tất cả những gì bạn cần. Nhưng đôi khi bạn muốn có một số biến thể nhỏ, vì vậy đáng để khám phá cách tự viết phương thức này.
Phương thức sẽ nhận hai mảng làm tham số và trả về kết quả boolean cho biết liệu hai mảng có bằng nhau hay không. Vì vậy, phương thức sẽ trông như thế này:
java
Copy code

```
public static boolean equals(int[] list1, int[] list2) {
    ...
}
```
Khi bạn bắt đầu viết một phương thức như thế này, bạn có thể nghĩ đến việc định nghĩa tính bằng nhau: "Hai mảng bằng nhau nếu độ dài của chúng bằng nhau và chúng chứa cùng một chuỗi các giá trị." Nhưng đây không phải là cách tiếp cận dễ nhất. Ví dụ, bạn có thể bắt đầu bằng cách kiểm tra xem độ dài có bằng nhau không, nhưng bạn sẽ làm gì tiếp theo?

java
Copy code
```
public static boolean equals(int[] list1, int[] list2) {
    if (list1.length == list2.length) {
        // tiếp theo làm gì?
        ...
    }
    ...
}
```
Các phương thức như thế này thường dễ viết hơn nếu bạn nghĩ theo hướng điều kiện ngược lại: Điều gì sẽ làm cho hai mảng không bằng nhau? Thay vì kiểm tra xem độ dài có bằng nhau không, hãy bắt đầu bằng cách kiểm tra xem độ dài có khác nhau không. Trong trường hợp đó, bạn biết chính xác phải làm gì. Nếu độ dài không bằng nhau, phương thức nên trả về giá trị false, và bạn sẽ biết rằng hai mảng không bằng nhau:

java
Copy code
```
public static boolean equals(int[] list1, int[] list2) {
    if (list1.length != list2.length) {
        return false;
    }
    ...
}
```

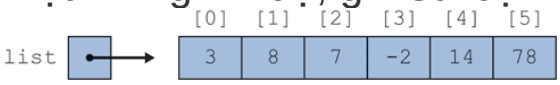
Nếu bạn vượt qua câu lệnh if, bạn sẽ biết rằng hai mảng có độ dài bằng nhau. Sau đó, bạn sẽ muốn kiểm tra xem chúng có chứa cùng một chuỗi giá trị hay không. Một lần nữa, hãy kiểm tra sự không bằng nhau thay vì sự bằng nhau, trả về false nếu có sự khác biệt:
java
Copy code
```
public static boolean equals(int[] list1, int[] list2) {
    if (list1.length != list2.length) {
        return false;
    }
    for (int i = 0; i < list1.length; i++) {
        if (list1[i] != list2[i]) {
            return false;
        }
    }
    ...
}
```
Nếu bạn vượt qua vòng lặp for, bạn sẽ biết rằng hai mảng có độ dài bằng nhau và chứa chính xác cùng một chuỗi giá trị. Trong trường hợp đó, bạn sẽ muốn trả về giá trị true để chỉ ra rằng hai mảng bằng nhau. Thêm điều này vào sẽ hoàn thành phương thức:
java
Copy code
```
public static boolean equals(int[] list1, int[] list2) {
    if (list1.length != list2.length) {
        return false;
    }
    for (int i = 0; i < list1.length; i++) {
        if (list1[i] != list2[i]) {
            return false;
        }
```

```
    }
    return true;
}
```

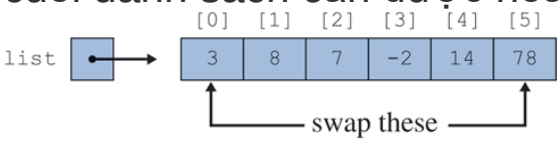
Đây là một mẫu phổ biến cho một phương thức như equals: Bạn kiểm tra tất cả các cách mà hai đối tượng có thể không bằng nhau, trả về false nếu bạn tìm thấy bất kỳ sự khác biệt nào, và trả về true ở cuối cùng để nếu tất cả các kiểm tra đều vượt qua, hai đối tượng được tuyên bố là bằng nhau.

Đảo ngược một mảng

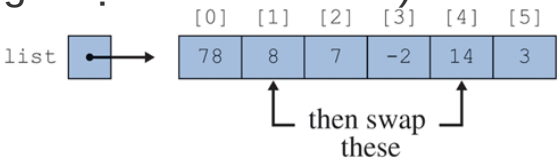
Như một ví dụ cuối cùng về các thao tác phổ biến, hãy xem xét nhiệm vụ đảo ngược thứ tự của các phần tử được lưu trữ trong một mảng. Ví dụ, giả sử bạn có một mảng lưu trữ các giá trị sau:



Một cách tiếp cận là tạo ra một mảng mới và lưu trữ các giá trị từ mảng đầu tiên vào mảng thứ hai theo thứ tự ngược lại. Mặc dù cách tiếp cận đó hợp lý, bạn có thể giải quyết vấn đề mà không cần tạo ra mảng thứ hai. Một cách khác là thực hiện một loạt các phép hoán đổi. Ví dụ, giá trị 3 ở đầu danh sách và giá trị 78 ở cuối danh sách cần được hoán đổi:



Sau khi hoán đổi cặp đó, bạn có thể hoán đổi cặp tiếp theo (các giá trị ở chỉ số 1 và 4):



Bạn có thể tiếp tục hoán đổi cho đến khi toàn bộ danh sách đã được đảo ngược. Trước khi xem xét đoạn mã thực hiện việc đảo ngược này, hãy xem xét vấn đề chung của việc hoán đổi hai giá trị.

Giả sử bạn có hai biến số nguyên x và y với giá trị là 3 và 78:

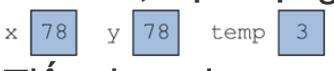
```
java
```

Copy code
int x = 3;
int y = 78;
Bạn sẽ hoán đổi các giá trị này như thế nào? Một cách tiếp cận đơn giản là chỉ cần gán các giá trị cho nhau:
java
Copy code
// sẽ không hoán đổi đúng
x = y;
y = x;
Thật không may, điều này không hoạt động. Bạn bắt đầu với điều sau:

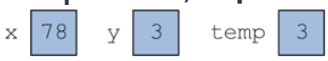
Khi câu lệnh gán đầu tiên được thực thi, bạn sao chép giá trị của y vào x:

Bạn muốn x cuối cùng sẽ bằng 78, nhưng nếu bạn cố gắng giải quyết vấn đề theo cách này, bạn sẽ mất giá trị cũ của x ngay khi bạn gán giá trị của y cho nó. Câu lệnh gán thứ hai sau đó sao chép giá trị mới của x, 78, quay lại y, điều này khiến bạn có hai biến bằng 78.
Giải pháp tiêu chuẩn là giới thiệu một biến tạm thời mà bạn có thể sử dụng để lưu trữ giá trị cũ của x trong khi bạn gán giá trị mới cho x. Sau đó, bạn có thể sao chép giá trị cũ của x từ biến tạm thời vào y để hoàn thành việc hoán đổi:
java
Copy code
int temp = x;
x = y;
y = temp;
Bạn bắt đầu bằng cách sao chép giá trị cũ của x vào temp:

Sau đó, bạn đặt giá trị của y vào x:



Tiếp theo, bạn sao chép giá trị cũ của x từ temp sang y:



Lúc này, bạn đã hoán đổi thành công giá trị của x và y, vì vậy bạn không cần temp nữa.

Trong một số ngôn ngữ lập trình, bạn có thể định nghĩa điều này như một phương thức hoán đổi có thể được sử dụng để hoán đổi hai giá trị int:

java

Copy code

```
// phương thức này sẽ không hoạt động
public static void swap(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
}
```

Như bạn đã thấy, loại phương thức này sẽ không hoạt động trong Java vì x và y được hoán đổi sẽ là bản sao của bất kỳ giá trị số nguyên nào được truyền cho chúng. Nhưng vì các mảng được lưu trữ dưới dạng các đối tượng, bạn có thể viết một phiên bản khác của phương thức này nhận một mảng và hai chỉ số làm tham số và hoán đổi các giá trị tại các chỉ số đó:

java

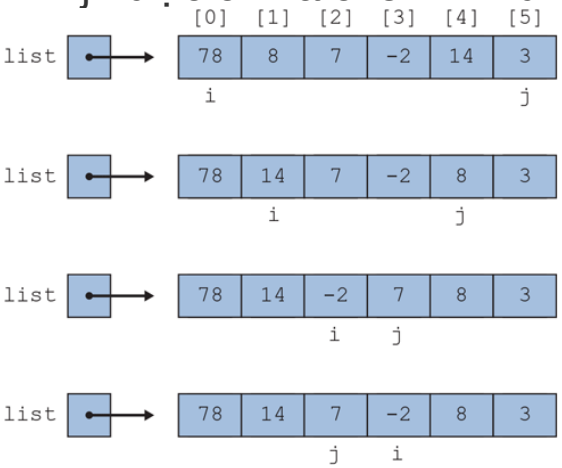
Copy code

```
public static void swap(int[] list, int i, int j) {
    int temp = list[i];
    list[i] = list[j];
    list[j] = temp;
}
```

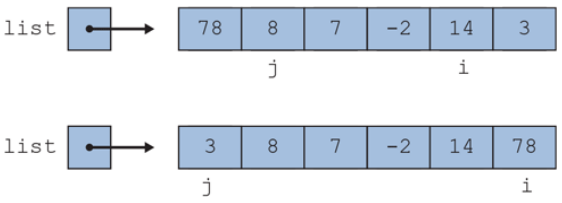
Mã trong phương thức này khớp với mã trong phương thức trước đó, nhưng thay vì sử dụng x và y, nó sử dụng list[i] và list[j].

Phương thức này sẽ hoạt động vì thay vì thay đổi các biến int đơn giản, phương thức này đang thay đổi nội dung của mảng. Với phương thức hoán đổi này, bạn có thể dễ dàng viết một phương thức đảo ngược. Bạn chỉ cần nghĩ về các kết hợp giá trị nào cần hoán đổi. Bắt đầu bằng cách hoán đổi giá trị đầu tiên và giá trị cuối cùng. Mảng mẫu có độ dài 6, nghĩa là bạn sẽ hoán đổi các giá trị tại các chỉ số 0 và 5. Nhưng bạn muốn viết mã sao cho nó hoạt động cho một mảng có độ dài bất kỳ. Nói chung, lần hoán đổi đầu tiên bạn muốn thực hiện là hoán đổi phần tử 0 với phần tử (list.length − 1):
java
Copy code
swap(list, 0, list.length − 1);
Sau đó, bạn sẽ muốn hoán đổi giá trị thứ hai với giá trị thứ hai từ cuối:
java
Copy code
swap(list, 1, list.length − 2);
Sau đó, bạn sẽ hoán đổi giá trị thứ ba với giá trị thứ ba từ cuối:
java
Copy code
swap(list, 2, list.length − 3);
Có một mẫu trong các lần hoán đổi này mà bạn có thể áp dụng bằng một vòng lặp. Nếu bạn sử dụng một biến i cho tham số đầu tiên của lệnh gọi swap và giới thiệu một biến cục bộ j để lưu trữ một biểu thức cho tham số thứ hai của swap, mỗi lệnh gọi này sẽ có dạng như sau:
java
Copy code
int j = list.length - i - 1;
swap(list, i, j);
Để thực hiện đảo ngược, bạn có thể đặt phương thức bên trong vòng lặp duyệt chuẩn:

java
Copy code
```
// không hoàn toàn hoạt động
for (int i = 0; i < list.length; i++) {
    int j = list.length - i - 1;
    swap(list, i, j);
}
```
Nếu bạn kiểm tra mã này, bạn sẽ thấy rằng nó dường như không có tác dụng gì. Danh sách lưu trữ các giá trị giống nhau sau khi thực hiện mã này như nó lưu trữ ban đầu. Vấn đề là vòng lặp này hoán đổi quá nhiều. Dưới đây là một dấu vết của sáu lần hoán đổi được thực hiện trên danh sách [3, 8, 7, −2, 14, 78], với chỉ số i và j được chỉ ra cho mỗi bước:



7.2-21 Full Alternative Text



Các giá trị của i và j sẽ giao nhau khi thực hiện nửa chừng quá trình này. Kết quả là, ba lần hoán đổi đầu tiên thành công trong việc đảo ngược mảng, và sau đó ba lần hoán đổi tiếp theo sẽ hoàn tác công việc của ba lần đầu tiên. Để khắc phục vấn đề này,

bạn cần dừng lại ở giữa quá trình. Nhiệm vụ này có thể dễ dàng thực hiện bằng cách thay đổi điều kiện kiểm tra:
java
Copy code
```
for (int i = 0; i < list.length / 2; i++) {
    int j = list.length - i - 1;
    swap(list, i, j);
}
```
Trong mảng mẫu, list.length là 6. Một nửa của nó là 3, điều này có nghĩa là vòng lặp này sẽ thực hiện đúng ba lần. Đây là điều bạn muốn trong trường hợp này (ba lần hoán đổi đầu tiên), nhưng bạn nên cẩn thận để xem xét các khả năng khác. Ví dụ, nếu list.length là 7 thì sao? Một nửa của nó cũng là 3, do phép chia làm tròn. Ba có phải là số lần hoán đổi chính xác cho một danh sách có độ dài lẻ không? Câu trả lời là có. Nếu có số lượng phần tử lẻ, giá trị ở giữa danh sách không cần phải hoán đổi. Vì vậy, trong trường hợp này, một phép chia đơn giản cho 2 lại là cách tiếp cận đúng.
Khi bao gồm đoạn mã này trong một phương thức, bạn sẽ có giải pháp tổng thể sau:
java
Copy code
```
public static void reverse(int[] list) {
    for (int i = 0; i < list.length / 2; i++) {
        int j = list.length - i - 1;
        swap(list, i, j);
    }
}
```
Thuật toán duyệt qua chuỗi:
Trong Java, chúng ta thường nghĩ về chuỗi như một đoạn văn bản, nhưng bạn cũng có thể coi nó là một dãy các ký tự riêng lẻ. Nhìn theo cách này, một chuỗi rất giống với một mảng. Hãy nhớ rằng các phần tử riêng lẻ của một chuỗi là kiểu char và bạn có

thể truy cập giá trị ký tự riêng lẻ bằng cách gọi phương thức charAt.
Các kỹ thuật mà chúng ta đã sử dụng để viết các thuật toán duyệt mảng cũng có thể được sử dụng để viết các thuật toán duyệt chuỗi. Cú pháp có hơi khác, nhưng logic là như nhau. Mẫu duyệt mảng của chúng ta trông như thế này:
java
Copy code
```
for (int i = 0; i < array.length; i++) {
    <làm gì đó với array[i]>;
}
```
Mẫu thuật toán tương ứng cho chuỗi trông như thế này:
java
Copy code
```
for (int i = 0; i < string.length(); i++) {
    <làm gì đó với string.charAt(i)>;
}
```
Lưu ý rằng với mảng, bạn tham chiếu đến length mà không cần dấu ngoặc đơn, nhưng với chuỗi, bạn cần sử dụng dấu ngoặc đơn. Cũng lưu ý rằng ký hiệu ngoặc vuông của mảng được thay thế bằng lệnh gọi phương thức charAt.
Ví dụ, bạn có thể đếm số lần xuất hiện của chữ cái "i" trong "Mississippi" bằng đoạn mã này:
java
Copy code
```
String s = "Mississippi";
int count = 0;
for (int i = 0; i < s.length(); i++) {
    if (s.charAt(i) == 'i') {
        count++;
    }
}
```

Đoạn mã này sẽ tính chính xác rằng có bốn lần xuất hiện của "i" trong chuỗi. Để lấy một ví dụ khác, hãy xem xét nhiệm vụ tính toán đảo ngược của một chuỗi. Bạn có thể duyệt qua chuỗi, xây dựng một phiên bản mới có các chữ cái theo thứ tự ngược lại bằng cách đặt mỗi ký tự mới ở đầu chuỗi mà bạn đang xây dựng. Dưới đây là một phương thức hoàn chỉnh sử dụng cách tiếp cận này:
java
Copy code
```
public static String reverse(String text) {
    String result = "";
    for (int i = 0; i < text.length(); i++) {
        result = text.charAt(i) + result;
    }
    return result;
}
```
Nếu bạn gọi reverse("Mississippi"), phương thức này sẽ trả về "ippississiM".

Cách tiếp cận hàm

Chương 19 mô tả một cách tiếp cận khác để thao tác mảng dẫn đến mã trông khá khác so với các ví dụ trong phần này. Nó dựa vào các tính năng được thêm vào ngôn ngữ lập trình Java bắt đầu từ phiên bản 8, cho phép bạn thao tác với mảng và các cấu trúc dữ liệu khác theo cách khai báo nhiều hơn. Thay vì chỉ rõ cách duyệt qua một mảng, bạn có thể nói cho Java biết bạn muốn làm gì với các phần tử của mảng và để Java tìm cách thực hiện việc duyệt qua. Việc bổ sung vòng lặp for-each bắt đầu từ phiên bản 5 của Java là một bước đi ban đầu theo hướng này, nhưng các tính năng mới tiến xa hơn nhiều.
Giả sử, ví dụ, bạn có một mảng các giá trị được định nghĩa như sau:
java

Copy code
int[] numbers = {8, 3, 2, 17};
Hãy xem mã bạn sẽ viết cho hai nhiệm vụ đơn giản: tìm tổng và in các giá trị. Sử dụng các vòng lặp duyệt tiêu chuẩn, bạn sẽ viết mã sau:
java
Copy code
// Tính tổng một mảng số và in chúng (for loop)
int sum = 0;
for (int i = 0; i < numbers.length; i++) {
    sum += numbers[i];
}
System.out.println("sum = " + sum);
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
Mã này tạo ra đầu ra như sau:
text
Copy code
sum = 30
8
3
2
17
Vòng lặp for-each đơn giản hóa mã này bằng cách chỉ rõ rằng bạn muốn thao tác với từng giá trị khác nhau trong mảng theo thứ tự, nhưng không yêu cầu bạn phải bao gồm một biến chỉ mục để chỉ rõ chính xác cách thực hiện điều đó.
java
Copy code
// Tính tổng một mảng số và in chúng (for-each loop)
int sum = 0;
for (int n : numbers) {

```
    sum += n;
}
System.out.println("sum = " + sum);
for (int n : numbers) {
    System.out.println(n);
}
```

Với các tính năng mới của Java 8, việc này trở nên đơn giản hơn nhiều. Nhiệm vụ tìm tổng của một dãy các giá trị phổ biến đến mức có một phương thức tích hợp sẵn để làm điều đó cho bạn. Và nhiệm vụ in từng giá trị bằng cách gọi phương thức println của System.out cũng có thể được diễn đạt một cách rất ngắn gọn.

java
Copy code

```
// Tính tổng một mảng số và in chúng (functional)
int sum = Arrays.stream(numbers).sum();
System.out.println("sum = " + sum);
Arrays.stream(numbers).forEach(System.out::println);
```

Mã này hoàn toàn không mô tả cách duyệt qua được thực hiện. Thay vào đó, bạn cho Java biết các thao tác bạn muốn thực hiện trên các giá trị trong mảng và để Java thực hiện việc duyệt qua. Xem Chương 19 để biết giải thích đầy đủ hơn về cách tiếp cận này.

7.3 Ngữ nghĩa Tham chiếu

Trong Java, mảng là các đối tượng. Chúng ta đã sử dụng đối tượng từ Chương 3 nhưng chưa thảo luận chi tiết về cách chúng được lưu trữ. Đã đến lúc khám phá các chi tiết này. Các đối tượng được lưu trữ trong bộ nhớ của máy tính theo cách khác so với dữ liệu nguyên thủy. Ví dụ, khi chúng ta khai báo biến số nguyên:

java
Copy code

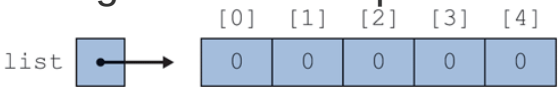
```
int x = 8;
```

biến này lưu trữ dữ liệu thực tế. Vì vậy, chúng ta đã vẽ các hình như sau:

Tình huống sẽ khác đối với mảng và các đối tượng khác. Đối với các đối tượng, biến không lưu trữ dữ liệu thực tế. Thay vào đó, dữ liệu được lưu trữ trong một đối tượng và biến lưu trữ một tham chiếu đến vị trí mà đối tượng đó được lưu trữ. Vì vậy, chúng ta có hai phần tử khác nhau trong bộ nhớ của máy tính: biến và đối tượng. Do đó, khi chúng ta tạo một đối tượng mảng như sau:
java
Copy code

```
int[] list = new int[5];
```

chúng ta sẽ có kết quả như sau:



Như sơ đồ cho thấy, hai giá trị khác nhau được lưu trữ trong bộ nhớ: mảng tự nó, xuất hiện ở phía bên phải của sơ đồ, và một biến có tên là list, lưu trữ một tham chiếu đến mảng (được biểu diễn trong hình này dưới dạng mũi tên). Chúng ta nói rằng list tham chiếu đến mảng đó.
Có thể bạn sẽ mất một thời gian để quen với hai cách tiếp cận khác nhau để lưu trữ dữ liệu, nhưng những cách tiếp cận này phổ biến đến mức các nhà khoa học máy tính đã có những thuật ngữ kỹ thuật để mô tả chúng. Hệ thống dành cho các kiểu nguyên thủy như int được gọi là "ngữ nghĩa giá trị" (value semantics), và những kiểu này thường được gọi là "kiểu giá trị" (value types). Hệ thống dành cho mảng và các đối tượng khác được gọi là "ngữ nghĩa tham chiếu" (reference semantics), và những kiểu này thường được gọi là "kiểu tham chiếu" (reference types).

Ngữ nghĩa Giá trị (Kiểu Giá trị)

Một hệ thống trong đó các giá trị được lưu trữ trực tiếp và việc sao chép được thực hiện bằng cách tạo ra các bản sao độc lập

của các giá trị. Các kiểu sử dụng ngữ nghĩa giá trị được gọi là kiểu giá trị.

Ngữ nghĩa Tham chiếu (Kiểu Tham chiếu)

Một hệ thống trong đó các tham chiếu đến giá trị được lưu trữ và việc sao chép được thực hiện bằng cách sao chép các tham chiếu này. Các kiểu sử dụng ngữ nghĩa tham chiếu được gọi là kiểu tham chiếu. Sẽ mất một thời gian để chúng ta khám phá tất cả các hệ quả của sự khác biệt này. Điều quan trọng cần nhớ là khi bạn làm việc với các đối tượng, bạn luôn làm việc với các tham chiếu đến dữ liệu thay vì chính dữ liệu đó.

Tại thời điểm này, có thể bạn đang tự hỏi tại sao Java lại có hai hệ thống khác nhau. Java được thiết kế cho lập trình hướng đối tượng, vì vậy câu hỏi đầu tiên cần xem xét là tại sao Sun quyết định rằng các đối tượng nên có ngữ nghĩa tham chiếu. Có hai lý do chính:

- **Hiệu quả.** Các đối tượng có thể phức tạp, điều này có nghĩa là chúng có thể chiếm rất nhiều không gian trong bộ nhớ. Nếu chúng ta tạo ra các bản sao của các đối tượng như vậy, chúng ta sẽ nhanh chóng hết bộ nhớ. Một đối tượng String lưu trữ một số lượng lớn ký tự có thể chiếm rất nhiều không gian trong bộ nhớ. Nhưng ngay cả khi đối tượng String rất lớn, một tham chiếu đến nó có thể khá nhỏ, tương tự như cách mà một dinh thự vẫn có một địa chỉ đường phố đơn giản. Một ví dụ khác là cách chúng ta sử dụng điện thoại di động để giao tiếp với mọi người. Các điện thoại có thể rất nhỏ và dễ dàng vận chuyển vì số điện thoại di động không chiếm nhiều không gian. Hãy tưởng tượng rằng thay vì mang theo một bộ số điện thoại, bạn lại cố gắng mang theo những người thực sự!
- **Chia sẻ.** Thường thì có một bản sao của thứ gì đó là không đủ tốt. Giả sử rằng giáo viên của bạn yêu cầu tất cả học sinh trong lớp đặt bài kiểm tra của mình vào một cái hộp nhất định. Hãy tưởng tượng sự vô nghĩa và khó hiểu sẽ xảy ra

nếu mỗi học sinh tạo ra một bản sao của cái hộp đó. Ý định rõ ràng là tất cả các học sinh sử dụng cùng một cái hộp. Ngữ nghĩa tham chiếu cho phép bạn có nhiều tham chiếu đến một đối tượng duy nhất, điều này cho phép các phần khác nhau của chương trình chia sẻ một đối tượng cụ thể.

Nếu không có ngữ nghĩa tham chiếu, các chương trình Java sẽ khó viết hơn. Vậy tại sao Sun lại quyết định bao gồm cả các kiểu nguyên thủy có ngữ nghĩa giá trị? Các lý do chủ yếu là lịch sử. Sun muốn tận dụng sự phổ biến của C và C++, các ngôn ngữ có các kiểu tương tự, và đảm bảo rằng các chương trình Java sẽ chạy nhanh chóng, điều này dễ dàng thực hiện hơn với các kiểu nguyên thủy truyền thống. Nếu các nhà thiết kế Java có cơ hội thiết kế lại Java ngày nay, có thể họ sẽ loại bỏ các kiểu nguyên thủy và sử dụng một mô hình đối tượng nhất quán chỉ với ngữ nghĩa tham chiếu.

Nhiều Đối tượng

Trong phần trước, bạn đã thấy cách thao tác với một đối tượng mảng duy nhất. Trong phần này, chúng ta sẽ tìm hiểu sâu hơn về các hệ quả của ngữ nghĩa tham chiếu bằng cách xem xét điều gì sẽ xảy ra khi có nhiều đối tượng và nhiều tham chiếu đến cùng một đối tượng.

Xem xét đoạn mã sau:

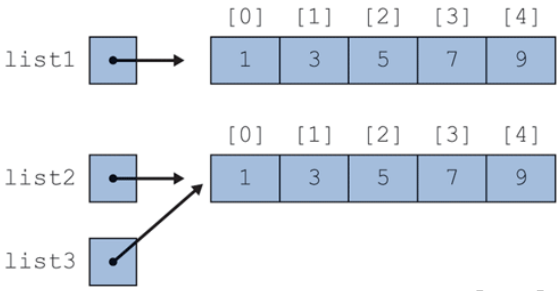
java

Copy code

```
int[] list1 = new int[5];
int[] list2 = new int[5];
for (int i = 0; i < list1.length; i++) {
    list1[i] = 2 * i + 1;
    list2[i] = 2 * i + 1;
}
int[] list3 = list2;
```

Mỗi lần gọi new tạo ra một đối tượng mới và đoạn mã này có hai lần gọi new, có nghĩa là chúng ta có hai đối tượng khác nhau.

Đoạn mã được viết theo cách mà list2 sẽ luôn có cùng độ dài và dãy giá trị như list1. Sau khi hai mảng được khởi tạo, chúng ta định nghĩa một biến mảng thứ ba được gán cho list2. Bước này tạo ra một tham chiếu mới nhưng không tạo ra một đối tượng mới. Sau khi máy tính thực thi đoạn mã, bộ nhớ sẽ trông như sau:



7.3-25 Văn bản Thay thế Đầy đủ

Chúng ta có ba biến nhưng chỉ có hai đối tượng. Các biến list2 và list3 đều tham chiếu đến cùng một đối tượng mảng. Sử dụng phép ẩn dụ với điện thoại di động, bạn có thể xem điều này giống như hai người cùng có số điện thoại di động của cùng một người. Điều đó có nghĩa là bất kỳ ai trong số họ đều có thể gọi cho người đó. Hoặc, như một phép ẩn dụ khác, giả sử cả bạn và một người bạn của bạn đều biết cách truy cập thông tin ngân hàng của bạn trực tuyến. Điều đó có nghĩa là cả hai bạn đều có quyền truy cập vào cùng một tài khoản và bất kỳ ai trong số bạn đều có thể thực hiện thay đổi tài khoản.

Hệ quả của phương pháp này là list2 và list3 theo một cách nào đó đều có khả năng như nhau trong việc sửa đổi mảng mà chúng tham chiếu đến. Dòng mã sau:

java

Copy code

```
list2[2]++;
```

sẽ có hiệu lực tương tự như dòng mã:

java

Copy code

```
list3[2]++;
```

Vì cả hai biến đều tham chiếu đến cùng một đối tượng mảng, bạn có thể truy cập mảng thông qua bất kỳ biến nào trong số đó. Ngữ nghĩa tham chiếu giúp chúng ta hiểu tại sao phép thử đơn giản == không cho ra kết quả như mong đợi. Khi thử nghiệm này được áp dụng cho các đối tượng, nó xác định liệu hai tham chiếu có giống nhau hay không (không phải các đối tượng mà chúng tham chiếu có tương đương với nhau hay không). Nói cách khác, khi chúng ta thử nghiệm xem hai tham chiếu có bằng nhau không, chúng ta đang kiểm tra liệu chúng có tham chiếu đến đúng cùng một đối tượng hay không.

Các biến list2 và list3 đều tham chiếu đến cùng một đối tượng mảng. Do đó, nếu chúng ta hỏi list2 == list3 thì câu trả lời sẽ là có (biểu thức đánh giá là true). Nhưng nếu chúng ta hỏi list1 == list2 thì câu trả lời sẽ là không (biểu thức đánh giá là false) mặc dù chúng ta nghĩ rằng hai mảng này theo một cách nào đó là tương đương.

Đôi khi bạn muốn biết liệu hai biến có tham chiếu đến đúng cùng một đối tượng hay không, và trong những trường hợp đó, so sánh đơn giản bằng == sẽ phù hợp. Nhưng bạn cũng sẽ muốn biết liệu hai đối tượng có tương đương với nhau về mặt giá trị hay không, trong trường hợp đó bạn nên gọi các phương thức như Arrays.equals hoặc phương thức equals của chuỗi.

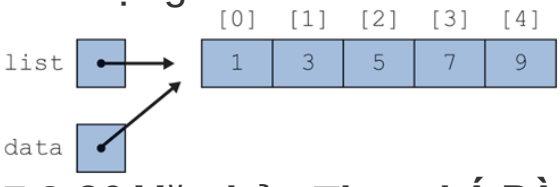
Hiểu ngữ nghĩa tham chiếu cũng cho phép bạn hiểu tại sao một phương thức có thể thay đổi nội dung của một mảng được truyền vào nó như một tham số. Hãy nhớ rằng ở phần trước của chương này, chúng ta đã xem xét phương thức sau:

java

Copy code

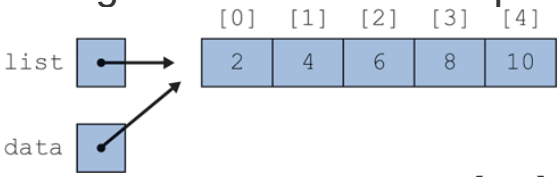
```
public static void incrementAll(int[] data) {
    for (int i = 0; i < data.length; i++) {
        data[i]++;
    }
}
```

Chúng ta đã thấy rằng khi biến list của chúng ta được khởi tạo với một mảng số lẻ, chúng ta có thể tăng tất cả các giá trị trong mảng bằng dòng mã sau:
java
Copy code
incrementAll(list);
Khi phương thức được gọi, chúng ta tạo ra một bản sao của biến list. Nhưng biến list không phải là mảng; thay vào đó, nó lưu trữ một tham chiếu đến mảng. Vì vậy, khi chúng ta tạo ra một bản sao của tham chiếu đó, chúng ta có hai tham chiếu đến cùng một đối tượng.



7.3-26 Văn bản Thay thế Đầy đủ

Bởi vì data và list đều tham chiếu đến cùng một đối tượng, khi chúng ta thay đổi data bằng cách nói data[i]++, chúng ta kết thúc bằng việc thay đổi đối tượng mà list tham chiếu đến. Đó là lý do tại sao, sau khi vòng lặp tăng giá trị của từng phần tử trong data, chúng ta kết thúc với kết quả sau:



7.3-27 Văn bản Thay thế Đầy đủ

Bài học quan trọng cần rút ra từ cuộc thảo luận này là khi chúng ta truyền một mảng làm tham số cho một phương thức, phương thức đó có khả năng thay đổi nội dung của mảng.
Trước khi chúng ta rời khỏi chủ đề ngữ nghĩa tham chiếu, chúng ta nên mô tả chi tiết hơn về khái niệm giá trị đặc biệt null. Đây là một từ khóa đặc biệt trong Java được sử dụng để biểu thị "không có đối tượng".

null

Một từ khóa trong Java biểu thị không có đối tượng.

Khái niệm null không có ý nghĩa với các kiểu giá trị như int và double lưu trữ giá trị thực tế. Nhưng có thể hợp lý khi đặt một biến lưu trữ tham chiếu thành null. Đây là một cách để nói với máy tính rằng bạn muốn có biến này, nhưng bạn chưa xác định được đối tượng mà nó nên tham chiếu đến. Vì vậy, bạn có thể sử dụng null cho các biến của bất kỳ kiểu đối tượng nào, chẳng hạn như một chuỗi (String) hoặc mảng:
java
Copy code
String s = null;
int[] list = null;
Có sự khác biệt giữa việc đặt một biến thành một chuỗi trống và đặt nó thành null. Khi bạn đặt một biến thành một chuỗi trống, có một đối tượng thực tế mà biến của bạn tham chiếu đến (mặc dù không phải là một đối tượng thú vị). Khi bạn đặt một biến thành null, biến đó chưa tham chiếu đến một đối tượng thực tế. Nếu bạn cố gắng sử dụng biến để truy cập đối tượng khi nó đã được đặt thành null, Java sẽ ném ra ngoại lệ NullPointerException.

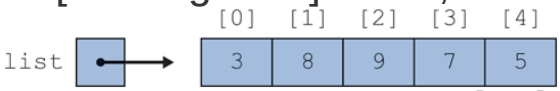
7.4 Các Kỹ Thuật Mảng Nâng Cao

Trong phần này, chúng ta sẽ thảo luận về một số cách sử dụng mảng nâng cao, chẳng hạn như các thuật toán không thể giải quyết bằng cách duyệt đơn giản. Chúng ta cũng sẽ xem cách tạo mảng lưu trữ các đối tượng thay vì các giá trị nguyên thủy.

Dịch Chuyển Giá Trị Trong Mảng

Bạn thường muốn di chuyển một loạt giá trị trong mảng. Ví dụ, giả sử bạn có một mảng các số nguyên lưu trữ dãy giá trị [3, 8, 9, 7, 5] và bạn muốn đưa giá trị ở đầu danh sách ra phía sau và giữ nguyên thứ tự của các giá trị khác. Nói cách khác, bạn muốn di chuyển số 3 ra phía sau, tạo ra danh sách [8, 9, 7, 5, 3]. Hãy cùng khám phá cách viết mã để thực hiện hành động đó.
Giả sử bạn có một biến kiểu int[] gọi là list với độ dài 5 lưu trữ các giá trị [3, 8, 9, 7, 5]:
java

Copy code
```
int[] list = {3, 8, 9, 7, 5};
int first = list[0];
for (int i = 1; i < list.length; i++) {
    list[i - 1] = list[i];
}
list[list.length - 1] = first;
```



7.4-28 Văn bản Thay thế Đầy đủ

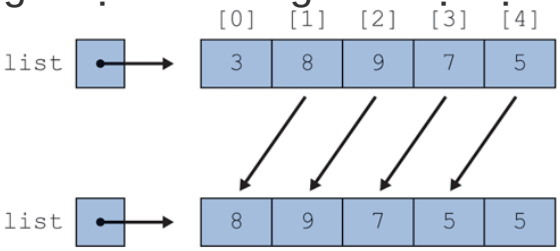
Phép dịch chuyển này tương tự như phép hoán đổi đã thảo luận trong phần trước, và bạn sẽ thấy rằng việc sử dụng một biến tạm thời ở đây cũng rất hữu ích. Số 3 ở đầu danh sách cần được chuyển đến cuối danh sách, và các giá trị khác sẽ được dịch về phía trước. Bạn có thể làm cho nhiệm vụ này dễ dàng hơn bằng cách lưu trữ giá trị ở đầu danh sách (trong ví dụ này là 3) vào một biến cục bộ:

java

Copy code

```
int first = list[0];
```

Với giá trị đó đã được lưu trữ an toàn, bây giờ bạn cần dịch các giá trị khác sang trái một vị trí:



7.4-29 Văn bản Thay thế Đầy đủ

Nhiệm vụ tổng thể này được chia thành bốn phép dịch chuyển khác nhau, mỗi phép là một câu lệnh gán đơn giản:

java

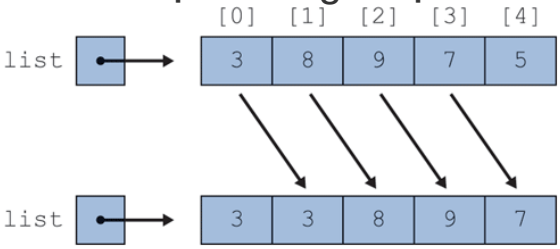
Copy code

```
list[0] = list[1];
list[1] = list[2];
```

```
list[2] = list[3];
list[3] = list[4];
```
Rõ ràng bạn sẽ muốn viết điều này dưới dạng một vòng lặp thay vì viết một loạt các câu lệnh gán riêng lẻ. Mỗi câu lệnh trên có dạng:
java
Copy code
```
list[i] = list[i + 1];
```
Bạn sẽ thay thế phần tử list[i] bằng giá trị hiện đang lưu trữ trong phần tử list[i + 1], điều này sẽ dịch chuyển giá trị đó sang trái. Bạn có thể đặt dòng mã này vào một vòng lặp duyệt tiêu chuẩn:
java
Copy code
```
for (int i = 0; i < list.length; i++) {
    list[i] = list[i + 1];
}
```
Vòng lặp này gần như đúng, nhưng nó có một lỗi "off-by-one". Vòng lặp này sẽ thực hiện năm lần đối với mảng mẫu, nhưng bạn chỉ muốn dịch chuyển bốn giá trị (bạn muốn thực hiện phép gán cho i bằng 0, 1, 2, và 3, nhưng không phải cho i bằng 4). Vì vậy, vòng lặp này đi quá một lần. Trong lần lặp cuối cùng của vòng lặp, khi i bằng 4, vòng lặp thực hiện dòng mã sau:
java
Copy code
```
list[i] = list[i + 1];
```
Dòng này trở thành:
java
Copy code
```
list[4] = list[5];
```
Không có giá trị list[5] vì mảng chỉ có năm phần tử, với các chỉ số từ 0 đến 4. Vì vậy, đoạn mã này tạo ra lỗi ArrayIndexOutOfBoundsException. Để khắc phục vấn đề, hãy thay đổi vòng lặp để nó dừng lại sớm hơn một lần:

java
Copy code
```
for (int i = 0; i < list.length - 1; i++) {
    list[i] = list[i + 1];
}
```
Thay vì sử dụng list.length như thường lệ, hãy sử dụng (list.length - 1). Bạn có thể coi dấu trừ một trong biểu thức này như là bù đắp cho dấu cộng một trong câu lệnh gán.
Tất nhiên, có một chi tiết cuối cùng bạn phải giải quyết. Sau khi dịch các giá trị sang trái, bạn đã tạo ra khoảng trống ở cuối danh sách cho giá trị mà trước đó đã ở đầu danh sách (hiện đang được lưu trữ trong một biến cục bộ gọi là first). Khi vòng lặp hoàn thành, bạn cần đặt giá trị này vào chỉ số 4:
java
Copy code
```
list[list.length - 1] = first;
```
Dưới đây là phương thức hoàn chỉnh:
java
Copy code
```
public static void rotateLeft(int[] list) {
    int first = list[0];
    for (int i = 0; i < list.length - 1; i++) {
        list[i] = list[i + 1];
    }
    list[list.length - 1] = first;
}
```
Một biến thể thú vị của phương thức này là xoay các giá trị sang phải thay vì xoay chúng sang trái. Để thực hiện phép đảo ngược này, bạn muốn lấy giá trị hiện đang ở cuối danh sách và đưa nó lên đầu, dịch các giá trị còn lại sang phải. Vì vậy, nếu một biến gọi là list ban đầu lưu trữ các giá trị [3, 8, 9, 7, 5], nó sẽ đưa số 5 lên đầu và lưu trữ các giá trị [5, 3, 8, 9, 7].

Bắt đầu bằng cách lưu trữ giá trị được xoay vào một biến tạm thời:
java
Copy code
int last = list[list.length - 1];
Sau đó dịch các giá trị khác sang phải:

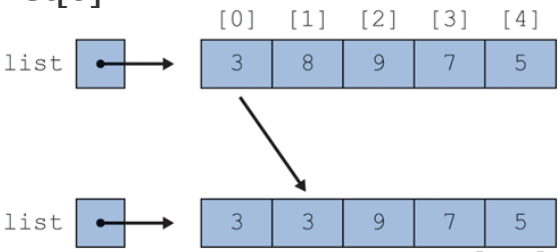


7.4-30 Văn bản Thay thế Đầy đủ
Trong trường hợp này, bốn câu lệnh gán riêng biệt sẽ là:

1. list[1] = list[0];
2. list[2] = list[1];
3. list[3] = list[2];
4. list[4] = list[3];

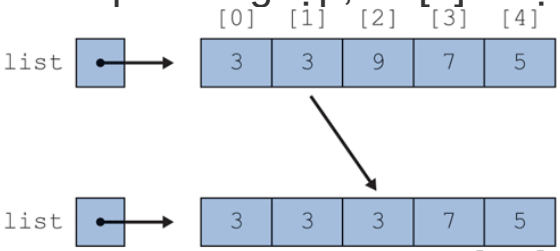
Một cách tổng quát hơn để viết điều này là dòng code sau:
list[i] = list[i - 1];
Nếu bạn đặt dòng code này vào trong vòng lặp for chuẩn, bạn sẽ có đoạn sau:
java
Copy code
// không hoạt động
for (int i = 0; i < list.length; i++) {
    list[i] = list[i - 1];
}
Có hai vấn đề với đoạn code này. Đầu tiên, có một lỗi sai chỉ số. Câu lệnh gán đầu tiên bạn muốn thực hiện sẽ đặt list[1] chứa giá trị hiện tại của list[0], nhưng vòng lặp này lại đặt list[0] thành list[-1]. Java sẽ tạo ra một ngoại lệ ArrayIndexOutOfBoundsException vì không có giá trị list[-1]. Bạn muốn bắt đầu i từ 1, không phải từ 0:
java

Copy code
```
// vẫn không hoạt động
for (int i = 1; i < list.length; i++) {
    list[i] = list[i - 1];
}
```
Tuy nhiên, phiên bản này của code cũng không hoạt động. Nó tránh được ngoại lệ, nhưng hãy nghĩ về điều mà nó thực hiện. Lần đầu tiên qua vòng lặp, nó sẽ gán list[1] bằng giá trị trong list[0].



7.4-31 Văn bản Thay thế Đầy đủ

Điều gì đã xảy ra với giá trị 8? Nó bị ghi đè bằng giá trị 3. Lần tiếp theo qua vòng lặp, list[2] được gán thành list[1].

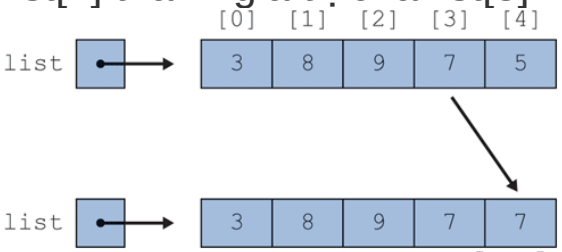


7.4-32 Văn bản Thay thế Đầy đủ

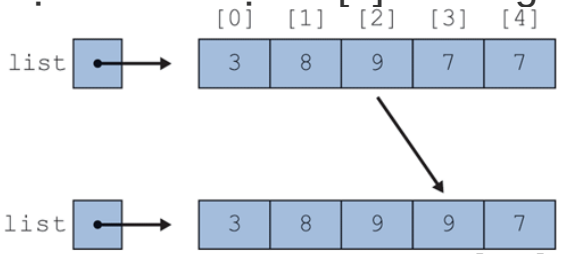
Bạn có thể nói: "Chờ một chút... list[1] không phải là 3, mà là 8." Đúng, nó là 8 khi bạn bắt đầu, nhưng lần lặp đầu tiên của vòng lặp đã thay thế 8 bằng 3, và bây giờ 3 đã được sao chép vào vị trí mà trước đó là 9.

Vòng lặp tiếp tục như vậy, đặt 3 vào mỗi ô của mảng. Rõ ràng, đó không phải là điều bạn muốn. Để làm cho đoạn mã này hoạt động, bạn phải chạy vòng lặp theo thứ tự ngược lại (từ phải sang trái thay vì từ trái sang phải). Vì vậy, hãy quay lại điểm xuất phát:

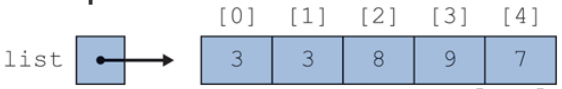
7.4-33 Văn bản Thay thế Đầy đủ
Chúng ta đã lưu giá trị cuối cùng của danh sách vào một biến cục bộ. Điều này giải phóng vị trí cuối cùng của mảng. Bây giờ, gán list[4] thành giá trị của list[3]:



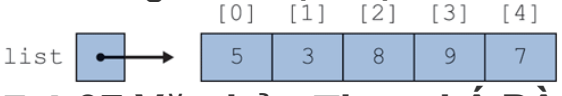
7.4-34 Văn bản Thay thế Đầy đủ
Điều này xóa giá trị 5 ở cuối danh sách, nhưng giá trị này đã được lưu trữ an toàn trong một biến cục bộ. Và sau khi bạn thực hiện câu lệnh gán này, bạn giải phóng list[3], điều đó có nghĩa là bạn có thể đặt list[3] thành giá trị hiện tại của list[2].



7.4-35 Văn bản Thay thế Đầy đủ
Quá trình tiếp tục như vậy, sao chép giá trị 8 từ chỉ số 1 sang chỉ số 2 và sao chép giá trị 3 từ chỉ số 0 sang chỉ số 1, để lại bạn với kết quả sau:



7.4-36 Văn bản Thay thế Đầy đủ
Tại thời điểm này, việc duy nhất còn lại là đặt giá trị 5 được lưu trữ trong biến cục bộ vào đầu danh sách:



7.4-37 Văn bản Thay thế Đầy đủ
Bạn có thể đảo ngược vòng lặp for bằng cách thay đổi i++ thành i-- và điều chỉnh phần khởi tạo và điều kiện kiểm tra. Phương thức cuối cùng như sau:
java

Copy code

```
public static void rotateRight(int[] list) {
    int last = list[list.length - 1];
    for (int i = list.length - 1; i >= 1; i--) {
        list[i] = list[i - 1];
    }
    list[0] = last;
}
```

7.4-31 Văn bản Thay thế Đầy đủ

Điều gì đã xảy ra với giá trị 8? Nó bị ghi đè bằng giá trị 3. Lần tiếp theo qua vòng lặp, list[2] được gán thành list[1].

7.4-32 Văn bản Thay thế Đầy đủ

Bạn có thể nói: "Chờ một chút... list[1] không phải là 3, mà là 8." Đúng, nó là 8 khi bạn bắt đầu, nhưng lần lặp đầu tiên của vòng lặp đã thay thế 8 bằng 3, và bây giờ 3 đã được sao chép vào vị trí mà trước đó là 9.

Vòng lặp tiếp tục như vậy, đặt 3 vào mỗi ô của mảng. Rõ ràng, đó không phải là điều bạn muốn. Để làm cho đoạn mã này hoạt động, bạn phải chạy vòng lặp theo thứ tự ngược lại (từ phải sang trái thay vì từ trái sang phải). Vì vậy, hãy quay lại điểm xuất phát:

7.4-33 Văn bản Thay thế Đầy đủ

Chúng ta đã lưu giá trị cuối cùng của danh sách vào một biến cục bộ. Điều này giải phóng vị trí cuối cùng của mảng. Bây giờ, gán list[4] thành giá trị của list[3]:

7.4-34 Văn bản Thay thế Đầy đủ

Điều này xóa giá trị 5 ở cuối danh sách, nhưng giá trị này đã được lưu trữ an toàn trong một biến cục bộ. Và sau khi bạn thực hiện câu lệnh gán này, bạn giải phóng list[3], điều đó có nghĩa là bạn có thể đặt list[3] thành giá trị hiện tại của list[2].

7.4-35 Văn bản Thay thế Đầy đủ

Quá trình tiếp tục như vậy, sao chép giá trị 8 từ chỉ số 1 sang chỉ

số 2 và sao chép giá trị 3 từ chỉ số 0 sang chỉ số 1, để lại bạn với kết quả sau:

7.4-36 Văn bản Thay thế Đầy đủ

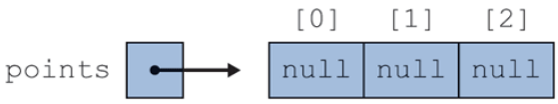
Tại thời điểm này, việc duy nhất còn lại là đặt giá trị 5 được lưu trữ trong biến cục bộ vào đầu danh sách:

7.4-37 Văn bản Thay thế Đầy đủ

Bạn có thể đảo ngược vòng lặp for bằng cách thay đổi i++ thành i-- và điều chỉnh phần khởi tạo và điều kiện kiểm tra. Phương thức cuối cùng như sau:

java
Copy code

```
public static void rotateRight(int[] list) {
    int last = list[list.length - 1];
    for (int i = list.length - 1; i >= 1; i--) {
        list[i] = list[i - 1];
    }
    list[0] = last;
}
```



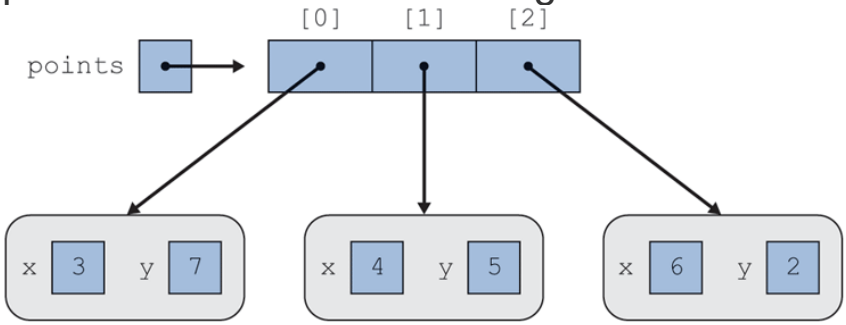
7.4-38 Văn bản Thay thế Đầy đủ

Các đối tượng Point thực tế phải được tạo riêng biệt bằng từ khóa new, như trong đoạn mã sau:

java
Copy code

```
Point[] points = new Point[3];
points[0] = new Point(3, 7);
points[1] = new Point(4, 5);
points[2] = new Point(6, 2);
```

Sau khi các dòng mã này được thực thi, chương trình của bạn sẽ tạo ra các đối tượng Point riêng biệt được tham chiếu bởi các phần tử khác nhau của mảng.



7.4-39 Văn bản Thay thế Đầy đủ

Chú ý rằng từ khóa new là cần thiết ở bốn vị trí khác nhau, vì có bốn đối tượng cần được tạo: chính mảng và ba đối tượng Point riêng lẻ. Bạn cũng có thể sử dụng ký hiệu dấu ngoặc nhọn để khởi tạo mảng, trong trường hợp này bạn không cần từ khóa new để tạo mảng:

java

Copy code

```
Point[] points = {new Point(3, 7), new Point(4, 5), new Point(6, 2)};
```

Đối số Dòng Lệnh

Như bạn đã thấy từ Chương 1, bất cứ khi nào bạn định nghĩa một phương thức main, bạn phải bao gồm tham số String[] args, là một mảng các đối tượng String. Java tự khởi tạo mảng này nếu người dùng cung cấp cái được gọi là "đối số dòng lệnh" khi gọi Java. Ví dụ, người dùng có thể thực thi một lớp Java có tên DoSomething từ dòng lệnh hoặc terminal bằng lệnh như sau:

bash

Copy code

```
java DoSomething
```

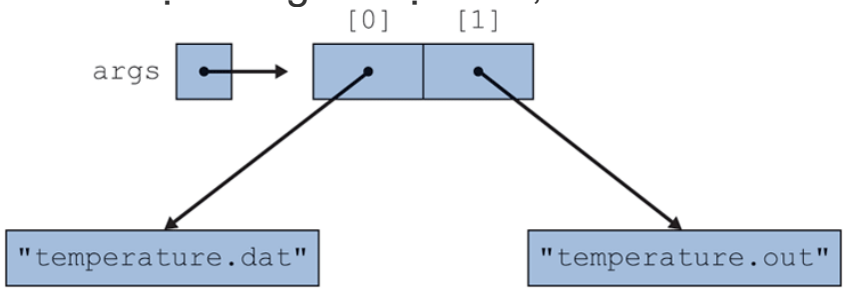
Người dùng có thể tùy chọn nhập thêm các đối số, như trong trường hợp sau:

bash

Copy code

```
java DoSomething temperature.dat temperature.out
```

Trong trường hợp này, người dùng đã chỉ định hai đối số bổ sung là các tên tệp mà chương trình sẽ sử dụng (ví dụ: tên của tệp đầu vào và tệp đầu ra). Nếu người dùng nhập các đối số này khi khởi chạy Java, tham số String[] args trong main sẽ được khởi tạo thành một mảng có độ dài 2, lưu trữ hai chuỗi này.



7.4-40 Văn bản Thay thế Đầy đủ
Thuật toán Vòng lặp Lồng nhau

Tất cả các thuật toán mà chúng ta đã thấy đều được viết với một vòng lặp duy nhất. Nhưng nhiều phép tính đòi hỏi vòng lặp lồng nhau. Ví dụ, giả sử bạn được yêu cầu in tất cả các phép đảo ngược trong một mảng số nguyên. Một phép đảo ngược được định nghĩa là một cặp số mà số đầu tiên trong danh sách lớn hơn số thứ hai.

Trong một danh sách đã sắp xếp như [1, 2, 3, 4], không có phép đảo ngược nào và không có gì để in ra. Nhưng nếu các số xuất hiện theo thứ tự ngược lại, [4, 3, 2, 1], thì có nhiều phép đảo ngược để in ra. Chúng ta mong đợi kết quả như sau:

scss

Copy code

(4, 3)
(4, 2)
(4, 1)
(3, 2)
(3, 1)
(2, 1)

Chú ý rằng bất kỳ số nào (ví dụ, số 4 trong danh sách trên) có thể tạo ra nhiều phép đảo ngược khác nhau, vì nó có thể được theo sau bởi nhiều số nhỏ hơn (1, 2, và 3 trong ví dụ). Với một danh

sách đã được sắp xếp một phần, như [3, 1, 4, 2], chỉ có một vài phép đảo ngược, vì vậy bạn sẽ nhận được kết quả như sau:
scss
Copy code
(3, 1)
(3, 2)
(4, 2)
Vấn đề này không thể được giải quyết bằng một lần duyệt vì chúng ta đang tìm các cặp số. Có nhiều giá trị đầu tiên và nhiều giá trị thứ hai trong cặp. Hãy phát triển một giải pháp bằng cách sử dụng mã giả.
Chúng ta không thể tạo tất cả các cặp bằng một vòng lặp duy nhất, nhưng chúng ta có thể sử dụng một vòng lặp để xem xét tất cả các giá trị đầu tiên có thể có:
java
Copy code
```
for (mỗi giá trị đầu tiên có thể) {
    in tất cả các phép đảo ngược liên quan đến giá trị đầu tiên này.
}
```
Bây giờ chúng ta chỉ cần viết mã để tìm tất cả các phép đảo ngược cho một giá trị đầu tiên cho trước. Điều đó yêu cầu chúng ta viết một vòng lặp thứ hai, lồng vào vòng lặp đầu tiên:
java
Copy code
```
for (mỗi giá trị đầu tiên có thể) {
    for (mỗi giá trị thứ hai có thể) {
        if (giá trị đầu tiên > giá trị thứ hai) {
            print(first, second).
        }
    }
}
```

Vấn đề này khá dễ chuyển thành mã Java, mặc dù giới hạn của vòng lặp có thể hơi phức tạp. Hiện tại, hãy sử dụng vòng lặp duyệt chuẩn cho mỗi giá trị:
java
Copy code
```
for (int i = 0; i < data.length; i++) {
    for (int j = 0; j < data.length; j++) {
        if (data[i] > data[j]) {
            System.out.println("(" + data[i] + ", " + data[j] + ")");
        }
    }
}
```
Mã trên chưa hoàn toàn đúng. Hãy nhớ rằng, để có một phép đảo ngược, giá trị thứ hai phải xuất hiện sau giá trị đầu tiên trong danh sách. Trong trường hợp này, chúng ta đang tính toán tất cả các kết hợp có thể của giá trị đầu tiên và giá trị thứ hai. Để chỉ xem xét các giá trị xuất hiện sau giá trị đầu tiên đã cho, chúng ta phải bắt đầu vòng lặp thứ hai tại i + 1 thay vì bắt đầu từ 0. Chúng ta cũng có thể cải thiện một chút bằng cách nhận ra rằng vì một phép đảo ngược yêu cầu một cặp giá trị, không có lý do gì để bao gồm số cuối cùng trong danh sách như một giá trị đầu tiên có thể. Vì vậy, vòng lặp ngoài liên quan đến i có thể kết thúc sớm hơn một lần lặp:
java
Copy code
```
for (int i = 0; i < data.length - 1; i++) {
    for (int j = i + 1; j < data.length; j++) {
        if (data[i] > data[j]) {
            System.out.println("(" + data[i] + ", " + data[j] + ")");
        }
    }
}
```

Khi bạn viết các vòng lặp lồng nhau như thế này, một quy ước phổ biến là sử dụng i cho vòng lặp ngoài, j cho vòng lặp bên trong vòng lặp ngoài, và k nếu có một vòng lặp bên trong vòng lặp j.

7.5 Mảng Đa Chiều

Các ví dụ về mảng trong các phần trước đều liên quan đến cái được gọi là mảng một chiều (một hàng hoặc một cột dữ liệu). Thông thường, bạn sẽ muốn lưu trữ dữ liệu theo cách đa chiều. Ví dụ, bạn có thể muốn lưu trữ một lưới hai chiều của dữ liệu có cả hàng và cột. May mắn thay, bạn có thể tạo ra các mảng với nhiều chiều tùy ý:

- double: một giá trị kiểu double
- double[]: một mảng một chiều các giá trị double
- double[][]: một lưới hai chiều các giá trị double
- double[][][]: một bộ sưu tập ba chiều các giá trị double
- ...

Mảng có hơn một chiều được gọi là mảng đa chiều.

Mảng Đa Chiều

Là mảng của các mảng, các phần tử của chúng được truy cập bằng nhiều chỉ số nguyên.

Mảng Hai Chiều Hình Chữ Nhật

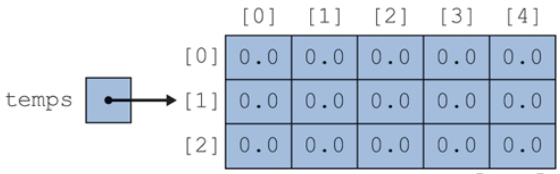
Ứng dụng phổ biến nhất của mảng đa chiều là mảng hai chiều với chiều rộng và chiều cao nhất định. Ví dụ, giả sử rằng trong ba ngày riêng biệt, bạn đã thực hiện một loạt năm lần đo nhiệt độ. Bạn có thể định nghĩa một mảng hai chiều có ba hàng và năm cột như sau:

java
Copy code

```
double[][] temps = new double[3][5];
```

Chú ý rằng cả ở phía bên trái và bên phải của câu lệnh gán này, bạn phải sử dụng một cặp dấu ngoặc vuông. Khi bạn mô tả kiểu dữ liệu ở phía bên trái, bạn phải làm rõ rằng đây không chỉ là một chuỗi một chiều các giá trị, mà thay vào đó là một lưới hai chiều các giá trị, có kiểu là double[][]. Ở phía bên phải, khi bạn tạo

mảng, bạn phải chỉ định kích thước của lưới. Quy ước thông thường là liệt kê hàng trước rồi đến cột. Mảng kết quả sẽ trông như sau:



7.5-41 Văn bản Thay thế Đầy đủ

Giống như mảng một chiều, các giá trị trong mảng được khởi tạo với giá trị 0.0 và các chỉ số bắt đầu từ 0 cho cả hàng và cột. Sau khi bạn tạo ra một mảng như vậy, bạn có thể tham chiếu đến các phần tử cụ thể bằng cách cung cấp số hàng và số cột (theo thứ tự đó). Ví dụ, để đặt giá trị thứ tư của hàng đầu tiên thành 98.3 và đặt giá trị đầu tiên của hàng thứ ba thành 99.4, bạn sẽ viết đoạn mã sau:

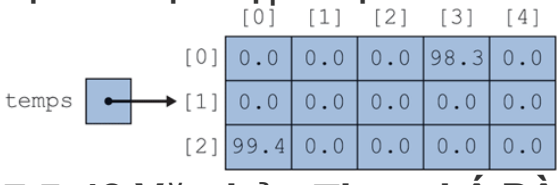
java

Copy code

```
temps[0][3] = 98.3;  // giá trị thứ tư của hàng đầu tiên
temps[2][0] = 99.4;  // giá trị đầu tiên của hàng thứ ba
```

Sau khi chương trình thực thi các dòng mã này, mảng sẽ trông như sau:

(Phần mô tả tiếp theo sẽ là hình minh họa của mảng với các giá trị đã được cập nhật theo các dòng lệnh trên).



7.5-42 Văn bản Thay thế Đầy đủ

Việc tham chiếu đến các phần tử riêng lẻ theo từng bước sẽ hữu ích, bắt đầu bằng tên của mảng. Ví dụ, nếu bạn muốn tham chiếu đến giá trị đầu tiên của hàng thứ ba, bạn sẽ thực hiện qua các bước sau:

text

Copy code

```
temps        // toàn bộ lưới
```

```
temps[2]      // toàn bộ hàng thứ ba
temps[2][0]   // phần tử đầu tiên của hàng thứ ba
```

Bạn có thể truyền mảng nhiều chiều như là tham số, giống như cách truyền mảng một chiều. Tuy nhiên, bạn cần chú ý về kiểu dữ liệu. Để truyền lưới nhiệt độ, bạn sẽ phải sử dụng tham số có kiểu double[][] (cả hai bộ dấu ngoặc vuông). Ví dụ, đây là một phương thức in lưới:

java
Copy code

```
public static void print(double[][] grid) {
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[i].length; j++) {
            System.out.print(grid[i][j] + " ");
        }
        System.out.println();
    }
}
```

Lưu ý rằng để lấy số hàng, bạn sử dụng grid.length, và để lấy số cột, bạn sử dụng grid[i].length.

Phương thức Arrays.toString được đề cập trước đó trong chương này có thể hoạt động trên mảng đa chiều, nhưng nó tạo ra kết quả không tốt. Khi sử dụng với mảng temps đã trình bày ở trên, nó tạo ra đầu ra như sau:

text
Copy code

```
[[D@14b081b, [D@1015a9e, [D@1e45a5c]
```

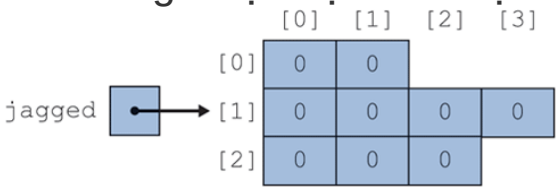
Kết quả không tốt này là do Arrays.toString hoạt động bằng cách nối các biểu diễn chuỗi của các phần tử trong mảng. Trong trường hợp này, các phần tử là các mảng nên chúng không chuyển đổi thành chuỗi một cách chính xác. Để giải quyết vấn đề này, bạn có thể sử dụng một phương thức khác gọi là Arrays.deepToString để trả về kết quả tốt hơn cho các mảng đa chiều:

java
Copy code
System.out.println(Arrays.deepToString(temps));
Lệnh gọi này sẽ tạo ra đầu ra như sau:
text
Copy code
[[0.0, 0.0, 0.0, 98.3, 0.0], [0.0, 0.0, 0.0, 0.0, 0.0], [99.4, 0.0, 0.0, 0.0, 0.0]]
Mảng có thể có bao nhiêu chiều tùy ý. Ví dụ, nếu bạn muốn có một khối lập phương ba chiều 4x4x4 gồm các số nguyên, bạn sẽ viết dòng mã sau:
java
Copy code
int[][][] numbers = new int[4][4][4];
Thông thường, thứ tự của các giá trị là số mặt phẳng, tiếp theo là số hàng, rồi đến số cột, mặc dù bạn có thể sử dụng bất kỳ quy ước nào miễn là mã của bạn được viết một cách nhất quán.

Mảng Không Đều (Jagged Arrays)

Các ví dụ trước đây liên quan đến các lưới hình chữ nhật có số hàng và cột cố định. Cũng có thể tạo một mảng không đều, trong đó số cột thay đổi từ hàng này sang hàng khác.
Để tạo một mảng không đều, chia việc xây dựng thành hai bước: Đầu tiên tạo mảng để giữ các hàng, sau đó tạo từng hàng riêng lẻ. Ví dụ, để tạo một mảng có hai phần tử trong hàng đầu tiên, bốn phần tử trong hàng thứ hai, và ba phần tử trong hàng thứ ba, bạn có thể viết các dòng mã sau:
java
Copy code
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[3];
Mã này sẽ tạo ra một mảng trông như sau:

(Phần mô tả tiếp theo sẽ là hình minh họa của mảng không đều với các giá trị được khởi tạo theo các dòng lệnh trên).



7.5-43 Văn bản Thay thế Đầy đủ

Chúng ta có thể khám phá kỹ thuật này bằng cách viết một chương trình tạo ra các hàng của cái gọi là tam giác Pascal. Các số trong tam giác này có nhiều tính chất toán học hữu ích. Ví dụ, hàng thứ n của tam giác Pascal chứa các hệ số nhận được khi bạn mở rộng phương trình:

(x+y)n(x + y)^n(x+y)n

Dưới đây là kết quả cho n từ 0 đến 4:

(x+y)0=1,(x+y)1=x+y,(x+y)2=x2+2xy+y2,(x + y)^0 = 1, \quad (x + y)^1 = x + y, \quad (x + y)^2 = x^2 + 2xy + y^2,(x+y)0=1,(x+y)1=x+y,(x+y)2=x2+2xy+y2, (x+y)3=x3+3x2y+3xy2+y3,(x+y)4=x4+4x3y+6x2y2+4xy3+y4(x + y)^3 = x^3 + 3x^2y + 3xy^2 + y^3, \quad (x + y)^4 = x^4 + 4x^3y + 6x^2y^2 + 4xy^3 + y^4(x+y)3=x3+3x2y+3xy2+y3,(x+y)4=x4+4x3y+6x2y2+4xy3+y4

Nếu bạn chỉ lấy ra các hệ số, bạn sẽ nhận được các giá trị sau:

markdown

Copy code

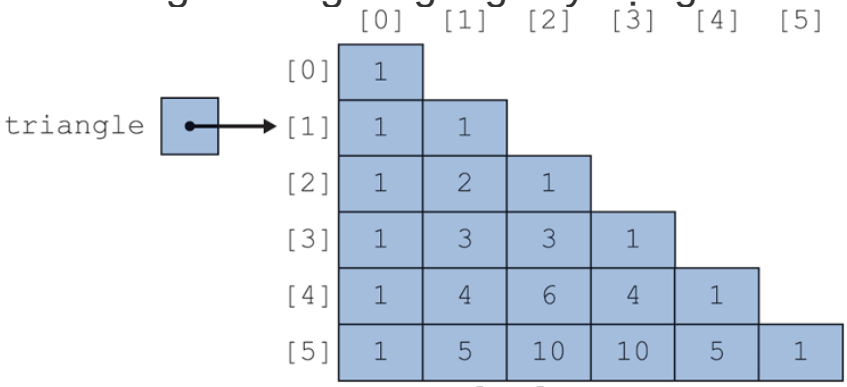
```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

Các hàng số này tạo thành một tam giác Pascal gồm năm hàng. Một trong những tính chất của tam giác là nếu bạn đã có một hàng bất kỳ, bạn có thể dùng nó để tính toán hàng tiếp theo. Ví dụ, hãy bắt đầu với hàng cuối cùng từ tam giác ở trên:

Copy code

1 4 6 4 1
Chúng ta có thể tính hàng tiếp theo bằng cách cộng các cặp giá trị liền kề với nhau. Vì vậy, chúng ta cộng cặp số đầu tiên (1 + 4), rồi cặp số thứ hai (4 + 6), và cứ tiếp tục như vậy:
Copy code
5 10 10 5
Sau đó, chúng ta đặt số 1 vào đầu và cuối của danh sách số này, và chúng ta có hàng tiếp theo của tam giác:
markdown
Copy code
```
      1
     1  1
    1  2  1
   1  3  3  1
  1  4  6  4  1
1  5  10  10  5  1
```
Tính chất này của tam giác cung cấp một kỹ thuật để tính toán nó. Chúng ta có thể xây dựng nó từng hàng, tính toán mỗi hàng mới từ các giá trị trong hàng trước đó. Nói cách khác, chúng ta viết vòng lặp sau (giả sử rằng chúng ta có một mảng hai chiều gọi là triangle để lưu trữ kết quả):
java
Copy code
```
for (int i = 0; i < triangle.length; i++) {
    // Xây dựng triangle[i] sử dụng triangle[i - 1].
}
```
Chúng ta chỉ cần bổ sung các chi tiết về cách xây dựng một hàng mới. Đây là một mảng không đều vì mỗi hàng có số phần tử khác nhau. Nhìn vào tam giác, bạn sẽ thấy rằng hàng đầu tiên (hàng 0) có một giá trị, hàng thứ hai (hàng 1) có hai giá trị, và cứ tiếp tục như vậy. Tổng quát, hàng i có (i + 1) giá trị, vì vậy chúng ta có thể tinh chỉnh giả mã như sau:
java

Copy code
```
for (int i = 0; i < triangle.length; i++) {
    triangle[i] = new int[i + 1];
    // Điền vào triangle[i] sử dụng triangle[i - 1].
}
```
Chúng ta biết rằng các giá trị đầu tiên và cuối cùng trong mỗi hàng phải là 1:
java
Copy code
```
for (int i = 0; i < triangle.length; i++) {
    triangle[i] = new int[i + 1];
    triangle[i][0] = 1;
    triangle[i][i] = 1;
    // Điền vào giữa của triangle[i] sử dụng triangle[i - 1].
}
```
Và chúng ta biết rằng các giá trị ở giữa lấy từ hàng trước đó. Để tìm hiểu cách tính toán chúng, hãy vẽ hình minh họa của mảng mà chúng ta đang cố gắng xây dựng.



7.5-44 Văn bản Thay thế Đầy đủ

Chúng ta đã viết mã để điền số 1 xuất hiện ở đầu và cuối mỗi hàng. Bây giờ, chúng ta cần viết mã để điền các giá trị ở giữa. Hãy xem hàng thứ 5 làm ví dụ. Giá trị 5 ở cột 1 có được từ tổng của các giá trị 1 ở cột 0 và 4 ở cột 1 trong hàng trước đó. Giá trị 10 ở cột 2 có được từ tổng của các giá trị ở cột 1 và 2 trong hàng trước đó.

Tổng quát hơn, mỗi giá trị ở giữa này là tổng của hai giá trị từ hàng trước, nằm ngay phía trên và bên trái của nó. Nói cách khác, với cột j, các giá trị được tính toán như sau:
triangle[i][j] = (giá trị phía trên và bên trái) + (giá trị phía trên).
Chúng ta có thể chuyển điều này thành mã thực tế bằng cách sử dụng các chỉ số mảng thích hợp:
triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
Chúng ta cần đưa câu lệnh này vào một vòng lặp for để nó gán tất cả các giá trị ở giữa. Vòng lặp for là bước cuối cùng để chuyển đổi giả mã của chúng ta thành mã thực tế:
java
Copy code
```
for (int i = 0; i < triangle.length; i++) {
    triangle[i] = new int[i + 1];
    triangle[i][0] = 1;
    triangle[i][i] = 1;
    for (int j = 1; j < i; j++) {
        triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
    }
}
```
Nếu chúng ta bao gồm đoạn mã này trong một phương thức cùng với phương thức in tương tự như phương thức in lưới đã mô tả trước đó, chúng ta sẽ có chương trình hoàn chỉnh sau:
java
Copy code
```
// Chương trình này xây dựng một mảng hai chiều không đều
// lưu trữ tam giác Pascal. Nó tận dụng
// thực tế rằng mỗi giá trị khác với các số 1 xuất hiện
// ở đầu và cuối mỗi hàng là tổng của hai giá trị
// từ hàng trước đó.

public class PascalsTriangle {
```

```
    public static void main(String[] args) {
        int[][] triangle = new int[11][];
        fillIn(triangle);
        print(triangle);
    }

    public static void fillIn(int[][] triangle) {
        for (int i = 0; i < triangle.length; i++) {
            triangle[i] = new int[i + 1];
            triangle[i][0] = 1;
            triangle[i][i] = 1;
            for (int j = 1; j < i; j++) {
                triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
            }
        }
    }

    public static void print(int[][] triangle) {
        for (int i = 0; i < triangle.length; i++) {
            for (int j = 0; j < triangle[i].length; j++) {
                System.out.print(triangle[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Chương trình này tạo ra kết quả sau:

Copy code

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
1 10 45 120 210 252 210 120 45 10 1

7.6 Mảng Pixel

Nhớ lại từ Phụ lục 3G rằng hình ảnh được lưu trữ trên máy tính dưới dạng một lưới hai chiều các điểm màu được gọi là pixel. Một trong những ứng dụng phổ biến nhất của mảng hai chiều (2D) là để thao tác với các pixel của một hình ảnh. Các ứng dụng phổ biến như Instagram cung cấp bộ lọc và tùy chọn để chỉnh sửa hình ảnh bằng cách áp dụng các thuật toán vào pixel của chúng; ví dụ, bạn có thể làm cho hình ảnh trở nên đen trắng, làm sắc nét, tăng cường màu sắc và độ tương phản, hoặc làm cho nó trông như một bức ảnh cũ bị phai màu. Tính chất hình chữ nhật hai chiều của một hình ảnh làm cho mảng 2D trở thành một cách tự nhiên để biểu diễn dữ liệu pixel của nó.

Phụ lục 3G đã giới thiệu lớp DrawingPanel mà chúng ta sử dụng để biểu diễn một cửa sổ vẽ các hình dạng và màu sắc 2D. Nhớ lại rằng một hình ảnh được tạo thành từ các pixel có vị trí được xác định bằng tọa độ số nguyên bắt đầu từ góc trên bên trái của hình ảnh tại (0, 0). Các lệnh vẽ khác nhau của đối tượng Graphics của bảng, chẳng hạn như drawRect và fillOval, thay đổi màu sắc của các vùng pixel. Màu sắc thường được chỉ định bằng các đối tượng Color, nhưng toàn bộ dải màu đến từ hỗn hợp của các phần tử đỏ, xanh lá cây và xanh dương được xác định bằng các số nguyên có giá trị từ 0 đến 255 (bao gồm cả 0 và 255). Mỗi tổ hợp của ba số nguyên này xác định một màu cụ thể và được gọi là giá trị RGB.

DrawingPanel bao gồm một số phương thức để lấy và đặt màu sắc của các pixel, được liệt kê trong Bảng 7.3. Bạn có thể tương tác với một pixel đơn lẻ, hoặc bạn có thể lấy tất cả các pixel của hình ảnh dưới dạng mảng 2D và thao tác với toàn bộ mảng. Mảng này có thứ tự hàng-đầu; tức là chỉ số đầu tiên của mảng là tọa độ y và chỉ số thứ hai là tọa độ x. Ví dụ, a[r][c] biểu thị pixel tại vị trí (x=c, y=r). Để hiệu quả, người ta thường khuyên dùng các phương thức dựa trên mảng; các phương thức pixel đơn lẻ chạy chậm khi được áp dụng lặp lại trên tất cả các pixel của một hình ảnh lớn.

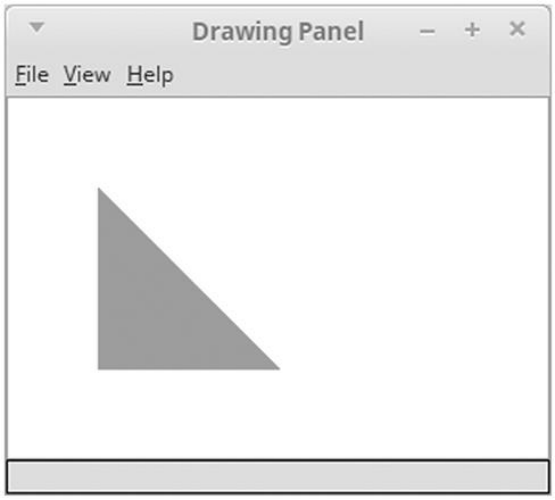
Bảng 7.3: Các phương thức của DrawingPanel liên quan đến pixel

Phương thức	Mô tả
getPixel(x, y)	Trả về màu sắc của một pixel dưới dạng đối tượng Color
getPixels()	Trả về màu sắc của tất cả các pixel dưới dạng mảng 2D các đối tượng Color, theo thứ tự hàng-đầu
setPixel(x, y, color)	Đặt màu sắc của một pixel thành màu của đối tượng Color đã cho
setPixels(pixels)	Đặt màu sắc của tất cả các pixel từ mảng 2D đã cho của các đối tượng Color, thay đổi kích thước bảng nếu cần để phù hợp với kích thước của mảng

Ví dụ chương trình DrawPurpleTriangle sau đây sử dụng getPixels và setPixels để tô một vùng hình tam giác của bảng với

màu tím. Hình 7.1 hiển thị đầu ra đồ họa của chương trình. Lưu ý rằng bạn phải gọi setPixels ở cuối để thấy hình ảnh đã được cập nhật; thay đổi mảng sẽ không tạo ra bất kỳ hiệu ứng nào trên màn hình cho đến khi bạn yêu cầu bảng cập nhật chính nó bằng nội dung mới của mảng.

Hình 7.1: Đầu ra của DrawPurpleTriangle



scss

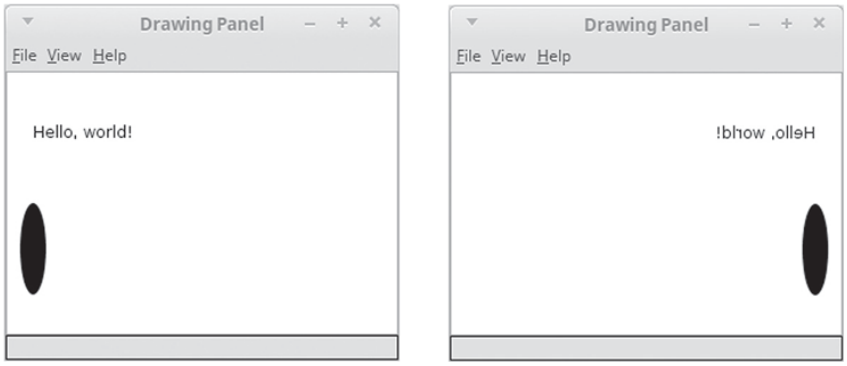
Copy code

```
// Chương trình này minh họa cách sử dụng các phương thức

// getPixels và setPixels của DrawingPanel

// để thao tác trên các điểm ảnh của một hình ảnh.


import java.awt.*;
```

```
public class DrawPurpleTriangle {

    public static void main(String[] args) {

        DrawingPanel panel = new DrawingPanel(300, 200);

        Color[][] pixels = panel.getPixels();


        for (int row = 50; row <= 150; row++) {

            for (int col = 50; col <= row; col++) {

                pixels[row][col] = Color.MAGENTA;

            }

        }


        panel.setPixels(pixels);

    }

}
```

Giải thích: Bạn có thể sử dụng getPixels và setPixels để vẽ một hình như tam giác màu tím trong chương trình này, nhưng cách sử dụng thông thường hơn của các phương thức này là để lấy trạng thái hiện tại của panel và thay đổi nó theo một cách thú vị. Chương trình Mirror sau đây minh họa việc sử dụng mảng 2D của các đối tượng Color. Phương thức mirror của chương trình

chấp nhận một tham số DrawingPanel và lật nội dung điểm ảnh theo chiều ngang, hoán đổi màu của mỗi điểm ảnh với điểm ảnh ở vị trí ngang đối diện. Đoạn mã sử dụng kích thước của mảng để đại diện cho kích thước của hình ảnh; pixels.length là chiều cao và pixels[0].length (chiều dài của hàng đầu tiên trong mảng 2D) là chiều rộng của nó. Hình 7.2 hiển thị đầu ra đồ họa của chương trình trước và sau khi gọi mirror.




```
// Chương trình này chứa một phương thức mirror

// lật ngang từng điểm ảnh của DrawingPanel.


import java.awt.*;


public class Mirror {

    public static void main(String[] args) {

        DrawingPanel panel = new DrawingPanel(300, 200);

        Graphics g = panel.getGraphics();
```

```
    g.drawString("Hello, world!", 20, 50);
    g.fillOval(10, 100, 20, 70);
    mirror(panel);
}

// Lật ngang các điểm ảnh của DrawingPanel đã cho.
public static void mirror(DrawingPanel panel) {
    Color[][] pixels = panel.getPixels();

    for (int row = 0; row < pixels.length; row++) {
        for (int col = 0; col < pixels[0].length / 2; col++) {
            // hoán đổi với điểm ảnh ở vị trí "đối xứng"
            int opposite = pixels[0].length - 1 - col;
            Color px = pixels[row][col];
            pixels[row][col] = pixels[row][opposite];
            pixels[row][opposite] = px;
        }
```

```
        }


        panel.setPixels(pixels);
    }
}
```

Giải thích: Thường bạn sẽ muốn trích xuất các thành phần đỏ, xanh lá cây, và xanh dương riêng lẻ của một màu để thao tác chúng. Mỗi đối tượng Color của điểm ảnh có các phương thức để giúp bạn thực hiện điều này. Các phương thức getRed, getGreen, và getBlue trích xuất các thành phần tương ứng từ một số nguyên RGB. Bảng 7.4 liệt kê các phương thức liên quan.

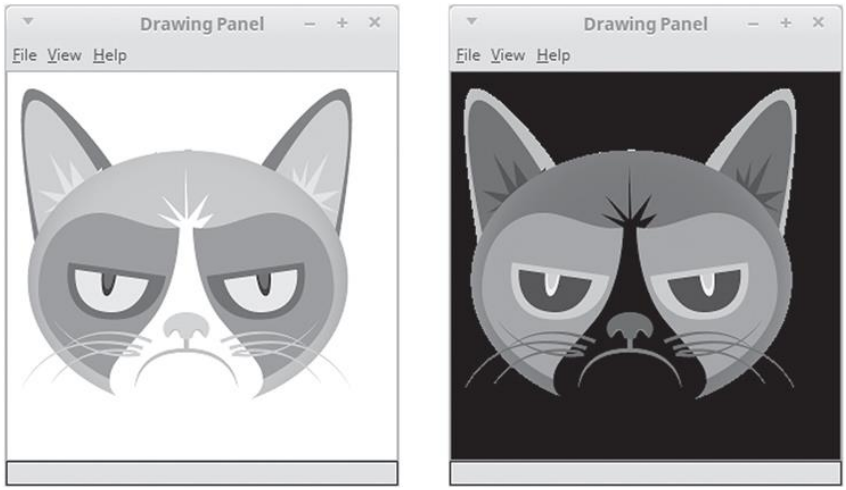
Bảng 7.4 Các phương thức của Color liên quan đến các thành phần RGB của điểm ảnh

Phương thức	Mô tả
getRed()	trả về thành phần màu đỏ từ 0-255
getGreen()	trả về thành phần màu xanh lá từ 0-255
getBlue()	trả về thành phần màu xanh dương từ 0-255

Đoạn mã sau đây cho thấy một phương thức tính toán ảnh âm bản, được tìm thấy bằng cách lấy giá trị RGB đối nghịch của mỗi màu. Ví dụ, đối nghịch của (đỏ = 255, xanh lá = 100, xanh dương = 35) là (đỏ = 0, xanh lá = 155, xanh dương = 220). Cách đơn

giản nhất để tính ảnh âm bản là trừ giá trị RGB của điểm ảnh từ giá trị màu tối đa là 255. Hình 7.3 cho thấy một ví dụ về đầu ra.

Hình 7.3 Âm bản của một hình ảnh (trước và sau khi xử lý)




```
// Tạo ảnh âm bản của hình ảnh bằng cách đảo ngược tất cả giá trị màu

// trong panel.


public static void negative(DrawingPanel panel) {

    Color[][] pixels = panel.getPixels();


    for (int row = 0; row < pixels.length; row++) {

        for (int col = 0; col < pixels[0].length; col++) {
```

```
            // trích xuất các thành phần đỏ/xanh lá/xanh dương từ 0-
255

            int r = 255 - pixels[row][col].getRed();

            int g = 255 - pixels[row][col].getGreen();

            int b = 255 - pixels[row][col].getBlue();



            // cập nhật mảng pixel với giá trị màu mới

            pixels[row][col] = new Color(r, g, b);

        }

    }



    panel.setPixels(pixels);

}
```

Giải thích: Tất cả các ví dụ trước đều liên quan đến việc thay đổi mảng điểm ảnh 2D tại chỗ. Nhưng đôi khi bạn muốn tạo một hình ảnh có kích thước khác hoặc muốn thiết lập từng điểm ảnh dựa trên các giá trị của các điểm ảnh xung quanh nó, do đó bạn cần tạo một mảng điểm ảnh mới. Ví dụ sau đây minh họa phương thức stretch mở rộng nội dung của một DrawingPanel thành gấp đôi chiều rộng hiện tại của nó. Để làm điều này, nó tạo ra một mảng mới newPixels có chiều rộng gấp đôi so với mảng hiện có. (Hãy nhớ rằng chỉ số đầu tiên của mảng 2D là y và chỉ số thứ hai

là x, vì vậy để mở rộng mảng, đoạn mã phải nhân đôi chiều thứ hai của mảng.) Phương thức setPixels sẽ thay đổi kích thước panel nếu cần để phù hợp với mảng điểm ảnh lớn hơn của chúng ta.

Vòng lặp để điền vào mảng mới đặt giá trị tại mỗi chỉ số dựa trên giá trị tại chỉ số x nhỏ hơn một nửa trong mảng gốc. Vì vậy, ví dụ, giá trị điểm ảnh của mảng gốc tại (52, 34) được sử dụng để điền vào các điểm ảnh của mảng mới tại (104, 68) và (105, 68). Hình 7.4 hiển thị đầu ra đồ họa của hình ảnh được kéo dài theo chiều ngang.

// Mở rộng panel được cho thành gấp đôi chiều rộng.

// Bất kỳ hình dạng và màu sắc nào đã vẽ trên panel cũng sẽ được kéo giãn để phù hợp.

```
public static void stretch(DrawingPanel panel) {

    Color[][] pixels = panel.getPixels();
```

```
    Color[][] newPixels = new Color[pixels.length][2 *
pixels[0].length];



    for (int row = 0; row < pixels.length; row++) {

        for (int col = 0; col < 2 * pixels[0].length; col++) {

            newPixels[row][col] = pixels[row][col / 2];

        }

    }



    panel.setPixels(newPixels);

}
```

Giải thích: Các phương thức thao tác với điểm ảnh được hiển thị trong phần này khá kém hiệu quả vì chúng tạo ra các mảng lớn của đối tượng Color, điều này tốn nhiều thời gian và bộ nhớ. Những phương thức này không đủ hiệu quả để sử dụng trong hoạt hình hoặc trò chơi. DrawingPanel cung cấp một số phương thức bổ sung như getPixelsRGB sử dụng các số nguyên được đóng gói đặc biệt để đại diện cho thông tin màu đỏ, xanh lá, và xanh dương thay vì các đối tượng Color, nhằm cải thiện tốc độ và sử dụng bộ nhớ với chi phí là tăng độ phức tạp của mã. Nếu bạn quan tâm, bạn có thể đọc về những phương thức bổ sung này trong tài liệu trực tuyến của DrawingPanel tại buildingjavaprograms.com.

7.7 Nghiên cứu trường hợp: Luật Benford

Hãy xem xét một ví dụ chương trình phức tạp hơn liên quan đến việc sử dụng mảng. Khi nghiên cứu dữ liệu thực tế, bạn thường gặp một kết quả kỳ lạ được biết đến với tên gọi Luật Benford, được đặt theo tên nhà vật lý Frank Benford, người đã phát biểu về nó vào năm 1938.

Luật Benford liên quan đến việc xem xét chữ số đầu tiên của một loạt các con số. Ví dụ, giả sử bạn sử dụng trình tạo số ngẫu nhiên để tạo ra các số nguyên trong khoảng từ 100 đến 999 và bạn xem xét tần suất các số bắt đầu bằng 1, tần suất các số bắt đầu bằng 2, và cứ như vậy. Bất kỳ trình tạo số ngẫu nhiên nào đủ tốt sẽ phân bố các câu trả lời đều nhau giữa chín vùng khác nhau, vì vậy chúng ta sẽ kỳ vọng thấy mỗi chữ số xuất hiện khoảng một phần chín thời gian (11,1%). Nhưng với nhiều dữ liệu thực tế, chúng ta thấy một phân bố rất khác.

Khi xem xét dữ liệu phù hợp với phân bố Benford, chúng ta thấy chữ số đầu tiên là 1 xuất hiện hơn 30% thời gian (gần một phần ba) và ở cực ngược lại, chữ số đầu tiên là 9 chỉ xuất hiện khoảng 4,6% thời gian (ít hơn một trong hai mươi trường hợp). Bảng 7.5 hiển thị phân bố dự kiến cho dữ liệu tuân theo Luật Benford.

Bảng 7.5 Phân bố dự kiến theo Luật Benford

Chữ số đầu tiên	Tần suất
1	30,1%
2	17,6%

Chữ số đầu tiên	Tần suất
3	12,5%
4	9,7%
5	7,9%
6	6,7%
7	5,8%
8	5,1%
9	4,6%

Tại sao phân bố lại như vậy? Tại sao có nhiều số 1 đến vậy? Tại sao lại có ít số 9? Câu trả lời là các dãy số mũ có đặc tính khác so với các dãy tuyến tính đơn giản. Đặc biệt, các dãy số mũ có nhiều số bắt đầu bằng 1 hơn.

Để khám phá hiện tượng này, hãy xem xét hai dãy số khác nhau: một dãy tăng tuyến tính và một dãy tăng theo số mũ. Nếu bạn bắt đầu với số 1 và cộng thêm 0,2 vào nó lặp đi lặp lại, bạn sẽ nhận được dãy tuyến tính sau: 1, 1.2, 1.4, 1.6, 1.8, 2, 2.2, 2.4, 2.6, 2.8, 3, 3.2, 3.4, 3.6, 3.8, 4, 4.2, 4.4, 4.6, 4.8, 5, 5.2, 5.4, 5.6, 5.8, 6, 6.2, 6.4, 6.6, 6.8, 7, 7.2, 7.4, 7.6, 7.8, 8, 8.2, 8.4, 8.6, 8.8, 9, 9.2, 9.4, 9.6, 9.8, 10

Trong dãy này có năm số bắt đầu bằng 1, năm số bắt đầu bằng 2, năm số bắt đầu bằng 3, và cứ thế. Đối với mỗi chữ số, có năm số bắt đầu bằng chữ số đó. Đó là điều chúng ta kỳ vọng sẽ thấy khi dữ liệu tăng lên theo một lượng không đổi mỗi lần.

Nhưng hãy xem xét điều gì xảy ra khi chúng ta biến nó thành một dãy số mũ. Hãy bắt đầu với số 1 và tiếp tục cho đến khi đạt đến số 10, nhưng lần này hãy nhân mỗi số kế tiếp với 1.05 (chúng ta sẽ chỉ hiển thị hai chữ số sau dấu thập phân, nhưng dãy thực tế sẽ tính toán tất cả các chữ số): 1.00, 1.05, 1.10, 1.16, 1.22, 1.28, 1.34, 1.41, 1.48, 1.55, 1.63, 1.71, 1.80, 1.89, 1.98, 2.08, 2.18, 2.29, 2.41, 2.53, 2.65, 2.79, 2.93, 3.07, 3.23, 3.39, 3.56, 3.73, 3.92, 4.12, 4.32, 4.54, 4.76, 5.00, 5.25, 5.52, 5.79, 6.08, 6.39, 6.70, 7.04, 7.39, 7.76, 8.15, 8.56, 8.99, 9.43, 9.91, 10.40

Trong dãy này có 15 số bắt đầu bằng 1 (31,25%), 8 số bắt đầu bằng 2 (16,7%), và cứ thế. Chỉ có 2 số bắt đầu bằng 9 (4,2%). Thực tế, phân bố các chữ số gần như chính xác với những gì bạn thấy trong bảng Luật Benford.

Có nhiều hiện tượng thực tế thể hiện tính chất số mũ. Ví dụ, dân số có xu hướng tăng theo số mũ ở hầu hết các khu vực. Có nhiều bộ dữ liệu khác cũng dường như thể hiện mô hình Benford, bao gồm các vết đen mặt trời, lương bổng, đầu tư, chiều cao của các tòa nhà, v.v. Luật Benford đã được sử dụng để cố gắng phát hiện gian lận kế toán dựa trên lý thuyết rằng khi ai đó tạo ra dữ liệu, họ có thể sử dụng một quy trình ngẫu nhiên hơn mà không tạo ra một phân bố theo kiểu Benford.

Đối với mục đích của chúng ta, hãy viết một chương trình đọc một tệp chứa các số nguyên và hiển thị phân bố của chữ số đầu tiên. Chúng ta sẽ đọc dữ liệu từ một tệp và chạy nó trên nhiều đầu vào mẫu. Tuy nhiên, trước tiên, hãy xem xét vấn đề tổng quát về việc thống kê.

Đếm giá trị

Trong lập trình, chúng ta thường cần đếm số lần xuất hiện của một tập hợp các giá trị. Ví dụ, chúng ta có thể muốn biết bao nhiêu người đạt điểm 100 trong một kỳ thi, bao nhiêu người đạt

điểm 99, bao nhiêu người đạt điểm 98, và cứ thế. Hoặc chúng ta có thể muốn biết có bao nhiêu ngày nhiệt độ trong một thành phố vượt quá 100 độ, bao nhiêu ngày nhiệt độ nằm trong khoảng 90 độ, bao nhiêu ngày nhiệt độ nằm trong khoảng 80 độ, v.v. Cách tiếp cận là gần như giống nhau cho mỗi nhiệm vụ đếm này. Hãy cùng xem xét một nhiệm vụ đếm nhỏ trong đó chỉ có năm giá trị cần đếm.

Giả sử rằng một giáo viên chấm điểm bài kiểm tra theo thang điểm từ 0 đến 4 và muốn biết phân bố điểm bài kiểm tra. Nói cách khác, giáo viên muốn biết có bao nhiêu điểm 0, bao nhiêu điểm 1, bao nhiêu điểm 2, bao nhiêu điểm 3, và bao nhiêu điểm 4. Giả sử rằng giáo viên đã ghi lại tất cả các điểm trong một tệp dữ liệu như sau:

Copy code

```
1 4 1 0 3 2 1 4 2 0
3 0 2 3 0 4 3 3 4 1
2 4 1 3 1 4 3 3 2 4
2 3 0 4 1 4 4 1 4 1
```

Giáo viên có thể tự đếm các điểm số, nhưng sẽ dễ dàng hơn nhiều nếu sử dụng máy tính để thực hiện việc đếm. Làm thế nào để bạn giải quyết vấn đề này? Trước tiên, bạn phải nhận ra rằng bạn đang thực hiện năm nhiệm vụ đếm riêng biệt: Bạn đang đếm số lần xuất hiện của số 0, số 1, số 2, số 3, và số 4. Bạn sẽ cần năm bộ đếm để giải quyết vấn đề này, điều đó có nghĩa là một mảng là một cách tuyệt vời để lưu trữ dữ liệu. Nói chung, bất cứ khi nào bạn nghĩ rằng bạn cần n loại dữ liệu nào đó, bạn nên nghĩ đến việc sử dụng một mảng có độ dài n.

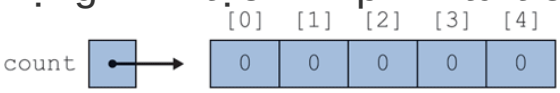
Mỗi bộ đếm sẽ là một số nguyên (int), vì vậy bạn cần một mảng gồm năm giá trị int:

java

Copy code

```
int[] count = new int[5];
```

Dòng mã này sẽ phân bổ một mảng gồm năm số nguyên và sẽ tự động khởi tạo mỗi phần tử trong mảng thành 0.



Bạn đang đọc dữ liệu từ một tệp, vì vậy bạn sẽ cần một đối tượng Scanner và một vòng lặp để đọc các điểm số cho đến khi không còn điểm nào để đọc:

java
Copy code

```
Scanner input = new Scanner(new File("tally.dat"));
while (input.hasNextInt()) {
    int next = input.nextInt();
    // xử lý next
}
```

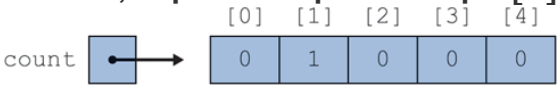
Để hoàn thiện đoạn mã này, bạn cần xác định cách xử lý từng giá trị. Bạn biết rằng next sẽ là một trong năm giá trị khác nhau: 0, 1, 2, 3, hoặc 4. Nếu nó là 0, bạn muốn tăng bộ đếm cho 0, tức là count[0]; nếu nó là 1, bạn muốn tăng bộ đếm cho 1, tức là count[1], và cứ thế. Chúng ta đã giải quyết các vấn đề tương tự như thế này bằng các câu lệnh if/else lồng nhau:

java
Copy code

```
if (next == 0) {
    count[0]++;
} else if (next == 1) {
    count[1]++;
} else if (next == 2) {
    count[2]++;
} else if (next == 3) {
    count[3]++;
} else { // next == 4
    count[4]++;
}
```

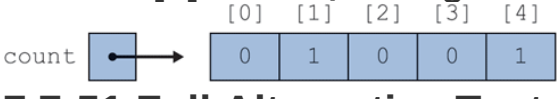
Nhưng với một mảng, bạn có thể giải quyết vấn đề này một cách trực tiếp hơn:
java
Copy code
count[next]++;
Dòng mã này ngắn gọn đến mức bạn có thể không nhận ra ngay rằng nó thực hiện cùng một việc như cấu trúc if/else lồng nhau. Hãy mô phỏng chính xác những gì xảy ra khi các giá trị khác nhau được đọc từ tệp.
Khi mảng được tạo ra, tất cả các bộ đếm đều được khởi tạo về 0:
java
Copy code
int[] count = new int[5];

Giá trị đầu tiên trong tệp đầu vào là 1, vì vậy chương trình sẽ đọc giá trị đó vào biến next. Sau đó, nó thực thi dòng mã sau:
java
Copy code
count[next]++;
Vì next là 1, dòng mã này trở thành:
java
Copy code
count[1]++;
Do đó, bộ đếm tại chỉ mục [1] sẽ được tăng thêm:



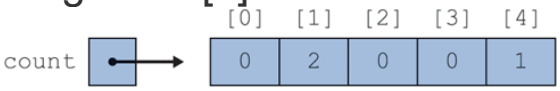
7.7-50 Full Alternative Text

Sau đó, một giá trị 4 được đọc từ tệp đầu vào, điều này có nghĩa là count[4] sẽ được tăng thêm:



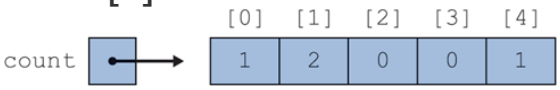
7.7-51 Full Alternative Text

Tiếp theo, một giá trị 1 khác được đọc từ tệp đầu vào, điều này tăng count[1]:



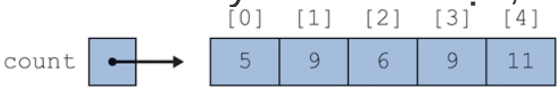
7.7-52 Full Alternative Text

Sau đó, một giá trị 0 được đọc từ tệp đầu vào, điều này tăng count[0]:



7.7-53 Full Alternative Text

Lưu ý rằng trong bộ dữ liệu ngắn này, bạn đã nhảy từ chỉ mục 1 đến chỉ mục 4, sau đó quay lại chỉ mục 1, rồi đến chỉ mục 0. Chương trình tiếp tục thực thi theo cách này, nhảy từ bộ đếm này sang bộ đếm khác khi đọc các giá trị từ tệp. Khả năng nhảy xung quanh trong cấu trúc dữ liệu này được gọi là truy cập ngẫu nhiên. Sau khi xử lý tất cả dữ liệu, mảng sẽ trông như sau:



7.7-54 Full Alternative Text

Sau khi vòng lặp này hoàn tất, bạn có thể báo cáo tổng số cho từng điểm số bằng cách sử dụng vòng lặp duyệt thông thường với lệnh println:

java

Copy code

```
for (int i = 0; i < count.length; i++) {
    System.out.println(i + "\t" + count[i]);
}
```

Với việc thêm tiêu đề cho đầu ra, chương trình hoàn chỉnh như sau:

java

Copy code

```
1  // Đọc một loạt giá trị và báo cáo tần suất xuất hiện của từng giá trị.
2
3
```

```
4  import java.io.*;
5  import java.util.*;
6
7  public class Tally {
8      public static void main(String[] args)
9          throws FileNotFoundException {
10         Scanner input = new Scanner(new File("tally.dat"));
11         int[] count = new int[5];
12         while (input.hasNextInt()) {
13             int next = input.nextInt();
14             count[next]++;
15         }
16         System.out.println("Giá trị\tSố lần xuất hiện");
17         for (int i = 0; i < count.length; i++) {
18             System.out.println(i + "\t" + count[i]);
19         }
20     }
21 }
```

Với tệp đầu vào mẫu được hiển thị trước đó, chương trình này sẽ tạo ra đầu ra sau:

bash

Copy code

```
Giá trị   Số lần xuất hiện
0         5
1         9
2         6
3         9
4         11
```

Điều quan trọng cần nhận ra là một chương trình được viết bằng mảng linh hoạt hơn nhiều so với các chương trình được viết bằng các biến đơn giản và câu lệnh if/else. Ví dụ, giả sử bạn muốn điều chỉnh chương trình này để xử lý một tệp đầu vào với các

điểm thi từ 0 đến 100. Thay đổi duy nhất bạn cần thực hiện là cấp phát một mảng lớn hơn:
java
Copy code
```
int[] count = new int[101];
```
Nếu bạn viết chương trình bằng cách tiếp cận if/else, bạn sẽ phải thêm 96 nhánh mới để xử lý phạm vi giá trị mới. Khi bạn sử dụng giải pháp mảng, bạn chỉ cần điều chỉnh kích thước tổng thể của mảng. Lưu ý rằng kích thước của mảng lớn hơn một so với điểm số cao nhất (101 thay vì 100) vì mảng được đánh chỉ số từ 0 và vì bạn có thể thực sự đạt được 101 điểm khác nhau trong bài kiểm tra, bao gồm cả 0.

Hoàn thành chương trình
Bây giờ chúng ta đã khám phá cách tiếp cận cơ bản để đếm số lần xuất hiện, chúng ta có thể dễ dàng thích ứng nó với bài toán phân tích một tệp dữ liệu để tìm sự phân bố của các chữ số đầu tiên. Như đã đề cập trước đó, chúng ta giả định rằng có một tệp chứa các số nguyên. Để đếm các chữ số đầu tiên, chúng ta cần lấy được chữ số đầu tiên của mỗi số. Nhiệm vụ này đủ chuyên biệt để xứng đáng có một phương thức riêng.
Vì vậy, hãy viết một phương thức tên là firstDigit trả về chữ số đầu tiên của một số nguyên. Nếu số đó là một chữ số, thì số đó sẽ là kết quả. Nếu số đó không phải là một chữ số, chúng ta có thể bỏ đi chữ số cuối cùng vì không cần đến nó. Nếu chúng ta thực hiện việc này trong một vòng lặp, cuối cùng sẽ còn lại một số một chữ số (chữ số đầu tiên). Điều này dẫn đến việc viết vòng lặp sau:
java
Copy code
```
while (result >= 10) {
    result = result / 10;
}
```

Chúng ta không mong đợi có số âm, nhưng cũng không phải ý tồi khi đảm bảo rằng không có số âm nào. Vậy nên khi đưa điều này vào một phương thức xử lý cả số âm, chúng ta có mã sau:
java
Copy code
```
public static int firstDigit(int n) {
    int result = Math.abs(n);
    while (result >= 10) {
        result = result / 10;
    }
    return result;
}
```
Trong phần trước, chúng ta đã khám phá cách tiếp cận chung để đếm số. Trong trường hợp này, chúng ta muốn đếm các chữ số từ 0 đến 9, nên chúng ta cần một mảng có độ dài 10. Giải pháp khác thì gần giống như chúng ta đã làm ở phần trước. Chúng ta có thể đưa mã đếm vào một phương thức để xây dựng mảng và trả về kết quả đếm:
java
Copy code
```
public static int[] countDigits(Scanner input) {
    int[] count = new int[10];
    while (input.hasNextInt()) {
        int n = input.nextInt();
        count[firstDigit(n)]++;
    }
    return count;
}
```
Lưu ý rằng thay vì đếm n trong thân vòng lặp, chúng ta thay vào đó đếm firstDigit(n) (chỉ chữ số đầu tiên, không phải toàn bộ số). Giá trị 0 có thể gây ra một vấn đề cho chúng ta. Luật Benford áp dụng cho dữ liệu đến từ một chuỗi số mũ. Nhưng ngay cả khi bạn tăng theo cấp số nhân, nếu bạn bắt đầu từ 0, bạn sẽ không bao

giờ vượt quá 0. Do đó, tốt nhất là loại bỏ các giá trị 0 khỏi tính toán. Thông thường chúng sẽ không xuất hiện chút nào.
Khi báo cáo kết quả, hãy bắt đầu bằng việc báo cáo các giá trị 0 bị loại bỏ nếu chúng tồn tại:
java
Copy code
```
if (count[0] > 0) {
    System.out.println("loại trừ " + count[0] + " số 0");
}
```
Với các chữ số khác, chúng ta muốn báo cáo số lần xuất hiện của mỗi chữ số và tỷ lệ phần trăm của từng chữ số. Để tính tỷ lệ phần trăm, chúng ta cần biết tổng số các giá trị. Đây là lúc thích hợp để giới thiệu một phương thức tính tổng của một mảng số nguyên. Đây là một bài toán duyệt mảng khá đơn giản có thể được giải quyết bằng vòng lặp for-each:
java
Copy code
```
public static int sum(int[] data) {
    int sum = 0;
    for (int n : data) {
        sum += n;
    }
    return sum;
}
```
Bây giờ chúng ta có thể tính tổng số các chữ số bằng cách gọi phương thức và trừ đi số lượng số 0:
java
Copy code
```
int total = sum(count) - count[0];
```
Và một khi chúng ta có tổng số chữ số, chúng ta có thể viết một vòng lặp để báo cáo tỷ lệ phần trăm của từng chữ số. Để tính tỷ lệ phần trăm, chúng ta nhân mỗi kết quả đếm với 100 và chia cho tổng số chữ số. Chúng ta phải cẩn thận nhân với 100.0 thay vì

100 để đảm bảo rằng kết quả tính toán được thực hiện bằng giá trị double. Nếu không, chúng ta sẽ nhận được kết quả bị cắt ngắn từ phép chia số nguyên và không có chữ số sau dấu thập phân:
java
Copy code
```
for (int i = 1; i < count.length; i++) {
    double pct = count[i] * 100.0 / total;
    System.out.println(i + " " + count[i] + " " + pct);
}
```
Lưu ý rằng vòng lặp bắt đầu từ 1 thay vì 0 vì chúng ta đã loại trừ các số 0 khỏi báo cáo.
Dưới đây là một chương trình hoàn chỉnh kết hợp các phần này lại với nhau. Nó cũng sử dụng câu lệnh printf để định dạng đầu ra và bao gồm một tiêu đề cho bảng và tổng số sau đó:
java
Copy code
```
1  // Chương trình này tìm sự phân bố của các chữ số đầu tiên trong một tập hợp
2  // các số nguyên dương. Chương trình hữu ích để khám phá hiện tượng
3  // được biết đến với tên gọi Luật Benford.
4
5  import java.io.*;
6  import java.util.*;
7
8  public class Benford {
9      public static void main(String[] args)
10             throws FileNotFoundException {
11         Scanner console = new Scanner(System.in);
12         System.out.println("Hãy đếm các chữ số đầu tiên...");
13         System.out.print("Tên tệp đầu vào? ");
14         String name = console.nextLine();
15         Scanner input = new Scanner(new File(name));
```

```
16          int[] count = countDigits(input);
17          reportResults(count);
18      }
19
20      // Đọc các số nguyên từ đầu vào, tính một mảng số lượng
21      // các lần xuất hiện của mỗi chữ số đầu tiên (0–9).
22      public static int[] countDigits(Scanner input) {
23          int[] count = new int[10];
24          while (input.hasNextInt()) {
25              int n = input.nextInt();
26              count[firstDigit(n)]++;
27          }
28          return count;
29      }
30
31      // Báo cáo tỷ lệ phần trăm cho mỗi chữ số đầu tiên, loại trừ
số 0
32      public static void reportResults(int[] count) {
33          System.out.println();
34          if (count[0] > 0) {
35              System.out.println("loại trừ " + count[0] + " số 0");
36          }
37          int total = sum(count) - count[0];
38          System.out.println("Chữ số  Số lần   Tỷ lệ (%)");
39          for (int i = 1; i < count.length; i++) {
40              double pct = count[i] * 100.0 / total;
41              System.out.printf("%5d %7d %8.2f\n", i, count[i], pct);
42          }
43          System.out.printf("Tổng %7d %8.2f\n", total, 100.0);
44      }
45
46      // trả về tổng các số nguyên trong mảng đã cho
47      public static int sum(int[] data) {
```

```
48          int sum = 0;
49          for (int n : data) {
50              sum += n;
51          }
52          return sum;
53      }
54
55      // trả về chữ số đầu tiên của số đã cho
56      public static int firstDigit(int n) {
57          int result = Math.abs(n);
58          while (result >= 10) {
59              result = result / 10;
60          }
61          return result;
62      }
63 }
```

Tóm tắt chương

Mảng là một đối tượng nhóm các giá trị nguyên thủy hoặc các đối tượng cùng loại dưới một tên duy nhất. Mỗi giá trị riêng lẻ, được gọi là phần tử, được truy cập bằng chỉ số nguyên bắt đầu từ 0 đến một giá trị nhỏ hơn độ dài của mảng.

Cố gắng truy cập một phần tử của mảng bằng một chỉ số nhỏ hơn 0 hoặc lớn hơn hoặc bằng độ dài của mảng sẽ khiến chương trình gặp lỗi ArrayIndexOutOfBoundsException.

Mảng thường được duyệt qua bằng cách sử dụng vòng lặp for. Độ dài của một mảng được tìm bằng cách truy cập vào trường length, vì vậy vòng lặp qua một mảng có thể xử lý các chỉ số từ 0 đến length - 1. Các phần tử của mảng cũng có thể được truy cập theo thứ tự bằng cách sử dụng một loại vòng lặp gọi là for-each.

Mảng có một số hạn chế, chẳng hạn như kích thước cố định và thiếu hỗ trợ cho các thao tác phổ biến như == và println. Để thực hiện các thao tác này, bạn phải sử dụng lớp Arrays hoặc viết các vòng lặp for xử lý từng phần tử của mảng.

Một số thuật toán mảng phổ biến, chẳng hạn như in một mảng hoặc so sánh hai mảng với nhau để kiểm tra sự bằng nhau, được thực hiện bằng cách duyệt qua các phần tử và kiểm tra hoặc sửa đổi từng phần tử.
Mảng trong Java là đối tượng và sử dụng ngữ nghĩa tham chiếu, trong đó các biến lưu trữ tham chiếu tới các giá trị thay vì chính các giá trị đó. Điều này có nghĩa là hai biến có thể tham chiếu đến cùng một mảng hoặc đối tượng. Nếu mảng bị sửa đổi thông qua một trong các tham chiếu của nó, sự sửa đổi cũng sẽ được thấy ở tham chiếu khác.
Mảng các đối tượng thực ra là mảng của các tham chiếu tới các đối tượng. Một mảng đối tượng mới được khai báo và khởi tạo thực sự lưu trữ null trong tất cả các chỉ số phần tử của nó, vì vậy mỗi phần tử phải được khởi tạo riêng lẻ hoặc trong một vòng lặp để lưu trữ một đối tượng thực sự.
Mảng đa chiều là mảng của các mảng. Chúng thường được sử dụng để lưu trữ dữ liệu hai chiều, chẳng hạn như dữ liệu trong các hàng và cột hoặc dữ liệu xy trong một không gian hai chiều.

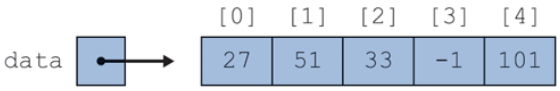
Các vấn đề tự kiểm tra

Phần 7.1: Kiến thức cơ bản về mảng

1. Trong các cú pháp sau đây, cú pháp nào đúng để khai báo một mảng chứa mười số nguyên?
    - a. int a[10] = new int[10];
    - b. int[10] a = new int[10];
    - c. []int a = [10]int;
    - d. int a[10];
    - e. int[] a = new int[10];
2. Biểu thức nào nên được sử dụng để truy cập phần tử đầu tiên của một mảng số nguyên có tên numbers? Biểu thức nào nên được sử dụng để truy cập phần tử cuối cùng của numbers, giả sử nó chứa 10 phần tử? Biểu thức nào có thể được sử dụng để truy cập phần tử cuối cùng của nó, bất kể độ dài của nó?

3. Viết mã tạo một mảng số nguyên có tên data với kích thước là 5 và có nội dung như sau:

java
Copy code
```
int[] data = {2, 4, 6, 8, 10};
```
4o



7.2-55 Văn bản thay thế đầy đủ

1. Viết mã lưu trữ tất cả các số lẻ từ -6 đến 38 vào một mảng bằng cách sử dụng vòng lặp. Kích thước của mảng cần vừa đủ để lưu trữ các số này. Sau đó, hãy thử tổng quát hóa mã của bạn để nó có thể hoạt động với bất kỳ giá trị tối thiểu và tối đa nào, không chỉ là -6 và 38.
2. Các phần tử của mảng numbers chứa những gì sau khi mã sau được thực thi?

java
Copy code
```
int[] numbers = new int[8];
numbers[1] = 4;
numbers[4] = 99;
numbers[7] = 2;
int x = numbers[1];
numbers[x] = 44;
numbers[numbers[7]] = 11; // sử dụng numbers[7] làm chỉ mục
```
3. Các phần tử của mảng data chứa những gì sau khi mã sau được thực thi?

java
Copy code
```
int[] data = new int[8];
data[0] = 3;
data[7] = -18;
```

```
data[4] = 5;
data[1] = data[0];
int x = data[4];
data[4] = 6;
data[x] = data[0] * data[1];
```

4. Có gì sai với mã sau?

```
java
Copy code
int[] first = new int[2];
first[0] = 3;
first[1] = 7;
int[] second = new int[2];
second[0] = 3;
second[1] = 7;

// in các phần tử của mảng
System.out.println(first);
System.out.println(second);

// kiểm tra xem các phần tử có giống nhau không
if (first == second) {
    System.out.println("Chúng chứa cùng các phần tử.");
} else {
    System.out.println("Các phần tử khác nhau.");
}
```

5. Trong các cú pháp sau, cú pháp nào đúng để khai báo một mảng chứa sáu giá trị số nguyên đã cho?
    - a. int[] a = {17, -3, 42, 5, 9, 28};
    - b. int a {17, -3, 42, 5, 9, 28};
    - c. int[] a = new int[6] {17, -3, 42, 5, 9, 28};
    - d. int[6] a = {17, -3, 42, 5, 9, 28};
    - e. int[] a = int [17, -3, 42, 5, 9, 28] {6};

6. Viết một đoạn mã khai báo một mảng có tên data với các phần tử 7, -1, 13, 24, và 6. Sử dụng chỉ một câu lệnh để khởi tạo mảng.
7. Viết một đoạn mã kiểm tra một mảng số nguyên và báo cáo giá trị lớn nhất trong mảng. Hãy cân nhắc đặt mã của bạn vào một phương thức có tên là max chấp nhận mảng làm tham số và trả về giá trị lớn nhất. Giả sử mảng chứa ít nhất một phần tử.
8. Viết một phương thức có tên average tính toán trung bình cộng (trung bình số học) của tất cả các phần tử trong một mảng số nguyên và trả về kết quả dưới dạng số thực (double). Ví dụ, nếu mảng chứa các giá trị [1, -2, 4, -4, 9, -6, 16, -8, 25, -10], trung bình tính được sẽ là 2.5. Phương thức của bạn chấp nhận mảng số nguyên làm tham số và trả về giá trị trung bình.

Phần 7.2: Thuật toán duyệt mảng

12. Duyệt mảng là gì? Hãy đưa ra một ví dụ về một vấn đề có thể được giải quyết bằng cách duyệt mảng.
13. Viết mã sử dụng vòng lặp for để in từng phần tử của mảng data chứa năm số nguyên:

- phần tử [0] là 14
- phần tử [1] là 5
- phần tử [2] là 27
- phần tử [3] là −3
- phần tử [4] là 2598 Hãy cân nhắc tổng quát hóa mã của bạn để nó có thể hoạt động trên một mảng với bất kỳ kích thước nào.

14. Các phần tử của mảng list chứa những gì sau khi mã sau được thực thi?

java
Copy code
```
int[] list = {2, 18, 6, −4, 5, 1};
for (int i = 0; i < list.length; i++) {
```

```
    list[i] = list[i] + (list[i] / list[0]);
}
```

15. Viết một đoạn mã in một mảng số nguyên theo thứ tự ngược lại, theo cùng định dạng như phương thức print từ Phần 7.2. Hãy cân nhắc đặt mã của bạn vào một phương thức có tên printBackwards chấp nhận mảng làm tham số.
16. Mô tả các sửa đổi cần thiết để thay đổi các phương thức count và equals đã phát triển trong Phần 7.2 để xử lý các mảng chuỗi (String) thay vì các mảng số nguyên.
17. Viết một phương thức có tên allLess chấp nhận hai mảng số nguyên và trả về true nếu mỗi phần tử trong mảng thứ nhất nhỏ hơn phần tử tại cùng chỉ số trong mảng thứ hai. Phương thức của bạn nên trả về false nếu các mảng không có cùng độ dài.

Mục 7.3: Tham khảo Semantics

18. Tại sao một phương thức hoán đổi hai phần tử trong mảng hoạt động đúng trong khi phương thức hoán đổi hai giá trị số nguyên thì không?

19. Kết quả của chương trình sau là gì?

java

Copy code

```
public class ReferenceMystery1 {
    public static void main(String[] args) {
        int x = 0;
        int[] a = new int[4];
        x = x + 1;
        mystery(x, a);
        System.out.println(x + " " + Arrays.toString(a));
        x = x + 1;
        mystery(x, a);
        System.out.println(x + " " + Arrays.toString(a));
    }

    public static void mystery(int x, int[] a) {
```

```
        x = x + 1;
        a[x] = a[x] + 1;
        System.out.println(x + " " + Arrays.toString(a));
    }
}
```

20. Kết quả của chương trình sau là gì?

java

Copy code

```
public class ReferenceMystery2 {
    public static void main(String[] args) {
        int x = 1;
        int[] a = new int[2];
        mystery(x, a);
        System.out.println(x + " " + Arrays.toString(a));
        x--;
        a[1] = a.length;
        mystery(x, a);
        System.out.println(x + " " + Arrays.toString(a));
    }

    public static void mystery(int x, int[] list) {
        list[x]++;
        x++;
        System.out.println(x + " " + Arrays.toString(list));
    }
}
```

21. Viết một phương thức tên là swapPairs nhận vào một mảng số nguyên và hoán đổi các phần tử ở các chỉ mục liền kề. Ví dụ, mảng [10, 20, 30, 40, 50] sẽ trở thành [20, 10, 40, 30, 50] sau khi gọi phương thức của bạn.

Mục 7.4: Kỹ thuật Mảng Nâng cao

22. Giá trị của các phần tử trong mảng numbers sau khi đoạn mã sau được thực thi là gì?

java

Copy code

```
int[] numbers = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
for (int i = 0; i < 9; i++) {
    numbers[i] = numbers[i + 1];
}
```

23. Giá trị của các phần tử trong mảng numbers sau khi đoạn mã sau được thực thi là gì?

java

Copy code

```
int[] numbers = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
for (int i = 1; i < 10; i++) {
    numbers[i] = numbers[i - 1];
}
```

24. Xem xét phương thức sau, mystery:

java

Copy code

```
public static void mystery(int[] a, int[] b) {
    for (int i = 0; i < a.length; i++) {
        a[i] += b[b.length - 1 - i];
    }
}
```

Giá trị của các phần tử trong mảng a1 sau khi đoạn mã sau được thực thi là gì?

java

Copy code

```
int[] a1 = {1, 3, 5, 7, 9};
int[] a2 = {1, 4, 9, 16, 25};
mystery(a1, a2);
```

25. Xem xét phương thức sau, mystery2:

java

Copy code

```
public static void mystery2(int[] a, int[] b) {
    for (int i = 0; i < a.length; i++) {
        a[i] = a[2 * i % a.length] - b[3 * i % b.length];
```

```
    }
}
```

Giá trị của các phần tử trong mảng a1 sau khi đoạn mã sau được thực thi là gì?

java

Copy code

```
int[] a1 = {2, 4, 6, 8, 10, 12, 14, 16};
int[] a2 = {1, 1, 2, 3, 5, 8, 13, 21};
mystery2(a1, a2);
```

26. Xem xét phương thức sau, mystery3:

java

Copy code

```
public static void mystery3(int[] data, int x, int y) {
    data[data[x]] = data[y];
    data[y] = x;
}
```

Giá trị của các phần tử trong mảng numbers sau khi đoạn mã sau được thực thi là gì?

java

Copy code

```
int[] numbers = {3, 7, 1, 0, 25, 4, 18, -1, 5};
mystery3(numbers, 3, 1);
mystery3(numbers, 5, 6);
mystery3(numbers, 8, 4);
```

27. Xem xét phương thức sau:

java

Copy code

```
public static int mystery4(int[] list) {
    int x = 0;
    for (int i = 1; i < list.length; i++) {
        int y = list[i] - list[0];
        if (y > x) {
            x = y;
        }
    }
```

```
    return x;
}
```

Giá trị mà phương thức trả về khi nhận vào các mảng sau là gì? a. {5} b. {3, 12} c. {4, 2, 10, 8} d. {1, 9, 3, 5, 7} e. {8, 2, 10, 4, 10, 9}

28. Xem xét phương thức sau:

java

Copy code

```
public static void mystery5(int[] nums) {
    for (int i = 0; i < nums.length - 1; i++) {
        if (nums[i] > nums[i + 1]) {
            nums[i + 1]++;
        }
    }
}
```

Nội dung cuối cùng của từng mảng sau nếu mỗi mảng được truyền vào phương thức trên là gì? a. {8} b. {14, 7} c. {7, 1, 3, 2, 0, 4} d. {10, 8, 9, 5, 5} e. {12, 11, 10, 10, 8, 7}

29. Viết một đoạn mã tính độ dài trung bình của các chuỗi trong một mảng chuỗi. Ví dụ, nếu mảng chứa {"belt", "hat", "jelly", "bubble gum"}, độ dài trung bình là 5.5.

30. Viết mã chấp nhận một mảng chuỗi làm tham số và xác định xem mảng đó có phải là chuỗi palindrome hay không—tức là mảng đó có đọc giống nhau từ trước ra sau và từ sau ra trước hay không. Ví dụ, mảng {"alpha", "beta", "gamma", "delta", "gamma", "beta", "alpha"} là một chuỗi palindrome.

Mục 7.5: Mảng Đa Chiều

31. Các phần tử trong mảng numbers chứa gì sau khi đoạn mã sau được thực thi?

java

Copy code

```
int[][] numbers = new int[3][4];
for (int r = 0; r < numbers.length; r++) {
    for (int c = 0; c < numbers[0].length; c++) {
        numbers[r][c] = r + c;
    }
}
```

32. Giả sử một mảng hai chiều hình chữ nhật của các số nguyên tên là data đã được khai báo với bốn hàng và bảy cột. Viết một vòng lặp để khởi tạo hàng thứ ba của data với các số từ 1 đến 7.

33. Viết một đoạn mã tạo ra một mảng hai chiều của các số nguyên với 5 hàng và 10 cột. Điền vào mảng bằng một bảng nhân, sao cho phần tử mảng [i][j] chứa giá trị i * j. Sử dụng các vòng lặp lồng nhau để xây dựng mảng.

34. Giả sử một mảng hai chiều hình chữ nhật của các số nguyên tên là matrix đã được khai báo với sáu hàng và tám cột. Viết một vòng lặp để sao chép nội dung của cột thứ hai vào cột thứ năm.

35. Xem xét phương thức sau:

java

Copy code

```
public static void mystery2d(int[][] a) {
    for (int r = 0; r < a.length; r++) {
        for (int c = 0; c < a[0].length - 1; c++) {
            if (a[r][c + 1] > a[r][c]) {
                a[r][c] = a[r][c + 1];
            }
        }
    }
}
```

Nếu một mảng hai chiều numbers được khởi tạo để lưu trữ các số nguyên như sau, nội dung của nó sau khi gọi phương thức trên sẽ là gì?

java

Copy code

```
int[][] numbers = {{3, 4, 5, 6},
                   {4, 5, 6, 7},
                   {5, 6, 7, 8}};
mystery2d(numbers);
```

36. Viết một đoạn mã tạo ra một mảng hai chiều dạng jagged (không đều) của các số nguyên với năm hàng và số lượng cột tăng dần ở mỗi hàng, sao cho hàng đầu tiên có một cột, hàng thứ hai có hai cột, hàng thứ ba có ba cột, v.v. Các phần tử mảng nên có

giá trị tăng dần từ trên xuống dưới, từ trái qua phải (còn gọi là thứ tự row-major). Nói cách khác, nội dung của mảng sẽ là:

text

Copy code

1

2, 3

4, 5, 6

7, 8, 9, 10

11, 12, 13, 14, 15

Sử dụng các vòng lặp lồng nhau để xây dựng mảng.

37. Khi xem xét một mảng 2D của các pixel, làm thế nào bạn có thể xác định được chiều rộng và chiều cao của hình ảnh ngay cả khi bạn không có quyền truy cập vào đối tượng DrawingPanel?

38. Hoàn thành đoạn mã sau cho một phương thức chuyển đổi một hình ảnh thành kênh màu đỏ của nó; tức là loại bỏ mọi màu xanh lá cây hoặc màu xanh dương từ mỗi pixel và chỉ giữ lại thành phần màu đỏ.

java

Copy code

```
public static void toRedChannel(DrawingPanel panel) {
    Color[][] pixels = panel.getPixels();
    for (int row = 0; row < pixels.length; row++) {
        for (int col = 0; col < pixels[0].length; col++) {
            // điền mã của bạn vào đây
        }
    }
    panel.setPixels(pixels);
}
```

39. Kết quả của đoạn mã sau là gì? Hình ảnh sẽ trông như thế nào?

java

Copy code

```
public static void pixelMystery(DrawingPanel panel) {
    Color[][] pixels = panel.getPixels();
    for (int row = 0; row < pixels.length; row++) {
```

```
        for (int col = 0; col < pixels[0].length; col++) {
            int n = Math.min(row + col, 255);
            pixels[row][col] = new Color(n, n, n);
        }
    }
    panel.setPixels(pixels);
}
```

Bài tập

1. Viết một phương thức gọi là lastIndexOf nhận một mảng số nguyên và một giá trị số nguyên làm tham số và trả về chỉ số cuối cùng mà giá trị đó xuất hiện trong mảng. Phương thức nên trả về -1 nếu không tìm thấy giá trị. Ví dụ, trong mảng [74, 85, 102, 99, 101, 85, 56], chỉ số cuối cùng của giá trị 85 là 5.
2. Viết một phương thức gọi là range trả về khoảng cách của các giá trị trong một mảng số nguyên. Khoảng cách được định nghĩa là 1 lớn hơn sự khác biệt giữa giá trị lớn nhất và giá trị nhỏ nhất trong mảng. Ví dụ, nếu một mảng gọi là list chứa các giá trị [36, 12, 25, 19, 46, 31, 22], lệnh gọi range(list) sẽ trả về 35 (46 - 12 + 1). Bạn có thể giả định rằng mảng có ít nhất một phần tử.
3. Viết một phương thức gọi là countInRange nhận một mảng số nguyên, một giá trị nhỏ nhất, và một giá trị lớn nhất làm tham số và trả về số lượng phần tử trong mảng nằm trong khoảng từ giá trị nhỏ nhất đến giá trị lớn nhất (bao gồm cả hai). Ví dụ, trong mảng [14, 1, 22, 17, 36, 7, –43, 5], với giá trị nhỏ nhất là 4 và giá trị lớn nhất là 17, có bốn phần tử có giá trị nằm trong khoảng từ 4 đến 17.
4. Viết một phương thức gọi là isSorted nhận một mảng số thực làm tham số và trả về true nếu danh sách được sắp xếp theo thứ tự không giảm, và false nếu không. Ví dụ, nếu các mảng có tên list1 và list2 lần lượt chứa [16.1, 12.3, 22.2,

14.4] và [1.5, 4.3, 7.0, 19.5, 25.1, 46.2], lệnh gọi isSorted(list1) sẽ trả về false và isSorted(list2) sẽ trả về true. Giả định rằng mảng có ít nhất một phần tử. Một mảng chỉ có một phần tử được coi là đã sắp xếp.

5. Viết một phương thức gọi là mode trả về phần tử xuất hiện nhiều nhất trong một mảng số nguyên. Giả sử rằng mảng có ít nhất một phần tử và mỗi phần tử trong mảng có giá trị từ 0 đến 100 (bao gồm cả hai). Nếu có nhiều giá trị xuất hiện cùng một số lần, chọn giá trị nhỏ hơn. Ví dụ, nếu mảng được truyền vào chứa các giá trị [27, 15, 15, 11, 27], phương thức của bạn sẽ trả về 15. (Gợi ý: Bạn có thể tham khảo chương trình Tally từ chương này để có ý tưởng giải quyết vấn đề này.) Bạn có thể viết một phiên bản của phương thức này mà không dựa vào các giá trị nằm trong khoảng từ 0 đến 100?
6. Viết một phương thức gọi là stdev trả về độ lệch chuẩn của một mảng số nguyên. Độ lệch chuẩn được tính bằng cách lấy căn bậc hai của tổng các bình phương của các sự khác biệt giữa mỗi phần tử và giá trị trung bình, chia cho một ít hơn số phần tử. Đơn giản là vậy! Một cách ngắn gọn và mang tính toán học, độ lệch chuẩn của một mảng a được viết như sau:

```
css
Copy code
stdev(a) = sqrt( (∑i=0 to a.length-1 (a[i] - average(a))^2) / (a.length-1) )
```

Ví dụ, nếu mảng được truyền vào chứa các giá trị [1, –2, 4, –4, 9, –6, 16, –8, 25, –10], phương thức của bạn sẽ trả về khoảng 11.237.

7. Viết một phương thức gọi là kthLargest nhận một số nguyên k và một mảng a làm tham số và trả về phần tử sao cho có k phần tử có giá trị lớn hơn hoặc bằng. Nếu k = 0, trả về phần tử lớn nhất; nếu k = 1, trả về phần tử lớn thứ hai, v.v. Ví dụ,

nếu mảng được truyền vào chứa các giá trị [74, 85, 102, 99, 101, 56, 84] và số nguyên k được truyền vào là 2, phương thức của bạn sẽ trả về 99 vì có hai giá trị ít nhất là lớn bằng 99 (101 và 102). Giả sử rằng 0 ≤ k < a.length. (Gợi ý: Xem xét việc sắp xếp mảng hoặc một bản sao của mảng trước.)

8. Viết một phương thức gọi là median nhận một mảng số nguyên làm tham số và trả về trung vị của các số trong mảng. Trung vị là số xuất hiện ở giữa danh sách nếu bạn sắp xếp các phần tử theo thứ tự. Giả định rằng mảng có kích thước lẻ (vì vậy mà chỉ một phần tử duy nhất làm trung vị) và rằng các số trong mảng nằm trong khoảng từ 0 đến 99 (bao gồm cả hai). Ví dụ, trung vị của [5, 2, 4, 17, 55, 4, 3, 26, 18, 2, 17] là 5 và trung vị của [42, 37, 1, 97, 1, 2, 7, 42, 3, 25, 89, 15, 10, 29, 27] là 25. (Gợi ý: Bạn có thể tham khảo chương trình Tally từ đầu chương này để có ý tưởng.)
9. Viết một phương thức gọi là minGap nhận một mảng số nguyên làm tham số và trả về sự khác biệt nhỏ nhất hoặc khoảng cách giữa các giá trị liền kề trong mảng, trong đó khoảng cách được định nghĩa là giá trị sau trừ giá trị trước. Ví dụ, trong mảng [1, 3, 6, 7, 12], khoảng cách đầu tiên là 2 (3-1), khoảng cách thứ hai là 3 (6-3), khoảng cách thứ ba là 1 (7-6), và khoảng cách thứ tư là 5 (12-7). Vì vậy, phương thức của bạn sẽ trả về 1 nếu được truyền vào mảng này. Khoảng cách nhỏ nhất có thể là số âm nếu danh sách không được sắp xếp. Nếu bạn được truyền một mảng có ít hơn hai phần tử, trả về 0.
10. Viết một phương thức gọi là percentEven nhận một mảng số nguyên làm tham số và trả về tỷ lệ phần trăm của các số chẵn trong mảng dưới dạng một số thực. Ví dụ, nếu mảng lưu trữ các phần tử [6, 2, 9, 11, 3], phương thức của bạn sẽ trả về 40.0. Nếu mảng không chứa phần tử chẵn hoặc không chứa phần tử nào, trả về 0.0.

11. Viết một phương thức gọi là isUnique nhận một mảng số nguyên làm tham số và trả về giá trị boolean cho biết các giá trị trong mảng có là duy nhất hay không (true là có, false là không). Các giá trị trong danh sách được coi là duy nhất nếu không có cặp giá trị nào giống nhau. Ví dụ, nếu truyền vào một mảng chứa [3, 8, 12, 2, 9, 17, 43, –8, 46], phương thức của bạn sẽ trả về true, nhưng nếu truyền vào [4, 7, 3, 9, 12, –47, 3, 74], phương thức của bạn sẽ trả về false vì giá trị 3 xuất hiện hai lần.
12. Viết một phương thức gọi là priceIsRight bắt chước quy tắc đoán giá từ trò chơi The Price Is Right. Phương thức nhận một mảng số nguyên đại diện cho giá đặt cược của các thí sinh và một số nguyên đại diện cho giá đúng. Phương thức trả về phần tử trong mảng bids gần nhất với giá đúng mà không lớn hơn giá đó. Ví dụ, nếu mảng có tên bids lưu trữ các giá trị [200, 300, 250, 1, 950, 40], lệnh gọi priceIsRight(bids, 280) sẽ trả về 250, vì 250 là giá gần nhất với 280 mà không vượt quá 280. Nếu tất cả các giá đều lớn hơn giá đúng, phương thức của bạn nên trả về -1.
13. Viết một phương thức gọi là longestSortedSequence nhận một mảng số nguyên làm tham số và trả về độ dài của dãy con tăng dần dài nhất. Một dãy con tăng dần là một tập hợp các phần tử trong danh sách liên tiếp tăng dần theo thứ tự từ giá trị này đến giá trị tiếp theo. Ví dụ, trong mảng [3, 8, 9, 7, 7, 12, 13, 14, 15, 10], độ dài của dãy con tăng dần dài nhất là 5 (7, 12, 13, 14, 15). Bạn có thể giả định rằng mảng có ít nhất một phần tử. Dãy con tăng dần dài nhất chỉ chứa một phần tử cũng hợp lệ; trong trường hợp này, bạn nên trả về 1.

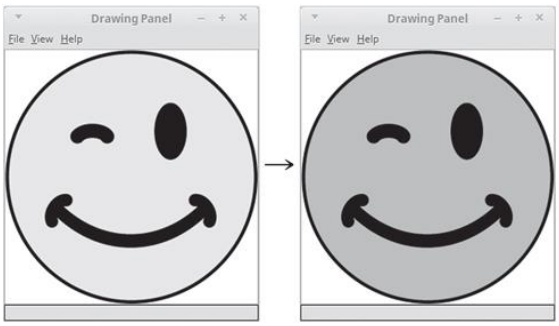
14. Viết một phương thức gọi là contains nhận hai mảng số nguyên a1 và a2 làm tham số và trả về một giá trị boolean cho

biết liệu dãy phần tử trong a2 có xuất hiện trong a1 hay không (true là có, false là không). Dãy phải xuất hiện liên tiếp và theo đúng thứ tự. Ví dụ, xem xét các mảng sau:
java
Copy code
int[] list1 = {1, 6, 2, 1, 4, 1, 2, 1, 8};
int[] list2 = {1, 2, 1};
Lệnh gọi contains(list1, list2) sẽ trả về true vì dãy giá trị trong list2 [1, 2, 1] xuất hiện trong list1 bắt đầu từ chỉ số 5. Nếu list2 chứa các giá trị [2, 1, 2], lệnh gọi contains(list1, list2) sẽ trả về false. Hai mảng có các phần tử giống hệt nhau được coi là chứa nhau. Mọi mảng đều chứa mảng rỗng, và mảng rỗng không chứa bất kỳ mảng nào khác ngoài chính nó.
15. Viết một phương thức gọi là collapse nhận một mảng số nguyên làm tham số và trả về một mảng mới chứa kết quả của việc thay thế mỗi cặp số nguyên bằng tổng của cặp đó. Ví dụ, nếu một mảng gọi là list lưu các giá trị [7, 2, 8, 9, 4, 13, 7, 1, 9, 10], thì lệnh gọi collapse(list) sẽ trả về một mảng mới chứa [9, 17, 17, 8, 19]. Cặp đầu tiên từ danh sách gốc được gộp thành 9 (7+2), cặp thứ hai được gộp thành 17 (8+9), v.v. Nếu danh sách chứa số lượng phần tử lẻ, phần tử cuối cùng sẽ không bị gộp. Ví dụ, nếu danh sách là [1, 2, 3, 4, 5], thì lệnh gọi sẽ trả về [3, 7, 5]. Phương thức của bạn không nên thay đổi mảng được truyền vào làm tham số.
16. Viết một phương thức gọi là append nhận hai mảng số nguyên làm tham số và trả về một mảng mới chứa kết quả của việc nối các giá trị của mảng thứ hai vào cuối mảng đầu tiên. Ví dụ, nếu các mảng list1 và list2 lưu trữ [2, 4, 6] và [1, 2, 3, 4, 5] tương ứng, thì lệnh gọi append(list1, list2) sẽ trả về một mảng mới chứa [2, 4, 6, 1, 2, 3, 4, 5]. Nếu lệnh gọi là append(list2, list1), phương thức sẽ trả về một mảng chứa [1, 2, 3, 4, 5, 2, 4, 6].

17. Viết một phương thức gọi là vowelCount nhận một chuỗi String làm tham số và tạo ra, trả về một mảng số nguyên đại diện cho số lượng mỗi nguyên âm trong chuỗi đó. Mảng trả về từ phương thức của bạn nên chứa năm phần tử: phần tử đầu tiên là số lượng A, phần tử thứ hai là số lượng E, phần tử thứ ba là I, phần tử thứ tư là O, và phần tử thứ năm là U. Giả sử rằng chuỗi không chứa chữ cái viết hoa. Ví dụ, lệnh gọi vowelCount("i think, therefore i am") sẽ trả về mảng [1, 3, 3, 1, 0].
18. Viết một phương thức gọi là evenBeforeOdd nhận một mảng số nguyên và sắp xếp lại các phần tử sao cho tất cả các giá trị chẵn xuất hiện trước tất cả các giá trị lẻ. Ví dụ, nếu mảng là [5, 4, 2, 11, 9, 10, 4, 7, 3], thì sau khi phương thức được gọi, một thứ tự chấp nhận được của các phần tử sẽ là [4, 2, 10, 4, 5, 11, 9, 7, 3]. Thứ tự chính xác của các phần tử không quan trọng, miễn là tất cả các giá trị chẵn xuất hiện trước tất cả các giá trị lẻ. Mảng có thể không chứa phần tử chẵn hoặc không chứa phần tử lẻ. Không sử dụng bất kỳ mảng tạm thời nào trong giải pháp của bạn và không gọi Arrays.sort.
19. Viết một phương thức gọi là wordLengths nhận một Scanner của một tệp đầu vào làm tham số. Phương thức của bạn nên mở tệp đã cho, đếm số lượng chữ cái trong mỗi từ trong tệp, và xuất ra một biểu đồ kết quả về số lượng từ chứa mỗi số chữ cái. Ví dụ, xét một tệp chứa văn bản sau:
mathematica
Copy code
Before sorting:
13
23 480 –18 75
hello how are you feeling today

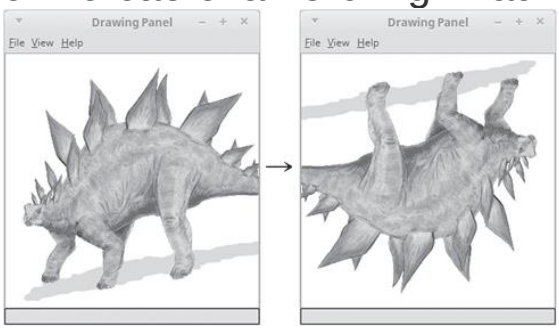
After sorting:
–18 13 23 75 480
are feeling hello how today you

Phương thức của bạn nên tạo ra kết quả sau trên console. Sử dụng tab để các dấu sao thẳng hàng:
makefile
Copy code
1: 0
2: 6 ******
3: 10 **********
4: 0
5: 5 *****
6: 1 *
7: 2 **
8: 2 **
Giả sử rằng không có từ nào trong tệp dài hơn 80 ký tự.
20. Viết một phương thức gọi là matrixAdd nhận một cặp mảng hai chiều số nguyên làm tham số, xử lý các mảng này như ma trận hai chiều, và trả về tổng của chúng. Tổng của hai ma trận A và B là một ma trận C, trong đó, với mỗi hàng i và cột j, Cij = Aij + Bij. Bạn có thể giả định rằng các mảng được truyền vào làm tham số có cùng kích thước.
21. Viết một phương thức gọi là isMagicSquare nhận một mảng hai chiều số nguyên làm tham số và trả về true nếu nó là một ma trận ma thuật. Một ma trận vuông là ma trận ma thuật nếu tất cả các tổng hàng, cột, và đường chéo đều bằng nhau. Ví dụ, [[2, 7, 6], [9, 5, 1], [4, 3, 8]] là một ma trận vuông vì tất cả tám tổng đều bằng 15.
22. Viết một phương thức gọi là grayscale chuyển đổi một hình ảnh màu sang hình ảnh đen trắng. Điều này được thực hiện bằng cách tính trung bình các thành phần đỏ, xanh lá, và xanh dương của mỗi điểm ảnh. Ví dụ, nếu một điểm ảnh có các giá trị RGB là (đỏ=100, xanh lá=30, xanh dương=80), trung bình của ba thành phần là (100+30+80)/3=70, vì vậy điểm ảnh đó trở thành (đỏ=70, xanh lá=70, xanh dương=70).



7.3-56 Văn bản thay thế đầy đủ

14. Viết một phương thức transpose nhận một DrawingPanel làm tham số và đảo ngược hình ảnh về cả hai trục x và y. Bạn có thể giả định rằng hình ảnh là hình vuông, nghĩa là chiều rộng và chiều cao của nó bằng nhau.

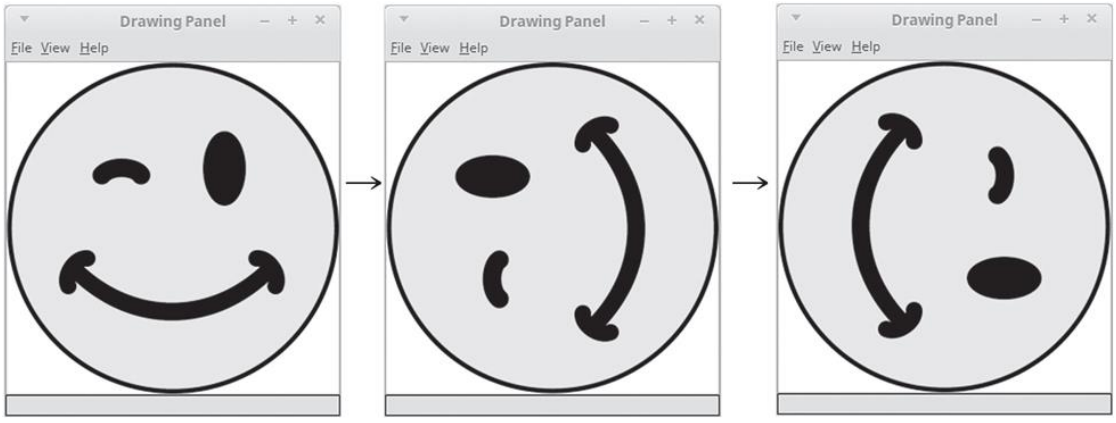


7.3-57 Văn bản thay thế đầy đủ

15. Viết một phương thức zoomIn nhận một DrawingPanel làm tham số và chuyển đổi nó thành một hình ảnh lớn gấp đôi ở cả hai chiều. Mỗi điểm ảnh từ hình ảnh gốc trở thành một nhóm 4 điểm ảnh (2 hàng và 2 cột) trong hình ảnh mới được phóng to.

7.3-58 Văn bản thay thế đầy đủ

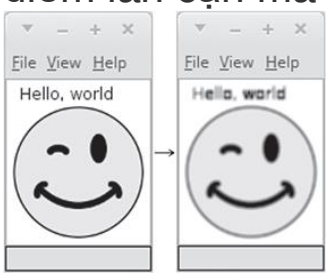
16. Viết các phương thức rotateLeft và rotateRight xoay các điểm ảnh của một hình ảnh ngược chiều kim đồng hồ hoặc theo chiều kim đồng hồ 90 độ tương ứng. Bạn không nên giả định rằng hình ảnh có hình vuông; chiều rộng và chiều cao của nó có thể khác nhau.



7.3-59 Văn bản thay thế đầy đủ

17. Viết một phương thức blur làm cho hình ảnh trở nên "mờ" bằng cách sử dụng thuật toán cụ thể sau. Đặt mỗi điểm ảnh là trung bình của chính nó và 8 điểm ảnh xung quanh. Nghĩa là, đối với điểm ảnh tại vị trí (x, y), đặt giá trị RGB của nó là trung bình

của các giá trị RGB tại các vị trí từ (x−1, y−1) đến (x+1, y+1). Cẩn thận không vượt ra khỏi giới hạn gần rìa của hình ảnh; nếu một điểm ảnh nằm dọc theo cạnh của hình ảnh, trung bình những điểm lân cận mà nó có.



7.3-60 Văn bản thay thế đầy đủ

Dự án lập trình

1. Java có giới hạn về kích thước số nguyên có thể lưu trữ trong kiểu int. Giới hạn này có thể được khắc phục bằng cách biểu diễn số nguyên dưới dạng mảng các chữ số. Hãy viết một chương trình tương tác để cộng hai số nguyên có tối đa 50 chữ số mỗi số.
2. Viết một trò chơi Hangman sử dụng mảng. Cho phép người dùng đoán các chữ cái và biểu diễn các chữ cái đã đoán trong một mảng.
3. Viết một chương trình chơi một biến thể của trò chơi Mastermind với người dùng. Ví dụ: chương trình có thể sử dụng các số ngẫu nhiên giả để tạo ra một số có bốn chữ số. Người dùng sẽ được phép đoán cho đến khi họ đoán đúng số. Cung cấp manh mối cho người dùng về số lượng chữ số của dự đoán là đúng và ở đúng vị trí, và số lượng chữ số đúng nhưng ở vị trí sai.
4. Viết một chương trình để chấm điểm các câu trả lời của người dùng cho bài kiểm tra tính cách Myers–Briggs cổ điển. Giả sử rằng bài kiểm tra có 70 câu hỏi xác định tính cách của một người trong bốn chiều. Mỗi câu hỏi có hai lựa chọn trả lời mà chúng ta sẽ gọi là "A" và "B". Các câu hỏi được tổ chức thành 10 nhóm, mỗi nhóm có bảy câu hỏi, với mô hình lặp lại sau trong mỗi nhóm:
    - Câu hỏi đầu tiên trong mỗi nhóm (các câu hỏi 1, 8, 15, 22, v.v.) cho biết liệu người đó có hướng nội hay hướng ngoại.
    - Hai câu hỏi tiếp theo (các câu hỏi 2 và 3, 9 và 10, 16 và 17, 23 và 24, v.v.) kiểm tra liệu người đó được hướng dẫn bởi các giác quan hay trực giác.
    - Hai câu hỏi tiếp theo (các câu hỏi 4 và 5, 11 và 12, 18 và 19, 25 và 26, v.v.) kiểm tra liệu người đó tập trung vào suy nghĩ hay cảm xúc.

- Hai câu hỏi cuối cùng trong mỗi nhóm (các câu hỏi 6 và 7, 13 và 14, 20 và 21, 27 và 28, v.v.) kiểm tra liệu người đó thích đánh giá hay bị hướng dẫn bởi nhận thức.

Nói cách khác, nếu chúng ta coi hướng nội/hướng ngoại (I/E) là chiều 1, giác quan/trực giác (S/N) là chiều 2, suy nghĩ/cảm xúc (T/F) là chiều 3, và đánh giá/nhận thức (J/P) là chiều 4, thì bản đồ các câu hỏi với các chiều tương ứng của chúng sẽ trông như thế này:

12233441223344122334412233441223344122334412233441223344122334412233441223344

BABAAAABAAAAAAABAAAABBAAAAAABAAAABABAABAAABABABAABAAAAAABAAAAAABAAAAAA

Dưới đây là một phần mẫu đầu vào của tệp tên và các câu trả lời:

Betty Boop

BABAAAABAAAAAAABAAAABBAAAAAABAAAABABAABAAABABABAABAAAAAABAAAAAABAAAAAA

Snoopy

AABBAABBBBBABABAAAAABABBAABBAAAABBBAAABAABAABABAAAABAABBBBAAABBAABABBB

Nếu ít hơn 50% các câu trả lời của một người là B cho một chiều tính cách nhất định, loại tính cách của người đó cho chiều đó sẽ là lựa chọn đầu tiên trong hai lựa chọn của nó. Nếu người đó có 50% hoặc nhiều hơn câu trả lời B, loại tính cách của người đó sẽ là lựa chọn thứ hai. Chương trình của bạn nên xuất tên của mỗi người, số lượng câu trả lời A và B cho mỗi chiều, tỷ lệ phần trăm của các câu trả lời B trong mỗi chiều và loại tính cách tổng thể. Dưới đây là kết quả mà chương trình của bạn nên xuất cho dữ liệu đầu vào trên:

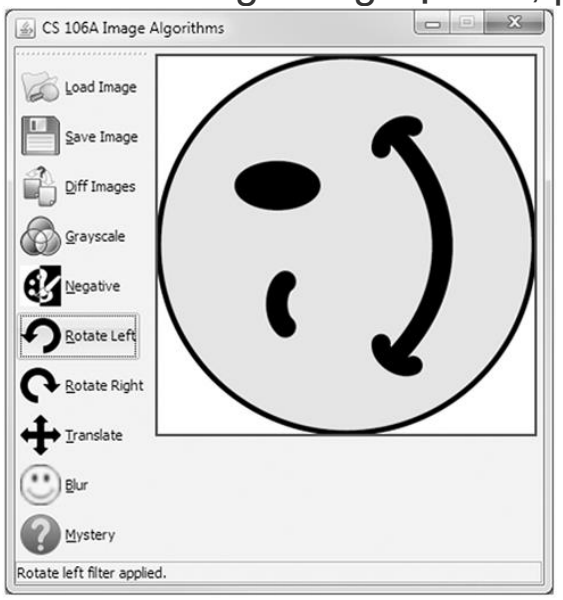
Betty Boop:

1A–9B 17A–3B 18A–2B 18A–2B

[90%, 15%, 10%, 10%] = ISTJ

Snoopy:
7A–3B 11A–9B 14A–6B 6A–14B
[30%, 45%, 30%, 70%] = ESTP

5. Sử dụng mảng hai chiều để viết trò chơi Tic-Tac-Toe biểu diễn bảng.
6. Viết một chương trình đọc tệp dữ liệu DNA và tìm kiếm các chuỗi protein. Dữ liệu DNA bao gồm các chuỗi dài các chữ cái A, C, G và T, tương ứng với các nucleotide hóa học gọi là adenine, cytosine, guanine và thymine. Protein có thể được xác định bằng cách tìm các chuỗi ba nucleotide đặc biệt chỉ định điểm bắt đầu và kết thúc của một dải protein. Lưu trữ dữ liệu liên quan trong các mảng khi bạn thực hiện tính toán. Tham khảo trang web sách giáo khoa của chúng tôi để tìm các tệp dữ liệu DNA ví dụ và chi tiết hơn về các quy tắc xác định protein.
7. Viết một chương trình Photoshop hoặc Instagram cơ bản với menu các thuật toán xử lý hình ảnh có sẵn tương tự như các bài tập trong chương này. Người dùng có thể tải một hình ảnh từ tệp và sau đó chọn thuật toán xử lý như chuyển đổi sang thang độ xám, phóng to, xoay, hoặc làm mờ.



Chapter 8 Classes

8.1 Object-Oriented Programming
• Classes and Objects
• [redacted] Objects
8.2 Object State and Behavior
• Object State: Fields
• Object Behavior: Methods
• The Implicit Parameter
• Mutators and Accessors
• The [redacted] Method
8.3 Object Initialization: Constructors
• The Keyword [redacted]
• Multiple Constructors
8.4 Encapsulation
• Private Fields
• Class Invariants
• Changing Internal Implementations
8.5 Case Study: Designing a [redacted] Class
• Object-Oriented Design Heuristics
• [redacted] Fields and Method Headers
• [redacted] Method and Constructor Implementation

Introduction

Now that you've mastered the basics of procedural-style programming in Java, you're finally ready to explore what Java was designed for: object-oriented

programming. This chapter introduces the basic terminology that you should use to talk about objects and shows you how to declare your own classes to create your own objects.

Objects are entities that contain state and behavior and that can be used as parts of larger programs. We'll discuss the concepts of abstraction and encapsulation, which allow you to use objects at a high level without understanding their inner details. We'll also discuss ideas for designing new classes of objects and implementing the programs that utilize them.

8.1 Object-Oriented Programming

Most of our focus so far has been on procedural decomposition, the technique of breaking complex tasks into smaller subtasks. This is the oldest style of programming, and even in a language like Java we still use procedural techniques. But Java also provides a different approach to programming that we call *object-oriented programming*.

Object-Oriented Programming (OOP)

Reasoning about a program as a set of objects rather than as a set of actions.

Object-oriented programming involves a particular view of programming that has its own terminology. Let's explore that terminology with nonprogramming examples first. Recall the definition of *object* from Chapter 3.

Object

A programming entity that contains state (data) and behavior (methods).

To truly understand this definition, you have to understand the terms “state” and “behavior.” These are some of the most fundamental concepts in object-oriented programming.

Let’s consider the class of objects we call radios. A radio can be in different states. It can be turned on or turned off. It can be tuned to one of many different stations, and it can be set to one of many different volumes. Any given radio has to “know” what state it is in, which means that it has to keep track of this information internally. We call the collection of such internal values the *state* of an object.

State

A set of values (internal data) stored in an object.

What are the behaviors of a radio? The most obvious one is that it produces sound when it is turned on and the volume is turned up. But there are actions that you can perform on a radio that manipulate its internal state. We can turn a radio on or off, and we can change the station or volume. We can also check what station the radio is set to right now. We call the collection of these operations the *behavior* of an object.

Behavior

A set of actions an object can perform, often reporting or modifying its internal state.

Objects themselves are not complete programs; they are components that are given distinct roles and responsibilities. Objects can be used as part of larger programs to solve problems. The pieces of code that create and use objects are known as *clients*.

Client (or Client Code)

Code that interacts with a class or objects of that class.

Client programs interact with objects by sending messages to them and asking them to perform behaviors. A major benefit of objects is that they provide reusable pieces of code that can be used in many client programs. You've already used several interesting objects, such as those of type , , , and . In other words, you and your programs have been clients of these objects. Java's class libraries contain thousands of existing classes of objects.

As you write larger programs, however, you'll find that Java doesn't always have a pre-existing object for the problem you're solving. For example, if you were creating a calendar application, you might want to use objects to represent dates, contacts, and appointments. If you were creating a three-dimensional graphical simulation, you might want objects to represent three-dimensional points, vectors, and matrices. If you were writing a financial program, you might want classes to represent your various assets, transactions, and expenses. In this chapter you'll learn how to create your own classes of objects that can be used by client programs like these.

Our definition of object-oriented programming is somewhat simplified. A full exploration of this programming paradigm includes other advanced concepts called polymorphism and inheritance that will be discussed in the next chapter.

Classes and Objects

In the previous chapters, we've considered the words "class" and "program" to be roughly synonymous. We wrote programs by creating new classes and placing static [redacted] methods into them.

But classes have another use in Java: to serve as blueprints for new types of objects. To create a new type of object in Java, we must create a class and add code to it that specifies the following elements:

- The state stored in each object
- The behavior each object can perform
- How to construct objects of that type

Once we have written the appropriate code, we can use the class to create objects of its type. We can then use those objects in our client programs. We say that the created objects are *instances* of the class because one class can be used to construct many objects. This is similar to the way that a blueprint works: One blueprint can be used to create many similar houses, each of which is an instance of the original blueprint.

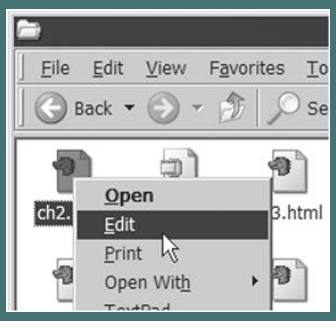
DID YOU KNOW?

Operating Systems History and Objects

In 1983 the IBM PC and its “clones” dominated the PC market, and most people ran an operating system called DOS. DOS uses what we call a “command-line interface,” in which the user types commands at a prompt. The console window is a similar interface. To delete a file in DOS, for example, you would give the command “del” (short for “delete”) followed by the file name:

This interface can be described in simple terms as “verb noun.” In fact, if you look at a DOS manual, you will find that it is full of verbs. This structure closely parallels the procedural approach to programming. When we want to accomplish some task, we issue a command (the verb) and then mention the object of the action (the noun, the thing we want to affect).

In 1984, Apple Computer released a new computer called a Macintosh that used what we call a graphical user interface, or GUI. The GUI interface uses a graphical “desktop” metaphor that has become so well known that people now tend to forget it is a metaphor. Later, Microsoft brought this functionality to IBM PCs with its Windows operating system.



8.1-1 Full Alternative Text

To delete a file on a Macintosh or on a Windows machine, you locate the icon for the file and click on it. Then you have several options. You can drag it to the trash/recycling bin, or you can select a “delete” command from the menu. Either way, you start with the object you want to delete and then give the command you want to perform. This is a reversal of the fundamental paradigm: With a GUI it’s “noun verb.” This different method of interaction is the core of object-oriented programming.

Most modern programs use GUIs because we have learned that people find it more natural to work this way. We are used to pointing at things, picking up things, grabbing things. Starting with the object is very natural for us. This approach has also proved to be a helpful way to structure our programs, enabling us to divide our programs into different objects that each can do a certain task, rather than dividing up the central task into subtasks.

Objects

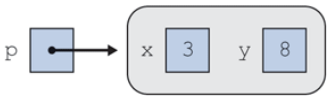
To learn about objects we will first examine an existing Java class, and then we will implement our own version of that class from scratch. The package has a class named . A

[redacted] object stores the (x, y) coordinates of a position in two-dimensional space. These coordinates are expressed as integers, although there are also variations for storing points using floating-point numbers. [redacted] objects are useful for applications that store many two-dimensional locations, such as maps of cities, graphical animations, and games.

Like most objects, [redacted] objects have to be explicitly constructed by calling a constructor. To construct a specific [redacted] object, you have to pass the values you want for [redacted] and [redacted]:

[redacted]

After the program executes the previous line of code, you have the following situation:



8.1-2 Full Alternative Text

Once you have constructed a [redacted] object, what can you do with it? One of the most common things you do with an object is print it to the console. A [redacted] object, like many Java objects, can be printed with the [redacted] statement.

[redacted]

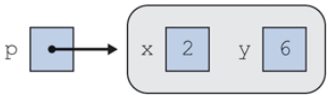
The [redacted] statement produces the following output. The format is a little ugly, but it lets you see the [redacted] and [redacted] values inside a given [redacted].

[redacted]

[redacted] objects also have a method called [redacted] that can be used to shift the coordinates by a specific delta-x and delta-y, which are passed as parameters. When you translate a [redacted], you shift its location by the specified amount. For example, you might say:

[redacted]

Given that the [redacted] started out with coordinates (3, 8), this translation would leave the [redacted] with coordinates (2, 6). Thus, after this line of code is executed, you'd end up with the following situation:



8.1-3 Full Alternative Text

One of the other things you can do with a [redacted] object is to refer to its [redacted] and [redacted] values using the dot notation:

[redacted]

[redacted]

You can even change these internal values directly:

[redacted]

Table 8.1 includes some useful methods of each [redacted] object.

Table 8.1 Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Translates the coordinates by the given amounts
[redacted]	Sets the coordinates to the given values
[redacted]	Returns the distance from this point to [redacted]

Here is a complete program that constructs a [redacted] object and translates its coordinates, using [redacted] statements to examine the coordinates before and after the call:

[redacted]

```
import


public class


public static void


new
```

This code produces the following output:

8.2 Object State and Behavior

In the next few sections, we'll explore the structure of classes by writing a new class incrementally. We'll write our own version of the [redacted] class that was just described.

Here are the main components of a class that we'll see in the sections that follow:

- Fields (the data stored in each object)
- Methods (the behavior each object can execute)
- Constructors (code that initializes an object as it is being constructed with the [redacted] keyword)
- Encapsulation (protects an object's data from outside access)

We'll focus on these concepts by creating several major versions of the [redacted] class. The first version will give us [redacted] objects that contain only data. The second version will add behavior to the objects. The third version will allow us to construct [redacted] at any initial position. The finished code will encapsulate each [redacted] object's internal data to protect it from unwanted outside access. The early versions of the class will be incomplete and will be used to illustrate each feature of a class in isolation. Only the finished version of the [redacted] class will be written in proper object-oriented style.

Object State: Fields

The first version of our [redacted] class will contain state only. To specify each object's state, we declare special variables inside the class called *fields*. There are many synonyms for "field" that come from other programming languages and environments, such as "instance variable," "data member," and "attribute."

Field

[redacted]

A variable inside an object that makes up part of its internal state.

The syntax for declaring a field is the same as the syntax for declaring normal variables: a type followed by a name and a semicolon. But unlike normal variables, fields are declared directly inside the [redacted] braces of your class. When we declare a field, we're saying that we want every object of this class to have that variable inside it.

In previous chapters we've seen that every class should be placed into its own file. The following code, written in the file [redacted], defines the first version of our [redacted] class:

[redacted]

```
public class

int

int
```

This code specifies that each object will contain two fields (an integer called and an integer called). It may look as though the code declares a pair of variables, and . But actually it indicates that *each* object will contain two variables inside it, called and . If we create 100 objects, we'll have 100 pairs of and fields, one in each instance of the class.

The class isn't itself an executable Java program; it simply defines a new class of objects for client programs to use. The client code that uses will be a separate class that we will store in a separate file. Client programs can create objects using the keyword and empty parentheses:

When a [redacted] object is constructed, its fields are given default initial values of [redacted], so a new [redacted] object always begins at the origin of (0, 0) unless you change its [redacted] or [redacted] value. This is another example of auto-initialization, similar to the way that array elements are automatically given default values.

The following lines of code form the first version of a client program that uses our [redacted] class (the code is saved in a file called [redacted], which should be in the same folder or project as [redacted] in order for the program to compile successfully):

```
public class

public static void
```

```
new

new

double
```

```
double
```

The code produces the following output:

The client program has some redundancy that we'll eliminate as we improve our [redacted] class in the sections that follow.

Our initial [redacted] class essentially serves as a way to group two [redacted] values into one object. This technique is somewhat useful for the client program, but the client could have been written using primitive [redacted]s instead. Using [redacted] objects is not yet substantially better than using primitive [redacted] values, because our [redacted] objects do not yet have any behavior. An object that contains state, but no behavior, is sometimes called a *record* or *struct*. In the sections that follow, we'll grow our [redacted] class from a minimal implementation into a proper Java class.

Object Behavior: Methods

The second version of our [redacted] class will contain both state and behavior. Behavior of objects is specified by writing *instance methods*. The instance methods of an object describe the messages to which that object can respond.

Instance Method

[redacted]

A method inside an object that operates on that object.

The objects introduced in previous chapters all contained instance methods that represented their behavior. For example, a [redacted] object has a [redacted] method and a [redacted] object has a [redacted] method. We think of these methods as being stored inside the object. As you recall, they use different call syntax than static methods: You write the object's name, then a dot, and then the

method's name and parameters. Each object's methods are able to interact with the data stored inside that object.

The preceding client program translates the position of two objects. It does this by manually adjusting their and values:

Since translating points is a common operation, we should represent it as a method. One option would be to write a static method in the client code that accepts a , a delta-, and a delta- as parameters. Its code would look like the following:

A call to the static method would look like the following line of code:

However, a static method isn't the best way to implement the behavior. The [redacted] class is supposed to be reusable so that many client programs can use it. If the [redacted] method is placed into our [redacted] client, other clients won't be able to use it without copying and pasting its code redundantly. Also, one of the biggest benefits of programming with objects is that we can put related data and behavior together. The ability of a [redacted] to translate data is closely related to that [redacted] object's (x, y) data, so it is better to specify that each [redacted] object will know how to translate itself. We'll do this by writing an instance method in the [redacted] class.

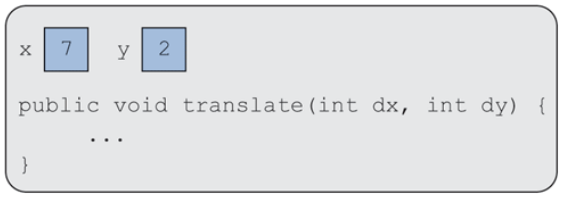
We know from experience with objects that you can call an instance method called [redacted] using "dot notation":

Notice that the instance method needs just two parameters: [redacted] and [redacted]. The client doesn't pass the [redacted] as a parameter, because the call begins by indicating which [redacted] object it wants to translate ([redacted]). In our example, the client is sending a [redacted] message to the object to which [redacted].

Instance method headers do not have the [redacted] keyword found in static method headers, but they still include the public keyword, the method's return type, its name, and any parameters that the method accepts. Here's the start of a [redacted] class with a [redacted] method, with the header declared but the body blank:

[redacted]

When we declare a [redacted] method in the [redacted] class, we are saying that *each* [redacted] object has its own copy of that method. Each [redacted] object also has its own [redacted] and [redacted] values. A [redacted] object would look like the following:



8.2-5 Full Alternative Text

Whenever an instance method is called, it is called on a particular object. So, when we're writing the body of the [redacted] method, we'll think of that code from the perspective of the particular [redacted] object that receives the message: "The client has given me a [redacted] and [redacted] and wants me to change my [redacted] and [redacted] values by those amounts." Essentially, we need to write code to match the following pseudocode:

```
public void translate(int dx, int dy) {

add dx to this Point object's x value.

add dy to this Point object's y value.
```

It's helpful to know that an object's instance methods can refer to its fields. In previous chapters we've talked about *scope*, the range in which a variable can be seen and used. The scope of a variable is the set of braces in which it is declared. The same rule applies to fields: Since they are declared directly inside a class, their scope is the entire class.

This rule means that the [redacted] method can directly refer to the fields [redacted] and [redacted]. For example, the statement [redacted]; would increase the [redacted] object's [redacted] value by [redacted].

Here is a working [redacted] method that adjusts the [redacted] object's location:

```
x += dx;

y += dy;
```

The [redacted] method can refer to [redacted] and [redacted] directly without being more specific about which object it is affecting. It's as though you were riding inside a car and wanted the driver to turn left; you'd simply say, "Turn left." Though there are millions of cars in the world, you wouldn't feel a need to specify which car you meant. It is implied that you mean the car you're currently occupying. Similarly, in instance methods we don't need to specify which object's [redacted] or [redacted] we're using, because it is implied that we want to use the fields of the object that receives the message.

Here's the complete [redacted] class that contains the [redacted] method. The Java style guidelines suggest declaring fields at the top of the class, with methods below, but in general it is legal for a class's contents to appear in any order.

```
```

Here is the general syntax for instance methods:

Methods like [redacted] are useful because they give our objects useful behavior that lets us write more expressive and concise client programs. Having the client code manually adjust the [redacted] and [redacted] values of [redacted] objects to move them is tedious, especially in

larger client programs that translate many times. By adding the [redacted] method, we have provided a clean way to adjust the location of a [redacted] object in a single statement.

The Implicit Parameter

The code for an instance method has an implied knowledge of the object on which it operates. This object is called the *implicit parameter*.

Implicit Parameter

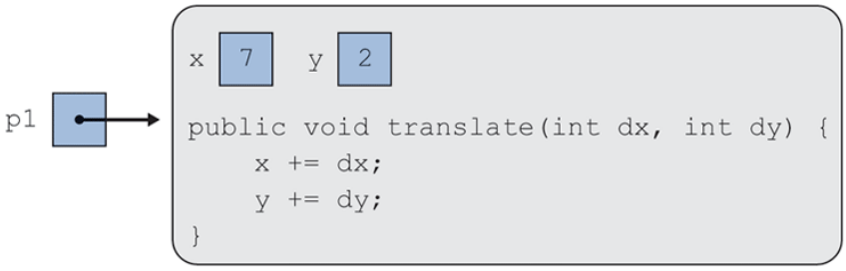
[redacted]

The object that is referenced during an instance method call.

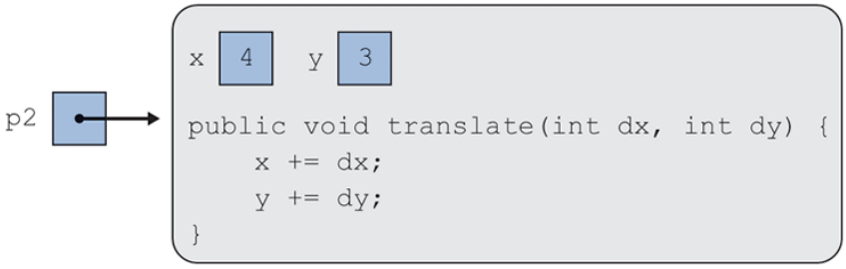
Let's walk through an example to demonstrate exactly how instance methods use the implicit parameter. The following client code constructs two [redacted] objects and sets initial locations for them:

[redacted]

After the preceding code has executed, the variables and objects in memory would appear as follows (remember that each object has its own copy of the [redacted] method):

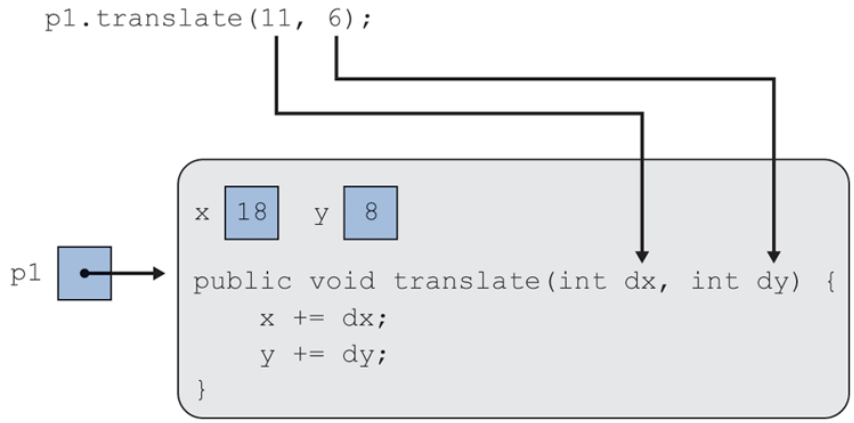


8.2-6 Full Alternative Text



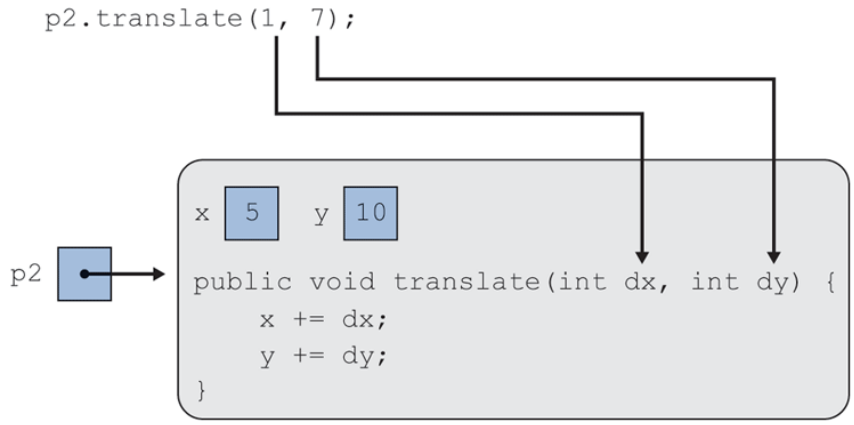
8.2-7 Full Alternative Text

Now we'll call the [redacted] method on each object. First, [redacted] is translated. During this call, [redacted]'s [redacted] method is passed the parameters [redacted] and [redacted]. The implicit parameter here is [redacted]'s object, so the statements [redacted]; and [redacted]; affect [redacted] and [redacted]:



8.2-8 Full Alternative Text

During the second method call [redacted]'s [redacted] method is executed, so the lines in the body of the [redacted] method change [redacted] and [redacted]:



8.2-9 Full Alternative Text

Mutators and Accessors

The [redacted] method is an example of a *mutator*.

Mutator

An instance method that modifies the object's internal state.

Generally, a mutator assigns a new value to one of the object's fields. Going back to the radio example, the mutators would be the switches and knobs that turn the radio on and off or change the station or volume.

It is a common convention for a mutator method's name to begin with "set," as in [redacted] or [redacted]. Usually, a mutator method has a [redacted] return type. Mutators often accept parameters that specify the new state of the object or the amount by which to modify the object's current state.

Accessors form a second important category of instance methods.

Accessor

An instance method that provides information about the state of an object without modifying it.

Generally, an accessor returns the value of one of the object's fields. Using our radio analogy, an accessor might return the current station or volume. Examples of accessor methods you have seen in previous chapters include the [redacted] and [redacted] methods of [redacted] objects and the [redacted] method of [redacted] objects.

Our client program computes the distance of two [redacted] from the origin, (0, 0). Since this is a common operation related to the data in a [redacted], let's give each [redacted] object an accessor called [redacted] that computes and returns that [redacted] object's distance from the origin. The method accepts no parameters and returns the distance as a [redacted].

The distance from the origin is computed using the Pythagorean Theorem, taking the square root of the sum of the squares of the [redacted] and [redacted] values. As we did when we used the [redacted] method, we'll refer to the [redacted] object's [redacted] and [redacted] fields directly in our computation:

Note that the [redacted] method doesn't change the [redacted] object's [redacted] or [redacted] value. Accessors are not used to change the state of the object—they only report information about the object. You can think of accessors as read-only operations while mutators are read/write operations.

A typical accessor will have no parameters and will not have a [redacted] return type, because it must return a piece of information. An accessor returns a value that is part of the state of the object or is

derived from it. The names of many accessors begin with “get” or “is,” as in [redacted] or [redacted].

Here’s the complete second version of our [redacted] class that now contains both state and behavior:

```
public class

int

int
```

```
public double




return
```

```
public void

int

int
```

The client program can now use the new behavior of the [redacted] class. The program produces the same output as before, but it is shorter and more readable than the original. The following lines show examples of the changes made to [redacted]:

The [redacted] Method

The designers of Java felt that it was important for all types of values to work well with [redacted]s. You've seen that you can concatenate [redacted]s with any other type of value, such as primitive [redacted]s or other objects.

Consider the following JShell interaction that creates some variables of different types and then prints them to the console. In this case JShell is acting as a client that uses the [redacted] class we have written in this chapter. To be able to access the [redacted] class we must use the [redacted] command, which reads code from a file into JShell.

```
jshell>


jshell>


jshell>


jshell>


jshell>


jshell>


jshell>


jshell>
```

Notice that printing [redacted] generated a strange result. We'd rather have it print the object's state of (0, 0), but Java doesn't know how to do so unless we write a special method in our [redacted] class.

When a Java program is printing an object or concatenating it with a [redacted], the program calls a special method called [redacted] on the object to convert it into a [redacted]. The [redacted] method is an instance method that returns a [redacted] representation of the object. A [redacted] method accepts no parameters and has a [redacted] return type:

If you don't write a [redacted] method in your class, your class will use a default version that returns the class name followed by an [redacted] sign and some letters and numbers related to the object's address in memory. If you define your own [redacted] method, it replaces this default version.

The following code, added to the [redacted] class in [redacted], implements a [redacted] method for our [redacted] objects and returns a [redacted] such as [redacted]":

Now that our class has this method, client code such as the preceding JShell interaction would produce the following output.

Note that the client code didn’t explicitly call the [redacted] method; the compiler did it automatically because the [redacted] object was being concatenated with a [redacted]. The [redacted] method is also implicitly called when printing an object by itself, as in the following code:

In order for this implicit calling behavior to work properly, your [redacted] method’s signature must exactly match the one shown

in this section. Changing the name or signature even slightly (for example, naming the method with a capital T, or) will cause the class to produce the old output (e.g., "). The reason has to do with concepts called inheritance and overriding that we will explore in the next chapter.

It is also legal to call explicitly if you prefer. The following client code uses an explicit call and produces the same output as the original client code:

```
.toString()
```

The Java guidelines recommend writing a method in every class you write.

COMMON PROGRAMMING ERROR

Statement in Method

Since the method is closely related to printing, some students mistakenly think that they should place statements in their methods, as in the following method:

A key idea to understand about is that it doesn't directly print anything: It simply returns a that the client can use in a statement.

In fact, many well-formed classes of objects do not contain any statements at all. The inclusion of statements in a class binds that class to a particular style of output. For example, the preceding code prints a object on its own line, making the class unsuitable for a client that doesn't want the output to appear exactly this way (say, a client that wants to print many objects on the same line).

You may wonder why the designers of Java chose to use a method rather than, say, a method that would output the object to the console. The reason is that is more versatile. You can use to output the object to a file, display it on a graphical user interface, or even send the text over a network.

8.3 Object Initialization: Constructors

Our third version of the [redacted] class will include the ability to create [redacted] objects at any initial location. The initial state of objects is specified by writing *constructors*, which were introduced in Chapter 3. Recall that a constructor is a piece of code that initializes the state of new objects as they are created.

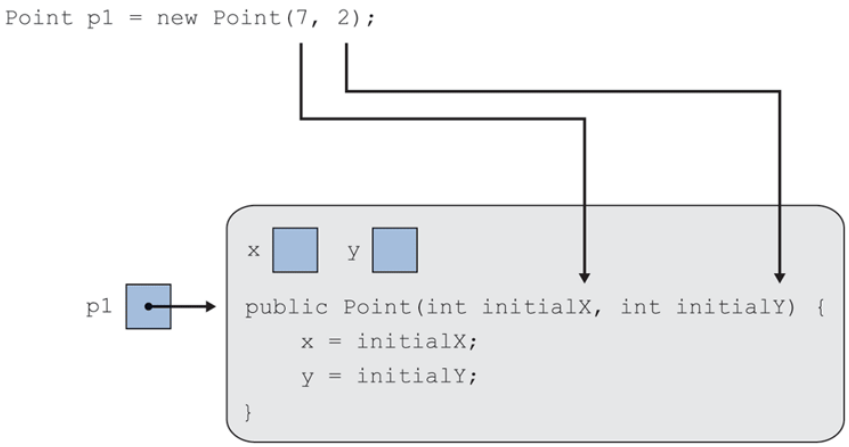
A clumsy aspect of our existing client code is that it takes three lines to create and initialize the state of one [redacted] object:

In general, when we have constructed objects, we have been able to declare and initialize them in a single statement. We might expect that we could initialize a [redacted] by writing its initial (x, y) values in parentheses as we constructed it:

However, such a statement wouldn't be legal for our class, because we haven't written any code specifying how to create a with an initial (x, y) location. We can specify how to do this by writing a constructor in our class. The constructor executes when the client uses the keyword to create a new object. When you write a constructor, you specify what parameters must be passed when clients use the keyword with your type and how those parameters should be used to initialize the newly created object.

A constructor's header begins with the keyword , followed by the class's name and any parameters. It looks like a method header with the same name as the class, except that you do not specify a return type. A constructor often has parameters that specify the object's initial state. Our constructor for the class will accept initial and values as parameters and store them into the new object's and fields:

Like instance methods, constructors execute on a particular object (the one that's being created with the [redacted] keyword) and can refer to that object's fields and methods directly. In this case, we store [redacted] and [redacted] parameter values into the new [redacted] object's [redacted] and [redacted] fields:



8.3-10 Full Alternative Text

Now that we are exploring constructors, it makes sense to think about the process of creating objects in more detail. When an executing Java program reaches a statement that creates a new [redacted] object, several operations occur:

1. A new [redacted] object is created and allocated in memory.
2. The [redacted] constructor is called on the newly created object, passing [redacted] and [redacted] as the [redacted] and [redacted] parameter values.
3. A [redacted] reference variable named [redacted] is created and set to refer to the newly created object.

Here is the complete code for the third version of our [redacted] class, which now contains a constructor:

[redacted]

```
public class

int

int

public

int

int
```

```
public double

return

public String

return
```

```
public void

int

int
```

Calling a constructor with parameters is similar to ordering a car from a factory: “I’d like the yellow one with power windows and leather seats.” You might not need to specify every detail about the car, such as the fact that it should have four wheels and headlights, but you do specify some initial attributes that are important to you.

The general syntax for constructors is the following:

When a class doesn’t have a constructor, as in our previous versions of the [redacted] class, Java automatically supplies a *default constructor* with no parameters. That is why it was previously legal to construct a [redacted]. The default constructor auto-initializes all fields to zero-equivalent values. However, Java doesn’t supply the default empty constructor when we supply a constructor of our own, so it is illegal to construct [redacted] objects without passing in the initial [redacted] and [redacted] parameters:

In the next sections, we’ll write additional code to restore this ability.

COMMON PROGRAMMING ERROR

Using with a Constructor

Many new programmers accidentally include the keyword in the header of a constructor, since they've gotten used to writing a return type for every method:

```
void
```

This is actually a very tricky and annoying bug. Constructors aren't supposed to have return types. When you write a return type such as , what you've created is not a constructor, but rather a normal instance method called that accepts and parameters and has a return type. This error is tough to catch, because the file still compiles successfully.

You will see an error when you try to call the constructor that you thought you just wrote, though, because it isn't actually a constructor. The client code that tries to construct the object will indicate that it can't find an constructor for a :

If you see "cannot find symbol" constructor errors and you were positive that you wrote a constructor, double-check its header to make sure there's no return type.

COMMON PROGRAMMING ERROR

Redeclaring Fields in a Constructor

Another common bug associated with constructors occurs when you mistakenly redeclare fields by writing their types. Here's an example that shows this mistake:

```
int

int
```

The preceding code behaves in an odd way. It compiles successfully, but when the client code constructs a object its initial coordinates are always (0, 0), regardless of the parameter values that are passed to the constructor:

The problem is that rather than storing and in the object's and fields, we've actually declared local variables called and inside the constructor. We store and in those local variables, which are thrown away when the constructor finishes running. No values are ever

assigned to the and fields in the constructor, so they are automatically initialized to . We say that these local and variables *shadow* our and fields because they obscure the fields we intended to set.

If you observe that your constructor doesn't seem to be setting your object's fields, check closely to make sure that you didn't accidentally declare local variables that shadow your fields. The key thing is not to include a type at the front of the statement when you assign a value to a field.

The Keyword

When we discussed instance methods we mentioned that an object's instance methods can refer to its other methods and fields, because the instance method code knows which object it's operating on. We called this idea the "implicit parameter." Now we will explore the mechanics behind the implicit parameter and introduce a keyword that allows us to refer to it directly.

The implicit parameter is actually a special reference that is set each time an instance method is called. You can access this reference in your code using the keyword .

this

A Java keyword that allows you to refer to the implicit parameter inside a class.

When you refer to a field such as [redacted] in your code, you are actually using shorthand. The compiler converts an expression such as [redacted] to [redacted]. You can use the longer form in your code if you want to be more explicit. For example, our [redacted] method could be rewritten as follows:

The code behaves the same way as the original version of the method. The explicit style is less common, but some programmers prefer it because it's clearer. It also more closely matches the style used in client code, where all messages to objects begin with a variable name and a dot.

The general syntax for using the keyword [redacted] to refer to fields is

Similarly, when you call an instance method such as [redacted], you're actually using shorthand for a call of [redacted]. You can use the longer form if you prefer. It has the following general syntax:

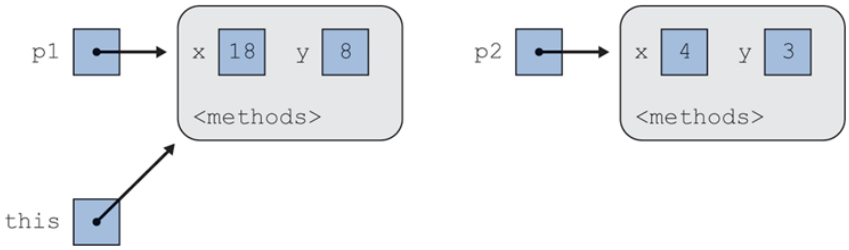
When the implicit parameter was introduced, we diagrammed the behavior of some method calls on two objects. Let's revisit the same example using the keyword . Consider the following two objects:

After constructing the objects, we make the following method calls:

Essentially, the behavior of these two method calls is the following:

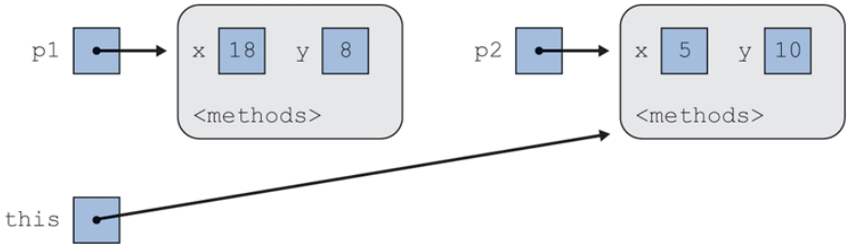
- Set to refer to the same object as , and execute the method with parameters .
- Set to refer to the same object as , and execute the method with parameters .

During the first call, [redacted] refers to the same object as [redacted]. Therefore, the method call adjusts the [redacted] coordinates of [redacted]'s object:



8.3-11 Full Alternative Text

During the second method call [redacted] refers to the same object as [redacted], so the lines in the body of the [redacted] method change the [redacted] and [redacted] fields of [redacted]'s object:



8.3-12 Full Alternative Text

One common usage of the keyword [redacted] is to deal with shadowed variables. As described earlier, shadowing occurs when a field is obscured by another variable with the same name. Shadowing can happen when a field has the same name as a parameter or local variable in a class. For example, the following is a legal header for our [redacted] method, even though our fields are also called [redacted] and [redacted]:

[redacted]

As explained at the beginning of this section, Java would normally interpret the expression [redacted] to mean [redacted]. However, if a parameter or local variable called [redacted] exists, the program will use that one instead if you just write [redacted] because the field [redacted] is shadowed by the parameter/variable. If you write [redacted], though, the program will always use the field [redacted].

Of course, you can avoid this situation by naming parameters and local variables differently from fields. However, some programmers prefer the style in which a variable takes the same name as a closely related field, because it saves them from having to concoct separate parameter names like [redacted] or [redacted].

In most cases, the compiler will not allow two variables to have the same name at the same point in a program. Fields are a special case that present the risk of shadowing. Java's designers decided to allow this risk so that parameter names could match their related fields.

Multiple Constructors

A class can have multiple constructors to provide multiple ways for clients to construct objects of that class. Each constructor must have a different signature (i.e., number and type of parameters).

Our existing [redacted] constructor requires two parameters (the [redacted] object's initial x- and y-coordinates). Before we added the constructor, we were able to construct [redacted] objects at (0, 0) without any parameters. When a class does not have a constructor, Java provides a parameterless default constructor that initializes all of the new object's fields to a zero-equivalent value. But when we added our two-parameter constructor, we lost the default constructor. This is unfortunate, because the default provided a useful shorter notation for constructing a [redacted] at the origin. We can restore this ability by adding a second, parameterless constructor to our [redacted] class. Our new constructor looks like this:

Now it's possible to construct [redacted] in two ways:

Returning to the analogy of purchasing cars, you can imagine that some customers want to specify many details about their new cars (e.g., “I’d like a yellow Civic with gold trim, upgraded stereo system, and a sun roof”), while other customers want to specify fewer details and want the car to contain default options instead. Having multiple constructors gives clients similar flexibility when they ask for new objects of your class.

Notice that both constructors perform similar actions; the only difference is what initial values the [redacted] and [redacted] fields receive. Another way of saying this is that the new constructor can be expressed in terms of the old constructor. The following two lines construct equivalent objects:

A common programming practice when writing classes with multiple constructors is for one constructor to contain the true initialization code and for all other constructors to call it. This means that every object created passes through a common code path. This system can be useful when you are testing and debugging your code later. The syntax for one constructor to call

another is to write the keyword [redacted], followed by the parameters to pass to the other constructor in parentheses:

This is really just the normal syntax for a method call, except that we use the special keyword [redacted] where we would normally put the name of a method. In our case, we want to pass parameter values of [redacted] and [redacted] to initialize each field:

8.4 Encapsulation

Our next version of the [redacted] class will protect its data from unwanted access using a concept known as *encapsulation*.

Encapsulation

[redacted]

Hiding the implementation details of an object from the clients of the object.

To understand the notion of encapsulation, recall the analogy of radios as objects. Almost everyone knows how to use a radio, but few people know how to build a radio or understand how the circuitry inside a radio works. It is a benefit of the radio's design that we don't need to know those details in order to use it.

The radio analogy demonstrates an important dichotomy of external versus internal views of an object. From the outside, we just see behavior. From the inside, we see the internal state that is used to accomplish that behavior (Figure 8.1).

Figure 8.1 The internal and external views of a radio.

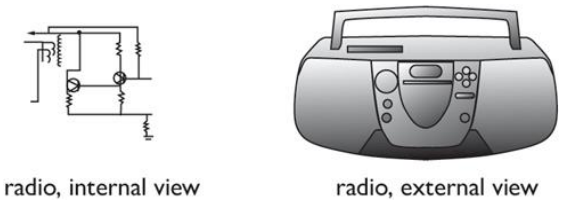


Figure 8.1 Full Alternative Text

Focusing on the radio's external behavior enables us to use it easily while ignoring the details of its inner workings that are unimportant to us. This is an example of an important computer science concept known as *abstraction*.

Abstraction

Focusing on essential properties rather than inner details.

In fact, a radio (like most other electronic devices) has a case or chassis that houses all of the electronics so that we don't see them from the outside. Dials, buttons, and displays on the case allow us to manipulate the radio without having to deal with all of the circuitry that makes it work. In fact, you wouldn't want someone to give you a fully functional radio that had wires and capacitors hanging out of it, because they would make the radio less pleasant to use.

In programming, the concept of hiding internal state from outside view is called encapsulation. When an object is properly encapsulated, its clients cannot directly access or modify its internal workings, nor do they need to do so. Only the implementer of the class needs to know about those details. Encapsulation leads to abstraction; an encapsulated object

presents a more pure abstraction than one that has data which can be accessed directly.

In previous chapters you have already taken advantage of the abstraction provided by well-encapsulated objects. For example, you have used [redacted] objects to read data from the console without knowing exactly how the [redacted] stores and tokenizes the input data, and you have used [redacted] objects to create random numbers without knowing exactly what algorithm the random number generator uses.

But so far, our [redacted] class is not encapsulated. We've built a working radio, but its wires (its [redacted] and [redacted] fields) are still hanging out. Using encapsulation, we'll put a casing around our [redacted] objects so that clients will only need to use the objects' methods and will not access the fields directly.

Private Fields

To encapsulate the fields of an object, we declare them to be private by writing the keyword [redacted] at the start of the declaration of each field. The fields of our [redacted] class would be declared as follows:

```
private

private
```

We haven’t yet shown a syntax template for fields because we wanted to show the preferred style with the fields private. The syntax for declaring encapsulated fields is

Fields can also be declared with an initial value:

Declaring fields encapsulates the state of the object, in the same way that a radio’s casing keeps the user from seeing the wires and circuitry inside it. Private fields are visible to all of the code inside the class (i.e., inside the file), but not anywhere else. This means that we can no longer directly refer to a object’s or fields in our client code. The following client code will not compile successfully:

The compiler produces error messages such as the following:

To preserve the functionality of our client program, we need to provide a way for client code to access a [redacted] object's field values. We will do this by adding some new accessor methods to the [redacted] class. If the value of an object's field might be useful externally, it is common to write an accessor to return that value. Here are the methods that provide access to a [redacted] object's [redacted] and [redacted] fields:

The client code to print a object's and values must be changed to the following:

```
p1.getX()

p1.getY()
```

It probably seems odd to grant access to a object's and fields when we said our goal was to encapsulate those fields, but having accessors like and doesn't actually violate the encapsulation of the object. The accessor methods just return a copy of the fields' values to the client, so that the client can see the or values but doesn't have any way to change them. In other words, these accessor methods give the client read-only access to the state of the object.

One drawback of encapsulating the class is that it is no longer easy for the client code to set a to a new location. For convenience, we'll add a new mutator to our encapsulated class that sets both the and fields of the object to new values passed as parameters:

Another way to set a [redacted] to a new location would be to write separate methods called [redacted] and [redacted]. We have chosen [redacted] partly for brevity and partly because it matches Java's actual [redacted] class.

Notice that the [redacted] class now has some redundancy between its two-parameter constructor and its [redacted] method. The two bodies are essentially the same, setting the [redacted] to have new x- and y-coordinates. We can eliminate this redundancy by having the constructor call [redacted] rather than setting the field values manually. It is legal for an object to call its own instance methods from a constructor or another instance method:

[redacted]

We can eliminate a bit more redundancy using this technique. Translating a [redacted] can be thought of as setting its location to the old location plus the [redacted] and [redacted], so we can modify the [redacted] method to call the [redacted] method:

[redacted]

DID YOU KNOW?

The Perils of Poor Encapsulation

[redacted]

Many novices (as well as many professional programmers) do not fully appreciate the concepts of abstraction and encapsulation. It is tempting to write classes that directly expose their data for

clients to use, since private fields introduce some complexity and restrictions into a program. However, there have been some famous examples where a lack of proper encapsulation and abstraction caused a large problem.

One such example is the “Y2K” (year 2000) or “millennium bug” scare of late 1999. The issue arose because a large number of computer programs represented years by using only two digits, such as 72 for 1972. They followed this convention largely to save memory, since many of them were older programs written in a language called COBOL during a time when memory was more scarce. Once the year became 2000, the programs would incorrectly think that the year was 1900, and this might cause them to fail.

Making matters worse was the fact that many of these programs contained their own handwritten logic for representing dates, which sometimes appeared in many places in the code. In order for the program to represent a year with more than two digits, many places in the code needed to be changed. In total, over $300 billion was spent on repairing old programs and systems to correct the Y2K problem.

If the old programs had used an encapsulated class that included a field representing the year, far less work would have been needed to fix the Y2K bug. This class could have been updated once and all the client code would have received the benefits.

Surprisingly, Java’s class libraries also contain examples of poorly encapsulated classes. In the package, for example, the and classes have public fields. (A object stores and fields to represent the size of an onscreen region.) Many client programs access the fields directly

when they use these objects. Java's developers regret this decision:

> Several classes in the Java platform libraries violate the advice that public classes should not expose fields directly. Prominent examples include the and classes in the package. Rather than examples to be emulated, these classes should be regarded as cautionary tales. [. . .] The decision to expose the internals of the class resulted in a serious performance problem that could not be solved without affecting clients.
> —*Joshua Bloch, Effective Java*

Now that we've introduced all the major elements of a well-encapsulated class, it's time to look at a proper syntax template for an entire class. The Java style guidelines suggest putting fields at the top of the class, followed by constructors, followed by methods:

Here is the fourth complete version of our [redacted] class, including encapsulated fields and accessor methods [redacted] and [redacted]:

```
public class

private

private

public

this
```

```
public

int

int

public double

return
```

```
public int


return


public int


return
```

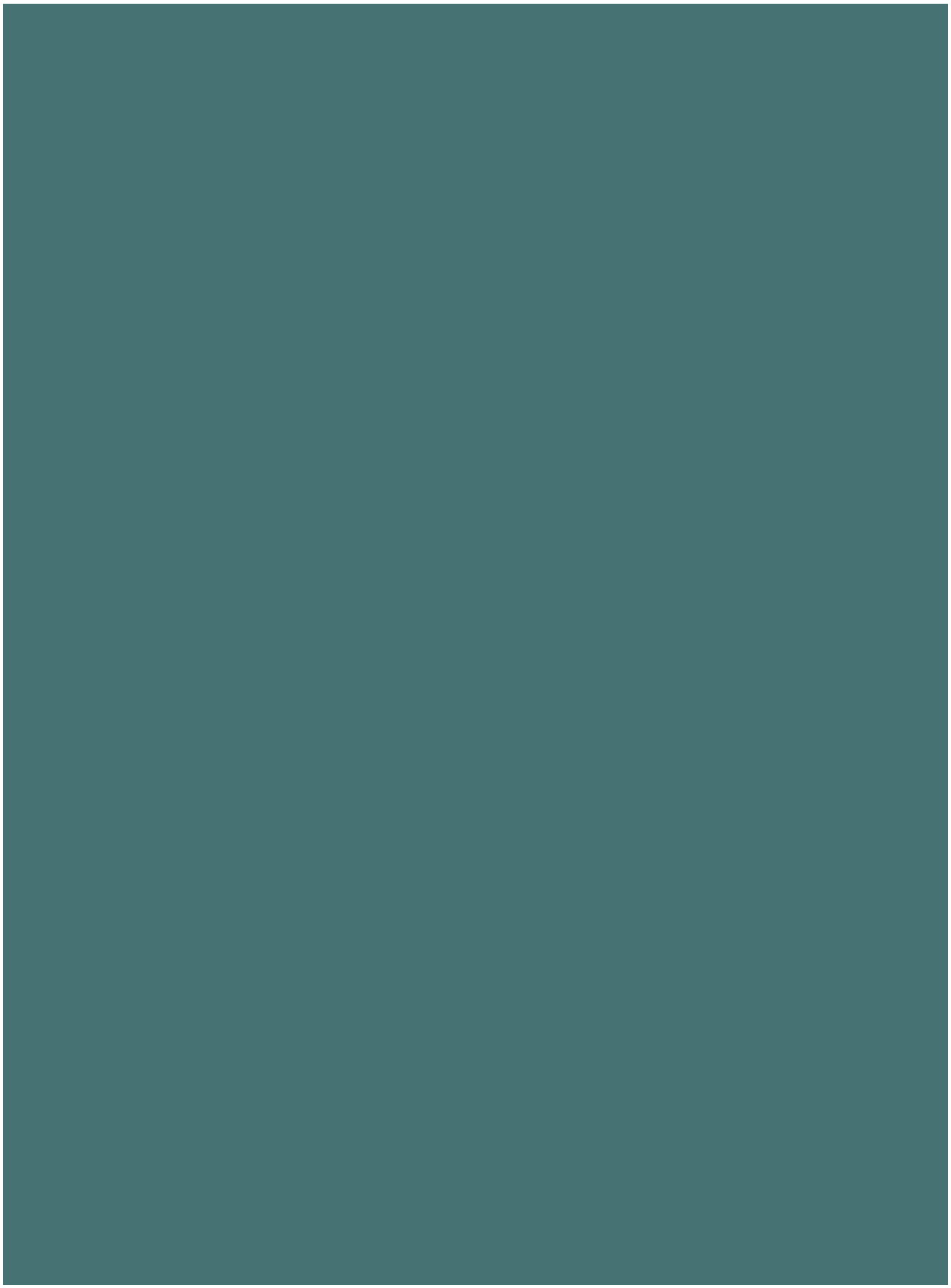
```
public void

int

int




this




this
```

```
public String


return
```

```
public void

int

int
```

Here’s the corresponding final version of our client program, which now uses the [redacted] constructors and methods appropriately:

```
public class


public static void




new


new
```



Class Invariants

In this section we will develop another new class to illustrate a particular benefit of encapsulation. Consider a program that measures or deals with elapsed intervals of time, such as a program for a stopwatch, a scheduler, a TV recorder, or an airline flight system. A useful abstraction in such a program would be an object representing an elapsed span of time.

Let's write a class called , in which each object represents an interval of elapsed hours and minutes. For example, we could construct a representing an interval of 6 hours and 15 minutes. We'll represent only hours and minutes, ignoring larger or smaller units such as days or seconds.

Since we're representing intervals of hours and minutes, it seems natural to use these two quantities as fields in our class. We will encapsulate the class properly from the start by declaring the fields as :

The constructor for a [redacted] object will accept [redacted] and [redacted] as parameters and store the values into the object's fields. However, there is a potential problem: What should we do about values of [redacted] that are [redacted] or greater? Should the client program be allowed to construct a [redacted] object representing 0 hours and 157 minutes? A more natural representation of this amount would be 2 hours and 37 minutes. And what about values for hours or minutes that are negative? It doesn't make sense to have a span of -2 hours or -35 minutes. Ideally we should not allow a [redacted] object to store such a value.

Let's make a design decision that we will allow [redacted] objects to store a value for [redacted] that is between 0 and 59 inclusive. If the client tries to construct a [redacted] object with a negative number of hours or minutes, we could resolve the problem by printing an error message or by setting the fields to 0. But in cases like this, the negative value often comes from a bug or a mistake in the client's understanding of our class. The best way to handle a violation like this is to throw an exception so that the client will know the parameter values passed were illegal.

If the user tries to construct a [redacted] object with more than 60 minutes, we will convert the excess minutes into hours. You might be tempted to use an [redacted] statement or a loop to handle minutes above 60, but there is a simpler solution. For a large number of minutes such as 157, dividing by 60 will produce the

hours (2) and using the % operator by 60 will produce the remaining minutes (37). The hours field should really store the hours parameter plus the minutes parameter divided by 60, and the minutes field should store the remaining minutes:

A useful behavior of a TimeSpan object would be the ability to add more hours and minutes to the span. An airline scheduling program might use this behavior to add the elapsed times for two back-to-back flights to determine the total travel time for a passenger's trip. Let's implement this behavior as a method called add that accepts hours and minutes as parameters. Here's an initial incorrect version of that method:

The problem with the preceding code is that it allows the client to put the object into an invalid state. If the client passes a value of [redacted] that is large enough to make the total minutes greater than 60, the [redacted] field will have an invalid value. For example, if the client adds 45 minutes to a time span of 1 hour and 30 minutes, the result will be 1 hour and 75 minutes.

We decided that we wanted every [redacted] object to store only valid numbers of minutes between 0 and 59. We wrote code in our constructor to ensure that this would be true of each object's initial state, but really we want to ensure that the condition is true for every object throughout its entire lifespan, not just when it is initially created. Such a property that is true of every object of a class is called a *class invariant*.

Class Invariant

An assertion about an object's state that is true for the lifetime of that object.

Class invariants are related to preconditions, postconditions, and assertions, as presented in Chapters 4 and 5. We cannot allow any mutator method such as [redacted] to break the invariants we have

decided upon; a class invariant should be treated as an implicit postcondition of every method in the class. Enforcing an invariant may cause you to add preconditions to the constructors and mutator methods of your class.

We can write code at the end of the [redacted] method to deal with invalid numbers of minutes and hours. First we want the program to throw an exception if the hours or minutes that are passed are negative. We will also make sure that we convert each group of 60 minutes into an hour. The following code implements the behavior:

The code now enforces its invariant in two places: the constructor and [redacted]. It would be preferable to solve this problem in one place rather than redundantly checking for it throughout the class. A more elegant solution is to have the constructor initialize the fields to [redacted] and then call the [redacted] method, which performs the necessary invariant checks and stores the values of [redacted] and [redacted]:

The fields would have been auto-initialized to [redacted] anyway, but many programmers prefer to explicitly initialize field values for clarity.

Another useful operation for [redacted] objects is the ability to print them on the console. We'll add this ability by including a [redacted] method that returns a [redacted] such as [redacted] for 2 hours and 35 minutes.

Here is the code for the complete [redacted] class that enforces its invariant:

```
public class

private int

private int

public

int

int
```

```
this



this
```

```
public void

int

int



if
```

```
throw new




this


this
```

```
public

return
```

Some additional features should be included in the class, such as accessors for the field values, but these are left as exercises.

Invariants bring to light the importance of proper encapsulation. If the [redacted] class weren't encapsulated, we would not be able to properly enforce our invariant. A buggy or malicious client would be able to make a [redacted] object's state invalid by setting its fields' values directly. When the class is encapsulated, it has much better control over how clients can use its objects, making it impossible for a misguided client program to violate the class invariant.

Changing Internal Implementations

Another important benefit of encapsulation is that it allows us to make internal design changes to a class without affecting its clients. A subtlety of classes is that the internal representation of a class does not necessarily have to match the external view that the client sees. A client of the [redacted] class thinks of each time span as a number of hours and a number of minutes. But the [redacted] object does not have to internally store its time using those two data fields. In fact, the code for [redacted] becomes simpler if we simply store a single field for the total number of minutes. For example, we can represent 2 hours and 15 minutes as 135 total minutes by converting each hour into 60 minutes.

Let's rewrite the [redacted] class to use only a single total minutes field:

Because our class is encapsulated, as long as our methods still produce the same results from an external point of view, we can change the implementation of the object's internal state and the clients will not need to be modified. We can implement the same constructor, [redacted], and [redacted] behavior using total minutes. For example, the [redacted] method needs to combine the [redacted] and

[redacted] together and add both of them into the [redacted]. We'll scale the hours by 60 as we add them to the total:

Notice that this new implementation makes it easier to enforce our class invariant about objects having valid state. We must still check for negative parameters, but we no longer need to worry about storing minutes of 60 or greater. All minutes added are properly grouped into the common total.

The constructor and [redacted] method also require minor modifications to account for our new representation. Here is the complete class, implemented with total minutes instead of hours and minutes. This version is shorter and simpler than the original:

```
public class


private int









public

int

int
```

```
public void

int

int



if



throw new
```

```
public

return
```

As another example, we could revisit our encapsulated [redacted] class and change its internal structure without having to modify the client code. For example, sometimes it is useful to express two-dimensional points in polar coordinates in terms of a radius r and an angle *theta*. In this representation, the (x, y) coordinates of a point are not stored directly but can be computed as (r cos(*theta*), *r* sin(*theta*)). When the [redacted] class is encapsulated, we can modify it to use r and *theta* fields internally, then modify

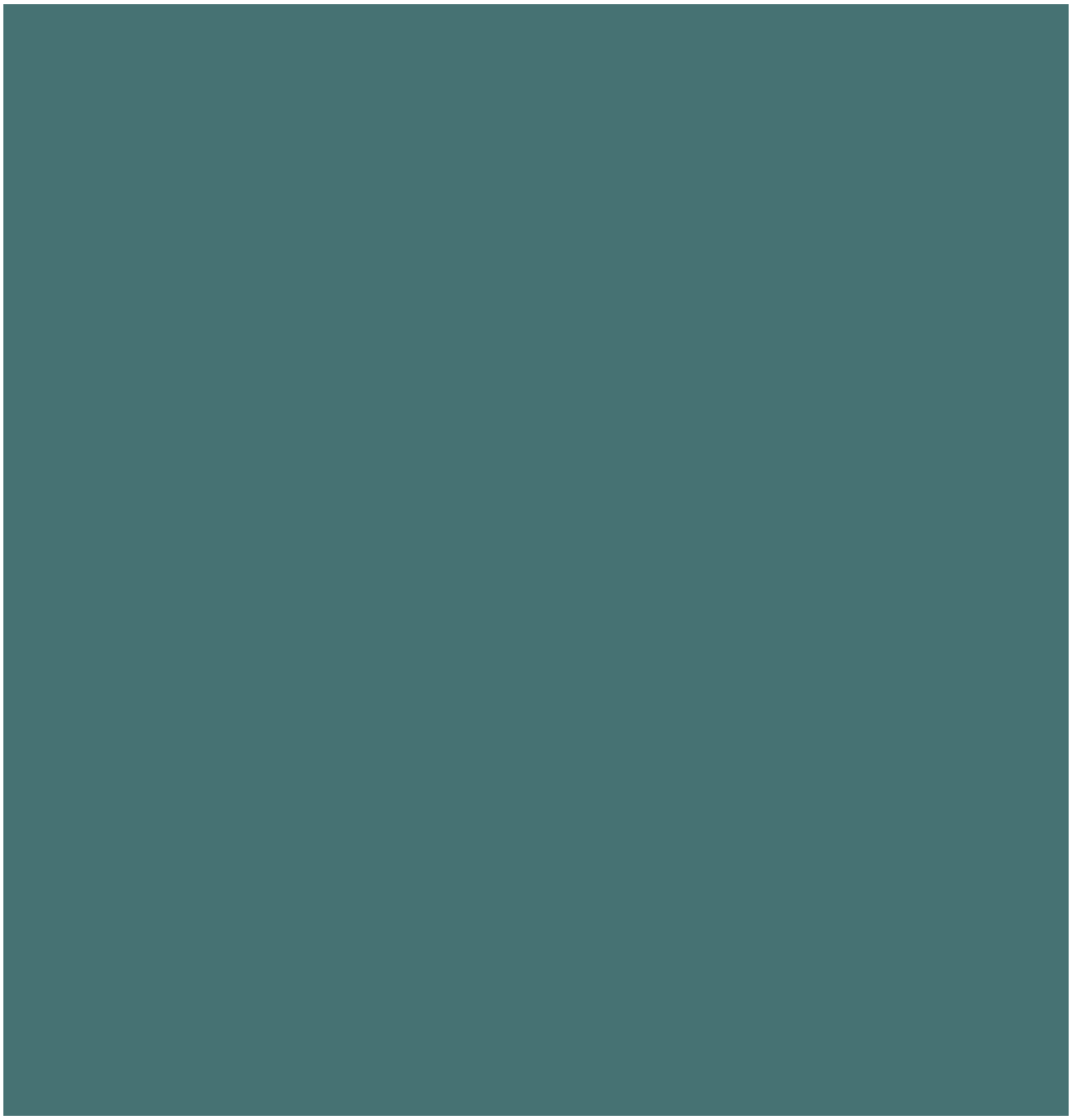
[redacted], [redacted], and other methods so that they still return appropriate values. The polar representation would not reap the same benefits we saw in our second implementation of [redacted], but it might make it easier for us to add certain functionality to the [redacted] class later.

8.5 Case Study: Designing a [redacted] Class

So far we have written several classes, but we have not talked about how to design a class or how to break apart a programming problem into classes. In this section we'll examine a larger programming problem and design a class and client to solve it. We will create a class called [redacted] and a client program that compares the performance of stocks that the user has purchased.

Consider the task of writing a financial program to record purchases of shares of two stocks and report which has the greatest profit. The investor may have made several purchases of the same [redacted] at different times and prices. The interaction with the program might look like this:

[redacted]



The program must perform several actions: prompting the user for input, calculating the amount spent on each purchase of stock, reporting profits, and so on. The client program could perform all of these actions and could keep track of the financial data using existing types such as [redacted] and [redacted]. However, recall that we began this chapter by talking about object-oriented reasoning. When you are studying complex programs, it is often useful to

think about the problem in terms of the relevant objects that could solve it, rather than placing all behavior in the client program. In this particular program, we must perform several computations that involve keeping track of purchases of shares of a particular stock, so it would be useful to store the purchase information in an object.

One possible design would be to create a [redacted] class that records information about a single purchase of shares of a particular stock. For example, if the user specified three purchases, the program should construct three [redacted] objects. However, a more useful abstraction here would be to hold the overall information about all purchases of one stock in one place. The investor may make many purchases of the same stock, so you want to have an easy way to accumulate these shares and their total cost into a single object.

Therefore, instead of a [redacted] class, we'll write a [redacted] class. Each [redacted] object will keep track of the investor's accumulated shares of one stock and can provide profit/loss information. Our [redacted] class will reside in a file called [redacted], and the client program itself will reside in a separate file called [redacted].

Object-Oriented Design Heuristics

We now face the important task of deciding on the contents of our [redacted] class. It can be tricky to choose a good set of classes and objects to solve a complex programming problem. Chapter 4's case study introduced a set of procedural design *heuristics*, or guidelines for good design, for effectively dividing a problem into

methods. There are similar guidelines for effectively breaking a large program into a set of classes and objects. The heuristics we'll discuss here are based on those listed in computer scientist Arthur Riel's influential book, *Object-Oriented Design Heuristics*.

First let's look at the overall set of responsibilities, the things that a class must know or do to solve the overall problem:

- Prompt the user for each stock's symbol and store the information somewhere.
- Prompt the user for the number of purchases of each stock.
- Read each purchase (number of shares and price per share) from the console and store the information somewhere.
- Compute the total profit/loss of each stock.
- Print the total profit/loss of each stock to the console.
- Compare the two total profits/losses and print a message to the console about which stock performed better.

It might be tempting to make most or all of these tasks responsibilities of our [redacted] class. We could make a [redacted] object store all the purchases of both stocks, prompt for information from the console, print the results, and so on. But a key guideline when writing classes is that they should have *cohesion*.

Cohesion

[redacted]

The extent to which the code for a class represents a single abstraction.

Placing all the responsibilities in the [redacted] class would not allow that class to represent a single clear abstraction. The abstraction we want to represent in the [redacted] class is the accumulated purchases of a single stock.

One set of responsibilities that [redacted] objects should not handle is producing the console input and output. We need to prompt the user for information and print messages, but these functions are specific to the current client program. Objects are meant to be reusable pieces of software, and other programs might want to track stock purchases without using these exact messages or prompts. If the [redacted] class handles the prompts and printing, it will be heavily intertwined with this client program and will not be easily reusable by other clients.

In general, we want to reduce unnecessary dependencies among classes. Dependencies among classes in an object-oriented program contribute to *coupling*, the degree to which one part of a program depends on another.

Striving to avoid unnecessary coupling is a second design heuristic commonly used in object-oriented programming. A design that avoids this problem is sometimes said to have *loose coupling*.

Let's divide some of the responsibilities now, based on our heuristics. Since the [redacted] client program will perform the console I/O, it should handle the following responsibilities:

[redacted]

- Prompt for each stock's symbol.
- Prompt for the number of purchases of each stock.

- Read each purchase (number of shares and price per share) from the console.
- Print the total profit/loss of each stock.
- Compare the two total profits/losses and print a message about which stock generated higher profits.

Since the [redacted] client program will be performing the console I/O, it might seem natural for it also to store the information about each stock purchase (that is, the number of shares and the price paid). But our [redacted] object should contain the functionality to compute a stock's total profit or loss, and it will need to have the data about all purchases to do so. This leads us to a third design heuristic: Related data and behavior should be in the same place. With that in mind, we can write out the responsibilities for the [redacted] class as follows:

[redacted]

- Store a stock's symbol.
- Store accumulated information about the investor's purchases of the stock.
- Record a purchase of the stock.
- Compute the total profit/loss for the stock.

When they are designing large object-oriented programs, many software engineers write information about classes as we've done here. A common technique to brainstorm ideas for classes is to write information on index cards. Each card is called a *CRC card* and lists the Class, its Responsibilities, and its Collaborators (other classes to which it is coupled).

The following list summarizes the design heuristics discussed in this section:

- A class should be cohesive, representing only one abstraction.
- A class should avoid unnecessary coupling.
- Related data and behavior should be in the same class.

Note that we began our design by looking at responsibilities rather than by specifying fields as we did when we developed our [redacted] class. We began writing the [redacted] class by discussing fields because the data associated with a point is simple and more obvious than the data associated with stock purchases. But in many larger problems like this one, working backward from behavior and responsibilities is a better technique.

[redacted] Fields and Method Headers

In this section we'll decide on a design for the method names and signatures the [redacted] should use to implement its behavior. We'll use this design to determine which fields are required to implement the behavior.

We've decided that a [redacted] object should allow clients to record purchases and request the total profit or loss. Each of these tasks can be represented as a method. The recording of a purchase can be represented as a method called [redacted]. The retrieval of the total profit or loss can be represented as a method called [redacted].

The [redacted] method should record information about a single purchase. A purchase consists of a number of shares that the user bought (which we can assume is a whole number) and a price per share (which can include real numbers with both dollars

and cents). Our [redacted] method should accept two parameters: an [redacted] for the number of shares bought and a [redacted] for the price per share. The method can use a [redacted] return type, since nothing needs to be returned after each purchase is recorded:

The [redacted] method will return the amount of money that the user made or lost on all accumulated purchases of this stock. Consider an investor who has made the following three purchases of a stock:

Purchase #1:20 shares * $10 per share=$ 200 costPurchase #2:20 shares * $30 per share=$ 600 costPurchase #3:10 shares * $20 per share=$ 200 cost50 total shares,$1000 total cost

If today’s price per share is $22.00, the current market value of the investor’s 50 shares is (50 * 22) or $1100. Since the investor paid $1000 total for the shares and they are now worth $1100, the investor has made ($1100−$1000)=$100 of profit. The general formula for the profit is the following:

profit = ([total shares]*[current share price])−(total cost)

The total number of shares and total cost figure needed for this calculation are the accumulated information from all the purchases that have been made of this stock. This means that information will need to be stored during each call of the [redacted] method to be used later in the [redacted] method. A key observation is that we do not need to store the number of shares, price per share, and cost for every purchase: We only need to

store cumulative sums of the total shares purchased so far and the total dollars spent so far to acquire those values.

The third value we need in order to calculate the profit is the current share price. We could choose to make this a field in the [redacted] class as well, but the share price is a dynamic value that changes regularly. We use it during a single call to the [redacted] method, but the next call may come at a later date when the price per share has changed.

This problem leads us to another design heuristic: Fields should represent values of core importance to the object and values that are used in multiple methods. Adding too many fields clutters a class and can make its code harder to read. If a value is used in only one method of the class, it's best to make it a parameter to that method rather than a field. Therefore, we'll make the share price a parameter to the [redacted] method.

One piece of state that we haven't discussed yet is that each stock has a symbol, such as [redacted]. We'll store the symbol as a [redacted] field in each [redacted] object.

Here's a skeleton of our [redacted] class so far:

[redacted] Method and Constructor Implementation

Now that we've decided on some of the [redacted]'s state and behavior, let's think about how to construct [redacted] objects. The client program will need the ability to create two [redacted] and record purchases of them.

It may be tempting to write a constructor that accepts three parameters: the symbol, the total number of shares purchased, and the total cost. But our [redacted] objects are accumulators of purchases, and we may want to be able to create new [redacted] objects before the program records initial purchases. Let's design our class to require only the symbol as a parameter and initialize the other fields to [redacted]:

When a constructor takes an object as a parameter (such as the [redacted]), it might make sense to check that parameter's value to make sure it isn't [redacted]. One possible way to handle this case would be to throw an exception if a [redacted] symbol is passed when the program creates a [redacted] object. We could do this by inserting the following lines at the start of the [redacted]'s constructor:

The Java convention is to throw a [redacted] when a parameter's value is [redacted] but should not be. For other invalid parameter values, throw an [redacted].

Now let's write the body of the [redacted] method. The task of recording the purchase consists of adding the new number of shares to the total number of shares and adding the new price paid for these shares to the total cost. The price paid is equal to the number of shares times the price per share. Here's the code for the [redacted] method to implement this behavior:

It might make sense here to check the parameters passed in to make sure they are valid, as we did with the constructor. In this case, valid numbers of shares and prices per share must not be

negative numbers. To perform this test, we can insert the following lines at the start of our [redacted] method:

Next, we'll write the body of the [redacted] method. As we noted previously, the profit of a [redacted] is equal to its current market value minus the amount that was paid for it:

profit=([total shares]*[current share price])−(total cost)

We can implement this formula in a straightforward manner using the [redacted] and [redacted] fields and the [redacted] parameter:

Note that parentheses are not needed in the code because multiplication has a higher precedence than subtraction.

As we did for the other methods, we should check for illegal parameter values. In this case, we shouldn't allow a negative current price per share. To ensure that this doesn't happen, we can place the following code at the start of the method:

After we've written all the fields, the constructor, and the methods of our , the class will look like this:

```
private int

private double

public

if

throw new
```

```
public double

double




if



throw new
```

```
double


return


public void

int

double
```

```
if


throw new
```

Here’s the client code to use the class:

```
```

```
import

public class

public static void

new

new
```

```
double

new

double

if
```

```
else if
```

```
else
```

```
public static double

int

for

int

int
```

```
double


double


double
```

```
return
```

It would be useful to have a few other methods in our [redacted] objects. For example, it would be good to implement accessors for the [redacted]'s data (the symbol, number of shares, and so on), and a [redacted] method to easily print [redacted] objects. We could even add a second constructor that would accept an initial number of shares and cost. These features are left for you to implement as exercises.

Chapter Summary

Object-oriented programming is a different philosophy of writing programs that focuses on nouns or entities in a program, rather than on verbs or actions of a program. In object-oriented programming, state and behavior are grouped into objects that communicate with each other.

A class serves as the blueprint for a new type of object, specifying the object's data and behavior. The class can be asked to construct many objects (also called "instances") of its type.

Java's [redacted] package has a class named [redacted]. Each object holds two [redacted] values, [redacted] and [redacted]. A [redacted] can be constructed, translated to a new location, and printed on the console.

The data for each object are specified using special variables called fields.

The behavior of each object is specified by writing instance methods in the class. Instance methods exist inside an object and can access that object's internal state.

To make objects easily printable, write a [redacted] method that returns the object's text representation.

A class can define special code called a constructor that initializes the state of new objects as they are created. The constructor will be called when external client code creates a new object of your type using the [redacted] keyword.

You can use the keyword [redacted] to have an object refer to itself. It is also used when a class has multiple constructors and one constructor wants to call another.

Objects can protect their internal data from unwanted external modification by declaring them to be [redacted], an action known as encapsulation. Encapsulation provides abstraction so that clients can use the objects without knowing about their internal implementation.

A class should represent only one key abstraction with related data and behavior, and it should be independent from its clients.

Self-Check Problems

Section 8.1: Object-Oriented Programming

1. Describe the difference between object-oriented programming and procedural programming.
2. What is an object? How is an object different from a class?
3. What is the state of a [redacted] object? What is its behavior?
4. What is the output of the following program?

5. Imagine that you are creating a class called [redacted]. A [redacted] object could be used to program a simple mathematical calculator device like the ones you have used in math classes in school. What state might a [redacted] object have? What might its behavior be?

Section 8.2: Object State and Behavior

6. Explain the differences between a field and a parameter. What is the difference in their syntax? What is the difference in their scope and the ways in which they may be used?
7. Create a class called [redacted] that represents a person's name. The class should have fields representing the person's first name, last name, and middle initial. (Your class should contain only fields for now.)
8. What is the difference between an accessor and a mutator? What naming conventions are used with accessors and mutators?
9. Suppose we have written a class called [redacted] with a method inside it, defined as:

[redacted]

If the client code has declared a [redacted] variable named [redacted], which of the following would be a valid call to the above method?

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]

10. Add a new method to the [redacted] class we developed in this chapter:

[redacted]

Returns the distance between the current [redacted] object and the given other [redacted] object. The distance between two points is equal to the square root of the sum of the squares of the differences of their x- and y-coordinates. In other words, the distance between two points (x1, y1) and (x2, y2) can be expressed as the square root of (x2−x1)2+(y2−y1)2. Two points with the same (x, y) coordinates should return a distance of [redacted].

11. (You must complete Self-Check Problem 7 before answering this question.)

Add two new methods to the [redacted] class:

[redacted]

Returns the person's name in normal order, with the first name followed by the middle initial and last name. For example, if the first name is [redacted], the middle initial is [redacted], and the last name is [redacted], returns [redacted].

[redacted]

Returns the person's name in reverse order, with the last name preceding the first name and middle initial. For example, if the first name is [redacted], the middle initial is [redacted], and the last name is [redacted], returns [redacted].

12. How do you write a class whose objects can easily be printed on the console?
13. The following [redacted] statement (the entire line) is equivalent to what?

[redacted]

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]

14. The [redacted] class in the [redacted] package has a [redacted] method that returns a [redacted] in the following format:

[redacted]

Write a modified version of the [redacted] method on our [redacted] class that returns a result in this format.

15. (You must complete Self-Check Problem 7 before answering this question.)

Write a [redacted] method for the [redacted] class that returns a [redacted] such as [redacted].

16. Finish the following client code so that it constructs two [redacted] objects, translates each, and then prints their coordinates.

Section 8.3: Object Initialization: Constructors

17. What is a constructor? How is a constructor different from other methods?
18. What are two major problems with the following constructor?

19. (You must complete Self-Check Problem 7 before answering this question.)

Add a constructor to the [redacted] class that accepts a first name, middle initial, and last name as parameters and initializes the [redacted] object's state with those values.

20. What is the meaning of the keyword [redacted]? Describe three ways that the keyword can be used.
21. Add a constructor to the [redacted] class that accepts another [redacted] as a parameter and initializes this new [redacted] to have the same (x, y) values. Use the keyword [redacted] in your solution.

Section 8.4: Encapsulation

22. What is abstraction? How do objects provide abstraction?
23. What is the difference between the [redacted] and [redacted] keywords? What items should be declared [redacted]?
24. When fields are made private, client programs cannot see them directly. How do you allow classes access to read these fields' values, without letting the client break the object's encapsulation?
25. Add methods named [redacted] and [redacted] to the [redacted] class that allow clients to change a [redacted] object's x- and y-coordinates, respectively.
26. (You must complete Self-Check Problem 7 before answering this question.)

Encapsulate the [redacted] class. Make its fields private and add appropriate accessor methods to the class.

27. (You must complete Self-Check Problem 26 before answering this question.)

Add methods called [redacted], [redacted], and [redacted] to your [redacted] class. Give the parameters the same names as your fields, and use the [redacted] keyword in your solution.

28. How does encapsulation allow you to change the internal implementation of a class?

Section 8.5: Case Study: Designing a Stock Class

29. What is cohesion? How can you tell whether a class is cohesive?
30. Why didn't we choose to put the console I/O code into the [redacted] class?
31. Add accessor methods to the [redacted] class to return the stock's symbol, total shares, and total cost.

Exercises

1. Add the following accessor method to the [redacted] class:

Returns which quadrant of the x/y plane the current [redacted] object falls in. Quadrant 1 contains all points whose [redacted] and [redacted] values are both positive. Quadrant 2 contains all points with negative [redacted] but positive [redacted]. Quadrant 3 contains all points with negative [redacted] and [redacted] values. Quadrant 4 contains all points with positive [redacted] but negative [redacted]. If the point lies directly on the x and/or y axis, return [redacted].

2. Add the following mutator method to the [redacted] class:

Negates and swaps the x/y coordinates of the [redacted] object. For example, if an object [redacted] initially represents the point(5,−3), after a call of [redacted] the object should represent(3,−5). If the same object initially represents the point (4, 17), after a call to [redacted] the object should represent(−17,−4).

3. Add the following accessor method to the [redacted] class:

Returns the “Manhattan distance” between the current [redacted] object and the given other [redacted] object. The Manhattan distance refers to the distance between two places if one can travel between them only by moving horizontally or vertically, as though driving on the streets of Manhattan. In our case, the Manhattan distance is the sum of the absolute values of the differences in their coordinates; in other words, the difference in [redacted] plus the difference in [redacted] between the points.

4. Add the following accessor method to the [redacted] class:

[redacted]

Returns [redacted] if the given [redacted] lines up vertically with this [redacted], that is, if their x-coordinates are the same.

5. Add the following accessor method to the [redacted] class:

[redacted]

Returns the slope of the line drawn between this [redacted] and the given other [redacted]. Use the formula (y2−y1)/(x2−x1) to determine the slope between two points (x1, y1) and (x2, y2). Note that this formula fails for points with identical x-coordinates, so throw an [redacted] in this case.

6. Add the following accessor method to the [redacted] class:

[redacted]

Returns whether this [redacted] is collinear with the given two other [redacted]. [redacted] are collinear if a straight line can be drawn that connects them. Two basic examples are three points that have the same x- or y-coordinate. The more general case can be determined by calculating the slope of the line between each pair of points and checking whether this slope is the same for all pairs of points. Use the formula (y2−y1)/(x2−x1) to determine the slope between two points (x1, y1) and (x2, y2). (Note that this formula fails for points with identical x-coordinates so this will have to be a special case in your code.) Since Java's [redacted] type is imprecise, round all slope values to a reasonable accuracy such as four digits past the decimal point before you compare them.

7. Add the following mutator method to the [redacted] class:

Adds the given amount of time to this time span.

8. Add the following mutator method to the [redacted] class:

Subtracts the given amount of time from this time span.

9. Add the following mutator method to the [redacted] class:

Scales this time span by the given factor. For example, 1 hour and 45 minutes scaled by 2 equals 3 hours and 30 minutes.

10. Add the following mutator method to the [redacted] class:

[redacted]

Resets this [redacted]'s number of shares purchased and total cost to [redacted].

11. Suppose the following [redacted] class has been created:

```
public class

double

public void

double
```

```
public void

double
```

Add a field to the [redacted] class named [redacted] for a real number representing an amount of money to deduct every time the user withdraws money. The default value is $0.00, but the client can change the value. Deduct the transaction fee money during every [redacted] call (but not from deposits). Make sure that the balance cannot go negative during a withdrawal. If the withdrawal (amount plus transaction fee) would cause it to become negative, don’t modify the balance at all.

12. Add a [redacted] method to the [redacted] class from the previous exercise. Your method should return a string that contains the account's name and balance separated by a comma and space. For example, if an account object named [redacted] has the name [redacted] and a balance of [redacted], the call [redacted] should return the string [redacted].
13. Add a [redacted] method to the [redacted] class from the previous exercises. Your method should move money from the current bank account to another account. The method accepts two parameters: a second [redacted] to accept the money, and a real number for the amount of money to transfer. There is a $5.00 fee for transferring money, so this much must be deducted from the current account's balance before any transfer. The method should modify the two [redacted] objects such that "this" current object has its balance decreased by the given amount plus the $5 fee, and the other account's balance is increased by the given amount. If this account object does not have enough money to make the full transfer, transfer whatever money is left after the $5 fee is deducted. If this account has under $5 or the amount is 0 or less, no transfer should occur and neither account's state should be modified. The following are some example calls to the method:

14. Write a class called [redacted] that represents a line segment between two [redacted]. Your [redacted] objects should have the following methods:

Constructs a new [redacted] that contains the given two [redacted].

Returns this [redacted]'s first endpoint.

Returns this [redacted]'s second endpoint.

Returns a [redacted] representation of this [redacted], such as [redacted].

15. Add the following accessor method to your [redacted] class:

Returns the slope of this [redacted]. The slope of a line between points (x1, y1) and (x2, y2) is equal to (y2−y1)/(x2−x1). If x2 equals x1 the denominator is zero and the slope is undefined, so you may throw an exception in this case.

16. Add the following constructor to your [redacted] class:

Constructs a new [redacted] that contains the given two [redacted].

17. Add the following accessor method to your [redacted] class:

Returns [redacted] if the given [redacted] is collinear with the [redacted] of this [redacted]—that is, if, when this [redacted] is stretched infinitely, it would eventually hit the given [redacted]. [redacted] are collinear if a straight line can be drawn that connects them. Two basic examples are

three points that have the same x- or y-coordinate. The more general case can be determined by calculating the slope of the line between each pair of points and checking whether this slope is the same for all pairs of points. Use the formula (y2−y1)/(x2−x1) to determine the slope between two points (x1,y1) and (x2, y2). (Note that this formula fails for points with identical x-coordinates, so this will have to be a special case in your code.) Since Java's [redacted] type is imprecise, round all slope values to a reasonable accuracy such as four digits past the decimal point before you compare them.

18. Write a class called [redacted] that represents a rectangular two-dimensional region. Your [redacted] objects should have the following methods:

[redacted]

Constructs a new [redacted] whose top-left corner is specified by the given coordinates and with the given [redacted] and [redacted]. Throw an [redacted] on a negative [redacted] or [redacted].

[redacted]

Returns this [redacted]'s height.

[redacted]

Returns this [redacted]'s width.

[redacted]

Returns this [redacted]'s x-coordinate.

[redacted]

Returns this [redacted]'s y-coordinate.

[redacted]

Returns a [redacted] representation of this [redacted], such as [redacted].

19. Add the following constructor to your [redacted] class:

[redacted]

Constructs a new [redacted] whose top-left corner is specified by the given [redacted] and with the given [redacted] and [redacted].

20. Add the following accessor methods to your [redacted] class:

[redacted]

Returns whether the given or coordinates lie inside the bounds of this .

21. Add the following method to your class:

Returns a new that represents the area occupied by the tightest bounding box that contains both this and the given other .

22. Add the following method to your class:

Returns a new that represents the largest rectangular region completely contained within both this and the given other . If the do not intersect at all, returns a with and both equal to .

23. Add the following method to your class:

Returns whether the given other rectangle lies entirely within the bounds of this rectangle.

Programming Projects

1. Write a class called [redacted] that represents a fraction with an integer numerator and denominator. A [redacted] object should have the following methods:

[redacted]

Constructs a new rational number to represent the ratio (numerator/denominator). The denominator cannot be [redacted], so throw an [redacted] if [redacted] is passed.

[redacted]

Constructs a new rational number to represent the ratio (0/1).

[redacted]

Returns this rational number's [redacted] value; for example, if the ratio is (3/5), returns [redacted].

[redacted]

Returns this rational number's [redacted] value; for example, if the ratio is (3/5), returns [redacted].

[redacted]

Returns a [redacted] representation of this rational number, such as [redacted]. You may want to omit [redacted] of [redacted], returning [redacted] instead of [redacted].
An extra challenge would be to maintain your [redacted] objects in reduced form, avoiding rational numbers such as [redacted] in favor of [redacted], or avoiding [redacted] in favor of [redacted]. Another possible extra feature would be methods to add, subtract, multiply, and divide two rational numbers.

2. Write a class called [redacted] that represents a date consisting of a year, month, and day. A [redacted] object should have the following methods:

[redacted]

Constructs a new [redacted] object to represent the given date.

[redacted]

Moves this [redacted] object forward in time by the given number of days.

[redacted]

Moves this [redacted] object forward in time by the given number of seven-day weeks.

[redacted]

Returns the number of days that this [redacted] must be adjusted to make it equal to the given other [redacted].

[redacted]

Returns the day value of this date; for example, for the date 2006/07/22, returns [redacted].

[redacted]

Returns the month value of this date; for example, for the date 2006/07/22, returns [redacted].

[redacted]

Returns the year value of this date; for example, for the date 2006/07/22, returns [redacted].

[redacted]

Returns [redacted] if the year of this date is a leap year. A leap year occurs every four years, except for multiples of 100 that are not multiples of 400. For example, 1956, 1844, 1600, and 2000 are leap years, but 1983, 2002, 1700, and 1900 are not.

[redacted]

Returns a [redacted] representation of this date in year/month/day order, such as [redacted].

3. Write a class named [redacted] that represents a list of items to buy from the market, and another class named [redacted] that represents a request to purchase a particular item in a given quantity (e.g., four boxes of cookies). The [redacted] class should use an array field to store the grocery items and to keep track of its size (number of items in the list so far). Assume that a grocery list will have no more than 10 items. A [redacted] object should have the following methods:

[redacted]

Constructs a new empty grocery list.

Adds the given item order to this list if the list has fewer than 10 items.

Returns the total sum cost of all grocery item orders in this list. The class should store an item quantity and a price per unit. A object should have the following methods:

Constructs an item order to purchase the item with the given name, in the given quantity, which costs the given price per unit.

Returns the total cost of this item in its given quantity. For example, four boxes of cookies that cost 2.30 per unit have a total cost of 9.20.

Sets this grocery item’s quantity to be the given value.

Chapter 9 Inheritance and Interfaces

9.1 Inheritance Basics
• Nonprogramming Hierarchies
• Extending a Class
• Overriding Methods
• 9.2 Interacting with the Superclass
• Calling Overridden Methods
• Accessing Inherited Fields
• Calling a Superclass's Constructor
• [redacted] Behavior
• The [redacted] Class
• The [redacted] Method
• The [redacted] Keyword
9.3 Polymorphism
• Polymorphism Mechanics
• Interpreting Inheritance Code
• Interpreting Complex Calls
9.4 Inheritance and Design
• A Misuse of Inheritance
• Is-a Versus Has-a Relationships
• [redacted]
9.5 Interfaces
• An Interface for Shapes
• Implementing an Interface
• Benefits of Interfaces
9.6 Case Study: Financial Class Hierarchy
• Designing the Classes
• Redundant Implementation
• Abstract Classes

Introduction

In this chapter, we will explore two of the most important techniques the Java language provides to help you write better structured solutions. Inheritance allows you to share code between classes to reduce redundancy and lets you treat different classes of objects in the same way. Interfaces allow you to treat several different classes of objects the same way without sharing code.

It is difficult to show the usefulness of inheritance and interfaces in simple examples, but in larger and more complex projects these techniques are invaluable. The Java class libraries make extensive use of these two features. Inheritance and interfaces make it possible to create well-structured code on a scale that is as large as the entire Java class libraries.

9.1 Inheritance Basics

We'll begin our discussion of inheritance by exploring how the concept originated and considering a nonprogramming example that will lead us toward programming with inheritance in Java.

Large programs demand that we write versatile and clear code on a large scale. In this textbook, we've examined several ways to express programs more concisely and elegantly on a small scale. Features like static methods, parameterization, loops, and classes help us organize our programs and extract common features that can be used in many places. This general practice is called *code reuse.*

Code Reuse

The practice of writing program code once and using it in many contexts.

DID YOU KNOW?

The Software Crisis

Software has been getting more and more complicated since the advent of programming. By the early 1970s, teams writing larger and more complex programs began to encounter some common problems. Despite much effort, software projects were running over budget and were not being completed on time; also, the software often had bugs, didn't do what it was supposed to do, or was otherwise of low quality. In his 1975 book *The Mythical Man-Month: Essays on Software Engineering*, software engineer Fred Brooks argued that adding manpower to a late software project often made it finish even later. Collectively, these problems came to be called the "software crisis."

A particularly sticky issue involved program maintenance. Companies found that they spent much of their time not writing new code but modifying and maintaining existing code (also called *legacy code*). This proved to be a difficult task, because it was easy to write disorganized and redundant code. Maintenance of such code was likely to take a long time and to introduce new bugs into the system.

The negative effects of the software crisis and maintenance programming were particularly noticeable when graphical user interfaces became prominent in the 1980s. User interfaces in graphical systems like Microsoft Windows and Apple's Mac OS X were much more sophisticated than the text interfaces that preceded them. The original graphical programs were prone to redundancy because they had to describe in detail how to implement buttons, text boxes, and other onscreen components. Also, the graphical components themselves contained a lot of common states and behavior, such as particular sizes, shapes, colors, positions, or scrollbars.

Object-oriented programming provides us with a feature called inheritance that increases our ability to reuse code by allowing one class to be an extension of another. Inheritance also allows us to write programs with hierarchies of related object types.

Nonprogramming Hierarchies

In order to use inheritance, you'll want to identify similarities between different objects and classes in your programs. Let's start by looking at a nonprogramming example: a hierarchy of employees at a company.

Imagine a large law firm that hires several classes of employees: lawyers, general secretaries, legal secretaries, and marketers. The company has a number of employee rules about vacation and sick days, medical benefits, harassment regulations, and so on. Each subdivision of the company also has a few of its own rules; for example, lawyers may use a different form to ask for vacation leave than do secretaries.

Suppose that all the employees attend a common orientation where they learn the general rules. Each employee receives a 20-page manual of these rules to read. A mixed group of employees could attend the orientation together: Lawyers, secretaries, and marketers all might sit in the same orientation group.

Afterward, the employees go to their subdivisions and receive secondary, smaller orientations covering any rules specific to those divisions. Each employee receives a smaller manual, two or three pages in length, covering that subdivision's specific rules.

Some rules are added to those in the general 20-page manual, and a few are replaced. For example, one class of employees may get three weeks of vacation instead of two, and one class may use a pink form to apply for time off rather than the yellow form listed in the 20-page manual. Each class has its own submanual with unique contents as shown in Figure 9.1.

Figure 9.1 A hierarchy of employee manuals

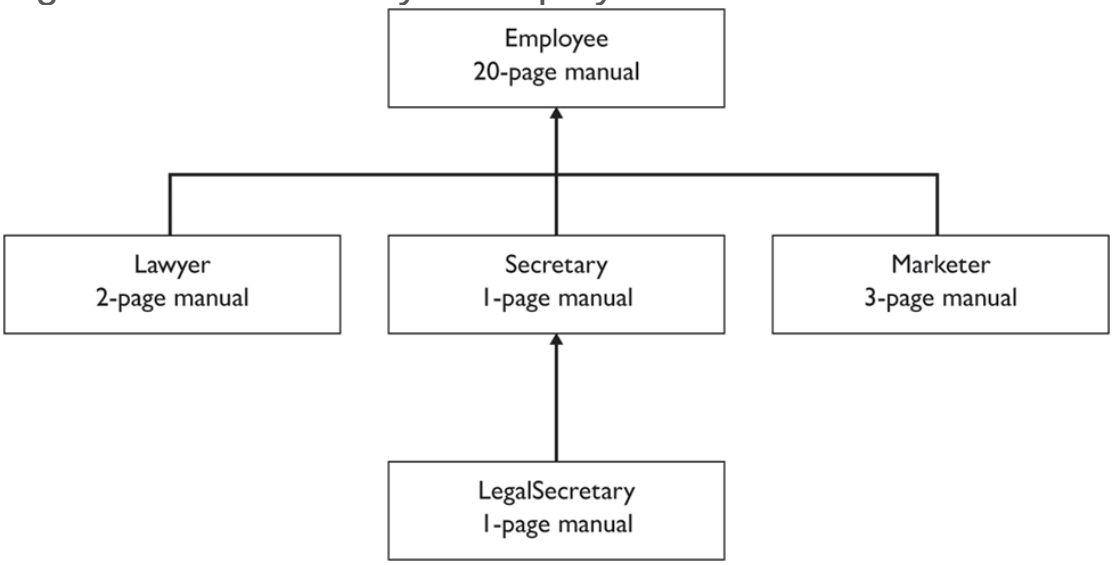


Figure 9.1 Full Alternative Text

An alternative solution would be to give every employee a large manual containing both the applicable general rules and the rules of their subdivisions. For example, there might be a 22-page manual for the lawyers, a 21-page manual for secretaries, and a 23-page manual for marketers. The consolidation might even save a few pages. So why does the company bother to generate two manuals for every employee?

The main reason has to do with redundancy and maintenance. The 22-page lawyer manual contains a lot of the same text as the 21-page secretary manual. If a common rule is changed under the one-manual scenario, all the manuals need to be updated

individually, which is a tedious process. Making the same change to many manuals is also likely to introduce errors, because it is easy to make the change in one copy but forget to do it in another.

In addition, there's a certain practical appeal to using the shorter, more specific manuals. Someone who wants to know all the rules that are specific to lawyers can simply read the 2-page lawyer manual, rather than combing through a 22-page lawyer manual trying to spot differences.

There are two key ideas here:

1. It's useful to be able to specify a broad set of rules that will apply to many related groups (the 20-page manual).
2. It's also useful to be able to specify a smaller set of rules specific to a particular group, and to be able to replace some rules from the broad set (e.g., "use the pink form instead of the yellow form").

An important thing to notice about the categories is that they are hierarchical. For example, every legal secretary is also a secretary, and every marketer is also an employee. In a pinch, you could ask a legal secretary to work as a standard secretary for a short period, because a legal secretary is a secretary. We call such a connection an *is-a relationship.*

Is-a Relationship

A hierarchical connection between two categories in which one type is a specialized version of the other.

An is-a relationship is similar to the idea of a role. A legal secretary can also fill the roles of a secretary and an employee. A lawyer can also fill the role of an employee. A member of a subcategory can add to or change behavior from the larger category. For example, a legal secretary adds the ability to file legal briefs to the secretary role and may change the way in which dictation is taken.

Each group of employees in our example is analogous to a class in programming. The different employee groups represent a set of related classes connected by is-a relationships. We call such a set of classes an *inheritance hierarchy.*

Inheritance Hierarchy

A set of hierarchical relationships between classes of objects.

As you'll see, inheritance hierarchies are commonly used in Java to group related classes of objects and reuse code between them.

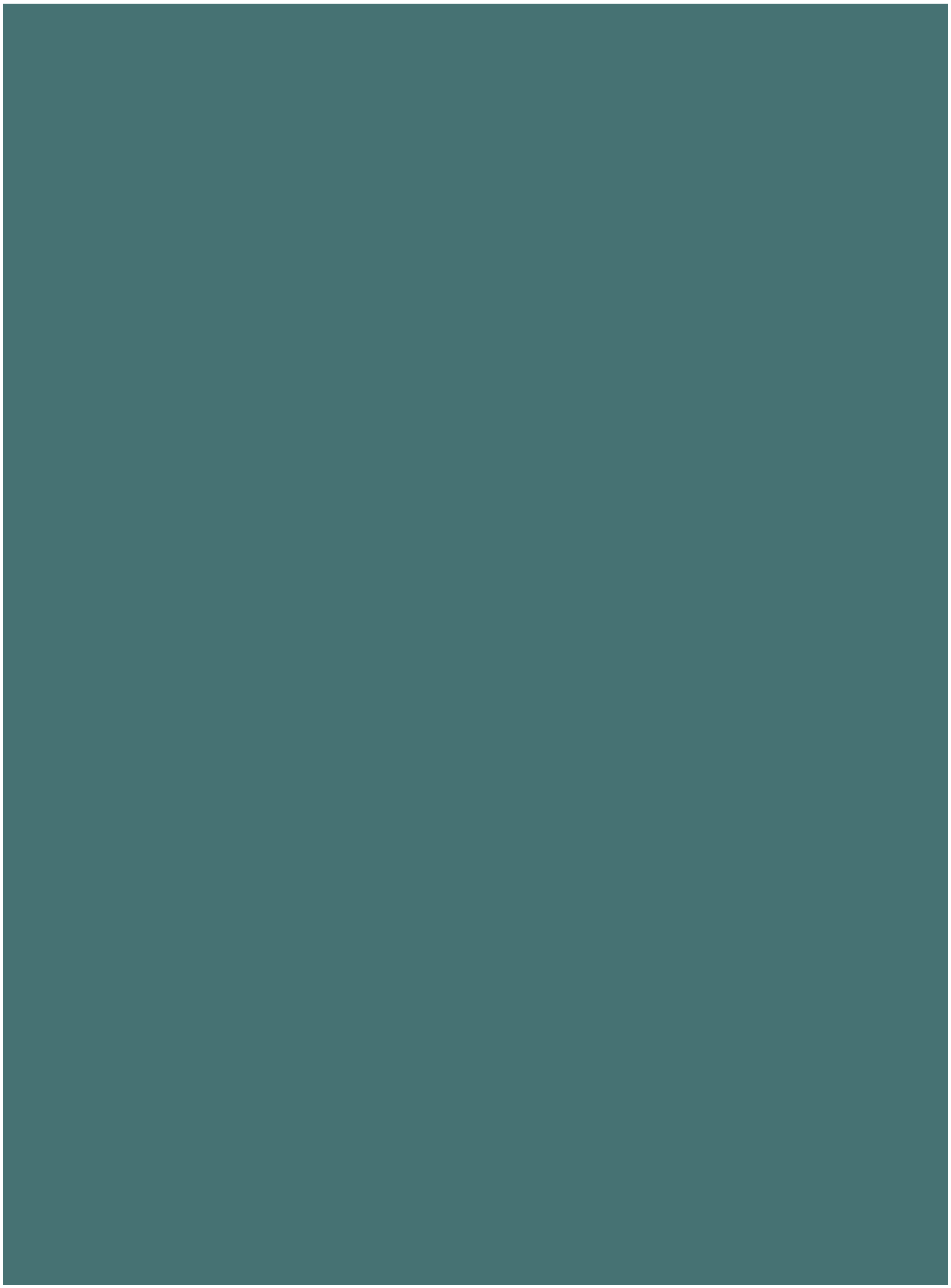
Extending a Class

The previous section presented a nonprogramming example of hierarchies. But as an exercise, we could write small Java classes to represent those categories of employees. The code will be a bit silly but will illustrate some important concepts.

Let's imagine that we have the following rules for our employees:

- Employees work 40 hours per week.
- All employees earn a salary of $40,000 per year, with the exception of marketers, who make $50,000 per year, and legal secretaries, who make $45,000 per year.
- Employees have two weeks of paid vacation leave per year, with the exception of lawyers, who have three weeks of vacation leave.
- Employees use a yellow form to apply for vacation leave, with the exception of lawyers, who use a special pink form.
- Each type of employee has unique behavior: Lawyers know how to handle lawsuits, marketers know how to advertise, secretaries know how to take dictation, and legal secretaries know how to file legal briefs.

Let's write a class to represent the common behavior of all employees. (Think of this as the 20-page employee manual.) We'll write methods called [redacted], [redacted], [redacted], and [redacted] to represent these behaviors. To keep things simple, each method will just return some value representing the default employee behavior, such as the $40,000 salary and the yellow form for vacation leave. We won't declare any fields for now. Here is the code for the basic [redacted] class:



Now let’s think about implementing the [redacted] subcategory. As we mentioned in the previous section, every [redacted] is also an [redacted] and, consequently, retains the abilities that [redacted] have. Secretaries also have one additional ability: the ability to take dictation. If we wrote [redacted] as a standalone class, its code would not reflect this relationship very elegantly. We would be forced to repeat all of the same methods from [redacted] with identical behavior. Here is the redundant class:

```
public int


return


public double


return
```

```
public int


return


public


return


public void
```

The only code unique to the [redacted] class is its [redacted] method. What we'd really like to do is to be able to copy the behavior from class [redacted] without rewriting it in the [redacted] class file.

```
public class Secretary {

    copy all the methods from the Employee class.

    // this is the only added behavior

    public void takeDictation() {

        System.out.println("I know how to take dictation.");

    }

}
```

Fortunately, Java provides a mechanism called *inheritance* that can help us remove this sort of redundancy between similar classes of objects. Inheritance allows the programmer to specify a

relationship between two classes in which one class includes (“inherits”) the state and behavior of another.

Inheritance (Inherit)

A programming technique that allows a derived class to extend the functionality of a base class, inheriting all of its state and behavior.

The derived class, more commonly called the *subclass*, inherits all of the state and behavior of its parent class, commonly called the *superclass.*

Superclass

The parent class in an inheritance relationship.

Subclass

The child, or derived, class in an inheritance relationship.

We say that the subclass *extends* the superclass because it not only receives the superclass’s state and behavior but can also add new state and behavior of its own. The subclass can also

replace inherited behavior with new behavior as needed, which we'll discuss in the next section.

A Java class can have only one superclass; it is not possible to extend more than one class. This is called *single inheritance.* On the other hand, one class may be extended by many subclasses.

To declare one class as the subclass of another, place the [redacted] keyword followed by the superclass name at the end of the subclass header. The general syntax is the following:

We can rewrite the [redacted] class to extend the [redacted] class. This will create an is-a relationship in which every [redacted] also is an [redacted]. [redacted] objects will inherit copies of the [redacted], [redacted], [redacted], and [redacted] methods, so we won't need to write these methods in the [redacted] class. This will remove the redundancy between the classes.

It's legal and expected for a subclass to add new behavior that wasn't present in the superclass. We said previously that secretaries add an ability not seen in other employees: the ability to take dictation. We can add this to our otherwise empty [redacted] class. The following is the complete [redacted] class:

```
public class

extends Employee



public void
```

This concise new version of the [redacted] class has the same behavior as the longer class shown before. Like the two-page specialized manual, this class shows only the features that are unique to the specific job class. In this case, it is very easy to see that the unique behavior of secretaries in our system is to take dictation.

The following client code would work with our new [redacted] class:

```
public class


public static void


new
```

new

The code would produce the following output:

Notice that the first four methods produce the same output for both objects, because [redacted] inherits that behavior from [redacted]. The fifth line of [redacted] output reflects the new extended behavior of the [redacted] method.

Overriding Methods

We can use inheritance in our other types of [redacted], creating [redacted], [redacted], and [redacted] classes that are subclasses of [redacted]. But while the [redacted] class merely adds behavior to the standard [redacted] behavior, these other classes also need to replace some of the standard [redacted] behavior with their own. Lawyers receive three weeks of vacation and use a pink form to apply for vacation, and they know how to handle lawsuits. Legal secretaries receive $45,000 a year (a $5,000 raise over the standard amount), they know how to take dictation (like regular secretaries), and they can file legal briefs. Marketers receive $50,000 a year (a $10,000 raise over the standard amount), and they know how to advertise.

We'd like these new classes to inherit most of the behavior from the [redacted] class, but we need to change or replace certain parts of the behavior. It's legal to replace superclass behavior by writing new versions of the relevant method(s) in the subclasses. The new version in the subclass will replace the one inherited from [redacted]. This idea of replacing behavior from the superclass is called *overriding.*

Override

[redacted]

To implement a new version of a method to replace code that would otherwise have been inherited from a superclass.

To override a method, just write the method you want to replace in the subclass. No special syntax is required, but the method's name and signature must exactly match those of the method from the superclass.

Here is the [redacted] class that extends [redacted] and overrides the relevant methods:

```
public class

extends
```

```
public int

return

public

return
```

```
public void
```

The [redacted] class could also be written to extend [redacted], but it has more in common with [redacted]. It is legal for a class to extend a class that itself extends a class, creating a multi-level hierarchy. So we will write the class for [redacted] as an extension of the class [redacted] so it inherits the ability to take dictation:

```
```

```
public class

extends









public double



return
```

```
public void
```

The following client program uses our [redacted] and [redacted] classes. Notice that the legal secretary inherits not only the normal employee behavior, but also the behavior to take dictation from the standard secretary:

```
public class

public static void
```

new

new

The program produces the following output:

Be careful not to confuse overriding with overloading. Overloading, introduced in Chapter 3, occurs when one class contains multiple methods that have the same name but different parameter signatures. Overriding occurs when a subclass substitutes its own version of an otherwise inherited method that uses exactly the same name and the same parameters.

9.2 Interacting with the Superclass

The classes in the previous sections demonstrated inheritance in classes containing only methods. But you'll want to write more meaningful classes that use inheritance with fields, methods, and constructors. These subclasses require more complex interaction with the state and behavior they inherit from their superclass. To show you how to perform this interaction properly, we'll need to introduce a new keyword called [redacted].

Calling Overridden Methods

Suppose things are going well at our example legal firm, so the company decides to give every employee a $10,000 raise. What is the best way to enact this policy change in the code we've written? We can edit the [redacted] class and change its [redacted] method to return 50000 instead of 40000. But some of the other types of employees have salaries higher than the original $40,000 rate (legal secretaries at $45,000 and marketers at $50,000), and these will need to be raised as well. We'll end up needing to change several files to enact this single overall raise.

The problem is that our existing code does not represent the relationship between the various salaries very well. For example, in the [redacted] class, instead of saying that we want to return 45000, we'd like to return the [redacted] class's salary plus $5000. You may want to write code like the following, but it does not work, because the [redacted] method has overridden the one from the superclass, meaning that the code that follows calls itself infinitely:

You also can't just call the [redacted] version of [redacted] by writing [redacted], because that is the syntax for executing static methods, not instance methods of objects. Instead, Java provides a keyword [redacted] that refers to a class's superclass. This keyword is used when calling a superclass method or constructor. Here is the general syntax for calling an overridden method using the [redacted] keyword:

The correct version of the legal secretary's salary code is the following. Writing the marketer's version is left as an exercise.

```
super.getSalary()
```

Accessing Inherited Fields

To examine the interactions of more complex classes in a hierarchy, let's shift to a more complex example than the employee classes we've been using so far. In Chapter 8's case study, we built a [redacted] class representing purchased shares of a given stock. Here's the code for that class, which has been shortened a bit for this section by removing tests for illegal arguments:

```
public class

private

private int

private double

public
```

```
this
```

```
public double
```

```
double
```

```
double
```

```
return
```

Now let’s imagine that you want to create a type of object for stocks which pay dividends. Dividends are profit-sharing payments that a corporation pays its shareholders. The amount that each shareholder receives is proportional to the number of shares that person owns. Not every stock pays dividends, so you wouldn’t want to add this functionality directly to the class. Instead, you should create a new class called that extends and adds this new behavior.

Each [redacted] object will inherit the symbol, total shares, and total cost from the [redacted] superclass. You'll simply need to add a field to record the amount of the dividends paid:

Using the [redacted] field, you can write a method in the [redacted] class that lets the shareholder receive a per-share dividend. Your first thought might be to write code like the following, but this won't compile:

A [redacted] cannot access the [redacted] field it has inherited, because [redacted] is declared private in [redacted]. A subclass may not refer directly to any private fields that were

declared in its superclass, so you'll get a compiler error like the following:

It may seem unnecessarily restrictive that a class isn't able to examine the fields it has inherited, since those fields are part of the object. The reason Java is built this way is to prevent a subclass from violating the encapsulation of the superclass. If a superclass object held sensitive data and subclasses were allowed to access that data directly, they could change it in malicious ways the superclass did not intend.

The solution here is to use accessor or mutator methods associated with our fields to access or change their values. The class doesn't have a public accessor method for the field, but you can now add a method to the class:

Here is a corrected version of the [redacted] method that uses the [redacted] method from [redacted]:

```
getTotalShares();
```

The [redacted] subclass is allowed to call the public [redacted] method, so the code now behaves properly. If we had a similar situation in which subclasses needed to modify total shares from a stock, the [redacted] class would need to provide a [redacted] method or something similar.

Calling a Superclass's Constructor

Unlike other behaviors, constructors are not inherited. You'll have to write your own constructor for the [redacted] class, and when you do so the problem of the inability to access private fields will arise again.

The [redacted] constructor should accept the same parameter as the [redacted] constructor: the stock symbol. It should have the same behavior as the [redacted] constructor but should also

initialize the [redacted] field to `0.0`. The following constructor implementation might seem like a good start, but it is redundant with [redacted]'s constructor and won't compile successfully:

The compiler produces four errors: one error for each line that tries to access an inherited private field, and a message about a missing [redacted] constructor:

The first problem is that even though a [redacted] does contain the [redacted], [redacted], and [redacted] fields by inheritance, it cannot refer to them directly because they were declared private in the [redacted] class.

The second problem—the missing [redacted] constructor—is a subtle and confusing detail of inheritance. A subclass's constructor must always begin by calling a constructor from the superclass. The reason is that a [redacted] object partially consists of a [redacted] object, and you must initialize the state of that [redacted] object first by calling a constructor for it. If you don't do so explicitly, the compiler assumes that [redacted] has a parameterless [redacted] constructor and tries to initialize the [redacted] data by calling this constructor. Since the [redacted] class doesn't actually have a parameterless constructor, the compiler prints a bizarre error message about a missing [redacted] constructor. (It's a shame that the error message isn't more informative.)

The solution to this problem is to explicitly call the [redacted] constructor that accepts a [redacted] symbol as its parameter. Java uses the keyword [redacted] for a subclass to refer to behavior from its superclass. To call a constructor of a superclass, write the keyword [redacted], followed by the constructor's parameter values in parentheses:

In the case of the [redacted] constructor, the following code does the trick. Use the [redacted] keyword to call the superclass constructor, passing it the same [redacted] value that was passed to the [redacted] constructor. This action will initialize the [redacted], [redacted], and [redacted] fields. Then set the initial dividends to `0.0`:

```
super(symbol)
```

The call to the superclass's constructor using [redacted] must be the first statement in a subclass's constructor. If you reverse the order of the statements in [redacted]'s constructor and set the dividends before you call [redacted], you'll get a compiler error like the following:

Here's the [redacted] class so far. The class isn't complete yet because you haven't yet implemented the behavior to make dividend payments:

[redacted]

[redacted] Behavior

To implement dividend payments, we'll begin by writing a method called [redacted] that accepts a dividend amount per share and adds the proper amount to [redacted]'s [redacted] field. The amount per share should be multiplied by the number of shares the shareholder owns:

The dividend payments that are being recorded should be considered to be profit for the stockholder. The overall profit of a [redacted] object is equal to the profit from the stock's price plus any dividends. This amount is computed as the market value (number of shares times current price) minus the total cost paid for the shares, plus the amount of dividends paid.

Notice that you don’t need to use [redacted] in the preceding code. You have to use the [redacted] keyword only when you are accessing overridden methods or constructors from the superclass. [redacted] doesn’t override the [redacted] method, so you can call it without the [redacted] keyword.

Because the profit of a [redacted] object is computed differently from that of a regular [redacted] object, you should override the [redacted] method in the [redacted] class to implement this new behavior. An incorrect initial attempt might look like the following code:

The preceding code has two problems. First, you can’t refer directly to the various fields that were declared in [redacted]. To get around this problem, you can add accessor methods for each field. The second problem is that the code is redundant: It duplicates much of the functionality from [redacted]’s [redacted] method, which was shown earlier. The only new behavior is the adding of dividends into the total.

To remove this redundancy, you can have [redacted]’s [redacted] method call [redacted]’s [redacted] method as part of its

computation. However, since the two methods share the same name, you must explicitly tell the compiler that you want to call 's version. Again, you do this using the keyword.

Here is the corrected code, which does compile and eliminates the previous redundancy:

```
super.getProfit(currentPrice)
```

And here is the code for the completed class:

```
public class

extends




private double

```

```
public

```

```
super

```

```
public double

double



return super









public void

double

```

It’s possible to have a deeper inheritance hierarchy with multiple layers of inheritance and overriding. However, the [redacted] keyword reaches just one level upward to the most recently overridden version of the method. It’s not legal to use [redacted] more than once in a row; you cannot make calls like [redacted]. If you need such a solution, you’ll have to find a workaround such as using different method names.

The [redacted] Class

A class called [redacted] serves as the ultimate superclass for all other Java classes, even those that do not declare explicit superclasses in their headers. In fact, classes with headers that do not have extends clauses are treated as though their headers say [redacted] when they are compiled. (You can explicitly write the [redacted] in a class header, but this is unnecessary and not a common style.)

The [redacted] class contains methods that are common to all objects. Table 9.1 summarizes the methods of the [redacted] class.

Note that some of the methods are not public and therefore cannot be called externally.

Table 9.1 Methods of the Object Class

[redacted]	Creates and returns a copy of the object (not a public method)
[redacted]	Indicates whether the other object is equal to this one
[redacted]	Called automatically by Java when objects are destroyed (not public)
[redacted]	Returns information about the type of the object
[redacted]	Returns a number associated with the object; used in some collections
[redacted]	Returns the state of the object as a [redacted]
[redacted]	Advanced methods for multithreaded programming

In Chapter 8, we mentioned that without a [redacted] method in a class, the objects will not print properly. For example, our [redacted] class printed [redacted] by default before we wrote its [redacted] method. This default message was the behavior of the [redacted] class's [redacted] method, which we inherited in our [redacted] class. The [redacted] class provides a generic [redacted] output that will work for every class: the class name followed by some

internal numeric information about the object. When we wrote our own [redacted] method, we overrode this default behavior.

It is sometimes useful to refer to the [redacted] class in your programs. For example, if you wish to write a method that can accept any object as a parameter, you can declare a parameter of type [redacted]:

Of course, since your parameter can be anything, you are only allowed to call the methods from the [redacted] class on it, such as [redacted] or [redacted]. It is also legal to have a method whose return type is [redacted].

The [redacted] class is used extensively in the Java class libraries. For example, the [redacted] method of the [redacted] class (the class of which [redacted] is an instance) accepts a parameter of type [redacted], which allows you to print any object to the console.

The [redacted] Method

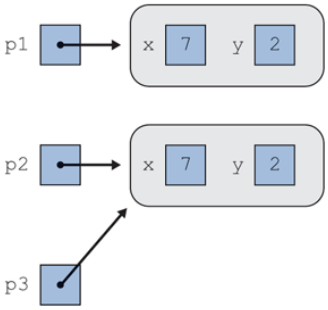
For several chapters now, you have used the == operator to compare for equality. You have seen that this operator does not behave as expected when used on objects, because it is possible to have two distinct objects with equivalent states, such as two objects with the coordinates (5, 2). This observation is a reminder that an object has an *identity* and is distinct from other objects, even if another object happens to have the same state.

A nonprogramming analogy would be if you and your friend both purchased identical pairs of shoes. They are in some ways equivalent, but you still consider them distinct and separate items. You might not want to share them. And the items' states are not linked in any way. Over time, their state might become visibly unequal, such as if one of you wore your new shoes more often than the other.

The == operator does not behave as expected with objects because it tests whether two objects have the same identity. The == comparison actually tests whether two variables refer to the same object, not whether two distinct objects have the same state. Consider the following three variable declarations:

The following diagram represents the state of these objects and the variables that refer to them. Notice that == is not a reference to a third object but a second reference to the object that was

referred to by [redacted]. This means that a change to [redacted], such as a call of its [redacted] method, would also be reflected in [redacted]:



9.2-2 Full Alternative Text

In the case of the preceding [redacted] objects, the expression [redacted] would evaluate to false because [redacted] and [redacted] do not refer to the same object. The object referred to by [redacted] has the same state as [redacted]'s object, but they have different identities. The expression [redacted] would evaluate to [redacted], though, because [redacted] does refer to the same object as [redacted].

Often, when comparing two objects, we want to know whether the objects have the same state. To perform such a comparison, we use a special method called [redacted]. Every Java object contains an [redacted] method that it uses to compare itself to other objects.

The previous section mentioned that a class without a [redacted] method receives a default version of the method. Similarly, a class without an [redacted] method receives a default version that uses the most conservative definition of equality, considering two objects to be equal only if they have the same identity. This means that the default [redacted] method behaves identically to the [redacted] operator. If you want a method that will behave differently, you must write your own [redacted] method to replace the default behavior.

A proper [redacted] method performs a comparison of two objects' states and returns [redacted] if the states are the same. With the

preceding [redacted] objects, we'd like the expressions [redacted], [redacted], and [redacted] to evaluate to [redacted] because the [redacted] all have the same (x, y) coordinates.

You can imagine a piece of client code that examines two [redacted] objects to see whether they have the same [redacted] and [redacted] field values:

[redacted]

But the [redacted] functionality should actually be implemented in the [redacted] class itself, not in the client code. Rather than having two [redacted], [redacted] and [redacted], the [redacted] method considers the first object to be the "implicit parameter" and accepts the second object as a parameter. It returns [redacted] if the two objects are equal.

The following code is an initial implementation of the [redacted] method that has several flaws:

[redacted]

An initial flaw we can correct is that the preceding code doesn't make good use of "Boolean Zen," described in Chapter 5. Recall that when your code uses an [redacted] statement to return a [redacted] value of [redacted] or [redacted], often you can directly return the value of the [redacted] statement's condition:

It's legal for the [redacted] method to access [redacted]'s fields directly, so we can optionally modify this further. Private fields are visible to their entire class, including other objects of that same class, so it is legal for one [redacted] object to examine the fields of another:

Some programmers respect [redacted]'s encapsulation even against other [redacted] objects and therefore would not make the preceding change.

To keep our [redacted] method consistent with other Java classes, we must also make a change to its header. The [redacted] method's parameter should not be of type [redacted]. The method must instead accept a parameter of type [redacted]:

```
Object o
```

A variable or parameter of type [redacted] can refer to any Java object, which means that any object may be passed as the parameter to the [redacted] method. Thus, we can compare [redacted] objects against any type of object, not just other [redacted]. For example, an expression such as [redacted] would now be legal. The [redacted] method should return [redacted] in such a case because the parameter isn't a [redacted].

You might think that the following code would correctly compare the two [redacted] objects and return the proper result. Unfortunately, it does not even compile successfully:

```
```

The Java compiler doesn't allow us to write an expression such as [redacted] because it doesn't know ahead of time whether [redacted]'s object will have a field called [redacted]. The preceding code produces errors such as the following for each of [redacted]'s fields that we try to access:

If we want to treat [redacted] as a [redacted] object, we must cast it from type [redacted] to type [redacted]. We've already discussed typecasting to convert between primitive types, such as casting [redacted] to [redacted]. Casting between object types has a different meaning. A cast of an object is a promise to the compiler. The cast is your assurance that the reference actually refers to a different type and that the compiler can treat it as that type. In our method, we'll write a statement that casts [redacted] into a [redacted] object so the compiler will trust that we can access its [redacted] and [redacted] fields:

Don't forget that if your object has fields that are objects themselves, such as a string or [redacted] as a field, then those fields

should be compared for equality using their [redacted] method and not using the [redacted] operator.

The [redacted] Keyword

By changing our [redacted] method's parameter to type [redacted], we have allowed objects that are not [redacted] to be passed. However, our method still doesn't behave properly when clients pass these objects. An expression in client code such as [redacted] will produce an exception like the following at runtime:

[redacted]

The exception occurs because it is illegal to cast a [redacted] into a [redacted]; these are not compatible types of objects. To prevent the exception, our [redacted] method will need to examine the type of the parameter and return [redacted] if it isn't a [redacted]. The following pseudocode shows the pattern that the code should follow:

```
public boolean equals(Object o) {

    if (o is a Point object) {
```

```
        compare the x and y values.

    } else {

        return false, because o is not a Point object.

    }

}
```

An operator called [redacted] tests whether a variable refers to an object of a given type. An [redacted] test is a binary expression that takes the following form and produces a [redacted] result:

```

```

The following JShell interaction shows some example expressions using [redacted] and their results. We declare the variables as type [redacted] to illustrate that an [redacted] variable can refer to any type of Java object.

```
jshell>



jshell>

```

```
jshell>


jshell>


jshell>


jshell>


jshell>
```

```
jshell>




jshell>
```

The [redacted] operator is unusual because it looks like the name of a method but is used more like a relational operator such as [redacted] or [redacted]. It is separated from its operands by spaces but doesn't require parentheses, dots, or any other notation. The operand on the left side is generally a variable, and the operand on the right is the name of the class against which you wish to test.

We must examine the parameter [redacted] in our [redacted] method to see whether it is a [redacted] object. The following code uses the [redacted] keyword to implement the [redacted] method correctly:

```
```

```
if (o instanceof Point) {




} else {


return false;
```

You might think that our [redacted] test would allow us to remove the type cast below it. After all, the [redacted] test ensures that the comparison occurs only when [redacted] refers to a [redacted] object. However, the type cast cannot be removed because the compiler doesn't allow the code to compile without it.

A nice side benefit of the [redacted] operator is that it produces a [redacted] result when [redacted] is [redacted]. Thus, if the client code contains an expression such as [redacted], it will correctly return [redacted] rather than throwing a [redacted].

Many classes implement an [redacted] method like ours, so much of the preceding [redacted] code can be reused as boilerplate code. The following is a template for a well-formed [redacted] method. The [redacted] test and type cast are likely the first two things you'll want to do in any [redacted] method that you write:

9.3 Polymorphism

One of the most powerful benefits of inheritance is that it allows client code to treat different kinds of objects in the same way. For example, with the employee class hierarchy described earlier, it's possible for client code to create an array or other data structure that contains both lawyers and legal secretaries, and then perform operations on each element of that array. The client code will behave differently depending on the type of object that is used, because each subclass overrides and changes some of the behavior from the superclass. This ability for the same code to be used with several different types of objects is called *polymorphism.*

Polymorphism

The ability for the same code to be used with several different types of objects and for the code to behave differently depending on the actual type of object used.

Polymorphism is made possible by the fact that the type of a reference variable (one that refers to an object) does not have to exactly match the type of the object it refers to. More specifically,

it is legal for a superclass variable to refer to an object of its subclass. The following is a legal assignment statement:

When we were studying the primitive types, we saw cases in which a variable of one type could store a value of another type (for example, an [redacted] value can be stored in a [redacted] variable). In the case of primitive values, Java converts variables from one type to another: [redacted] values are automatically converted to [redacted] when they are assigned.

When a subclass object is stored in a superclass variable, no such conversion occurs. The object referred to by [redacted] really is a [redacted] object, not an [redacted] object. If we call methods on it, it will behave like a [redacted] object. For example, the call of [redacted] returns [redacted], which is the [redacted]'s behavior, not the [redacted]'s.

This ability for variables to refer to subclass objects allows us to write flexible code that can interact with many types of objects in the same way. For example, we can write a method that accepts an [redacted] as a parameter, returns an [redacted], or creates an array of [redacted] objects. In any of these cases, we can substitute a [redacted], [redacted], or other subclass object of [redacted], and the code will still work. Even more importantly, code will actually behave differently depending on which type of object is used, because each subclass overrides and changes some of the behavior from the superclass. This is polymorphism at work.

Here is an example test file that uses [redacted] objects polymorphically as parameters to a static method:

```
public class

public static void

new

new
```

```
new


new


public static void
```

Notice that the method lets us pass many different types of [redacted] as parameters, and it produces different behavior depending on the type that is passed. Polymorphism gives us this flexibility. The last token of output printed for each employee is the employee object itself, which calls the [redacted] method on the object. Our classes don't have [redacted] methods, so the program uses the default behavior, which prints the class name plus some extra hexadecimal characters. This allows us to distinguish the classes in the output. The program produces output such as the following:

The word “polymorphism” comes from the Greek words “poly” and “morph,” which mean “many” and “forms,” respectively. The lines of code in the [redacted] method are polymorphic because their behavior will take many forms depending on what type of employee is passed as the parameter.

The program doesn’t know which [redacted] or [redacted] method to call until it’s actually running. When the program reaches a particular call to an object’s method, it examines the actual object to see which method to call. This concept has taken many names over the years, such as *late binding*, *virtual binding*, and *dynamic dispatch.*

When you send messages to an object stored in a variable of a superclass type, it is legal only to call methods that are known to the superclass. For example, the following code will not compile because the [redacted] class does not have a [redacted] or [redacted] method:

The compiler does not allow this code because the variable [redacted] could theoretically refer to any kind of employee, including one that does not know how to take dictation or file legal briefs. Even though we know it must refer to a [redacted] because the code is so simple, the compiler enforces this rule strictly and returns an error message. The same thing happens if you write a method that accepts an [redacted] as a parameter; you cannot call subclass-only methods such as [redacted], [redacted], or [redacted] on the object that is passed in, even if the actual object might have those methods.

The variable's type can be any type equal or higher in the inheritance hierarchy compared to the actual object. So we could store a legal secretary in a variable of type [redacted], which would allow us to execute any standard secretary behavior, including taking dictation:

It is legal to cast a variable into a different type of reference in order to make a call on it. This does not change the type of the object, but it promises the compiler that the variable really refers

to an object of the other type. For example, the following code works successfully:

```
Employee
```

You can only cast a reference to a compatible type, one above or below it in its inheritance hierarchy. The preceding code will compile, but it would crash at runtime if the variable did not actually refer to an object of type or one of its subclasses.

Polymorphism Mechanics

Inheritance and polymorphism introduce some complex new mechanics and behavior into programs. One useful way to get the hang of these mechanics is to perform exercises to interpret the behavior of programs with inheritance. The main goal of these exercises is to help you understand in detail what happens when a Java program with inheritance executes.

The program developed in the previous section serves as a template for inheritance questions of the following form: Given a certain hierarchy of classes, what behavior would result if we created several objects of the different types and called their various methods on them?

In order to use polymorphism and keep our program compact, we'll store the objects in an array. In the case of the [redacted] hierarchy, it's legal for an object of class [redacted], [redacted], or any other subclass of [redacted] to reside as an element of an [redacted].

The following program produces the same output as the [redacted] program from the previous section:

```
public class

public static void

new

new
```

new

new

for

Even if you didn’t understand inheritance, you might be able to deduce some things about the hierarchy from the classes’ names and the relationships among employees in the real world. So let’s take this exercise one step further. Instead of using descriptive names for the classes, we’ll use letters so that you have to read the code in order to determine the class relationships and behavior.

Assume that the following classes have been defined:

```
public class
```

```
public void
```

```
public void
public
return
```

```
public class

extends




public void
```

```
public class

extends
```

```
public void
public
return
```

```
public class

extends



public void
```

Consider the following client code that uses these classes. It takes advantage of the fact that every other class extends class [redacted] (either directly or indirectly), so the array can be of type [redacted]. When you call methods on the elements of the array, polymorphic behavior will result:

```
public class


public static void


new

new

new


for

int
```

It’s difficult to interpret such code and correctly determine its output. In the next section we’ll present techniques for doing so.

Interpreting Inheritance Code

To determine the output of a polymorphic program like the one in the previous section, you must determine what happens when

each element is printed (i.e., when its [redacted] method is called) and when its [redacted] and [redacted] methods are called. You can draw a diagram of the classes and their methods to see the hierarchy ordering and see which methods exist in each class. Draw each class as a box listing its methods, and connect subclasses to their superclasses with arrows, as shown in Figure 9.2. (This is a simplified version of a common design document called a UML class diagram.)

Figure 9.2 Hierarchy of classes A, B, C, and D

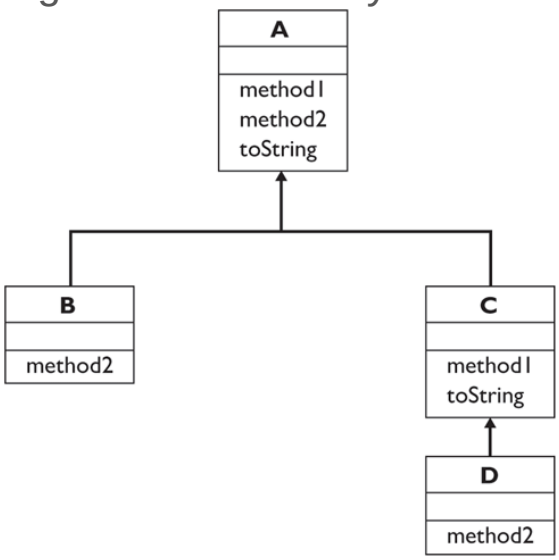


Figure 9.2 Full Alternative Text

A good second step is to write a table that lists each class and its methods' output. Write the output not just for the methods defined in each class, but also for the ones that the class inherits. Since the [redacted] class is at the top of the hierarchy, we'll start there.

When someone calls [redacted] on an [redacted] object, the resulting output is [redacted]. When someone calls [redacted] on an [redacted] object, the resulting output is [redacted]. When someone prints an [redacted] object with [redacted], the resulting output is [redacted]. We can fill in the first column of our table as shown in Table 9.2.

Table 9.2 Method Output for Class

`method1`	
`method2`	

The next layer in the hierarchy is the class, which inherits all the behavior from , except that it overrides the output to be . That means you can fill in the output on your table identically to the output, except that you replace with . Table 9.3 shows the table so far.

Table 9.3 Method Output for Classes and

The class also inherits all the behavior from , but it overrides the output to be and it overrides the method to return . Thus, the output in the table will have the

same second line as the [redacted] output, but we replace [redacted] with [redacted] and [redacted] with [redacted]. Table 9.4 shows the updated table.

Table 9.4 Method Output for Classes [redacted], [redacted], and [redacted]

[redacted]			
[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]

The [redacted] class inherits all the behavior from [redacted], except that it overrides the [redacted] output to say [redacted]. The final output data are shown in Table 9.5.

Table 9.5 Method Output for Classes [redacted], [redacted], [redacted], and [redacted]

[redacted]				
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]	[redacted]

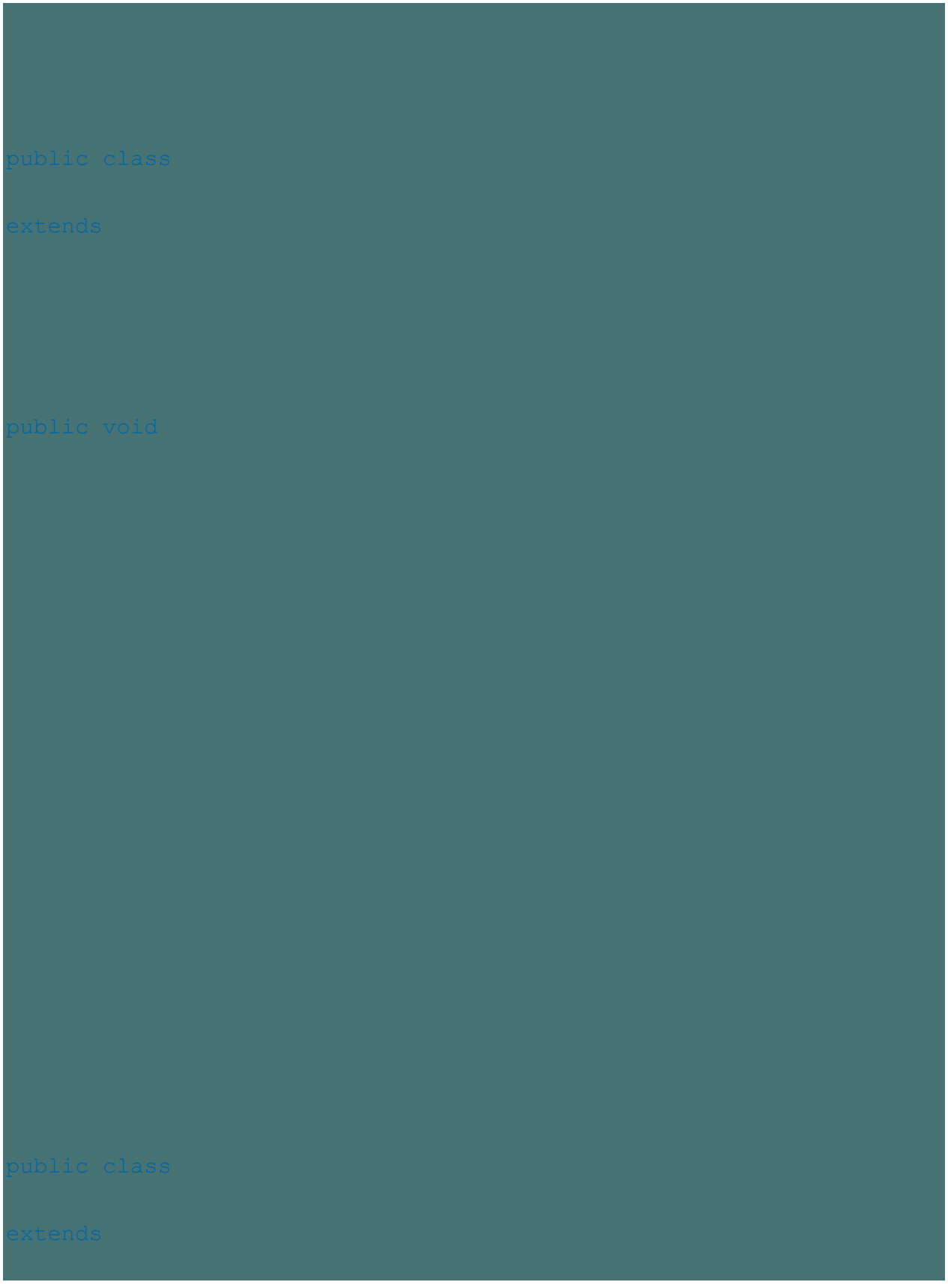
Once you've created your table, you can find the output of the client code. The array contains an [redacted] object, a [redacted] object, a [redacted] object, and a [redacted] object. For each of these it prints the [redacted] output,

then calls [redacted], then calls [redacted], then prints a blank line. When a method gets called on an object, you can look up the output of that method for that type in the table. The following is the complete output for the exercise:

Interpreting Complex Calls

In a more complicated version of the previous inheritance exercise, a class's methods might call each other or interact with the superclass. Consider the following classes. Notice that they are listed in random order and that their methods' behavior is more complicated than the previous example:

```
public class

extends



public void
```



```
public class

extends
```

```
public

return

public void

super
```

```
public class

public

return

public void
```

```
public void

public class

extends

public void
```

In order to determine the best order in which to examine the classes, you could draw a diagram like the one in Figure 9.2. Then make a table and fill in each class's behavior from top to bottom in the hierarchy. First identify the top class in the hierarchy. Look for the one whose header has no clause. In this example, that is the class. Its methods have simple behavior, so we can fill in that row of the table immediately, as shown in Table 9.6.

Table 9.6 Method Output for Class

The next class in the hierarchy is , which extends . The is not overridden, so its output is as it was in the superclass. does override to return . It also

overrides [redacted] to print [redacted] and then call the superclass's ([redacted]'s) version of [redacted]. When there is a call to a superclass's method, we can evaluate its output immediately and put it into our table by looking at the superclass. This means that the [redacted] class's [redacted] prints [redacted]. Table 9.7 shows this information.

Table 9.7 Method Output for Classes [redacted] and [redacted]

[redacted]		
`toString`	[redacted]	[redacted]
`method1`	[redacted]	[redacted]
`method2`	[redacted]	[redacted]

The next class to tackle is [redacted], which extends [redacted]. It does not override [redacted] or [redacted], so these methods produce the same output as they do in superclass [redacted]. Class [redacted] does override [redacted] to print [redacted] and then call [redacted]. Since [redacted] prints [redacted], calling [redacted] on an [redacted] object prints [redacted].

But here's where things get tricky: You shouldn't write this output in your table. The reason will become clear when we look at the [redacted] class, which is a subclass of [redacted] that overrides [redacted]. Because of polymorphism, if you call [redacted] on an [redacted] object, when it makes the inner call to [redacted], it will use the version from the [redacted] class. What you should write into your table for [redacted]'s [redacted] output is that it prints [redacted] and then calls [redacted]. Table 9.8 shows the information for class [redacted].

Table 9.8 Method Output for Classes , , and

`toString`			
`method1`			
`method2`			

Lastly we will examine class , which extends . It does not override , so it produces the same output as in superclass . It overrides only to print . These methods are simple (they don't call any others), so we can write the and output into the table immediately.

Class doesn't override , so you may think that you can copy over its output from superclass . But remember that the inherited prints and then calls . In this case, we are thinking from the perspective of an object. So that inner call to now prints instead of the old output. This means that calling on an object actually prints . Table 9.9 shows the final output for all methods.

Table 9.9 Method Output for Classes , , , and

`toString`				

method1				
method2				

9.4 Inheritance and Design

Inheritance affects the thought processes you should use when designing object-oriented solutions to programming problems. You should be aware of similarities between classes and potentially capture those similarities with inheritance relationships and hierarchies. The designers of the Java class libraries have followed these principles, as we'll see when we examine a graphical subclass in this section.

However, there are also situations in which using inheritance seems like a good choice but turns out to produce poor results. Misuse of inheritance can introduce some pitfalls and problems that we'll now examine.

A Misuse of Inheritance

Imagine that you want to write a program that deals with points in three-dimensional space, such as a three-dimensional game, rendering program, or simulation. A [redacted] class would be useful for storing the positions of objects in such a program.

This seems to be a case in which inheritance will be useful to extend the functionality of existing code. Many programmers would be tempted to have [redacted] extend [redacted] and simply add the new code for the z-coordinate. Here's a quick implementation of a minimal [redacted] class that extends [redacted]:

```
public class

extends



private int









public
```

```
int

int

int




super



this
```

```
public int


return
```

On the surface, this seems to be a reasonable implementation. However, consider the method defined in the class. It compares the x- and y-coordinates of two objects and returns if they are the same:

You might also want to write an method for the class. Two objects are equal if they have the same x-, y-,

and z-coordinates. The following is a working implementation of [redacted] that is correct but stylistically unsatisfactory:

The preceding code compiles and runs correctly in many cases, but it has a subtle problem that occurs when you compare [redacted] objects to [redacted] objects. The [redacted] class's [redacted] method tests whether the parameter is an instance of [redacted] and returns [redacted] if it is not. However, the [redacted] operator will return [redacted] not only if the variable refers to that type, but also if it refers to any of its subclasses. Consider the following test in the [redacted] method of the [redacted] class:

The test will evaluate to [redacted] if [redacted] refers to a [redacted] object or a [redacted] object. The [redacted] operator is like an is-a test, asking whether the variable refers to any type that can fill the role of a [redacted]. By contrast, [redacted]'s [redacted] method tests whether the parameter is an instance of [redacted] and rejects it if it is not. A [redacted] cannot fill the role of a [redacted] (not every [redacted] is a [redacted]), so the method will return [redacted] if the parameter is of type [redacted].

Consequently, the [redacted] behavior is not symmetric when it is used with a mixture of [redacted] and [redacted] objects. The following client code demonstrates the problem:

The code produces the output that follows. The first test returns [redacted] because a [redacted] can accept a [redacted] as the parameter to [redacted], but the second test returns [redacted] because a [redacted] cannot accept a [redacted] as its parameter to [redacted]:

This is a problem, because the contract of the [redacted] method requires it to be a symmetric operation. You'd encounter other problems if you added more behavior to [redacted], such as a [redacted] or [redacted] method.

Proper object-oriented design would not allow [redacted] to extend [redacted], because any code that asks for a [redacted] object should be able to work correctly with a [redacted] object as well. We call this principle *substitutability.* (It is also sometimes called the Liskov substitution principle, in honor of the Turing Award–winning author of a 1993 paper describing the idea.)

Substitutability

The ability of an object of a subclass to be used successfully anywhere an object of the superclass is expected.

Fundamentally, a [redacted] isn't the same thing as a [redacted], so an is-a relationship with inheritance is the wrong choice. In this case, you're better off writing [redacted] from scratch and avoiding these thorny issues.

Is-a Versus Has-a Relationships

There are ways to connect related objects without using inheritance. Consider the task of writing a [redacted] class, in which each [redacted] object is specified by a center point and a radius. It might be tempting to have [redacted] extend [redacted] and add the [redacted] field. However, this approach is a poor choice because a class is only supposed to capture one abstraction, and a circle simply isn't a point.

A point does make up a fundamental part of the state of each [redacted] object, though. To capture this relationship in the code, you can have each [redacted] object hold a [redacted] object in a field to represent its center. One object containing another as state is called a *has-a relationship.*

Has-a Relationship

[redacted]

A connection between two objects where one has a field that refers to the other. The contained object acts as part of the containing object's state.

Has-a relationships are preferred over is-a relationships in cases in which your class cannot or should not substitute for the other class. As a nonprogramming analogy, people occasionally need legal services in their lives, but most of them will choose to *have* a lawyer handle the situation rather than to *be* a lawyer themselves.

The following code presents a potential initial implementation of the [redacted] class:

```
public class

private

private double

public
```

```
double



this



this


public double


return


```

This design presents a [redacted] object as a single clear abstraction and prevents awkward commingling of [redacted] and [redacted] objects.

[redacted]

Use of inheritance is prevalent in the Java class libraries. One notable example is in the drawing of two-dimensional graphics. In this section we'll discuss a class that uses inheritance to draw complex two-dimensional shapes and to assign colors to them.

In the Chapter 3 supplement on graphics, we introduced an object called [redacted] which acts like a pen that you can use to draw shapes and lines onto a window. When Java's designers wanted additional graphical functionality, they extended the [redacted] class into a more powerful class called [redacted]. This is a good example of one of the more common uses of inheritance: to extend and reuse functionality from a powerful existing object.

Why didn't the designers of Java simply add the new methods into the existing [redacted] class? The [redacted] class already worked properly, so they decided it was best not to perform unnecessary surgery on it. John Vlissides, part of a famous foursome of software engineers affectionately called the "Gang of Four," once described the idea this way: "A hallmark—if not the hallmark—of good object-oriented design is that you can modify and extend a system by adding code rather than by hacking it. In short, change is additive, not invasive."

Making [redacted] extend [redacted] retains *backward compatibility.* Backward compatibility is the ability of new code to work correctly with old code without modifying the old code. Leaving [redacted] untouched ensured that old programs would keep working properly and gave new programs the option to use the new [redacted] functionality.

The documentation for [redacted] describes the purpose of the class as follows. “This [redacted] class extends the [redacted] class to provide more sophisticated control over geometry, coordinate transformations, color management, and text layout. This is the fundamental class for rendering 2-dimensional shapes, text, and images on the Java platform.” To be specific, [redacted] adds the ability to perform transformations such as scaling and rotation when you’re drawing. These capabilities can lead to some fun and interesting images on the screen.

If you used the [redacted] class from Chapter 3’s graphical supplement, you previously wrote statements like the following to get access to the panel’s [redacted] object:

[redacted]

Actually, the [redacted] method doesn’t return a [redacted] object at all, but rather a [redacted] object. Because of polymorphism, though, it is legal for your program to treat it as a [redacted] object, because every [redacted] object “is” a [redacted] object. To use it as a [redacted] object instead, simply write the following line of code:

[redacted]

Table 9.10 lists some of [redacted]'s extra methods.

Table 9.10 Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Rotates subsequently drawn items by the given angle in radians with respect to the origin
[redacted]	Adjusts the size of any subsequently drawn items by the given factors (1.0 means equal size)
[redacted]	Gives a slant to any subsequently drawn items
[redacted]	Shifts the origin by (dx, dy) in the current coordinate system

The following program demonstrates [redacted]. The [redacted] method's parameter is an angle of rotation measured in radians instead of degrees. Rather than memorizing the conversion between degrees and radians, we can use a static method from the [redacted] class called [redacted] that converts a degree value into the equivalent radian value:

[redacted]

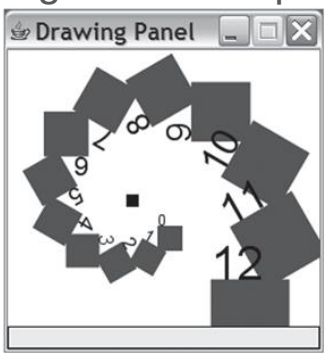
```
import

public class

public static void

new
```

```
for

int
```

[redacted]

Figure 9.3 shows the program's output.

Figure 9.3 Output of [redacted]



9.5 Interfaces

Inheritance is a very useful tool because it enables polymorphism and code sharing, but it does have several limitations. Because Java uses single inheritance, a class can extend only one superclass. This makes it impossible to use inheritance to set up multiple is-a relationships for classes that share multiple characteristics, such as an employee who is both part-time and a secretary. (Some languages, such as C++, do allow multiple inheritance, but this can be complicated and cause subtle problems, so the designers of Java left it out of the language.) There are also situations in which you want is-a relationships and polymorphism but you don't want to share code, in which case inheritance isn't the right tool for the job.

To this end, Java provides a feature called an *interface* that can represent a common supertype between several classes without code sharing.

Interface

A type that consists of a set of method declarations; when classes promise to implement an interface, you can treat those classes similarly in your code.

An interface is like a class, but it contains only method headers without bodies. A class can promise to *implement* an interface, meaning that the class promises to provide implementations of all the methods that are declared in the interface. Classes that implement an interface form an is-a relationship with that interface. In a system with an interface that is implemented by several classes, polymorphic code can handle objects from any of the classes that implement that interface.

A nonprogramming analogy for an interface is a professional certification. It's possible for a person to become certified as a teacher, nurse, accountant, or doctor. To do this, the person must demonstrate certain abilities required of members of those professions. When employers hire someone who has received the proper certification, they expect that the person will be able to perform certain job duties. An interface acts as a certification that classes can meet by implementing all the behavior described in the interface. Code that receives an object implementing an interface can rely on the object having certain behavior.

Interfaces are also used to define roles that objects can play; for example, a [redacted] class might implement the [redacted] interface to indicate that [redacted] objects can be compared to each other, or a [redacted] class might implement the [redacted] interface to indicate that a [redacted] object can be replicated.

Interfaces are used in many other places in Java's class libraries. Here are just a few of Java's important interfaces:

- The [redacted] interface in the [redacted] package is used to assign behavior to events when a user clicks on a button or other graphical control.

- The [redacted] interface in the [redacted] package denotes classes whose objects may be saved to files and transferred over a network.
- The [redacted] interface allows you to describe how to compare objects of your type to determine which are less than, greater than, or equal to each other. This technique can be used to search or sort a collection of objects.
- The [redacted] interface lets objects describe different ways that they can be printed by the [redacted] command.
- The [redacted] interface is used for multithreading, which allows a program to execute two pieces of code at the same time.
- Interfaces such as [redacted], [redacted], [redacted], and [redacted] in the [redacted] package describe data structures that you can use to store collections of objects.

We will cover some of these interfaces in later chapters.

Beginning with Java 8, interfaces are now allowed to contain implementations of “default” methods and static methods, but we will not explore those features in this chapter.

An Interface for Shapes

In this section, we’ll use an interface to define a polymorphic hierarchy of shape classes without sharing code between them. Imagine that we are creating classes to represent many different types of shapes, such as rectangles, circles, and triangles. We might be tempted to use inheritance with these shape classes

because they seem to share some common behavior (all shapes have an area and a perimeter, for example).

There is an is-a relationship here, because a rectangle, a circle, and a triangle are all shapes. But code sharing isn't useful in this case because each class implements its behavior differently. As depicted in Figure 9.4 and Table 9.11, each shape computes its area and perimeter in a totally different way. The w and h represent the rectangle's width and height; the r represents the circle's radius; and the a, b, and c represent the lengths of the triangle's three sides.

Figure 9.4 Three types of shapes

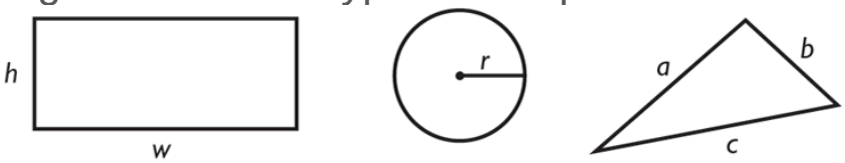


Figure 9.4 Full Alternative Text

Table 9.11 Formulas for Area and Perimeter of Each Shape Type

Area	w*h	πr2	s(s−a)(s−b)(s−c)
			where s=a+b+c2
Perimeter	2(w+h)	2πr	a+b+c

Since no code is shared among these classes, we should not create a common superclass to represent their is-a relationship. Java uses single inheritance, and we don't want to use up our

only potential inheritance relationship here. A better solution would be to write an interface called [redacted] to represent the common functionality of all shapes: the ability to ask for an area and a perimeter. Our various shape classes will implement this interface.

To write an interface, we create a new file with the same name as the interface's name; our [redacted] interface, for example, would be stored in [redacted]. We give the interface a header with the keyword [redacted] in place of the word [redacted]:

Inside the interface, we write headers for each method that we want a [redacted] to contain. But instead of writing method bodies with braces, we simply place a semicolon at the end of each header. We don't specify how the methods are implemented. Instead, we're requiring that any class that wants to be considered a shape must implement these methods. In fact, it isn't legal for an interface to contain method bodies; an interface can only contain method headers and class constants.

The following is the complete code for our [redacted] interface. It declares that shapes have methods to compute their areas and perimeters as type [redacted]:

```
public interface



public double



public double
```

The methods of an interface are sometimes called *abstract methods* because we only declare their names and signatures; we don’t specify how they will be implemented.

Abstract Method

A method that is declared (as in an interface) but not implemented. Abstract methods represent the behavior that a class promises to implement when it implements an interface.

Writing the [redacted] keyword on an interface's method headers is optional. We chose to include the [redacted] keyword so that the declarations in the interface would match the headers of the method implementations in the classes. The general syntax we'll use for declaring an interface is the following:

Although superficially, classes and interfaces look alike, an interface cannot be *instantiated;* that is, you cannot create objects of its type. In our case, any code trying to create a [redacted] would not compile. It is, however, legal to declare variables of type [redacted] that can refer to any object that implements the [redacted] interface, as we'll explore in a moment.

Implementing an Interface

Now that we've written a [redacted] interface, we want to connect the various classes of shapes to it. To connect a class to our interface with an is-a relationship, we must do two things:

1. Declare that the class "implements" the interface.
2. Implement each of the interface's methods in the class.

The general syntax for declaring that a class implements an interface is the following:

We must modify the headers of our various shape classes to indicate that they implement all of the methods in the [redacted] interface. The file [redacted], for example, should begin like this:

When we claim that our [redacted] class implements [redacted], we are promising that the [redacted] class will contain implementations of the [redacted] and [redacted] methods. If a class claims to implement [redacted] but does not have a suitable [redacted] or [redacted] method, it will not compile. For example, if we leave the body of our [redacted] class empty and try to compile it, the compiler will give errors like the following:

In order for the code to compile, we must implement the [redacted] and [redacted] methods in our [redacted] class. We'll define a [redacted] object by a width and a height. Since the area of a rectangle is equal to its width times its height, we'll implement the [redacted] method by multiplying its fields. We'll then use the perimeter formula `2 * (w + h)` to implement [redacted]. Here is the complete [redacted] class that implements the [redacted] interface:

```
public class

implements



private double



private double
```

```
public

double

double
```

```
this
```

```
this
```

```
public double
```

```
return
```

```
public double


return
```

The other classes of shapes are implemented in a similar fashion. We'll define a [redacted] object to have a field called [redacted]. (We'll abandon the center [redacted] object used in the [redacted] class earlier in the chapter, since we don't need it here.) We can determine the area by multiplying π by the radius squared. To find the perimeter, we'll use the equation `2 * π` [redacted]. Notice that there is no common code between [redacted] and [redacted], so inheritance is unnecessary. Here is the complete [redacted] class:

```
```

```
public class


private double


public
double


this
```

```
public double

return

public double

return
```

Finally, we'll specify a triangle's shape by its three side lengths, a, b, and c. The perimeter of the triangle is simply the sum of the three side lengths. The [redacted] method is a bit trickier, but there is a useful geometric formula called Heron's formula which says that the area of a triangle with sides of lengths a, b, and c is related to a value s that is equal to half the triangle's perimeter:

area=s(s−a)(s−b)(s−c) where s=a+b+c2

Here is a complete version of the [redacted] class:

```
public class

implements



private double



private double
```

```
private double

public
double
double
double

this


this
```

```
this
```

```
public double
```

```
double
```

```
return
```

```
public double
```

```
return
```

Using these shape classes and their common interface, a client program can now construct an array of shapes, write a method that accepts a general shape as a parameter, and otherwise take advantage of polymorphism.

Benefits of Interfaces

Classes that implement a common interface form a type hierarchy similar to those created by inheritance. The interface serves as a parent type for the classes that implement it. The following diagram represents our type hierarchy after our modifications. The way we represent an interface is similar to the way we represent a class, but we use the word “interface” for clarity. The methods are italicized to emphasize that they are abstract. Figure 9.5 shows our use of dashed lines to connect the classes and the interface they implement.

Figure 9.5 Hierarchy of shape classes

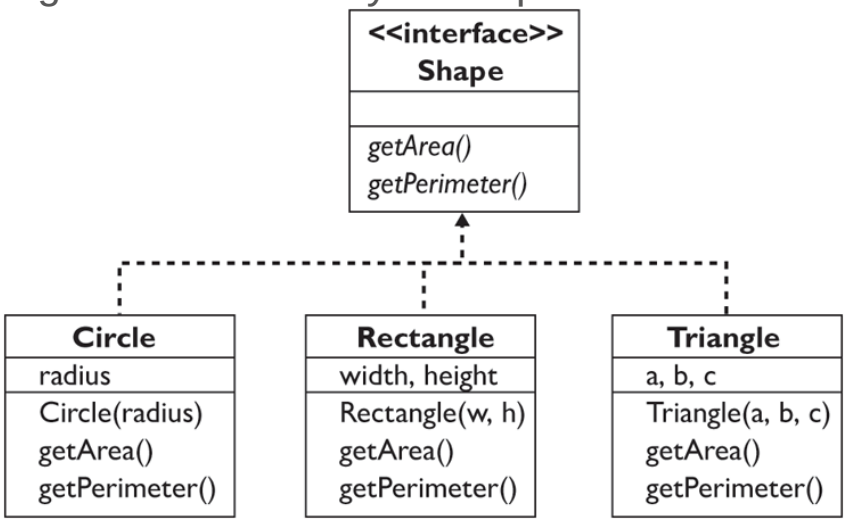


Figure 9.5 Full Alternative Text

The major benefit of interfaces is that we can use them to achieve polymorphism. We can create an array of [redacted], pass a [redacted] as a parameter to a method, return a [redacted] from a method, and so on. The following program uses the shape classes in an example that is similar to the polymorphism exercises in Section 9.3:

```
public class

public static void
```

```
new

new

new

new

for

int
```

This program produces the following output:

It would be fairly easy to modify our client program if another shape class, such as [redacted] or [redacted], were added to the hierarchy. This is another example of the desired property of “additive, not invasive” change that we mentioned earlier in this chapter.

It may seem odd that we can have interface variables, arrays, and parameters when it isn't possible to construct an object of an interface type, but all this capacity means is that any object of a type which implements that interface may be used. In our case, any type that implements [redacted] (such as [redacted], [redacted], or [redacted]) may be used.

Also recall that interfaces help us cope with the limitations of single inheritance. A class may extend only one superclass but may implement arbitrarily many interfaces. The following is the general syntax for headers of classes that extend a superclass and implement one or more interfaces:

Many classes in the Java class libraries both extend a superclass and implement one or more interfaces. For example, the [redacted] class (of which [redacted] is an instance) has the following header:

9.6 Case Study: Financial Class Hierarchy

As you write larger and more complex programs, you will end up with more classes and more opportunities to use inheritance and interfaces. It is important to practice devising sensible hierarchies of types, so that you will be able to solve large problems by breaking them down into good classes in the future.

When you are designing an object-oriented system, you should ask yourself the following questions:

- What classes of objects should I write?
- What behavior does the client want each of these objects to have?
- What data do the objects need to store in order to implement this behavior?
- Are the classes related? If so, what is the nature of the relationships?

Having good answers to these questions, along with a good knowledge of the necessary Java syntax, is a good start toward designing an object-oriented system. Such a process is called *object-oriented design*. We discussed some object-oriented design heuristics in the case study of Chapter 8.

Object-Oriented Design (OOD)

Modeling a program or system as a collection of cooperating objects, implemented as a set of classes using class hierarchies.

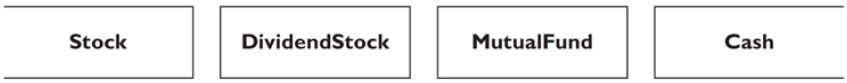
Let’s consider the problem of gathering information about a person’s financial investments. We’ve already explored a [redacted] example in this chapter and the previous chapter, as well as a [redacted] class to handle stocks that pay dividends. But stocks are not the only type of asset that investors might have in their financial portfolios. Other investments might include mutual funds, real estate, or cash.

How would you design a complete portfolio system? What new types of objects would you write? Take a moment to consider the problem. We’ll discuss an example design next.

Designing the Classes

Each type of asset deserves its own class. We already have the [redacted] class from the last chapter and its [redacted] subclass from earlier in this chapter, and we can add classes like [redacted] and [redacted]. Each object of each of these types will represent a single investment of that type. For example, a [redacted] object will represent a purchase of a mutual fund, and a [redacted] object will represent a sum of money in the user’s portfolio. The available types are shown in Figure 9.6. What data and behavior are necessary in each of these types of objects? Take a moment to consider it.

Figure 9.6 Financial classes



Though each type of asset is unique, the types do have some common behavior: Each asset should be able to compute its current market value and profit or loss, if any. These values are computed in different ways for different asset types, though. For instance, a stock's market value is the total number of shares that the shareholder owns, times the current price per share, while cash is always worth exactly its own amount.

In terms of data, we decided previously that a [redacted] object should store the stock's symbol, the number of shares purchased, the total cost paid for all shares, and the current price of the stock. Dividend stocks also need to store the amount of dividends paid. A [redacted] object should store the same data as a [redacted] object, but mutual funds can hold partial shares. Cash only needs to store its own amount. Figure 9.7 updates our diagram of types to reflect this data and behavior.

Figure 9.7 Financial classes with state and behavior

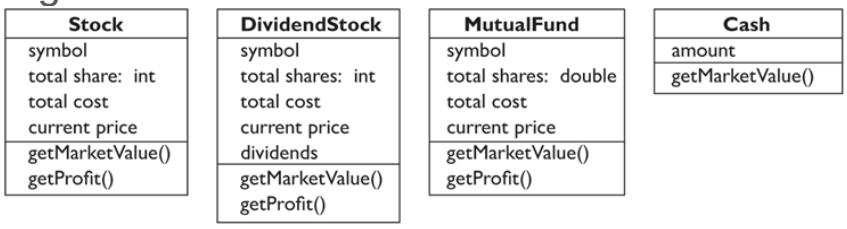


Figure 9.7 Full Alternative Text

The asset types are clearly related. Perhaps we'd want to gather and store a person's portfolio of assets in an array. It would be convenient to be able to treat any asset the same way, insofar as they share similar functionality. For example, every asset has a market value, so it would be nice to be able to compute the total

market value of all assets in an investor's portfolio, without worrying about the different types of assets.

Because different assets compute their market values in different ways, we should consider using an interface to represent the notion of an asset and have every class implement the asset interface. Our interface will demand that all assets have methods to calculate the market value and profit. The interface is a way of saying, "Classes that want to consider themselves assets must have [redacted] and [redacted] methods." Our interface for financial assets would be saved in a file called [redacted] and would look like this:

```
public interface



public double
```

```
public double
```

We'll have our various classes certify that they are assets by making them implement the [redacted] interface. For example, let's look at the [redacted] class. We didn't write a [redacted] method in our previous diagram of [redacted], because the value of cash doesn't change and therefore it doesn't have a profit. To indicate this quality, we can write a [redacted] method for [redacted] that returns `0.0`. The [redacted] class should look like this:

```
public class

implements



private double
```

```
public

double




this
```

```
public double
```

```
return

public double

return

public void

double
```

```
this
```

As we discussed earlier in this chapter, a [redacted] is very similar to a normal [redacted], but it has a small amount of behavior added. Let's display [redacted] as a subclass of [redacted] through inheritance, matching the design from earlier in the chapter. Figure 9.8 shows how our hierarchy should now look.

Figure 9.8 Financial class hierarchy

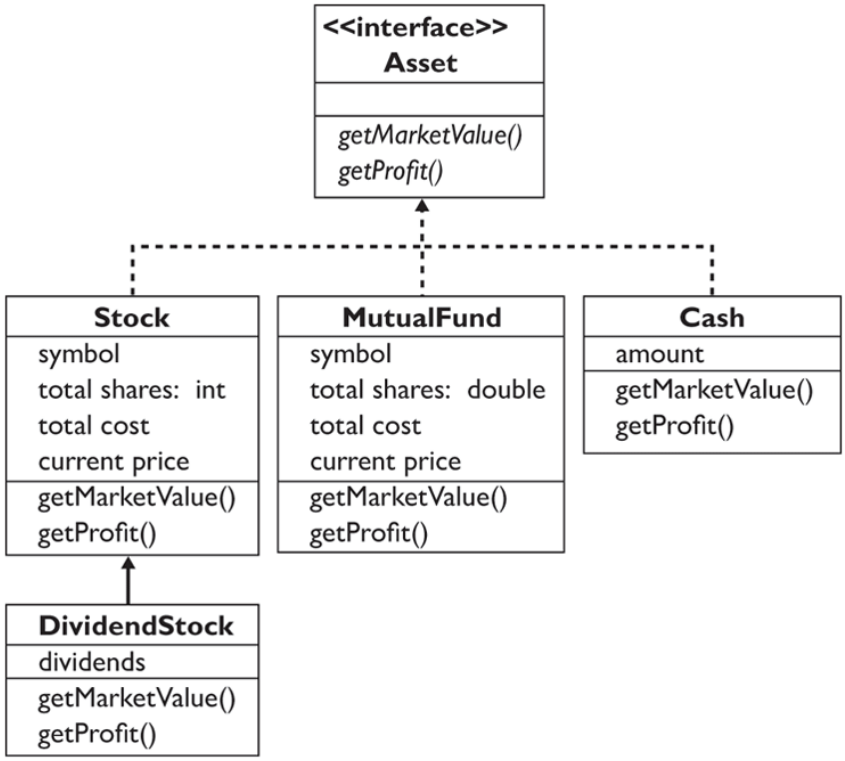


Figure 9.8 Full Alternative Text

What about the similarity between mutual funds and stocks? They both store assets that are based on shares, with a symbol, total cost, and current price. It wouldn't work very well to make one of them a subclass of the other, though, because the type of shares (integer or real number) isn't the same, and also because it's not a sensible is-a relationship: stocks aren't really mutual funds, and vice versa. It might seem excessive to have a separate class for mutual funds when the only difference is the existence of partial shares. But conceptually, these are separate types of investments, and many investors want to keep them separate. Also, there are some aspects of mutual funds, such as tax ramifications and Morningstar ratings, that are unique and that we might want to add to the program later.

Let's modify our design by making a new superclass called [redacted] which represents any asset that has shares and that contains the common behavior of [redacted] and [redacted]. Then

we can have both [redacted] and [redacted] extend [redacted], to reduce redundancy.

Our previous versions of the [redacted] and [redacted] classes each had a [redacted] method that required a parameter for the current price per share. In order to implement the [redacted] interface with its parameterless [redacted] and [redacted] methods, we'll change our design and create a field for the current price. We'll also add methods to get and set the value of the current price. The updated type hierarchy is shown in Figure 9.9.

Figure 9.9 Updated financial class hierarchy

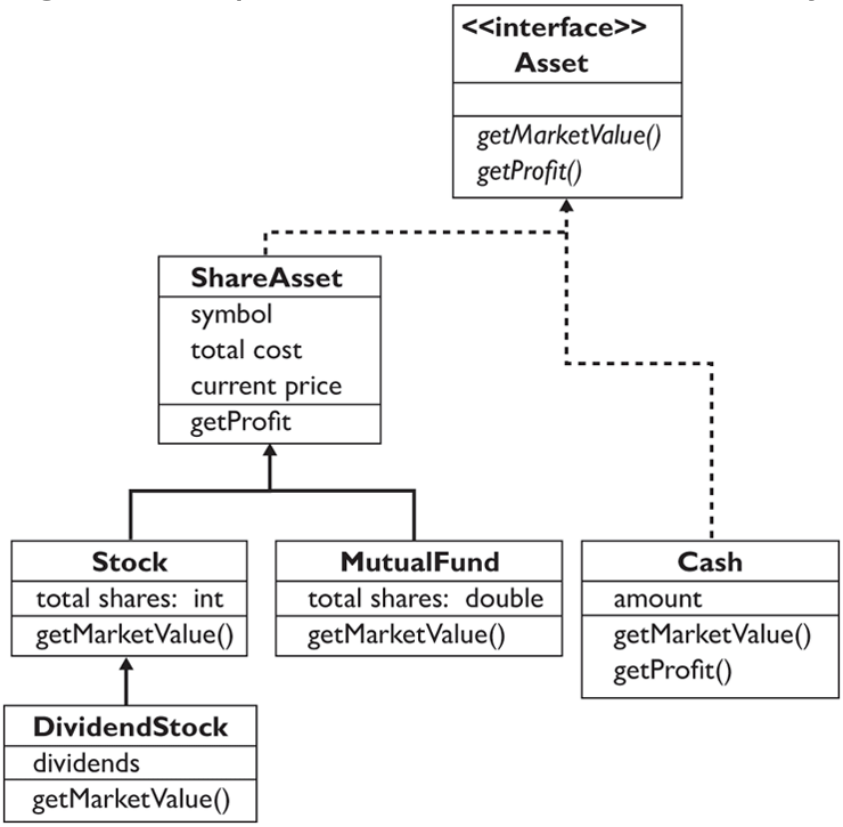


Figure 9.9 Full Alternative Text

This practice of redesigning code to meet new requirements is sometimes called *refactoring.*

Refactoring

Changing a program's internal structure without modifying its external behavior to improve simplicity, readability, maintainability, extensibility, performance, etc.

Redundant Implementation

Here's some potential code for the class:

```
public class

private
```

```
private double

private double

public
double

this
```

```
this
```

```
public void
```

```
double
```

```
public double
```

```
return

public double

return

public void

double
```

```
this
```

We stole some code from the [redacted] class and then made a few changes so that the code would fit this interface. Our [redacted] code accepted the current share price as a parameter to its [redacted] method. Since the [redacted] method cannot accept any parameters if we wish to implement the interface, we'll instead store the current share price as a field in the [redacted] class and supply a [redacted] mutator method that can be called to set its proper value. We also include a constructor that can initialize a [redacted] object with any number of shares and a total cost.

One last modification we made in creating [redacted] was to include an [redacted] method, which we'll use to add a given amount to the asset's total cost. We will need this because purchases of [redacted] and [redacted] need to update the [redacted] field, but they cannot do so directly because it is private.

The [redacted] class can now extend [redacted] to implement its remaining functionality. Notice that we both extend [redacted] and implement the [redacted] interface in the class's header:

```
public class

extends

implements



private int
```

```
public

double




super

```

```
public double

```

```
return

```

```
public int

return

public double

return
```

```
public void

int

double
```

The class receives similar treatment, but with a for its total shares (the two classes are highly redundant; we'll improve them in the next section):

```
```

```
public class

extends

implements



private double

public
```

```
double


super


public double


return
```

```
public double

return

public double

return
```

```
public void

double

double
```

The [redacted] simply adds an amount of dividend payments to a normal [redacted], which affects its market value. We don’t need to override the [redacted] method in [redacted], because [redacted] already inherits a [redacted] method with the following body:

```
```

Notice that [redacted]'s body calls [redacted]. The [redacted] class overrides the [redacted] method, with the convenient side effect that any other method that calls [redacted] (such as [redacted]) will also behave differently. This occurs because of polymorphism; since [redacted] is overridden, [redacted] calls the new version of the method. The profit will be correctly computed with dividends because the dividends are added to the market value.

The following code implements the [redacted] class:

```
public class

private double
```

```
public

double




super
```

```
public double




return
```

```
public void

double
```

Abstract Classes

So far we have written classes, which are concrete implementations of state and behavior, and interfaces, which are completely abstract declarations of behavior. There is an entity that exists between these two extremes, allowing us to define some concrete state and behavior while leaving some abstract, without defined method bodies. Such an entity is called an *abstract class.*

Abstract Class

A Java class that cannot be instantiated, but that instead serves as a superclass to hold common code and declare abstract behavior.

You probably noticed a lot of redundancy between the and code in the last section. For example, although the and methods have identical code, they can't be moved up into the superclass because they depend on the number of shares, which is different in each child class. Ideally, we should get rid of this redundancy somehow.

There is also a problem with our current class. A isn't really a type of asset that a person can buy; it's just a concept that happens to be represented in our code. It would be undesirable for a person to actually try to construct a object—we wrote the class to eliminate redundancy, not for clients to instantiate it.

We can resolve these issues by designating the [redacted] class as abstract. Writing abstract in a class's header will modify the class in two ways. First, the class becomes noninstantiable, so client code will not be allowed to construct an object of that type with the new keyword. Second, the class is enabled to declare abstract methods without bodies. Unlike an interface, though, an abstract class can also declare fields and implement methods with bodies, so the [redacted] class can retain its existing code.

The general syntax for declaring an abstract class is

[redacted]

Thus, our new [redacted] class header will be

[redacted]

An attempt to create a [redacted] object will now produce a compiler error such as the following:

Really, the [redacted] class introduced earlier in this chapter should also have been an abstract class. We did not especially want client code to construct [redacted] objects. No one is *just* an employee; the [redacted] class merely represented a general category that we wanted the other classes to extend.

Abstract classes are allowed to implement interfaces. Rather than requiring all subclasses of [redacted] to implement the [redacted] interface, we can specify that [redacted] implements [redacted]:

This indication will save [redacted] subclasses from having to write [redacted] in their class headers.

[redacted] does not implement the [redacted] method required by [redacted]; that functionality is left for its subclasses. We can instead declare [redacted] as an abstract method in the [redacted] class. Abstract methods declared in abstract classes need to have the keyword [redacted] in their headers in order to compile properly. Otherwise, the syntax is the same as when we declare an abstract method in an interface, with a semicolon replacing the method's body:

The general syntax for an abstract method declaration in an abstract class is the following:

Another benefit of this design is that code in the abstract class can actually call any of its abstract methods, even if they don't have implementations in that class. This is allowed because the abstract class can count on its subclasses to implement the abstract methods. Now that [redacted] implements [redacted], we can move the common redundant [redacted] code up to [redacted] and out of [redacted] and [redacted]:

objects can call from their methods even though that method isn't present in . The code compiles because the compiler knows that whatever class extends will have to implement .

The following is the final version of the abstract class:

```
implements




private




private double




private double


















public

double
```

this

this

public void

double

```
public double

return

public abstract double

public double
```

```
return

public double

return

public void
```

```
double



this
```

An abstract class is a useful hybrid that can contain both abstract and nonabstract methods. All methods in the interfaces we have written in this chapter have been implicitly abstract; we could have declared them with the [redacted] keyword. Declaring them without the [redacted] keyword as we have done in this chapter is a commonly used shorthand for the longer explicit form. Unfortunately, abstract classes disallow this shorthand to avoid ambiguity.

Nonabstract classes like [redacted] and [redacted] are sometimes called *concrete classes* to differentiate them from abstract classes. We can modify the [redacted] and [redacted] classes to take advantage of [redacted] and reduce redundancy. The following are the final versions of the [redacted] and [redacted] classes. ([redacted] is unmodified.) Notice that the subclasses of [redacted] must implement [redacted], or we'll receive a compiler error:

```
public class

extends



private int

















public

double

```

```
super
```

```
public double
```

```
return
```

```
public int


return
```

```
public void

int

double
```

```
public class

extends



private double
```

```
public

double




super
```

```
public double


return
```

```
public double


return


public void

double

double
```

Abstract classes can do everything interfaces can do and more, but this does not mean that it is always better to use them than interfaces. One important difference between interfaces and abstract classes is that a class may choose to implement arbitrarily many interfaces, but it can extend just one abstract class. That is why an interface often forms the top of an inheritance hierarchy, as our [redacted] interface did in this design. Such placement allows classes to become part of the hierarchy without having it consume their only inheritance relationships.

Chapter Summary

Inheritance is a feature of Java programs that allows the creation of a parent–child relationship between two types.

The child class of an inheritance relationship (commonly called a subclass) will receive a copy of (“inherit”) every field and method from the parent class (superclass). The subclass “extends” the superclass, because it can add new fields and methods to the ones it inherits from the superclass.

A subclass can override a method from the superclass by writing its own version, which will replace the one that was inherited.

Treating objects of different types interchangeably is called polymorphism.

Subclasses can refer to the superclass’s constructors or methods using the [redacted] keyword.

The [redacted] class represents the common superclass of all objects and contains behavior that every object should have, such as the [redacted] and [redacted] methods.

Inheritance provides an “is-a” relationship between two classes. If the two classes are not closely related, inheritance may be a poor design choice and a “has-a” relationship between them (in which one object contains the other as a field) may be better.

An interface is a list of method declarations. An interface specifies method names, parameters, and return types but does not include the bodies of the methods. A class can implement (i.e., promise to implement all of the methods of) an interface.

Interfaces help us achieve polymorphism so that we can treat several different classes in the same way. If two or more classes both implement the same interface, we can use either of them interchangeably and can call any of the interface’s methods on them.

An abstract class cannot be instantiated. No objects of the abstract type can be constructed. An abstract class is useful because it can be used as a superclass and can also define abstract behavior for its subclasses to implement.

An abstract class can contain abstract methods, which are declared but do not have bodies. All subclasses of an abstract class must implement the abstract superclass’s abstract methods.

Self-Check Problems

Section 9.1: Inheritance Basics

1. What is code reuse? How does inheritance help achieve code reuse?
2. What is the difference between overloading and overriding a method?
3. Which of the following is the correct syntax to indicate that class A is a subclass of B?
   a.
   b.
   c.
   d.
   e.

Section 9.2: Interacting with the Superclass

4. Explain the difference between the [redacted] keyword and the [redacted] keyword. When should each be used?
5. For the next three problems, consider the following class:

```
public class

private

private int

public
```

```
int


this


this


public void


this
```

Also consider the following partial implementation of a subclass of [redacted] to represent undergraduate students at a university:

Can the code in the [redacted] class access the [redacted] and [redacted] fields it inherits from [redacted]? Can it call the [redacted] method?

6. Write a constructor for the [redacted] class that accepts a name as a parameter and initializes the [redacted]'s state with that name, an [redacted] value of [redacted], and a [redacted] value of [redacted].
7. Write a version of the [redacted] method in the [redacted] class that not only sets the [redacted] but also increments the [redacted] field's value by one.
8. Consider the following two automobile classes:

Given the following declared variables, what is the output from the following statements?

9. Suppose the [redacted] code from the previous problem changes to the following:

Using the same variables from the previous problem, what is the output from the following statements?

Section 9.3: Polymorphism

10. Consider the following classes:

Which of the following are legal statements?

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]
f. [redacted]

11. Using the [redacted], [redacted], [redacted], and [redacted] classes from this section, what is the output of the following code fragment?

12. Assume that the following classes have been defined:

```
public class

extends



public void
```

```
public


return


public class

extends


public void
```

```
public class

extends




public void
```

```
public class
```

```
public void

public void

public String
```

```
return
```

What is the output produced by the following code fragment?

13. Using the classes from the previous problem, write the output that is produced by the following code fragment.

14. Assume that the following classes have been defined:

```
public class

extends
```

```
public void
```

```
public class
```

```
public void
```

```
public void

public

return

public class
```

```
extends



public void









public



return









public class
```

```
public void




public


return
```

What output is produced by the following code fragment?

```
```

15. Using the classes from the previous problem, write the output that is produced by the following code fragment:

16. Assume that the following classes have been defined:

```
public class

extends



public void









super









public void
```

```
public class


public void


public void
```

```
public void

public class

public void
```

```
public class

extends



public void
```

What output is produced by the following code fragment?

```
```

17. Suppose that the following variables referring to the classes from the previous problem are declared:

Which of the following statements produce compiler errors? For the statements that do not produce errors, what is the output of each statement?

Section 9.4: Inheritance and Design

18. What is the difference between an is-a and a has-a relationship? How do you create a has-a relationship in your code?
19. Imagine a [redacted] class with objects that represent two-dimensional rectangles. The [redacted] has [redacted] and [redacted] fields with appropriate accessors and mutators, as well as [redacted] and [redacted] methods.

You would like to add a [redacted] class into your system. Is it a good design to make [redacted] a subclass of [redacted]? Why or why not?

20. Imagine that you are going to write a program to play card games. Consider a design with a [redacted] class and 52 subclasses, one for each of the unique playing cards (for example, [redacted] and [redacted]). Is this a good design? If so, why? If not, why not, and what might be a better design?
21. In Section 9.2 we discussed adding functionality for dividend payments to the [redacted] class. Why was it preferable to create a [redacted] class rather than editing the [redacted] class and adding this feature directly to it?

Section 9.5: Interfaces

22. What is the difference between implementing an interface and extending a class?
23. Consider the following interface and class:

What must be true about the code for class [redacted] in order for that code to compile successfully?

24. What's wrong with the code for the following interface? What should be changed to make a valid interface for objects that have colors?

25. Modify the [redacted] class from Chapter 8 so that it implements the [redacted] interface and [redacted] have colors. (You may wish to create a [redacted] class that extends [redacted].)
26. Declare a method called [redacted] in the [redacted] interface that returns the number of sides that the shape has. Implement the method in all shape classes. A circle is defined to have 0 sides.

Section 9.6: Case Study: Financial Class Hierarchy

27. What is an abstract class? How is an abstract class like a normal class, and how does it differ? How is it like an interface?
28. Consider the following abstract class and its subclass. What state and behavior do you know for sure will be present in the subclass? How do you know?

29. Consider writing a program to be used to manage a collection of movies. There are three kinds of movies in the collection: dramas, comedies, and documentaries. The collector would like to keep track of each movie's title, the name of its director, and the year the movie was made. Some operations are to be implemented for all movies, and there will also be special operations for each of the three different kinds of movies. How would you design the class(es) to represent this system of movies?

Exercises

1. Write the class [redacted] to accompany the other law firm classes described in this chapter. Marketers make $50,000 ($10,000 more than general employees) and have an additional method called advertise that prints [redacted] Make sure to interact with the superclass as appropriate.
2. Write a class [redacted] to accompany the other law firm classes described in this chapter. Janitors work twice as many hours per week as other employees (80 hours/week), they make $30,000 ($10,000 *less* than general employees), they get half as much vacation as other employees (only 5 days), and they have an additional method [redacted] that prints [redacted] [redacted] Make sure to interact with the superclass as appropriate.
3. Write a class [redacted] to accompany the other law firm classes described in this chapter. Harvard lawyers are like normal lawyers, but they make 20% more money than a normal lawyer, they get 3 days more vacation, and they have to fill out four of the lawyer's forms to go on vacation. That is, the [redacted] method should return [redacted]. Make sure to interact with the superclass as appropriate.
4. Write a class [redacted] that relates to the [redacted] and [redacted] classes from Self-Check Problems 9 and 10 and whose methods have the following behavior. Whenever possible, use inheritance to reuse behavior from the superclasses.

5. For the next four problems, consider the task of representing types of tickets to campus events. Each ticket has a unique number and a price. There are three types of tickets: walk-up tickets, advance tickets, and student advance tickets. Figure 9.10 illustrates the types:

Figure 9.10 Classes of tickets that are available to campus events

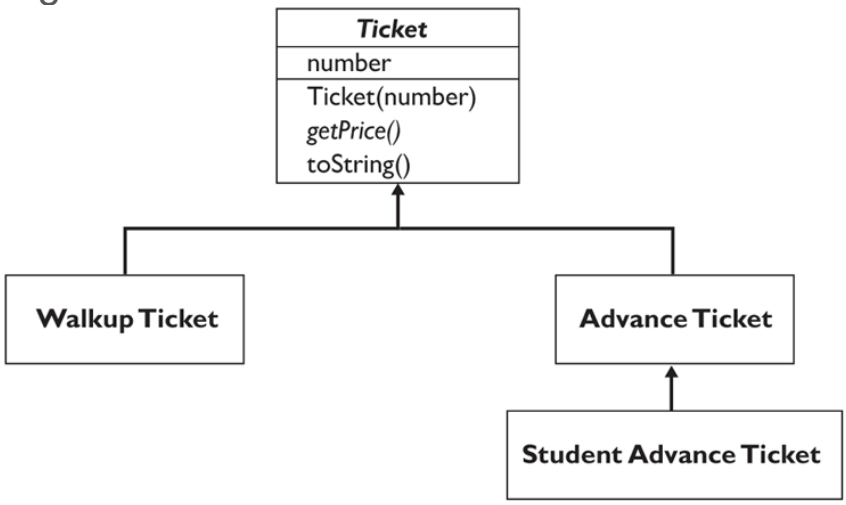


Figure 9.10 Full Alternative Text

- Walk-up tickets are purchased the day of the event and cost $50.

- Advance tickets purchased 10 or more days before the event cost $30, and advance tickets purchased fewer than 10 days before the event cost $40.
- Student advance tickets are sold at half the price of normal advance tickets: When they are purchased 10 or more days early they cost $15, and when they are purchased fewer than 10 days early they cost $20.

Implement a class called [redacted] that will serve as the superclass for all three types of tickets. Define all common operations in this class, and specify all differing operations in such a way that every subclass must implement them. No actual objects of type [redacted] will be created: Each actual ticket will be an object of a subclass type. Define the following operations:

- The ability to construct a ticket by number.
- The ability to ask for a ticket's price.
- The ability to [redacted] a ticket object as a [redacted]. An example [redacted] would be [redacted].

6. Implement a class called [redacted] to represent a walk-up event ticket. Walk-up tickets are also constructed by number, and they have a price of $50.
7. Implement a class called [redacted] to represent tickets purchased in advance. An advance ticket is constructed with a ticket number and with the number of days in advance that the ticket was purchased. Advance tickets purchased 10 or more days before the event cost $30, and advance tickets purchased fewer than 10 days before the event cost $40.
8. Implement a class called [redacted] to represent tickets purchased in advance by students. A student advance ticket is constructed with a ticket number and with the number of days in advance that the ticket

was purchased. Student advance tickets purchased 10 or more days before the event cost $15, and student advance tickets purchased fewer than 10 days before the event cost $20 (half of a normal advance ticket). When a student advance ticket is printed, the [redacted] should mention that the student must show his or her student ID (for example, [redacted]).

9. [redacted]. A company has written a large class with many methods including:

[redacted]	Constructs a [redacted] object using information in [redacted]
[redacted]	Records the given debit
[redacted]	Records the given credit
[redacted]	Returns the current balance in pennies

Design a new class [redacted] whose instances can be used in place of a bank account but include new behavior of remembering the minimum and maximum balances ever recorded for the account. The class should have a constructor that accepts a [redacted] parameter. The bank account's constructor sets the initial balance on the basis of the startup information. Assume that only debits and credits change an account's balance. Include these new methods in your class:

[redacted]	Returns the minimum balance in pennies
[redacted]	Returns the maximum balance in pennies

10. [redacted]. Suppose a class [redacted] keeps track of a list of items being purchased at a market:

[redacted]	Constructs a grocery bill object for the given clerk
[redacted]	Adds the given item to this bill
[redacted]	Returns the total cost of these items
[redacted]	Prints a list of items

Grocery bills interact with [redacted] objects, each of which has the public methods that follow. A candy bar item might cost 1.35 with a discount of 0.25 for preferred customers, meaning that preferred customers get it for 1.10. (Some items will have no discount, 0.0.) Currently the preceding classes do not consider discounts. Every item in a bill is charged full price, and item discounts are ignored.

[redacted]	Returns the price for this item
[redacted]	Returns the discount for this item

Define a class [redacted] that extends [redacted] to compute discounts for preferred customers. Its constructor accepts a parameter for whether the customer should get the discount. Your class should also adjust the total reported for preferred customers. For example, if the total would have been $80 but a preferred customer is getting $20 in discounts, then [redacted] should report the total as $60 for that customer. Also keep track of the number of items on which a customer is getting a nonzero discount and the sum of these discounts, both as a total amount and as a percentage of the original bill. Include the extra methods that follow, which allow a client to ask about the discount. Return 0.0 if the customer is not a preferred customer or if no items were discounted.

[redacted]	Constructs bill for given clerk
[redacted]	Returns the number of items that were discounted, if any
[redacted]	Returns the total discount for this list of items, if any
[redacted]	Returns the percent of the total discount as a percent of what the total would have been otherwise

11. [redacted]. A cash processing company has a class called [redacted] used to process transactions:

[redacted]	Constructs an account using client information
[redacted]	Processes the next transaction, returning true if the transaction was approved and false otherwise

Account objects interact with [redacted] objects, which have many methods including

[redacted]	Returns the value of this transaction in pennies (could be negative, positive or zero)

Design a new class called [redacted] whose instances can be used in place of normal accounts but which include the extra behavior of not processing transactions with a value of 0. More specifically, the new class should indicate that a zero-valued transaction was approved but shouldn't call the [redacted] method for it. Your class should have a single constructor that accepts a

parameter of type [redacted], and it should include the following method:

[redacted]	Returns the percent of transactions filtered out (between 0.0 and 100.0); returns 0.0 if no transactions are submitted

12. Add an [redacted] method to the [redacted] class introduced in Chapter 8. Two time spans are considered equal if they represent the same number of hours and minutes.
13. Add an [redacted] method to the [redacted] class introduced in this chapter. Two cash objects are considered equal if they represent the same amount of money.
14. Add an [redacted] method to each of the [redacted], [redacted], and [redacted] classes introduced in this chapter. Two shapes are considered equal if their fields have equivalent values.
15. Write a class named [redacted] whose objects represent regular octagons (eight-sided polygons). Your class should implement the [redacted] interface defined in this chapter, including methods for its area and perimeter. An [redacted] object is defined by its side length. (You may need to search online to find formulas for the area and perimeter of a regular octagon.)
16. Write a class named [redacted] whose objects represent regular hexagons (6-sided polygons). Your class should implement the [redacted] interface defined in this chapter.
17. Write a class named [redacted] whose objects represent regular dodecagons (12-sided polygons). Your class should implement the [redacted] interface defined in this chapter. A [redacted] object is defined by its side length as passed to its constructor. The common formulas for the area and perimeter of a regular dodecagon are based on its circumradius, which is the radius of the smallest

circle that would fully contain the dodecagon. We suggest reading about these formulas on a web site such as Wikipedia or Wolfram MathWorld.

18. Declare an interface called [redacted] which represents items that store an integer that can be incremented in some way. The interface has a method called [redacted] that increments the value and a method called [redacted] that returns the value. Once you have written the interface, write two classes called [redacted] and [redacted] that implement the interface. The [redacted] begins its value at [redacted] and increases it by 1 each time it is incremented. The [redacted] begins its value at a random integer and changes it to a new random integer each time it is incremented.

Programming Projects

1. Write an inheritance hierarchy of three-dimensional shapes. Make a top-level shape interface that has methods for getting information such as the volume and surface area of a three-dimensional shape. Then make classes and subclasses that implement various shapes such as cubes, rectangular prisms, spheres, triangular prisms, cones, and cylinders. Place common behavior in superclasses whenever possible, and use abstract classes as appropriate. Add methods to the subclasses to represent the unique behavior of each three-dimensional shape, such as a method to get a sphere's radius.
2. Write a set of classes that define the behavior of certain animals. They can be used in a simulation of a world with many animals moving around in it. Different kinds of animals will move in different ways (you are defining those differences). As the simulation runs, animals can "die" when two or more of them end up in the same location, in which case the simulator randomly selects one animal to survive the collision. See your course web site or www.buildingjavaprograms.com for supporting files to run such a simulation.

The following is an example set of animals and their respective behavior:

Bird	B	Moves randomly 1 step in one of the four directions each time

Frog	F	Moves randomly 3 steps in one of the four directions
Mouse	M	Moves west 1 step, north 1 step (zig-zag to the northwest)
Rabbit	V	Move north 2 steps, east 2 steps, south 2 steps (“hops” to the right)
Snake	S	Moves south 1 step, east 1 step, south 1 step, west 2 steps, south 1 step, east 3 steps, south 1 step, west 4 steps, ... (“slithers” left and right in increasing length)
Turtle	T	Moves south 5 steps, west 5 steps, north 5 steps, east 5 steps (clockwise box)
Wolf	W	Has custom behavior that you define

Your classes should be stored in files called Bird.java, Frog.java, Mouse.java, Rabbit.java, Snake.java, Turtle.java, and Wolf.java.

3. Write an inheritance hierarchy that stores data about sports players. Create a common superclass and/or interface to store information common to any player regardless of sport, such as name, number, and salary. Then create subclasses for players of your favorite sports, such as basketball, soccer, or tennis. Place sport-specific information and behavior (such as kicking or vertical jump height) into subclasses whenever possible.
4. Write an inheritance hierarchy to model items at a library. Include books, magazines, journal articles, videos, and electronic media such as CDs. Include in a superclass and/or interface common information that the library must have for every item, such as a unique identification number and title. Place behavior and information that is specific to items, such as a video’s runtime length or a CD’s musical genre, into the subclasses.

Chapter 10 ArrayLists

10.1
• Basic Operations
• Searching Methods
• A Complete Program
• Adding to and Removing from an
• Initializing an
• Using the For-Each Loop with
• Wrapper Classes
10.2 The Interface
• Natural Ordering and
• Implementing the Interface
10.3 Case Study: Ranked Choice Voting
• Ballot Class
• Counting Votes
• Multiple Rounds

Introduction

One of the most fundamental data structures you will encounter in programming is a list. You'll want to store lists of words, lists of numbers, lists of names, and so on. Chapter 7 demonstrated using arrays to store sequences of values, but arrays are fixed-size structures that require you to declare in advance exactly how many elements you want to store. In this

chapter we'll explore a new structure, known as an [redacted], that provides more flexibility than an array. An [redacted] is a dynamic structure with a variable length, so it can grow and shrink as the program executes.

The [redacted] structure is the first example we have discussed of a generic structure that can be used to store values of different types. As a result, we will need to explore some issues related to generic structures in this chapter. We will also look at how to use primitive data with such structures, using what are known as the wrapper classes. Finally, we will demonstrate how to use the [redacted] interface to put values of a particular type into sorted order and how to write classes that implement the [redacted] interface.

10.1 [redacted]

In our daily lives, we often manipulate lists of one kind or another. For example, on social networking sites like Facebook.com, people list the bands they like. Suppose someone listed the following bands:

[redacted]

You saw in Chapter 7 that you can declare an array to store a sequence of values. For example, to store the preceding list, you could declare an array of [redacted]s of length 5. But what happens if you want to change the list later (say, to remove Tool and U2 from the list)? You would have to shift values over, and you'd be left with empty array slots at the end. And what if you wanted to add to the list, so that it ended up with more than five names? You wouldn't have room to store more than five values in the original array, so you would have to construct a new array with a larger size to store the list.

Most of us think of lists as being more flexible than that. We don't want to have to worry about the kind of low-level details that come up in the course of manipulating an array. We want to be able to just say, "Add something to the list" or "Remove this value from the list," and we want the lists to grow and shrink over time as we add or remove values. Computer scientists would say that we have in mind a *list abstraction* that enables us to specify certain operations to be performed (add, remove) without having to worry

about the details of how those operations are performed (shifting, constructing new arrays).

Java provides this functionality in a class called [redacted]. Internally, each [redacted] object uses an array to store its values. As a result, an [redacted] provides the same fast random access as an array. But unlike with an array, with an [redacted] you can make simple requests to add or remove values, and the [redacted] takes care of all of the details for you: If you add values to the list it makes the array bigger, and if you remove values it handles any shifting that needs to be done.

Remember that you can declare arrays of different types. If you want an array of [redacted] values, you declare a variable of type [redacted]. For an array of [redacted] values, you use the type [redacted]. This is a special syntax that works just for arrays, but the [redacted] class has almost the same flexibility. If you read the API documentation for [redacted], you'll see that it is actually listed as [redacted]. This is an example of a *generic class* in Java.

Generic Class (Generic)

[redacted]

A class such as [redacted] that takes a type parameter to indicate what kind of values it will use.

The "E" in [redacted] is short for "Element," and it indicates the type of elements that will be included in the [redacted]. Generic classes are similar to parameterized methods. Remember from Chapter 3 that you can use a parameter to define a family of related tasks that differ just by a particular characteristic like height or width. In this case, the parameter is a

type and it is used to declare another type. The type [redacted] represents a family of types that differ just by the type of element they store. You would use [redacted] to store a list of [redacted], [redacted] to store a list of [redacted], [redacted] to store a list of [redacted], and so on. Notice that you would never actually declare something to be of type [redacted]. As with any parameter, you have to replace the [redacted] with a specific value to make it clear which of the many possible [redacted] types you are using.

Basic [redacted] Operations

The [redacted] class is part of the [redacted] package, so to include it in a program, you must include an [redacted] declaration:

[redacted]

The syntax for constructing an [redacted] is more complicated than what we've seen before because of the required type parameter. For example, you could construct an [redacted] of [redacted]s as follows:

[redacted]

This code constructs an empty list of strings. The syntax is complicated, but it will be easier to remember if you keep in mind that the [redacted] notation is actually part of the type: This isn't simply an [redacted], it is an [redacted], often read as "an [redacted] of [redacted]." Notice how the type appears on both sides of the [redacted] sign, when you declare the variable and when you call the constructor.

If you think in terms of the type being [redacted], you'll see that this line of code isn't all that different from the code used to construct an object like a [redacted]. In both cases, you write the variable's type on the left side of the [redacted] sign before its name and after the [redacted] sign and the keyword [redacted] (which calls that type's constructor to create an object of that type):

It can be cumbersome to list the element type [redacted] twice, so Java version 7 introduced a new shorter syntax for declaring collections called the "diamond operator" whereby the element type may be omitted on the right side of the statement and replaced by [redacted], such as:

```
<>
```

The diamond operator syntax is shorter and more convenient, and its behavior is identical to that of the longer code. So we will use the short form for the rest of our examples in this text.

In the rest of this section we'll manipulate a list of a person's favorite music groups represented as strings. Once you've constructed an `[redacted]`, you can add values to it by calling its `[redacted]` method:

```
[redacted]
```

When you ask an [redacted] to add a new value to the list, it appends the new value to the end of the list. Unlike with a simple array, an [redacted] can be thought of as having a dynamic size that grows and shrinks to fit the elements you put inside it. So you can think of the add method as performing two major tasks: expanding the list's size by one to accommodate the new element, and placing the new element at the end of the list. Figure 10.1 shows the state of our list of bands after each of the preceding three elements is added:

Figure 10.1 Adding elements to a list

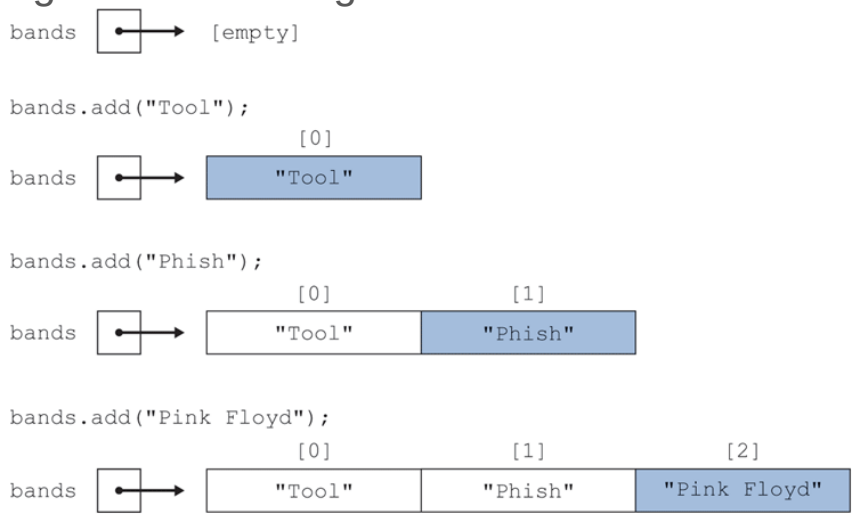


Figure 10.1 Full Alternative Text

Java will make sure that you add values of an appropriate type. In this case, because you requested an [redacted], you can add [redacted] elements to the list. If you try to add a value of a different type, such as an integer or [redacted], the code will generate a compiler error.

Unlike with simple arrays, printing an [redacted] is straightforward because the [redacted] class overrides Java's [redacted] method. The [redacted] version of [redacted] constructs a string that includes the contents of the list inside square brackets with the

values separated by commas. Remember that the [redacted] method is called when you print an object or concatenate an object to a string. As a result, you can print an [redacted] with a simple [redacted] statement:

For example, you can add [redacted] statements as you add each element to the list. Or if you are testing [redacted] features in JShell, you can just type the list's name to print its contents, as in the following interaction:

```
jshell>


jshell>


jshell>


jshell>
```

Notice that you can print the list even when it is empty and each time that new values are added to the end of the list. The [redacted] class also provides an overloaded version of the [redacted] method for adding a value at a particular index in the list. It preserves the order of the other list elements, shifting values to the right to make room for the new value. This version of [redacted] takes two parameters: an index and a value to insert. [redacted]s use zero-based indexing, just as arrays and strings do. For

example, given the preceding list, consider the effect of inserting a value at index 1:

```
jshell>




jshell>


jshell>
```

The call on instructs the list to insert the new element at index 1. Therefore, the old value at index 1 and everything that follows it gets shifted to the right. You can think of this method as performing three operations: expanding the list's size by 1, shifting elements right to make room for a new element, and inserting the new element. Figure 10.2 shows the state of our list before and after inserting U2 at index 1.

Figure 10.2 Inserting an element at a given index

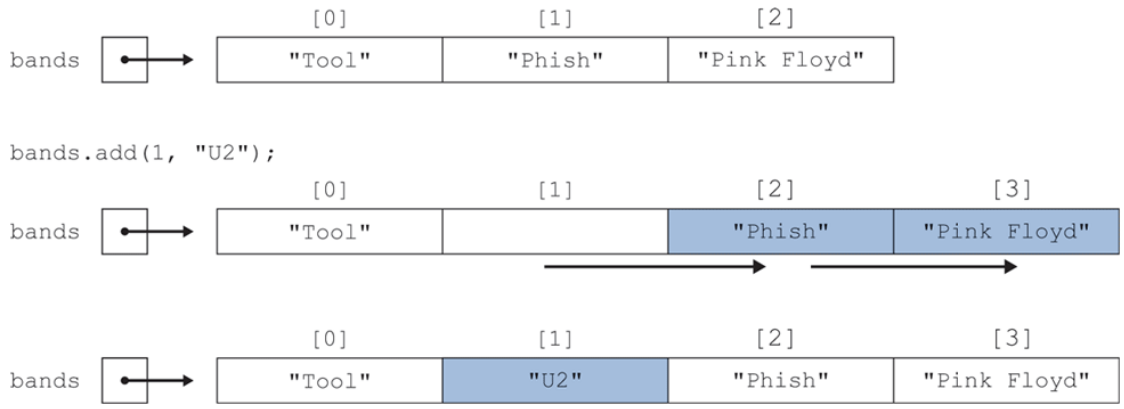


Figure 10.2 Full Alternative Text

The [redacted] also has a [redacted] method that accepts an integer index as a parameter and removes the value at that index. The method preserves the order of the list by shifting values to the left to fill in any gap. For example, consider what happens to the previous list if we remove the value at position 0 and then remove the value at position 1:

```
jshell>


jshell>


jshell>
```

```
jshell>
```

This result is a little surprising. We asked the list to remove the value at position 0 and then to remove the value at position 1. You might imagine that this would get rid of the strings [redacted] and [redacted], since they were at positions 0 and 1, respectively, before this code was executed. However, an [redacted] is a dynamic structure whose values move around and shift into new positions in response to your commands. The order of events is demonstrated more clearly if we include a third [redacted] statement in the code:

```
jshell>



jshell>



jshell>
```

```
jshell>



jshell>
```

The first call on remove removes [redacted] because it's the value currently in position 0. But once that value has been removed, everything else shifts over: The element [redacted] moves to the front (to position 0), the element [redacted] shifts into position 1, and the element [redacted] moves into position 2. So, when the second call on [redacted] is performed, Java removes [redacted] from the list because it is the value that is in position 1 at that point in time. Figure 10.3 depicts the two removals from the list.

Figure 10.3 Removing an element from a given index

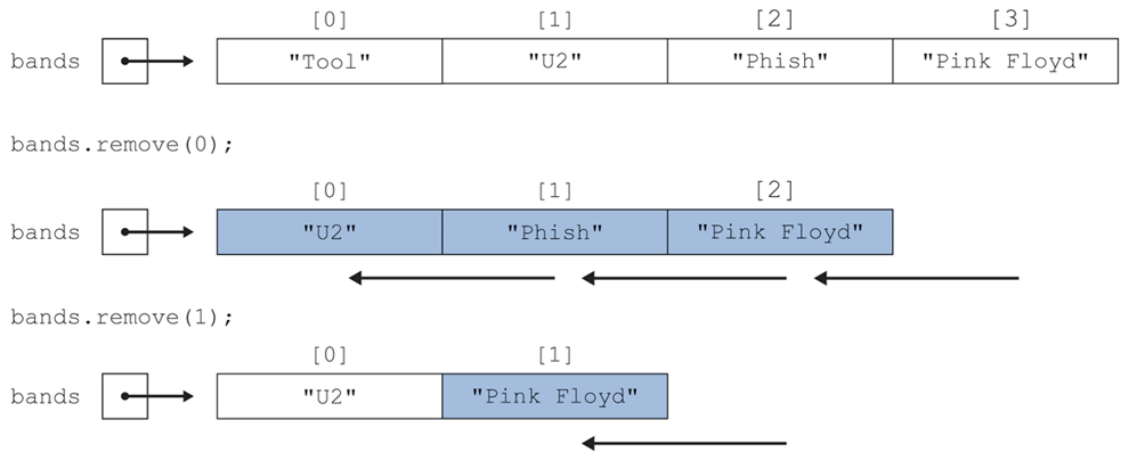


Figure 10.3 Full Alternative Text

If you want to find out the number of elements in an [redacted], you can call its [redacted] method. If you want to obtain an individual item from the list, you can call its [redacted] method, passing it a specific index. These two methods are often used together. For example, the following loop would add up the lengths of the strings in the preceding list of bands:

```
bands.size()
```

```
bands.get(i)
```

This loop looks similar to the kind of loop you would use to access the various elements of an array, but instead of asking for the as you would for an array, you ask for , and instead of asking for as you would with an array, you ask for .

Calling and can be expensive in terms of time because the computer has to shift the values around. If all you want to do is to replace a value, you can use the list's method, which takes an index and a value and replaces the value at the given index with the given value without doing any shifting. For example, you could replace the value at the front of the sample list by writing the following line of code:

As noted earlier, when you construct an it will initially be empty. After you have added values to a list, you can remove them one at a time. But what if you want to remove all of the values from the list? In that case, you can call the list's method.

Table 10.1 summarizes the [redacted] operations introduced in this section. A more complete list can be found in the online Java documentation.

Table 10.1 Basic [redacted] Methods

[redacted]		
[redacted]	adds the given value at the end of the list	[redacted]
[redacted]	adds the given value at the given index, shifting subsequent values right	[redacted]
[redacted]	removes all elements from the list	[redacted]
[redacted]	gets the value at the given index	[redacted]
[redacted]	removes the value at the given index, shifting subsequent values left	[redacted]
[redacted]	replaces the value at the given index with the given value	[redacted]

[redacted]	returns the current number of elements in the list	[redacted]

[redacted] Searching Methods

Once you have built up an [redacted], you might be interested in searching for a specific value in the list. The [redacted] class provides several mechanisms for doing so. If you just want to know whether or not something is in the list, you can call the [redacted] method, which returns a Boolean value. For example, suppose you have an input file of names that has some duplicates, and you want to get rid of the duplicates. The file might look like this:

You can construct an [redacted] to hold these names and use the [redacted] method to ensure that there are no duplicates:

Given the sample input file, this code produces the following output:

Notice that only 8 of the original 13 names appear in this list, because the various duplicates have been eliminated.

Sometimes it is not enough to know that a value appears in the list. You may want to know exactly where it occurs. For example, suppose you want to write a method to replace the first occurrence of one word in an [redacted] with another word. You can call the [redacted] method to replace the value, but you have to know where it appears in the list. You can find out the location of a value in the list by calling the [redacted] method.

The [redacted] method takes a particular value and returns the index of the first occurrence of the value in the list. If it doesn't find the value, it returns [redacted]. So, you could write a [redacted] method as follows:

Notice that the return type of this method is [redacted], even though it changes the contents of an [redacted] object. Some novices think that you have to return the changed [redacted], but the method doesn't actually create a new [redacted]; it merely changes the contents of the list. As you've seen with arrays and other objects, a parameter is all you need to be able to change the current state of an object because objects involve reference semantics in which the method is passed a reference to the object.

You can test the method with the following code:

This code produces the following output:

There is also a variation of [redacted] known as [redacted]. As its name implies, this method returns the index of the last occurrence of a value. There are many situations where you might be more interested in the last occurrence rather than the first occurrence. For example, if a bank finds a broken automated teller machine, it might want to find out the name and account number of the last customer to use that machine. Table 10.2 summarizes the [redacted] searching methods.

Table 10.2 [redacted] Searching Methods

[redacted]		
[redacted]	returns [redacted] if the given value appears in the list	[redacted]
[redacted]	returns the index of the first occurrence of the given value in the list ([redacted] if not found)	[redacted]

[redacted]	returns the index of the last occurrence of the given value in the list ([redacted] if not found)	[redacted]

All of the [redacted] searching methods call the [redacted] method for comparing values. The method names are fairly standard and appear elsewhere in the Java class libraries. For example, the [redacted] class also has methods called [redacted] and [redacted] that allow you to search for the position of a character or substring inside a string.

A Complete [redacted] Program

Before we go further, let's look at a complete [redacted] program. Search engines like Google ignore the *stop words* in users' queries. The idea is that certain words like "a" and "the" appear so often that they aren't worth indexing. Google won't disclose the exact list of words it uses, although a few examples are listed on the web site and people have speculated about what they think is on the list.

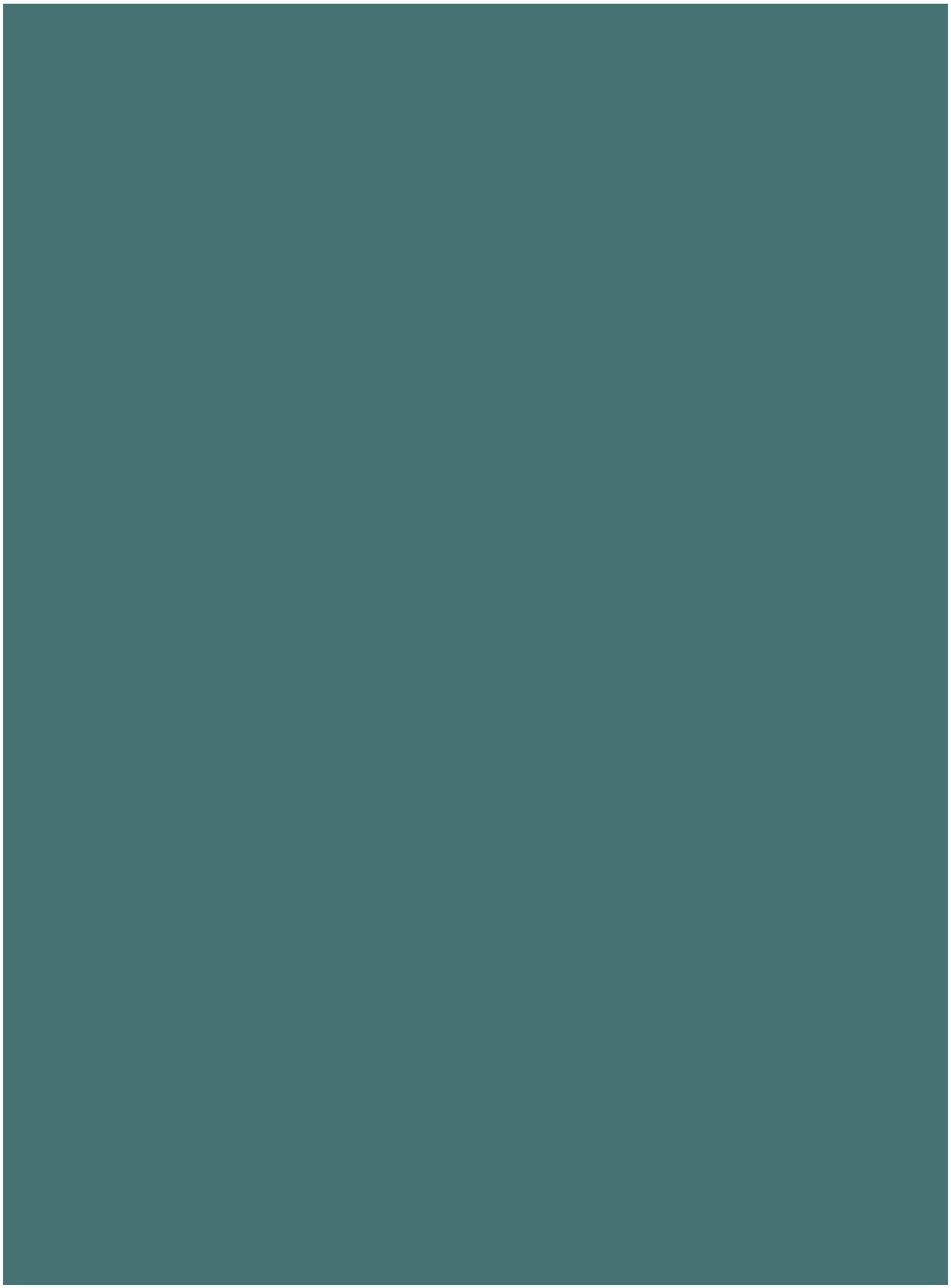
Google's full list of stop words is believed to have at least 35 entries, but we'll settle for 15 of the most obvious choices. To explore how removing stop words can affect a text, our program will read a file called [redacted] that contains the first part of Hamlet's famous speech:

[redacted]

The program constructs a list of stop words and then reads the file word by word, printing every word that is not a stop word. To avoid issues of case, the stop words are all in lowercase and the call on [redacted] is passed a lowercase version of each word from the input file. Here is the complete program:

```
import

import
```

```
public class

public static void

throws

new
```



```
new

new



while









if
```

The program produces the following output:

This output represents the search view of the original text (the core set of words that will be used by a search engine).

Adding to and Removing from an

VideoNote

In this section, we will explore some of the issues that come up when you dynamically add values to or remove values from the middle of an . The results are often surprising, so it is worth exploring the common pitfalls.

Consider the following code, which creates an and stores several words in it:

This code produces the following output:

We'll explore the problem of inserting a tilde ("~") in front of each word, doubling the size of the list. Inserting tildes isn't the most exciting operation you can imagine doing with a list, but we want to keep things simple so we can focus on the programming issues, and you'll find that you often want to perform operations

like this. For example, if you put a tilde in front of a search term, Google does a different search that includes synonyms of the word. Searching for “~four ~score” yields more than 10 times as many pages as searching for just “four score.”

In our case, we want to keep the tildes separate from the words themselves, so we want to insert a new [redacted] containing just a tilde in front of each word in the list. Here is a first attempt that makes sense intuitively but doesn’t work:

This [redacted] loop is a slight variation of the standard array-traversal loop. It has an index variable [redacted] whose value starts at [redacted] and goes up by one each time. In this case, the loop is inserting a tilde at position [redacted] each time it executes the loop. The problem is that the loop never terminates. (If you’re patient enough, you will find that the program does eventually terminate with an “out of memory” error.)

The loop fails to terminate because the [redacted] structure is dynamic in nature. Let’s think about this carefully to see what’s

happening. Initially we have the following list, with the "[redacted]" in position 0:

The first time the program executes the loop, it inserts a tilde at position 0. To make room for the tilde at position 0, the [redacted] has to shift all the other values one place to the right. As a result, the [redacted] "[redacted]" ends up in position 1:

Then we come around the [redacted] loop, increment [redacted] to be [redacted], and insert a tilde at position 1. But because the word "[redacted]" is currently at position 1, this second tilde also goes in front of the word "[redacted]", shifting it into position 2:

We then go around the loop again, incrementing [redacted] to be [redacted] and inserting a tilde at that position, which is once again in front of the word "[redacted]":

This loop continues indefinitely, because we keep inserting tildes in front of the first word in the list. The [redacted] loop test compares [redacted] to the size of the list, but because the list is growing, the size keeps going up. So, this process continues until all the computer's available memory is exhausted.

To fix this loop, we have to take into account the fact that inserting a tilde at position [redacted] is going to shift everything one place to the right. So, on the next iteration of the loop, we will want to insert the tilde in the position that is two to the right, not one to the right. We can fix the code simply by changing the update part of the [redacted] loop to add 2 to [redacted] instead of adding 1 to [redacted]:

When we execute this version of the code, we get the following output:

As another example, let’s consider what code we would need to write to undo this operation. We want to write code that will remove every other value from the list, starting with the first value—in other words, the values that are currently at indexes 0, 2, 4, 6, 8, and 10. We might write code like the following:

Looking at the loop, you can see that [redacted] starts at [redacted] and goes up by 2 each time, which means it produces a sequence of even values ([redacted], [redacted], [redacted], and so on). That seems to be what we want, given that the values to be removed are at those indexes. But this code doesn't work. It produces the following output:

Again, the problem comes from the fact that in the [redacted] values are shifted dynamically from one location to another. The first tilde we want to remove is at index 0:

But once we remove the tilde at position 0, everything is shifted one position to the left. The second tilde moves into index 1:

So the second remove should be at index 1, not index 2. And once we perform that second remove, the third tilde will be in index 2:

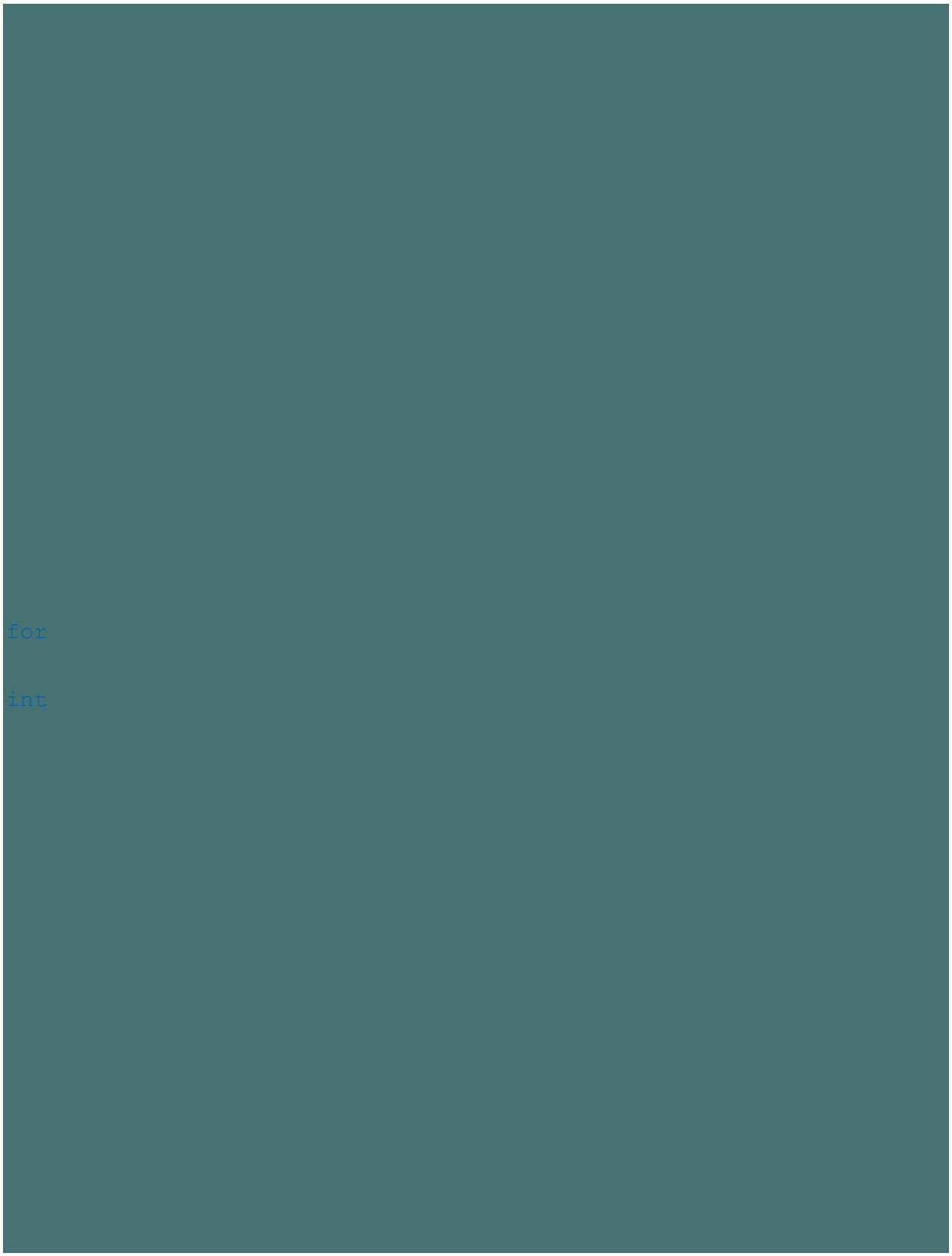
In this case, we don’t want to increment by 2 each time through the loop. Here, the simple loop that increments by 1 is the right choice:

After the program executes this code, it produces the following output:

Putting all of these pieces together gives us the following complete program:

```
import

public class

public static void
```

```
new
for
int
```


for
int

If we want to write the loops in a more intuitive manner, we can run them backwards. The loops we have written go from left to right, from the beginning of the list to the end of the list. We could instead go from right to left, from the end of the list to the beginning of the list. By going backward, we ensure that any changes we are making occur in parts of the list that we have already visited.

For example, we found that the following loop did not work properly even though it seemed like the intuitive approach:

But if we turn this loop around and have it iterate backward rather than going forward, it does work properly:

The problem with the original code was that we were inserting a value into the list and then moving our index variable onto that spot in the list. If instead we work backward, the changes that we make affect only those parts of the list that we have already processed.

Similarly, we tried to write the second loop as follows:

Again, the problem was that we were changing a part of the list that we were about to process. We can keep the overall structure intact by running the loop backward:

Initializing an [redacted]

In Chapter 7 we saw different ways to construct arrays and initialize the values of each element of an array. We saw the typical syntax for creating an array of a given length and setting each of its elements was the following:

If we wanted an [redacted] of these elements rather than an array, the corresponding code would be the following:

It can be tedious to set up an array or list of values in the preceding way because it takes so many lines to store the element values. In Chapter 7 we also showed a syntax for quickly initializing an array from a given list of element values:

Unfortunately the [redacted] does not support the [redacted] syntax we used with arrays. But there is a static method called [redacted] that accepts any number of element values as parameters and returns a list object containing those elements. We can then pass this to the constructor of an [redacted] to quickly initialize the [redacted] to store the values we want:

Previous versions of Java used a method called [redacted] that behaves much the same way as [redacted]. But as of Java 8 the latter method is preferred, partly because it is shorter to type and partly because it is more consistent with methods for creating other types of collections. In the next chapter we will learn about collections called sets and maps, and those collection types have methods called [redacted] and [redacted] that help you create collections. Using a consistent convention like this makes the method names easier to remember.

Using the For-Each Loop with [redacted]s

You saw in Chapter 7 that you can use a for-each loop to iterate over the elements of an array. You can do the same with an [redacted]. For example, earlier we mentioned that the following code could be used to add up the lengths of the [redacted] stored in an [redacted] called [redacted]:

We can simplify this code with a for-each loop. Remember that the syntax of this kind of loop is as follows:

Thus, the preceding loop to add up the lengths of the [redacted] can be rewritten as follows:

This loop is another way of saying, “For each [redacted] contained in [redacted] ....”

Because the for-each loop has such a simple syntax, you should use it whenever you want to process each value stored in a list sequentially. You will find, however, that the for-each loop is not appropriate for more complex list problems. For example, there is no simple way to skip around in a list using a for-each loop. You must process the values in sequence from first to last. Also, you cannot modify the list while you are iterating over it.

Consider, for example, the following sample code:

This code prints a [redacted] from the list and then attempts to remove the value at the front of the list. When you execute this code, the program halts with a [redacted]. Java is letting you know that you are not allowed to iterate over the list and to modify the list at the same time (concurrently). Because of this limitation, neither of the problems discussed in the previous section could be solved using a for-each loop.

Chapter 7 also mentioned that starting with version 8 of Java, there is a different approach to manipulating structures like arrays and lists that allows you to write more concise code. The following two lines of code perform the same task as the two loop versions above.

You can read more about this approach in Chapter 19.

Wrapper Classes

So far, all of the examples we have studied have involved of objects. What if you wanted to form a list of integers? Given that is a generic class, you'd think that Java would allow you to define an , but that is not the case. The in can be filled in with any object or reference type (i.e., the name of a class). The primitive types (e.g., , , , and), cannot be used as type parameters for an .

Instead, Java defines a series of *wrapper classes* that allow you to store primitive data as objects.

Wrapper Class

A class that "wraps" (stores) primitive data as an object.

To understand the role of a wrapper class, think of a piece of candy packaged in a wrapper. Pieces of candy can be sticky and inconvenient to handle directly, so we put them inside wrappers that make handling them more convenient. When we want to eat the actual candy, we open up the wrapper to get the candy out. The Java wrapper classes fill a similar role.

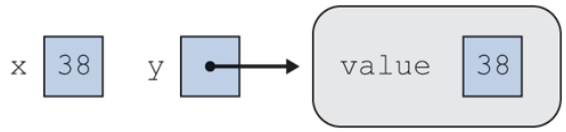
For example, consider simple integers, which are of type , a primitive type. Primitive types are not objects, so we can't use values of type in an object context. To allow such use, we must wrap up each into an object of type . objects are very simple. They have just one field: an value. When we construct an , we pass an value to be

wrapped; when we want to get the [redacted] back, we call a method called [redacted] that returns the [redacted].

To understand the distinction between [redacted] and [redacted], consider the following variable declarations:

[redacted]

This code leads to the following situation in memory:



Primitive data is stored directly, so the variable [redacted] stores the actual value [redacted]. Objects, by contrast, are stored as references, so the variable [redacted] stores a reference to an object that contains [redacted].

If we later want to get the [redacted] out of the object (to unwrap it and remove the candy inside), we call the method [redacted]:

[redacted]

The wrapper classes are of particular interest in this chapter because when you use an , the needs to be a reference type. You can't make an , but you can form an . For example, you can write code like the following that enters several integer values into a list and adds them together:

This code produces the following output:

The code takes advantage of a mechanism that Java provides for simplifying code which involves the use of wrapper classes. For example, Java will convert between [redacted] values and [redacted] values for you when your intent seems clear. Given the declaration of the variable [redacted] as an [redacted], Java would normally expect you to add values of type [redacted] to the list. But in the preceding code you were adding simple [redacted] values, as in:

When it reaches this line of code, Java sees that you are adding an [redacted] to a structure that is expecting an [redacted]. Because Java understands the relationship between [redacted] and [redacted] (each [redacted] is simply an [redacted] wrapped up as an object), it will automatically convert the [redacted] value into a corresponding [redacted] object. This process is known as *boxing*.

Boxing

An automatic conversion from primitive data to a wrapped object of the appropriate type (e.g., an boxed to form an).

Similarly, you don’t have to do anything special to unwrap an to get the inside. You could write code like the following:

This code multiplies two values from the and stores the result in a variable of type . The calls on will return an object, so normally these values would be incompatible. However, because Java understands the relationship between and it will unwrap the objects for you and give you the values stored inside. This process is known as *unboxing*.

Unboxing

An automatic conversion from a wrapped object to its corresponding primitive data (e.g., an unboxed to yield an).

Notice that you can write a for-each loop to use a variable of type [redacted] even though the [redacted] stores values of type [redacted]. Java will unbox the objects and perform the appropriate conversions for you.

Because Java has boxing and unboxing, the only place you generally need to use the wrapper class is when you describe a type like [redacted]. You can't actually declare it to be of type [redacted], but you can manipulate it as if it is of type [redacted].

Table 10.3 lists the major primitive types and their corresponding wrapper classes.

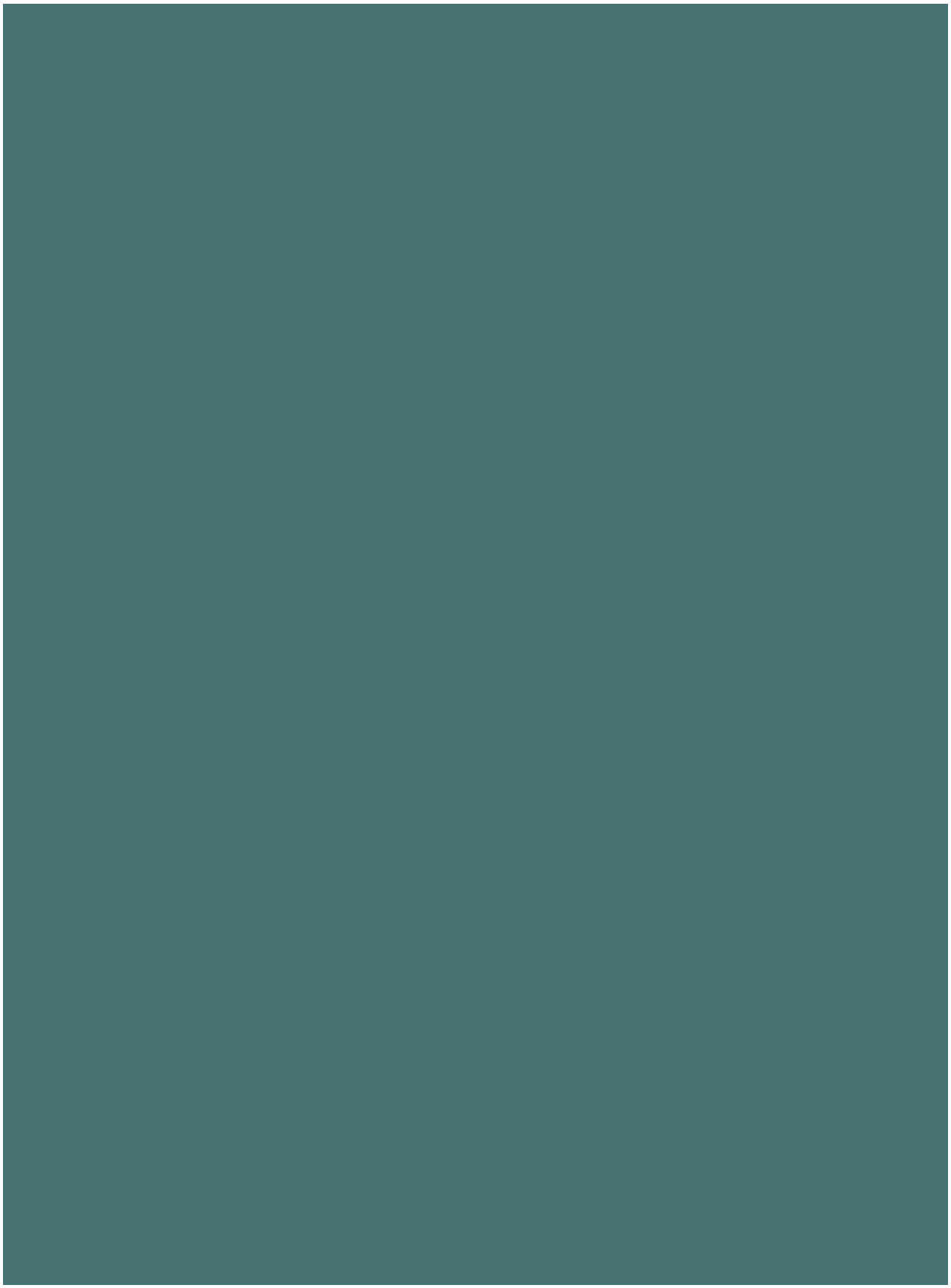
Table 10.3 Common Wrapper Classes

[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]

10.2 The [redacted] Interface

The method [redacted] can be used to sort an [redacted]. It is part of the [redacted] package. The following short program demonstrates how to use [redacted]:

```
import

public class

public static void

new
```



This program produces the following output:

In Chapter 13, we will explore how this sorting method actually works. For now we are simply going to be clients of the method without worrying about how it works.

If you try to make a similar call to an , you will find that the program does not compile. Why is it possible to sort a list of objects but not a list of objects? The answer is that the class implements the interface, while the class does not. In this section we will explore the details of the interface and explain how to write classes that implement it.

DID YOU KNOW?

Controversy over Boxing and Unboxing

Not all software developers are happy with the decision to add boxing and unboxing to the Java language. The ability to manipulate an almost as if it were an can simplify code, and everyone agrees that simplification is good. The disagreement comes from the fact that it is *almost* like an . Some argue that “almost” isn’t good enough. Because it comes close, programmers are likely to use it and eventually come to count on it. That can prove disastrous when “almost” isn’t “always.”

As an analogy, suppose someone told you that you could use a device that is almost like a potholder to pick up hot objects. In most cases, it will protect your hand from heat. So you start using it, and while you might be nervous at first, you soon find that it seems to work just fine. And then one day you pick up a new object and you get burned. You can think of similar analogies with aircraft landing gear that almost works or vests that are almost bulletproof.

For a programming example, consider the following code:

It's difficult to know exactly what this code will do. If you assume that is "almost" like an , you'd probably think that the code would print the message that the two values are equal. In fact, there is no guarantee as to what it will do. In the current version of Java, it prints the message "unequal."

Remember that testing for object equality is not as simple as testing for equality of primitive data. Two s might store the same text but might not be the same object, which is why we call the method to compare s. The same principle applies here: The two elements might store the same but might not be the same object. The code prints "unequal" in the current release of Java because the program creates two different objects that each store the value . However, to add to the confusion, if we change the value from to , the program will print that the two values are equal.

The Java Language Specification guarantees that this code will work for any value of between and , but it provides no guarantee as to how the code will behave for other values of . For those other values, it could print either message, and this might change from one implementation of Java to another. It

might be that in the next version of Java released, the code will print “equal” for but not for a value like .

Some people have argued that because boxing and unboxing cover up what is happening underneath, it is better not to use them at all. Boxing and unboxing don’t necessarily simplify anything if they work only “sometimes,” because you have to be able to understand the cases in which they don’t work.

Natural Ordering and

We are all familiar with many kinds of data that can be sorted. For example, we are used to putting numbers in order from lowest to highest or alphabetizing lists of names. We describe types that can be sorted as having a *natural ordering* of values. To have such an ordering of values, a type needs to have a well-defined *comparison function* that indicates the relationship between any pair of values.

Comparison Function

A well-defined procedure for deciding, given a pair of values, the relative order of the two values (less than, equal to, or greater than).

Natural Ordering

The order imposed on a type by its comparison function.

Not all types have natural orderings because not all types have comparison functions. For example, in this chapter we have been exploring how to construct a variety of objects. How would you compare two objects to determine whether one is less than another? What would it mean for one to be less than another? You might decide to use the lengths of the lists to determine which one is less, but what would you do with two objects of equal length that store different values? You wouldn't want to describe them as "equal." There is no agreed-upon way of ordering s, and therefore there is no comparison function for this type. As a result, we say that the type does not have a natural ordering.

Java has a convention for indicating the natural ordering of a type. Any type that has such an ordering should implement the interface:

This interface provides a second example of a generic type in Java. In the case of , Java uses the letter "E," which is

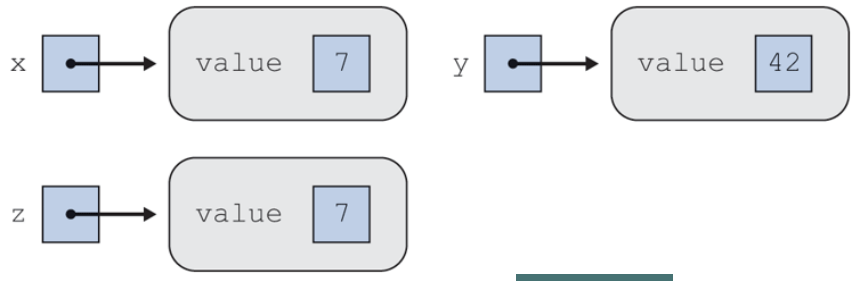
short for “Element.” In the case of [redacted], Java uses the letter “T,” which is short for “Type.”

The [redacted] method is the comparison function for the type. A boolean return type can’t be used because there are three possible answers: less than, equal to, or greater than. The convention for [redacted] is that an object should return one of the following results:

- A negative number to indicate a less-than relationship
- [redacted] to indicate equality
- A positive number to indicate a greater-than relationship

Let’s look at a few examples to help you understand this concept. We have seen that Java has [redacted] objects that serve as wrappers for individual [redacted] values. We know how to compare [redacted] values to determine their relative order, so it is not surprising that the [redacted] class implements the [redacted] interface. Consider the following code:

This code begins by constructing three [redacted] objects:



Then it includes a series of [redacted] statements that report the results of various pairwise comparisons. In the first [redacted] statement, we compare [redacted] to [redacted], which involves comparing the [redacted] value [redacted] to the [redacted] value [redacted]. This pair has a less-than relationship because [redacted] is less than [redacted], so the method call returns a negative integer. In the second [redacted] statement, we compare [redacted] to [redacted], which involves comparing one occurrence of the [redacted] value [redacted] with another occurrence of the [redacted] value [redacted]. This second pair has an equality relationship because [redacted] equals [redacted], so the method call returns [redacted]. In the final [redacted] statement, we compare [redacted] to [redacted], which involves comparing the [redacted] value [redacted] to the [redacted] value [redacted]. This final pair has a greater-than relationship because [redacted] is greater than [redacted], so the method call returns a positive integer.

Here is the actual output of the code:

[redacted]

The values [redacted], [redacted], and [redacted] are the standard values returned, but the [redacted] method is not required to return these specific values. For example, consider a similar piece of code that compares [redacted] values:

The [redacted] method of the [redacted] class compares strings alphabetically, so there are similar relationships in this code: [redacted] is less than [redacted] because in an alphabetical list [redacted] comes before [redacted], [redacted] is equal to [redacted] because the two occurrences of [redacted] are equal, and [redacted] is greater than [redacted] because in an alphabetical list [redacted] comes after [redacted]. But the output produced is slightly different from that produced by the [redacted] example:

Instead of [redacted] and [redacted], we get [redacted] and [redacted]. You don't really need to know where these numbers come from—the only important fact is whether they are negative or positive—but for those of you who are curious, the [redacted] and [redacted] represent the distance between the positions of the characters [redacted] and [redacted] in type [redacted]. [redacted] appears 15 positions later than [redacted].

So while the values [redacted] and [redacted] are often returned by a comparison function, that won't always be the case. The important thing to remember is that "less-than" relationships are indicated by a negative number and "greater-than" relationships are indicated by a positive number.

Also keep in mind that the relationship operators that we've been using since Chapter 4 have a different syntax. For example, you've seen that if two variables [redacted] and [redacted] are of type [redacted] or [redacted], you can compare them by using operators like [redacted] and [redacted]:

Even though the [redacted] class implements the [redacted] interface, you can't use the relational operators to compare [redacted]. The following code will not compile:

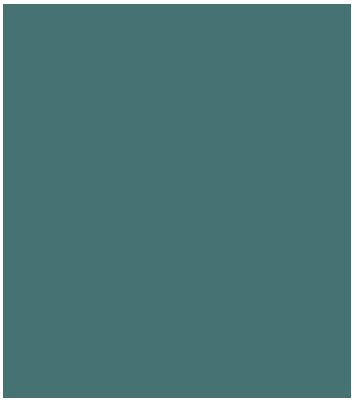
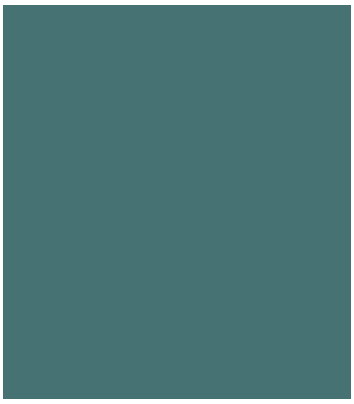
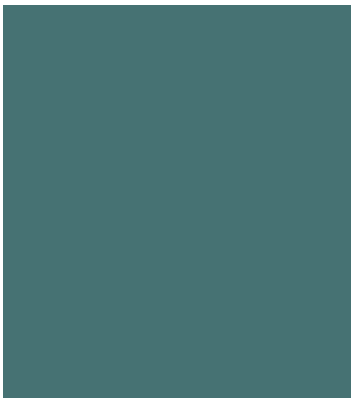
Instead, call the [redacted] method, as in:

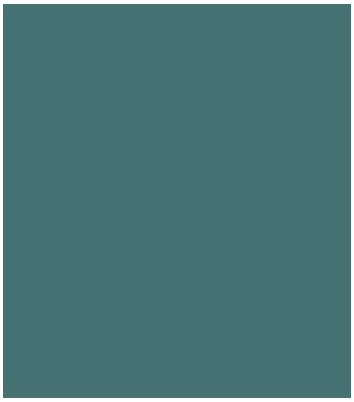
```
s1.compareTo(s2) < 0
```

You can use a relational operator in this context because the [redacted] method returns an [redacted]. Notice that the specific value of [redacted] isn't used for [redacted] because you are only guaranteed to get a negative value for a less-than relationship. Table 10.4 summarizes the standard way to compare objects that implement the [redacted] interface.

Table 10.4 Comparing Values Summary

less than	[redacted]	[redacted]
less than or equal to	[redacted]	[redacted]

equal to		
not equal to		
greater than		

greater than or equal to		

Implementing the Interface

Many of the standard Java classes, such as , implement the interface. You can have your own classes implement the interface as well. Implementing the interface will open up a wealth of off-the-shelf programming solutions that are included in the Java class libraries. For example, there are built-in methods for sorting lists and for speeding up searches. Many of these features will be discussed in the next chapter.

As a fairly simple example, let's explore a class that can be used to keep track of a calendar date. The idea is to keep track of a particular month and day, but not the year. For example, the United States celebrates its independence on July 4 each year. Similarly, an organization might want a list of its employees' birthdays that doesn't indicate how old they are.

DID YOU KNOW?

Why Not , , and for ?

As we discussed earlier, some types like return the values , , and when you call . These are the *canonical* values for to return, because they correspond to a function in mathematics known as the *signum* function (sometimes abbreviated “sgn”). However, other types like do not return the standard values. You might wonder why Java doesn’t require that all classes return , , and when you call .

One answer to this question is that Java doesn’t have a convenient ternary type. For any binary decision, we can use as the return type. But what type do we use if we want to return exactly one of three different values? There is no predefined type that has just three values, so it’s more honest in a sense to use the rule that any negative number will do and any positive number will do. Suppose that Java said that should return only , , and . What should happen when someone writes a that returns something else? Ideally, any code calling that particular would throw an exception when it gets an illegal return value, but that would require programmers to write a lot of error-checking code. By saying that all negatives will be interpreted one way, all positives will be interpreted a second way, and will be interpreted a third way, Java provides a complete definition for all values of type

, which makes it easier for programmers to work with the method.

A second reason for having behave this way is that it then is easy to express many comparison tasks directly. It is often convenient to express the comparison as a difference between two values. This pattern occurs in many places. For example, the class uses *lexicographic* order (also called “dictionary” or “alphabetic” order). To determine the relationship between two , you scan through them until you find the first pair of letters that differ. For example, if you were comparing and you’d find that the first pair of characters that differ is the third pair (versus). You would then return the difference between the character values (' '). If you don’t find such a pair, then you return the difference between the lengths. For example, " " is considered greater than " " on the basis of length.

The behavior for the class can be described with the following pseudocode:

```
search for a pair of characters in corresponding positions that
differ.

if (such a pair exists) {


return the difference between the two characters.

} else {


return the difference between the two lengths.

}
```

Notice that this approach returns in just the right case, when there are no character pairs that differ and when the strings have the same length. Having the flexibility to return any negative integer for “less than” and any positive integer for “greater than” makes it easier to implement this approach.

The final reason not to specify the return values for is efficiency. By having a less strict rule, Java allows programmers to write faster methods. The method in the class is one of the most frequently called methods. All sorts of data comparisons are built on comparisons, and performing a task like sorting thousands of records will lead to thousands of calls on the class’s method. As a result, it’s important for the method to run quickly. We wouldn’t want to unnecessarily complicate the code by requiring that it always return , , or .

We can implement this class of dates with two fields to store the month and day:

Remember that to implement an interface, you include an extra notation in the class header. Implementing the [redacted] interface is a little more challenging because it is a generic interface ([redacted]). We can't simply write code like the following:

We have to replace the [redacted] in [redacted]. Whenever you implement [redacted], you compare pairs of values from the same class. So a class called [redacted] should implement [redacted]. If you look at the header for the [redacted] class, you will find that it implements

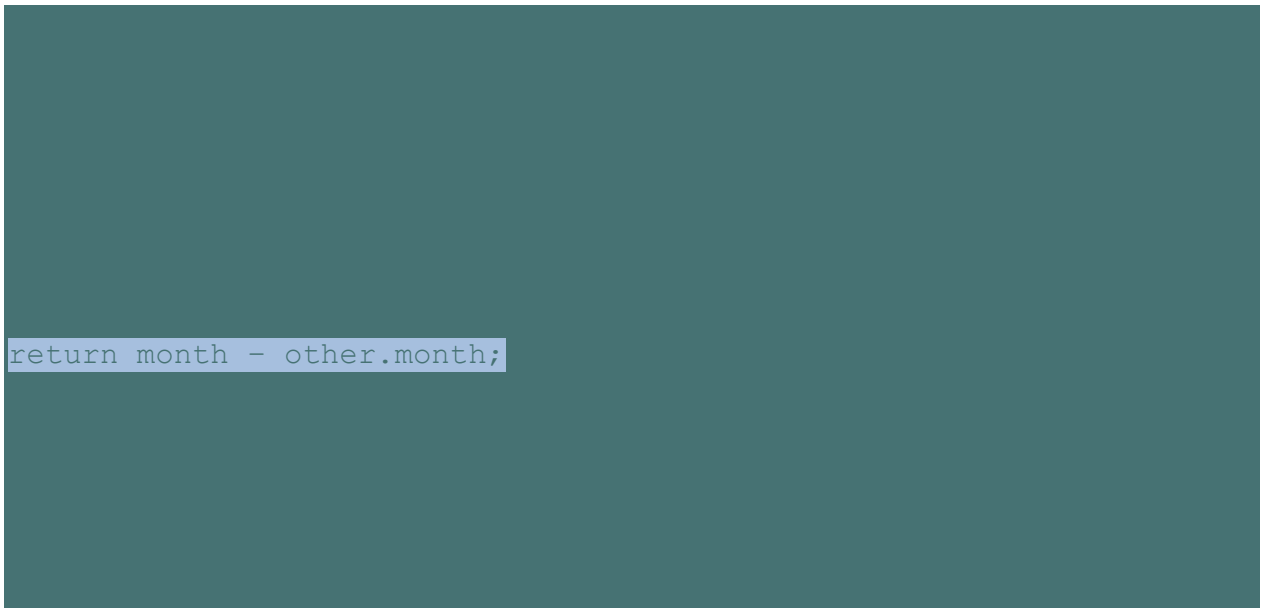
. Likewise, the class implements . So we need to change the header to the following one:

Of course, claiming to implement the interface is not enough. We also have to include appropriate methods. In this case, the interface includes just a single method:

Because we are using in place of , we need to write a method that takes a parameter of type :

Now we have to figure out how to compare two dates. Each [redacted] object will contain fields that store the month and day. With calendars, the month takes precedence over the day. If we want to compare January 31 (1/31) with April 5 (4/5), we don’t care that 5 comes before 31; we care instead that January comes before April. So, as a first attempt, consider the following method that compares only the months:

We need to consider more than just the month, but before we do that, we can improve on what we have here. This code uses a nested construct to return the standard values of , , and , but a simpler option is available. We can simply return the difference between and , because it will be negative when is less than , it will be when they are equal, and it will be positive when is greater than . So, we can simplify the code as follows:



It is a good idea to keep things simple when you can, so this version is preferable to the original one. It returns slightly different values than the earlier version, but it satisfies the contract of the

[redacted] interface just as well. However, the code still has a problem.

Consider, for example, a comparison of April 1 (4/1) and April 5 (4/5). The current version of [redacted] would subtract the months and return a value of [redacted], indicating that these two dates are equal. However, the dates aren't equal: April 1 comes before April 5.

The day of the month becomes important only when the months are equal. If the months differ, we can use the months to determine order. Otherwise (when the months are equal), we must use the day of the month to determine order. This common ordering principle is present in many tasks. We can implement it as follows:

```
if (month != other.month) {



} else {

return day - other.day;

}
```

It is still possible for this code to return . Suppose that we have two objects that both store the date April 5 (4/5). The months are equal, so the program returns the difference between the days. That difference is 0, so the program returns , which correctly indicates that the two dates are equal.

Here is a complete class with the method, two accessor methods, and a method:

```
private int


public

int

int


this


this
```

```
public int


if


return


else


return


public int
```

```
return

public int

return

public

return
```

One of the major benefits of implementing the interface is that it gives you access to built-in utilities like . As we mentioned previously, you can use to sort an but not to sort an , because the class does not implement . The class implements the interface, so, as the following short program demonstrates, we can use for an :

```
import

public class
```

```
public static void
```

```
new
```

```
new
```

```
new
```

```
new
```

```
new
```

This program produces the following output:

Notice that the dates appear in increasing calendar order after the call on [redacted].

10.3 Case Study: Ranked Choice Voting

In this section, we will use [redacted]s to solve a complex problem. We will develop a program that implements what is known as *ranked choice voting*. Ranked choice voting has been getting more attention in the United States in recent years with cities like San Francisco and Minneapolis using it along with the state of Maine.

Ranked choice voting (also called “instant runoff”) allows voters to specify what order they prefer for various candidates. Suppose that you have eight voters and four candidates. To keep things simple, we will refer to candidates using the letters A, B, C, and D. Below is a list of voting preferences. Each line has a ranked list of the four candidates and represents a single voter’s preferences.

The usual voting procedure is to tally the first choice of each voter, which means looking at the votes in the first column. If you tally those votes, nobody ends up with a clear majority. Candidate A has three votes or 37.5% of the eight total votes. In ranked choice voting when no candidate has a majority, we eliminate the candidate with the lowest vote total, which in this case is candidate D. That leaves us with these preferences:

There is still no candidate with a majority, so we eliminate the one with the low vote total, which is candidate B leaving us with:

Now we see that candidate C has five votes and wins the election. Notice that in the first round of voting, candidate C was tied for second place. But more voters liked candidate C as a substitute if they couldn't get the candidates they preferred more.

We are going to develop a program that performs the ranked choice voting algorithm assuming we have complete preferences from each voter. As with other case studies, we will develop it in stages:

1. We will develop a class called [redacted] to store information about one voter's preferences.

2. We will write a program that reads an input file of voter preferences and that tallies the results for each voter's first choice pick.
3. We will introduce code to repeatedly eliminate the candidate with the lowest vote total until some candidate gets a majority or we end up with a tie with two candidates each with the same vote total.

Ballot Class

In solving this problem, you can identify two kinds of data: the data for a single voter which we are calling a ballot, and the list of ballots. It makes sense to introduce a class for storing the information for a single ballot.

Assume that as with the lists shown previously, each voter's preferences will be listed as a single line of data with the candidates listed from most desirable to least desirable. One common way to store a file with text like this is to separate the

different names with a tab character. Spreadsheet programs have an option to save data as a tab-delimited file that would have this structure. You will see that it is fairly easy to break up such a string into an array of strings. It makes sense, then, to assume the constructor will take an array of strings as a parameter:

You need to store this sequence of names. You could use an array, but you know that the algorithm involves eliminating candidates who get lower vote totals. It would be better to store the names in an [redacted] that can be stored as a field of the class:

To complete the constructor, you need to construct this list and add each name to your list. This is a nice place to use a for-each loop:

The ballot includes all of the candidates, but the code that processes a ballot needs to know which candidate is the current top choice. That will always be the first element of your list, the one located at index 0. You can therefore provide a method for requesting the current choice of candidate by looking at the first element of the list:

As noted earlier, this approach will work only if each ballot contains information for all candidates. If you don’t have complete information, then eventually the list of candidates can become empty. This case study assumes complete information, but you could extend the code to deal with incomplete information (e.g., when a voter doesn’t want to include some candidate).

Another important behavior you have to implement is to eliminate a candidate from consideration. The [redacted] comes in handy for this because you can simply call its [redacted] method to remove a value:

You need one final behavior for this class. To make it easier to count votes, it would be helpful if a list of ballots could be sorted by candidate name. That way all of the votes for a given candidate will be grouped together. That means that you need to have the [redacted] class implement the [redacted] interface. The idea is to make it easier to count votes, so you want to use the

candidate names to put them in order. The names are stored as strings and the [redacted] class implements [redacted]. That means you can use the [redacted] method of the individual strings that represent the candidate names to perform this comparison:

Putting all of these pieces together, you end up with the following complete class:

```
public class

implements




private

                    

                    

                    

                    

                    

public

                    

                    

new

                    

                    

for

                    

                    

                    

                    

                    

                    
```

```
public

return

public void
```

```
public int


return
```

Counting Votes

Now that we have code to represent a ballot, we can turn to writing the overall program in a separate file called [redacted]. Assume that votes are stored in a file called [redacted]. The first version of your program is supposed to read the information from this file to create an [redacted] of [redacted] objects. You can then sort the list by candidate name and count the votes. As with the calendar sample program, you can call

[redacted] to sort the list. The reading and vote counting operations deserve to be written as methods. That leads to the following [redacted] method:

The [redacted] method involves reading the input file line by line and adding [redacted] objects to an [redacted]. The [redacted] constructor expects an array of strings to specify the candidate choice. Recall that the plan is to have each input line list the candidates with tabs between candidate names. The [redacted] class has a method called [redacted] that provides a convenient way to break up this string into the individual names. It takes a parameter that specifies what separator or delimiter to use. The reading method, then, can be written as follows:

The [redacted] method takes a bit more effort to complete. The [redacted] method calls [redacted] to put the ballots in order by candidate name. That means that all of the votes for a particular candidate will be grouped together. If you use simple names like A, B, C, and D for candidates, then all of the votes for candidate A will come first followed by all of the votes for candidate B, and so on.

This problem involves two levels of structure that can be useful to separate into different methods. You need an outer loop that processes the first candidate, then the second candidate, and so

on. For each candidate, you need to count how many votes that candidate got.

If you use the sample set of eight votes mentioned in the beginning of the section and you store them in an [redacted] ordered by the name of the candidate, you end up with the sequence shown in Figure 10.4.

Figure 10.4 ArrayList of eight ballots

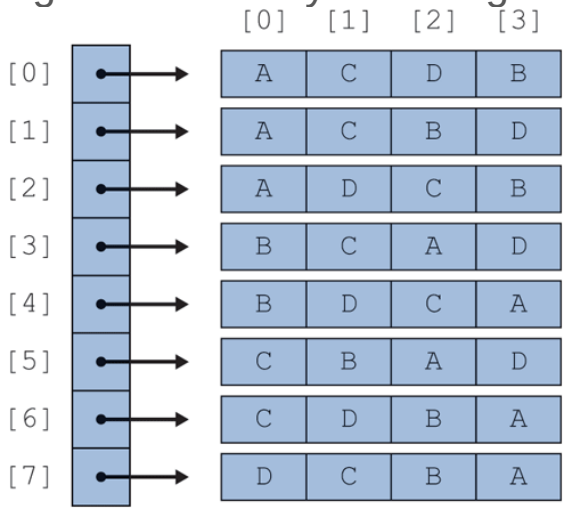


Figure 10.4 Full Alternative Text

The list elements 0 through 2 represent votes for candidate A, which means that A got 3 votes. The list elements 3 through 4 represent votes for candidate B, which is two votes. To process this list candidate by candidate, you can keep track of a current index and then compute the number of votes for the next candidate. This vote count can then added to the index to position to the beginning of the votes for the next candidate. For example, you can start the index at 0 for the first candidate and given that the first candidate got 3 votes, you know to increase the index to 3 to process votes for the second candidate.

In counting the votes, you need access to the list of ballots and you also need to know which index to start at. It isn't a bad idea to also provide the name of the candidate. The following code implements this strategy:

The second level of structure is to implement the [redacted] method that counts the votes for a single candidate. Remember that the method is passed the starting index, candidate name, and list of ballots as parameters:

It needs to have a loop that increments the index while the next name in the list matches the given candidate, counting as it goes. The following first attempt is not quite right:

To see the bug in the preceding code, ask yourself, what happens when you reach the last candidate? You can’t count on finding another element of the list with a different name. That means you

need an additional test to make sure the index hasn't become too high. You have to put the new test first so that you avoid getting an [redacted]:

```
index < ballots.size() &&
```

Once you know the count for this candidate, you can compute and report their percentage of the vote. Since we want to round the results, this is a good place to use a [redacted] statement:

The final step for the method is to return the count. Putting these pieces together, you get the following complete program:

```
import

import

public class

public static void

throws
```

```
new

new









public static


new
```

```
while

new

return

public static void
```

```
int


while


int
```

```
public static int

int









int



while

```

```
double
```

```
return
```

When you run the program using the short input file of eight votes, you get the following output:

The program does not yet eliminate candidates to find the overall election winner, but we'll add that functionality in the next section.

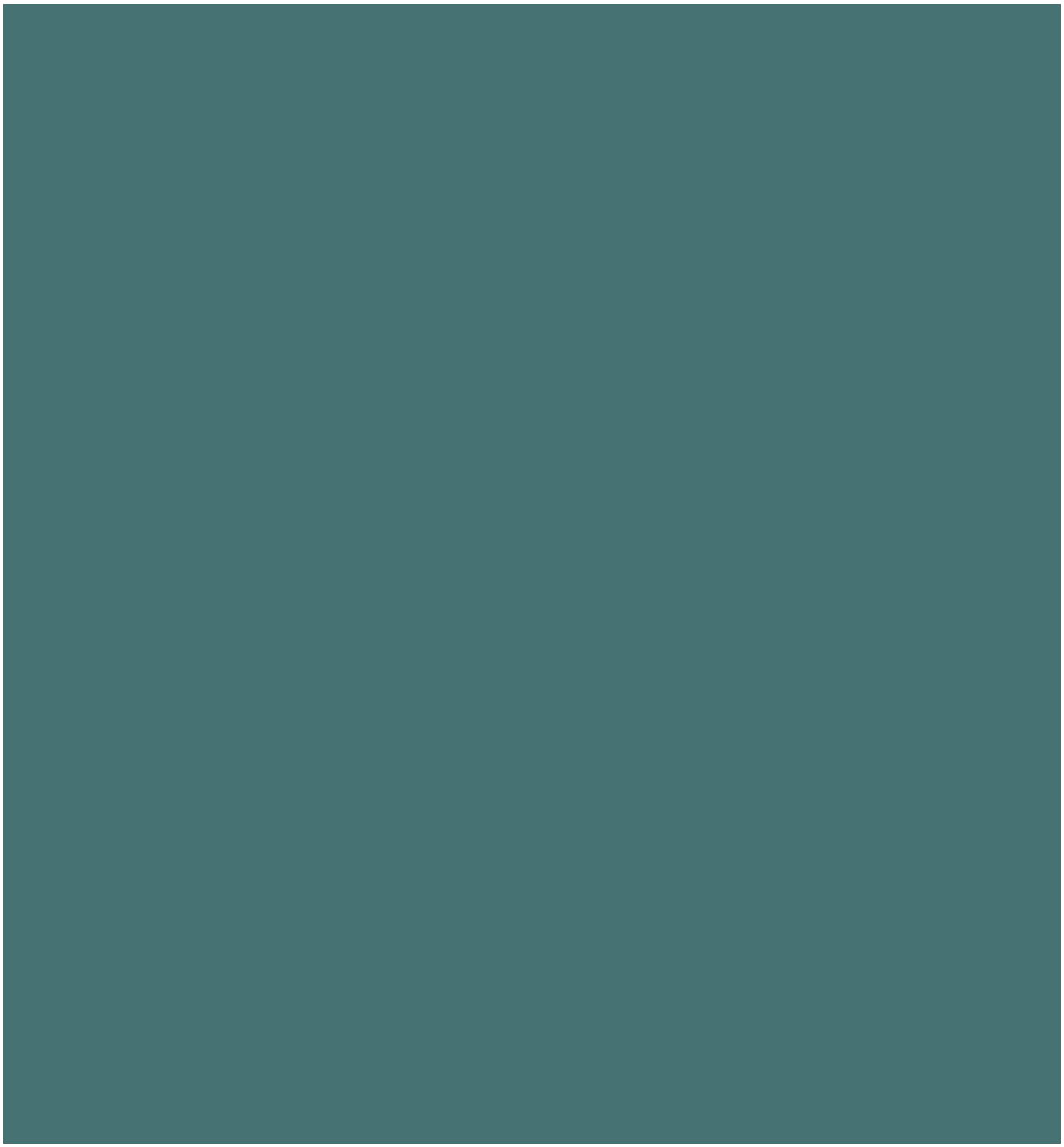
Multiple Rounds

The current version of the program counts the votes for each candidate and returns those counts to the method called [redacted]. The remainder of the program involves multiple rounds where candidates are eliminated until a winner is found. You can modify the [redacted] method to return a [redacted] value indicating whether or not the voting has ended. If you add some code for reporting the round number and include a line of dashes to separate the different rounds of output, you end up with the following modifications to our [redacted] program's [redacted] method. Notice that we have to sort the ballots after each round to order them by each voter's next preferred candidate:

The header of the [redacted] method will have to be changed to reflect the fact that it returns a [redacted] result.

The method has a loop that processes one candidate at a time. To determine whether there is a winner, you need to know which candidate got the most votes. If there is no winner, you have to eliminate the candidate with the lowest vote total, which means you would need to also know which candidate got the lowest number of votes. These are classic max/min computations. You need to know both the candidate names and their vote totals, so

there are a lot of variables involved, but otherwise this is a straightforward task of including the standard max/min code to the existing loop. You can initialize the minimum to one more than the total number of ballots and initialize the maximum to 0, and you can initialize the candidate names to , which leads to the following code:



There are three possible outcomes. If the count for the top candidate is more than 50%, then that candidate has won. Otherwise you normally would eliminate the candidate with the lowest vote total. But there is one special case that is important to consider. When you get down to just two candidates, there might be a tie where each of the remaining candidates earns exactly 50% of the vote. In that case, there is no winner and you would want to stop the algorithm from continuing. The task of eliminating a candidate deserves to be its own method. The rest of the code can be written as a nested [redacted] that distinguishes these three cases, producing an appropriate message and returning either [redacted] or [redacted] depending upon whether the voting has ended. That leads to the following code:

The task of eliminating a candidate simply involves calling the [redacted] object's [redacted] method for each of the ballots in the list. This is a good place to use a for-each loop:

Putting these pieces together, you get the following complete program:

```
import

import


public class
```

```
public static void



throws



new

new









int



boolean

false



while


```

```
public static

new
```

```
while

new

return
```

```
public static boolean

int

int

int

while
```

```
int

if

if
```

```
if


return true


else if


return true


else
```

```
return false
```

```
public static int
```

```
int
```

```
while
```

```
double
```

```
return
```

```
public static void




for
```

Given the previous short input file of eight votes, it produces the following output:

```
```

Chapter Summary

The [redacted] class in Java's [redacted] package represents a growable list of objects implemented using an array. You can use an [redacted] to store objects in sequential order. Each element has a zero-based index.

[redacted] is a generic class. A generic class accepts a data type as a parameter when it is created, like [redacted].

An [redacted] maintains its own size for you; elements can be added to or removed from any position up to the size of the list. Other [redacted] operations include [redacted], [redacted], [redacted], and [redacted].

[redacted] can be searched using methods named [redacted], [redacted], and [redacted].

Java's for-each loop can be used to examine each element of an [redacted]. The list cannot be modified during the execution of the for-each loop.

When you are storing primitive values such as [redacted] or [redacted] into an [redacted], you must declare the list with special wrapper types such as [redacted] and [redacted].

The [redacted] interface defines a natural ordering for the objects of a class. Objects that implement [redacted] can be placed into an [redacted] and sorted. Many common types (such as [redacted] and [redacted]) implement [redacted].

You can implement the [redacted] interface in your own classes by writing a method [redacted].

Self-Check Problems

Section 10.1:

1. What is an ? In what cases should you use an rather than an array?
2. Which of the following is the correct syntax to construct an to store integers?
   a.
   b.
   c.
   d.
   e.

3. The next five questions refer to the following elements:

Write the code to declare an containing these elements. What is the size of the list? What is its type?

4. Write code to insert two additional elements, "" and "", at the proper places in the list to produce the following as the result:

5. Write code to change the second element's value to "[redacted]", producing the following [redacted] as the result:

[redacted]

6. Write code to remove from the list any [redacted] that contain the letter "[redacted]". The following should be the list's contents after your code has executed:

[redacted]

7. Write code to declare an [redacted] holding the first 10 multiples of 2: 0, 2, 4,..., 18. Use a loop to fill the list with the proper elements.
8. Write a method called [redacted] that takes an [redacted] of [redacted]s as a parameter and that returns the length of the longest [redacted] in the list. If your method is passed an empty [redacted], it should return [redacted].
9. Write code to print out whether or not a list of [redacted]s contains the value "[redacted]". Do not use a loop.
10. Given the [redacted] from problem 4, write code to print out the index at which your list contains the value "[redacted]" and the index at which it contains "[redacted]". Do not use a loop.

11. Given the [redacted] from problem 4, write a for-each loop that prints the uppercase version of each [redacted] in the list on its own line.
12. When the code that follows runs on an [redacted] of [redacted]s, it throws an exception. Why?

13. The code that follows does not compile. Why not? Explain how to fix it.

14. What is a wrapper class? Describe the difference between an [redacted] and an [redacted].
15. Write the output produced when the following method is passed each of the following lists:

a. [redacted]
b. [redacted]
c. [redacted]

16. Write the output produced when the following method is passed each of the following lists:

a. [redacted]
b. [redacted]
c. [redacted]

17. Write the output produced when the following method is passed each of the following lists:

a. [redacted]
b. [redacted]
c. [redacted]

18. Write the output produced when the following method is passed each of the following lists:

[redacted]

a. [redacted]
b. [redacted]
c. [redacted]

Section 10.2: The [redacted] Interface

19. Describe how to arrange an [redacted] into sorted order. What must be true about the type of elements in the list in order to sort it?
20. What is a natural ordering? How do you define a natural ordering for a class you've written?
21. Consider the following variable declarations:

[redacted]

Indicate whether the result of each of the following comparisons is positive, negative, or [redacted]:

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]
f. [redacted]

22. Use the [redacted] method to write code that reads two names from the console and prints the one that comes first in alphabetical order. For example, the program's output might look like the following:

[redacted]

23. Write code to read a line of input from the user and print the words of that line in sorted order, without removing duplicates. For example, the program output might look like the following:

[redacted]

Exercises

1. Write a method called [redacted] that takes an [redacted] of strings as a parameter and returns the average number of vowel characters (a, e, i, o, u) in all Strings in the list. If your method is passed an empty [redacted], it should return [redacted].
2. Write a method called [redacted] that switches the order of values in an [redacted] of strings in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, then the next two, and so on. If the number of values in the list is odd, the method should not move the final element. For example, if the list initially stores [redacted], your method should change the list's contents to [redacted].

3. Write a method called [redacted] that takes an [redacted] of strings as a parameter and removes all of the strings of even length from the list.
4. Write a method called [redacted] that takes an [redacted] of strings as a parameter and replaces every string with two of that same string. For example, if the list stores the values [redacted] before the method is called, it should store the values [redacted] after the method finishes executing.
5. Write a method called [redacted] that takes an [redacted] of integers as a parameter and replaces every integer of

value k with k copies of itself. For example, if the list stores the values [redacted] before the method is called, it should store the values [redacted] after the method finishes executing. Zeroes and negative numbers should be removed from the list by this method.

6. Write a method called [redacted] that takes an [redacted] of integers as a parameter and moves the minimum value in the list to the front, otherwise preserving the order of the elements. For example, if a variable called [redacted] stores [redacted], the value [redacted] is the minimum, so your method should modify the list to store the values [redacted].
7. Write a method called [redacted] that takes as a parameter a sorted [redacted] of strings and eliminates any duplicates from the list. For example, if the list stores the values [redacted] before the method is called, it should store the values [redacted] after the method finishes executing. Because the values will be sorted, all of the duplicates will be grouped together. Assume that the [redacted] contains only [redacted] values, but keep in mind that it might be empty.
8. Write a method called [redacted]s that takes as a parameter an [redacted] of integers and eliminates any occurrences of the number [redacted] from the list. For example, if the list stores the values [redacted] before the method is called, it should store the values [redacted] after the method finishes executing.
9. Write a method called [redacted]s that takes as a parameter an [redacted] of integers and returns the

number of indexes apart the two farthest occurrences of the number [redacted] are. For example, if the list stores the values [redacted] when the method is called, it should return [redacted], because the occurrences of [redacted] that are farthest apart are at indexes 2 and 7, and the range 2 through 7 has six elements. If only one [redacted] occurs in the list, your method should return [redacted]. If no [redacted]s occur, your method should return [redacted].

10. Write a method called [redacted] that accepts three parameters, an [redacted] of strings, a beginning string, and an ending string, and removes from the list any strings that fall alphabetically between the start and end strings. For example, if the method is passed a list containing the elements [redacted], [redacted] as the start String, and [redacted] as the end String, the list's elements should be changed to [redacted]. The [redacted] and [redacted] should be removed because they occur alphabetically between [redacted] and [redacted]. You may assume that the start string alphabetically precedes the ending string.
11. Write a method called [redacted] that accepts an [redacted] of strings and an integer k as parameters and that replaces every string with k copies of that string. For example, if the list stores the values [redacted] before the method is called and k is 4, it should store the values [redacted] after the method finishes executing. If k is 0 or negative, the list should be empty after the call.

12. Write a method called [redacted] that accepts an [redacted] of strings as a parameter and that places a string of four asterisks "[redacted]" in front of every string of length 4. For example, suppose that a variable called [redacted] contains the values [redacted]. The call of [redacted] should change the list to store the values [redacted].
13. Write a method called [redacted] that accepts an [redacted] of integer values as a parameter and reverses each successive sequence of three values in the list. If the list has extra values that are not part of a sequence of three, those values are unchanged. For example, if a list stores values [redacted], after the call the list should store the values [redacted]. The first sequence of three [redacted] has been reversed to be [redacted]. The second sequence [redacted] has been reversed to be [redacted], and so on. Notice that [redacted] and [redacted] are unchanged because they were not part of a sequence of three values.
14. Write a method called [redacted] that accepts an [redacted] of strings as a parameter and removes from each pair of values the shorter string in the pair. If the list is of odd length, the final element is unchanged. For example, suppose that a list contains [redacted]. In the first pair [redacted] the shorter string is "[redacted]". In the second pair [redacted] the shorter string is "[redacted]". In the third pair [redacted] the shorter string is "[redacted]". Your method should remove these shorter strings, changing the list to store [redacted]. If both strings in a

pair have the same length, remove the first string in the pair.

15. Write a method called [redacted] that accepts an [redacted] of integers and two integer values *min* and *max* as parameters and removes all elements whose values are in the range *min* through *max* (inclusive). For example, if a variable called [redacted] stores the values [redacted], the call of [redacted] should remove all values between [redacted] and [redacted], changing the list to store [redacted]. If no elements in range *min-max* are found in the list, or if the list is initially empty, the list's contents are unchanged.
16. Write a method called [redacted] that accepts an [redacted] of strings as a parameter and replaces each pair of strings with a single string that consists of the two original strings in parentheses separated by a space. If the list is of odd length, the final element is unchanged. For example, suppose that a list contains [redacted]. Your method should change the list to store [redacted].
17. Write a method called [redacted] that accepts two [redacted] of integers *a1* and *a2* as parameters and inserts the elements of *a2* into *a1* at alternating indexes. If the lists are of unequal length, the remaining elements of the longer list are left at the end of *a1*. For example, if *a1* stores [redacted] and *a2* stores [redacted], the call of [redacted] should change *a1* to store [redacted]. If *a1* had stored [redacted] and *a2* had stored [redacted], the call of [redacted] would change *a1* to store [redacted].

18. Write a method called [redacted] that accepts an [redacted] of strings as a parameter and produces a mirrored copy of the list as output, with the original values followed by those same values in the opposite order. For example, if an [redacted] variable called [redacted] contains the values [redacted], after a call of [redacted]; it should contain [redacted].
19. Modify the [redacted] class from Chapter 8 so that it defines a natural ordering by implementing the [redacted] interface. Compare the [redacted] by y-major order; that is, points with smaller y-coordinate values should come before those with higher y-coordinate values. Break ties by comparing x-coordinate values.

20. Modify the [redacted] class from Chapter 8 to include a [redacted] method that compares time spans by their length. A time span that represents a shorter amount of time is considered to be “less than” one that represents a longer amount of time. For example, a span of 3 hours and 15 minutes is greater than a span of 1 hour and 40 minutes.
21. Modify the [redacted] class from this chapter to include a year field, and modify its [redacted] method to take years into account when making comparisons. Years take precedence over months, which take precedence over days. For example, July 18, 1995, comes before March 2, 2001.

Programming Projects

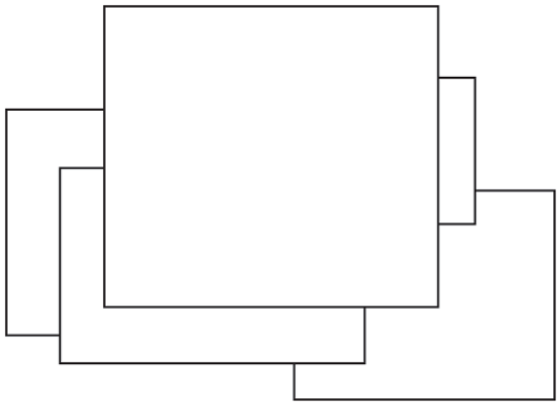
1. Write classes to model a shopping list. Make an [redacted] class that represents a grocery item's name and price, such as tissues for $3. Also implement an [redacted] class that represents a shopper's desire to purchase a given item in a given quantity, such as five boxes of tissues. You might wish to implement bulk-discounted items, such as two boxes of tissues for $4, which would bring the cost of the given item order of 2+2+1 boxes of tissues to $4+$4+$3, or $11.00. Lastly, implement a [redacted] class that stores [redacted]s in an [redacted] and allows item orders to be added to, removed from, or searched for in the cart. The cart should be able to report the total price of all item orders it currently carries.
2. Write a program to reverse the lines of a file and also to reverse the order of the words in each line of the file. Use [redacted]s to help you.
3. Write a family database program. Create a class to represent a person and to store references to the person's mother, father, and any children the person has. Read a file of names to initialize the name and parent–child relationships of each [redacted]. (You might wish to create a file representing your own family tree.) Store the overall list of [redacted]s as an [redacted]. Write an overall main user interface that asks for a name and prints the maternal and paternal family line for that person.

Here's a hypothetical execution of the program, using as an input file the line of English Tudor monarchs:

4. Write a class that models a list of possibly overlapping rectangular two-dimensional window regions, like the windows for the programs open on your computer. The order of the rectangles in the list implies the order in which they would display on the screen (sometimes called the “z-order”), from [redacted] on the bottom to [redacted] on the top.

Each rectangle stores its (x, y) position, width, and height. Your rectangle list class should have a method that takes a [redacted] as a

parameter, treats it as though the user clicked that [redacted] on the screen, and moves the topmost rectangle touching that [redacted] to the front of the list.



5. Extend the ranked choice voting case study to allow for incomplete preferences. Voters are normally allowed to fill out the ballot in an incomplete manner. For example, with the sample ballots with four candidates, you might have ballots that specify only a first choice or only two choices. When you have partial information, you can end up with empty ballots because the voter might have chosen candidates who are all eliminated.

Modify the [redacted] class to have an [redacted] method that indicates that there are no candidates left. (Modify the [redacted] method to return [redacted] in this case.) Then modify the [redacted] program to account for potentially empty ballots. It is possible that the input file will contain blank lines because a voter didn't pick any candidates, so include a test that ignores blank lines in the input file. Modify the code that eliminates candidates so that it removes from the list of ballots any ballots that become empty. That way the percentages will be reported correctly on the next round of tallying.

Chapter 11 Java Collections Framework

11.1 Lists
• Collections
• [redacted] versus [redacted]
• Iterators
• Abstract Data Types (ADTs)
• [redacted] Case Study: [redacted]
11.2 Sets
• Set Concepts
• [redacted] versus [redacted]
• Set Operations
• Set Case Study: [redacted]
11.3 Maps
• Basic Map Operations
• Map Views ([redacted] and [redacted])
• [redacted] versus [redacted]
• Map Case Study: [redacted]
• Collection Overview

Introduction

The previous chapter explored the [redacted] class. An [redacted] is one of many ways to store data in Java. In this chapter we'll explore Java's framework of collections, including lists, sets, and maps. We'll see how to use these structures together to

manipulate and examine data in many ways to solve programming problems. This chapter will examine a trio of smaller interesting programs as case studies rather than presenting a unified case study at the end of the chapter.

We'll introduce a new type of list called a linked list that stores its data differently from an [redacted] *but supports the same operations. We'll also discuss collections called sets that don't allow duplicate elements and that are easy to search. Another collection type we'll explore is the map, which creates associations between pairs of data values. We'll also delve into the notion of abstract data types as a way to separate the capabilities of a collection from the details of its implementation.*

11.1 Lists

The [redacted] class from Chapter 10 has several advantages over an array: It keeps track of its own size for you, it allows you to insert and remove data at arbitrary places in the array, and it resizes itself for you if it gets full.

In this section we'll learn about an object called a [redacted], which is similar to an [redacted]. We'll also look at generalizing collections and discuss a useful object called an iterator that lets you examine the elements of any collection.

Collections

In Chapters 7 and 8, we discussed ways to use arrays and classes to store data. The notion of organizing and structuring data is an important one that helps us solve complex problems. Entities that store and manage data are also called *data structures*. Data structures can be used to implement sophisticated data storage objects called *collections*.

Collection

An object that stores a group of other objects, called its *elements*.

An [redacted] is an example of a collection. A collection uses a data structure internally to store its elements, such as an array or a set of objects that refer to one another. For example, an [redacted] is implemented using an array as its data structure, and a [redacted] (a collection introduced later in this chapter) is implemented using a data structure called a binary search tree.

Collections are categorized by the types of elements they store, the operations they allow you to perform on those elements, and the speed or efficiency of those operations. Here are some examples of collections:

- List: An ordered collection of elements, often accessed by integer indexes or by iteration.
- Stack: A collection in which the last element added is the first one to be removed.
- Queue: A collection in which elements are removed in the same order in which they were added.
- Set: A collection of elements that is guaranteed to contain no duplicates.
- Map: A collection of key/value pairs in which each key is associated with a corresponding value.

Java provides a large group of useful collections that allow you to store, access, search, sort, and manipulate data in a variety of ways. Together, these collections and classes are known as the *Java Collections Framework*. This framework is largely contained in the package [redacted].

The [redacted] package contains an interface called [redacted] that is implemented by all collections except [redacted]. This interface specifies the operations that most collections support. Table 11.1 lists those operations.

Table 11.1 Useful Methods of the [redacted] Interface

[redacted]	
[redacted]	Adds the specified element to this collection
[redacted]	Adds all elements from the given collection to this collection
[redacted]	Removes all elements from this collection
[redacted]	Returns [redacted] if this collection contains the given element
[redacted]	Returns [redacted] if this collection contains all elements of the given collection
[redacted]	Returns [redacted] if this collection contains no elements
[redacted]	Returns an object that can be used to traverse the elements of this collection
[redacted]	Removes one occurrence of the specified element, if it is contained in this collection
[redacted]	Removes all elements of the given collection from this collection
[redacted]	Removes all elements not found in the given collection from this collection

[redacted]	Returns the number of elements in this collection
[redacted]	Returns an array containing the elements of this collection

The [redacted] interface is extended and implemented by the other interfaces and classes in the Java Collections Framework. Figure 11.1 summarizes the various interfaces and the classes that implement them, all of which will be discussed in this chapter.

Figure 11.1 An abridged view of the Java Collections Framework

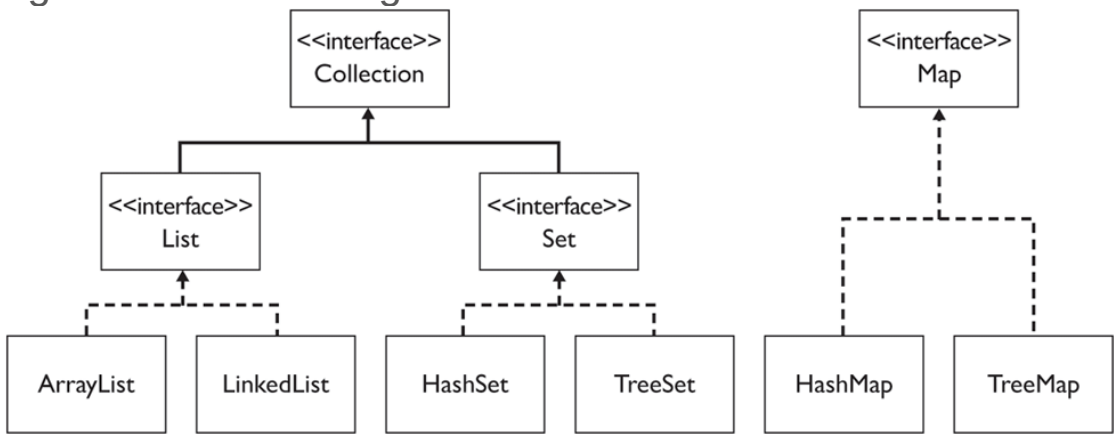


Figure 11.1 Full Alternative Text

[redacted] versus [redacted]

Now we'll look at a collection called [redacted] and compare and contrast it with [redacted].

[redacted] is a powerful and useful collection, but there are some cases in which using an [redacted] isn't ideal. For example, suppose we want to write a program to remove each [redacted] of even length from an [redacted] of [redacted]. We can do this by

using a loop that examines each element of the list and either removes it if its length is even or advances to the next string if its length is odd:

The preceding code is correct, but it doesn’t perform well when the list has a lot of elements. On a relatively modern machine, it can take several minutes to process a list of a million elements. The reason it is so slow is that every time we remove an element

from the list, we have to shift all subsequent elements to the left by one. This repeated shifting results in a slow program.

Another case in which an [redacted] behaves slowly is when it's used to model a waiting line or queue, where elements (customers) are always added to the end of the list (line) and always removed from the front. As customers arrive, they are added to the end of the list. Customers are removed from the front of the list and processed in turn. Removing an element from the front of a large [redacted] is a slow operation because all the other elements have to be shifted to the left.

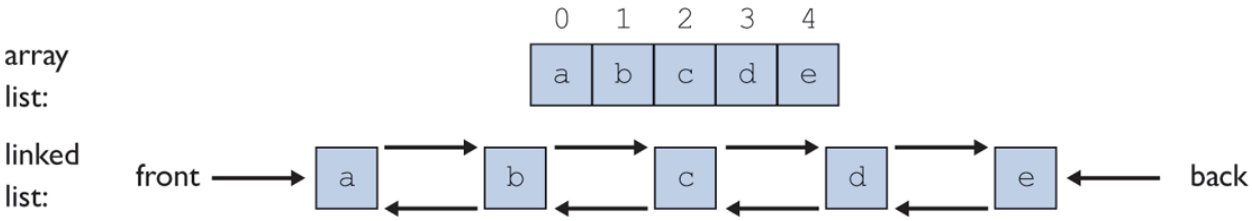
Another type of collection, called a *linked list*, can give better performance in problems like these that involve a lot of additions to or removals from the front or middle of a list. A linked list provides the same operations as an array list, such as [redacted], [redacted], [redacted], [redacted], and [redacted]. But a linked list stores its elements in a fundamentally different way. Elements of a linked list are stored in small individual containers called *nodes*. The nodes are "linked" together, each node storing a reference to the next node in the list. The overall linked list object keeps references to the front and back nodes.

Linked List

[redacted]

A collection that stores a list of elements in small object containers called *nodes*, which are linked together.

You can envision a linked list as an array list that's been "broken apart," with each element then stored in a small box (a node) connected to its neighboring box by an arrow.



11.1-2 Full Alternative Text

One major advantage of using a linked list is that elements can generally be added at the front of the list quickly, because rather than shifting all the elements in an array, the list just creates a new node object and links it with the others in the list. We don't have to do this ourselves; we simply call methods on the list, which takes care of it for us internally. Figure 11.2 shows what happens inside a linked list when an element is added at the front.

Figure 11.2 Adding an element to the front of a linked list

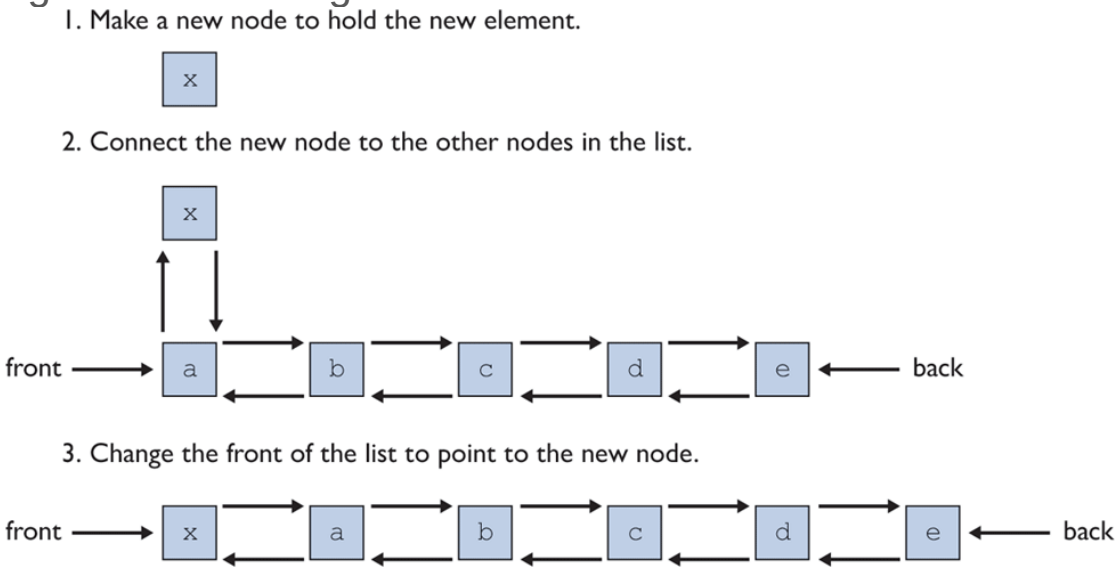


Figure 11.2 Full Alternative Text

To use a linked list in Java, create an object of type [redacted] instead of type [redacted]. A [redacted] object has the same methods you've used when working with [redacted]:

We could write a version of our [redacted] method that accepts a [redacted] as its parameter rather than an [redacted]. However, this change alone won't have much impact on performance. Since the [redacted] method examines each element of the list in sequential order, we can make the code more efficient by employing another type of object called an iterator.

Iterators

As we discussed in Chapter 7, arrays provide a nice property called *random access*, meaning that we can efficiently access or modify arbitrary array elements in any order. This access is possible because arrays are stored as large contiguous blocks of memory, so the computer can quickly compute the memory location of any of the array's elements. Linked lists, unfortunately, do not provide fast random access. A linked list is made up of many small node objects and generally keeps a direct reference

only to one or two particular elements, such as the front and back elements. Therefore it is not possible to quickly access arbitrary elements of the list. A linked list is somewhat like a VHS tape or audio cassette tape in this way; if we wish to access an element, we must “fast-forward” or “rewind” through the list to the proper position.

When you call a method like [redacted], [redacted], [redacted], or [redacted] on a linked list, the code internally creates a temporary reference that begins at the front of the list and traverses the links between nodes until it reaches the desired index. The amount of time that this takes depends on the index you chose: If you ask for the element at index 5 it will be returned quickly, but if you ask for the element at index 9,000 the loop inside the [redacted] method must advance through 9,000 nodes, which will take much longer.

These methods tend to be slow when you use them on a linked list, especially if you call them many times or call them on a list with many elements. Imagine that after the preceding example call of [redacted] you decided to call [redacted]. The linked list doesn’t remember its previous position, so it starts from the front again and advances 9,001 times. The only case in which the methods run quickly is when you pass an index near the front or back of the list.

Earlier in this chapter, we wrote a method to remove strings of even length from an [redacted]. If we adapted this code to use a [redacted] and made no other modifications we would find that it still runs very slowly on large lists, because it calls the [redacted] and [redacted] methods many times:

[redacted]

However, there’s an efficient way to examine every element of a linked list if we want sequential access (i.e., if we want to examine each element in order from the front to the back). To perform this task, we can use a special object called an *iterator* that keeps track of our current position in the list.

Iterator

An object that allows the efficient retrieval of the elements of a list in sequential order.

Using an iterator, when we move from one element to the next, we don’t have to go back to the beginning of the list at each call and follow the links all the way from the front of the list to the desired index. As we’ll see in this chapter, iterators are central to the Java Collections Framework. Every collection provides iterators to access its elements. In other words, there’s a familiar interface for examining the elements of any collection.

An iterator object has the methods listed in Table 11.2. The methods [redacted] and [redacted] should be familiar from the [redacted] class that you have already studied, and their behavior is similar here.

Table 11.2 Methods of Iterator Objects

[redacted]	
[redacted]	Returns [redacted] if there are more elements to be examined
[redacted]	Returns the next element from the list and advances the position of the iterator by one
[redacted]	Removes the element most recently returned by [redacted]

To get an iterator from most collections, such as an [redacted] or a [redacted], you call the method [redacted] on the list, which returns an object of type [redacted] for examining that list’s elements. (You don’t use the new keyword.) Generally, a variable named [redacted] storing elements of type [redacted] uses an iterator in the following way:

The example of removing strings that have even length from a collection can be implemented much more efficiently using an iterator:

```
Iterator<String> i = list.iterator();

(i.hasNext())

i.next()

i.remove();
```

Whereas the original [redacted] version required up to several minutes to process a list of one million elements on a modern computer, this new code finishes a million-element list in under one-tenth of a second. It performs so quickly because the iterator retains the current position in the list between calls to get or remove elements.

Iterators are also used internally by Java's for-each loop. When you use a for-each loop like the following, Java is actually accessing the elements using an iterator under the hood:

COMMON PROGRAMMING ERROR

Calling [redacted] on an Iterator Too Many Times

Iterators can be a bit confusing to new programmers, so you have to be careful to use them correctly. The following code attempts to use an iterator to find and return the longest string in a linked list, but it has a bug:

The problem with the previous code is that its loop calls the method on the iterator in two places: once when it tests the length of the string, and again when it tries to store the string as the

longest. Each time you call , the iterator advances by one position, so if it's called twice in the loop, you'll skip an element when you find a match. For example, if the list contains the elements (, , , ,), the program might see the and intend to store it, but the second call to would actually cause it to store the element that follows , namely, .

The solution is to save the result of the call into a variable. The following code would replace the loop in the previous code:

```
String current = itr.next();

longest = current;
```

As the compiler processes the for-each loop, it essentially converts the loop into the following code:

There’s a more advanced version of [redacted] called [redacted] that works only on lists. A [redacted] provides operations like adding elements, setting element values, and reversing iteration from back to front. Because it is more complex, we won’t discuss [redacted] in detail in this book. You can read more about it online in the Java API and Java Tutorial pages.

In summary, the following table compares the major benefits of [redacted] and [redacted]:

[redacted]	• Random access: any element can be accessed quickly • Adding and removing at the end of the list is fast
[redacted]	• Adding and removing at either the beginning or end of the list is fast • Adding and removing during a sequential access with an iterator is fast

	• Unlike with arrays, there is no need to expand when full • Can be more easily used as a queue than arrays can

Java 8 has introduced some powerful new features for processing lists and other collections. The following Java 8 code would produce a new list that does not contain any even-length strings:

You can read more about this approach in Chapter 19.

Abstract Data Types (ADTs)

It's no accident that the [redacted] collection provides the same methods as the [redacted]. They both implement the same kind of collection: a *list*. At a high level, the most important thing isn't the way the list is implemented internally, but the operations we

can perform on it. This set of operations is an example of an *abstract data type*, or ADT.

Abstract Data Type (ADT)

A specification of a type of data and the operations that can be performed on it.

An ADT specifies operations that can be performed on data without specifying exactly how those operations are implemented. Linked lists and array lists are both examples of the list ADT because they both provide the same operations, such as storing data by index, adding and removing data at particular indexes, and so on.

In Java, ADTs are specified by interfaces. Each ADT's operations are specified by the methods of its interface. For example, both and implement an interface in the package called . The interface declares all the common methods that both types of lists implement.

It's a good practice to declare any variables and parameters of a collection type using the appropriate interface type for that ADT rather than the actual class's type. For example, the following code constructs a object but stores it in a variable of type :

```
List<Integer>
```

Joshua Bloch, one of the authors of the Java Collections Framework, says that this programming practice is “strongly recommended” because “it gives you the flexibility to change implementations.” Note that you cannot create an object of type [redacted], but you can use [redacted] as the type of a variable.

You can also use the interface types for ADTs like [redacted] when you declare parameters, return types, or fields. Doing so is useful when you’re writing a method that accepts a collection as a parameter, because it means that the method will be able to operate successfully on any collection that implements that ADT’s interface. For example, the following method can accept a [redacted] or an [redacted] as its actual parameter:

```
List<String> list
```

It works with either type of list and is efficient for both. This flexibility is another benefit of polymorphism (discussed in Chapter 9). In fact, you could make the method even more general by having it accept a parameter of type [redacted] rather than [redacted], since every collection has an [redacted] method.

The [redacted] package has a class called [redacted] that contains several useful methods related to all collections. (Note that this class is not the same as the [redacted] interface that many collection classes implement.) The [redacted] class contains static methods that operate on lists. These methods' headers specify parameters of type [redacted] rather than [redacted] or [redacted]. The methods perform common tasks on lists, such as sorting, shuffling, and searching. Table 11.3 presents a short list of useful methods from the [redacted] class that operate on lists.

Table 11.3 Useful Static Methods of the [redacted] Class

[redacted]	
[redacted]	Searches a sorted list for a given element value and returns its index
[redacted]	Copies all elements from the source list to the destination list
[redacted]	Replaces every element in the given list with the given value
[redacted]	Returns count of occurrences of value in the collection
[redacted]	Returns the element with the highest value
[redacted]	Returns the element with the lowest value
[redacted]	Replaces all occurrences of the old value with the new value
[redacted]	Reverses the order of the elements in the given list
[redacted]	Shifts each element to the right by the given number of indexes, moving the final element to the front
[redacted]	Rearranges the elements into random order
[redacted]	Rearranges the elements into sorted (nondecreasing) order
[redacted]	Switches the element values at the given two indexes

Notice that these methods are static, so they must be called by writing the word followed by a dot and the method's name. For example, if you had a variable called and you wanted to reverse the list's contents, you'd write

```

```

Since Java 8, the interface contains some useful static methods for creating and manipulating lists. Perhaps the most useful is the method, which takes element values as parameters and returns a fixed-length list containing those elements, similar to the method shown in Chapter 10. This is a nice syntax for quickly creating a list of elements. The following JShell interaction creates such a list:

```
jshell>




jshell>
```

The list returned by [redacted] is a special list that cannot be changed. Your program will crash if you try to modify the list:

```
jshell>
```

If you want a modifiable version of such a list, create a regular [redacted] or [redacted] and pass the result of [redacted] as the parameter to its constructor. Doing so will make a copy of the result of [redacted] that is able to be changed, as shown in the following JShell interaction:

```
jshell>



...>




jshell>
```

```
jshell>
```

There are similar methods [redacted] and [redacted] for the [redacted] and [redacted] collections we'll see later in this chapter.

In addition to [redacted], there are several other interfaces, such as [redacted], [redacted], and [redacted], representing ADTs in the Java Collections Framework. We'll explore several of them in this chapter.

[redacted] Case Study: [redacted]

Consider the task of finding all prime numbers up to a given maximum. Prime numbers are integers that have no factors other than 1 and themselves. The number 2 is the smallest prime number.

To build a list of prime numbers, you could just write a brute-force solution using [redacted] loops:

```
for (each number from 2 to maximum) {

if (number is prime) {

add number to list of prime numbers.

}

}
```

But you would need a way to figure out whether each number is prime. One option would be to write another [redacted] loop that tested all lower integers to see whether they were factors of that number. However, there's an easier way.

The *sieve of Eratosthenes*, named for the Ancient Greek mathematician who devised it, is a classic algorithm for finding prime numbers. The sieve algorithm starts by creating two lists of numbers: one list of numbers to process (some of which may be prime), and another list of numbers known to be prime. Initially, the list of numbers to process can contain every number from 2 to the maximum, while the list of primes will be empty. Here are the initial two lists for a maximum of 25:

The sieve algorithm begins by removing the first element from the [redacted] list and adding it to the [redacted] list. This number is found to be prime because of the nature of the algorithm. Next, the algorithm filters out all the other elements from the [redacted] list that are multiples of this prime number. On the first pass of the algorithm, for example, the program selects 2 from the [redacted] list, places it into the [redacted] list, and removes all multiples of 2 from the [redacted] list. Now the number at the front of the [redacted] list is 3. This number will be placed into the [redacted] list during the next pass of the algorithm, and all the multiples of 3 that appear in the [redacted] list will be removed.

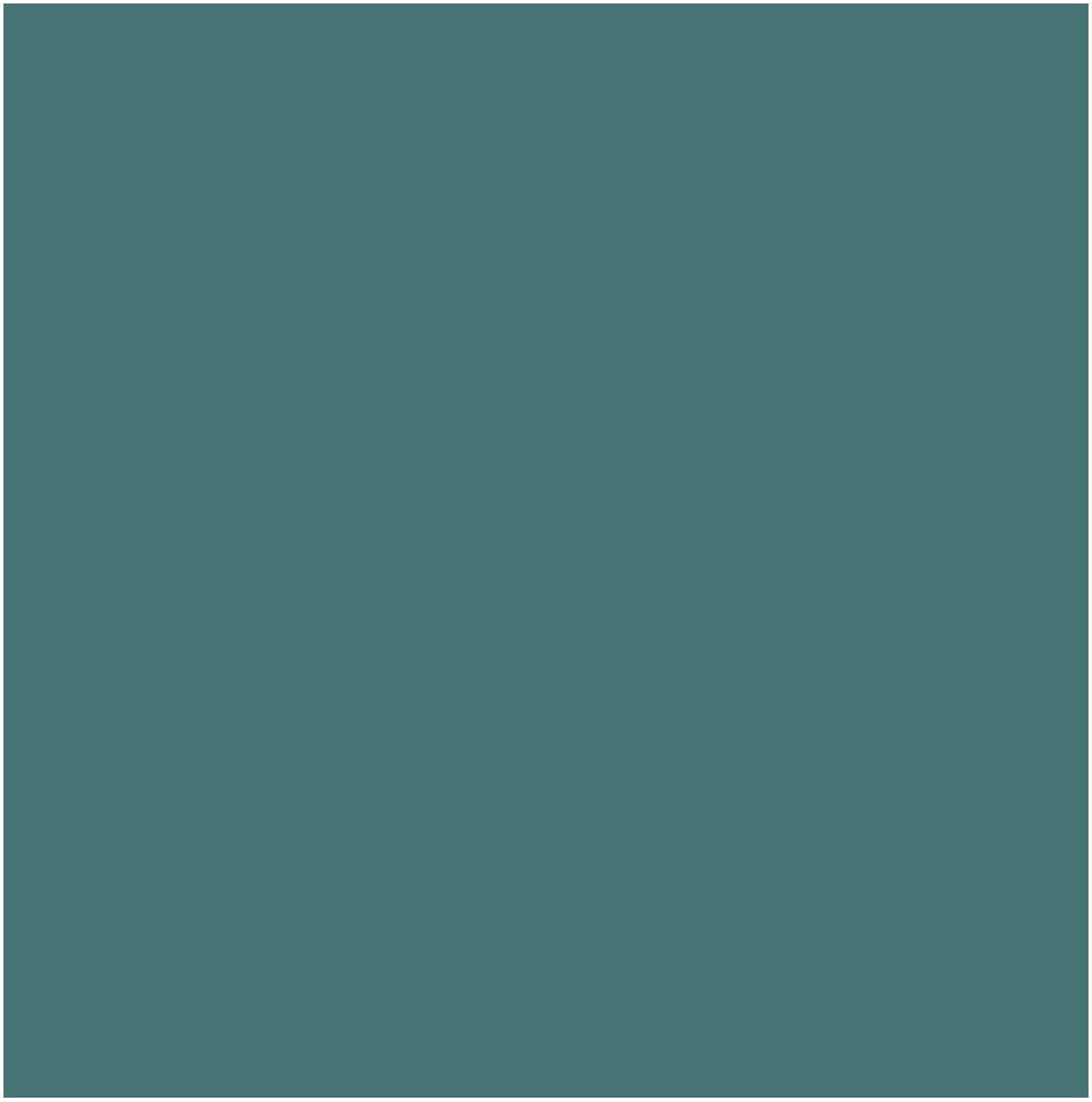
The numbers taken from the front of the [redacted] list are guaranteed to be prime. A nonprime number cannot reach the front of the [redacted] list because every nonprime number must be a multiple of some prime number, and any such multiples will have been removed by a previous pass of the algorithm.

Here are the two lists after the first three passes of the algorithm:

Now let’s implement the sieve algorithm. We’ll use [redacted] to represent the lists of numbers and primes. This tool is preferable to [redacted] because, as we discussed previously, removing elements from the front of an [redacted] is inefficient.

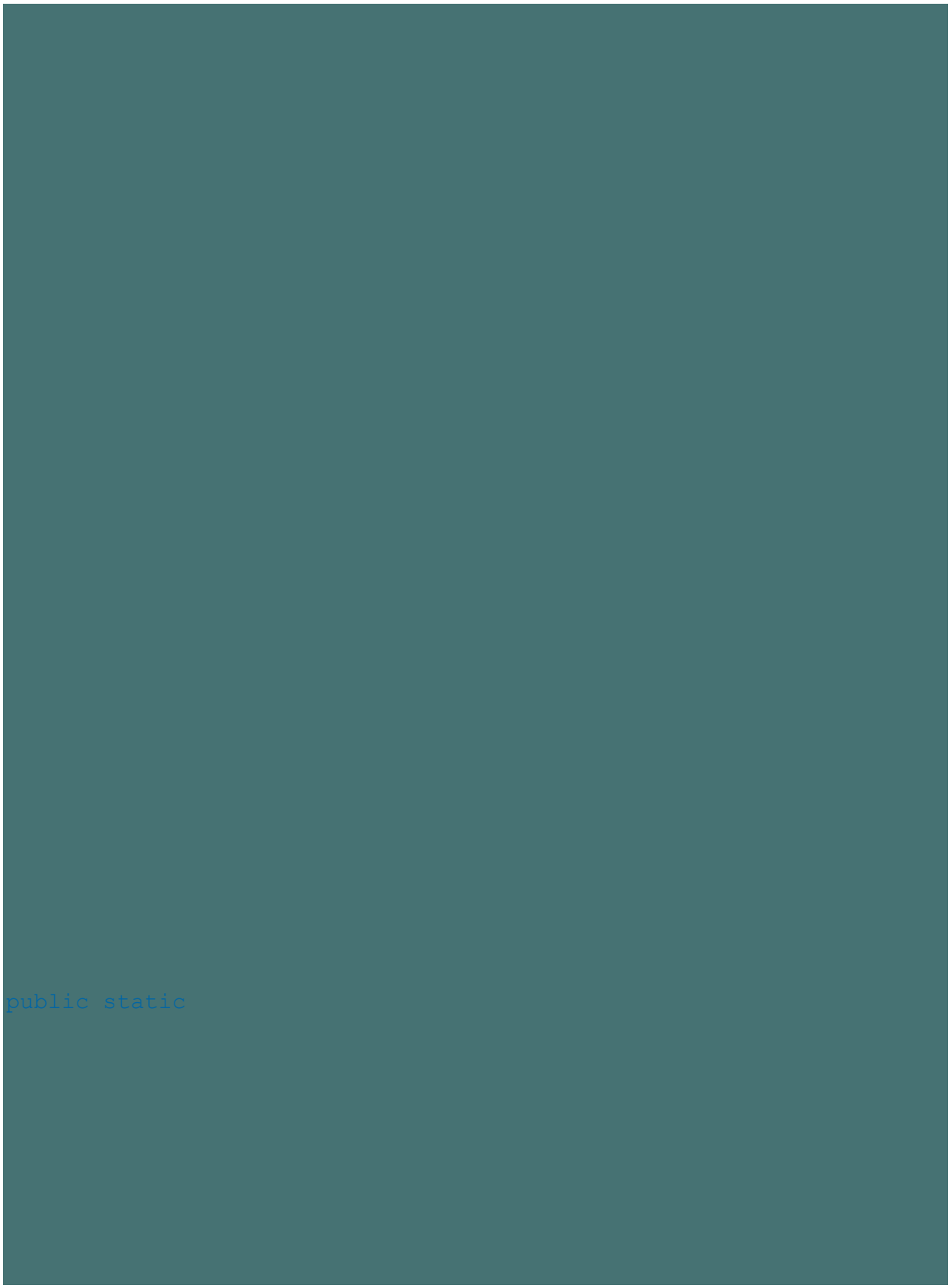
First we’ll create an empty list of primes and a list of all the numbers up to the given maximum. Since we’ve discussed ADTs and the [redacted] interface, we’ll declare our variables as the ADT interface type [redacted]:

Next, we’ll process the list of numbers. We’ll use an iterator to make passes over the [redacted] list and remove elements that are multiples of the [redacted] element:



The lines of code that follow are the complete program. The most significant addition is a [redacted] method that prompts the user for the maximum number:

```
import


public class


public static void
```


public static

```
new


for

int


while


int
```

```
while
```

```
int
```

```
if
```

```
return
```

The following is a sample log of execution of the program:

Our version of the sieve algorithm has been simplified. The real algorithm stops when the first element of the [redacted] list is greater than the square root of the maximum, because any number this large that remains in the list cannot have any multiples remaining in the list. For example, when the maximum is 25, once the item at the front of the [redacted] list exceeds 5, all remaining numbers in the list are known to be prime and can be placed in the [redacted] list.

The algorithm can be improved in other ways. For example, the initial list of numbers doesn’t need to store every integer from 2 through the maximum. It can instead store 2 and each odd integer up to the maximum, because no other even numbers are prime. This modification makes the algorithm more efficient because fewer numbers need to be processed. These two improvements to the algorithm are left as an exercise.

11.2 Sets

A major limitation of both linked and array lists is that searching them takes a long time. Generally, if you want to search a list, you have to look at each element sequentially to see whether you've found the target. This can take a long time for a large list.

Another limitation of lists is that it's not easy to prevent a list from storing duplicate values. In many cases this isn't a problem, but if, for example, you are storing a collection to count the number of unique words in a book, you don't want any duplicates to exist. To prevent duplicates in a list, you have to sequentially search the list every time you add to it, in order to make sure you aren't adding a word that's already present.

When you want to maintain a collection of elements that can be searched quickly and that prevents duplicates, you're better off using another abstract data type called a *set*.

Set

A collection that cannot contain duplicates.

The [redacted] collection is very much like the mathematical notion of a set. Sets do not support all the operations you can perform on lists (namely, any operation that requires an index), but they do offer the benefits of fast searching and effortless elimination of duplicates.

Set Concepts

The two primary implementations of the Java Collections Framework's [redacted] interface are called [redacted] and [redacted]. [redacted] is the general-purpose set class, whereas [redacted] offers a few advantages that will be discussed later. If you wanted to store a set of [redacted] values, you could write code like the following:

After the code executes, the set will contain only four elements, because [redacted] will be placed into the set only once. Notice that, as with lists, you can declare your collection variable to be of the interface type rather than the class type (type [redacted] rather than type [redacted]).

A [redacted] provides all of the operations from the [redacted] interface introduced earlier in this chapter, such as [redacted], [redacted], and [redacted]. It's generally assumed that the [redacted] performs these operations efficiently, so you can add many elements to a [redacted] and search it many times without experiencing poor performance. A [redacted] also provides a [redacted] method that lets you see its elements. Printing the preceding [redacted] set would produce the following output:

[redacted]

One of the most important benefits of using a [redacted] such as [redacted] is that it can be searched incredibly quickly. Recall that the [redacted] method of an [redacted] or [redacted] must examine every element of the list in order until it finds the target value. By contrast, the [redacted] method of a [redacted] is implemented in such a way that it often needs to examine just one element, making it a much more efficient operation.

A [redacted] is implemented using a special internal array called a *hash table* that places elements into specific positions based upon integers called *hash codes*. (Every Java object has a hash code that can be accessed through its [redacted] method.) You don't need to understand the details of [redacted]'s implementation to use it—the bottom line is that it's implemented in such a way

that you can add, remove, and search for elements very quickly. We will discuss hash set implementations in detail in Chapter 18.

One drawback of [redacted] is that it stores its elements in an unpredictable order. The elements of the [redacted] set were not alphabetized, nor did they match the order in which they were inserted. This storage practice is a tradeoff for the [redacted]'s fast performance. (Java now includes a variation called [redacted] that is almost as fast as HashSet but stores its elements in the order they were inserted.)

Sets allow you to examine lots of data while ignoring duplicates. For example, if you wanted to see how many unique words appear in the book *Moby-Dick*, you could write code such as the following:

This code produces the following output when run on the text of *Moby-Dick* (available from http://www.gutenberg.org):

The [redacted] class has a convenient constructor that accepts another collection as a parameter and puts all the unique elements from that collection into the [redacted]. One clever usage of this constructor is to find out whether a [redacted] contains any duplicates. To do so, simply construct a [redacted] from the list and see whether the sizes differ:

One drawback of a [redacted] is that it doesn't store elements by indexes. The following loop doesn't compile on a [redacted], because it doesn't have a [redacted] method:

Instead, if you want to loop over the elements of a , you must use an iterator. Like other collections, a has an method that creates an object to examine its elements. You can then use the familiar loop to examine each element:

A shorter alternative to the preceding code is to use a for-each loop over the elements of the . As mentioned previously, the code behaves the same way but is easier to write and read than the version with the loop:

versus

The examples in the preceding section used , but there's another class called that also implements the interface. A uses an internal linked data structure called a *binary search tree* to store its elements in sorted order. We will discuss binary tree implementations in detail in Chapter 17. A is efficient for adding, removing, and searching, though it is a bit slower than a .

A can be useful if you want to print the set and have the output ordered. For example, the following code displays the sorted set of all three-letter words in *Moby Dick* that start with "a":

The code produces the following output:

A [redacted] can be used with data that has a natural ordering. This means that it will work if its elements are of any type that implements the [redacted] interface, such as [redacted] or [redacted]. You can also provide your own object that specifies how to compare elements, called a *comparator*. Comparators will be discussed in Chapter 13 when we cover searching and sorting.

You should not try to construct a [redacted] of objects without a natural ordering, such as [redacted] objects:

The preceding code compiles (unfortunately), but it generates an exception when you run it because it doesn't know how to order the [redacted] objects in the [redacted]:

You'd be better off using a [redacted] in this case.

In summary, the following are some of the major differences between [redacted] and [redacted]:

[redacted]	
[redacted]	• Extremely fast performance for add, contains, and remove tasks • Can be used with any type of objects as its elements

[redacted]	• Elements are stored in sorted order • Must be used with elements that can be compared (such as [redacted], [redacted])

Set Operations

Consider the task of figuring out how many unique elements appear in two given sets. You cannot just add the sets' sizes, since they might have some elements in common that should not be counted twice in your total. Instead, you could count all elements from the first set and then count only the unique elements of the second, by checking to see whether each element from the second is also in the first:

However, a more elegant way to perform this calculation is to compute a *union* between the sets. The union of two sets A and B is the set of all elements contained in either A, B, or both. Union is an example of a *set operation*. Other examples of set operations are *intersection* (the set of all elements that are in both A and B) and *difference* (the set of all elements that are in A but not in B). Many set operations combine two sets to produce a new set as their result.

Set operations are often depicted by drawings called Venn diagrams, which depict sets as circles and set operations as shaded overlapping between the circles. Figure 11.3 shows some examples.

Figure 11.3 Set operation Venn diagrams

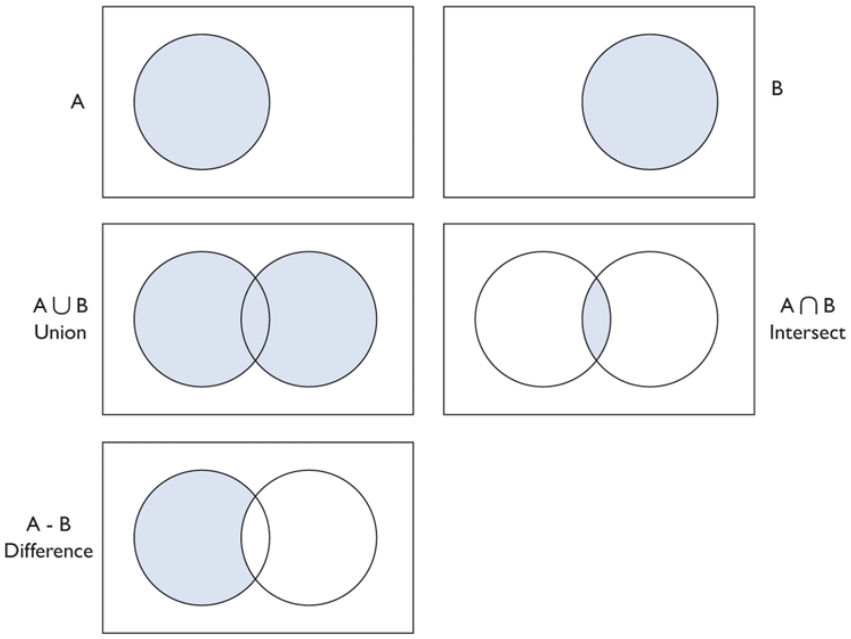


Figure 11.3 Full Alternative Text

You can write code to perform set operations by calling methods whose names end with “All” using the relevant pair of sets. Table 11.4 summarizes which methods correspond to which set operations.

Table 11.4 Common Set Operations, Given Two Sets A and B

union	[redacted]	Set of all elements that are in A, B, or both
intersection	[redacted]	Set of all elements that are in both A and B
difference	[redacted]	Set of all elements that are in A but not in B
superset, subset	[redacted]	Returns [redacted] if A is a superset of (contains all elements of) B

For example, we could rewrite the [redacted] code to use a union with the [redacted] method:

It’s important to note that the set operations in Java modify the existing sets on which you call them, rather than creating new sets for you. Notice that in the preceding code, we initialize a new [redacted] that contains all the elements from [redacted] and then add the contents of [redacted] to the new set, rather than combining [redacted] and [redacted] directly. We do this because the caller might not want us to disturb the sets’ original contents.

Set Case Study: [redacted]

Consider the task of writing a lottery program. The program should randomly generate a winning lottery ticket, then prompt the

player to enter lotto numbers. Depending on how many numbers match, the player wins various prizes.

Sets make excellent collections for storing the winning lotto numbers and the player's numbers. They prevent duplicates, and they allow us to efficiently test whether a number in one set exists in the other. These features will help us to count the number of winning numbers the player has entered.

The following code uses a [redacted] object to initialize a set of six winning lottery numbers between 1 and 40. The code uses a [redacted] loop because the same number might be randomly generated more than once:

Once the program has generated the winning number set, we'll read the player's lottery numbers into a second set. To figure out how many numbers the player has chosen correctly, we could search the winning number set to see whether it contains each number from the ticket. However, a more elegant way to perform this test is to determine the intersection between the winning

numbers set and the player's ticket set. The following code creates the intersection of the player's ticket and the winning numbers by copying the ticket and then removing any elements from it that aren't winning numbers:

Once we have the intersection, we can ask for its size to see how many of the player's numbers were winning numbers; we can then calculate the appropriate cash prize amount for the player on the basis of that number. (Our version starts with a $100 prize and doubles that figure for each winning number.)

Here is a complete implementation of the lottery program. We've created a few static methods for structure and added a few constants to represent the number of numbers, maximum number, and lotto prize amounts:

```
import


public class


public static final int


public static final int


public static final int


public static void
```

new

if

```
double
```

```
public static
```

```
new
```

```
while
```

```
int

return

public static

new

new
```

```
while

int

return
```

Here’s one example output from running the program:

11.3 Maps

Consider the task of writing a telephone book program that allows users to type a person's name and search for that person's phone number. You could store the data in an array, a list, or a set. Perhaps you'd make a small class called [redacted] that stores a person's name and phone number and a list to contain the [redacted] objects. When you want to search for a phone number, you'd traverse the list, looking for the [redacted] that matches the name entered by the user, and return the associated phone number.

A solution such as the one just described isn't very practical. If the list of records is large, it would take the program a long time to look at each one to find the right record to retrieve the phone number.

In many data-processing tasks, it's useful to link pairs of objects (such as a name and a telephone number). We often find ourselves saying, "I'd like to associate every A with a B." Perhaps we'd like to associate names with addresses so that when the user types a name, we can quickly look up that person's address. Or perhaps we want to count the number of occurrences of every word in a book by associating each word with its count of occurrences.

The abstract data type *map* describes a collection that allows you to create one-way associations between pairs of objects to solve these kinds of problems.

Map

A collection that associates objects called keys with objects called values.

Maps can be used to solve a surprisingly large number of problems. A map can group all the words in a book by length and report how many words there are of each length. Maps can associate chat users with their set of friends and buddies. Maps can even represent a family tree associating each person with his or her mother and father.

A map associates *keys* with *values*. You can store a key/value pair into a map; later in your code, if you supply just the key to the map, it will give you back the value associated with that key. A key can map to only one value, but it's possible for multiple keys to map to the same value. The Java Collections Framework includes an interface called representing this ADT.

You can think of a map as a pair of connected collections: a set of keys and a collection of values associated with those keys. Figure 11.4 is an example of this idea that maps first names to last names.

Figure 11.4 Mapping keys (first names) to values (last names)

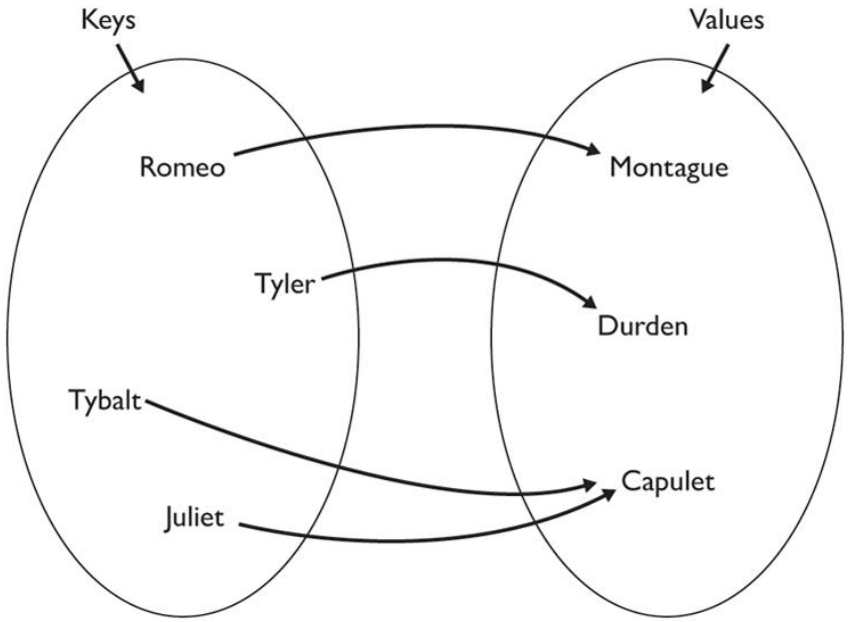


Figure 11.4 Full Alternative Text

Basic Map Operations

The two primary classes that implement the [redacted] interface are called [redacted] and [redacted]. These class names parallel the ones used in the set collections, because the maps and sets have similar internal implementations. [redacted] is the more general-purpose map; a [redacted] stores comparable keys in sorted order.

A [redacted] is constructed with not one but two generic type parameters, separated by a comma. The first type parameter represents the type of the keys, and the second represents the type of the values. The inclusion of two parameters makes the declaration line in your code lengthy. The line of code that follows is an example of constructing a salary map that associates people's names with their salaries. Notice that we have to use the wrapper type [redacted] rather than the primitive type [redacted]:

Notice the use of the diamond operator as discussed in Chapter 10. The operator will infer the type of the keys and values from the two types given on the left side of the statement.

Key/value pairings are added to a map using its [redacted] method, which is roughly similar to the [redacted] method of most other collections. The [redacted] method accepts a key and a value as parameters and stores a mapping between the key and value in the map. If the key was previously associated with some other value, the new association replaces the old one. We can add key/value pairs to our salary map using code like the following:

Once you've added key/value pairs to a map, you can look up a value later by calling the map's [redacted] method, which accepts a key as a parameter and returns the value associated with that key:

To see whether a map contains a mapping for a given key, you can use the method, or you can call the method and test for a result:

```
salaryMap.containsKey(name)
```

Table 11.5 lists several useful methods.

Table 11.5 Useful Methods of Maps

[redacted]	Removes all keys and values from a map
[redacted]	Returns [redacted] if the given key maps to some value in this map
[redacted]	Returns [redacted] if some key maps to the given value in this map
[redacted]	Returns the value associated with this key, or [redacted] if not found
[redacted]	Returns the value associated with this key, or the given default value if not found
[redacted]	Returns [redacted] if this collection contains no keys or values
[redacted]	Returns a [redacted] of all keys in this map
[redacted]	Associates the given key with the given value
[redacted]	Adds all key/value mappings from the given map to this map
[redacted]	Adds a key/value pair if this key was not present
[redacted]	Removes the given key and its associated value from this map
[redacted]	Updates a key/value pair only if the key was present
[redacted]	Returns the number of key/value mappings in this map

[redacted]	Returns a [redacted] of all values in this map

A [redacted]'s [redacted] method displays a comma-separated list of its key/value pairs. The order in which the keys appear depends on the type of map used, which we'll discuss in a moment. Here's what the salary map declared previously would look like when the program prints it:

[redacted]

Map Views ([redacted] and [redacted])

Unlike most collections, a map doesn't have an [redacted] method, because it wouldn't be clear what you wanted to examine. The keys? The values? Both? Instead, maps have a pair of methods called [redacted] and [redacted] that respectively return a [redacted] of all keys in the map and a [redacted] of all values in the map. These are sometimes called *collection views* of a map because each is a collection that exists conceptually within the map.

For example, consider a map that associates people's social security numbers with their names. In other words, the map's keys are nine-digit social security numbers and its values are names. We could create the map with the following code:

If we wanted to write a loop that printed the social security numbers of every person in the map, we could then call the [redacted] method on the map. This method returns a [redacted] containing every key from the hash table—in this case, every [redacted] for a person's number. If you store the [redacted] in a variable, you should declare that variable as type [redacted], with the type of the map's keys between the [redacted] and [redacted] brackets:

The preceding code would produce the following output (the keys are in an unpredictable order since a [redacted] is used):

If we instead wanted to loop over every name (every value) stored in the map, we'd call the [redacted] method on the map. The [redacted] method returns a reference of type [redacted], not of type [redacted], because the values may contain duplicates (it's legal for two keys to map to the same value). If you store the [redacted] result in a variable, you should declare that variable as type [redacted], with the type of the map's values between the [redacted] and [redacted]:

The preceding code would produce output such as the following:

You can combine the keys and the values by looping over the keys and then getting the value for each key:

Notice that this code doesn’t declare a variable to store the key set, but instead calls [redacted] directly in the for-each loop. The code produces the following output:

A related method called [redacted] returns objects of a type called [redacted] that represents key/value pairs, but we won’t explore this method here.

versus

Just as there are two set implementations, and , there are two flavors of collections in Java: and . A performs a bit faster than a and can store any type of data, but it keeps its keys in a somewhat haphazard order. A can store only comparable data and performs a bit slower, but it keeps its keys in sorted order.

We could store the social security number information from the previous section in a as follows:

```
TreeMap
```

The keys would be ordered differently, leading to the following output when printing the map:

Notice that the social security numbers (the map's keys) are sorted in numeric order. This sorting can be useful for certain applications, but [redacted] is still recommended for general use over [redacted]. Many applications don't care about the order of the keys and benefit from the better performance of [redacted]. [redacted] works even on data that does not have a natural ordering.

Map Case Study: [redacted]

In an earlier example, we counted the number of unique words in the book *Moby Dick*. What if we wanted to find the words that occur most frequently in the book? To do this, we should count how many times each word in the book occurs, then examine all of those counts and print the ones with the largest values.

Maps are very useful for solving these kinds of problems. We can create a word-count map in which each key is a word and its associated value is the number of occurrences of that word in the book:

```
wordCountMap = empty.

for (each word from file) {
```

```
if (I have never seen this word before) {

set this word's count to 1.

} else {

increase this word's count by one.

}

}
```

We'll need a [redacted] to read the appropriate file and a [redacted] to store the word counts. We'll use a [redacted] so that the frequently used words will be shown in alphabetical order:

We can now read the file's contents and store each word that the program encounters in the map. If we come across a word that we have already seen, we retrieve its old [redacted] value, increment it by 1, and put the new value back into the map. Recall that when you put a key/value mapping into a map that already contains that

key, the old mapping is replaced. For example, if the word [redacted] was mapped to the number 25 and we put in a new mapping from [redacted] to 26, the old mapping from [redacted] to 25 would be replaced; we don't have to remove it manually. Here's the code to build up the map:

Once we've built the word-count map, if we want to print all words that appear more than, say, 2,000 times in the book, we can write code like the following:

Here’s the complete program, with a method added for structure and a constant added for the number of occurrences that are needed for a word to be considered among the most frequent:

```
import

import
```

```
public class


public static final int


public static void


throws
```

```
new

for

int

if
```

```
public static

new

while

if
```

```
int

else

return
```

The program produces the following output for *Moby-Dick*:

Collection Overview

We've discussed three major abstract data types in this chapter: lists, sets, and maps. It's important to understand the differences between them and to know when each should be used. Table 11.6 summarizes the different types of collections and their pros and cons.

Table 11.6 Comparison of Lists, Sets, and Maps

[redacted]	[redacted]	A sequence of elements arranged in order of insertion	Slow to search, slow to add/remove arbitrary elements	List of accounts; prime numbers; the lines of a file
[redacted]	[redacted]	A set of unique elements that can be searched quickly	Does not have indexes; user cannot retrieve arbitrary elements	Unique words in a book; lottery ticket numbers
[redacted]	[redacted]	A group of associations between pairs of “key” and “value” objects	Not a general-purpose collection; cannot easily map backward from a value to its key	Word counting; phone book creation

When you approach a new programming problem involving data, you can ask yourself questions to decide what type of collection is most appropriate. Here are some examples:

- What are you going to do with the data? Do you intend to add and remove many elements, search the data many times, or connect this data to other data?
- Do you plan to search the data? If so, how? If you want to examine elements in order of insertion, you want a list. If you intend to search for arbitrary elements, a set may be better. If you need to find an object, given partial information about it (for example, a PIN or ID number, to find a user’s bank account object), a map may be best.
- In what order should the elements be stored? Lists hold their elements in order of insertion, whereas the tree collections ([redacted] and [redacted]) order elements by

their natural ordering. If order doesn’t matter, you may want a hash table collection such as a [redacted].

Chapter Summary

A collection is an object that stores a group of other objects. Examples of collections are [redacted], [redacted], and [redacted]. Collections are used to structure, organize, and search data.

A linked list is a collection that's similar to an [redacted] but that is implemented internally by storing each element in a small container object called a node. Linked lists can perform certain operations faster than array lists, such as adding data to and removing data from the front or middle of the list.

An iterator is an object that keeps track of the current position in a list and expedites the examination of its elements in sequential order. Linked lists are often used with iterators for increased efficiency.

An abstract data type (ADT) is a specification of the operations that can be performed on data. Two examples of ADTs are [redacted] and [redacted]. ADTs in the Java Collections Framework are represented as interfaces (e.g., the [redacted] interface, which is implemented both by [redacted] and by [redacted]).

A set is a collection that doesn't allow duplicates. Sets generally can be searched very quickly to see whether they contain a particular element value. The [redacted] interface represents sets.

There are two major set classes in Java: [redacted] and [redacted]. A [redacted] holds [redacted] data in a sorted order; a [redacted] can hold any data and can be searched faster, but its elements are stored in an unpredictable order.

A map is a collection that associates key objects with value objects. Maps are used to create relationships of association between pieces of data, such as a person's name and phone number.

There are two major map classes in Java: [redacted] and [redacted]. A [redacted] holds [redacted] keys in a sorted order; a [redacted] can hold any data as its keys and performs value lookups faster, but its keys are stored in an unpredictable order.

Self-Check Problems

Section 11.1: Lists

1. When should you use a [redacted] instead of an [redacted]?
2. Would a [redacted] or an [redacted] perform better when run on the following code? Why?

3. What is an iterator? Why are iterators often used with linked lists?
4. Write a piece of code that counts the number of duplicate elements in a linked list, that is, the number of elements whose values are repeated at an earlier index in the list. Assume that all duplicates in the list occur consecutively. For example, the list [`1, 1, 3, 5, 5, 5, 5, 7, 7, 11`]

contains five duplicates: one duplicate of element value [redacted], three duplicates of element value [redacted], and one duplicate of element value [redacted].

5. Write a piece of code that inserts a [redacted] into an ordered linked list of [redacted], maintaining sorted order. For example, for the list [redacted], inserting [redacted] in order would produce the list [redacted].
6. Write a method called [redacted] that accepts a linked list of integers as a parameter and removes all occurrences of a particular value. You must preserve the original relative order of the remaining elements of the list. For example, the call [redacted] would change the list [redacted] to [redacted].
7. Write a method called [redacted] that accepts a linked list of integers as a parameter and moves the first half of the list to the back of the list. If the list contains an odd number of elements, the smaller half is wrapped (in other words, for a list of size N, the middle element, N/2, becomes the first element in all cases). For example, calling [redacted] on the list [redacted], would change that list into [redacted]. For the list [redacted], the result would be [redacted].
8. What is an abstract data type (ADT)? What ADT does a linked list implement?
9. Self-Check Problem 4 asked you to write code that would count the duplicates in a linked list. Rewrite your code as a method called [redacted] that will allow either an [redacted] or a [redacted] to be passed as the parameter.

Section 11.2: Sets

10. A [redacted] has every method that a [redacted] has, and more. So why would you use a [redacted] rather than a [redacted]?
11. When should you use a [redacted], and when should you use a [redacted]?
12. A [redacted] doesn't have the [redacted] and [redacted] methods that an [redacted] has. How do you examine every element of a [redacted]?
13. What elements are contained in the following set after this code executes?

14. How do you perform a union operation on two sets? An intersection? Try to give an answer that doesn't require any loops.
15. Write the output produced when the following method is passed each of the following lists:

a.

b. [redacted]
c. [redacted]

Section 11.3: Maps

16. Write the code to declare a [redacted] that associates people's names with their ages. Add mappings for your own name and age, as well as those of a few friends or relatives.
17. A [redacted] doesn't have the [redacted] and [redacted] methods that an [redacted] has. It doesn't even have an iterator method like a [redacted] does, nor can you use a for-each loop on it directly. How do you examine every key (or every value) of a [redacted]?
18. What keys and values are contained in the following map after this code executes?

19. Write the output produced when the following method is passed each of the following maps:

[redacted]

a. [redacted]

b. [redacted]

c. [redacted]

d. [redacted]

20. Write the output produced when the following method is passed each of the following maps:

[redacted]

[redacted]

a. [redacted]

b. [redacted]

c. [redacted]

d. [redacted]

21. Write the map returned when the following method is passed the following maps:

[redacted]

a. [redacted]

b. [redacted]

c. [redacted]

22. Modify the [redacted] program so that it prints the most frequently occurring words sorted by number of occurrences. To do this, write code at the end of the program to create a reverse map from counts to words that is based on the original map. Assume that no two words of interest occur the exact same number of times.

Exercises

1. Modify the [redacted] program developed in Section 11.1 to make two optimizations. First, instead of storing all integers up to the maximum in the [redacted] list, store only 2 and all odd numbers from 3 upward. Second, write code to ensure that if the first number in the [redacted] list ever reaches the square root of the maximum, all remaining values from the [redacted] list are moved into the [redacted] list. (Why is this a valid operation?)
2. Write a method called [redacted] that accepts two [redacted] as its parameters and returns a new [redacted] containing alternating elements from the two lists, in the following order:
   - First element from first list
   - First element from second list
   - Second element from first list
   - Second element from second list
   - Third element from first list
   - Third element from second list
   - . . .

If the lists do not contain the same number of elements, the remaining elements from the longer list should be placed consecutively at the end. For example, for a first list of [redacted] and a second list of [redacted], a call of [redacted] should return a list containing [redacted].

3. Write a method called [redacted] that accepts four parameters: a [redacted], an element value, a starting index, and an ending index.The method's behavior is to

remove all occurrences of the given element that appear in the list between the starting index (inclusive) and the ending index (exclusive). Other values and occurrences of the given value that appear outside the given index range are not affected.

For example, for the list [redacted], a call of [redacted] should produce the list [redacted]. Notice that the zeros located at indexes between 5 inclusive and 13 exclusive in the original list (before any modifications were made) have been removed.

4. Write a method called [redacted] that accepts a list of integers and an integer value [redacted] as its parameter, and rearranges (partitions) the list so that all the elements with values less than [redacted] occur before all elements with values greater than [redacted]. The exact order of the elements is unimportant, so long as all elements less than [redacted] appear before all elements greater than [redacted]. For example, for the linked list [redacted], one acceptable ordering of the list after a call of [redacted] would be [redacted]. You may assume that the list contains no duplicates and does not contain the element value [redacted].
5. Write a method called [redacted] that accepts a list of integers as its parameter and rearranges the list's elements into sorted ascending order, as well as removing all duplicate values from the list. For example, the list [redacted] would become [redacted] after a call to your method. Use a [redacted] as part of your solution.

6. Write a method [redacted] that accepts a list of integers as a parameter and returns the number of unique integer values in the list. Use a set as auxiliary storage to help you solve this problem. For example, if a list contains the values [redacted], your method should return [redacted]. The empty list contains [redacted] unique values.
7. Write a method [redacted] that accepts two lists of integers as parameters and returns the number of unique integers that occur in both lists. Use one or more sets as storage to help you solve this problem. For example, if one list contains the values `[3, 7, 3, -1, 2, 3, 7, 2, 15, 15]` and the other list contains the values [redacted], your method should return [redacted] because the elements [redacted] and [redacted] occur in both lists.
8. Write a method [redacted] that accepts a set of strings as a parameter and that returns the length of the longest string in the list. If your method is passed an empty set, it should return [redacted].
9. Write a method [redacted] that accepts a set of integers as a parameter and returns [redacted] if the set contains at least one odd integer and [redacted] otherwise. If passed the empty set, your method should return [redacted].
10. Write a method [redacted] that accepts a set of strings as a parameter and that removes all of the strings of even length from the set.
11. Write a method called [redacted] that accepts two [redacted] as parameters and returns a new [redacted] containing their symmetric set difference (that is, the set of elements contained in either of the two sets but not in both). For example, the symmetric difference between the

sets [redacted] and [redacted] is [redacted].

12. Write a method [redacted] that accepts a list of strings as a parameter and returns [redacted] if any single string occurs at least 3 times in the list, and [redacted] otherwise. Use a map as auxiliary storage.
13. Write a method [redacted] that accepts a map whose keys and values are strings as a parameter and returns [redacted] if no two keys map to the same value (and [redacted] if any two or more keys do map to the same value). For example, if the map contains the following key/value pairs, your method would return [redacted]: [redacted]. But calling it on the following map would return [redacted], because of two mappings for [redacted] and [redacted]: {[redacted].
14. Write a method [redacted] that accepts two maps whose keys are strings and whose values are integers as parameters and returns a new map containing only the key/value pairs that exist in both of the parameter maps. In order for a key/value pair to be included in your result, not only do both maps need to contain a mapping for that key, but they need to map it to the same value. For example, if the two maps passed are [redacted] and [redacted], your method would return the following new map (the order of the key/value pairs does not matter): [redacted].

15. Write a method [redacted] that accepts a list of integers as a parameter and returns the number of times the most frequently occurring integer (the “mode”) occurs in the list. Solve this problem using a map as auxiliary storage. If the list is empty, return [redacted].
16. Write a method called [redacted] that accepts a map whose keys and values are strings as its parameter and returns [redacted] if no two keys map to the same value. For example, [redacted] should return [redacted], but [redacted] should return [redacted]. The empty map is considered 1-to-1 and returns [redacted].
17. Write a method called [redacted] that accepts two maps from strings to strings as its parameters and returns [redacted] if every key in the first map is also contained in the second map and maps to the same value in the second map. For example, [redacted] is a submap of [redacted]. The empty map is a submap of every map.
18. Write a method called [redacted] that accepts a map from strings to strings as a parameter and returns a new map that is the reverse of the original. The reverse of a map is a new map that uses the values from the original as its keys and the keys from the original as its values. Since a map’s values need not be unique but its keys must be, you should have each value map to a set of keys. In other words, if the original map maps keys of type K to values of type V, the new map should map keys of type V to values that are [redacted] containing elements of type K. For example, the map [redacted] has a reverse of

[redacted]. (The order of the keys and values does not matter.)

19. Write a method called [redacted] that accepts a map whose keys are strings and whose values are integers as a parameter and returns the integer value that occurs the fewest times in the map. If there is a tie, return the smaller integer value. If the map is empty, throw an exception.
20. Write a modified version of the [redacted] program developed in Chapter 10 that uses sets rather than [redacted] to store its words. (The program will be noticeably shorter and will run faster!)
21. Write a method called [redacted] that accepts a list of strings representing individual words and counts the number of occurrences of all 2-character sequences of letters in those words. For example, suppose the list contains [redacted]. This list contains the following two-character pairs: [redacted] from [redacted] from [redacted] from [redacted]; and [redacted] from [redacted]. (Note that [redacted] is only one character long, so it contains no pairs.) Your method should count the occurrences of these pairs of letters and return the counts as a [redacted]. In this case you would return [redacted].

Notice that pairings that occur more than once in the same word should be counted as separate occurrences. For example, [redacted] and [redacted] each occur twice in [redacted].

Programming Projects

1. Write a program that computes the edit distance (also called the Levenshtein distance, for its creator Vladimir Levenshtein) between two words. The edit distance between two strings is the minimum number of operations that are needed to transform one string into the other. For this program, an operation is a substitution of a single character, such as from “brisk” to “brick”. The edit distance between the words “dog” and “cat” is 3, following the chain of “dot”, “cot”, and “cat” to transform “dog” into “cat”. When you compute the edit distance between two words, each intermediate word must be an actual valid word. Edit distances are useful in applications that need to determine how similar two strings are, such as spelling checkers.

Read your input from a dictionary text file. From this file, compute a map from every word to its immediate neighbors, that is, the words that have an edit distance of 1 from it. Once this map is built, you can walk it to find paths from one word to another.
A good way to process paths to walk the neighbor map is to use a linked list of words to visit, starting with the beginning word, such as “dog”. Your algorithm should repeatedly remove the front word of the list and add all of its neighbors to the end of the list, until the ending word (such as “cat”) is found or until the list becomes empty, which indicates that no path exists between the two words.

2. Write a program that solves the classic “stable marriage” problem. This problem deals with a group of men and a group of women. The program tries to pair them up so as to generate as many stable marriages as possible. A set

of marriages is unstable if you can find a man and a woman who would rather be married to each other than to their current spouses (in which case the two would be inclined to divorce their spouses and marry each other).

The input file for the program will list all of the men, one per line, followed by a blank line, followed by all of the women, one per line. The men and women are numbered according to their positions in the input file (the first man is #1, the second man is #2, and so on; the first woman is #1, the second woman is #2, and so on). Each input line (except for the blank line separating men from women) lists the person's name, followed by a colon, followed by a list of integers. These integers are the marriage partner preferences of this particular person. For example, see the following input line in the men's section:

This line indicates that the person is named "Joe" and that his first choice for marriage is woman #10, his second choice is woman #8, and so on. Any women not listed are considered unacceptable to Joe.

The stable marriage problem is solved by the following algorithm:

```
assign each person to be free.

while (some man M with a nonempty preference list is free) {


W = first woman on M's list.


if (some man P is engaged to W) {
```

```
assign P to be free.

}

assign M and W to be engaged to each other.

for (each successor Q of M who is on W's list) {

delete W from Q's preference list.

delete Q from W's preference list.

}

}
```

Consider the following input:

The following is a stable marriage solution for this input:

3. Write a program that solves the classic “random writer” problem. This problem deals with reading input files of text and examining the frequencies of characters. On the basis of those frequencies, you can generate randomized output that appears to match the writing style of the original document. The longer the chains you link together, the more accurate the random text will sound. For example, level 4 random text (text with chains of 4 letters long) generated from *Tom Sawyer* might look like this: “en themself, Mr. Welshman, but him awoke, the balmy shore. I’ll give him that he couple overy because in the slated snufflindeed structure’s kind was rath. She said that the wound the door a fever eyes that WITH him.” Level 10 random text from the same source might look like this: “you understanding that they don’t come around

in the cave should get the word beauteous was over-fondled, and that together and decided that he might as we used to do—it’s nobby fun. I’ll learn you.” Search the Internet for “Random Writer” to learn more about this problem, such as the specification posed by computer scientist Joseph Zachary.

Chapter 12 Recursion

12.1 Thinking Recursively
• A Nonprogramming Example
• An Iterative Solution Converted to Recursion
• Structure of Recursive Solutions
12.2 A Better Example of Recursion
• Mechanics of Recursion
12.3 Recursive Functions and Data
• Integer Exponentiation
• Greatest Common Divisor
• Directory Crawler
• Helper Methods
12.4 Recursive Graphics
12.5 Recursive Backtracking
• A Simple Example: Traveling North/East
• 8 Queens Puzzle
• Solving Sudoku Puzzles
12.6 Case Study: Prefix Evaluator
• Infix, Prefix, and Postfix Notation
• Evaluating Prefix Expressions
• Complete Program

Introduction

This chapter focuses on a programming technique known as recursion that allows us to solve certain complex problems in a highly elegant manner. The

chapter begins by comparing recursion with the problem-solving techniques you already know. Then it discusses the low-level mechanics that make recursion work in Java. Finally, we examine a number of problems that are easily expressed using this technique.

Recursion turns out to have a surprising range of useful applications, including recursive graphics that are known as fractals. But programming recursively also requires some special techniques that we'll have to explore. Recursive programming also requires a different mind-set in general, so the chapter explores a large set of example problems to reinforce this new way of thinking.

12.1 Thinking Recursively

The problem-solving techniques we have employed so far fall under the heading of classical *iteration*, also known as the *iterative approach.*

Iteration (Iterative)

A programming technique in which you describe actions to be repeated using a loop.

In this chapter, we will explore a new technique known as *recursion.*

Recursion (Recursive)

A programming technique in which you describe actions to be repeated using a method that calls itself.

You have spent so much time writing solutions iteratively that it will take a while to get used to thinking about problems recursively. This chapter will help you get acclimated.

A Nonprogramming Example

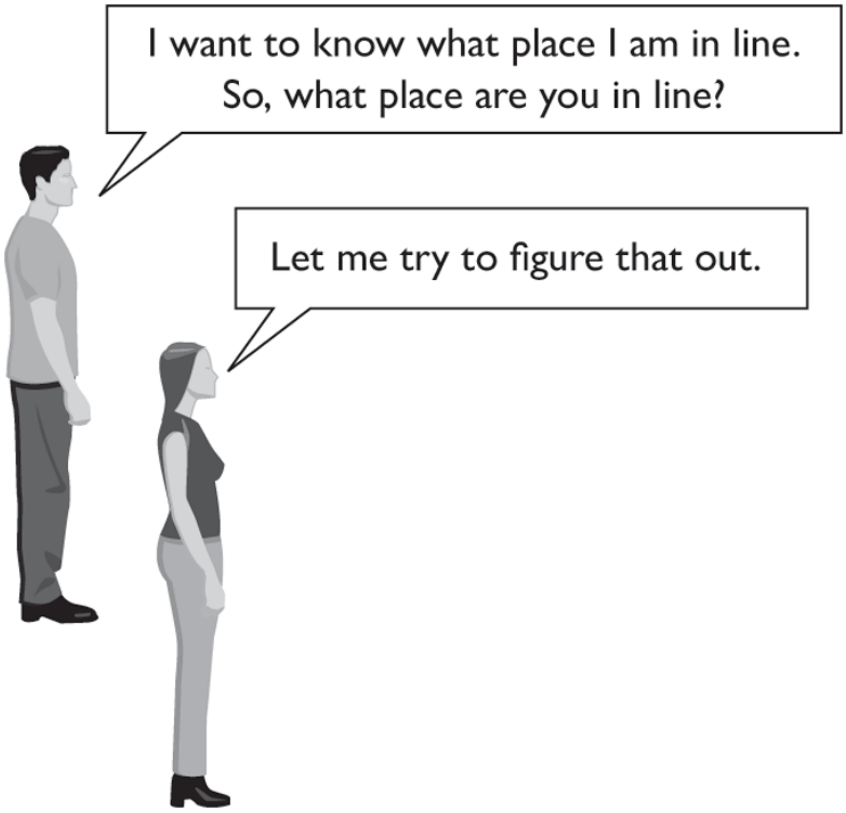
If you're standing in line, you might wonder what position you're in. Are you number 10 in the line? Number 20? How would you find out?

Most people would solve this problem iteratively, by counting the people in the line: one, two, three, and so on. This approach is like a [redacted] loop that continues while there are more people left to count. The iterative approach is a natural one, but it has some limitations. For example, what if the person in front of you is taller than you? Will you be able to see past that person to count all the people in front of him? And what if the line goes around the block and you can't see around the corner to count the people there?

Can you think of another way to determine your position in the line? To think about the problem recursively, you have to imagine that all the people standing in line work together to solve the problem: Instead of having one person do all of the counting, each person is responsible for a little piece.

One cooperative approach would be to ask the person in front of you what your place in line is. That person might ask another person, who might ask another person. But that doesn't help much, because it just leads to a bunch of people saying, "This guy wants to know what place he is in line. Does anyone know?" Someone would probably eventually start counting and solve the problem iteratively.

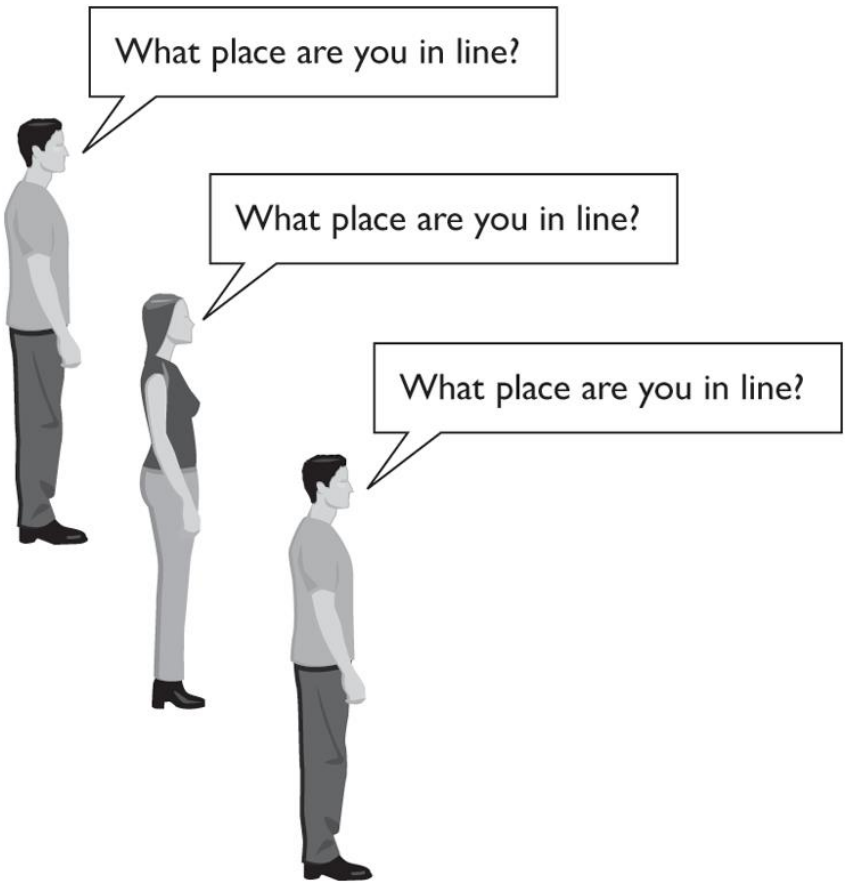
You have to make the problem simpler. Instead of asking the person in front of you what place *you* are in line, ask that person what place *he* or *she* is in line:



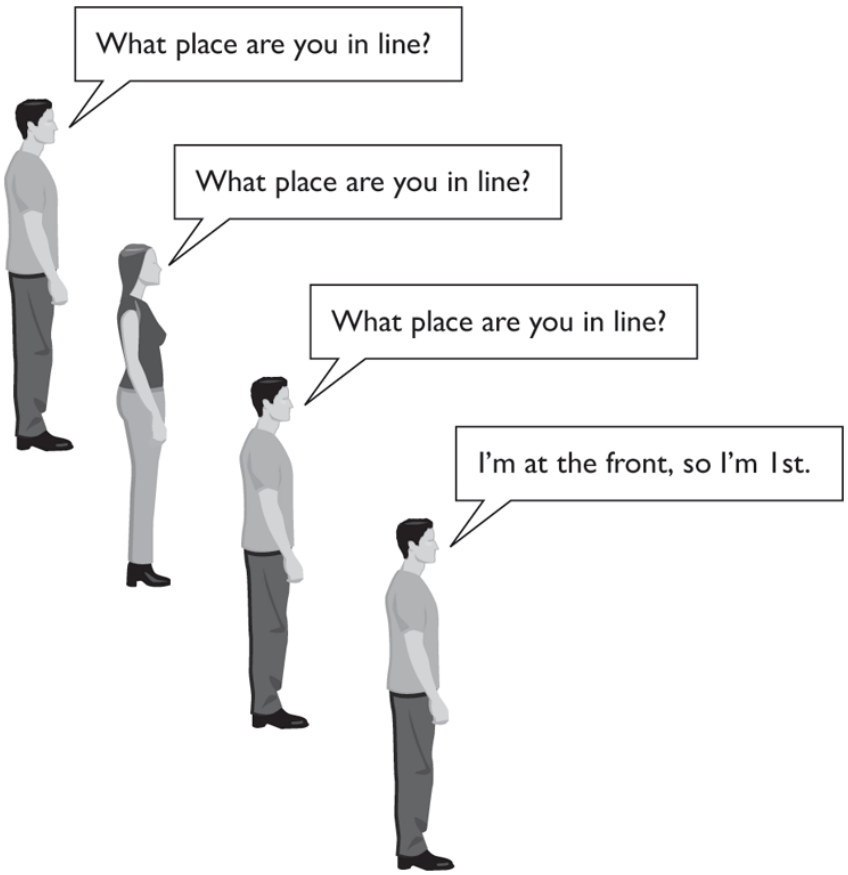
12.1-1 Full Alternative Text

The key difference is that the person in front of you is closer to the front of the line. Suppose, for example, that you're 4th in line. The person in front of you is 3rd in line, which is closer to the front. But notice that you're asking the person in front of you to think about the exact same *kind* of question you're considering: You're both trying to figure out your places in line. That's where recursion comes in—the problem *recurs* because each of you wants to answer the same question.

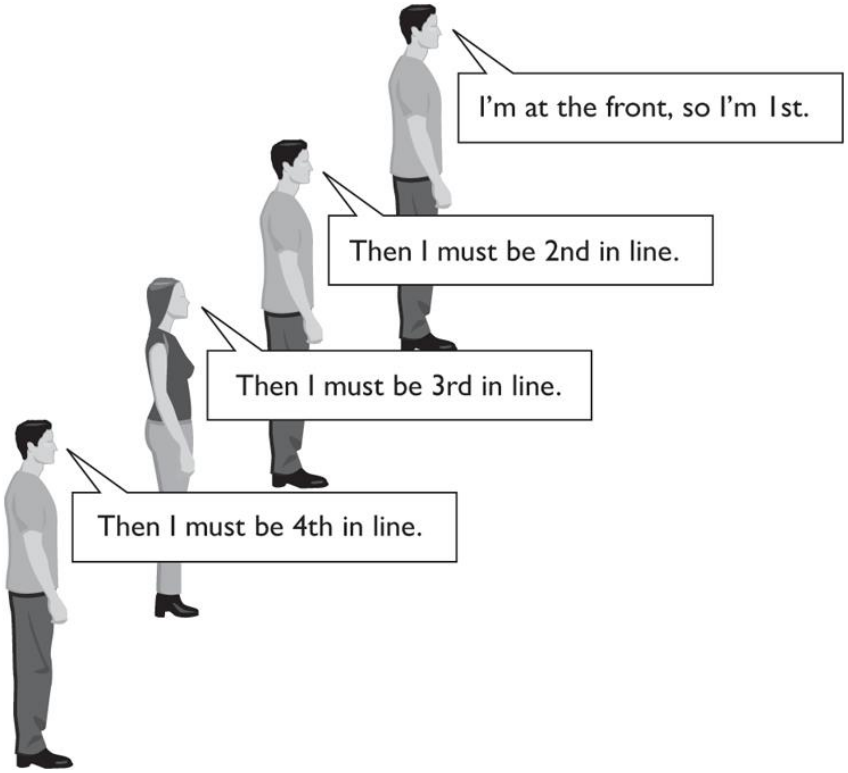
The idea is to set up a chain reaction of people, all asking the person in front the same question:



This process has to end eventually, when someone asks the person who is first in line:



At this point you've reached what is sometimes referred to as the *bottom* of the recursion. You've gotten a bunch of people involved in collectively solving the problem, and you've finally reached a point where you can start assembling the answer. The person at the front is in position 1. That means the person just before is at position 2, and the person just before that person is at position 3, and so on. Once you reach the bottom of the recursion, you *unwind* it to figure out the answer to your initial problem:



12.1-4 Full Alternative Text

These diagrams included just 4 individuals for the sake of brevity, but this process would still work even if there were 30 or even 300 people in the line.

One of the key aspects to notice here is that recursion involves many cooperating entities, each of which solves a little bit of the problem. Instead of one person doing all of the counting, each individual asks one question as we go toward the front of the line and answers one question as we come back down the line.

In programming, the iterative solution of having one person do all the counting is like having a loop that repeats some action. The recursive solution of having many people each do a little bit of work translates into many different method calls, each of which performs a little bit of work. Let's look at an example of how a simple iterative solution can be turned into a recursive solution.

An Iterative Solution Converted to Recursion

As a first example, we will explore a problem that has a simple iterative solution. It won't be a very impressive use of recursion because the problem is easily solved with iteration. But exploring a problem that has a straightforward iterative solution allows us to compare the two solutions.

Suppose you want to create a method called [redacted] that will take an integer parameter [redacted] and will produce a line of output with exactly [redacted] stars on it. You can solve this problem with a simple [redacted] loop:

The action being repeated here is the call on [redacted] that prints a star. To write the stars recursively, you need to think

about different cases. You might ask the method to produce a line with 10 stars, 20 stars, or 50 stars. Of all of the possible star-writing tasks you might ask it to perform, which is the simplest?

Students often answer that printing a line of one star is very easy and they're right that it's easy. But there is a task that is even easier. Printing a line of zero stars requires almost no work at all. You can create such a line by calling [redacted], so you can begin your recursive definition with a test for this case:

The code in the [redacted] part will deal with lines that have more than zero stars on them. Your instinct will probably be to fill in the [redacted] part with the [redacted] loop shown earlier, but you'll have to fight the instinct to solve the entire problem that way. To solve this second part of the problem, it is important to think about how you can do just a small amount of work that will get you closer to the solution. If the number of stars is greater than zero, you know you have to print at least one star, so you can add that action to the code:

```
System.out.print("*");
```

At this point in the process you have to make a leap of faith: You have to believe that recursion actually works. Once you've printed a single star, what's left to do? The answer is that you want to write [redacted] more stars, along with a [redacted]. In other words, after writing one star, the task that remains is to write a line of [redacted] stars. You may think, "If only I had a method that would produce a line of [redacted] stars, I could call that method." But you *do* have such a method—the method you're writing. So, after your method writes a single star, you can call the [redacted] method itself to complete the line of output:

```
[redacted]
```

```
writeStars(n - 1);
```

Many novices complain that this seems like cheating. You're supposed to be writing the method called [redacted], so how can you call [redacted] from inside [redacted]? Welcome to the world of recursion.

In the earlier example, we talked about people standing in a line and solving a problem together. To understand a recursive method like [redacted], it is useful to imagine that each method invocation is like a person in the line. The key insight is that there isn't just one person who can do the [redacted] task; there's an entire army of people, each of whom can do the task.

Let's think about what happens when you call the method and request a line of three stars:

Imagine that you're calling up the first person from the [redacted] army and saying, "I want a line of three stars." That person looks at the code in the method and sees that the way to write a line of three stars is to execute the following lines:

In other words, the first member of the army writes a star and calls up the next member of the army to write a line of two stars, and so on down the line. Just as in the earlier example you had a series of people figuring out their places in line, now you have a series of people each printing one star and then calling on someone else to write the rest of the line. In the line example, you eventually reached the person at the front of the line. In this case, you eventually reach a request to write a line of zero stars, which leads you into the [redacted] branch rather than the [redacted] branch. At this point, you complete the task with a simple [redacted].

Here is a trace of the calls that would be made to print the line:

A total of four different calls are made on the method. Continuing the analogy, you could say that four members of the army are called up to solve the task together. Each one solves a star-writing task, but the tasks are slightly different (three stars, two stars, one star, zero stars). This is similar to the example in which the various people standing in line were all answering the same question but were solving slightly different problems because their positions in line were different.

Structure of Recursive Solutions

Writing recursive solutions requires you to make a certain leap of faith, but there is nothing magical about recursion. Let’s look a bit more closely at the structure of a recursive solution. The following method is not a solution to the task of writing a line of ▮ stars:

This version never finishes executing, a phenomenon called *infinite recursion.* For example, if you ask the method to write a line of 10 stars, it tries to accomplish that by asking the method to write a line of 10 stars, which asks the method to write a line of 10 stars, which asks the method to write a line of 10 stars, and so on. This solution is the recursive equivalent of an infinite loop.

Every recursive solution that you write will have two key ingredients: a *base case* and a *recursive case.*

Base Case

A case within a recursive solution that is so simple that it can be solved directly without a recursive call.

Recursive Case

A case within a recursive solution that involves reducing the overall problem to a simpler problem of the same kind that can be solved by a recursive call.

Here is the method again, with comments indicating the base case and recursive case:

```
// base case

// recursive case
```

The base case is the task of writing a line of zero stars. This task is so simple that it can be done immediately. The recursive case is the task of writing lines with one or more stars. To solve the recursive case, you begin by writing a single star, which reduces

the remaining task to that of writing a line of [redacted] stars. This is the task that the [redacted] method is designed to solve and it is simpler than the original task, so you can solve it by making a recursive call.

As an analogy, suppose you're at the top of a ladder with [redacted] rungs on it. If you have a way to get from one rung to the one below and if you can recognize when you've reached the ground, you can handle a ladder of any height. Stepping from one rung to the one below is like the recursive case in which you perform some small amount of work that reduces the problem to a simpler one of the same form (get down from rung [redacted] versus get down from rung [redacted]). Recognizing when you reach the ground is like the base case that can be solved directly (stepping off the ladder).

Some problems involve multiple base cases and some problems involve multiple recursive cases, but there will always be at least one of each case in a correct recursive solution. If you are missing either, you run into trouble. Without the ability to step down from one rung to the one below, you'd be stuck at the top of the ladder. Without the ability to recognize when you reach the ground, you'd keep trying to step down onto another rung even when there were no rungs left in the ladder.

Keep in mind that your code can have infinite recursion even if it has a proper recursive case. Consider, for example, this version of [redacted]:

This version correctly reduces from the case of [redacted] to the case of [redacted], but it has no base case. As a result, it goes on infinitely. Instead of stopping when it reaches the task of writing zero stars, it instead recursively tries to write −1 stars, then −2 stars, then −3 stars, and so on.

Because recursive solutions include some combination of base cases and recursive cases, you will find that they are often written with [redacted] statements, nested [redacted] statements, or some minor variation thereof. You will also find that recursive programming generally involves a case analysis, in which you categorize the possible forms the problem might take into different cases and write a solution for each case.

12.2 A Better Example of Recursion

Solving the [redacted] task with recursion may have been an interesting exercise, but it isn't a very compelling example. Let's look in detail at a problem in which recursion simplifies the work to be done.

Suppose you have a [redacted] that is tied to an external input file and you want to print the lines of the file in reverse order. For example, the file might contain the following four lines of text:

[redacted]

Printing these lines in reverse order would produce this output:

[redacted]

To perform this task iteratively, you'd need some kind of data structure for storing the lines of text, such as an [redacted]. However, recursion allows you to solve the problem without using a data structure.

Remember that recursive programming involves thinking about cases. What would be the simplest file to reverse? A one-line file would be fairly easy to reverse, but it would be even easier to reverse an empty file. So, you can begin writing your method as follows:

In this problem, the base case is so simple that there isn't anything to do. An empty file has no lines to reverse. Thus, in this case it makes more sense to turn around the [redacted] statement so that you test for the recursive case. That way you can write a simple [redacted] statement that has an implied "[redacted] there is nothing to do":

Again, the challenge is to solve only a little bit of the problem. How do you take just one step that will get you closer to completing the task? You can read one line of text from the file:

For the sample file, this code would read the line [redacted] into the variable [redacted] and leave you with the following three lines of text in the [redacted]:

Recall that your aim is to produce the following overall output:

You might be asking yourself questions like, “Is there another line of input to process?” But that’s not recursive thinking. If you’re

thinking recursively, you'll be thinking about what a call on the method will get you. Since the [redacted] is positioned in front of the three lines [redacted], [redacted], and [redacted], a call on [redacted] should read in those lines and produce the first three lines of output that you're looking for. If that works, you'll only have to write out the line [redacted] afterward to complete the output.

This is where the leap of faith comes in—you have to believe that the [redacted] method actually works. If it does, this code can be completed as follows:

```
reverse(input);

System.out.println(line);
```

This code does work. To reverse a sequence of lines, simply read in the first one, reverse the others, and then write out the first one.

Mechanics of Recursion

Novices seem to understand recursion better when they know more about the underlying mechanics that make it work. Before we examine a recursive method in detail, let's review how nonrecursive methods work. Consider the following simple program:

```
public class

public static void
```

```
public static void
```

```
public static void
```

The program prints three triangles:

How do we describe the method calls that take place in this program? Imagine that each method call has been written on a different piece of paper. We begin program execution with the

[redacted] method, so imagine that we're grabbing the sheet of paper with [redacted] on it:

```
public static void main(String[] args) {
    drawTriangle();
    drawTwoTriangles();
}
```

12.2-5 Full Alternative Text

We then execute each of the statements in [redacted] in turn, from first to last. First, we execute the call on [redacted]:

```
public static void main(String[] args) {
➔   drawTriangle();
    drawTwoTriangles();
}
```

12.2-6 Full Alternative Text

We know that at this point, the computer stops executing [redacted] and turns its attention to the [redacted] method. This step is analogous to picking up the piece of paper with [redacted] written on it and placing it over the piece of paper with [redacted] on it:

```
public static void main(String[] args) {
public static void drawTriangle() {
    System.out.println("  *");
    System.out.println(" ***");
    System.out.println("*****");
    System.out.println();
}
```

12.2-7 Full Alternative Text

Now we execute each of the statements in [redacted] from first to last, then go back to [redacted], removing the [redacted] sheet. When we go back to [redacted] we will have finished the call on [redacted], so the next step is the call on [redacted]:

```
public static void main(String[] args) {
    drawTriangle();
➔   drawTwoTriangles();
}
```

12.2-8 Full Alternative Text

So we grab the piece of paper with [redacted] on it and place it over the paper with [redacted] on it:

```
public static void main(String[] args) {
public static void drawTwoTriangles() {
    drawTriangle();
    drawTriangle();
}
```

12.2-9 Full Alternative Text

The first thing to do here is the first call on [redacted]

```
public static void main(String[] args) {
public static void drawTwoTriangles() {
➔   drawTriangle();
    drawTriangle();
}
```

12.2-10 Full Alternative Text

To execute this method, we take out the sheet of paper with [redacted] on it and place it on top of the [redacted] sheet:

```
public static void main(String[] args) {
public static void drawTwoTriangles() {
public static void drawTriangle() {
    System.out.println("  *");
    System.out.println(" ***");
    System.out.println("*****");
    System.out.println();
}
```

12.2-11 Full Alternative Text

This diagram makes it clear that we started with the method [redacted], which called the method [redacted], which called the method [redacted]. So, at this moment in time, three different methods are active. The one on top is the one that we are actively executing. Once we complete it, we'll go back to the one underneath, and once we finish that one, we'll go back to [redacted]. We could continue with this example, but you probably get the idea by now.

The idea of representing each method call as a piece of paper and putting each one on top of the others as it is called is a metaphor for Java's *call stack.*

Call Stack

[redacted]

The internal structure that keeps track of the sequence of methods that have been called.

If you envision the call stack as a stack of papers with the most recently called method on top, you'll have a pretty good idea of how it works.

Let's use the idea of the call stack to understand how the recursive file-reversing method works. To visualize the call stack, we need to put the method definition on a piece of paper:

```
public static void reverse(Scanner input) {
    if (input.hasNextLine()) {
        String line = input.nextLine();
        reverse(input);
        System.out.println(line);
    }
}
line
```

12.2-12 Full Alternative Text

Notice that the paper includes a place to store the value of the local variable [redacted]. This is an important detail.

Suppose that we call this method with the earlier sample input file, which contains the following four lines of text:

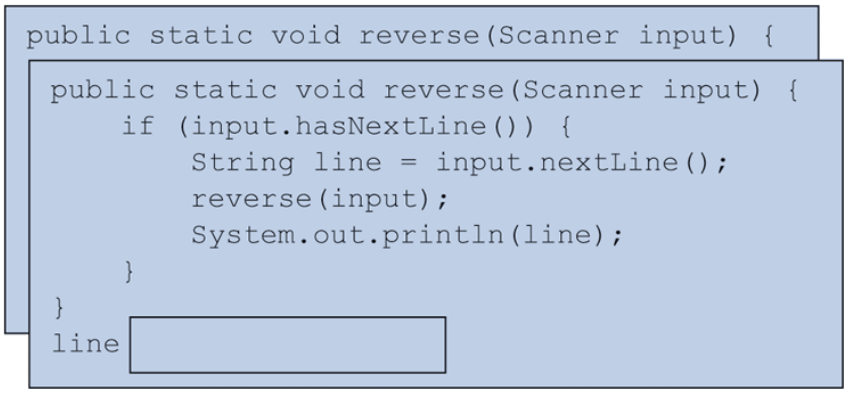
[redacted]

When we call the method, it reads the first line of text into its [redacted] variable, and then it reaches the recursive call on [redacted]:

```
public static void reverse(Scanner input) {
    if (input.hasNextLine()) {
        String line = input.nextLine();
→       reverse(input);
        System.out.println(line);
    }
}
line "this"
```

12.2-13 Full Alternative Text

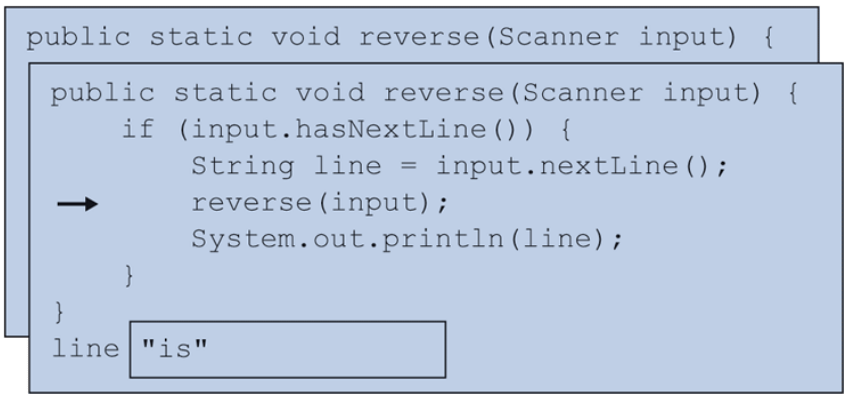
Then what happens? In the [redacted] program, we took the sheet of paper for the method being called and placed it on top of the current sheet of paper. But here we have the method [redacted] calling the method [redacted]. To understand what happens, you have to realize that each method invocation is independent of the others. We don't have only a single sheet of paper with the [redacted] method written on it; we have as many copies as we want. So, we can grab a second copy of the method definition and place it on top of the current one:



12.2-14 Full Alternative Text

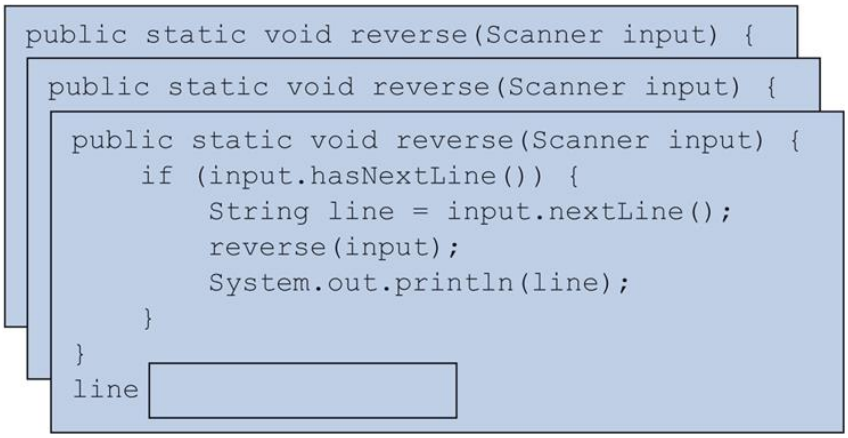
This new version of the method has a variable of its own, called [redacted], in which it can store a line of text. Even though the previous version (the one underneath this one) is in the middle of its execution, this new one is at the beginning of its execution. Think back to the analogy of being able to employ an entire army of people to write out a line of stars. Just as you could call on as many people as you needed to solve that problem, you can bring up as many copies of the [redacted] method as you need to solve this problem.

The second call on [redacted] reads another line of text (the second line, [redacted]). After the program reads the second line, it makes another recursive call on [redacted]:



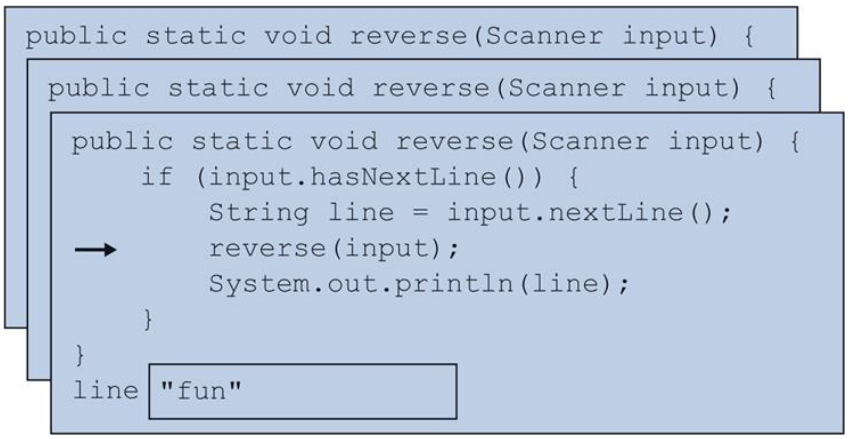
12.2-15 Full Alternative Text

So Java sets aside this version of the method as well and brings up a third version:



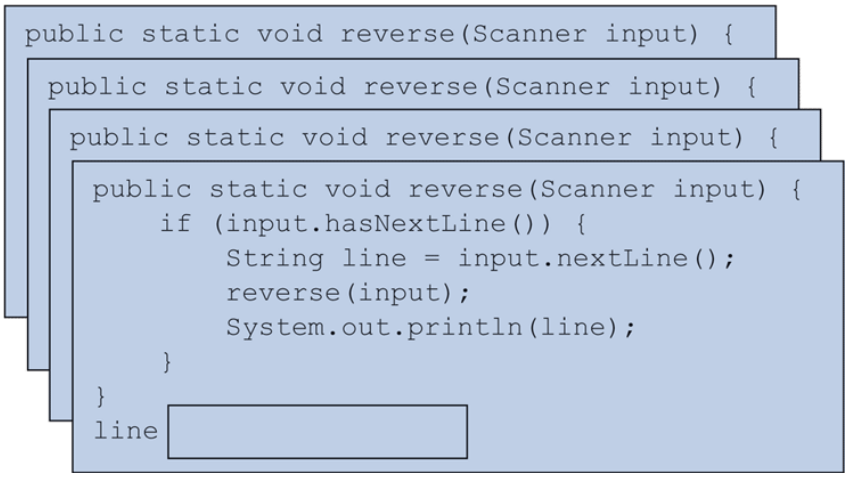
12.2-16 Full Alternative Text

Again, notice that this version has its own variable called [redacted] that is independent of the other variables called [redacted]. This version of the method also reads in a line (the third line, [redacted]) and reaches a recursive call on [redacted]:



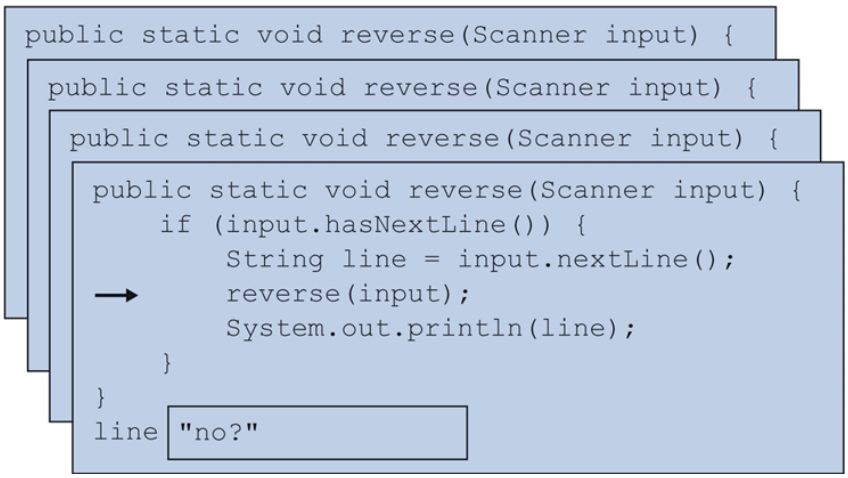
12.2-17 Full Alternative Text

This brings up a fourth version of the method:



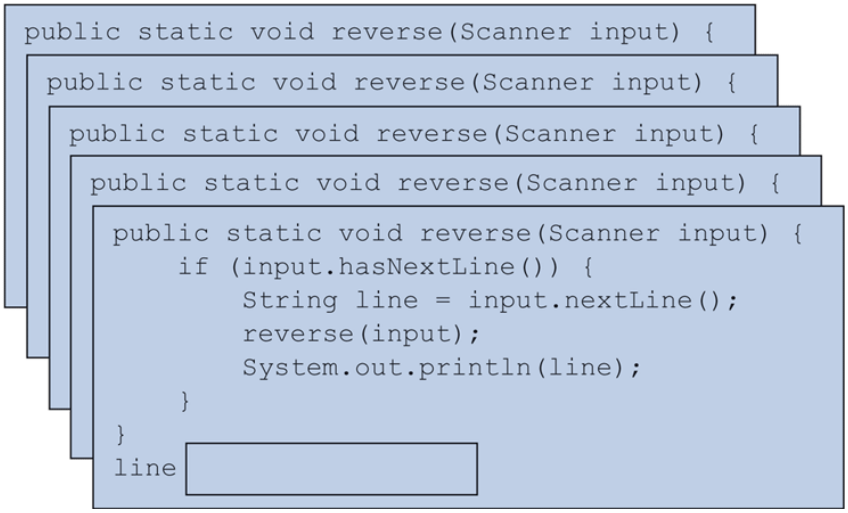
12.2-18 Full Alternative Text

This version finds a fourth line of input ( ), so it reads that in and reaches the recursive call:



12.2-19 Full Alternative Text

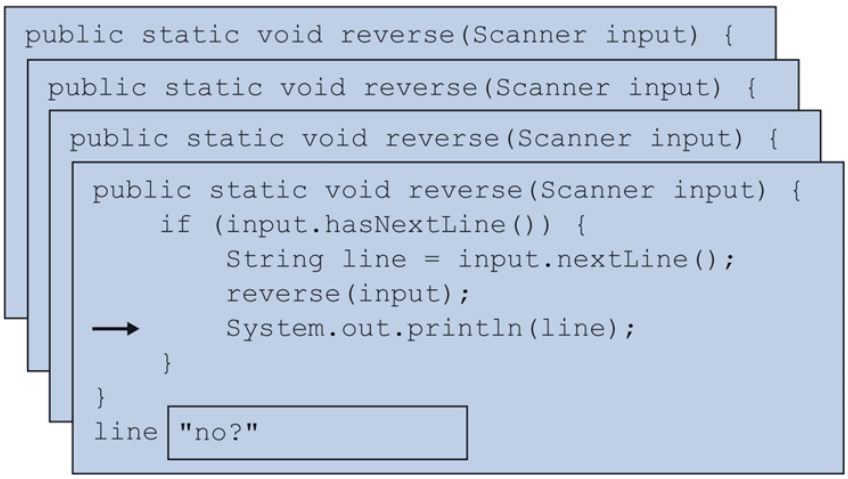
This call brings up a fifth version of the method:



12.2-20 Full Alternative Text

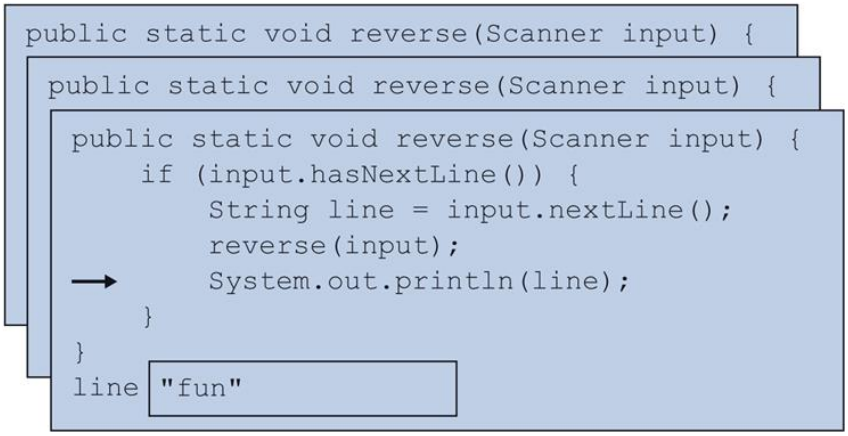
This version turns out to have the easy task, like the final person who was asked to print a line of zero stars. This time around the [redacted] is empty ([redacted] returns [redacted]). The program has reached the very important base case that stops this process from going on indefinitely. This version of the method recognizes that there are no lines to reverse, so it simply terminates.

Then what? Having completed this call, we throw it away and return to where we were just before executing the call:



12.2-21 Full Alternative Text

We've finished the call on [redacted] and are positioned at the [redacted] right after it, so we print the text in the [redacted] variable ([redacted]) and terminate. Where does that leave us? This method has been executed and we return to where we were just before:



12.2-22 Full Alternative Text

We then print the current line of text, which is [redacted], and this version also goes away:

```
public static void reverse(Scanner input) {
    if (input.hasNextLine()) {
        String line = input.nextLine();
        reverse(input);
        System.out.println(line);
    }
}
line "is"
```

12.2-23 Full Alternative Text

Now we execute this [redacted], for the text [redacted], and eliminate one more call:

```
public static void reverse(Scanner input) {
    if (input.hasNextLine()) {
        String line = input.nextLine();
        reverse(input);
        System.out.println(line);
    }
}
line "this"
```

12.2-24 Full Alternative Text

Notice that we've written out three lines of text so far:

Our leap of faith was justified. The recursive call on [redacted] read in the three lines of text that followed the first line of input and

printed them in reverse order. We complete the task by printing the first line of text, which leads to this overall output:

Then this version of the method terminates, and the program has finished executing.

12.3 Recursive Functions and Data

Both of the examples of recursion we have studied so far have been action-oriented methods with a return type of [redacted]. In this section, we will examine some of the issues that arise when you want to write methods that compute values and return a result. Such methods are similar to mathematical functions in that they accept a set of input values and produce a set of possible results. We'll also explore an example that involves manipulating recursive data and an example that requires a helper method.

Integer Exponentiation

Java provides a method called [redacted] that allows you to compute an exponent. If you want to compute the value of xy, you can call [redacted]. Let's consider how we could implement the [redacted] method. To keep things simple, we'll limit ourselves to the domain of integers. But because we are limiting ourselves to integers, we have to recognize an important precondition of our method: We won't be able to compute negative exponents because the results would not be integers.

The method we want to write will look like the following:

We could obviously solve this problem by writing a loop, but we want to explore how to write the method recursively. Again, we should start by thinking about different cases. What would be the easiest exponent to compute? It’s pretty easy to compute x1, so that’s a good candidate, but there is an even more basic case. The simplest possible exponent is 0. By definition, any integer to the 0 power is considered to be 1. So we can begin our solution with the following code:

In the recursive case, we know that [redacted] is greater than [redacted]. In other words, there will be at least one factor of [redacted] in the result. We know from mathematics that

$$xy = x \cdot xy-1$$

This equation expresses [redacted] to the [redacted] power in terms of [redacted] to a smaller power, [redacted]. Therefore, it can serve as our recursive case. All we have to do is to translate it into its Java equivalent:

```
return x * pow(x, y - 1);
```

This is a complete recursive solution. Tracing the execution of a recursive function is a little more difficult than using a [redacted] method, because we have to keep track of the values that are returned by each recursive call. The following is a trace of execution showing how we would compute 35:

```
pow(3, 5) = 3 * pow(3, 4)
|     pow(3, 4) = 3 * pow(3, 3)
|     |     pow(3, 3) = 3 * pow(3, 2)
|     |     |     pow(3, 2) = 3 * pow(3, 1)
|     |     |     |     pow(3, 1) = 3 * pow(3, 0)
|     |     |     |     |     pow(3, 0) = 1
|     |     |     |     pow(3, 1) = 3 * 1 = 3
|     |     |     pow(3, 2) = 3 * 3 = 9
|     |     pow(3, 3) = 3 * 9 = 27
|     pow(3, 4) = 3 * 27 = 81
pow(3, 5) = 3 * 81 = 243
```

12.3-25 Full Alternative Text

Notice that we make a series of six recursive calls in a row until we reach the base case of computing 3 to the 0 power. That call returns the value [redacted] and then the recursion unwinds, computing the various answers as it returns from each method call.

It is useful to think about what will happen if someone violates the precondition by asking for a negative exponent. For example, what if someone asked for [redacted]? The method would recursively ask for [redacted], which would ask for [redacted], which would ask for [redacted], and so on. In other words, it would lead to an infinite recursion. In a sense, it's okay for this to occur, because the person calling the method should pay

attention to the precondition and should not enter a negative exponent. But it’s not much work for us to handle this case in a more elegant manner. Our solution is structured as a series of cases, so we can simply add a new case for illegal exponents:

```
if (y < 0) {

throw new IllegalArgumentException("negative exponent: " + y);

} else
```

One of the advantages of writing functions recursively is that if we can identify other cases, we can potentially make the function

more efficient. For example, suppose that you want to compute 216. In its current form, the method will multiply 2 by 2 by 2 a total of 16 times. But we can do better than that. If is an even exponent, then

xy=(x2)y2

So instead of computing 216, we can compute 48, which is simpler. Adding this case to our method is relatively easy:

```
} else if (y % 2 == 0) {

// recursive case with y > 0, y even

return pow(x * x, y / 2);

}
```

This version of the method is more efficient than the original. The following is a trace of execution for computing 216:

```
pow(2, 16) = pow(4, 8)
|    pow(4, 8) = pow(16, 4)
|    |    pow(16, 4) = pow(256, 2)
|    |    |    pow(256, 2) = pow(65536, 1)
|    |    |    |    pow(65536, 1) = 65536 * pow(65536, 0)
|    |    |    |    |    pow(65536, 0) = 1
|    |    |    |    pow(65536, 1) = 65536 * 1 = 65536
|    |    |    pow(256, 2) = 65536
|    |    pow(16, 4) = 65536
|    pow(4, 8) = 65536
pow(2, 16) = 65536
```

12.3-26 Full Alternative Text

Without the special case for even exponents, this call would have required 17 different calls on [redacted] (16 recursive cases and one base case).

Greatest Common Divisor

In mathematics, we often want to know the largest integer that goes evenly into two different integers, which is known as the

greatest common divisor (or GCD) of the two integers. Let's explore how to write a GCD method recursively.

For now, let's not worry about negative values of [redacted] and [redacted]. We want to write the following method:

To introduce some variety, let's try to figure out the recursive case first and then figure out the base case. Suppose, for example, that we are asked to compute the GCD of 20 and 132. The GCD is 4, because 4 is the largest integer that goes evenly into both numbers.

There are many ways to compute the GCD of two numbers. One of the most efficient algorithms dates back at least to the time of Euclid and perhaps even farther. This algorithm eliminates any multiples of the smaller integer from the larger integer. In the case of 20 and 132, we know that

$$132=20\cdot6+12$$

There are six multiples of 20 in 132, with a remainder of 12. Euclid's algorithm says that we can ignore the six multiples of 20

and just focus on the value 12. In other words, we can replace 132 with 12:

We haven’t figured out the base case yet, but no matter what the base case ends up being, we’re making progress if we can reduce the numbers using Euclid’s algorithm. When you’re dealing with nonnegative integers, you can’t reduce them forever.

The proof of this principle is beyond the scope of this book, but that is the basic idea. This algorithm is easy to express in Java terms because the mod operator gives us the remainder when one number is divided by another. Expressing this principle in general terms, we know that

Again, the proof is beyond the scope of this book, but given this basic principle we can produce a recursive solution to the problem. We might try to write the method as follows:

This isn’t a bad first attempt, but it has a problem: It’s not enough for the solution to be mathematically correct; we also need our recursive solution to keep reducing the overall problem to a simpler problem. If we start with the numbers 132 and 20, the method makes progress on the first call, but then it starts repeating itself:

This pattern will lead to infinite recursion. The Euclidean trick helped the first time around, because for the first call [redacted] was greater than [redacted] (132 is greater than 20). But the algorithm makes progress only if the first number is larger than the second number.

Here is the line of code that is causing the problem:

When we compute [redacted], we are guaranteed to get a result that is smaller than [redacted]. That means that on the recursive call, the first value will always be smaller than the second value. To make the algorithm work, we need the opposite to be true. We can achieve this goal simply by reversing the order of the arguments:

On this call, we are guaranteed to have a first value that is larger than the second value. If we trace this version of the method for computing the GCD of 132 and 20, we get the following sequence of calls:

At this point we have to decide what the GCD of 4 and 0 is. It may seem strange, but the answer is 4. In general, [redacted] is [redacted]. Obviously, the GCD can't be any larger than [redacted], and [redacted] goes evenly into [redacted]. But [redacted] also goes evenly into [redacted], because [redacted] can be written as an even multiple of [redacted]: [redacted].

This observation leads us to the base case. If [redacted] is [redacted], the GCD is [redacted]:

```
if (y == 0) {

// base case with y == 0

return x;

}
```

This base case also solves the potential problem that the Euclidean formula depends on not being . However, we still have to think about the case in which either or both of and are negative. We could keep the precondition and throw an exception when this occurs, but it is more common in mathematics to return the GCD of the absolute value of the two values. We can accomplish this by including one extra case for negatives:

```
if (x < 0 || y < 0) {

// recursive case with negative value(s)

return gcd(Math.abs(x), Math.abs(y));

} else
```

COMMON PROGRAMMING ERROR

Infinite Recursion

Everyone who uses recursion to write programs eventually accidentally writes a solution that leads to infinite recursion. For example, the following is a slight variation of the method that doesn't work:

This solution is just slightly different than the one we wrote previously. In the test for negative values, this code tests whether and are less than or equal to . The original code tests whether they are strictly less than . It doesn't seem like this variation should make much difference, but it does. If we execute this version of the code to solve our original problem of finding the GCD of 132 and 20, the program produces many lines of output that look like the following:

The first time you see this, you are likely to think that something has broken on your computer because you will get so many lines of output. The number of lines of output will vary from one system to another, but it’s likely to be hundreds, if not thousands. If you scroll all the way back up, you’ll see that the output begins with this message:

Java is letting you know that the call stack has gotten too big. Why did this happen? Remember the trace of execution for this case:

Consider what happens at this point, when we call . The value of is , which is our base case, so normally we would expect the method to return the value and terminate. But the method begins by checking whether either or is less than or equal to . Since is , this test evaluates to , so the method makes a recursive call with the absolute values of and . But the absolute values of 4 and 0 are 4 and 0. So the method decides that must be equal to , which must be equal to :

In other words, this version generates infinitely many recursive calls. Java allows you to make a lot of recursive calls, but eventually it runs out of space. When it does, it gives you a back trace to let you know how you got to the error. In this case, the back trace is not nearly as helpful as usual because almost all of the calls will involve the infinite recursion.

Again, think in terms of stacking pieces of paper on top of each other as methods are called. You'd wind up with a stack containing hundreds or even thousands of sheets, and you would have to look back through all of these to find the problem.

To handle these situations, you have to look closely at the line number to see which line of your program generated the infinite recursion. In this simple case, we know that it is the recursive call for negative and values. That alone might be enough to allow us to pinpoint the error. If the problem isn't obvious, though, you might need to include statements to figure out what is going on. For example, in this code, we could add a just before the recursive call:

```
System.out.println("x = " + x + " and y = " + y);
```

When we run the program with that in place, the code produces hundreds of lines of output of the form:

If we examine that case closely, we'll see that we don't have negative values and will realize that we have to fix the test we are using.

Directory Crawler

Recursion is particularly useful when you're working with data that is itself recursive. For example, consider how files are stored on a computer. Each file is kept in a folder or directory. But directories can contain more than just files: Directories can contain other directories, those inner directories can contain directories, and even those directories can contain directories. Directories can be nested to an arbitrary depth. This storage system is an example of recursive data.

In Chapter 6, you learned to use objects to keep track of files stored on your computer. For example, if you have a file called , you can construct a object that can be used to get information about that file:

Several useful methods for the File class were introduced in Chapter 6. For example, the exists method indicates whether or not a certain file exists, and the isDirectory method indicates whether or not the name supplied corresponds to a file or a directory.

Let's write a program that will prompt the user for the name of a file or directory and recursively explore all the files that can be reached from that starting point. If the user provides the name of a file, the program should simply print the name. But if the user gives the name of a directory, the program should print the directory name and list all the directories and files inside that directory.

We can write a fairly simple main method that prompts for a file or directory name and checks to make sure it exists. If it does not, the program tells the user that there is no such file or directory. If it does exist, the program calls a method to print the information for that File object:

Our job will be to write the [redacted] method. The method should begin by printing the name of the file or directory:

If the [redacted] object [redacted] represents a simple file, we have completed the task. But if [redacted] represents a directory, we also want to print the names of all the files contained in the directory:

```
if (f.isDirectory()) {

// print contents of directory

}
```

We can accomplish this task with the [redacted] method, which returns an array of [redacted] objects that represent the contents of the directory. We can use a for-each loop to process each object:

```
for (File subF : f.listFiles()) {

// print information for subF

}
```

To complete the method, we have to figure out how to print information about each of the individual objects in the directory. An obvious approach is to simply print the names of the subfiles:

```
System.out.println(subF.getName());
```

This method works in that it prints the names of the subfiles inside the directory. But remember that there can be directories inside this directory, so some of those subfiles might actually be directories whose contents also need to be printed. We could try to fix our code by adding a test:

```
if (subF.isDirectory()) {

// print contents of subdirectory

}
```

But even this won’t work, because there might be directories within those inner directories, and those directories might have subdirectories. There is no simple way to solve this problem with standard iterative techniques.

The solution is to think recursively. You might be tempted to envision many different cases: a file, a directory with files, a

directory with subdirectories, a directory with subdirectories that have subdirectories, and so on. However, there are really only two cases to consider: Each object is either a file or a directory. If it's a file, we simply print its name. If it's a directory, we print its name and then print information about every file and directory inside of it. How do we get the code to recursively explore all of the possibilities? We call our own method to process whatever appears inside a directory:

```
print(subF)
```

This version of the code recursively explores the structure. Each time the method finds something inside a directory, it makes a recursive call that can handle either a file or a directory; that recursive call might make yet another recursive call to handle either a file or directory, and so on.

If we run this version of the method, we'll get output like the following:

The problem with this output is that it doesn't indicate the structure to us. We know that the first line of output is the name of the starting directory ( ) and that everything that follows is inside that directory, but we can't easily see the substructure. It would be more convenient if the output used indentation to indicate the inner structure, as in the following lines of output:

In this output, we can more clearly see that the directory called [redacted] contains three elements, two of which are directories that have their own files ([redacted] and [redacted]). We can get this output by including an extra parameter in the [redacted] method that indicates the desired level of indentation. On the initial call in [redacted], we can pass the method an indentation of 0. On each recursive call, we can pass it a value one higher than the current level. We can then use that parameter to print some extra spacing at the beginning of the line to generate the indentation.

Here is our program, including the new version of the method with indentation:

```
import

import

public class

public static void

new

new
```

```
if

else

public static void

int
```

```
for

int

if

for
```

Helper Methods

Sometimes when you solve a problem recursively, you need to introduce an extra method to solve the problem. We call such methods *helper* methods.

For example, consider the task of writing a method called [redacted] that takes an array of integers as a parameter and that returns the sum of the numbers in the array. Even though this is a fairly simple task, it turns out to be challenging to implement it using recursion. We could make a base case for an empty array (an array of length 0):

If we write the code this way, what would we use for a recursive case? Suppose, for example, that we are asked to work with an array of length 10. How do we simplify an array of length 10 to bring it closer to being an empty array? The only way to do that would be to construct a new array with a shorter length, but that would be wasteful.

It is useful to consider how you would solve this iteratively. Here is an iterative version that follows a classic cumulative sum approach:

The key to the iterative approach is to work with an index variable that starts at 0 and continues iterating until it equals the length of the array. We can apply this same idea to a recursive solution. Remember that you want to think about how to pick off some small piece of the problem. Consider the following line of pseudocode:

```
sum(entire list) = list[0] + sum(list starting at 1)
```

This code states that we have to add in [redacted]. That leaves us with the task of adding up the remaining values of the list (the values starting at index 1). And how would we do that? We can similarly say that:

```
sum(list starting at 1) = list[1] + sum(list starting at 2)
```

This approach leads to a series of recursive calls, each of which handles one element of the array:

```
sum(entire list) = list[0] + sum(list starting at 1)

    sum(list starting at 1) = list[1] + sum(list starting at 2)

        sum(list starting at 2) = list[2] + sum(list starting at 3)

            ...
```

When does this process end? We could end when we get to the last element of the array:

```
sum(list starting at list.length - 1) = list[list.length - 1]
```

Unfortunately, this choice of ending point assumes that a last element exists. That won't be true of an array of length 0. An easier approach is to end when we reach an index beyond the last element of the array. The [redacted] loop version of this method executes for as long as the index is less than the length of the array. Once the index becomes equal to the length of the array, it stops. Our recursion can similarly stop when the index becomes equal to the length of the array. And what does that add up to? It adds up to 0:

```
sum(list starting at list.length) = 0
```

Returning 0 in this case is the recursive equivalent of initializing the cumulative sum to 0.

We are almost ready to write the recursive method, but to express this idea as a method, we have to pass the method both the array and the starting index. So we want our method to look like the following:

```

```

Using those two parameters, you'll find that the code is fairly simple to write using the approach described before:

Of course, there's the issue of that extra parameter. We were asked to write a method that is passed just the array. We could complain that we were asked to solve the wrong problem, but that's a bad attitude to have. We should allow a client to call methods using parameters that are convenient. We need the second parameter to write our recursive solution, but we shouldn't burden the client with the need to understand that detail.

This is a common situation. We often find ourselves wanting to write a recursive method that has extra parameters relative to the public method we are being asked to implement. The solution is to introduce a private method that has the parameter passing we want. So the client gets the convenient public method with the kind of parameter passing that the client wants, and we get our convenient private method with the kind of parameter passing that we want. We call the private method a helper method because it helps us to write the public method we've been asked to implement.

Here is the code that includes the private method:

```
public static int sum(int[] list) {

return sum(list, 0);

}



private
```

The public method simply calls the private method, passing it an appropriate value for the second parameter. Both methods can be called [redacted] because they have different signatures (one parameter versus two). We make the second method a private method because we don’t want to clutter up the public interface seen by the client.

There is a final benefit to this two-method approach. Sometimes you want to perform an action once, either before or after the recursive method executes. By dividing the recursive method into a public and a private method, you can include this one-time code in the public method either just before or just after the call on the private method.

12.4 Recursive Graphics

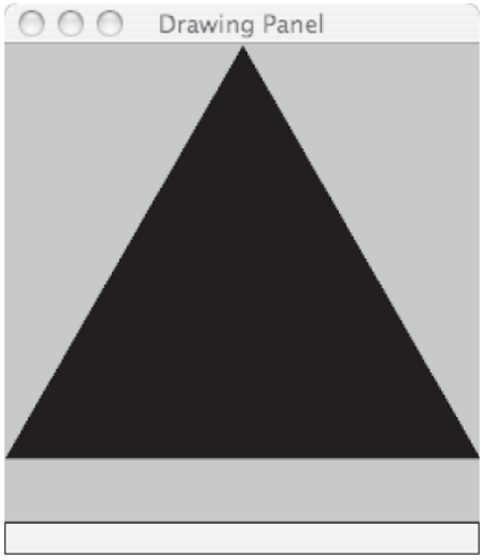
There has been a great deal of interest in the past 30 years about an emerging field of mathematics called *fractal geometry.* A fractal is a geometric object that is recursively constructed or self-similar. A fractal shape contains smaller versions of itself, so that it looks similar at all magnifications.

Benoit Mandelbrot founded the field of fractals in 1975 with his first publication about these intriguing objects, particularly a specific fractal that has come to be known as the Mandelbrot set. The most impressive aspect of fractal geometry is that extremely intricate and complex phenomena can be described with a simple set of rules. When Mandelbrot and others began drawing pictures of their fractals, they were an instant hit.

Many fractals can be described easily with recursion. As an example, we will explore a recursive method for drawing what is known as the Sierpinski triangle. We can't draw the actual fractal, because it is composed of infinitely many subtriangles. Instead, we will write a method that produces various levels that approximate the actual fractal.

At level 1, we draw an equilateral triangle as shown in Figure 12.1:

Figure 12.1 Sierpinski triangle, level 1



Proceeding to level 2, we draw three smaller triangles that are contained within the original triangle (see Figure 12.2):

Figure 12.2 Sierpinski triangle, level 2

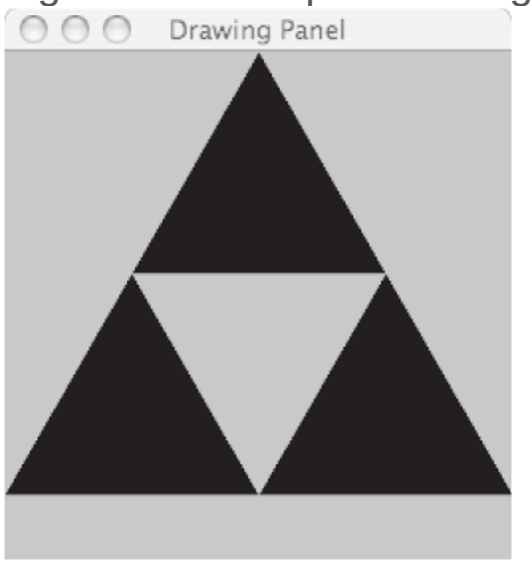


Figure 12.2 Full Alternative Text

We apply this principle in a recursive manner. Just as we replaced the original triangle with three inner triangles, we replace each of these three triangles with three inner triangles to obtain Figure 12.3 with nine triangles in level 3:

Figure 12.3 Sierpinski triangle, level 3

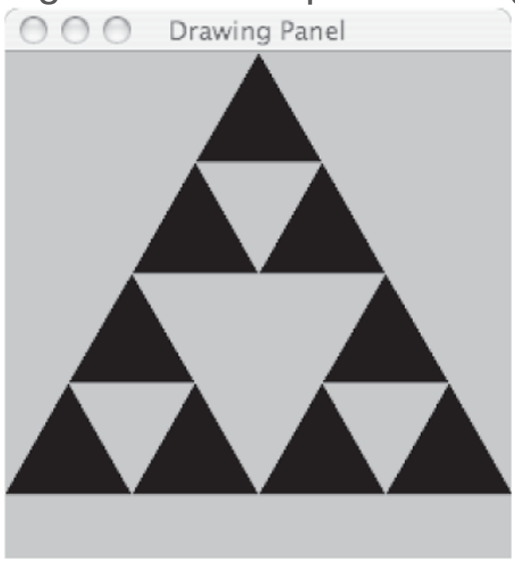


Figure 12.3 Full Alternative Text

This process continues indefinitely, making a more intricate pattern at each new level. Figure 12.4 shows the result at level 7:

Figure 12.4 Sierpinski triangle, level 7

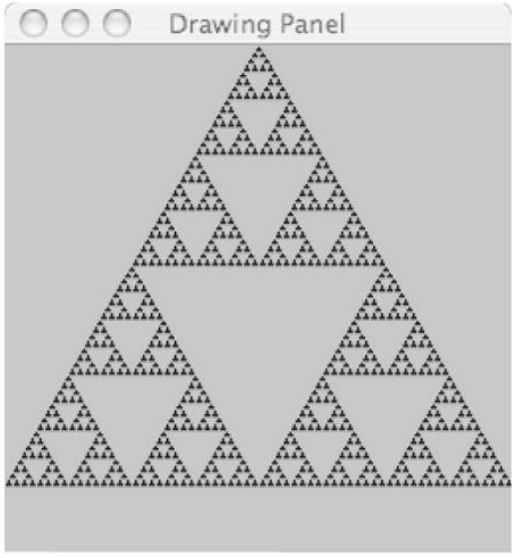


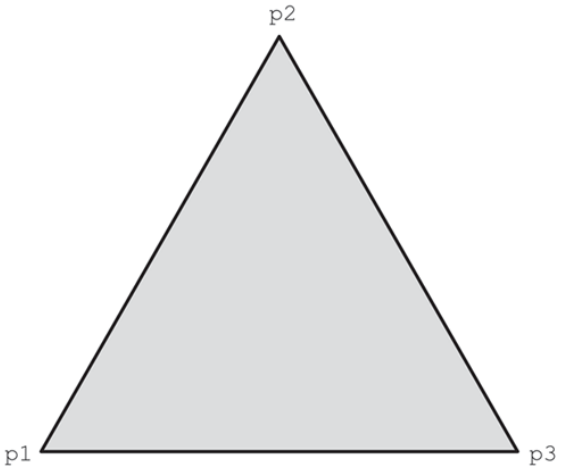
Figure 12.4 Full Alternative Text

We can solve this problem using the [redacted] class from Supplement 3G. We'll pass the [redacted] object for the panel to the method that is to draw the triangles. The method will also need to know the level to use and the three vertices of the triangle, which we can pass as [redacted] objects. Our method will look like this:

Our base case will be to draw the basic triangle for level 1. The [redacted] class has methods for filling rectangles and ovals, but not for filling triangles. Fortunately, there is a [redacted] class in the [redacted] package. We can construct a [redacted] and then add a series of points to it. Once we've specified the three vertices, we can use the [redacted] method of the [redacted] class to fill the polygon. Our base case will look like this:

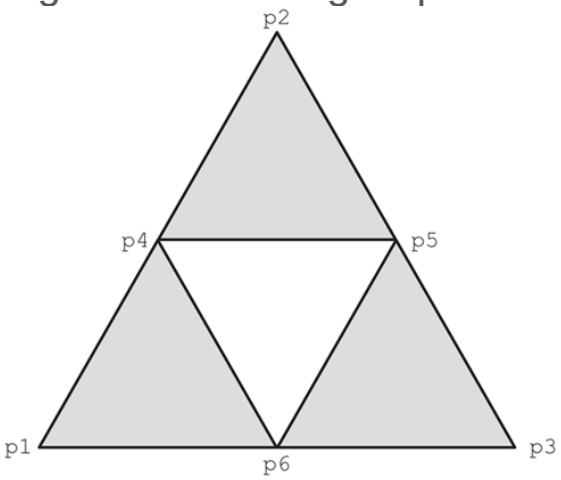
Most of the work happens in the recursive case. We have to split the triangle into three smaller triangles. We'll label the vertices of the overall triangle (see Figure 12.5).

Figure 12.5 Triangle before splitting



We then need to compute three new points that are the midpoints of the three sides of this triangle as shown in Figure 12.6.

Figure 12.6 Triangle split into subtriangles



From Figure 12.6, we see that

- p4 is the midpoint of p1 and p2
- p5 is the midpoint of p2 and p3
- p6 is the midpoint of p1 and p3

Once we have computed those points, we can describe the smaller triangles as follows:

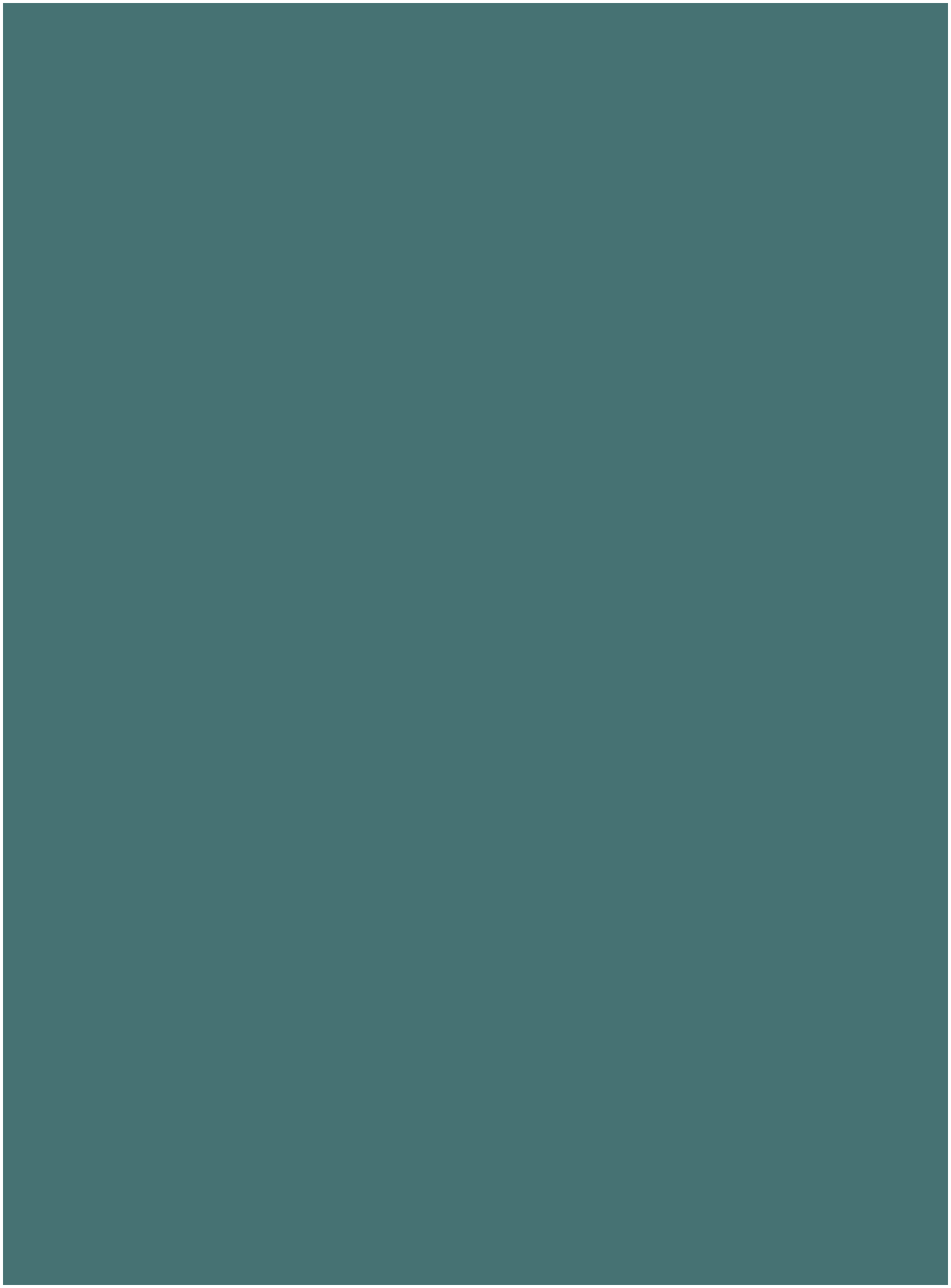
- In the lower-left corner is the triangle formed by p1, p4, and p6.
- On top is the triangle formed by p4, p2, and p5.
- In the lower-right corner is the triangle formed by p6, p5, and p3.

There are three different midpoint computations involved here, so clearly it will be helpful to first write a method that will compute the midpoint of a segment given two endpoints. Computing the midpoints involves finding the arithmetic average (halfway point) of the x values and the y values:

Given this method, we can easily compute the three midpoints.

The final detail we have to think about is the level. If you look again at the level 2 version of the figure (Figure 12.2), you will notice that it is composed of three simple triangles. In other words, the level 2 figure is composed of three level 1 figures. Similarly, the level 3 figure is composed of three level 2 figures, each of which in turn is composed of three level 1 figures. In general, if we are asked to draw a level figure, we do so by drawing three level figures.

We can turn these observations into code to complete the recursive method:



There is a limit to the number of levels down that we can go. The [redacted] has a finite resolution, so at some point we won't be able to subdivide our triangles any further. Also bear in mind that at each new level the number of triangles that we draw triples, which means that the number of triangles increases exponentially with the level.

The complete program is available from http://www.buildingjavaprograms.com. Remember that you need to also download [redacted] to execute the program.

12.5 Recursive Backtracking

Many programming problems can be solved by systematically searching a set of possibilities. For example, if you want to find a path through a maze from a starting point to an exit point, you can explore all possible paths through the maze until you find one that works. For many games like tic-tac-toe, you can explore all possible moves and countermoves to see if there is some move that guarantees that you win.

Many of these exhaustive search problems can be solved with an approach called *backtracking*. It is a particular approach to problem solving that is nicely expressed using recursion. As a result, it is sometimes referred to as *recursive backtracking*.

(Recursive) Backtracking

A general algorithm for finding solutions to a problem by exploring possible candidate solutions and abandoning (“backtracking”) once a given candidate is deemed unsuitable.

Backtracking involves searching all possibilities, so it can be an expensive technique to use. But many problems are small enough in scope that they are nicely solved with a backtracking approach.

A Simple Example: Traveling North/East

To introduce the basic concepts and terminology of backtracking, let's explore a simple example. Consider a standard Cartesian plane with (x, y) coordinates. Suppose that you start at the origin, (0, 0), and you are allowed to repeatedly make one of these three moves:

- You can move North (abbreviated "N"), which will increase your y-coordinate by 1.
- You can move East (abbreviated "E"), which will increase your x-coordinate by 1.
- You can move Northeast (abbreviated "NE"), which will increase both your x-coordinate and y-coordinate by 1.

Starting from the origin, these three different moves would leave you in the locations shown in Figure 12.7. We can think of this as a traveling problem where we can make a series of moves that take us from the origin to some other (x, y) point. For example, the sequence of moves N, NE, N would leave us at (1, 3).

Figure 12.7 Traveling north, east, or northeast from the origin

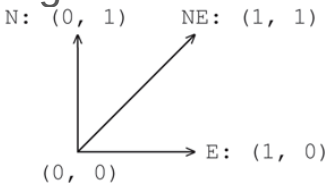


Figure 12.7 Full Alternative Text

Every backtracking problem involves a *solution space* of possible answers that you want to explore. We try to view the problem as a sequence of *choices*, which allows us to think of the solution space as a *decision tree*. For our traveling problem the choices are the sequence of moves that we make. Figure 12.8 shows a

decision tree showing all of the possible ways to make two moves and where each sequence leaves us. These decision trees can be quite large even for a small problem like this.

Figure 12.8 Decision tree for two moves

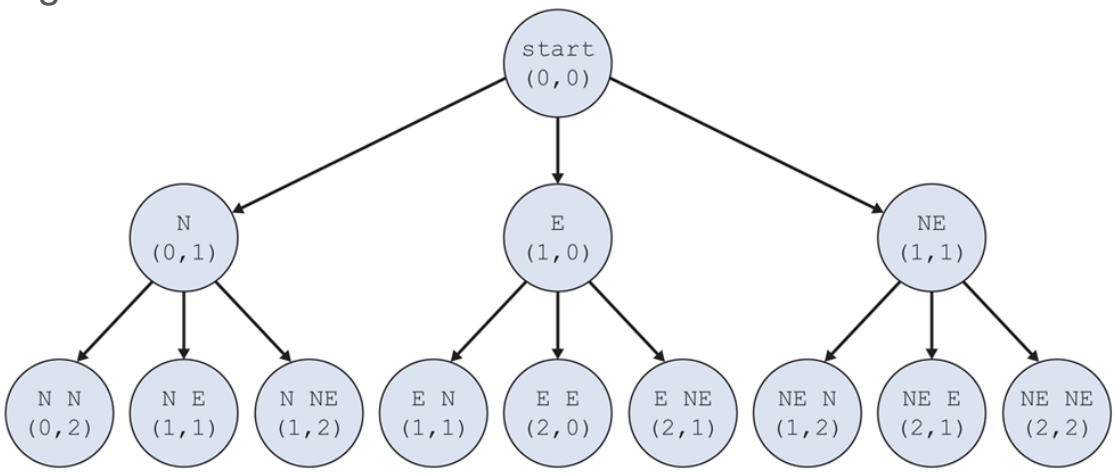


Figure 12.8 Full Alternative Text

Consider the problem of traveling from the origin to the point (1, 2). What are the possible sequences of moves that would get you there? People are fairly good at solving problems like these, so you can probably pretty easily come up with all five of the possibilities:

- N, N, E
- N, E, N
- N, NE
- E, N, N
- NE, N

How would we write a computer program to find these solutions? For this simple problem, we could devise a specialized algorithm that takes into account the properties of these paths, but backtracking provides a convenient way to exhaustively search all possibilities.

For most backtracking solutions we end up writing two methods. We generally want a public method that is passed the details of the problem to be solved. But we almost always need extra parameters for the backtracking, so we almost always have an extra private method that does the actual backtracking.

The basic form that our backtracking solution will take is a method to explore all possible choices recursively:

```
private static void explore(a scenario) {

    ...

}
```

Because we are using recursion, we need to identify base cases and recursive cases. Backtracking solutions generally involve two different base cases. You tend to stop the backtracking when you find a solution, so that becomes one of our base cases. Often what you want to do when you find a solution is to report it:

```
private static void explore(a scenario) {

    if (this is a solution) {

        report it.

    } else {

        ...
```

```
    }

}
```

We don’t want to search forever, so we also have to be on the lookout for what is called a *dead-end*. We might make a set of choices that lead us to a scenario where it is clear that no solution is possible with this set of choices. We make this a base case in our recursion so that we stop exploring when we reach a dead-end:

```
private static void explore(a scenario) {

    if (this is a solution) {

        report it.

    } else
if (this is not a dead-end)
 {

        ...

    }

}
```

If our backtracking search has led us to a scenario where we haven’t yet solved the problem and haven’t yet reached a dead-end, then we want to explore each possible choice available to

us. For each possible choice, we recursively explore the scenario of making that choice:

```
private static void explore(a scenario) {
    if (this is a solution) {
        report it.
    } else if (this is not a dead-end) {

use recursive calls to explore each available choice.
    }
}
```

This pseudocode captures the essence of the backtracking approach. Not all backtracking solutions will take exactly this form, but they will all have some variation of each of these elements.

We can flesh out the pseudocode a bit by filling in some details for this particular problem. We are considering the problem of moving from a current position to a target position, both specified with (x, y) coordinates. And we have three available moves from any given position: N, E, and NE:

```
private static void explore(current (x, y) and target (x, y)) {
```

```
    if (this is a solution) {

        report it.

    } else if (this is not a dead-end) {


explore(moving N).


explore(moving E).


explore(moving NE).

    }

}
```

Often it can be challenging to figure out what parameters to pass to the exploration method. In this case, we need a current x and y and a target x and y. We also need some way of keeping track of the choices that we have made so that we can report the path that we have taken. There are many ways to do this, but for simplicity, let's build up a string that stores the sequence of moves.

We can test whether we have a solution by testing whether the current and target coordinates match. But how do we test for a dead-end? In this problem, our x-coordinates and y-coordinates never go down. If we reach a point where the current x is greater than the target x or the current y is greater than the target y, then we know that we have traveled too far in that direction and we will never reach the target.

Putting these details together, we end up with the following exploration method:

We can complete the solution by providing a public method that starts the recursive search by passing the initial coordinates (0, 0) and an appropriate value to appear at the front of the string that stores the path:

If we ask the method to find all ways of traveling to (1, 2), it produces the following output:

Figure 12.9 shows the complete decision tree that is explored by this particular call with the five solutions shaded.

Figure 12.9 Complete decision tree for paths to (1, 2)

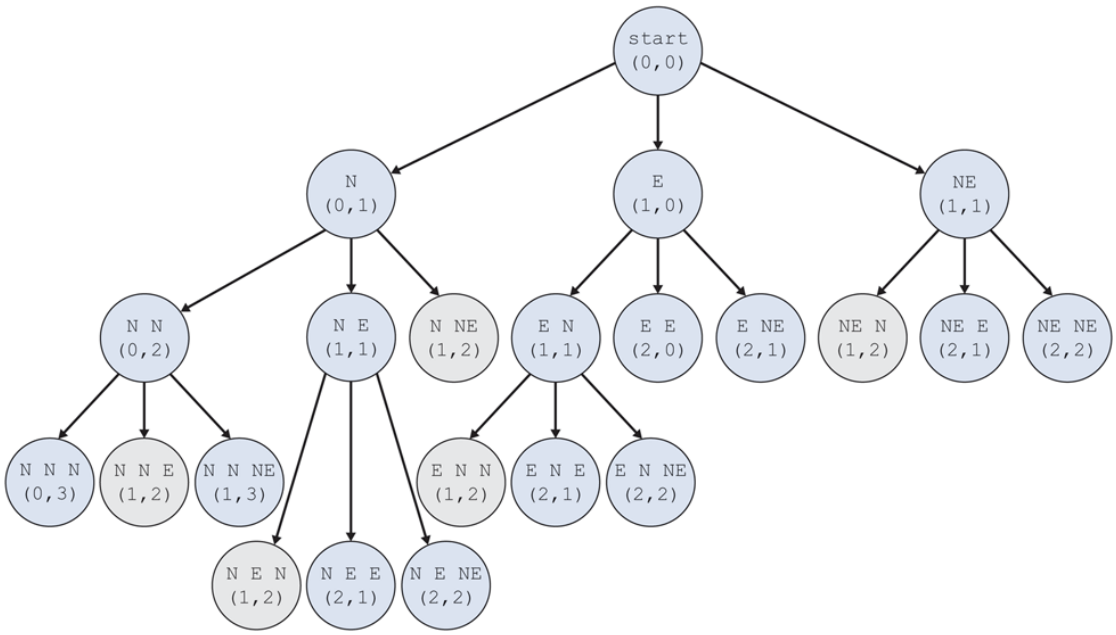


Figure 12.9 Full Alternative Text

Notice that the tree does not contain all of the possible combinations of three moves. That is because we don't explore past a solution and we don't explore past a dead-end (such as (2, 1)).

It is useful to consider the order in which the recursive method searches these possibilities. In general, as with most recursive solutions, the recursive calls lead to a depth-first traversal. From the starting point at the top of the tree it first considers moving North to (0, 1). That entire branch of the tree is explored before any other branches are explored. The next call considers moving North again to (0, 2). Then it considers another move North to (0, 3). So the initial sequence of calls leads it down to the leftmost node of the three. This is a dead-end, so it stops exploring. At that point, it returns to where it last had another choice to consider. It backs up to where it had chosen two North moves in a row (N N) and it considers moving East instead of moving North. That turns out to be a solution (N N E), so it reports a success. Then it backs

up again to where it last had another choice to make and it considers the remaining possibilities.

This property of going back to where it still has other choices to explore is the source of the term “backtracking.” As it finds solutions and dead-ends, it goes back (backtracks) to where it last had some other move to consider. It continues to search until it has exhausted all possible sequences of choices.

8 Queens Puzzle

A classic puzzle that backtracking solves well is to find a way to place 8 queens on a chessboard so that no two queens threaten each other. Because queens can move horizontally, vertically, or diagonally, it is a challenge to find a way to put 8 different queens on the board so that no two of them are in the same row, column, or diagonal. Figure 12.10 shows one example placement.

Figure 12.10 The 8 queens problem

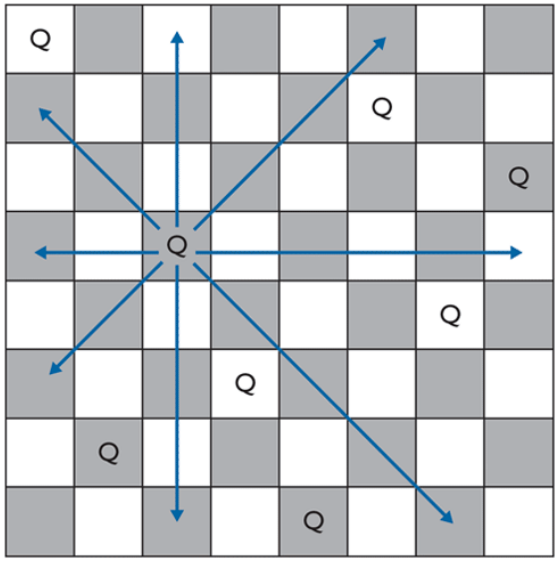


Figure 12.10 Full Alternative Text

To solve this problem with backtracking, we have to think of it in terms of a sequence of choices. The simplest way to do this is to think in terms of choosing where the first queen goes, then choosing where the second queen goes, and so on. There are 64 places to put the first queen because the chessboard is an 8-by-8 board. At the top of the tree, there are 64 different choices you could make for placing the first queen. Then once you've placed one queen, there are 63 squares left to choose from for the second queen, then 62 squares for the third queen, and so on.

Because backtracking searches all possibilities, it can take a long time to execute as there are many possibilities to explore. If we explore them all, we'll need to look at 64 * 63 * 62 * . . . * 57 states, which is too many even for a fast computer. We need to be as smart as we can about the choices we explore. In the case of 8 queens, we can do better than to consider 64 choices followed by 63 choices followed by 62 choices, and so on. We know that most of these aren't worth exploring.

One approach is to observe that if there is any solution at all to this problem, then the solution will have exactly one queen in

each row and exactly one queen in each column. That's because you can't have two in the same row or two in the same column and there are 8 of them on an 8-by-8 board. We can search more efficiently if we go row-by-row or column-by-column. It doesn't matter which choice we make, so let's explore column by column. Eliminating undesirable candidates from being explored is also called *pruning* the decision tree.

In this new way of looking at the search space, the first choice is for column 1. We have 8 different rows where we could put a queen in column 1. At the next level we consider all of the places to put a queen in column 2, and so on. Figure 12.11 shows the top of our decision tree.

Figure 12.11 Decision tree for first column

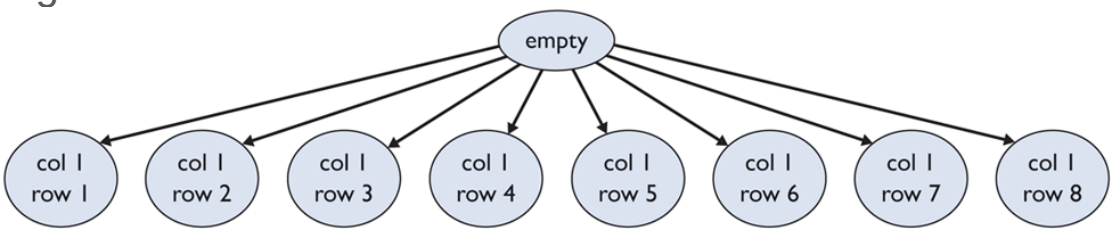


Figure 12.11 Full Alternative Text

There are eight different branches for each column. Under each of these branches, we have eight branches for each of the possible rows where we might place a queen in column 2. For example, if we think just about the possibility of placing the first queen in column 1 and row 5 and then think about all of the ways to place a second queen, we end up with an extra level of the tree as shown in Figure 12.12.

Figure 12.12 A decision tree for second column

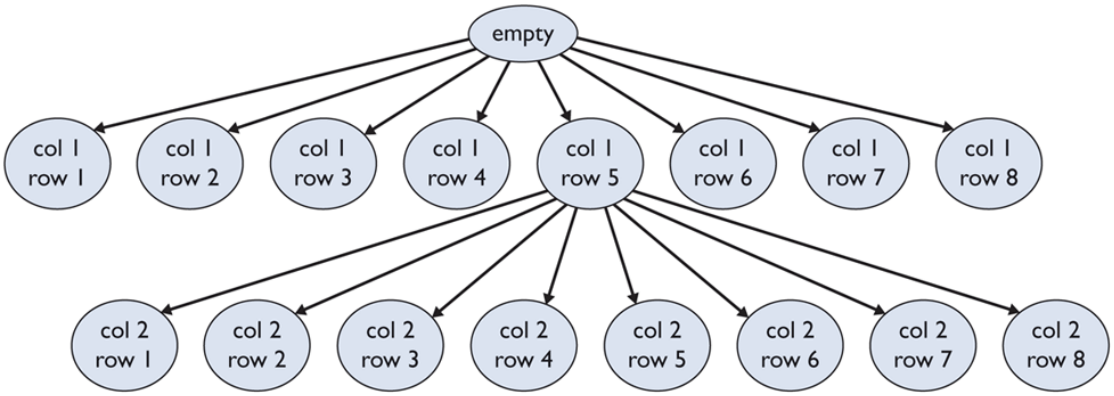


Figure 12.12 Full Alternative Text

These pictures don't capture the whole story because the tree is so large. There are eight branches at the top. From each of these eight branches there are eight branches. And from each of those branches there are eight branches. This continues eight levels deep (one level for each column of the board).

It's clear that the eight choices could be coded fairly nicely in a loop, something along the lines of:

But what we need for backtracking is something more like a deeply nested loop:

That’s not a bad way to think of what backtracking does, although we will use recursion to write this in a more elegant way.

Before we explore the backtracking code, we need to consider the low-level details of how to keep track of a board that allows us to place queens in specific locations. It is helpful to split off the low-level details into a separate class. Let’s plan on writing a [redacted] class that allows us to construct a [redacted] object to keep track of the state of the chessboard.

What kind of methods would we want to have for a [redacted] object? Obviously it would need some kind of constructor. We want to pass it an integer n so we could solve the more general “n queens” problem with an n-by-n board. We need to be able to test whether it’s safe to place a queen at a particular location. We need a way to place a queen on the board. We need a way to remove a queen because the backtracking involves trying different possibilities. We need a way to display output showing where the queens have been placed. Lastly, we’d like to be able to ask the board what size it is. These can be implemented as the following constructor and methods:

Let's assume that we have a [redacted] class that implements all of these methods. That's not the interesting code. The interesting code is the backtracking code which, given this class, we can now write in a very straightforward manner.

Once again we will introduce a public/private pair of methods to perform the backtracking. The public method can be passed a [redacted] object and it will call the private recursive method that performs the backtracking:

Recall that we had this general pseudocode for backtracking solutions:

```
private static void explore(a scenario) {

    if (this is a solution) {
```

```
        report it.

    } else if (this is not a dead-end) {

        use recursive calls to explore each available choice.

    }

}
```

The 8-queens backtracking problem differs in several important ways from the simple backtracking we saw before. Let's consider each of the differences and see how to adapt our pseudocode for each of them.

In the traveling problem we had just three possibilities to consider, so it made sense to write three recursive calls to explore each possibility. Here we have eight possibilities for the 8-queens problem and potentially a different number of possibilities if the board size is something other than 8. In this case, we want to use a loop to consider the different possibilities. Many backtracking problems will require using a loop instead of individual calls; it is useful to adapt our pseudocode to fit that approach.

```
private static void explore(a scenario) {

    if (this is a solution) {

        report it.

    } else {
```

```
for (each available choice) {

if (this is not a dead-end) {

 use a recursive call to explore that choice.

}

}

    }

}
```

We can also be more specific about what it means to explore a choice. In our simple example, we were building up a string that stored the path. That meant that we didn't have to undo a choice to move on to the next choice. More complex backtracking problems require a cleanup step where you undo a choice. That will be true of 8-queens. We will place a queen on the board to explore that branch and when we come back from the recursive exploration, we will need to remove the queen to get ready to explore the next possible choice. So the pseudocode can be expanded to include the pattern of making a choice, recursively exploring, and then undoing the choice:

```
private static void explore(a scenario) {

    if (this is a solution) {
```

```
        report it.
    } else {
        for (each available choice) {
            if (this is not a dead-end) {

make the choice.

recursively explore subsequent choices.

undo the choice.
            }
        }
    }
}
```

This pseudocode is general enough that it can be used for many backtracking problems. Some programmers refer to this as the “choose, explore, un-choose” pattern for backtracking.

This is appropriate code to use if you want to search all possibilities. In the case of 8-queens, there are many solutions (over 90 different solutions). We don’t really want to see all of these solutions. We’re happy to have just one. For this backtracking problem, we will consider a variation that stops when

it finds a solution. That means that our recursive method will need to have a way to let us know whether a certain path worked out or whether it turned out to be a dead-end. A good way to do this is to have a boolean return type and to have the method return true if it succeeds, false if it is a dead-end.

```
private static
boolean
 explore(a scenario) {

    if (this is a solution) {

        report it.


return true;

    } else {

        for (each available choice) {

            if (this is not a dead-end) {

                make the choice.


if (recursive call to explore subsequent choices) {


return true;


}

            undo the choice.
```

```
        }

    }

return false;

}
```

This pseudocode is also general enough that it can be used for many backtracking problems when you want to stop the process after finding a solution.

We can adapt this to the 8-queens problem by filling in the details. What parameters will it need to specify a scenario? It certainly needs the Board object. Recall that each level of the decision tree involves a different column of the board. The first invocation will handle column 1, the second will handle column 2, and so on. Therefore, in addition to the board, the method also needs to know the column to work on. That leaves us with this header:

```
(Board b, int col)
```

How do we know if we've found a solution? This backtracking code doesn't explore dead-ends. As a result, it has the following precondition:

What would be a nice column to get to? A tempting answer is 8. It would be nice to get to column 8 because it would mean that 7 of the 8 queens have been placed properly. But an even better answer is 9, because the precondition tells us that if we ever reach column 9, then queens have been safely placed in each of the first 8 columns.

This turns out to be our test for whether we have found a solution:

The pseudocode indicates that we should print the answer before returning true. We could do that, but because the solution is stored in the object, we can simply return and allow the calling method to print out the solution.

Now we have to fill in the details of the loop that explores the various possibilities. We have eight possibilities to explore (the eight rows of this column where we might place a queen). A loop works nicely to explore the different row numbers. The pseudocode indicates that we should test to make sure it is not a dead-end. We can do that by making sure that it is safe to place a queen in that row and column:

```
if (b.isSafe(row, col)) {



}
```

We now need to fill in the three steps that are involved in exploring a choice: making the choice, recursively exploring subsequent choices, and undoing the choice. We make the choice by telling the board to place a queen in that row and column:

```
b.place(row, col);
```

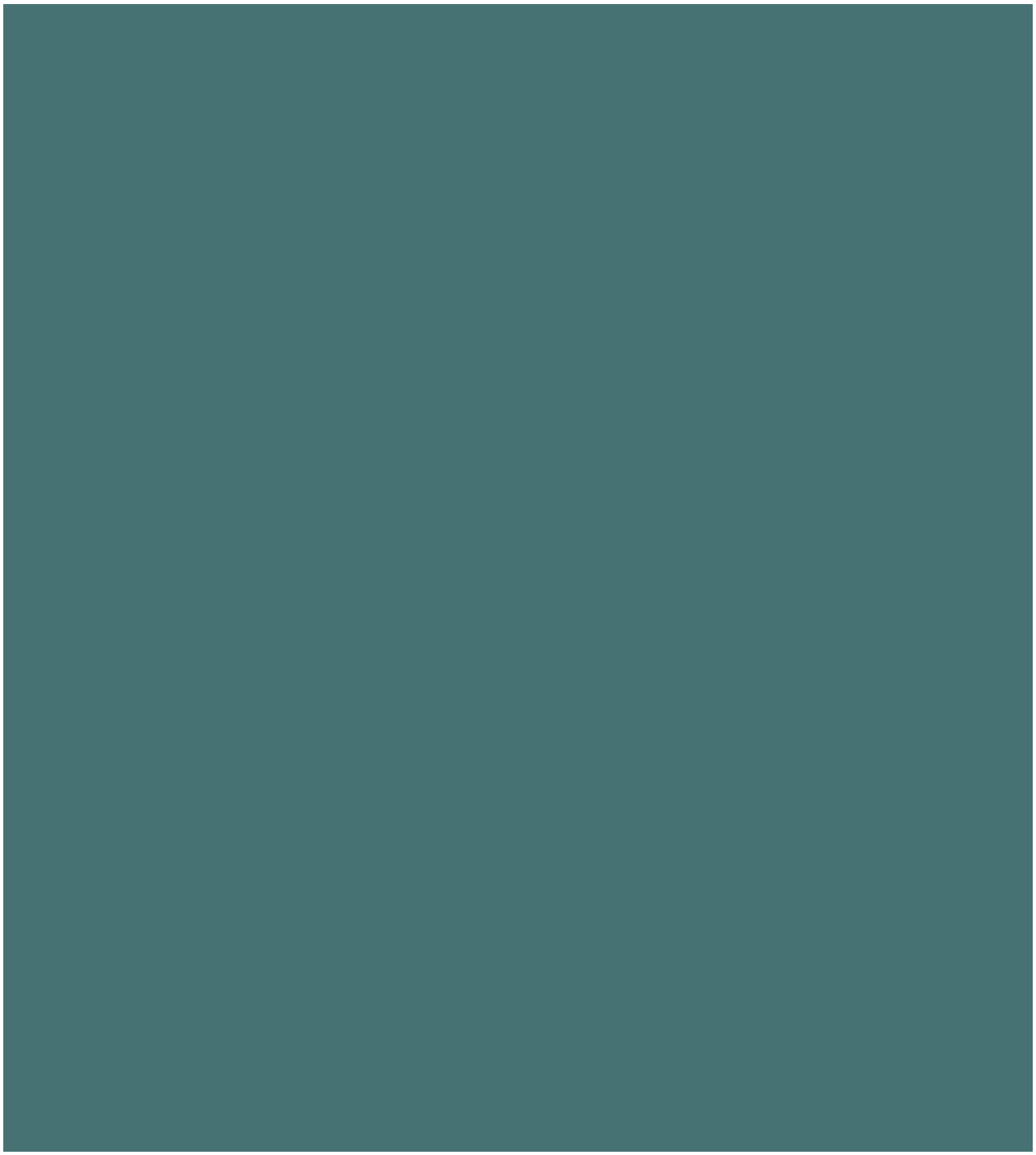
Then we recursively explore subsequent choices (later columns) and return true to stop the process if it finds a solution:

```
if (explore(b, col + 1)) {

return true;

}
```

Finally, we have to undo our choice in case that turns out to be a dead-end:

```
b.remove(row, col);
```

Putting it all together we get:



We need some code in the [redacted] method that starts the recursion in column 1 and that either prints the solution or a message about there not being solutions:

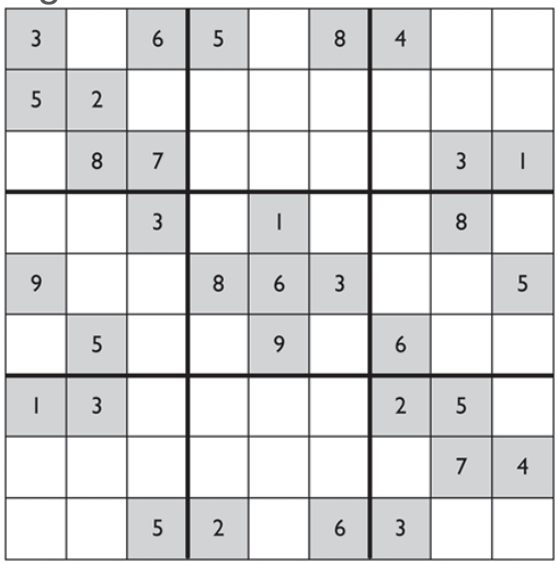
On our web site http://www.buildingjavaprograms.com/ you will find the [redacted] class and a full runnable version of the program that you can run to see that it finds a solution to 8 queens, 4 queens, and so on. In the case of 2 queens, there are no solutions. Our graphical variation of the program includes an animation generated by the backtracking itself.

Solving Sudoku Puzzles

Let's look at one more example of backtracking. Sudoku puzzles have become very popular. The puzzle involves a 9-by-9 grid that is to be filled in with the digits 1 through 9. Each row and column is required to have exactly one occurrence of each of the nine

digits. The grid is further divided into nine 3-by-3 grids, and each grid is also required to have exactly one occurrence of each of the nine digits. A specific Sudoku puzzle will fill in some of the cells of the grid with specific digits, such as the puzzle shown in Figure 12.13.

Figure 12.13 A Sudoku board



When people play Sudoku, they use all sorts of heuristics and intuitions to find a way to fill in the empty cells of the grid while still meeting the constraints. With a backtracking approach, we just use brute force to try all possibilities.

The choices involved are what digit to put in each of the unoccupied cells of the grid. There are nine possibilities to explore for each empty cell (the digits 1 through 9). This approach to Sudoku is not necessarily going to run quickly, but backtracking provides a framework for quickly writing a complete solution.

As with many backtracking tasks, the real work involves keeping track of the details of this particular scenario. With the 8-queens problem we introduced a [redacted] class to keep track of the

chessboard. In this case we should develop a [redacted] class that keeps track of the state of the Sudoku grid.

In the case of the Sudoku grid, we have to start with some of the cells filled in. That will mean that our [redacted] class will need to read the initial configuration. Let's assume that each puzzle is stored in a text file with a 9-by-9 grid of numbers and dashes, as in:

That means that our [redacted] class constructor will need to read the initial configuration:

We immediately run into another problem. If some of the cells are filled in, then it's not as simple as the 8-queens case where we could just systematically explore each different cell of an empty board. In the Sudoku case, we have to figure out which cell to explore first, which cell to go to after that, and so on. There are many ways we could approach this, but the simplest is to give this responsibility to the [redacted] class. We can introduce a method that allows us to ask the grid to give us the next unassigned location in the grid.

Then we are faced with the question of how to specify a grid location. An obvious approach would be to give row and column numbers, but a method can return only one value. If the grid is going to identify the unassigned locations, then we don't really need to keep track of rows and columns ourselves. To keep thing simple, let's assume that the grid returns a cell number as a simple [redacted]:

Eventually the grid will fill up, so we have to adopt a convention for what happens when there is no unassigned location left. Let's assume that the method returns [redacted] in that case to indicate that the grid is full.

In addition to these methods, we will want similar methods to the ones we had in 8-queens. We want to be able to test whether it's safe to set a particular cell to a particular digit. This is the method that will make sure that the grid we are building up is legal. So it will have to check to see if this new digit doesn't already appear in the given row, column, or subgrid. We'll need a method to set a cell to a specific digit. An "undo" method to remove a digit from a cell will be necessary. We will also want a method to print our solution. These are implemented as the following methods, respectively:

As with 8-queens, we'll want to stop searching once we find a solution, so we start with the standard pseudocode for the single-solution backtracking:

```
private static boolean explore(a scenario) {

    if (this is a solution) {

        report it.

        return true;
```

```
    } else {
        for (each available choice) {
            if (this is not a dead-end) {
                make the choice.
                if (recursive call to explore subsequent choices) {
                    return true;
                }
                undo the choice.
            }
        }
        return false;
    }
}
```

In this case, the information about a scenario is stored in the grid object. We know that we have a solution if we manage to fill every cell of the grid. To test whether we have a solution we can ask the grid to give us the next unassigned location and see if the method returns (the value that signals that the grid is full):

```
```

The available choices are the digits 1 through 9 and we have methods for performing each of the operations described in the pseudocode, so we can fill in the blanks to complete the method as follows:

You will find the [redacted] class and a full runnable version of this program on our web site http://www.buildingjavaprograms.com/ along with a graphical version that shows the choices as they are being considered by the backtracking.

12.6 Case Study: Prefix Evaluator

In this section, we will explore the use of recursion to evaluate complex numeric expressions. First, we will examine the different conventions for specifying numeric expressions and then we will see how recursion makes it relatively easy to implement one of the standard conventions.

Infix, Prefix, and Postfix Notation

When we write numeric expressions in a Java program, we typically put numeric operators like ▮ and ▮ between the two operands, as in the following:

Putting the operator between the operands is a convention known as *infix notation.* A second convention is to put the operator in front of the two operands, as in the following examples:

Putting the operator in front of the operands is a convention known as *prefix notation.* Prefix notation looks odd for symbols like and , but it resembles mathematical function notation, in which the name of the function goes first. For example, if we were calling methods instead of using operators, we would write

There is also a third convention, in which the operator appears after the two operands, as in the following examples:

This convention is known as *postfix notation.* It is also sometimes referred to as reverse Polish notation, or RPN. For many years Hewlett-Packard has sold scientific calculators that use RPN rather than normal infix notation.

We are so used to infix notation that it takes a while to get used to the other two conventions. One of the interesting facts you will discover if you take the time to learn the prefix and postfix conventions is that infix is the only notation that requires parentheses. The other two notations are unambiguous.

Table 12.1 summarizes the three notations.

Table 12.1 Arithmetic Notations

infix	operator between operands	

prefix	operator before operands (functional notation)	
postfix	operator after operands (reverse Polish notation)	

Evaluating Prefix Expressions

Of the three standard notations, prefix notation is most easily implemented with recursion. In this section, we will write a method

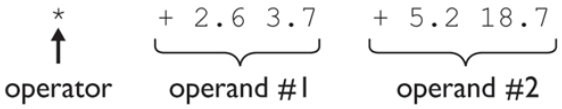
that reads a prefix expression from a [redacted] and computes its value. Our method will look like this:

Before we can begin writing the method, we have to consider the kind of input that we are going to get. As the precondition indicates, we will assume that the [redacted] contains a legal prefix expression. The simplest possible expression would be a number like

There isn’t much to evaluate in this case—we can simply read and return the number. More complex prefix expressions will involve one or more operators. Remember that the operator goes in front of the operands in a prefix expression. A slightly more complex expression would be two numbers as operands with an operator in front:

This expression could itself be an operand in a larger expression. For example, we might ask for

At the outermost level, we have a multiplication operator with two operands:



12.6-28 Full Alternative Text

In other words, this expression is computing the product of two sums. Here is the same expression written in the more familiar infix notation:

These expressions can become arbitrarily complex. The key observation to make about them is that they all begin with an operator. In other words, every prefix expression is of one of two forms:

- A simple number

- An operator followed by two operands

This observation will become a road map for our recursive solution. The simplest prefix expression will be a number, and we can distinguish it from the other case because any other expression will begin with an operator. So we can begin our recursive solution by checking whether the next token in the [redacted] is a number. If it is, we have a simple case and we can simply read and return the number:

Turning our attention to the recursive case, we know that the input must be an operator followed by two operands. We can begin by reading the operator:

```
String operator = input.next();
```

At this point, we reach a critical decision. We have read in the operator, and now we need to read in the first operand and then the second operand. If we knew that the operands were simple numbers, we could write code like the following:

```
double operand1 = input.nextDouble();


double operand2 = input.nextDouble();
```

But we have no guarantee that the operands are simple numbers. They might be complex expressions that begin with operators. Your instinct might be to test whether or not the original operator is followed by another operator (in other words, whether the first operand begins with an operator), but that reasoning won't lead you to a satisfactory outcome. Remember that the expressions can be arbitrarily complex, so either of the operands might contain dozens of operators to be processed.

The solution to this puzzle involves recursion. We need to read two operands from the [redacted], and they might be very complex. But we know that they are in prefix form and we know that they aren't as complex as the original expression we were asked to

evaluate. The key is to recursively evaluate each of the two operands:

```
evaluate(input);

evaluate(input);
```

This simple solution works. Of course, we still have the task of evaluating the operator. After the two recursive calls have executed, we will have an operator and two numbers (say, `*`, `3.4`,

and `2.6`). It would be nice if we could just write a statement like the following:

Unfortunately, Java doesn't work that way. We have to use a nested statement to test what kind of operator we have and to return an appropriate value:

We can include this code in its own method so that our recursive method stays fairly short:

```
return apply(operator, operand1, operand2);
```

Complete Program

When you program with recursion, you'll notice two things. First, the recursive code that you write will tend to be fairly short, even though it might be solving a very complex task. Second, most of your program will generally end up being supporting code for the recursion that does low-level tasks. For our current task of evaluating a prefix expression, we have a short and powerful prefix evaluator, but we need to include some supporting code that explains the program to the user, prompts for a prefix

expression, and reports the result. We also found that we needed a method that would apply an operator to two operands. The nonrecursive parts of the program are fairly straightforward, so they are included in the following code without detailed discussion:

```
import


public class


public static void
```

```
new
```

```
double
```

```
public static double



if



return



else









double



double



return
```

```
public static double

double



double



if



return
```

```
else if

return

else if

return

else if

return

else if

return
```

```
else

throw new
```

The program can handle simple numbers, as in the following sample execution:

It can also handle expressions with a single operator, as in the following execution:

And it handles the case we considered that involved a product of two sums:

In fact, it can handle arbitrarily complex expressions, as in the following sample execution:

The expression being computed in the previous example is the prefix equivalent of the following infix expression:

Chapter Summary

Recursion is an algorithmic technique in which a method calls itself. A method that uses recursion is called a recursive method.

Recursive methods include two cases: a base case that the method can solve directly without recursion, and a recursive case in which the method reduces a problem into a simpler problem of the same kind using a recursive call.

Recursive method calls work internally by storing information about each call into a structure called a call stack. When the method calls itself, information about the call is placed on top of the stack. When a method call finishes executing, its information is removed from the stack and the program returns to the call underneath.

A recursive method without a base case, or one in which the recursive case doesn’t properly transition into the base case, can lead to infinite recursion.

A helper method is written to help solve a subtask of an overall problem. Recursive helper methods often have parameters in addition to the ones passed to the overall recursive method that calls them, to allow them to more easily implement the overall recursive solution.

Recursion can be used to draw graphical figures in complex patterns, including fractal images. Fractals are images that are recursively self-similar, and they are often referred to as “infinitely complex.”

Self-Check Problems

Section 12.1: Thinking Recursively

1. What is recursion? How does a recursive method differ from a standard iterative method?
2. What are base cases and recursive cases? Why does a recursive method need to have both?
3. Consider the following method:

For each of the following calls, indicate the output that is produced by the method:

a.
b.
c.
d.
e.
f.

g. [redacted]

4. Consider the following method:

[redacted]

For each of the following calls, indicate the output that is produced by the method:

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]

5. Consider the following method:

[redacted]

For each of the following calls, indicate the output that is produced by the method:

a.
b.
c.
d.
e.

6. Consider the following method:

For each of the following calls, indicate the output that is produced by the method:

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]

7. Convert the following iterative method into a recursive method:

Section 12.2: A Better Example of Recursion

8. What is a call stack, and how does it relate to recursion?
9. What would be the effect if the code for the [redacted] method were changed to the following?

10. What would be the effect if the code for the [redacted] method were changed to the following?

Section 12.3: Recursive Functions and Data

11. The following method is an attempt to write a recursive [redacted] method to compute exponents. What is wrong with the code? How can it be fixed?

12. What are the differences between the two versions of the [redacted] method shown in Section 12.3? What advantage does the second version have over the first version? Are both versions recursive?
13. Consider the following method:

For each of the following calls, indicate the value that is returned:

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]

14. Consider the following method:

For each of the following calls, indicate the value that is returned:

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]−23, −48)
e. [redacted]

15. Consider the following method:

[redacted]

For each of the following calls, indicate the value that is returned:

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]

16. Convert the following iterative method into a recursive method:

17. The following method has a bug that leads to infinite recursion. What correction fixes the code?

18. Sometimes the parameters that a client would like to pass to a method don't match the parameters that are best for writing a recursive solution to the problem. What should a programmer do to resolve this issue?
19. The Fibonacci sequence is a sequence of numbers in which the first two numbers are 1 and each subsequent number is the sum of the previous two Fibonacci numbers. The sequence is 1, 1, 2, 3, 5, 8, 13, 21, 34, and so on. The following is a correct, but inefficient, method to compute the nth Fibonacci number:

The code shown runs very slowly for even relatively small values of ; it can take minutes or hours to compute even the 40th or 50th Fibonacci number. The code is inefficient because it makes too many recursive calls. It ends up recomputing each Fibonacci number many times. Write a new version of this method that is still recursive and has the same header but is more efficient. Do this by creating a helper method that accepts additional parameters, such as previous Fibonacci numbers, that you can carry through and modify during each recursive call.

Section 12.4: Recursive Graphics

20. What is a fractal image? How does recursive programming help to draw fractals?
21. Write Java code to create and draw a regular hexagon (a type of polygon).

Section 12.5: Recursive Backtracking

22. Why is recursion an effective way to implement a backtracking algorithm?
23. What is a decision tree? How are decision trees important for backtracking?
24. Draw the decision tree that would have resulted for Figure 12.9 if the backtracking solution had explored NE first instead of last in the recursive explore method. (Hint: the tree changes at every level.)
25. The original North/East backtracking solution printed the following ways of traveling to (1, 2) in this order. In what order would they be printed if the solution had explored NE first instead of last?

26. Figure 12.12 shows only part of the decision tree for the first two levels. How many entries are there at the second level of the full tree? How many are at level 3 of the full tree?
27. If our 8 Queens algorithm tried every possible square on the board for placing each queen, how many entries

are there at the 8th and final level of the full tree? What does our algorithm do to avoid having to explore so many possibilities?

28. The 8 Queens [redacted] method stops once it finds one solution to the problem. What part of the code causes the algorithm to stop once it finds a solution? How could the code be modified so that it would find and output every solution to the problem?

Exercises

1. Write a recursive method called [redacted] that accepts an integer as a parameter and prints to the console a string of stars (asterisks) that is 2n (i.e., 2 to the nth power) long. For example,
   - [redacted] should print [redacted] (because 20 == 1)
   - [redacted] should print [redacted] (because 21 == 2)
   - [redacted] should print [redacted] (because 22 == 4)
   - [redacted] should print [redacted] (because 23 == 8)
   - [redacted] should print [redacted] (because 24 == 16)

The method should throw an [redacted] if passed a value less than [redacted].

2. Write a method called [redacted] that takes an integer [redacted] as a parameter and prints to the console the first n integers starting with 1 in sequential order, separated by commas. For example, consider the following calls:

[redacted]

These calls should produce the following output:

Your method should throw an [redacted] if passed a value less than [redacted].

3. Write a method called [redacted] that accepts an integer n as a parameter and prints to the console a symmetric sequence of n numbers composed of descending integers that ends in 1, followed by a sequence of ascending integers that begins with 1. The following table indicates the output that should be produced for various values of n:

Notice that when n is odd the sequence has a single 1 in the middle, whereas for even values it has two 1s in the middle. Your method should throw an [redacted] if it is passed a value less than [redacted].

4. Write a recursive method called [redacted] that accepts an integer n as a parameter and returns the integer obtained by replacing every digit of n with two of that digit. For example, [redacted] should return [redacted]. The call [redacted] should return [redacted]. Calling [redacted] on a negative number should return the negation of calling [redacted] on the corresponding positive number; for example, [redacted] should return [redacted].
5. Write a recursive method called [redacted] that accepts an integer as a parameter and writes its binary representation to the console. For example, [redacted] should print [redacted].
6. Write a recursive method called [redacted] that accepts an integer parameter n and prints the first n squares separated by commas, with the odd squares in descending order followed by the even squares in ascending order. For example, [redacted] prints the following output:

[redacted]

A call of [redacted] prints [redacted]. The method should throw an [redacted] if it is passed a value less than 1.

7. Write a recursive method called [redacted] that accepts an integer parameter n and that prints out a total of n characters. The middle character of the output should always be an asterisk ([redacted]). If you are asked to write out an even number of characters, then there will be two asterisks in the middle ([redacted]). Before the asterisk(s) you should write out less-than characters ([redacted]). After the asterisk(s) you should write out greater-than characters ([redacted]). Your method should throw an [redacted] if it is passed a value less than 1. For example, the following calls produce the following output:

[redacted]	
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]

8. Write a recursive method called that returns the product of the first n even integers. For example, returns and returns (because 2*4*6*8=384). The method should throw an if it is passed a value less than or equal to 0.
9. Write a recursive method called that accepts an integer parameter n and returns a real number representing the sum of the first n reciprocals. In other words, returns (1+1/2+1/3+1/4+...+1/n). For example, should return `1.5`. The method should return `0.0` if it is passed the value 0 and throw an if it is passed a value less than 0.
10. Write a recursive method called that accepts two nonnegative integers as parameters and that returns the number of digits that match between them. Two digits match if they are equal and have the same position relative to the end of the number (i.e., starting with the ones digit). In other words, the method should compare the last digits of each number, the second-to-last digits of each number, the third-to-last digits of each number, and so forth, counting how many pairs match. For example, for the call of , the method would compare as follows, and return because four of the pairs match (2-2, 5-5, 8-8, and 1-1).

11. Write a recursive method called [redacted] that accepts a string s and an integer n as parameters and that returns s concatenated together n times. For example, [redacted] returns [redacted], and [redacted] returns [redacted], and [redacted] returns [redacted]. String concatenation is an expensive operation, so for an added challenge try to solve this problem while performing fewer than n concatenations.
12. Write a recursive method called [redacted] that accepts two strings as parameters and returns [redacted] if the two strings contain the same sequence of characters as each other but in the opposite order (ignoring capitalization), and [redacted] otherwise. For example, the call of [redacted] would return [redacted]. The empty string, as well as any one-letter string, is considered to be its own reverse.
13. Write a recursive method called [redacted] that accepts two strings as parameters and that returns the starting index of the first occurrence of the second string inside the first string (or −1 if not found). For example, the call of [redacted] would return 8. (Strings already have an [redacted] method, but you may not call it in your solution.)

14. Write a recursive method called [redacted] that takes a string as a parameter and that returns a new string obtained by replacing every sequence of repeated adjacent letters with just one of that letter. For example, the string [redacted] has three repeated adjacent letters ("oo", "kkkkk", and "ee"), so the call of [redacted] should return [redacted]. Do not call the string [redacted] or [redacted] methods in your solution.
15. Write a recursive method [redacted] that takes a string as a parameter and returns a string in which all of the vowels have been moved to the end. The vowels should appear in reverse order of what they were in the original word. For example, the call of [redacted] should return [redacted].
16. Write a recursive method called [redacted] that accepts an integer parameter and that returns the integer formed by removing the odd digits from it. For example, [redacted] returns 8426 and [redacted] returns [redacted]. If the number is 0 or has no even digits, such as 35159 or 7, return [redacted]. Leading zeros in the result should be ignored.
17. Write a recursive method called [redacted] that accepts two integers n and r as parameters and returns the number of unique permutations of r items from a group of n items. For given values of n and r, this value `P(n, r)` can be computed as follows:

$$P(n,r)=n!(n-r)!$$

For example, [redacted] should return [redacted]. It may be helpful to note that [redacted] returns [redacted], or 840 / 7.

18. The Sierpinski carpet is a fractal that is defined as follows: The construction of the Sierpinski carpet begins with a square. The square is cut into nine congruent subsquares in a 3-by-3 grid, with the central subsquare removed. The same process is then applied recursively to the eight other subsquares. Figure 12.14 shows the first few iterations of the carpet.

Figure 12.14 Sierpinski carpet

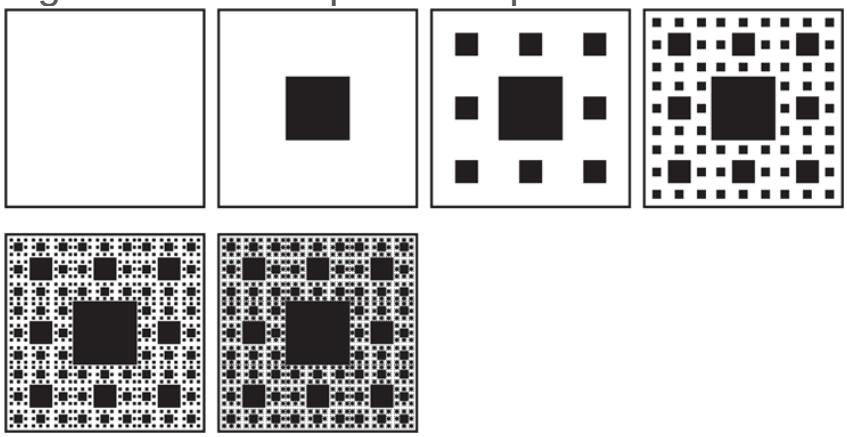


Figure 12.14 Full Alternative Text

Write a program to draw the carpet on a [redacted] recursively.

19. The Cantor set is a fractal that is defined by repeatedly removing the middle thirds of line segments as shown in Figure 12.15.

Figure 12.15 Cantor set

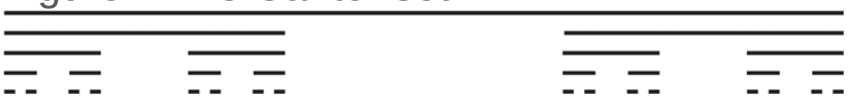


Figure 12.15 Full Alternative Text

Write a program to draw the Cantor set on a [redacted] recursively.

20. Write a recursive method called [redacted] that takes a positive integer value representing a number of stairs and prints each unique way to climb a staircase of that height, taking strides of one or two stairs at a time. Do not use any loops. Output each way to climb the stairs on its own line, using a 1 to indicate a small stride of 1 stair, and a 2 to indicate a large stride of 2 stairs. The order in which you output the possible ways to climb the stairs is not important, so long as you list the right overall set of ways. For example, the call [redacted]; should produce the following output:

[redacted]

The call [redacted]; should produce the following output:

[redacted]

21. Write a recursive method called [redacted] that accepts an integer n as a parameter and that prints all binary numbers that have exactly n digits in ascending order, each on its own line. All n digits should be shown for all numbers, including leading zeros if necessary. Assume that n is nonnegative. If n is 0, a blank line should be produced.

[redacted]	
[redacted]	[redacted]
	[redacted]
[redacted]	[redacted]
	[redacted]
	[redacted]
	[redacted]
[redacted]	[redacted]
	[redacted]
	[redacted]
	[redacted]
	[redacted]
	[redacted]

22. Write a recursive method called [redacted] to find every possible sub-list of a given list. A sub-list of a list L contains 0 or more of L's elements. Your method should accept a list of strings as its parameter and print every sub-list that could be created from elements of that list, one per line. For example, if the list stores [redacted], the output from your method would be:

The order in which you show the sub-lists does not matter, and the order of the elements of each sub-list also does not matter. The key thing is that your method should produce the correct overall set of sub-lists as its output. Notice that the empty list is considered one of these sub-lists. You may assume that the list passed to your method is not null and that the list contains no duplicates. Do not use any loops.

23. Write a recursive method called [redacted] that accepts a list of integers, L, and an integer limit n as parameters and uses backtracking to find the maximum sum that can be generated by adding elements of L that do not exceed n. For example, if you are given the list [redacted] and the limit of 19, the maximum sum that can be generated that does not exceed is 16, achieved by adding 7, 8, and 1. If the list L is empty, or if the limit is not a positive integer, or all of L's values exceed the limit, return [redacted].

Each index's element in the list can be added to the sum only once, but the same number value might occur more than once in a list, in which case each occurrence might be added to the sum. For example, if the list is [redacted] you may use up to one 6 in the sum, but if the list is [redacted] you may use up to two sixes.

You may assume that all values in the list are nonnegative. Your method may alter the contents of the list L as it executes, but L should be restored to its original state before your method returns. Do not use any loops.

24. Write a recursive method called [redacted] to find all ways to express an integer as a sum of squares of unique positive integers. For example, the call [redacted] should produce the following output:

[redacted]

Some numbers (such as 128 or 0) cannot be represented as a sum of squares, in which case your method should produce no output. Keep in mind that the sum has to be formed with unique integers. Otherwise you could always find a solution by adding 12 together until you got to whatever number you are working with. As with any backtracking problem, this one amounts to a set of choices, one for each integer whose square might or might not be part of your sum. You may generate the choices by doing a for loop over an appropriate range of numbers. Note that the maximum possible integer that can be part of a sum of squares for an integer n is the square root of n.

Programming Projects

1. Write a recursive program to solve the “Missionaries and Cannibals” problem. Three missionaries and three cannibals come to a river and find a boat that holds two. If the cannibals ever outnumber the missionaries on either bank, the missionaries will be eaten. How might they cross safely?

Your output should include the initial problem, the moves you make, and a “picture” of the current state of the puzzle after each move. Your final output should produce only the moves and the picture for the states that are on the solution path.

2. Write a recursive program to solve the Towers of Hanoi puzzle. The puzzle involves manipulating disks that you can move between three different towers. You are given a certain number of disks (four in this example) stacked on one of the three towers. The disks have decreasing diameters, with the smallest disk on the top (see Figure 12.16).

Figure 12.16 Towers of Hanoi

Figure 12.16 Full Alternative Text

The object of the puzzle is to move all of the disks from one tower to another (say, from A to B). The third tower is provided as a temporary storage space as you move disks around. You are

allowed to move only one disk at a time, and you are not allowed to place a disk on top of a smaller one (i.e., one with a smaller diameter).
Examine the rather simple solutions for one, two, and three disks, and see if you can discern a pattern. Then write a program that will solve the Towers of Hanoi puzzle for any number of disks. (Hint: Moving four disks is a lot like moving three disks, except that one additional disk is on the bottom.)

3. Write a recursive program to generate random sentences from a given BNF grammar. A BNF grammar is a recursively defined file that defines rules for creating sentences from tokens of text. Rules can be recursively self-similar. The following grammar can generate sentences such as “Fred honored the green wonderful child”:

4. The Koch snowflake is a fractal that is created by starting with a line segment, then recursively altering it as follows:
    1. Divide the line segment into three segments of equal length.
    2. Draw an equilateral triangle that has the middle segment from step 1 as its base.
    3. Remove the line segment that is the base of the triangle from step 2.

Figure 12.17 shows the first several iterations of the snowflake.

Figure 12.17 Koch snowflake

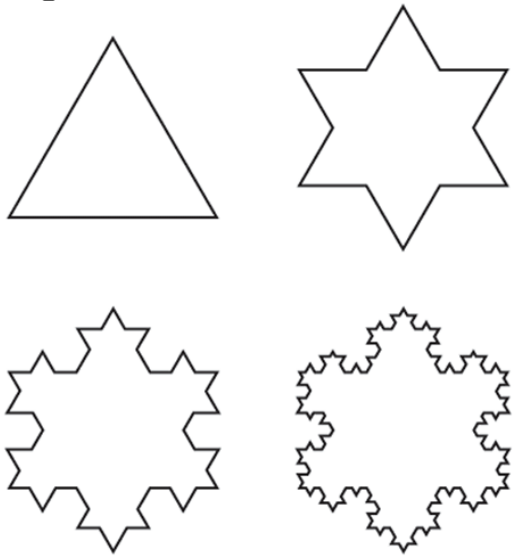


Figure 12.17 Full Alternative Text

Write a program to draw the Koch snowflake on a [redacted] recursively.

5. Write a program that uses recursive backtracking to generate all anagrams from a phrase typed by the user.

An anagram is a word or phrase made by rearranging the letters of another word or phrase. For example, the words “midterm” and “trimmed” are anagrams. If you ignore spaces and capitalization and allow multiple words, a multi-word phrase can be an anagram of some other word or phrase. For example, the phrases “Clint Eastwood” and “old west action” are anagrams. Your program will read a dictionary file of words and search for all words that can be formed using the letters in the user’s phrase. Use backtracking to choose each word, explore what can be made out of the remaining letters, then un-choose the word afterward. Here is a possible example dialogue:

6. Write a program that uses recursive backtracking to play the game of Boggle. Boggle is a word game played on a 4×4 grid where the player tries to find all valid dictionary words that can be made by tracing a path between adjacent letters from the board. Each link in the path can be horizontal, vertical, or diagonal. Figure 12.18 shows an example path to form the word “ensure”. Use recursive backtracking to explore each possible word that can be made using the letters on the board. Your algorithm should choose a starting square, explore what can be made from there, and un-choose the square afterward. (Hint: You will need a way to “mark” squares as being chosen or not chosen.)

Figure 12.18 An example Boggle board

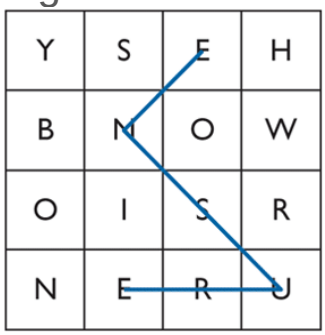
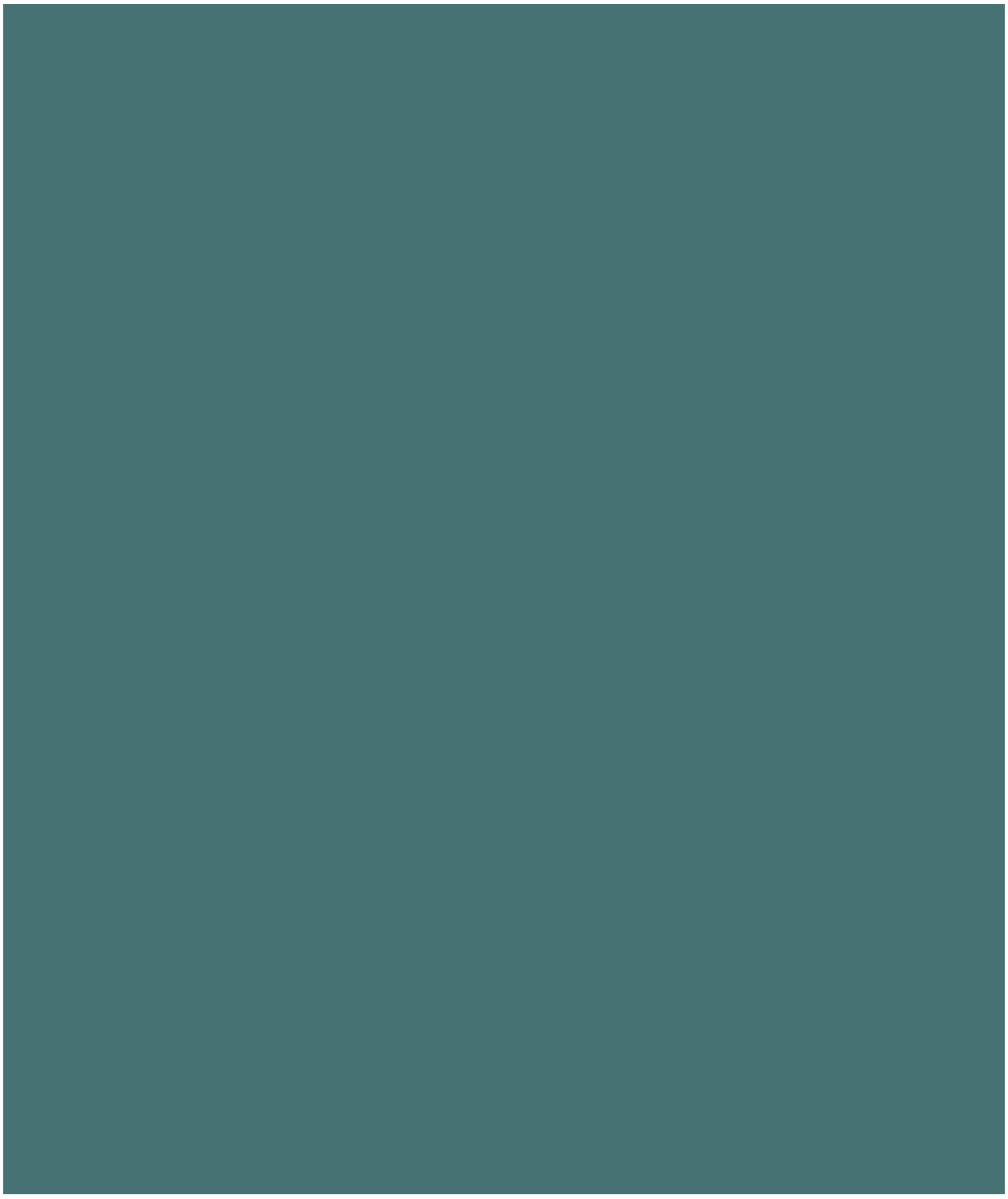


Figure 12.18 Full Alternative Text

7. Write a program that uses recursive backtracking to find all ancestors and descendants of a person given a file of familial relationships. For ancestors it must show all parents, all grandparents, all great grandparents, etc. For descendants it must show all children, all grandchildren, all great grandchildren, etc. The program also must use indentation to make it clear who is a parent of whom and who is a child of whom. Write two recursive methods that use backtracking to explore for ancestors and

descendants respectively. Here is a possible example dialogue:



Chapter 13 Searching and Sorting

13.1 Searching and Sorting in the Java Class Libraries
• Binary Search
• Sorting
• Shuffling
• Custom Ordering with Comparators
13.2 Program Complexity
• Empirical Analysis
• Complexity Classes
13.3 Implementing Searching and Sorting Algorithms
• Sequential Search
• Binary Search
• Recursive Binary Search
• Searching Objects
• Selection Sort
13.4 Case Study: Implementing Merge Sort
• Splitting and Merging Arrays
• Recursive Merge Sort
• Complete Program

Introduction

When you are dealing with large amounts of data, you'll often want to search the data for particular values. For example, you might want to search the Internet for a web page that contains a certain

keyword, or you might want to search a phone book for a person's phone number.

It's also useful to be able to rearrange a collection of data into sorted order. For example, you might want to sort a list of students' course grade data by name, student ID, or grade.

In this chapter we'll look at ways to use Java's class libraries to search and sort data. We'll practice implementing some searching and sorting algorithms and talk more generally about how to observe and analyze the runtimes of algorithms.

13.1 Searching and Sorting in the Java Class Libraries

To search an [redacted] or [redacted], you can call its [redacted] method. This method examines each element of the list, looking for a target value. It returns the first index at which the target value occurs in the list, or [redacted] if the target doesn't occur.

Imagine a large [redacted] containing all the words in a book. The following code could read a large text file into such a list:

You could use the `indexOf` method to see whether a given word appears in the book and, if so, at what index the word appears:

```
words.indexOf(word)
```

The [redacted] method performs a *sequential search*, examining each element of the list in sequence until it finds the one that the user is looking for. If it reaches the end of the list without finding the requested word, it returns [redacted]. When it searches a 1,000,000-element list for an element at index 675,000, a sequential search would have to examine all 675,000 elements that came before the desired element.

If you have an array of data instead of a list, there's no prewritten method to sequentially search the array. You'll have to write the code yourself (as we'll do later in this chapter) or put the array's elements into a [redacted] first and search the [redacted] with [redacted].

Binary Search

Sometimes you'll want to search through elements of an array or list that you know is in sorted order. For example, if you wanted to know whether "queasy" was a real English word, you could search the contents of an alphabetized dictionary text file. Likewise, you might find yourself looking for a book written by Robert Louis Stevenson in a list of books sorted by the author's last name. If the dictionary is large or the list of books is long, you probably won't want to sequentially examine all the items it contains.

There’s a better algorithm called *binary search* that searches sorted data much faster than a sequential search. A normal sequential search of a million-element array may have to examine all the elements, but a binary search will need to look at only around 20 of them. Java’s class libraries contain methods that implement the binary search algorithm for arrays and lists.

Binary Search

An algorithm that searches for a value in a sorted list by repeatedly dividing the search space in half.

The binary search algorithm begins by examining the center element of the array or list. If the center element is smaller than the target you’re searching for, there’s no reason to examine any elements to the left of the center (at lower indexes). If the center element is larger than the target you’re searching for, there’s no reason to examine any elements to the right of the center (at greater indexes). Each pass of the algorithm eliminates half the search space from consideration, so in most cases the algorithm finds the target value much faster than a sequential search would have found it.

The logic of the binary search algorithm is similar to the strategy that people use in a high/low guessing game in which the computer generates a random number between 1 and 100 and the user tries to guess it. After each incorrect guess, the program gives a hint about whether the user’s guess was too high or too low. A poor algorithm for this game is to guess 1, 2, 3, and so on. A smarter algorithm is to guess the middle number and cut the

range in half each time on the basis of whether the guess was too high or too low. Figure 13.1 shows how this works.

Figure 13.1 Passes of a binary search for a number between 1 and 100

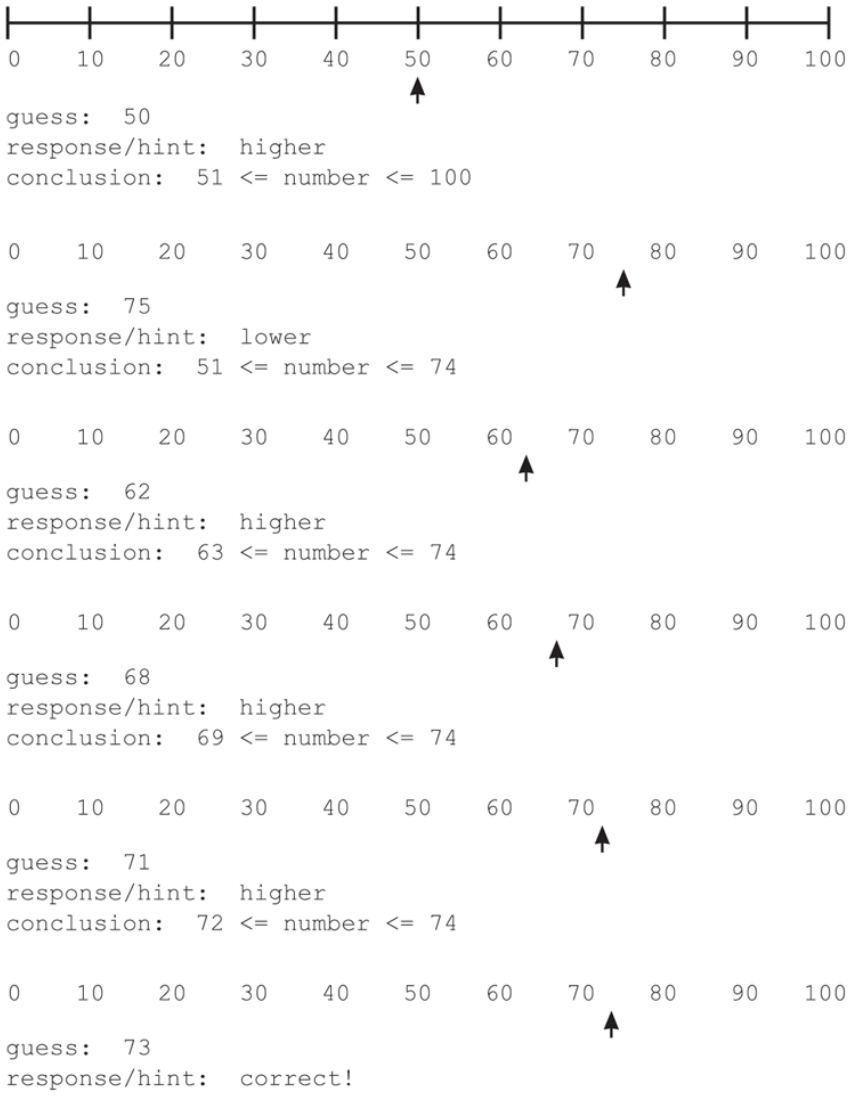


Figure 13.1 Full Alternative Text

A binary search uses this same approach when it searches a sorted array for a target value. The algorithm scales extremely well to large input data. The [redacted] class in the [redacted] package contains a static method called [redacted] that

implements the binary search algorithm. It accepts an array of any suitable type and a target value as its parameters and returns the index where you can find the target element. If the element isn't found, the method returns a negative index.

The following code uses the [redacted] method to find a number in an array of integers. It needs to examine only indexes 4, 6, and then 5 in order to find the target value at index 5:

If you're using a list such as an [redacted] instead, you can call the static method [redacted] to search the list of elements. If you had an [redacted] called [redacted] containing the same elements as the array in the previous example, the following code would similarly search the elements at indexes 4, 6, and then 5 before assigning the value [redacted] to the variable [redacted]:

If you want to use the [redacted] method on either an array or a list, the data must be in sorted order, because the method relies on the ordering to quickly find the target value. If you call [redacted] on unsorted data, the results are undefined and the algorithm doesn't guarantee that it will return the right answer.

Let's look at a short program that benefits from the speed of binary search. In the game of Scrabble, players form words on a board using letter tiles to earn points. Sometimes a player tries to spell a word that is not a legal word in the dictionary, so another player "challenges" the word. The challenger looks up the word in the dictionary, and depending on whether it is found, the move may be revoked from the board. The following program helps resolve Scrabble word challenges by performing binary searches for words in a dictionary file. The input file's words occur in sorted order, so the list can be properly searched using [redacted]. (If the file had not been sorted, we could have sorted it with [redacted]).

```
import

import


public class

public static void

throws
```

```
new

new




new




while
```

```
new
```

```
while

int

if

else
```

Here is a sample execution of the program and its resulting output, using a Scrabble players' dictionary that contains 172,823 words:

Sorting

When you use a computer, you often need to sort data. When you browse your hard drive, for example, you might sort your files by file name, extension, and date. When you play music, you might sort your song collection by artist, year, or genre. You might also want to sort arrays and lists so that they can be searched efficiently with the binary search algorithm.

The Java class libraries provide sorting methods for arrays and lists. You can sort an array with the [redacted] method:

The preceding code produces the following output:

Java 8 introduced a new sorting method called [redacted] that generally runs faster than [redacted]. It takes advantage of computers with multiple processors or multi-core processors to sort multiple portions of the array at the same time, leading to faster performance. To use it, just write [redacted] instead of [redacted] in your code. Other new features of Java 8 are discussed in more detail in Chapter 19.

The array must be of a type that can be compared—that is, a type that stores either primitives or objects that implement the

[redacted] interface discussed in Chapter 10. For example, you can sort an array of integers or [redacted], but you can't easily sort an array of [redacted] objects or [redacted] objects because those classes don't implement the [redacted] interface.

To sort a list you can use the [redacted] method, discussed briefly in Chapter 10, which accepts a list such as an [redacted] as a parameter and puts its elements into sorted order. The following code produces the same output as the preceding array code:

When the [redacted] method is used with primitive data, it uses an algorithm called quicksort. [redacted] and [redacted] use a different algorithm, called merge sort, when they deal with object data. We'll discuss the implementation of merge sort in detail later in this chapter.

Shuffling

The task of shuffling data, or rearranging the elements into a random order, is perhaps the opposite of sorting. Why would one want to do this?

One application for shuffling is a card game program. You might have a card deck stored as a list of [redacted] objects. If the cards are in a predictable order, the game will be boring. You'd like to shuffle the deck of cards, rearranging them into a random ordering each time. This is a case in which chaos is preferable to order.

Another application is a situation in which you want a random permutation of a list of numbers. You can acquire a random permutation of the numbers from 1 through 5, for example, by storing those numbers into a list and shuffling the list.

The [redacted] class has a method called [redacted] that accepts a list as its parameter and rearranges its elements randomly. The

following example creates a deck of card strings, shuffles it, and examines the card at the top of the deck:

The code randomly produces output such as the following, with different outputs on different runs:

Table 13.1 briefly summarizes the useful static methods in Java's class libraries for searching, sorting, and shuffling.

Table 13.1 Searching and Sorting in Java's Class Libraries

[redacted]	Returns the index of the given value in the given array, assuming that the array's elements are currently in sorted order, or returns a negative number if the given value is not found.
[redacted]	Arranges the given array's elements into sorted order.
[redacted]	Returns the index of the given value in the given list, assuming that the list's elements are currently in sorted order, or returns a negative number if the given value is not found.
[redacted]	Arranges the given list's elements into a random order.
[redacted]	Arranges the given list's elements into sorted order.

Custom Ordering with Comparators

Sometimes you'll want to search or sort a collection of objects in an ordering that is different from that of its [redacted] implementation. For example, consider the following code, which sorts an array of [redacted] and prints the result:

This code produces the following output, which may not be what you expected:

Notice that the elements are in case-sensitive alphabetical ordering, with all the uppercase strings placed before the lowercase ones.

Recall from Chapter 10 that many types (including [redacted]) have natural orderings that are defined by comparison functions, implemented as the [redacted] method from the [redacted] interface. The [redacted] method of [redacted] objects uses a case-sensitive ordering, but in this case we would prefer a case-insensitive comparison. We might also want an option to sort in other orders, such as by length or in reverse alphabetical order.

In situations such as these, you can define your own external comparison functions with an object called a *comparator*. Comparators perform comparisons between pairs of objects. While a class can have only one natural ordering, arbitrarily many comparators can be written to describe other ways to order the class's objects. A comparator can also supply an external ordering on a class that does not implement a natural ordering of its own. The [redacted], [redacted], and [redacted] methods all have variations that accept a comparator as an additional parameter and use the comparator to guide the searching or sorting process.

Comparators are implemented as classes that implement the interface [redacted] in the [redacted] package. This interface has a method called [redacted] that accepts a pair of objects as parameters and compares them. Like [redacted], the [redacted] method returns a negative number, zero, or a positive number to indicate a less-than, equal-to, or greater-than relationship, respectively:

Implementing [redacted] is like implementing the [redacted] interface, except that instead of placing the code inside the class to be compared, you write a separate comparator class. The [redacted] method is similar to the [redacted] method from the [redacted] interface, except that it accepts both of the objects to compare as parameters. (The [redacted] method accepts only one object as a parameter because the other object compared is the implicit parameter.)

Like [redacted], [redacted] is actually a generic interface [redacted] that requires you to specify the type of objects you'll be comparing. For example, if you wish to write a comparison function for [redacted], you must write a class that implements [redacted].

Before we implement a comparator of our own, let's use one that is part of Java's class libraries. If you want to sort [redacted] in case-insensitive alphabetical order, you can use a constant [redacted] in the [redacted] class called [redacted]. The following code uses it to sort an array of mixed-case strings:

```
, String.CASE_INSENSITIVE_ORDER)
```

This code produces the following output:

```

```

Now let's write a [redacted] that orders strings by their lengths. Our [redacted] method should return a negative number if the first string is shorter than the second, [redacted] if they are equal in length, and a positive number if the first string is longer than the second. Recall from Chapter 10 that a [redacted] method examining integer data can often simply subtract one number from the other and return the result. The following [redacted] method is also based on this idea:

```
import
```

```
public

implements



public int



return
```

Now that we've written a length comparator, we can pass one when we sort an array or list of [redacted] objects:

```
```

```
, new LengthComparator()
```

Here's the output when we run this code on the array from earlier in this section:

```

```

Notice that the strings appear in order of increasing length.

Sometimes you'll want to search or sort a collection of objects that don't implement the interface. For example, the class doesn't implement , but you might want to sort an array of objects by x-coordinate, using the y-coordinate to break ties. The following code is an example that compares objects in this way:

```
import
```

```
import


public class
implements


public int


int


if


int
```

```
return

else

return
```

It’s important to understand that we are creating a separate class as our comparator; we are not modifying the [redacted] class itself. This is one of the main benefits of comparators: you don’t need to modify the class being ordered. The following code uses the [redacted] to sort an array of four [redacted] objects:

```
```

This code sorts the points into the following order: (–1, 15), (3, 7), (3, 9), (4, –2). This comparator also allows you to use [redacted] objects in the “tree” collections that depend on ordering, [redacted] and [redacted]. Both of these collections have constructors that accept a comparator as a parameter and use it to order their elements internally.

If you have a group of objects with a natural ordering and you want to sort them in reverse order, you can call the method [redacted] to receive a [redacted] that inverts the objects’ natural ordering. For example, the following code would sort the array of [redacted] that was used earlier in reverse alphabetical order:

For types that do not have a natural ordering, you can ask for the reverse of a [redacted]'s order by calling [redacted] and passing the [redacted] to be reversed. For example, the following code would sort the array of [redacted] objects by decreasing x-coordinate:

```
new PointComparator()
```

The resulting order would be as follows: (4, –2), (3, 9), (3, 7), (–1, 15).

Table 13.2 summarizes several useful places that comparators appear in the Java class libraries.

Table 13.2 Useful Comparators and Methods

[redacted]	Returns the index of the given value in the given array, assuming that the array is currently sorted in the ordering of the given comparator, or returns a negative number if the given value is not found.
[redacted]	Sorts the given array in the ordering of the given comparator.

[redacted]	Returns the index of the given value in the given list, assuming that the list is currently sorted in the ordering of the given comparator, or returns a negative number if the given value is not found.
[redacted]	Returns the largest value in the collection according to the ordering of the given comparator.
[redacted]	Returns the smallest value in the collection according to the ordering of the given comparator.
[redacted]	Returns a comparator that compares objects in the opposite of their natural order.
[redacted]	Returns a comparator that compares objects in the opposite of the ordering of the given comparator.
[redacted]	Sorts the given list in the ordering of the given comparator.
[redacted]	Sorts strings alphabetically, ignoring capitalization.

13.2 Program Complexity

In Chapter 8, we talked about client code, the code that interacts with a class or object. In this chapter, we've studied how to be a client of Java's methods for searching and sorting data. Since searching and sorting are important programming ideas, it's worthwhile to understand how they are implemented. But before we dive into this, let's discuss some background ideas about how to analyze the efficiency of code.

As you progress in this textbook, you're writing increasingly complex programs. You're also seeing that there are often many ways to solve the same problem. How do you compare different solutions to the same problem to see which is better?

We desire algorithms that solve problems quickly or with high efficiency. The technical term that refers to algorithms' runtime is *complexity*. An algorithm with higher complexity uses more time or resources to solve a problem.

Complexity

A measure of the computing resources that are used by a piece of code, such as time, memory, or disk space.

Usually when we talk about the efficiency of a program we are talking about how long the program takes to run, or its *time complexity*. The time complexity for a program to be “fast enough” depends on the task. A program running on a modern computer that requires five minutes to look up a dictionary word is probably too slow. An algorithm that renders a complex three-dimensional movie scene in five minutes is probably very fast.

One way to determine an algorithm’s approximate time complexity is to program it, run the program, and measure how long it takes to run. This is sometimes called an *empirical analysis* of the algorithm. For example, consider two algorithms to search an array: one that sequentially searches for the desired target element, and one that first sorts the array and then performs a binary search on the sorted array. You could empirically analyze the algorithms by writing both as programs, running them on the same input, and timing them.

But empirically analyzing an algorithm isn’t a very reliable measure, because on a different computer with a different processor speed and more or less memory the program might not take the same amount of time. Also, in order to empirically test an algorithm, you must write it and time it, which can be a chore.

A more neutral way to measure a program’s performance is to examine its code or pseudocode and roughly count the number of statements that are executed. This is a form of *algorithm analysis*, the practice of applying techniques to mathematically approximate the performance of various computing algorithms. Algorithm analysis is an important tool in computer science. One of the fundamental principles of science in general is that we can make predictions and hypotheses using formal models, which we can then test by experimentation.

Not all statements require the same amount of time to execute. For example, a CPU can handle addition faster than multiplication, and a method call generally takes more time than a statement that evaluates the Boolean test of an [redacted] statement. But for the purposes of simplification, let's assume that the following actions require an equal and fixed amount of time to execute:

- Variable declarations and assignments
- Evaluating mathematical and logical expressions
- Accessing or modifying an individual element of an array
- Simple method calls (where the method does not perform a loop)

One kind of variable that does not require a fixed amount of time to initialize is an array. When an array is constructed, Java zeroes each array element, which takes more time for longer arrays. Some types of objects also have lengthy code in their constructors that makes them take longer to construct.

From the preceding simple rules, we can extrapolate the runtimes of larger and more complex pieces of code. For example, the runtime of a group of statements in sequential order is the sum of the individual runtimes of the statements:

```
statement1.
statement2.   } 3
statement3.
```

The runtime of a loop is roughly equal to the runtime of its body times the number of iterations of the loop. For example, a loop with a body that contains K simple statements and that repeats N times will have a runtime of roughly (K*N):

```
for (N times) {
    statement1.
    statement2.
    statement3.
}
```
3N

13.2-4 Full Alternative Text

The runtime of multiple loops placed sequentially (not nested) with other statements is the sum of the loops' runtimes and the other statements' runtimes:

```
statement1.
for (N times) {
    statement2.
}

for (M times) {
    statement3.
}
statement4.
statement5.
```
N, M, M + N + 3

13.2-5 Full Alternative Text

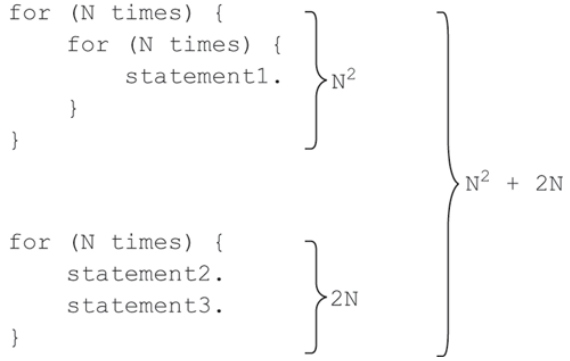
The runtime of a loop containing a nested loop is roughly equal to the runtime of the inner loop multiplied by the number of repetitions of the outer loop:

```
for (M times) {
    for (N times) {
        statement1.
        statement2.
        statement3.
    }
}
```
3, 3N, 3MN

13.2-6 Full Alternative Text

Normally, the loops in long-running algorithms are processing some kind of data. Many algorithms run very quickly if the input dataset is small, so we generally worry about the performance

only for large datasets. For example, consider the following set of loops that process an array of N elements:



13.2-7 Full Alternative Text

When we analyze code like this, we often think about which line is most frequently executed in the code. In programs with several sequential blocks of code that all relate to some common value N (such as the size of an input dataset), the block raised to the highest power of N usually dominates the overall runtime. In the preceding code, the first N2 loop executes its statement far more times than the second N loop executes its two statements. For example, if N is 1000, [redacted] executes (1000*1000) or 1,000,000 times, while [redacted] and [redacted] each execute only 1000 times.

When we perform algorithm analysis, we often ignore all but the most frequently executed part of the code, because the runtime of this statement will outweigh the combined runtimes of the other parts of the code. For example, we might refer to the preceding code as being “on the order of” N2, ignoring the extra 2N statements altogether. We’ll revisit this idea later in the chapter.

One key concept to take away from this brief discussion of algorithm analysis is how expensive it is to perform nested loops over large sets of input data. Algorithms that make many nested passes over a very large dataset tend to perform poorly, so it’s

important to come up with efficient algorithms that don't loop over data needlessly.

Now let's take a look at algorithm complexity in action, observing the runtimes of some actual algorithms that can be used to solve a programming problem on a large dataset.

Empirical Analysis

Consider the task of computing the range of numbers in an array. The range is the difference between the lowest and highest numbers in the array. An initial solution might use nested loops to examine every pair of elements in the array, computing their difference and remembering the largest difference found:

```
max = 0.

for (each index i) {

for (each index j) {

update max, if elements i and j differ by more than max.

}

}
```

The following code implements the [redacted] method as described:

Since the code has two nested [redacted] loops, each of which processes the entire array, we can hypothesize that the algorithm executes roughly N2 statements, or some multiple thereof.

We can measure the speed of this range algorithm in milliseconds by calling [redacted] on various arrays and measuring the time elapsed. We measure the time by acquiring the current time before and after calling [redacted] on a large array and subtracting the start time from the end time.

As you can see in Figure 13.2, as the input size N doubles, the runtime of the [redacted] method approximately quadruples. This is consistent with our hypothesis. If the algorithm takes N2 statements to run and we increase the input size to 2N, the new runtime is roughly (2N)2 or 4N2, which is four times as long as the original runtime.

Figure 13.2 Runtimes for first version of range algorithm

N	Runtime (ms)
1000	15
2000	47
4000	203
8000	781
16000	3110
32000	12563
64000	49937

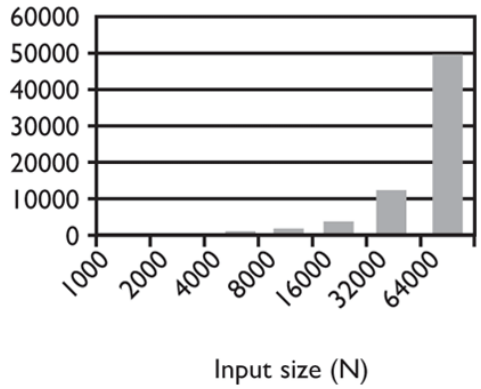


Figure 13.2 Full Alternative Text

Our code isn't very efficient for a large array. It requires over 12 seconds to examine 32,000 integers on a modern computer. In real-world data-processing situations, we would expect to see far larger input datasets than this, so this runtime isn't acceptable for general use.

Studying a piece of code and trying to figure out how to speed it up can be deceptively difficult. It's tempting to approach the

problem by looking at each line of code and trying to reduce the amount of computation it performs. For example, you may have noticed that our [redacted] method actually examines every pair of elements in the array twice: For unique integers [redacted] and [redacted], we examine the pair of elements at indexes ([redacted], [redacted]) as well as the pair at ([redacted], [redacted]).

We can perform a minor modification to our [redacted] method's code by starting each inner [redacted] loop ahead of [redacted], so that we won't examine any pair ([redacted], [redacted]) where [redacted]. Performing minor modifications like this is sometimes called tweaking an algorithm. The following code implements our tweaked version of the range algorithm:

```
i + 1
```

Since about half of the possible pairs of [redacted] values are eliminated by this tweak, we'd hope that the code would run about twice as fast. Figure 13.3 shows its actual measured runtime. As we estimated, the second version is about twice as fast as the first. We could implement other minor tweaks, such as replacing the [redacted] call with a simple [redacted] test (which would speed up the algorithm by around 10% more), but there's a more important point to be made. When the input size doubles, the runtime of either version of the algorithm roughly quadruples. Consequently, regardless of which version we use, if the input array is very large the method will be too slow.

Figure 13.3 Runtimes for second version of range algorithm

N	Runtime (ms)
1000	16
2000	16
4000	110
8000	406
16000	1578
32000	6265
64000	25031

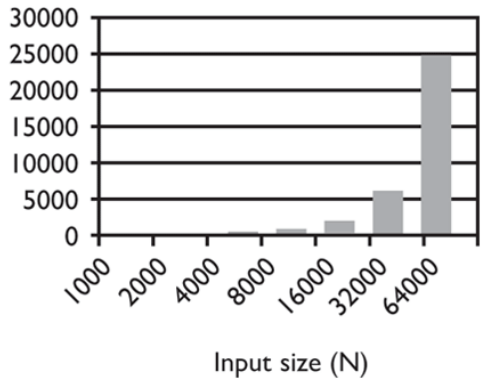
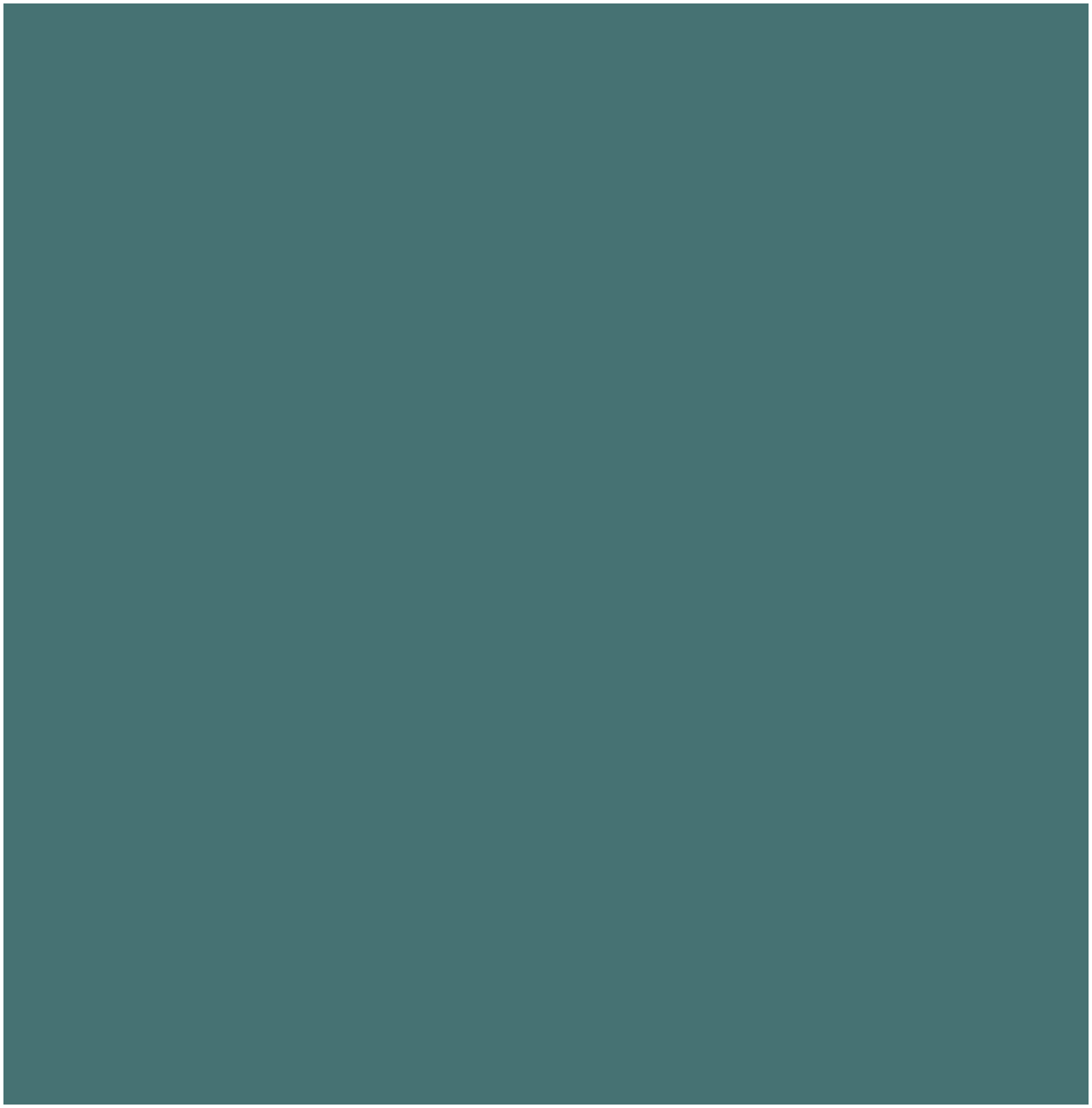


Figure 13.3 Full Alternative Text

Rather than trying to further tweak our nested loop solution, let's try to think of a more efficient algorithm. As we noted earlier, the range of values in an array is the difference between the array's largest and smallest elements. We don't really need to examine

all pairs of values to find this range; we just need to discover the pair representing the largest and smallest values. We can discover both of these values in a single loop over the array by using a min/max loop, discussed in Chapter 4. The following new algorithm demonstrates this idea:



Since this algorithm passes over the array only once, we'd hope that its runtime would be proportional to the array's length. If the array length doubles, the runtime should double, not quadruple. Figure 13.4 shows its runtime.

Figure 13.4 Runtimes for third version of range algorithm

N	Runtime (ms)
1000	0
2000	0
4000	0
8000	0
16000	0
32000	0
64000	0
128000	0
256000	0
512000	0
1e6	0
2e6	16
4e6	31
8e6	47
1.67e7	94
3.3e7	188
6.5e7	453
1.3e8	797
2.6e8	1578

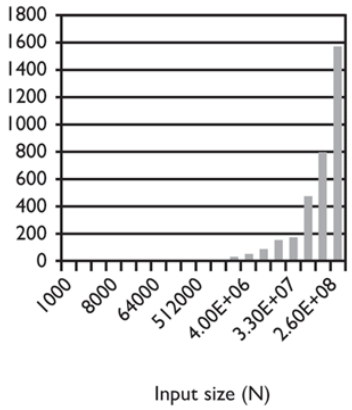


Figure 13.4 Full Alternative Text

Our runtime predictions were roughly correct. As the size of the array doubles, the runtime of this new range algorithm approximately doubles as well. The overall runtime of this algorithm is much better; we can examine over a hundred million integers in less than a second.

There are some important observations to take away from this exercise:

- Tweaking an algorithm's code often isn't as powerful an optimization as finding a better algorithm.
- An algorithm's rate of growth, or the amount by which its runtime increases as the input dataset grows, is the standard measure of the complexity of the algorithm.

Complexity Classes

We categorize rates of growth on the basis of their proportion to the input data size *N*. We call these categories *complexity classes* or *growth rates*.

Complexity Class

A set of algorithms that have a similar relationship between input data size and resource consumption.

The complexity class of a piece of code is determined by looking at the most frequently executed line of code, determining the number of times it is executed, and extracting the highest power of *N*. For example, if the most frequent line executes (2N3+4N) times, the algorithm is in the “order N3” complexity class, or O(N3) for short. The shorthand notation with the capital O is called *big-Oh notation* and is used commonly in algorithm analysis.

Here are some of the most common complexity classes, listed in order from slowest to fastest growth (i.e., from lowest to highest complexity):

- *Constant-time*, or O(1), algorithms have runtimes that don’t depend on input size. Some examples of constant-time algorithms would be code to convert Fahrenheit

temperatures to Celsius or numerical functions such as [redacted].

- *Logarithmic*, or O(log N), algorithms typically divide a problem space in half repeatedly until the problem is solved. Binary search is an example of a logarithmic-time algorithm.
- *Linear*, or O(N), algorithms have runtimes that are directly proportional to N (i.e., roughly they double when N doubles). Many algorithms that process each element of a data set are linear, such as algorithms that compute the count, sum, average, maximum, or range of a list of numbers.
- *Log-linear*, or O(N log N), algorithms typically perform a combination of logarithmic and linear operations, such as executing a logarithmic algorithm over every element of a dataset of size *N*. Many efficient sorting algorithms, such as merge sort (discussed later in this chapter), are log-linear.

DID YOU KNOW?

Timing Code and the Epoch

[redacted]

Java's method [redacted] returns the number of milliseconds that have passed since 12:00 AM on January 1, 1970. This method can be used to measure the runtime of an algorithm. Over one trillion milliseconds have passed since the indicated time, so the value is too large to store in a simple value. The milliseconds are instead returned as a value of type

, which is a primitive type that's similar to but that is capable of holding much larger values. A can store any number up to 9,223,372,036,854,775,807, or roughly 263.

The following lines of code show how a piece of code can be timed:

The choice of January 1, 1970, as the point of reference for system times is an example of an *epoch*, or an instant chosen as the origin of a particular time scale. This particular epoch was chosen because it matches the epochs of many popular operating systems, including Unix.

For historical reasons, many older Unix operating systems store the time that has passed since the epoch as a 32-bit integer value. However, unspecified problems may occur when this number exceeds its capacity, which is not necessarily a rare event. The clocks of some Unix systems will overflow on January 19, 2038, creating a Year 2038 problem similar to the famous Year 2000 (Y2K) problem.

- *Quadratic*, or O(N2), algorithms have runtimes that are proportional to the square of the input size. This means that quadratic algorithms' runtimes roughly quadruple

when N doubles. The initial versions of the range algorithm developed in the previous section were quadratic algorithms.

- *Cubic*, or O(N3), algorithms have runtimes that are proportional to the cube of the input size. Such algorithms often make triply nested passes over the input data. Code to multiply two N×N matrices or to count the number of colinear trios of points in a large [redacted] array would be examples of cubic algorithms.
- *Exponential*, or O(2N), algorithms have runtimes that are proportional to 2 raised to the power of the input size. This means that if the input size increases by just one, the algorithm will take roughly twice as long to execute. One example would be code to print the “power set” of a dataset, which is the set of all possible subsets of the data. Exponential algorithms are so slow that they should be executed only on very small input datasets.

Table 13.3 presents several hypothetical algorithm runtimes as the input size N grows, assuming that each algorithm requires 100 ms to process 100 elements. Notice that even though they all start at the same runtime for a small input size, as N grows the algorithms in higher complexity classes become so slow that they would be impractical.

Table 13.3 Algorithm Runtime Comparison Chart

100	100 ms	100 ms	100 ms	100 ms	100 ms	100 ms	100 ms
200	100 ms	115 ms	200 ms	240 ms	400 ms	800 ms	32.7 sec
400	100 ms	130 ms	400 ms	550 ms	1.6 sec	6.4 sec	12.4 days
800	100 ms	145 ms	800 ms	1.2 sec	6.4 sec	51.2 sec	36.5 million years
1600	100 ms	160 ms	1.6 sec	2.7 sec	25.6 sec	6 min 49.6 sec	4.21*1024 years
3200	100 ms	175 ms	3.2 sec	6 sec	1 min 42.4 sec	54 min 36 sec	5.6*1061 years

When you look at the numbers in Table 13.3 , you might wonder why anyone bothers to use O(N3) or O(2N), algorithms when O(1) and O(N) algorithms are so much faster. The answer is that not all problems can be solved in O(1) or even O(N) time. Computer scientists have been studying classic problems such as searching and sorting for many years, trying to find the most efficient algorithms possible. However, there will likely never be a constant-time algorithm that can sort 1,000,000 elements as quickly as it can sort 10 elements.

For large datasets it’s very important to choose the most efficient algorithms possible (i.e., those with the lowest complexity classes). Algorithms with complexity classes of O(N2) or worse will take a long time to run on extremely large datasets. Keeping this in mind, we’ll now examine algorithms to search and sort data.

13.3 Implementing Searching and Sorting Algorithms

In this section, we'll implement methods that search and sort data. We'll start by writing code to search for an integer in an array of integers and return the index where it is found. If the integer doesn't appear in the array, we'll return a negative number. We'll examine two major searching algorithms, sequential and binary search, and discuss the tradeoffs between them.

There are literally hundreds of algorithms to sort data; we'll cover two in detail in this chapter. The first, seen later in this section, is one of the more intuitive algorithms, although it performs poorly on large datasets. The second, examined as a case study in the next section, is one of the fastest general-purpose sorting algorithms that is used in practice today.

Sequential Search

Perhaps the simplest way to search an array is to loop over the elements of the array and check each one to see if it is the target number. As we mentioned earlier, this is called a sequential search because it examines every element in sequence.

We implemented a sequential search of an array of integers in Chapter 7. The code uses a [redacted] loop and is relatively

straightforward. The algorithm returns -1 if the loop completes without finding the target number:

Using the rules stated in the previous section, we predict that the sequential search algorithm is a linear O(N) algorithm because it contains one loop that traverses at most N elements in an array. (We say “at most” because if the algorithm finds the target element, it stops and immediately returns the index.) Next, we’ll time it to verify our prediction.

Figure 13.5 shows actual results of running the sequential search algorithm on randomly generated arrays of integers. Searches were conducted for a value known to be in the array and for a value not in the array.

Figure 13.5 Runtimes for sequential search algorithm

N	Runtime when element is found (ms)	Runtime when element is not found (ms)
100000	0	0
200000	0	0
400000	0	0
800000	0	0
1.6e6	0	16
3.2e6	15	16
6.4e6	16	31
1.3e7	32	47
2.6e7	37	93
5.1e7	45	203
1.0e8	72	402
2.0e8	125	875

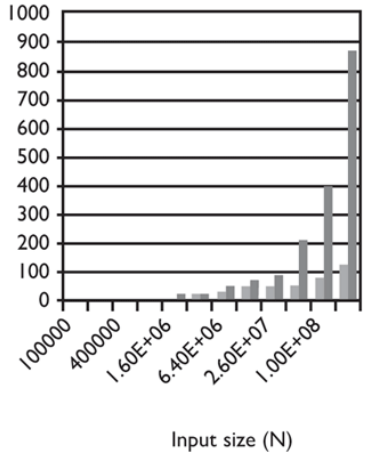


Figure 13.5 Full Alternative Text

When the algorithm searches for an integer that isn't in the array, it runs somewhat slower, because it can't exit its loop early by finding the target. This scenario raises the question of whether we should judge the algorithm by its fastest or slowest runtime. Often what's most important is the expected behavior for a typical input, or the average of its runtime over all possible inputs. This is called an *average case analysis*. But under certain conditions, we also care about the fastest possible outcome, the *best case analysis*, and/or the slowest possible outcome, the *worst case analysis*. In this algorithm, the average search looks at approximately half of the array, which is linear or O(N).

Binary Search

Consider a modified version of the searching problem, in which we can assume that the elements of the input array are in sorted order. Does this ordering affect our algorithm? Our existing algorithm will still work correctly, but now we know that we can stop the search if we ever get to a number larger than our target without finding the target. For example, if we're searching an array containing the elements [redacted] for the target value [redacted], we can stop searching once we see the [redacted].

We might think that such a modification to our sequential search algorithm would significantly speed up the algorithm, but in actuality it doesn't make much difference. The only case in which it speeds up the algorithm noticeably is when it is searching for a relatively small value that isn't found in the array. In fact, when the modified algorithm is searching for a large value that requires the code to examine most or all of the array elements, the algorithm actually performs slower than the original because it has to perform a few more Boolean tests. Most importantly, the algorithm is still O(N), which isn't the optimal solution.

Once again, tweaking the algorithm won't make as much difference as finding another, more efficient algorithm. If the input array is in sorted order, a sequential search isn't the best choice. If you had to instruct a robot how to look up a person's phone number in a phone book, would you tell the robot to read through all the entries on the first page, then the second, and so on until it found the person's name? Not unless you wanted to torture the poor robot. You know that the entries are sorted by name, so you'd tell the robot to flip open the book to somewhere near the

middle, then narrow its search toward the first letter of the person's name.

The binary search algorithm discussed previously in this chapter takes advantage of the ordering of the array. A binary search keeps track of the range of the array that is currently of interest. (Initially, this range is the whole array.) The algorithm repeatedly examines the center element of the array and uses its value to eliminate half of the range of interest. If the center element is smaller than the target, the lower half of the range is eliminated; if the center element is larger than the target, the upper half is eliminated.

As the algorithm runs, we must keep track of three indexes:

- The minimum index of interest ([redacted])
- The maximum index of interest ([redacted])
- The middle index, halfway between the minimum and maximum, which will be examined during each pass of the algorithm ([redacted])

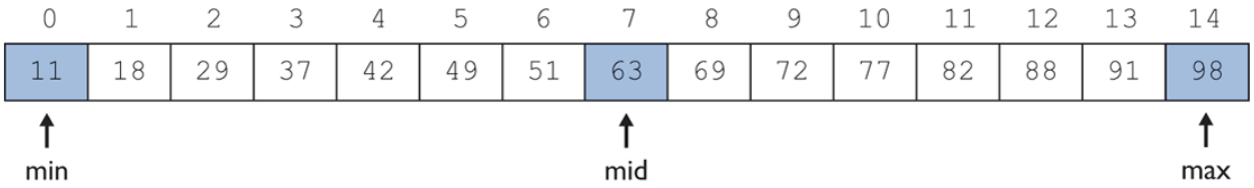
The algorithm repeatedly examines the element at the middle index and uses it to trim the range of indexes of interest in half. If we examine the middle element and find it's too small, we will eliminate all elements between [redacted] and [redacted] from consideration. If the middle element is too large, we will eliminate all elements between [redacted] and [redacted] from consideration.

Consider the following array:

```
int[] numbers = {11, 18, 29, 37, 42, 49, 51, 63,
                 69, 72, 77, 82, 88, 91, 98};
```

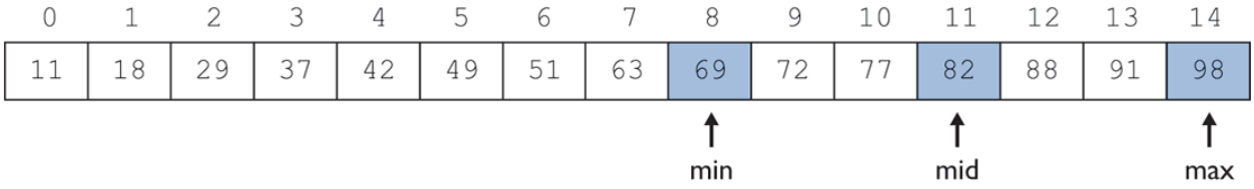
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
11	18	29	37	42	49	51	63	69	72	77	82	88	91	98

Let's run a binary search on the array for a target value of [redacted]. We'll start at the middle element, which is at index 14/2, or 7. The following diagrams show the [redacted], [redacted], and [redacted] at each step of the algorithm:



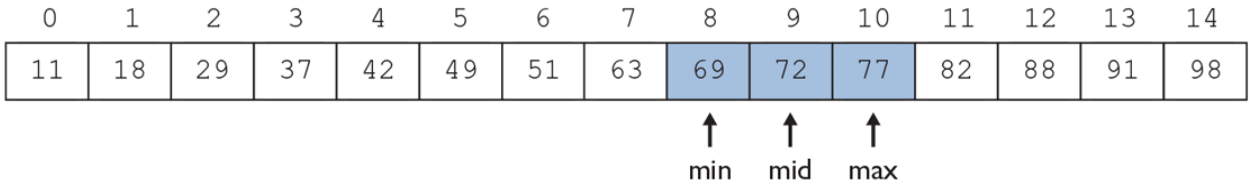
observation: `numbers[7] < 77`
decision: search indexes 8 – 14

13.3-10 Full Alternative Text



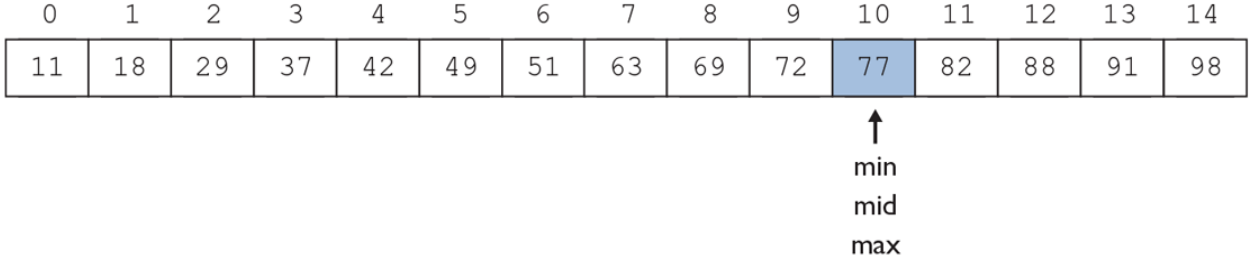
observation: `numbers[11] > 77`
decision: search indexes 8 – 10

13.3-11 Full Alternative Text



observation: `numbers[9] < 77`
decision: search index 10

13.3-12 Full Alternative Text



observation: `numbers[10] == 77`
decision: return 10

13.3-13 Full Alternative Text

What about when we're searching for an element that isn't found in the array? Let's say we're searching for the value [redacted] instead of [redacted]. The steps of the algorithm will be the same, except that on the fourth pass the algorithm will reach [redacted] instead of the desired value, [redacted]. The algorithm will have eliminated the entire range without finding the target and will know that it should stop. Another way to describe the process is that the algorithm loops until the [redacted] and [redacted] have crossed each other.

The following code implements the binary search algorithm. Its loop repeats until the target is found or until the [redacted] and [redacted] have crossed:

We won’t bother to show a runtime chart for the binary search algorithm, because there would be nothing to draw on the chart. This algorithm is so fast that the computer’s clock has trouble measuring its runtime. On a modern computer, even an array of over 100,000,000 elements registers as taking 0 ms to search!

While this is an impressive result, it makes it harder for us to empirically examine the runtime. What is the complexity class of the binary search algorithm? The fact that it finishes so quickly tempts us to conclude that it’s a constant-time, or O(1), algorithm. But it doesn’t seem right that a method with a loop in it would take a constant amount of time to execute. There is a relation between the runtime and the input size, because the larger the input is, the more times we must divide our range in half to arrive at a

single element. We could say that 2 raised to the number of repetitions is approximately equal to the input size *N*:

$$2^{repetitions} \cong N$$

Using some algebra and taking a logarithm base-2 of both sides of the equation, we find that

$$repetitions \cong \log_2 N$$

We conclude that the binary search algorithm is in the logarithmic complexity class, or O(log N).

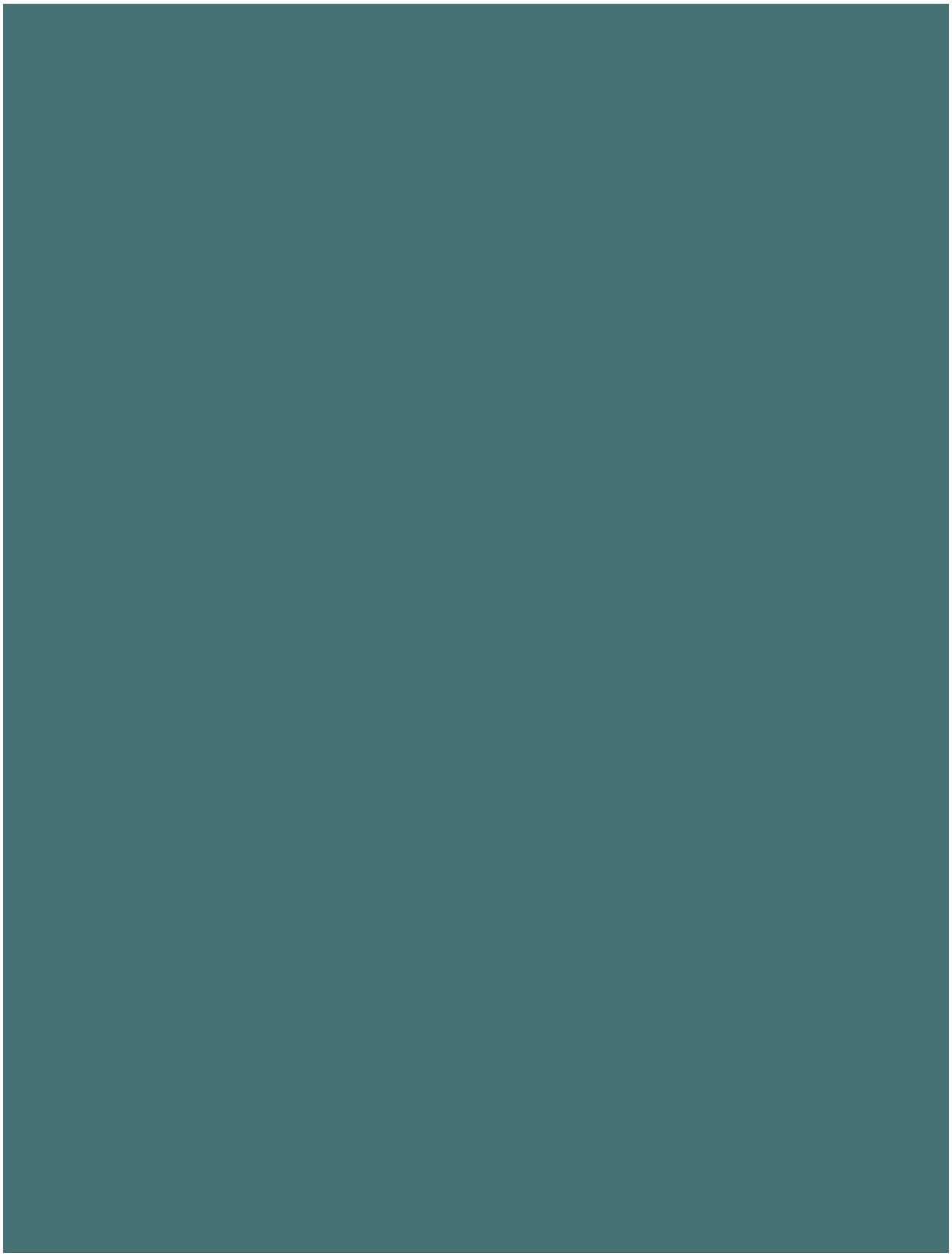
The runtime of the binary search algorithm doesn't differ much between the best and worst cases. In the best case, the algorithm finds its target value in the middle on the first check. In the worst case, the code must perform the full (log N) comparisons. But since logarithms are small numbers (log2 1,000,000 is roughly 20), the performance is still excellent in the worst case.

Recursive Binary Search

In the previous section, binary search was implemented using an iterative algorithm with a [redacted] loop. But the algorithm can also be implemented elegantly using the concept of recursion introduced in Chapter 12. The recursive version of the method should accept the same parameters as the standard binary search:

But this header will not make it easy to write a recursive solution to the problem. Recall that the essence of a recursive solution is to break down the problem into smaller pieces and then solve the sub-problem(s). In this algorithm, the way to shrink the problem is to examine a smaller and smaller portion of the array until we find the right index. To do this, we can change our method to accept additional parameters for the range of indexes (min and max) that currently are being examined. Rather than changing the header just shown, we can add a private recursive helper with the additional parameters as described in Chapter 12. The public method can start the recursive process by calling the helper with [redacted] and [redacted] as its minimum and maximum indexes to examine, respectively:

Our recursive binary search method accepts the minimum and maximum indexes of interest as parameters. On each pass of the recursive algorithm, the code examines the middle element. If this element is too small, the code recursively examines only the right half of the array. If the middle element is too large, the code recursively examines only the left half. This process repeats, recursively calling itself with different values of [redacted] and [redacted], until it finds the target or until the entire array has been eliminated from consideration. The following code implements the algorithm:



Some instructors don’t like recursive versions of methods like binary search because there is a nonrecursive solution that’s fairly easy to write and because recursion tends to have poor runtime performance because of the extra method calls it generates. However, that doesn’t pose a problem here. The runtime of the recursive version of our binary search method is still O(log N), because it’s essentially performing the same computation; it’s still cutting the input in half at each step. In fact, the recursive version is fast enough that the computer still can’t time it accurately. It produces a runtime of 0 ms even on arrays of tens of millions of integers.

In general, analyzing the runtimes of recursive algorithms is tricky. Recursive runtime analysis often requires a technique called *recurrence relations*, which are mathematical relations that describe an algorithm’s runtime in terms of itself. That’s a complex topic for a later course that won’t be covered in this textbook.

DID YOU KNOW?

Binary Search Details

There are a few interesting things about Java's implementation of binary search in and that we haven't mentioned yet. Take a look at this text from the Javadoc documentation of the method:

> The array must be sorted (as by the method, above) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no guarantee which one will be found.

Binary search depends on the array being sorted. If it isn't sorted, the documentation says the results are undefined. What does that mean? Why doesn't the algorithm just sort the array for you if it's unsorted?

There are two problems with that idea, both essentially related to runtime performance. For one, sorting takes much longer (O(N log N) time) than a binary search (O(log N) time). Second, to even discover that you need to sort the array, you'd need to look at each element to determine whether they're in order, which takes O(N) time. Essentially, the cost of examining the array and sorting it if necessary would be too great.

Even if the cost of sorting the array weren't so large, the client of the method probably won't want its array modified by the method. Searching is supposed to be a read-only operation, not one that rearranges the array.

Let's look at the other part of the previous quote: "If the array contains multiple elements with the specified value, there is no guarantee which one will be found." We didn't mention this earlier when we were discussing the algorithm, but in the case of duplicates, binary search isn't guaranteed to find the first occurrence of the element, because the moment it finds an occurrence it stops.

Here's another interesting blurb from the documentation:

Returns: index of the search key, if it is contained in the list; otherwise, . The insertion point is defined as the point at which the key would be inserted into the list: the index of the first element greater than the key, or , if all elements in the list are less than the specified key. Note that this guarantees that the return value will be if and only if the key is found.

Rather than returning for an unsuccessful search, the and methods return ( ), where is the last index the algorithm examined before giving up, which is also the index of the first element whose value is greater than the target. The documentation for these methods calls this value the *insertion point* because if you wanted to add the target to the array in sorted position, you'd place it at that index. For example, because a binary search of the following array for the value would finish at index 5, the algorithm would return :

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
11	18	29	37	42	49	51	63	69	72	77	82	88	91	98

If you were maintaining this sorted array and wanted to add to it at the proper index to retain the sorted order, you could call , get the result of , negate it, and subtract 1 from it to get the index at which you should insert the value. This is much faster than linearly searching for the place to add the value or adding the value at the end and resorting the array.

We could modify our own binary search code to match this behavior by changing the last line of the method's body to the following line:

Searching Objects

Searching for a particular object in an array of objects requires a few modifications to our searching code from the previous sections. Let's look at a sequential object search first, because it will work with any type of object. The most important modification to make to the code is to ensure that it uses the method to compare objects for equality:

If we want to do a binary search on objects, the objects must have an ordering (in other words, must be of a type that implements the interface or must be provided with a comparator) and the elements in the array or collection must be in sorted order. One common example would be an array of . Since we can't use relational operators like and on objects, we must call the method on pairs of objects and examine the value it returns. The following code implements a binary search on an array of s:

```
String
```

```
String
```

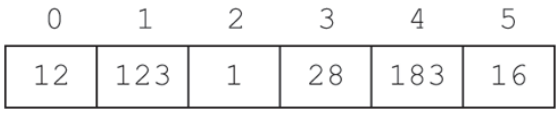
```
strings[mid].compareTo(target)
```

```
compare == 0
```

```
compare < 0
```

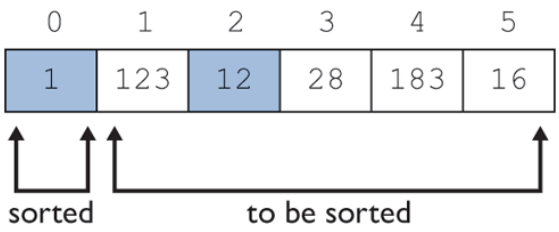
Selection Sort

Selection sort is a well-known sorting algorithm that makes many passes over an input array to put its elements into sorted order. Each time it runs through a loop, it selects the smallest value and puts it in the proper place near the front of the array. Consider the following array:



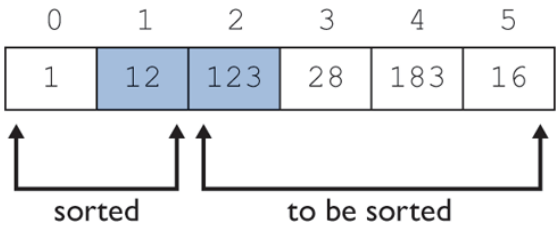
13.3-15 Full Alternative Text

How would you put its elements into order from smallest to largest? The selection sort algorithm conceptually divides the array into two pieces: sorted elements at the front and unsorted elements at the end. The first step of the selection sort makes a pass over the array and finds the smallest number. In the sample array, the smallest is [redacted], which equals [redacted]. The algorithm then swaps the smallest value with the value in the first position in the array, so that the smallest value will be at the front of the array. In this case, [redacted] and [redacted] are swapped:



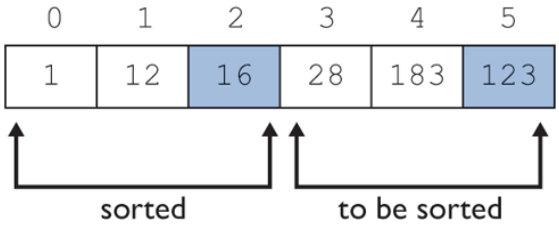
13.3-16 Full Alternative Text

The element at index 0 now has the right value, and only the elements at indexes 1 through 5 remain to be ordered. The algorithm repeats the process of scanning the unsorted portion of the array and looking for the smallest element. On the second pass, it scans the remaining five elements and finds that [redacted], which equals [redacted], is the smallest element. The program swaps this value with [redacted]. After this swap, the sorted area of the array consists of its first two indexes:



13.3-17 Full Alternative Text

Now [redacted] and [redacted] have the correct values. The third pass of the algorithm scans the remaining four unsorted elements and finds that the smallest one is [redacted], which equals [redacted]. It swaps this element with [redacted].



13.3-18 Full Alternative Text

The algorithm continues this process until all the elements have the proper values. Each pass involves a scan followed by a swap. The scan/swap occurs five times to process six elements. You don’t need to perform a sixth scan/swap because, if the first five elements have the correct values, the sixth element will be correct as well.

Here is a pseudocode description of the execution of the selection sort algorithm over an array `nums` that has six elements:

```
for (each i from 0 to 4) {

scan nums[i] through nums[5] for the smallest value.

swap nums[i] with the smallest element found in the scan.

}
```

You can write pseudocode like the following for the scan:

```
smallest = lowest array index of interest.

for (all other index values of interest) {

if (nums[index] < nums[smallest]) {

smallest = index.

}
```

```

```

You can then incorporate this pseudocode into your larger pseudocode as follows:

```
for (each i from 0 to 4) {

smallest = i.

for (each index between (i + 1) and 5) {

if (nums[index] < nums[smallest]) {

smallest = index.

}

}

swap nums[i] with nums[smallest].

}
```

You can translate this pseudocode almost directly into Java, except for the swap process. In Chapter 7, we wrote a method to swap two elements of an array. We can reuse it here:

We can also modify the code to work with arrays of any size. The following code implements the overall selection sort algorithm:

Since selection sort makes roughly N passes over an array of N elements, its performance is O(N2). Technically, it examines N+(N−1)+(N−2)+. . .+3+2+1 elements, because each pass starts one index ahead of where the last one started. Chapter 3 mentioned a mathematical identity which states that the sum of all integers from 1 to any maximum value N equals (N)(N+1)/2, which is just over ½N2. Figure 13.6 supports this analysis, because the runtime quadruples every time the input size N is doubled, which is characteristic of an N2 algorithm. The algorithm becomes impractically slow once the number of elements reaches tens of thousands.

Figure 13.6 Runtimes for selection sort algorithm

N	Runtime (ms)
1000	0
2000	16
4000	47
8000	234
16000	657
32000	2562
64000	10265
128000	41141
256000	164985

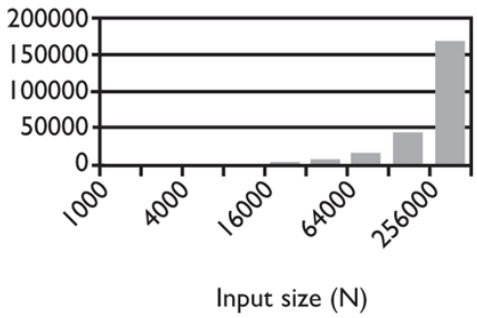


Figure 13.6 Full Alternative Text

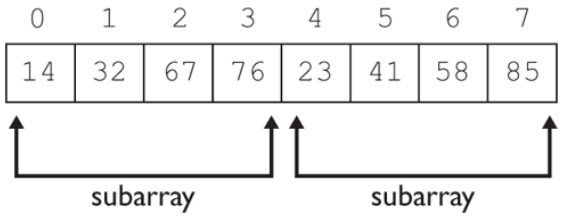
The current [redacted] code will sort arrays of integer values, but you could adapt it to sort [redacted] objects such as [redacted] using the techniques covered in the previous section on searching objects. For example, you could use the [redacted] and [redacted] methods to compare objects rather than relational operators like [redacted] and [redacted].

13.4 Case Study: Implementing Merge Sort

There are other algorithms similar to selection sort that make many passes over the array and swap various elements on each pass. An algorithm that searches for inverted pairs of elements and swaps them into order in this way cannot run faster than O(N2), on average. However, there is a better algorithm that breaks this barrier.

The *merge sort* algorithm is named for the observation that if you have two sorted subarrays, you can easily merge them into a single sorted array. For example, consider the following array:

You can think of it as two subarray halves, each of which (because of the element values we chose) happens to be sorted:



13.4-19 Full Alternative Text

The following pseudocode provides the basic idea of the merge sort algorithm:

```
split the array into two halves.

sort the left half.

sort the right half.

merge the two halves.
```

Let’s look at splitting the array and merging halves first; then we’ll talk about the sorting.

Splitting and Merging Arrays

Splitting one array into its two halves is relatively straightforward. We’ll set a midpoint at one half of the length of the array and consider everything before this midpoint to be part of the “left” half and everything that follows it to be in the “right” half. We can use the method [redacted] to extract the halves of an array as new arrays. The “left” half is from range 0 to half the length, and the “right” half is from half the length to the full length:

```
```

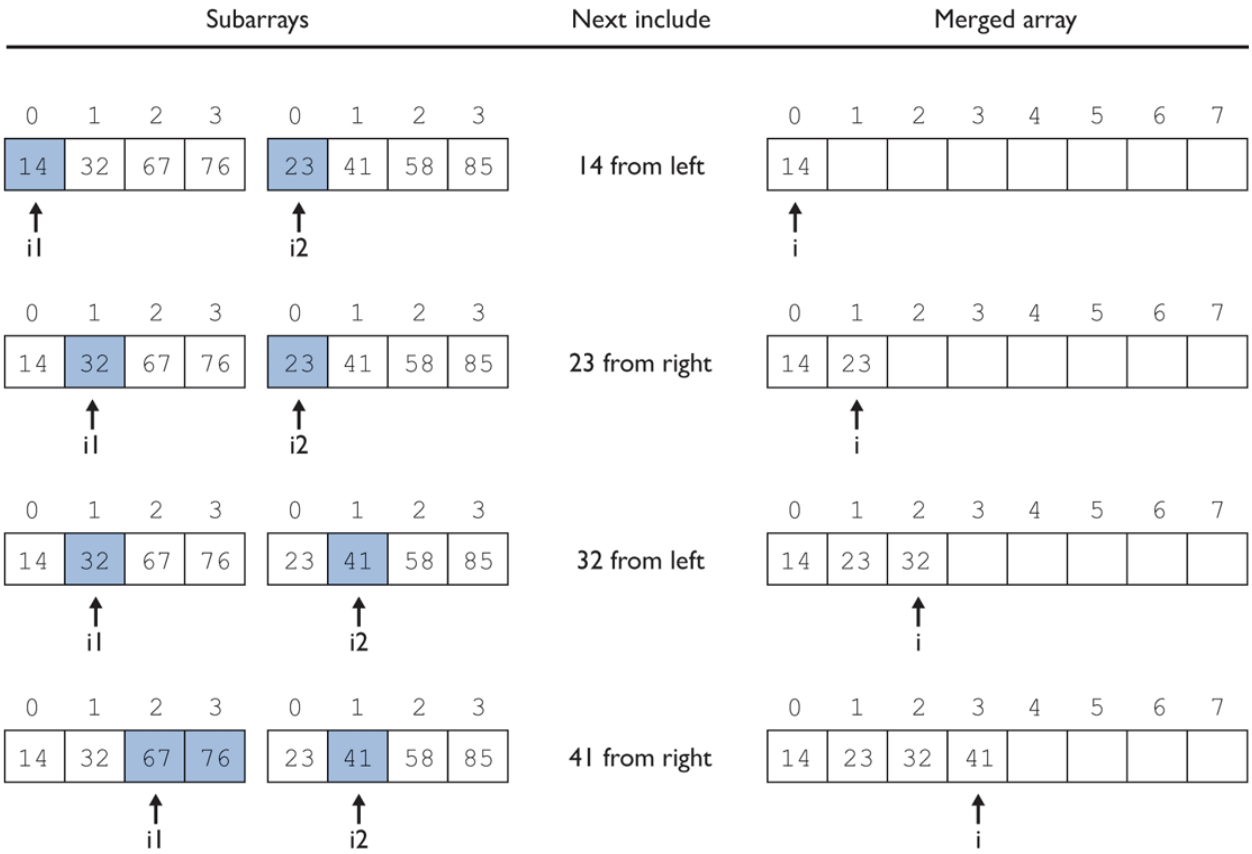
We will need to sort these left/right halves, then merge them into a sorted whole. For now, let's think about how we would merge two sorted subarrays. (We'll come back to sorting them later.) Suppose that you have two stacks of exam papers, each sorted alphabetically by name, and you need to combine them into a single stack sorted by name. The simplest algorithm is to place both stacks in front of you, look at the top paper of each stack, pick up the paper that comes first in alphabetical order, and put it face down into a third pile. You then repeat this process, comparing the papers on the top of each stack and placing the one that comes first face down on the merged stack, until one of your two original stacks is empty. Once one is empty, you just grab the entire remaining stack and place it on your merged pile.

The idea behind merging two sorted arrays is similar, except that instead of physically removing papers (integers) from the piles (subarrays), we'll keep an index for each subarray and increment that index as we process a given element. Here is a pseudocode description of the merging algorithm:

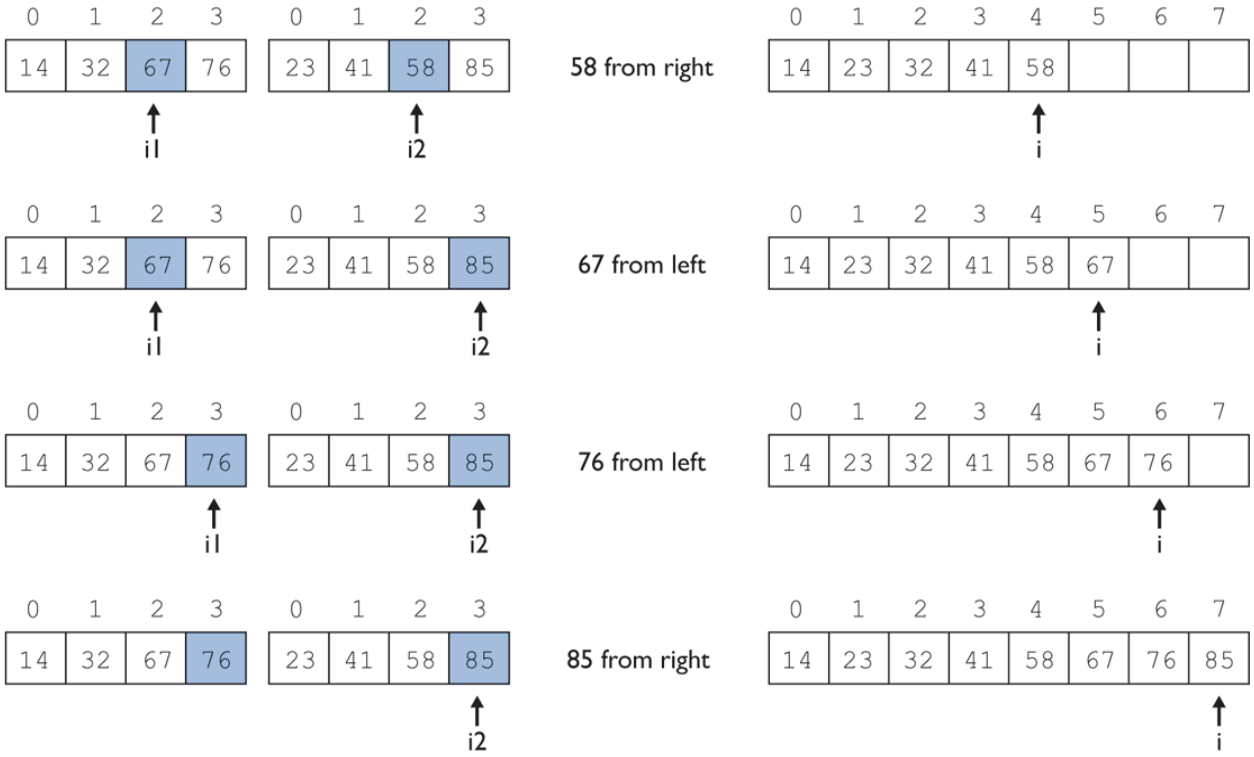
```
i1 = 0.    // left index

i2 = 0.    // right index

for (number of elements in entire array) {


if (left value at i1 <= right value at i2) {
```

```
include value from left array in new array.

i1++;

} else {

include value from right array in new array.

i2++;

}

}
```

Here is a trace of the eight steps to merge the two subarrays into a sorted array:



13.4-20 Full Alternative Text



13.4-21 Full Alternative Text

The following code is an initial attempt to implement the merge algorithm that was just described:

The preceding code is incorrect and will cause an out-of-bounds exception. After the program completes the seventh step of the preceding diagram, all of the elements in the left subarray will have been consumed and the left index [redacted] will run off the end of the subarray. Then, when the code tries to access element [redacted], it will crash. A similar problem would occur if the right index [redacted] exceeded the bounds of the right array.

We need to modify our code to remain within the bounds of the arrays. The [redacted] logic needs to ensure that the index [redacted] or [redacted] is within the array bounds before the program attempts to access the appropriate element. The simple test in the pseudocode needs to be expanded:

```
if (i2 has passed the end of the right array, or
```

```
left element at i1 <= right element at i2) {


take from left.

} else {


take from right.

}
```

The following second version of the code correctly implements the merging behavior. The preconditions and postconditions of the method are documented in comments:

```
i2 >= right.length || (i1 < left.length &&
```

Recursive Merge Sort

We've written the code to split an array into halves and to merge the sorted halves into a sorted whole. The overall merge sort method now looks like this:

The last piece of our program is the code to sort each half of the array. How can we sort the halves? We could call the [redacted] method created earlier in this chapter on the two halves. But in the previous chapter we discussed the recursive "leap of faith," the belief that our own method will work properly to solve a smaller version of the same problem. In this case, a better approach is to merge sort the two smaller halves. We can recursively call our own [redacted] method on the array halves, and if it's written correctly, it'll put each of them into sorted order. Our original pseudocode can now be rewritten as the following pseudocode:

```
merge sort
 the left half.

merge sort
 the right half.

merge the two halves.
```

If we're making our merge sort algorithm recursive, it needs to have a base case and a recursive case. The preceding pseudocode specifies the recursive case, but for the base case, what are the simplest arrays to sort? An array with either no elements or just one element doesn't need to be sorted at all. At least two elements must be present in order for them to appear in the wrong order, so the simple case would be an array with a length less than 2. This means that our final pseudocode for the merge sort method is the following:

```
if (array length is more than 1) {

split the array into two halves.

merge sort the left half.

merge sort the right half.
```

```
merge the two halves.

}
```

No [redacted] case is needed because if the array size is 0 or 1, we don’t need to do anything to the array. This recursive algorithm has an empty base case.

The following method implements the complete merge sort algorithm:

```
```

```
mergeSort(left);


mergeSort(right);
```

To get a better idea of the algorithm in action, we'll temporarily insert a few [redacted] statements into its code and run the method on the eight-element sample array shown previously in this section. We'll insert the following [redacted] statement at the start of the [redacted] method:

```
```

We'll also put the following [redacted] statement at the start of the [redacted] method:

Here is the output from running [redacted] on the example array:

It’s also important to test the code on an array that doesn’t divide into subarrays of exactly equal size (i.e., one whose overall length is not a power of 2). Because it employs integer division, our code makes the left subarray one element smaller than the right subarray when the size is odd. Given an initial five-element list of [ ] the algorithm prints the following output:

Complete Program

The complete program containing the merge sort code follows. Its method constructs a sample array and sorts it using the algorithm:

```
import

public class

public static void

int
```

```
public static void

int



if (a.length > 1) {



// split array into two halves



int
```

```
int
```

```
public static void


int


int


for
int


if
```

```
else
```

The program produces the following output:

Figure 13.7 demonstrates the performance of our merge sort algorithm on a modern computer. The merge sort algorithm's performance is much better than that of the selection sort algorithm. For example, whereas our selection sort test run needed over 41 seconds to sort 128,000 elements, the merge sort algorithm handled the same job in a blistering 47 milliseconds. But what is merge sort's complexity class? It looks almost like an O(N) algorithm, because the runtime only slightly more than doubles when we double the array size.

Figure 13.7 Runtimes for merge sort algorithm

N	Runtime (ms)
1000	0
2000	0
4000	0
8000	0
16000	0
32000	15
64000	16
128000	47
256000	125
512000	250
1e6	532
2e6	1078
4e6	2265
8e6	4781
1.6e7	9828
3.3e7	20422
6.5e7	42406
1.3e8	88344

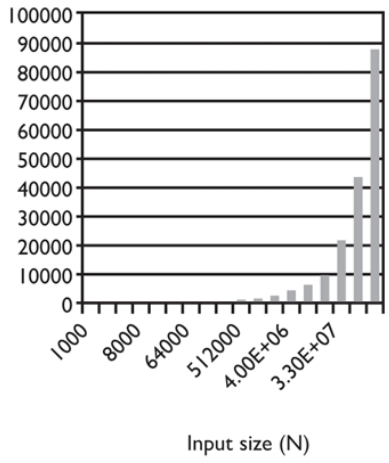


Figure 13.7 Full Alternative Text

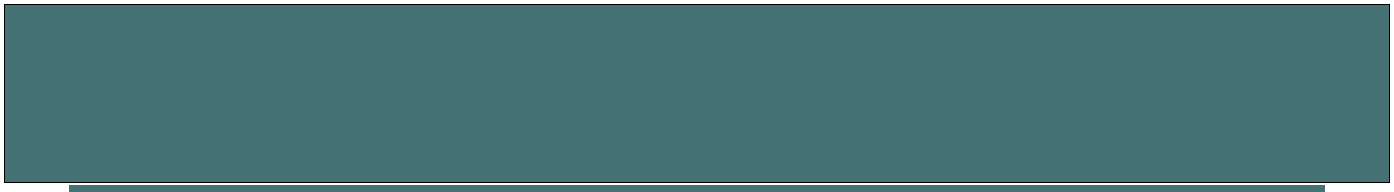
However, merge sort is actually an O(N log N) algorithm. A formal proof of this statement is beyond the scope of this book, but a

common-sense chain of reasoning is as follows: We have to split the array in half repeatedly until we hit the algorithm's base case, in which the subarrays each contain 1 element. For an array of size N, we must split the array log2 N times. At each of those log N steps, we have to do a linear operation of order N (merging the halves after they're sorted). Multiplying these operations' runtimes together produces a O(N log N) overall runtime.

The preceding algorithm runtime analysis is informal and not rigorous. As with other recursive algorithms, a precise analysis of merge sort's performance is complicated and requires mathematical techniques such as recurrence relations, which are not discussed in this book.

DID YOU KNOW?

Cost of Java Array Allocation



Earlier in the chapter, we mentioned that variable declaration and assignment can be thought of as taking some constant amount of time. That is usually true, but there is one important exception: initializing arrays. Suppose you were to execute this line of code:

Executing this line of code requires the computer to allocate a block of memory for the array. That can generally be done in a constant amount of time. But Java also insists on auto-initializing all array elements to and that takes time proportional to .

This subtlety of Java has stung several textbook authors who were translating textbooks from C and C++ into Java. When they were writing a solution to the merge sort algorithm, they included code that would allocate a temporary array as large as the entire array every time they did a merging operation.

For an array of size , there are different merging operations. If each one requires the computer to auto-initialize an array of length , then the overall sorting algorithm becomes an O(N2)) algorithm. That is considerably slower than the O(N log N) behavior that we expect for a merge sort algorithm.

The authors made this mistake because C and C++ don't auto-initialize arrays. That means that their code ran fast in those languages, and when they tested their code they found that it behaved as expected. They apparently translated their code into Java without retesting whether it was fast. More than one textbook author has made this mistake, but out of respect for our fellow authors (and recognizing that we make mistakes ourselves), we won't name any names!

Chapter Summary

Searching is the task of attempting to find a particular target value in a collection or array.

Sorting is the task of arranging the elements of a list or array into a natural ordering.

Java's class libraries contain several methods for searching and sorting arrays and lists, such as [redacted] and [redacted].

A [redacted] object describes a customized way to compare objects, enabling arrays or lists of these objects to be searched and sorted in many orders.

Empirical analysis is the technique of running a program or algorithm to determine its runtime. Algorithm analysis is the technique of examining an algorithm's code or pseudocode to make inferences about its complexity.

Algorithms are grouped into complexity classes, which are often described using big-Oh notation such as O(N) for a linear algorithm.

Sequential search is an O(N) searching algorithm that looks at every element of a list until it finds the target value.

Binary search is an O(log N) searching algorithm that operates on a sorted dataset and successively eliminates half of the data until it finds the target element.

Selection sort is an O(N2) sorting algorithm that repeatedly finds the smallest unprocessed element of the array and moves it to the frontmost remaining slot of the array.

Merge sort is an O(N log N) sorting algorithm, often implemented recursively, that successively divides the input dataset into two halves, recursively sorts the halves, and then merges the sorted halves into a sorted whole.

Self-Check Problems

Section 13.1: Searching and Sorting in the Java Class Libraries

1. Describe two ways to search an unsorted array of [redacted] objects using the Java class libraries.
2. If you perform a binary search on an array of one million integers, which of the following is closest to the number of elements that the search algorithm will need to examine?
   a. all 1,000,000 of the integers
   b. roughly 3/4 (750,000) of the integers
   c. roughly half (500,000) of the integers
   d. roughly 1/10 (100,000)
   e. less than 1% (10,000 or fewer)
3. Should you use a sequential or binary search on an array of [redacted] objects, and why?
4. Under what circumstances can the [redacted] and [redacted] methods be used successfully?
5. In what order does the [redacted] method arrange a list of strings? How could you arrange them into a different order?
6. Why wouldn't the [redacted] method work when used on a list of [redacted] objects? How can you make it so that the sort method can be used on [redacted] or any other type of objects?
7. The following [redacted] class is attempting to arrange [redacted] objects by account name, breaking ties by account balance. But the code has some syntax errors and some logic errors. What is wrong with the code? How would you correct it?

```
import


public class


public int


if


return this


else


return this

```

8. In this section, we wrote a [redacted] that would allow a list or array of strings to be sorted in ascending order by length. How could we easily sort the collection in descending order by length, from longest to shortest, without modifying the code of [redacted]?

Section 13.2: Program Complexity

9. Approximate the runtime of the following code fragment, in terms of n:

10. Approximate the runtime of the following code fragment, in terms of n:

11. Approximate the runtime of the following code fragment, in terms of n:

12. Approximate the runtime of the following code fragment, in terms of n:

13. Approximate the runtime of the following code fragment, in terms of n:

14. Determine the complexity classes of the algorithms that could be used to perform the following tasks:
    a. Finding the average of the numbers in an array of integers
    b. Finding the closest distance between any pair of points in an array of
    c. Finding the maximum value in an array of real numbers
    d. Counting the median length of the in an array
    e. Raising an integer to a power—for example, AB
    f. Examining an array of to see how many trios of points are colinear—that is, how many groups of three points could be connected by a straight line
    g. Counting the number of lines in a file
    h. Determining whether a given integer representing a year stores a leap year (a year divisible by 4, but not divisible by 100 unless also divisible by 400)
15. Suppose an algorithm takes exactly the given number of statements for each value below, in terms of an input size N. Give a tight big-Oh bound for each algorithm, representing the closest complexity class for that algorithm based on that runtime.
    a. ½N log N+log N
    b. N2−(N+N log N+1000)
    c. N2 log N+2N
    d. ½(3N+5+N)
    e. (2N+5+N4)/N

f. log (2N)
g. N!+2N

Section 13.3: Implementing Searching and Sorting Algorithms

16. What is the runtime complexity class of a sequential search on an unsorted array? What is the runtime complexity class of the modified sequential search on a sorted array?
17. Why does the binary search algorithm require the input to be sorted?
18. How many elements (at most) does a binary search examine if the array contains 60 elements?
19. What indexes will be examined as the middle element by a binary search for the target value [redacted] when the search is run on the following input arrays? What value will the binary search algorithm return?

    a. [redacted]

    b. [redacted]

    c. [redacted]

    d. [redacted]

20. What indexes will be examined as the middle element by a binary search for the target value [redacted] when the search is run on the following input array? Notice that the input array isn't in sorted order. What can you say about the binary search algorithm's result?

[redacted]

21. Consider the following sorted array of integers. When a binary search is performed on this array for each of the following integer values, what indexes are examined in order? What result value is returned?

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]

22. Consider the following sorted array of integers. When a binary search is performed on this array for each of the following integer values, what indexes are examined in order? What result value is returned?

a. [redacted]
b. [redacted]
c. [redacted]

d. [redacted]

23. What modifications would you have to make to the [redacted] method to cause it to sort an array of [redacted] values rather than one of integer values?
24. Consider the following array:

[redacted]

After a single pass of the selection sort algorithm (a single swap), what would be the state of the array?

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]
e. [redacted]

25. Trace the execution of the selection sort algorithm as shown in this section when run on the following input arrays. Show each element that will be selected by the algorithm and where it will be moved, until the array is fully sorted.

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]

Section 13.4: Case Study: Implementing Merge Sort

26. How many calls on the [redacted] method are generated by a call to sort a list of length 32?
27. Consider the following array of [redacted] elements:

    a. Show the state of the elements after five passes of the outermost loop of selection sort have occurred.
    b. Show a trace that is two levels deep of the merge sort algorithm. Show the splitting of the overall array, plus one level of the recursive calls.
28. Consider the following array of [redacted] elements:

    a. Show the state of the elements after five passes of the outermost loop of selection sort have occurred.
    b. Show a trace that is two levels deep of the merge sort algorithm. Show the splitting of the overall array, plus one level of the recursive calls.
29. Which one of the following statements about sorting and big-Oh is true?

a. Selection sort can sort an array of integers in O(N) time.
b. Merge sort achieves an O(N log N) runtime by dividing the array in half at each step and then recursively sorting and merging the halves back together.
c. Merge sort runs faster than selection sort because it is recursive, and recursion is faster than loops.
d. Selection sort runs in O(N) time if the array is already sorted to begin with, or O(N2) if it is not.
e. Sorting algorithms that rely on comparing elements can only be used with type [redacted], because values from other types of data cannot be compared to each other.

30. Trace the complete execution of the merge sort algorithm when called on each array below. Show the sub-arrays that are created by the algorithm and show the merging of sub-arrays into larger sorted arrays.

a. [redacted]
b. [redacted]
c. [redacted]

Exercises

1. Suppose the following array has been declared:

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

   a. [redacted]
   b. [redacted]
   c. [redacted]
   d. [redacted]

2. Suppose the following array has been declared:

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]

3. Suppose the following array has been declared:

[redacted]

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

a. [redacted]
b. [redacted]
c. [redacted]
d. [redacted]

4. To which complexity class does the following algorithm belong? Consider N to be the length or size of the array or collection passed to the method. Explain your reasoning.

[redacted]

5. To which complexity class does the following algorithm belong?

6. To which complexity class does the following algorithm belong?

7. To which complexity class does the following algorithm belong?

8. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

9. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.
10. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

11. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.
12. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

13. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.
14. Write code to read a dictionary from a file, then prompt the user for two words and tell the user how many words in the dictionary fall between those two words. Here is a sample run of the program:

Use the binary search algorithm in your solution.

15. Write a [redacted] that compares [redacted] objects by their distance from the origin of (0, 0). Points that are closer to the origin are considered to come before those which are further from the origin.
16. Write a [redacted] that compares [redacted] objects by the number of words they contain. Consider any nonwhitespace string of characters to be a word. For example, “hello” comes before “I see”, which comes before “You can do it”.
17. Write a [redacted] that compares [redacted] objects of a particular format. Each string is of a form such as [redacted], beginning with a numeric token that is followed by additional text tokens. Your job is to treat the first tokens as integers and compare them in numerical order. You cannot simply compare them by using the strings’ [redacted] method, since it would treat the numbers as text and not as integers. For example, [redacted] is greater than [redacted]. Use a [redacted] to tokenize the strings while comparing them.
18. Write a modified version of the selection sort algorithm that selects the largest element each time and moves it to the end of the array, rather than selecting the smallest element and moving it to the beginning. Will this algorithm be faster than the standard selection sort? What will its complexity class (big-Oh) be?
19. Write a modified “dual” version of the selection sort algorithm that selects both the largest and smallest

elements on each pass and moves each of them to the appropriate end of the array. Will this algorithm be faster than the standard selection sort? What predictions would you make about its performance relative to the merge sort algorithm? What will its complexity class (big-Oh) be?

20. Implement an algorithm to shuffle an array of numbers or objects. The algorithm for shuffling should be the following:

```
for (each index i) {

choose a random index j where j >= i.

swap the elements at indexes i and j.

}
```

(The constraint about ▮ being greater than or equal to ▮ is actually quite important, if you want your shuffling algorithm to shuffle fairly. Why?)

21. Implement a “bogus” sorting algorithm called *bogo sort* that uses your shuffling algorithm from the previous exercise to sort an array of numbers. The bogo sort algorithm is the following:

```
while (array is not sorted) {
```

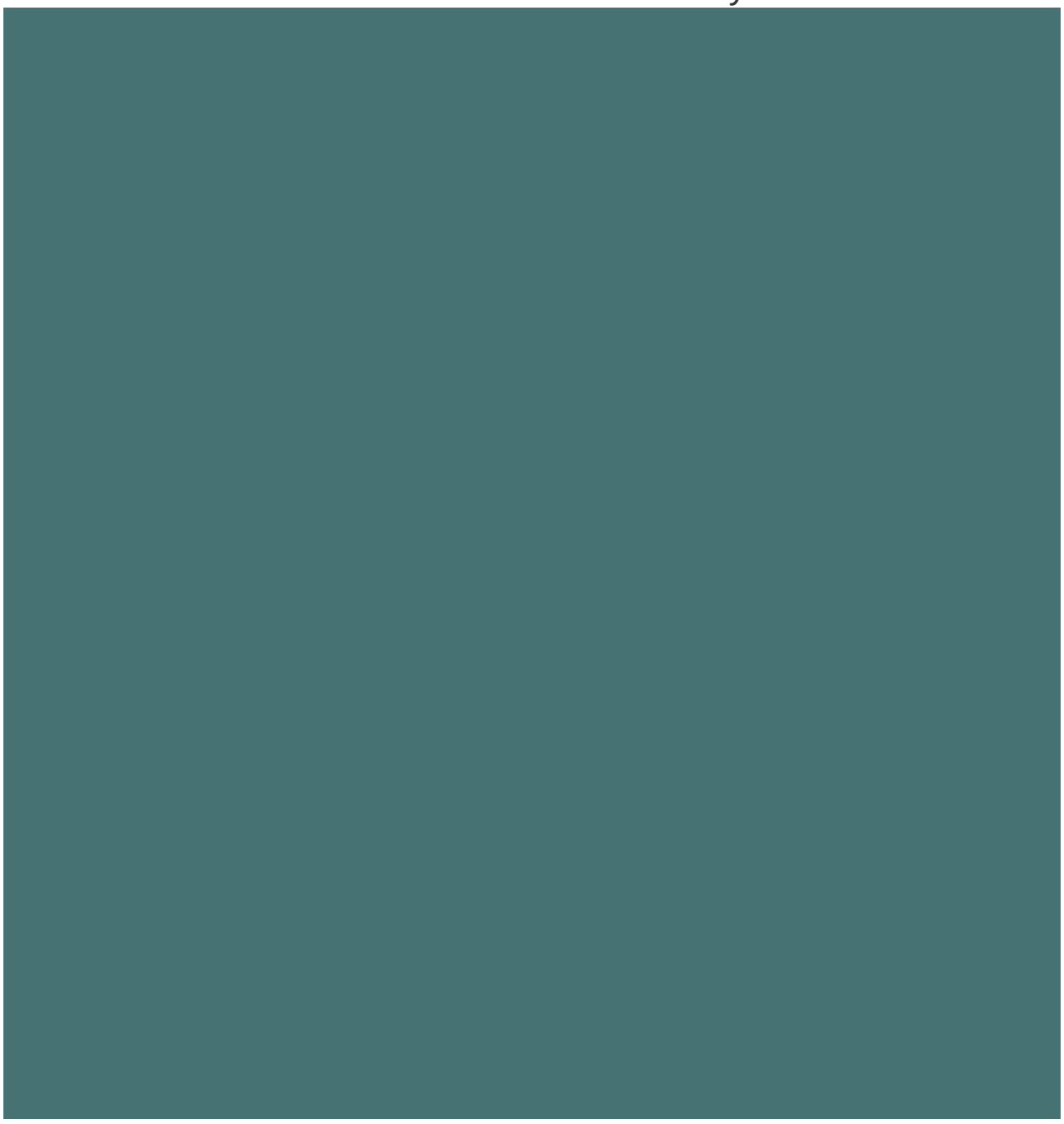
```
shuffle array.

}
```

Obviously, this is not a very efficient sorting algorithm, but it eventually does shuffle the array into order if you let it run long enough. Try running it on a very small array, such as 8 or 10 elements, to examine its runtime. What is your best guess about the complexity class (big-Oh) of this silly algorithm?

Programming Projects

1. Write a program that reads a series of input lines and sorts them into alphabetical order, ignoring the case of words. The program should use the merge sort algorithm so that it efficiently sorts even a large file.
2. Perform a “Sort Detective” challenge to run several sorting algorithms without knowing which is which. Try to figure out which sorting algorithm is which on the basis of the runtime and characteristics of each algorithm. Search the web for “sort detective” for more ideas on such a project.
3. Write a program that processes a data file of students’ course grade data. The data arrive in random order; each line stores information about a student’s last name, first name, student ID number, grade as a percentage, and letter grade. For example, here are a few lines of data:

Your program should be able to sort the data by any of the columns. Use [redacted] to achieve the sort orderings. Make the data sortable by last name, student ID, and grade percentage in ascending and descending order. For example, here are the lines of student data sorted a few different ways:



4. Write a program that discovers all anagrams of all words listed in an input file that stores the entries in a large dictionary. An anagram of a word is a rearrangement of its letters into a new legal word. For example, the anagrams of “share” include “shear”, “hears”, and “hares”. Assume that you have a file available to you that lists many words, one per line. Your program should first read in the dictionary file and sort it, but instead of sorting in alphabetical order it should sort according to each word’s canonical form. The canonical form of a word contains the same letters as the original, but in sorted order. Thus, the canonical form of “computer” is “cemoprtu”, and the canonical form of “program” is “agmoprr”. When your dictionary file is sorted, the word “program” would be placed before the word “computer”, because its canonical form comes first in alphabetical order. Write code to retrieve a word’s canonical form and a [redacted] that compares words by using their canonical forms.

Chapter 14 Stacks and Queues

14.1 Stack/Queue Basics
• Stack Concepts
• Queue Concepts
14.2 Common Stack/Queue Operations
• Transferring between Stacks and Queues
• Sum of a Queue
• Sum of a Stack
14.3 Complex Stack/Queue Operations
• Removing Values from a Queue
• Comparing Two Stacks for Similarity
14.4 Case Study: Expression Evaluator
• Splitting into Tokens
• The Evaluator

Introduction

In Chapter 11, we saw that the Java Collections Framework incorporates the idea of an abstract data type (ADT) with interfaces defined for the most fundamental ADTs and a variety of implementations for each. In this chapter we are going to explore two of the most fundamental ADTs in computer science, called stacks and queues. They are so simple that they almost seem not worth studying. They are like

the programming equivalent of drawers and shelves. Drawers and shelves are very simple and, therefore, sort of boring, and yet we find uses for them everywhere we turn.

It is useful to study stacks and queues as a way to understand a minimal kind of data structure. We'll find, for example, that they are less powerful than the list structures we have been looking at. But we often find ourselves wanting to think in terms of the simplest possible solution to a problem, as in "Could you solve that with just a stack?" They are also part of the basic culture of computer science, so you will find it helpful to practice using these primitive structures and learning the terminology associated with each.

14.1 Stack/Queue Basics

Like a list, a stack or queue stores an ordered sequence of values. A minimal set of operations for such a structure would require at least:

- some way to put values into the structure (an “add” operation)
- a way to take values out (a “remove” operation)
- a way to test whether there is anything left in the structure (is the structure empty?)

These three operations are the bare bones that you’d need for such a structure and in their purest form, stacks and queues have just these three operations. Java’s version of these also includes a [redacted] method that lets you ask for the number of elements in the structure and a [redacted] method that lets you examine the next value to be removed without actually removing the value.

Stacks and queues are similar in that they each store a sequence of values in a particular order. Stacks are LIFO structures (Last-In, First-Out), meaning that the most recently inserted element is the one that can be accessed first. Queues are FIFO structures (First-In, First-Out), meaning that the first inserted element is the one that can be accessed first.

L[redacted]	F[redacted]
I[redacted]	I[redacted]

First	First
Out	Out

Stack Concepts

With a stack, all of the action occurs at one end of the structure that we refer to as the top of the stack, as shown in Figure 14.1. The adding operation is called a *push* and the removing operation is called a *pop*. We push values onto the top and we pop them off the top. A good analogy for stacks is to think of a cafeteria and how trays are stacked up for the customers. When you go to get a tray, you take the one on the top of the stack. You don't bother to try to get the one on the bottom, because you'd have to move a lot of trays to get to it. Similarly, if someone brings clean trays to add to the stack, they are added on the top rather than on the bottom. The result is that stacks tend to reverse things. Each new value goes to the top of the stack, and when we take them values back out, we draw from the top, so they come back out in reverse order.

Figure 14.1 A stack

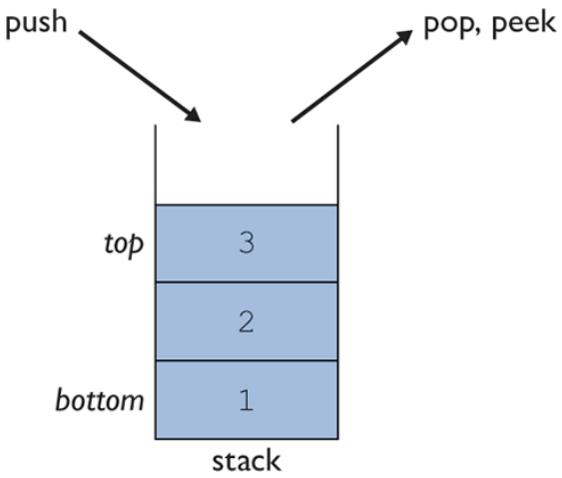


Figure 14.1 Full Alternative Text

There are also methods for testing whether the stack is empty, requesting the current size of the stack, and peeking at the next value to be removed. The basic operations for the [redacted] class are listed in Table 14.1.

Table 14.1 Useful Methods of the [redacted] Class

[redacted]	
[redacted]	Pushes the given value onto the top of the stack
[redacted]	Removes and returns the value at the top of the stack
[redacted]	Returns [redacted] if this stack contains no values
[redacted]	Returns the value at the top of the stack without removing it
[redacted]	Returns the number of values in the stack

Stacks are generic structures like the other collections we have seen, so it is really [redacted], where [redacted] is filled in with some type. The [redacted] class is one of the oldest classes in the Java Collections Framework. It was included with the original release of Java. As a result, it was not designed as well as the later ADTs in the framework. In particular, there is no separate interface that specifies the ADT. Instead, it extends a class called [redacted], which is an early version of the [redacted] class. Because of this inheritance relationship, you can treat the stack like a list. For example, you can add and remove values in the middle.

Stack

[redacted]

An ordered collection that allows adding and removing the “top” element, providing Last-In, First-Out (LIFO) access.

Even though it is badly designed, we think it is important to use the built-in version of this class. We highly recommend that you restrict yourself to the stack-like methods of Table 14.1 and not use other methods that would allow you to treat the stack like a list. The following [redacted] is a complete program that shows basic manipulations of a stack.

[redacted]

```
import


public class


public static void


new


for
```

```
while
```

It produces the following output:

```
```

[redacted]

You can see from the first line of output that the [redacted] class has a [redacted] method that formats the contents the same way a list is formatted. The first value listed inside brackets is the value at the bottom of the stack.

The second line of output uses the [redacted] method to report that there are 6 values in the stack. The third line of output uses the [redacted] method to report that the top of the stack is [redacted]. Keep in mind that this peeking operation does not change the stack. The final line of output shows the sequence of values obtained by popping the stack until it becomes empty. Notice that the values come out in reverse order relative to the order in which they were added.

Queue Concepts

For a queue we manipulate both ends of the structure, adding new values at the back of the queue and removing values from the front, as shown in Figure 14.2. Queues have the same basic operations that stacks have, but they have different names. The adding and removing operations are simply called add and remove, although sometimes we use the more formal terms *enqueueing* and *dequeueing* to describe these.

Figure 14.2 A queue

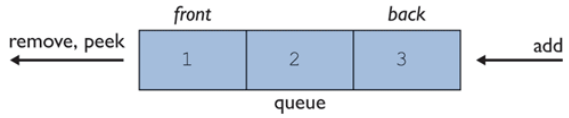


Figure 14.2 Full Alternative Text

A good analogy for queues is to think about standing in line at the grocery store. As new people arrive, they are told to go to the back of the line. When the store is ready to help another customer, the person at the front of the line is helped. In fact, the British use the word “queue” the way Americans use the word “line,” telling people to “queue up” or to “go to the back of the queue.” Table 14.2 includes the five fundamental operations we will use for manipulating queues.

Table 14.2 Useful Methods of the [redacted] Interface

[redacted]	
[redacted]	Adds the given value at the back of the queue
[redacted]	Removes and returns the value at the front of the queue
[redacted]	Returns [redacted] if this queue contains no values
[redacted]	Returns the value at the front of the queue without removing it
[redacted]	Returns the number of values in the queue

Queues were added to the Java Collections Framework later and they were properly designed. There is a separate [redacted] interface that describes the ADT and various implementations. We will use the [redacted] implementation for all of the queues that we construct.

Queue

An ordered collection that allows adding the “back” element and removing the “front” element, providing First-In, First-Out (FIFO) access.

The following is a simple client program that uses an array of data to initialize and manipulate a queue. The program’s output is similar to the earlier program, except for the order in which the elements are printed.

```
import

public class

public static void
```

```
new
```

```
while
```

The queue example produces the following output:

In the first line of output you can see that the first value inside brackets is the value at the front of the queue. That is why the [redacted] method returns [redacted]. And notice that the final line of output lists the values in the same order in which they were added to the structure.

14.2 Common Stack/Queue Operations

In this section we will become more familiar with stacks and queues by writing some client code for manipulating stacks and queues of [redacted] values. Recall that we have to use the [redacted] wrapper class to do so by constructing values of type [redacted] and [redacted]. Let's write some code that would add 10 random values between 0 and 99 to a queue using a [redacted] object:

Notice that our variable is of type [redacted] (using the interface). We only use the name of the implementation ([redacted]) when we are constructing a new object with the [redacted]

keyword. All variables, parameters, and return types should be defined using the interface.

This produces output like the following:

We can put this code for generating a queue with random values into a method. It's helpful to have a [redacted] parameter indicating how many values to add to the queue. The return type for the method should be [redacted]:

We can change our client code to call this method and print the result:

We can create a variation of [redacted] called [redacted] for making a stack of random values. This is a fairly straightforward modification where we simply switch queue operations with stack operations:

Transferring between Stacks and Queues

Consider the task of writing a method to transfer all values from a queue to a stack. We would need a loop that will remove values from the queue as long as there are more values left to remove. We can accomplish this with a [redacted] loop and the [redacted] and [redacted] methods:

Notice that the parameters are of type [redacted] and [redacted].

This code removes values from the queue, but to add them to the stack, we have to include a call on [redacted] inside the loop. The following is an incorrect attempt:

The problem with this version is that it calls [redacted] twice each time through the loop. It should be calling [redacted] with the value of [redacted]:

We could eliminate the line that involves [redacted] altogether and have just the second line of code:

The following client code reports what was in the stack and queue after calling [redacted]:

It produces output like the following:

We can write a similar method [redacted]:

Sum of a Queue

How would we write a method to find the sum of the values in a queue? It is a cumulative sum task, which involves initializing a [redacted] variable to [redacted] outside the loop and then adding each value to the sum as we progress through the loop. A first attempt might look like this:

If you call this version of the method and print the queue afterward, you'll find that the queue is empty. As a side effect of calculating the sum, we are destroying the contents of the queue. This is generally not acceptable behavior.

So how do we examine each of the queue elements while preserving the state of the structure? You could ask the queue for an iterator or use a for-each loop to get each value from the structure, but we are interested in understanding the queue in its most basic form, so we aren't going to use either iterators or for-each loops when working with stacks and queues.

The only other way to find out what is in a queue is to remove each of the values. We just need to find a way to do that while restoring the queue to its original form when we are done. One solution is to use a second queue as auxiliary storage:

```
Queue<Integer> temp = new LinkedList<>();


temp.add(n);


while (!temp.isEmpty()) {     // restore the queue

q.add(temp.remove());

}
```

This approach works, but there is an easier way. Why not use the original queue itself? As we remove values to be processed, we can add them back into the queue at the end. Think of it as cycling through values where the value at the front goes to the back.

Of course, then the queue never becomes empty. So instead of a loop looking for an empty queue, we would write a loop using the size of the queue:

```
q.add(n);
```

Sum of a Stack

Now let's write a [redacted] method for stacks:

This method can also be called [redacted] because the two methods have different signatures. Remember that a signature of a method is its name plus its parameters. These are both called [redacted] and they both have just a single parameter, but the parameter types are different, so this is okay.

So how do we write the [redacted] method for stacks? We can start out by trying to simply substitute stack operations for queue operations:

Unfortunately, this code doesn’t work. We’d see output like this when we test it:

The sum of these numbers is not 850. We’re getting that sum because the loop pops the value 85 off the stack 10 different times and then pushes it back onto the top of the stack. With a queue, values go in at one end and come out the other end. But with a stack, all the action is at one end of the structure (the top). So this approach isn’t going to work.

In fact, you can’t solve this in a simple way with just a stack. You’d need something extra like an auxiliary structure. Consider how we could solve it if we were allowed to use one queue available (and only one queue; no other auxiliary structures) as temporary storage. Then we can put things into the queue as we take them out of the stack and after we have computed the sum,

we can transfer things from the queue back to the stack using our [redacted] method. Here is a second attempt:

```
Queue<Integer> q = new LinkedList<>();

q.add(n);

queueToStack(q, s);
```

This also doesn't work. Here is a sample execution:

There are two problems here. Only half of the values were removed from the stack and those values now appear in reverse order. Why only half? We are using a [redacted] loop that compares a variable [redacted] against the size of the stack. The variable [redacted] is going up by 1 while the size is going down by 1 every time. The result is that halfway through the process, [redacted] is large enough relative to size to stop the loop. This is a case where we want a [redacted] loop instead of a [redacted] loop:

Even this is not correct. It finds the right sum, but it ends up reversing the values in the stack. If we could use a stack instead of a queue as auxiliary storage, then this problem would go away. In many of our sample problems, we purposely restrict you to a particular kind of structure so that you can practice working within constraints. It becomes almost a brain teaser to think of how to solve the problem with a given structure. A stack would be more convenient, but that doesn't mean that you have to use a stack to solve the problem.

The problem is that by transferring the data from the stack into the queue and then back into the stack, we have reversed the order. The fix is to do it again so that it goes back to the original order. So we add two extra calls at the end of the method that move values from the stack back into the queue and then from the queue back into the stack:

```
stackToQueue(s, q);

queueToStack(q, s);
```

This is not the most efficient way to solve the problem, but it demonstrates that it can be done with a queue.

14.3 Complex Stack/Queue Operations

In this section, we will examine two harder stack/queue problems that allow us to explore some of the issues and common bugs that come up when manipulating these structures.

Removing Values from a Queue

Consider the task of removing all occurrences of a certain value from a queue of integers. For example, suppose that a variable called stores a reference to the following queue:

and we make the following call:

We would want the queue to store the following values after the call:

This seems like a fairly straightforward task, but it leads to a subtle bug if we aren't careful. This task would be easier to solve if we could use a second queue. Then we could copy the values we want to keep from the first queue to the second and then copy them back when we are done. But this problem can be solved without an auxiliary data structure.

We saw that we can cycle through the elements of a queue by repeatedly removing the front value and then adding it back at the end of the queue. For example, the following loop cycles through the values in the queue exactly once:

As a first attempt, we could modify this loop so that it doesn't add the value back if it is the value we want to remove:

```
if (n != value) {



}
```

This version doesn’t work. For the sample queue, if we remove [redacted], we end up with these values in the queue after the method finishes executing:

It’s obvious that it didn’t work because the queue now starts with a [redacted]. Even worse, the order of the elements has been damaged. The original queue started with the value [redacted], which is now the third value in the queue. So it didn’t finish the task and it has changed the order of the elements in the queue.

The problem is that our loop is using a counter to keep track of how many times to execute and it is comparing the counter against the size of the queue. But the size is changing. As we come across values to be removed, we skip them rather than adding them back into the queue, which makes the queue size go down by 1.

What happened in this specific case is that we came across two values of [redacted] that weren't added back in. That reduced the queue size by 2. As a result, the final two values from the original queue were never processed. That allowed a [redacted] to get past our filtering process, and it changed the order of the values in the queue because we didn't make a full pass through the data.

This is a subtle bug that can come up often when working with queues. You may think that you are making a complete cycle through the values when you are actually off by some amount that causes you to cycle either too little or too much.

In case the bug still isn't clear, let's look at a more extreme case. Suppose the queue consists entirely of six occurrences of the value [redacted]:

If we call [redacted] and pass this queue and the value [redacted], the code should completely empty the queue. But the loop continues until the counter [redacted] reaches the queue's [redacted]. The algorithm goes through the following iterations:

After three loop iterations and elements removed, the value of has reached the value of , so the loop stops executing. The problem is that the queue is shrinking, so the method will return different values on different iterations of the loop. Regardless of how many elements are removed, we always want the loop to repeat once for each element in the original queue. So we need to store the initial size of the queue in a variable and use that to control the loop:

```
int oldSize = q.size();
```

```
oldSize
```

This version works properly.

Comparing Two Stacks for Similarity

Consider the task of comparing two stacks of integers to see if they have the same pattern of parity (odd versus even). In other words, we want to test whether two stacks have even numbers and odd numbers in the same positions. To be considered equivalent, we will also require that the stacks have the same size. For example, suppose variables called [redacted] and [redacted] store these values:

We expect the call of [redacted] to return [redacted] because both stacks store values with an (odd, even, odd, even, ...) parity pattern. To solve this problem, we will need an auxiliary data structure. It would be most convenient to have two auxiliary

stacks, but suppose that we have been restricted to one auxiliary stack. The basic form of our method will be to set up our auxiliary stack and examine values until we find a solution. We can also add some code to make sure that the two stacks are of the same size. We can begin our solution with this code:

Our task is to remove values from the two stacks and make sure they have the same parity. But we also have to be able to restore the two stacks to their original state. We have to use the third stack to store the values that we remove from the other two. If we find a pair of values that have different parity, then we would want to stop examining values. You might imagine writing code like the following:

The preceding approach won’t work. The code properly removes one value from each stack and stores the values into the third stack. But it returns from the method when it finds a difference. You have to be careful that you don’t exit the method prematurely. Remember that we have to restore the stacks to their original state. So even though we know that we want to return [redacted], we have to be careful to restore the stacks before we exit. We can accomplish this by introducing a [redacted] flag to keep track of whether we have seen a difference and stopping the loop when a difference has been encountered:

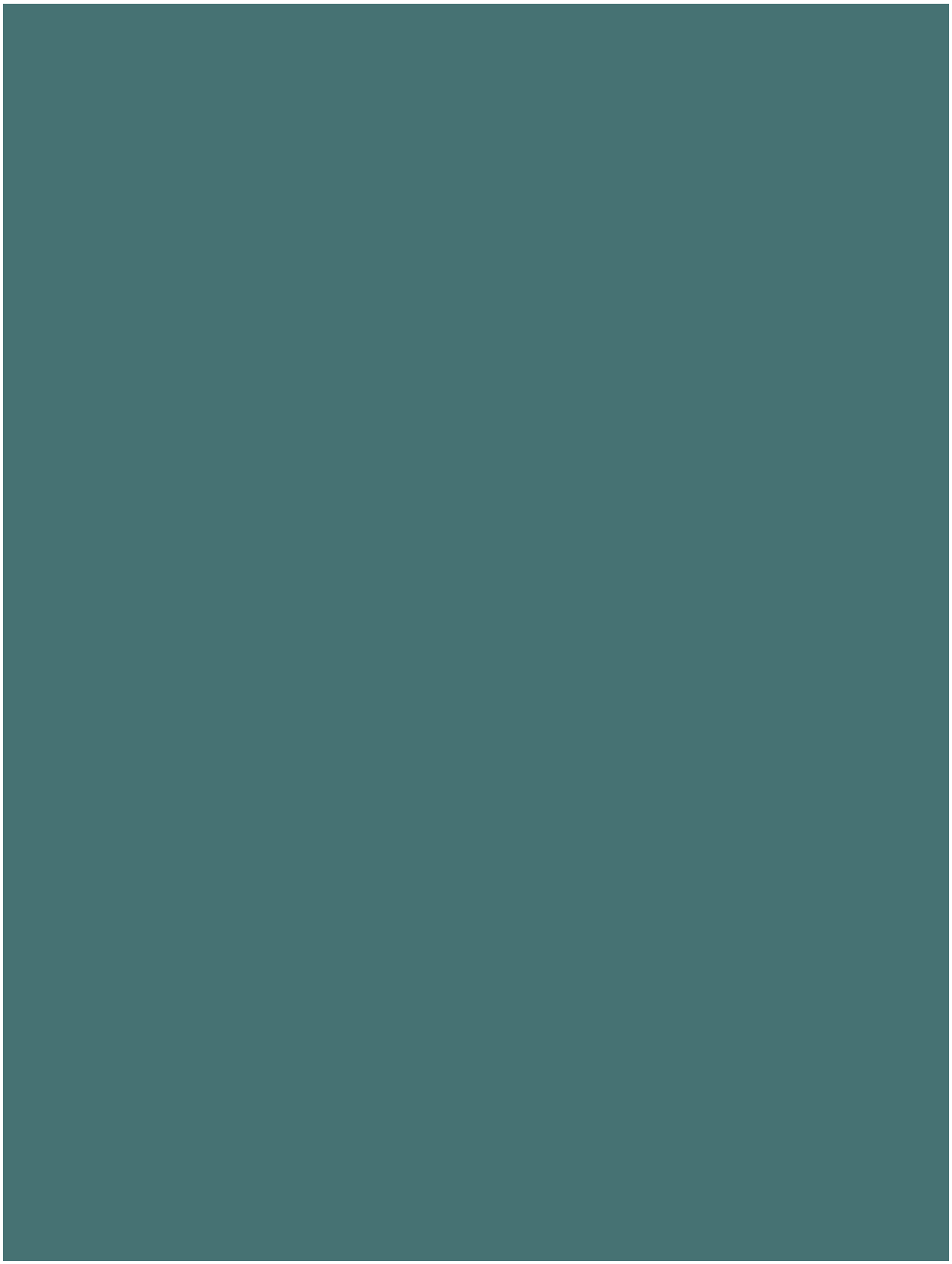
```
same &&



same = false;
```

Now we need a loop that will restore the stacks to their original state. Our third stack has pairs of values that were originally in the other two stacks. We can repeatedly put each pair of values back to the appropriate stacks. In each pair, we first pushed a value from [redacted] and then pushed a value from [redacted], so you might think that the correct loop to write is:

```

```

The problem with this is that even though we pushed a value from [redacted] and then a value from [redacted], they will come back out of the stack in the opposite order. So we need to reverse the order of the two lines inside our loop:

This completes our method other than the return statement. Once we have restored the stacks, we can use our [redacted] flag to report whether or not the stacks had the same pattern. The following is a complete solution to the method:



14.4 Case Study: Expression Evaluator

In Chapter 12, we saw how to write a program that uses recursion to evaluate prefix expressions. We also saw that recursive solutions take advantage of the method call stack. But what if you didn't have recursion available to you? It turns out that many problems that are easily solved with recursion are also easily solved with a stack.

We are going to write a program that evaluates fully parenthesized arithmetic expressions using the standard operators for addition, subtraction, multiplication, and division, and a special operator for exponentiation ( ).

For example, if we wanted to evaluate this expression:

18.4−2.3×8.519.5+2.74.9

We would write it as a fully parenthesized expression as:

We have conventions that allow us to leave out many of the parentheses in this expression, but it is very challenging to process such expressions because then we have to handle precedence issues. We will keep it simple by always requiring parentheses for every operator.

In the Chapter 12 case study, we assumed that spaces would be used to separate all of the individual tokens. That allowed us to use a [redacted] for reading the tokens, but it can be very annoying to have to include spaces for each individual element. For example, the expression above would have to be rewritten as:

It turns out that separating a string like this into tokens leads to an interesting application for a queue. So we will write a more sophisticated version this time that allows you to have as little or as much spacing as you'd like.

We also should be careful to report most errors if the user leaves off parentheses or doesn't have them matched properly. This error checking is a nice complement to the other operations we will be performing to evaluate these expressions.

But we will avoid one thorny issue. We won't allow numbers to have a minus sign in front of them. So even though it makes sense to form an expression like this:

We won't allow that minus sign in front of the [redacted]. Instead you'd have to form an expression like:

Splitting into Tokens

We have seen that the [redacted] class can be used to tokenize a string using whitespace, but we want to allow a user to leave out the whitespace and still have his or her input properly tokenized. We are going to build a supporting class called [redacted] that will solve this fairly specialized task. It will behave somewhat like a [redacted], but it won't require whitespace to break apart the string.

We can use the standard method names from the [redacted] class for getting a next token and asking if there is another token. We should also introduce a [redacted] method that allows the client to ask about the next token without actually reading it. And we will need a constructor that takes the string to split as a parameter. So the public methods will be:

This task is going to involve scanning through the letters of a string from beginning to end and returning to the client of the class the individual tokens that we find. Our class has to keep track of what has been read already and what it has left to read. It will also require a lot of peeking ahead to figure out how to separate the individual characters into tokens. For example, if the string is [redacted], we will want to produce this sequence of tokens:

As we are reading a number like [redacted], we will need to peek ahead to see what comes next so that we can recognize the end of the number.

It turns out that a queue is a great structure for solving this problem. It allows us to examine all of the characters from beginning to end and it keeps track of what we have already looked at and what we have left to look at. It also allows us to peek ahead.

The big difference between our splitter and a simple [redacted] is that we need to recognize certain special characters as being tokens. If we encounter a parenthesis or one of the arithmetic operators, we have to make that a token whether or not it is surrounded by whitespace. It is best to include these special characters in a string constant for the class.

In terms of fields, since we are reading a character at a time, we will want a [redacted] to store the individual characters of the string and a field for storing the next token. Because we want to allow the client to peek ahead, we should have the field for the token always store the next token to be processed (that way we can allow the client to peek at it as much as he or she wants without actually reading anything).

We have to provide the client with a way to ask whether there is a next token to be processed. We could use an extra field for this, but we could also just set the token field to [redacted] when there are no tokens left to process.

Here is a basic outline of our class including our two fields, our special constant, and a constructor that adds the letters of the string to our queue of characters:

Notice that the constructor ends with a call on a method called [redacted]. The idea is that we want a private method that will process the queue and give an appropriate value to the field called [redacted]. Most of the work of writing this class is to write that method.

To write the [redacted] method, we have to consider all of the possible cases and make sure that we handle each one. We want to skip whitespace just as the [redacted] does, so we will have to include code to do that. After skipping any leading whitespace, we would normally be ready to build up the next token. But we have to consider the case where we run out of characters, in which case there are no more tokens to produce. If we do have a token to produce, we'll have to remove characters one at a time from the queue and add them to the token that we are building up. The basic approach can be described with the following pseudocode:

```
token = null;

} else {


initialize token to the next queue character.


while (next queue character is part of this token) {


token = token + (next character from the queue).


}

}
```

To skip the leading whitespace, we can peek ahead in the queue to see if the next character is a whitespace character. We'll utilize a helpful static method from the [redacted] wrapper class called [redacted] that returns [redacted] if a given character is whitespace such as a space, tab, or line break. But we have to keep in mind we can't peek ahead when the queue is empty, so we have to include a special test for that as well:

```
```

Now we need to write the code for building up a token character by character. We can initialize the token to the next character in the queue by saying:

Then we run into the problem of knowing how many more characters to include in this token. We have a set of special characters that are supposed to be one-character tokens, so if the token is any of those, then we need to stop adding characters to the token. We can use our string constant and a call on to handle that special case:

Now we need a loop that will append characters to the current token until it finds something that isn’t part of the token. We would want to stop if we came across a whitespace character. But we would also want to stop if we came across any of the special characters. And we have an extra problem in that we might run out of characters completely if this is the last token to be processed.

The logic gets fairly complicated in this case, so it is helpful to introduce a [redacted] variable that keeps track of whether or not we are done:

This completes the private method for finding the next token. As mentioned earlier, the other parts of the class are fairly straightforward. The following is the complete class definition. Notice that the class has an extra private method called [redacted] that throws a [redacted] if the client calls the [redacted] or [redacted] methods when there are no more tokens left.

```
import

public class

private

private

public static final

public
```

```
new


for

int


public boolean


return
```

```
public
```

```
return
```

```
public

return

private void
```

```
while

if

else

if

boolean
```

```
while

char

if

else
```

```
private void

if

throw new
```

The Evaluator

Now that we have a support class that will allow us to read the tokens of a string, we can work on the code that will evaluate the tokens that we find in a fully parenthesized expression. We are going to implement a variation of a famous algorithm known as the shunting-yard algorithm that was invented by Edsger Dijkstra. It uses two stacks to save intermediate results. One stack stores numbers and the other stack stores symbols.

The basic idea is that we store values in the two stacks until we are ready to process them. As we see left parentheses and operators, we push them onto the symbol stack. As we see numbers, we push them onto the number stack. And when we see a right parenthesis, we know we have all of the information for a given sub-expression and we go ahead and evaluate it. We then push the result back onto the number stack.

Consider a simple case of evaluating [redacted]. Table 14.3 shows how the initially empty stacks have elements added to them until we encounter the right parenthesis, at which point we evaluate the sum and push the result onto the number stack.

Table 14.3 Evaluation of [redacted]

	Push onto symbol stack		
	Push onto number stack		
	Push onto symbol stack		
	Push onto number stack		
	Evaluate expression and push result onto number stack		

Notice that the overall value is the one and only value stored in the number stack when we are done and the symbol stack is empty when we are done. If there are other values left in either stack, then we know that we had an illegal expression.

This seems like a lot of work to do for a fairly simple computation. But remember that we can form complex sub-expressions that need to be evaluated as well. For example, what if the expression to evaluate had been ? This expression will have the same value because evaluates to and evaluates to . With the two stack approach, we can keep track of each part of this expression until we are ready to process it. Table 14.4 shows the evaluation of the more complex expression.

Table 14.4 Evaluation of

[redacted]	Push onto symbol stack	[redacted]	[redacted]
[redacted]	Push onto symbol stack	[redacted]	[redacted]
[redacted]	Push onto number stack	[redacted]	[redacted]
[redacted]	Push onto symbol stack	[redacted]	[redacted]
[redacted]	Push onto number stack	[redacted]	[redacted]
[redacted]	Evaluate expression and push result onto number stack	[redacted]	[redacted]
[redacted]	Push onto symbol stack	[redacted]	[redacted]
[redacted]	Push onto symbol stack	[redacted]	[redacted]
[redacted]	Push onto number stack	[redacted]	[redacted]
[redacted]	Push onto symbol stack	[redacted]	[redacted]
[redacted]	Push onto number stack	[redacted]	[redacted]
[redacted]	Evaluate expression and push result onto number stack	[redacted]	[redacted]
[redacted]	Evaluate expression and push result onto number stack	[redacted]	[redacted]

To write the code, we just have to implement the algorithm. It is easiest if we assume the input has no errors, but it's better to recognize errors when we can. We can't really recover from an error and it can get complex to report the nature of each error. So

let’s strike a middle ground of recognizing as many errors as we can, but giving just a simple error message if we encounter a mistake.

We know that we want to process tokens until we either encounter an error or run out of tokens. It is helpful to introduce a [redacted] flag that keeps track of whether an error has been seen. We expect to reach a point where the symbol stack is empty and the number stack has exactly one value in it. In that case, we could report that one number as the overall result. The basic structure of our solution is:

Our remaining task is to fill in the body of the [redacted] loop for processing tokens. Two of the cases are relatively simple. When we see a left parenthesis or operator, we push it on the symbol stack. When we see a number, we push it on the number stack. The hard part is when we see a right parenthesis. The body of our [redacted] loop will look like this:

Notice that in the final case we call the method [redacted] to convert the token from a string into a [redacted]. As noted above, two out of three of these cases are simple. The third case is the hard one. How do we process a right parenthesis?

If an expression is legal, then we should have encountered a left parenthesis at the beginning of it and two numbers to work with and an operator to evaluate. So we expect that the symbol stack will have an operator on the top of the stack and a left parenthesis just below it. The two values should be on the number stack. In general we will remove the operator and left parenthesis, remove the two numbers, and then apply the operator. What makes this complicated is that there might be all sorts of errors. We have to be careful to check at every step that we have what we are supposed to have.

One thing we know is that the only values that are pushed onto the symbol stack are legal operators and left parentheses. We can use the stack sizes for several of our tests, but we have to make sure that we have an operator and not a left parenthesis and that below it on the stack is a left parenthesis. The following code checks for these errors and evaluates one operator, pushing the result back onto the numbers stack:

The code above involves a call on a method called [redacted]. We wrote this method for the Chapter 12 case study. It takes an operator and two operands and returns the result of applying that operator to the two operands. Putting all of this together, we end up with the following complete program:

```
public class

public static void

new
```

```
while
```

```
public static void
```

```
new
```

new

new

boolean

while

if

if

```
else

if

else

double

double
```

```
double
```

```
else if
```

```
else
```

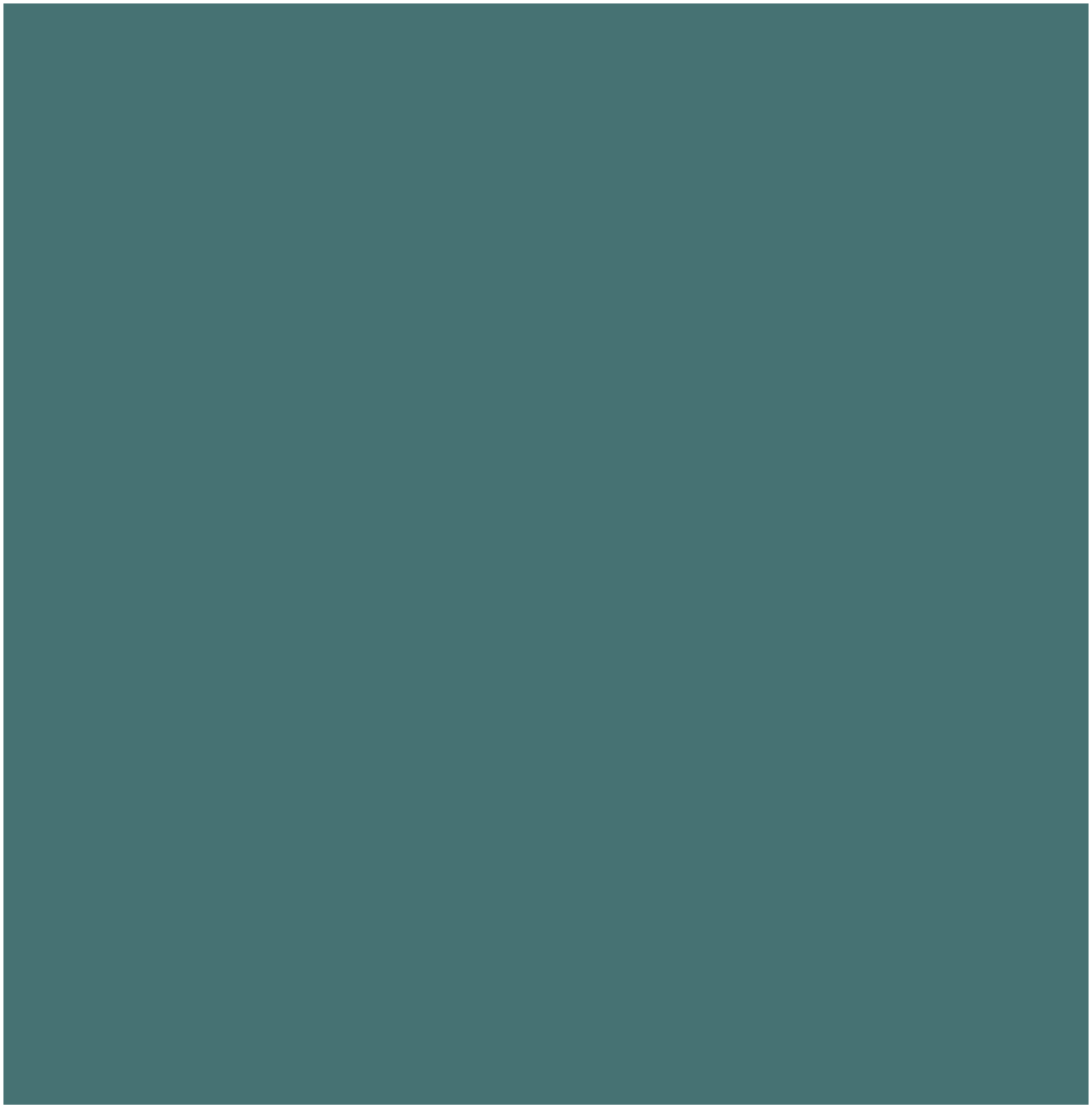
```
if
```

```
else

public static double

double
```

```
double

if

return

else if

return

else if

return

else if
```

```
return


else if


return


else


throw new
```

Below is a sample log of execution:



This program is fairly robust. The one bit of error checking it doesn't do is to handle illegal tokens. Any illegal tokens will be assumed to be numbers. The main token-processing loop includes a call on [redacted] that will throw a [redacted] if it encounters such a token. This could

be fixed by adding a [redacted] block that sets the error flag to true if the [redacted] is thrown.

Chapter Summary

A stack is a collection that allows you to add and remove elements from its top, providing “Last-In, First-Out” (LIFO) access.

The common operations of a stack include adding (“push”), removing (“pop”), testing whether the stack is empty, asking for the stack’s size, and “peeking” at the top element without removing it.

A queue is a collection that allows you to add elements to the back and remove elements from the front, providing “First-In, First-Out” (FIFO) access.

The common operations of a queue include adding (“enqueue”), removing (“dequeue”), testing whether the queue is empty, asking for the queue’s size, and “peeking” at the front element without removing it.

To process all elements of a stack, the collection must be emptied. If you want to examine the contents without damaging the collection, you must keep a backup and restore the data afterward.

To process all elements of a queue, you must either backup and restore the data or cycle values to the end of the queue as you process them.

The size of a stack or queue changes as its elements are processed and removed, so many algorithms to process these collections need to keep track of the collection’s size separately to avoid common bugs and pitfalls.

Self-Check Problems

Section 14.1: Stack/Queue Basics

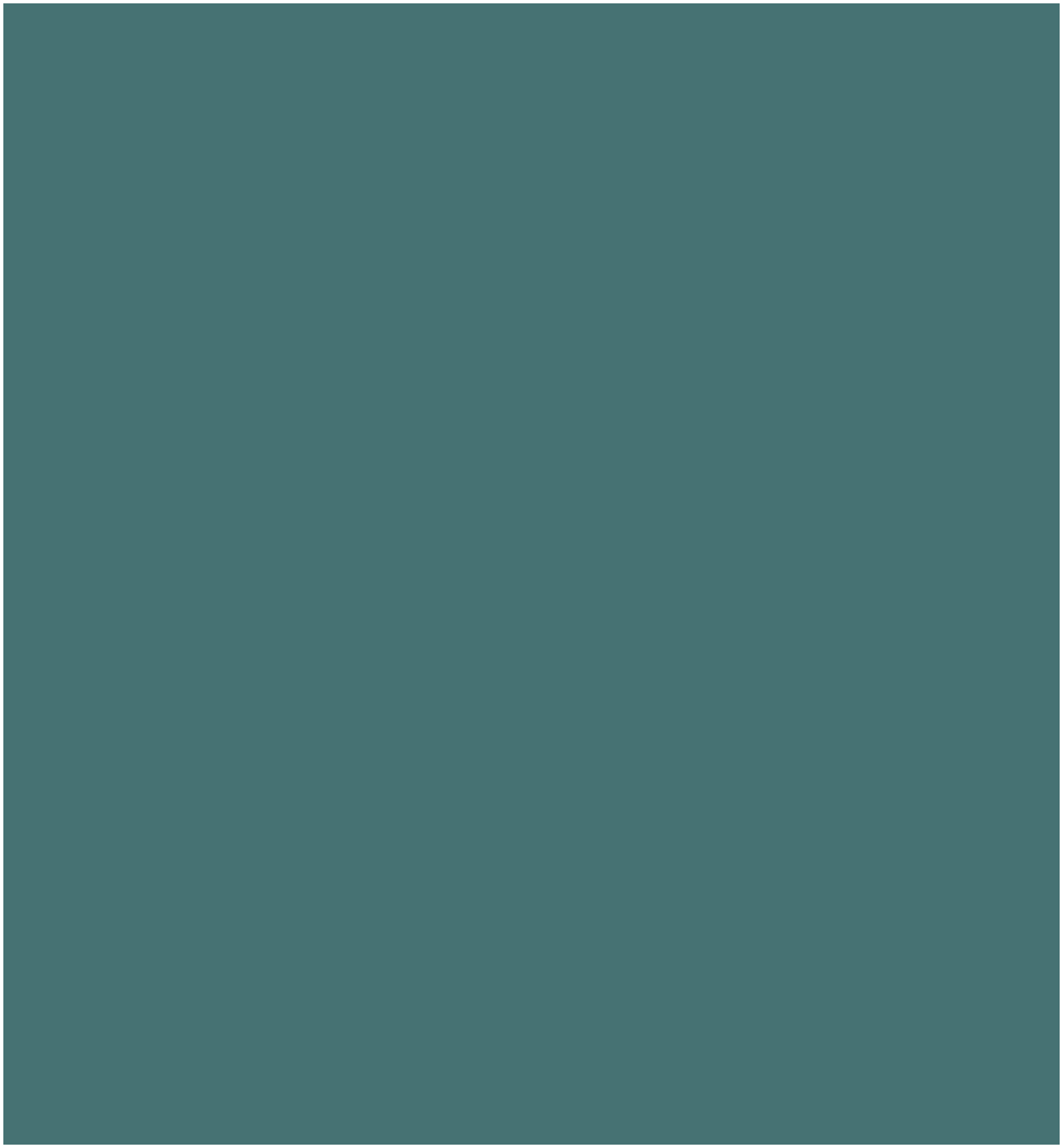
1. Which of the following statements about stacks and queues is true?
   a. Stacks and queues can store only integers as their data.
   b. A stack returns elements in the same order as they were added (first-in, first-out).
   c. A queue's [redacted] method removes and returns the element at the front of the queue.
   d. Stacks and queues are similar to lists, but less efficient.
   e. The [redacted] method allows access to the element at the bottom of a stack.
2. What is a real-world example of data that could be modeled using a stack? Using a queue?
3. When you call [redacted] on a stack, where is the new element placed relative to the other elements in the stack? When you call [redacted], which element from the stack is returned?
4. When you call [redacted] on a queue, where is the new element placed relative to the other elements in the queue? When you call [redacted], which element from the queue is returned?
5. If you create a new empty stack and push the values 1, 2, and 3 in that order, and call [redacted] on the stack once, what value will be returned?
6. If you create a new empty queue and add the values 1, 2, and 3 in that order, and call [redacted] on the queue once, what value will be returned?
7. The following piece of code incorrectly attempts to declare a queue of integers. What is wrong with the code, and how would you fix it?

8. Write a piece of code that declares a stack of strings and fills it with the following contents, such that [redacted] is at the top of the stack and [redacted] is at the bottom: [redacted]. You can print the stack to verify its state.
9. Write a piece of code that declares a stack of integers and uses a loop to fill it with the multiples of 2 from 0 through 100 inclusive, such that [redacted] is at the top of the stack and [redacted] is at the bottom: [redacted]. You can print the stack to verify its state.
10. Write a piece of code that declares a queue of strings and fills it with the following contents, such that [redacted] is at the front of the queue and [redacted] is at the back: [redacted]. You can print the queue to verify its state.

Section 14.2: Common Stack/Queue Operations

11. Stacks and queues do not have index-based methods such as [redacted] from [redacted]. How can you access elements in the middle of a stack or queue?
12. Stacks and queues have less functionality than other similar collections like lists and maps. Why are they still useful despite lacking functionality? What possible advantages are there of using a less powerful collection?
13. What is the output of the following code?

14. What is the output of the following code?



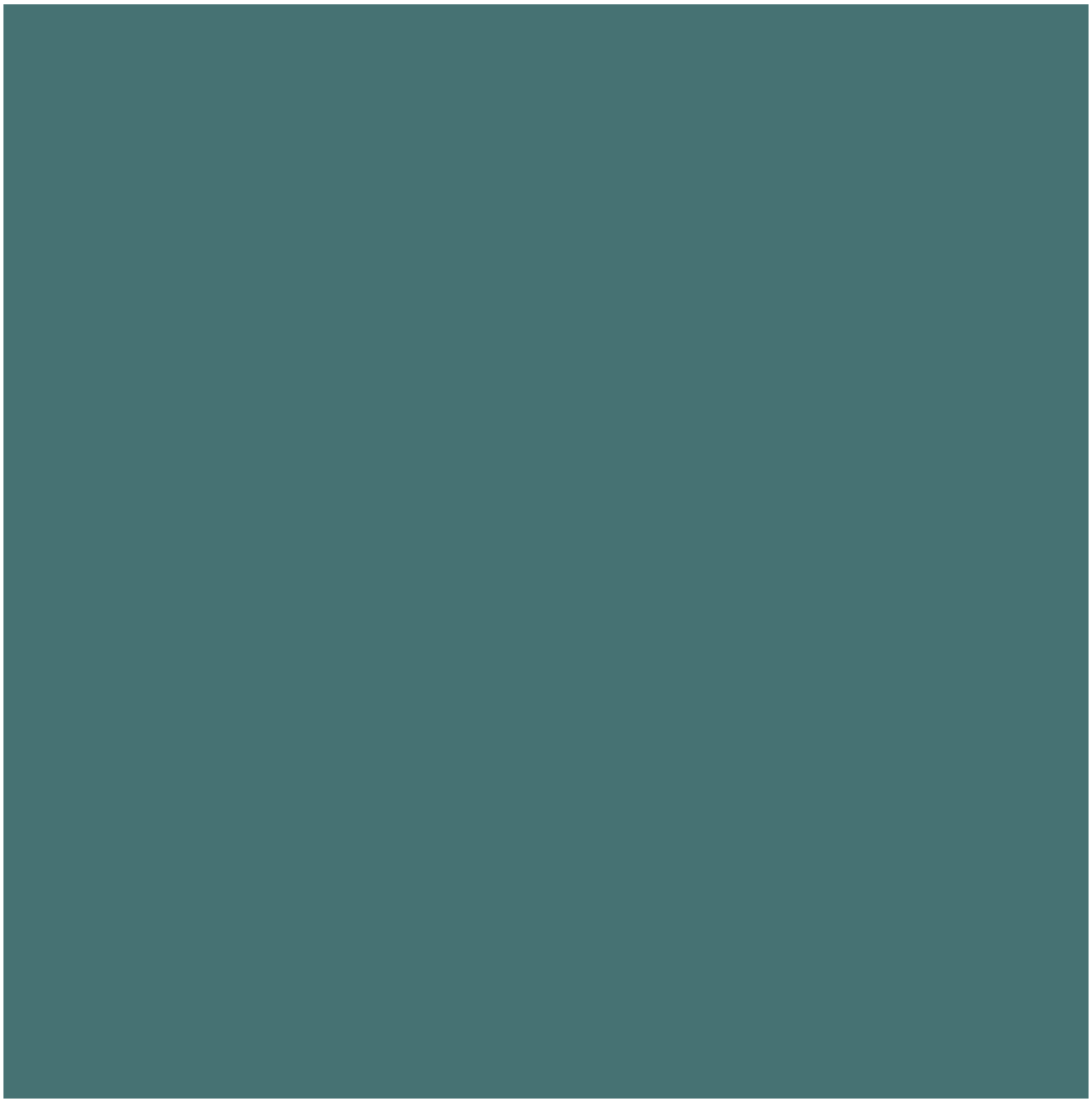
15. What is the output of the following code?

16. Write the output produced when the following method is passed each of the following stacks:

a. [redacted]
b. [redacted]
c. [redacted]

Section 14.3: Complex Stack/Queue Operations

17. Write the output produced when the following method is passed each of the following queues:



a. [redacted]
b. [redacted]
c. [redacted]

18. Write the output produced when the following method is passed each of the following queues:

[redacted]

a. [redacted]
b. [redacted]
c. [redacted]

19. The following piece of code incorrectly attempts to find the largest value in a queue of integers. What is wrong with the code, and how would you fix it?

20. The following piece of code incorrectly attempts to compute the sum of all positive values in a queue of integers. What is wrong with the code, and how would you fix it?

21. The following piece of code incorrectly attempts to remove all even values from a stack of integers. What is wrong with the code, and how would you fix it?

22. Write a piece of code that prints the elements of a queue of integers, one per line. When your code is done running, the queue should still contain the same contents as it had at the start. In other words, don't destroy the queue as you print it. If you like, put your code into a method called [redacted] that accepts the queue as a parameter.
23. Write a piece of code that finds and prints the longest string in a stack of strings. For example, in the stack [redacted], the longest string is [redacted]. When your code is done running, the stack should still contain the same contents as it had at the start. In other words, if you destroy the stack as you examine it, restore its state afterward. If you like, put your code into a method called [redacted] that accepts the stack as a parameter.

Exercises

Each problem will indicate what kind of structure to use as auxiliary storage. You should not use any other auxiliary data structures to solve the problems, although you can create as many simple variables as you'd like.

It is the authors' intent that you use stacks/queues in stack/queue-like ways only when solving these problems. For example, you should not call index-based methods such as [redacted], [redacted], or [redacted] (or use a for-each loop) on a stack/queue. You may call only [redacted], [redacted], [redacted], [redacted], [redacted], [redacted], and [redacted]. It is also possible to solve all of the exercises without using [redacted] if you want an extra challenge.

For problems that accept a stack or queue as a parameter, unless otherwise specified, you should make sure that your method does not damage the state of the parameter. That is, if you modify the parameter stack or queue's elements in your method, you should restore the parameter collection to its original state before your method returns.

Some of these problems have elegant recursive solutions, but the authors' intent is generally that you should not solve these problems recursively, because recursion can circumvent some of the tricky stack/queue manipulation that you are supposed to practice.

1. Write a method called [redacted] that accepts a stack of integers as a parameter and rearranges its elements so that all the negatives appear on the bottom of the stack

and all the nonnegatives appear on the top. If after this method is called you were to pop numbers off the stack, you would first get all the nonnegative numbers and then get all the negative numbers. It does not matter what order the numbers appear in as long as all the negatives appear lower in the stack than all the nonnegatives. For example, if the stack stores [redacted], an acceptable result from your method would be [redacted]. Use a single queue as auxiliary storage.

2. Write a method called [redacted] that accepts a stack of integers as a parameter and replaces every value in the stack with two occurrences of that value. Preserve the original relative order. For example, if the stack stores [redacted], your method should change it to store [redacted]. Use a single queue as auxiliary storage.
3. Write a method called [redacted] that accepts a stack of integers as a parameter and returns a copy of the original stack (i.e., a new stack with the same values as the original, stored in the same order as the original). Your method should create the new stack and fill it up with the same values that are stored in the original stack. When your method is done executing, the original stack must be restored to its original state. Use one queue as auxiliary storage.
4. Write a method called [redacted] that accepts a stack of integers as a parameter and that collapses it by replacing each successive pair of integers with the sum of the pair. For example, if the stack stores [redacted], the first pair should be collapsed into 9(7+2), the second pair should be collapsed into 17(8+9), and so on. If the stack stores an odd number of elements, such as the [redacted] at the end of our example stack, the final

element is not collapsed. So for this stack your method would yield [redacted]. Use one queue as auxiliary storage.

5. Write a method called [redacted] that accepts two stacks of integers as parameters and returns [redacted] if the two stacks store exactly the same sequence of integer values in the same order. Your method must restore the two stacks to their original state before returning. Use one stack as auxiliary storage.
6. Write a method called [redacted] that accepts a queue of integers as a parameter and rearranges the order of the values so that all of the even values appear before the odd values and that otherwise preserves the original order of the queue. For example, if the queue stores [redacted], your method should rearrange it to store [redacted]. Notice that all of the evens appear at the front followed by the odds and that the relative order of the evens and odds is the same as in the original. Use one stack as auxiliary storage.
7. Write a method called [redacted] that accepts a queue of integers as a parameter and reverses the order of all the elements in odd-numbered positions (position 1, 3, 5, etc.), assuming that the first value in the queue has position 0. For example, if the queue stores [redacted], your method should change it to store [redacted]. Notice that numbers in even positions (positions 0, 2, 4, 6) have not moved. That subsequence of integers is still (1, 7, 9, 12). But notice that the numbers in odd positions (positions 1, 3, 5, 7) are now in reverse order relative to the original. In other words, the original subsequence (8, 2, 18, 0) has

become (0, 18, 2, 8). Use a single stack as auxiliary storage.

8. Write a method called [redacted] that accepts a queue of integers as a parameter and returns [redacted] if the numbers in the queue are the same in reverse order. For example, if the queue stores [redacted], your method should return [redacted] because this sequence is the same in reverse order. If the queue stores [redacted], your method would return [redacted] because this sequence is not the same in reverse order (the [redacted] and [redacted] in the middle don't match). The empty queue should be considered a palindrome. Your method must restore the parameter queue to its original state before returning. Use one stack as auxiliary storage.
9. Write a method called [redacted] that accepts a stack of integers as a parameter and swaps neighboring pairs of numbers starting at the bottom of the stack. For example, if the stack initially stores [redacted], your method should swap the first pair (1, 2), the second pair (8, 6), the third pair (-1, 15), and so on. If the stack contains an odd number of elements, the element at the top should remain unmodified. So the final state of the stack would be [redacted]. Use one queue as auxiliary storage.
10. Write a method called [redacted] that accepts a stack of integers as a parameter and that returns [redacted] if the stack contains a sequence of consecutive integers starting from the bottom of the stack. Consecutive integers are integers that come one after the other, as in 3, 4, 5, etc. If the stack stores [redacted], your method should return [redacted]. If the stack had instead contained [redacted], your method should return [redacted] because the numbers [redacted] and [redacted] are not

consecutive. Notice that we look at the numbers starting at the bottom of the stack. Any stack with fewer than two values should be considered to be a list of consecutive integers. Your method must restore the parameter stack to its original state before returning. Use one queue as auxiliary storage.

11. Write a method called [redacted] that accepts a queue of integers as a parameter and that puts the integers into sorted (nondecreasing) order, assuming that the queue is already sorted by absolute value. For example, if the queue stores [redacted], notice that the values appear in sorted order if you ignore the sign of the numbers. Your method should reorder the values so that the queue stores [redacted]. Use one stack as auxiliary storage.
12. Write a method called [redacted] that accepts a stack of integers and an integer n as parameters and that shifts n values from the bottom of the stack to the top of the stack. For example, if the stack named [redacted] stores [redacted], and we make the call [redacted] your method should shift the three values at the bottom of the stack to the top of the stack and leave the other values in the same order, producing [redacted]. Notice that the value that was at the bottom of the stack is now at the top, the value that was second from the bottom is now second from the top, the value that was third from the bottom is now third from the top, and that the five values not involved in the shift are now at the bottom of the stack in their original order. Use one queue as auxiliary storage. You may assume that the parameter n is ≥0 and not larger than the number of elements in the stack.

13. Write a method called [redacted] that accepts a stack of integers as a parameter and makes sure that the stack's elements are in nondecreasing order from top to bottom, by removing from the stack any element that is smaller than any element(s) on top of it. For example, if the stack stores [redacted], the element values [redacted] and [redacted] should be removed because each has an element above it with a larger value. So your method should change the stack to store [redacted]. Notice that now the elements are in nondecreasing order from top to bottom. If the stack is empty or has just one element, nothing changes. Use one queue or stack (but not both) as auxiliary storage.
14. Write a method called [redacted] that accepts an integer k and a queue of integers as parameters and reverses the order of the first k elements of the queue, leaving the other elements in the same relative order. For example, if a queue named [redacted] stores [redacted], the call of [redacted] should change the queue to store [redacted]. If k is 0 or negative, no change should be made. If the queue does not contain at least k elements, your method should throw an [redacted]. Use one queue or stack (but not both) as auxiliary storage.
15. Write a method called [redacted] that accepts a stack of integers as a parameter and returns [redacted] if the elements in the stack occur in ascending (nondecreasing) order from top to bottom. That is, the smallest element should be on top, growing larger toward the bottom. For example, if the stack stores [redacted], your method should return [redacted]. An empty or one-

element stack is considered to be sorted. Your method must restore the parameter stack to its original state before returning. Use one queue or stack (but not both) as auxiliary storage.

16. Write a method called [redacted] that accepts a stack of integers as a parameter and replaces the stack contents with itself plus a mirrored version of itself (the same elements in the opposite order). For example, if the stack stores [redacted], your method should change it to store [redacted]. If passed an empty stack, your result should be an empty stack. Use one stack or one queue (but not both) as auxiliary storage to solve this problem.
17. Write a method called [redacted] that accepts a stack of integers as a parameter and that replaces each sequence of duplicates with a pair of values: a count of the number of duplicates, followed by the actual duplicated number. For example, if the stack stores [redacted], your method should change it to store [redacted]. This new stack indicates that the original had [redacted] occurrences of [redacted] at the bottom of the stack followed by 3 occurrences of [redacted] followed by 1 occurrence of 82, and so on. If the stack is empty, your method should not change it. Use one queue as auxiliary storage.
18. Write a method called [redacted] that accepts a queue of integers as a parameter and replaces each half of that queue with itself plus a mirrored version of itself (the same elements in the opposite order). For example, if the queue stores [redacted], your method should change it to store [redacted]. If your method is passed

an empty queue, the result should be an empty queue. If your method is passed a queue whose size is not even, throw an [redacted]. Use one stack or one queue (but not both) as auxiliary storage.

19. Write a method called [redacted] that accepts a stack of integers as a parameter and removes and returns the smallest value from the stack. For example, if the stack stores [redacted], your method should remove and return [redacted], leaving the stack storing [redacted]. If the minimum value appears more than once, all occurrences of it should be removed. For example, given the same stack, if we again call [redacted] on it, the method would return [redacted] and leave the stack storing [redacted]. Use one queue as auxiliary storage.
20. Write a method called [redacted] that accepts a queue of integers as a parameter and rearranges the elements by alternating the elements from the first half of the queue with those from the second half of the queue. For example, if the queue stores [redacted], your method should change it to store [redacted]. To understand the result, consider the two halves of this queue. The first half is [redacted] and the second half is [redacted]. These are combined in an alternating fashion to form a sequence of pairs: the first values from each half ([redacted] and [redacted]), then the second values from each half ([redacted] and [redacted]), and so on. Your method should throw an [redacted] if the queue does not have an even size. Use one stack as auxiliary storage.
21. Write a method called [redacted] that takes a stack of integers as a parameter and moves the largest value in the stack to the top of the stack, leaving all other values in

their original order. You may assume that the stack does not contain any duplicates. For example, if a stack [redacted] stores [redacted], the call of [redacted] should change it to store [redacted]. If the stack is empty, your method should not change it. Use one queue as auxiliary storage.

Programming Projects

1. Write a Primes program that finds prime numbers using the Sieve of Eratosthenes, an algorithm devised by a Greek mathematician of the same name who lived in the third century BC. The algorithm finds all prime numbers up to some maximum value n, as described by the following pseudocode:

```
create a queue of
numbers
 to process.

fill the queue with the integers 2 through
n
 inclusive.

create an empty
result
 queue to store primes.


repeat the following steps:


obtain the next prime
p
 by removing the first value


from the queue of
numbers
.
```

```
put
p
 into the
result
 queue of primes.


loop through the queue of
numbers
,


eliminating all numbers that are divisible by p.

while (
p
 is less than the square root of
n
).


all remaining values in the
numbers
 queue are prime,


so transfer them to the
result
 primes queue.
```

Several web sites have nice descriptions and animations of this algorithm in action; consider searching for “Sieve of Eratosthenes” in your web browser.

2. Write an HTML Validator program that reads files of HTML data and uses stacks and queues to verify whether the tags in the file are properly matched. A tag consists of a named element between less-than, , and greater-than, , symbols. Many tags apply to a range of text, in which case a pair of tags is used: an opening tag indicating the start of the range and a closing tag with a slash indicating the end of the range. For example, you can make some text bold like this. Tags can be nested to combine effects, bold italic. Some tags, such as the tag for inserting a line break or for inserting an image, do not cover a range of text and are considered to be “self-closing.” Self-closing tags do not need a closing tag; for a line break, only a tag of is needed. Some web developers write self-closing tags with an optional before the , such as .

The following HTML file has some errors: the tag is not closed; the tag appears twice; an extraneous tag appears; and the tag is not properly closed.

3. Modify the expression evaluator case study program from this chapter to make a new program that accepts as input a fully parenthesized infix expression and returns a string representing an equivalent postfix expression. Postfix expressions are ones where each operator follows its two operands, such as [redacted] rather than [redacted]. Postfix expressions are elegant in that they do not need parentheses. For example, the given infix expression:

is equivalent to the following postfix expression:

Your algorithm should read the expression token by token, using a stack to store operators. (You don't need the values stack from the original case study anymore.) Instead, each time a number is encountered, append it to a string that you are building up. Each time a right parenthesis is encountered, pop the stack to get an operator and append it to the string you are building up. Leave in place the rest of the code to preserve the error checking. Return the string [redacted] if an error is encountered.

4. Write a Maze Explorer program that uses stacks and queues to implement an algorithm to escape from a maze. The overall pseudocode of the algorithm is the following. The algorithm can be implemented using a stack or a queue. What are the pros and cons of each?

```
create an empty stack of locations to explore.

push the start location onto the stack.


while (stack is not empty) {


pop a location
L
 from the stack.


if we have we pulled
L
 from the stack before:


no need to explore it again, so skip
L
.
```

```
if
L
 is the end location:


the end was reachable!


else,
L
 is a new reachable non-finish location, so explore it:


add all non-wall adjacent maze locations to the stack.


record the fact that we have explored
L
.



if the stack is empty, the finish is unreachable.
```

5. Write a Guitar Hero program that uses a queue to simulate the creation of guitar notes. When a guitar string is plucked, it vibrates to generate sound. The sound starts from the note’s initial note pitch or frequency, then it undergoes a wave-like oscillation and gradually fades in volume over time.

The oscillation and fading of notes can be computed using an algorithm called Karplus-Strong. The algorithm represents sound as time slices called samples (44,100 samples per second, in this

case). You can compute samples that sound similar to the vibrations of a real guitar as displacements from the guitar string's original frequency. First create a queue of random displacements between −½ and ½. The length of the queue should be the sampling rate, 44,100, divided by the note's frequency. Then repeatedly remove the first queue element, average it with the next front element, slightly fade the volume by multiplying it by 0.996, then add the result back into the queue as shown in Figure 14.3.

Figure 14.3 Karplus-Strong algorithm samples

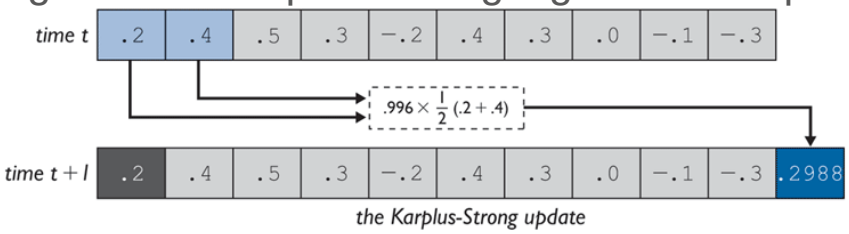


Figure 14.3 Full Alternative Text

This program requires support code to send output to your computer's sound card. The code is provided on our web site at http://buildingjavaprograms.com/.

Chapter 15 Implementing a Collection Class

15.1 Simple [redacted]
• Adding and Printing
• Thinking about Encapsulation
• Dealing with the Middle of the List
• Another Constructor and a Constant
• Preconditions and Postconditions
15.2 A More Complete [redacted]
• Throwing Exceptions
• Convenience Methods
15.3 Advanced Features
• Resizing When Necessary
• Adding an Iterator
15.4 [redacted]

Introduction

In Chapters 10, 11, and 14 we saw how to use various data structures that are part of the collections framework. As a Java programmer, you will find it useful to have these off-the-shelf solutions available to you, but you can learn a lot from examining how these structures are implemented. In

this chapter we will explore in detail how to implement one such structure.

Our goal is to understand the [redacted] *class that was described in detail in Chapter 10. Recall that* [redacted] *is a generic class that is best described as* [redacted]*, where* [redacted] *is filled in with a specific element type. Because generic classes are not easy to implement, we will first explore the implementation of a class called* [redacted] *that can be used to store a list of simple integer values. There aren't a lot of applications for a simple* [redacted] *object, but the code we write for it will be similar to the code we need for the generic version. The chapter ends with a section that explores how to turn the more specific* [redacted] *into a generic* [redacted]*.*

15.1 Simple [redacted]

In this section, we will develop an initial version of the [redacted] class that contains appropriate fields for storing a list and a minimal set of operations for manipulating it.

Recall from Chapter 8 that there are two important perspectives of any class. The first is the external view that a client of the class will have. Clients in general don't want to understand all of the internal details of the class. They just want to know what it does. As a result, we want to have a clear specification for the client of what the [redacted] class is supposed to do. We think of this as a *contract* that we have with the client.

Contract

[redacted]

A clear specification that tells a client how an object behaves without describing the details of how it is implemented.

The second view is the internal view of the implementer. We have to figure out how to make the object work—how to satisfy the contract. When we are thinking as an implementer, we will consider the nitty-gritty details of the innards of an object.

It is sometimes confusing to switch between these two different perspectives, but you will get better at it as you practice.

Adding and Printing

Let’s start by developing a version of the class which has a single mutator method that will append values to the end of the list and a single accessor method that will display the contents of the list. Assume that the client code will look something like the following:

```
public class


public static void


new
```

new

The preceding code will produce the following output:

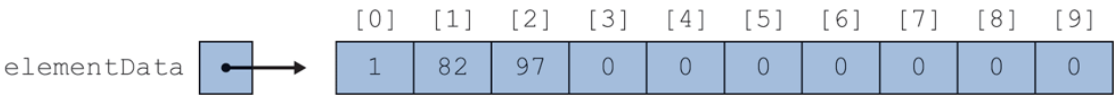
You must implement three methods in this program: the constructor, the [redacted] method, and the [redacted] method that [redacted] will call. First you have to figure out what kind of fields you need. We are trying to emulate the [redacted] class, which is built on top of a simple array, so you should do the same thing for your [redacted] class. All of the examples of arrays in Chapter 7 involved what could be called *filled arrays* in which each element of the array is in use. A filled array works fine in many applications, but this is not one of them. You are implementing a dynamic structure that will grow and shrink as the client adds and

removes values, so you don't want to be forced to use an array that is exactly the same size as your list.

Instead, you should use an *unfilled array* in which some of the array elements are in use and some are not. This approach is similar to the way that a hotel is run. A hotel might have 100 guest rooms, but they don't all have to be occupied at once. There might be 90 rooms currently in use by customers and 10 vacant rooms that are available for later use.

This concept raises the question of how to distinguish between occupied cells and vacant cells in the array. One simple approach is to maintain a [redacted] variable that keeps track of the number of occupied cells and use the front of the array for cells that are currently in use and the back of the array for cells that are vacant. Unlike a hotel that has vacant occupied rooms interspersed, this kind of unfilled array groups all of the occupied cells together at the front of the array and all of the vacant cells at the end of the array.

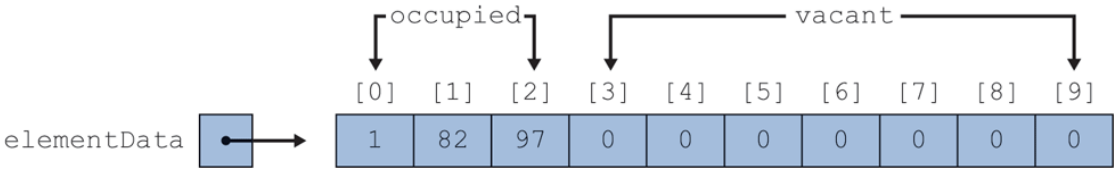
For example, the sample client code at the beginning of this section constructs a list and adds the values [redacted], [redacted], and [redacted] to the list. That would mean that three of the array cells are occupied and any other cells are vacant. Suppose that you construct an array variable called [redacted] and you add these three values as the first three entries in the array:



15.1-1 Full Alternative Text

While it's true that the first vacant cell has the value [redacted] stored in it, you wouldn't want to count on this being the case because the client might want to store the value [redacted] in the list. Without a [redacted] variable, you wouldn't know for sure whether this list ends with [redacted]

or whether it ends with one of the zeros. With the [redacted] variable, you can keep track of exactly how many cells are occupied and you know that all others are vacant:



15.1-2 Full Alternative Text

In the preceding example, the array has a length of 10. We refer to this length as the *capacity* of the list. In our hotel analogy, the hotel has 100 rooms, which means that it has the capacity to rent up to 100 rooms. But often the hotel is not filled to capacity. The same will be true for your list. The [redacted] variable keeps track of how much of the list is currently occupied whereas the capacity tells you how large the list can grow.

The fields you want to declare for your class are an array and a [redacted] variable:

[redacted]

Your constructor should initialize these fields. The array in the preceding example had a capacity of 10, but let's increase it to 100:

```
public ArrayIntList() {

elementData = new int[100];

size = 0;

}
```

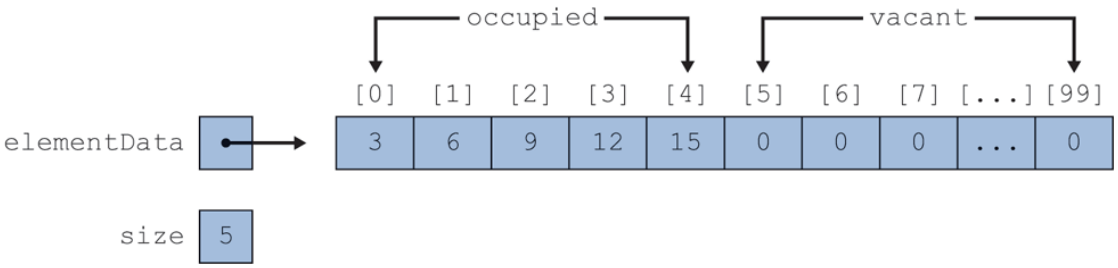
Recall from Chapter 8 that fields are initialized to the zero-equivalent for the type, which means that the [redacted] field will be initialized to [redacted] with or without the line of code in the constructor. Some Java programmers prefer to leave out the line of code because they are familiar with the default initialization. Others

prefer to be more explicit about the initialization. This is somewhat a matter of personal taste.

Now that you have the fields and constructor defined, you can turn your attention to the [redacted] method. It takes an integer as a parameter. The idea is that you should append the given value to the end of the list. The method will look like this:

[redacted]

This turns out to be a fairly simple method to implement. Suppose, for example, that the list stores the values [redacted]



15.1-3 Full Alternative Text

As indicated in the preceding figure, the current size of the list is 5. If you wanted to add the value [redacted] to the end of this list, you'd store it in the first vacant array element, which has index 5. In other words, when the list has a size of 5, you append the next value into index 5. And once you've added that value into index 5, then the list will have a size of 6 and the next vacant element will be at index 6.

Generally speaking, the size of the list will always match the index of the first vacant cell. This seems a bit odd, but it is a result of zero-based indexing. Because the first value is stored in index 0, the last value of a sequence of length n will be stored in index n−1.

Therefore you should begin the `add` method by appending the given value at index `size`:

```
elementData[size] = value;
```

Storing the value into the array isn't enough. You have to increment the `size` to keep track of the fact that one more cell is now occupied:

```
size++;
```

If your program did not increment [redacted], it would keep adding values in the same index of the array. This would be like our hypothetical hotel checking every customer into the same hotel room because the hotel didn’t keep track of which rooms are occupied.

It seems likely that this method will work, but so far the program doesn’t have a good way of verifying it. You need some kind of method for displaying the contents of the list.

Chapter 7 explored the following code for printing the contents of an array of integers:

```

```

The code involves a fencepost loop because we use commas to separate values. If there are n numbers, they will be separated by n−1 commas. The code uses the classic fencepost solution of printing the first value before the loop. The [redacted] is used because there is a special case if the array is empty. This code can be fairly easily adapted to your [redacted] class. You have to convert the static method into an instance method. As a result, instead of having a parameter called [redacted], you refer to the field called [redacted]. You also want to modify the code so that it uses the size of the list rather than the length of the array:

```
()

size

elementData[0]

size
```

```
elementData[i]
```

But you don’t want a method to print the list. If you want to make a truly useful class, you should provide flexible mechanisms that will work in a wide variety of situations. A better solution is to return a string representation of the list rather than printing it. As we saw in Chapter 8, Java has a standard name for such a method: `toString()`. So you should rewrite the preceding method to construct and return a string representation of the list:

Here is the complete class:

```
public

new

public void

int
```

```
public

if

return

else

for

int
```

```
return
```

This version of the class correctly executes the sample client program, producing the correct output.

Thinking about Encapsulation

When we declared the fields for [redacted], we followed the usual convention of declaring them to be private. This can cause a problem for a client of the class. Suppose that the client has made a series of calls on the [redacted] method and wants to know how many elements are in the list. For example, the client might want to write code like the following:

The information the client wants is stored internally in the field called [redacted]. But because the field is declared to be private, the client can't access the value.

There are several clumsy ways to solve this problem. One solution would be to change the field from private to public, but as we saw in Chapter 8, making the field public breaks encapsulation and leaves the list object open to unwanted modifications. It's not a problem to allow the client to examine the value of the field, but suppose the client tried to set it to an illegal value, as in the following two examples:

Allowing the client to reach into the object opens the door for the client to put the object into a bad state.

Another solution would be to force the client to keep track of the size, but that is a rather silly solution. Why should both the object and the client keep track of this information? It puts an undue burden on the client and it leads to unnecessary duplication of effort.

The right solution is to keep the field private, but to introduce an accessor method that allows the client to examine the value of the field:

Then the client can write code like the following:

This line of code looks fairly similar to code that accesses the field directly. The difference here is that we are calling a method called . The method allows the client to examine the field without giving the client the ability to change the value of the field.

A client will also want the ability to access individual elements of the list. Internally, you access the list elements by referring to the array called [redacted]. Again, you don't want to break encapsulation by giving a client direct access to this field. Instead you can introduce another accessor method for examining the value stored at a given index:

Eventually, we want to make sure that this method can't be used by a client to access any of the vacant array elements (elements beyond the size of the list), but this simple version will do for now.

Dealing with the Middle of the List

So far we have explored how to add a value at the end of a list, but often a client is interested in dealing with values in the middle of the list. The client might want to search for the location of values or add them to or remove them from the middle of the list.

Let’s begin with the task of searching for the location of a value in a list. The client could use the [redacted] and [redacted] methods to locate a value. But if it seems likely that a client will want to perform frequent searches for the locations of values, then it makes more sense to include the ability to do this as a method inside the class. That way, it will be readily available to all clients of the class.

In Chapter 7, we wrote the following code to find the index of the first occurrence of a value in an array of integers:

This method is fairly easily converted from a static method to an instance method, as you did with [redacted]. To achieve the conversion, replace the array parameter with references to the [redacted] field and, instead of using the length of the array as a loop bound, use the [redacted] field so that the method searches only elements of the array that are currently in use:

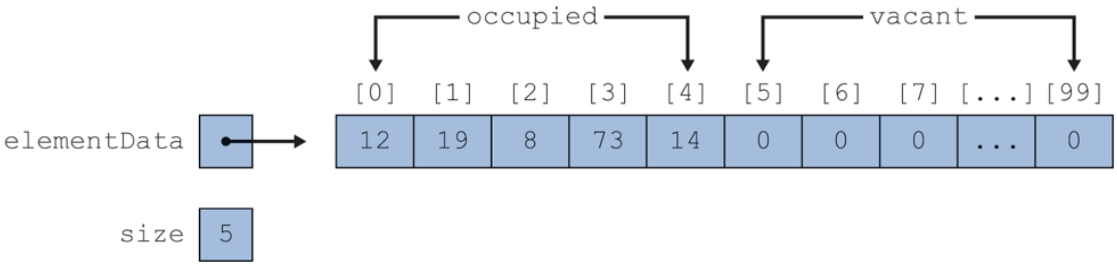
```
(int value)
```

```
size
```

```
elementData[i]
```

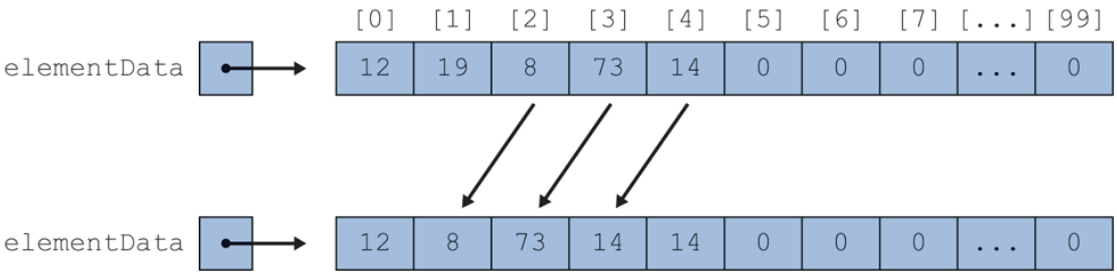
You haven’t yet given the client the ability to remove a value from the list. Let’s call the method `remove`. It should take as a parameter the index of the value to be removed:

Suppose, for example, that a list contains the five values :



15.1-4 Full Alternative Text

Suppose that you want to remove the value , which is stored at index 1. This action will create a gap in the list unless you shift values over to fill in the gap. The three values that come after the value each have to be shifted down by one position:



15.1-5 Full Alternative Text

In order to shift the values over, you obviously need some kind of loop. You want to shift a value into index 1, another value into index 2, and another value into index 3. Why start at index 1? Because that's the target index, the index of the value you've been asked to remove. And why stop at index 3? Because that's when you run out of occupied cells to shift. The following code is a good first guess at the loop bounds:

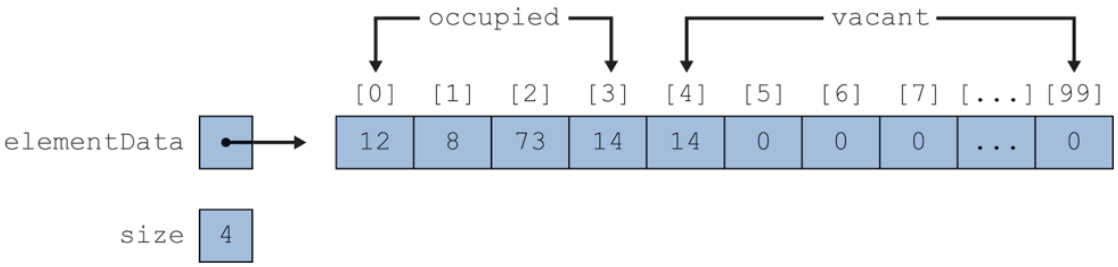
We saw in Chapter 7 that we can use code like the following to accomplish the shifting task:

This code is almost correct. The problem is that the loop executes once too often. As we have noted, you want to shift values into indexes 1, 2, and 3. But when the [redacted] is [redacted], as in our example, the loop will do one extra shift, shifting a value into index 4. You want to stop the loop one step earlier so that it won't perform this final shift. You can accomplish that by subtracting one from the final loop bound:

This modification almost completes the method. The only other detail you need to worry about is that once you have removed this value, you need to decrement the size of the list. Here is the complete method:

```
size--;
```

Let’s take a look at the final state of the list in our example. After we have removed the value at index 1, shifted three values left, and decremented the size, we end up with the following list:



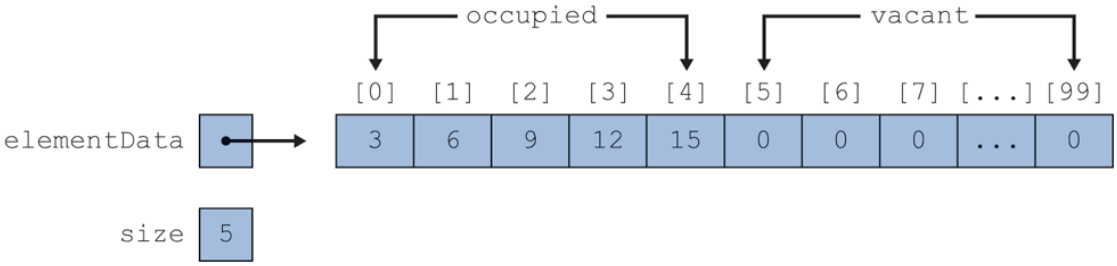
15.1-6 Full Alternative Text

Notice that the first cell among the vacant cells has the value [redacted] in it. You might imagine that you have to set it back to [redacted]. In general, this isn't necessary for your [redacted]. Because the value is among the vacant cells, you know that it isn't a *real* value of the list. And if you ever need to use that cell again, you'll overwrite the [redacted] with some new value. It doesn't matter whether you overwrite a [redacted] or overwrite a [redacted]. In fact, in the final version of our program, we'll make sure that a client can't ever see that extra value of [redacted]. The situation is slightly different when you deal with the generic [redacted], but we'll save that discussion for the end of the chapter.

Another operation that a client is likely to want to perform is to add a value in the middle of a list. You already wrote a simple [redacted] method that appends at the end of the list:

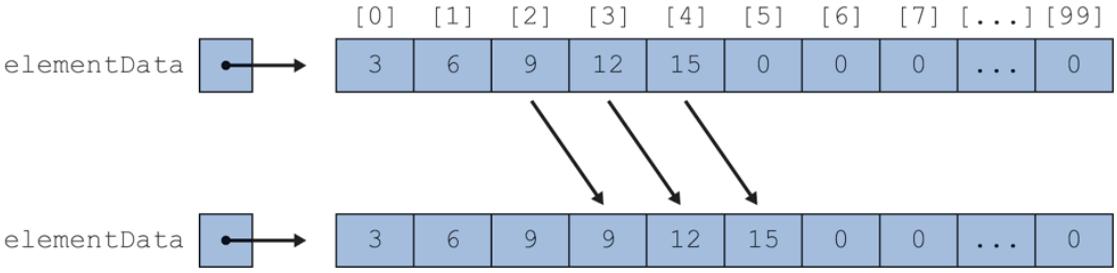
You can call the new method [redacted] as well, but it will have different parameters. You will still need to know the value to add, as with the appending method. But this new method will also need to know the index where the new value is to be added. Thus, it will look like the following:

Implementing this method is more complicated than appending a value at the end of the list because you have to shift values over to make room for the value to be inserted. Suppose, for example, that your list contains the five values :



15.1-7 Full Alternative Text

Suppose that you want to insert the value at index 2. That insertion will require you to shift each of the values that are currently in indexes 2 through 4 to the right by one:



15.1-8 Full Alternative Text

This insertion is similar to the remove operation. You want to deal with cells between the target index (where the new value is to be added) and the end of the sequence of occupied cells. Therefore,

you know that the loop bounds will be something like the following:

We saw in Chapter 7 that you can shift values to the right by writing code like the following:

There are several problems with this code. First, it starts too early. In the example, you are trying to shift values so that you can add a new value at index 2. The first shift you want to do is to shift the value that is currently at index 2 into index 3. But the preceding loop will first shift the value at index 1 into index 2. The loop needs to start one index later:

```
index + 1
```

Even after you make this change, the loop bounds are still not correct. In the sample for a list of 5 elements, the final shift moved a value into the cell with index 5. The preceding code will stop the loop once `i` is 4 (while it is still strictly less than the size). The loop needs to allow `i` to be equal to the size:

The final problem, as described in detail in Chapter 7, is that this loop has to be run backward rather than forward. Otherwise you overwrite your list values with the value that is stored at the target index:

This modification completes the shifting, but you also have to store the new value at the target index now that there is room for it, and you have to increment the size of the list:

```
elementData[index] = value;


size++;
```

Another Constructor and a Constant

The [redacted] class is shaping up nicely, but so far you have just a single constructor that constructs a list with a capacity of 100:

This constructor isn’t very flexible. What if clients want to manipulate a list of 200 values? What are they supposed to do? You don’t want to force them to rewrite the code just to use your class. That would be like having to open up a radio and rewire the insides just to change the volume or the station. If there is some value like the capacity that the client is likely to want to change, then you want to be sure to build in the flexibility to allow the client to do so.

You can accomplish this flexibility by changing the constructor to take a parameter that specifies the capacity of the list. You can then use that value when you construct the array:

This modification allows a client to write lines of code like the following:

Unfortunately, if this is your only constructor, then the client loses the ability to write lines of code like the following:

You could include both constructors, but then you have redundant code. As we saw in Chapter 8, you can avoid the duplication by having one constructor call the other. The constructor that takes the capacity is the more general constructor, so you can have the constructor that takes no arguments call it using the notation:

You might wonder how Java can tell the two constructors apart. The answer is that they have different signatures. One constructor takes no arguments, whereas the second constructor takes an integer as an argument. When Java sees the call on [redacted] in the first constructor, it knows that it is calling the second constructor because the call includes an integer value as a parameter.

Another improvement we can make is to introduce a constant for the rather arbitrary value of [redacted]:

It is a good idea to make this constant public because the client might want to be able to refer to the value to know what capacity a list has if the client doesn’t specify a specific value to use. You would then rewrite the first constructor to use the constant instead of the specific value:

Preconditions and Postconditions

We are almost ready to put all of these pieces together into a complete class. But before we do so, we should consider the issue of documentation. When you document the class, you want to think in terms of important information that should be conveyed to the client of the class. You want to describe what each method does and you want to describe any limitations of each method. This is a great place to use preconditions and postconditions, as described in Chapter 4.

Recall from Chapter 4 that preconditions are assumptions the method makes. They are a way of describing any dependencies that the method has ("this has to be true in order for me to do my work"). Also, recall that postconditions describe what the method accomplishes, assuming that the preconditions are met ("I'll do this as long as the preconditions are met"). The combination of preconditions and postconditions is a way of describing the contract that a method has with the client.

Consider, for example, the [redacted] method. It is a fairly simple method that allows the client to access individual elements of the list:

This method makes sense only if the value of [redacted] is in the range of occupied cells. In other words, it has to be greater than or equal to [redacted] and it has to be less than the size of the list. We can describe these constraints in a precondition comment:

The postcondition should describe what the method accomplishes, assuming that the precondition is met. You can do that with a short description of the method’s behavior:

There are quite a few preconditions for the methods you have written. All of the methods that take an index as a parameter assume that it is a legal index. The constructor that takes a capacity can't work with a negative value. And there is another important precondition lurking out there. Remember that your list has a fixed capacity. What happens if the client calls the [redacted] method so many times that the capacity is exceeded? You should warn the client about that scenario to make the contract clear.

In the next section we'll examine how to handle bad values more directly, but for now we'll settle for documenting the various preconditions.

It is worth noting that the method comments for this class do not contain comments about implementation. For example, we wouldn't mention that the [redacted] method shifts values in the array. That's a detail of the implementation that won't generally interest the client. If you think that the implementation is complicated enough to deserve comments, then include those comments with the code itself inside the method. The comment on the method itself should be written from the client perspective, not from the perspective of the implementer.

Professional Java programmers write their comments in Javadoc format so that they can be extracted from the actual program files and published as html files. If you've been referring to the Java API documentation, then you've been reading such comments.

We don’t use Javadoc format for our comments, but you can read about it in Appendix B.

Here is a complete version of the [redacted] class incorporating all of the pieces discussed in this section:

```
public class

private int

private int
```

```
public static final int

public

this

public ArrayIntList

int
```

```
new int
```

```
public int
```

```
return
```

```
public int

int




return
```

```
public
```

```
if
```

```
return
```

```
else
```

```
for

int

return
```

```
public int

int




for

int




if




return

```

```
return

```

```
public void

int
```

```
public void

int




for

int
```

```
public void

int



for

int
```

15.2 A More Complete [redacted]

In this section, we will extend the [redacted] class to throw exceptions when preconditions are violated and to include a more inclusive set of methods that parallel the methods available for the built-in [redacted].

Throwing Exceptions

In the previous version of [redacted], we documented the various preconditions of the methods. It is a good practice to clearly specify the contract with potential clients, but you shouldn't assume that clients will always obey the preconditions. As we saw in Chapter 4, the convention in Java is to throw an exception when a precondition is violated.

For example, consider the constructor that takes a capacity as a parameter:

[redacted]

The precondition indicates that the capacity is not supposed to be negative. You can go further than just documenting it by adding code to throw an exception. In this case, it is appropriate to throw an :

You have the option to include a string that will be displayed with the exception. In this case, it would be useful to tell the client the value that was being passed for :

You should also mention the exception in the comments for the method. It is important to mention exactly which type of exception will be thrown and under what circumstances. Here, then, is the final version of the method with the extra code to throw an exception when the precondition is violated:

```
(throws IllegalArgumentException if not)




if (capacity < 0) {

throw new IllegalArgumentException("capacity: " + capacity);

}
```

In order to construct an exception object to be thrown, you have to decide what type of exception to use. All of the examples we have seen so far have constructed exceptions of type . This is a kind of generic exception that is used to indicate that some value passed as an argument

was not legal. But there are other exception types that you can use. The convention in Java is to pick the most specific exception that you can. Table 15.1 lists of some of the most common exception types.

Table 15.1 Common Exception Types

[redacted]	A null value has been used in a case that requires an object.
[redacted]	A value passed as an index to an array is illegal.
[redacted]	A value passed as an index to some nonarray structure is illegal.
[redacted]	A method has been called at an illegal or inappropriate time.
[redacted]	A value passed as an argument to a method is illegal.
[redacted]	A call was made on an iterator's next method when there are no values left to iterate over.

[redacted] and [redacted] are thrown by Java automatically, so you don't generally write code to specify those exceptions. Java programmers often use the other four exceptions, however.

Both of the [redacted] methods in your [redacted] class have a precondition that the size of the list must be strictly less than the

capacity. Otherwise you will attempt to store a value in an array index that doesn't exist. This is an appropriate situation for an [redacted]. It is not appropriate to add values to the list once it has reached its capacity. In this case, you want to let the client know that the problem doesn't come from the values passed as arguments, but rather, that the [redacted] method was called at an inappropriate time.

You could add code to each of the [redacted] methods to check for this exception, but to avoid redundancy, it is better to introduce a private method that each of the methods calls. You'll be introducing some other methods that add more than one value at a time, so it will be useful to write the private method in a fairly flexible way. You can write a method that takes a required capacity as a parameter and that checks to make sure that the array has that capacity. In other words, you'll tell the method, "I need room for this many elements," and it will test whether the array has that much room. If it does not, the method throws an [redacted]:

You can add the following line of code to each of the methods to check that the array has the capacity to add one more value to the list:

You also need to modify the comments to indicate that this exception is thrown. For example, here is the rewritten appending method:

```
(throws IllegalStateException if not)




checkCapacity(size + 1);
```

Several methods specify the index of a value. The [redacted] method, for example, is supposed to return the value at a particular index. If you provide a value that is outside the bounds of the underlying array, then the method will throw an [redacted]. This exception is better than nothing, but it doesn't cover all cases. For example, your list might have a capacity of 100 but have only 10 values stored in it. The client might try to access a value at index 10 or 11 or 50, and your method will return a result when you'd prefer it to throw an exception to let the client know that the index is not legal for a list of 10 elements.

In this case, it is better to throw an [redacted] and to include the illegal index to let the client see exactly what bad value was passed as a parameter. There are multiple methods that refer to index values, so it is again useful to introduce a private method that can be called by each method:

This private method will properly throw an exception for [redacted] and [redacted]. It isn't quite the right test to use for the [redacted] method that takes an index as a parameter, because [redacted] has a slightly different precondition than the other methods. Normally, it doesn't make sense to refer to a value at index [redacted] because that index is beyond the last value stored in the list. For example, if the list stores 10 values, then you don't want to call [redacted] or [redacted] for a nonexistent value at index 10. But for [redacted], it makes sense to refer to this index. If there are currently 10 values in the list, then you would be allowed to add a new value at index 10 because it is a new value not currently in the list. Instead of calling the private method, the [redacted] method will have its own code for checking the index and throwing an [redacted].

There is a further benefit to adding all of this code to throw exceptions: Adding the code will ensure that your object is never in a corrupt state. It will always satisfy certain data invariants. (Data invariants were described in Chapter 8.) In particular, you know that

- [redacted] always (your calls on [redacted] make sure of that)
- the array elements that you examine are always in the range of 0 to [redacted] (your calls on [redacted] make sure of that)
- your code will never generate an [redacted]

It is extremely useful to know that your class will always satisfy these constraints. This knowledge is an added benefit of encapsulation. If a client could reach in and change the [redacted] or [redacted] fields, then you couldn't make these kind of guarantees.

Convenience Methods

The built-in [redacted] class has many other methods besides the ones we have implemented so far, so it makes sense to go ahead and add some of those methods to your [redacted] class. These methods are largely for the convenience of the client, since you could achieve the same functionality by calling existing methods on the list.

Your class has a method called [redacted] that can be used to search for the location of a value in the list. Sometimes, though, a client just wants to ask the simpler question of whether a particular value appears in the list somewhere. The convention in Java is to use a method called [redacted] for this purpose. It has a [redacted] return type:

How do you write this method? You don't want to duplicate the code that you included in [redacted], so instead you'll call [redacted]. Remember that it returns a value of [redacted] if the value is not found, so you can test whether or not [redacted] returned an index that is greater than or equal to [redacted]:

This version violates Boolean Zen, which was covered in Chapter 5. You can simply return the value of the expression, rather than including it in an statement:

Thus, the method ends up being a single line of code:

Another common method in the Java Collections Framework is called [redacted]. It returns a [redacted] value indicating whether or not the list is empty. This is another method that can be written concisely as a one-line method using the value of the [redacted] field:

Again, you don't need an [redacted] statement; you can simply return the value of the Boolean expression.

So far you have included methods to add values to and remove values from the list, but sometimes you simply want to replace the value at a certain location with some new value. This operation is referred to as the [redacted] method and is easy to implement. You have to remember to include a call on [redacted] because the method has a precondition that is similar to that of [redacted] and [redacted]:

Providing this method is important because it allows a client to avoid unnecessary inefficiency. For example, suppose that the list has 10,000 values stored in it and the client wants to replace the first value with something new. The [redacted] method does it very quickly. The alternative would be to call [redacted], which would shift over the other 9999 values, and then to call [redacted], which would shift the other 9999 values back to where they were earlier. Using [redacted] is much more efficient.

You might think that the introduction of the [redacted] method breaks encapsulation, because it allows a client to change a value in the array. But it doesn't really break encapsulation because you force the client to get this access by calling a method. This fact allows your method to be in control of the client request. So, for example, you can call the [redacted] method to ensure that the array index is legal.

Another common operation that a client might wish to perform is to empty the structure of all values. This is referred to as the [redacted] operation and can be accomplished quickly by resetting the [redacted] field back to its original value of [redacted]:

You might imagine that you have to reset all of the array elements back to [redacted], but that's not necessary. You have written the code in such a way that a client can only get access to array elements 0

through [redacted]. When [redacted] is reset to [redacted], the client can't access any of the array elements. The only way those array elements will be used again is if the client makes calls on the [redacted] method, in which case the old values will be overwritten with new values supplied by the client. But just as with the [redacted] method, you will find at the end of the chapter that you solve this problem slightly differently for the generic [redacted] class.

The last method we will add is a "bulk add" method called [redacted] that adds all of the values from a second [redacted]. It may seem a little odd to have one [redacted] deal with another [redacted], but this actually happens fairly often. The idea is that the first [redacted] is supposed to add all of the values from the second [redacted], which means that the header for the method looks like the following:

[redacted]

You can call the appending [redacted] method to add values to the list. You just need a loop that iterates over the values in the second list. You also need to call your [redacted] method to make sure that the array has sufficient capacity to store these new values:

[redacted]

This code refers to [redacted] and [redacted], which you might imagine would generate an error because they are private fields. In fact, the code does not generate an error. Recall from the Chapter 8 examples that the word “private” means that it is “private to the class.” This is not the way that we as humans understand the meaning of private. (If something is private to me, then it shouldn’t be available to other humans.) But in Java, one [redacted] object can access private elements of another object because they both belong to the same class.

You can find the complete third version of the [redacted] class on the web page http://buildingjavaprograms.com.

15.3 Advanced Features

In this section, we explore adding some extra advanced functionality to the [redacted] class. First we will see how to implement the class so that it has no fixed capacity and will grow larger if necessary. Then we will learn how to provide an iterator over the list.

Resizing When Necessary

The built-in [redacted] class has a notion of capacity, as does our [redacted] class. But instead of throwing an exception when the capacity is exceeded, the class creates a larger array. In other words, its capacity grows as needed to accommodate the addition of new values to the list.

It isn't generally easy to make an array bigger. Java doesn't allow you to stretch an array that was constructed previously. Instead, you have to allocate a brand-new array and copy values from the old array to the new array. An analogy would be the way shops and other businesses work in the real world. If you need some extra space for your store, you can't generally break down the wall and grab some of the space from the store next door. More often, you have to relocate your store to a larger space.

Obviously you don't want to construct a new array too often. For example, suppose you had space for 1000 values and found you

needed space for one more. A poor solution would be to allocate a new array of length 1001 and copy the 1000 values over. Then, if you find you need space for one more, you could make an array that is 1002 in length and copy the 1001 old values over. This kind of growth policy would be very expensive.

A better idea would be to double the size of the array when you run out of space. If you have filled up an array of length 1000, you double its size to 2000 when the client adds something more. That particular call on [redacted] is expensive because it has to copy 1000 values from the old array to the new array. But you won't need to copy again for a while. You can add another 999 values before you'd need extra space. As a result, we think of the expense as being spread out or *amortized* over all 1000 calls on [redacted]. When the cost is spread out over 1000 adds, it is fairly low (a constant).

The built-in [redacted] class does something similar. The documentation is a little coy about this: "The details of the growth policy are not specified beyond the fact that adding an element has constant amortized time cost." If you look at the actual code, you'll find that it increases the capacity by 50% each time (a multiplier of 1.5).

So how do you add this functionality to your [redacted] class? You included a method called [redacted] that throws an exception if the array isn't big enough. You can simply replace this method with a new method that makes the array larger if necessary:

This version of the method works, but you can get a slight improvement by calling a built-in method called [redacted] that returns a copy of an array. It has the same functionality as [redacted], but it is likely to run faster because this operation is what is known as a *block copy* operation that can be optimized to run faster. Thus, the method can be rewritten as follows:

```
Arrays.copyOf(elementData, newCapacity)
```

The [redacted] method was declared to be private, but the preceding method has been declared to be public, because a client might also want to make use of that method. For example, if the client recognizes that the capacity needs to be significantly increased, then it is useful to be able to call this method to resize it once rather than resizing several times. The [redacted] class has this method available as a public method.

Adding an Iterator

In Chapter 11, we saw that it is common in the collections framework to use an iterator object to traverse a collection. Recall that an iterator should provide three basic operations:

- , which returns true if there are more elements to be examined
- , which returns the next element from the list and advances the position of the iterator by one
- , which removes the element most recently returned by

We will develop a class called that implements this functionality for an . The usual convention in Java is to ask the collection to construct the iterator by calling the method :

Once we have obtained an iterator from the list, we can use its three methods to traverse the list. For example, the following program constructs an and then computes the product of the list:

```
public class


public static void


int


new


for
int
```

```
int


while


int
```

The program produces the following output:

Here is a variation that removes any 0 values from the list, computing the product of the nonzero values:

```
public class
```

```
public static void

int

new

for

int
```

```
int

while

int

if

else
```

The program produces the following output:

So how do you implement the [redacted] class? The main function that the iterator performs is to keep track of a

particular position in a list, so the primary field will be an integer variable for storing this position:

Initially you start [redacted] at [redacted] so that it will refer to the first value in the list. The [redacted] method is supposed to determine whether any values remain to iterate over. To do so, it will have to compare this position to the size of the list:

To perform this comparison, the iterator needs to be able to find out the size of the list. To do so, the iterator will need to keep track of the list over which it is iterating. That means that you will need a second field and the constructor will have to be passed a reference to the list to iterate over:

```
private ArrayIntList list;



ArrayIntList list

this.list = list;
```

Using this field, you can now easily write the method:

What about the [redacted] method? It is supposed to return the next value from the list and then reset the position to be one later in the sequence, which you can accomplish by simply incrementing the value:

But the method has an important precondition that you must consider. What if a client calls [redacted] when the iterator has run out of values to return? The method should throw an exception in that case. The convention in Java is to throw a [redacted]:

```
if (!hasNext()) {

throw new NoSuchElementException();

}
```

The final operation performed by an iterator is the [redacted] method. The method is supposed to remove the most recent value that was returned by [redacted]. The [redacted] field keeps track of the next value to be returned by the iterator, so the value to be removed is at index [redacted]:

```
```

Keep in mind what happens when you ask the [redacted] to remove that value. All of the other values will be shifted one to the left in the list. That means that [redacted] will no longer be positioned at the next value in the list. That value has been shifted one to the left, so you have to decrement [redacted] to account for the shift that has taken place:

```
position--;
```

This method also has an important precondition to consider. A client is supposed to call [redacted] before calling [redacted]. One possibility is that the client will call [redacted] before making any call on [redacted]. If that happens, it will be obvious from the fact that [redacted] will be zero. Another possibility is that the client will call [redacted] twice in a row without calling [redacted] in between. That is not a legal operation either. You won't know just from looking at the value of [redacted] whether the client has violated this precondition.

In this case, you need an extra bit of state for the object. You need to keep track of whether it is currently legal to remove a

value, so this is a good time to add a field. It will be of type [redacted] and you can call it [redacted]. You can use this field to throw an exception if the precondition is violated. And once a call on [redacted] has been performed, you have to remember that it is no longer legal to remove a value until [redacted] is called again:

```
if (!removeOK) {

throw new IllegalStateException();

}




removeOK = false;
```

Notice that you throw an [redacted] because a call on the method is not appropriate if it is not okay to remove a value. You can then add code to the constructor to initialize this field to [redacted], and you can reset it to [redacted] whenever [redacted] is called.

Here is the final class definition:

```
import

public class

private

private int
```

```
private boolean


public


this
```

```
public boolean




return
```

```
public int




if
```

```
throw new
```

```
int


return


public void


if
```

```
throw new
```

The program imports the [redacted] package because the class for one of the exceptions we want to throw, [redacted], comes from that package.

You also have to modify the [redacted] class. Remember that it needs to have a method called [redacted] that constructs an iterator, which means it will look like this:

So which list should you mention in the call on the constructor? The is supposed to construct an iterator that is looking at itself. You can use the keyword to say, “Construct an iterator that is iterating over me”:

You can find a complete listing of this fourth version of the class along with the class on the web page for this textbook at http://buildingjavaprograms.com.

15.4 [redacted]

In this section, we will explore how to convert the version of [redacted] from the previous section into a generic [redacted]. To start, you can simply replace all occurrences of [redacted] with [redacted] and change references to [redacted] that refer to values to [redacted]. Of course, there are other uses of [redacted] that specify capacity, size, and indexes, and these don't change.

This approach almost works, but there are a few places where you have to be careful. For example, when you define constructors, you don't use the generic [redacted] when you're naming the constructor. So the zero-argument constructor becomes:

[redacted]

You also run into trouble in the second constructor. After you perform the simple substitution, you end up with the following method:

[redacted]

When you try to compile this version, you get an error indicating that you are not allowed to construct a generic array. This is a limitation of generic types. You can construct an array of type , but not an array of type . You can solve this problem by introducing a cast. Replace the following line of code:

In its place, include this line of code:

This new version compiles, but it generates a warning about using unsafe or unchecked types. There’s no real way to get around this because Java won’t let you construct a generic array. That means

you can ignore this warning. In this case, it is good to include an *annotation* for the method to indicate that you don't want it to generate the warning for this method:

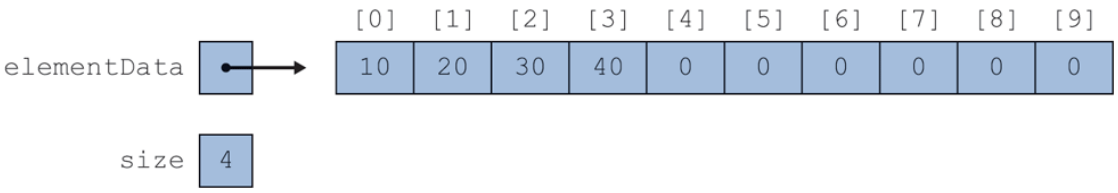
Annotations are special Java syntax that mark metadata on a class or method. They give instructions to the compiler. This particular annotation is a way of telling the compiler, "Don't generate the warning for this method because I'm aware of the problem already." There are many different kinds of annotations that you can include in your Java programs, but we won't take time to explore them in this book.

There is also a problem with the [redacted] method. After you perform the substitution of [redacted] for [redacted], here is the result:

This version compiles and would work, but it has a very strict definition of equality. It would require that the actual object you are searching for appears in the list. More often, you want to use an `==` comparison to see whether the value for which you are searching is equal to some value in the list. You'll want to replace the following bit of code from your method:

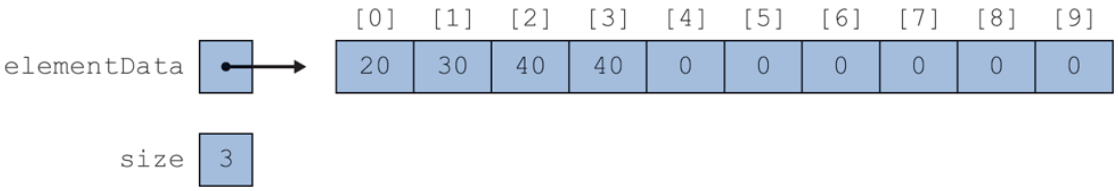
You can replace it with the following:

One final issue has to do with memory allocation. Consider what happens when a value is removed from the simple [redacted]. An array element that used to correspond to a list element is no longer being used. In general, this isn't a problem, and we haven't bothered to do any cleaning up afterward. For example, suppose that you have a list with a capacity of 10 and you store the values [redacted] in the list. The array looks like this:



15.4-10 Full Alternative Text

If you then remove the value at index [redacted], you shift the other three values left and decrement [redacted]:



15.4-11 Full Alternative Text

Notice that you now have two occurrences of [redacted] in the list. That isn't generally a problem because you know from the value of your [redacted] field that the [redacted] stored in index [redacted] isn't being used.

You can't be so cavalier when it comes to objects. We have to think about what is known as the *garbage collector*:

Garbage Collector

[redacted]

A process that is part of the Java Runtime Environment that periodically frees the memory used by objects that are no longer referenced.

In some programming languages, you have to explicitly destroy objects when you don’t need to use them any longer. Java saves you the trouble of doing this. Instead, its garbage collector looks for objects that are no longer being used (i.e., objects that are no longer referenced). You want to make sure that your doesn’t interfere with what the garbage collector is trying to accomplish.

If your is keeping a reference to some object that is no longer being used, then the garbage collector might not recognize that it can reclaim that space. So you have to explicitly set that array element back to .

A direct translation of the code produces the following method:

You need to add an extra line of code after the shifting code that sets the unused array element back to [redacted]:

```
elementData[size - 1] = null;
```

Similarly, the [redacted] method needs to set all values to [redacted] before resetting the [redacted] field to [redacted]:

```
for (int i = 0; i < size; i++) {
```

```
elementData[i] = null;


}
```

One final change that you will see in this class is that the iterator class has been converted to what is known as an *inner class*:

Inner Class

A class declared inside another class. Objects of the inner class have access to the methods and fields of the outer class.

In other words, the structure becomes

The generic syntax can be a bit confusing with inner classes. We declare our inner class without an type parameter, because the type is already declared as part of the outer list class. But we do have to say that it implements to match the interface from . Accidentally declaring the inner class as actually creates a second generic type and leads to confusing compiler errors.

Inner classes are normally declared to be private. Using an inner class for the iterator is the more usual approach. It allows you to eliminate the field that keeps track of the list. As we indicated in the definition box, when you declare an inner class, the instances of the inner class have access to the methods and fields of the instance of the outer class that constructed it. For example, the old version of the method refers to a field called that was used to keep track of the list over which the iterator was iterating:

This code can be simplified. Because the iterator class is now an inner class, you can write the method in a simple way:

```
size()
```

This method is calling the [redacted] method even though the iterator has no such method. Because it has no such method, it calls the [redacted] method of the outer class.

This can cause problems in one part of the code. When you write the [redacted] method for the iterator, you need to call the [redacted] method of the list:

Unfortunately, this version does not compile, because both the iterator and the list have a method called [redacted]. One has no parameters and the other has one parameter, so you'd think that Java could tell the difference, but it can't. You have to use a special notation to make it clear that you want to refer to the outer object. You do that by referring to the [redacted] value of the outer class, which you refer to as [redacted]. Thus, you can rewrite the following line of code:

Here is the new version of the code:

Before we finish this exploration, we should consider at least briefly the issue of interfaces. As we saw in Chapters 9 and 11, interfaces allow clients to describe variables in a more generic way. For example, suppose that you want to iterate over an [redacted] of [redacted] values. You'll replace the following line of code:

Instead, you'll use the [redacted] interface:

In order for this code to compile, it is important for the [redacted] class to implement the [redacted] interface and for the return type for the [redacted] method to use the interface rather than the class name. These changes have been incorporated into the final version of the class. It turns out that there are other interfaces to consider as well, but we will save that discussion until the end of the next chapter. At that time we will see a final version of the [redacted] class that is even closer to the built-in version.

You will find the complete code for the [redacted] class on our web page at http://buildingjavaprograms.com.

Chapter Summary

In this chapter, we implemented an array list class to store lists of integers.

Collection classes have two views: the external view seen by client code and the internal view seen by the implementer.

An array list uses an unfilled array and a [redacted] field in which the first *size* elements are meaningful and the rest are empty zeroes that are not considered to be part of the list. The entire array length represents its capacity for storing values.

When we add values to or remove values from the front or middle of an array list, we must shift the values right or left respectively to account for the newly added or removed element.

Our array list has preconditions that the client will not pass an illegal capacity on construction or illegal indexes when accessing elements. If the client does try to do so, Java will throw an exception.

Our subsequent versions of the list class resize to a larger capacity when the array becomes full.

The array list has an iterator for examining its elements in sequence.

Our array list of integers can be converted into a generic class that can store a list of any type of objects. The code is similar, but we must make a few changes—for example, when we construct arrays of type [redacted] or compare objects for equality.

An inner class is declared inside the braces of another (outer) class and has access to the state of an object of that outer class. Our final list iterator is an inner class.

Self-Check Problems

Section 15.1: Simple [redacted]

1. What is the difference between an array list's size and its capacity? What is the relationship between the two values? (Is one always larger or smaller than the other, for instance?)
2. What fields must be included in the [redacted] class, and why is each field important? Would the class still work correctly if we removed any of these fields?
3. How would the output of the [redacted] program shown in this section change if each field from [redacted] were declared [redacted]?
4. In this version of the list class, what happens if the client adds too many values to fit in the array?
5. Why does the list class use a [redacted] method rather than a [redacted] method?
6. We wrote the class to have public methods called [redacted] (to read the number of elements of the list) and [redacted] (to access the element value at a specific index). Why is this approach better than declaring the fields (such as [redacted]) [redacted]?
7. An element can be inserted at the beginning, middle, or end of an array list. Which of the three insertion points is the most computationally expensive, and why? Which is the most expensive location to remove an element from the list?
8. Write methods called [redacted] and [redacted] that return the smallest and largest values in the list respectively. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should return [redacted] and the call of [redacted] should return [redacted]. If the list is empty, the methods should throw an [redacted].

Section 15.2: A More Complete [redacted]

9. Describe the overall preconditions placed on the [redacted] class in this section. What assumptions do we make about how clients will use the list?
10. What is the purpose of the [redacted] method? Where is it called in the [redacted] class? Describe a way that the client can utilize an [redacted] that will be caught by [redacted].
11. What is the purpose of the [redacted] method? Where is it called in the [redacted] class? Describe a way that the client can utilize an [redacted] that will be caught by [redacted].
12. Once we check thoroughly for preconditions in the code, what data invariants can we now assume about the list?
13. Why do we bother to add the [redacted], [redacted], and [redacted] methods to the [redacted] class, when the client can already perform this same functionality with the [redacted], [redacted], and [redacted] methods, respectively?

Section 15.3: Advanced Features

14. When this new version of the class fills to its capacity, it resizes. How much does it grow? Why choose this growth rate, rather than increasing the capacity by a single element or other constant amount?
15. What is the benefit of adding an iterator to the list class?
16. What state does the array list iterator store?
17. How does the array list iterator know if there are more elements left to examine? What does it do if the client tries to examine a next element but there are none left to examine?
18. What is a precondition of the iterator's [redacted] method? How does the iterator enforce this precondition, and what does it do if the precondition is violated?
19. Write a method called [redacted] that returns the sum of all values in the list. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should return [redacted]. If the list is empty, [redacted] should return [redacted].
20. Write a method called [redacted] that returns the average of the values in the list as a real number. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should return `10.5`. If the list is empty, [redacted] should return `0.0`.

Section 15.4: [redacted]

21. What problem do we encounter when we try to construct an array of type [redacted]? How do we resolve this problem?
22. Since our list stores an unfilled array, the empty elements were filled with the value [redacted] when our array was full of integers. What value occupies the empty cells when our list stores values of type [redacted]?
23. What changes need to be made to the [redacted] method to search for objects of type [redacted] in the new [redacted] class, and why are these changes necessary?
24. What is an annotation? How are annotations useful in writing our [redacted] class?
25. Why is it important to set empty elements to [redacted] when we are clearing or removing from the list of type [redacted], when we didn't need to clear out these elements in the previous [redacted]?
26. What is one benefit of making the list iterator into an inner class?

Exercises

Each of the following exercises is a method to be added to the [redacted] class from this chapter.

1. Write a method called [redacted] that accepts an integer as a parameter and returns the index in the list of the last occurrence of that value, or −1 if the value is not found in the list. For example, if the list stores [redacted], then the last index of 18 is 6 and the last index of 3 is −1.
2. Write a method called [redacted] that accepts another list L as a parameter and returns the starting index of where L first appears in this list, or −1 if it is not found. All elements of L must appear in sequence and in the same order. For example, if variables called [redacted] and [redacted] store [redacted] and [redacted], respectively, the call of [redacted] should return [redacted].
3. Write a method called [redacted] that accepts two integer values as parameters and replaces all occurrences of the first value in the list with the second value. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should change the list to store [redacted].
4. Write a method called [redacted] that reverses the order of the elements in the array list. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should change the list to store [redacted]

. An empty or one-element list is not changed by a call to this method.

5. Write a method called that returns a new that contains a running total of the original list. In other words, the ith value in the new list should store the sum of elements 0 through i of the original list. For example, given a variable that stores , consider what happens when the following call is made:

The variable should store . The original list should not be changed by the method. If the original list is empty, the result should be an empty list. The new list should have the same capacity as the original. Remember that there is a list constructor that accepts a capacity as a parameter.

6. Write a method called that accepts an integer value as a parameter and replaces every value in the list with that value. For example, if a variable called initially stores and the call of is made, the list will be changed to store .
7. Write a method called that returns whether or not a list of integers is pairwise sorted. A list is considered pairwise sorted if each successive pair of numbers is in nondecreasing order. For example, if a variable called stores , then the call of should return because the successive pairs of this list are all sorted:

[redacted]. The extra value [redacted] at the end had no effect on the result because it is not part of a pair. If the list had instead stored [redacted], then the method should return [redacted] because the pair [redacted] is not in sorted order. If a list is so short that it has no pairs, then it is considered to be pairwise sorted.

8. Write a method called [redacted] that accepts an element value as a parameter and returns the number of occurrences of that value in the list. For example, suppose a variable named [redacted] stores [redacted]. A call of [redacted] should return [redacted] because there are four occurrences of that value in the list.
9. Write a method called [redacted] that returns the number of occurrences of the most frequently occurring value in a sorted list of integers. Because the list will be sorted, all duplicates will be grouped together, which will make it easier to count duplicates. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should return [redacted] because the most frequently occurring value (7) occurs four times. It is possible that there will be a tie for the most frequently occurring value, but that doesn't affect the outcome because you are just returning the count, not the value. If there are no duplicates in the list, then every value will occur exactly once and the max count is [redacted]. If the list is empty, the method should return [redacted].
10. Write a method called [redacted] that returns the length of the longest sorted sequence within a list of integers. For example, if a variable called [redacted] stores [redacted], then

the call of [redacted] would return [redacted] because it is the length of the longest sorted sequence within this list (the sequence [redacted]). If the list is empty, your method should return [redacted]. Notice that for a nonempty list the method will always return a value of at least [redacted] because any individual element constitutes a sorted sequence.

11. Write a method called [redacted] that removes and returns the last value from a list of integers. For example, if a variable called list stores [redacted], a call of [redacted] should return [redacted] and change the list's state to [redacted]. The next call would return [redacted] and remove [redacted] from the list, and so on. If the list is empty, throw a [redacted].
12. Write a method called [redacted] that takes an integer n as a parameter and that removes the first n values from a list of integers. For example, if a variable called [redacted] stores [redacted] and a call of [redacted] is made, the list's contents should become [redacted]. You may assume that the parameter value passed is between [redacted] and the size of the list inclusive.
13. Write a method [redacted] that accepts an integer value as a parameter and that removes all occurrences of the given value from the list.
14. Write a method called [redacted] that lists all inversions in a list of integers. An inversion is a pair of numbers in which the first appears before the second in the list, but the first is greater than the second. Thus, for a sorted list such as [redacted] there are no inversions at all, and the method would produce no output. Suppose that a variable called [redacted] stores the values [redacted]

[redacted]. The call of [redacted] would print many inversions:

[redacted]

The inversions can appear in any order, so this is just one possible correct output. You must reproduce this format exactly, but the inversions can appear in any order. You may assume that the list has no duplicates.

15. Write a method called [redacted] that doubles the size of a list by appending the mirror image of the original sequence to the end of the list. The mirror image is the same sequence of values in reverse order. For example, if a variable called [redacted] stores [redacted] and the client calls [redacted] then the list should be changed to store [redacted]. Notice that it has doubled in size because the original sequence now appears in reverse order at the end of the list.
16. Write a method called [redacted] that replaces every value with two of that value. For example, if the list initially stores [redacted], after the call it should store [redacted].

17. Write a method called [redacted] that takes an integer n as a parameter and that increases a list of integers by a factor of n by replacing each integer in the original list with n copies of that integer. For example, if a variable called [redacted] stores [redacted] and we make the call of [redacted] the list should be changed to store [redacted]. If n is zero or negative, the list should become empty.
18. Write a method called [redacted] that doubles the size of a list by appending a copy of the original sequence to the end of the list. For example, if the list stores [redacted], your method should change it to store [redacted].
19. Write a method called [redacted] that replaces every pair of elements in the list with a single element equal to the sum of the pair. If the list is of odd size, leave the last element unchanged. For example, if the list stores [redacted], your method should change it to store [redacted] (1+7, then 3+9, then 4+6, then 5).
20. Write a method called [redacted] that moves the value at the front of a list of integers to the end of the list. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should move the value [redacted] from the front of the list to the back of the list, producing [redacted].
21. Write a method called [redacted] that switches the order of values in the list in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, switch the order of the next two, and so on. If the list contains an odd number of values, the final element is not moved. For example, if the list initially stores [redacted],

your method should switch the first pair (and), the second pair (and), and the third pair (5 and) to yield .

22. Write a method called that converts an of counts into a new of values as follows. Assume that the list stores a sequence of integer pairs that each pair indicates a count and a number. For example, suppose that an called stores . We will interpret this sequence of pairs to mean that you have 5 occurrences of 2, followed by two occurrences of −5 followed by 4 occurrences of 3, and so on. So the call of:

should cause the variable to store .

Programming Projects

1. The actual [redacted] interface in the [redacted] package has several methods beyond the ones implemented in this chapter. Write a version of [redacted] that adds some or all of these methods. The methods to add are as follows (some headers are slightly modified; see the Java API Specification for descriptions of each method):
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
2. The [redacted] package has an interface called [redacted] that extends the [redacted] interface and includes additional methods specific to iterating through the elements of lists forward or backward. Write a class called [redacted] that adds some or all of these methods. The methods to add are as follows (see the Java API Specification for descriptions of each method):
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]

- [redacted]

3. The actual [redacted] class in the [redacted] package has a method called [redacted] that returns a view of a subportion of a list through a given range of indexes. It can be useful to think of part of a list as if it were its own list, complete with its own set of indexes and values. The sublist is “backed” by the original list, meaning that it is not a copy; if any change is made to the sublist, the original list is also affected.

In order to implement this method, you will need to write an inner class inside [redacted] that extends [redacted] and implements the behavior of the sublist. Override the methods for getting and setting values at particular indexes, as well as the [redacted] method, so that they reflect the sublist’s index range and size. Also, modify the outer [redacted] class so that it always refers to its own elements through the use of these methods. The outer class should be given the following new method that returns an object of your new inner sublist class:

[redacted]

4. Based on the implementation of [redacted] or [redacted] write a class [redacted] or [redacted] that provides most of the same operations but maintains its elements in sorted order. When a new value is added to the sorted list, rather than appending it to the end of the list, it is placed in the appropriate index to maintain sorted order of the overall list.

For efficiency, you should discover the appropriate place to add new values to the list by using a binary search. Shift elements as needed and add the element in the proper index to maintain

sorted order. (Do not manually re-sort the elements such as by calling [redacted].) You should also modify the class's [redacted] method to use a binary search to locate elements.
Since the list must remain sorted, your sorted list should not retain the following operations from [redacted] or [redacted]:

[redacted]

Chapter 16 Linked Lists

16.1 Working with Nodes
• Constructing a List
• List Basics
• Manipulating Nodes
• Traversing a List
16.2 A Linked List Class
• Simple [redacted]
• Appending [redacted]
• The Middle of the List
16.3 A Complex List Operation
• Inchworm Approach
16.4 An [redacted] Interface
16. 5[redacted]
• Linked List Variations
• Linked List Iterators
• Other Code Details

Introduction

In the previous chapter, we explored how to build a list structure using an array as the underlying storage mechanism. In this chapter we will explore a different structure known as a linked list. Linked lists serve as a useful contrast to arrays because their strengths and weaknesses are completely opposite.

And just as Java has an [redacted] class as part of the collections framework, it also has a [redacted] class.

Even though Java provides an off-the-shelf implementation of a linked list, it is useful to study how it is implemented because it will give you good insights into the properties of linked lists. It is also important for you to understand the concept of a linked structure. Almost all data structures are implemented using some combination of arrays or linking or both, which means that it is important to understand each approach.

We will once again develop a structure for storing simple [redacted] values, as we did with the development of [redacted] in the last chapter. Our new structure will be called [redacted]. Keeping the data simple will allow us to focus on the data structure issues and to learn how to manipulate linked lists in general. Then we will explore some issues related to interfaces. Finally, at the end of the chapter we will explore how to turn the fairly simple [redacted] into a more general [redacted] class.

16.1 Working with Nodes

Because arrays are stored in one large contiguous block of memory, we can quickly locate any particular element of the array. Recall that this ability to quickly jump around in the array is called random access. The weaknesses of the array approach are that we can't easily insert values in or remove values from the middle without shifting other values, and we can't easily enlarge the size of the structure without constructing a brand-new array with a larger capacity.

The linked list structure has the opposite properties. It is not a random access structure, so it is not easy to jump around in the structure. It has the kind of sequential access we associate with a cassette tape. If you are playing a tape and want to skip forward 10 songs, you have to fast forward through the songs. You can't quickly skip to the position where you want to be on the tape. Linked lists have this property as well.

But where arrays are weak, linked lists are strong. We can quickly insert values in or delete values from the middle of a linked list without any shifting. It is also easy to make the list larger or smaller.

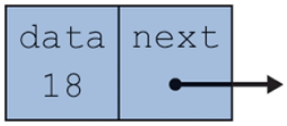
Linked lists are composed of individual elements called *nodes*.

Node

A single element of a structure such as a linked list; each node contains one data value.

A node is like a Lego building block. It looks unimpressive by itself, but once you put a bunch of them together, it can form an interesting structure.

A basic list node looks like this:



It's an object with two fields: one for storing a single item of data and one for storing a reference to the next node in the list. To store a list of integer values we'd declare the node class as follows:

This class does not produce a nicely encapsulated object with private fields, but the technique of using public fields is the usual approach to defining nodes. In the next section we'll discuss why it is acceptable to avoid encapsulation in this case.

This is a recursive data structure. The [redacted] class is defined in terms of itself because it has a field of type [redacted]. As a result, it is often possible to solve linked list programming problems more effectively by writing recursive methods, as described in Chapter 12.

Constructing a List

Let's begin by constructing a series of nodes that store the sequence of values [redacted]. There are three values, which means you'll need three nodes that are linked together. When you're creating linked lists, if you keep a reference to the front of the list, then you can get to anything in the list. You'll usually use a single variable of type [redacted] that refers to (or points to) the front of the list. You begin with this declaration:

[redacted]

The variable [redacted] is not itself a node. It's a variable that is capable of referring to a node. You haven't yet given it a value, so you would draw the following picture for memory at this point in the program:

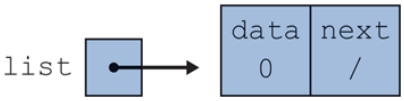
The "?" will be replaced with a reference to a node, which means that this box does not have a [redacted] field or a [redacted] field. It's a box where you can store a reference to such an object. You don't

have an actual node yet. To understand this distinction between an object and a variable that stores a reference to an object, you might want to review the section on reference semantics in Chapter 7.

The way to get an actual node is to call [redacted]:

[redacted]

This call constructs a new node and tells Java to have the variable [redacted] refer to it:

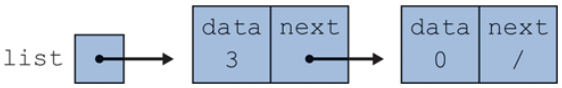


Recall that when objects are constructed, Java initializes the fields to the zero-equivalent for the type. That is why in the node above the [redacted] field has the value [redacted] and the [redacted] field has the value [redacted]. Notice that we use a slash through the box to indicate a [redacted] value.

What do you want to do with the node you have constructed? You want to store [redacted] in its [redacted] field ([redacted]) and you want its [redacted] field to point to a new node:

[redacted]

The addition of these lines of code makes our linked list appear as follows:



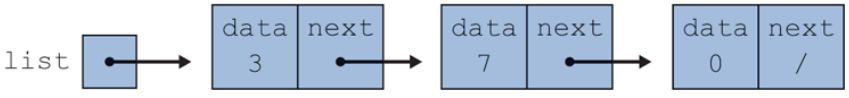
16.1-4 Full Alternative Text

When you program linked lists, you have to understand how to access the different elements of the structure. The variable [redacted] stores a reference to the first node. You can get inside that node with the dot notation ([redacted] and [redacted]). So [redacted] is the way to refer to the [redacted] box of the first node. You wrote code to assign it to refer to a new node, which is why [redacted] is pointing at this second node.

Now you want to assign the second node's data field ([redacted]) to the value [redacted] and assign the second node's [redacted] field to refer to a third node:

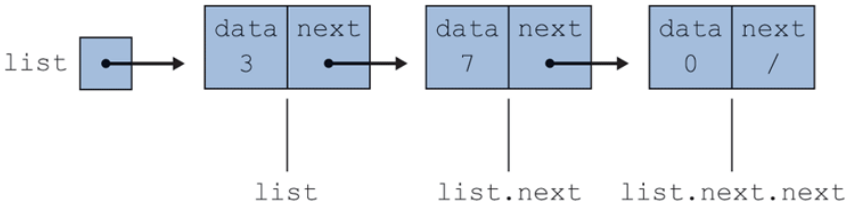
[redacted]

Now the linked list looks like this:



16.1-5 Full Alternative Text

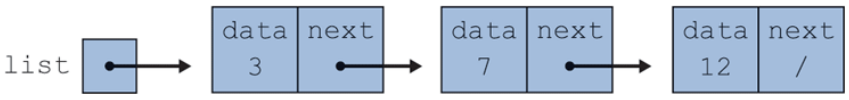
Again, pay close attention to the difference between [redacted], [redacted], and [redacted], and remember which box is associated with each of these:



16.1-6 Full Alternative Text

Finally, you want to set the [redacted] field of this third node to [redacted] (stored in [redacted]) and you want to set its [redacted] field to [redacted]:

Our linked list now looks like this:



16.1-7 Full Alternative Text

The assignment of the [redacted] field to [redacted] is actually unnecessary because Java initializes it to that value, but it's not a bad idea to be explicit about the value that you want to store in the field.

The dot notation is often very confusing for novices, so it is worth pausing for a moment to make sure that you understand the

different combinations and the elements to which they refer. Table 16.1 includes each of the possibilities for the preceding diagram.

Table 16.1 Referring to Elements of a Sample List

[redacted]	[redacted]	Our variable that refers to the first node in the list
[redacted]	[redacted]	[redacted] field of the first node ([redacted])
[redacted]	[redacted]	[redacted] field of the first node
[redacted]	[redacted]	[redacted] field of the second node ([redacted])
[redacted]	[redacted]	[redacted] field of the second node
[redacted]	[redacted]	[redacted] field of the third node ([redacted])
[redacted]	[redacted]	[redacted] field of the third node

Here is a complete program that includes all this code for constructing a three-element list along with code that prints the list:

```
public class

public static void

new

new

new
```

The program produces the following output:

Obviously, this program represents a very tedious way to manipulate a list. It’s much better to write code that involves loops to manipulate lists. But it takes a while to get used to this idea, so we’re first going to practice doing some raw list operations without a loop.

List Basics

Our previous version of the node class has just two fields:

In general, we like to keep the node class simple, so we don’t want to add much to this. But it is a good idea to include some constructors:

```
public int


public


public


this


public

int
```

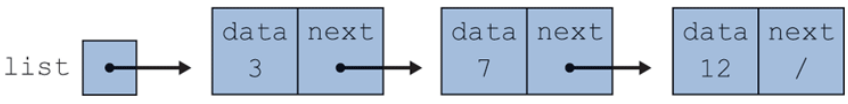
```
this


public

int


this


this
```

Like the other classes we've seen, this class has one "real" constructor (the one that takes two arguments). The other two constructors use the [redacted] notation to call the third constructor with default values (0 for the data, [redacted] for next). In the new version of the class, it is possible to write a single statement to construct the three-element list we have been studying:

```
public class

public static void

new
```

```
new

new
```

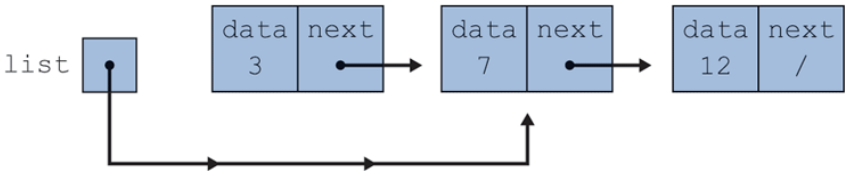
In some programming languages you have to explicitly free up memory when you are no longer using it. That is not the case in Java. In Java you can simply stop referring to a node when you no longer want to use it. For example, consider the following three-element list that we have constructed:



16.1-9 Full Alternative Text

How would you get rid of the first node? You can reassign the variable [redacted] to point to the node that comes after it. In effect, you leapfrog over the node that stores [redacted] and instead have [redacted] point to the node that stores [redacted] by writing the following line of code:

Now the linked list contains the following:



16.1-10 Full Alternative Text

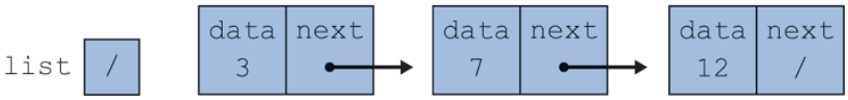
Now the variable [redacted] is pointing at a two-element list because it points at the node storing [redacted], which in turn points to the node that stores [redacted]. The node that stores [redacted] is no longer pointed to by any variable.

What happens to a node when no variable points to it? As we mentioned at the end of the previous chapter, the Java runtime system periodically invokes the garbage collector to look for objects like this node and to reclaim the space so that it can be used again. Java programmers enjoy the convenience of relying on this process, which is sometimes called *automatic garbage collection*.

A good analogy is to think of each node as a helium balloon and the arrows as the strings that we use to hold on to the balloons. If you let go of a string, then the balloon floats away. But the garbage collector will find all of those stray balloons and reclaim the space.

As one final example, consider what happens if you were to reset the [redacted] variable to [redacted]:

Our linked list would look like this:



16.1-11 Full Alternative Text

The value [redacted] is used to represent the empty list. There are three nodes floating around from the work we did earlier, but those are like helium balloons that have floated away because their strings were let go. They will eventually be recycled by the garbage collector.

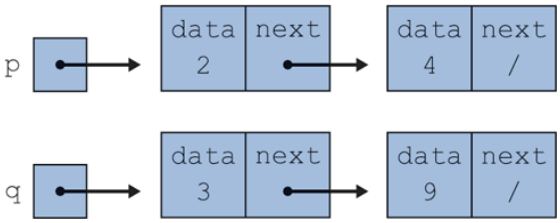
You have to be careful about manipulating an empty list. For example, if the variable [redacted] is [redacted], and you execute code that refers to [redacted], Java will halt your program by throwing a [redacted]. You will quickly discover that [redacted] is one of the most common problems that you run into as you try to debug your linked list code. Just keep in mind that Java throws that exception when you attempt to dereference a [redacted] value. In other words, it occurs when you ask for [redacted] where [redacted] has the value [redacted]. When Java throws the exception, it will show you the exact line number where it occurred. Look at that line of code carefully to find all occurrences of the dot notation, because one of them involves a [redacted] value.

Manipulating Nodes

VideoNote

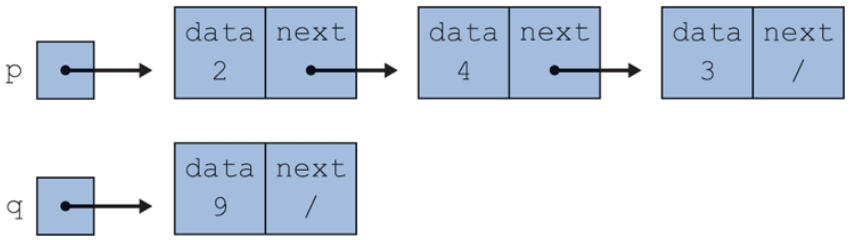
In the next section, we will explore how to use loops to write more generalized code, but first it is useful to practice basic node manipulation. A good way to practice is to make up an exercise that involves a “before” picture and an “after” picture. The challenge is to write code that gets you from the first state to the second state.

As an example, suppose that you have two variables of type [redacted] called [redacted] and [redacted] and that the following is the “before” situation:



16.1-12 Full Alternative Text

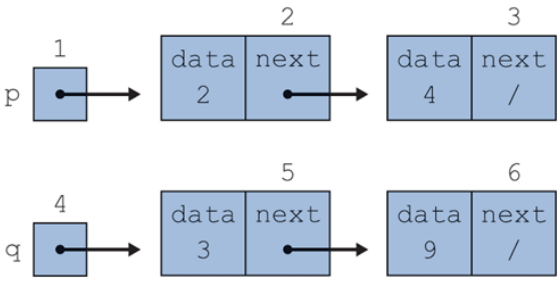
Suppose that you want to get to the following “after” situation:



16.1-13 Full Alternative Text

In order to accomplish this, you will have to rearrange the links of these two lists. As a starting point, think about how many

variables of type [redacted] there are. You might say two because there are two named variables, [redacted] and [redacted]. Or you might say four because you notice that each of the four nodes has a field of type [redacted]. Actually, there are six different variables of type [redacted]. The following diagram numbers each of the six variables:



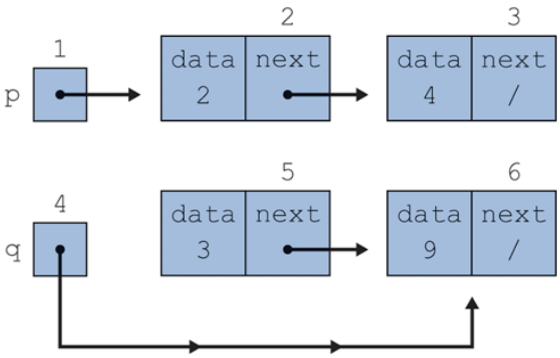
16.1-14 Full Alternative Text

Having the variables numbered makes it easier to discuss the task at hand. Which of these variables has to change in value to get from the before picture to the after picture? The boxes numbered 3, 4, and 5 have to change. If we change them appropriately, we'll be done.

We have to be careful about how we change the links. The order can be important. For example, suppose we start by changing box 4 (the variable [redacted]). In the final situation, it's supposed to point at the node with [redacted] in it. We can accomplish this by leapfrogging over the current node to which it is pointing:

[redacted]

But if we start with this change, what happens to the node that contains ? We lose track of it, as the following picture indicates:



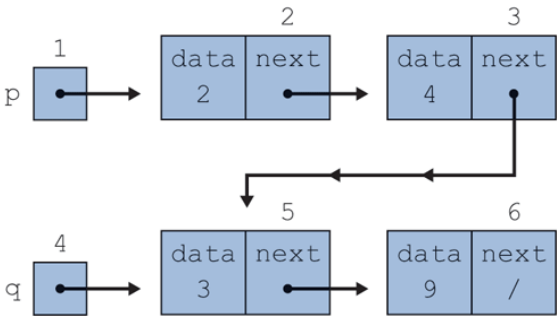
16.1-15 Full Alternative Text

Once we have lost track of the node containing , we have no way to get back to it. It's like a helium balloon that has floated away because we let go of the string.

This is a common problem in linked list programming that you have to consider carefully. One solution is to introduce a temporary variable that would keep track of the node to which the variable used to point (the node containing). Often, though, we can instead solve the problem by carefully choosing the order of the changes that we make.

Of the three values we have to change to solve this problem, the one that is safe to change is box 3 because it's currently . We begin by setting it to point to the node containing :

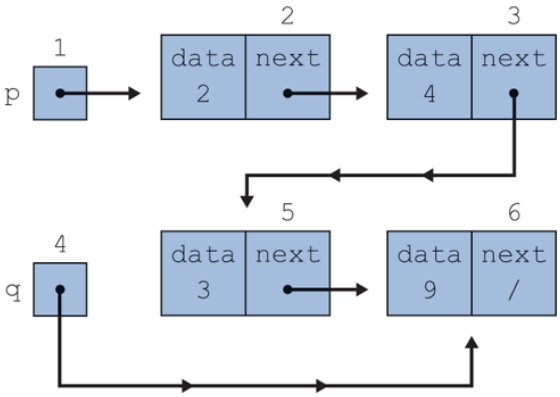
Our linked list now looks like this:



16.1-16 Full Alternative Text

Now that we've used the value of box 4 to reset box 3, we can reset box 4. It's supposed to point to the node that has in it. Now we can leapfrog over the current node to which it is pointing:

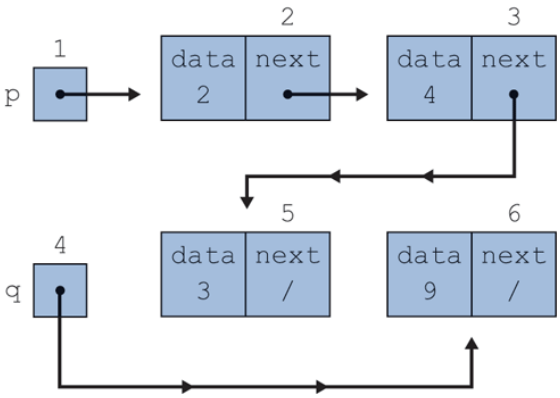
Our linked list now looks like this:



16.1-17 Full Alternative Text

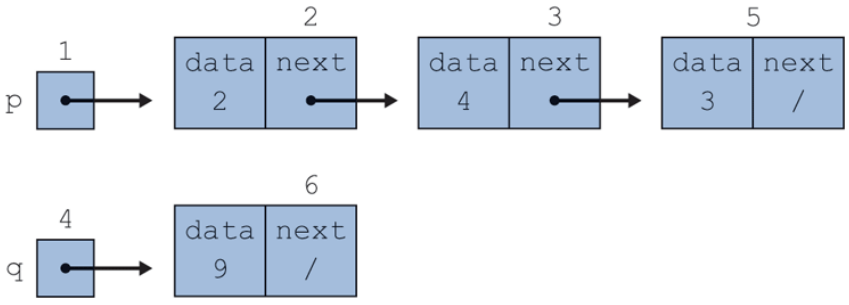
Now we just have to reset box 5. In the original picture, we would have referred to box 5 as , but now that we've changed the value of , we have to change the way we refer to this box. We can still get to box 5 by starting with the variable :

Our linked list now looks like this:



16.1-18 Full Alternative Text

We can simplify this diagram by redrawing the three nodes that are now part of the first list all in a row. Keep in mind, though, that the nodes themselves haven't moved around in memory. We have only rearranged various links. But we can more easily understand the structure if we draw the picture in this way:



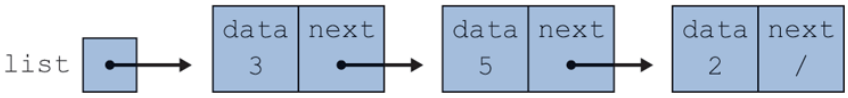
16.1-19 Full Alternative Text

And now we can see that this diagram matches the "after" picture we were given. The three lines of code that are needed to get from the initial state to the final state are as follows:

Obviously, this process can be very confusing. It is essential to draw pictures to keep track of what is pointing where and what is going on when this code executes. It's the only way to master linked list code.

Traversing a List

Consider the problem of printing each value in a list on a different line. For example, suppose we have a variable called [redacted] that stores a reference to the list [redacted]:



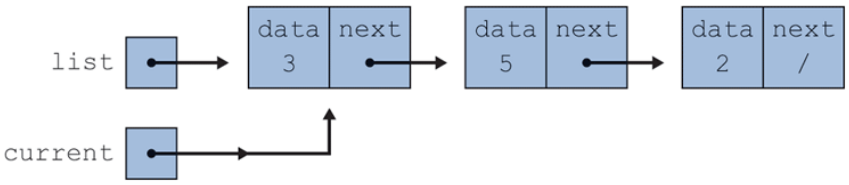
16.1-20 Full Alternative Text

Using the techniques from the previous section you could refer to each of the three data fields: [redacted], [redacted], [redacted], [redacted], and [redacted]. This approach can work for very short lists, but obviously won't work when you have hundreds or thousands of nodes to process. In that case, you'd want to write a loop.

You have just one variable to work with, the variable [redacted], so that's clearly where you have to start. You could use it to move along the list and print things out, but then you would lose the original value of the variable, which would mean that you would have lost the list. Instead, it's better to declare a local variable of type [redacted] that you use to access the different nodes of the list:

[redacted]

You can use any variable name you like, although it is common in linked list programming to use a name like [redacted] or [redacted]. The preceding line of code initializes [redacted] to point to the same value as [redacted] (the first node in the list):



16.1-21 Full Alternative Text

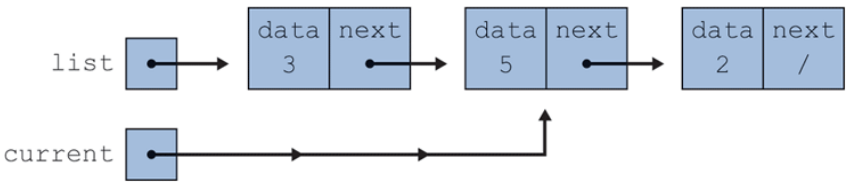
You want a loop that prints the various values, and you want it to keep going as long as there is more data to print. So how do you structure your loop? The variable [redacted] will refer to each different node in turn. The final node has the value [redacted] in its [redacted] field, so eventually the variable [redacted] will become equal to [redacted] and that's when you know your loop is finished. Thus, your basic loop structure will be as follows:

```
ListNode current = list;

while (current != null) {


process next value.


move current forward.
```

To process a node, you need to print out its value, which you can get from [redacted], and you need to move [redacted] to the next node over. The position of the next node is stored in [redacted], so moving to that next node involves resetting [redacted] to [redacted]:

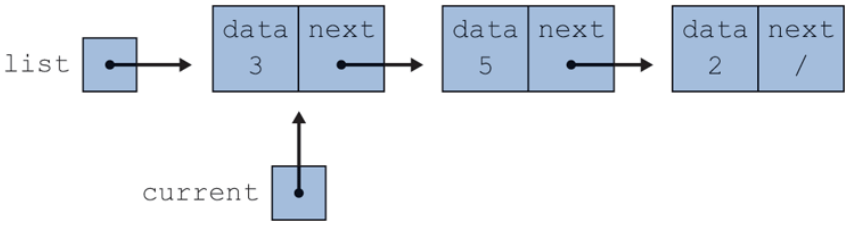
```
```

The first time through this loop, [redacted] is referring to the node with the [redacted] in it. It prints this value and then resets [redacted], which causes [redacted] to refer to (or point to) the second node in the list:



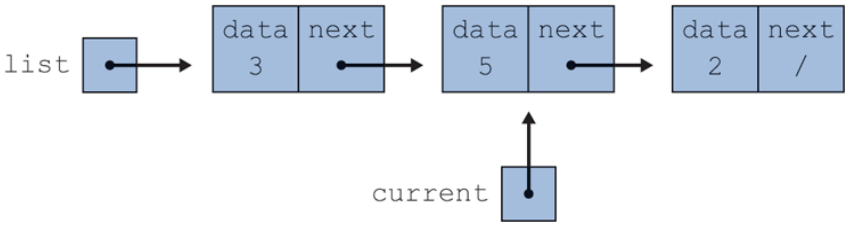
16.1-22 Full Alternative Text

Some people prefer to visualize this differently. Instead of envisioning the variable [redacted] sitting still while its arrow moves, some people prefer to envision the variable itself moving. So, for the initial situation, you draw the following picture:



16.1-23 Full Alternative Text

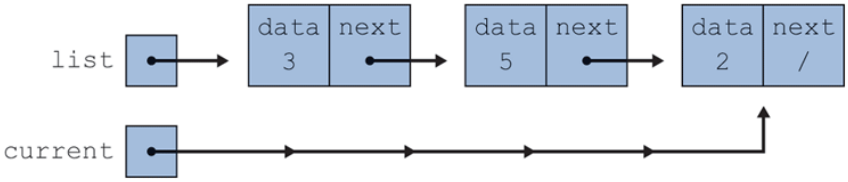
After execution of the statement [redacted], the linked list would look like this:



16.1-24 Full Alternative Text

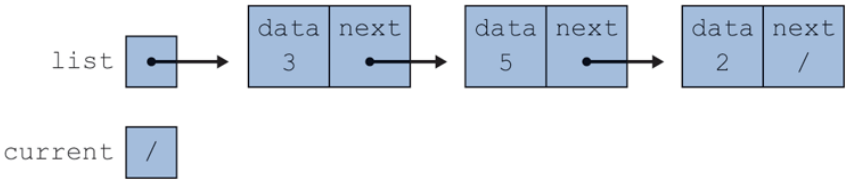
Either way of thinking about this scenario works. Because in this new situation the variable [redacted] is not [redacted], you once again

enter the loop, print out [redacted] (which is now [redacted]), and move [redacted] along again:



16.1-25 Full Alternative Text

Once again [redacted] is not [redacted], so you enter the loop a third time, print the value of [redacted], and reset [redacted]. But this time [redacted] has the value [redacted], so when you reset [redacted], the computer's memory will look like this:



16.1-26 Full Alternative Text

Because [redacted] has become [redacted], when you break out of the loop the program has produced the following output:

[redacted]

The corresponding array code would look like this:

[redacted]

Assuming that you have some comfort with array-style programming, this example might give you some useful insight into linked list programming. There are direct parallels in terms of typical code, as shown in Table 16.2.

Table 16.2 Array/Linked List Equivalents

Go to front of the list		
Test for more elements		
Current value		
Go to next element		

You may recall that in Chapter 7 we introduced a standard array traversal approach:

You can write a similar standard linked list traversal pattern:

Knowing that we like to use [redacted] loops for array processing, you can imagine writing [redacted] loops to process linked lists as well. Your previous code could be rewritten as the following code:

Some people like to write their list code this way, but most tend to use [redacted] loops. It’s an issue of personal taste.

16.2 A Linked List Class

In this section, we will explore how to write a class called [redacted] that will be a parallel of the [redacted] class of Chapter 15.

Simple [redacted]

So far we have been discussing the low-level details of how to manipulate the nodes of a class. In general, we want to provide a potential client with a simple interface that doesn't require the client to understand these low-level details. For example, the [redacted] class that we examined in Chapter 15 uses an array as its underlying structure, but a client of the class never has to know that. In a similar way, we'd like to define a [redacted] class that a client can access without having to understand that it is implemented by means of a linked list of nodes.

First we have to consider the set of fields that we need. As we've seen, we can get to any element in the list as long as we have a reference to the front of the list. So at a minimum we need a reference to the front of the list:

There are several other fields we could add to improve efficiency. Two of the most commonly added fields keep track of the length of the list and keep a reference to the back of the list. But for this first version, we will keep it simple and store just a reference to the front of the list.

Notice that for this class, the field [redacted] is declared to be private. We do that to guarantee that the class is well encapsulated. But what about those public fields in the node class? In general public fields are a bad idea. But they're not of great concern in this case because we're going to make sure that only our [redacted] object will ever manipulate individual nodes. By the time we complete this program, we will have two classes: one for individual nodes of a list and one for the list itself. We'll be careful to have a clean, well-encapsulated list object, but we don't have to worry about doing the same thing for the node class.

Some people prefer to encapsulate even node objects to keep things simple, but if you want to learn how complex classes are written, it is important to understand this convention. When we are writing code to interact with a client, we want everything to be

completely encapsulated. But when we are writing code that is seen only by the implementation, we can be more relaxed. As an analogy, think of how you behave in public versus in your own home. In your own home you might walk around in a bathrobe and slippers because you know that only you and your family have access to your home. You wouldn't tend to wear a bathrobe and slippers out in public. As another example, think of the post office boxes you find at a post office. Those post office boxes are locked for security, just like an encapsulated object, because anyone has access to the public post office. But in a corporate office setting, mailboxes often are not locked because only a limited number of people have access to them and, therefore, people are willing to be more informal.

Later in the chapter, we will see an even better way of handling this case by including the node class as a static inner class. But for now we'll work with the two classes to keep things simple.

As a first exercise, let's add a method to our class that will allow us to examine the contents of the list. We saw that code like the following could be used to print a list:

We could turn that into a method of the class by initializing the local variable to the value of our field :

```
public void print() {

front









}
```

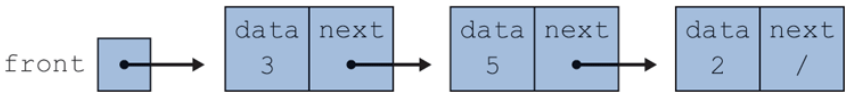
But as we saw in Chapter 15, it is a better idea to make this a method that returns a representation of the list. For the class we wrote the following method:

The code is very similar for the [redacted] class because we will have the same special case for an empty list and the same fencepost loop to make sure that values are separated by commas. If we translate each array operation into its linked list equivalent, we end up with the following code:

Notice that in this code we initialize [redacted] to [redacted] because we handle the data for the first node outside the loop.

Appending [redacted]

Next let's consider how to write a method that will append a value to the end of the list. To do so, we have to locate the end of the list. Let's think about the general case in which we are appending a value to the end of a list that already contains a few values. For example, suppose the list stores [redacted]:

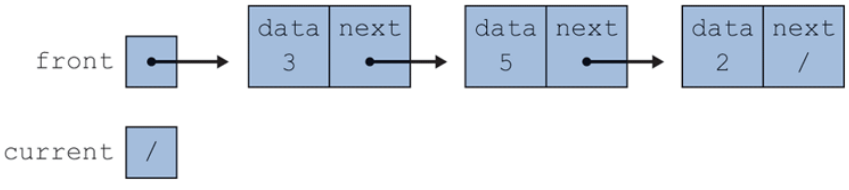


16.2-28 Full Alternative Text

Suppose that you want to add the value [redacted] at the end of the list. First you have to get to the correct location. So here's a start:

```
[redacted]
```

What happens when this code executes is that the variable [redacted] moves along the list from the first to the last node until the loop test fails—that is, until [redacted] becomes [redacted]. The computer's memory then looks like this:

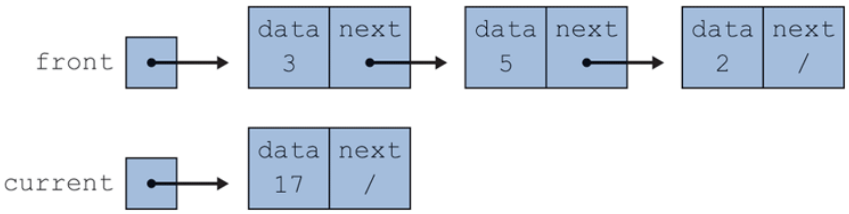


16.2-29 Full Alternative Text

It's tempting to think that you could then execute the following line of code to complete the task:

[redacted]

But that won't work. It leaves the computer's memory looking like this:



16.2-30 Full Alternative Text

The preceding code allocates a new node, but this new node has no connection to the original list. The list is still composed of three nodes linked together. This fourth node has been constructed, but it hasn't been properly linked into the list.

As you learn about linked list programming, you'll find that there are only two ways to change the contents of a list:

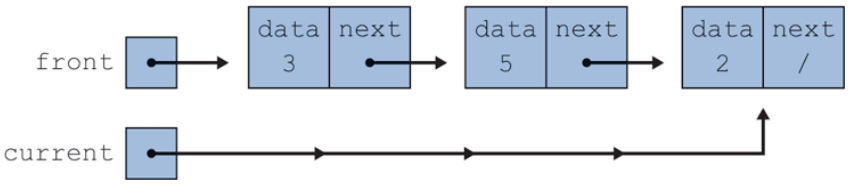
- change the value of [redacted], in which case you are changing the starting point for the list, or
- change the value of [redacted] (for some variable), which changes one of the current links of the list.

To solve this problem, you have to stop one position early. You don't want to run off the end of the list as you did with the [redacted] and [redacted] code. Instead, you want to position [redacted] to the

final element. You can do this by changing your test. Instead of running the loop until [redacted] becomes [redacted], you want to run it until [redacted] is [redacted], because only the last node of the list will have a [redacted] field that is [redacted]:

```
.next
```

After the loop executes, [redacted] will be pointing at the last node in the list:

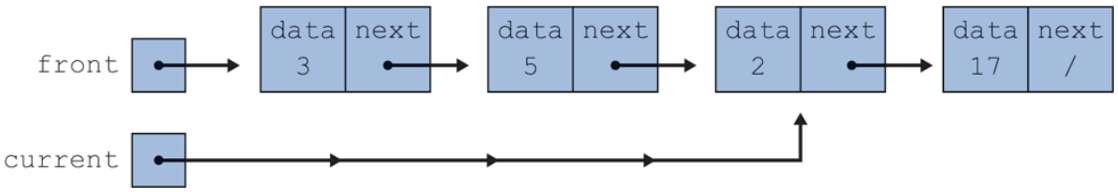


16.2-31 Full Alternative Text

At that point, you can assign [redacted] to be a new node with [redacted] in it:

```
[redacted]
```

In this case, you are changing the currently [redacted] value in that last node to instead link to a new node with [redacted] in it:



16.2-32 Full Alternative Text

You have been preparing to write code for the appending [redacted]. So this code would be included inside a method, and you would have to alter it to use the name of the parameter:

[redacted]

Even this code isn't quite correct, because you have to deal with the special case in which the list is empty:

```
if (front == null) {

front = new ListNode(value);

} else {









}
```

If you combine the [redacted] method and the [redacted] method, along with a constructor that initializes the front of the list to [redacted], you end up with the following first version of the [redacted] class:

```
public class

private

public
```

```
public

if

return

else

while
```

```
return result;
```

```
public void
```

```
if
```

```
new
```

```
else

while

new
```

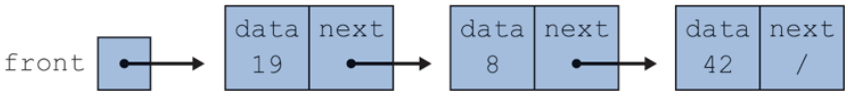
The idea of stopping one step early in processing a list isn't limited to this appending operation. It is a fundamental pattern that comes up over and over in linked list programming, as you'll see with the other examples in the chapter.

The Middle of the List

If you want your [redacted] to have the same capabilities as the [redacted] that we developed in Chapter 15, you have to add several new methods. For example, you will want to have a method called [redacted] that returns a value at a given index. When the underlying structure was an array, you could simply ask for the array element at that index. But for a linked list, you have access only to the front of the list. So, to find a value at a particular index, you have no choice but to start at the front and examine each value until you get to the desired location:

As we noted earlier, this means that your linked list does not have the fast random access property that arrays and our [redacted] have. Instead, we say that the linked list has sequential access, meaning that you have to go through the values in sequence until you get to the value of interest.

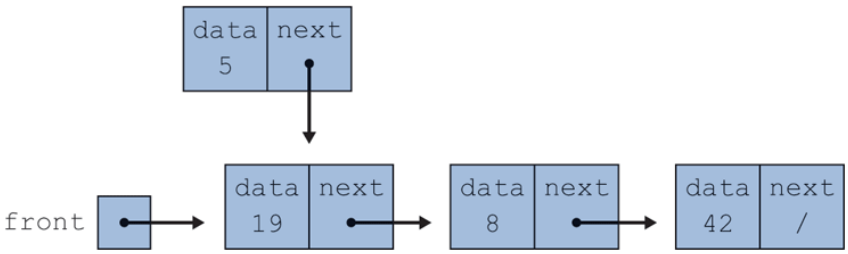
Let's now consider the task of adding a value at a particular index. There is a special case when you add at index 0, at the front of the list. For example, suppose a list contains the values [redacted]:



16.2-33 Full Alternative Text

Suppose you want to add the value [redacted] at the front of this list. Then you will need to construct a new node that has the value [redacted] and that points to the current front of the list:

Constructing this node generates the following situation:

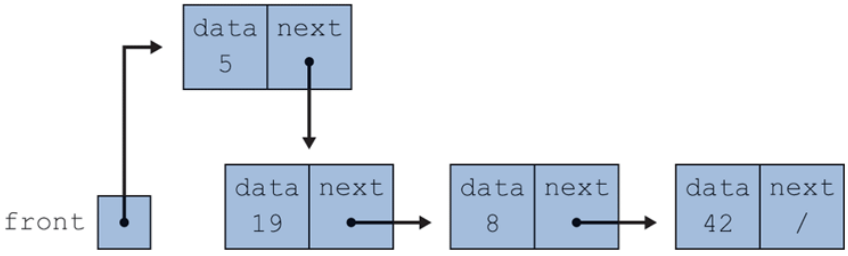


16.2-34 Full Alternative Text

But that's only part of what you need to do. To link this node into the list, you have to reset [redacted] to point to this new node:

[redacted]

Executing this line of code generates the following list:



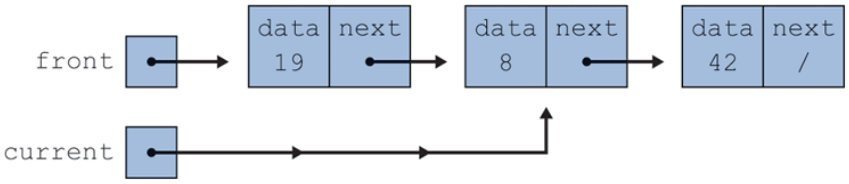
16.2-35 Full Alternative Text

This is a special case for your [redacted] method that applies only when the index is [redacted]. So your method to add at an index should begin like this:

[redacted]

If you aren’t adding at the front of the list, then you have to use a temporary variable to position yourself to the appropriate spot in the list. This is another case in which you have to stop one step early. In the case of the method, you wanted to position a local variable to the node with the given index. Here you want to stop one step before that index:

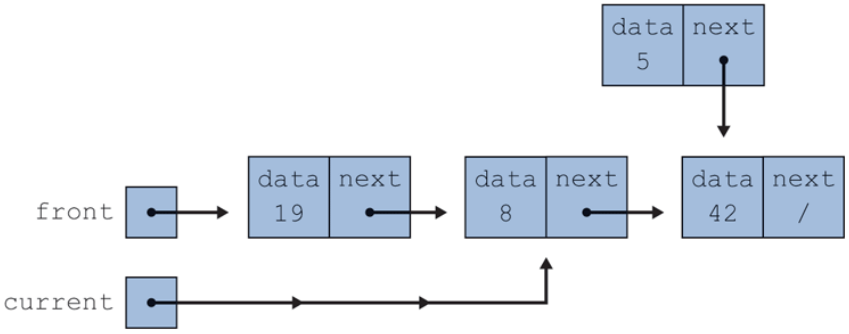
Notice that the loop test uses instead of so that it stops one step early. For example, suppose that you had the same starting list of and that you want to insert the value at index 2. You'd want to position the variable to point at the value with index 1 (the node storing):



16.2-36 Full Alternative Text

Now you can change the value of to add the new node. This new node should have a data value of . To what should its link refer? It should refer to the node that has in it, which is stored in . So you'll want to construct the node as follows:

Just calling the constructor leaves the computer's memory looking like this:

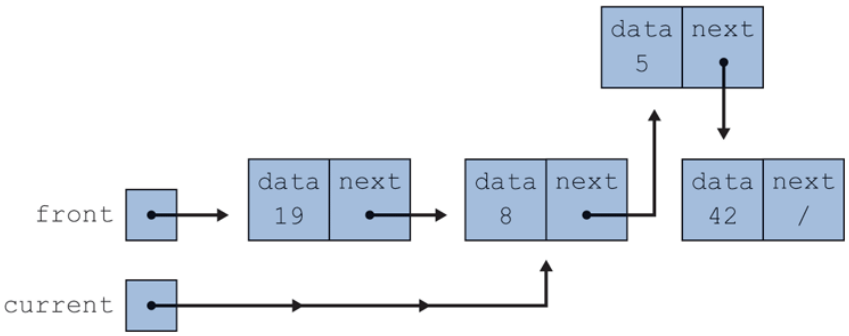


16.2-37 Full Alternative Text

Your code is not complete. You've constructed a node that points at the list, but nothing in the list points at the node. Thus, you've taken care of half of what you need to do. The other half of the task is to change a link of the list to point to the new node. The link to change is [redacted]:

[redacted]

After this modification has been made, the list will look like this:



16.2-38 Full Alternative Text

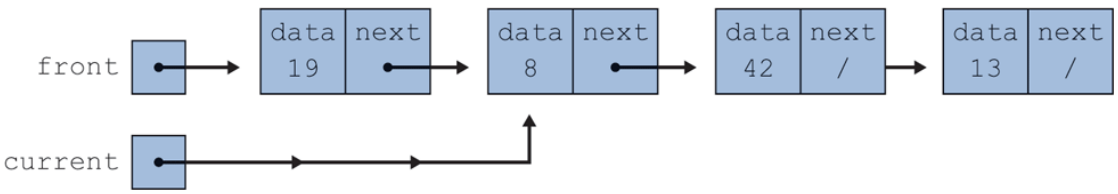
Some people prefer to write the code for creating the new node in two steps with a temporary variable, as in the following lines of code:

Both approaches work. After you have incorporated this code into your method, the code looks like this:

You can use a similar approach to writing the [redacted] method. You need to include a special case for the front of the list, in which case you can simply leapfrog over the first element:

For values that appear later in the list, you need a loop similar to the one that you used in the [redacted] method to reach the position at the node just before the one to be removed:

If you are working with the list [redacted] and you are removing the value from index 2, your list will look like the following:

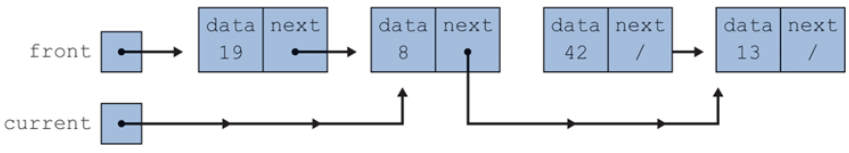


16.2-39 Full Alternative Text

To remove the value from index 2, you need to change [redacted] to point to the value that comes next, which is stored in [redacted]:

```
current.next = current.next.next;
```

Now the linked list looks like this:



16.2-40 Full Alternative Text

The code has properly linked around the node that stores the value [redacted] (which is the value that was stored at index 2), so the list now stores the sequence of values [redacted].

As you write methods that involve manipulating values in the middle of the list, it becomes clear that you often end up writing the same kind of loop to position to a particular node in the list. This is a great place to introduce a private method:

The following is a complete implementation of the [redacted] that includes the methods we have just developed, along with the other basic operations that we included in our first version of [redacted]. Notice that the private [redacted] method is used in three different methods ([redacted], [redacted] at an index, and [redacted]). Here is the code:

```
public class

private

public
```

```
public int

int

while

return
```

```
public int

int



return
```

```
public
```

```
if


return
```

```
else

while

return
```

```
public int

int


int


while


if
```

```
return
```

```
return
```

```
public void
```

```
int
```

```
if
```

```
new

else

while

new
```

```
public void

int

int



if



new



else



new
```

```
public void


if


else
```

```
private

int

for

int
```

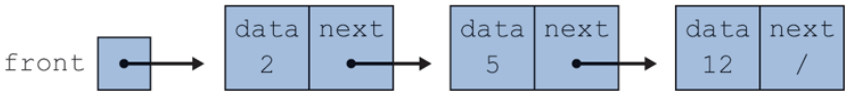
```
return
```

16.3 A Complex List Operation

Suppose that you want to write a new method for your [redacted] class called [redacted] that would add values to the list in a way which preserves sorted order. In other words, the method would be specified as follows:

Our exploration of this task will point out important cases to consider and significant pitfalls that you must be careful to avoid when you program with linked lists.

Let's first look at the general case of adding a value to the middle of a list. Suppose that the list currently stores the values [redacted]:



16.3-41 Full Alternative Text

Now suppose that you call the [redacted] method and pass it the value [redacted]. How do you add the value [redacted] to this list to preserve sorted order? First you need to find the right spot to insert it. It belongs between the [redacted] and [redacted], because it is larger than [redacted] and smaller than [redacted]. And how do you write that as a loop? You have to compare the value against the various data values stored in the list, starting with the first value. The new node doesn't belong in front of the node with [redacted] in it because [redacted] is less than [redacted]. Likewise, it doesn't belong in front of the node with [redacted] in it because [redacted] is less than [redacted]. But it does belong in front of the node with [redacted] in it, because [redacted] is not less than [redacted]. Your first attempt at writing the code might look like the following:

[redacted]

This code has the core of the right idea, but it has many problems. First of all, it ends up positioning you in the wrong spot.

You want to stop one position early in order to add something to the list, just as you did with the two [redacted] methods. Remember that there are only two ways to change the structure of a linked list: Either change [redacted], or change one of the [redacted] fields of one of the nodes.

In this case, you want to change the [redacted] field of the node that has [redacted] in it. So you don't want your variable [redacted] to end up referring to the node that has [redacted] in it. You want [redacted] to point to the node that has [redacted] in it. You have to modify the code to stop one position early. You can do this by changing the test to use [redacted] instead of [redacted]:

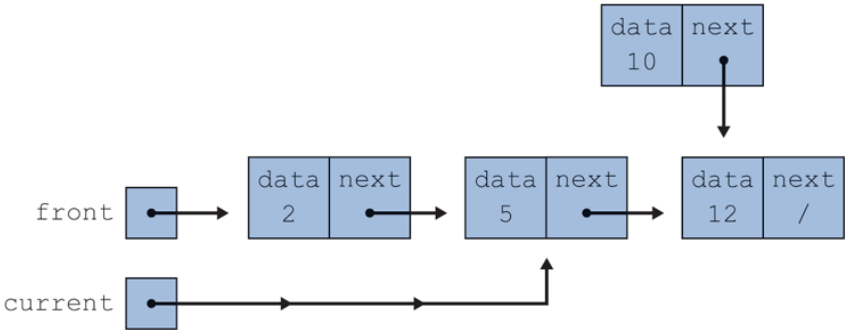
```
current.next.data
```

In effect, your test now says, "While the value of the data one position to the right of my current position is less than [redacted], keep advancing [redacted]." This loop stops when [redacted] refers to the node with [redacted] in it, which means that you can link in the new node by changing [redacted]. This new node should have a data value of [redacted] ([redacted] in our example). To what should its next link refer? It should refer to the node that has [redacted] in it, which is stored

in [redacted]. So you'll want to construct the node in the following way:

[redacted]

Just calling the constructor leaves the computer's memory looking like this:

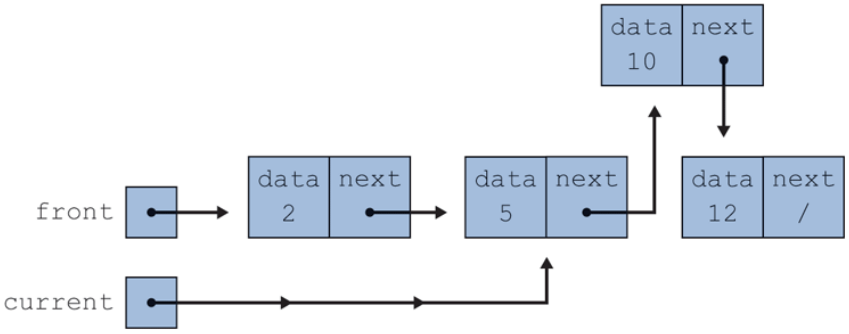


16.3-42 Full Alternative Text

But now you have to link this into the list by changing [redacted]:

[redacted]

Now the computer's memory looks like this:



16.3-43 Full Alternative Text

This diagram isn't the easiest picture to read, but if you follow the links carefully, you'll see that, starting at the front, the sequence of values is [redacted], which is what you want.

The code works for most cases, but it is incomplete. What if you want to insert the value [redacted]? Remember your loop test:

[redacted]

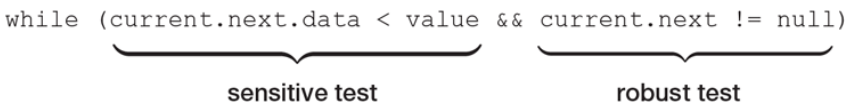
This code depends on finding a value in the list that is less than the value you are trying to insert. What if there is no such value, as in the case of [redacted]? This code keeps moving [redacted] forward until [redacted] is [redacted]. At that point, when you try to ask for the value of [redacted], you are asking for [redacted], which throws a [redacted] because Java is looking for the [redacted] field of a nonexistent object.

If the value that you wish to insert is greater than everything else in the list, then it belongs after the last node in the list. So you want to stop when [redacted] gets to the last node in the list. Thus,

a second (still incorrect) attempt at writing the code for the test would be

```
&& current.next != null
```

This code still doesn't work. The test for [redacted] being [redacted] should stop changing [redacted] at the right place, but when [redacted] is [redacted], it is not legal to ask for the value of [redacted]. That test will throw a [redacted]. This test is an example of a combination of a sensitive and robust test:



16.3-44 Full Alternative Text

You need to switch the order of these tests to make them work properly.

```
current.next != null &&
```

Recall from Chapter 5 that Java uses what is known as short-circuited evaluation, which means that if the first test evaluates to [redacted], Java doesn't bother to perform the second test. So the first test, in effect, protects you from the potential problem generated by the second test (the [redacted]).

Incorporating these changes, you end up with the following code:

But even this code is not enough. The first test in this loop is the robust test, but it isn't very robust. If [redacted] is [redacted], then it throws a [redacted]. So you want to execute this code only in the case in which [redacted] isn't [redacted].

There is another special case. If the value to be inserted belongs at the very front of the list, then this code will place it in the wrong spot. It always inserts after a node currently in the list, never in front of all nodes.

For example, suppose that you want to insert the value [redacted] in the previous list that begins with the value [redacted]. The code that you have

written starts [redacted] at the front of the list and inserts the value *after* that node by changing the value of [redacted]. So the value [redacted] would be inserted after the value [redacted], which is clearly wrong.

For the “front of the list” case, you have to write code that changes [redacted] rather than changing [redacted]. In what case would you want to do that? When the value is less than [redacted]. And what exactly do you want to do? You want to set [redacted] to a new list node that points at the old front of the list:

[redacted]

There is yet another special case. If you ran this program, you’d find that, after all of your hard work, the program throws a [redacted] on the very first call to [redacted]. Java throws an exception if the list is empty because a line in the code asks for [redacted] and [redacted] is [redacted].

So you need to include yet another test in the code:

[redacted]

```
|| front == null)
```

Of course, we have purposely written this addition in the wrong way as well. This is another example of a sensitive test (referring to) and a robust test (testing for). So you have to reverse the order of these two tests to make the modification to the code work properly.

The final version of the method is as follows:

The statement in this code deals with the two special cases we just mentioned. If the list is currently empty ( ) or if the value belongs at the front of the list ( ), then you insert the value at the front of the list rather than using the other code that you developed. The order in which this test appears is important because the test involving will throw a if is .

This code is a good example to study because it has so many special cases. In the course of writing the code, you had to deal with the following cases:

- middle: the “typical” case in which a value is inserted in the middle of the list
- back: the special case in which we had to insert at the back of the list
- front: the special case in which we had to insert at the front of the list
- empty: the special case in which we had to insert into an empty list

The first two cases are handled by the branch of the code:

```
if (front == null || front.data >= value)
```

empty list front of list

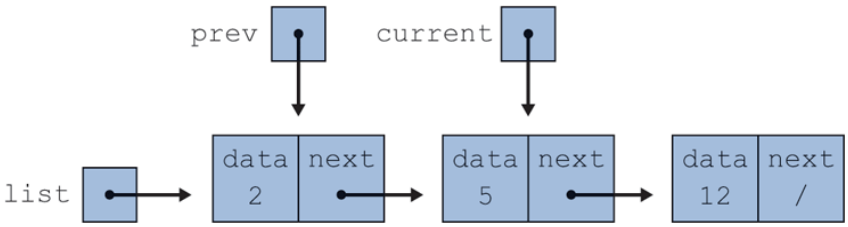
16.3-45 Full Alternative Text

The second two cases are handled inside the branch of the code:

16.3-46 Full Alternative Text

Inchworm Approach

Some people find it confusing to write code that involves expressions like [redacted]. Another approach is to use a pair of pointers that keep track of a current and a previous position in the list. In effect, the pointers keep track of a two-element window on the list, as in the following diagram:

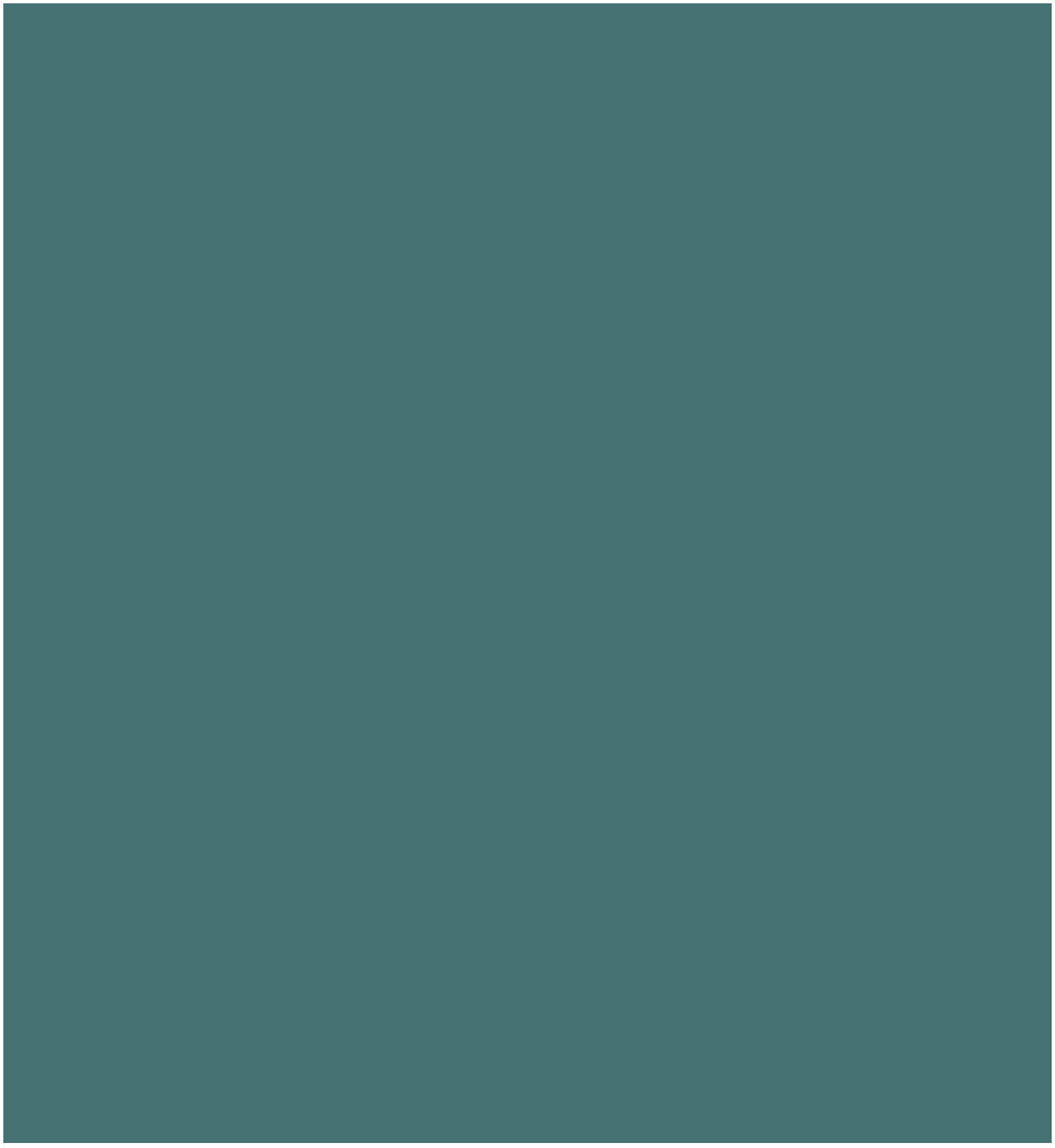


16.3-47 Full Alternative Text

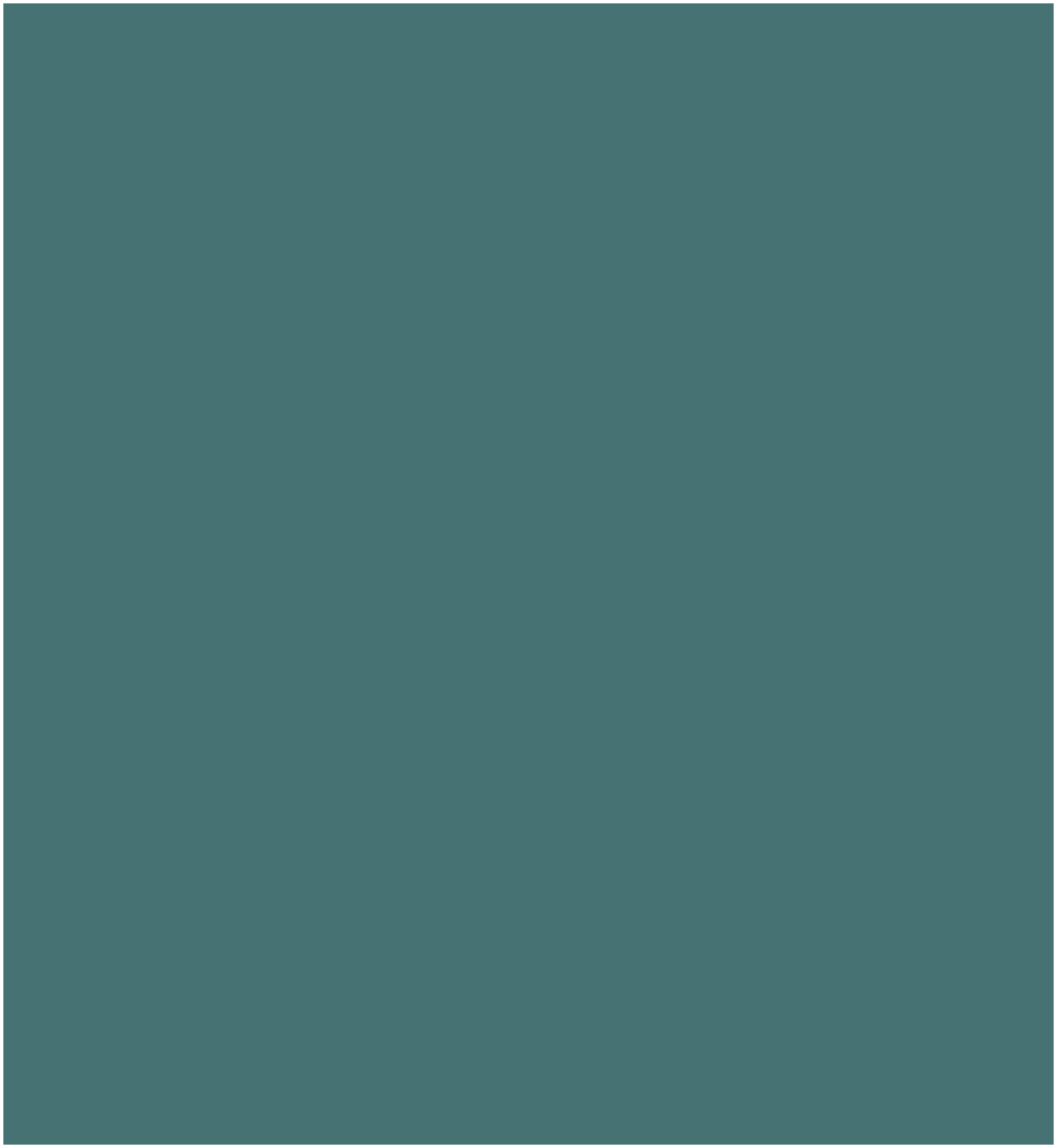
As an analogy, consider an inchworm that is two nodes in length. When the inchworm stretches out, its back half is on one node and the front half is on another node. To move forward, it scoots its back half up to where the front half is, then scoots the front half onto the next node. This is exactly analogous to the code you'd use to move this pair of variables forward one spot:

[redacted]

Here is a solution to the [redacted] problem using this approach:



Yet another variation is to set [redacted] equal to [redacted] initially in order to eliminate the special case for the front of the list. Then you have to test after the loop to see whether [redacted] is still [redacted]:



16.4 An Interface

The class from Chapter 15 and the class we have explored in this chapter have very similar methods. They each have the following methods:

- a method
- a method
- a method
- an method
- a one-argument method (the appending)
- a two-argument method (add at an index)
- a method (remove at an index)

This similarity isn't an accident. When we began studying linked lists, we purposely implemented new versions of these methods that worked for linked lists. The point is that these classes are similar in terms of what they can do, but they are very different in the ways that they do it.

Consider the following client code, which performs parallel operations on two different lists, adding three values, removing one, and printing the list before and after the

```
public class
```

```
public static void




new
```

```
new
```

The program produces the following output:

As we expected, the two kinds of list behave the same way. So what if you wanted to introduce a static method to eliminate this redundancy? What type would you use for the parameter? You can't use [redacted] because then you wouldn't be able to pass the [redacted] as a parameter, and vice versa.

As we saw in Chapters 9 and 11, the right way to approach this task is to introduce an interface. You want to think of these lists as being the same kind of thing. Even though you recognize that there are significant differences between the two, you can imagine an "integer list" abstraction of which these are two possible implementations. They're the same in the sense that they provide basic "integer list" functionality like an appending [redacted]. But they are different in the sense that they are implemented quite differently (one using an array and the other using a linked list).

You can introduce an interface with just the method headers to capture this notion of the integer list:

```
public interface


public int
```

```
public int

int




public




public int

int




public void

int




public void

int

int




public void
```

```
int
```

Using the interface, you can write a [redacted] method to use in the client program:

```
public class

public static void

new

processList(list1);
```

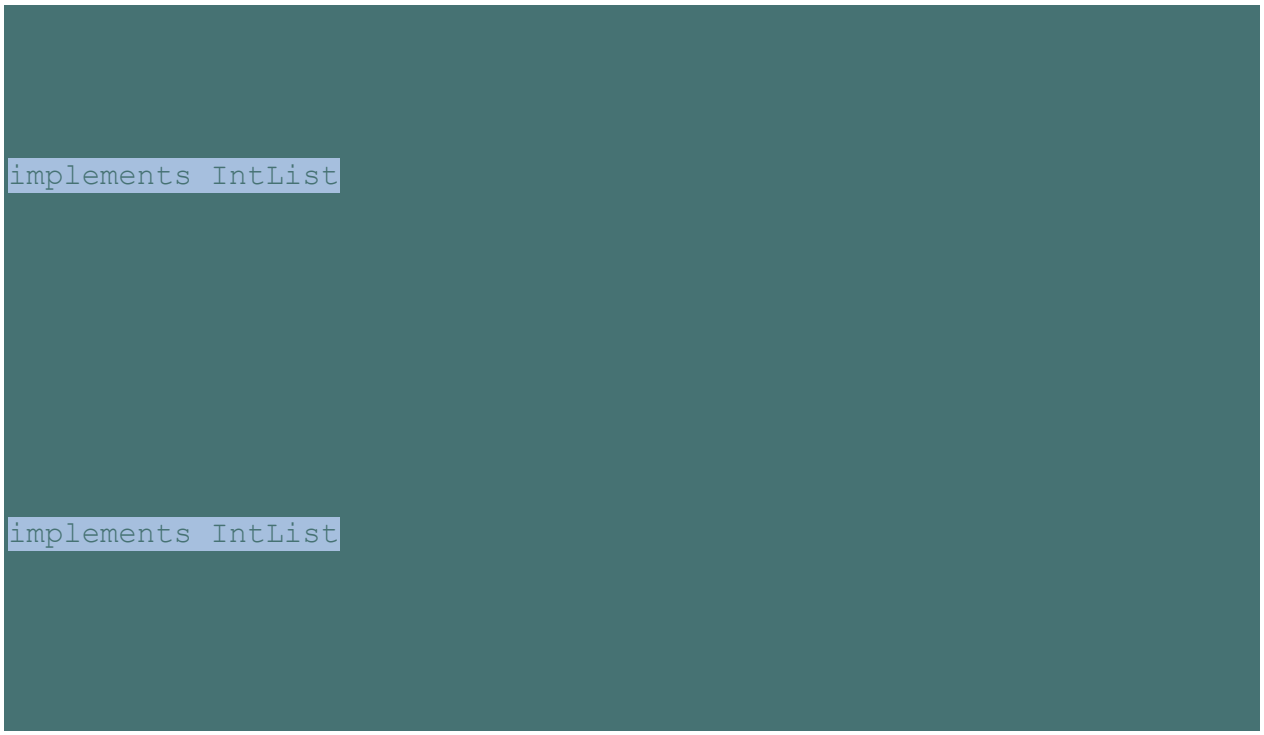
```
new
```

```
processList(list2);
```

```
public static void
```

```
IntList
```

If this is the only change you make, then your program will include two errors instead of one. Now both calls on the method cause an error. That seems a bit odd, because both [redacted] and [redacted] have the methods mentioned in the [redacted] interface. But recall that Java requires classes to explicitly state which interfaces they implement. So you have to modify the two classes to include this notation:



After you make this change, the code compiles and executes properly.

It is important to keep in mind that you can’t create instances of the interface type. The following line of code is an error:

Interfaces cannot be instantiated because they are incomplete. They specify behaviors, but they don’t say how those behaviors are implemented. So when you create an instance, you have to pick one of the actual implementations.

Even though you can’t create an instance of an interface, it is a good idea to use interfaces when you define the types of variables. Therefore, method in the client code should be rewritten as follows:

```
IntList
```

These variables are more flexible than the old variables. The variable [redacted], for example, can now refer to any [redacted] object, not just one of type [redacted].

16.5

In this section, we will use the that we have developed to produce a class that is similar to Java's class. In the collections framework, there is a close relationship between the class and the class. We'll mirror that relationship in the version that we create. The task will require making a few changes to the that we developed at the end of the previous chapter.

In particular, we'll introduce an interface that both classes will implement. We can include all of the methods that we implemented for the .

We can also introduce a slight improvement. Various structures can use the for-each loop that was introduced in Chapter 7. If you want to be able to use a for-each loop for your own class, you have to implement a special interface known as . It has one and only one method:

In other words, we need to have a method that constructs an iterator for our collection. We'll discuss later in this section how to do that for a linked list. Under the hood, the for-each loop actually calls the [redacted] method on the collection and yields each next element to the loop's body.

In this case, we want our two classes to implement both the [redacted] interface and the [redacted] interface. The usual way to accomplish that in Java is to have one interface extend another. This is similar to inheritance for classes and you use the same [redacted] keyword. Here is the complete [redacted] specification:

```
public interface

extends



public int



public
```

```
int


public int


public boolean


public boolean


public void


public void
int


public void


public void
```

```
int



public void

int



public void
```

Notice that the header mentions that it extends the [redacted] interface. That means that any class implementing the interface must provide a method for constructing an iterator, which will allow the class to be used in for-each loops.

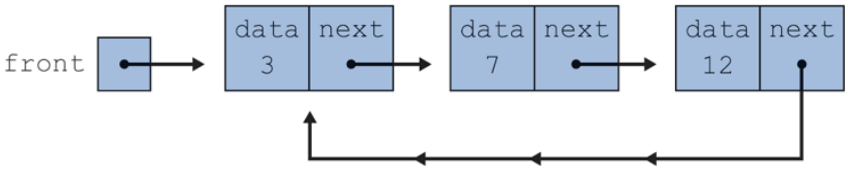
Before we write the actual code for [redacted], we'll explore some variations on linked lists and discuss the issue of iterators.

Linked List Variations

So far in this chapter, we have discussed the simplest possible linked list. Each node in this list has had a single link to the next value in the list, and we have terminated each list with the value [redacted]. We have used the value [redacted] to represent an empty list.

However, there are many variations on linked lists. Several of these ideas are used in Java's implementation of [redacted], and we will use several for our own implementation. There are three primary variations: circular lists, lists with dummy nodes, and doubly linked lists. Let's consider each variation in turn.

Instead of making the final element of the list store the value [redacted], we can have the final element of the list point back to the first value in the list. We refer to such a list as *circular*. For example, here is a circular list that stores the values [redacted]:



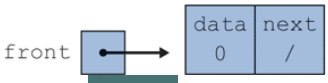
16.5-48 Full Alternative Text

There are many applications in which it can be helpful to have the list point back to the front in this manner. To manipulate such a list, you use very different code. You can't test for [redacted] because there are no [redacted] links in the list. Instead, you test whether you have returned to the point at which you started. It is often helpful to use a [redacted] loop for processing a circular list. For example, the following code prints the values of the list, one per line:

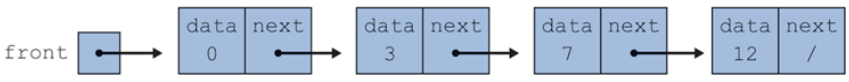
[redacted]

One of the tricky aspects of working with a circular list is dealing with an empty list. The preceding code, for example, generates a [redacted] if the variable [redacted] has the value [redacted].

This brings us to our second variation. It is often helpful to have extra nodes in the list that do not store meaningful elements and that are not considered part of the list. We refer to such a node as a *dummy node*. For example, in a circular list it is often convenient to represent the empty list as a single dummy node that points back to itself. Dummy nodes are useful even with the kind of null-terminated lists we have been writing. By using a dummy header node, we represent the empty list as having just the dummy node:

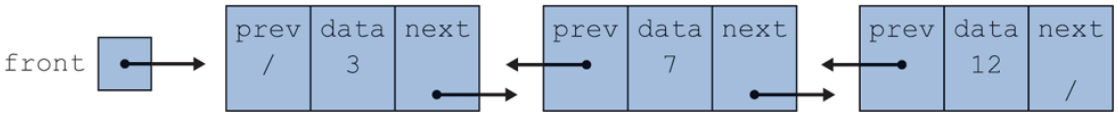


The [redacted] field in the preceding list stores [redacted], but it doesn't matter what value we store in the dummy node. The advantage of using a dummy header node is that we no longer have to write special code to insert a value at the front of the list. Now the front of the list will always be this dummy header node. For example, if we were to insert the values [redacted], [redacted], and [redacted], they would be inserted after the dummy node:



16.5-50 Full Alternative Text

The final linked list variation is called a *doubly linked list*. The idea is that instead of storing links in a single direction, we store them in both directions. That means that our node class has fields for both [redacted] and [redacted] and that we maintain the links in both directions. For example, the three-element list [redacted] would be stored as follows:

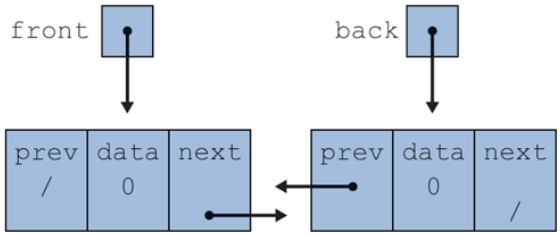


16.5-51 Full Alternative Text

Notice that each node now has two different links pointing in different directions. The list also has two [redacted] links. The final node has a [redacted] field that is [redacted] and the first node has a [redacted] field that is [redacted] to indicate in each case that there are no other nodes in that direction.

In practice, programmers use some combination of these techniques. For example, Java's [redacted] is implemented as a doubly linked, circular list.

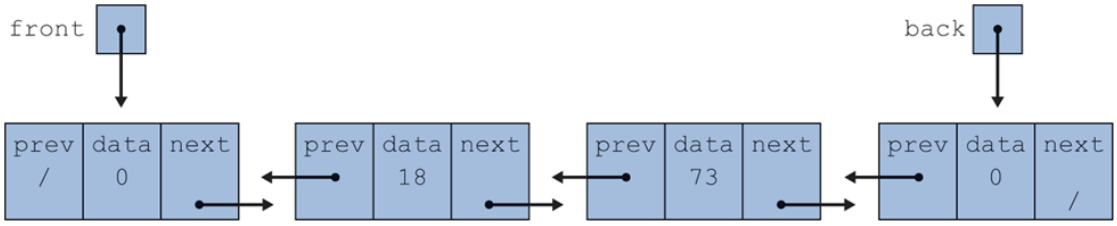
In our implementation, we will use two of these techniques. Our list will be doubly linked and will have two dummy nodes, one for the front of the list and one for the back of the list. We will also introduce an extra field to keep track of the back of the list. Thus, in our version, an empty list will look like this:



16.5-52 Full Alternative Text

This list doesn't look very empty, but that's the idea. Even when it is empty, our list will have two dummy header nodes. That way,

the fields [redacted] and [redacted] will always point to exactly the same dummy node and we won't have to write any special case code for changing their values. Instead, all insertions will occur between the two dummy nodes. For example, if we insert the values [redacted] and [redacted], our list will look like this:



16.5-53 Full Alternative Text

You can think of the dummy nodes as bookends that appear on either side of the actual nodes.

Linked List Iterators

To match our [redacted] class and to be able to implement the [redacted] interface that allows us to use for-each loops, we need to implement an iterator for our [redacted] class. Just as we did with [redacted], we can make the iterator class an inner class that has access to the fields of the outer object. In the case of the [redacted], the primary field for our iterator was an integer index. For a linked list, we want to keep track of a current list node. We still need the [redacted] field that we used for [redacted]. So our iterator class begins as follows:

[redacted]

Notice that our node class is now also generic ( ). As in the last chapter, we do not declare our class as , because the type is already declared as part of the outer linked list class. But we do place an type parameter after the and declarations.

To implement the iterator class, you have to take into consideration the fact that you will have two dummy nodes, one at the front of the list and one at the back. The first node you will want to process will come after that dummy node at the front, so the proper initialization for your iterator is as follows:

To test whether you have a next value to examine, you will use the dummy nodes again. You'll want to continue to move [redacted] forward until it reaches the dummy node at the end of the list. That means that you'll implement [redacted] as follows:

For the other two operations, you can simply translate what you did with the array implementation into appropriate code for your linked list implementation. For the [redacted] method, your code will look like the following:

For the [redacted] method, your code will look like this:

Notice that, to remove a node, you have to reset both a [redacted] and a [redacted] pointer in the list because now you are working with a doubly linked list.

Before we leave the subject of iterators, you should consider briefly how to implement the [redacted] method for your [redacted] class. Keep in mind that its header will refer to your [redacted] interface:

You have to write this code in such a way that it can take either a [redacted] or an [redacted] as a parameter. That means that you have to use methods mentioned in the interface. One misguided approach would be to use a [redacted] loop and repeatedly call [redacted] to retrieve each element:

This code will work, but it will be extremely inefficient when [redacted] is a [redacted]. Each time you call the [redacted] method, you have to start at the front of the list and skip nodes until you get to the right spot. It is much better to use an iterator to solve this problem. You can make it particularly simple by using a for-each loop and letting Java manage the iterator for you. This technique greatly simplifies the code:

You should use this same implementation for the method in the [redacted] class to make sure that it also runs efficiently for all types of lists. It turns out that several of the methods can be implemented using the same code for both the array list and

linked list. A better implementation would introduce an abstract list superclass to represent this similarity. This is left as a programming project idea.

Other Code Details

We are almost ready to put the pieces together. But first, let's consider the fields for our [redacted] class. We have already discussed keeping track of both the front and the back of the list. It is also useful to keep track of the current size in a field so that you don't have to traverse the list to find out its size. So there will be three fields:

To initialize the list, you want to construct the two dummy nodes and then set up everything to refer to an empty list. You also have to implement the [redacted] method that is supposed to return to an empty list, so this is a good opportunity to write the code once in the [redacted] method and to have the constructor call it. The constructor will look like this:

Notice that you need to include [redacted] when you construct the nodes. For the [redacted] method, you simply make these two dummy nodes point to each other and reset the size:

When you implement the [redacted] method, you can introduce an optimization. In the case of a singly linked list, you always start at the beginning of the list and move forward until you find the appropriate spot in the list. This works well when the value appears toward the beginning of the list. What if the value appears near the end? Because your list is doubly linked, you can start at the end and work backward, following the [redacted] links until you get to the appropriate spot. The best thing to do is to start from the front when the index is in the first half of the list and to start from the back when it is in the second half of the list. This optimization applies to any of the methods that involve going to a specific index ([redacted], [redacted], [redacted]). All of these methods call the private [redacted] method, so the optimization appears in just one place in the code:

In our final implementation of , we didn’t include methods for , , or . The method can simply check whether the size is , as you did for the , and the method can check the result of calling .

To write , you loop over the list, looking for the value and returning its index if you find it. Because of your dummy nodes, the loop ends up looking slightly different than the other loops you have written:

You also have to be careful to call the `equals(value)` method for comparisons.

The code involves ordinary list operations that you've seen throughout this chapter:

Many of the methods that you wrote for [redacted] can be used with only slight modifications for the [redacted]. Let's look at [redacted] as an example. Because of the dummy nodes, you don't have to make a special case for an empty list. But because of the doubly linking, you have to reset two links (a [redacted] and a [redacted]) when you remove the node:

You'll want to make similar changes to the [redacted] method.

The [redacted] class includes one final improvement. Instead of keeping the node class as a separate class, the [redacted] class includes it as a nested class. In this case, the individual node objects don't need access to the outer object, so it is best to declare the class to be a static inner class. The details of this code are beyond the scope of this book, but the quick explanation is that providing access to the outer class takes some extra storage and because we're likely to construct thousands and perhaps even millions of node objects, we want to be more efficient about the space that they take up. Declaring the class to be static accomplishes that goal.

The complete code for the [redacted] class can be found at http://buildingjavaprograms.com. You will also find the [redacted] interface and an updated version of [redacted] there.

Chapter Summary

A linked list is a data structure that stores an ordered sequence of elements using a chain of objects called nodes. Each node refers to at least one other node in the list. The overall list object keeps references to a small number of these nodes, such as the front or back node.

A node has a [redacted] and [redacted] field. You can connect one node to another by assigning its [redacted] field to refer to the other node. It is possible to make chains of nodes of arbitrary length in this way.

The end of a chain of nodes, the [redacted] reference of the last node, is [redacted].

When you are trying to rearrange one ordering of nodes into another, you must be careful about the order in which you change the references. If you change the ordering so that no variable or field refers to a given node, the node is lost and cannot be recovered.

You can traverse a linked list by creating a temporary node reference (called [redacted] in our examples) and looping through the overall list of nodes. Such a loop can stop when the program finds a particular value or encounters a [redacted] next node (the end of the list).

It is often desirable to stop one node before the relevant part of the list that you are searching for, because, in order to change the list, you often must change the [redacted] field of the prior node.

When you are performing complex list operations such as adding to a sorted list, it is important to think about all of the possible cases in which the method might be called. Is the list empty, does it have a single element, or does it have many elements? Will our element of interest be added at or removed from the beginning, middle, or end of the list?

When you create two tests separated by an [redacted] or [redacted], be sure to place the more “robust” test before the more “sensitive” test, especially if the robust test’s failure would cause the sensitive test to throw an exception.

One way to traverse a list is with two current references that are one node apart, also called the inchworm approach.

A list interface can be created to represent an abstract data type that is implemented by both array lists and linked lists. The list interface allows client code to treat both types of lists the same way.

A linked list iterator keeps a reference to its current position in the list. This class is often written as an inner class inside the overall list class so that the iterator has direct access to the list’s nodes.

A generic linked list class can store objects of any type rather than just integers.

Self-Check Problems

Section 16.1: Working with Nodes

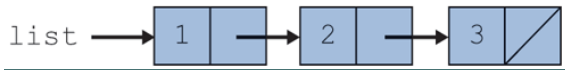
1. What is the difference between a linked list and an array list? How are they similar?
2. What is the difference between a linked node and a linked list? How are they related and connected?
3. What value is stored as the [redacted] field of the last node of a list? What value will a node's [redacted] field have if none is specified?
4. What happens if you or the client try to go past the last element in a linked list? Be specific.

For each of the next four problems, draw a picture of what the given linked nodes would look like after the given code executes.

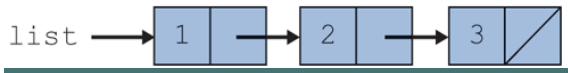
5.

6.

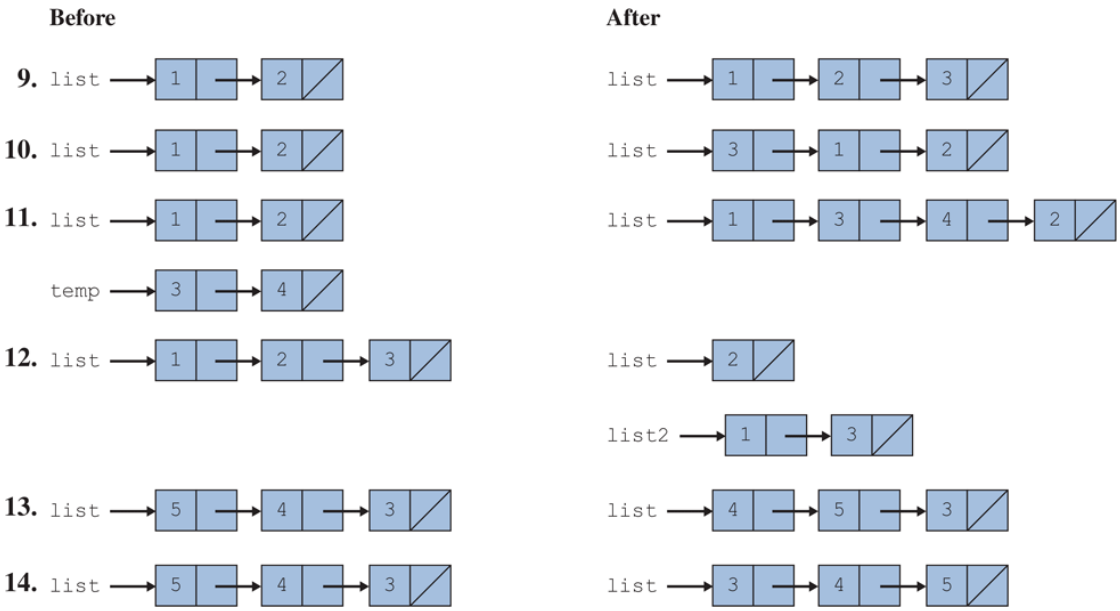
7.



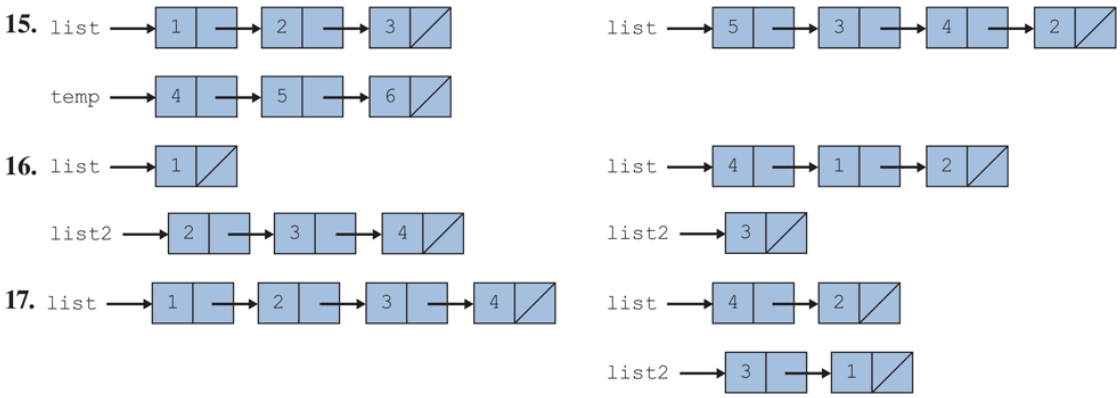
8.



For each of the next nine problems, you'll see pictures of linked nodes before and after changes. Write the code that will produce the given result by modifying links between the nodes shown and/or creating new nodes as needed. There may be more than one way to write the code, but you may not change any existing node's field value. If a variable does not appear in the "after" picture, it doesn't matter what value it has after the changes are made.



16.2-58 Full Alternative Text



16.2-59 Full Alternative Text

Section 16.2: A Linked List Class

18. What are the two ways to change the contents of a linked list?
19. An element can be inserted at or removed from the beginning, middle, or end of a linked list. Which of the three locations is the most computationally expensive, and why? How does this compare against the result for an array list?
20. When you add or remove the element found at index i of a list, you must create a temporary [redacted] node reference and advance it through the list. At which index's node should the loop stop, relative to i?
21. In an array list, it is possible to overrun the capacity of the array, at which point the list must be resized to fit. Is resizing necessary on a linked list? What limits the number of elements that a linked list can have?

For each of the next three problems, you'll see pictures of long chains of linked nodes before and after changes. (The . . . in the middle of the chain signifies an indeterminate large number of nodes.) Write the code that will produce the given result by modifying links between the nodes shown and/or creating new nodes as needed. You will need to write loops to advance to the end of the list in order to reach the node(s) to modify.

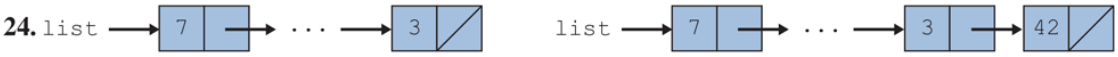
22.

(change all nodes to store the value `42`)

23.

(change the last node to store the value `42`)

24.



(add a node at the end that stores the value `42`)

16.2-60 Full Alternative Text

25. Write methods called [redacted] and [redacted] that return the smallest and largest values in the linked list, respectively. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should return [redacted] and the call of [redacted] should return [redacted]. If the list is empty, throw an [redacted].

Section 16.3: A Complex List Operation

26. What are the four cases examined in the [redacted] method?
27. What is the “inchworm approach”? What advantages does this approach have over other approaches for examining a linked list?
28. Write methods called [redacted] and [redacted] that return the sum of all values in the list and the average value as a real number, respectively. For example, if a variable called [redacted] stores [redacted], the call of [redacted] should return [redacted] and the call of [redacted] should return [redacted]. If the list is empty, [redacted] should return [redacted] and [redacted] should return [redacted].

Section 16.4: An [redacted] Interface

29. What are some advantages of creating an [redacted] interface and having both types of lists implement it?
30. Write a method called [redacted] that can accept either type of integer list as a parameter and that moves the first element of the list to the end. For example, if a variable called [redacted] contains the values [redacted], the call of [redacted] will change the list to store [redacted].

Section 16.5: [redacted]

31. What are some changes that need to be made to the linked list class to convert it from storing integers to storing objects of type [redacted]?
32. Why is an iterator especially useful with linked lists?
33. What state does the linked list iterator store? How does the iterator know if there are more elements left to examine?

Exercises

Each of the following exercises is a method to be added to the [redacted] class from this chapter.

1. Write a method called [redacted] that accepts an index and a value and sets the list's element at that index to have the given value. You may assume that the index is between 0 (inclusive) and the size of the list (exclusive).
2. Write a method called [redacted] that returns the minimum value in a list of integers. If the list is empty, it should throw a [redacted].
3. Write a method called [redacted] that returns [redacted] if the list is in sorted (nondecreasing) order and returns [redacted] otherwise. An empty list is considered to be sorted.
4. Write a method called [redacted] that accepts an integer value as a parameter and that returns the index in the list of the last occurrence of that value, or −1 if the value is not found in the list. For example, if a variable [redacted] stores the values [redacted], then the call of [redacted] should return [redacted]. If the call had instead been [redacted], the method would return [redacted].
5. Write a method called [redacted] that returns the number of duplicates in a sorted list. The list will be in sorted order, so all of the duplicates will be grouped together. For example, if a variable [redacted] stores the values [redacted], the call of [redacted] should return [redacted] because there are 2 duplicates of [redacted], 1 duplicate

of [redacted], 1 duplicate of [redacted], 2 duplicates of [redacted], and 1 duplicate of [redacted].

6. Write a method called [redacted] that returns whether or not a list of integers has two adjacent numbers that are consecutive integers ([redacted] if such a pair exists and [redacted] otherwise). For example, if a variable [redacted] stores the values [redacted], then the call [redacted] should return [redacted] because the list contains the adjacent numbers ([redacted]), which are a pair of consecutive numbers.
7. Write a method called [redacted] that deletes the last value (the value at the back of the list) and returns the deleted value. If the list is empty, throw a [redacted].
8. Write a method called [redacted] that switches the order of values in the list in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, switch the order of the next two, and so on. If the list contains an odd number of values, the final element is not moved. For example, if the list initially stores [redacted], your method should switch the first pair ([redacted] and [redacted]), the second pair ([redacted] and [redacted]), and the third pair ([redacted] and [redacted]) to yield [redacted].
9. Write a method called [redacted] that doubles the size of a list by replacing every integer in the list with two of that integer. For example, suppose a variable [redacted] stores the values [redacted], after a call of [redacted], it should store [redacted].
10. Write a method called [redacted] that takes an integer n as a parameter and that increases a list of integers by a factor of n by replacing each integer in the original list

with n copies of that integer. For example, if a variable called [redacted] stores [redacted] and we make the call of [redacted] the list should be changed to store [redacted]. If n is zero or negative, the list should become empty.

11. Write a method called [redacted] that replaces every pair of elements in the list with a single element equal to the sum of the pair. If the list is of odd size, leave the last element unchanged. For example, if the list stores [redacted], your method should change it to store [redacted] (1+7, then 3+9, then 4+6, then 5).
12. Write a method called [redacted] that rearranges the elements of a list so that all of the negative values appear before all of the nonnegatives. For example, suppose a variable [redacted] stores the values [redacted]. The call of [redacted] should rearrange the list to put the negatives first: [redacted]. It doesn't matter what order the numbers are in, only that the negatives appear before the nonnegatives, so this is only one possible solution. You must solve the problem by rearranging the links of the list, not by swapping [redacted] values or creating new nodes. You also may not use auxiliary structures like arrays or [redacted] to solve this problem.
13. Write a method called [redacted] that accepts a second linked list as a parameter and that moves values from the second list to this list. You are to attach the second list's elements to the end of this list. You are also to empty the second list. For example, suppose two lists called [redacted] and [redacted] store [redacted] and [redacted], respectively. The call of [redacted] should change [redacted] to

[redacted] and [redacted] to an empty list, [redacted]. The order of the arguments matters; [redacted] should change [redacted] to an empty list, [redacted], and [redacted] to [redacted]. Either of the two lists could be empty, but you can assume that neither list is [redacted]. You are not to create any new nodes. Your method should simply change links of the lists to join them together.

14. Write a method called [redacted] that removes all occurrences of a particular value. For example, if a variable [redacted] stores the values [redacted], the call of [redacted] would change the list to store [redacted].
15. Write a method called [redacted] that accepts a second list as a parameter, returns [redacted] if the two lists are equal, and returns [redacted] otherwise. Two lists are considered equal if they store exactly the same values in exactly the same order and have exactly the same length.
16. Write a method called [redacted] that removes the values in even-numbered indexes from a list, returning a new list that contains those values in their original order. For example, consider a variable [redacted] that stores the values [redacted] and imagine that the following call is made:

[redacted]

After the call, [redacted] should store [redacted] and [redacted] should store [redacted]. You may not call any methods of the class other than the constructor to solve this

problem. You may not create any new nodes nor change the values stored in [redacted] fields to solve this problem. You must solve it by rearranging the links of the list.

17. Write a method called [redacted] that accepts a starting and ending index as parameters and removes the elements at those indexes (inclusive) from the list. For example, if a variable [redacted] stores the values [redacted], the call of [redacted] should remove values between index 3 and index 8 (the values [redacted] and [redacted]), leaving the list of [redacted]. The method should throw an [redacted] if either of the positions is negative. Otherwise you may assume that the positions represent a legal range of the list (0≤start≤end<size).
18. Write a method called [redacted] that doubles the size of a list by appending a copy of the original sequence to the end of the list. For example, if a variable [redacted] stores the values [redacted] and we make the call of [redacted] then after the call it should store [redacted]. Notice that the list has been doubled in size by having the original sequence appear twice in a row. You may not make assumptions about how many elements are in the list. You may not call any methods of the class to solve this problem. If the original list contains N nodes, then you should construct exactly N nodes to be added. You may not use any auxiliary data structures such as arrays or [redacted] to solve this problem. Your method should run in O(N) time where N is the number of nodes in the list.
19. Write a method called [redacted] that moves the value at the front of a list of integers to the end of the list. For example, if a variable called list stores the values [redacted]

[redacted], then the call of [redacted] should move the value [redacted] from the front of the list to the back of the list, changing the list to store [redacted]. If the method is called for a list of 0 elements or 1 element, it should have no effect on the list. You may neither construct any new nodes to solve this problem nor change any of the [redacted] values stored in the nodes. You must solve the problem by rearranging the links of the list.

20. Write a method called [redacted] that rearranges the elements of a list of integers by moving to the end of the list all values that are in odd-numbered positions and otherwise preserving list order. For example, suppose that a variable [redacted] stores the values [redacted]. The call of [redacted] should rearrange the list to store [redacted]. It doesn't matter whether the value itself is odd or even; what matters is whether the value appears in an odd index (index 1, 3, 5, etc.). Also notice that the original order of the list is otherwise preserved. You may not construct any new nodes nor use any auxiliary data structures to solve this problem. You also may not change any [redacted] fields of the nodes; you must solve this problem by rearranging the links of the list.
21. Write a method called [redacted] that takes an integer [redacted] and an integer [redacted] as parameters and surrounds all nodes in the list containing the value [redacted] with new nodes containing the value [redacted]. In particular, each node that contains the value [redacted] as data should have a new node just before it and just after it that each contain the value [redacted]. If no nodes in the list contain the value [redacted], then the list should not be modified. For example, suppose that the

variables [redacted], [redacted], and [redacted] store the following sequences of values:

and we make the following calls:

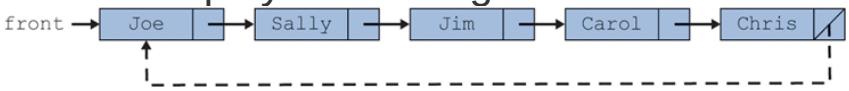
then the variables will now store these sequences:

22. Write a method called [redacted] that reverses the order of the elements in the list. (This is very challenging!) For example, if the variable [redacted] initially stores the values [redacted], the call of [redacted] should change the list to store [redacted].

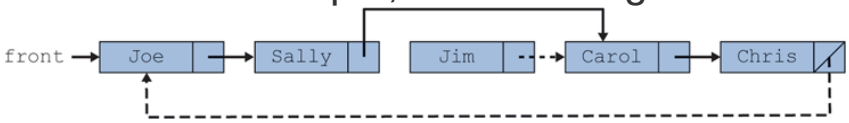
Programming Projects

1. The actual [redacted] interface in the [redacted] package has several methods beyond the ones implemented in this chapter. Write a version of [redacted] that adds some or all of these methods. The methods to add are the following (some headers are slightly modified; see the Java API Specification for descriptions of each method):
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
2. The [redacted] package has an interface called [redacted] that extends the [redacted] interface and includes additional methods that are specific to iterating through the elements of lists forward or backward. Write a class called [redacted] that adds some or all of these methods for iterating over a doubly linked list. The methods to add are the following (see the Java API Specification for descriptions of each method):
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]

3. The implementation of several methods is (or can be) the same between our [redacted] and [redacted]. Write a common abstract superclass called [redacted] that implements the common behavior and is extended by both [redacted] and [redacted]. Factor out the common code from the two list classes into this abstract superclass so that no code duplication occurs between the two. Use iterators wherever possible in the abstract code to ensure that the implementation is efficient for both types of lists.
4. “Assassin” is a real-life game in which a group of players all try individually to find and touch (or “kill”) one other player. You can use a linked list to represent this “kill ring” of players in the game:



If a “kill” is made, the ring adjusts by removing that person from the list. For example, the following occurs if Sally kills Jim:

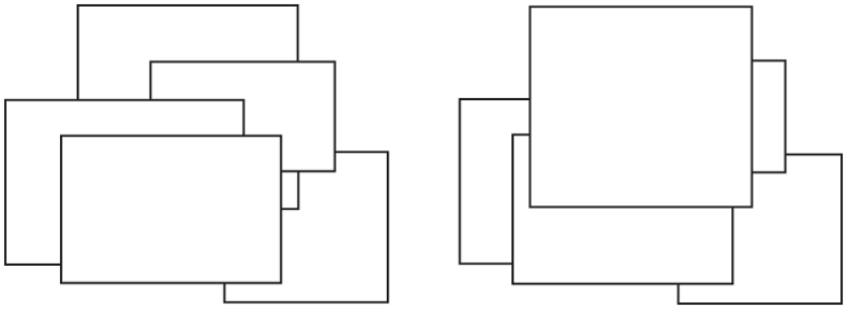


16.4-62 Full Alternative Text

Write a program that models a game of Assassin. The game reads the names of the initial kill ring from a file and puts them into a linked list in random order. Then the game repeatedly prompts for the name of a person that has been killed. The game continues until only one player remains and is declared the winner. The program should also have methods for printing the current contents of the kill ring and printing a “graveyard” of all players who have been killed.

5. Write a graphical program that shows a set of overlapping rectangular regions, each in a different random color. The regions are represented internally as a linked list. The

regions have a “z-ordering” in which rectangles closer to the end of the list are closer to the top, closer to the user’s field of view. When the user clicks the mouse, the topmost region that touches the mouse pointer moves to the very top (end) of the linked list. For example, the following diagrams show the top-left rectangle before and after the mouse is clicked:



(See the Nifty Assignments page at http://nifty.stanford.edu for a more detailed description of this project, as written by its original author, Prof. Michael Clancy of the University of California, Berkeley.)

Chapter 17 Binary Trees

17.1 Binary Tree Basics
• Node and Tree Classes
17.2 Tree Traversals
• Constructing and Viewing a Tree
17.3 Common Tree Operations
• Sum of a Tree
• Counting Levels
• Counting Leaves
17.4 Binary Search Trees
• The Binary Search Tree Property
• Building a Binary Search Tree
• The Pattern [redacted]
• Searching the Tree
• Binary Search Tree Complexity
17.5 [redacted]

Introduction

In this chapter, we will explore a new data structure known as a binary tree. Like linked lists, binary trees are composed of interconnected nodes. But unlike linked lists, which involve a one-dimensional (straight line) sequence, binary trees can branch in two directions, which gives them a two-dimensional structure.

A surprising number of data relationships can be represented using binary trees. Any relationship that involves a division into two paths can be represented with a binary tree. Thus, binary trees can store information that involves a yes/no relationship, a high/low relationship, or a first and second relationship. For example, arithmetic expressions have operators like [redacted] and [redacted] that have a first operand and a second operand. Binary trees are a natural structure for storing such relationships. Such trees are often referred to as expression trees because they capture the structure of the arithmetic expressions they represent.

Because binary trees are linked structures, they share many of the useful properties of linked lists. It is fairly easy to grow or shrink a binary tree by rearranging the links of the nodes.

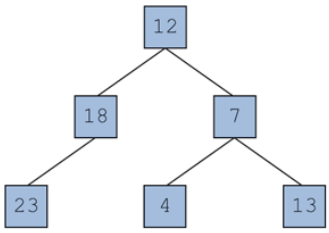
After looking at basic binary tree terminology and basic binary tree operations, we will explore a particular kind of binary tree known as a binary search tree. Binary search trees are used to capture high/low relationships and provide an efficient structure for keeping track of data that have a natural ordering. The [redacted] and [redacted] structures

that are part of the collections framework are implemented as binary search trees.

As we study binary trees, we will find that recursion comes into play quite often. Many binary tree operations are best written recursively.

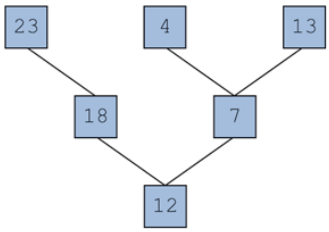
17.1 Binary Tree Basics

Here is a diagram of a simple binary tree of integer values.



17.1-1 Full Alternative Text

As we did with linked lists, we refer to each different data value as a node. This tree has a total of six nodes. Some people joke that computer scientists view the world upside down, so imagine turning the diagram around the other way:



17.1-2 Full Alternative Text

Viewed upside down, the diagram looks more like a tree. At the bottom of this structure is a node storing the value [redacted]. We refer to this value as the *root* of the tree. All nodes are connected either directly or indirectly to the root node. In this diagram, the nodes that store the values [redacted], [redacted], and [redacted] have connections branching out and up. These nodes are referred to as *branch nodes*. The nodes storing the values [redacted], [redacted], and [redacted] are at the top of this tree and have nothing above them. They are referred to as leaf *nodes* or *leaves* of the tree.

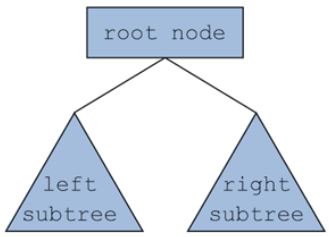
Before we try to formally define these terms, let's begin with a formal definition of a binary tree.

Binary Tree

A binary tree is either

- an empty tree or
- a root node (typically storing some data) that refers to two other trees known as the left subtree and the right subtree.

The key phrase in this definition is "subtree." The tree is composed of smaller trees. This definition is recursive. The base case or simple case is an empty tree that has no nodes at all. The recursive case describes a tree of the following form:



This recursive definition will be a useful way to think about trees as we write binary tree code. The definition will help us to more precisely define the concepts of a branch node and a leaf node.

Branch (Branch Node)

A node that has one or more nonempty subtrees.

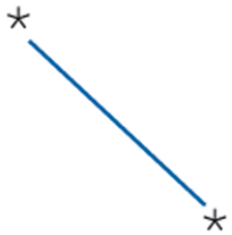
Leaf (Leaf Node)

A node that has two empty subtrees.

Using our recursive definition, let’s explore how we can form various kinds of trees. The simplest kind of tree is an empty tree, which can’t really be drawn because it contains no values.

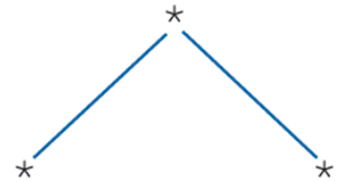
Once you understand the concept of an empty tree, you can use the second part of the definition to create a new kind of tree that is composed of a root node with left and right subtrees that are both empty. In other words, that tree would be a single leaf node, which you could represent with a star:

Now you can use the recursive rule to say that a tree can be a root node with a left tree that is empty and a right tree that is a leaf:

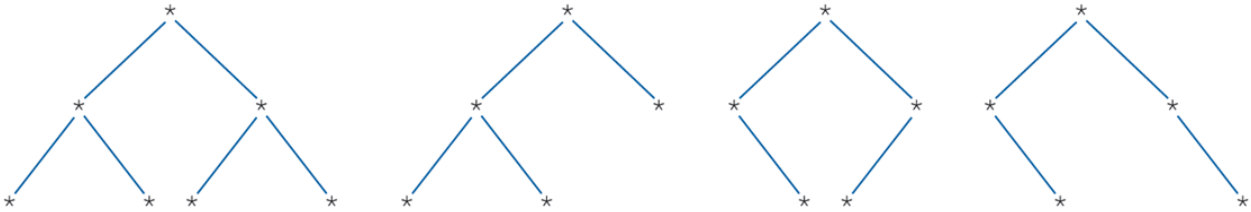


Or a tree can have an empty right and a leaf to the left:

Or a tree can have a leaf on either side:



These different forms now become possibilities to use for the recursive definition, allowing you to construct even more kinds of trees:



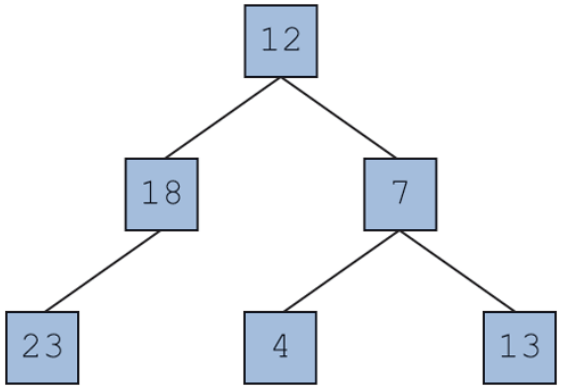
17.1-7 Full Alternative Text

Using the recursive definition of a tree, we can define the concept of parent/child relationships in the tree:

Parent/Child/Sibling/Ancestor/Descendant

For every node p that has a nonempty subtree with root node q, we say that p is the parent of q and q is the child of p, which leads logically to ancestor relationships (parent of parent of . . .), descendant relationships (child of child of . . .), and sibling relationships (two nodes with the same parent).

Let us return to our original tree:



17.1-8 Full Alternative Text

Let's start at the overall root node, which stores the value [redacted]. It has two nonempty subtrees. The left subtree stores the value [redacted] at its root and the right subtree stores the value [redacted] at its root. The nodes storing [redacted] and [redacted] are children of the node storing [redacted] and they are siblings of each other. Similarly, the node storing [redacted] is a child of the node storing [redacted], and the nodes storing [redacted] and [redacted] are children of the node storing [redacted], which makes them descendants of the overall root. The overall root is the ancestor of each of the other nodes.

Using this parent/child terminology, we can more precisely define our notion of the overall root of the tree.

Root of a Tree (Overall Root)

[redacted]

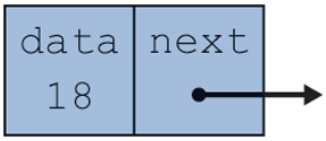
The node at the top of a binary tree, the only node in the tree that has no parent.

Another important aspect of binary trees is the distribution of nodes into different levels of the tree. Our sample tree has three

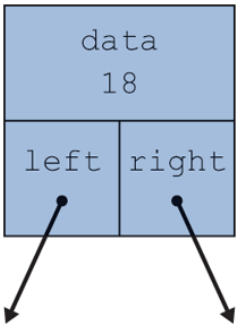
different levels: the overall root, the children of the overall root, and the grandchildren of the overall root. This notion of levels leads us to the concept of the *depth* of an individual node (how many levels away from the root it is) and the overall *height* of the tree (how many levels it has).

Node and Tree Classes

When we studied linked lists, we found that each was built from a fairly simple building block that looks like this:



Our basic building block for a binary tree will be similar, but instead of one link, it will have two:



As we did with linked lists, we will first study how to manipulate a binary tree of simple values. Here is a class definition for our binary tree nodes.

```
public class

public int

public

public

public

this
```

public

this

this

this

Like our linked list node, this node is very simple; it has some public fields and a few constructors. The node has a field of type [redacted] for storing the data contained in this node and two links of type [redacted] for the left and right subtrees. The first constructor constructs a leaf node (using [redacted] for left and right). The second constructor is appropriate for a branch node where you want to specify the left and right subtrees.

The node class is not well encapsulated, but as we saw with linked lists, this is common practice in Java programming. In this chapter, we'll define a tree class that is well encapsulated and we will make sure that a client of the tree class never sees these nodes. In the final example of the chapter we will completely hide this node class by making it a static inner class, as we did with the final version of the linked list class.

Just as we could reach every node of a linked list by starting at the front and moving forward, we can reach every node of a binary tree by starting at the overall root and moving down in the tree. As a result, we need only one field in the tree class: a reference to the root of the tree:

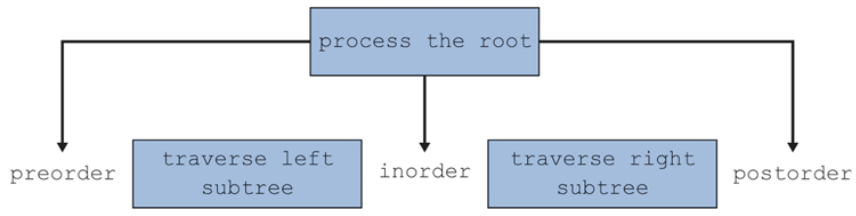
We purposely use the name [redacted] to distinguish this root from all of the other roots. There is only one overall root. But each subtree is itself the root of a tree, and in our recursive methods we'll often use the parameter name [redacted] to indicate any of the roots.

As we did with our linked lists, we'll represent the empty tree by storing the value [redacted] in the [redacted] field:

17.2 Tree Traversals

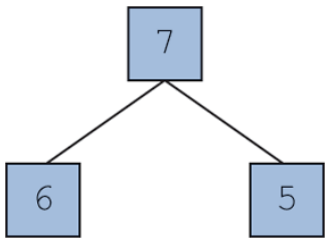
We can't do much with our tree class unless we have some way of seeing what's inside. We want to traverse the tree in such a way that we visit each node exactly once. There are many different ways to do this. Because the tree is defined recursively, it is easiest to use a recursive approach to this problem. As a result, we want to traverse the entire left subtree without dealing with any of the elements from the right and, in a separate operation, traverse the entire right subtree without dealing with any of the elements from the left. Given this decision, there are three classic binary tree traversals. Because we read from left to right, we traverse the left subtree before the right subtree. The question becomes, at what point do you deal with the root of the tree?

There are three possible answers you might give. You can process the root before you traverse either subtree, after you traverse both subtrees, or in between traversing the two subtrees. These three approaches are known as *preorder, postorder*, and *inorder* traversals respectively.



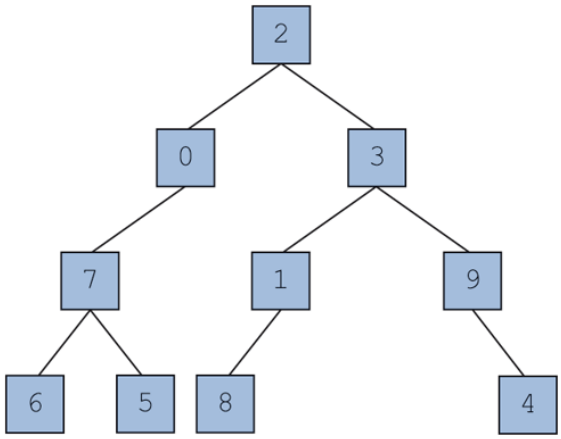
17.2-12 Full Alternative Text

For example, consider the following simple tree:



A preorder traversal visits the root, then the left subtree, then the right subtree, yielding the sequence [redacted]. An inorder traversal visits the left subtree, then the root, then the right subtree, yielding the sequence [redacted]. A postorder traversal visits the left subtree, then the right subtree, then the root, which yields [redacted]. Notice in this example that we always list [redacted] before [redacted] (left before right) and that the only value which changes position is the root value [redacted].

When we traverse a more complex tree, we simply use the recursive nature of this definition to carry out the traversal. For example, consider the following tree.



17.2-14 Full Alternative Text

The overall root stores the value [redacted], so we know that the different traversals will look like this:

- Preorder traversal: [redacted]
- Inorder traversal: [redacted]
- Postorder traversal: [redacted]

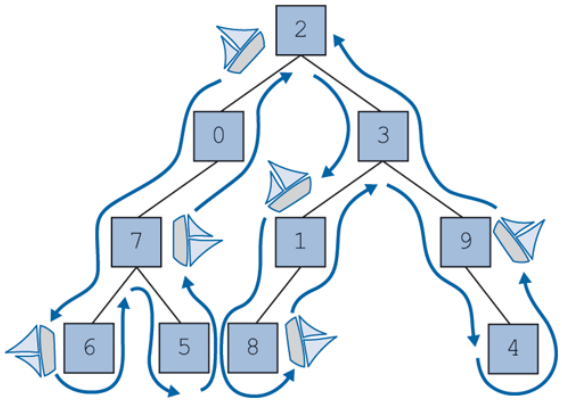
The left subtree of this tree stores the value [redacted] and has an empty right subtree. We know that each of the occurrences of [redacted] is going to be replaced by an appropriate traversal:

- Preorder traversal: [redacted]
- Inorder traversal: [redacted]
- Postorder traversal: [redacted]

Then each occurrence of [redacted] would be replaced with an appropriate traversal for the subtree that has [redacted] as its root. The following are the complete traversal results:

- Preorder traversal: [redacted]
- Inorder traversal: [redacted]
- Postorder traversal: [redacted]

There is another way to find the three different traversals. Imagine that each node in the tree is an island and that each link is a solid wall. Then imagine a sailboat sailing in from the upper left part of the tree and sailing around the tree, always hugging the coastline, and continuing around until it sails past the root on the right-hand side:



17.2-15 Full Alternative Text

The sailboat sails past the root node three different times. It passes it first on the left, then underneath, and then on the right. If you think about it the right way, you'll realize that the sailboat passes each of the nodes three times. Even for a leaf node like

the one on the left that stores , it has to pass on the left, underneath, and on the right. It does them all in a row for a leaf node, but it still passes by in all three directions.

If you print the values as the sailboat passes them on the left, you get a preorder traversal. If you print the values as the sailboat passes underneath, you get an inorder traversal. And if you print the values as the sailboat passes them on the right, you get a postorder traversal. This technique is useful for double-checking your understanding of the different traversals.

Let's now see how we can write public methods for the class that will perform these traversals. Let's begin by writing a public method that will print the preorder traversal of a tree. Your public method will look like this:

At this point we run into a problem that we discussed in Chapter 12. Often when you write a recursive solution, you have to introduce a second method that we call a helper method. When you do so, it is best to write the second method as a private method. For these binary tree methods, it will almost always be the case that you actually have to write a pair of methods to solve any of these problems.

The issue is that the client makes a request to print the entire tree. But to solve this problem recursively, you need a method that works on every subtree, not just the overall tree. In other words, you need a method that takes an [redacted] as a parameter so that the recursive method will match the recursive structure of the tree. That way each recursive call will be able to pass on smaller and smaller subtrees, which allows you to get closer to reaching a base case. As a result, you will see a common pattern in binary tree code that looks like this:

Binary tree code won't always follow this pattern exactly because there might be other actions to perform in the public method, the return type might not be [redacted] or there might be other parameters. But this general pattern will be there even with all of these variations.

To solve the problem of printing in preorder, you need a private method that has an [redacted] parameter:

You start the recursive process by passing it the overall root. You can also include code to print text at the beginning and end of the line of output that you want to produce:

How do you write the private method? It’s good to go back to the basic definition of a binary tree. Remember that it is either an empty tree or a root node with left and right subtrees. If it’s an empty tree, then there isn’t anything to print. That means you could begin your private method this way:

But since this case leaves the method with nothing to do, it’s better to test the negation of this statement:

And what do you do if [redacted] is not [redacted]? In that case, you have a root node with some data in it and you have left and right subtrees that need to be printed. In a preorder traversal you handle the root node first, which means you’d print out the data for the root. You can include a space to separate it from other values on the line of output:

```
System.out.print(" " + root.data);
```

What do you do after you print the data for this node? You want to print the left subtree in a preorder manner and then print the right subtree in a preorder manner:

This is a time to make the leap of faith that is so essential in writing recursive methods. Thinking recursively, you'll think, "If only I had a method to print a subtree in a preorder manner . . . but I do have such a method . . . the one I'm writing." Therefore, this method becomes:

```
printPreorder(root.left);

printPreorder(root.right);
```

That modification completes the method. It may seem odd that it takes so little code to perform this task, but as we saw in Chapter 12, recursive code often ends up being very short.

Now let's consider how to modify the code to print the tree in an inorder manner. The new code will have a similar public/private pair of methods:

The code will also have a similar test for an empty tree. But how do you change the body of the private printing method to make it print using an inorder traversal rather than a preorder traversal? You put the [redacted] in between the two recursive calls:

```
System.out.print(" " + root.data);
```

This simple rearrangement of the three calls causes the code to print in an inorder manner rather than a preorder manner.

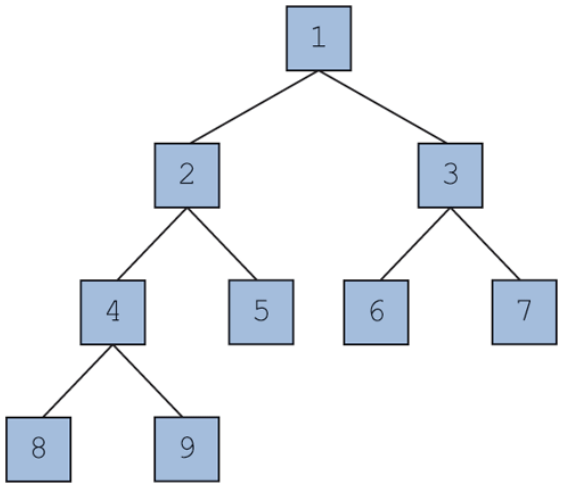
You can create another set of methods for printing in postorder by again rearranging the three operations so that the two recursive calls are made before the [redacted].

It is important to understand why this seemingly small change of moving the [redacted] statement relative to the recursive calls generates such different behavior. It's because each recursive call potentially processes a large amount of data (an entire subtree). You'll find that this is a common property of binary tree code. A minor change to the code can produce very different results.

Constructing and Viewing a Tree

It's difficult to know whether our traversal methods work unless we can construct a tree and examine its structure. Most often we develop binary tree code with a specific application in mind. We're trying to keep things simple initially, so for now, let's construct a very specific tree that will allow us to test our code.

Using recursion, we can fairly easily construct a tree that has nodes numbered in a sequential manner level by level, as in this tree of nine nodes:



17.2-16 Full Alternative Text

Notice how the number 1 is on the first level, then 2 and 3 on the next level, then 4, 5, 6, and 7, and so on. In this structure, we will fill in every possible node up to some maximum node number. Let's refer to this as a *sequential tree* to simplify our discussion.

Assume that the constructor for the class will be passed the maximum node number to include in the tree:

This is another case in which you need to introduce a private method to do most of the work for you. In our previous pattern, we wrote methods that take a parameter of type [redacted] and

our initial call passed the value of the field [redacted] to start the ball rolling. In this case, there is no tree to traverse. The whole point of the method is to construct the tree. Instead of passing [redacted] as a parameter, we want to use the method to give a value to [redacted]. And instead of passing a parameter of type [redacted], we want the method to return a value of type [redacted] (a reference to the tree it has built). Thus, the pair of methods will look something like this:

This is a good first approximation, but it isn't right. As you learn to write code for binary trees, you will discover that one of the most difficult tasks is figuring out the appropriate parameters to pass. As you practice more, you will see various patterns emerge.

One obvious parameter to include here is the value [redacted] that is passed to the public method. How can the private method do its job properly if it doesn't know when to stop? You can modify your pair to include that parameter:

```
max



int max
```

This still isn’t enough information for the private method to do its job. Think of it this way: The private method has to construct each of the different subtrees that are involved. One call on the method will construct the overall tree. Another will construct the left subtree of the overall root. Another will construct the right subtree of the overall root. How is the method supposed to know which of these tasks to solve?

We must include an extra parameter telling the method the node number to use for the root of the tree that it is constructing. When the method is passed the value , it will construct the overall root. When it is passed the value , it will construct the left subtree of the overall root, and so on. Including this second parameter, we end up with the following code:

```
1
```

```
int n
```

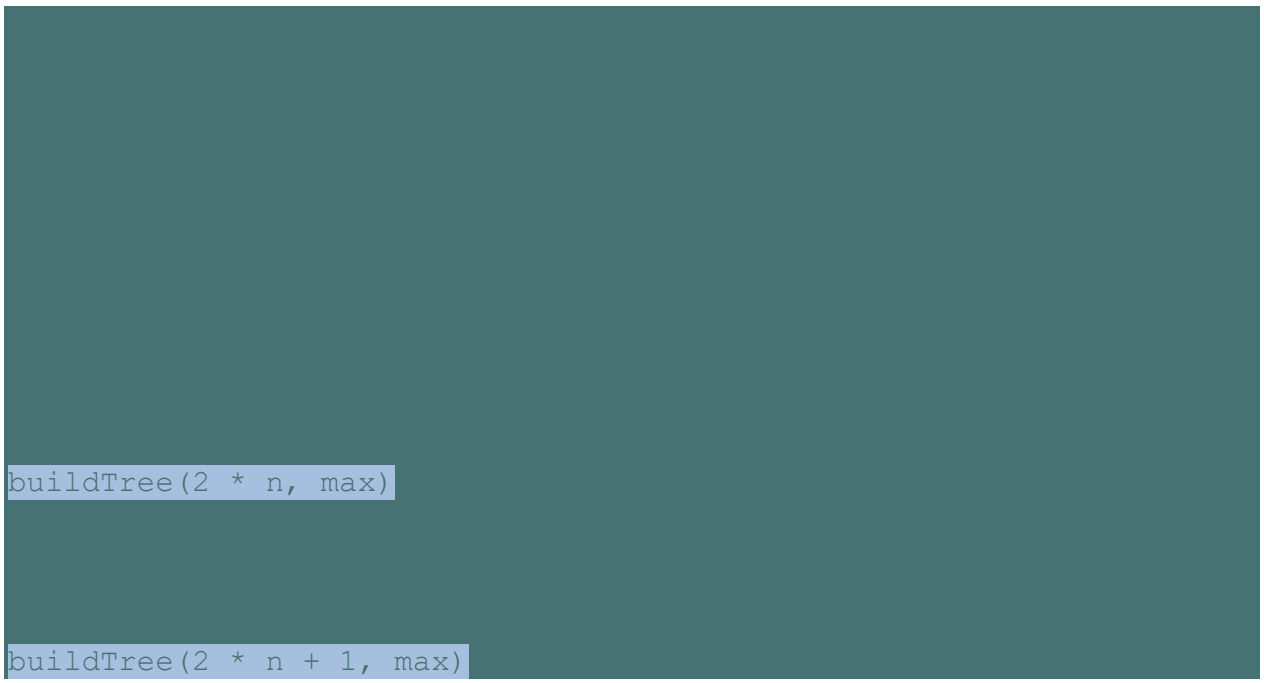
Notice that in the public method you need to pass the value [redacted] to indicate that the first call should be generating a tree with [redacted] as its root value. And as the comments indicate, you only want to construct the tree if the value of [redacted] is less than or equal to the value of [redacted].

The requirement that you want to stop building the tree for values of [redacted] that are greater than [redacted] gives you an appropriate base case for the private method:

Now you just have to fill in the recursive case. You know that you need to construct a node that stores the value [redacted]. The less obvious part is how to construct the left and right subtrees. For the moment, you can introduce some local variables for the two subtrees; later, you will fill in the details about how to construct them. The method so far looks like this:

We suggested building this particular tree because there is a simple mathematical relationship between each node in this tree and its children. If a node is numbered n, then its left child is numbered 2n and its right child is numbered 2n+1. For example, the overall root is numbered [redacted] and its children are numbered [redacted] and [redacted]. The left subtree is numbered [redacted] and its children are numbered [redacted] and [redacted], and so on.

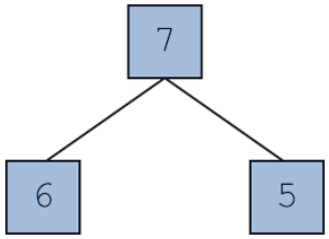
Getting back to our code, now that we know what value to store in each of the two children, we can use recursive calls to construct the left and right subtrees:



The preceding code works and is very clear in terms of the steps performed. In the recursive case (the [redacted] branch), you can see that you construct a left subtree, then construct a right subtree, and finally put these pieces together into a new [redacted] that is returned by the method. Although this code is clear, many people prefer to write it even more concisely to eliminate the local variables. You can rewrite the method above as follows:

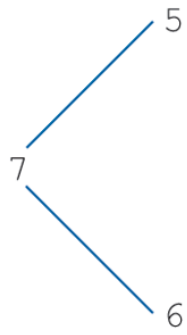
Even though this tree ends up having a very predictable structure, it is helpful to develop a method that shows the structure more clearly. Some programming environments like jGRASP have "structure viewers" that allow you to see a visual representation of a binary tree. If you don't have access to such a program, you might want to take a simplified approach to viewing the structure of the tree.

You can accomplish this by writing a simple variation of your inorder traversal. Instead of printing values all on a line, you will print them one per line and use indentation to indicate the level of the tree. For example, consider the following simple tree:



You'd want your program to produce output like this:

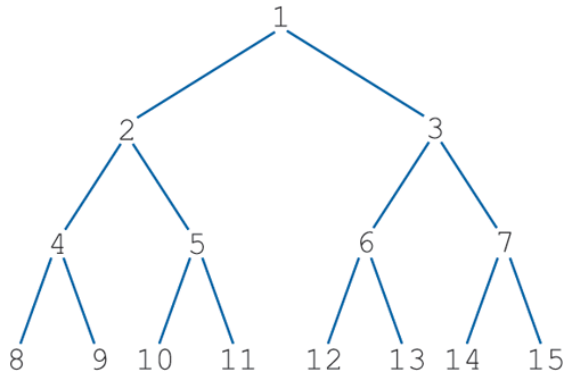
To read this output, you have to imagine rotating the page 90 degrees in a clockwise fashion or tilting your head slightly to the left. The output is a little easier to see if we throw in some lines:



The lines are a little tricky to produce, so let's settle for producing the simple output and you'll have to mentally add the lines. For example, you might get output like the following for a tree with four levels:

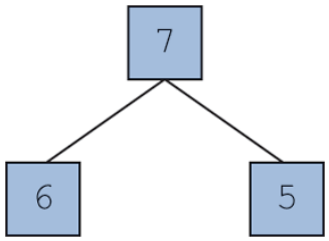
Again, imagine rotating this output 90 degrees (or rotating the page on which it is printed). The result looks like this:

If you add lines to this diagram, you'll see the tree:



17.2-19 Full Alternative Text

The output has the root node between its two subtrees, so this traversal is a variation of the inorder traversal. Oddly enough, you want to print the right subtree before the left subtree. For example, consider the following tree:



Your method would produce the following output:

Notice that the right subtree value of [redacted] is printed first, then the root, then the left subtree value of [redacted]. This happens because you are rotating the image.

You can fairly easily modify your inorder traversal code to produce this new set of methods:

But this version of the code does not indent the various levels of the tree. To obtain the proper indentation, you need to introduce an extra parameter to let the private method know the level at which each node appears. The initial call on the private method should pass the value [redacted] because the overall root should be indented [redacted] spaces. Each recursive call should pass an indentation level that is one higher for its children. The following code produces the indentation of the tree:

```
, 0
```

```
, int level
```

```
for (int i = 0; i < level; i++) {
```

```
System.out.print("    ");
```

```
}
```

Here is some client code that calls the various methods we have developed:

```
public class


public static void


new
```

This method produces the following output:

Here is the complete [redacted] class:

```
public class


private


public
int


if


throw new
```

```
private

int

int



if



return
```

```
else



return new
```

```
public void
```

```
private void

if
```

```
public void

private void

if
```

```
public void
```

```
private void



if
```

```
public void

private void

if

for

int
```

17.3 Common Tree Operations

This section discusses several common binary tree operations. All of these operations are built on top of a standard tree traversal and involve reasoning recursively about the subtrees.

Sum of a Tree

First let's write a method to find the sum of the values stored in a binary tree of . The method header will look like this:

As usual, you will need to introduce a private method that allows you to pass a node as a parameter so that you can specify which subtree of the overall tree to work with. The public method should

call the private method, pass it the overall root, and return the result:

The recursive definition tells us that a tree is either empty or a root node with left and right subtrees. This definition provides an excellent basis for recursive code. You know that the empty tree has a sum of 0, so you can begin with that case:

If the tree is not empty, then it is a root node that has left and right subtrees. To find the sum of the values, you have to combine all three parts: the data stored in the root node and the sums of the left and right subtrees. This task translates very easily into recursive code:

```
return root.data + sum(root.left) + sum(root.right);
```

Counting Levels

As a second example, let's write a method called [redacted] that returns the number of levels in a tree. For our purposes, we consider the root node to be at level [redacted], its children to be at level [redacted], its grandchildren to be at level [redacted], and so on. The [redacted] method should return the level of the node that is furthest from the root.

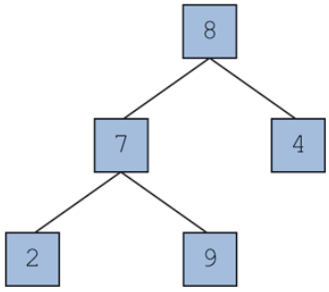
You can again solve this task by writing a public/private pair of methods. The public method returns the result of invoking the private method with the overall root:

You can again use the definition that a tree is either empty or a root node with left and right subtrees. If it is empty, then it has no levels at all (0 levels):

If the tree is not empty, then it is a root node with left and right subtrees. In this case, the data stored in the tree don’t matter. We are asking a question about the structure of the tree. To solve this problem, you should think about what a recursive call on the subtrees would return. It would tell you the number of levels in the left subtree and the number of levels in the right subtree. Those answers might match (they might have the same number of levels), but often you’ll find that one subtree has more levels than the other. In that case, what matters more is the subtree that has more levels, because that determines the number of levels in the overall tree. Here’s a first attempt at writing the method:

```
return Math.max(countLevels(root.left), countLevels(root.right));
```

Let’s think about a specific case. Consider the following tree:



If the method has been called with the overall root of the tree as a parameter, then you expect the recursive calls to return [redacted] as the number of levels for the left subtree and [redacted] as the number of levels for the right subtree. The call on [redacted] will correctly choose the [redacted] over the [redacted] for the overall answer, but the overall tree doesn’t have two levels. It has three levels. You have to account for the root node, which is an extra level, by adding [redacted] to the previous expression:

```
```

Without the “ ” in this expression, the method would always return an answer of . This is another example of a case in which a minor change in a recursive definition makes a big difference in how the method behaves.

The *height* of a binary tree is 1 less than its number of levels. When we count levels, we say that the empty tree has levels and a tree composed of a single node has level. According to the standard definition of binary tree height, a tree of one node is considered to have a height of and the empty tree is considered either to have no height (undefined height) or a height of .

Counting Leaves

As a final example, let’s write a method that returns a count of the number of leaf nodes in a tree. It will have the familiar public/private pair of methods that includes an initial call with the overall root. You can once again use the recursive definition of a tree to help you write this code. If a tree is empty, then it has no

nodes at all, let alone any leaf nodes. Thus, you can begin your method with the following lines of code:

The [redacted] case involves a root node with left and right subtrees. How many leaf nodes are in such a tree? Think about what recursive calls on the left and right subtrees will return. They will tell you how many leaves are in the subtrees. Each of those leaves is a leaf of the overall tree, so you might think that the answer is as simple as returning the sum of the subtree leaves:

```
return countLeaves(root.left) + countLeaves(root.right);
```

If you were to test this version of the code, you'd find that it always returns 0 as the number of leaves in a tree. That's because you forgot one important case. When you have a root node that has left and right subtrees, the number of leaves in that tree is generally the sum of the number of leaves in the two subtrees, except in one important case. What if the subtrees are both empty? That would mean that the node you are looking at is itself a leaf node. That particular tree has exactly one leaf node (the root). You have to introduce a special case that handles that particular situation:

```
else if (root.left == null && root.right == null) {

return 1;

}
```

It may seem odd, but that simple change makes this code work. Each leaf node returns an answer of 1 and those answers are added together by the other calls to produce a count of the various leaf nodes in the tree.

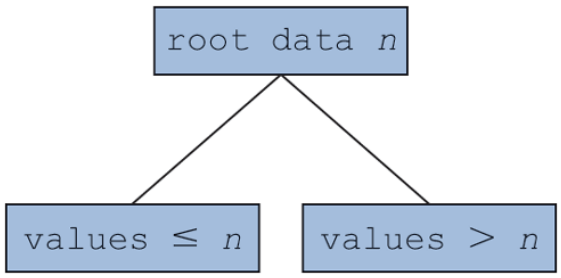
17.4 Binary Search Trees

In this section, we will study a particularly useful kind of binary tree known as a *binary search tree*. Recall from Chapter 13 that we can efficiently search through a sorted array using the binary search algorithm. That algorithm is highly efficient because on each iteration it divides a range of sorted values in half, which allows you to eliminate half of the possibilities.

A binary search tree is a structural parallel to the binary search algorithm. At each level of the tree we divide the data in half, storing approximately half the values in the left subtree and approximately half the values in the right subtree. The result is a highly efficient structure for storing a sorted sequence of values that can be quickly searched for particular values. As we noted in the introduction, both the [redacted] and [redacted] classes in the Java Collections Framework are binary search trees. In this section, we will explore how to add values to a binary search tree and how to search the tree. We will then explore some of the complexity issues related to binary search trees. We'll see that binary search trees can be very efficient, although they also can lose their efficiency if they become unbalanced. If you continue to study computer science, you will learn various techniques which guarantee that the tree stays balanced.

The Binary Search Tree Property

The values in a binary search tree are guaranteed to be arranged in such a way that the values in the left subtree are all less than the root data and the values in the right subtree are all greater than the root data. Sometimes we want to allow duplicates, in which case we have to adopt a convention about which subtree they appear in. It doesn't matter what convention we adopt, as long as we are consistent. For example, we can decide that if duplicates are allowed, then they should appear in the left subtree, which would lead to the following property:



This property of nodes is known as the *binary search tree property*.

Binary Search Tree Property

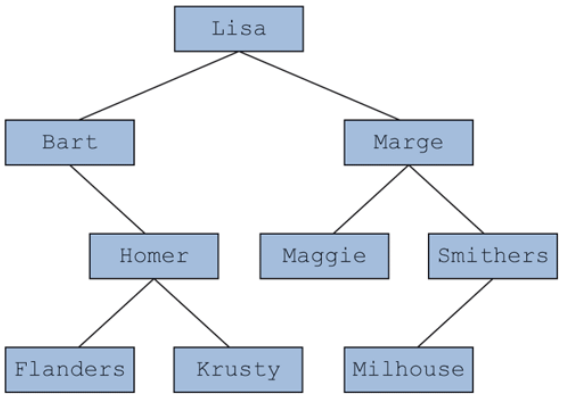
Property of a node which stores data n if all values stored in the left subtree are less than n and all values in the right subtree are greater than n. If duplicates are allowed, then all values in the left subtree must be less than or equal to n.

For a tree to qualify as a binary search tree, each node of the tree must have this property. This is not just a property of the overall root.

Binary Search Tree

A binary tree in which every node has the binary search tree property.

Here is a binary search tree of names:



17.4-23 Full Alternative Text

The names appear in the tree using alphabetical ordering (also known as *lexicographic ordering*). For example, the overall root stores [redacted], which means that all of the names in the left subtree are alphabetically less than or equal to [redacted] and all names in the right subtree are alphabetically greater than [redacted].

Consider the result you obtain when you perform an inorder traversal of the tree:

[redacted]

This list of names is in alphabetical order. This result isn’t terribly surprising when you think about the binary search tree property. In a binary search tree, you know that values which are alphabetically less than or equal to the root appear in the left subtree and that values which are alphabetically greater appear in the right subtree. For any given subtree, an inorder traversal traverses the left subtree first, then the root, then the right subtree, which means you visit the nodes in this order:

Therefore, the root value is printed at just the right point in time (after the values that are alphabetically less than or equal to it and before the values that are alphabetically greater than it). And because every node in the tree has the binary search tree property, you know that the value stored in each node is printed in the correct position relative to the values that appear below it in the tree. The overall result is that each data value is printed at just the right point in time relative to the rest of the data.

This observation highlights one of the useful properties of a binary search tree. Once it is constructed, we can traverse the values in sorted order by performing an inorder traversal of the tree.

Building a Binary Search Tree

We can borrow many elements of the [redacted] class that we wrote in the previous section to produce a new [redacted] class. The primary traversal in which we will be interested is the inorder traversal, so we can include it in our new class and rename it with the simpler name [redacted].

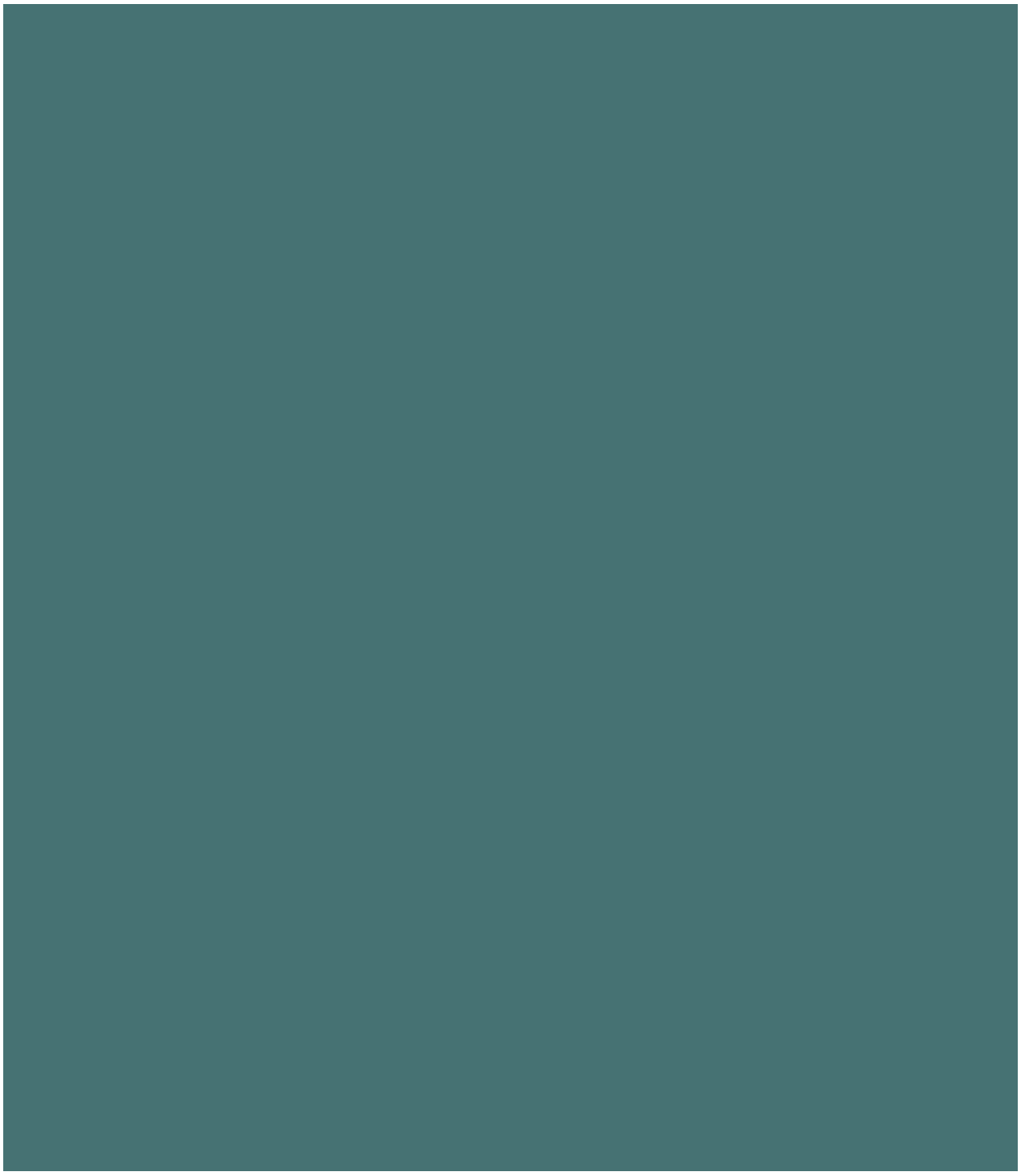
For this new class, it is best to use the concept of a data invariant, as introduced in Chapter 8. We can guarantee that the binary search tree property is true for every node in our tree by making sure that we never violate this invariant relationship. That will simplify our code because we won't have to test to make sure that our tree is a binary search tree.

We imagine that a client program will involve code like the following:

```
import

public class

public static void

new

new
```

```
int


while
```

This program prompts for a series of integers that are added one at a time into the tree until the user enters the sentinel value 0. The program then prints the structure of the tree and the contents of the tree. The following is a sample execution. Notice that the numbers are printed in increasing order:



To begin with, you will need a constructor that constructs an empty tree. Remember that the empty tree is represented by the value `null`. In fact you could simply not have a constructor and Java would provide you with a zero-argument constructor that

initializes the field [redacted] to [redacted]. But it's not a bad idea to include the constructor anyway for clarity:

Then you have to write the [redacted] method. Because of our data invariant, you can assume that, each time the method is called, the existing tree is a binary search tree. You have to make sure that you add the value in an appropriate place so as to preserve the binary search tree property.

The [redacted] method will have the usual structure of a public method that the client calls with no mention of tree nodes and a private recursive method that takes a node as a parameter and that does the actual work. Thus, your pair of methods will look like this:

Recall that a binary tree is either empty or a root node with left and right subtrees. If it is empty, then you want to insert the value here. For example, initially the overall tree is empty and you insert the first value at the top of the tree (replacing the [redacted] value with a reference to a new leaf node that contains the given value). So the private [redacted] method would start like this:

But what if it's not an empty tree? Then it must have a root node with some data. Let's assume that duplicates are allowed, so that you want to insert this value no matter what. In the case in which [redacted] is not [redacted], you know that the new value has to be added to one of the subtrees. Which one? You can compare the value you are inserting to the data in the root to figure that out. If the value is less than or equal to the root's data, then you insert the value into the left subtree; otherwise, you insert into the right subtree. Your method will look something like this:

```
else if (value <= root.data) {

// add to left

} else {

// add to right

}
```

This is the general structure that you want, but exactly how do you add to the left or right subtree? Some novices try to test things about the left or right subtree, as in the following example:

```
```

```
if (root.left == null) {

root.left = new IntTreeNode(value);

} else {

// even more stuff

}
```

This is not a good way to approach the problem. If you're thinking recursively, you'll realize that it's another insertion task into either the left or the right subtree. You can call the `insert` method itself:

```
```

```
add(root.left, value);



add(root.right, value);
```

The logic of this code is almost correct. Unfortunately, in this form the tree always remains empty. The [redacted] method never inserts a value. The problem has to do with the parameter called [redacted]. The parameter [redacted] will store a copy of whatever value is passed into it. As a result, when you reassign [redacted], it has no effect on the value that is passed into it.

There is a particular approach to solving this problem that is very helpful in writing binary tree code. Let's explore this issue and the technique in some detail before we try to fix this code.

The Pattern [redacted]

Consider the following short program:

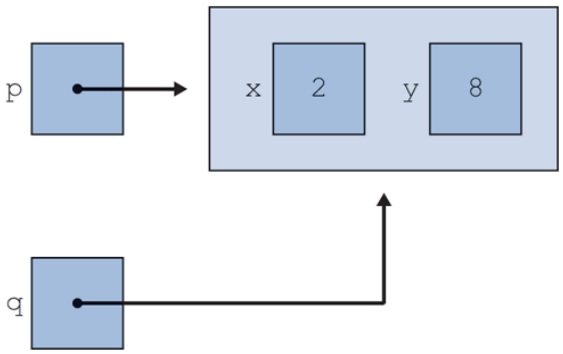
```
import


public class


public static void


new
```

```
public static void


new
```

The code in [redacted] constructs a [redacted] object and passes it as a parameter. Inside the method we translate the coordinates of the [redacted] object. This change is reflected in the original [redacted] object because when we pass an object as a parameter, the parameter gets a copy of the reference to the object.

But what about the final line of the method that constructs a new ? Does that change the method's variable ? The answer is no. The overall output for this version of the program is as follows:

One of the changes (translating the coordinates) has an effect, but the other (constructing a new) does not. This happens because when we pass an object as a parameter, the parameter gets a copy of the reference to the object. When we call the method, the variable is set up as a copy of the variable :



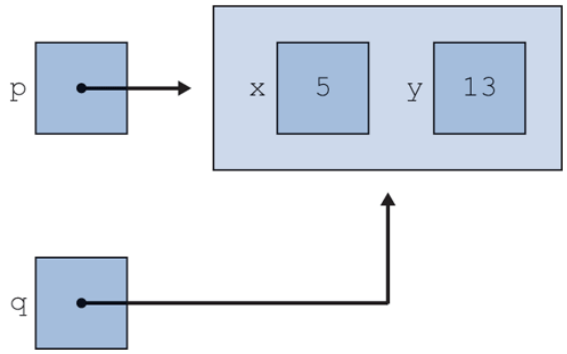
17.4-24 Full Alternative Text

This is a situation we have seen before. There are two references to the same object. We discussed this scenario in detail in the section on reference semantics in Chapter 7. There is only one object here and we have two different ways of referring to it: using the variable or using the variable . That's why the

method is able to change the object to which [redacted] refers when it executes this line of code:

[redacted]

There are two different variables referring to the same object, so either variable is capable of changing the object:

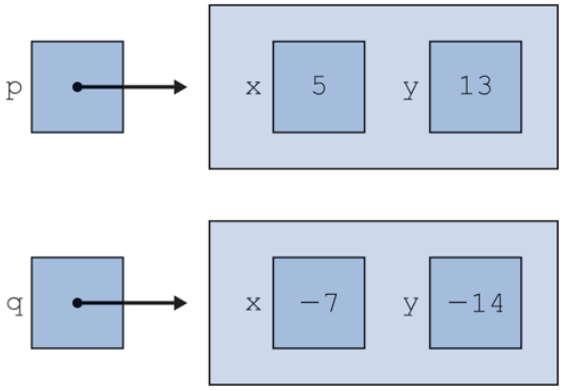


17.4-25 Full Alternative Text

But that doesn't mean that [redacted] has the power to change [redacted] itself. It can change the object to which [redacted] refers (the [redacted]), but it can't change the value that is stored in [redacted] (the reference, or pointer; or the arrow, or cell phone number). When we change [redacted] with the following line of code, it has no effect on [redacted]:

[redacted]

This code gives a new value to [redacted], but not to [redacted], as you can see in the following diagram:



17.4-26 Full Alternative Text

There are many ways to try to fix this, but there is a particular approach called the [redacted] idiom that solves the problem rather nicely. We change the method so that it returns the value of the parameter just before it exits. This modification also requires us to change the return type for the method:

```
Point



return q;
```

Then we change the call on the method to the [redacted] form:

With these changes, the program produces the following output:

The change that occurs in the method propagates back to the [redacted] method.

Now let's return to the binary tree [redacted] method to see how we could apply this technique there. Currently the [redacted] method has a return type of [redacted]:

We can change it so that the last thing we do in the method is to return the value of [redacted], which means we have to change the return type to [redacted]:

```
IntTreeNode
```

```
return root;
```

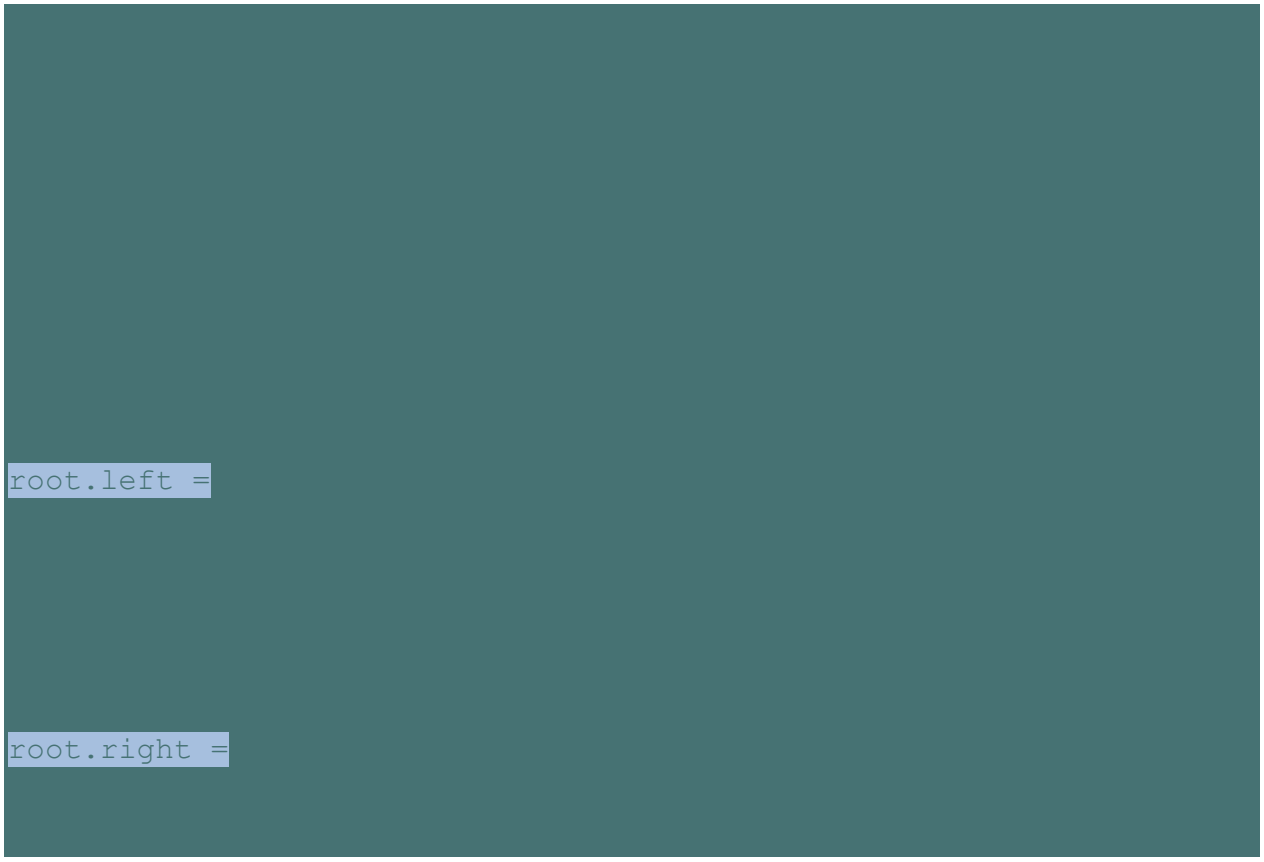
Then we change every call on to match the form. Recall our old public method:

The code now becomes:

```
overallRoot =
```

The idea is that we pass the value of [redacted] to the [redacted] method and it passes back the value of the parameter, which might be the old value or might be a new value. We reassign [redacted] to this value passed back by [redacted]. That way, if the method changes the value of the parameter, [redacted] gets updated to that new value. If it doesn't change the value of [redacted], then we are simply assigning a variable to the value that it already has (effectively saying [redacted]), which has no effect.

Two other calls on [redacted] inside the method itself need to be updated in a similar manner:



After we make these changes, the [redacted] method works properly.

This [redacted] idiom is very powerful and simplifies a lot of binary tree code, so it is important to learn how to use it well.

Searching the Tree

Once we have built a binary search tree, we will want to do more than just printing its contents or structure. One of the most common operations we might want to perform is to search the tree to see whether it contains a particular value.

The method to perform this search should return a [redacted] result and you will once again want to have a public/private pair of methods, so the basic form of your solution will be as follows:

Now you just have to write the body of the private method. An empty tree again serves as the easy case. If you're asked whether the empty tree contains a value, the answer is no ( ):

For the method, you had two other cases. You either added to the left subtree or added to the right subtree. For this method, there are three cases, cases for searching the left and right subtree and the root:

```
else if (value == root.data) {

...

} else if (value < root.data) {

...

} else { // value > root.data

...

}
```

Now you just need to fill in the three different cases. If the value is less than [redacted], then it is either in the left subtree or not in the tree at all. In other words, you want to search the left subtree to see whether it contains the value. You can accomplish this with a recursive call on [redacted]. You can make a similar recursive

call on [redacted] to search the right subtree when the value is greater than [redacted]. In the final case in which the value is equal to [redacted], you should simply report that you found the value. In other words, the answer is [redacted]. Putting these pieces together, you end up with the following code:

```
return true;


return contains(root.left, value);


return contains(root.right, value);
```

A clever reader might notice that this method violates the principle of Boolean Zen that we introduced in Chapter 5. This method can

be written more concisely as a Boolean expression. For example, it can be written as a single expression as follows:

In this case, it is debatable whether the single expression is more readable than the [redacted] version of the code. This is somewhat a matter of personal taste, so different programmers will prefer one version over the other, but our conclusion was that the longer version is actually easier to read in this case.

Here is the complete implementation of the [redacted] class:

```
public class

private

public

public void
```

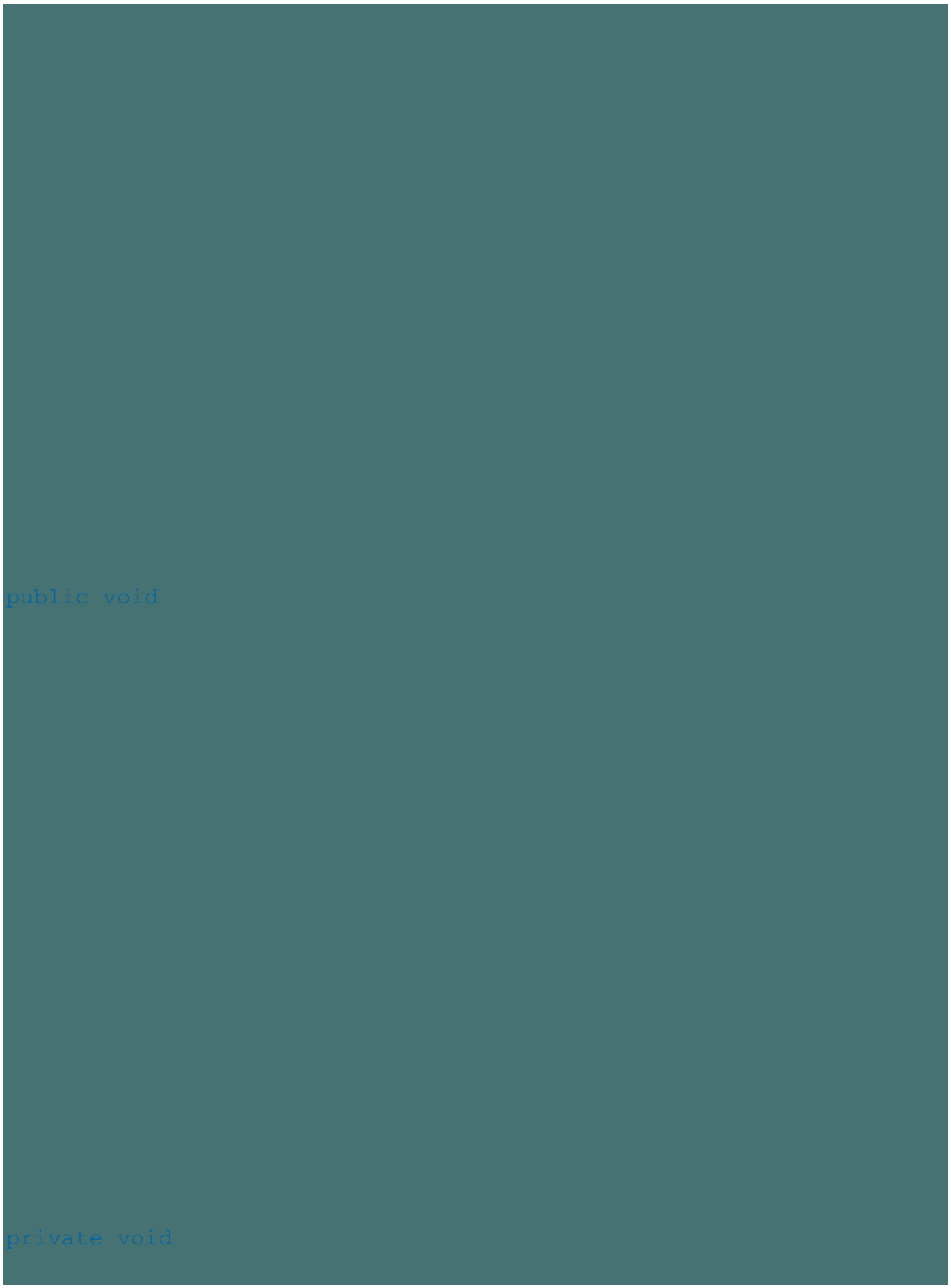
```
int

private

int

if

new

else if
```

```
else

return

public boolean

int

return
```

```
private boolean

if

return

else if

return

else if
```

```
return

else

return

public void
```

```
private void


if
```


public void
private void

```
if

for

int
```

Binary Search Tree Complexity

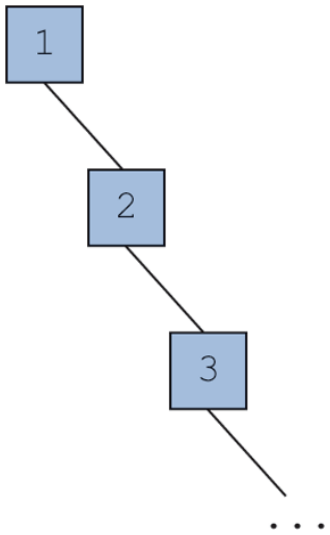
Each level of a binary search tree can hold twice as many nodes as the level above it. The one root node can have two children, four grandchildren, eight great-grandchildren, and so on. As a result, we can store a lot of values in a tree that has a relatively small number of levels.

The number of nodes and the number of levels have an important relationship. If you have k levels in a tree, then you can store approximately 2k values in the tree. Going in the other direction, if you want to include N nodes in a tree, you will need approximately log N levels for the tree. We normally use this second way of thinking about the relationship.

It may not seem immediately obvious how powerful this is because you may not realize how much smaller log N is than N. Consider some examples. To store a thousand values, you would need around 10 levels. Increase the number of values to a million, and you need 20 levels. Increase the number to a billion, and you need 30 levels. To store a trillion values, you need only 40 levels for the tree. As a result, binary search trees tend to be relatively short in height, even when they store vast amounts of data.

It is also important to realize that the insertion algorithm we have developed in this chapter does not guarantee that the tree will store data efficiently in the minimum number of levels. While it’s true that you can store a trillion values with just 40 levels, you might end up with considerably more levels than that. Consider, for example, what happens if the data are inserted in sorted order.

If you build a binary search tree by inserting the sequential numbers 1 through 1000, you end up with a tree that looks like this one:



We refer to this as a *degenerate tree*. It's technically still a tree, but it's not a very good tree because it makes no use of the two branching possibilities. This is really a linked list turned on its side. You can get similar degenerate trees if the values are inserted in decreasing order, which produces a tree that expands indefinitely in the left direction.

If the values are inserted into a binary search tree in a random order, then the odds of getting a degenerate tree are fairly low. If you take a more advanced course in data structures, you will study techniques for ensuring that the tree is balanced. For example, the [redacted] and [redacted] structures in the Java class libraries are implemented as what are called red/black trees to guarantee that the resulting binary search tree is balanced.

Let's set aside the issue of degenerate trees and think about the case for randomized data or for binary search trees that guarantee balance. One important property of the binary search tree is that each insertion into the tree requires you to descend

from level to level until you find an open spot to insert the value. Thus, insertion into a balanced binary search tree is an O(log N) operation. Similarly, finding a value in a balanced binary search tree involves descending from level to level, so it also is an O(log N) operation. To search for a value in a normal binary tree, you would have to search the entire tree. Although we haven't shown the solution here, we'll mention that removing a value from a balanced binary search tree can also be accomplished as an O(log N) operation.

The binary search tree, then, offers insertion, search, and removal all in O(log N) time. That means that if you are working with a trillion values, for example, you can add a new value, search for a value, or remove a value, all within around 40 steps. That makes the binary search tree a very efficient structure to use for these operations.

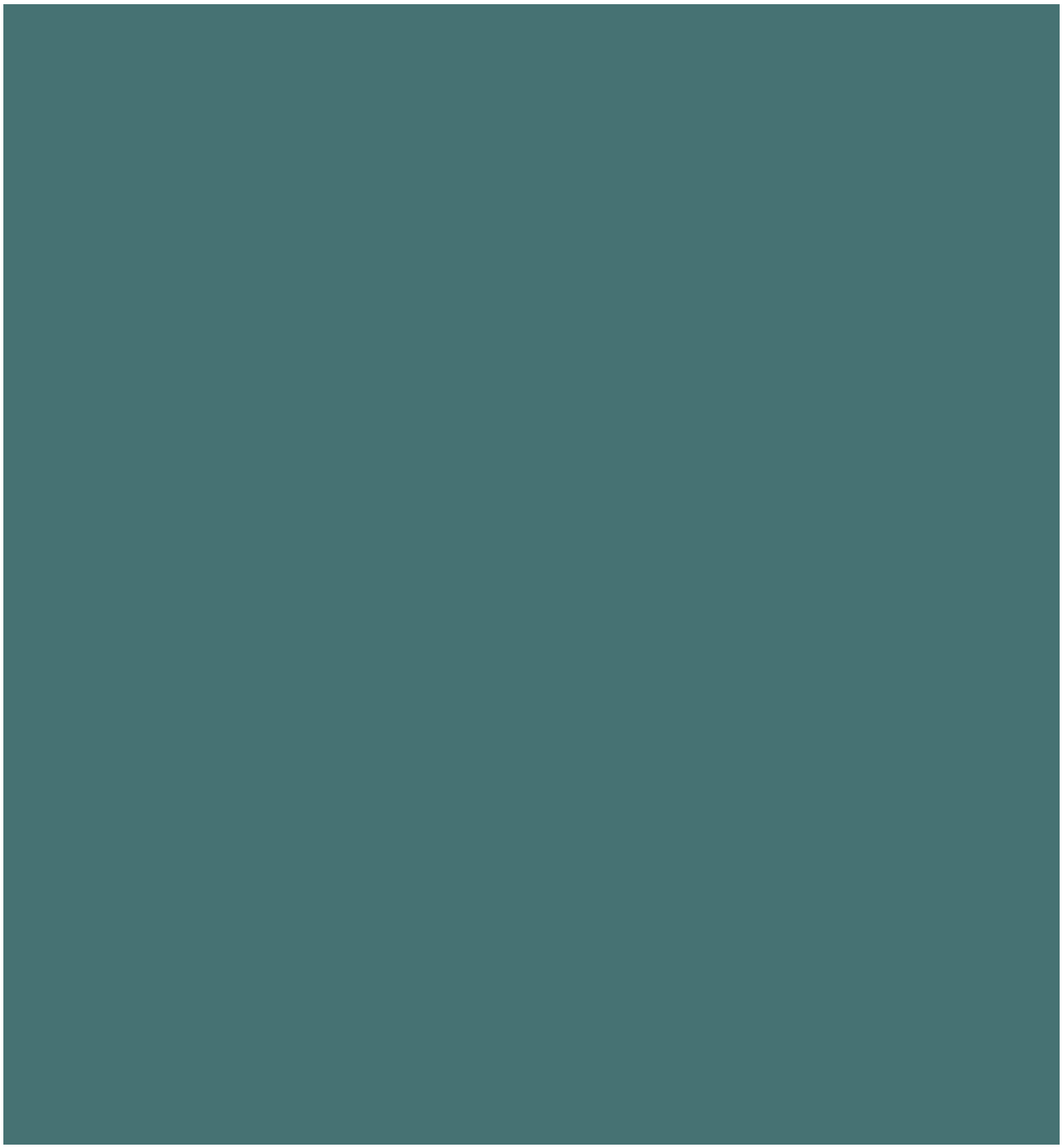
17.5 [redacted]

In the previous section, we developed code for a binary search tree of integers. You might want to build other kinds of binary search trees with different kinds of data and you wouldn't want to make many copies of essentially the same code (one for integers, one for strings, etc.). Instead, you want to write the code in a more generic way.

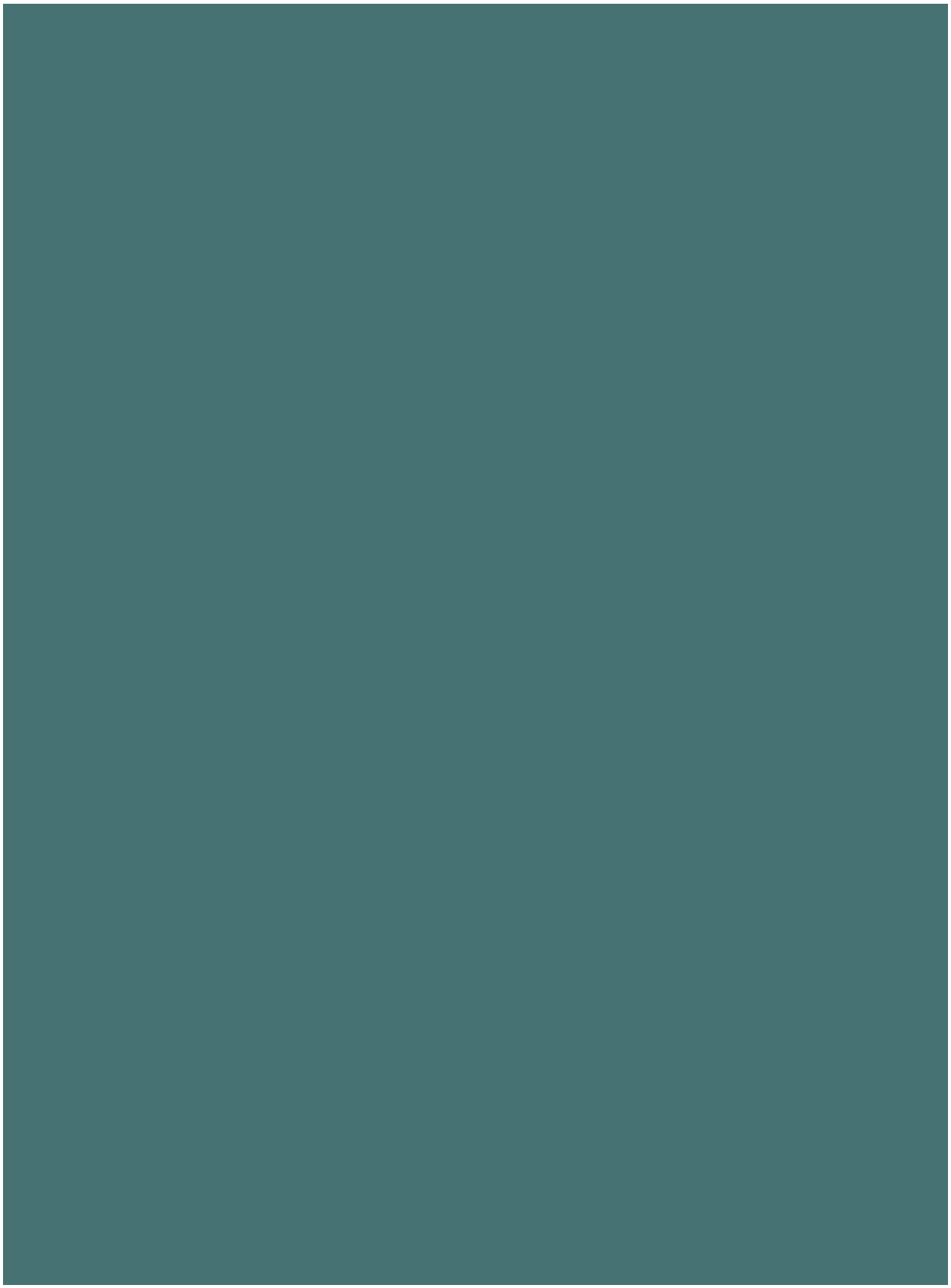
We want to be able to build a search tree for any class that implements the [redacted] interface. We want a class that we could call [redacted] (for some element type [redacted]). Here is a client program which constructs a [redacted] that puts words into alphabetical order and a [redacted] that puts numbers into numerical order.

```
public class

public static void

new

new

while
```

```
new

int

while
```



Here is a sample log of execution:



To generate the [redacted] class, we can start with the [redacted] class and convert it into generic form.

First we need a node class for the tree. Our node class is almost the same as the [redacted] class, but instead of having data of type [redacted], we have data of type [redacted], and instead of saying [redacted], we have to say [redacted] in most places other than the constructor headers:

```
public class

public

public

public

public

this
```

```
public

this

this

this
```

So what about the [redacted] class? You can get a pretty close approximation of the code for this class by replacing all occurrences of [redacted] with [redacted] and replacing all occurrences of [redacted] with [redacted]. But you have to make a few adjustments. First, the private [redacted] method looks like this after the substitution:

The problem here is that you have to call the [redacted] method rather than using the [redacted] operator. You'll replace the line of code:

with:

You have to make a similar change in the private [redacted] method. You have to use calls on the [redacted] method rather than simple comparisons. Because the method has more than one comparison, in this case it makes sense to call [redacted] once and store it in a local variable:

```
int compare = value.compareTo(root.data);


compare == 0
```

```
compare < 0
```

You have to make one final change. Java knows only that these data values are of some generic type . As a result, the calls on in your and methods generate compiler errors. You could fix this problem with a cast. For example, you could replace the line:

```

```

with:

```

```

A better approach is to modify the class header to include this information. You want to add the constraint that the class implements the interface. You specify that constraint by modifying the header as follows:

```
extends Comparable<E>
```

It's odd that Java has you use the keyword because you want it to implement the interface, but that's how generics work in Java. If you are defining a class, you make a distinction between the cases in which the new class extends another class and those in which it implements an interface. But in generic declarations, you use the word for both kinds of extension.

This is a fairly quick explanation of a complex topic. We don't have time to explain the details of programming with generics in detail in this book, although there are many excellent online tutorials that you can find by entering "Java generics" into your favorite web browser.

The complete code for the class can be found at http://www.buildingjavaprograms.com.

Chapter Summary

A binary tree is a linked structure in which each node refers to two other nodes. A tree is either empty ( ) or a node with references to two other trees, called its subtrees.

The root is the top node of a tree. A leaf node is a node that has no children. A branch node is a node that has at least one child. Each node has a level related to the number of links that are between it and the root of the tree.

A tree node object stores a data value and left/right references to other nodes. An overall tree object stores a reference to a single tree node as its overall root.

A traversal is an examination of all elements in a binary tree. Traversals are commonly done in three orders: preorder, inorder, and postorder. The difference between the orders relates to when a given node is examined relative to its left/right subtrees.

Tree methods are often recursive and are often implemented with the use of a public/private pair, in which the private method accepts a reference to a tree node as a parameter. This technique allows the method to recursively operate on a given portion of the overall tree.

The elements of a binary search tree are arranged in order, with smaller elements on the left and larger elements on the right. Search trees are useful for implementing collections that can be searched quickly, such as sets and maps.

To add a node to a binary tree, traverse the tree to the left if the new element is smaller than the current node or to the right if it is larger. The end of this traversal will reveal the proper place to add the new node.

To search for a value in a binary tree, traverse the tree going left if the target value is smaller than the current node or right if it is larger. Because of the way in which the tree is ordered, the end of this traversal will either find the target's node or find a [redacted] dead end, in which case the value is not in the tree.

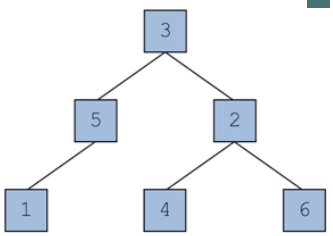
The [redacted] is a pattern in which a recursive method is passed an object or value in its initial state (called [redacted] here) and returns the new state of [redacted]. This technique is often used in recursive tree methods; the recursive call accepts a node parameter and later returns the potentially changed node.

A tree can be made to store elements of any object type [redacted] rather than just integers. To do this, the tree class must be made into a generic class, and the generic type parameter [redacted] must be constrained so that it is a class which implements the [redacted] interface.

Self-Check Problems

Section 17.1: Binary Tree Basics

1. How many roots can a tree have? Does a tree have more branches or leaves?
2. Draw a tree that has twice as many leaves as branches and one that has the same number of leaves as branches.
3.
    a. How many levels does the given tree have?
    b. How many branches does it have, and which nodes are they?
    c. How many leaves does it have, and which nodes are they?
    d. Which node is the root of the tree?
    e. Which node(s) are the sibling(s) of the node storing the value ? Which nodes are its children?

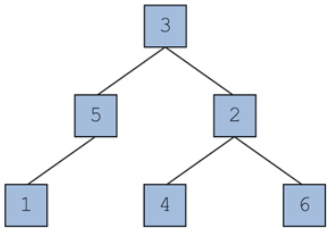


17.2-28 Full Alternative Text

Section 17.2: Tree Traversals

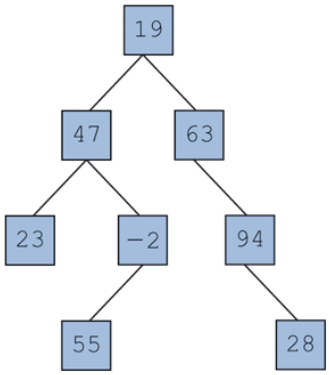
For each of the next three problems, write the elements of the given tree in the order in which they would be seen by a preorder, inorder, and postorder traversal.

4.



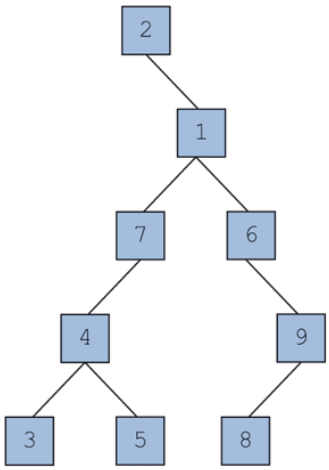
17.2-29 Full Alternative Text

5.



17.2-30 Full Alternative Text

6.



17.2-31 Full Alternative Text

7. What would happen if we removed the [redacted] test from the [redacted] method?
8. Write a method called [redacted] that could be added to the [redacted] class and that prints a postorder traversal of the tree.
9. Write a method called [redacted] that could be added to the [redacted] class and that prints a backward inorder traversal of the tree. That is, for a given node, it examines the right subtree, then the node itself, then the left subtree.

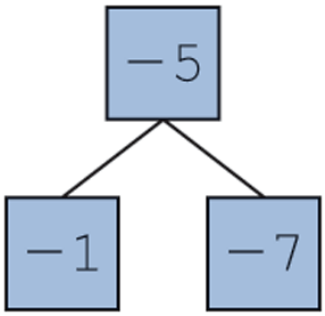
Section 17.3: Common Tree Operations

10. Why do many recursive tree methods use a public/private pair? What is the difference between the header of the public method and that of the private method?
11. Write a method called [redacted] that could be added to the [redacted] class and that returns the total number of nodes in the tree.
12. Write methods called [redacted] and [redacted] that could be added to the [redacted] class and that return the smallest and largest values in the tree, respectively. For example, if a variable called [redacted] stores the values shown in Self-Check Problem 5, the call of [redacted] should return [redacted] and the call of [redacted] should return [redacted]. If the tree is empty, the methods should throw an [redacted].
13. Write a method called [redacted] that could be added to the [redacted] class and that returns the total number of branch nodes in the tree. A branch node is any node that is not a leaf node. (Hint: Look at the code for [redacted] written in Section 17.3.)

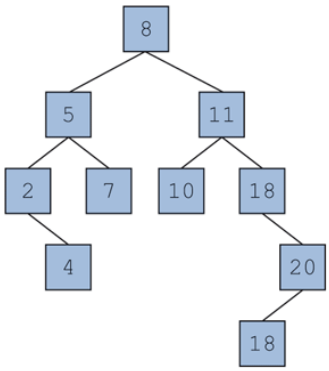
Section 17.4: Binary Search Trees

14. What is the difference between a regular binary tree and a binary search tree?
15. Which of the following trees are valid binary search trees?

a.



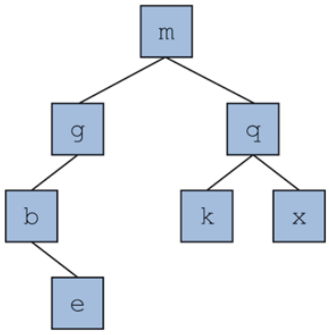
b.



17.2-33 Full Alternative Text

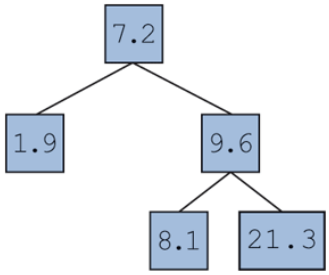
c.

d.



17.2-35 Full Alternative Text

e.



17.2-36 Full Alternative Text

16. What will be true about the results of an inorder traversal of a binary search tree?

For each of the next four problems, draw the binary search tree that would result if the given elements were added to an empty binary search tree in the given order. Then write the elements of the tree in the order that they would be visited by each kind of traversal (preorder, inorder, and postorder).

17. Leia, Boba, Darth, R2D2, Han, Luke, Chewy, Jabba
18. Meg, Stewie, Peter, Joe, Lois, Brian, Quagmire, Cleveland
19. Kirk, Spock, Scotty, McCoy, Chekov, Uhuru, Sulu, Khaaaan!
20. Lisa, Bart, Marge, Homer, Maggie, Flanders, Smithers, Milhouse
21. Why does the [redacted] method of a binary search tree need to return the newly added/created node?

22. What is the [redacted] pattern, and how is it used with binary trees?
23. How many nodes at most would be examined in a call to [redacted] on a perfect binary search tree of height N?
24. Consider the following implementation of the [redacted] method. How does it differ from the one we showed in Section 17.4? Is it better or worse, and why?

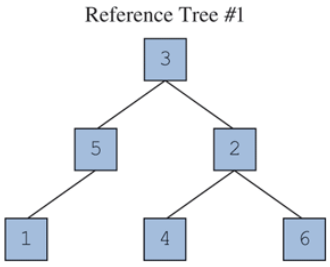
25. Rewrite the [redacted] and [redacted] methods from Self-Check Problem 12 so that they will work on a binary search tree. The methods should take advantage of the fact that the tree is sorted and should not examine nodes unless necessary.

Section 17.5: [redacted]

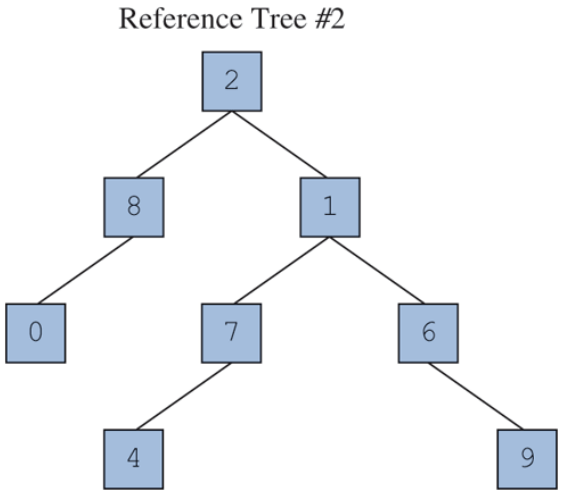
26. What are some changes that need to be made to the tree class to convert it from storing integers to storing objects of type [redacted]?
27. What kind of changes would you make to add an iterator to our binary tree class?

Exercises

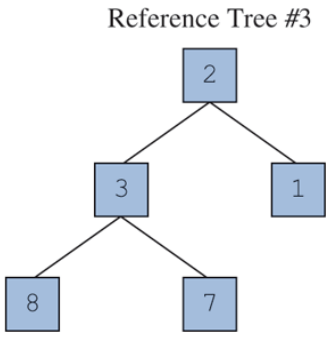
Each of the following exercises is a method to be added to the [redacted] class from this chapter. You may define additional private methods to implement your public method if necessary. Several problem descriptions refer to the following reference binary trees:



17.3-37 Full Alternative Text



17.3-38 Full Alternative Text



17.3-39 Full Alternative Text

1. Write a method called [redacted] that returns the number of left children in the tree. A left child is a node that appears as the root of the left-hand subtree of another node. For example, reference tree #1 has 3 left children (the nodes storing the values [redacted], and [redacted]).
2. Write a method called [redacted] that returns the number of empty branches in a tree. An empty tree is considered to have one empty branch (the tree itself). For nonempty trees, your methods should count the total number of empty branches among the nodes of the tree. A leaf node has two empty branches, a node with one nonempty child has one empty branch, and a node with two nonempty children has no empty branches. For example, reference tree #1 has 7 empty branches (two under the value [redacted], one under [redacted], and two under each of [redacted] and [redacted]).
3. Write a method called [redacted] that returns the sum of the values stored in a binary tree of integers weighted by the depth of each value. The method should return the value at the root, plus 2 times the values stored at the next level of the tree, plus 3 times the values stored at the next level of the tree, and so on. For example, the depth sum of reference tree #1 would be computed as [redacted].

4. Write a method called [redacted] that returns the number of branch nodes in a binary tree that contain even numbers. A branch node has one or two children (i.e., it is not a leaf). For example, if a variable [redacted] refers to reference tree #2, then the call [redacted] should return [redacted] because there are three branch nodes with even values ([redacted], [redacted], and [redacted]). Notice that leaf nodes with even values are not included (the nodes storing [redacted] and [redacted]).
5. Write a method called [redacted] that accepts an integer parameter [redacted] and prints the values at level [redacted] from left to right, one per line. We will use the convention that the overall root is at level [redacted], its children are at level [redacted], and so on. If there are no values at the level, your method should produce no output. Your method should throw an [redacted] if it is passed a value for a level that is less than 1. For example, if a variable [redacted] refers to reference tree #2, then the call of [redacted] would produce the following output:

[redacted]

6. Write a method called [redacted] that prints to [redacted] the leaves of a binary tree from right to left. More specifically, the leaves should be printed in the *reverse* order that they would be printed using any of the standard traversals. If the tree is empty, your method should produce the output [redacted]. For example, if a

variable [redacted] refers to reference tree #2, the call of [redacted] should produce the following output:

[redacted]

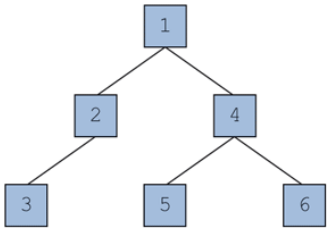
7. Write a method called [redacted] that returns [redacted] if a binary tree is full and [redacted] if it is not. A full binary tree is one in which every node has 0 or 2 children. For example, reference trees #1 and #2 are not full, but #3 is full. By definition, the empty tree is considered full.
8. Write a [redacted] method for a binary tree of integers. The method should return [redacted] for an empty tree. For a leaf node, it should return the data in the node as a string. For a branch node, it should return a parenthesized [redacted] that has three elements separated by commas: the data at the root, a string representation of the left subtree, and then a string representation of the right subtree. For example, if a variable [redacted] refers to reference tree #2, then the call [redacted] should return the following [redacted] (without the surrounding quotes):

[redacted]

9. Write a method called [redacted] that accepts another binary tree of integers as a parameter and compares the two trees to see whether they are equal to each other. For example, if variables of type [redacted] called [redacted] and [redacted]

have been initialized, then [redacted] will return [redacted] if the trees are equal and [redacted] otherwise. Two empty trees are considered to be equal to each other.

10. Write a method called [redacted] that doubles all data values greater than 0 in a binary tree of integers.
11. Write a method called [redacted] that changes the data stored in a binary tree, assigning sequential integers starting with [redacted] to each node so that a preorder traversal will produce the numbers in order ([redacted] etc.). For example, if a variable [redacted] refers to reference tree #1, the call of [redacted] would overwrite the existing data, assigning values from [redacted] to [redacted] to the nodes so that a preorder traversal of the tree would produce [redacted] as shown in the following diagram. Do not change the structure of the tree, only the values stored in the data fields. Your method should return the number of nodes in the tree.

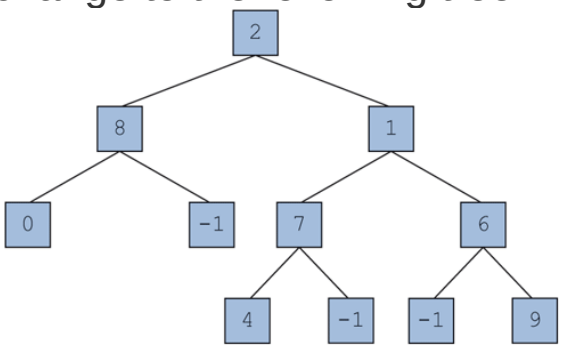


17.3-40 Full Alternative Text

12. Write a method called [redacted] that removes the leaves from a tree. A leaf is a node that has empty left and right subtrees. If your method is called on an empty tree, the method does not change the tree because there are no nodes of any kind (leaf or not).
13. Write a method called [redacted] that returns a reference to a new [redacted] that is an independent copy of the original tree. Do not change the original tree.
14. Write a method called [redacted] that accepts an integer n as a parameter and that adds nodes to a tree

to complete the first n levels. A level is complete if every possible node at that level is not [redacted]. We will use the convention that the overall root is at level [redacted], its children are at level [redacted], and so on. You should preserve any existing nodes in the tree. Any new nodes added to the tree should contain the value [redacted]. Your method should throw an [redacted] if it is passed a value for a level that is less than [redacted].

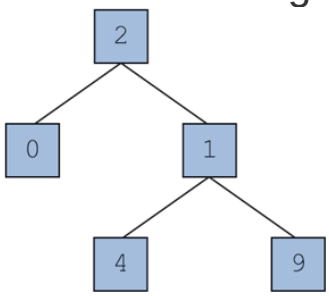
For example, if a variable called [redacted] refers to reference tree #2 and you make the call of [redacted], the tree should change to the following tree:



17.3-41 Full Alternative Text

15. Write a method called [redacted] that accepts minimum and maximum integers as parameters and removes from the tree any elements that are not within that range inclusive. For this method you should assume that your tree is a binary search tree and that its elements are in valid binary search tree order. Your method should maintain the binary search tree ordering property of the tree. This property is important for solving this problem.
16. Write a method called [redacted] that eliminates branch nodes that have only one child. Each such node should be replaced by its only child. (This can lead to multiple replacements because the child might itself be replaced.)

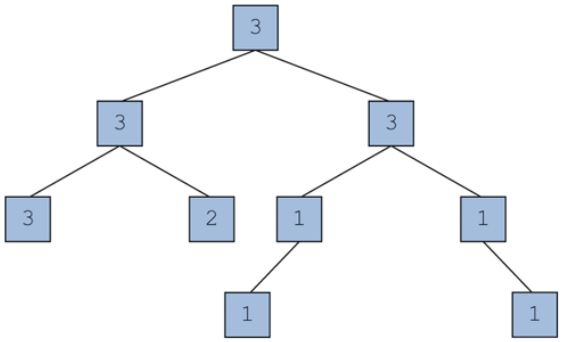
For example, if a variable called [redacted] refers to reference tree #2, the call of [redacted] should leave [redacted] storing the following tree.



17.3-42 Full Alternative Text

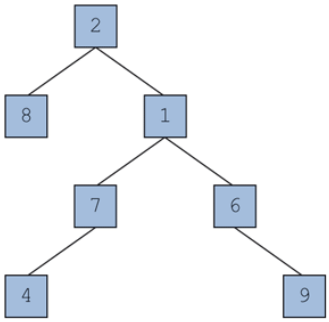
17. Write a method called [redacted] that accepts another binary tree of integers as a parameter and combines the two trees into a new third tree that is returned. The new tree's structure should be a union of the structures of the two original trees; it should have a node in any location where there was a node in either of the original trees (or both). The nodes of the new tree should store an integer indicating which of the original trees had a node at that position ([redacted] if just the first tree had the node, [redacted] if just the second tree had the node, and [redacted] if both trees had the node). Your method should not change the structure or contents of either of the two original trees that are being combined.

For example, suppose [redacted] variables [redacted] and [redacted] refer to reference trees #2 and #3, respectively. The call of [redacted] will return a reference to the following new tree. Keep in mind that nodes numbered [redacted] are those that appear only in [redacted], nodes numbered [redacted] appear only in [redacted], and nodes numbered [redacted] appear in both.



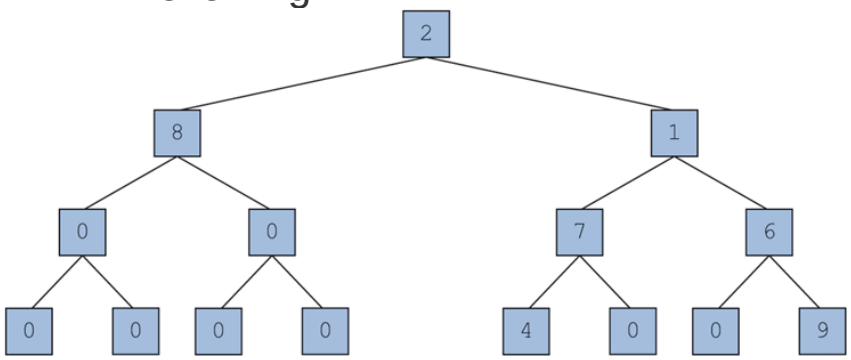
17.3-43 Full Alternative Text

18. Write a method called [redacted] that returns a list containing the sequence of values obtained from an inorder traversal of your binary tree of integers. For example, if a variable [redacted] refers to reference tree #3, then the call [redacted] should return the list [redacted]. If the tree is empty, your method should return an empty list.
19. Write a method called [redacted] that makes sure that all branches end on an even level. If a leaf node is on an odd level it should be removed from the tree. We will define the root as being on level [redacted]. For example, if a variable [redacted] refers to reference tree #2, then the call [redacted] should change the tree's state to the following, removing the leaf value [redacted] because it is on an odd level:



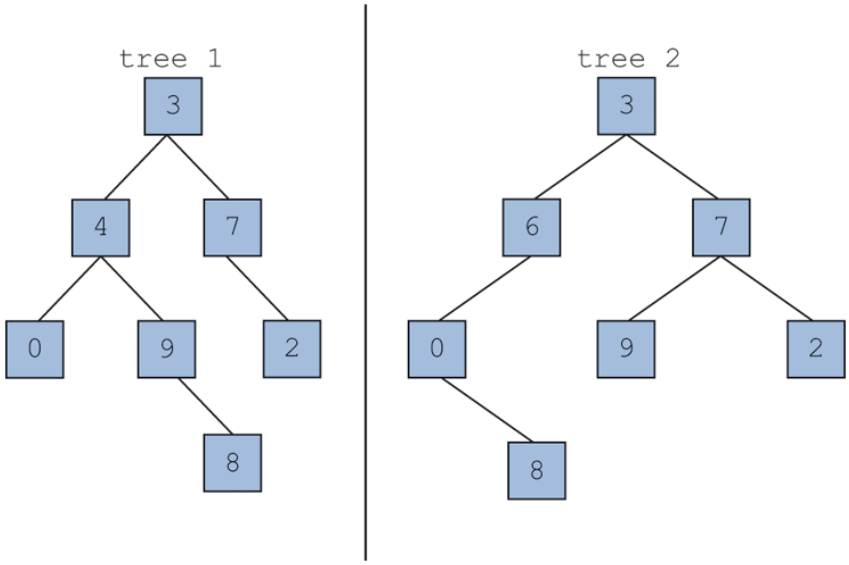
17.3-44 Full Alternative Text

20. Write a method called [redacted] that adds nodes until the binary tree is a *perfect* tree. A perfect binary tree is one where all leaves are at the same level. Another way of thinking of it is that you are adding dummy nodes to the tree until every path from the root to a leaf is the same length. A perfect tree's shape is exactly triangular and every branch node has exactly two children, and all of the leaves are at the same level. Each new node you add to the tree should store the value [redacted]. For example, if a variable [redacted] refers to reference tree #2, then the call [redacted] should change the tree's state to the following:



17.3-45 Full Alternative Text

21. Write a method called [redacted] that returns a count of the number of nodes in one tree that match nodes in another tree. A match is defined as a pair of nodes that are in the same position in the two trees relative to their overall root and that store the same data. Consider, for example, the following trees:



17.3-46 Full Alternative Text

The overall root of the two trees match (both are 3). The nodes at the top of the left subtrees of the overall root do not match (one is 4 and one is 6). The top of the right subtrees of the overall root match (both are 7). The next level of the tree has 2 matches for the nodes storing 0 and 2 (there are two nodes that each store 9 at this level, but they are in different positions relative to the overall root of the tree). The nodes at the lowest level both store 8, but they aren't a match because they are in different positions. Therefore, these two trees have a total of 4 matches. Therefore the calls of [redacted] and [redacted] would each return [redacted].

Programming Projects

1. Implement a yes/no guessing game called 20 Questions using a binary tree. At the beginning of each round of the game, the human player thinks of an object. The computer tries to guess the object by asking a series of no more than 20 yes-or-no questions. Eventually the computer guesses what the object is. If this guess is correct, the computer wins; if not, the human player wins.

The computer keeps track of a binary tree with nodes representing questions and answers. A “question” node contains a left “yes” subtree and a right “no” subtree. An “answer” node is a leaf. This tree can be traversed to ask the human player questions until it reaches a leaf node, at which point the computer asks whether that answer is the correct one.

Initially the computer is not very intelligent, but it grows more intelligent with each game. If the computer’s answer guess is incorrect, the player must give it a new question to help it in future games. This is similar to the game found at the web site http://animalgame.com/. You should write methods to construct a tree, play a game, and save/load the tree’s state from a file.

2. Write a program that encodes and decodes Morse code files using a binary tree. Morse code is a system that encodes the 26 English letters and 10 numeric digits into sequences of dots, dashes, and spaces. Your tree can store the encodings of each letter and number, going left for a dot and right for a dash. When your program reads a sequence of Morse code characters, it should traverse the tree in the appropriate direction for each character

and output the letter or number it reaches at the end of each sequence.

3. Write a program that performs Huffman encoding and compression using a binary tree. Huffman coding is an algorithm devised by David A. Huffman of MIT in 1952 for compressing text data to make a file occupy a smaller number of bytes. The idea of Huffman coding is to abandon the rigid 8-bits-per-character requirement of ASCII encoding and use different-length binary encodings for different characters. The advantage of doing this is that if a character occurs frequently in the file, such as the letter e, it could be given a shorter encoding (fewer bits), making the file smaller.

Your program should contain methods to read an input text file, count its characters, and use these counts to build a Huffman tree for the characters by frequency. Use this Huffman tree to output an encoded and compressed version of the file. Also write a method to read and decompress this file later. You may want to use helper classes for reading and writing a file one bit at a time. You can find classes called [redacted] and [redacted] at our web site, http://buildingjavaprograms.com.

4. The actual [redacted] interface in the [redacted] package has several methods that are beyond those implemented by our search tree in this chapter. Write a version of [redacted] that adds some or all of these methods. The methods to add are the following (some headers are slightly modified; see the Java API Specification for descriptions of each method):
   - [redacted]
   - [redacted]
   - [redacted]
   - [redacted]

- [redacted]
- [redacted]
- [redacted]
- [redacted]
- [redacted]

5. Add an iterator to the search tree. Write a class called [redacted] that has the methods that follow for iterating over a binary tree. You will also need to modify the tree nodes to store parent references so that the iterator can properly walk “up” the tree as necessary. (See the Java API Specification for descriptions of each method.) Here are the methods:
   - [redacted]
   - [redacted]
   - [redacted]
6. Write a program that evaluates numeric expressions using a binary expression tree. A leaf node represents an operand (a number). A branch node represents an operator; its two subtrees represent its operands.

Chapter 18 Advanced Data Structures

18.1 Hashing
• Array Set Implementations
• Hash Functions and Hash Tables
• Collisions
• Rehashing
• Hashing Non-Integer Data
• Hash Map Implementation
18.2 Priority Queues and Heaps
• Priority Queues
• Introduction to Heaps
• Removing from a Heap
• Adding to a Heap
• Array Heap Implementation
• Heap Sort

Introduction

In this chapter, we will explore the implementation of two powerful data structures called hash tables and heaps. Both of these structures are used in Java's Collections Framework to implement collections such as [redacted], [redacted], and [redacted]. These clever data structures allow for efficient operations

such as adding, removing, searching, and ordering elements.

Upon completion of this chapter you will have seen all of the major implementation strategies used in the primary collections in the Java Collections Framework: arrays ([redacted], [redacted]), linked lists ([redacted]), binary trees ([redacted], [redacted]), hash tables ([redacted], [redacted]), and heaps ([redacted]).

18.1 Hashing

Hashing is a technique for efficiently mapping data elements to indexes in an array so that they can be added, removed, and searched very quickly. Hashing is the underlying technique used to implement the [redacted] and [redacted] classes. Hashing makes it possible to have a collection that can add, remove, and search for data elements in a constant amount of time (also called O(1), as discussed in Chapter 13), making it extremely fast for many common use cases.

Array Set Implementations

Suppose we want to store a set of integers using an array as the underlying data structure and that the most common operations we want to enable are insertion ([redacted]), deletion ([redacted]), and testing for membership ([redacted]). Further suppose that the order in which the elements are stored does not matter, so long as we can implement all of these operations efficiently. The key concern is that the structure should be fast for these three operations.

We could follow the same general idea as [redacted], using an unfilled array and a [redacted] field to represent the data in the set:

[redacted]

As each element is inserted, it is stored at the next available index and the [redacted] field is incremented. So if the array's length is 10 and the client adds the values [redacted], [redacted], [redacted], and [redacted], the set would have the internal state shown in Figure 18.1. The blank array cells actually contain 0s, but they are unimportant because they are past the size.

Figure 18.1 Representing a set as an unfilled, unsorted array

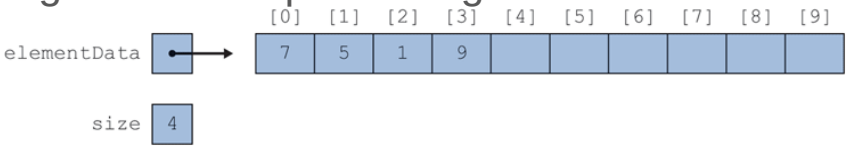


Figure 18.1 Full Alternative Text

Is this a good way to implement a set? Think about how the common operations would be written. Adding an element is efficient (O(1)); simply place it in the next free slot and increment

the [redacted] field. Removal is an inefficient linear search through the array for the value to remove. Once the value is found, since order is unimportant, you can quickly remove it by replacing it by the last element in the array, rather than having to shift any following elements left by one index; but the search to find it is still slow. We might have to examine all of the elements (on average, about half of them), so removal is O(N). Testing for membership is also slow, a linear search over all elements to try to find the value of interest. To figure out whether the set contains the value [redacted], we must look at the [redacted], [redacted], [redacted], and [redacted] before it is found.

Removal and searching from the unsorted array are slow because there's no particular way to guess where the element value will be found, if it is in the array at all. But with some creativity we can improve upon this situation. We've seen some strategies that take advantage of sorted data, such as the binary search algorithm. Binary search does not need to examine every index of the array to find an element; at each step the algorithm eliminates half of the remaining search space, resulting in a O(log N) runtime.

We could base our set on a sorted array, as shown in Figure 18.2. As elements are added to this sorted array, they must be placed into the proper indexes to maintain sorted order. The implementation of insertion or removal requires two steps: binary searching for the right index for the element, then shifting any following elements left or right. For example, if the value [redacted] is added, the [redacted], [redacted], and [redacted] must be shifted right by one index to make room for it.

Figure 18.2 Representing a set as an unfilled, sorted array

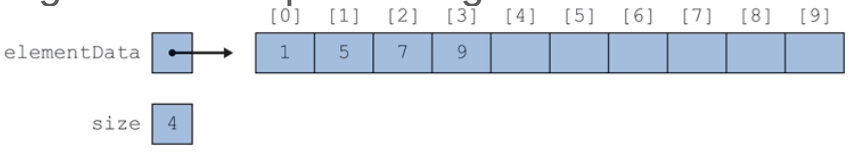


Figure 18.2 Full Alternative Text

The sorted array structure would be faster to search because we could implement our [redacted] method as a binary search. Adding and removing would be O(N) in the worst case because of the shifting. Still, this is arguably a better implementation than the unsorted array if the primary concern is the speed of the [redacted] operation.

Hash Functions and Hash Tables

Implementing a set using a sorted array is a step up from an unsorted one, but it's possible to do better. Let's explore how to use a special mapping strategy called a hash function to organize the elements of our set and search it quickly.

Recall that the order of the elements doesn't matter, as long as the set can add, remove, and find elements. The elements can be placed anywhere, as long as they can be found later. Here's an odd but powerful idea: What if we stored element value k at index k? For example, if you tell the set to add the value [redacted], store it at index 5. If we used this technique, the set storing [redacted], [redacted], [redacted], and [redacted] would have the structure shown in Figure 18.3.

Figure 18.3 Storing set elements at corresponding indexes

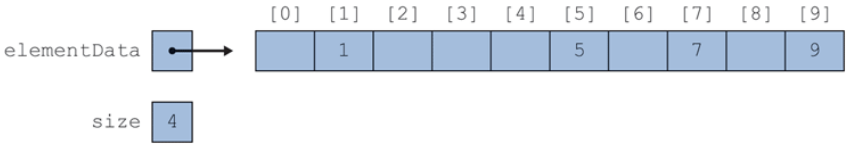


Figure 18.3 Full Alternative Text

If we stored our set data using this technique, the three basic set operations become extremely efficient to implement. Inserting a value k simply involves going to index k and storing k there; no searching or shifting is needed. Removing the value k requires going to index k and changing the value back to [redacted]. Testing for membership of a value k ([redacted]) requires looking at index k to see whether k is stored there; if so, the value is part of the set, and if not, it isn't.

You may already be thinking of some problems with this implementation strategy. What if the client tries to add a value that is outside the range 0 to 9, such as [redacted]? We could start with a larger array; if [redacted] has length 100, our [redacted] will fit nicely. But no matter how large the array becomes, the client can always add a larger integer outside its bounds. Negative numbers also pose a problem because arrays never have negative indexes.

To get around these issues, let's patch our storage technique. Instead of always storing element value k at index k, we'll limit k's value by modding it by the array capacity. So if the array length is 10, the value [redacted] would be inserted at index 3 because [redacted] equals [redacted]. To fix negative numbers, we'll take the absolute value of k and apply the same technique. So the value [redacted] would be inserted at index 8.

Figure 18.4 shows what the array would look like after the insertion of [redacted] and [redacted] using our new mod technique. The set still has the same speedy runtime for the common operations

because each element still has a single index into which it will be placed.

Figure 18.4 Use of hash function with wrap-around

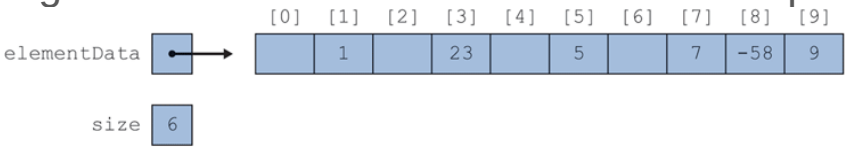


Figure 18.4 Full Alternative Text

Since our mapping from element value to preferred index now has a bit of complexity to it, we might turn it into a method that accepts the element value as a parameter and returns the right index for that value. Such a method is referred to as a *hash function*, and an array that uses such a function to govern insertion and deletion of its elements is called a *hash table*. The individual indexes in the hash table are also sometimes informally called *buckets*.

Our hash function so far is the following:

Hash Function

A method for rapidly mapping between element values and preferred array indexes at which to store those values.

Hash Table

An array that stores its elements in indexes produced by a hash function.

Collisions

There is still a problem with our current hash table. Because our hash function wraps values to fit in the array bounds, it is now possible that two values could have the same preferred index. For example, if we try to insert into the hash table, it maps to index 5, conflicting with the existing value . This is called a *collision*. Our implementation is incomplete until we have a way of dealing with collisions. If the client tells the set to insert , the value must be added to the set somewhere; it's up to us to decide where to put it.

Collision

When two or more element values in a hash table produce the same result from its hash function, indicating that they both prefer to be stored in the same index of the table.

One common way of resolving collisions is called *probing*, which involves looking for another index to use if the preferred index is taken. For example, if the client wants to add [redacted] and index 5 is in use, we could just put [redacted] at the next available index, which is 6 in our example. (Looking forward one index at a time for the next free index is called *linear probing*, but there are other kinds of probing such as *quadratic probing*, which involves jumping around to various places in the hash table.)

Figure 18.5 shows the hash table's state if the values [redacted], [redacted], and [redacted] are added and linear probing is used to resolve the collisions. The [redacted] conflicts with [redacted] and is put into index 6. The [redacted] conflicts with existing value [redacted] and is put into index 2. The [redacted] conflicts with [redacted] and [redacted] and must be put into index 3.

Figure 18.5 Hash collisions resolved by linear probing

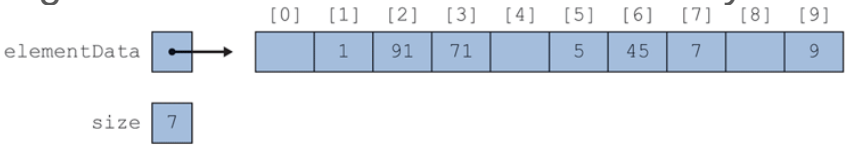


Figure 18.5 Full Alternative Text

Probing gets around the collision problem, but it introduces new problems of its own. For one, it is no longer as simple to find an element's value. A value whose hash function evaluates to k might be stored at index k, but if something other than k is there, it might be at index k+1, k+2, etc. So we would need to patch our other methods such as [redacted] accordingly. We can make the appropriate modifications, but we also have to think about the original goals of this implementation. Searching for elements is

supposed to be fast, and if we have to probe through a lot of elements to find anything, we're losing the efficiency we sought after in the first place.

Probing

Resolving hash collisions by placing elements at other indexes in the table rather than their preferred indexes.

Another problem is that the hash table can get full, resulting in no free slots to store a value. The example array of size 10 can hold only 10 elements. If the client tries to add an eleventh element, the array must be resized. Resizing a hash table is a nontrivial operation that we'll discuss in the next section.

Even if the array is not entirely full, if many elements are next to each other it can slow down the runtime for later operations. For example, to add the value [redacted] to the hash table in Figure 18.5 requires looking at four buckets: indexes 5, 6, 7, and 8 (where the value is finally placed). After doing so, a search for the value [redacted] (which is not found in the table) would need to examine indexes 5, 6, 7, 8, 9, and 0 before finally giving up. When elements clump up near each other like this, it is called *clustering*, and it is desirable to have as little clustering as possible in a hash table for it to perform efficiently.

Another way of dealing with the collision problem is to change our internal data structure. Collisions would not be a problem if each array index could store more than one value. This is possible if we change the array to be an array of lists rather than an array of integers. The list at index k will store all elements that the hash

function maps to k. Figure 18.6 demonstrates this idea. To add an element to the hash table, we go to its preferred index and add the element value to the list stored at that index. Resolving collisions by storing hash elements in lists is called *separate chaining*. The real [redacted] and [redacted] provided in [redacted] use separate chaining internally to resolve collisions.

Figure 18.6 Hash collisions resolved by separate chaining

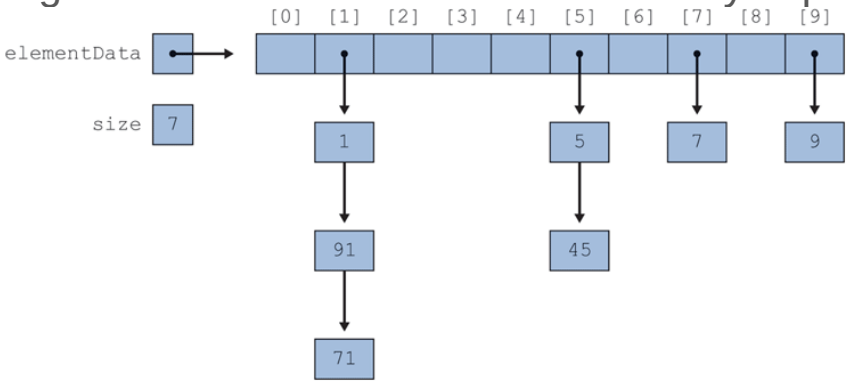


Figure 18.6 Full Alternative Text

Separate Chaining

[redacted]

Resolving hash collisions by having each index of the table store a list of values rather than a single value.

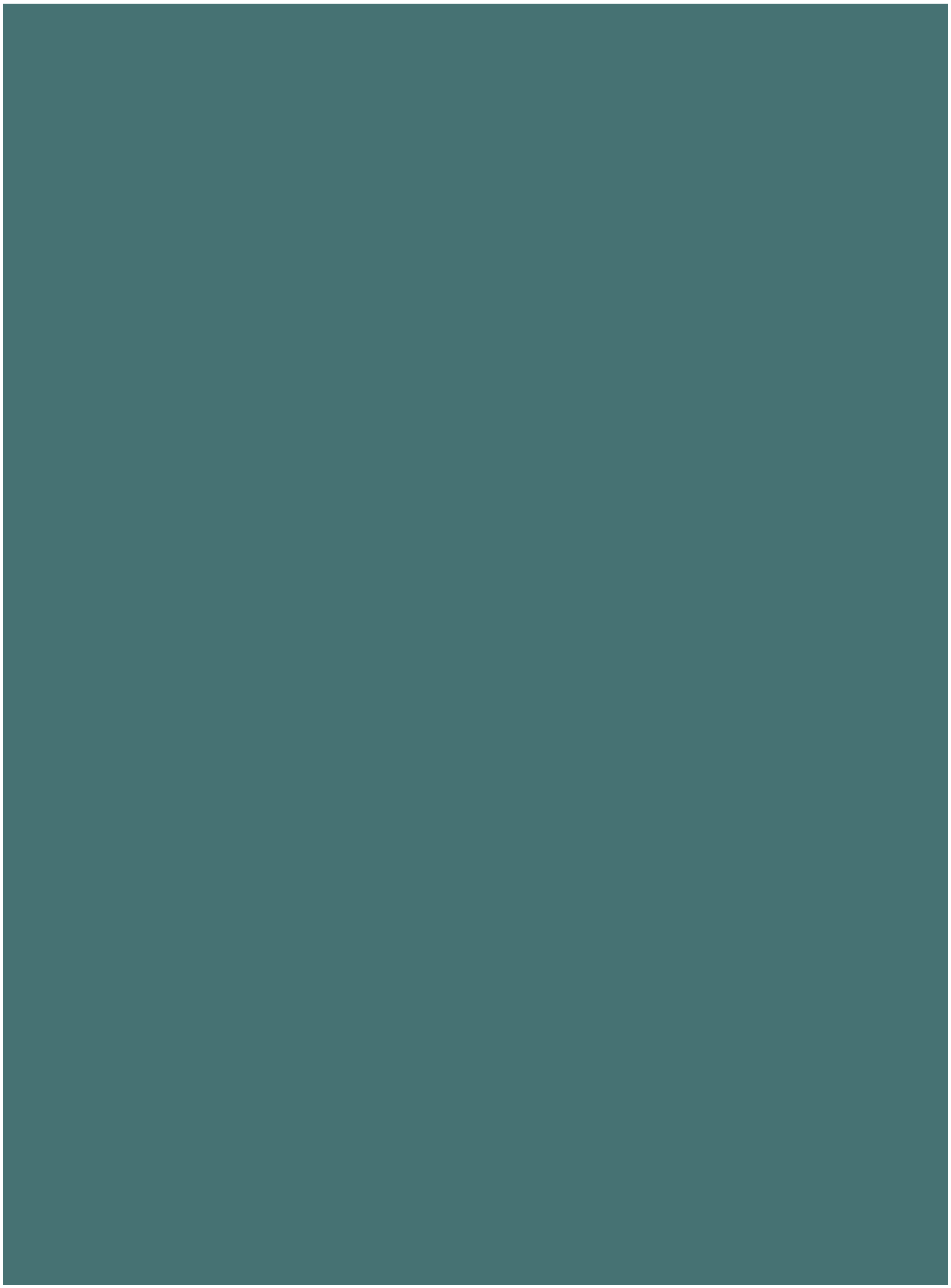
As with probing, separate chaining faces an efficiency issue if the size of the set becomes large enough, especially if the numbers happen to have a lot of collisions with each other. For example, if the client adds a lot of numbers to the set that end with 1, there will be a long list of elements at index 1 in the hash table, and we'll have to perform a long search through the list to find values. One way to mitigate this issue is to use a larger array to store the

element data, which will result in fewer collisions and shorter lists at each index. Another good idea is to use an array size that is not a multiple of 10 or any other round number, to avoid number patterns that are likely to collide. Many hash table implementations use a prime number for the table capacity.

We'll implement the lists of elements as lists of node objects, similar to the implementation of linked lists in Chapter 16. An inner class [redacted] represents a single node in such a list:

Using this inner class we can now implement the [redacted] and [redacted] operations on our set. The [redacted] method adds a new entry to the table if the element is not already found in the table. Before adding an element to the set, we make sure it is not a duplicate by calling [redacted]. The [redacted] method looks through the list at the appropriate hash bucket index to see if that value is found in the list. When elements are added, we insert them at the front of the appropriate linked list. This is faster than traversing the links all the way to the end of the list to insert the element.

We can also implement the [redacted] method now. Removal is a bit trickier than adding, because we have to make sure to handle all of the possible cases. If the element to be removed is at the front of its linked list, we must adjust the front reference; otherwise we must change the [redacted] reference of some existing node in the list. As always with linked lists, we must be careful to check for [redacted] and not try to traverse past the end of a list.



Removing from a hash table that uses probing has different complications. If we start with the hash table from Figure 18.5 and decide to remove the value , we can't just replace it with a blank value, because there is a chain of other elements (and) that probed to the following indexes 2 and 3. If a later call is made on the set, the code might mistakenly think that bucket 1 is empty and therefore incorrectly decide that is not contained in the set. What is often done instead is to replace the removed value with a special marker value to indicate that a removal occurred. That way when or is called later, it can recognize that value and realize that other values might follow the removed bucket. Flagging buckets with this special marker is more efficient than trying to shift values back in the hash table. Figure 18.7 shows an example removal of the value .

Figure 18.7 Removal of an element when probing is used

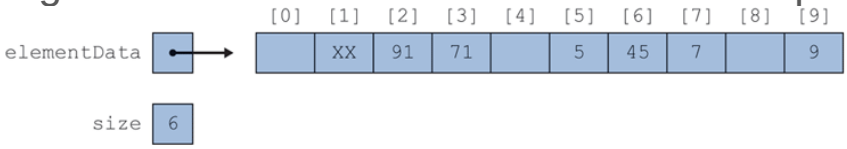


Figure 18.7 Full Alternative Text

We will focus on our implementation that uses separate chaining rather than showing the code for the probing version. You can find both versions on our web site at http://buildingjavaprograms.com.

There are other ways of resolving collisions. For example, if the hash function results in a conflict, a secondary hash function can be used to find a new hash bucket. This is sometimes called *double hashing*. Double hashing and other methods of collision resolution are outside the scope of this chapter.

Rehashing

When a hash table becomes too full, it can be enlarged to give it additional capacity for storing elements. Consider the full hash table in Figure 18.8 that uses linear probing to resolve collisions. In addition to the elements previously added, the client has inserted [redacted], [redacted], and [redacted], causing a completely full hash table. When the client tries to add an eleventh value, the hash table must be enlarged to make room for it.

Figure 18.8 Full hash table with linear probing

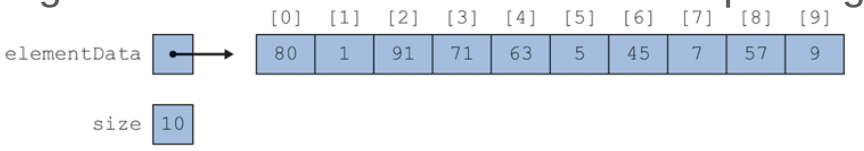


Figure 18.8 Full Alternative Text

Resizing a hash table is not as simple as creating a larger array and copying each element over into it. The reason is because the element values may not have the same hash function result in the new larger array. For example, the value [redacted] maps to index 1 in the array of length 10 because [redacted] equals [redacted]. But the same value [redacted] maps to index 11 in an array of length 20 because [redacted] equals 11. Therefore, when resizing a hash table array, it is important to reprocess each element using the hash function and

place it into the new appropriate index. This process is called *rehashing*.

Suppose the client tries to add the value [redacted] to the full hash table. Figure 18.9 shows a properly resized and rehashed table. Notice that [redacted], [redacted], [redacted], and [redacted] now sit in the second half of the array because their hash function results fall in that range. The value [redacted] also gets to sit in its preferred index of 3 because the element [redacted] that previously collided with it has moved to index 12. The rehashing also helps to decrease clustering; now the longest chain of consecutively occupied buckets is three.

Figure 18.9 Enlarged hash table after rehashing

Figure 18.9 Full Alternative Text

Rehashing

[redacted]

Resizing a hash table to increase its capacity and enabling it to store more elements, or store them more efficiently.

Hash tables that use separate chaining can also be rehashed, but the issues are slightly different. Technically a chained hash table never becomes completely “full” in the sense of becoming unable to add any more elements, because a new element can always be added to the list at a given index. But if the lists at each index become too long, the set operations become too slow. So it is still important to enlarge the hash table as its size grows. Figure 18.10 shows a chained table with the same elements as in Figure 18.8.

Figure 18.10 “Full” hash table with separate chaining

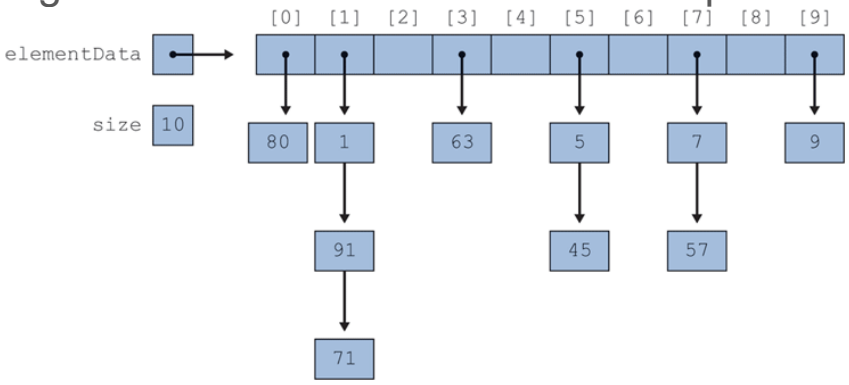


Figure 18.10 Full Alternative Text

Figure 18.11 shows the same hash table after a rehashing to length 20 and the insertion of value [redacted]. The values [redacted], [redacted], [redacted], and [redacted] are now stored in lists in the second half of the table.

Figure 18.11 Enlarged chained hash table after rehashing

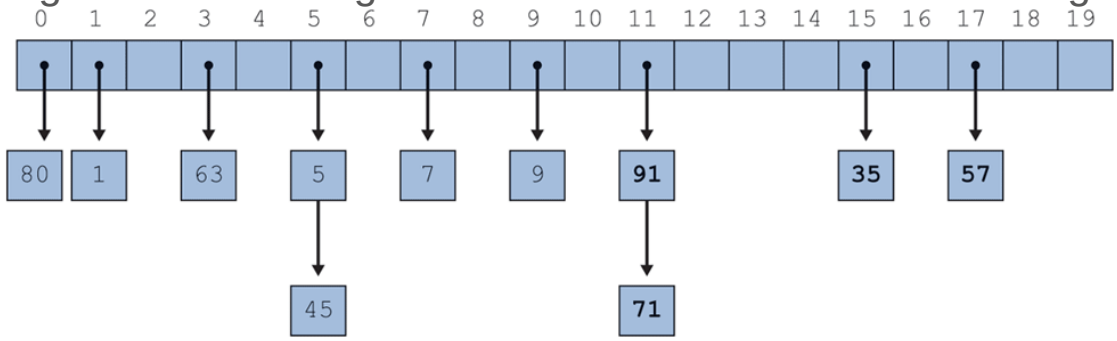


Figure 18.11 Full Alternative Text

In our examples we chose to resize the hash table once its size exceeded its capacity. But most hash tables do not wait so long to rehash, because a nearly or entirely full table starts to exhibit poor performance. A better alternative is to monitor the table’s size relative to the array capacity, also called the *load factor* of the table, and resize once this ratio exceeds some threshold. The load factor is fairly easy to compute:

The real Java [redacted] uses a default load factor of 0.75; in other words, once the number of elements is three fourths as large as the capacity, the table is enlarged. We can incorporate this logic into our own hash set with a constant and a check at the start of the [redacted] method:

```
rehash();


}
```

The simplest way to implement the actual rehashing operation is to replace the set's internal hash table with a new larger one, then loop over the old table and read all of its contents back into the new hash table. We can call the set's own method to reinsert all of the data to avoid duplicating code:

```
elementData = new HashEntry[2 * oldElementData.length];
```

```
add(current.data);
```

Hashing Non-Integer Data

Our hash set works well for storing integer data, but we have not yet discussed how to apply hashing techniques to non-integer data. The key idea is still the same: We need a way of mapping from values to preferred indexes at which to store them. It is possible to come up with hash functions for other kinds of data that convert them into integers. For example, we could convert a string into an integer by adding the integer ASCII values of its characters in some way.

The idea of hashing, and the importance of all kinds of data being able to be stored in a hash table, was close to the hearts of the original designers of Java. Recall that every Java class has an ancestor named [redacted] that contains various basic behaviors that are common to every object. One of the methods of the [redacted] class is named [redacted]. You can call [redacted] on any object to convert the object into an integer for use in a hash table. A

modified version of our hash function that works for any type of objects is the following:

```
Object


value.hashCode()
```

According to the official Java documentation for the [redacted] method, a proper hash function must be consistent. That is, when called multiple times on an object whose state is not changing, it should return the same value each time. Similarly, when [redacted] is called on two objects with equivalent states (such that [redacted] would return [redacted] for the two objects), it should return the same value.

In general it is also desirable for [redacted] to return unequal values when two objects have different states. But it is not mandatory for unequal objects to always have different hash codes. In fact, it is impossible to guarantee such a thing; for example, there are only four billion unique integer values, but there are infinitely many unique strings, so some strings must naturally have the same hash code as others. Unequal objects having equal hash codes does not break the hash table, but it does mean that those objects will hash to the same index and will

collide, so they will be placed into the same list if separate chaining is used on the hash table.

The default version of [redacted] from the [redacted] class computes an integer based on the memory address of the object. This function satisfies the consistency requirement, but it is not ideal because it does not take into account the state of each object. For this reason many classes override [redacted] with their own versions that compute an integer by somehow combining the object's state. Good hash functions try to do their best about spreading the hash values over a wide range of numbers to minimize conflicts.

When writing your own class, if your class has an [redacted] method, you are expected to also provide a [redacted] method to override the one provided by the [redacted] superclass. The reason is that objects with "equal" state are supposed to have equal hash codes, and this would not be guaranteed by the version of [redacted] inherited from class [redacted]. Your [redacted] method should combine the object's state in some way into a single integer and return it. The state can be combined in any way as long as it consistently produces the same integer for the same state.

The following is a possible [redacted] method for the [redacted] class implemented in Chapter 8. The code adds the [redacted] and [redacted] values but scales the [redacted] value up by a large number to spread out the space of integers returned. This is to avoid collisions between points like (4, 7) and (7, 4). Simply adding [redacted] and returning the result would also be correct but would not distribute the integer codes as much.

Many classes in the Java class libraries have their own [redacted] methods. For example, the [redacted] class code has a [redacted] method whose code is roughly the following. The code seems odd in that it multiplies by 31 at each pass through the loop. If the characters were simply added, there would be conflicts between various kinds of strings. For example, strings of similar length would all be clustered within a small range of hash codes, and strings that are anagrams, such as “file” and “life”, would always collide. Multiplying the result by some number at each pass spreads out the space of codes and reduces these potential collisions.

If you are writing a [redacted] method for a class that has fields that are not integers, it is less clear how to combine the fields' state into a single hash code integer. Remember that if the fields are objects, they have their own [redacted] methods that you can call. And primitives like [redacted] can be converted into their object wrapper equivalents, such as [redacted], and you can call [redacted] on those. Here is an example class [redacted] with non-integer fields and a possible [redacted] method based on its state. Notice that we again multiply the various fields by large integer constants to spread out the results of the function.

If we want our set to be able to store any kind of object as its data, we can convert our [redacted] into a generic [redacted], giving it a type parameter [redacted] for its elements. This requires small changes throughout the code, such as changing methods to accept parameters of type [redacted] rather than [redacted], and comparing values using [redacted] rather than [redacted]. The process is very similar to our conversions of [redacted] and [redacted] to generic collections in previous chapters. The complete generic version of the class is not shown, but a few of the relevant lines to change are found in the following abbreviated code:

```
E

.equals(

)

E

E
```

Hash Map Implementation

The ideas shown so far form the core of the implementation of the class. The has a very similar internal structure. Recall that a map stores key/value pairs rather than simple element values. A is still implemented using a hash table, but the hash entry objects in each linked list are changed to store a pair of data fields, representing one key/value pair.

```
private Object key;

private Object value;
```

Suppose a map is created with integers as the type for its keys and values, and the following pairs are added to it:

Figure 18.12 shows the internal state of the hash table containing these pairs. Each index stores a list of objects.

Figure 18.12 Hash map

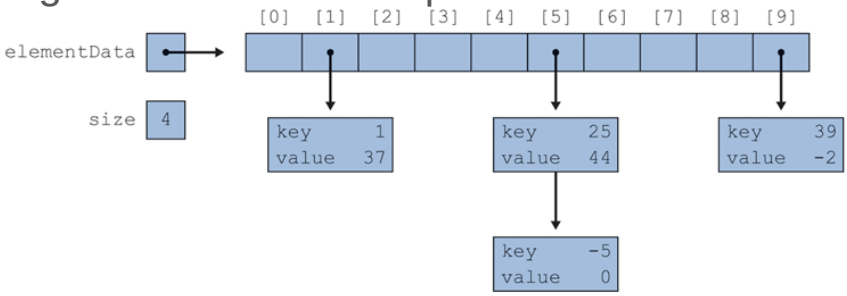


Figure 18.12 Full Alternative Text

To be able to accept any type of data for the keys and values, most hash map implementations are generic classes that accept a pair of type parameters, for the key type and for the value type. The hash map therefore has the following class header:

```
<K, V>
```

```
private K key;

private V value;
```

You can find complete code for the [redacted] and [redacted] classes on our web page at http://buildingjavaprograms.com.

18.2 Priority Queues and Heaps

In Chapter 14, you saw how queues are useful for storing data in First-In, First-Out (FIFO) order. There are other situations in which a queue-like structure is needed but with an ordering other than FIFO. In this section we will discuss a collection called a priority queue that is useful in these situations. We will also explore an underlying data structure called a heap that can be used to efficiently implement a priority queue.

Priority Queues

Consider the task of simulating the waiting line of patients at an emergency room. In general it seems that the patients should be served in the order in which they arrived. But some patients require urgent care and should be able to step in front of others in line. The primary consideration in the line's ordering is the importance of treating that particular patient.

A similar situation is the task of processing the set of customers waiting to purchase their goods at a food market. Again the customers naturally want to gather into lines, but real markets have found that they can serve more customers if they have "express lines" and other features to allow some customers to go ahead of others; in this case, customers with fewer items can skip ahead.

The general principle in situations like these is that of a queue, but organized by urgency or priority of each element rather than strictly on the order of insertion. There is a collection called a *priority queue* that is well suited to tasks like these. You can add elements to a priority queue in any order, but when you remove them, they will come out in order of priority or importance.

Priority Queue

A collection that associates each added element with a priority or importance and allows access or removal of the elements in order of priority.

Java's Collections Framework in includes a class called that arranges elements into their natural ordering from smallest to largest. For primitive values like integers and real numbers this means arranging the elements in ascending order.

The following example briefly illustrates the usage of a priority queue. Seven integers are added to the queue in no particular order. When the integers are removed from the queue, they come out in ascending order.

For objects, the priority queue arranges the elements by the ordering of their method. If the elements of the queue are not (or if you would like to arrange them in an order other than their natural ordering), you can pass a to the queue's constructor to indicate the relative ordering to use. The most commonly desired reordering is to reverse the queue so that the elements come out in descending rather than ascending order. This can be done easily by calling , a method that returns a that reverses the natural ordering of any comparable class:

The useful methods of [redacted] objects are listed in Table 18.1.

Table 18.1 [redacted] Methods

[redacted]	constructs a new empty priority queue
[redacted]	constructs a new empty priority queue, using the given [redacted] to order its elements
[redacted]	adds the given element to the queue
[redacted]	removes all elements from the queue
[redacted]	returns [redacted] if there are no elements in the queue
[redacted]	returns an [redacted] over the queue's elements
[redacted]	returns the minimum element in the queue
[redacted]	removes and returns the minimum element in the queue; throws a [redacted] if queue is empty

[redacted]	returns number of elements in the queue
[redacted]	returns an array containing the queue's elements
[redacted]	string representation of queue, such as [redacted]

A priority queue is a bit like a sorted list. You could achieve similar behavior to a priority queue by storing your data in an [redacted], calling [redacted] after adding each element, and always removing the element from index 0 every time. One key difference with a priority queue compared to a sorted list is that it manages the ordering automatically without the need for the client to sort the elements manually. Another big advantage is that the priority queue implements this behavior in a much more efficient way. Sorting a list is a slow operation that requires O(N log N) time, and removing the first element from index 0 leads to bulky shifting of the elements, requiring O(N) time for each removal.

The priority queue, when implemented properly, can support its most common operations much more quickly than a sorted list. Adding and removing elements requires O(log N) time, and peeking at the minimum element requires constant O(1) time. The key to such an efficient implementation is a clever underlying data structure called a heap, which we will explore in the following sections.

Introduction to Heaps

A *heap* is a tree that arranges its elements with a vertical ordering. In a minimum heap or *min-heap*, child nodes always

store values greater than or equal to the parent's value. In a maximum heap or *max-heap*, a parent node's value is always greater than or equal to those of its children. In other words, in a min-heap, the smallest element values are closer to the top (root) of the tree; in a max-heap, the largest element values are nearest to the root.

The fact that these elements are located at the top of the tree makes them easy to access and remove quickly and makes a heap an excellent structure to use to implement a priority queue. Since a priority queue is always finding and removing the smallest element, having that element at the very top of the tree makes finding it trivial. For this reason Java's [redacted] collection is implemented using a min-heap internally.

Heap

[redacted]

A tree that maintains a vertical ordering between parents and children, where parent nodes store smaller values than all of their children (min-heap) or parents store greater values than all of their children (max-heap).

Figure 18.13 shows an example of a min-heap. Notice how the smallest element value, [redacted], is at the top of the tree. Larger elements are placed lower on the tree. You may be surprised initially by some of the ordering; [redacted] appears higher in the tree than [redacted], and [redacted] is higher than [redacted] and [redacted], and so on. But the key property of a heap is that a parent's value must be smaller than that of its children, and the nodes just listed are not in the same subtree as each other, so they share no direct parent–child

relationship. Therefore, this example tree does satisfy the requirements to be a valid heap.

Figure 18.13 A min-heap

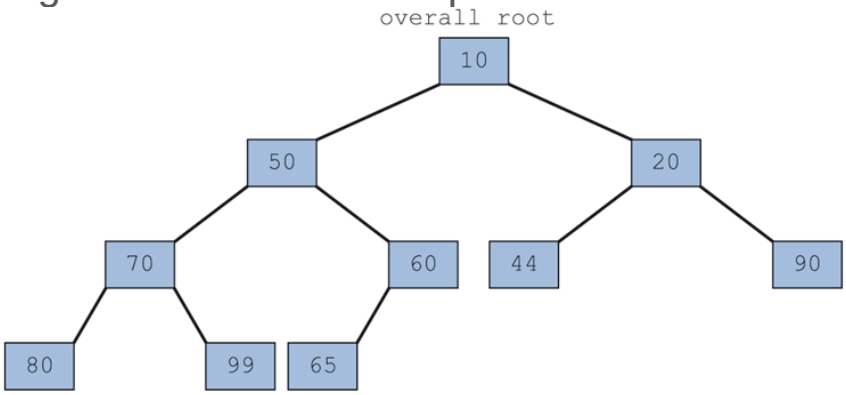


Figure 18.13 Full Alternative Text

A heap can be implemented with nodes having any number of children, but as we did with trees in the previous chapter, we will focus on heaps whose nodes have at most two children, which are called *binary heaps*.

A heap is usually maintained as a *complete tree*, which is one where every level except possibly the lowest level is completely filled with all nodes having both children, and if the last level of the tree is not completely filled, all children are at the left-most edge of the tree. This minimizes the height of the tree relative to the number of elements; a wider and shorter tree is faster for traversing from the root to any given leaf.

Some people become confused about the difference between a priority queue and a heap. A priority queue is an abstract data type, a description of a set of operations that a collection should have. A heap is one possible efficient way of implementing those operations, but it is not the only way of doing so. The Java class libraries add to this confusion by naming their class

[redacted] when it should probably be represented as an interface named [redacted] that is implemented by a class called [redacted], to match other pairs like [redacted]/[redacted] or [redacted]/[redacted].

The key operations for a priority queue are adding elements and finding/removing the minimum value. Let's explore how those operations can be implemented efficiently using a heap. Finding the minimum value ([redacted]) is trivial since it is simply the data stored in the overall root element of the tree. So we'll focus on adding and removing values from a heap.

Removing from a Heap

Suppose we have the heap shown in Figure 18.13 and want to remove the minimum value, 10. We can't simply yank the node out of the tree because it leaves a hole at the tree's root that must be filled. When dealing with binary search trees with their left-to-right ordering, removal usually involves moving one of the removed element's children up to occupy its spot and rearranging the tree as necessary to ensure that the ordering is left intact. With a heap we employ a different algorithm that begins not with the removed element's children but with a leaf.

To remove the root of a heap, first we remove the rightmost leaf of the tree. Then we place this leaf's value into the tree at the root, replacing the previous root value. This is likely to place the tree in a temporarily invalid state, since the leaf is presumably one of the larger element values and therefore is not suitable as a new root. But we then repair this problem by pushing the invalid root

down the tree by swapping it with its smallest child until it is in a legal location, a process called *bubbling* or *percolating* the element down the tree.

Bubble (Percolate)

Moving a heap element upward or downward until the heap's state satisfies the required vertical ordering.

The overall remove-min algorithm is described by the following pseudocode:

```
find and remove rightmost leaf node.

insert leaf's value into root node.

current node = root.

while (current node's value is not smaller than both children) {

    choose smaller of two children.

    swap current node's value with smaller child value.

    current node = smaller child node.

}
```

Figure 18.14 demonstrates the removal of the minimum value, [redacted], from our previous heap. First the rightmost leaf, [redacted], is inserted as the root. Then since [redacted] is not smaller than its children, it is swapped for its smaller child repeatedly until it is in proper order. The first swap is with [redacted], because [redacted]. Then [redacted] is compared with its new children, [redacted] and [redacted]. The [redacted] is smaller so the [redacted] is swapped there. Now the heap is back into proper order so the removal is complete.

Figure 18.14 Removing from min-heap with bubble-down

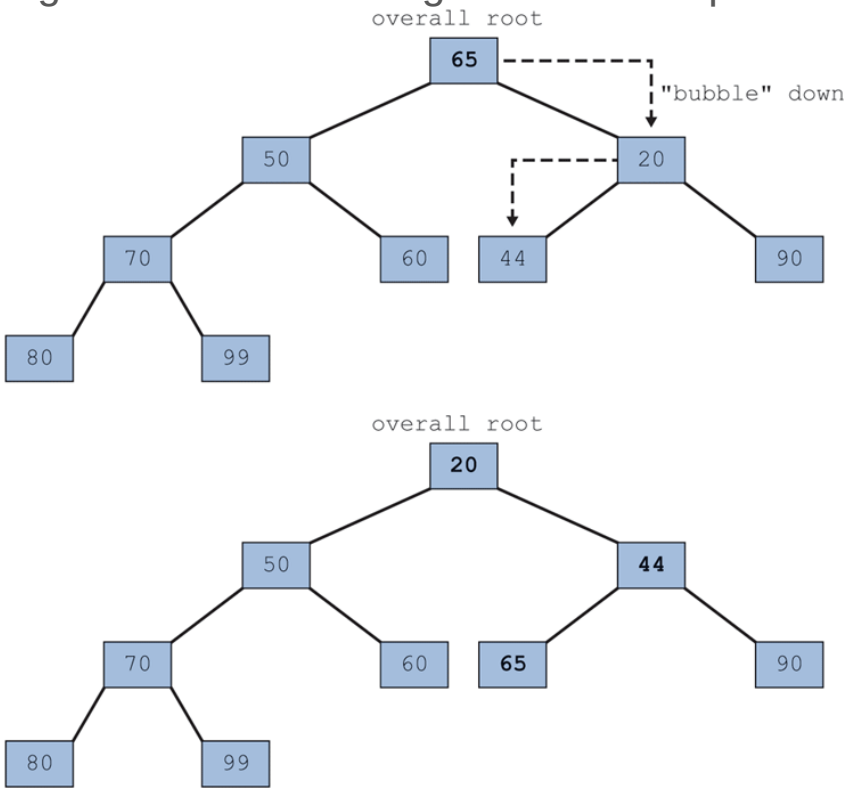


Figure 18.14 Full Alternative Text

The runtime of the remove-min operation is related to the number of swaps that occur as the new root is bubbled down to its proper location. Since our tree is a complete tree, it is as wide as possible for its height; the number of nodes in a tree of height H is roughly equal to 2H. Thus, the height of a tree containing N

elements is very close to log2 N. This means that the number of bubble-down swaps, and therefore the overall runtime of removing the minimum element from a heap, is O(log N).

Adding to a Heap

Adding a value to a heap is similar to removing the minimum element but in reverse. Suppose we have the heap shown in Figure 18.14 and want to add a new value, ■. The value needs to be placed properly to maintain the heap's overall ordering. The algorithm for adding an element to a heap begins at the bottom of the tree.

To add an element to a heap, we first insert it as a new rightmost leaf. (If the tree is completely full at its bottom level, the new value becomes the left child of the leftmost leaf, starting its own new level in the tree.) This is likely to place the tree in a temporarily invalid state, since the newly inserted leaf may not be larger than all of its parents and therefore not suitable to be placed there. We repair this problem by swapping the new value upward with its parents until it is in a suitable location, bubbling it upward much like we bubbled values downward upon a removal. The overall pseudocode for adding to a heap is the following:

```
insert new element value as a new rightmost leaf node.
```

```
current node = the new leaf node.

while (current node's value is not greater than its parent) {

    swap current node's value with parent's value.

    current node = parent node.

}
```

Figure 18.15 demonstrates adding the value [redacted] to our example heap from Figure 18.14. The value bubbles up twice, swapping with [redacted] and then with [redacted], because those values are greater than [redacted]. It stops there because the next parent is [redacted], which is smaller than [redacted] and should therefore remain higher in the tree.

Figure 18.15 Adding to min-heap with bubble-up

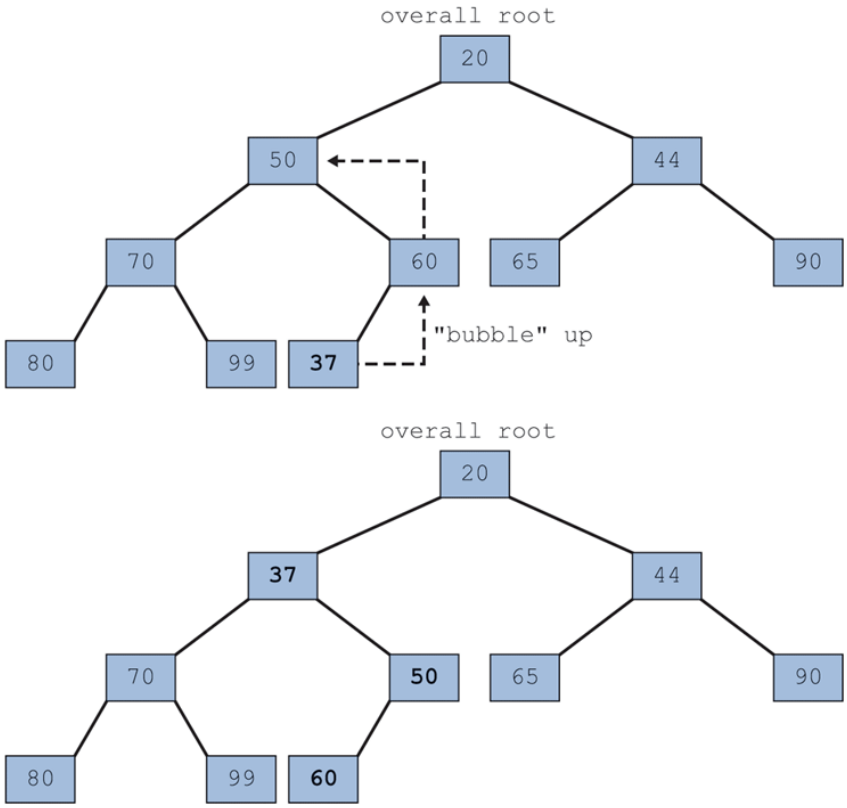


Figure 18.15 Full Alternative Text

The runtime complexity of adding to a heap is similar to that of removing the minimum element. In an ideal case, the newly added element will be larger than its parent and no upward bubbling will be required, which would yield a constant O(1) runtime. In the worst case, the newly added node is smaller than all of the values in the heap and must bubble all the way to the root. Since the height of a heap with N elements is almost exactly log2 N, the worst case for adding to the heap is O(log N). In general the newly added element must bubble itself some portion of the distance up the tree, so we say that the [redacted] operation has a O(log N) average runtime.

Array Heap Implementation

A heap can be implemented as a tree of linked node objects much like the binary trees implemented in the binary trees chapter. But a few of the operations we want to perform don't lend themselves well to the node implementation. For example, bubbling upward requires us to be able to reach a node's parent, and our tree implementation did not contain references from a node to its parent. Also, the add and remove operations both involve manipulating the rightmost leaf in the tree, which is not trivial to access in a tree implemented with nodes.

There is another clever implementation of heaps that provides great efficiency for the operations we want to implement. It turns out that any complete binary tree can actually be stored as an array, where index 1 stores the root element, and the children of index i are stored at indexes 2i and 2i+1. So the children of index 1 (the root) are at indexes 2 and 3; the children of index 2 are at indexes 4 and 5; the children of index 3 are indexes 6 and 7; the children of index 4 are indexes 8 and 9; the children of index 5 are indexes 10 and 11; and so on. We leave index 0 blank and start our root at index 1 so that the indexes line up conveniently in this way. Figure 18.16 shows an array representation of the same heap depicted in Figure 18.13, with the value as its root.

Figure 18.16 Heap implemented as array

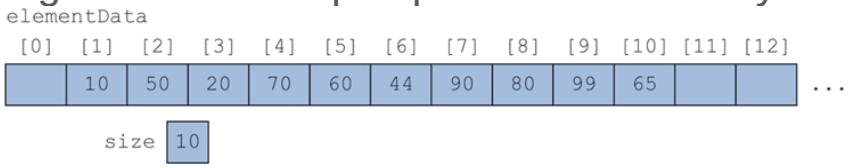


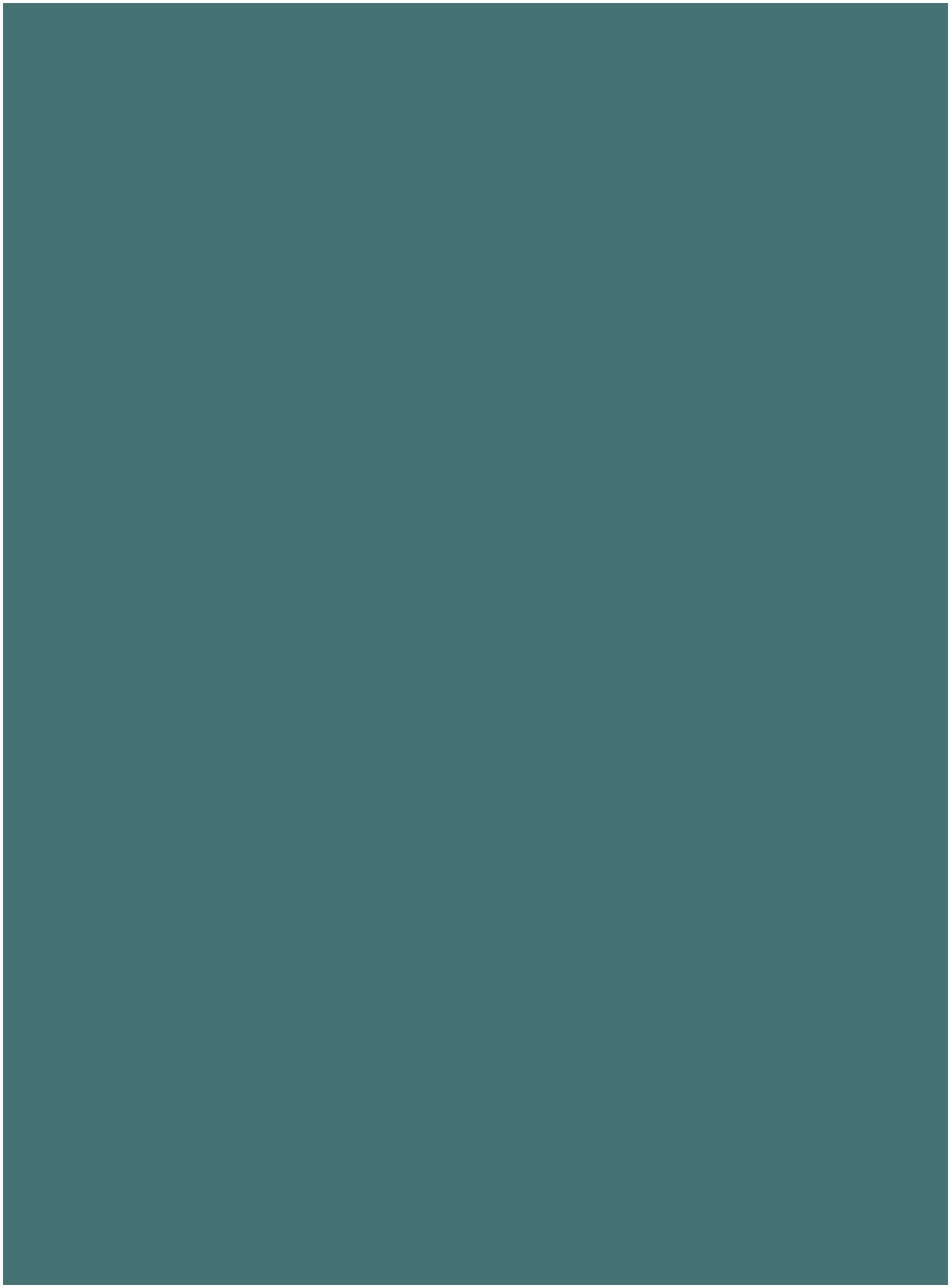
Figure 18.16 Full Alternative Text

The array representation is excellent for implementing a heap because it is easy to jump from a node to its parent; the parent of

the element at index i is stored at index i/2. It is also trivial to find the rightmost child in the heap; the rightmost child is always located at index *size*, and any new rightmost child we want to insert should be inserted at index size+1. For example, in the array heap shown in Figure 18.16, the rightmost leaf is , stored in index 10, and a new leaf would be added at index 11.

Let's implement a skeletal version of a priority queue using a heap represented as an array.

To simplify the implementation of the add and remove operations, we'll introduce several short helper methods for navigating upward and downward in the tree from parents to children and vice versa, as well as a method to swap elements in the array, since swapping is used in bubbling.



First let's implement the [redacted] operation, since it is a bit simpler than removal. Figure 18.17 shows the adding of element value [redacted] to the heap. It bubbles upward as far as necessary until the value is not smaller than its parent value or until it reaches the root of the tree.

Figure 18.17 Array heap [redacted] operation

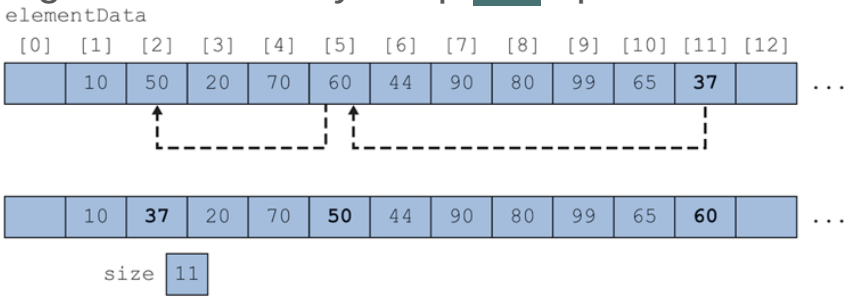
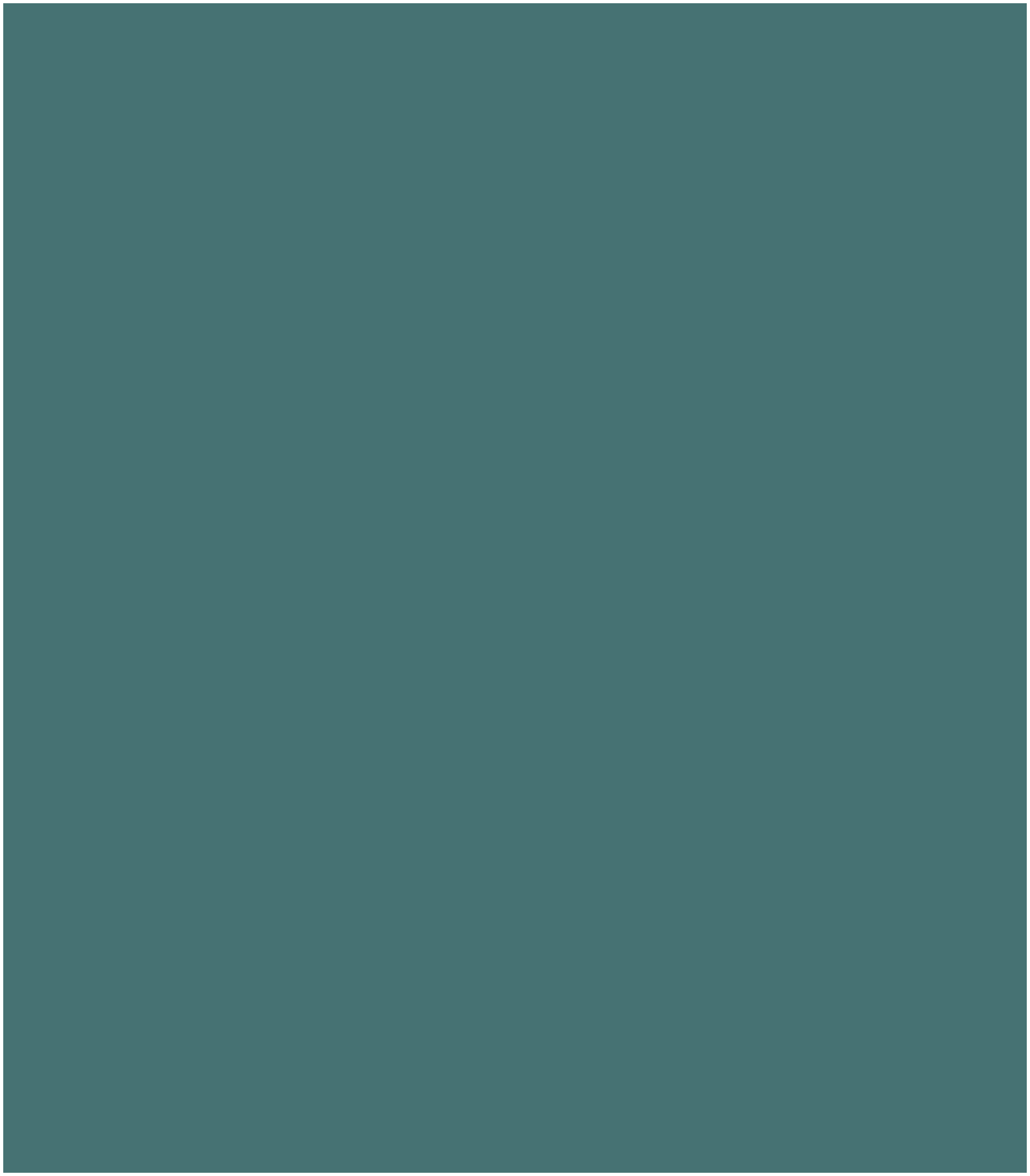


Figure 18.17 Full Alternative Text

The following code implements the [redacted] operation on our min-heap. The code uses the helper methods defined previously such

as [redacted] and [redacted]. Notice that the node is initially inserted at the end of the array (index size+1) and then bubbled upward to parent indexes as long as the node’s value is less than that of its parent.



As with an array list or hash table, the array can run out of space. A proper method should also check to make sure that the array has enough capacity to hold the element being added. The following code can be inserted at the start of the method to handle this:

Now let’s implement the remove-min operation. Figure 18.18 shows the removal of the root element value from the heap from Figure 18.17. The rightmost leaf, , is moved to the root, then it is bubbled downward, swapping it with its smaller child as far as necessary until the value is not larger than either of its children (or until it is a leaf and has no children). In the example, swaps with its right child, , which is smaller than ; then it swaps with its left child, , which is less than its right child, .

Figure 18.18 Array heap remove-min operation

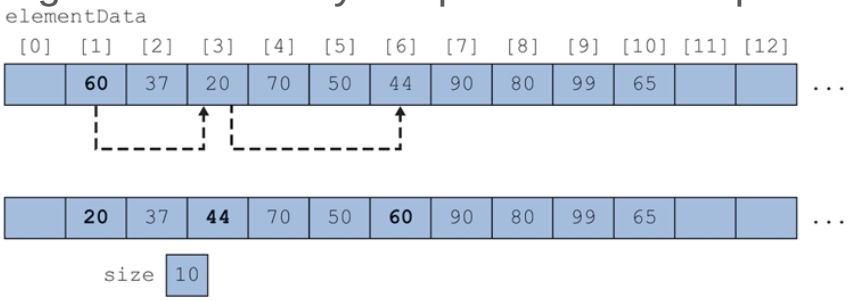
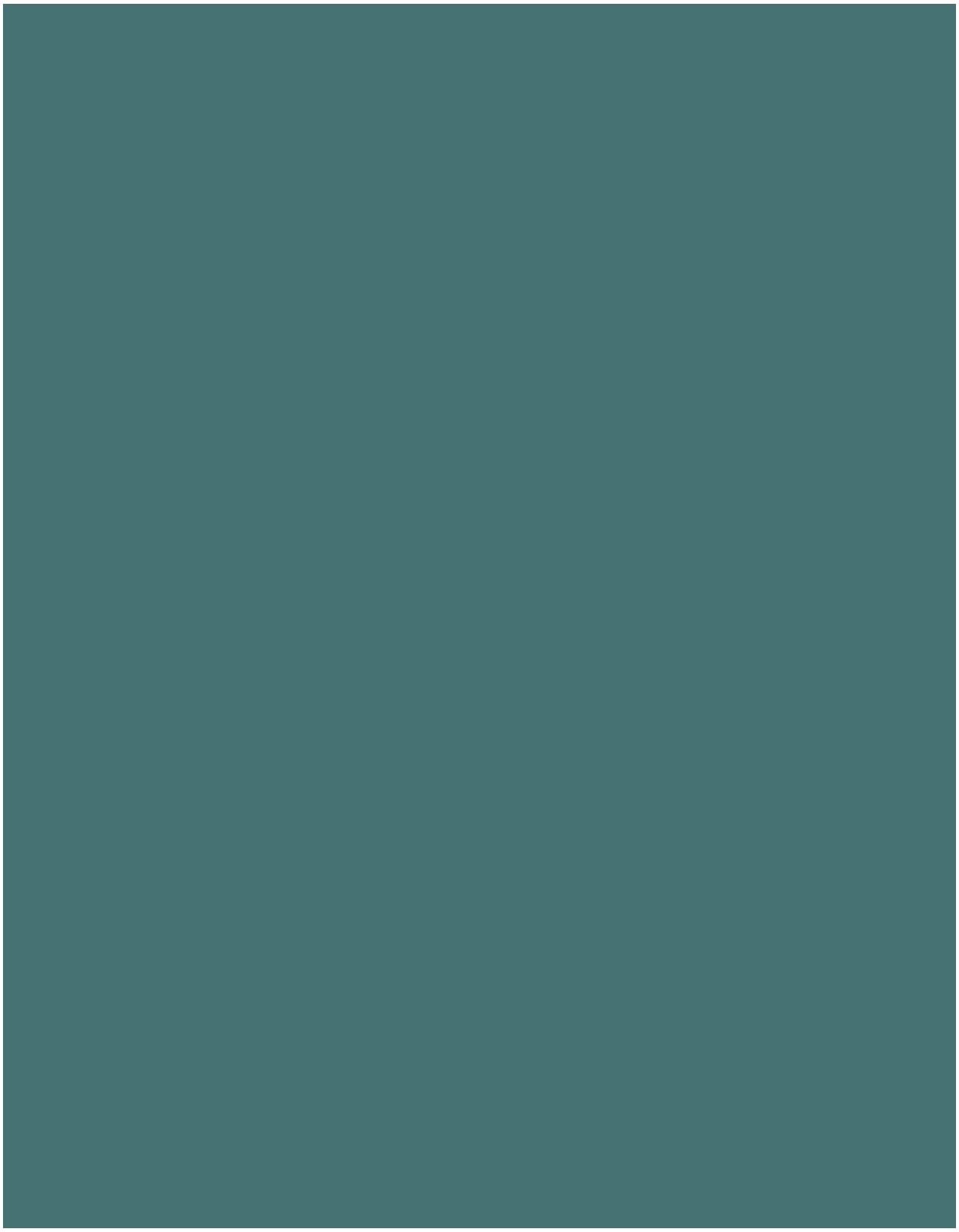


Figure 18.18 Full Alternative Text

The following code implements the remove-min operation on our min-heap. The code uses the helper methods defined previously, such as [redacted] and [redacted]. The code inside the loop initially chooses to swap with the left child, but if the right one is smaller than the left, the right one is chosen. The loop stops when the node has no children (when it becomes a leaf) or when it is not larger than its child and has therefore reached the correct position.

[redacted]



A heap that stores objects rather than integers is very similar to the code we have already shown, except that comparisons are made using the [redacted] method rather than [redacted] and [redacted] operators. You can find complete code for the [redacted] class, as well as a [redacted] that stores objects, on our web page at http://buildingjavaprograms.com.

Heap Sort

In a previous chapter, we discussed details of several sorting algorithms, such as selection sort and merge sort. There is a simple algorithm for sorting called *heap sort* that takes advantage of the ordering of heaps to arrange data quickly. If you add all elements of an array to a priority queue and then remove them, they will come out in ascending (sorted) order. The heap sort algorithm can be described by the following pseudocode:

```
heapSort(A):
    H = create new heap.
    for each element n in A:
        add n to H.
    while (H not empty):
```

```
        remove element from H.

        add element back into A.
```

Here is a simple implementation of the heap sort for sorting an array of integers using a :

When sorting an array of N elements, heap sort performs N add and N remove operations on a heap. Each add or remove has a O(log N) complexity, so the overall heap sort algorithm has O(N log N) complexity. Heap sort is a fairly efficient algorithm, certainly much faster than selection sort and with comparable performance to merge sort.

One drawback of the heap sort algorithm as shown is the memory required. To sort the array we must create another large data structure (the priority queue, and the array heap inside it) to temporarily store the data. For very large data sets this can be undesirable.

It is possible to implement heap sort “in-place” so that it uses the original array itself as the heap. The basic idea is to treat the array as an initially invalid heap and “repair” it into a proper heap by repeatedly bubbling elements into proper positions. Once the array represents a heap, performing [redacted] operations repeatedly on the array and moving the removed elements to the end of the array end up fully sorting the array contents. The algorithm arranges the elements into ascending order if a max-heap is used and descending order if a min-heap such as our [redacted] is used.

The in-place version of the heap sort algorithm would require a modified version of our [redacted] implementation with a few tweaks. The details of implementing an in-place heap sort algorithm are outside the scope of this chapter, but a full implementation of the algorithm is posted on our web page at http://buildingjavaprograms.com.

Chapter Summary

Hashing is a technique for mapping between element values and integers representing array indexes at which to store those elements. Hashing allows for the implementation of sets and maps with extremely efficient O(1) runtime for adding, removing, and searching for elements.

A hash function is an algorithm that maps element values to array indexes. A common technique for integer data is to mod (%) the value by the array's length.

When multiple element values have the same hash function value, there is a collision. Collisions can be resolved by moving to another index (probing) or by storing a list of matched elements at each index (separate chaining).

Rehashing is the process of resizing a hash table, generally to enlarge it so that it can efficiently store more elements. The array cannot simply be enlarged; the elements must be readded because their hash function values may change in the new larger array.

Non-integer elements can be hashed by calling their [redacted] method.

A priority queue allows elements to be inserted in any order and removed in ascending order.

A structure called a heap is often used to implement priority queues. A heap is a tree with a vertical ordering, where parents store smaller values than their children (min-heap) or larger values than their children (max-heap).

When adding a value to a heap, it is added as a leaf and then “bubbled” (swapped) up the tree until it is in a proper place in the vertical ordering. When removing a value from a heap, the heap’s rightmost leaf is moved to the root and then “bubbled” down the tree until it is in a proper place in the vertical ordering.

Because of its complete structure, a heap can be implemented efficiently using an array, rather than as a linked tree of node objects.

Heaps can be used to help sort data, an algorithm called heap sort.

Self-Check Problems

Section 18.1: Hashing

1. What is hashing, and why is hashing a good way of implementing a set?
2. Which of the following statements about hash tables is true?
   a. A hash table has O(log N) average time to add and search for elements.
   b. The higher a hash table's load factor, the more quickly elements can be found.
   c. Once a hash table's load factor reaches 0.75, no more elements can be added.
   d. A hash function maps element values to integer indexes in the hash table.
   e. A good hash function returns the same value as much as possible for all data.
3. How do you know when a hash table has become "full"? At what point does it become impossible to add further elements to a hash table of a given size? Does it depend on the collision resolution strategy used?
4. The following statement is an incorrect attempt to resize a hash table to twice its current size. Why is this code incorrect? What is the proper way to enlarge a hash table?

5. What is the final state of a hash table of size 10 after adding 35, 2, 15, 80, 42, 95, and 66? Assume that we are using the standard "mod" hash function shown in the

chapter and linear probing for collision resolution. Do not perform any resizing or rehashing. Draw the entire array and the contents of each index.

6. If separate chaining is used for collision resolution, and the same elements from the previous problem (35, 2, 15, 80, 42, 95, and 66) are added to a hash table of size 10, what is the final state of the hash table? Do not perform any resizing or rehashing. Draw the entire array and the contents of each index.
7. Suppose we have a hash set that uses the standard “mod” hash function shown in the chapter and uses linear probing for collision resolution. The starting hash table length is 5, and the table chooses to rehash to twice its former size once the load factor reaches or exceeds 0.5. If we begin with an empty set, what will be the final state of the hash table after the following elements are added and removed? Draw the entire array and the contents of each index, including any resizing and rehashing necessary. Write “X” in any index in which an element is removed and not replaced by another element. Also write the size, capacity, and load factor of the final hash table.

8. Suppose we have a hash set that uses the standard “mod” hash function shown in the chapter and uses linear probing for collision resolution. The starting hash table length is 11, and the table does not rehash during this problem. If we begin with an empty set, what will be the final state of the hash table after the following elements are added and removed? Draw the entire array and the contents of each index. Write “X” in any index in which an element is removed and not replaced by another element. Also write the size, capacity, and load factor of the final hash table.

9. For each of the following possible [redacted] implementations: Is the following a legal [redacted] method for a [redacted] class, according to the general contract of that method? Does it distribute the hash codes well between objects? Why or why not?

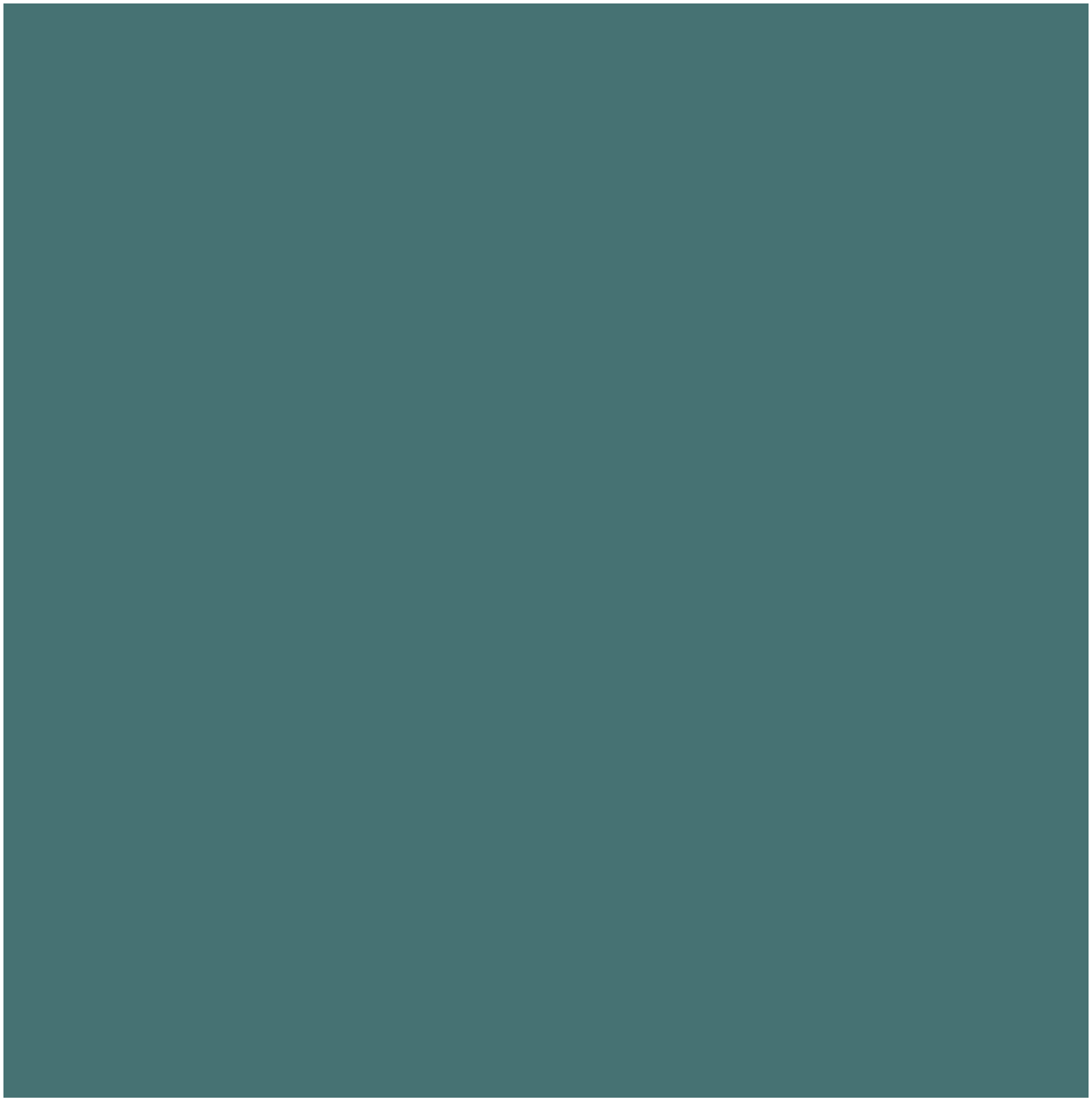
a.

b.

c.

10. Write a [redacted] method for a [redacted] class, whose fields are a year, month, and day, as integers. Follow the general contract for the [redacted] method.
11. Write a [redacted] method for a [redacted] class, whose fields are a name (a string), age (an integer), student ID number (an integer), and weight in pounds (a real number). Follow the general contract for the [redacted] method.
12. Suppose we have a hash map that uses the standard "mod" hash function shown in the chapter and uses linear probing for collision resolution. The starting hash table length is 5, and the table chooses to rehash to twice its former size once the load factor reaches or exceeds 0.5. If we begin with an empty map, what will be the final state of the hash table after the following key/value pairs are added and removed? Draw the entire array and the contents of each index, including any resizing and

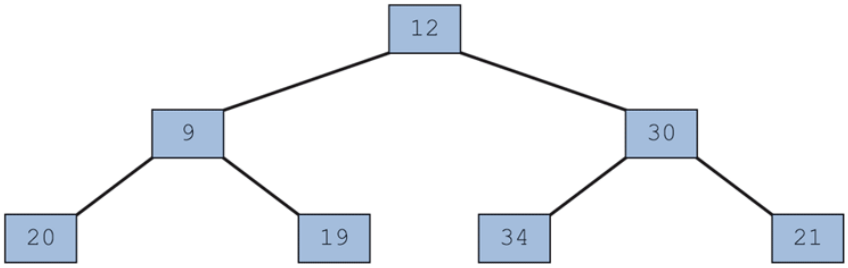
rehashing necessary. Write “X” in any index in which an element is removed and not replaced by another element. Also write the size, capacity, and load factor of the final hash table.



Section 18.2: Priority Queues and Heaps

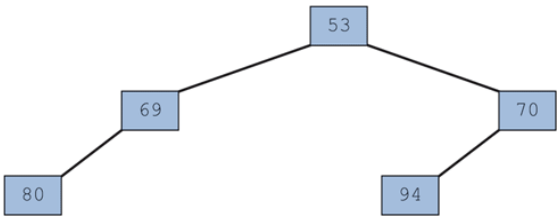
13. Which of the following statements about min-heaps is true?
    a. Smaller values are on the left and larger values are on the right.
    b. The smallest value is the root.
    c. The smallest value is one of the leaves.
    d. Every value is smaller than all values at lower levels of the tree. For example, if there is a [redacted] at level 3, there will not be any elements with values less than 25 at levels 4 and beyond.
    e. The [redacted] operation returns the largest value in the heap.
14. If a binary heap has 26 nodes, what is its height? If it has 73 nodes, what is the height? How do you know for sure?
15. Which of the following are valid min-heaps? For the one(s) that are invalid, what makes them invalid?

a.



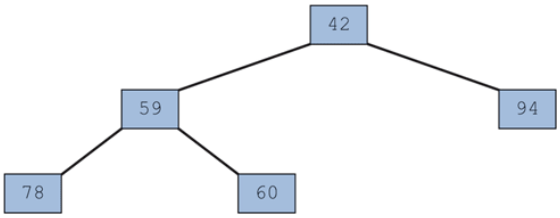
18.2-2 Full Alternative Text

b.



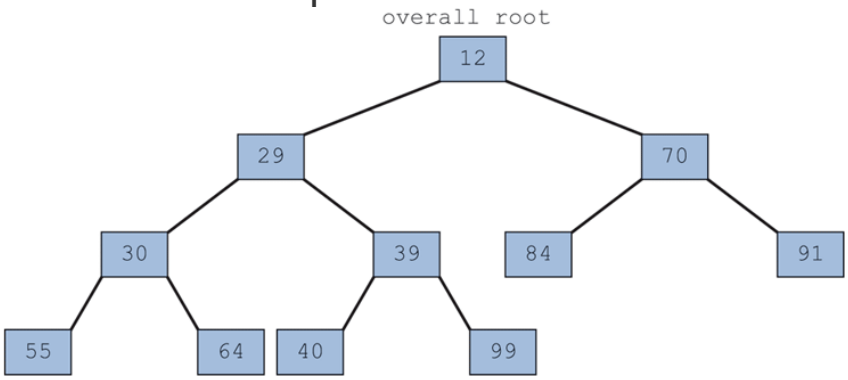
18.2-3 Full Alternative Text

c.



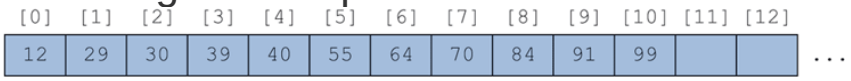
18.2-4 Full Alternative Text

16. In an array heap as implemented in this section, for the element at index 8 of the array, what are the indexes of its left and right children? What is the index of its parent? If the element is at index 23, what are the indexes of its children and parent?
17. Simulate the adding of the value [redacted] to the following min-heap:



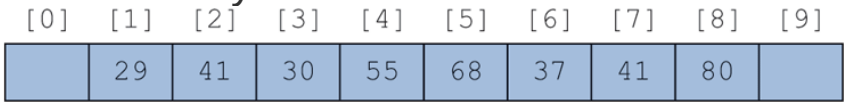
18.2-5 Full Alternative Text

18. Simulate the adding of value [redacted] to the same heap from the previous problem, after the [redacted] has already been added.
19. Draw the tree for the binary min-heap that results from inserting 4, 9, 3, 7, 2, 5, 8, 6 in that order into an initially empty heap.
20. Perform 3 removals on the heap you drew in the previous problem. Show the complete state of the tree after each removal.
21. Draw the tree for the binary min-heap that results from inserting 11, 9, 12, 14, 3, 15, 7, 8, 1 in that order into an initially empty heap.
22. Perform 3 removals on the heap you drew in the previous problem. Show the complete state of the tree after each removal.
23. The following is an incorrect diagram of the array representations of the heap shown in Self-Check Problem 17 using the array representation described in this section of the chapter (before any adds or removals are made from it). What is wrong with the diagram? What changes must be made to fix it so that it accurately represents the given heap?



18.2-6 Full Alternative Text

24. Draw the min-heap being represented by the given array:



18.2-7 Full Alternative Text

25. Draw the array representation of the heap you computed as your answer to Self-Check Problem 19 (after all of the elements are added to it).
26. Draw the array representation of the heap you computed as your answer to Self-Check Problem 21 (after all of the elements are added to it).

Exercises

1. Write a method in the [redacted] class called [redacted] that accepts another hash set as a parameter and adds all of the elements from the other set into the current set. For example, if the set stores [redacted] and the method is passed [redacted], your set would store [redacted].
2. Write a method in the [redacted] class called [redacted] that accepts another hash set as a parameter and returns [redacted] if your set contains every element from the other set. For example, if the set stores [redacted] and the method is passed [redacted], your method would return [redacted]. If the method were passed [redacted], your method would return [redacted] because your set does not contain the value [redacted].
3. Write a method in the [redacted] class called [redacted] that accepts another hash set as a parameter and returns [redacted] if the two sets contain exactly the same elements. The internal hash table size and ordering of the elements does not matter, only the sets of elements themselves.
4. Write a method in the [redacted] class called [redacted] that accepts another hash set as a parameter and ensures that this set does not contain any of the elements from the other set. For example, if the set stores [redacted] and the method is passed [redacted], your set would store [redacted].
5. Write a method in the [redacted] class called [redacted] that accepts another hash set as a parameter and removes all elements from this set that are not contained

in the other set. For example, if the set stores [redacted] and the method is passed [redacted], your set would store [redacted].

6. Write a method in the [redacted] class called [redacted] that returns the elements of the set as a filled array. The order of the elements in the array is not important as long as all elements from the set are present in the array, with no extra empty slots before or afterward.
7. Write a method in the [redacted] class called [redacted] that returns a string representation of the elements in the set, such as [redacted]. The order of the elements in the string does not matter as long as they are all present in the proper format. Do not list any empty or meaningless indexes in the string.
8. Write a method called [redacted] that accepts an array of integers and rearranges the integers in the array to be in descending order using a [redacted] as a helper. For example, if the array passed stores [redacted], after the call the array should store [redacted].
9. Write a method called [redacted] that accepts a [redacted] of integers and an integer k as parameters and returns the kth-smallest integer from the priority queue. For example, if the queue passed stores the integers [redacted] and k is 4, return the fourth-smallest integer, which is [redacted]. If k is 0 or negative or greater than the size of the queue, throw an [redacted]. If your method modifies the state of the queue during its computation, it should restore the queue before it returns. You may use one stack or queue as auxiliary storage.
10. Write a method called [redacted] that accepts a [redacted] of integers as a parameter and returns

[redacted] if the queue contains a sequence of consecutive integers starting from the front of the queue. Consecutive integers are integers that come one after the other, as in 5, 6, 7, 8, 9, etc., so if the queue stores [redacted], your method should return [redacted]. If your method modifies the state of the queue during its computation, it should restore the queue before it returns. You may use one stack or queue as auxiliary storage.

11. Write a method called [redacted] that accepts a [redacted] of integers as a parameter and modifies the queue's state so that any element that is equal to another element in the queue is removed. For example, if the queue stores [redacted], your method should modify the queue to store [redacted]. You may use one stack or queue as auxiliary storage.
12. Write a method called [redacted] that accepts a [redacted] of integers as a parameter and replaces every value in the queue with two occurrences of that value. For example, if the queue stores [redacted], your method should modify the queue to store [redacted]. You may use one stack or queue as auxiliary storage.
13. Write a method called [redacted] that accepts a [redacted] of integers as a parameter and adds elements to it until every element in its range of smallest to largest is represented once if it was not previously found in the priority queue. For example, if the queue stores [redacted], your method should modify the queue to store [redacted]. You may use one collection as auxiliary storage.
14. Write a method in the [redacted] class called [redacted] that returns the elements of the queue as

a filled array. The order of the elements in the array is not important as long as all elements from the queue are present in the array, with no extra empty slots before or afterward.

15. Write a method in the [redacted] class called [redacted] that returns a string representation of the elements in the queue, such as [redacted]. The order of the elements in the string does not matter as long as they are all present in the proper format.
16. Write a method in the [redacted] class called [redacted] that accepts another [redacted] as a parameter and adds all elements from the other queue into the current queue, maintaining proper heap order such that the elements will still come out in ascending order when they are removed. Your code should not modify the queue passed in as a parameter. (Recall that objects of the same class can access each other's private fields.)

Programming Projects

1. Add an iterator to the [redacted] class written in this chapter. To do this you will need to write an inner class that can iterate over the elements of the set, remembering its position as it moves along. Also write a method in the [redacted] class called [redacted] that returns such an iterator object.
2. In this chapter, we implemented a [redacted] that used separate chaining to resolve collisions. Implement your own version of [redacted] that uses linear probing to resolve collisions. Perform lazy removals, placing a special “removed” marker value into any bucket where an element is removed.
3. Modify the [redacted] class written in this chapter to make it configurable in ways similar to Java’s [redacted] class. Make it possible for the heap to be a min-heap or max-heap. (If you create a heap of objects, you could also modify it to accept a [redacted] parameter to its constructor.)
4. Modify the [redacted] class written in this chapter to make it into a three-heap. A three-heap is similar to a binary heap, except that each node is considered to have three children rather than two children. A three-heap is a wider tree though not as tall as a binary heap. The heap can still be represented as an array, but the algorithms for traversing it must be modified to consider all three children when bubbling as appropriate, etc. (Do you expect that a three-heap will be faster or slower than a binary heap for insertion, and for

removal? Why? You can create a test to verify the performance of each operation.)

5. Write a program that implements the “Huffman coding” compression algorithm using priority queues and binary trees. Huffman coding is an algorithm devised by David A. Huffman of MIT in 1952 for compressing text data to make a file occupy a smaller number of bytes. Normally text data is stored in a standard format of 8 bits per character, commonly using an encoding called ASCII that maps every character to a binary integer value from 0–255. The idea of Huffman coding is to abandon the rigid 8-bits-per-character requirement and use different-length binary encodings for different characters. The advantage of doing this is that if a character occurs frequently in the file, such as the letter “e”, it could be given a shorter encoding (fewer bits), making the file smaller.

The steps involved in Huffman coding a given text source file into a destination compressed file are the following:

a. Examine the source file’s contents and count the number of occurrences of each character (consider using a map).
b. Place each character and its frequency (count of occurrences) into a priority queue ordered in ascending order by character frequency.
c. Convert the contents of this priority queue into a binary tree with a particular structure. Create this tree by repeatedly removing the two front elements from the priority queue (the two nodes with the lowest frequencies) and combining them into a new node with these two nodes as its children and the two nodes’ combined frequencies as its frequency. Then reinsert this combined node back into the priority queue.

Repeat until the priority queue contains just one single node.

d. Traverse the tree to discover the binary encodings of each character. Each left branch represents a ‘0’ in the character’s encoding and each right branch represents a “1”.
e. Reexamine the source file’s contents, and for each character, output the encoded binary version of that character to the destination file to compress it.

Chapter 19 Functional Programming with Java 8

19.1 Effect-Free Programming
19.2 First-Class Functions
• Lambda Expressions
19.3 Streams
• Basic Idea
• Using Map
• Using Filter
• Using Reduce
• Optional Results
19.4 Function Closures
19.5 Higher-Order Operations on Collections
• Working with Arrays
• Working with Lists
• Working with Files
19.6 Case Study: Perfect Numbers
• Computing Sums
• Incorporating Square Roots
• Just Five and Leveraging Concurrency

Introduction

Every 10 to 15 years the computer science education community updates the suggested curriculum for computer science majors. In the most

recent curriculum revision in 2013, the ACM/IEEE joint task force updated the suggestions for coverage of programming languages. As they explained in their introduction to that section:

Software developers must understand the programming models underlying different languages and make informed design choices in languages supporting multiple complementary approaches. Computer scientists will often need to learn new languages and programming constructs, and must understand the principles underlying how programming language features are defined, composed, and implemented.

They went on to describe specific concepts from various programming paradigms that should be studied by every undergraduate computer science major. Two of the paradigms that they emphasize are the procedural and object-oriented paradigms that have been discussed thoroughly in this book. Java is a hybrid language that supports both approaches. We began the book by studying procedural programming with static methods that we used to decompose a large program into action-oriented components. Then starting in Chapter 8 we moved on to the object-oriented approach in which a large program is decomposed into a set of interacting objects each with their own class defining their state and behavior.

In this chapter, we will explore a third approach to programming that is known as *functional programming*. Java uses the term "method" for what many other languages call a "function," so in the discussion that follows, you can mentally substitute "method" for "function."

Functional Programming

A style of programming that emphasizes the use of functions (methods) to decompose a complex task into subtasks.

The Java community has spent years discussing how best to add functional features to the language, culminating with the release of Java 8 in 2014. The new features added to the language in this release allow Java programmers to take advantage of this different approach to programming. The new curriculum guidelines list five specific topics that undergraduates should study related to functional programming:

1. Effect-free programming
2. Processing structured data via functions
3. First-class functions
4. Function closures
5. Higher-order operations on collections

The first three are categorized as Core Tier-1 and items 4 and 5 are categorized as Core Tier-2. As the guidelines explain, "computer-science curricula should cover all the Core Tier-1 topics" and "all or almost all of the Core Tier-2 topics." Each of these concepts has its own section in this chapter, except for Item

2 which does not translate easily to Java. In addition, this chapter includes a separate section that discusses the central Java 8 concept of streams because much of the functional programming support is provided using that mechanism. This chapter ends with a short case study that picks up an additional Core Tier-1 concept of using concurrency to speed the execution of a complex computation. As we will see, the functional programming approach is particularly well suited to safe and reliable concurrency, which is becoming increasingly important in modern computing where the average PC now comes with multiple processors (sometimes described as “multi-core processing”). The terminology can sometimes be intimidating, but as we will see, these concepts are fairly straightforward to discuss using Java 8 constructs.

19.1 Effect-Free Programming

Consider the following line of code:

This code computes a result by calling a function (a method) two different times and adding together the results. Offhand it seems that we would get the exact same behavior with this line of code:

Sometimes the two are equivalent and sometimes not. The question to consider is whether the function produces a *side effect*.

Side Effect

A change to the state of an object or program variable produced by a call on a function (i.e., a method).

As an example, consider the following program:

```
public class


public static int


public static int

int
```

```
return

public static void

int
```

This confusing program begins with the declaration of a static variable called [redacted] that has the entire class as its scope. Such variables are called *global variables*. We have discouraged the use of global variables because they can lead to the kind of confusing code we have here.

The program begins by initializing the global variable [redacted] to the value 5. Then on line 14 the program calls the function [redacted] twice, passing it the value of [redacted] each time. But [redacted] doubles the value of [redacted] before returning the sum of the value passed as a parameter and [redacted]. If you find this all highly confusing, you're not alone. This style of programming is very bad.

On the first call to [redacted], the variable [redacted] has the value 5. This is passed from [redacted] as the parameter called [redacted]. Inside function [redacted], we first double [redacted], which gives it the value 10. It then returns the sum of [redacted], which is now 10, and [redacted], which is 5. So it returns 15 for the first call. On the second call, it passes the current value of [redacted], which is 10, to the function and uses this to initialize [redacted] to be 10. Then it doubles [redacted] from 10 to 20 and returns the sum of [redacted] and [redacted], which are 20 and 10. So it returns the value 30 on the second call. The variable [redacted] is set to the sum of these two returns values, which is 15 plus 30, or 45.

What happens if we substitute our other line of code for setting [redacted]?

[redacted]

The first call on the function behaves as it did before, returning the value 15. We double that to 30 and set [redacted] to 30. So we

end up with a program that has different behavior because the function [redacted] has a side effect.

Because functional programming focuses so much on individual functions, the community of programmers who use functional programming regularly have concluded that side effects should be avoided when possible, leading to the kind of “effect free” functions that serve as the title of this section.

The object-oriented community has been less wary of functions with side effects. A central idea of OOP is that objects have state that changes as methods are called. Recall from Chapter 8 that instance methods are classified as *accessors* that can be thought of as read-only operations accessing the object’s state and *mutators* that can be thought of as read/write operations that often change the state of the object. Using the terminology from functional programming, we would say that accessors are usually effect-free functions while mutators are functions with a side effect. Even accessors can have side effects because they might change the state of other objects.

One of the simplest and most pervasive sources of side effects we have seen is the printing of values. Continuing our example of a function [redacted] that might be called once or twice, if the function includes a call on [redacted], then it will produce more output when called twice than when it is called once. So even though the variable [redacted] in our example might be set to the same value, there is still a possible side effect caused by the function if it produces output.

As we will see in the case study at the end of this chapter, there are great advantages to writing code that is free of side effects. It is easier to take advantage of concurrency using such code and it

is easier to prove formal properties of programs that are written using the effect-free approach.

DID YOU KNOW?

Functional Programming Influences on Java

One of the most successful functional programming languages is the Scheme language developed at MIT in the 1970s by Guy Steele and Gerry Sussman. Jim Gosling who designed the Java programming language was well aware of Scheme and it is clear that many of the ideas from Scheme influenced the design of Java.

As mentioned in Chapter 3, objects of the class are immutable, which means that all of their methods are free of side effects. This has greatly simplified the use of objects. Consider, for example, a class with a field called name of type and an accessor method for examining the value of that field:

In Java, you don’t have to worry about the possibility that a client of this class might alter your string object. In other programming languages, you might make a defensive copy to give to the client to avoid any possible interference.

The idea of effect-free programming is also evident in the class that has methods for constructing an “unmodifiable” version of a collection, such as .

Another example of the influence of Scheme can be found in Java’s concept of interning of strings (read the description of the method if you are curious). It is also worth noting that Guy Steele, the co-designer of Scheme, was hired by Sun Microsystems in 1994 to work on the Java team.

19.2 First-Class Functions

When you study any particular programming language, you will soon learn what the basic elements of the language are. For example, almost every programming language allows you to manipulate numbers as basic elements. You can store a number in a variable, you can pass a number as a parameter, you have various operations for manipulating numbers, and you can return a number from a method call. Strings are also basic elements of Java because they are easy to declare, store, and pass as parameters.

Not every data type is a first-class citizen of every language. For example, not every programming language allows you to manipulate Boolean values as basic elements the way Java does. For decades the C programming language did not have a proper type for storing Boolean values and programmers instead used the integer values 1 and 0 to represent true and false. Some languages consider collections like lists and maps to be first-class citizens, providing special syntax for declaring and manipulating them; in Java, arrays receive first-class treatment and special syntax, but the collection objects do not.

First-Class Citizen

An element of a programming language that is tightly integrated with the language and supports the full range of operations generally available to other entities in the language.

What would it mean to provide first-class status for functions? Let’s begin with an example that points out the motivation. Consider the task of performing drill and practice with a user where you make up addition problems involving two numbers between 1 and 12 and you keep track of how many answers the user got right. Below is a typical interaction.

Suppose that the above quiz is represented as a method called [redacted] that is passed a [redacted] that reads from the console and a number of problems to provide to the user, as in:

This is a straightforward task to solve building on material from Chapters 1 through 5.

We now have a method that allows us to have a user practice addition problems. What if we wanted to also practice multiplication problems? Most of the code stays the same, but we would have to change the addition in these two lines of code:

to be multiplication:

In a sense, we want to replace the plus with an asterisk in both of these lines of code. If we want the code to work for both addition and multiplication, the first line of code is easier to generalize. The only part that needs to change is the text to be printed, and we have the ability to manipulate text as a basic element of Java using a [redacted]. We can change the method header to take a third parameter specifying the text to print:

```
String text



text
```

We can then make two calls on the method to have it use a plus the first time and an asterisk the second time:

So far so good. But now we run into the problem that the method is computing the right answer as the sum of the two numbers, even if an asterisk is passed as the text. That will make for a very frustrating experience on the second call where the console indicates to the user that the problem is a multiplication problem.

The question is how to change the computation of the right answer. It would be great if we could say:

If Java somehow filled in “+” or “*” appropriately and then used the corresponding operator for the computation, then it would work. But Java doesn’t work this way. The way that we usually get around this in Java is by introducing an [redacted] structure that tests whether the string is a plus or an asterisk. But then the code works for only those two operators, and additional branches must be added to the code later to make it support subtraction and other operations.

What we really want is the ability to pass an additional parameter that specifies the calculation to perform. We want to say, “Use the addition operation the first time and the multiplication operation the second time.” A functional programmer would say that what we want to be able to do is to pass in a function. This is an example of what we mean by elevating functions to first class status in the language. We want to be able to introduce a fourth parameter that specifies the function to use for computing the right answer. That requires thinking of the function as a thing in the language that can be passed as a parameter.

Lambda Expressions

Java 8 provides a nice mechanism for doing exactly that. We can form a *lambda expression*.

Lambda Expression (Lambda)

An expression that describes a function by specifying its parameters and the value that it returns.

The term “lambda” was coined by a logician named Alonzo Church in the 1930s. The term is used consistently across many programming languages, so it is worth becoming familiar with it. The Python programming language, for example, uses “lambda” as a keyword for forming this type of anonymous function.

Lambda expressions are formed in Java by specifying the parameters of the function and an expression that represents the value to return separated by the special operator “ ”.

For example, we can use the following lambda expression to represent a function that adds together two arguments:

Notice that the parameters are enclosed in parentheses. In reading this expression, we typically describe it as, “Given the parameters x and y of type int, we return x+y.” We can also write this as a method with a name, as in:

Notice how the lambda expression takes the parenthesized parameter list from the method header and the expression used in

the return statement to form a simple expression. Once you get used to reading lambda expressions, you will find that it is a concise way to read and reason about the underlying computation being performed.

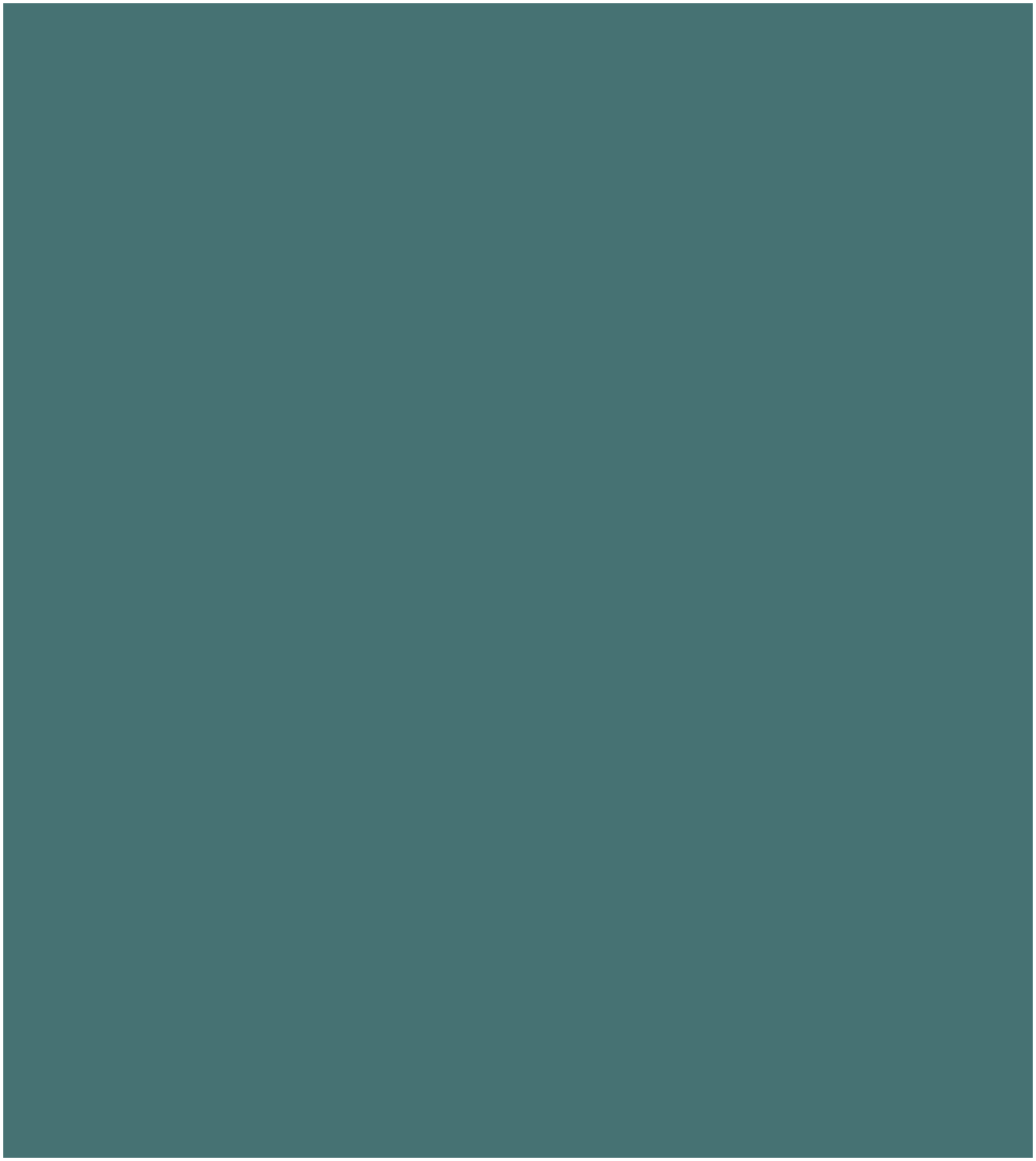
It is also often possible to eliminate the types for the parameters because they can usually be inferred by the surrounding context. For our sample code, we will be able to use this lambda expression to describe addition:

And this expression to describe multiplication:

Given this new option, we can rewrite our client code as follows to perform 3 each of addition and multiplication problems.

```
(x, y) -> x + y


(x, y) -> x * y
```

Below is a sample log of execution.



This ability to pass a lambda expression as a parameter points out the benefit of treating functions as first-class elements of the language. Just as we can provide a different number of problems to perform or a different text to use for displaying the problems, we can also provide a different function for computing the right answer. This is a much more flexible approach than having to write tedious [redacted] constructs that say exactly what to do for each different possibility. Instead we provide a simple definition of the function we want to use and the function is stored in a parameter of the method.

It is more important that you learn how to become a client of these functional programming features of Java than to learn how to implement them yourself. But for those who are interested in seeing the implementation, the following is the revised method code:

```
String text, IntBinaryOperator operator
```

```
text
```

```
operator.applyAsInt(x, y);
```

There are other variations on lambda expression syntax. For example, if a lambda expression accepts only a single parameter, the parentheses around it are not required. The following is a lambda expression that accepts an integer and returns that integer plus 1:

Another syntax variation is that if the computation is not a simple expression, you can include multiple statements enclosed in curly braces, such as:

Our discussion of first-class functions is a little generous to Java because it turns out that functions in Java 8 are not truly first-class. The language designers have done some fancy work behind the scenes to make it feel like Java has first-class functions, but they aren't really first-class because you can't do basic things like storing them directly in a variable. Instead Java takes advantage of interfaces that have a single abstract method in them (known as *functional interfaces*) and constructs an object for you that implements the interface's method using the elements of the lambda expression. As a result, Java's implementation of functional programming is more clunky and restrictive than in a true functional programming language. But for our purposes, the lambda expressions and functional interfaces act enough like first-class functions that we can explore the concept, even though it is a bit of an illusion.

You can try out lambdas in JShell, but the syntax is a bit clunky compared to some other programming languages. The following JShell interaction creates a lambda function called and calls it on two integers:

```
jshell>


jshell>
```

19.3 Streams

You don’t truly appreciate the benefits of functional programming in Java until you explore streams, which are the primary mechanism that Java provides for this style of programming. We have seen this concept before when we studied files in Chapter 6. We saw input streams that are a source of data and output streams that are a destination for results. The streams in this chapter are a generalization of that idea. Oracle describes a Java stream as a sequence of elements of data on which various functional programming operations can be performed.

Stream

A sequence of elements from a data source that supports aggregate operations.

Basic Idea

The best way to think about a stream is to visualize it as a flow of data from a source to a terminator with possible modifiers in between, as shown in Figure 19.1.

Figure 19.1 Streams

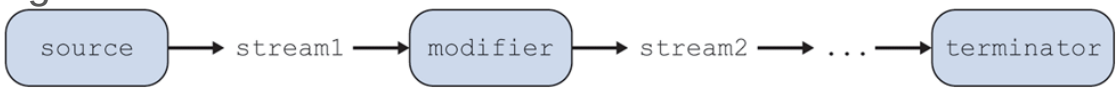


Figure 19.1 Full Alternative Text

There is always one source and one terminator, but there can be any number of modifiers (including none) in between. As the diagram indicates, think of each modifier as transforming the stream in some way. One sequence of values flows in and a different sequence of values flows out. This way, we solve a complex programming task by identifying the source of the data to process, the final result we want to compute, and a series of transformations in between that move us closer to completing the task. Each of the modifiers will be specified by a function, which means that we are decomposing the overall task into a series of subtasks that each involve a single transformation specified by a function.

As a first example, suppose we want to find the sum of the squares of the integers 1 through 5. We could use a classic cumulative sum to accomplish this:

This code specifies exactly how to perform this computation, using a loop variable called [redacted] that varies from 1 to 5 and accumulating the final answer in a variable called [redacted]. You will see that when we use streams, we describe more *what* we want computed instead of specifying *how* to compute it. This can make the coding itself simpler, but more importantly, it gives the computer more flexibility to decide how to implement the computation. As we will see in the case study at the end of this chapter, this can allow the computer to optimize the solution to run faster.

Using a stream approach, we first have to identify a source of data. We don't have a convenient source for the squares of the positive integers, but there is a static method called [redacted] that produces a stream of sequential integers in a particular range. As with the [redacted] method of the [redacted] class, the [redacted] method has a first parameter that is inclusive and a second parameter that is exclusive. So we will make the call [redacted] to produce a stream with the integers [1, 2, 3, 4, 5]. There is a variation of the method called [redacted] that would allow us to pass [redacted] as parameters, but the [redacted] method uses the same convention we have studied in Java for substrings and it is also more commonly used in other programming languages. Python, for example, has a [redacted] function that works the same way.

We also need to pick an appropriate terminator. In this case Java provides one for us in the form of a method called [redacted] that adds up the values in a stream of numbers. For now, let's just add up the integers and store the result in a variable. So we would write this line of code:

This sets the variable [redacted] to 15 (1+2+3+4+5). In this case, we have the required source of data and the required terminator, but there are no modifications along the way. Figure 19.2 shows a diagram of what is going on.

Figure 19.2 Stream operations on range of integers

```
IntStream.range(1, 6) -> [1, 2, 3, 4, 5] -> sum -> 15
```

In our computation, the initial call to [redacted] is the source, and [redacted] is the terminator. The call on [redacted] produces a stream of five integers. This stream is fed into the [redacted] method, which adds them up to produce the final result of 15. But recall that we want the sum of the first five squares, not the first five integers. We can accomplish that by introducing a modifier in between the range creation and the sum operation.

You can see optional values in JShell. It displays that the result is optional, along with its value, if any. If you want to split up long lines with multiple stream operations, you have to type the dot character before you press Enter for JShell to know you haven't finished writing the stream expression yet:

```
jshell>
```

```
jshell>
```

Using Map

Stream objects have a method called [redacted] that takes a function as a parameter. We can provide a function that squares a number. What [redacted] does is to produce a new stream that has the result of applying the given function to each element of the original stream. So if the original stream had five numbers, then the new stream will also have five numbers, but they will be new numbers obtained by calling the function passed as a parameter on each of the five elements in the stream. So our line of code becomes:

```
.map(n -> n * n)
```

There is a convention popular among Java programmers to list each step of this operation on a different line of code. The first line of code should have the source and the final line should have the terminator and any modifications should be listed as separate lines in between. So the code above becomes:

This is just a formatting convention to make it easier to read the code. We would read this in a high-level way as, “To assign the variable , first form the given range of integers, then map the given function over those integers, and then find their sum.” We can update our diagram as shown in Figure 19.3, again using the convention of including one step on each line.

Figure 19.3 Stream operations with

```
IntStream.range -> [1, 2, 3, 4, 5]
          -> map -> [1, 4, 9, 16, 25]
          -> sum -> 55
```

Figure 19.3 Full Alternative Text

As the diagram indicates, it is useful to think in terms of a flow of data. First the call on produces the sequence [1, 2, 3, 4, 5].

This is then passed through the [redacted] modifier that changes the stream by applying the squaring function to each value, generating the new stream [1, 4, 9, 16, 25]. This new stream is then sent into the [redacted] terminator that adds them up to produce the value 55.

This example has the classic structure of one source of data, one modification, and one terminator that computes a result. It is useful to keep in mind these three basic elements of a typical stream computation. But you can have more than one modification along the way, so let's look at an example that has many.

Using Filter

Suppose that instead of using sequential integers, we decide to work with the first 10 digits of *pi*. We can form a specific stream of integers by calling the method [redacted] and listing the individual values, so let's use the same code as before but using those 10 digits:

[redacted]

This sets [redacted] to be 207, as indicated in Figure 19.4.

Figure 19.4 Stream operations on digits of *pi*

```
IntStream.of -> [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
      -> map -> [9, 1, 16, 1, 25, 81, 4, 36, 25, 9]
      -> sum -> 207
```

Figure 19.4 Full Alternative Text

But suppose that we don't want to add up the squares of all of these digits. Suppose we are only interested in the odd digits. We could rewrite the function we used in [redacted] to return 0 for even numbers and the square of the number for odd numbers, but there is a better way. There is a modifier known as [redacted] that can be used to restrict a stream to those values that pass some test. So while the [redacted] modifier gives you a stream of new values of the same length as the original, the [redacted] modifier gives you a stream of unchanged values, but not necessarily of the same length because not all values pass the given test.

The [redacted] method takes as its argument a Boolean-valued predicate. In our case, we will test that the remainder when divided by 2 is not 0.

```
.filter(n -> n % 2 != 0)
```

Updating our diagram for the flow of data we get the result shown in Figure 19.5.

Figure 19.5 Stream operations with

```
IntStream.of -> [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
   -> filter -> [3, 1, 1, 5, 9, 5, 3]
   -> map    -> [9, 1, 1, 25, 81, 25, 9]
   -> sum    -> 151
```

Figure 19.5 Full Alternative Text

We now have two different stream modifiers; let's throw in a third. Suppose that we don't want to include any given number more than once in computing this sum. That would normally require some kind of structure like a set to remember values that we have seen so as to eliminate duplicates. But Java provides a specific modifier for this known as . The stream that it produces will have values from the original stream in the same order, but with all duplicates removed (only the first occurrence of each value will be included). So we can add this to our code to get:

```
.distinct()
```

Figure 19.6 shows an updated diagram of this new code.

Figure 19.6 Stream operations with

```
IntStream.of -> [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
 -> filter   -> [3, 1, 1, 5, 9, 5, 3]
 -> distinct -> [3, 1, 5, 9]
 -> map      -> [9, 1, 25, 81]
 -> sum      -> 116
```

Figure 19.6 Full Alternative Text

This is a fairly complex operation that we have described without saying much about how it is accomplished. We are requesting the sum of the squares of the odd numbers of this sequence after duplicate numbers have been removed. The stream mechanism and these individual modifiers take care of the rest of the work for us.

As another example, suppose we want to write a method to test whether a number n is prime. By definition a prime is a number that is divisible by 1 and itself but nothing else. Another way of saying that is that primes have exactly two factors. We can express that using streams by producing the values 1 through n, filtering on factors, and seeing whether there are exactly two of them.

Remember that the second parameter to [redacted] is exclusive, which is why we use n+1 to have the stream go all the way up to and including n. It is worth considering the special case of 1. This method says that it is not prime, because there is only one value between 1 and 1 that is divisible by 1. That is the right answer because the value 1 is not considered a prime. This computation is inefficient because it only needs to test values up to the square root of n, but we will delay that discussion for the case study at the end of this chapter.

Using Reduce

Now let's consider the task of computing the factorial of an integer n, which is defined as the product of the integers 1 through n. This is a slightly different task and requires a slightly different approach. In this case, we want to take all of the integers in a stream and combine them into one integer using multiplication. There is a special stream terminator known as [redacted] that combines elements from a stream. Below is a method that computes the factorial using this approach.

You will notice that the call on [redacted] begins with an extra parameter [redacted]. This is the starting value to use for the computation. Multiplying by 1 does not change the overall result and it allows the expression to guarantee that it returns a value even if the stream is empty. That means that 0 factorial is correctly reported as 1. It also means that negative factorials are reported as 1 which we could fix with a test that throws an exception in that case because the factorial is undefined for negatives.

There are many details we have left out about streams. For example, we have not yet discussed the problem that some computations do not produce a result. You can generally compute an average or a maximum or a minimum of a sequence of values, but not if the sequence is empty. Java has a notion of an option type that sometimes has a result and sometimes does not, which we will briefly explore in the next section. We will close with a simple variation of our computation on the first ten squares that uses a built-in terminator known as [redacted] in place of [redacted]:

which produces the following output:

Each of these statistics has a corresponding terminator, such as and . You can read more about them and the other modifiers and terminators in the documentation for the class.

Optional Results

Some of the provided stream terminators have a subtlety that merits investigation. For example, if you want to find and print the largest multiple of 10 in a certain group of integers, you might perform the following stream operations to filter out all the non-multiples of 10 and then print the largest of the remaining numbers:

But you'd find that the preceding code generates a compiler error such as the following:

This is because the results of some stream terminators like [redacted], [redacted], [redacted], and [redacted] are undefined if the stream does not contain any data. What if there were no multiples of 10 after doing the filtering? There would be no maximum to return. The terminator could have chosen a default value like 0, but this is not guaranteed to be what the programmer would want.

For this reason, those terminators do not actually return the type of result you would expect, such as an [redacted] or [redacted]. The designers of the library needed a way to represent that the result might be undefined. So these terminators actually return values of types called [redacted] and [redacted]. These "optional" types are small wrapper objects for storing a single [redacted] or [redacted] result respectively. Each type has a corresponding method called [redacted] and [redacted] to extract the value. Those methods will throw a [redacted] if the value is undefined. (There is also a more general optional type called [redacted] that is outside the scope of this chapter.)

The following code properly stores the largest multiple of 10 as an integer:

```
.getAsInt()
```

A full discussion of these optional types is outside the scope of this chapter, but you can read more about them in the Java 8 API documentation.

19.4 Function Closures

We have seen that Java has extensive rules about the scope of variables that guarantees that every reference to a variable will work out. Consider, for example, the following program that attempts to count how many factors of 10 there are.

```
public class

public static boolean

int
```

```
return
```

```
public static void
```

```
int
```

```
int
```

```
for
int
```

```
if
```

This program does not compile. Java complains that the variable [redacted] referred to in the method [redacted] is not defined. You might argue that it is defined in the [redacted] method and is set up before the function is called. But as we have seen, Java has strict rules about this and requires that the variable [redacted] be visible in the scope of the method or in the outer class scope. Being available in the scope of a different method isn't good enough.

The usual fix is to pass [redacted] as a parameter:

```
```

```
public class


public static boolean

int

int n


return
```

```
public static void



int



int



for

int



if

n
```

This program now compiles and correctly reports that there are four factors of 10 (1, 2, 5, and 10). But consider the following code that uses a stream, a call on [redacted], and a call on the terminator [redacted] which returns a value of type [redacted].

It also correctly reports that there are four factors of 10. There is similar code used in the [redacted] method included in the previous section. But how does it work? Think about the lambda expression being passed as a parameter to [redacted]:

This is supposed to represent an independent function. In fact, it looks a lot like the [redacted] method from the [redacted] class. It has a single parameter called [redacted] and it returns a [redacted] value based on whether the variable [redacted] is divisible by [redacted]. It has access to the variable [redacted] because it is the parameter, but how does it have access to the variable [redacted]?

You might say that you can see the variable [redacted] defined in the scope in which this lambda expression occurs and that is the key to understanding this, but it is important to recognize that something special is going on here. Remember that you are passing a function to the [redacted] method that it will execute. That [redacted] function is in a different class that doesn't have access to the variable [redacted] that is defined here. So somehow the value of [redacted] is being passed along with the code.

When you form a lambda expression, some of the variables you refer to will be included as parameters to the function. We call such references *bound variables* because they are each connected to a parameter. Any reference that is not to a parameter in a lambda expression is considered a *free variable*. Variables cannot be completely free of definition because then they would be undefined. But because a lambda expression occurs in the middle of code that potentially has variables defined, the lambda expression can refer to free variables that are defined in its outer scope.

Bound/Free Variable

In a lambda expression, parameters are bound variables while variables in the outer containing scope are free variables.

In our sample lambda expression, the outer code defines a local variable called , which means that the reference to in the lambda expression makes sense. But Java has to perform some work behind the scenes to include this variable definition along with the code. We refer to such a combination as a *closure*.

Function Closure

A block of code defining a function along with the definitions of any free variables that are defined in the containing scope.

Let’s use another example that goes to an extreme to make the point. Suppose we have a method called that takes a function of two arguments as a parameter. We might call it like this:

This lambda expression refers to and which are both parameters, which means they are bound. There are no free

variables in this expression. But suppose the code instead had been:

This lambda expression has references to [redacted] and [redacted] just like the other one, but those aren't a problem because they are bound because they are parameters. But it also has references to variables called [redacted], [redacted], and [redacted], which are not parameters of the lambda expression, which means they are free variables. Java has to include those definitions along with the code in order for the compute method to be able to do its job.

For a visual representation, imagine that Java puts together the code of the lambda expression along with the definitions of the parameters and these three free variables into one big object, as shown in Figure 19.7.

Figure 19.7 Diagram of a function closure

```
parameters     : (x, y)
free variables: min = 10, max = 50, multiplier = 3
code           : Math.max(x, min) * Math.max(y, max) * multiplier
```

Figure 19.7 Full Alternative Text

This object is the closure. It contains both the code and the relevant context in which the code occurs. The free variables [redacted], [redacted], and [redacted] have been included so that the code can be executed. We say that they have been *captured* in forming this closure.

The word closure comes from mathematics (e.g., the concept of a transitive closure), but here is a way to think about it that might be helpful to remember the concept. We all seek closure in our lives, which means that we don't want any loose ends from our past left around to upset us. Free variables are loose ends in a lambda expression that would make it impossible to evaluate the code. By including their definition in the closure, we are tying up those loose ends so that there isn't anything left that is undecided. Once we've done that, we've reached a kind of closure with this computation.

We run into potential problems if any of the free variables included in a closure change in value. If the value of a free variable changes, then the computation isn't well defined. For example, it means that the order in which we perform the individual computations for a stream can change the overall result. This relates to the concept of effect-free programming described earlier in this chapter. We want to write code that does not have these potential sources of inconsistency.

Java requires that any free variable included in a lambda expression has to effectively be a constant. The easiest way to guarantee this is to use values that are defined with the keyword [redacted]. But the Java compiler will allow you to use ordinary variables as long as your code never assigns those variables a value more than once. This is a helpful restriction, but it still leaves open the possibility that you will have a reference to an

object whose state changes. That leads to unpredictable results and should be avoided whenever possible.

19.5 Higher-Order Operations on Collections

You may have noticed that methods like [redacted], [redacted], and [redacted] are functions that accept other functions as arguments. As a result, they are known as higher-order functions.

Higher-Order Function

[redacted]

A function that takes another function as an argument.

We have already discussed these higher-order functions previously with examples of streams composed of simple [redacted] values. In this section, we will explore how similar operations can be performed on arrays and other Java collections. This section also discusses stream terminators that collect stream elements into a collection.

Working with Arrays

Any array can serve as the source for a stream by passing it as a parameter to the method [redacted]. For example, suppose that you have an array of simple [redacted] values defined as follows:

Suppose that we want to find the sum of the absolute values of the even integers in this list and we want to exclude any duplicates. That sounds a lot like what we were doing before and we can do it in the same way but using the array as the source of data.

We can once again use a diagram to explore what happens on each step of this computation, as shown in Figure 19.8.

Figure 19.8 Stream operations on an array

```
Arrays.stream -> [3, -4, 8, 4, -2, 17, 9, -10, 14, 6, -12]
  -> map      -> [3, 4, 8, 4, 2, 17, 9, 10, 14, 6, 12]
  -> filter   -> [4, 8, 4, 2, 10, 14, 6, 12]
  -> distinct -> [4, 8, 2, 10, 14, 6, 12]
  -> sum      -> 56
```

Figure 19.8 Full Alternative Text

Order can matter for these components that modify the stream. For example, if we call [redacted] before we use [redacted] to convert to the absolute value, then we would miss the fact that there is a -4 and a 4 in the original list. There is a minor efficiency improvement in performing the [redacted] operation before the [redacted] operation because [redacted] is easier to perform on a shorter sequence, which is what [redacted] provides.

One of these lambda expressions is of particular interest. We call map passing it an expression that converts a number to its absolute value:

[redacted]

Think about this expression. It describes a function that takes a value [redacted] as a parameter and that returns the value of [redacted]. But isn't that exactly what [redacted] already does? In other words, [redacted] is the function we want to pass to [redacted]. We don't need to make a new function that is basically a second version of [redacted]. Java allows you to avoid this redundancy by directly referring to the [redacted] method. We use the name of the class followed by two colons followed by the name of the method. This syntax in Java is called a *method reference*. That means that our code can instead be written as:

```
Math::abs
```

Method reference

A syntax to refer to a method so that it can be used as part of a functional expression.

Things work slightly differently for arrays of objects, but we'll explore that in the next section on working with lists. Java cares about the values in a stream but not where those values come from. So a stream whose source is an array of objects would be manipulated in exactly the same way as a stream whose source is a list of objects.

It is also worth noting that the various stream classes have a special terminator called that collects the stream contents into an array. For example, the following code:

```
int[] sublist
```

```
.toArray()
```

will set the variable [redacted] to the following array:

Working with Lists

As noted earlier, Java doesn't care what the source of a stream is. It only cares about the kind of data in the stream. But so far we have only looked at streams that contain simple [redacted] data. Lists store objects, so they tend to be manipulated in a slightly different way.

The [redacted] interface now includes a method called [redacted] that can be used to create a stream of values from a list. For example, suppose that we want to process a list of words. We can use the [redacted] method to create a list of specific words to manipulate:

Suppose that we simply want to print these words on a line of output. We could use a for-each loop to do so:

This code produces the following output.

Think about what is going on here. We are using a for-each loop to go through every element of the list and to perform some specific task on each of those elements. We can accomplish the same effect by creating a stream from the list and calling the method of the stream class to perform the same task expressed using a lambda expression. The stream method

[redacted] accepts a lambda that, in turn, accepts a single element from the stream and performs some action on it.

It seems odd to use [redacted] instead of [redacted]. Isn't this a mapping operation? The answer is yes and this is one of the cases where Java is showing it's clunky implementation of functional programming. There are two reasons that we need to use [redacted] instead of [redacted]. Remember that [redacted] is a modifier that takes one stream and produces a new stream. We are calling the [redacted] method, which has a [redacted] return type. Because it doesn't return anything, it would not be an appropriate method to use for a [redacted] operation. More importantly, each stream needs a terminator. In this case, the application of the [redacted] method is the last thing we want to do with the stream, so we need a terminator rather than a modifier. It's not a bad idea to think of [redacted] as a variation of [redacted] that serves as a terminator rather than a modifier.

We can make this code a bit more functional by noticing that we are modifying each string to have a space in front of it and then applying the print method to each of those strings. We can describe this more clearly with a call on [redacted] that modifies each

string and then a call on the [redacted] method that can be referred to directly using a method reference:

```
.map(s -> " " + s)

System.out::print
```

As usual, we can add more modifiers to this stream computation. For example, we can apply the [redacted] method to eliminate any differences in case, the [redacted] method to eliminate duplicates, and the [redacted] method to put the results into sorted order.

```
.map(String::toLowerCase)

.distinct()
```

```
.sorted()
```

This code produces the following output:

```
```

There is enough going on here that a diagram is again helpful to understand each individual step. Figure 19.9 shows the various steps in the computation.

Figure 19.9 Stream operations on a list of strings

```
words.stream -> ["To", "be", "or", "Not", "to", "be"]
  -> map     -> ["to", "be", "or", "not", "to", "be"]
  -> distinct -> ["to", "be", "or", "not"]
  -> sorted   -> ["be", "not", "or", "to"]
  -> map      -> [" be", " not", " or", " to"]
  -> forEach  -> prints: " be not or to"
```

Figure 19.9 Full Alternative Text

Let's explore one last example that shows one of the potential pitfalls that comes up when working with streams. Suppose we want to know the sum of the lengths of the strings in the original list. We can apply the [redacted] method of the [redacted] class to each

of the strings and then ask for the sum of those values. This seems fairly straightforward:

Unfortunately, this code does not compile. It says that there is no method called [redacted] for the given stream. But doesn't the [redacted] method of the [redacted] class return an [redacted]? So wouldn't the call on [redacted] produce a stream of [redacted] values? If so, it should have the [redacted] method we were using in the previous section.

The problem is that Java has different stream types and these types are preserved by modifying components unless you explicitly tell Java to change the stream type. The source is a stream of objects, so the [redacted] command will produce a stream of objects. In particular, it boxes each of the [redacted] values returned by the [redacted] method and gives you a stream of [redacted] objects. If you really want it to produce a stream of simple [redacted] values, you have to instead call the [redacted] method that changes the stream type:

This version of the code works properly, setting the variable [redacted] to [redacted]. Making a diagram is again helpful to consider exactly what is happening here, as shown in Figure 19.10.

Figure 19.10 Stream operations on a list of strings mapped to integers

```
words.stream -> ["To", "be", "or", "Not", "to", "be"]
  -> mapToInt -> [2, 2, 2, 3, 2, 2]
  -> sum      -> 13
```

Figure 19.10 Full Alternative Text

Notice that the original stream contains strings, but after the call on [redacted], we have a stream of [redacted] values. That stream can be terminated with a call on [redacted], as we did in the examples in the last section.

As noted earlier, this short chapter is not meant to cover all of the details of these Java 8 constructs. We are exploring the important concepts with a representative subset of these operations. But if you keep in mind the idea of streams changing their types, then you will probably understand why there are methods with names like [redacted], [redacted], [redacted], and [redacted] and why it is sometimes handy to call the modifier called [redacted] that converts a stream of primitive values into its corresponding wrapped objects.

We saw that you can collect the values in a stream into an array by using the terminator called [redacted]. There is a similar method called [redacted] that allows you to collect the results in a collection. You need to construct an appropriate [redacted] object by

calling an appropriate method such as [redacted] or [redacted]. For example, using our words example from before, we can convert all of the words to lowercase and collect the results in a set.

This code produces the following output:

If you want to specify exactly which type of collection to produce, you can do so with a call that passes in a reference to the constructor of the appropriate class. For example, if you want to guarantee that you get a [redacted] that keeps the keys in sorted order, you would instead say:

In which case the output becomes:

As with the other examples, we are only scratching the surface here of the details of how all of this works, but these examples serve as good indicators of the kind of operations that can be performed using streams.

As you can see from the examples in this section, functional style programming is particularly helpful for processing of collections like arrays and lists. The trio of higher-order functions , , and provide a particularly powerful set of tools for solving a wide variety of computational tasks on collections.

Working with Files

Java 8 provides a new facility for reading the lines of a file into a stream of strings. Importing the package will give you access to the class, which contains a static method that returns a stream of the lines of a file. The syntax is the following:

The [redacted] method throws an [redacted] if the file does not exist or cannot be read, so you must surround it with a [redacted] block or use a throws clause in your method heading. For example, suppose we have a file in the current directory named [redacted] that stores the following three lines of text:

The [redacted] method would return the file's three lines as a stream containing three string elements. The following code would read and print every line of [redacted] using a stream:

If you want to process each line, the standard stream operations like [redacted], [redacted], and [redacted] can be used on the strings in the stream. For example, the following code finds out the length of longest line in the file:

Figure 19.11 summarizes the stream operations and their results. Recall that we need to use [redacted] at the end of our stream calls because the [redacted] terminator returns an [redacted] to represent the possibility that there might be no lines in the file.

Figure 19.11 Stream operations on the lines of a file

```
Files.lines -> ["haiku are funny",
                "but sometimes they don't make sense",
                "refrigerator"]
-> mapToInt -> [15, 35, 12]
-> max      -> 35
```

Figure 19.11 Full Alternative Text

19.6 Case Study: Perfect Numbers

To complete our exploration of functional programming, we will look at a classic problem from mathematics that is known to take a lot of computational power to solve. We are going to write a program that looks for perfect numbers. A perfect number is defined as one that is equal to the sum of its divisors other than itself. For example, the divisors of 6 are [1, 2, 3, 6]. If you exclude 6, the other divisors add up to 6(1+2+3). In fact, 6 is the smallest perfect number. The next perfect number is 28 whose divisors are [1, 2, 4, 7, 14, 28].

If you search for “perfect number” in your web browser, you will find several web sites that chronicle the fascination that many people have had over the years with this concept. Some ancient Greeks believed that the world was created in 6 days because 6 is the first perfect number. St. Augustine repeats this claim in his writings. The ancient Greeks also believed that the moon completes an orbit every 28 days because 28 is the second perfect number.

The first four perfect numbers have been known since ancient times, but the earliest known references to the fifth perfect number date from the fifteenth century. In this section we will write a program to find the fifth perfect number. In doing so, we will be able to explore one of the biggest benefits of functional programming: that it can speed up the execution of programs that require a lot of computational power.

Computing Sums

This problem, as with many problems from mathematics, lends itself naturally to a functional approach. In this first version, let's write code that finds the sum of the divisors of a number not including the number itself and include debugging code that will allow us to have confidence that it is working correctly.

We can use [redacted] to produce a stream of integers and filter it with a test for divisibility and then compute the sum. Let's put it into a method so that we can call it easily:

Normally we would have range go up to [redacted] so that it will include [redacted], but in this case we want to exclude [redacted] from the sum because we are looking for the sum of the divisors other than the number itself.

In the [redacted] method, we can compute the sums for the first 10 integers and print them out again using a call on [redacted] and a call on [redacted] to apply our function:

Notice that we terminate this stream with a call on [redacted] that prints each number with a space after it. This will produce an extraneous space at the end of the line, but that's not a problem for us at this stage in development. Also notice that the call on [redacted] takes a method reference to the function to be applied. We use the name of the class (the one we are writing) along with the double colon operator and the name of the method from this class that we want to call.

The code above produces the following output:

We could hand-check that these values are correct, but it is helpful to build some debugging code into our solution that will allow us to see more clearly exactly what is going on in this computation. Java provides a special method called [redacted] that is particularly helpful for this kind of debugging. In some sense, the [redacted] method has no effect on a stream at all. It produces as

output the same stream that it is given as input. But it allows you to pass it a function to be executed on each value in the stream. In our case, we would like to do some printing of individual stream values. We see 10 numbers above and can figure out which one goes with each input value to the function, but we can use [redacted] to confirm this more directly.

The code below includes a call on [redacted] that prints the number and an equals sign before it maps that number to its divisor sum and prints it.

```
.peek(n -> System.out.print(n + "="))
```

It produces the following output.

```
[redacted]
```

This output indicates that the sum of divisors for 1 is 0, the sum of divisors for 2 is 1, the sum of divisors for 3 is 1, and so on. This

sounds wrong, but remember that we are excluding the number itself from these divisors. We can make this even clearer by adding a call on to our stream that is computing divisors:

```
.peek(x -> System.out.print(x + ","))
```

This version of the code produces the following output:

This output is a little hard to read as well, but keep in mind what we have added. As the method that sums up the divisors encounters a new divisor, it prints it with a comma afterward. For example, when it is finding the divisors of 10 it finds the divisors 1, 2, and 5. So it produces the output “1,2,5,” which includes an extra comma at the end. Then the code we have in prints the actual result, which is 8. We can see that 1+2+5 is equal to 8,

so we know that this output verifies that the code is working properly.

It is useful to look at a diagram for the specific computation of the divisors of 10. Figure 19.12 shows the steps of the stream computation.

Figure 19.12 Stream operations to compute sum of divisors of 10

```
IntStream.range -> [1, 2, 3, 4, 5, 6, 7, 8, 9]
      -> filter -> [1, 2, 5]
      -> peek   -> [1, 2, 5] with side effect of printing "1,2,5,"
      -> sum    -> 8
```

Figure 19.12 Full Alternative Text

As the diagram above indicates, the call on [redacted] is producing a side effect. Normally we want to write code that is effect-free without such side effects, but in this case it is a useful way to make the computation more visible for debugging purposes.

Consider a similar diagram for the overall computation being performed by the [redacted] method. Figure 19.13 shows the stream operations.

Figure 19.13 Stream operations to compute sum of divisors of 10

```
IntStream.range -> [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
     -> peek    -> [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
                   with side effect of printing "1=2=3=4=5=6=7=8=9=10="
     -> map     -> [0, 1, 1, 3, 1, 6, 1, 7, 4, 8]
     -> forEach -> prints to System.out: "0 1 1 3 1 6 1 7 4 8"
```

Figure 19.13 Full Alternative Text

Figure 19.13 is confusing. It implies that it prints all of the text from the call on [redacted] before it calls [redacted] and only later prints all of the answers through the call on [redacted]. A diagram can only convey so much of a complex process like the processing of a

stream. The diagram is showing the resulting stream on each step, but that is different from the order of operations. This can get very tricky to understand, but here is a brief explanation. Nothing happens until Java encounters the terminator. When it gets to the call on [redacted], it works backward to get a value from the stream to process. It goes all the way back to the source which was the call on [redacted]. The [redacted] method sends out the value 1, which is given to [redacted] to print “1=”, and then it passes to [redacted] which converts it to 0, and then it makes it back to [redacted] that prints the 0. At this point in time we have as output “`1=0`”. Then the [redacted] method asks for another value and we begin the process over again using the value 2, then 3, then 4, all the way up to 10. So the right way of thinking of this is that individual values of the stream pass through each of the different modifiers in sequence before the next value from the stream is processed.

There is a great moral of the story here. The main reason this is confusing is that the output generated by [redacted] is mixed in with the output generated by the terminator in a surprising way. And that output generated by [redacted] is a side effect. If we restrict ourselves to effect-free code that doesn’t have side effects, then we rarely need to think about these issues of ordering.

Below is the complete code for this version that prints the sums for the numbers 1 through 10 including debugging information.

```
import

public class

public static void
```

```
public static int

int




return
```

Incorporating Square Root

It seems like we are almost done. We can modify [redacted] so that it looks at a lot of integers (say, up to 1 million) and filters on those that are equal to their sum of divisors. We can also use the [redacted] method introduced in Chapter 13 to keep track of how much time we spend computing the result. We can temporarily turn off our debugging code by commenting it out and include the following code in [redacted].

When one of the authors ran this on his laptop computer, it reported taking 2260.112 seconds to complete, which is over 37 minutes. It correctly reported the first four perfect numbers, but it didn't manage to find the fifth. And it isn't going to find the fifth in a reasonable amount of time.

We need to improve the efficiency of our computation. The easiest way to do this is to notice that each divisor that is less than the square root of a number is paired with a divisor that is greater than the square root. If, for example, we are looking for the divisors of 100, we find that the divisor 1 is paired with 100, the divisor 2 is paired with 50, the divisor 4 is paired with 25, and so on. So instead of finding each divisor by checking all the way up to the number, we can instead find pairs of divisors by checking up to the square root. If you search for the divisors in this way, for each divisor x you find, you know that you have also found its pair, whose value is n / x; so our modified code contains an additional call to [redacted] that adds in each divisor's pair to our sum.

Remember that we want to exclude the number n itself from the sum. It is always paired with 1, so an easy way to do this is to start our search for divisors starting at 2 rather than 1. If we do that, however, then we have to remember to add 1 to the final result because 1 is supposed to be included in the sum. The code below implements this strategy and updates the call on [redacted] to report the pairs of divisors.

We can go back to our old client code that shows output for the numbers 1 through 10, which produces the following output.

This output is different from before in that it doesn't include 1 at the beginning of each list of divisors. That's because we are handling 1 as a special case, as described previously. But there is something else to notice. It is getting the wrong answers for some of these values. In particular, it has the wrong answer for 1, 4, and 9. You might be able to notice what those three numbers have in common, but you can also figure this out by looking at one of them in detail. Let's take 4. Its divisors other than itself are 1 and

2, so that should add up to 3. Instead we are getting an indication that it has a pair of divisors 2 and 2 and that it adds up to 5 when you include the special case of 1.

The problem comes for numbers that are perfect squares. It is true that every divisor that is strictly less than the square root of a number has a different divisor that it is paired with that is strictly larger than the square root. But for numbers that are perfect squares, their square root isn't paired with another divisor. This is a special case that we can handle after the main computation of by adding the following code.

Below is the complete program that checks numbers up to 1 million. This new version takes only 4.493 seconds to run, which is quite a bit better than 37 minutes.

```
import

public class

public static void

long
```

```
double
```

```
public static int
```

```
int
```

```
int
```

```
int
```

```
int

if

return
```

Just Five and Leveraging Concurrency

We are basically ready to see if we can actually find the fifth perfect number. We have checked up to 1 million and didn't find it. We could try going up to 10 million or 100 million or some other large number. But Java provides a nice alternative.

We have been using a call on [redacted] to specify a specific range of numbers to use. We can instead use the method [redacted] to produce a stream containing all positive integers. The [redacted] method takes a starting value and an update function. So the source of our stream will be the following call.

This is potentially an infinite stream of numbers because we say to start at 1 and to update by replacing n with n+1, but we never say how it stops. When you work with a stream like this, you have to include a special modifier that limits how many values it produces. Not surprisingly, this modifier is called [redacted]. The following code prints the first five integers that are equal to the sum of their divisors (in other words, the first five perfect numbers):

This is another place where it is important to understand in general the order of operations. If you think of the call on [redacted] as producing all of its result before you ever get to the call on [redacted], then this code makes no sense. You have to understand this code in terms of individual values being generated by [redacted] and then passed to [redacted]. When one of those values passes the test, it is passed on to [redacted] and then to [redacted] which prints it. After the [redacted] method has produced five values, it stops requesting new values to be checked from the [redacted] method.

We can throw in a call on the modifier [redacted] because it makes the [redacted] method easier to optimize so that it potentially runs a bit faster. Combining all of this and running the program with the timing code, it ends up producing output like the following. The program finds the fifth perfect number and takes less than 11 minutes on one of the authors' laptops.

But we can do even better. Computer scientists have realized for a while now that speeding up a single processor cannot be achieved indefinitely and that eventually we will have to leverage the power of concurrency. It is typical now for laptops to be dual-core or quad-core or some other form of multi-core where there is more than one processor. But it isn't easy to take advantage of multiple processors.

The pitfalls of concurrency are beyond the scope of this discussion, but think about something as simple as preparing a meal to be served at a restaurant. It is easy to see how one chef would complete the task by doing everything. Adding additional chefs might be helpful if you can divide up the task somehow. For example, one might prepare an appetizer while another prepares the main dish and a third prepares a dessert. Even in this case the timing matters because if the dessert is completed quickly and the main dish comes out last, then it is possible that the dessert will spoil because the customer won't want to eat it until the end of the meal. And if you try to have a dozen cooks work on the meal you generally end up with the well-known disaster of "too many cooks in one kitchen."

When Google was faced with the problem of performing massive calculations over vast databases, they quickly turned to concurrency with multiple machines to speed things up. They realized that many computations can be decomposed into a mapping operation and a reducing operation similar to the

approach we have described in this chapter. For example, if you want to count how many web pages have a particular search phrase, you can map a function over the pages that returns 0 or 1 depending on whether the search term appears in that particular page and then you can reduce all of those 0s and 1s using simple addition. They built a system that they call MapReduce that uses exactly this approach. Typical computations are executed with hundreds of processors working on each problem. An open source version of their system called Hadoop has also been popular for applying concurrency effectively to large-scale computations.

The increasing importance of concurrency has led many to realize that functional programming is a powerful way to take advantage of parallel computation. Its emphasis on effect-free programming allows it to avoid many of the common pitfalls of running code on multiple processors. The model of expressing a computation in terms of mapping, filtering, and reducing lends itself naturally to a concurrent solution because usually these operations can be performed in parallel without affecting the overall result.

Many people enjoy functional programming because they like to express problems in a functional manner. Others appreciate the fact that you can write short code that has a certain elegance to it because complex computations can be expressed very concisely. But in the world of modern computing, the truly compelling reason to study functional programming is that it provides a practical solution to the problem of taking advantage of concurrency. In other words, concurrency is the “killer app” for functional programming that convinced the IEEE/ACM joint task force among others that every undergraduate majoring in computer science needs to understand the basics of functional programming.

It is therefore fitting to end this chapter by noting that the final version of the program has one minor change. We insert a modifier called [redacted] in the block in [redacted] that is searching for perfect numbers:

```
.parallel()
```

This extra call tells Java that it would be okay to perform this computation in parallel. It's okay for Java to use as many cooks as it has available to work on the individual problems. Imagine, for example, that a thousand different processors are working on this. There is no reason you can't say, "Processor #1, you figure out whether 1 is a perfect number while processor #2 figures out whether 2 is and processor #3 figures out whether 3 is, etc., etc."

Notice, again, that this style of programming is characterized by the lack of specificity of how it is to be accomplished. We describe the source of the numbers to examine, the test to perform, the fact that we want five answers, the fact that we want to print each one, and the fact that the computations can be performed in parallel. But we leave it up to the computer to figure out how best

to optimize the actual computations. Many programmers resist this style because many of us are control freaks who want to say exactly how everything is done. It turns out that loosening the reins and allowing the computer to optimize it for us often leads to much better results if we have expressed what we want in a way that preserves flexibility for how to get there. That is one of the things that functional programming is best suited for.

When this final version was run on the same laptop, it produced the following output on one of the authors' computers.

This is 3.7 times faster than it was before. Not surprisingly, the author's laptop is a quad-core machine with four different processors. The final program appears below.

```
import

public class

public static void
```

```
long
```

```
double
```

```
public static int

int


int

int


int
```

```
if

return
```

Chapter Summary

Functional programming is a style that emphasizes the use of functions or methods to decompose problems. Java 8 added new constructs to the language to support functional programming.

A side effect is a change made to the state of a program that occurs when a function or method is called, such as modifying a global variable or printing output. Functional programmers try to avoid side effects as much as possible.

A first-class function is one that can be treated like other types of data, such as being passed as a parameter or composed with other functions.

Java provides a shorthand syntax for defining anonymous functions called lambda expressions or lambdas for short.

A stream is a sequence of elements from a data source that supports aggregate operations. An array, collection, string, range of integers, or many other sources of data can be converted into a stream.

Typical operations performed on a stream include map (apply an operation to each element), filter (keep or remove some elements

based on various criteria), and reduce (combine multiple elements into a single element).

A closure is a function definition along with the definitions of any variables declared outside the function (“free variables”) that the function utilizes.

A higher-order function is one that accepts another function as an argument. Java supports a limited form of higher-order functions through a feature called method references.

Self-Check Problems

19.1 Effect-Free Programming

1. Why do functional programmers want to avoid side effects?
2. Why is calling [redacted] considered a side effect? Does this imply that calling [redacted] is a bad thing?
3. What side effect does the following function have? How could it be rewritten to avoid side effects?

4. Rewrite the [redacted] program from this section so that it does not contain any side effects. Rather than modifying a global variable, make the function accept the value of [redacted] to use as a parameter.

19.2 First-Class Functions

5. What change must be made to the math drilling program from this section for it to support subtraction problems?
6. Write a lambda expression that converts an integer into the square of that integer; for example, 4 would become 16.
7. Write a lambda expression that accepts two integers and chooses the larger of the two; for example, if given 4 and 11, it would return the 11.
8. Write a lambda expression that accepts two strings representing a first and last name and concatenates them together into a string in “Last, First” format. For example, if passed [redacted] and [redacted], it would return [redacted].

19.3 Streams

9. Is a stream the same as an array? How are they similar, and how are they different?
10. What value is stored into the variable [redacted] by the following code?

[redacted]

11. Write a piece of code that uses stream operations to compute the sum the negations of a stream of integers. For example, if the stream contains [redacted], the sum to compute is −2+−4+1+−8=-13.
12. Write a piece of code that uses stream operations to count the number of even integers in a stream. For example, if the stream contains [redacted], there are 4 even integers.
13. What value is stored into the variable [redacted] by the following code?

[redacted]

14. The following code does not compile. Why not? How must it be modified?

[redacted]

19.4 Function Closures

15. What is the difference between a free variable and a bound variable?
16. What are the free variables and bound variables in the lambda function in the following code?

17. The following code does not compile. Why not? What is upsetting the compiler?

19.5 Higher-Order Operations on Collections

18. What is the output of the following code?

19. Modify the code from the previous problem so that it does not print any duplicate values.
20. Suppose you have an array of integers called [redacted]. Write a piece of code that uses stream operations to make a new array called [redacted] that stores only the positive integers from [redacted].
21. Suppose you have a list of strings declared as follows. Write code to use stream operations to print all of the four-letter words in the list.

22. Write code to print all lines from the file [redacted] that are at least 40 characters long.

23. The following code contains four problems causing it to fail to compile. What are they?

Exercises

1. Write a method [redacted] that uses stream operations to print twice the value of each element of array of integers. For example, if the array passed is [redacted], print `4 -2 8 32`.
2. Write a method [redacted] that uses stream operations to compute the sum of the absolute values of an array of integers. For example, the sum of [redacted] is [redacted].
3. Write a method [redacted] that uses stream operations to find and return the largest even number from an array of integers. For example, if the array is [redacted], your method should return [redacted]. You may assume that the array contains at least one even integer.
4. Write a method [redacted] that uses stream operations to compute and return the sum of the areas of a group of circles, rounded to the nearest whole number. Your function accepts an array of real numbers representing the radii of the circles. For example, if the array is [redacted], return `289.0`. Recall that the area of a circle of radius r is π r2.
5. Write a method [redacted] that uses stream operations to count how many numbers in a given array of integers are negative. For example, if the array is [redacted], return [redacted].
6. Write a method [redacted] that uses stream operations to convert a [redacted] parameter into its “Pig Latin” form. For this problem we’ll use a simple definition of Pig Latin where the first letter should be moved to the end of the

word and followed by “ay.” For example, if the string passed is [redacted], return [redacted].

7. Write a method [redacted] that uses stream operations to count the number of vowels in a given string. A vowel is an A, E, I, O, or U, case-insensitive. For example, if the string is [redacted], there are seven vowels.
8. Write a method [redacted] that uses stream operations to convert a [redacted] parameter into a sorted form with its letters in alphabetical order. For example, if the string passed is [redacted], return [redacted].
9. Write a method [redacted] that computes the standard deviation of an array of real numbers. The formula for computing a standard deviation σ of N values is the following, where xi represents each ith element and μ represents the arithmetic mean (average) of all the elements:

$$\sigma = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$$

10. Write a method [redacted] that accepts a [redacted] of strings as its parameter and uses stream operations to return a single string consisting of the list’s elements concatenated together in reverse order. For example, if the list stores [redacted], you should return [redacted].
11. Write a method [redacted] that accepts a file name as a parameter and uses stream operations to return a count of the number of lines in the file that start with the word “The”, case-insensitive.
12. Write a method [redacted] that accepts a file name as a parameter and returns a count of the number

of unique lines in the file that are exactly four letters long. Assume that each line in the file contains a single word.

13. Write a method [redacted] that accepts a file name string as its parameter and uses stream operations to print the first five non-blank lines of the file. Do not use any loops or collections; perform the work entirely using streams.

Programming Projects

1. Write a file searching program that uses streams to efficiently search a set of files for a given substring. Write two versions of the code, one that sequentially reads each file with a [redacted] and checks each line to see if it contains the substring, and a second that uses streams to open all of the files and search the lines using stream operations. The most efficient version of the stream code will open all of the files in parallel. Test how much more efficient the streams are than the [redacted] by using [redacted] to measure the elapsed time for both versions of the code when run on a collection of large files, printing output such as the following:

2. Write a program that prompts the user for an integer value n and that reports the sum of the first n prime

numbers, reporting a sum of 0 if the user enters a value less than 1. Structure your program to be similar to the case study, using an iterating function to produce the sequence 1, 2, 3, . . . , and filtering using the [redacted] method that appears at the end of section 19.3. Don't include the [redacted] modifier because we want the first n primes, not some other combination of n primes. Include timing code and print how long it takes to compute the sum.

Once you have a working program, explore the following efficiency improvements and note how the time changes with each using a fairly large value of n such as 10,000:

- Modify [redacted] to check only up to the square root, as in the case study (remember that 1 is not a prime).
- Modify the iterating function to examine only odd numbers and manually add 2 to the sum (because 2 is the only even prime).
- Modify the overall iteration to include the [redacted] modifier, as in the case study. Note that the benefits of parallel execution might not be evident for this computation because the problem size is fairly constrained (values of n greater than 20,043 will lead to integer overflow).

Appendix A Java Summary

Java Keywords

Primitive Types

[redacted]	integers (whole numbers)	[redacted]
[redacted]	real numbers	[redacted]
[redacted]	single characters	[redacted]
[redacted]	logical values	[redacted]

Arithmetic Operators

[redacted]	addition	[redacted]	[redacted]
[redacted]	subtraction	[redacted]	[redacted]
[redacted]	multiplication	[redacted]	[redacted]
[redacted]	division	[redacted]	[redacted]
[redacted]	remainder or mod	[redacted]	[redacted]

Relational Operators

[redacted]	equal to	[redacted]	[redacted]
[redacted]	not equal to	[redacted]	[redacted]
[redacted]	less than	[redacted]	[redacted]
[redacted]	greater than	[redacted]	[redacted]
[redacted]	less than or equal to	[redacted]	[redacted]
[redacted]	greater than or equal to	[redacted]	[redacted]

Logical Operators

[redacted]	AND (conjunction)	[redacted]	[redacted]
[redacted]	OR (disjunction)	[redacted]	[redacted]
[redacted]	NOT (negation)	[redacted]	[redacted]

Operator Precedence

unary operators	
multiplicative operators	
additive operators	
relational operators	
equality operators	
logical AND	
logical OR	
assignment operators	

Wrapper Classes

[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]

Syntax Templates

Variable declaration without initialization:

Variable declaration with initialization:

Assignment:

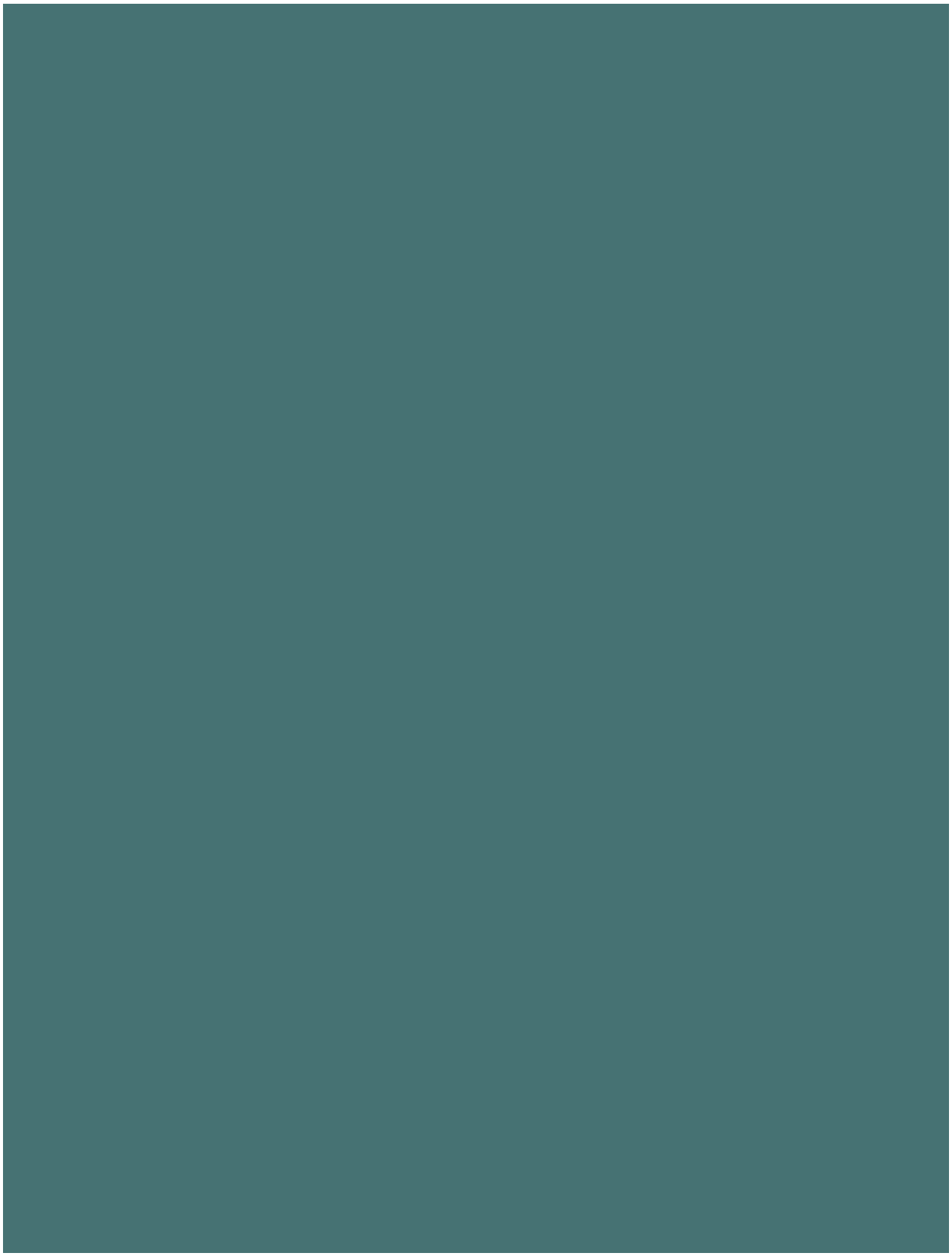
Constant declaration:

Static method definition:

Call on static method:

Call on instance method:

Class definition:



Constructor calling another constructor:

Instance method calling superclass method:

Constructor calling superclass constructor:

Specifying an inheritance relationship:

Implementing an interface:

Specifying inheritance relationship and implementing interfaces:

Interface definition:

Abstract class:

Abstract method:

statement:

statement:

[redacted] statement:

Array declaration:

Array initialization:

Simple [redacted]:

[redacted]:

Nested [redacted] ending in [redacted]:

Nested ending in :

loop:

[redacted]

[redacted] loop:

[redacted]

[redacted] loop:

[redacted]

[redacted] loop:

statement:

Useful Methods of [redacted] Objects

[redacted]		
[redacted]	Adds the given value at the end of the list	[redacted]
[redacted]	Adds the given value at the given index, shifting subsequent values right	[redacted]
[redacted]	Removes all elements from the list	[redacted]
[redacted]	Returns [redacted] if the given value appears in the list	[redacted]
[redacted]	Gets the value at the given index	[redacted]
[redacted]	Returns the index of the first occurrence of the given value in the list ([redacted] if not found)	[redacted]
[redacted]	Returns the index of the last occurrence of the given value in the list ([redacted] if not found)	[redacted]

[redacted]	Removes the value at the given index, shifting subsequent values left	[redacted]
[redacted]	Replaces the given value at the given index with the given value	[redacted]
[redacted]	Returns the current number of elements in the list	[redacted]

Useful Methods of the [redacted] Class

[redacted]		
[redacted]	Converts a character that looks like a number into that number	[redacted] returns [redacted]
[redacted]	Tests whether the character is one of the digits ‘0’ through ‘9’	[redacted] returns [redacted]
[redacted]	Tests whether the character is in the range ‘a’ to ‘z’ or ‘A’ to ‘Z’	[redacted] returns [redacted]
[redacted]	Tests whether the character is a lowercase letter	[redacted] returns [redacted]
[redacted]	Tests whether the character is an uppercase letter	[redacted] returns [redacted]
[redacted]	Converts a character into	[redacted] returns [redacted]

	the lowercase version of the given letter	
[redacted]	Converts a character into the uppercase version of the given letter	[redacted] returns [redacted]

Useful Methods of the [redacted] Interface

[redacted]	
[redacted]	Adds the specified element to this collection
[redacted]	Adds all elements from the given collection to this collection
[redacted]	Removes all elements from this collection
[redacted]	Returns [redacted] if this collection contains the given element
[redacted]	Returns [redacted] if this collection contains all elements of the given collection
[redacted]	Returns [redacted] if this collection contains no elements
[redacted]	Returns an object that can be used to traverse the elements of this collection
[redacted]	Removes one occurrence of the specified element, if it is contained in this collection
[redacted]	Removes all elements of the given collection from this collection
[redacted]	Removes all elements not found in the given collection from this collection
[redacted]	Returns the number of elements in this collection

[redacted]	Returns an array containing the elements of this collection

Useful Methods of the [redacted] Class

[redacted]	
[redacted]	Searches a sorted list for a given element value and returns its index
[redacted]	Copies all elements from the source list to the destination list
[redacted]	Replaces every element in the given list with the given value
[redacted]	Returns count of occurrences of value in the collection
[redacted]	Returns the element with the highest value
[redacted]	Returns the element with the lowest value
[redacted]	Replaces all occurrences of the old value with the new value
[redacted]	Reverses the order of the elements in the given list
[redacted]	Shifts each element to the right by the given number of indexes, moving the final elements to the front
[redacted]	Rearranges the elements into random order
[redacted]	Rearranges the elements into sorted (nondecreasing) order

	Switches the element values at the given two indexes

Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Returns a reference to the [redacted] object that can be used to draw onto the panel
[redacted]	Sets the background color of the panel to the given color (the default is white)

Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Deletes the given file
[redacted]	Indicates whether this file exists on the system
[redacted]	Returns the full path specifying the location of this file
[redacted]	Returns the name of this file as a [redacted], without any path attached
[redacted]	Indicates whether this file represents a directory/folder on the system
[redacted]	Indicates whether this file represents a file (nonfolder) on the system
[redacted]	Returns the number of characters in this file
[redacted]	Creates the directory represented by this file, if it does not exist
[redacted]	Changes this file's name to be the given file's name

Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Draws a line between the points ([redacted]) and ([redacted])
[redacted]	Draws the outline of the largest oval that fits within the specified rectangle
[redacted]	Draws the outline of the specified rectangle
[redacted]	Draws the given text with its lower-left corner at ([redacted])
[redacted]	Fills the largest oval that fits within the specified rectangle using the current color
[redacted]	Fills the specified rectangle using the current color
[redacted]	Sets this graphics context's current color to the specified color (all subsequent graphics operations using this graphics context use this specified color)
[redacted]	Sets this graphics context's current font to the specified font (all subsequent strings drawn using this graphics context use this specified font)

Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Returns [redacted] if there are more elements to be examined
[redacted]	Returns the next element from the list and advances the position of the iterator by one
[redacted]	Removes the element most recently returned by [redacted]

Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Removes all keys and values from a map
[redacted]	Returns [redacted] if the given key maps to some value in this map
[redacted]	Returns [redacted] if some key maps to the given value in this map
[redacted]	Returns the value associated with this key, or [redacted] if not found
[redacted]	Returns the value associated with this key, or the given default value if not found
[redacted]	Returns [redacted] if this collection contains no keys or values
[redacted]	Returns a [redacted] of all keys in this map
[redacted]	Associates the given key with the given value
[redacted]	Adds all key/value mappings from the given map to this map
[redacted]	Adds a key/value pair if this key was not present
[redacted]	Removes the given key and its associated value from this map
[redacted]	Updates a key/value pair only if the key was present
[redacted]	Returns the number of key/value mappings in this map

[redacted]	Returns a [redacted] of all values in this map

Constants and Useful Methods of the [redacted] Class

[redacted]	
[redacted]	base used in natural logarithms (2.71828 . . .)
[redacted]	ratio of circumference of a circle to its diameter (3.14159 . . .)

[redacted]		
[redacted]	absolute value	[redacted] returns [redacted]
[redacted]	ceiling (rounds upward)	[redacted] returns [redacted]
[redacted]	cosine (radians)	[redacted] returns [redacted]
[redacted]	exponent base e	[redacted] returns [redacted]
[redacted]	floor (rounds downward)	[redacted] returns [redacted]
[redacted]	logarithm base e	[redacted] returns [redacted]
[redacted]	logarithm base 10	[redacted] returns [redacted]
[redacted]	maximum of two values	[redacted] returns [redacted]

[redacted]	minimum of two values	[redacted] returns [redacted]
[redacted]	power (general exponentiation)	[redacted] returns [redacted]
[redacted]	random value	[redacted] returns a random double value k such that $0.0 \leq k < 1.0$
[redacted]	sine (radians)	[redacted] returns [redacted]
[redacted]	square root	[redacted] returns [redacted]
[redacted]	converts radian angles to degrees	[redacted] returns [redacted]
[redacted]	converts degree angles to radians	[redacted] returns [redacted]

Useful Methods of the [redacted] Class

[redacted]	
[redacted]	Creates and returns a copy of the object (not a public method)
[redacted]	Indicates whether the other object is equal to this one
[redacted]	Called automatically by Java when objects are destroyed (not a public method)
[redacted]	Returns information about the type of the object
[redacted]	Returns a number associated with the object; used with certain data structures
[redacted]	Returns the state of the object as a [redacted]
[redacted]	Advanced methods for multithreaded programming

Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Returns the distance from this [redacted] to [redacted]
[redacted]	Sets the coordinates to the given values
[redacted]	Translates the coordinates by the given amounts

Useful Methods of [redacted] Objects

[redacted]	
[redacted]	random logical value of [redacted] or [redacted]
[redacted]	random real number between [redacted] (inclusive) and [redacted] (exclusive)
[redacted]	random integer between [redacted]31 and [redacted]31 – 1)
[redacted]	random integer between [redacted] and [redacted]

Useful Methods of [redacted] Objects

[redacted]	
[redacted]	Reads and returns the next token as a [redacted]
[redacted]	Reads and returns a [redacted] value
[redacted]	Reads and returns an [redacted] value
[redacted]	Reads and returns the next line of input as a [redacted]
[redacted]	Returns [redacted] if there is another token to be read
[redacted]	Returns [redacted] if there is another token to be read and if it can be interpreted as a [redacted]
[redacted]	Returns [redacted] if there is another token to be read and if it can be interpreted as an [redacted]
[redacted]	Returns [redacted] if there is another line of input to be read

Useful Methods of [redacted] Objects

[redacted]		
[redacted]	Returns character at a specific index	[redacted] returns [redacted]
[redacted]	Tests whether the string ends with some text	[redacted] returns [redacted]
[redacted]	Returns index of a particular character or [redacted] ([redacted] if not present)	[redacted] returns [redacted]
[redacted]	Returns number of characters in the string	[redacted] returns [redacted]
[redacted]	Replace all occurrences of one substring with another	[redacted] returns [redacted]
[redacted]	Tests whether the string starts with some text	[redacted] returns [redacted]
[redacted]	Returns characters from start index to just before stop index	[redacted] returns [redacted]
[redacted]	Returns a new string with all lowercase letters	[redacted] returns [redacted]
[redacted]	Returns a new string with all uppercase letters	[redacted] returns [redacted]

Appendix B The Java API Specification and Javadoc Comments

The Java API Specification

Java's Application Programming Interface (API) Specification is a set of web pages that describe the classes, interfaces, and methods of the Java class libraries. You can use these pages to learn class and method names or to find details about a particular method. The API pages can be thought of as a contract between the authors of the class libraries and you, the client of those classes.

The API pages exemplify the idea of the public view of a class versus its private implementation. Each class has a set of constructors, constants, and methods that its clients can access and use. The class also has private fields, methods, and method bodies that are used to actually implement the behavior specified in the public interface. The main benefit of this separation is that you don't need to know the private implementation of the class libraries to be able to use them.

As of this writing, the current Java API Specification pages can be accessed from the following URL:

http://docs.oracle.com/javase/10/docs/api/

(Each new version of Java has a new specification, so this URL will change as new versions are released.) When you visit the API pages you'll see a screen that looks like Figure B.1.

Figure B.1 The Java API Specification

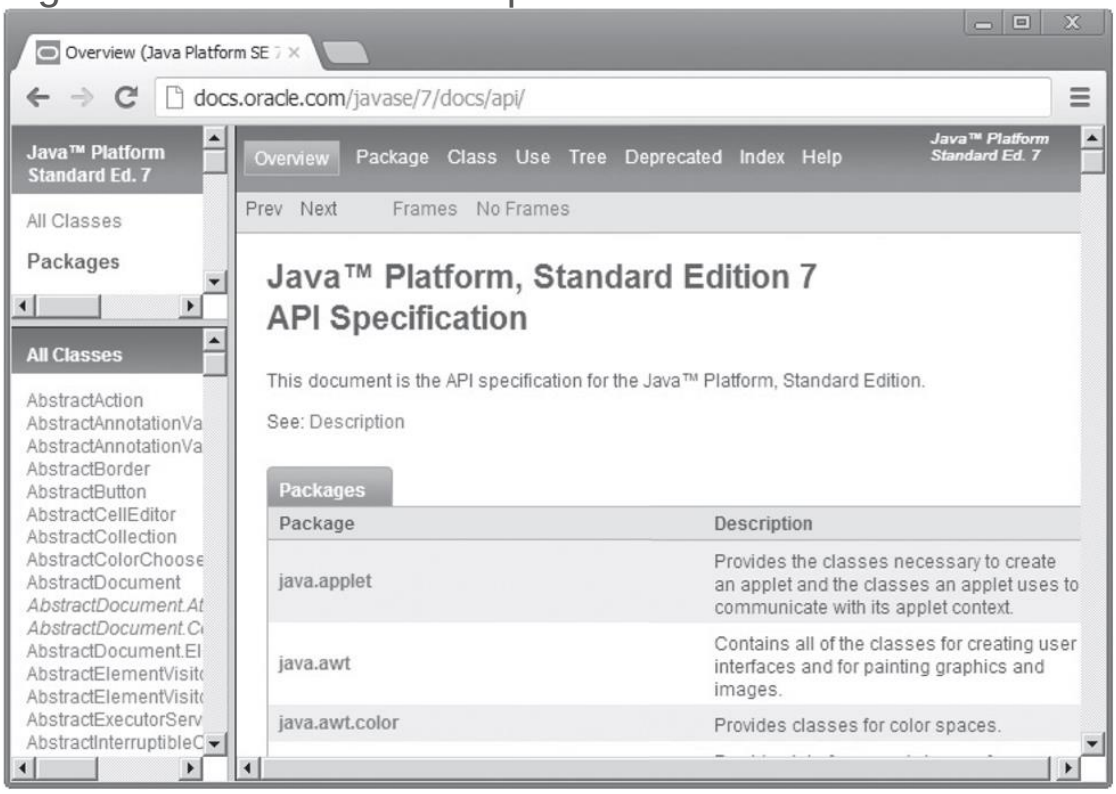


Figure B.1 Full Alternative Text

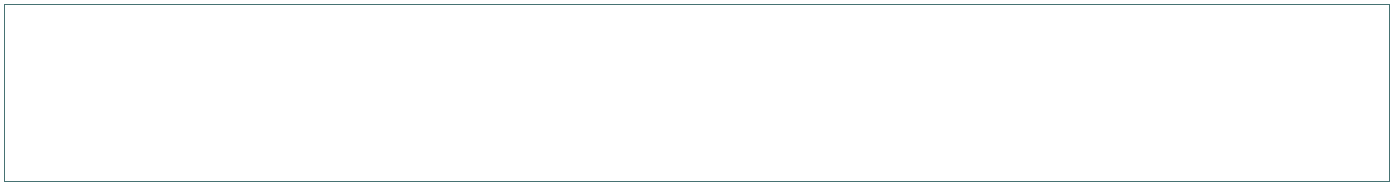
The main frame of the page (on the right) shows information about classes that you can select using the frames on the left. The lower-left frame lists all of the classes and interfaces, and the upper-left frame lists all of the packages in the class libraries. Recall that packages are groups of related classes that you can use in your program by importing them. If you're looking for a class from a particular package, you can click that package's name in the top-left frame to filter the results in the bottom-left frame.

Once you click a class name in the bottom-left frame, information about that class appears in the main frame. At the start of the page you'll see a tree showing the names of any superclasses and any interfaces that it implements, the class's header, and a summary description of the class. The summary description gives general information about the purpose and usage of the class and may link to other relevant documentation or tutorials.

An ordered list of the contents of the class follows the summary. Listed first will be any public fields and class constants. Next will be the class's constructors, followed by its methods. Each field, method, and constructor has a line showing information such as its name, parameters, and return type, followed by a one-sentence description. You can click the name of the item to see more details about it.

For example, the following is the summary information for the [redacted] class's constructor to read input from a file:

Constructor Summary



[redacted] [redacted]

Constructs a new [redacted] that produces values scanned from the specified file.

Clicking on a method can provide useful details about how to use it, including a longer description, a detailed explanation of each parameter that the method requires, a description of what kind of value (if any) the method returns, and a listing of any exceptions

the method may throw. These detail views allow you to learn the preconditions and postconditions of the method and to see examples of calls and their results.

For example, the following are the details about the [redacted] method of the [redacted] class:

Returns a new string that is a substring of this string. The substring begins at the specified [redacted] and extends to the character at index [redacted]. Thus the length of the substring is [redacted].

Examples:

Parameters

[redacted]—the beginning index, inclusive

[redacted]—the ending index, exclusive

Returns

the specified substring

Throws

[redacted]—if the [redacted] is negative, if [redacted] is larger than the length of this [redacted] object, or if [redacted] is larger than [redacted]

Some of the methods in the API specs are marked as *deprecated.*

Deprecated

Discouraged from use.

Deprecated items are ones that Java's designers want to discourage you from using. Generally these are methods that Oracle has either renamed or decided shouldn't be called because they didn't behave correctly or safely. You might think that they should just remove these deprecated methods from Java, but doing so would break any older Java programs that call these methods. An example of a deprecated method is the [redacted] method in the [redacted] class of the [redacted] package:

Deprecated. As of *JDK version 1.1, replaced by* [redacted].

Writing Javadoc Comments

If you write your comments in a special style called *Javadoc*, they can be used to automatically produce web pages like those in the API Specification. In fact, Oracle generates the API Specification by writing Javadoc comments in each class of the class libraries. The syntax to signify a Javadoc comment is to begin a multiline comment with rather than the usual :

Javadoc comments may be added to class headers, methods, class constants, public fields, and any other visible members of a class.

Many Javadoc comments also specify additional information using special syntax called tags. A *tag* is an indicator for specific information such as a description of a parameter or return value, the author of a class, the version of a file, and so on. The information in a tag can include boundary conditions, acceptable parameter ranges, and examples.

The syntax for tags is to write an [redacted] sign and a tag name, followed by any additional information. Several common Javadoc tags are described in the following table:

[redacted]	
[redacted]	Name(s) of the author(s) who wrote this class
[redacted]	Details about the given parameter to this method
[redacted]	Details about the value that is returned by this method
[redacted]	A type of exception that this method may throw and a description of the circumstances under which it will do so
[redacted]	Version or revision number of the file; may be a number such as 1.2.5 or a more complex string such as a date

The [redacted] and [redacted] tags are often used on class headers. For example, the following comment can precede the header for a [redacted] class:

The [redacted] and [redacted] tags are often used on methods and constructors. Non[redacted] methods may also use the [redacted] tag. It is best to use the three preceding tags if they supply valuable additional information that cannot be discerned from the method's header or overall comment header.

For example, the following comment can precede the header for the [redacted] method of the [redacted] class:

As we mentioned previously, Javadoc comments can be converted into web documentation pages. Some Java editing environments include this functionality; check your editor to see whether it is provided. If not, the JDK includes a command-line tool for generating the pages. To use it, open a terminal window to the directory of your source code files and type the following command:

The web page that is generated for the preceding [redacted] class is shown in Figure B.2.

Figure B.2 Generated Javadoc pages for [redacted] class

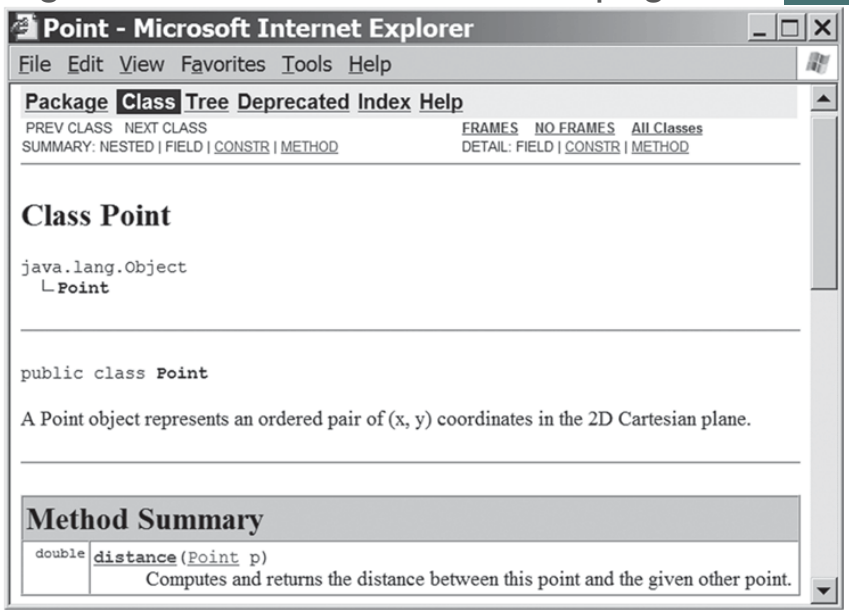


Figure B.2 Full Alternative Text

Clicking on the name of a method brings up the method's detail page, as in the API Specification. The web page that is generated for the preceding [redacted] method looks like this:

Method Detail

distance

[redacted] distance(Point [redacted]

Computes and returns the distance between this point and the given other point.

Parameters:

[redacted] — the point to which the distance is computed

Returns:

the distance, computed as the square root of the sums of the squares of the differences between the two points' x-coordinates ([redacted]) and between their y-coordinates ([redacted])

Throws:

[redacted] — if [redacted] is null

Because Javadoc comments are converted into HTML, they can contain HTML tags that will show up in any generated Javadoc pages. You may use these tags to format your comments.

Javadoc comments provide documentation to clients that lets them know how to use your class without needing to read its source code. However, they are lengthy and can take time to write. Writing Javadoc comments is most useful when you know that your class will be used by many clients.

Oracle maintains a web site with much more information about writing Javadoc comments at the following URL:

http://www.oracle.com/technetwork/java/javase/documentation/index-137868.html

Appendix C Additional Java Syntax

This appendix briefly covers several features of Java that are not otherwise shown in the textbook. It is not intended to be a complete reference; see the Java Language Specification and Java Tutorial online for more detailed coverage of these language features.

Primitive Types: [redacted], [redacted], [redacted], [redacted]

In this textbook, we have focused our attention on four primitive data types: [redacted], [redacted], [redacted], and [redacted]. Java has four additional primitive types that are used in certain special situations. Three of these additional types ([redacted], [redacted], and [redacted]) are variants of [redacted] that use a different amount of memory, providing a tradeoff between memory consumption and the range of numbers that can be represented. The fourth, [redacted], is a variant of [redacted] that uses half the memory. It can be useful in certain applications where reduced memory usage and faster computation is more important than extremely high numeric accuracy, such as in computer games.

[redacted]			
[redacted]	8-bit (1-byte) integer	-128 to 127	Reading data from an input source one byte at a time
[redacted]	16-bit (2-byte) integer	$-32,768$ to 32,767	Saving memory when creating many integers
[redacted]	64-bit (8-byte) integer	-263 to $(263-1)$	Representing very large integers that may not fit into the range of [redacted]
[redacted]	32-bit (4-byte) real number	roughly $-3.4E+38$ to $3.4E+387$	Saving memory when creating many real numbers

Many of the operators and much of the syntax that you have used with [redacted] and [redacted] work with these types. Two of these types use a suffix letter at the end of their literal values: [redacted] for float, and [redacted] for [redacted]. The following code declares a variable of each type:

Ternary Operator ? :

Java has a *ternary operator* that allows you to choose between two expressions based on the value of a [redacted] test. (“Ternary” means “having three sections.”) Think of it as an abbreviated form of an [redacted] statement, except that an [redacted] chooses between two blocks of statements, while a ternary expression chooses between two expressions or values:

A ternary expression is most useful when you want to assign a variable one of two values, or when you want to pass one of two values as a parameter or return value. For example:

```
(pals == 1 ? "buddy" :

"buddies");




(a > b) ? a : b
```

Exiting a Loop: [redacted] and [redacted]

Java has a statement called [redacted] that will immediately exit a [redacted], [redacted], or [redacted] loop. One common usage of [redacted] is to write a loop that performs its exit test in the middle of each iteration rather than at the start or end. The common template is to form what appears to be an infinite loop:

Because the [redacted] literal [redacted] always evaluates to [redacted], this loop appears to execute indefinitely. But a test occurs in the

middle of the loop. This technique is useful for solving fencepost and sentinel problems. For example, Chapter 5 examines a sentinel problem for summing numbers until −1. This problem can be elegantly solved with `break` as follows:

```
while (true)

    break;
```

The `break` statement immediately ends the current iteration of a loop. The code execution will return to the loop's header, which

will perform the loop's test again (along with the update step, if it is a [redacted] loop). The following loop uses [redacted] to avoid negative integers:

```
continue;
```

Many programmers discourage use of the [redacted] and [redacted] statements because these statements cause the code execution to jump from one place to another, which some people find nonintuitive. Any solution that uses [redacted] or [redacted] can be rewritten to work without them.

It is possible to label statements and to [redacted] or [redacted] executing from a particular label. This can be useful for immediately exiting to particular levels from a set of nested loops. This syntax is outside the scope of this textbook.

The [redacted] Statement

The [redacted] statement is a control structure that is similar to the [redacted] statement. It chooses one of many paths (“cases”) to execute on the basis of the value of a given variable or expression. It uses the following syntax:

The [redacted] used in a [redacted] statement must be an integral type ([redacted], [redacted], [redacted], or [redacted]) or an enumerated type ([redacted], discussed later in this appendix). In Java version 7 and up, you may also switch on a [redacted] value.

The benefit of the [redacted] syntax is that you do not need to repeat the [redacted] syntax or the variable's name for each path; you simply write [redacted] plus the next value to test. If the value does not match any of the cases, none of them is executed. The following code prints a runner's medal; if the runner was not in first through third place, no message is printed.

The optional [redacted] case holds code to execute if the expression’s value does not match any of the other cases, as shown in the code that follows.

The tricky part of [redacted] is remembering to write a [redacted] statement at the end of each case. If you omit the [redacted] at the end of a case, the code “falls through” into the next case and also executes its code. A programmer may cause this to happen intentionally, as in the following code, but often it is done by accident and leads to bugs:

```
default:
```

Many programmers avoid the [redacted] statement because it is so easy to produce a bug such as that just described. In older programming languages the [redacted] statement was a more efficient way to execute the [redacted] statement, but this benefit is not noticeable in Java.

The [redacted] Statement

The [redacted] statement “tries” to execute a given block of code (called the “[redacted] block”). The statement also specifies a second “[redacted] block” of code that should be executed if any code in the “[redacted] block” generates an exception of a particular type. It uses the following syntax:

For example, the following code attempts to read an input file and prints an error message if the operation fails:

```
try
```

```
catch (FileNotFoundException e)
```

If you wrap all potentially unsafe operations in a method with the [redacted] syntax, you do not need to use a [redacted] clause on that method's header. For example, you do not need to declare that your [redacted] method throws a [redacted] if you handle it yourself using a [redacted] block.

Some variations of the [redacted] syntax are not shown here. It is possible to have multiple [redacted] blocks for the same [redacted] block, to handle multiple kinds of exceptions. It is also possible to add another block called a "[redacted] block" that contains code to execute in all cases, whether an error occurs or not. This technique can be useful to consolidate cleanup code that should be run after the [redacted] block finishes. These syntax variations are outside the scope of this textbook.

The [redacted] Statement

In Chapters 4 and 5, we discussed preconditions, postconditions, and logical assertions. Sometimes programmers want to test logical assertions (Boolean expressions) as sanity checks in their own code. As the code runs, it will check each assertion that it reaches. If the assertion's expression is not true, the program will halt with an error.

Java supports the testing of assertions with its [redacted] statement, which uses the following syntax:

[redacted]

For example, to test that a variable x is nonnegative, you could write the following line of code:

[redacted]

In general, we expect these assertions to succeed. When an assertion fails, it signals a problem. It means that the program has a logic error that is preventing the assumptions from holding true.

Testing of assertions can be expensive, so Java lets you control whether this feature is enabled or disabled. You can enable assertion checking in your Java editor while you are developing and testing a program to make sure it works properly. Then you can disable it when you’re fairly confident that the program works and you want to speed it up. By default, assertion checking is disabled.

Enumerations:

Since Chapter 2 we have seen the usefulness of class constants. Sometimes we want to create a type that has only a small number of predefined constant values. For example, suppose we are writing a program for a card game. Each card has a suit: Clubs, Diamonds, Hearts, or Spades. We could represent these values as integers (0 through 3) or strings, but these are clumsy solutions because the range of integers and strings is large, so it may be possible to slip in an invalid suit value.

In such situations, it can be useful to create an *enumerated type*, which is a simple type that has only a small number of constant values.

For example, to create an enumerated type for card suits, you could write the following code in a file named :

The following client code uses the enumerated type:

It would also be possible to create a class that has a field for the card's rank and a field of type for the card's suit. This way an invalid suit can never be passed; it must be one of the four constants declared. You can test whether a variable of an enumerated type stores a particular value with the and operators:

The [redacted] concept is borrowed from past programming languages such as C, in which enumerated constants were actually represented as 0-based integers on the basis of the order in which they were declared. If you want to get an integer equivalent value for a Java [redacted] value, call its [redacted] method. For example, the call of [redacted] returns 1 because [redacted] is the second constant declared. Every [redacted] type also has a [redacted] method that returns all possible values in the enumeration as an array. For example, the call of [redacted] returns the following array:

Packages

Since Chapter 3 we have used the [redacted] statement to make use of classes from the Java class libraries. The classes in the libraries are organized into groups called *packages*. Packages are valuable when you are working with large numbers of classes. They give Java multiple *namespaces*, meaning that there may be two classes with the same name so long as they are in different packages. Packages also help physically separate your code files into different directories, which makes it easier to organize a large Java project.

Every Java class is part of a package. If the class does not specify which package it belongs to, it is part of a nameless default package. To specify that the class belongs to some other package, place a package declaration statement as the first statement at the top of the file, even before any [redacted] statements or the class's header. A package declaration has the following syntax:

For example, to specify that the class [redacted] belongs to the [redacted] package, you would write the following statement at the top of the [redacted] file:

```
package homework4;
```

Packages may be nested, indicated by dots. The packages in the Java class libraries are nested at least two levels deep, such as [redacted] or [redacted]. For example, to specify that a graphical user interface file for Homework 4 belongs to the [redacted] subpackage within the [redacted] package, you would write the following statement:

```
homework4.gui
```

Packages are reflected by the directory structure of the files in your Java project. For example, if a file claims to belong to the [redacted] package, it must be placed in the [redacted] directory relative to the root directory of your project. A file in the [redacted] package must be in the [redacted] directory relative to the project's root.

If you write classes that are part of different packages and a class from one package wants to use a class from another, you must use an import statement for the compiler to find the classes. Remember that packages are not nested, so importing the [redacted] package does not automatically import [redacted] and vice versa.

```
general
```

Packages are generally not necessary for small projects, though some editors add them to the top of all files automatically. Users of basic Java IDEs or text editors often avoid packages because they can make it harder to compile the entire project successfully. More sophisticated Java editors such as Eclipse or NetBeans handle packages better and usually provide a one-click button for compiling and executing all of the packages of a project.

Protected and Default Access

In Chapter 8's discussion of encapsulation we learned about two kinds of access: [redacted] (visible only to one class) and [redacted] (visible to all classes). In general, you should follow the convention shown in this book of declaring all fields [redacted] and most methods [redacted] (other than internal helper methods). But Java has two other levels of access:

- [redacted] access:

Visible to this class and all of its subclasses, as well as to all classes in the same package as this class.

- default (package) access:

Visible to all classes in the same package as this class.

Some programmers prefer to declare their fields as [redacted] when writing classes that are part of an inheritance hierarchy. This declaration provides a relaxed level of encapsulation, because subclasses can directly access protected fields rather than having to go through accessor or mutator methods. For example, if a subclass [redacted] chose to extend the following [redacted] class, it would be able to directly get or set its [redacted] or [redacted] field values:

```
protected
```

```
protected
```

The downside is that any class in the same package can also access the fields, which is generally discouraged. A compromise recommended by Joshua Bloch, author of *Effective Java*, is to give subclasses access to private data through protected methods instead of protected fields. This gives the class author the freedom to change the implementation of the class later if necessary.

Default access is given to a field when it has no access modifier in front of its declaration, such as in the initial versions of our class in Chapter 8 (prior to properly encapsulating it). A field or method with default access can be directly accessed by any class in the same package. We generally discourage the use of default access in most situations, since it can needlessly violate the encapsulation of the object.

Index

A

- [redacted] method, 161, 270, 1140–1141
- abstract classes, 647–652
  - defined, 648
  - interfaces *vs*., 647–652
  - [redacted] class, 648–650
  - syntax for declaring, 648
  - uses, 651
- abstract data types. See ADTs (abstract data types)
- abstraction
  - defined, 560, 561
  - representation, 574
- abstract methods, 634, 648
- Abstract Window Toolkit (AWT), 202
- access
  - default, 1190
  - inherited fields, 603–605
  - private, 561–567, 604
  - protected, 1190
- accessors, 548–549, 1122
- [redacted] interface, 632
- actual parameters, 145, 149–150
- addition (+) operator, 84–86
- additive operators, 73, 74, 88, 247
- [redacted] method, 671, 747
- [redacted] method, 1002–1008
- ADTs (abstract data types), 731–733, 892
  - defined, 731
  - interfaces and, 731

    - map, 746
- algorithmic thinking, 3
- algorithms
    - analysis, 852, 854
    - array-traversal, 470–484
    - binary search, 862–865
    - concept of, 2–3
    - constant-time, 858
    - cubic, 860
    - cumulative, 259–270
    - cumulative sum, 259–261
    - cumulative text, 272–274
    - defined, 2
    - Euclid’s, 785
    - exponential, 860
    - fencepost, 333–339
    - linear, 859, 861
    - logarithmic, 858
    - log-linear, 859
    - merge sort, 873
    - nested loops, 495–497
    - pseudocode, 108–110
    - quadratic, 859
    - range, 855–858
    - searching and sorting, 861–872
    - selection sort, 869–872
    - sequential, 447
    - string traversal, 481–482
- AND (&&) operator, 258, 288
    - precedence, 343
    - short-circuited evaluation, 346
    - truth table for, 341
- annotation, 964

- API (Application Programming Interface) Specification, 8
- Apple Computer, 538
- Application Programming Interface (API) Specification, 1176–1178
- applications, 4
- arguments. See parameters
- arithmetic notations, 819
- arithmetic operators, 69–72, 1161
- [redacted], 455, 472, 491, 950
- [redacted] class, 932, 941, 990, 1008, 1084
  - adding an iterator, 957–963
  - advanced features, 955–963
  - constructor and constant, 944–945
  - convenience methods, 952–955
  - converting into [redacted], 963–968
  - encapsulation, 938–939
  - garbage collector, 965
  - inner class, 966–968
  - preconditions and postconditions, 945–948
  - printing/adding, 932–937
  - resizing, 955–957
  - throwing exceptions, 949–955
- [redacted], 667–721, 723, 841
  - adding to, 679–683
  - adding values to, 671–672
  - benefits of, 730
  - [redacted] method, 689
  - complete program, 677–679
  - defined, 667, 668
  - for-each loop, 684–686
  - [redacted] *vs.*, 724–726
  - [redacted] loop and, 680–683
  - methods, 675

  - natural ordering of values, 691–694
  - operations, 669–674
  - printing, 670
  - removing from, 679–683
  - searching methods, 674–677
  - syntax for constructing, 669
  - wrapper classes, 686–688
- [redacted] objects, methods of, 1168
- arrays, 447–534, 1140–1141
  - accessing, 452–455
  - advanced techniques, 488–497
  - auto-initialization, 450
  - basics of, 448–470
  - buffer overruns, 457–458
  - case study, 509–519
  - command-line arguments, 494–495
  - constructing, 448–452
  - declaring, 449
  - defined, 447, 448
  - equality, testing for, 475–477
  - filled, 933
  - for-each loop, 467–468
  - heap implementation, 1106–1108
  - index, 448
  - initializing, 455–456
  - jagged, 499–503
  - loops, 467–468
  - merging, 873–876
  - and methods, 464–466
  - multidimensional, 497–503
  - nested loop algorithms, 495–497
  - of objects, 493–494
  - of pixels, 504–509

    - printing, 471–473
    - program, 456–461
    - random access, 461–462
    - rectangular two-dimensional, 497–499
    - reversing, 477–481
    - searching and replacing, 473–475
    - sequential search, 861–862
    - shifting values in, 488–493
    - sorted, 1084
    - splitting, 873–876
    - traversal algorithms, 470–484
    - traversing, 448–452
    - unfilled, 933
    - unsorted, 1084
- [redacted] method, 1141–1142
- [redacted] class, 468–469
    - [redacted] method, 469
    - [redacted] method, 469
    - [redacted] method, 499
    - [redacted] method, 469, 476–477
    - [redacted] method, 845
    - [redacted] method, 845
    - [redacted] method, 469–470, 499
    - [redacted] method, 1140–1141
- ASCII, 275
- assertions, 358–366
    - defined, 358
    - example, 362–366
    - provable, 360
    - reasoning about, 360–362
- [redacted] statement, 1187
- assignment operators, 83, 88, 247
- assignment statements, 78, 79–80, 81–82, 83

- asterisk (*), 65, 69
- auto-initialization, 450
- automatic garbage collection, 981
- average case analysis, 861

B

- backslash character (\ ), 16, 69
- back trace, 170
- backtracking, recursive, 801–818
- backward compatibility, 629
- base case, 770–771, 780, 782–785, 799, 804
- behavior, object, 536, 540
- Benford’s Law case study, 509–519
- best case analysis, 862
- big-Oh notation, 858
- binary files, 395
- binary heaps, 1102
- binary numbers, 4–5
- binary search, 842–845, 862–865
  - defined, 842
  - details, 867–868
  - implementing, 862–865
  - recursive, 865–867
  - runtime of, 865
- [redacted] method, 842, 867–868
- binary search tree, 740, 1052–1067
  - building, 1055–1058
  - complexity, 1066–1067
  - defined, 1052, 1054
  - property, 1053–1055
  - searching, 1062–1066
- binary tree, 1028–1071
  - branch nodes, 1029, 1030
  - counting leaves, 1051–1052

  - counting levels, 1049–1050
  - defined, 1029
  - degenerate, 1067
  - height of, 1051
  - kinds of, 1031
  - leaf nodes, 1030–1031
  - operations of, 1048–1052
  - parent/child/sibling/ancestor/descendant, 1031
  - root of, 1029, 1031–1032
  - sum of, 1048–1049
  - traversals, 1033–1048
- block copy operation, 956
- [redacted] object, 809–810
- Body Mass Index (BMI), 76
  - case study, 290–305
- boilerplate code, 427
- Boole, George, 339
- Boolean expression, 339–341, 353–354
- [redacted] flag, 908
- [redacted] type, 64, 69, 339–354
  - defined, 320
  - flags, 348–350
  - values, 339–340
  - variables, 340, 348–350
- Boolean Zen, 350–352
- bounds checking, 458
- bound variables, 1137, 1138
- boxing, 688, 690–691
- branch nodes, 1029
- [redacted] statement, 1183–1184
- bubbling, 1103
- buckets, 1087
- buffer overruns, 457–458

- bugs, 28
  - off-by-one, 462–463
  - Y2K (millennium), 564
  - Zune, 345
- bytecodes, 7
- [redacted] type, 64

C

- calls, 602–603
  - complex, interpretation, 622–625
  - constructor, 605–607
  - recursive, 783, 806, 810
- call stack, 776
- [redacted] method, 425
- capacity, list, 934
- case studies
  - Benford’s Law, 509–519
  - Body Mass Index (BMI), 290–305
  - [redacted], 40–46
  - financial class hierarchy, 639–642
  - hourglass figure, 115–123
  - lottery program, 743–745
  - merge sort, implementation, 873–881
  - [redacted], 366–375
  - perfect numbers, 1146–1156
  - prefix evaluator, 818–824
  - projectile trajectory, 178–186
  - pyramids, 224–230
  - sieve algorithm, 734–737
  - [redacted] class, 573–582
  - vocabulary comparison, 701–714
  - [redacted], 750–753
  - zip code lookup, 427–434
- casting, 74–76, 612
- central processing unit (CPU), 3
- chaining, 304–305

- [redacted] class, methods of, 1169
- characters, literals, 69
- [redacted] method, 168, 285, 482
- [redacted] type
  - defined, 64
  - example, 64, 69
  - [redacted] *vs.*, 271–272
  - literal values, 270
  - [redacted] *vs.*, 271
- checked exception, 398
- [redacted] method, 953, 954
- [redacted] class, 628, 636
- circles, area and perimeter, 633
- Class, Responsibilities, Collaborators (CRC) card, 576
- classes, 535–591
  - abstract, 647–652
  - [redacted]. See [redacted] class
  - [redacted], 468–470
  - case study, 573–582
  - [redacted], 273
  - cohesion, 576
  - [redacted], 208–210
  - complexity, 858–860
  - concrete, 651
  - constants, 113–115, 119–122
  - defined, 10, 166
  - design(ing), 573–582, 640–644
  - [redacted], 202–203
  - generic, 668–669
  - [redacted], 629–631
  - [redacted], 282
  - inner, 966–968
  - [redacted], 897–898

    - invariants, 567–571
    - [redacted]. See [redacted] class
    - [redacted], 975, 1011–1019
    - [redacted], 158–161
    - [redacted], 609–610
    - objects. See objects
    - [redacted], 420–424
    - [redacted], 910
    - [redacted], 571–573
    - wrapper. See wrapper classes
- class headers, 11
- client code, 537
- clients, defined, 537
- [redacted] interface, 632
- clustering, 1088
- code, 6
    - boilerplate, 427
    - client, 537
    - evaluation, 917–920
    - legacy, 593
    - reuse, 593
- cohesion, 302–303, 576
- [redacted] interface, 732
    - useful methods of, 724, 1169
- collections, 723–724. See also lists; maps; sets
    - higher-order operations on, 1139–1146
    - overview, 753–754
- [redacted] class, methods of, 732
- [redacted] method, 689
- collisions, 1087–1092
- [redacted] class, 208–210
    - constants, 209
- [redacted] object, 506–509

  - methods, 507
- colors
  - background, 208
  - graphics, 208–210
- command-line arguments, 494–495
- comments
  - defined, 22
  - forms, 22–23
  - Javadoc, 23, 1178-1181
  - pre/post, 281–282
  - programming error, 29
  - single-line, 23
  - writing, 1178–1181
- [redacted] interface, 632, 689–701, 848
  - implementation, 692, 694–701
- comparators
  - custom ordering with, 848–851
  - defined, 740
  - implementation, 848–849
  - and methods, 851
- [redacted] method, 848–849
- [redacted] method, 692, 695–697, 848–849
- compilation errors, 24
- compilers, 6–7
- complex expressions, 66
- complexity, 852–860
  - binary search tree, 1066–1067
  - classes, 858–860
  - defined, 852
  - empirical analysis, 855–858
  - managing, 103–115
  - time, 852
- computers, 2–10

- concatenation, string, 84–87
- concrete classes, 651
- conditional execution, 243–319
  - case study, 290–305
  - cumulative algorithms, 259–270
  - [redacted] statements, 244–258
  - methods with, 279–290
  - return values and, 283–287
  - text processing, 270–278
- conditions
  - multiple, 258
  - postconditions, 279
  - preconditions, 279
- console
  - defined, 9
  - input, 9, 173
  - interaction, 9
  - output, 9
  - window, 9
- constants
  - class, 113–115, 119–122
  - declaration, 114
  - [redacted] class, 217
  - [redacted] class, 160
  - parameters *vs*., 156
- constant-time algorithm, 858, 865
- constructors, 173, 552–560, 980
  - default, 555
  - headers, 553
  - implementation, 578–582
  - multiple, 559–560
  - redeclaring fields in, 556
  - syntax, 555

    - using [redacted] keyword with, 555–556
- consuming input, 406–407
- [redacted] method, 675
- [redacted] statement, 1183–1184
- contract, 932
- control structure, 94
- control variable, 106
- [redacted] method, 469, 956
- [redacted] method, 469
- counter, 266
- coupling
    - defined, 303–304
    - loose, 575
    - minimizing, 303
- C programming language, 14, 65, 458
- C# programming language, 458
- C++ programming language, 14, 458
- CPU (central processing unit), 3
- CRC (Class, Responsibilities, Collaborators) card, 576
- cubic algorithm, 860
- cumulative algorithms, 259–270
    - cumulative product, 280–281
    - cumulative sum, 259–261, 265–267
    - defined, 259
    - [redacted] loops, 261–265
    - roundoff errors, 267–270
    - text, 272–274
- cumulative concatenation, 273
- cumulative product, 280–281
- cumulative sum, 259–261
    - execution, 260–261
    - with [redacted] statements, 265–267
    - pseudocode, 259

- cumulative text algorithms, 272–274
- curly brace characters ({}), 11, 97–98
- current directory, 409
- cursor
  - input, 405
  - output, 17, 18

D

- data
  - availability, 393
  - munging, 428
  - ownership, 305
  - processing, origin of, 394
  - recursive functions and, 781–797
- data structures, 723
- data types, 64–65. See also specific types
- dead-end, 804, 811, 813
- debugging, 24, 28
- decimal numbers, 4, 5
- decision tree, 802–803, 806
- declarations
  - array, 449
  - import, 178
  - static method, 153
- declarations, variable, 77–78
  - defined, 77
  - variations, 81–82
- decomposition. See also procedural decomposition
- decrementing, 81, 87, 99
- decrement (--) operator, 87–90
- [redacted] method, 499
- default access, 1190
- default constructor, 555
- defensive programming, 283
- definite loops, 63, 321
- degenerate tree, 1067

- deprecated items, 1178
- dequeueing, 896
- design
  - class, 640–644
  - inheritance and, 625–631
  - object-oriented. See object-oriented design (OOD)
  - procedural, 300–305
- diamond operator, 669
- difference operation, 742
- digital, 4
- digit-matching algorithm, 369
- directories, 409–412
  - current, 409
  - defined, 409
  - recursion, 790–794
  - working, 409
- [redacted] class, 606–608
- division operator (/), 69–70
- DOS, 538
- dot notation, 159, 544
- double backslash characters (\\), 21
- double hashing, 1092
- [redacted] method, 151, 152
- [redacted] type, 65
  - conversion of [redacted] into, 75–76
  - defined, 64
  - in scientific notation, 69
- doubly linked list, 1014
- [redacted] loops, 330–332. See also loops
  - flow of, 331
  - syntax, 331
- [redacted] method, 33–36, 156–157
- [redacted] case study, 40–46

  - final version, 43–44
  - flow of execution, 44–46
  - structured version, 41–42
- [redacted] objects, 202–203, 218
  - case study, 224–230
  - methods, 202, 504, 1172
  - size, 211
- [redacted] method, 203
- [redacted] method, 207, 208
- [redacted] method, 206, 208, 215
- [redacted] method, 215, 220
- [redacted] method, 220
- [redacted] method, 774–776
- dummy node, 1013
- dummy values, 327

E

- effect-free programming, 1120–1122
- elements, 723. See also arrays
  - auto-initialization, 450
  - defined, 448
- empirical analysis, 852
  - complexity, 855–861
- encapsulation, 560–573
  - class invariants, 567–571
  - defined, 560
  - internal implementation changes, 571–572
  - [redacted] class, 611
  - poor, 564
  - private access, 561–567
  - of superclass, 604
- enqueueing, 896
- enumerated types (enum), 1185–1186
- epoch, 859
- equality
  - array, 475–476
  - object, 255–256
- equality operator (==), 246, 247
- [redacted] method, 256
- [redacted] method, 256, 469, 475–477, 610–613, 868
- Erle, Schuyler, 428
- errors. See also exceptions
  - compilation, 24
  - logic, 24, 28
  - off-by-one, 462–463

    - program, 24–28
    - roundoff, 267–270
    - runtime, 24, 39–40
    - syntax, 24–28
    - user, 354–358
- escape sequences, 15–16
- Euclid’s algorithm, 785
- evaluation
    - code, 917–920
    - expression, 65, 908–920
- evaluator, 914–920
- exceptions. See also errors
    - checked, 398
    - defined, 170
    - thrown, 170, 279–283, 949–955
    - types, 950
- execution
    - conditional. See conditional execution
    - flow analysis, 44–46
    - [redacted] loops, 96
    - program, 6
- exponential algorithm, 860
- exponentiation, integer, 781–783
- expressions
    - Boolean, 339–340, 341, 353–354
    - complex, 66
    - defined, 65
    - evaluating, 65
    - operators, 66
    - values, calculating, 79
- [redacted] keyword, 596
- extensions
    - class, 596–600

- file, 8–9

F

- factorials
  - computing, 280
  - defined, 280
  - mathematical definition of, 281
- factoring
  - defined, 257
  - [redacted] statements, 256–258
- fencepost loop, 333–337. See also loops
  - defined, 333
  - with [redacted] statement, 334–337
  - sentinel, 337–338
- fields, 541–543
  - defined, 541
  - encapsulated, 562
  - inherited, accessing, 603–605
  - private, 561–567
  - redeclaring in constructors, 556
- FIFO (First-In, First-Out), 893
- figures, drawing, 40–46
- [redacted], 398–399, 402
- [redacted] objects, 396, 400, 790–791, 1173
- files, 1145–1146
  - absolute path, 409
  - advanced processing, 420–427
  - binary, 395
  - defined, 393
  - extensions, 8–9, 394–395
  - input, 412–414

  - line-based processing, 413–419
  - output, 420–425
  - paths, 409–410
  - processing, 392–446
  - reading, 393–400
  - relative path, 409
  - structure of, 403–407
  - text, 395
  - token-based processing, 401–413
- [redacted] method, 1145
- [redacted] method, 208
- [redacted] method, 799
- [redacted] method, 208, 220
- [redacted] modifier, 1132
- [redacted] keyword, 114
- financial class hierarchy case study, 639–642
- [redacted] method, 911
- first-class citizen, 1123–1124
- first-class functions, 1123–1124
- flags, boolean, 348–349
- floating-point numbers, 72
- [redacted] type, 64, 65
- flow of control, 34–36
  - of [redacted] loops, 331
  - of execution analysis, 44–46
  - of [redacted] statement, 246
  - of [redacted] statement, 248
  - of [redacted] loop, 94
  - of nested [redacted] statements, 250, 251
  - of [redacted] loop, 321
- folders, 409
- [redacted] class, 216
  - constants, 217

- fonts
  - common names, 217
  - defined, 216
  - graphics, 215–218
- for-each loop, 791
  - [redacted] and, 684–685
  - arrays, 467–468, 475
  - iterators, 729–730
  - syntax, 467
  - uses, 467
- [redacted] method, 1142
- [redacted] loop, 63, 92–102, 212, 767, 794, 812–813
  - and [redacted], 679–680
  - body, 93
  - as control structures, 94
  - decrementing, 99
  - execution, 96, 99
  - flow of, 94
  - nested, 100
  - patterns, 98–99
  - scope, 105
  - syntax, 93
  - tracing, 94–97
- formal parameter, 145, 149
- formal verification, 360
- format specifiers, 276–277
- format string, 276
- [redacted] interface, 632
- forward slash (/), 69–70
- fractal geometry, 797
- free variables, 1137, 1138
- functional interfaces, 1129
- functional programming, 1119–1156

    - defined, 1120
    - effect-free programming, 1120–1122
    - first-class functions, 1123–1129
    - function closures, 1136–1139
    - higher-order function, 1139–1146
    - influences on Java, 1122–1123
    - perfect numbers case study, 1146–1156
    - streams, 1129–1136
- function closures, 1136–1139
- functions, recursive, 781–797
    - advantages of, 783
    - execution of, 782–783

G

- garbage collection, 981
- garbage collector, 965
- Gauss, Carl Friedrich, 161
- GB (gigabytes), 5, 6
- GCD (greatest common divisor), 784–787
- generic class, 668–669
- [redacted] method, 507
- [redacted] method, 203, 629
- [redacted] method, 507
- [redacted] method, 427
- [redacted] method, 673, 727, 747
- [redacted] method, 504–505
- [redacted] method, 507
- gigabytes (GB), 5, 6
- [redacted] method, 1124–1125
- global variables, 1121
- graphics
  - case study, 224–230
  - colors, 208–210
  - defined, 201
  - [redacted] class, 202–203
  - fonts, 215–218
  - images, 218–220
  - introduction to, 202–209
  - loops for, 211–215
  - procedural decomposition with, 220–224
  - shapes, 203–207
  - text, 215–218

- [redacted] class, 798–799
- [redacted] class, 629–631
- [redacted] objects, 204–210, 1171
- greater than (>) operator, 246
- greater than or equal to (>=) operator, 246
- greatest common divisor (GCD), 784–787
- growth rates, 857

H

- hailstone sequences, 263
- hard disk, 3
- hardware, 3
- has-a relationship, is-a relationship *vs*., 628
- [redacted] method, 1095–1097
- hash codes, 738
- hash function, 1087
- hashing, 1084–1099
  - array set implementations, 1084–1085
  - collisions, 1087–1092
  - double, 1092
  - functions/tables, 1085–1087
  - map implementation, 1098–1099
  - non-integer data, 1095–1098
  - rehashing, 1092–1095
  - separate chaining, 1088–1092
- [redacted] interface, 746
- [redacted] class, 738–741
- hash table, 738
- [redacted] method, 355, 356
- [redacted] method, 414
- headers
  - class, 11
  - constructors, 553
  - instance methods, 544–545
  - method, 12
- [redacted], 1111
- heaps

    - adding to, 1103–1105
    - array implementation, 1106–1109
    - binary, 1102
    - complete tree, 1102
    - defined, 1102
    - min/max, 1101
    - priority queue, 1099–1101
    - [redacted] methods, 1101
    - removing from, 1103–1104
    - sorting algorithms, 1110–1111
- Hello, world!, 10–14, 421–422
- helper methods, 794–797
- heuristics
    - defined, 300
    - object-oriented design, 574–576
    - procedural design, 300–305
- hierarchies
    - class, 620
    - inheritance, 595–596
    - nonprogramming, 594–596
    - shape class, 638
- higher-order function, 1139–1146
- hinting, 369–372
- Hollerith, Herman, 394
- Hopper, Grace, 28
- hourglass figure case study, 115–123
- [redacted] method, 164

- IBM, 538
- identifiers, 18–19
- IDEs (Integrated Development Environments), 9
- IEEE (Institute of Electrical and Electronics Engineers), 280
- [redacted] statements, 244–258, 772–773, 852, 875–876, 936
  - cumulative sum with, 265–267
  - defined, 245
  - factoring, 256–257
  - flow of, 246
  - multiple conditions, 258
  - nested, 248
  - object equality, 255–256
  - overview, 245
  - relational operators, 246–248
  - syntax, 245
  - wrong construct selection, 253–255
- [redacted] statement
  - defined, 244
  - with fencepost, 334–337
  - flow of, 244, 248
  - sequential, flow of, 249
  - syntax, 244
- [redacted], 282, 578, 949
- [redacted], 951
- illegal values, testing for, 290
- images, 218–220

- immutable objects, 171
- implicit parameter, 546–548, 611
- import declarations, 178
- inchworm approach, 1007–1008
- incrementing, 81, 87
- increment (++) operator, 87–90
- indefinite loops, 321
- indexes
  - arrays, 448
  - defined, 168
  - [redacted], 286
- [redacted] method, 284–285, 675, 841, 953
- [redacted], 952
- infinite loops, 107–108, 320–321
- infinite recursion, 770, 786, 787–790
- infix notation, 818, 819
- inheritance
  - basics, 593–602
  - code reuse, 593
  - defined, 598
  - and design, 625–631
  - extending class, 596–600
  - hierarchies, 595–596
  - interfaces. See interfaces
  - misuse of, 625–627
  - nonprogramming hierarchies, 594–596
  - overriding methods, 600–602
  - polymorphism and, 615–625
  - single, 598
  - subclass, 598
  - superclass, 598, 602–615
- initialization
  - arrays, 450, 456–457

    - objects, 552–560
    - parameters, 151
    - variables, 95
- inner class, 966–968
- inorder traversals, 1034
- input
    - console, 9, 173
    - consuming, 405–406
    - cursor, 405–406
    - files, 412–414
    - tokenizing, 175
- insertion point, 868
- instance methods, 543–546
    - accessors, 548–549, 1122
    - defined, 543
    - headers, 544–545
    - implicit parameters, 546–547
    - mutators, 548–549, 1122
    - syntax, 546
- [redacted] keyword, 613–615
- instances, objects as, 166
- [redacted] class, 897–899
- integers exponentiation, 781–783
- Integrated Development Environments (IDEs), 9
- interactive programs, 173–182
    - defined, 173
    - sample, 176–178
- interfaces, 631–639
    - abstract classes *vs*., 647–652
    - and abstract data types, 731
    - [redacted], 632
    - benefits of, 637–639
    - [redacted], 632

    - [redacted], 724, 732
    - [redacted], 632, 689–701, 848
    - defined, 631
    - [redacted], 632
    - [redacted], 746
    - implementation, 632, 634–637
    - [redacted], 1008–1011
    - [redacted], 731, 1141–1142
    - [redacted], 632
    - [redacted], 632
    - for shapes, 632–634
    - syntax for declaring, 634
    - [redacted], 746
- Internet worms, 458
- intersection operation, 742
- [redacted] interface, 1008–1011
- [redacted] method, 1153
- [redacted] method, 1132–1133
- [redacted] method, 1130, 1131, 1147–1148
- [redacted] type
    - conversion into [redacted], 75–76
    - defined, 64
    - integer literals, 68
- [redacted] method, 686
- invariants, class, 567–571
    - defined, 569
    - enforcing, 569–570
- is-a relationship, 595, 598
    - *vs*. has-a relationship, 628
- [redacted] method, 790
- [redacted] method, 953, 1018
- [redacted] method, 1136, 1137
- iterations, 95, 764

    - solution converted to recursion, 767–769
- iterative approach, 764
- iterative enhancement, 31
- [redacted] interface, 632
- [redacted] objects, methods of, 1174
- iterators, 727–730
    - [redacted], 957–963
    - calling [redacted] on, 729–730
    - defined, 728
    - in for-each loops, 729–730
    - linked lists, 1015–1017
    - methods, 728

J

- jagged arrays, 499–503
- Java
  - bytecodes, 7
  - class libraries, 8
  - coding conventions, 13
  - defined, 7
  - need for, 7–8
  - platform independence, 8
  - programming environment, 8–10
  - runtimes, 7
- [redacted] package, 208, 493, 539, 632
- Java Collections Framework
  - lists, 723–737
  - maps, 745–753
  - sets, 737–745
- Javadoc comments, 23, 1178–1181
- [redacted] package, 632
- [redacted] package, 178
- [redacted], 1145
- Java Runtime Environment (JRE), 7
- [redacted] package, 178, 632, 669, 723, 724, 842, 1088
- Java Virtual Machine (JVM), 7
- JRE (Java Runtime Environment), 7
- JShell, 67–69
- JVM (Java Virtual Machine), 7

K

- KB (kilobytes), 5, 6
- Kernighan, Brian, 14, 28, 103
- [redacted] method, 748–749
- keywords, 1161
  - defined, 19
  - [redacted], 598
  - [redacted], 114
  - [redacted], 613–615
  - [redacted], 174, 494
  - [redacted], 561
  - [redacted], 32
  - reserved, 19–20
  - [redacted], 33, 544
  - [redacted], 557–559
  - [redacted], 33
- kilobytes (KB), 5, 6
- Knuth, Don, 2, 3

L

- lambda expression, 1126–1129
- [redacted] method, 677
- leaf nodes, 1030–1031
  - counting, 1051–1052
- legacy code, 593
- [redacted] method, 543
- less than (<) operator, 246
- less than or equal to (<=) operator, 246
- lexicographic ordering, 1054
- libraries, defined, 8
- LIFO (Last-In, First-Out), 893, 894
- linear algorithm, 859, 861
- linear probing, 1087, 1088
- line-based file processing, 413–419
  - defined, 413
  - token-based combination, 415–419
- lines, drawing, 203–208
- [redacted] class, 1008
  - [redacted] method, 1002–1008
  - appending [redacted], 991–995
  - inchworm approach, 1007–1008
  - middle of the list, 995–1002
  - simple, 989–991
- [redacted] class, 724–726, 841, 989–1002
  - case study, 734–737
- linked lists, 975–1027. See also lists
  - advantage of, 726
  - as array list, 725

    - benefits of, 730
    - code details, 1017–1019
    - construction of, 977–979
    - defined, 725
    - doubly, 1014
    - iterators, 727–730, 1015–1017
    - nodes, 725, 976–988
    - variations, 1012–1014
- list abstraction, 668
- [redacted] method, 791
- [redacted] interface, 632, 731, 1141–1142
- [redacted], 730
- lists, 723–737, 1141–1145
    - capacity, 934
    - circular, 1012–1013
    - complex operations, 1002–1008
    - defined, 723
    - equivalents, 988
    - linked. See linked lists
    - sets/maps *vs*., 753
    - traversing, 985–988
- literals, 68–69
    - [redacted], 69
    - characters, 69
    - defined, 65, 68
    - integers, 68
    - string, 14, 29
- load factor, 1094
- localizing variables, 103
- local variables, 103, 150
- logarithmic algorithm, 859
- logical operators, 340–343, 1162
    - AND (&&), 341

    - defined, 340
    - NOT (!), 341
    - OR (| |), 341
    - short-circuit evaluation, 343–348
- logic errors, 24, 28
- log-linear algorithm, 859
- [redacted] type, 64
- loops
    - [redacted]. See [redacted] loop
    - array, 467–468
    - definite, 63, 321
    - [redacted], 331–333
    - exiting, 1183–1184
    - for-each, 467–468, 475, 684–686
    - for graphics, 211–215
    - indefinite, 63, 321
    - infinite, 107, 324–325
    - iterations, 95
    - [redacted], 261–265
    - priming, 327–328
    - runtime(s), 853
    - sentinel, 337–339
    - traversing, 470, 474
    - [redacted]. See [redacted] loop
- loose coupling, 575
- lottery case study, 743–745

M

- machine language, 6
- Macintosh computer, 538
- magic numbers, 113
- [redacted] method, 36, 147, 774
  - as static method, 32
  - syntax, 12
- managing complexity, 103–115
- Mandelbrot, Benoit, 797
- [redacted] interface, 632
- [redacted] objects, methods of, 1172
- maps, 745–753
  - case study, 750–753
  - construction, 746–747
  - defined, 723, 746
  - [redacted] method, 748–749
  - key/value pair, 746–747
  - methods of, 748
  - operations, 746–747
  - sets/lists *vs*., 753
  - [redacted] *vs*. [redacted], 749–750
  - [redacted] method, 748–749
  - views, 748–749
- [redacted] class, 158–161
  - constants, 160
  - methods of, 1173
  - static methods in, 160–161
- megabytes (MB), 5, 6
- memory

    - defined, 3
    - storage, units of, 6
- merge sort
    - case study, 873–881
    - defined, 873
    - recursive, 876–879
- merging arrays, 873–876
- method call, 33
- method headers, 12
    - parameters in, 144
    - stock fields and, 576–577
- methods
    - [redacted], 161, 270, 1140–1141
    - abstract, 634
    - [redacted], 670
    - [redacted], 1002–1008
    - [redacted], 675
    - [redacted] objects, 1168
    - arrays and, 464–466
    - [redacted], 1141–1142
    - [redacted] class, 470
    - [redacted], 1140–1141
    - [redacted], 842
    - calling another method, 36–38
    - [redacted], 425
    - [redacted] class, 273, 1169
    - [redacted], 168, 285
    - [redacted], 953, 954
    - [redacted] class, 732
    - [redacted], 689
    - [redacted] object, 507
    - [redacted], 848–849
    - [redacted], 692, 848–849

- with conditional execution, 279–290
- [redacted], 675
- [redacted], 469
- [redacted], 469
- coupling, 303–304
- [redacted], 499
- defined, 11
- [redacted], 151
- [redacted], 33–36, 156–157
- [redacted] class, 202, 504, 1170
- [redacted], 203
- [redacted], 206, 208
- [redacted], 206, 208, 215
- [redacted], 215, 220
- [redacted], 220
- [redacted], 774–776
- [redacted], 256, 475–477, 610–613, 868
- [redacted], 256
- [redacted], 1145
- [redacted], 208
- [redacted], 799
- [redacted], 208, 220
- [redacted], 911
- [redacted], 673, 727, 747
- [redacted], 507
- [redacted], 203, 629
- [redacted], 507
- [redacted], 427
- [redacted], 504–505
- [redacted], 507
- [redacted], 1124–1125
- [redacted], 630
- graphics objects, 1171

- [redacted], 1095–1097
- [redacted], 355, 356
- [redacted], 414
- helper, 794–797
- [redacted], 164
- with indentation, 793–794
- [redacted], 284–285, 676–677, 841, 953
- instance, 543–546
- [redacted], 1132–1133
- [redacted], 1130, 1131
- [redacted], 686
- [redacted], 790
- [redacted], 953, 1018
- [redacted] objects, 1172
- iterators, 728
- [redacted], 748–749
- [redacted], 677
- [redacted], 543
- [redacted], 791
- [redacted], 12, 32, 147, 774
- [redacted] objects, 1172
- maps, 748
- [redacted] class, 160–161, 1173
- [redacted], 401
- [redacted], 175, 177, 179, 401, 405, 406
- [redacted], 174, 179, 326, 401
- [redacted], 174, 414
- [redacted] class, 609, 1174
- overloading, 156–157
- overriding, 600–602
- [redacted], 845
- [redacted] objects, 540
- [redacted], 781, 783

- [redacted], 790, 792–793
- [redacted], 1101
- [redacted], 747
- [redacted] interface, 896
- [redacted], 325
- [redacted] objects, 326
- [redacted], 855
- [redacted], 288
- [redacted], 727, 939
- [redacted], 726
- [redacted], 475
- return values, 157–164
- [redacted], 772, 777–780
- [redacted], 851
- [redacted], 209–210
- [redacted], 209–210, 213
- [redacted], 216
- [redacted], 563
- [redacted], 504–505
- [redacted], 847
- signatures, 156–157
- [redacted], 673
- [redacted], 845
- [redacted], 158
- [redacted] class, 894
- static, 31–34, 153
- [redacted] objects, 172
- [redacted], 169–170, 344
- [redacted], 162, 1130
- [redacted], 479, 871
- [redacted], 597
- [redacted], 171, 172
- [redacted]. See [redacted] method

    - [redacted], 171, 172
    - [redacted], 539, 544, 545–546
    - [redacted], 748–749
    - [redacted], 153–154
    - [redacted], 143–146
    - [redacted], 767–769
- mod operator (%), 70, 71
- multidimensional arrays, 497–503. See also arrays
    - defined, 497
    - jagged, 499–503
    - rectangular two-dimensional, 497–499
- multiple constructors, 559–560
- multiple objects, 486–488
- multiplication operator (*), 69–70
- multiplicative operators, 73, 74, 88, 247
- mutators, 548–549, 1122

N

- namespaces, 1188
- native compilers, 6
- natural ordering, 691–694
- nested [redacted] statements, 248–252
  - final branch, 252
  - flow of, 250, 251
  - options, 253
- nested loops
  - algorithms, 495–497
  - code, 101
  - defined, 100
  - [redacted] loop, 100–102
  - trace of, 108
- *The New Hacker's Dictionary*, 38–39
- [redacted] keyword, 174, 494
- [redacted] method, 175, 177, 179, 401, 405–407
- [redacted] method, 174, 179, 326, 401
- [redacted] method, 174, 414
- [redacted] method, 401
- nodes, 976–988
  - branch, 1029
  - defined, 725, 976
  - dummy, 1013
  - leaf, 1030–1031
  - manipulating, 982–985
  - and tree classes, 1032–1033
- nonabstract classes, 651
- nonprogramming hierarchies, 594–596

- [redacted], 912, 960
- not equal to (!=) operator, 246
- NOT operator (!), 341
- [redacted], 578, 615, 950, 1004, 1006
- [redacted] case study, 366–375
- [redacted] parameter, 144, 148
- numbers
  - binary, 4–5
  - decimal, 4, 5
  - magic, 113
  - perfect, case study, 1146–1156
  - pseudorandom, 325
  - random, 325–329
  - smallest divisor, finding, 322–323

O

- [redacted] class, 609–610
  - methods of, 609, 1174
- object-oriented design (OOD), 639, 640
  - heuristics, 574–576
- object-oriented programming (OOP), 485, 536–540, 594
  - defined, 536
- objects, 165–178
  - array, 484, 493–494
  - behavior, 536
  - class, 166
  - defined, 166, 536
  - equality, 255–256
  - exception, 282
  - [redacted], 396, 400
  - [redacted], 203–208
  - immutable, 171
  - initialization, 552–560
  - as instances, 166
  - multiple, 486–488
  - [redacted]. See [redacted] objects
  - [redacted], 420–421
  - [redacted], 326
  - [redacted], 142, 173–178
  - searching, 868–869
  - state, 536, 541–543
  - [redacted], 166–173
- off-by-one bug, 462–463
- offset, 222

- OOD. See object-oriented design (OOD)
- OOP. See object-oriented programming (OOP)
- operands, 66, 819–820
- operating system, 4
  - history, and objects, 538
- operators. See also specific operators
  - additive, 73, 74, 88, 247
  - arithmetic, 69–72, 1161
  - assignment, 83, 88, 247
  - decrement (--), 87–90
  - defined, 66
  - increment (++), 87–90
  - [redacted], 613
  - logical. See logical operators
  - multiplicative, 73, 88, 247
  - precedence, 72–74, 88, 247, 343, 1162
  - relational, 246–248, 1162
  - ternary, 1183
  - unary, 74, 88, 247
- OR operator (| |), 258, 341–343
  - precedence, 343
  - short-circuited evaluation, 346
  - truth table for, 341
- output
  - cursor, 17, 18
  - files, 420–424
  - of program, 9
- overloading methods, 156–157
- overriding methods, 600–602

P

- packages, 1188–1190. See also specific packages
  - declaration syntax, 1189
  - defined, 178
- [redacted] method, 846
- parameters, 143–157
  - actual, 145, 149–150
  - defined, 143
  - formal, 145, 149–150
  - implicit, 546–548
  - initialization, 151
  - limitations of, 150–152
  - local manipulations of, 152
  - mechanics of, 146–150
  - in method headers, 144
  - multiple, 153–155
  - *vs*. constants, 156
- parentheses (()), 93
- Pascal’s triangle, 500–501
- paths
  - defined, 409
  - files, 409–410
  - reasoning about, 288–290
  - relative, 410
- PB (petabyte), 6
- Peirce, Charles, 3
- percent sign (%)
  - mod operator, 70, 71
- percolating, 1103

- perfect numbers case study, 1146–1156
- petabyte (PB), 6
- Pirsig, Robert, 350
- pixels, 202
  - arrays of, 504–509
- [redacted] class, 850
  - encapsulation, 561–567, 612
  - state and behavior, 540–546
- [redacted] class, 625–628
- [redacted] objects, 539–540, 545, 851
  - constructing, 494
  - defined, 539
  - methods of, 540, 1176
- [redacted] class, 799
- polymorphism
  - complex calls interpretation, 622–625
  - inheritance and, 615–625
  - inheritance code interpretation, 620–622
  - mechanics, 618–620
- pop operation, stack, 893
- postconditions, 279
- postdecrement (x--) operator, 88
- postfix notation, 818–819
- postincrement (x++) operator, 88
- postorder traversals, 1034
- [redacted] method, 781, 783
- precedence, 72–74, 88, 247, 343, 1162
  - defined, 72
  - logical operator, 342
- preconditions, 279
- predecrement (--x) operator, 88–89
- prefix evaluator case study, 818–824
- prefix notation, 818–819

- preincrement (++x) operator, 88
- preorder traversals, 1034
- priming loops, 327
- primitive types, 64–65, 616, 686, 1163, 1182
- [redacted] statement, 276, 692
- printing
  - [redacted] class, 932–937
  - [redacted], 669
  - arrays, 471–472
- [redacted] statements, 16–18, 79, 471, 671
  - in [redacted] method, 551–552
- [redacted] method, 790, 792–793
- [redacted] statement, 17–18, 98
- [redacted] class, 420–424
- [redacted] method, 1101
- private access, 561–567, 604
- [redacted] keyword, 561
- probing, 1087, 1088
- procedural decomposition
  - defined, 28
  - example, 30
  - flow of control, 34–36
  - with graphics, 220–224
  - static methods, 31–34
- procedural design heuristics, 300–305
- program errors, 24–28
- programming
  - defensive, 283
  - functional. See functional programming
  - Java environment, 8–10
  - need for, 2–3
  - object-oriented. See object-oriented programming (OOP)

  - process, 6–7
- programs
  - array, 456–460
  - [redacted], 677–678
  - chaining, 304–305
  - client, 537
  - cohesion, 302–303
  - complexity. See complexity
  - defined, 3
  - errors, 24–28
  - execution, 6
  - flow of control, 34–36
  - Hello, world!, 10–14, 421–422
  - interactive, 173–174
  - output of, 9
  - overview of, 10–23
  - performance, 852
  - readability, 21–23
- projectile trajectory case study, 178–186
- prompt, 177
- protected access, 1190
- provable assertion, 359–360
- pruning, decision tree, 808
- pseudocode, 108–113, 357, 367, 795, 804
  - for backtracking solutions, 810–813
  - defined, 108
  - sort algorithm, 871–872
- pseudorandom numbers, 325
- [redacted] keyword, 33
- push operation, stack, 893
- [redacted] method, 747
- pyramids case study, 224–230
- Pythagorean theorem, 163

Q

- quadratic algorithm, 859
- quadratic probing, 1087
- [redacted] interface, methods of, 896
- queues
  - concepts, 896–897
  - defined, 723, 896
  - [redacted] wrapper class, 897–898
  - operations, 897–898
  - priority, 1099–1101
  - removing values from, 904–906
  - sum of, 900–901
  - transfer values to stack, 899–900
- [redacted] method, 902

R

- RAM (random access memory), 3
- random access, 461–462, 727
- random access memory (RAM), 3
- [redacted] method, 325
- random numbers, 325–329
- [redacted] objects, 325–327
  - methods of, 326, 1176
  - misusing, 329
- range algorithm, 855–858
- [redacted] method, 855
- [redacted] method, 288
- Raymond, Eric, 38
- readability, program, 21–23
- reading files, 393–400
  - beyond end, 399–400
  - with [redacted] objects, 396–399
  - structure of files, 403–407
- reasoning, assertion, 360–362
- records, 543, 604–605
- [redacted] class, 635
- rectangles, area and perimeter, 633
- rectangular two-dimensional arrays, 497–499
- recurrence relations, 867
- recursion, 763–839
  - backtracking, 801–818
  - base case, 770–771
  - binary search, 865–868
  - bottom of, 766

    - case study, 818–824
    - data, 781–797
    - defined, 764
    - directories, 790–794
    - example, 771–781
    - file-reversing method, 776–781
    - functions, 781–797
    - graphics, 797–801
    - helper methods, 794–797
    - infinite, 770, 786, 787–789
    - iterative solution converted to, 767–769
    - mechanics of, 773–781
    - merge sort, 876–879
    - nonprogramming example, 764–767
    - recursive case, 770–771
    - solution structure, 769–771
- recursive case, 770–771, 782, 785, 820
- recursive merge sort, 876–879
- [redacted], stream terminator, 1134–1135
- refactoring, 643–644
- reference semantics, 484–488
    - defined, 484–485
    - multiple objects, 486–488
    - reasons for, 485–486
- rehashing, 1092–1095
- relational operators, 246–248, 1164
    - limitation of, 248
    - types, 247
- relationships
    - has-a, 628
    - is-a, 595, 598
- relative file path, 409
- remainder operator (%), 70–71

- [redacted] method, 726
- [redacted] method, 727, 940
- [redacted] method, 475
- [redacted] statement, 161–162, 288
- return type, 158
- return values, 157–165
  - ignoring, 163
- [redacted] method, 772, 777–780
- [redacted] method, 851
- RGB value, 504, 507
- Ritchie, Dennis, 14
- robust programs, 354
- root, of binary tree, 1029, 1031–1032
- roundoff errors, 267–270
  - defined, 268
  - outcomes, 268–269
- [redacted] interface, 632
- running programs, 6
- runtimes, 613
  - of algorithm, 855–858
  - binary search algorithm, 864–865
  - errors, 24, 39–40
  - of loop, 853
  - statements, 853

S

- [redacted] class, 173–178, 910
  - interactive program with, 176–178
  - lookahead, 354–356
- [redacted] objects, 142
  - in file processing, 407
  - methods, 174, 1177
  - parameters, 407–409
  - reading files with, 396–399
- scope, 103–107, 545
  - defined, 103, 545
  - implications, 103
  - outer, 105
- searching
  - [redacted], 673–677
  - arrays, 472–475
  - binary, 842–845, 862–865, 867–868
  - implementing algorithms, 861–872
  - in Java’s class libraries, 847
  - libraries, 841–851
  - objects, 868–869
  - recursive binary, 865–867
  - sequential search, 861–862
- [redacted], 1067–1071
- selection sort, 869–872
- self-check problems
  - [redacted], 714–717
  - arrays, 520–527
  - classes, 583–585

    - conditional execution, 306–314
    - file processing, 435–444
    - functional programming, 1157–1159
    - graphics, 230–232
    - Java Collections Framework, 754–758
    - Java programming introduction, 47–54
    - linked lists, 1020–1023
    - parameters and objects, 187–195
    - primitive data and definite loops, 124–132
    - program logic and indefinite loops, 376–386
    - recursion, 825–831
    - searching and sorting, 883–887
    - stacks and queues, 920–924
- semicolon (;), 12, 15, 26–27
- sentinel loops, 337–339
- separate chaining, 1088–1092
- sequences
    - escape, 15–16
    - hailstone, 263
- sequential access, 461
- sequential algorithms, 447
- sequential search, 841
    - of array, 861–862
- [redacted] interface, 632
- [redacted] method, 209–210
- [redacted] collection, 737
- [redacted] method, 209–210, 213
- [redacted] method, 216
- [redacted] interface, 632
- [redacted] method, 563
- [redacted] method, 504–505
- sets, 737–745
    - case study, 743–745

    - concepts, 738–740
    - defined, 723, 737
    - maps/lists *vs*., 753
    - operations, 741–743
    - [redacted] *vs*. [redacted], 738–741
- shapes
    - area and perimeter formulas, 633
    - drawing, 203–208
    - interfaces for, 632–634
- [redacted] class, 645–646
- short-circuit evaluation, 343–348
- [redacted] type, 64
- [redacted] method, 846
- shuffling, 846–847
- side effect, 1121
- Sierpinski triangle, 797–798
- sieve algorithm case study, 734–737
- signatures, method, 156–157
- simulations, 329–331
- single inheritance, 598
- [redacted] method, 673
- smallest divisor, 322–323
- software, 4
    - crisis, 593
- sorted array, 1084
- sorting, 845–846, 852
    - [redacted] method, 845
    - [redacted] method, 845
    - [redacted] method, 846
    - implementing algorithms, 861–872
    - in Java’s class libraries, 847
    - selection sort, 869–872
- [redacted] method, 845

- spaces
  - [redacted] objects, 168
  - writing, 143–146
- space-writing task, 143–144
- specifiers, format, 276–277
- splitting arrays, 873–876
- [redacted] method, 158–159
- stack, 893–895
  - comparison for similarity, 906–908
  - defined, 723, 894
  - [redacted] wrapper class, 897–898
  - operations, 897–904
  - sum of, 901–904
  - transfer values to queue, 899–900
- [redacted] class, methods of, 894
- stack trace, 170
- state, objects, 541–543
  - defined, 536
- statements
  - [redacted], 1187
  - assignment, 79–83
  - [redacted], 1185–1186
  - [redacted], 1183–1184
  - defined, 12
  - [redacted], 772–773. See [redacted] statements
  - [redacted], 17–18, 98
  - [redacted], 707
  - [redacted], 16–18, 79, 671
  - prompt, 177
  - [redacted], 161–162, 288
  - runtimes, 853
  - [redacted], 1184–1186
  - [redacted], 281

    - [redacted], 1186–1187
- [redacted] keyword, 33, 544
- static methods, 31–34
    - defined, 32
    - in [redacted] class, 160–161
    - structure, 32
    - syntax, 153
- Steele, Guy, 1122
- stepwise refinement, 31
- [redacted] class case study, 573–582
- streams, 1129–1136
    - defined, 1129
    - using filter, 1132–1134
    - using map, 1131–1132
    - using [redacted], 1134–1135
- [redacted] class, 693–694
- string concatenation, 84–87
- [redacted], 347
- string literals, 14, 29
- [redacted] objects, 166–173
    - index, 168
    - length, 167–168
    - methods of, 172, 1177
    - spaces, 168
- strings
    - defined, 14
    - format, 276
    - line breaks inside, 15–16
    - programming error, 29
    - traversal algorithm, 481–482
- struct, 543
- subclasses, 598. See also inheritance
- subset, 742

- substitutability, 627
- [redacted] method, 169–170, 344, 1130
- [redacted] method, 162, 1130
- superclasses. See also inheritance
  - accessing inherited fields, 603–605
  - constructors, calling, 605–607
  - declaration syntax, 598
  - defined, 598
  - interacting with, 602–615
  - overridden methods, 602–603
- superset, 742
- Sussman, Gerry, 1122
- [redacted] method, 479, 871
- [redacted] statement, 1172–1174
- syntax
  - abstract classes, 648
  - abstract method declaration, 649
  - array declaration and construction, 449
  - array initialization, 455–456
  - constant definitions, 114
  - constructors, 555
  - [redacted] loops, 331
  - encapsulated field declaration, 562
  - errors, 24–28
  - for-each loop, 467–468
  - instance methods, 546
  - interface declaration, 634
  - [redacted] loop, 93
  - packages declaration, 1189
  - [redacted] statement, 162
  - static method declaration, 153
  - superclass declaration, 598
  - template, 11

    - templates, 1163–1167
    - keyword, 557–559
    - statement, 281
    - loop, 321
- method, 1150–1151
- , 173
- , 274–278
- , 172

T

- tags, Javadoc comments, 1181
- [redacted] method, 597
- tallying values, 510–514
- terabyte (TB), 6
- terminal interaction, 9
- ternary operator, 1183
- text
  - cumulative algorithms, 272–274
  - files, 395
  - processing, 270–278
- [redacted] keyword, 557–559
- throwing exceptions, 170, 279–283, 949–952
- [redacted] clause, 398, 425
- [redacted] statement, 281
- time complexity, 852
- [redacted] class, 571
- token-based processing, 401–413. See also files
  - defined, 401
  - input file, 412–413
  - line-based combination, 415–419
  - paths and directories, 409–411
  - [redacted] parameters, 407–409
  - structure of files, 403–407
- tokens
  - defined, 174
  - reading wrong kind of, 404
  - splitting, 909–914
  - whitespace-separated, 405

- [redacted] method, 171–172, 1143
- [redacted] method, 469, 499, 671, 738, 747, 895, 936, 994
  - implementing, 551
  - [redacted] statement in, 551–552
- [redacted] method, 171–172
- [redacted] method, 539, 544, 545–546
- traversing loops, 470, 474
- [redacted] class, 746, 850, 1053
  - *vs.* [redacted] class, 749–750
- [redacted] class, 723, 738–741, 850, 1053
- tree traversals, 1033–1048. See also binary tree
  - constructing and viewing, 1039–1048
  - preorder/postorder/inorder, 1034
- triangle, area and perimeter, 633
- [redacted] class, 637
- [redacted] statement, 1186–1187
- two-dimensional arrays, 497–499
- types
  - [redacted], 64, 69, 320, 339–354
  - [redacted], 64
  - [redacted], 64
  - [redacted], 64–65
  - [redacted], 64–65
  - [redacted], 64, 271–272
  - [redacted], 64
  - mixing, 74–76
  - primitive, 64–65
  - [redacted], 64
- type-safe language, 64

U

- unary operators, 74, 88, 247
- unboxing, 688, 690–691
- underscore (_) character, 19
- Unicode, 275
- uninitialized variables, 78
- union operation, 742
- unsorted array, 1084
- user errors, 354–358

V

- [redacted] method, 748–749
- values
  - adding to [redacted], 671–672
  - comparison function, 691
  - natural ordering, 691–694
  - removing from [redacted], 672
  - semantics, 484–485
  - shifting in arrays, 488–493
  - tallying, 510–514
  - types, 484
- value semantics, 485
- variables, 76–92
  - assigning, 78–80
  - assignment/declaration variations, 81–84
  - [redacted] type, 339, 348–350
  - bound, 1137–1138
  - declarations, 77–78
  - decrementing, 81, 87
  - defined, 76
  - free, 1137–1138
  - global, 1121
  - incrementing, 81, 87
  - initial value, 81
  - local, 103, 150
  - localizing, 103
  - loop, 95
  - mixing types and, 90–91
  - in [redacted] statement, 79

    - uninitialized, 78
- Venn diagrams, 742
- verification, program, 360
- [redacted] keyword, 33, 555–556

W

- [redacted] loop, 321–333. See also loops
  - defined, 320
  - [redacted] loops, 331–333
  - flow of, 321
  - priming, 327
  - random numbers, 325–328
  - simulations, 329–331
  - smallest divisor, 322–323
  - syntax of, 321
- whitespace, 174, 405
- [redacted] case study, 750–753
- working directory, 409
- worst case analysis, 862
- wrapper classes, 686–688, 1186
  - boxing, 688
  - common, 688
  - defined, 686
  - unboxing, 688
- [redacted] methods, 153–154
- [redacted] method, 143–145
- [redacted] method, 767–769

Y

- Y2K (millennium) bug, 564

Z

- Zen, Boolean, 350–352
- zero-based indexing, 448, 671
- zip code lookup case study, 427–434

Contents

1. Building Java Programs A Back to Basics Approach
2. Preface
   A. New to This Edition
   B. Features from Prior Editions
   C. Layers and Dependencies
   D. Supplements
   E. MyLab Programming
   F. VideoNotes
3. Location of Video Notes in the Text
   http://www.pearson.com/cs-resources
4. Brief Contents
5. Contents
6. Chapter 1 Introduction to Java Programming
   A. Introduction
   B. 1.1 Basic Computing Concepts
      1. Why Programming?
      2. Hardware and Software
      3. The Digital Realm
      4. The Process of Programming
      5. Why Java?
      6. The Java Programming Environment
   C. 1.2 And Now—Java
      1. String Literals (Strings)
      2. [redacted]
      3. Escape Sequences
      4. [redacted] versus [redacted]
      5. Identifiers and Keywords
      6. A Complex Example: [redacted]
      7. Comments and Readability

    D. 1.3 Program Errors
        1. Syntax Errors
        2. Logic Errors (Bugs)
    E. 1.4 Procedural Decomposition
        1. Static Methods
        2. Flow of Control
        3. Methods That Call Other Methods
        4. An Example Runtime Error
    F. 1.5 Case Study: [redacted]
        1. Structured Version
        2. Final Version without Redundancy
        3. Analysis of Flow of Execution
    G. Chapter Summary
    H. Self-Check Problems
        1. Section 1.1: Basic Computing Concepts
        2. Section 1.2: And Now—Java
        3. Section 1.3: Program Errors
        4. Section 1.4: Procedural Decomposition
    I. Exercises
    J. Programming Projects
7. Chapter 2 Primitive Data and Definite Loops
    A. Introduction
    B. 2.1 Basic Data Concepts
        1. Primitive Types
        2. Expressions
        3. JShell
        4. Literals
        5. Arithmetic Operators
        6. Precedence
        7. Mixing Types and Casting
    C. 2.2 Variables
        1. Assignment/Declaration Variations
        2. String Concatenation

3. Increment/Decrement Operators
4. Variables and Mixing Types

D. 2.3 The [redacted] Loop
1. Tracing [redacted] Loops
2. [redacted] Loop Patterns
3. Nested [redacted] Loops

E. 2.4 Managing Complexity
1. Scope
2. Pseudocode
3. The Table Technique
4. Class Constants

F. 2.5 Case Study: Hourglass Figure
1. Problem Decomposition and Pseudocode
2. Initial Structured Version
3. Adding a Class Constant
4. Further Variations

G. Chapter Summary

H. Self-Check Problems
1. Section 2.1: Basic Data Concepts
2. Section 2.2: Variables
3. Section 2.3: The [redacted] Loop
4. Section 2.4: Managing Complexity

I. Exercises

J. Programming Projects

8. Chapter 3 Introduction to Parameters and Objects

A. Introduction

B. 3.1 Parameters
1. The Mechanics of Parameters
2. Limitations of Parameters
3. Multiple Parameters
4. Parameters versus Constants
5. Overloading of Methods

C. 3.2 Methods That Return Values

1. The [redacted] Class
2. Defining Methods That Return Values

D. 3.3 Using Objects
1. [redacted] Objects
2. Interactive Programs and Scanner Objects
3. Sample Interactive Program

E. 3.4 Case Study: Projectile Trajectory
1. Unstructured Solution
2. Structured Solution

F. Chapter Summary

G. Self-Check Problems
1. Section 3.1: Parameters
2. Section 3.2: Methods That Return Values
3. Section 3.3: Using Objects

H. Exercises

I. Programming Projects

9. Supplement 3G Graphics (Optional)

A. Introduction

B. 3G.1 Introduction to Graphics
1. DrawingPanel
2. Drawing Lines and Shapes
3. Colors
4. Drawing with Loops
5. Text and Fonts
6. Images

C. 3G.2 Procedural Decomposition with Graphics
1. A Larger Example: [redacted]

D. 3G.3 Case Study: Pyramids
1. Unstructured Partial Solution
2. Generalizing the Drawing of Pyramids
3. Complete Structured Solution

E. Chapter Summary

F. Self-Check Problems

1. Section 3G.1: Introduction to Graphics
G. Exercises
H. Programming Projects
10. Chapter 4 Conditional Execution
A. Introduction
B. 4.1 [redacted] Statements
1. Relational Operators
2. Nested [redacted] Statements
3. Object Equality
4. Factoring [redacted] Statements
5. Testing Multiple Conditions
C. 4.2 Cumulative Algorithms
1. Cumulative Sum
2. Min/Max Loops
3. Cumulative Sum with [redacted]
4. Roundoff Errors
D. 4.3 Text Processing
1. The [redacted] Type
2. [redacted] versus [redacted]
3. Cumulative Text Algorithms
4. [redacted]
E. 4.4 Methods with Conditional Execution
1. Preconditions and Postconditions
2. Throwing Exceptions
3. Revisiting Return Values
4. Reasoning about Paths
F. 4.5 Case Study: Body Mass Index
1. One-Person Unstructured Solution
2. Two-Person Unstructured Solution
3. Two-Person Structured Solution
4. Procedural Design Heuristics
G. Chapter Summary
H. Self-Check Problems

1. Section 4.1: [redacted] Statements
2. Section 4.2: Cumulative Algorithms
3. Section 4.3: Text Processing
4. Section 4.4: Methods with Conditional Execution

I. Exercises
J. Programming Projects

11. Chapter 5 Program Logic and Indefinite Loops

A. Introduction
B. 5.1 The [redacted] Loop
   1. A Loop to Find the Smallest Divisor
   2. Random Numbers
   3. Simulations
   4. [redacted] Loop
C. 5.2 Fencepost Algorithms
   1. Fencepost with [redacted]
   2. Sentinel Loops
D. 5.3 The [redacted] Type
   1. Logical Operators
   2. Short-Circuited Evaluation
   3. [redacted] Variables and Flags
   4. Boolean Zen
   5. Negating Boolean Expressions
E. 5.4 User Errors
   1. [redacted] Lookahead
   2. Handling User Errors
F. 5.5 Assertions and Program Logic
   1. Reasoning about Assertions
   2. A Detailed Assertions Example
G. 5.6 Case Study: [redacted]
   1. Initial Version without Hinting
   2. Randomized Version with Hinting
   3. Final Robust Version
H. Chapter Summary

I. Self-Check Problems
1. Section 5.1: The [redacted] Loop
2. Section 5.2: Fencepost Algorithms
3. Section 5.3: The boolean Type
4. Section 5.4: User Errors
5. Section 5.5: Assertions and Program Logic
J. Exercises
K. Programming Projects
12. Chapter 6 File Processing
A. Introduction
B. 6.1 File-Reading Basics
1. Data, Data Everywhere
2. Files and File Objects
3. Reading a File with a [redacted]
C. 6.2 Details of Token-Based Processing
1. Structure of Files and Consuming Input
2. Scanner Parameters
3. Paths and Directories
4. A More Complex Input File
D. 6.3 Line-Based Processing
1. [redacted] [redacted] and Line/Token Combinations
E. 6.4 Advanced File Processing
1. Output Files with [redacted]
2. Guaranteeing That Files Can Be Read
F. 6.5 Case Study: Zip Code Lookup
G. Chapter Summary
H. Self-Check Problems
1. Section 6.1: File-Reading Basics
2. Section 6.2: Details of Token-Based Processing
3. Section 6.3: Line-Based Processing
4. Section 6.4: Advanced File Processing
I. Exercises
J. Programming Projects

13. Chapter 7 Arrays
    A. Introduction
    B. 7.1 Array Basics
        1. Constructing and Traversing an Array
        2. Accessing an Array
        3. Initializing Arrays
        4. A Complete Array Program
        5. Random Access
        6. Arrays and Methods
        7. The For-Each Loop
        8. The [redacted] Class
    C. 7.2 Array-Traversal Algorithms
        1. Printing an Array
        2. Searching and Replacing
        3. Testing for Equality
        4. Reversing an Array
        5. String Traversal Algorithms
        6. Functional Approach
    D. 7.3 Reference Semantics
        1. Multiple Objects
    E. 7.4 Advanced Array Techniques
        1. Shifting Values in an Array
        2. Arrays of Objects
        3. Command-Line Arguments
        4. Nested Loop Algorithms
    F. 7.5 Multidimensional Arrays
        1. Rectangular Two-Dimensional Arrays
        2. Jagged Arrays
    G. 7.6 Arrays of Pixels
    H. 7.7 Case Study: Benford's Law
        1. Tallying Values
        2. Completing the Program
    I. Chapter Summary

J. Self-Check Problems
    1. Section 7.1: Array Basics
    2. Section 7.2: Array-Traversal Algorithms
    3. Section 7.3: Reference Semantics
    4. Section 7.4: Advanced Array Techniques
    5. Section 7.5: Multidimensional Arrays
K. Exercises
L. Programming Projects

14. Chapter 8 Classes
A. Introduction
B. 8.1 Object-Oriented Programming
    1. Classes and Objects
    2. [redacted] Objects
C. 8.2 Object State and Behavior
    1. Object State: Fields
    2. Object Behavior: Methods
    3. The Implicit Parameter
    4. Mutators and Accessors
    5. The [redacted] Method
D. 8.3 Object Initialization: Constructors
    1. The Keyword [redacted]
    2. Multiple Constructors
E. 8.4 Encapsulation
    1. Private Fields
    2. Class Invariants
    3. Changing Internal Implementations
F. 8.5 Case Study: Designing a [redacted] Class
    1. Object-Oriented Design Heuristics
    2. [redacted] Fields and Method Headers
    3. [redacted] Method and Constructor Implementation
G. Chapter Summary
H. Self-Check Problems
    1. Section 8.1: Object-Oriented Programming

2. Section 8.2: Object State and Behavior
3. Section 8.3: Object Initialization: Constructors
4. Section 8.4: Encapsulation
5. Section 8.5: Case Study: Designing a Stock Class

I. Exercises
J. Programming Projects

15. Chapter 9 Inheritance and Interfaces

A. Introduction
B. 9.1 Inheritance Basics
1. Nonprogramming Hierarchies
2. Extending a Class
3. Overriding Methods
C. 9.2 Interacting with the Superclass
1. Calling Overridden Methods
2. Accessing Inherited Fields
3. Calling a Superclass's Constructor
4. [redacted] Behavior
5. The [redacted] Class
6. The [redacted] Method
7. The [redacted] Keyword
D. 9.3 Polymorphism
1. Polymorphism Mechanics
2. Interpreting Inheritance Code
3. Interpreting Complex Calls
E. 9.4 Inheritance and Design
1. A Misuse of Inheritance
2. Is-a Versus Has-a Relationships
3. [redacted]
F. 9.5 Interfaces
1. An Interface for Shapes
2. Implementing an Interface
3. Benefits of Interfaces
G. 9.6 Case Study: Financial Class Hierarchy

1. Designing the Classes
2. Redundant Implementation
3. Abstract Classes

H. Chapter Summary
I. Self-Check Problems
1. Section 9.1: Inheritance Basics
2. Section 9.2: Interacting with the Superclass
3. Section 9.3: Polymorphism
4. Section 9.4: Inheritance and Design
5. Section 9.5: Interfaces
6. Section 9.6: Case Study: Financial Class Hierarchy

J. Exercises
K. Programming Projects

16. Chapter 10 ArrayLists
A. Introduction
B. 10.1 [redacted]
1. Basic [redacted] Operations
2. [redacted] Searching Methods
3. A Complete [redacted] Program
4. Adding to and Removing from an [redacted]
5. Initializing an [redacted]
6. Using the For-Each Loop with [redacted]s
7. Wrapper Classes

C. 10.2 The [redacted] Interface
1. Natural Ordering and [redacted]
2. Implementing the [redacted] Interface

D. 10.3 Case Study: Ranked Choice Voting
1. Ballot Class
2. Counting Votes
3. Multiple Rounds

E. Chapter Summary
F. Self-Check Problems
1. Section 10.1: [redacted]

2. Section 10.2: The [redacted] Interface
G. Exercises
H. Programming Projects
17. Chapter 11 Java Collections Framework
A. Introduction
B. 11.1 Lists
1. Collections
2. [redacted] versus [redacted]
3. Iterators
4. Abstract Data Types (ADTs)
5. [redacted] Case Study: [redacted]
C. 11.2 Sets
1. Set Concepts
2. [redacted] versus [redacted]
3. Set Operations
4. Set Case Study: [redacted]
D. 11.3 Maps
1. Basic Map Operations
2. Map Views ([redacted] and [redacted])
3. [redacted] versus [redacted]
4. Map Case Study: [redacted]
5. Collection Overview
E. Chapter Summary
F. Self-Check Problems
1. Section 11.1: Lists
2. Section 11.2: Sets
3. Section 11.3: Maps
G. Exercises
H. Programming Projects
18. Chapter 12 Recursion
A. Introduction
B. 12.1 Thinking Recursively
1. A Nonprogramming Example

2. An Iterative Solution Converted to Recursion
3. Structure of Recursive Solutions

C. 12.2 A Better Example of Recursion
1. Mechanics of Recursion

D. 12.3 Recursive Functions and Data
1. Integer Exponentiation
2. Greatest Common Divisor
3. Directory Crawler
4. Helper Methods

E. 12.4 Recursive Graphics

F. 12.5 Recursive Backtracking
1. A Simple Example: Traveling North/East
2. 8 Queens Puzzle
3. Solving Sudoku Puzzles

G. 12.6 Case Study: Prefix Evaluator
1. Infix, Prefix, and Postfix Notation
2. Evaluating Prefix Expressions
3. Complete Program

H. Chapter Summary

I. Self-Check Problems
1. Section 12.1: Thinking Recursively
2. Section 12.2: A Better Example of Recursion
3. Section 12.3: Recursive Functions and Data
4. Section 12.4: Recursive Graphics
5. Section 12.5: Recursive Backtracking

J. Exercises

K. Programming Projects

19. Chapter 13 Searching and Sorting

A. Introduction

B. 13.1 Searching and Sorting in the Java Class Libraries
1. Binary Search
2. Sorting
3. Shuffling

4. Custom Ordering with Comparators
C. 13.2 Program Complexity
1. Empirical Analysis
2. Complexity Classes
D. 13.3 Implementing Searching and Sorting Algorithms
1. Sequential Search
2. Binary Search
3. Recursive Binary Search
4. Searching Objects
5. Selection Sort
E. 13.4 Case Study: Implementing Merge Sort
1. Splitting and Merging Arrays
2. Recursive Merge Sort
3. Complete Program
F. Chapter Summary
G. Self-Check Problems
1. Section 13.1: Searching and Sorting in the Java Class Libraries
2. Section 13.2: Program Complexity
3. Section 13.3: Implementing Searching and Sorting Algorithms
4. Section 13.4: Case Study: Implementing Merge Sort
H. Exercises
I. Programming Projects
20. Chapter 14 Stacks and Queues
A. Introduction
B. 14.1 Stack/Queue Basics
1. Stack Concepts
2. Queue Concepts
C. 14.2 Common Stack/Queue Operations
1. Transferring between Stacks and Queues
2. Sum of a Queue
3. Sum of a Stack

D. 14.3 Complex Stack/Queue Operations
    1. Removing Values from a Queue
    2. Comparing Two Stacks for Similarity
E. 14.4 Case Study: Expression Evaluator
    1. Splitting into Tokens
    2. The Evaluator
F. Chapter Summary
G. Self-Check Problems
    1. Section 14.1: Stack/Queue Basics
    2. Section 14.2: Common Stack/Queue Operations
    3. Section 14.3: Complex Stack/Queue Operations
H. Exercises
I. Programming Projects
21. Chapter 15 Implementing a Collection Class
A. Introduction
B. 15.1 Simple [redacted]
    1. Adding and Printing
    2. Thinking about Encapsulation
    3. Dealing with the Middle of the List
    4. Another Constructor and a Constant
    5. Preconditions and Postconditions
C. 15.2 A More Complete [redacted]
    1. Throwing Exceptions
    2. Convenience Methods
D. 15.3 Advanced Features
    1. Resizing When Necessary
    2. Adding an Iterator
E. 15.4 [redacted]
F. Chapter Summary
G. Self-Check Problems
    1. Section 15.1: Simple [redacted]
    2. Section 15.2: A More Complete [redacted]
    3. Section 15.3: Advanced Features

4. Section 15.4: [redacted]
H. Exercises
I. Programming Projects
22. Chapter 16 Linked Lists
A. Introduction
B. 16.1 Working with Nodes
1. Constructing a List
2. List Basics
3. Manipulating Nodes
4. Traversing a List
C. 16.2 A Linked List Class
1. Simple [redacted]
2. Appending [redacted]
3. The Middle of the List
D. 16.3 A Complex List Operation
1. Inchworm Approach
E. 16.4 An [redacted] Interface
F. 16.5 [redacted]
1. Linked List Variations
2. Linked List Iterators
3. Other Code Details
G. Chapter Summary
H. Self-Check Problems
1. Section 16.1: Working with Nodes
2. Section 16.2: A Linked List Class
3. Section 16.3: A Complex List Operation
4. Section 16.4: An [redacted] Interface
5. Section 16.5: [redacted]
I. Exercises
J. Programming Projects
23. Chapter 17 Binary Trees
A. Introduction
B. 17.1 Binary Tree Basics

1. Node and Tree Classes
C. 17.2 Tree Traversals
1. Constructing and Viewing a Tree
D. 17.3 Common Tree Operations
1. Sum of a Tree
2. Counting Levels
3. Counting Leaves
E. 17.4 Binary Search Trees
1. The Binary Search Tree Property
2. Building a Binary Search Tree
3. The Pattern
4. Searching the Tree
5. Binary Search Tree Complexity
F. 17.5
G. Chapter Summary
H. Self-Check Problems
1. Section 17.1: Binary Tree Basics
2. Section 17.2: Tree Traversals
3. Section 17.3: Common Tree Operations
4. Section 17.4: Binary Search Trees
5. Section 17.5:
I. Exercises
J. Programming Projects
24. Chapter 18 Advanced Data Structures
A. Introduction
B. 18.1 Hashing
1. Array Set Implementations
2. Hash Functions and Hash Tables
3. Collisions
4. Rehashing
5. Hashing Non-Integer Data
6. Hash Map Implementation
C. 18.2 Priority Queues and Heaps

1. Priority Queues
2. Introduction to Heaps
3. Removing from a Heap
4. Adding to a Heap
5. Array Heap Implementation
6. Heap Sort

D. Chapter Summary
E. Self-Check Problems
1. Section 18.1: Hashing
2. Section 18.2: Priority Queues and Heaps

F. Exercises
G. Programming Projects

25. Chapter 19 Functional Programming with Java 8
A. Introduction
B. 19.1 Effect-Free Programming
C. 19.2 First-Class Functions
1. Lambda Expressions

D. 19.3 Streams
1. Basic Idea
2. Using Map
3. Using Filter
4. Using Reduce
5. Optional Results

E. 19.4 Function Closures
F. 19.5 Higher-Order Operations on Collections
1. Working with Arrays
2. Working with Lists
3. Working with Files

G. 19.6 Case Study: Perfect Numbers
1. Computing Sums
2. Incorporating Square Root
3. Just Five and Leveraging Concurrency

H. Chapter Summary

I. Self-Check Problems
1. 19.1 Effect-Free Programming
2. 19.2 First-Class Functions
3. 19.3 Streams
4. 19.4 Function Closures
5. 19.5 Higher-Order Operations on Collections
J. Exercises
K. Programming Projects
26. Appendix A Java Summary
A. Java Keywords
B. Primitive Types
C. Arithmetic Operators
D. Relational Operators
E. Logical Operators
F. Operator Precedence
G. Wrapper Classes
H. Syntax Templates
I. Useful Methods of [redacted] Objects
J. Useful Methods of the [redacted] Class
K. Useful Methods of the [redacted] Interface
L. Useful Methods of the [redacted] Class
M. Useful Methods of [redacted] Objects
N. Useful Methods of [redacted] Objects
O. Useful Methods of [redacted] Objects
P. Useful Methods of [redacted] Objects
Q. Useful Methods of [redacted] Objects
R. Constants and Useful Methods of the [redacted] Class
S. Useful Methods of the [redacted] Class
T. Useful Methods of [redacted] Objects
U. Useful Methods of [redacted] Objects
V. Useful Methods of [redacted] Objects
W. Useful Methods of [redacted] Objects

27. Appendix B The Java API Specification and Javadoc Comments
    A. The Java API Specification
        1. Parameters
        2. Returns
        3. Throws
        4. Deprecated
    B. Writing Javadoc Comments
28. Appendix C Additional Java Syntax
    A. Primitive Types: [redacted], [redacted], [redacted], [redacted]
    B. Ternary Operator ? :
    C. Exiting a Loop: [redacted] and [redacted]
    D. The [redacted] Statement
    E. The [redacted] Statement
    F. The [redacted] Statement
    G. Enumerations: [redacted]
    H. Packages
    I. Protected and Default Access
29. Index
    A. A
    B. B
    C. C
    D. D
    E. E
    F. F
    G. G
    H. H
    I. I
    J. J
    K. K
    L. L
    M. M
    N. N

O. O
P. P
Q. Q
R. R
S. S
T. T
U. U
V. V
W. W
X. Y
Y. Z

List of Illustrations

Landmarks

1. Brief Contents
2. Frontmatter
3. Start of Content
4. backmatter
5. List of Illustrations

1. i
2. ii
3. iii
4. iv
5. v
6. vi

7. vii
8. viii
9. ix
10. x
11. xi
12. xii
13. xiii
14. xiv
15. xv
16. xvi
17. xvii
18. xviii
19. xix
20. xx
21. xxi
22. xxii
23. xxiii
24. xxiv
25. xxv
26. xxvi
27. xxvii
28. xxviii
29. 1
30. 2
31. 3
32. 4
33. 5
34. 6
35. 7
36. 8
37. 9
38. 10
39. 11

40. 12
41. 13
42. 14
43. 15
44. 16
45. 17
46. 18
47. 19
48. 20
49. 21
50. 22
51. 23
52. 24
53. 25
54. 26
55. 27
56. 28
57. 29
58. 30
59. 31
60. 32
61. 33
62. 34
63. 35
64. 36
65. 37
66. 38
67. 39
68. 40
69. 41
70. 42
71. 43
72. 44

73. 45
74. 46
75. 47
76. 48
77. 49
78. 50
79. 51
80. 52
81. 53
82. 54
83. 55
84. 56
85. 57
86. 58
87. 59
88. 60
89. 61
90. 62
91. 63
92. 64
93. 65
94. 66
95. 67
96. 68
97. 69
98. 70
99. 71
100. 72
101. 73
102. 74
103. 75
104. 76
105. 77

106.78
107.79
108.80
109.81
110.82
111.83
112.84
113.85
114.86
115.87
116.88
117.89
118.90
119.91
120.92
121.93
122.94
123.95
124.96
125.97
126.98
127.99
128.100
129.101
130.102
131.103
132.104
133.105
134.106
135.107
136.108
137.109
138.110

139.111
140.112
141.113
142.114
143.115
144.116
145.117
146.118
147.119
148.120
149.121
150.122
151.123
152.124
153.125
154.126
155.127
156.128
157.129
158.130
159.131
160.132
161.133
162.134
163.135
164.136
165.137
166.138
167.139
168.140
169.141
170.142
171.143

172.144
173.145
174.146
175.147
176.148
177.149
178.150
179.151
180.152
181.153
182.154
183.155
184.156
185.157
186.158
187.159
188.160
189.161
190.162
191.163
192.164
193.165
194.166
195.167
196.168
197.169
198.170
199.171
200.172
201.173
202.174
203.175
204.176

205.177
206.178
207.179
208.180
209.181
210.182
211.183
212.184
213.185
214.186
215.187
216.188
217.189
218.190
219.191
220.192
221.193
222.194
223.195
224.196
225.197
226.198
227.199
228.200
229.201
230.202
231.203
232.204
233.205
234.206
235.207
236.208
237.209

238.210
239.211
240.212
241.213
242.214
243.215
244.216
245.217
246.218
247.219
248.220
249.221
250.222
251.223
252.224
253.225
254.226
255.227
256.228
257.229
258.230
259.231
260.232
261.233
262.234
263.235
264.236
265.237
266.238
267.239
268.240
269.241
270.242

271.243
272.244
273.245
274.246
275.247
276.248
277.249
278.250
279.251
280.252
281.253
282.254
283.255
284.256
285.257
286.258
287.259
288.260
289.261
290.262
291.263
292.264
293.265
294.266
295.267
296.268
297.269
298.270
299.271
300.272
301.273
302.274
303.275

304. 276
305. 277
306. 278
307. 279
308. 280
309. 281
310. 282
311. 283
312. 284
313. 285
314. 286
315. 287
316. 288
317. 289
318. 290
319. 291
320. 292
321. 293
322. 294
323. 295
324. 296
325. 297
326. 298
327. 299
328. 300
329. 301
330. 302
331. 303
332. 304
333. 305
334. 306
335. 307
336. 308

337. 309
338. 310
339. 311
340. 312
341. 313
342. 314
343. 315
344. 316
345. 317
346. 318
347. 319
348. 320
349. 321
350. 322
351. 323
352. 324
353. 325
354. 326
355. 327
356. 328
357. 329
358. 330
359. 331
360. 332
361. 333
362. 334
363. 335
364. 336
365. 337
366. 338
367. 339
368. 340
369. 341

370.342
371.343
372.344
373.345
374.346
375.347
376.348
377.349
378.350
379.351
380.352
381.353
382.354
383.355
384.356
385.357
386.358
387.359
388.360
389.361
390.362
391.363
392.364
393.365
394.366
395.367
396.368
397.369
398.370
399.371
400.372
401.373
402.374

403.375
404.376
405.377
406.378
407.379
408.380
409.381
410.382
411.383
412.384
413.385
414.386
415.387
416.388
417.389
418.390
419.391
420.392
421.393
422.394
423.395
424.396
425.397
426.398
427.399
428.400
429.401
430.402
431.403
432.404
433.405
434.406
435.407

436.408
437.409
438.410
439.411
440.412
441.413
442.414
443.415
444.416
445.417
446.418
447.419
448.420
449.421
450.422
451.423
452.424
453.425
454.426
455.427
456.428
457.429
458.430
459.431
460.432
461.433
462.434
463.435
464.436
465.437
466.438
467.439
468.440

469.441
470.442
471.443
472.444
473.445
474.446
475.447
476.448
477.449
478.450
479.451
480.452
481.453
482.454
483.455
484.456
485.457
486.458
487.459
488.460
489.461
490.462
491.463
492.464
493.465
494.466
495.467
496.468
497.469
498.470
499.471
500.472
501.473

502.474
503.475
504.476
505.477
506.478
507.479
508.480
509.481
510.482
511.483
512.484
513.485
514.486
515.487
516.488
517.489
518.490
519.491
520.492
521.493
522.494
523.495
524.496
525.497
526.498
527.499
528.500
529.501
530.502
531.503
532.504
533.505
534.506

535.507
536.508
537.509
538.510
539.511
540.512
541.513
542.514
543.515
544.516
545.517
546.518
547.519
548.520
549.521
550.522
551.523
552.524
553.525
554.526
555.527
556.528
557.529
558.530
559.531
560.532
561.533
562.534
563.535
564.536
565.537
566.538
567.539

568.540
569.541
570.542
571.543
572.544
573.545
574.546
575.547
576.548
577.549
578.550
579.551
580.552
581.553
582.554
583.555
584.556
585.557
586.558
587.559
588.560
589.561
590.562
591.563
592.564
593.565
594.566
595.567
596.568
597.569
598.570
599.571
600.572

601.573
602.574
603.575
604.576
605.577
606.578
607.579
608.580
609.581
610.582
611.583
612.584
613.585
614.586
615.587
616.588
617.589
618.590
619.591
620.592
621.593
622.594
623.595
624.596
625.597
626.598
627.599
628.600
629.601
630.602
631.603
632.604
633.605

634.606
635.607
636.608
637.609
638.610
639.611
640.612
641.613
642.614
643.615
644.616
645.617
646.618
647.619
648.620
649.621
650.622
651.623
652.624
653.625
654.626
655.627
656.628
657.629
658.630
659.631
660.632
661.633
662.634
663.635
664.636
665.637
666.638

667.639
668.640
669.641
670.642
671.643
672.644
673.645
674.646
675.647
676.648
677.649
678.650
679.651
680.652
681.653
682.654
683.655
684.656
685.657
686.658
687.659
688.660
689.661
690.662
691.663
692.664
693.665
694.666
695.667
696.668
697.669
698.670
699.671

700.672
701.673
702.674
703.675
704.676
705.677
706.678
707.679
708.680
709.681
710.682
711.683
712.684
713.685
714.686
715.687
716.688
717.689
718.690
719.691
720.692
721.693
722.694
723.695
724.696
725.697
726.698
727.699
728.700
729.701
730.702
731.703
732.704

733.705
734.706
735.707
736.708
737.709
738.710
739.711
740.712
741.713
742.714
743.715
744.716
745.717
746.718
747.719
748.720
749.721
750.722
751.723
752.724
753.725
754.726
755.727
756.728
757.729
758.730
759.731
760.732
761.733
762.734
763.735
764.736
765.737

766.738
767.739
768.740
769.741
770.742
771.743
772.744
773.745
774.746
775.747
776.748
777.749
778.750
779.751
780.752
781.753
782.754
783.755
784.756
785.757
786.758
787.759
788.760
789.761
790.762
791.763
792.764
793.765
794.766
795.767
796.768
797.769
798.770

799.771
800.772
801.773
802.774
803.775
804.776
805.777
806.778
807.779
808.780
809.781
810.782
811.783
812.784
813.785
814.786
815.787
816.788
817.789
818.790
819.791
820.792
821.793
822.794
823.795
824.796
825.797
826.798
827.799
828.800
829.801
830.802
831.803

832.804
833.805
834.806
835.807
836.808
837.809
838.810
839.811
840.812
841.813
842.814
843.815
844.816
845.817
846.818
847.819
848.820
849.821
850.822
851.823
852.824
853.825
854.826
855.827
856.828
857.829
858.830
859.831
860.832
861.833
862.834
863.835
864.836

865.837
866.838
867.839
868.840
869.841
870.842
871.843
872.844
873.845
874.846
875.847
876.848
877.849
878.850
879.851
880.852
881.853
882.854
883.855
884.856
885.857
886.858
887.859
888.860
889.861
890.862
891.863
892.864
893.865
894.866
895.867
896.868
897.869

898.870
899.871
900.872
901.873
902.874
903.875
904.876
905.877
906.878
907.879
908.880
909.881
910.882
911.883
912.884
913.885
914.886
915.887
916.888
917.889
918.890
919.891
920.892
921.893
922.894
923.895
924.896
925.897
926.898
927.899
928.900
929.901
930.902

931.903
932.904
933.905
934.906
935.907
936.908
937.909
938.910
939.911
940.912
941.913
942.914
943.915
944.916
945.917
946.918
947.919
948.920
949.921
950.922
951.923
952.924
953.925
954.926
955.927
956.928
957.929
958.930
959.931
960.932
961.933
962.934
963.935

964.936
965.937
966.938
967.939
968.940
969.941
970.942
971.943
972.944
973.945
974.946
975.947
976.948
977.949
978.950
979.951
980.952
981.953
982.954
983.955
984.956
985.957
986.958
987.959
988.960
989.961
990.962
991.963
992.964
993.965
994.966
995.967
996.968

997. 969
998. 970
999. 971
1000. 972
1001. 973
1002. 974
1003. 975
1004. 976
1005. 977
1006. 978
1007. 979
1008. 980
1009. 981
1010. 982
1011. 983
1012. 984
1013. 985
1014. 986
1015. 987
1016. 988
1017. 989
1018. 990
1019. 991
1020. 992
1021. 993
1022. 994
1023. 995
1024. 996
1025. 997
1026. 998
1027. 999
1028. 1000
1029. 1001

1030. 1002
1031. 1003
1032. 1004
1033. 1005
1034. 1006
1035. 1007
1036. 1008
1037. 1009
1038. 1010
1039. 1011
1040. 1012
1041. 1013
1042. 1014
1043. 1015
1044. 1016
1045. 1017
1046. 1018
1047. 1019
1048. 1020
1049. 1021
1050. 1022
1051. 1023
1052. 1024
1053. 1025
1054. 1026
1055. 1027
1056. 1028
1057. 1029
1058. 1030
1059. 1031
1060. 1032
1061. 1033
1062. 1034

1063. 1035
1064. 1036
1065. 1037
1066. 1038
1067. 1039
1068. 1040
1069. 1041
1070. 1042
1071. 1043
1072. 1044
1073. 1045
1074. 1046
1075. 1047
1076. 1048
1077. 1049
1078. 1050
1079. 1051
1080. 1052
1081. 1053
1082. 1054
1083. 1055
1084. 1056
1085. 1057
1086. 1058
1087. 1059
1088. 1060
1089. 1061
1090. 1062
1091. 1063
1092. 1064
1093. 1065
1094. 1066
1095. 1067

1096. 1068
1097. 1069
1098. 1070
1099. 1071
1100. 1072
1101. 1073
1102. 1074
1103. 1075
1104. 1076
1105. 1077
1106. 1078
1107. 1079
1108. 1080
1109. 1081
1110. 1082
1111. 1083
1112. 1084
1113. 1085
1114. 1086
1115. 1087
1116. 1088
1117. 1089
1118. 1090
1119. 1091
1120. 1092
1121. 1093
1122. 1094
1123. 1095
1124. 1096
1125. 1097
1126. 1098
1127. 1099
1128. 1100

1129. 1101
1130. 1102
1131. 1103
1132. 1104
1133. 1105
1134. 1106
1135. 1107
1136. 1108
1137. 1109
1138. 1110
1139. 1111
1140. 1112
1141. 1113
1142. 1114
1143. 1115
1144. 1116
1145. 1117
1146. 1118
1147. 1119
1148. 1120
1149. 1121
1150. 1122
1151. 1123
1152. 1124
1153. 1125
1154. 1126
1155. 1127
1156. 1128
1157. 1129
1158. 1130
1159. 1131
1160. 1132
1161. 1133

1162. 1134
1163. 1135
1164. 1136
1165. 1137
1166. 1138
1167. 1139
1168. 1140
1169. 1141
1170. 1142
1171. 1143
1172. 1144
1173. 1145
1174. 1146
1175. 1147
1176. 1148
1177. 1149
1178. 1150
1179. 1151
1180. 1152
1181. 1153
1182. 1154
1183. 1155
1184. 1156
1185. 1157
1186. 1158
1187. 1159
1188. 1160
1189. 1161
1190. 1162
1191. 1163
1192. 1164
1193. 1165
1194. 1166

1195. 1167
1196. 1168
1197. 1169
1198. 1170
1199. 1171
1200. 1172
1201. 1173
1202. 1174
1203. 1175
1204. 1176
1205. 1177
1206. 1178
1207. 1179
1208. 1180
1209. 1181
1210. 1182
1211. 1183
1212. 1184
1213. 1185
1214. 1186
1215. 1187
1216. 1188
1217. 1189
1218. 1190
1219. 1191
1220. 1192
1221. 1193
1222. 1194
1223. 1195
1224. 1196
1225. 1197
1226. 1198
1227. 1199

1228. 1200
1229. 1201
1230. 1202
1231. 1203
1232. 1204

Long description

The screenshot shows two sample windows of the platform “MyLab (T M) Programming” to present the various features that come under “Workbench.” The first window shows the workbench for an exercise named “Exercise 2 0 5 0 1” with buttons to go back to the previous page or move to the next page. Three tabs, “Work Area,” “Results,” and “Solutions” appear below the exercise name with “Results” in the selected mode. A list of items appears below it under the head “Codelab Analysis: Compiler Error(s).” The items are shown further sub-grouped into “Remarks” and “More Hints.” Two buttons, “Print Results” and “Contact Support” are also shown appearing at the top-right section of the window. The second window is shown divided into two panels. The left panel on the window is titled “Table of Contents” and enlists a large list of items. It shows the chapter names for the book titled, “Starting out with C plus plus.” The right panel of the window shows the “Workbench” with two buttons at the top-left section allowing the user to go back to the previous page or move to the next page. It also features a brief instruction addressed to the instructor telling how to navigate, use the table of contents and bulletins, and get help.

Long description

The flowchart shows the interdependency of the chapters of the book “Building Java Programs: A Back to Basics Approach” as follows:

- The chapters from 1 through 6 cover the “Programming Fundamentals.”
- The chapters that are interdependent on the first six chapters are: Chapter 7, “Arrays”; Chapter 8, “Classes”; Chapter 9, “Inheritance Interfaces”; Chapter 10, “Array Lists”; Chapter 11, “Collections”; Chapter 15, “Implementing Collections”; Chapter 16, “Linked Lists”; Chapter 17, “Binary Trees.”
- Chapter 12, “Recursion” and chapter 13, “Searching and Sorting” are interdependent on chapter 7.
- Chapter 14, “Stacks, Queues” is interdependent on chapter 14.
- Chapter 18, “Hashing and Heaps” is interdependent on chapter 13.
- Chapter 19 is interdependent on chapter 11 and chapter 9, Functional Programming, except Section 19.5.

Long description

The flowchart shows the steps to create a new Java program, “Hello dot java.” The steps are as follows:

- Step 1: Use editor to create or edit the source file to get the output, “Hello dot java.”
- Step 2: Submit the source file to the Java compiler. If step 2 turns out to be a failure, step 1 would be repeated and if it is executed successfully, it would give the output, “hello dot class.”
- Step 3: Execute the Java class file.

Long description

The flowchart shows “Make cake” as the highest-level operation. The highest level operation is divided into four lower-level operations namely “Make batter,” “Bake,” “Make frosting,” and “Frost cake.” The “Make batter” operation is further divided into four lower-level operations namely “Mix dry ingredients,” “Cream butter sugar,” “Beat in eggs,” and “Stir in dry ingredients.”

Long description

The image shows nine print l n statements executed. It shows the control first flowing to the drawBox method and executing the four print l n statements. It then returns to main and executes a print l n statement. It further transfers the control to drawBox and again executes the remaining four statements.

Long description

The image shows the flow of controls first flowing to the word “foo” and executing a print l n statement, before it returns to the word “bar.” It further transfers the control for the second time to the word “baz” and executes another print l n statement. The flow of control finally returns to the word “mumble” to execute the final print l n statement.

Long description

The image shows the flow of controls first flowing to the word “foo” and executing a print l n statement, before it returns to the word “bar.” It then transfers the control to drawBox and executes a print l n statement for the word “baz.” It again returns to the word “bar,” before it finally executes a print l n statement for the word “mumble.”

Long description

The flowchart shows “draw X” as the main activity of the decomposition and “drawDiamond” and “drawRocket” as its subtasks. The “drawDiamond” is divided into “drawCone” and “draw V.” The “draw X” is divided into “draw V” and “draw Cone.” The “drawRocket” is divided into “drawCone” and “drawbox.”

Long description

The diagram shows two simple expressions made up of operands and operators. The first expression, “3 plus 29” has 3 and 29 marked as the operands and the plus sign marked as the operator. The second expression, “4 times 5” has 4 and 5 marked as the operands and the multiplication sign marked as the operator.

Long description

The image shows two simple expressions, “3 plus 29 minus 4 times 5.” In the addition operator, 3 and 29 are labeled as “Operand” and the plus sign between 3 and 29 is labeled as “Operator.” In the multiplication operator, 4 and 5 are labeled as “Operand” and the multiplication sign is labeled as “Operator.” The minus sign between the addition operands and the multiplication operands is labeled as “Operator.”

Long description

The image shows “13 times 2 plus 239 divided by 10 percent of 5 minus 2 times 2” as a complex expression, with six operators. It shows 13 times 2 grouped together using parenthesis and labeled “26.” It shows 239 divided by 10 grouped together using parenthesis and labeled “23.” 23 percent of 5 is shown grouped together using parenthesis and labeled “3.” It further shows 2 times 2 grouped together using parenthesis and labeled “4.”

Long description

The image shows three variables namely “height,” “weight,” and “b m i.” There is a cell located next to each variable representing their respective values. The value for the variable height is given as “70.0.” The values for the variables, weight and b m i are not known and are represented by question marks.

Long description

The image shows three variables namely “height,” “weight,” and “b m i.” There is a cell located next to each variable representing their respective values. The value for the variable height is given as “70.0.” The value for the variable weight is given as “195.0.” The value for the variable b m i is not known and is represented by a question mark.

Long description

The image shows three variables namely “height,” “weight,” and “b m i.” There is a cell located next to each variable representing their respective values. The value for the variable height is given as “70.0.” The value for the variable weight is given as “195.0.” The value for the variable b m i is given as “27.976530612244897.”

Long description

The image shows three variables namely “height,” “weight,” and “b m i.” There is a cell located next to each variable representing their respective values. The value for the variable height is given as “70.0.” The value for the variable weight is given as “195.0.” The value for the variable b m i is given as “27.976530612244897.”

Long description

The image shows three variables namely “height,” “weight,” and “b m i.” There is a cell located next to each variable representing their respective values. The value for the variable height is given as “70.0.” The value for the variable weight is given as “180.0.” The value for the variable b m i is given as “27.976530612244897.”

Long description

The image shows three variables namely “height,” “weight,” and “b m i.” There is a cell located next to each variable representing their respective values. The value for the variable height is given as “70.0.” The value for the variable weight is given as “180.0.” The value for the variable b m i is given as “25.82448979591837.”

Long description

The flowchart shows the steps followed by Java to execute a “for” loop. The first step in the process is labeled as, “Perform initialization once.” The second step labeled “Is the test true?” is further divided into “Yes” or “No.” If the test is true, two more sequential steps are followed. These two steps are given as “Execute the controlled statement(s)” and “Perform the update.” If the test is not true, then the final step to be followed is labeled as “Execute statement after for loop.”

Long description

The image shows the detailed trace of the “for” loop in five executions as follows:

- Initialize i equals 1
- Test left-parenthesis i less than or equal to 5 question mark yes right-parenthesis; body left-parenthesis print ln 1 times 1 right-parenthesis; update left-parenthesis i becomes 2 right-parenthesis
- Test left-parenthesis i less than or equal to 5 question mark yes right-parenthesis; body left-parenthesis print ln 2 times 2 right-parenthesis; update left-parenthesis i becomes 3 right-parenthesis
- Test left-parenthesis i less than or equal to 5 question mark yes right-parenthesis; body left-parenthesis print ln 3 times 3 right-parenthesis; update left-parenthesis i becomes 4 right-parenthesis
- Test left-parenthesis i less than or equal to 5 question mark yes right-parenthesis; body left-parenthesis print ln 4 times 4 right-parenthesis; update left-parenthesis i becomes 5 right-parenthesis
- Test left-parenthesis i less than or equal to 5 question mark yes right-parenthesis; body left-parenthesis print ln 5 times 5 right-parenthesis; update left-parenthesis i becomes 6 right-parenthesis
- Test left-parenthesis i less than or equal to 5 question mark no right-parenthesis
- Exit loop

Long description

The image shows individual characters used to develop the top half section of an hourglass namely, bar, spaces, backslash, dots, slash, spaces, and bar. The bars are denoted by vertical lines, the spaces are denoted by blanks, the backslash is shown as a backward-sloping diagonal line, and the dots are shown as points.

Long description

The image shows a simple calling mechanism that reads, “write Spaces left-parenthesis 10 right-parenthesis semicolon.” An arrow starting from the mechanism points toward a rectangular box displaying the method. The arrow indicates the flow of information from the call to the method. The method reads as follows:

Line 1: public static void write Spaces left brace int number right-parenthesis left-parenthesis

Line 2: (indented) ellipsis

Line 3: right-parenthesis

Long description

The image shows the two methods “method main” and “method write Spaces” appearing in two boxes, respectively. The box representing “method main” displays two variables, “spaces 1” and “spaces 2” with their values entered in the respective cells as “3” and “5,” respectively. The box representing “method write Spaces” displays the variable, “number” with its value entered in the respective cell as “3.”

Long description

The image shows the two methods “method main” and “method write Spaces” appearing in two boxes, respectively. The box representing “method main” displays two variables, “spaces 1” and “spaces 2” with their values entered in the respective cells as “3” and “5,” respectively. The box representing “method write Spaces” displays the variable, “number” with its value in the cell changed from “3” to “5.”

Long description

The image shows the two methods “method main” and “method write Spaces” appearing in two boxes, respectively. The box representing “method main” displays two variables, “spaces 1” and “spaces 2” with their values entered in the respective cells as “3” and “5,” respectively. The box representing “method write Spaces” displays the variable, “number” with its value in the cell changed from “5” to “8.”

Long description

The image shows an expression with two string literals and an integer. The expression reads as, “spaces 1 times spaces 2 minus 5.” It shows “spaces 1” is marked with a parenthesis and labeled “3” while “spaces 2” is marked with a parenthesis and labeled “5.” The new expression formed is “3 times 5 minus 5.” It further shows “3 times 5” is grouped together using parenthesis and labeled “15.” The new expression formed is “15 minus 5.” The expression “15 minus 5” is grouped together using parenthesis and labeled “10.” The value of the expression is shown as 10.

Long description

The image shows the two methods “method main” and “method write Spaces” appearing in two boxes, respectively. The box representing “method main” displays two variables, “spaces 1” and “spaces 2” with their values entered in the respective cells as “3” and “5,” respectively. The box representing “method write Spaces” displays the variable, “number” with its value in the cell changed from “8” to “10.”

Long description

The image shows the assignment of a value to an integer variable under the method main. It shows a box titled “method main” displaying the initialization of an integer variable with the value 17 as “x 17.” It shows another box titled “method double Number” with the value of the “number” variable entered as “17” in the respective cell.

Long description

The image shows the assignment of a value to an integer variable under the method main. It shows a box titled “method main” displaying the initialization of an integer variable with the value 17 as “x 17.” It shows another box titled “method double Number” with the value of the “number” variable doubled from “17” to “34.”

Long description

The image shows a box titled “method main” displaying the initialization of an integer variable with the value 17 as “x 17,” indicating the value of “x” as 17. It further displays the initialization of a variable “number” with its value as “42.”

Long description

The image shows a box titled “method main” displaying the initialization of an integer variable with the value 17 as “x 17,” indicating the value of “x” as 17. It further displays the initialization of a variable “number” with its value as “42.” It shows another box titled “method double Number” displaying the value of the “number” variable as “42” entered in its respective cell.

Long description

The image shows a box titled “method main” displaying the initialization of an integer variable with the value 17 as “x 17,” indicating the value of “x” as 17. It further displays the initialization of a variable “number” with its value as “42.” It shows another box titled “method double Number” displaying the initial value of the “number” variable 42, doubled to “84.”

Long description

The image shows a box titled “method main” displaying the initialization of an integer variable with the value 17 as “x 17,” indicating the value of “x” as 17. It further displays the initialization of a variable “number” with its value as “42” indicating the value of number 84 returning to the main method.

Long description

The image shows parameters lined up in sequential order to define a method.

The calling mechanism reads, “write Chars left-parenthesis ‘equal sign’, 20 right-parenthesis.” It shows two arrows starting from the actual parameters in the mechanism and pointing toward their respective formal parameters. It shows the “equal sign” going into “ch” and “20” going into “number” in the method. The lines in the method read as follows:

Line 1: public static void write Chars left brace char ch, int number right-parenthesis left-parenthesis

Line 2: (indented) ellipsis

Line 3: right-parenthesis

Long description

The image shows the result obtained by executing the code “int answer equals sum left brace 100 right brace,” with the use of the sum method by the main method. It shows the method invoked with parameter “n” initialized to 100, by plugging the value 100 into the following formula:

Line 1: public static int sum left-parenthesis int n right-parenthesis left-parenthesis

Line 2: (indented) return left-parenthesis n plus 1 right-parenthesis times n over 2 semicolon

Line 3: right-parenthesis

The value obtained from this formula is shown as “5050.” The value is shown further sent back to be stored in the variable “answer” of the code.

Long description

The image shows the characters of the string “hello” assigned with index values, starting with 0. The index values for the characters of the string are assigned as follows: h, 0; e, 1; l, 2; l, 3; o, 4.

Long description

The image shows the characters of the string “how are you?” assigned with index values, starting with 0. The index values for the characters of the string are assigned as follows:

- h: 0
- o: 1
- w: 2
- space: 3
- a: 4
- r: 5
- e: 6
- space: 7
- y: 8
- o: 9
- u: 10
- question mark: 11

Long description

The image shows the characters of the string “how are you?” assigned with index values, starting with 0. The index values for the characters of the string are assigned as follows:

- h: 0
- o: 1
- w: 2
- space: 3
- a: 4
- r: 5
- e: 6
- space: 7
- y: 8
- o: 9
- u: 10
- question mark: 11

Long description

The illustration shows the initial velocity and the final velocity of a projectile, with the help of arrows forming right-angled triangles. The triangle denoting the initial velocity of the projectile has its height labeled “v sub 0 times y,” hypotenuse labeled “v sub 0,” and the base labeled “v sub 0 times x.” An angle of theta is shown formed opposite the right angle. The initial velocity is shown rising upward, as denoted by upward arrows. The triangle denoting the final velocity of the projectile has its height labeled “v sub t times y,” hypotenuse labeled “v sub t,” and the base labeled “v sub t times x.” An angle of theta is shown formed opposite the right angle. The final velocity is shown rising downward, as denoted by downward arrows.

Long description

The diagram shows a rectangular drawing-panel drawn on a xy-plane with the vertical axis labeled “y plus” and the horizontal axis labeled “x plus.” The upper-left corner of the panel is lying at ordered pair 0 comma 0, and the lower-right corner is lying at ordered pair 199 comma 199.

Long description

The screenshot shows a rectangular-shaped window titled, “Drawing Panel.” It displays a linear line rising upward from the lower-left part of the window at 25 over from the left and 75 down from the top to the upper-right part of the window at 175 over from the left and 25 down from the top.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying a circle, with a square at its lower-left, on the left section of the window and a rectangle, with an oval at its bottom, on the right section of the window. The background of the window is white, and the square and the oval are shaded in black.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying a circle, with a square at its lower-left, on the left section of the window and a rectangle, with an oval at its bottom, on the right part of the window. The background of the window is light blue, and the square and the oval are shaded in white.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying a series of four rectangles with black outlines, extended from the upper-left corner of the window to the lower-right corner of the window. Each of the rectangles consists of an oval, shaded in white color with white outlines.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying a series of four rectangles, extended from the upper-left corner of the window to the lower-right corner of the window. The first rectangle has a black outline and the remaining three rectangles have white outlines. Each of the rectangles consists of an oval, shaded in white color with white outlines.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying a series of rectangles extended from the upper-left corner of the window to the lower-right corner of the window. The rectangles are shaded in different colors from black to white, on moving from the upper-left corner to the lower-right corner of the window.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying three messages in three different fonts and styles, on a pink background. The first message reads, “Too big.” It is displayed in the font, “Monospaced” and the text style in the message is bolded and italicized. The second message reads, “Too small.” It is displayed in the font, “Sans Serif” and the text style in the message is kept plain. The third message reads, “Just right.” It is displayed in the font, “Serif” and the text style in the message is italicized.

Long description

The image shows a square depicting a diamond, enclosed by a square-shaped box with its top-left corner lying at ordered pair 0 comma 0. The vertices of the square are lying at ordered pair 25 comma 0, ordered pair 50 comma 25, ordered pair 25 comma 50, and ordered pair 0 comma 25.

Long description

The image shows a square depicting a diamond, enclosed by a square-shaped box with its top-left corner lying at ordered pair 78 comma 22. The vertices of the square are lying at ordered pair 103 comma 22, ordered pair 128 comma 47, ordered pair 103 comma 72, and ordered pair 78 comma 47.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying three pyramids in white, blue, and red color, with different numbers of stairs stacked one above the other, in the decreasing order of their lengths. The first pyramid consists of 10 stairs, each shaded in white with a black outline. The second pyramid consists of 20 stairs, each shaded in blue with a black outline. The third pyramid consists of 5 stairs, each shaded in red with a black outline.

Long description

The diagram shows two stairs stacked one above the other with decreasing lengths but the same heights. The top-left corner is lying at ordered pair 0 comma 0. The bottom stair is 20 pixels wide and the top stair is 10 pixels wide. The empty space on either side of the bottom stair is 40 pixels wide and the empty space on either side of the top stair is 45 pixels wide.

Long description

The diagram shows two stairs stacked one above the other with decreasing length but same length. The top-left corner is lying at ordered pair 80 comma 140. The bottom stair is 40 pixels wide and the top stair is 20 pixels wide. The empty space on either side of the bottom stair is 30 pixels wide and the empty space on either side of the top stair is 40 pixels wide.

Long description

A screenshot shows a window titled “Drawing Panel,” displaying a red-colored square with two blue-colored ovals at its top-left corner and the top-right corner, on a yellow background. The ovals are drawn such that their centers are lying at the corners of the squares. A solid black-colored horizontal line is passing through the center of the square.

Long description

A screenshot shows a window titled “Drawing Panel,” displaying a set of two red-colored squares, each with two blue-colored ovals at its top-left corners and the top-right corners, on a yellow background. The ovals are drawn such that their centers are lying with the corners of the squares. A solid black-colored horizontal line is passing through the center of each square. The second red square is lying at the bottom-right of the first red square.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying five squares one inside the other sharing a common vertex, on a cyan background. The horizontal and vertical sides of the squares are drawn in red color and a diagonal line is drawn from the top-left to the bottom-right in black color.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying three sets of five squares drawn one inside the other sharing a common vertex, on a cyan background. The horizontal and vertical sides of the squares are drawn in red color and a diagonal line in black color is drawn from the top-left to the bottom-right of the squares in each set.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying three sets of five squares drawn one inside the other sharing a common vertex, on a cyan background. The horizontal and vertical sides of the squares are drawn in red color and a diagonal line in black color is drawn from the top-left to the bottom-right of the squares in each set. The first set of squares is at the left with their sizes set at 100 pixels. The second set of squares is at the top-right with their sizes set at 50 pixels. The third set of squares is at the bottom-right with their sizes set at 180 pixels.

Long description

The first screenshot shows a window titled “Drawing Panel,” displaying 10 stairs stacked one above the other in the increasing order of their lengths. The stairs are aligned to the left. The second screenshot shows a window titled “Drawing Panel,” displaying 10 stairs stacked one above the other in the increasing order of their lengths. The stairs are aligned to the left. The third screenshot shows a window titled “Drawing Panel,” displaying 10 stairs stacked one above the other in the increasing order of their lengths. The stairs are aligned to the right.

Long description

The screenshot shows a window titled “Drawing Panel,” displaying a rectangle with an oval drawn inside it from the bottom-left corner to the upper-right corner. There are multiple straight lines drawn between the empty spaces from the horizontal and vertical edges of the rectangle to the edges of the oval.

Long description

The first figure shows a square consisting of five concentric yellow circles with black outlines drawn one inside the other. A vertical and a horizontal black-colored line are drawn inside the square such that they intersect each other at the centers of the circles. The background of the circles is white and the background of the square is cyan. The second figure shows a large square divided into 25 small squares, each consisting of four concentric yellow circles with black outlines drawn one inside the other. A vertical and a horizontal line are drawn inside each small square such that they intersect each other at the centers of the circles. The background of the circles is white and the background of the squares is cyan. The third figure shows a large square divided into 36 small squares, each consisting of five concentric yellow circles with black outlines drawn one inside the other. A vertical and a horizontal line are drawn inside each small square such that they intersect each other at the centers of the circles. The background of the circles is white and the background of the squares is cyan. The fourth figure shows a large square divided into nine small squares, each consisting of three concentric yellow circles with black outlines drawn one inside the other. A vertical and a horizontal line are drawn inside each small square such that they intersect each other at the centers of the circles. The background of the circles is white and the background of the squares is cyan.

Long description

The screenshot shows a window titled “Drawing Panel” displaying a square divided into nine small square grids, each with a pattern of an image. The first grid on the top-left shows two ovals with black outlines, consisting of circles shaded in gray color. The second grid on the top-center shows a pattern of rectangles, drawn one after the other along the length and the width of the grid, in gray color. The third grid on the top-right shows a text that reads, “I Love Java” in cyan color. The first grid on the center-left shows a pattern of rectangles, drawn one after the other along the length and the width of the grid, in cyan color. The second grid on the center shows a text that reads, “I Love Java” in a light colored font. The third grid on the center-right shows two ovals with black outlines, consisting of circles shaded in cyan color. The first grid on the bottom-left shows a text that reads, “I Love Java” in gray color. The second grid on the bottom-center shows two ovals with black outlines, consisting of circles shaded in red color. The third grid on the bottom-right shows a pattern of rectangles, drawn one after the other along the length and the width of the grid, in a light color.

Long description

The first figure shows a square with a black outline divided into 25 square grids, each shaded in gray and white in alternate. The second figure shows a square with a black outline divided into 100 square grids, each consisting of five small squares, one at the center and the other four at the vertices. The grids have black outlines while the small squares are shaded in gray. The third figure shows a square with a black outline, divided into 25 square grids with black outlines. Each of the 25-grid is further divided into 25 smaller grids, each shaded in gray and white in alternate.

Long description

The first figure shows a series of eight straight squares shaded in black and white colors in alternate. The squares shaded in black have two diagonals intersecting each other. The second figure shows a series of ten straight squares shaded in black and white colors in alternate. The squares shaded in black have two diagonals intersecting each other. The third figure shows eight series of eight straight squares, placed one below the other. Each square in the series is shaded in black and white colors in alternate. The squares shaded in black have two diagonals intersecting each other. The fourth figure shows six series of six straight squares placed one below the other with every alternate series slightly indented toward the right. Each square in the series is shaded in black and white colors in alternate. The squares shaded in black have two diagonals intersecting each other. The fifth figure shows four series of four straight squares placed one below the other with every alternate series beginning below the second squares of the previous series. Each square in the series is shaded in black and white colors in alternate. The squares shaded in black have two diagonals intersecting each other. The sixth figure shows ten series of ten straight squares placed one below the other with every alternate series slightly indented toward the right. Each square in the series is shaded in black and white colors in alternate. The squares shaded in black have two diagonals intersecting each other.

Long description

The flowchart shows the statement “Is the test true?” in a diamond structure. If the response to the statement is “Yes,” the subsequent step would be to “Execute the controlled statement(s).” If the response is “No,” the subsequent step would be to “Execute the statement after the ‘if’ statement.”

Long description

The flowchart shows the statement “Is the test true?” in a diamond structure. If the response to the statement is “Yes” the subsequent step would be to “Execute the ‘if’ controlled statement(s).” If the response is “No,” the subsequent step would be to “Execute the ‘else’ controlled statement(s).” The final step to be followed after implementing either of the subsequent steps is to “Execute the statement after the ‘if/else’ statement.”

Long description

The image shows a sample expression “3 plus 2 times equals 9.” It shows “2 times 2” grouped together using a parenthesis and labeled “4.” The new expression formed is “3 plus 4 equals 9.” “3 plus 4” is further grouped together using a parenthesis and labeled “7.” “7 equals 9” is grouped together using a parenthesis and labeled “false.”

Long description

The flowchart shows three conditional statements in diamond structures, arranged in the vertical alignment one below the other. Each statement is assigned a positive and a negative response and the sequential step to be followed based on the response. The first conditional statement is “Is test 1 true?” If the response to the statement is “Yes,” the sequential step to be followed is “Statement 1.” Statement 1 leads to the next conditional statement. If the response is “No,” then the computer would move to the second conditional statement. The second conditional statement is “Is test 2 true?” If the response to the statement is “Yes,” the sequential step to be followed is “Statement 2.” Statement 2 leads to the next conditional statement. If the response is “No,” then the computer would move to the third conditional statement. The third conditional statement is “Is test 3 true?” If the response to the statement is “Yes,” the sequential step to be followed is “Statement 3.” If the response is “No,” then the computer would move to the next conditional statement.

Long description

The flowchart shows three conditional statements in diamond structures, arranged such that they are placed one inside the other. Each statement is assigned a positive and a negative response and the sequential step to be followed based on the response. The first conditional statement is “Is test 1 true?” If the response to the statement is “Yes,” the sequential step to be followed is “Statement 1.” If the response is “No,” then the computer would move to the second conditional statement. The second conditional statement is “Is test 2 true?” If the response to the statement is “Yes,” the sequential step to be followed is “Statement 2.” If the response is “No,” then the computer would move to the third conditional statement. The third conditional statement is “Is test 3 true?” If the response to the statement is “Yes,” the sequential step to be followed is “Statement 3.” If the response is “No,” then the computer would move to the next conditional statement. The statements, 1, 2, and 3 are shown independent of any further steps. The nested structure of the flowchart indicates that one of the controlled statements, the first one that is evaluated to be true can be executed. None of the statements can be executed if no test is evaluated to be true.

Long description

The flowchart shows two conditional statements in diamond structures, arranged such that they are placed one inside the other. Each statement is assigned a positive and a negative response and the sequential step to be followed based on the response. The first conditional statement is “Is test 1 true?” If the response to the statement is “Yes,” the sequential step to be followed is “Statement 1.” Here, statement 1 can be directly executed. If the response is “No,” then the computer would move to the second conditional statement. The second conditional statement is “Is test 2 true?” If the response to the statement is “Yes,” the sequential step to be followed is “Statement 2.” Here, statement 2 can be directly executed. If the response is “No,” then the sequential step to be followed is “Statement 3.” The structure of the flowchart indicates that one of the controlled statements, the first one that is evaluated to be true can be executed. None of the statements can be executed if no test is evaluated to be true.

Long description

The flowchart shows the main branch labeled “Depending upon money.” The main branch is further branched out into three more branches. The three branches are labeled, “Money less than 500,” “500 less than or equal to Money less than 1000,” and “Money greater than or equal to 1000.” A series of actions are shown performed in each branch as follows:

- Money less than 500
  - Report money
  - Warn about low money
  - Prompt for bet
  - Read bet
- 500 less than or equal to Money less than 1000
  - Report money
  - Report okay money
  - Prompt for bet
  - Read bet
- Money greater than or equal to 100
  - Report money
  - Report good money
  - Prompt for bet
  - Read bet

Long description

The flowchart shows the main branch labeled “Report money: Depending upon money.” The main branch is further branched out into three more branches labeled, “Money less than 500,” “500 less than or equal to Money less than 1000,” and “Money greater than or equal to 1000.” A consequent action is shown performed in each branch as follows:

- Money less than 500: Warn about low money
- 500 less than or equal to Money less than 1000: Report okay money
- Money greater than or equal to 100: Report good money

All the three actions are shown together leading to two final actions, “Prompt for bet” and “Read bet.”

Long description

The flowchart shows the “Director” at the top of the organizational structure. The “Marketing administrator,” “Design manager,” and the “Engineering manager” are the employees grouped under the Director. The “Design manager” is shown further linked to four more employees namely the “Secretary,” “Architect,” “Engineer,” and the “Administrator.”

Long description

The diagram shows a hypothetical code with chaining, on the left and without chaining on the right. The design on the left with chaining is titled “main.” It shows each method returns an object, allowing the calls to be chained together in a single statement without requiring variables to store the intermediate results. It results in the formation of a chain under the main method, named method 1, method 2, method 3, method 4, and method 5. The methods from 1 through 3 are executed one after the other, each passing parameters down the chain. The last two methods, method 4 and 5 are executed in synchronization with each other. The design on the right without chaining is titled “main.” It does not feature a chainlike structure and the methods call the other for their execution. It shows method 2 calling method 3, with method 2 as a subtask of method 1. This leads to the execution of method 4 and method 5, one following the other.

Long description

The diagram shows the coordinate plane, with the horizontal axis labeled “x-axis” and the vertical axis labeled “y-axis.” The axes intersect each other such that they divide the plane into four infinite regions called quadrants. On moving in the counterclockwise direction or from the upper-right region, the first quadrant is labeled “Quadrant I,” the second quadrant is labeled “Quadrant 2,” the third quadrant is labeled “Quadrant 3,” and the fourth quadrant is labeled “Quadrant 4.”

Long description

The flowchart shows the “while” loop performing its test at the top of the loop. The condition of the loop “Is the test true?” appears at the top of the chart in a diamond structure. There are two responses to the condition and their sequential steps. If the response to the condition is “Yes,” the consequent step would be to “Execute the controlled statement(s).” After the controlled statement or statements are executed they are shown returning to the condition “Is the test true?” If the response to the condition is “No,” the consequent step would be to “Execute the statement after the “while” loop.

Long description

The flowchart shows the first step in the process as “Execute the controlled statement(s).” Once the controlled statements are executed, the resulting condition takes place. The condition is to evaluate “Is the test true?” The condition is provided with two responses and the consequent step for each response. If the response to the condition is “Yes,” the controlled statement(s) are executed. If the response is “No,” the consequent step would be to “Execute the statement after the do/while loop.”

Long description

The diagram shows the words “four score” of a string arranged in a horizontal strip divided into ten parts. Each part is assigned a letter in the following pattern: f, o, u, r, space, s, c, o, r, e. Each letter is further assigned with an index as follows: f, 0; o, 1; u, 2; r, 3; space, 4; s, 5; c, 6; o, 7; r, 8; e, 9. Ellipses appear at the right of the indexes and at the end of the vertical strip containing the letters.

Long description

The diagram shows a scanner as an input cursor or pointer in an input file. It treats the file as a one-dimensional string of characters and further converts their contents into a series of separated tokens. A text in the input file reads, “Hello. What up? I’m fine!” The pointer points toward the scanner. The cursor further moves forward to “Hello.” The direction of the cursor indicates that it would further move to “What” and “up?”

Long description

The image shows a file that reads as follows: “308 point 2 space 14 point 9 space 7 point 4 backslash n 2 point 8 backslash n backslash n backslash n 3 point 9 4 point 7 space negative 15 point 4 backslash n 2 point 8 backslash n.” An upward arrow labeled “input cursor” is shown pointing at the first character “308.2” of the file.

Long description

The image shows a file that reads as follows: “308 point 2 space 14 point 9 space 7 point 4 backslash n 2 point 8 backslash n backslash n backslash n 3 point 9 4 point 7 space negative 15 point 4 backslash n 2 point 8 backslash n.” An upward arrow labeled “input cursor” is shown pointing at the space after the first character “308.2” of the file.

Long description

The image shows a file that reads as follows: “308 point 2 space 14 point 9 space 7 point 4 backslash n 2 point 8 backslash n backslash n backslash n 3 point 9 4 point 7 space negative 15 point 4 backslash n 2 point 8 backslash n.” An upward arrow labeled “input cursor” is shown pointing at the space between 14 point 9 and 7 point 4 of the file.

Long description

The image shows a file that reads as follows: “308 point 2 space 14 point 9 space 7 point 4 backslash n 2 point 8 backslash n backslash n backslash n 3 point 9 4 point 7 space negative 15 point 4 backslash n 2 point 8 backslash n.” An upward arrow labeled “input cursor” is shown pointing at the “n” placed before 2.8 of the file.

Long description

The image shows a file that reads as follows: “308 point 2 space 14 point 9 space 7 point 4 backslash n 2 point 8 backslash n backslash n backslash n 3 point 9 4 point 7 space negative 15 point 4 backslash n 2 point 8 backslash n.” An upward arrow labeled “input cursor” is shown pointing at the “n” placed after 2.8 of the file.

Long description

The image shows a file that reads as follows: “308 point 2 space 14 point 9 space 7 point 4 backslash n 2 point 8 backslash n backslash n backslash n 3 point 9 4 point 7 space negative 15 point 4 backslash n 2 point 8 backslash n.” An upward arrow labeled “input cursor” is shown pointing at the space between 3 point 9 and 4 point 7 of the file.

Long description

The image shows a file that reads as follows: “308 point 2 space 14 point 9 space 7 point 4 backslash n 2 point 8 backslash n backslash n backslash n 3 point 9 4 point 7 space negative 15 point 4 backslash n 2 point 8 backslash n.” An upward arrow labeled “input cursor” is shown pointing at the last character “n” of the file.

Long description

The diagram shows an array of three double values constructed as the array of temperatures. It shows the variable “temperature” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at three values arranged in three cells next to each other. The value “0.0” is entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the three double values in the array are as follows: 0.0, left square bracket 0 right square bracket; 0.0, left square bracket 1 right square bracket; 0.0, left square bracket 2 right square bracket.

Long description

The diagram shows an array of three values constructed as the array of temperatures. It shows the variable “temperature” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at three values arranged in three cells next to each other. The values, 74.3, 68.4, and 70.3 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 74.3, left square bracket 0 right square bracket
- 68.4, left square bracket 1 right square bracket
- 70.3, left square bracket 2 right square bracket

Long description

The diagram shows an array of values constructed as the array of temperatures. It shows the variable “temperature” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at values arranged in cells next to each other. The value, 0.0 is entered in each cell. Each value in the cell is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 0.0, left square bracket 0 right square bracket
- 0.0, left square bracket 1 right square bracket
- 0.0, left square bracket 2 right square bracket
- 0.0, left square bracket 3 right square bracket
- 0.0, left square bracket 4 right square bracket
- Ellipsis, left square bracket ellipsis right square bracket
- 0.0, left square bracket 99 right square bracket

Long description

The diagram shows an array of length 5, with the array elements initialized to 0 for the variable list of type integer left square bracket right square bracket. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. The value “0” is entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 0, left square bracket 0 right square bracket
- 0, left square bracket 1 right square bracket
- 0, left square bracket 2 right square bracket
- 0, left square bracket 3 right square bracket
- 0, left square bracket 4 right square bracket

Long description

The diagram shows an array of length 5, with the array elements featuring successive odd numbers. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. The values 1, 3, 5, 7, and 9 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 1, left square bracket 0 right square bracket
- 3, left square bracket 1 right square bracket
- 5, left square bracket 2 right square bracket
- 7, left square bracket 3 right square bracket
- 9, left square bracket 4 right square bracket

Long description

The diagram shows an array of length 5 with the values, 1, 3, 5, 7, and 9. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values are 0, 1, 2, 3, 4, and 5. The length of the array from 1 to 9 is marked together and labeled “legal indexes 0 to 4.” The region at the left of the array is marked with an up-arrow and labeled “index less than 0 out of bounds.” The region at the right of the array is marked with an up-arrow and labeled “index 5 or more out of bounds.”

Long description

The diagram shows the string “James T Kirk” with 12 characters written into a buffer divided into twelve small parts. Each part is assigned a character, on moving from left to right in the following pattern: J, a, m, e, s, space, T, space, K, i, r, k. The characters are shown marked together and labeled “12-character buffer.” It shows spaces between “James” and “T” and between “T” and “Kirk.”

Long description

The diagram shows the string “Jean Luc Picard” with 15 characters written into a buffer divided into fifteen small parts. Each part is assigned a character, on moving from left to right in the following pattern: J, e, a, n, space, L, u, c, space, P, i, c, a, r, d. The characters from “J” in Jean to “a” in Picard are shown marked together and labeled “12-character buffer.” The last three characters, “a,” “r,” and “d” are marked together and labeled “overrun” indicating that there is an overrun on writing three extra characters into the buffer.

Long description

The diagram shows an array of integer values with their indexes, to be printed using print ln statement. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at seven values arranged in seven cells next to each other. The values 17, negative 3, 42, 8, 12, 2, and 103 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 17, left square bracket 0 right square bracket
- Negative 3, left square bracket 1 right square bracket
- 42, left square bracket 2 right square bracket
- 8, left square bracket 3 right square bracket
- 12, left square bracket 4 right square bracket
- 2, left square bracket 5 right square bracket
- 103, left square bracket 6 right square bracket

Long description

The diagram shows an array of integer values, with few values repeating multiple times in the array. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at seven values arranged in seven cells next to each other. The values 8, 7, 19, 82, 8, 7, and 8 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 8, left square bracket 0 right square bracket
- 7, left square bracket 1 right square bracket
- 19, left square bracket 2 right square bracket
- 82, left square bracket 3 right square bracket
- 8, left square bracket 4 right square bracket
- 7, left square bracket 5 right square bracket
- 8, left square bracket 6 right square bracket

Long description

The diagram shows an array of six values with their indexes from 0 to 5. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at six values arranged in six cells next to each other. The values 3, 8, 7, negative 2, 14, and 78 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 3, left square bracket 0 right square bracket
- 8, left square bracket 1 right square bracket
- 7, left square bracket 2 right square bracket
- Negative 2, left square bracket 3 right square bracket
- 14, left square bracket 4 right square bracket
- 78, left square bracket 5 right square bracket

Long description

The diagram shows an array of six values with the first and the last value in the list swapped with each other. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at six values arranged in six cells next to each other. The values 3, 8, 7, negative 2, 14, and 78 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 3, left square bracket 0 right square bracket
- 8, left square bracket 1 right square bracket
- 7, left square bracket 2 right square bracket
- Negative 2, left square bracket 3 right square bracket
- 14, left square bracket 4 right square bracket
- 78, left square bracket 5 right square bracket

A double-headed arrow is, with one head pointing at the first value “3” in the array and the other pointing at the last value “78” in the array and labeled “swap these.”

Long description

The diagram shows an array of six values with the second and the fifth value in the list swapped with each other. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at six values arranged in six cells next to each other. The values 78, 8, 7, negative 2, 14, and 3 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 78, left square bracket 0 right square bracket
- 8, left square bracket 1 right square bracket
- 7, left square bracket 2 right square bracket
- Negative 2, left square bracket 3 right square bracket
- 14, left square bracket 4 right square bracket
- 3, left square bracket 5 right square bracket

A double-headed arrow is, with one head pointing at the second value “8” in the array and the other pointing at the fifth value “14” in the array and labeled “then swap these.”

Long description

The diagram shows an array of six values with four swaps between the values. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at six values arranged in six cells next to each other. The values 78, 8, 7, negative 2, 14, and 3 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets from 0 through 5. The arrays of values appear four times in the diagram one below the other, indicating four swaps. In the first swap, the first value “78” and the sixth value “3” are labeled “i” and “j,” respectively. In the second swap, the second value “14” and the fifth value “8” are labeled “i” and “j,” respectively. In the third swap, the third value “negative 2” and the fourth value “7” are labeled “i” and “j,” respectively. In the fourth swap, the third value “7” and the fourth value “negative 2” are labeled “j” and “i,” respectively.

Long description

The diagram shows an array of six values with two swaps between the values. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at six values arranged in six cells next to each other. The values 78, 8, 7, negative 2, 14, and 3 are entered in each cell. The arrays of values appear twice in the diagram one below the other, indicating two swaps. In the first swap, the second value “8” and the fifth value “14” are labeled “j” and “i,” respectively. In the second swap, the first value “3” and the sixth value “78” are labeled “j” and “i,” respectively.

Long description

The diagram shows an array of length 5, with the array elements initialized to 0. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. The value “0” is entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 0, left square bracket 0 right square bracket
- 0, left square bracket 1 right square bracket
- 0, left square bracket 2 right square bracket
- 0, left square bracket 3 right square bracket
- 0, left square bracket 4 right square bracket

Long description

The diagram shows two array objects of length 5, with the elements initialized to 0. It shows three variables, “list 1,” “list 2,” and “list 3,” each standing alone inside a cell. A rightward arrow is starting from each cell and pointing at the objects containing five values, 1, 3, 5, 7, and 9 arranged in five cells next to each other. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The variable “list 1” refers to the first array object containing the values 1, 3, 5, 7, and 9, with indexes assigned from 0 through 4, respectively. The variables, “list 2” and “list 3” refer to the same array object containing the values, 1, 3, 5, 7, and 9, with indexes assigned from 0 through 4.

Long description

The diagram shows an array object of length 5, with the elements initialized to 0. It shows two variables, “list” and “data” each standing alone inside a cell. An arrow is starting from each cell and pointing at the object containing five values, 1, 3, 5, 7, and 9 arranged in five cells next to each other. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The variable “list” refers to the array object containing the values 1, 3, 5, 7, and 9. The variable “data” also refers to the same array object containing the values, 1, 3, 5, 7, and 9, with indexes assigned from 0 through 4.

Long description

The diagram shows an array object of length 5, with the elements initialized to 0. It shows two variables, “list” and “data” each standing alone inside a cell. An arrow is starting from each cell and pointing at the object containing five values, 2, 4, 6, 8, and 10 arranged in five cells next to each other. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The variable “list” refers to the array object containing the values 2, 4, 6, 8, and 10. The variable “data” also refers to the same array object containing the values, 2, 4, 6, 8, and 10, with indexes assigned from 0 through 4.

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. The values 3, 8, 9, 7, and 5 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are as follows:

- 3, left square bracket 0 right square bracket
- 8, left square bracket 1 right square bracket
- 9, left square bracket 2 right square bracket
- 7, left square bracket 3 right square bracket
- 5, left square bracket 4 right square bracket

Long description

The diagram shows two array objects of length 5, with the elements in the first initialized to 0. It shows two variables, each labeled “list” and both standing alone inside a cell. An arrow is starting from each cell and pointing at the object containing five values. The first array object stores the values, 3, 8, 9, 7, and 5. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The second array object stores the values, 8, 9, 7, 5, and 5. Four arrows are drawn from four values in the first array and point at four values in the second, indicating the shifting of four values to the left in the second arrow by one position. The values in the first array from which the arrows are drawn are 8, 9, 7, and 5. These arrows point at the values, 8, 9, 7, and 5, respectively in the second array.

Long description

The diagram shows two array objects of length 5, with the elements in the first initialized to 0. It shows two variables, each labeled “list” and both standing alone inside a cell. An arrow is starting from each cell and pointing at the object containing five values. The first array object stores the values, 3, 8, 9, 7, and 5. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The second array object stores the values, 3, 3, 8, 9, and 7. Four arrows are drawn from four values in the first array and point at four values in the second, indicating the shifting of four values to the right in the second arrow by one position. The values in the first array from which the arrows are drawn are 3, 8, 9, and 7. These arrows point at the values, 3, 8, 9, and 7, respectively in the second array.

Long description

The diagram shows two array objects of length 5, with the elements in the first initialized to 0. It shows two variables, each labeled “list” and both standing alone inside a cell. An arrow is starting from the first cell of the first array and pointing at the second value in the second array object. The first array object stores the values, 3, 8, 9, 7, and 5. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The second array object stores the values, 3, 3, 9, 7, and 5. An arrow is drawn from the value “3” in the first array and points at “3” in the second cell of the second array, indicating the value “3” being overwritten in place of 8.

Long description

The diagram shows two array objects of length 5, with the elements in the first initialized to 0. It shows two variables, each labeled “list” and both standing alone inside a cell. An arrow is starting from the second value of the first array object and pointing at the third value in the second array object. The first array object stores the values, 3, 3, 9, 7, and 5. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The second array object stores the values, 3, 3, 3, 7, and 5. An arrow is drawn from the second value “3” in the first array and points at the third value “3” in the second array, indicating the value “3” being overwritten in place of 9.

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. The values 3, 8, 9, 7, and 5 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array range from 0 through 4.

Long description

The diagram shows two array objects of length 5, with the elements in the first initialized to 0. It shows two variables, each labeled “list” and both standing alone inside a cell. An arrow is starting from the fourth value of the first array object and pointing at the fifth value in the second array object. The first array object stores the values, 3, 8, 9, 7, and 5. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The second array object stores the values, 3, 8, 9, 7, and 7. An arrow is drawn from the fourth value “7” in the first array and points at the fifth value “7” in the second array, indicating that the final value “5” in the list has been wiped out.

Long description

The diagram shows two array objects of length 5, with the elements in the first initialized to 0. It shows two variables, each labeled “list” and both standing alone inside a cell. An arrow is starting from the third value of the first array object and pointing at the fourth value in the second array object. The first array object stores the values, 3, 8, 9, 7, and 7. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array are assigned from 0 through 4. The second array object stores the values, 3, 8, 9, 9, and 7. An arrow is drawn from the third value “9” in the first array and points at the fourth value “9” in the second array, indicating that the former value “7” has been replaced with “9.”

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. The values 3, 3, 8, 9, and 7 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array range from 0 through 4.

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “list” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. The values 5, 3, 8, 9, and 7 are entered in each cell. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array range from 0 through 4.

Long description

The diagram shows a variable “points” referring to an array of length 3 storing references to point objects. It shows the variable “points” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at three point objects arranged in three cells next to each other. Each point object is labeled “null.” Each object is further assigned an index placed at the top and between square brackets as 0, 1, and 2, respectively.

Long description

The diagram shows a variable “points” referring to an array of length 3, storing references to point objects. It shows the variable “points” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at point objects arranged in three cells next to each other. The cells storing point objects appear to be empty, with each cell assigned an index placed at the top and between square brackets as 0, 1, and 2, respectively. The point object in index 0 refers to an array representing the value of x and y as 3 and 7, respectively. The point object in index 1 refers to an array representing the value of x and y as 4 and 5, respectively. The point object in index 2 refers to an array representing the value of x and y as 6 and 2, respectively.

Long description

The diagram shows a variable “args” referring to an array of length 2, storing references to arguments. It shows the variable “args” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at arguments arranged in two cells next to each other. The cells storing arguments appear to be empty, with each cell assigned an index placed at the top and between square brackets as 0 and 1, respectively. The point object in index 0 refers to an array with the argument labeled “temperature dot dat.” The point object in index 1 refers to an array with the argument labeled “temperature dot out.”

Long description

The diagram shows a variable “temps” referring to a two-dimensional grid with three rows and five columns. It shows the variable “temps” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at the cells in the grid. The cells in the grid are set to the same value, 0.0. The columns in the grid are assigned an index placed at the top of each column and between square brackets from 0 through 4. The rows in the grid are assigned an index placed at the left of each row and between square brackets from 0 through 2.

Long description

The diagram shows a variable “temps” referring to a two-dimensional rectangular grid with three rows and five columns. It shows the variable “temps” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at the cells in the grid. The columns in the grid are assigned an index placed at the top of each column and between square brackets from 0 through 3. The rows in the grid are assigned an index placed at the left of each row and between square brackets from 0 through 2. The fourth value of the first row is set to 98.3 while the first value of the third row is set to 99.4. The values in the remaining cells in the grid are set to 0.0.

Long description

The diagram shows a variable “jagged” referring to an array of arrays with the member arrays in different sizes and producing rows of jagged edges. It shows the variable “jagged” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at the cells in the array. The columns in the grid are assigned an index placed at the top of each row and between square brackets from 0 through 4. The rows in the grid are assigned an index placed at the left of each row and between square brackets from 0 through 2. Each cell in the array is set to the same value, 0.0.

Long description

The diagram shows a variable “triangle” referring to an array consisting of six rows with values. It shows the variable “triangle” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at the cells in the array. The columns in the array are assigned an index placed at the top of each row and between square brackets from 0 through 5. The rows in the array are assigned an index placed at the left of each row and between square brackets from 0 through 5. The values in each row appear as follows. Row 1: 1; row 2: 1, 1; row 3: 1, 2, 1; row 4: 1, 3, 3, 1; row 5: 1, 4, 6, 4, 1; row 6: 1, 5, 10, 10, 5, 1.

Long description

The screenshot shows a “Drawing Panel” window displaying the output of “Draw Purple Triangle.” The title bar of the window features a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” A right-angled triangle appears on the window screen in purple color.

Long description

The screenshot shows two “Drawing Panel” windows displaying the output of “Mirror” before and after mirroring. The title bar of the window features a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” The window on the left displays a drawing of a mirror on the extreme left, with a text above it that reads, “Hello, world!” The window on the right displays a drawing of a mirror on the extreme right, with a mirror image of the text “Hello, world!” above it.

Long description

The screenshot shows two “Drawing Panel” windows displaying the negative of an image. The title bar of the window features a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” The window on the left displays a drawing of the face of a cat in a light background. The window on the right displays a drawing of the face of a cat in a dark background.

Long description

The screenshot shows two “Drawing Panel” windows displaying an image before it is horizontally stretched and after it is horizontally stretched. The title bar of the window features a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” The window on the left displays a drawing of the face of a cat in normal shape. The window on the right displays a drawing of the face of a cat stretched horizontally.

Long description

The diagram shows an array of five values with their indexes assigned from 0 to 4. It shows the variable “count” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. Each cell is set to the same value, 0. Each value is further assigned an index placed at the top and between square brackets.

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “count” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. Each cell is set to the same value, 0. Each value is further assigned an index placed at the top and between square brackets.

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “count” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. Each value is further assigned an index placed at the top and between square brackets. The indexes for the values in the array range from 0 through 4. On moving from left to right, the values in the cells are set as, 0, 1, 0, 0, and 0, respectively, indicating that the counter at index 1 is incremented.

Long description

The diagram shows an array of five values with their indexes assigned from 0 to 4. It shows the variable “count” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. Each value is assigned an index placed at the top and between square brackets. On moving from left to right, the values in the cells are set as, 0, 1, 0, 0, and 1, respectively, indicating that the counter at index 1 and the counter at index 4 are incremented.

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “count” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. Each value is assigned an index placed at the top and between square brackets. The indexes for the values in the array range from 0 through 4. On moving from left to right, the values in the cells are set as, 0, 2, 0, 0, and 1, respectively; indicating that the counter at index 1 is set to 2 and the counter at index 4 is set to 1.

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “count” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. Each value is assigned an index placed at the top and between square brackets. On moving from left to right, the values in the cells are set as, 0, 2, 0, 0, and 1, respectively; indicating that the counter at index 1 is set to 2 and the counter at index 4 is set to 1.

Long description

The diagram shows an array of five values with their indexes from 0 to 4. It shows the variable “count” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. Each value is assigned an index placed at the top and between square brackets. On moving from left to right, the values in the cells are set as, 5, 9, 6, 9, and 11.

Long description

The diagram shows an array of length 5, storing five integer values, with their indexes from 0 to 4. It shows the variable “data” standing alone inside a cell. A rightward arrow is starting from the cell and pointing at five values arranged in five cells next to each other. Each value is assigned an index placed at the top and between square brackets. On moving from left to right, the values in the cells are set as, 27, 51, 33, negative 1, and 101.

Long description

The screenshot shows two “Drawing Panel” windows displaying an image in colored and black-and-white form. The title bars of the windows feature a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” The window on the left displays a colored image of a winking smiley face in a light background. The window on the right displays a black-and-white image of a winking smiley face, with the face in grayscale.

Long description

The screenshot shows two “Drawing Panel” windows displaying an image about both the x and y-axes. The title bars of the windows feature a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” The window on the left displays the normal image of a dinosaur while the window on the right displays the inverted image of a dinosaur, such that it is flipped vertically.

Long description

The screenshot shows two “Drawing Panel” windows displaying two images of different sizes. The title bars of the windows feature a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” The window on the left displays a very small image of a winking smiley face. The window on the right displays a zoomed image of the winking smiley face.

Long description

The screenshot shows two “Drawing Panel” windows displaying three images of different sizes. The title bars of the windows feature a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” The first window displays a normal image of a winking smiley face. The second window displays an image of a winking smiley face rotated left by 90 degrees. The third window displays an image of a winking smiley face rotated right by 90 degrees.

Long description

The screenshot shows two “Drawing Panel” windows displaying two images, one in normal view and the other in blurred view. The title bars of the windows feature a drop-down button on the left and three buttons to minimize the window, add new window, and close the window on the right. A menu bar appears immediately below the title bar featuring three options, “File,” “View,” and “Help.” The window on the left displays a clearly drawn winking smiley face, with a legible text atop it that reads, “Hello, world.” The window on the right displays a blurry image of the winking smiley face, with an illegible text atop it that reads, “Hello, world.”

Long description

The screenshot shows a window titled, “C S 1 0 6 A Image Algorithms.” A set of three buttons to minimize the window, maximize the window, and close the window appear on the right of the title bar. The window is divided into two panels, with the left panel displaying a menu of items. It features options to “Load Image,” “Save Image,” “Diff Images,” “Grayscale,” “Negative,” “Rotate Left,” “Rotate Right,” “Translate,” “Blur,” and “Mystery,” with the option “Rotate Left” in the selected mode. The right panel displays an image of a winking smiley face rotated leftward at 90 degrees. A text at the bottom of the window reads, “Rotate left filter applied.”

Long description

The screenshot shows the partial view of a window, displaying the icons for multiple files. A partial view of a pop-up menu of right-click appears for one of the icons with few of the commands appearing as “Open,” “Edit,” “Print,” and “Open With.” The cursor of a mouse is placed on the command, “Edit.”

Long description

The diagram shows an array of a point object in terms of x and y with reference to variable, p. It shows “P” inside a cell, with a rightward arrow pointing at a cell storing the values for x and y. The value of x is entered as “3” and the value of y is entered as “8.”

Long description

The diagram shows an array of a point object in terms of x and y with reference to variable, p. It shows “P” inside a cell, with a rightward arrow pointing at a cell storing the values for x and y. The value of x is entered as “2” and the value of y is entered as “6.”

Long description

The diagram shows an array of a point object in terms of x and y, with reference to variable, p 1. It shows a cell labeled “p 1” with a rightward arrow pointing at a cell storing the values for x and y, and the body of the “translate” method. The value of x is “7” and the value of y is “2.” The code written in the body of the “translate” method is as follows:

Line 1: public void translate left-parenthesis int d x, int d y right-parenthesis left brace

Line 2: (right indented) x plus equals d x semicolon

Line 3: (right indented) y plus equals d y semicolon

Line 4: right brace

Long description

The diagram shows an array of a point object in terms of x and y, with reference to variable, p 2. It shows a cell labeled “p 2” with a rightward arrow pointing at a cell storing the values for x and y, and the body of the “translate” method. The value of x is “4” and the value of y is “3.” The code written in the body of the “translate” method is as follows:

Line 1: public void translate left-parenthesis int d x, int d y right-parenthesis left brace

Line 2: (right indented) x plus equals d x semicolon

Line 3: (right indented) y plus equals d y semicolon

Line 4: right brace

Long description

The diagram shows an array of a point object in terms of x and y, with reference to variable, p 1. It shows a cell labeled “p 1” with a rightward arrow pointing at a cell storing the values for x and y, and the body of the “translate” method. The value of x is “18” and the value of y is “8.” The code written in the body of the “translate” method is as follows:

Line 1: public void translate left-parenthesis int d x, int d y right-parenthesis left brace

Line 2: (right indented) x plus equals d x semicolon

Line 3: (right indented) y plus equals d y semicolon

Line 4: right brace

A line at the top of the array reads “p 1 dot translate left-parenthesis 11 comma 6 right-parenthesis semicolon.” The parameters 11 and 6 are passed for “d x” and “d y,” respectively.

Long description

The diagram shows an array of a point object in terms of x and y, with reference to variable, p 2. It shows a cell labeled “p 2” with a rightward arrow pointing at a cell storing the values for x and y, and the body of the “translate” method. The value of x is “5” and the value of y is “10.” The code written in the body of the “translate” method is as follows:

Line 1: public void translate left-parenthesis int d x, int d y right-parenthesis left brace

Line 2: (right indented) x plus equals d x semicolon

Line 3: (right indented) y plus equals d y semicolon

Line 4: right brace

A line at the top of the array reads “p 2 dot translate left-parenthesis 1 comma 7 right-parenthesis semicolon.” The parameters 1 and 7 are passed for “d x” and “d y,” respectively.

Long description

The diagram shows an array of a point object in terms of x and y, with reference to variable, p 1. It shows a cell labeled “p 1” with a rightward arrow pointing at a cell storing the values for x and y, and the body of the “constructor” for a point class. The values for x and y are left blank. The code written in the body of the “constructor” reads as follows:

Line 1: public point left-parenthesis int initial X, int initial Y right-parenthesis left brace

Line 2: (right indented) x equals initial X semicolon

Line 3: (right indented) y equals initial Y semicolon

Line 4: right brace

A line at the top of the array reads “Point p 1 equals new point left-parenthesis 7 comma 2 right-parenthesis semicolon.” The parameters 7 and 2 are passed for “X” and “Y,” respectively.

Long description

The first diagram shows an array of a point object in terms of x and y with reference to variables, “p 1” and “this.” It shows “p 1” and “this” inside two separate cells, with two rightward arrows pointing at a cell storing the values for x and y. The value of x is entered as “18” and the value of y is entered as “8.”The lines in the body of the “translate” method are written below the values of x and y.The second diagram shows an array of a point object in terms of x and y with reference to variable, “p 2.” It shows “p 2” inside a separate cell, with a rightward arrow pointing at a cell storing the values for x and y. The value of x is entered as “4” and the value of y is entered as “3.” The lines in the body of the “translate” method are written below the values of x and y.

Long description

The first diagram shows an array of a point object in terms of x and y with reference to variable, “p 1.” It shows “p 1” inside a separate cell, with a rightward arrow pointing at a cell storing the values for x and y. The value of x is entered as “18” and the value of y is entered as “8.” The lines in the body of the “translate” method are written below the values of x and y. The second diagram shows an array of a point object in terms of x and y with reference to variables “p 2” and “this.” It shows “p 2” and “this” inside two separate cells, with two rightward arrows pointing at a cell storing the values for x and y. The value of x is entered as “5” and the value of y is entered as “10.” The lines in the body of the “translate” method are written below the values of x and y.

Long description

The diagram shows an array of a point object in terms of x and y, with reference to a variable. It shows a cell storing the values for x and y, and the body of the “translate” method. The value for x is “7” and the value of y is “2.” The code written in the body of the “translate” method reads as follows:

Line 1: public void translate left-parenthesis int d x, int d y right-parenthesis left brace

Line 2: (right indented) Ellipsis

Line 3: Right brace

Long description

The internal view of a radio shows the connections made between its internal parts such as diodes, resistors, capacitors, transistors, coils, and transformers. The external view of the radio shows its outer frame, providing mechanical support to the internal parts.

Long description

The flowchart shows the number of pages in a manual for different classes of employees in an organization. It shows a 20-page manual given to each employee. The different classes are further given the following number of pages in a manual:

- Lawyer: 2-page manual
- Secretary: 1-page manual
- Marketer: 3-page manual
- Legal Secretary: 1-page manual

Long description

The first diagram shows an array of a point object in terms of x and y with reference to variable, “p 1.” It shows “p 1” inside a separate cell, with a rightward arrow pointing at a cell storing the values for x and y. The value of x is entered as “7” and the value of y is entered as “2.” The second diagram shows an array of a point object in terms of x and y with reference to variables, “p 2” and “p 3.” It shows “p 2” and “p 3” inside two separate cells, with two rightward arrows pointing at a cell storing the values for x and y. The value of x is entered as “18” and the value of y is entered as “8.”

Long description

The flowchart shows classes, A, B, C, and D represented by boxes listing the methods and subclasses connected to their superclasses using arrows. Class A is at the top of the hierarchy and connected to its subclasses, B and C. Class D is connected to its superclass, class C. The methods listed in the boxes of the respective classes appear as follows:

- Class A: method 1, method 2, to string
- Class B: method 2
- Class A: method 1, to string
- Class D: method 2

Long description

The first diagram shows a square with its length labeled “w” and width labeled “h.” The second diagram shows a circle with its radius labeled “r.” The third diagram shows a right-angled triangle with its base labeled “b,” height labeled “a,” and hypotenuse labeled “c.”

Long description

The flowchart shows three shapes, “Circles,” “Rectangle,” and “Triangle” with their interfaces listed in boxes and connected to a box at the top of the hierarchy titled “interface Shape” by a dashed arrow. The box at the top shows the common interfaces to be implemented by the different shapes as follows:

- Get Area left-parenthesis right blank parenthesis
- Get Perimeter left-parenthesis right blank parenthesis

The attribute considered for the shape, “Circle” is radius. The different interfaces implemented by it are as follows:

- Circle (radius)
- Get Area left-parenthesis right blank parenthesis
- Get Perimeter left-parenthesis right blank parenthesis

The attributes considered for the shape, “Rectangle” are width and height. The different interfaces implemented by it are as follows:

- Rectangle (w, h)
- Get Area left-parenthesis right blank parenthesis
- Get Perimeter left-parenthesis right blank parenthesis

The attributes considered for the shape, “Triangle” are a, b, and c. The different interfaces implemented by it are as follows:

- Triangle (a, b, and c)
- Get Area left-parenthesis right parenthesis
- Get Perimeter left-parenthesis right parenthesis

Long description

The chart lists the names of different financial classes as “Stock,” “Dividend Stock,” “Mutual Fund,” and “Cash.” The state and behavior of the class, “Stock” are as follows:

- State:
  - Symbol
  - Total share: i n t
  - Total cost
  - Current price
- Behavior:
  - get Market Value left-parenthesis right parenthesis
  - get Profit left-parenthesis right parenthesis

The state and behavior of the class, “Dividend Stock” are as follows:

- State:
  - Symbol
  - Total share: i n t
  - Total cost
  - Current price
  - Dividends
- Behavior:
  - get Market Value left-parenthesis right parenthesis
  - get Profit left-parenthesis right parenthesis

The state and behavior of the class, “Mutual Fund” are as follows:

- State:
  - Symbol
  - Total share: double
  - Total cost
  - Current price

- Behavior:
  - get Market Value left-parenthesis right parenthesis
  - get Profit left-parenthesis right parenthesis

The state and behavior of the class, “Cash” are as follows:

- State:
  - amount
- Behavior:
  - get Market Value left-parenthesis right parenthesis

Long description

The flowchart shows different financial classes, “Stock,” “Mutual Fund,” “Cash,” and “Dividend Stock” with their state and behavior listed in boxes and connected to a box at the top of the hierarchy titled “interface Assets” by dashed arrows. The financial class, “Dividend Stock” is the subclass of “Stock” and it is connected to it by a solid arrow. The box at the top shows the common behaviors to be shown by the different financial classes as follows:

- Get MarketValue left-parenthesis right blank parenthesis
- Get Profit left-parenthesis right blank parenthesis

The state and behavior of the class, “Stock” are as follows:

- State:
  - Symbol
  - Total share: i n t
  - Total cost
  - Current price
- Behavior:
  - get Market Value left-parenthesis right parenthesis
  - get Profit left-parenthesis right parenthesis

The state and behavior of the class, “Dividend Stock” are as follows:

- State:
  - dividends
- Behavior:
  - get Market Value left-parenthesis right parenthesis
  - get Profit left-parenthesis right parenthesis

The state and behavior of the class, “Mutual Fund” are as follows:

- State:

    - Symbol
    - Total share: double
    - Total cost
    - Current price
- Behavior:
    - get Market Value left-parenthesis right parenthesis
    - get Profit left-parenthesis right parenthesis

The state and behavior of the class, “Cash” are as follows:

- State:
    - amount
- Behavior:
    - get Market Value left-parenthesis right parenthesis
    - get Profit left-parenthesis right parenthesis

Long description

The flowchart shows different financial classes, “Stock,” “Mutual Fund,” and “Dividend Stock” along with their state and behavior listed in boxes and connected to a box representing their superclass, “Share Asset” through a solid arrow. The classes, “Share Asset” and “Cash” are the subclasses of a box at the top of the hierarchy titled “interface Assets” by dashed arrows. The financial class, “Dividend Stock” is the subclass of “Stock” and is connected to it by a solid arrow. The box at the top shows the common behavior to be shown by the different financial classes as follows:

- Get MarketValue left-parenthesis right blank parenthesis
- Get Profit left-parenthesis right blank parenthesis

The box titled, “Share Asset” shows the following common state and behavior to be shown by the different financial classes.

- State:
  - symbol
  - total cost
  - current price
- Behavior:
  - get Profit

The state and behavior of the class, “Stock” are as follows:

- State:
  - Total share: i n t
- Behavior:
  - get Market Value left-parenthesis right parenthesis

The state and behavior of the class, “Dividend Stock” are as follows:

- State:
  - dividends
- Behavior:
  - get Market Value left-parenthesis right parenthesis

The state and behavior of the class, “Mutual Fund” are as follows:

- State:
  - Total share: double
- Behavior:
  - get Market Value left-parenthesis right parenthesis

The state and behavior of the class, “Cash” are as follows:

- State:
  - amount
- Behavior:
  - get Market Value left-parenthesis right parenthesis
  - get Profit left-parenthesis right parenthesis

Long description

The flowchart shows the classes of tickets represented by boxes, with a box at the top of the hierarchy titled, “Ticket” as the superclass for the tickets, “Walkup Ticket” and “Advance Ticket.” The ticket, “Student Advance Ticket” is the subclass for the ticket “Advance Ticket.” The common operations performed by the classes are as follows:

- Ticket number
- get Price left-parenthesis right parenthesis
- to string left-parenthesis right parenthesis

Long description

The diagram shows four bands with elements added to each, on moving downward. The first bands variable is referring at an empty element. The second band variable is assigned a code, “bands dot add left-parenthesis start quote Tool right-parenthesis end quote semicolon,” referring at an element placed newly as a list. The new element is labeled “Tool.” The index assigned to the element is 0. The second bands variable is assigned a code, “bands dot add left-parenthesis start quote Phish right-parenthesis end quote semicolon,” referring at an array, with element “Tool” and a newly placed element “Phish” at the end of the list. The index assigned to the new element is 1. The third bands variable is assigned a code, “bands dot add left-parenthesis start quote Pink Floyd right-parenthesis end quote semicolon,” referring at an array, with elements “Tool” and “Phish” and a newly placed element “Pink Floyd” at the end of the list. The index assigned to the new element is 2.

Long description

The diagram shows three bands with elements added to each. The first bands variable is referring at an array of elements “Tool,” “Phish,” and “Pink Floyd” with indexes “0,” “1,” and “2,” respectively. The second bands variable is assigned a code, “bands dot add left-parenthesis 1 comma start quote U 2 end quote right-parenthesis semicolon,” referring at an array of length 4. The first element in the array is “Tool” and assigned with index “0.” The second element in the array is empty and assigned with index “1.” The third element in the array is “Phish” and assigned with index “2.” The fourth element in the array is “Pink Floyd” and assigned with index “3.” Two rightward arrows are shown pointing from index 1 to index 2 and from index 2 to index 3. The elements in indexes 2 and 3 are highlighted. The third bands variable is referring at an array of length 4. The first element in the array is “Tool” and assigned with index “0.” The second element in the array is “U 2” and assigned with index “1.” The third element in the array is “Phish” and assigned with index “2.” The fourth element in the array is “Pink Floyd” and assigned with index “3.” The element in index 1 is highlighted.

Long description

The diagram shows three bands with elements added to each. The first bands variable is referring at an array of elements “Tool,” “U 2,” “Phish,” and “Pink Floyd” with indexes “0,” “1,” “2,” and “3,” respectively. The second bands variable is assigned a code, “bands dot remove left-parenthesis 0 comma right-parenthesis semicolon,” referring at an array of length 3. The first element in the array is “U 2” and assigned with index “0.” The second element in the array is “Phish” and assigned with index “1.” The third element in the array is “Pink Floyd” and assigned with index “2.” Three leftward arrows are pointing from infinity toward index 2, from index 2 to index 1, and from index 1 to index 0. All the elements are highlighted. The third bands variable is assigned a code “bands dot remove left-parenthesis 1 right-parenthesis semicolon” referring at an array of length 2. The first element in the array is “U 2” and assigned with index “0.” The second element in the array is “Pink Floyd” and assigned with index “1” indicating that the element in index 1 is removed. A leftward is pointing from infinity toward index 1.

Long description

The diagram shows an “Array List” with a sample set of eight ballots. The list elements range from 0 through 7. Each list element is referring at an array of length 4, with indexes from 0 through 4, respectively. The elements assigned to each list element are as follows:

- List element 0: Index 0, A; index 1, C; index 2, D; index 3, B.
- List element 1: Index 0, A; index 1, C; index 2, B; index 3, D.
- List element 2: Index 0, A; index 1, D; index 2, C; index 3, B.
- List element 3: Index 0, B; index 1, C; index 2, A; index 3, D.
- List element 4: Index 0, B; index 1, D; index 2, C; index 3, A.
- List element 5: Index 0, C; index 1, B; index 2, A; index 3, D.
- List element 6: Index 0, C; index 1, D; index 2, B; index 3, A.
- List element 7: Index 0, D; index 1, C; index 2, B; index 3, A.

Long description

The diagram shows the abridged view of the “Collection” interface extended and implemented by other interfaces and classes in the Java Collections Framework. It shows two flowcharts, one with the main branch labeled “interface Collection” and the other branch labeled “interface Map.” The main branch “interface Collection” is divided into “interface List” and “interface Set.” The branch “interface List” is shown further subdivided into “Array List” and “Linked List.” The branch “interface Set” is further subdivided into “Hash Set” and “Tree Set.” The main branch “interface Map” is divided into “Hash Map” and “Tree Map.”

Long description

The diagram shows a linked list presented as an array list and broken apart. The array list has the elements “a,” “b,” “c,” “d,” and “e” in small boxes set next to each other. Each element is assigned with the indexes 0, 1, 2, 3, and 4, respectively. It further shows a linked list with each element separately stored in a small box, connected to its neighboring box by leftward and rightward arrows. A text pointing at element “a” reads “front” and a text pointing at element “e” reads “back.”

Long description

The diagram illustrates the process of adding an element to the front of a linked list in three stages. The first stage reads, “Make a new node to hold the new element.” A variable “x” is assigned to this stage. The second stage reads, “Connect the new node to the other nodes in the list” with the variable “x.” It further shows a linked list with the elements, “a,” “b,” “c,” “d,” and “e” separately stored in small boxes, connected to the neighboring box by leftward and rightward arrows. A text pointing at element “a” reads “front” and a text pointing at element “e” reads “back.” The third stage reads, “Change the front of the list to point to the new node.” It shows a linked list with the elements, “x,” “a,” “b,” “c,” “d,” and “e” separately stored in small boxes, connected to the neighboring box by leftward and rightward arrows. A text pointing at element “e” reads “front” and a text pointing at element “e” reads “back.”

Long description

The diagram shows five set operations depicted by Venn diagrams. The first set operation shows a shaded circle inside a rectangular box labeled “A.” The second set operation shows a shaded circle inside a rectangular box labeled “B.” The third set operation shows two shaded circles overlapping each other inside a rectangular box, indicating the union of the sets A and B. The fourth set operation shows two circles overlapping each other inside a rectangular box, indicating the intersection of the sets A and B. The region common to both the circles is shaded. The fifth set operation shows two circles overlapping each other inside a rectangular box, indicating the difference between set A and set B. The portion of the first circle denoting set A that is not common to set B is shaded.

Long description

The diagram illustrates the mapping of keys denoted by first names to values denoted by last names. The first names appear in a circle labeled “Keys” and the last names appear in another circle placed alongside labeled “Values.” The first names are Romeo, Tyler, Tybalt, and Juliet. The last names are Montague, Durden, and Capulet. The first names are linked to the last names in the following pattern: Romeo Montague, Tyler Durden, Tybalt Capulet, and Juliet Capulet.

Long description

The diagram shows two persons standing at a distance from each other. A speech bubble attached to the first person reads “I want to know what place I am in line. So, what place are you in line?” A speech bubble attached to the second person reads “Let me try to figure that out.”

Long description

The diagram shows four persons standing at a distance from each other. A speech bubble attached to the first person reads “I’m at the front, so I’m first.” A speech bubble attached to the second person reads “Then I must be second in line.” A speech bubble attached to the third person reads “Then I must be third in line.” A speech bubble attached to the fourth person reads “Then I must be fourth in line.”

Long description

The image shows a sheet of paper with the “main” method written on it. The program for “main” reads as follows:

Line 1: public static void main left-parenthesis String left square bracket right square bracket args right-parenthesis left curly brace

Line 2: (indented rightward) draw Triangle left-parenthesis right-parenthesis semicolon

Line 3: (indented rightward) draw Two Triangles left-parenthesis right-parenthesis semicolon

Line 4: Right curly brace

Long description

The image shows a sheet of paper with the “main” method written on it, with the call on “draw Triangle” being executed. The program for “main” reads as follows:

Line 1: public static void main left-parenthesis String left square bracket right square bracket args right-parenthesis left curly brace

Line 2: (indented rightward) draw Triangle left-parenthesis right-parenthesis semicolon

Line 3: (indented rightward) draw Two Triangles left-parenthesis right-parenthesis semicolon

Line 4: Right curly brace

An arrow points at the command “draw Triangle left-parenthesis right-parenthesis semicolon” in line 2, indicating that the call on “draw Triangle” is being executed.

Long description

The image shows two sheets of paper with “main” and “draw Triangle” written on them. The papers are placed such that one overlaps the other. The paper at the bottom has “main” written on it. The paper with the method “draw Triangle” written on it is placed over the paper with “main” on it. The program for “draw Triangle” reads as follows:

Line 1: public static void draw Triangle left-parenthesis right-parenthesis left curly brace

Line 2: (indented rightward) System dot out dot print ln left curly brace start quote spaces asterisk end quote right curly brace semicolon

Line 3: (indented rightward) System dot out dot print ln left curly brace start quote spaces asterisk asterisk asterisk end quote right curly brace semicolon

Line 4: (indented rightward) System dot out dot print ln left curly brace start quote spaces asterisk asterisk asterisk asterisk asterisk end quote right curly brace semicolon

Line 5: (indented rightward) System dot out dot print ln left-parenthesis right-parenthesis semicolon

Line 6: Right curly brace

Long description

The image shows two sheets of paper with “main” and “draw Two Triangles” written on them. The papers are placed such that one overlaps the other. The paper at the bottom has “main” written on it. The paper with the method “draw Two Triangles” written on it is placed over the paper with “main” on it. The program for “draw Two Triangle” reads as follows:

Line 1: public static void draw Triangle left-parenthesis right-parenthesis left curly brace

Line 2: (indented rightward) draw Triangle left-parenthesis right-parenthesis semicolon

Line 3: (indented rightward) draw Triangle left-parenthesis right-parenthesis semicolon

Line 4: Right curly brace

Long description

The image shows two sheets of paper with “main” and “draw Two Triangles” written on them. The papers are placed such that one overlaps the other. The paper at the bottom has “main” written on it. The paper with the method “draw Two Triangles” written on it is placed over the paper with “main” on it. The program for “draw Two Triangle” reads as follows:

Line 1: public static void draw Triangle left-parenthesis right-parenthesis left curly brace

Line 2: (indented rightward) draw Triangle left-parenthesis right-parenthesis semicolon

Line 3: (indented rightward) draw Triangle left-parenthesis right-parenthesis semicolon

Line 4: Right curly brace

The command in line 2 has a rightward arrow pointing at it, indicating that it is the first call.

Long description

The image shows a sheet of paper with the method “draw Two Triangles” written on it. The program for “draw Two Triangle” reads as follows:

Line 1: public static void draw Triangle left-parenthesis right-parenthesis left curly brace

Line 2: (indented rightward) draw Triangle left-parenthesis right-parenthesis semicolon

Line 3: (indented rightward) draw Two Triangles left-parenthesis right-parenthesis semicolon

Line 4: Right curly brace

The command in line 3 has a rightward arrow pointing at it, indicating that it is the second call.

Long description

The image shows three sheets of paper with “main,” “draw Two Triangles,” and “draw Triangles” written on them. The papers are placed such that one overlaps the other. The paper at the bottom has “main” written on it. The paper in between has “draw Two Triangles” written on it. The paper with the method “draw Triangle” written on it is placed at the top. The program for “draw Triangle” reads as follows:

Line 1: public static void draw Triangle left-parenthesis right-parenthesis left curly brace

Line 2: (indented rightward) System dot out dot print ln left curly brace start quote spaces asterisk end quote right curly brace semicolon

Line 3: (indented rightward) System dot out dot print ln left curly brace start quote spaces asterisk asterisk asterisk end quote right curly brace semicolon

Line 4: (indented rightward) System dot out dot print ln left curly brace start quote spaces asterisk asterisk asterisk asterisk asterisk end quote right curly brace semicolon

Line 5: (indented rightward) System dot out dot print ln left-parenthesis right-parenthesis semicolon

Line 6: Right curly brace

Long description

The image shows a sheet of paper with the recursive file-reversing method written on it. The program for the recursive file-reversing method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line.”

Long description

The image shows a sheet of paper with the recursive file-reversing method written on it. The program for the recursive file-reversing method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line” as “this.”

An arrow points at line 4 indicating “reverse input” as the recursive call.

Long description

The image shows two sheets of paper, with the paper with the first call on reverse method written on it placed atop the other sheet. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

Line 8: line (placeholder)

Long description

The image shows two sheets of paper, with the paper with the second call on reverse method written on it placed atop the other sheet. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line” as “is.”

An arrow points at line 4 indicating “reverse input” as the recursive call.

Long description

The image shows three sheets of paper, with the paper with the third call on reverse method written on it placed atop the other sheets. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

Line 8: line (placeholder)

Long description

The image shows three sheets of paper, with the paper with the third call on reverse method written on it placed atop the other sheets. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line” as “fun.”

An arrow points at line 4 indicating “reverse input” as the recursive call.

Long description

The image shows four sheets of paper, with the paper with the fourth call on reverse method written on it placed atop the other three sheets. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

Line 8: line (placeholder)

Long description

The image shows four sheets of paper, with the paper with the fourth call on reverse method written on it placed atop the other three sheets. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line” as “no?”

An arrow points at line 4 indicating “reverse input” as the recursive call.

Long description

The image shows five sheets of paper, with the paper with the fifth call on reverse method written on it placed atop the other four sheets. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

Line 8: line (placeholder)

Long description

The image shows four sheets of paper, with the paper with the fourth call on reverse method written on it placed atop the other three sheets. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line” as “no?”

An arrow points at line 5 indicating “system dot out ln left-parenthesis line right-parenthesis semicolon” as the recursive call.

Long description

The image shows three sheets of paper, with the paper with the third call on reverse method written on it placed atop the other two sheets and the call positioned at the print ln right after it. The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line” as “fun.”

An arrow points at line 5 indicating “system dot out ln left-parenthesis line right-parenthesis semicolon” as the recursive call.

Long description

The image shows two sheets of paper, with the paper with the second call on reverse method written on it, placed atop the other sheet and the line variable assigned the value “is.” The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line” as “is.”

An arrow points at line 5 indicating “system dot out ln left-parenthesis line right-parenthesis semicolon” as the recursive call.

Long description

The image shows a sheet of paper with the line variable assigned the value “this.” The program for the reverse method reads as follows:

Line 1: public static void reverse left-parenthesis Scanner input right-parenthesis left curly brace

Line 2: (indented rightward) if left curly brace input dot has Next Line left-parenthesis right-parenthesis left curly brace

Line 3: (indented rightward) String line equals input dot next Line left-parenthesis right-parenthesis semicolon

Line 4: reverse left-parenthesis input right-parenthesis semicolon

Line 5: System dot out dot print ln left-parenthesis line right-parenthesis semicolon

Line 6: Right curly brace

Line 7: Right curly brace

A placeholder appears in the bottom-left section of the paper to store the value of the local variable “line” as “this.”

An arrow points at line 5 indicating “system dot out ln left-parenthesis line right-parenthesis semicolon” as the recursive call.

Long description

The diagram illustrates the tracing of the execution of a recursive function. It shows a series of recursive calls in a row reaching the base case of computing 3 to the power of 5. The recursive calls are shown connected to each other in the following pattern:

- pow (3, 5) equals 3 times pow (3, 4) connected to pow (3, 5) equals 3 times 81 equals 243
- pow (3, 4) equals 3 times pow (3, 3) connected to pow (3, 4) equals 3 times 27 equals 81
- pow (3,3) equals 3 times pow (3, 2) connected to pow (3, 3) equals 3 times 9 equals 27
- pow (3, 2) equals 3 times pow (3, 1) connected to pow (3, 2) equals 3 times 3 equals 9
- pow (3, 1) equals 3 times pow (3, 0) connected to pow (3, 1) equals 3 times 1 equals 3
- pow (3, 0) equals 1

It shows the call returning the final value 1, thus unwinding the recursion.

Long description

The diagram illustrates the tracing of the execution of a recursive function. It shows a series of recursive calls in a row reaching the base case of computing 2 to the power of 16. The recursive calls are shown connected to each other in the following pattern:

- pow (2 comma 16) equals pow (4 comma 8) connected to pow (2 comma 16) equals 65536
- pow (4 comma 8) equals pow (16 comma 4) connected to pow (4 comma 8) equals 65536
- pow (16 comma 4) equals pow (256 comma 2) connected to pow (16 comma 4) equals 65536
- pow (256 comma 2) equals pow (65536 comma 1) connected to pow (256 comma 2) equals 65536
- pow (65536 comma 1) equals 65536 times pow (65536 comma 0) connected to pow (65536 comma 1) equals 65536 times 1 equals 65536
- pow (65536 comma 0) equals 1

It shows the call returning the final value 1, thus unwinding the recursion.

Long description

The screenshot of the “Drawing Panel” window displays level 2 of the Sierpinski triangle. It shows the triangle divided into four smaller equilateral triangles using the midpoints of the three sides of the original triangle as the new vertices. The interior of the middle triangle is removed and shaded in white, resulting in three smaller triangles (shaded in black) inside the equilateral triangle.

Long description

The screenshot of the “Drawing Panel” window displays level 3 of the Sierpinski triangle. It shows the triangle containing three smaller triangles. Each smaller triangle is divided into four smaller equilateral triangles using the midpoints of the three sides of the original triangle as the new vertices. The interior of the middle triangle is removed and shaded in white, resulting in nine smaller triangles (shaded in black) inside the equilateral triangle.

Long description

The screenshot of the “Drawing Panel” window displays level 7 of the Sierpinski triangle. It shows the triangle containing three self-similar pieces corresponding to the three functions performed in level 1, level 2, and level 3. The triangle is formed by dividing the line segments of the largest triangle into four equal parts. Once new triangles are drawn the pattern is repeated.

Long description

The diagram shows a Cartesian plane with (x comma y) coordinates. It marks the point of origin as ordered pair 0 comma 0. Three different moves are made from the origin in different directions. The y-axis pointing toward the north is labeled “N: ordered pair 0 comma 1.” The x-axis pointing toward the east is labeled “E: ordered pair 1 comma 0.” A linear line is shown pointing toward the northeast and labeled “N E: ordered pair 1 comma 1.”

Long description

The diagram shows a decision tree with all of the possible ways to make two moves. The moves beginning from “start (0, 0)” appears in the decision tree in the following pattern:

- Start (0, 0) is divided into three moves: N (0, 1); E (1, 0); NE (1, 1)
- N (0, 1) is divided into three moves: N N (0, 2); N E (1, 1); N NE (1, 2)
- E (1, 0) is divided into three moves: E N (1, 1); E E (2, 0); E NE (2, 1)
- NE (1, 1) is divided into three moves: NE N (1, 2); NE E (2, 1); NE NE (2, 2)

Long description

The diagram shows a decision tree with all of the possible ways to travel to (1, 2). The moves beginning from “start (0, 0)” appears in the decision tree in the following pattern:

- Start (0, 0) is divided into three moves: N (0, 1); E (1, 0); NE (1, 1)
- N (0, 1) is divided into three moves: N N (0, 2); N E (1, 1); N NE (1, 2)
- N N (0, 2) is divided into three moves: N N N (0, 3); N N E (1, 2); N N NE (1, 3)
- N E is divided into three moves: N E N 91, 2); N E E (2, 1); N E NE (2, 2)
- E (1, 0) is divided into three moves: E N (1, 1); E E (2, 0); E NE (2, 1)
- E N (1, 1) is divided into three moves: E N N (1, 2); E N E (2, 1); E N NE (2, 2)
- NE (1, 1) is divided into three moves: NE N (1, 2); NE E (2, 1); NE NE (2, 2)

Long description

The diagram shows the 8 queens problem on an 8-by-8 chessboard, with the alternate squares in black and white color. The positions of the queens in each row appear as follows:

- First Q: First square in the first row
- Second Q: Sixth square in the second row
- Third Q: Eighth square in the third row
- Fourth Q: Third square in the fourth row
- Fifth Q: Seventh square in the fifth row
- Sixth Q: Fourth square in the sixth row
- Seventh Q: Second square in the seventh row
- Eighth Q: Fifth square in the eighth row

Eight arrows are shown starting from the Q in the fourth row, all pointing at different directions on the chessboard.

Long description

The diagram shows a decision tree with eight possible choices for the first column on an 8-by-8 board. The main branch of the decision tree is labeled “empty.” The possible choices branched out from it are as follows:

- Column 1, row 1
- Column 1, row 2
- Column 1, row 3
- Column 1, row 4
- Column 1, row 5
- Column 1, row 6
- Column 1, row 7
- Column 1, row 8

Long description

The diagram shows a decision tree with eight possible choices for the first and second column on an 8-by-8 board. The main branch of the decision tree is labeled “empty.” The possible choices branched out from it are as follows:

- Column 1, row 1
- Column 1, row 2
- Column 1, row 3
- Column 1, row 4
- Column 1, row 5
- Column 1, row 6
- Column 1, row 7
- Column 1, row 8

It further shows eight choices branched out from row 5 of column 1 in the following pattern:

- Column 2, row 1
- Column 2, row 2
- Column 2, row 3
- Column 2, row 4
- Column 2, row 5
- Column 2, row 6
- Column 2, row 7
- Column 2, row 8

Long description

The diagram shows a multiplication operator with two operands at the outermost level. It shows a statement that reads “multiplication sign plus 2.6 3.7 plus 5.2 18.7.” The multiplication sign is labeled “operator,” the expression “plus 2.6 3.7” is labeled “operand 1,” and the expression “plus 5.2 18.7” is labeled “operand 2.”

Long description

The image shows a few iterations of the Sierpinski carpet. The construction of the Sierpinski carpet begins with a square. The square is cut into nine congruent subsquares in a 3-by-3 grid, with the central subsquare removed. The same procedure is then applied recursively to the remaining eight subsquares, again and again.

Long description

The diagram shows the Canter set, with a set of points and small lines lying below a single and long line segment. The middle thirds of the long line segment are removed from the remaining pieces to form smaller segments. The process is repeated again and again, thus leaving very small dashed line segments in the fifth step.

Long description

The diagram shows the Towers of Hanoi puzzle, with three vertical lines A, B, and C, denoting three towers, respectively, at a distance from each other. Four disks of different sizes are stacked one above the other in the ascending order of their sizes and placed on tower A.

Long description

The diagram shows four iterations of the Koch snowflake. It shows the snowflakes built by starting with an equilateral triangle and recursively altering each line segment as follows. It divides the line segment into three segments of equal length and draws an equilateral triangle having the middle segment from the previous step as its base and points outward. It further removes the line segment that acts as the base of the triangle from the previous step, with the process being repeated again and again.

Long description

The diagram shows an example of the 4-by-5 Boggle board, with alphabets filled in each row as follows. First row: Y, S, E, H. Second row: B, N, O, W. Third row: O, I, S, R. Fourth row: N, E, R, U. A line starting from E in the first arrow is connected to E in the last row through N in the second row, S in the third row, and U in the last row.

Long description

The diagram shows a number line extended from 0 to 100 in increments of 10, with an arrow pointing at 50. A binary algorithm reads the following lines for the first guess.

Line 1: guess 50

Line 2: response/hint: higher

Line 3: conclusion: 51 less than equals to number less than equals to 100

The numbers from 0 to 100 in increments of 10, with an arrow pointing between the number 70 and 80 is shown in the second guess. A binary algorithm reads the following lines:

Line 1: guess 75

Line 2: response/hint: lower

Line 3: conclusion: 51 less than equals to number less than equals to 74.

The numbers from 0 to 100 in increments of 10, with an arrow pointing just to the right of number 60 is shown in the third guess. A binary algorithm reads the following lines:

Line 1: guess 62

Line 2: response/hint: higher

Line 3: conclusion: 63 less than equals to number less than equals to 74

The numbers from 0 to 100 in increments of 10, with an arrow pointing just before the number 70 is shown in the fourth guess. A binary algorithm reads the following lines:

Line 1: guess 68

Line 2: response/hint: higher

Line 3: conclusion: 69 less than equals to number less than equals to 74

The numbers from 0 to 100 in increments of 10, with an arrow pointing just after the number 70 is shown in the fifth guess. A binary algorithm reads the following lines:

Line 1: guess 71

Line 2: response/hint: higher

Line 3: conclusion: 72 less than equals to number less than equals to 74

The numbers from 0 to 100 in increments of 10, with an arrow pointing toward the middle of the numbers 70 and 80 is shown in the sixth guess. A binary algorithm reads the following lines:

Line 1: guess 73

Line 2: response/hint: correct

Long description

The image shows the loop with three statements repeating N times. The code written in the loop reads as follows:

Line 1: for left-parenthesis n times right-parenthesis left brace

Line 2: statement 1 dot

Line 3: statement 2 dot

Line 4: statement 3 dot

Line 5: right brace

The lines from 1 to 5 are collectively labeled “3 N.”

Long description

The image shows multiple loops placed sequentially with other statements. The codes written in multiple loops are as follows:

Line 1: statement 1 dot

Line 2: for left-parenthesis N times right-parenthesis left brace

Line 3: statement 2 dot

Line 4: right brace

Line 5: empty

Line 6: for left-parenthesis M times right-parenthesis left brace

Line 7: statement 3 dot

Line 8: empty

Line 9: statement 4 dot

Line 10: statement 5 dot

The lines from 2 to 4 are collectively labeled “N” and the lines from 6 to 8 are collectively labeled “M.” The lines from 1 to 10 are collectively labeled “M plus N plus 3.”

Long description

The image shows the codes for a loop with an inner loop, as follows:

Line 1: for left-parenthesis M times right-parenthesis left brace

Line 2: for left-parenthesis N times right-parenthesis left brace

Line 3: statement 1 dot

Line 4: statement 2 dot

Line 5: statement 3 dot

Line 6: right brace

Line 7: right brace

The lines from 3 to 5 are collectively labeled “3” and the lines from 2 to 6 are collectively labeled “M.” The lines from 1 to 7 are collectively labeled “3 M N.”

Long description

The image shows the loops placed N times one below the other. The codes written in the body of the loop read as follows:

Line 1: for left-parenthesis N times right-parenthesis left brace

Line 2: for left-parenthesis N times right-parenthesis left brace

Line 3: statement 1 dot

Line 4: right brace

Line 5: right brace

Line 6: empty

Line 7: empty

Line 8: for left-parenthesis N times right-parenthesis left brace

Line 9: statement 2 dot

Line 10: statement 3 dot

Line 11: right brace

The lines from 1 to 5 are collectively labeled “N squared” and the lines from 8 to 11 are collectively labeled “2 N.” The lines from 1 to 10 are collectively labeled “N squared plus 2 N.”

Long description

The table lists the data for input size, N in the first column and the runtimes of the “range” method in the adjacent column. The data for the runtimes of the “range” method at different input sizes is as follows:

- N, 1000; Runtime, 15
- N, 2000; Runtime, 47
- N, 4000; Runtime, 203
- N, 8000; Runtime, 781
- N, 16000; Runtime, 3110
- N, 32000; Runtime, 12563
- N, 64000; Runtime, 49937

The bar graph plots the values of the runtime of “range” method for different input sizes. The vertical axis represents the values of runtimes and ranges from 0 to 60000 in increments of 10000. The horizontal axis represents the input size, N and ranges from 1000 to 64000 in multiples of 2. It shows seven bins plotted at 1000, 2000, 4000, 8000, 16000, 32000, and 64000. The heights of the bins are 15, 47, 203, 781, 3110, 12563, and 10000, respectively.

Long description

The table lists the data for input size, N in the first column and the runtimes of the “range” method in the adjacent column. The data for the runtimes of the “range” method at different input sizes is as follows:

- N, 1000; Runtime, 16
- N, 2000; Runtime, 16
- N, 4000; Runtime, 110
- N, 8000; Runtime, 406
- N, 16000; Runtime, 1578
- N, 32000; Runtime, 6265
- N, 64000; Runtime, 25031

The bar graph plots the values of the runtime of “range” method for different input sizes. The vertical axis represents the values of runtimes and ranges from 0 to 30000 in increments of 5000. The horizontal axis represents the input size, N and ranges from 1000 to 64000 in multiples of 2. It shows seven bins at 1000, 2000, 4000, 8000, 16000, 32000, and 64000. The heights of the bins are 16, 16, 110, 406, 1578, 6265, and 25031, respectively.

Long description

The table lists the data for input size, N in the first column and the runtimes of the “range” method in the adjacent column. The data for the runtimes of the “range” method at different input sizes is as follows:

- N, 1000; Runtime, 0
- N, 2000; Runtime, 0
- N, 4000; Runtime, 0
- N, 8000; Runtime, 0
- N, 16000; Runtime, 0
- N, 32000; Runtime, 0
- N, 64000; Runtime, 0
- N, 128000; Runtime, 0
- N, 256000; Runtime, 0
- N, 512000; Runtime, 0
- N, 1 e 6; Runtime, 0
- N, 2 e 6; Runtime, 16
- N, 4 e 6; Runtime, 31
- N, 8 e 6; Runtime, 47
- N, 1.67 e 7; Runtime, 94
- N, 3.3 e 7; Runtime, 188
- N, 6.5 e 7; Runtime, 453
- N, 1.3 e 8; Runtime, 797
- N, 2.6 e 8; Runtime, 1578

The bar graph plots the values of the runtime of “range” method for different input sizes. The vertical axis represents the values of runtimes and ranges from 0 to 1800 in increments of 200. The horizontal axis represents the input size, N and ranges from 1000 to 2.60 e 8. The heights of the bins drawn between 4 e 6 and 8 e 6, 8 e 6 and 1.67 e 7, 1.67 e 7 and 3.3 e 7, 3.3 e 7 and 6.5 e 7,

6.5 e 7 and 1.3 e 8, and 1.3 e 8 and 2.6 e 8 are 16, 31, 47, 94, 188, 453, 797, and 1578, respectively.

Long description

The table lists the data for input size, N in the first column for the runtimes when an element is found and the runtimes when an element is not found in the adjacent columns. The data for the runtimes are as follows:

- N, 100000; Runtime when element is found, 0; Runtime when element is not found, 0
- N, 200000; Runtime when element is found, 0; Runtime when element is not found, 0
- N, 400000; Runtime when element is found, 0; Runtime when element is not found, 0
- N, 800000; Runtime when element is found, 0; Runtime when element is not found, 0
- N, 1.6 e 6; Runtime when element is found, 0; Runtime when element is not found, 16
- N, 3.2 e 6; Runtime when element is found, 15; Runtime when element is not found, 16
- N, 6.4 e 6; Runtime when element is found, 16; Runtime when element is not found, 31
- N, 1.3 e 7; Runtime when element is found, 32; Runtime when element is not found, 47
- N, 2.6 e 7; Runtime when element is found, 37; Runtime when element is not found, 93
- N, 5.1 e 7; Runtime when element is found, 45; Runtime when element is not found, 203
- N, 1.0 3 8; Runtime when element is found, 72; Runtime when element is not found, 402
- N, 2.0 e 8; Runtime when element is found, 125; Runtime when element is not found, 875

The bar graph plots the results of running the sequential search algorithm for different input sizes. The vertical axis represents the

values of runtimes and ranges from 0 to 1000 in increments of 100. The horizontal axis represents the input size, N and ranges from 100000 to 1.00 e 8. The heights of the bins drawn for runtimes when the element is found at 1.6 e 6, 3.2 e 6, 6.4 e 6, 1.3 e 7, 2.6 e 7, 5.1 e 7, 1.0 e 8, and 2.0 e 8 are 0, 15, 16, 32, 37, 45, 72, and 125, respectively. The heights of the bins drawn for runtimes when the element is not found at 1.6 e 6, 3.2 e 6, 6.4 e 6, 1.3 e 7, 2.6 e 7, 5.1 e 7, 1.0 e 8, and 2.0 e 8 are 16, 16, 31, 47, 93, 203, 402, and 875, respectively.

Long description

The diagram shows the array with numbers 11, 18, 29, 37, 42, 49, 51, 63, 69, 72, 77, 82, 88, 91, and 98 arranged in 15 cells respectively. Each cell is indexed with a number at the top from 0 to 14, in increments of 1. The numbers 11, 63, and 98 are shaded and labeled “min,” “mid,” and “max,” respectively.

Long description

The diagram shows the array with numbers 11, 18, 29, 37, 42, 49, 51, 63, 69, 72, 77, 82, 88, 91, and 98 arranged in 15 cells respectively. Each cell is indexed with a number at the top from 0 to 14, in increments of 1. The numbers 69, 82, and 98 are shaded and labeled “min,” “mid,” and “max,” respectively.

Long description

The diagram shows the array with numbers 11, 18, 29, 37, 42, 49, 51, 63, 69, 72, 77, 82, 88, 91, and 98 arranged in 15 cells respectively. Each cell is indexed with a number at the top from 0 to 14, in increments of 1. The numbers 69, 72, and 77 are shaded and labeled “min,” “mid,” and “max,” respectively.

Long description

The diagram shows the array with numbers 11, 18, 29, 37, 42, 49, 51, 63, 69, 72, 77, 82, 88, 91, and 98 arranged in 15 cells respectively. Each cell is indexed with a number at the top from 0 to 14, in increments of 1. The number, 77 is shaded and labeled “min,” “mid,” and “max.”

Long description

The diagram shows the array with numbers 1, 123, 12, 28, 183, and 16 arranged in 6 cells respectively. Each cell is indexed with a number at the top from 0 to 5, in increments of 1. The cell with index 0 is labeled “sorted” and the cells with indexes from 1 to 5 are labeled “to be sorted.” The numbers, 1 and 12 with indexes, 0 and 2, respectively are shaded.

Long description

The diagram shows the array with numbers 1, 123, 12, 28, 183, and 16 arranged in 6 cells respectively. Each cell is indexed with a number at the top from 0 to 5, in increments of 1. The cell with index 0 is labeled “sorted” and the cells with indexes from 1 to 5 are labeled “to be sorted.” The numbers, 1 and 12 with indexes, 0 and 2, respectively are shaded.

Long description

The diagram shows the array with numbers 1, 12, 123, 28, 183, and 16 arranged in 6 cells respectively. Each cell is indexed with a number at the top from 0 to 5, increments of 1. The cell with indexes 0 and 1 are labeled “sorted” and the cells with indexes from 2 to 5 are labeled “to be sorted.” The numbers, 12 and 123 with indexes, 1 and 2, respectively are shaded.

Long description

The diagram shows the array with numbers 1, 12, 16, 28, 183, and 123 arranged in 6 cells respectively. Each cell is indexed with a number at the top from 0 to 5, increments of 1. The cells with indexes from 0 to 2 are labeled “sorted” and the cells with indexes from 3 to 5 are labeled “to be sorted.” The numbers, 16 and 123 with indexes, 2 and 5, respectively are shaded.

Long description

The table lists the data for input size, N in the first column and the runtimes of the selection sort algorithm in the adjacent column. The data for the runtimes at different input sizes is as follows:

- N, 1000; Runtime, 0
- N, 2000; Runtime, 16
- N, 4000; Runtime, 47
- N, 8000; Runtime, 234
- N, 16000; Runtime, 657
- N, 32000; Runtime, 2562
- N, 64000; Runtime, 10265
- N, 128000; Runtime, 41141
- N, 256000; Runtime, 164985

The bar graph plots the results of the runtimes of the selection sort algorithm at different input sizes, N. The vertical axis represents the values of runtimes and ranges from 0 to 200000 in increments of 50000. The horizontal axis represents the input size, N and ranges from 1000 to 256000 in multiples of 4. It shows seven bins at 1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000, and 256000. The heights of the bins are 0, 16, 47, 234, 657, 2562, 10265, 41141, and 164985, respectively.

Long description

The diagram shows the array with numbers 14, 32, 67, 76, 23, 41, 58, and 85 arranged in 8 cells respectively. Each cell is indexed with a number at the top from 0 to 7, in increments of 1. The cells with indexes from 0 to 3 are labeled “sub array” and the cells with indexes from 4 to 7 are labeled “sub array.”

Long description

The chart is divided into three columns, with column heads labeled “Subarrays,” “Next include,” and “Merged array.” Each row of the first column has a set of two arrays of numbers, divided into four cells. The left array has the numbers 14, 32, 67, and 76 while the right array has the numbers 23, 41, 58, and 85. Each cell of both the arrays is indexed with numbers from 0 to 4, in increments of 1. The second column of the table provides information about the next numbers to be merged. The third column of the table shows the merged array, with merged numbers in the cells. Each cell of the merged array is indexed with numbers from 0 to 7, in increments of 1.

The first row depicts the first step with the number, 14 in the left array and the number, 23 in the right array shaded and labeled “i 1” and “i 2,” respectively. The number 14 in the left array is the next number to be included in the merged array. The merged array has number 14 in the first cell with an arrow pointing at it.

The second row depicts the second step with the number, 32 in the left array and the number, 23 in the right array shaded and labeled “i 1” and “i 2,” respectively. The number 23 in the right array is the next number to be included in the merged array. The merged array has the numbers, 14 and 23 in the cells indexed 0 and 1, respectively with an arrow pointing at 23.

The third row depicts the third step with the number, 32 in the left array and the number, 41 in the right array shaded and labeled “i 1” and “i 2,” respectively. The number 32 in the left array is the next number to be included in the merged array. The merged array has the numbers, 14, 23, and 32 in the cells indexed from 0 to 2, respectively with an arrow pointing at 32.

The fourth row depicts the fourth step with the numbers, 67 and 76 in the left array and the number 41 in the right array shaded. The number 67 in the left array and the number 41 in the right array are labeled “i 1” and “i 2,” respectively. The number 41 in the right array is the next number to be included in the merged array. The merged array has the numbers 14, 23, 32, and 41 in the cells indexed from 0 to 3, respectively with an arrow pointing at 41.

Long description

The chart shows the last four steps to merge two subarrays into a sorted arrangement as presented in three columns, “Subarrays,” “Next include,” and “Merged array.” Each row of the first column has a set of two arrays of numbers, divided into four cells. The left array has the numbers 14, 32, 67, and 76 while the right array has the numbers 23, 41, 58, and 85. Each cell of both the arrays is indexed with numbers from 0 to 4, in increments of 1. The second column of the table provides information about the next numbers to be merged. The third column of the table shows the merged array, with merged numbers in the cells. Each cell of the merged array is indexed with numbers from 0 to 7, in increments of 1.

The first row depicts the fifth step with the number, 67 in the left array and the number, 58 in the right array shaded and labeled “i 1” and “i 2,” respectively. The number 58 in the right array is the next number to be included in the merged array. The merged array has the numbers 14, 23, 32, 41, and 58 in the cells indexed from 0 to 4, respectively with an arrow pointing at 41.

The second row depicts the sixth step with the number, 67 in the left array and the number, 85 in the right array shaded and labeled “i 1” and “i 2,” respectively. The number 67 in the left array is the next number to be included in the merged array. The merged array has the numbers 14, 23, 32, 41, 58, and 67 in the cells indexed from 0 to 5, respectively with an arrow pointing at 67.

The third row depicts the seventh step with the number, 76 in the left array and the number, 85 in the right array shaded and labeled “i 1” and “i 2,” respectively. The number 76 in the left array is the next number to be included in the merged array. The

merged array has the numbers 14, 23, 32, 41, 58, 67, and 76 in the cells indexed from 0 to 6, respectively with an arrow pointing at 76.

The fourth row depicts the eighth step with the number, 76 in the left array and the number, 85 in the right array shaded. The number, 85 in the right array is labeled “i 2.” The number 85 in the right array is the next number to be included in the merged array. The merged array has the numbers 14, 23, 32, 41, 58, 67, 76, and 85 in the cells indexed from 0 to 7, respectively with an arrow pointing at 85.

Long description

The table lists the data for input sizes, N in the first column and the runtimes of merge sort algorithm in the adjacent column. The data for the runtimes at different input sizes is as follows:

- N, 1000; Runtime, 0
- N, 2000; Runtime, 0
- N, 4000; Runtime, 0
- N, 8000; Runtime, 0
- N, 16000; Runtime, 0
- N, 32000; Runtime, 15
- N, 64000; Runtime, 16
- N, 128000; Runtime, 47
- N, 256000; Runtime, 125
- N, 512000; Runtime, 250
- N, 1 e 6; Runtime, 532
- N, 2 e 6; Runtime, 1078
- N, 4 e 6; Runtime, 2265
- N, 8 e 6; Runtime, 4781
- N, 1.6 e 7; Runtime, 9828
- N, 3.3 e 7; Runtime, 20422
- N, 6.5 e 7; Runtime, 42406
- N, 1.3 e 8; Runtime, 88344

The bar graph plots the results for the runtimes of merge sort algorithm at different input sizes, N. The vertical axis represents the values of runtimes and ranges from 0 to 100000, in increments of 10000. The horizontal axis represents the input size, N and ranges from 1000 to 1.3 e 8. The heights of the bins drawn at 32000, 64000, 128000, 256000, 512000, 1 e 6, 2 e 6, 4 e 6, 8 e 6, 1.6 e 7, 3.3 e 7, 6.5 e 7, and 1.3 e 8 are 15, 16, 47, 125, 250, 532, 1078, 2265, 4781, 9828, 20422, 42406, and 88344, respectively.

Long description

The diagram shows a stack of objects placed one above the other, with the objects from bottom to top numbered 1, 2, and 3, respectively. The object numbered 1 is labeled “bottom” while the object numbered “3” is labeled “3.” An arrow at the top pointing at the top of the stack is labeled “push” while an arrow pointing out from the top of the stack is labeled “pop, peek.”

Long description

The diagram illustrates a queue as a vertical strip divided into three equal parts and numbered 1, 2, and 3, respectively. The part numbered 1 is labeled “front” while the part numbered 3 is labeled “front. An arrow starting from the left of the strip and pointing leftward is labeled “remove, peek.” An arrow pointing toward the right of the strip is labeled “add.”

Long description

The diagram shows a sample of the Karplus-Strong algorithm. It shows two queues, one labeled “time t” and the other labeled “time t plus 1.” The queue labeled “time t” is divided into five parts and assigned with values, 0.2, 0.4, 0.5, 0.3, negative 0.2, 0.4, 0.3, 0.0, negative 0.1, and negative 0.3 from left to right. The first two values, 0.2 and 0.4 in the queue are shaded. The queue labeled “time t plus 1” is divided into eleven parts and assigned with values, 0.2, 0.4, 0.5, 0.3, negative 0.2, 0.4, 0.3, 0.0, negative 0.1, negative 0.3, and 0.2988. An arrow from the first two values 0.2 and 0.4 of the first queue points toward an expression that reads, “0.996 times start fraction 1 over 2 end fraction left-parenthesis 0.2 plus 0.4 right-parenthesis.” An arrow starting from the expression further points toward the value, 0.2988 in the second queue. A text at the bottom reads “The Karplus-Strong update.”

Long description

The diagram shows an array of length 9 for the variable “element data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 1, 82, 97, 0, 0, 0, 0, 0, 0, and 0. The values are assigned indexes from 0 through 9, respectively.

Long description

The diagram shows an array of length 9 for the variable “element data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 1, 82, 97, 0, 0, 0, 0, 0, 0, and 0. The values are assigned indexes from 0 through 9, respectively. The values 1, 82, and 97 assigned with indexes 0, 1, and 2, respectively are together labeled “occupied” while the remaining values with indexes from 3 to 9 respectively are together labeled “vacant.”

Long description

The diagram shows an array for the variable “element data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 3, 6, 9, 12, 15, 0, 0, 0, ellipsis, and 0. The values are assigned indexes from 0 through 99, respectively. The values 3, 6, 9, 12, and 15 are together labeled “occupied” while the remaining values with indexes from 5 through 99 respectively are together labeled “vacant.” Another variable “size” is shown with its value entered as “5.”

Long description

The diagram shows an array for the variable “element data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 12, 19, 8, 73, 14, 0, 0, 0, ellipsis, and 0. The values are assigned indexes from 0 through 99, respectively. The values 12, 19, 8, 73, and 14 are together labeled “occupied” while the remaining values with indexes from 5 through 99 respectively are together labeled “vacant.” Another variable “size” is shown with its value entered as “5.”

Long description

The diagram shows an array for the variable “element Data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 12, 19, 8, 73, 14, 0, 0, 0, ellipsis, and 0. The values are assigned indexes from 0 through 99, respectively. It shows another array for the variable “element Data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 12, 8, 73, and 14, 14, 0, 0, 0, ellipsis, and 0. Three arrows starting from the values 8, 73, and 14 in the first array are shown pointing at the values, 8, 73, and 14 in the second array, indicating that the values that come after the value 19 each have been shifted done by one position.

Long description

The diagram shows an array for the variable “element data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 12, 8, 73, 14, 14, 0, 0, 0, ellipsis, and 0. The values are assigned indexes from 0 through 99, respectively. The values 12, 19, 8, 73, and 14 are together labeled “occupied” while the remaining values with indexes from 4 through 99 respectively are together labeled “vacant.” Another variable “size” is shown with its value entered as “5.”

Long description

The diagram shows an array for the variable “element data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 3, 6, 9, 12, 15, 0, 0, 0, ellipsis, and 0. The values are assigned indexes from 0 through 99, respectively. The values 3, 6, 9, 12, and 15, are together labeled “occupied” while the remaining values with indexes from 5 through 99 respectively are together labeled “vacant.” Another variable “size” is shown with its value entered as “5.”

Long description

The diagram shows an array for the variable “element Data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 3, 6, 9, 12, 15, 0, 0, 0, ellipsis, and 0. The values are assigned indexes from 0 through 99, respectively. It shows another array for the variable “element Data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 3, 6, 9, 9, 12, 15, 0, 0, ellipsis, and 0. Three arrows starting from the values 9, 12, and 15 in the first array are shown pointing at the values, 9, 12, and 15 in the second array.

Long description

The diagram shows an array for the variable “element data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 10, 20, 30, 40, 0, 0, 0, 0, 0, and 0. The values are assigned indexes from 0 through 9, respectively. Another variable “size” is shown with its value entered as “4” indicating 10, 20, 30, and 40 as the stored values.

Long description

The diagram shows an array for the variable “element data.” The variable is shown standing alone in a cell. A rightward arrow from the cell is pointing at the array consisting of the values, 20, 30, 40, 40, 0, 0, 0, 0, 0, and 0. The values are assigned indexes from 0 through 9, respectively. Another variable “size” is shown with its value entered as “3” indicating 20, 30, and 40 as the stored values.

Long description

The diagram shows the variable “list” referring to a node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “0” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “list” referring to a node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “7” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “0” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “list” referring to a node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “7” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “0” and the other storing the value of variable “next” as a forward slash. The first node is labeled “list.” The second node is labeled “list dot next.” The third node is labeled “list dot next dot next.”

Long description

The diagram shows the variable “list” referring to a node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “7” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “list” referring to a node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “7” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “list” referring to a node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “7” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash. An arrow starting from the variable list points at the first node, indicating that the node storing 3 has been leapfrogged.

Long description

The diagram shows the variable “list” storing a slash indicating “null. This is followed by three nodes, with the first node consisting of two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “7” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “p” referring to a node with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “4” and the other storing the value of variable “next” as a forward slash. The variable “q” refers to a node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “9” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “p” referring to a node with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “4” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “3” and the other storing the value of variable “next” as a forward slash. The variable “q” refers to a node with two cells, one storing the value of variable data as “9” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “p,” numbered 1, referring to the first node, numbered 2, with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 3, with two cells, one storing the value of variable data as “4” and the other storing the value of variable “next” as a forward slash. The variable “q,” numbered 4, refers to the first node, numbered 5, with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 6, with two cells, one storing the value of variable data as “9” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “p,” numbered 1, referring to the first node, numbered 2, with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 3, with two cells, one storing the value of variable data as “4” and the other storing the value of variable “next” as a forward slash. The variable “q,” numbered 4, refers to the first node, numbered 5, with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 6, with two cells, one storing the value of variable data as “9” and the other storing the value of variable “next” as a forward slash. An arrow starting from the node numbered 4 points at the node numbered 6.

Long description

The diagram shows the variable “p,” numbered 1, referring to the first node, numbered 2, with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 3, with two cells, one storing the value of variable data as “4” and the other storing the value of variable “next” as a forward slash. The variable “q,” numbered 4, refers to the first node, numbered 5, with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 6, with two cells, one storing the value of variable data as “9” and the other storing the value of variable “next” as a forward slash. An arrow starting from the node numbered 3 points at the node numbered 5.

Long description

The diagram shows the variable “p,” numbered 1, referring to the first node, numbered 2, with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 3, with two cells, one storing the value of variable data as “4” and the other storing the value of variable “next” as a forward slash. The variable “q,” numbered 4, refers to the first node, numbered 5, with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 6, with two cells, one storing the value of variable data as “9” and the other storing the value of variable “next” as a forward slash. An arrow starting from the node numbered 4 points at the node numbered 6. An arrow starting from the node numbered 3 points at the node numbered 5.

Long description

The diagram shows the variable “p,” numbered 1, referring to the first node, numbered 2, with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 3, with two cells, one storing the value of variable data as “4” and the other storing the value of variable “next” as a forward slash. The variable “q,” numbered 4, refers to the first node, numbered 5, with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 6, with two cells, one storing the value of variable data as “9” and the other storing the value of variable “next” as a forward slash. An arrow starting from the node numbered 4 points at the node numbered 6. An arrow starting from the node numbered 3 points at the node numbered 5.

Long description

The diagram shows the variable “p,” numbered 1, referring to the first node, numbered 2, with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node, numbered 3, with two cells, one storing the value of variable data as “4” and the other storing a reference of the variable “next” pointing at the third node. The third node, numbered 5, consists of two cells, one storing the value of variable data as “3” and the other storing the value of variable “next” as a forward slash. The variable “q,” numbered 4, refers to a node, numbered 6, with two cells, one storing the value of variable data as “9” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “list” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “list” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the first node, storing the value of variable data as 3.

Long description

The diagram shows the variable “list” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the second node, storing the value of variable data as 5.

Long description

The diagram shows the variable “list” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the first node, storing the value of variable data as 3.

Long description

The diagram shows the variable “list” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the first node, storing the value of variable data as 3.

Long description

The diagram shows the variable “list” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the third node, storing the value of variable data as 2.

Long description

The diagram shows the variable “list” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. Another variable “current” appears in the next line storing a slash indicating “null.”

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. Another variable “current” appears in the next line storing a slash indicating “null.”

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. The variable “current” refers to a node with two cells, one storing the value of variable data as “17” and the other storing the value of variable “next” as a forward slash.

Long description

Another variable “current” refers to the data field, list dot next dot next dot data, storing the value 2.

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the third node, storing the value of variable data as 2.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “2” and the other storing a reference of the variable “next” pointing at the fourth node. The fourth node consists of two cells, one storing the value of variable data as “17” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the fourth node, storing the value of variable data as 17.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “19” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “8” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “42” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “19” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “8” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “42” and the other storing the value of variable “next” as a forward slash. A new node appears at the top of the first node with two cells, one storing the value of variable “data” as “5” and the other storing a reference of the variable “next” pointing at the first node.

Long description

The diagram shows the variable “front” storing a reference to the new node with two cells, one storing the value of variable “data” as “5” and the other storing a reference of the variable “next” pointing at the first node. The first node has two cells, one storing the value of variable data as “19” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “8” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “42” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “19” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “8” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “42” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the second node, storing the value of variable data as 8.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “19” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “8” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “42” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the second node, storing the value of variable data as 42. A new node appears at the top of the third node with two cells, one storing the value of variable “data” as “5” and the other storing a reference of the variable “next” pointing at the third node.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “19” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “8” and the other storing a reference of the variable “next” pointing to the new node with two cells, one storing the value of variable “data” as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “42” and the other storing the value of variable “next” as a forward slash. A variable “current” appearing in the next line, stores a reference to the second node, storing the value of variable data as 8.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “19” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “8” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “42” and the other storing a reference of the variable “next” pointing at the fourth node. The fourth node consists of two cells, one storing the value of variable data as “13” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the second node, storing the value of variable data as 8.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “19” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “8” and the other storing a reference of the variable “next” pointing at the fourth node. The third node consists of two cells, one storing the value of variable data as “42” and the other storing a reference of the variable “next” also pointing at the fourth node. The fourth node consists of two cells, one storing the value of variable data as “13” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the second node, storing the value of variable data as 8.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash. Another variable “current” appearing in the next line, stores a reference to the second node, storing the value of variable data as 5. A new node appears at the top of the third node with two cells, one storing the value of variable “data” as “10” and the other storing a reference of the variable “next” pointing at the third node.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing to the new node with two cells, one storing the value of variable “data” as “10” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash. A variable “current” appearing in the next line, stores a reference to the second node, storing the value of variable data as 5.

Long description

The image shows a code that reads, “while left-parenthesis current dot next dot data less than value ampersand ampersand current dot next exclamation sign equals null right-parenthesis.” The expression “current dot next dot data less than value” in the code is labeled “sensitive test” while the expression “current dot next exclamation sign equals null” in the code is labeled “robust test.”

Long description

The image shows a code that reads, “if left-parenthesis front equals equals null vertical bar vertical bar front dot data greater than or equal to value right-parenthesis.” The expression “front equals equals null” in the code is labeled “empty list” while the expression “front dot data greater than or equal to value” is labeled “front of list.”

Long description

The image shows a code that reads, “while left-parenthesis current dot next exclamation sign equals null ampersand ampersand current dot next dot data less than value right-parenthesis.” The expression “current dot next exclamation sign equals null” in the code is labeled “back of list” while the expression “current dot next dot data less than value” in the code is labeled “middle of list.”

Long description

The diagram shows the variable “list” referring to the first node with two cells, one storing the value of variable data as “2” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “5” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash. Two new nodes appear at the top of the first and the second node respectively. The new node for variable “prev” stores a reference pointing at the first node while the new node for variable “current” stores a reference pointing at the second node.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “3” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “7” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a reference pointing at the second node with data 3.

Long description

The diagram shows the variable “front” referring to the first node with two cells, one storing the value of variable data as “0” and the other storing a rightward arrow pointing outward. The list next is referring to the second node with two cells, one storing the value of variable data as “3” and the other storing a reference of the variable “next” pointing at the third node. The third node consists of two cells, one storing the value of variable data as “7” and the other storing a reference of the variable “next” pointing at the fourth node. The fourth node consists of two cells, one storing the value of variable data as “12” and the other storing the value of variable “next” as a forward slash.

Long description

The diagram shows the variable “front” referring to the first three-element list with variables prev, data, and next stored as null, 3, and reference to the next list, respectively. The second three-element list has values for variables prev, data, and next stored as reference to the first list, 7, and reference to the next list, respectively. The third three-element list has values for variables prev, data, and next stored as reference to the second list, 12, and null, respectively.

Long description

The diagram shows the variable “front” referring to the first three-element list for variables prev, data, and next stored as null, 0, and reference to the second list, respectively. The variable “back” refers to the second three-element list with variables prev, data, and next stored as reference to the first list, 0, and null, respectively.

Long description

The diagram shows the variable “front” referring to the first three-element list for variables prev, data, and next stored as null, 0, and reference to the second list, respectively. The second three-element list has values for variables prev, data, and next stored as reference to the first list, 18, and reference to the next list, respectively. The third three-element list has values for variables prev, data, and next stored as reference to the second list, 73, and reference to the fourth list, respectively. The fourth three-element list has values for variables prev, data, and next stored as reference to the third list, 0, and null, respectively. The variable “back” refers to the fourth three-element list with variables prev, data, and next stored as reference to the third list, 0, and null, respectively.

Long description

The diagram shows the variable “list” referring to nodes in the left column labeled “Before” for seven times and in the right column “After” for seven times. The linked nodes in the left column appear in the following pattern:

- A variable “list” referring to two nodes, with the first node referring to the second node. The “data” fields in both the nodes store the values 1 and 2, respectively while the “next” fields in both the nodes are null.
- A variable “list” referring to two nodes, with the first node referring to the second node. The “data” fields in both the nodes store the values 1 and 2, respectively while the “next” fields in both the nodes are null.
- A variable “list” referring to two nodes, with the first node referring to the second node. The “data” fields in both the nodes store the values 1 and 2, respectively while the “next” fields in both the nodes are null.
- A variable “temp” referring to two nodes, with the first node referring to the second node. The “data” fields in both the nodes store the values 3 and 4, respectively while the “next” fields in both the nodes are null.
- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 1, 2, and 3 respectively while the “next” field in the third node is null.
- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 5, 4, and 3 respectively while the “next” field in the third node is null.
- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields

in the three nodes store the values 5, 4, and 3 respectively while the “next” field in the third node is null.

The linked nodes in the right column appear in the following pattern:

- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 1, 2, and 3 respectively while the “next” field in the third node is null.
- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 3, 1, and 2 respectively while the “next” field in the third node is null.
- A variable “list” referring to four nodes, with the first three nodes referring to the succeeding node. The “data” fields in the four nodes store the values 1, 3, 4, and 2, respectively while the “next” field in the fourth node is null.
- A variable “list” referring to a node, with the value of “data” stored as 2 and the value of “next” stored as “null.”
- A variable “list 2” referring to two nodes, with the first node referring to the second node. The “data” fields in both the nodes store the values 1 and 3, respectively while the “next” fields in both the nodes are null.
- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 4, 5, and 3 respectively while the “next” field in the third node is null.
- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 3, 4, and 5 respectively while the “next” field in the third node is null.

Long description

The diagram shows the variable “list” referring to nodes in the left column labeled “Before” for five times and in the right column “After” for five times. The linked nodes in the left column appear in the following pattern:

- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 1, 2, and 3 respectively while the “next” field in the third node is null.
- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 4, 5, and 6 respectively while the “next” field in the third node is null.
- A variable “list” referring to a node, with the value of “data” stored as 1 and the value of “next” stored as “null.”
- A variable “list 2” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 2, 3, and 4 respectively while the “next” field in the third node is null.
- A variable “list” referring to four nodes, with the first three nodes referring to the succeeding node. The “data” fields in the four nodes store the values 1, 2, 3, and 4, respectively while the “next” field in the fourth node is null.

The linked nodes in the right column appear in the following pattern:

- A variable “list” referring to four nodes, with the first three nodes referring to the succeeding node. The “data” fields in the four nodes store the values 5, 3, 4, and 2, respectively while the “next” field in the fourth node is null.

- A variable “list” referring to three nodes, with the first two nodes referring to the succeeding node. The “data” fields in the three nodes store the values 4, 1, and 2 respectively while the “next” field in the third node is null.
- A variable “list 2” referring to a node, with the value of “data” stored as 3 and the value of “next” stored as “null.”
- A variable “list” referring to two nodes, with the first node referring to the second node. The “data” fields in both the nodes store the values 4 and 2, respectively while the “next” fields in both the nodes are null.
- A variable “list 2” referring to two nodes, with the first node referring to the second node. The “data” fields in both the nodes store the values 3 and 1, respectively while the “next” fields in both the nodes are null.

Long description

The diagram shows the variable “list” referring to nodes in the left column for three times and in the right column for three times. The linked nodes in the left column appear in the following pattern:

- A variable “list” referring to two nodes, with the first node referring to the last node. The “data” fields in both the nodes store the values 7 and 16, respectively while the “next” fields in both the last node are null. The two nodes are separated by epsilon.
- A variable “list” referring to two nodes, with the first node referring to the last node. The “data” fields in both the nodes store the values 7 and 3, respectively while the “next” fields in both the last node are null. The two nodes are separated by epsilon.
- A variable “list” referring to two nodes, with the first node referring to the last node. The “data” fields in both the nodes store the values 7 and 3, respectively while the “next” fields in both the last node are null. The two nodes are separated by epsilon.

The linked nodes in the left column appear in the following pattern:

- A variable “list” referring to two nodes, with the first node referring to the last node. The “data” fields in both the nodes store the values 42 and 42, respectively while the “next” fields in both the last node are null. The two nodes are separated by epsilon. A text at the bottom reads “change all nodes to store the value 42.”
- A variable “list” referring to two nodes, with the first node referring to the last node. The “data” fields in both the nodes store the values 7 and 42, respectively while the

“next” fields in both the last node are null. The two nodes are separated by epsilon. A text at the bottom reads “change the last node to store the value 42.”

- A variable “list” referring to three nodes, with the first node referring to the last two nodes. The “data” fields in the nodes store the values 7, 3, and 42, respectively while the “next” fields in both the last node are null. The first node and the second-last node are separated by epsilon. A text at the bottom reads “add a node at the end that stores the value 42.”

Long description

The last element “Chris” further refers to the first element “Joe” and the second element “Sally” refers to the fourth element “Carol.”

Long description

The diagram of the binary tree shows the binary tree with six nodes. The main node stores the value 12, which is divided into two nodes storing the values, 18 and 7, respectively. The node storing the value 18 is further divided into a node storing the value 23 while the node storing the value 7 is further divided into two nodes storing the values, 4 and 13, respectively.

Long description

The diagram of the binary tree shows the binary tree with six nodes. On moving from bottom to top, the tree has a node storing the value 12, which is divided into two nodes storing the values, 18 and 7, respectively. The node storing the value 18 is further divided into a node storing the value 23 while the node storing the value 7 is further divided into two nodes storing the values, 4 and 13.

Long description

The diagram shows a set of four binary trees, with the first tree branching out into two nodes and each node further branching out into two more nodes. The second tree branches out into two nodes, and the left node further branches out into two more nodes. The third tree branches out into two nodes, and each node further node branching out to another node. The fourth tree branches out into nodes, and each node further branching out to another node.

Long description

The diagram of the binary tree shows the binary tree with six nodes. The main node stores the value 12, which is divided into two nodes storing the values, 18 and 7, respectively. The node storing the value 18 is further divided into a node storing the value 23 while the node storing the value 7 is further divided into two nodes storing the values, 4 and 13, respectively.

Long description

The flowchart shows the main approach “process the root” divided into three approaches, “preorder,” “inorder,” and “postorder.”A text between “preoder” and “inorder” is labeled “traverse left subtree” and a text between “inorder” and “postorder” is labeled “traverse right subtree.”

Long description

The diagram of the binary tree shows the binary tree with ten nodes. The main node stores the value 2, which is divided into two nodes storing the values, 0 and 3, respectively. The node storing the value 0 is further divided into a node storing the value 7 while the node storing the value 3 is further divided into two nodes storing the values, 1 and 9, respectively. The value 7 is further divided into two nodes storing the values, 6 and 5, respectively. The value 1 is further divided into a node storing the value 8 and the value 9 is divided into a node storing the value 4.

Long description

The diagram of the binary tree shows the binary tree with ten nodes. The main node stores the value 2, which is divided into two nodes storing the values, 0 and 3, respectively. The node storing the value 0 is further divided into a node storing the value 7 while the node storing the value 3 is further divided into two nodes storing the values, 1 and 9, respectively. The value 7 is further divided into two nodes storing the values, 6 and 5, respectively. The value 1 is further divided into a node storing the value 8 and the value 9 is divided into a node storing the value 4. It shows each node in the tree as an island and each link connecting the nodes as a solid wall. The movement of a sailboat is marked sailing in from the upper left part of the tree and sailing around the tree, hugging the coastline, and further continuing until it sails past the root carrying the value 4, on the right-hand side. It shows the sailboat sailing past the root node three times. It passes the first on the left, then underneath and then on the right.

Long description

The diagram of the binary tree shows the binary tree with nine nodes. The main node stores the value 1, which is divided into two nodes storing the values, 2 and 3, respectively. The node storing the value 2 is further divided into two nodes storing the values 4 and 5, respectively. The node storing the value 3 is further divided into two nodes storing the values 6 and 7, respectively. The node storing the value 4 is divided into two nodes, storing the value 8 and 9, respectively.

Long description

The diagram shows the complex binary tree with the main node storing the value 1. 1 is divided into two nodes, 2 and 3, respectively. The value 2 is further divided into two nodes, 4 and 5, respectively. 4 is further divided into two nodes 8 and 9, respectively while 5 is further divided into two nodes 10 and 11, respectively. The value 3 is divided into two nodes 6 and 7, respectively. 6 is divided into two nodes 12 and 13 while 7 is divided into 14 and 15, respectively.

Long description

The diagram shows a binary search tree of names, with the main node “Lisa” branched out into two names “Bart” and “Marge.” Bart is branched out into “Homer” which is further subdivided into two nodes named “Flanders” and “Krusty,” respectively. Marge is branched out into “Maggie” and “Smithers,” with “Smithers” further branched out to “Milhouse.”

Long description

The diagram shows a variable “p” referring to the variables “x” and “y” storing the values “2” and “8,” respectively. Another variable “q” appearing in the next line is referring to the “x” and “y” variables.

Long description

The diagram shows a variable “p” referring to the variables “x” and “y” storing the values “5” and “13,” respectively. Another variable “q” appearing in the next line is referring to the “x” and “y” variables.

Long description

The diagram shows two variables, with variable “p” referring to two variables “x” and “y” storing the values “5” and “13,” respectively. Variable “q” is referring to two variables storing the values “negative 7” and “negative 14,” respectively.

Long description

The diagram shows the binary tree with the main node storing the value “3,” which is divided into two nodes, “5” and “2,” respectively. The node storing the value “5” is further linked to another node storing the value “1.” The node storing the value “2” is branched out into two nodes storing the values, “4” and “6,” respectively.

Long description

The diagram shows the binary tree with the main node storing the value “3,” which is divided into two nodes, “5” and “2,” respectively. The node storing the value “5” is further linked to another node storing the value “1.” The node storing the value “2” is branched out into two nodes storing the values, “4” and “6,” respectively.

Long description

The diagram shows the binary tree with the main node storing the value “19,” which is branched out into two nodes storing the values, “47” and “63,” respectively. The node storing the value “47” is branched out into two nodes storing the values “23” and “negative 2,” respectively. The node storing the value “negative 2” is branched out to another node storing the value “55.” The node storing the value “63” is branched out into a node storing the value “94,” which is further branched out into a node storing the value “28.”

Long description

The diagram shows the binary tree with the main node storing the value “2” which is branched out into another node storing the value “1.” The value “1” is branched out into two nodes, one storing the value “7” and the other storing the value “6.” The value “7” is further branched out into a node storing the value “4.” The value “4” is further branched out into two nodes storing the values “3” and “5,” respectively. The value “6” is branched out into a node storing the value “9,” which is further branched out into a node storing the value “8.”

Long description

The diagram shows the binary tree with the main node storing the value “8,” which is further branched out into two nodes storing the values “5” and “11,” respectively. The node storing the value “5” is branched out into two nodes storing the values “2” and “7,” respectively. The value “2” is further branched out into a node storing the value “4.” The node storing the value “11” is further branched out into two nodes storing the values “10” and “18,” respectively. The value “18” is branched out into a node storing the value “20.” The value “20” is further branched out into a node storing the value “18.”

Long description

The diagram shows the binary tree, with the main node storing the value “m,” which is branched out into two nodes storing the values “g” and “q,” respectively. The value “g” is branched out into another node “b” which is further branched out into a node storing the value “e.” The node storing the value “q” is branched out into two nodes storing the values “k” and “x,” respectively.

Long description

The diagram shows the binary tree, with the main node storing the value “7.2,” which is branched out into two nodes storing the values “1.9” and “9.6,” respectively. The value “9.6” is branched out into two nodes storing the values “8.1” and “21.3,” respectively.

Long description

The diagram shows the binary tree “Reference Tree 1” with the main node storing the value “3,” which is branched out into two nodes storing the values “5” and “2,” respectively. The value “5” is branched out into another node “1.” The node storing the value “2” is branched out into two nodes storing the values “4” and “6,” respectively.

Long description

The diagram shows the binary tree “Reference Tree 2” with the main node storing the value “2” branched out into two nodes “8” and “1,” respectively. The node storing the value “8” is further branched out into another node storing the value “0.” The node storing the value “1” is further branched out into two nodes storing the values “7” and “6,” respectively. The node storing the value “7” is branched out into a node storing the value “4” while the node storing the value “6” is branched out into another node storing the value “9.”

Long description

The diagram shows the binary tree “Reference Tree 3” with the main node storing the value “2” which is further branched out into two nodes storing the values “3” and “1,” respectively. The node storing the value “3” is further branched out into two nodes storing the values “8” and “7,” respectively.

Long description

The diagram shows the binary tree with the main node storing the value “1,” which is branched out into two nodes storing the values “2” and “4,” respectively. The value “2” is branched out into another node “3.” The node storing the value “4” is branched out into two nodes storing the values “5” and “6,” respectively.

Long description

The diagram shows the complex binary tree with the main node storing the value “2” branched out into two nodes storing the value “8” and “1,” respectively. The node storing the value “8” is branched out into two nodes storing the values “0” and “negative 1,” respectively. The node storing the value “1” is branched out into two nodes storing the values “7” and “6,” respectively. The value “7” is further branched out into two nodes “4” and “negative 1” while the value “6” is branched out into two nodes “negative 1” and “9.”

Long description

The diagram shows the binary tree with the main node storing the value “2,” which is branched out into two nodes storing the values “0” and “1,” respectively. The value “1” is branched out into two nodes storing the values “4” and “9,” respectively.

Long description

The diagram shows the complex binary tree with the main node storing the value “3” branched out into two nodes storing the value “3” and “3,” respectively. The node storing the value “3” at the left is branched out into two nodes storing the values “3” and “2,” respectively. The node storing the value “3” at the right is branched out into two nodes storing the values “1” and “1,” respectively, each further branched out into another node storing the value “1.”

Long description

The diagram shows the binary tree with the main node storing the value “2,” which is branched out into two nodes storing the values “8” and “1,” respectively. The value “1” is branched out into two nodes storing the values “7” and “6,” respectively. The value “7” is further branched out into another node storing the value “4” while the value “6” is branched out into another node storing the value “9.”

Long description

The diagram shows the binary tree with the main node storing the value “2” which is branched out into two nodes “8” and “1,” respectively. The value “8” is further branched out into two nodes “0” and “0,” respectively. Each value “0” is further branched out into two nodes, each storing the value “0.” The value “1” is further branched out into two nodes storing the values “7” and “6,” respectively. The value “7” is further branched out into two nodes “4” and “0,” respectively. The value “6” is further branched out into two nodes storing the values “0” and “9,” respectively.

Long description

The diagram shows two binary trees, one labeled “tree 1” and the other labeled “tree 2.” The binary tree 1 has the main node storing the value “3” branched out into two nodes storing the values “4” and “7,” respectively. The value “4” is divided into two nodes “0” and “9,” with the value “9” further branched out into another node storing the value “8.” The value “7” is further branched out into another node storing the value “2.” The binary tree 2 has the main node storing the value “3” branched out into two nodes storing the values “6” and “7,” respectively. The node storing the value “6” is branched out into another node “0” which is further branched out into another node storing the value “8.” The node storing the value “7” is branched out into two nodes, one storing the value “9” and the other storing the value “2.”

Long description

The diagram shows the variable “element Data” referring to an array of length 10. The values 7, 5, 1, and 9 are stored in the first four cells of the array with the other cells left blank. All ten cells in the array are assigned with an index starting from 0 to 9. The cells from index 4 to index 9 are blank. Another variable “size” has its value entered as “4,” indicating that the size of the array is “4.”

Long description

The diagram shows the variable “element Data” referring to an array of length 10. The values 1, 5, 7, and 9 are stored in the first four cells of the array with the other cells left blank. All ten cells in the array are assigned with an index starting from 0 to 9. The cells from index 4 to index 9 are blank. Another variable “size” has its value entered as “4,” indicating that the size of the array is “4.”

Long description

The diagram shows the variable “element Data” referring to an array of length 10. All ten cells in the array are assigned with an index starting from 0 to 9. The values stored in the cells with indexes 1, 5, 7, and 9 are 1, 5, 7, and 9, respectively. The other cells in the array are left empty. Another variable “size” has its value entered as “4,” indicating that the size of the array is “4.”

Long description

The diagram shows the variable “element Data” referring to an array of length 10. All ten cells in the array are assigned with an index starting from 0 to 9. The values stored in the cells with indexes 1, 3, 5, 7, 8, and 9 are 1, 23, 5, 7, negative 58, and 9, respectively. The other cells in the array are left empty. Another variable “size” has its value entered as “6,” indicating that the size of the array is “6.”

Long description

The diagram shows the variable “element Data” referring to an array of length 10. All ten cells in the array are assigned with an index starting from 0 to 9. The values stored in the cells with indexes 1, 2, 3, 5, 6, 7, and 9 are 1, 91, 71, 5, 45, 7, and 9 respectively. The other cells in the array are left empty. Another variable “size” has its value entered as “7,” indicating that the size of the array is “7.”

Long description

The diagram shows the variable “element Data” referring to an array of length 10. All ten cells in the array are assigned with an index starting from 0 to 9. The cells assigned with indexes 1, 5, 7, and 9 are referring to a vertical chain of nodes in the following pattern:

- The cell with index 1 is referring to 1. 1 is referring to 91, which is further referring to 71.
- The cell with index 5 is referring to 5, which is further referring to 45.
- The cell with index 7 is referring to 7.
- The cell with index 9 is referring to 9.

Long description

The diagram shows the variable “element Data” referring to an array of length 10. All ten cells in the array are assigned with an index starting from 0 to 9. The values stored in the cells with indexes 1, 2, 3, 5, 6, 7, and 9 are X X, 91, 71, 5, 45, 7, and 9 respectively. The other cells in the array are left empty. Another variable “size” has its value entered as “6,” indicating that the size of the array is “6.” The “X X” in the cell with index 1 indicates the value has been removed.

Long description

The diagram shows the variable “element Data” referring to an array of length 10. All ten cells in the array are assigned with an index starting from 0 to 9. On moving from left to right, the values stored in the cells are 80, 1, 91, 71, 63, 5, 45, 7, 57, and 9. Another variable “size” has its value entered as “10,” indicating that the size of the array is “10.”

Long description

The diagram shows a properly resized and rehashed table of length 20 assigned with indexes from 0 through 19. The cells assigned with indexes 0, 1, 3, 5, 6, 7, 11, 12, 15, and 17 stores the values 80, 1, 63, 5, 45, 7, 9, 91, 71, 35, and 57, respectively.

Long description

The diagram shows the variable “element Data” referring to an array of length 10. All ten cells in the array are assigned with an index starting from 0 to 9. The cells assigned with indexes 0, 1, 3, 5, 7, and 9 are referring to a vertical chain of nodes in the following pattern:

- The cell with index 0 is referring to 80.
- The cell with index 1 is referring to 1, which is referring to 91. 91 is further referring to 71.
- The cell with index 3 is referring to 63.
- The cell with index 5 is referring to 5, which is further referring to 45.
- The cell with index 7 is referring to 7, which is further referring to 57.
- The cell with index 9 is referring to 9.

Long description

The diagram shows an enlarged chained hash table of length 20 after rehearsing. The cells in the table are assigned with indexes from 0 through 19. The cells assigned with indexes 0, 1, 3, 5, 7, 9, 11, 15, and 17 are referring to a vertical chain of nodes in the following pattern:

- The cell with index 0 is referring to 80.
- The cell with index 1 is referring to 1
- The cell with index 3 is referring to 63.
- The cell with index 5 is referring to 5, which is further referring to 45.
- The cell with index 7 is referring to 7.
- The cell with index 9 is referring to 9.
- The cell with index 11 is referring to 91, which is further referring to 71.
- The cell with index 15 is referring to 35.
- The cell with index 17 is referring to 57.

Long description

The diagram shows the internal state of a hash table with pairs. It shows the variable “element Data” referring to an array of length 10. All ten cells in the array are assigned with an index starting from 0 to 9. The cells assigned with indexes 1, 5, and 9 are referring to a vertical chain of nodes in the following pattern:

- The cell with index 0 is referring to the pair with the value of “key” and “value” given as “1” and “37,” respectively.
- The cell with index 5 is referring to the pair with the value of “key” and “value” given as “25” and “44,” respectively. It further refers to the pair with the value of “key” and “value” given as “negative 5” and “0,” respectively.
- The cell with index 0 is referring to the pair with the value of “key” and “value” given as “39” and “negative 2,” respectively.

Long description

The diagram shows an example of a min-heap, with the main node “overall root” storing the value 10. It is branched out into two nodes storing the values “50” and “20,” respectively. The value “50” is branched out into two values “70” and “60” with “70” further branching out into two nodes storing the values “80” and “99,” respectively. The value 60 is further branching out into another node storing the value “65.” The value “20” is branched out into two nodes storing the values “44” and “90,” respectively.

Long description

The diagram shows two binary trees illustrating the values being removed from min-heap with bubble-down. The first tree with the main node “overall root” stores the value 65. It is branched out into two nodes storing the values “50” and “20,” respectively. The value “50” is branched out into two values “70” and “60,” with “70” further branching out into two nodes storing the values “80” and “99,” respectively. The value “20” is further branched out into two nodes storing the values “44” and “90,” respectively. A dashed arrow from the main node storing the value “65” is drawn pointing toward “20” and labeled “bubble down.” Another dashed arrow is drawn from “20” pointing at “44.” The second tree with the main node “overall root” stores the value 20. It is branched out into two nodes storing the values “50” and “44,” respectively. The value “50” is branched out into two values “70” and “60,” with “70” further branching out into two nodes storing the values “80” and “99,” respectively. The value “44” is branched out into two nodes storing the values “65” and “90,” respectively. The values “44” and “65” appear in bold.

Long description

The diagram shows two binary trees illustrating the values being added to min-heap with bubble-down. The first tree with the main node “overall root” stores the value 20. It is branched out into two nodes storing the values “50” and “44,” respectively. The value “50” is branched out into two values “70” and “60,” with “70” further branching out into two nodes storing the values “80” and “99,” respectively. The value “60” is branched out into another node storing the value “37.” The value “44” is further branched out into two nodes storing the values “65” and “90,” respectively. A dashed arrow from the value “37” is drawn pointing toward “60” and labeled “bubble up.” Another dashed arrow is drawn from “60” pointing at “50.” The second tree with the main node “overall root” stores the value 20. It is branched out into two nodes storing the values “37” and “44,” respectively. The value “37” is branched out into two values “70” and “50,” with “70” further branching out into two nodes storing the values “80” and “99,” respectively. The value “50” is branched out into another node storing the value “60.” The value “44” is branched out into two nodes storing the values “65” and “90,” respectively. The values “50” and “60” appear in bold.

Long description

The diagram shows an array representation with the heap implemented as array for the variable “element Data.” The values in the array are assigned with indexes from 0 to 12, with cells in the indexes 1, 2, 3, 4, 5, 6, 7, 8, 9, and 10 storing the values 10, 50, 20, 70, 60, 44, 90, 80, 99, and 65. The end of the array is shown with ellipsis to depict continuity. Another variable “size” appears in the next line with its value entered as “10,” indicating that the size of the array is 10.

Long description

A diagram of an array representation for the variable “element Data” shows the adding of element value 37 to the heap. The values in the array are assigned with indexes from 0 to 12, with cells in the indexes 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, and 11 storing the values 10, 50, 20, 70, 60, 44, 90, 80, 99, 65, and 37. The end of the array is shown with ellipsis to depict continuity. It shows another array assigned with indexes from 0 to 12, with cells in the indexes 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, and 11 storing the values 10, 37, 20, 70, 50, 44, 90, 80, 99, 65, and 60. The end of the array is shown with ellipsis to depict continuity. Another variable “size” appears in the next line with its value entered as “11,” indicating that the size of the array is 11.

Long description

A diagram of an array representation for the variable “element Data” shows the removing of the root element value from the heap. The values in the array are assigned with indexes from 0 to 12, with cells in the indexes 1, 2, 3, 4, 5, 6, 7, 8, 9, and 10 storing the values 60, 37, 20, 70, 50, 44, 90, 80, 99, and 65. The end of the array is shown with ellipsis to depict continuity. It shows another array assigned with indexes from 0 to 12, with cells in the indexes 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, and 11 storing the values 20, 37, 44, 70, 50, 60, 90, 80, 99, and 65. The end of the array is shown with ellipsis to depict continuity. Another variable “size” appears in the next line with its value entered as “10,” indicating that the size of the array is 10.

Long description

The diagram shows the binary tree with the main node storing the value “12,” further branching out into two nodes storing the values “9” and “30,” respectively. The value “90” is branching out into two nodes “20” and “19” while the value “30” is branching out into two nodes “34” and “21,” respectively.

Long description

The diagram shows the binary tree with the main node storing the value “53,” further branching out into two nodes storing the values “69” and “70,” respectively. The value “69” is further branching out into another node storing the value “80” while the value “70” is storing the value “94.”

Long description

The diagram shows the binary tree with the main node storing the value “42,” further branching out into two nodes storing the values “59” and “94,” respectively. The value “59” is further branching out into two nodes storing the values “78” and “60,” respectively.

Long description

The diagram shows an example of a min-heap, with the main node “overall root” storing the value “12.” It is branched out into two nodes storing the values “29” and “70,” respectively. The value “29” is branched out into two values “30” and “39,” with “30” further branching out into two nodes storing the values “55” and “64,” respectively. The value 39 is further branching out into two nodes storing the values “40” and “99,” respectively. The value “70” is branched out into two nodes storing the values “84” and “91,” respectively.

Long description

The diagram shows an array representation with the heap implemented as array. The values in the array are assigned with indexes from 0 to 12, with cells in the indexes from 0 to 10 storing the values 12, 29, 30, 39, 40, 55, 64, 70, 84, 91, and 99. The end of the array is shown with ellipsis to depict continuity.

Long description

The diagram shows an array representation with the heap implemented as array. The values in the array are assigned with indexes from 0 to 9, with cells in the indexes from 1 to 8 storing the values 29, 41, 30, 55, 68, 37, 41, and 80.

Long description

The diagram shows a stream as a horizontal flow of data from a “source” to a “modifier” through “stream 1”. It further flows from the “modifier” to the “terminator” through “stream 2” and other streams in between, denoted by ellipses.

Long description

The image shows a code for stream operations with map using the convention of including one step on each line. The code reads as follows:

Line 1: IntStream dot range gives left square bracket 1, 2, 3, 4, 5 right square bracket

Line 2: Gives map gives left square bracket 1, 4, 9, 16, 25 right square bracket

Line 3: Gives sum gives 55

Long description

The image shows a code for stream operations on the first 10 digits of pi. The code reads as follows:

Line 1: IntStream dot of gives left square bracket 3, 1, 4, 1, 5, 9, 2, 6, 5, 3 right square bracket

Line 2: Gives map gives left square bracket 9, 1, 16, 1, 25, 81, 4, 36, 25, 9 right square bracket

Line 3: Gives sum gives 207

Long description

The image shows a code for stream operations with filter for flow of data. The code reads as follows:

Line 1: IntStream dot of gives left square bracket 3, 1, 4, 1, 5, 9, 2, 6, 5, 3 right square bracket

Line 2: Gives filter left square bracket 3, 1, 1, 5, 9, 5, 3 right square bracket

Line 3: Gives map gives left square bracket 9, 1, 1, 25, 81, 25, 9 right square bracket

Line 4: Gives sum gives 151

Long description

The image shows a code for stream operations with distinct. The code reads as follows:

Line 1: IntStream dot of gives left square bracket 3, 1, 4, 1, 5, 9, 2, 6, 5, 3 right square bracket

Line 2: Gives filter left square bracket 3, 1, 1, 5, 9, 5, 3 right square bracket

Line 3: Gives distinct gives left square bracket 3, 1, 5, 9 right square bracket

Line 4: Gives map gives left square bracket 9, 1, 25, 81 right square bracket

Line 5: Gives sum gives 116

Long description

The diagram shows the code for a function closure that reads as follows:

Line 1: parameters colon left-parenthesis x comma y right-parenthesis

Line 2: free variables colon min equals 10, max equals 50, multiplier equals 3

Line 3: code colon Math dot max left-parenthesis x comma min right-parenthesis times Math dot max left-parenthesis y comma max right-parenthesis times multiplier

Long description

The image shows a code for stream operations on an array. The code reads as follows:

Line 1: Arrays dot stream dot of gives left square bracket 3, negative 4, 8, 4, negative 2, 17, 9, negative 10, 14, 6, negative 12 right square bracket

Line 2: Gives map gives left square bracket 3, 4, 8, 4, 2, 17, 9, 10, 14, 6, 12 right square bracket

Line 3: Gives filter left square bracket 4, 8, 4, 2, 10, 14, 6, 12 right square bracket

Line 4: Gives distinct gives left square bracket 4, 8, 2, 10, 14, 6, 12 right square bracket

Line 5: Gives sum gives 56

Long description

The image shows a code for stream operations on a list of strings. The code reads as follows:

Line 1: words dot stream gives left square bracket “To”, “be”, “or”, “Not”, “to” “be” right square bracket

Line 2: Gives map gives left square bracket “to”, “be”, “or”, “not”, “to” “be” right square bracket

Line 3: Gives distinct gives left square bracket “to”, “be”, “or”, “not” right square bracket

Line 4: Gives map gives left square bracket “be”, “not”, “or”, “to”, right square bracket

Line 5: Gives for Each gives prints colon “be not or to”

Long description

The image shows a code for stream operations on a list of strings mapped to integers. The code reads as follows:

Line 1: words dot stream gives left square bracket “To”, “be”, “or”, “Not”, “to” “be” right square bracket

Line 2: Gives map To Int gives left square bracket 2, 2, 2, 3, 2, 2 right square bracket

Line 3: Gives sum gives 13

Long description

The image shows a code for stream operations on the lines of a file. The code reads as follows:

Line 1: Files dot lines gives left square bracket “haiku are funny”,

Line 2: (indented rightward) “but sometimes they don’t make sense”,

Line 3: (indented rightward) “refrigerator” right square bracket

Line 4: Gives map To Int gives left square bracket 15, 35, 12 right square bracket

Line 5: Gives max gives 35

Long description

The image shows a code for stream operations to compute the sum of divisors of 10. The code reads as follows:

Line 1: IntStream dot range gives left square bracket 1, 2, 3, 4, 5, 6, 7, 8, 9 right square bracket

Line 2: Gives filter left square bracket 1, 2, 5 right square bracket

Line 3: Gives peek gives left square bracket 1, 2, 5 right square bracket with side effect of printing “1, 2, 5,”

Line 4: Gives sum gives 8

Long description

The image shows a code for stream operations to compute the sum of divisors of 10. The code reads as follows:

Line 1: IntStream dot range gives left square bracket 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 right square bracket

Line 2: Gives peek gives left square bracket 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 right square bracket “1 equals 2 equals 3 equals 4 equals 5 equals 6 equals 7 equals 8 equals 9 equals 10 equals”

Line 3: Gives map gives left square bracket 0, 1, 1, 3, 1, 6, 1, 7, 4, 8 right square bracket

Line 4: Gives for Each gives prints to system dot out colon “0 1 1 3 1 6 1 7 4 8”

Long description

The screenshot of the webpage presents an overview of the A P I specification for the “Java (trademark) Platform, Standard Edition 7.” The page is shown divided into two panels, with the left panel further divided into two frames. The upper-left frame features two items, “All Classes” and “Packages” with “Packages” in the selected mode. The lower-left frame enlists a list of items under the heading “All Classes,” with the items sorted in the alphabetical order. The right panel features a table divided into two columns, with the first column “Packages” listing the names of the various packages in the class library and the second column “Description” providing a brief overview of each package. Both the panels are shown with vertical and horizontal scroll bars, indicating that there are more items on the list.

Long description

The screenshot shows a sample Javadoc page titled “Point – Microsoft Internet Explorer” for a preceding “Point” class, with buttons to minimize the window, maximize the window, and close the window at the top-right section of the window. A menu bar appears immediately below the title bar with tabs for “File,” Edit,” “View,” “Favorites” “Tools,” and “Help.” It also shows six items, “Package,” “Class,” “Tree,” Deprecated,” “Index,” and “Help” arranged below the menu bar, with “Class” in the selected mode. It also has options to go to the previous class, next class, identify frames, and access other information like summary and details for the chosen class. The line of code for the chosen class point is given as “java dot lang dot object” with a pointer labeled “Point” indicating toward “java.” The page further provides descriptive information about the “public class point” followed by a table titled “Method Summary.” A vertical scroll bar at the right of the page indicates that there are a number of items in the table.